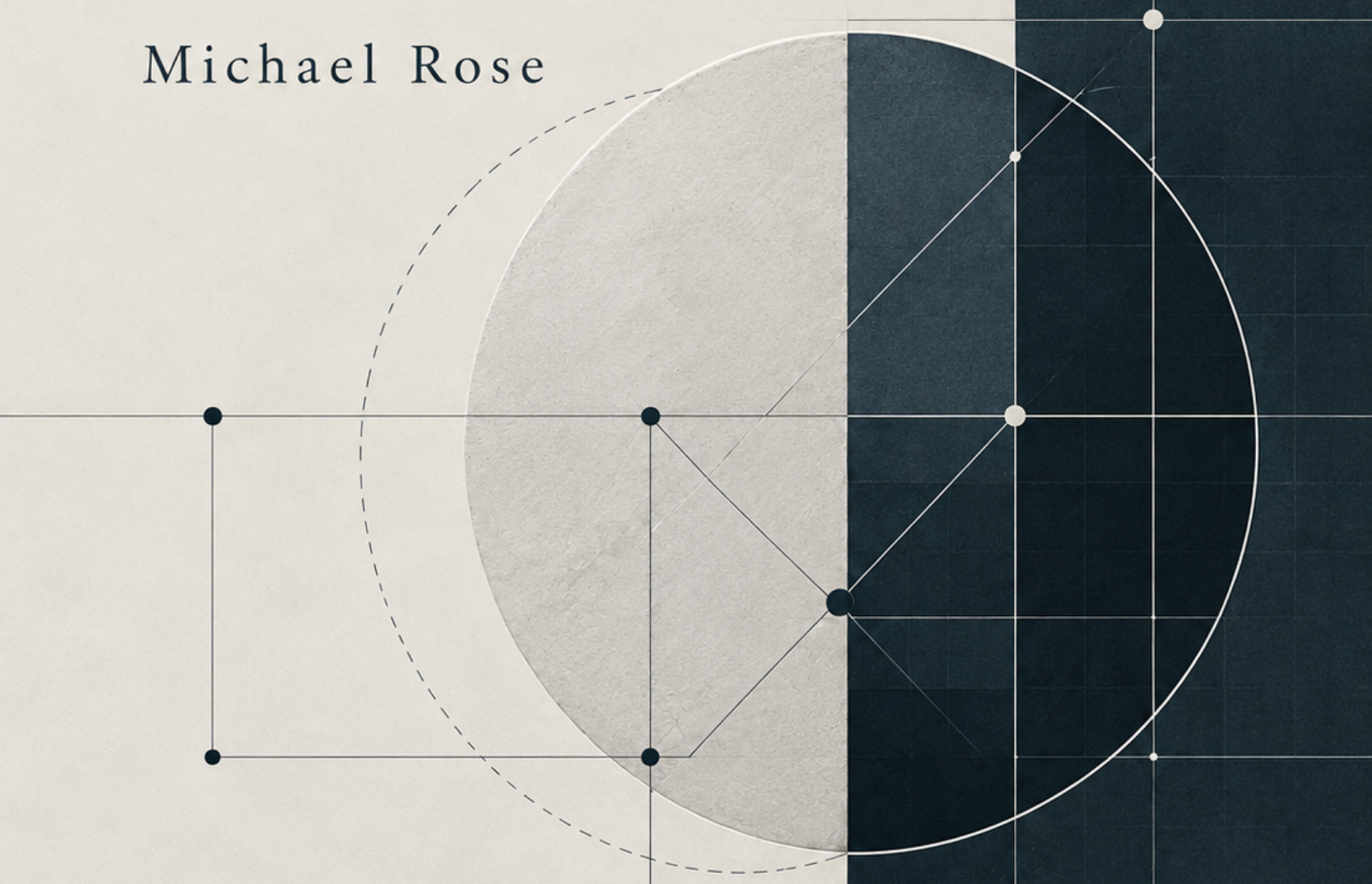
Gesso
Unified Design

Canonical architecture, assurance model, and reference application

Michael Rose



2026-09-22

Part I — Foundations

Canonical document identity and provenance

This document specifies the candidate R5 Gesso unified design as an intended end-state, not a claim that the current implementation already satisfies its requirements. Canonical R4 remains the preservation baseline until a complete cross-part requirements and acceptance audit explicitly accepts R5.

The published design retains twelve ordered parts. During review, a part may be maintained as smaller numbered editorial source sections (for example, 01.01 and 01.02) assembled mechanically into that published part. These fragments have no independent normative status. The candidate remains `r5-draft` until its assembly, cross-references, and requirement changes have been reviewed; it does not implicitly supersede R4.

Historical source plans remain provenance. They do not carry independent normative revision identities inside this specification.

The principal source lineage is:

Source	Revision / role
Gesso Choreography Design	v4.5; foundational choreography semantics
Gesso Provable Hardening Plan	v7.376; compositional rejection and proof direction
Gesso/HumanHelp Biff 2 Migration Plan	native Biff 2 substrate and lifecycle integration
Gesso/HumanHelp Preflight Recovery Plan	v662; no-silent-downgrade and production-path closure
Gesso Unified Design	canonical r2; prior consolidation
Gesso Unified Design	canonical r3; earlier predecessor
Gesso Unified Design	canonical r4; direct predecessor and preservation baseline
Gesso Unified Design Revision and Validation Plan	plan revision 2; r3 review discipline

Historical plans explain the origin of the design. The reviewed R4 assembly is the preservation reference for this R5 proposal. Existing obligations are retained when their wording is consolidated; a requirement is not deleted just because its original example or paragraph was removed. Each significant R4 omission must be classified as preserved elsewhere, deliberately superseded, or unresolved for repair. A change in implementation does not by itself change a requirement, and an implementation restriction does not become a design restriction merely because it made one proving experiment tractable.

This specification describes the intended architecture from zero. It does not classify features as implemented or unfinished, prescribe patches to a current checkout, or treat a passing implementation test as a normative decision. Implementation-specific remediation belongs in a separate document.

Revision scope

Candidate R5 specifies the semantic, construction, runtime, acceptance, and assurance boundaries of Gesso as one coherent architecture. Its requirements remain owned by the sections that define them; this summary is a navigation aid.

The revision emphasizes:

- the full host language within checked managed application construction;
- explicit contracts for opaque computations, with assumptions preserved through every dependent guarantee;
- typed semantic transitions, derived obligations, and controlled realization of admitted effects;
- backend contracts supported by versioned qualification and adversarial conformance evidence;
- owner-relative admission for asynchronous events, including shared events;
- scope-preserving acquisition requirements and precisely stated resource bounds;
- independent required-capability acceptance alongside assembly closure and production-path enforcement;
- participant-relative command uncertainty after local retirement or lost delivery;
- explicit HumanHelp adoption policy and testable product outcomes;
- production/formal correspondence classified by the evidence actually supplied;
- a single normative home for each obligation, with examples that preserve it;
- connected reference realizations and controlled defects on production paths.

Unresolved product choices remain explicit. A completion claim that depends on one of those choices is unavailable until the choice and its acceptance conditions are specified. Generic Gesso semantics do not silently choose HumanHelp business policy.

Document organization and sectional assembly

The published document has the following logical parts. Their two-digit order is the whole-document concatenation order. Within a split part, editorial source sections have a separately specified, deterministic assembly order.

Order	Part	Scope
01	I — Foundations	Purpose, ownership direction, normative interpretation, correctness scopes, assurance, evidence vocabulary, construction principles
02	II — Semantic model	Identity, time, resource instances, trust, knowledge, provenance, observation evidence, agreement
03	III — Choreography language and projection	Choreography, operation types, Malli, branch knowledge, projection, long logical workflows
04	IV — Authoritative operations and progression	Model integration, FX, atomic transitions, delivery debt, XTDB basis and scoped progression, command/execution identity
05	V — Provisional state and executable presentation	Optimism, continuity, views, portable execution, ExecutablePlan and diagnostic sidecars
06	VI — Browser realization and recovery	Adapter refinement, owner-relative event admission, acquisition coordination, stale plans, managed effects, absence, failure and convergence
07	VII — Whole-application construction	ApplicationAssembly, closure, artifacts, routes, capabilities, settlement/acquisition, static and dynamic preflight, Biff lifecycle, diagnostics
08	VIII — HumanHelp proving application	Production model ownership, example integration, lifecycle, creation, adoption, board behavior, product decisions and independent acceptance
09	IX — Compositional certification	Semantic interfaces, typed rules, certificate hierarchy, dependency identity, proof kernel, mutation corpus, HumanHelp proof pilot
10	X — Formal correspondence and system boundaries	Assurance targets, theorem families, formalization, production correspondence, TCB, validation, multi-node semantics
11	XI — Release and completion	Incremental builds, release evidence, non-goals, progress measures and the four completion scopes
12	XII — Implementation order and final invariants	Ordered stages, dependency relationships, cross-stage acceptance and final unified principle

Only the beginning of published Part I carries the document-wide Org title, date, options, and revision metadata. Subsequent published parts begin with their top-level part heading and use subordinate headings within that part. Editorial source sections preserve their existing heading depth; they are joined before the twelve published parts are assembled. Concatenation must not introduce repeated headers, include directives, or independent normative fragments. A generated published part must match its reviewed source sections.

Internal links may refer forward to sections in later parts. Such references are resolved against the complete assembly. Every referenced anchor must occur exactly once in that assembly. Historical Part references are replaced by the corresponding R5 section links or current part names during revision.

Cross-cutting requirement references

The following links are a navigation aid, not a second semantic registry. The linked sections are the authoritative homes of the corresponding requirements.

Requirement / decision	Authoritative section
Assurance classification	Assurance model
Full-language construction	Full-language application logic and checked construction
Admission and proof results	Admission policy and unresolved proof obligations
Backend assumptions	Backend contracts and qualification
Canonical transition construction	Canonical semantic construction and controlled effects
Semantic category and admission	Semantic category, representation, and admission
Trusted-code boundary	Trusted application code and accidental misuse
Observation evidence	Observation claims, receipts, and authority
Progression scope	Progression requirements are scoped
Advisory discharge	Advisory obligations and authoritative acquisition
Participant-relative command uncertainty	Participant-relative command uncertainty
Provisional resolution	Provisional resolution and local retirement
Scoped authoritative installation	Scoped authoritative installation
Application assurance claims	Three separate application assurance claims
Assembly construction phases	Assembly construction and admitted generations
Production-path enforcement	Production-path enforcement evidence
HumanHelp selective adoption	Selective board adoption
HumanHelp Refresh coverage	Refresh coverage and concurrent activity
HumanHelp remote lifecycle policy	Remote lifecycle changes

Purpose

This document specifies the intended end-state design of Gesso choreography, proof, execution, application assembly, and its use by HumanHelp.

The goal is not merely to provide a convenient syntax for distributed state machines. Gesso should provide a useful correctness boundary for distributed application operations. It should make coherent programs straightforward to express, malformed programs difficult to assemble, and contradictions visible as early and as locally as the available information permits.

The central design target is:

> A Gesso application may still express a coherent but domain-wrong business rule, but it should be difficult to assemble a structurally contradictory distributed program that appears to work.

Gesso therefore distinguishes two very different failure classes.

A **well-formed but domain-wrong** program may express the wrong policy while remaining internally coherent. For example, an application may validly encode a rule that an Organization chose incorrectly. Gesso cannot infer product intent merely from syntax.

A **malformed** program contradicts its own declared semantic structure. Examples include:

- a role consuming a fact it cannot know;
- a browser affordance naming an operation that has no projected realization;
- an operation requiring authority that no trusted server path can establish;
- a provisional trajectory with no permitted resolution or recovery path;
- a projected plan whose required semantic dependency no longer matches the certified dependency used to produce it;
- a lifecycle operation silently degrading to a weaker ordinary request because required semantic metadata is absent;
- a proof or diagnostic artifact being accepted as runtime authority;
- a stale callback mutating a newer execution merely because both refer to the same logical DOM target.

Those are framework-recognizable contradictions and must be rejected when the applicable supported checker has the information needed to establish them. Diagnostics must identify the violated obligation and its owning boundary.

A computation being outside the supported proof fragment is a different matter. It is not thereby malformed or prohibited. Ordinary Clojure remains available within checked construction, with precisely stated interfaces, effects, and assumptions. The framework must distinguish an invalid construction from an unproved proposition and from an explicitly accepted asserted contract.

The distinction is easier to see in a small but nontrivial application slice. Suppose an expense system models submitted expenses and a current approval policy. The following code is illustrative rather than a frozen public API, but it shows the amount of semantic structure Gesso should be able to reason about without requiring the application author to reproduce framework internals.

```
(def Money
  [:int
   {:min 0
    :max 9007199254740991
    :semantic/type :money/usd-cents}])

(def Expense
  [:map
   [:expense/id uuid?]
   [:expense/employee-id uuid?]
   [:expense/department keyword?]
   [:expense/amount Money]
   [:expense/status
    [:enum :draft
           :submitted
           :approved
           :rejected
           :needs-finance]]])

(def ApprovalPolicy
  [:map
   [:policy/department keyword?]
   [:policy/manager-limit Money]
   [:policy/finance-threshold Money]])

(defrule manager-may-approve?
  {:inputs {:expense Expense
            :policy ApprovalPolicy}
   :returns :boolean}
  [:and
   [:= [:expense/status :expense] :submitted]
   [:= [:expense/department :expense]
       [:policy/department :policy]]
   [:< [:expense/amount :expense]
       [:policy/manager-limit :policy]]])

(defrule needs-finance?
  {:inputs {:expense Expense
            :policy ApprovalPolicy}
   :returns :boolean}
  [:and
   [:= [:expense/status :expense] :submitted]
   [:≥ [:expense/amount :expense]
       [:policy/finance-threshold :policy]]])

;; Suppose this is what the Organization actually intends.
(def accounting-policy
  {:policy/department :accounting
   :policy/manager-limit 50000
   :policy/finance-threshold 50000})
```

This introductory `defrule` form, with expressions such as `[:expense/status :expense]`, is **schematic application-authoring notation**, not an expression admitted by Part IX's closed `[:read reader-id]` rule grammar. The baseline Part IX grammar also does not directly admit an arbitrary `Money` nominal value or infer its semantics from a Malli metadata key. A supported frontend must validate the source interfaces and provide a checked elaboration into the admitted typed-rule/reader semantics, with any required nominal codec or registered extension. Otherwise a selected checked-rule profile rejects the unsupported form or retains an expressly permitted opaque contract with its premises; it must not pretend to type-check or execute an unknown operator as part of the closed proof language. No specific `defrule` constructor or elaborator is asserted to exist by this example.

Here `Money` **illustrates** an intended nominal semantic type backed by nonnegative, exactly represented integer USD cents. The

schema and rule declarations must cross their construction checks before downstream code treats them as admitted semantic interfaces; a metadata key or a Var name does not itself admit them. The maximum is part of the example's intended portable numeric profile. Arithmetic must preserve that bound or return a declared failure; a JavaScript floating-point approximation must not silently replace exact monetary semantics. A different currency or decimal profile needs its own declared encoding and operations. Malli shape validation alone does not establish nominal semantic identity.

Now imagine that the application author accidentally changes only the policy data:

```
(def accounting-policy
  {:policy/department :accounting
   :policy/manager-limit 500000
   :policy/finance-threshold 500000})
```

The program may still be perfectly coherent. `500000` is a valid amount in this USD-cent representation: $5,000 rather than $500. The rule is well typed. The operation may project correctly. The authoritative transaction may apply the rule exactly as declared. Gesso has no basis for inventing the unstated product intention that the limit should have been `50000` cents.

Contrast that with a malformed rule whose own semantic structure is contradictory:

```
(defrule malformed-manager-rule
  {:inputs {:expense Expense
            :policy ApprovalPolicy}
   :returns :boolean}
  [:and
   ;; A status enum is not an ordered monetary value.
   [:< [:expense/status :expense]
       [:policy/manager-limit :policy]]

   ;; A money value is not a status variant.
   [:= [:expense/amount :expense]
       :submitted]])
```

Once a supported frontend has admitted the nominal types and **checked** an elaboration of these schematic expressions to Part IX's typed reader grammar, the checker has enough information to reject their incompatible operand categories before the rule enters choreography or a model operation. If elaboration is unsupported, the correct first result is an unsupported-source diagnostic, not a fabricated successful type check or a claimed proof of these particular operand mismatches. When the typed terms actually have been admitted, an illustrative diagnostic can identify both the location and the expected semantic categories:

```
{:error/type :gesso.rule/type-error
 :rule       'malformed-manager-rule
 :at         [:and 0]
 :operator   :<
 :left       {:type [:enum :draft :submitted :approved
                     :rejected :needs-finance]
              :source [:expense/status :expense]}
 :right      {:type :money/usd-cents
              :source [:policy/manager-limit :policy]}
 :required   {:operator :<
              :operands [:ordered :ordered]}}

{:error/type :gesso.rule/type-error
 :rule       'malformed-manager-rule
 :at         [:and 1]
 :operator   :=
 :left       {:type :money/usd-cents
              :source [:expense/amount :expense]}
 :right      {:type [:enum-value :submitted]}
 :required   {:operator :=
              :operands [:same-semantic-category
                         :same-semantic-category]}}
```

The diagnostic map shapes are illustrative output, not the public error API or independent evidence that a real checker ran. The distinction is the foundational one this document relies on everywhere else: the first application can be wrong because humans chose the wrong coherent policy. The second is malformed under the intended typed semantics because its own declarations do not compose. Admission of its concrete authoring syntax and proof of the checker implementation remain separate obligations. Later examples apply the same distinction to knowledge, authority, browser ownership, application assembly, and proof evidence.

The choreography system should provide the following broad properties:

- one global choreography describes an allowed distributed interaction;
- the global choreography crosses checked construction before execution, with the supported semantic judgments established and accepted assumptions retained;
- the choreography is projected into actor-local machines;
- local machines execute independently in Clojure and ClojureScript;
- actors acquire justified knowledge through declared provenance paths such as inputs, communication, authoritative observation, deterministic derivation, and explicit trusted assertion;
- authoritative model transitions remain atomic at their model transaction boundaries;
- clients may hold explicit provisional state without confusing it with authoritative state;
- actors may temporarily observe different frontiers of an evolving world;
- where convergence is required, the protocol provides a path toward justified knowledge of the relevant authoritative facts;
- durable application facts normally rendezvous through model authority rather than requiring direct client-to-client delivery;
- browser-local concerns such as continuity remain implementation concerns rather than becoming distributed truth;
- sleeping, reconnecting, stale, or incompatible browser projections have a defined recovery path rather than being assumed compatible forever;
- browser actors are treated as adversarial participants that may replace or modify every byte of client JavaScript and protocol state;
- authoritative security never depends on an honest browser projection;
- portable choreography and browser-adapter semantics are explicit transition systems suitable for machine-checked reasoning;
- the production browser adapter uses the same semantic transition model that is checked or proved rather than independently reimplementing a paper model;
- HTMX remains responsible for the ordinary declarative browser lifecycle it already owns;
- Gesso-owned browser effects remain behind a narrow explicit boundary;
- verification and projection produce one canonical executable artifact plus separate optional proof and diagnostic artifacts;
- proof or diagnostic artifacts cannot participate in runtime semantic decisions;
- whole-application assembly checks the correspondence between the semantic operation, projected plan, browser artifact, transport, trusted server capability, authoritative operation, settlement/progression, and permitted authoritative reacquisition;
- safety properties called proved identify their checked derivations or sound exhaustive/refinement arguments; asserted contracts and tested properties remain separately identified;
- tests remain essential for trusted assumptions, implementation correspondence, real browsers, external libraries, operational behavior, and application policy.

The design should make downstream application code easier to reason about even when Gesso itself becomes substantially more sophisticated.

Correctness, explicit boundaries, debuggability, strong guarantees, and clear repair paths are more important than minimizing implementation size.

Full-language application logic and checked construction

Gesso supports ordinary Clojure application logic. Functions, recursion, collections, libraries, and computations outside a decidable or supported proof fragment remain available. The checked rule language is a reasoning representation, not the boundary of the application programming language.

The normal application path is checked managed construction. An application does not move to an unmanaged second architecture because one policy predicate or computation is opaque to a prover. Its operation identities, contracts, authority boundaries, effects, settlement, and recovery still compose through the same admission system.

Full-language support is relative to the declared execution target. A JVM-specific Clojure function does not become browser-executable merely because its interface can be serialized. Server logic may remain behind a public model operation; a portable or browser-local realization must separately satisfy its target contract. Projection preserves the declared placement and communication boundaries rather than pretending to translate arbitrary host code.

An opaque implementation exports a semantic contract. The contract states the input and output meanings, relevant preconditions and postconditions, permitted effects, and the assumptions on which dependent reasoning relies. Claims of purity, determinism, termination, or bounded resource use are required only when the enclosing guarantee depends on them. Their evidence must then be explicit. No such property follows merely from the function's name or declared result type.

There are three complementary ways to establish a needed property:

- derive it from supported semantic definitions and checked inference rules;
- enforce an instance-specific premise at the appropriate runtime boundary;
- accept a named asserted contract under the applicable admission policy and retain it in every dependent claim.

These mechanisms may be combined within one operation. An opaque business predicate need not prevent proof that a transition preserves an unrelated field, that a command identifier survives retries, or that a retired execution cannot mutate a replacement owner. A conclusion that depends on that predicate implementing the intended policy remains conditional on its contract.

Ordinary Clojure behind an explicit semantic interface

The following sample Organization policy requires the helper to hold the skills listed for the Request's category and requires claiming to be enabled. This is an illustrative policy for the example, not a mandatory HumanHelp product rule. HumanHelp's actual policy remains application-owned.

```
(defn organization-allows?
  [{:keys [organization membership request]}]
  (let [policy (:organization/policy organization)
        required (get-in policy
                         [:skills-by-category (:request/category request)]
                         #{})
        held (set (:membership/skills membership))]
    (and (= :enabled (:claim policy))
         (every? #(contains? held %) required))))

(def organization-claim-policy-contract
  {:contract/id :organization/claim-policy
   :inputs {:organization Organization
            :membership Membership
            :request Request}
   :returns :boolean
   :state-inputs #{:organization/policy
                   :membership/skills
                   :request/category}
   :effects #{}
   :meaning
   {:true-iff [:and
               [:policy/claim-enabled]
               [:membership/holds-required-category-skills]]}})

(def organization-claim-policy-binding
  {:contract organization-claim-policy-contract
   :implementation #'organization-allows?
   :assertions
   [{:id :organization/policy-implementation-contract
     :proposition [:implementation/refines-contract
                   :organization/claim-policy]
     :owner :humanhelp/model-maintainer}
    {:id :organization/policy-effect-contract
     :proposition [:implementation/has-no-undeclared-effects
                   :organization/claim-policy]
     :owner :humanhelp/model-maintainer}]})
```

The two symbolic policy clauses refer to the displayed policy meaning: claiming is enabled, and every required category skill is held. The contract vocabulary and binding syntax are illustrative. Admission must resolve each symbol and schema, validate its interface, and bind the assertions to the exact contract, implementation realization, and applicability scope. Merely placing an assertion map next to a function is not admission.

The framework can check declared argument/result categories and the surrounding operation's use of them. It can evaluate the function on bound current inputs. It must not label the implementation's policy correctness or lack of hidden effects proved solely because those declarations are well formed.

For this effect-free contract, the declared state inputs determine which authoritative facts must be protected when its result is used to authorize a new commit. Data obtained before submission is suitable only when the commit mechanism validates all relevant dependencies or recomputes the decision under an equivalent atomic contract. A caller assertion that a pre-read is current does not close that race.

An effectful computation instead declares its effects and their realization boundaries. External service calls, clocks, randomness, mutable process state, and asynchronous completion must not be hidden inside a contract claiming deterministic evaluation over supplied values. Full-language support permits these operations; their semantics and assumptions remain explicit.

A small authoring surface and a richer internal representation

Application authors provide domain meaning and genuine implementation choices. Gesso derives the information it can derive soundly, including references, footprints for supported expressions, obligations, and evidence dependencies. Generated artifacts must be independently validated at their required boundary.

For opaque code, a declared effect or read footprint is an explicit contract, not automatically an exact static analysis result. The assembly must retain the evidence or assumption that connects it to the implementation. Where a managed capability or candidate validator enforces a narrower property, that property can have stronger evidence without upgrading the rest of the contract.

Ordinary authors must not have to write theorem-prover languages, proof DAGs, transitive certificate maps, or hand-maintained copies of derivable metadata. Proof-producing elaboration and dedicated checking belong inside Gesso. Explicit data and ordinary functions are the canonical authoring basis; helper macros may improve ergonomics but must not define a competing semantic system.

Design priority and dependency direction

Gesso is organized around a strict distinction between semantic truth, distributed coordination, physical realization, and evidence about that realization.

At the highest level:

Layer	Responsibility
Domain model	Own business meaning and authoritative state transitions
Public model operations	Expose admitted operations and their contracts
Global choreography	Coordinate operations and participant knowledge
Role projections	Derive actor-local machines from the choreography
Executable plans	Carry canonical executable semantics and identities
Runtime realization	Perform declared effects through adapters and transport

Evidence flows alongside this path:

Declarations export normalized semantic interfaces. Checking derives and validates their obligations. Certificates, proof sidecars, and explanations record the resulting judgments, evidence, and premises.

Evidence may justify acceptance of an executable artifact, but evidence does not become an executable authority channel.

That distinction separates **admission evidence** from **runtime authority**.

A certificate, derivation, proof sidecar, exact artifact digest, preflight report, or other assurance artifact may establish that an `ExecutablePlan` was constructed from the declared semantics, depends on the expected semantic interfaces, satisfies the checks claimed for it, and is therefore eligible to run. Once admitted, however, that evidence does not grant the running program new principals, permissions, authoritative facts, capabilities, branch discriminators, or current authorization.

For example, a certificate may establish:

> A new incorporation of this Claim command can occur only through a declared path requiring its current claim-authority premise. A compatible replay reaches its recorded result through a separately declared disclosure-authority check.

The certificate does not establish:

> This particular HTTP request is currently made by principal P and P is currently authorized to claim Request R.

Those concrete premises still come from trusted authentication, current model state, runtime authorization, and other declared runtime boundaries.

A fuller HumanHelp Claim slice makes that separation visible in code. The operation is defined once and reused by later assembly and view examples. The declaration syntax is illustrative; its distinctions are required.

```
(def ClaimInput
  [:map [:request/id uuid?]])

(def claimable-rule
  {:rule/id :request/new-claim-allowed
   :inputs {:principal Principal
            :arguments ClaimInput
            :request Request
            :membership Membership
            :organization Organization}
   :returns :boolean
   :expression
   [:and
    [:= [:request/id :request] [:request/id :arguments]]
    [:= [:membership/user-id :membership] [:user/id :principal]]
    [:= [:membership/organization-id :membership]
        [:request/organization-id :request]]
    [:= [:organization/id :organization]
        [:request/organization-id :request]]
```

```
    [:= [:request/status :request] :open]
    [:= [:membership/status :membership] :active]
    [:no-conflicting-active-primary-assignment :request]
    [:call :organization/claim-policy
     {:organization :organization
      :membership :membership
      :request :request}]]})

(def claim-request
  {:operation/id :request/claim
   :actor :helper
   :input ClaimInput
   :entry-knowledge [[:knows? :helper [:request/id]]]

   :invocation
   {:requires [:authenticated-principal :admitted-command-binding]
    :model-operation :request/claim}

   :new-incorporation
   {:guard claimable-rule
    :policy-binding organization-claim-policy-binding
    :transition
    {:request {:status :claimed}
     :primary-assignment
     {:effect :establish-active-primary
      :request [:request/id :arguments]
      :helper :authenticated-principal}
     :preserve [:unrelated-request-fields
                :unrelated-assignment-facts]}
    :atomic-with [:command/result-record
                  :required-durable-publication-intent]

   :repetition
   {:binding [:operation :semantic-arguments :principal :authority-scope]
    :requires :command/result-disclosure-authority
    :result :permitted-recorded-outcome
    :incorporate-again? false}

   :effect
   [:model/invoke :request/claim
    {:request-id [:request/id :arguments]
     :principal :authenticated-principal
     :command-id :gesso/command-id}]

   :provisional
   {:view :request/claiming
    :owner :gesso/execution-id
    :correlate :gesso/command-id}

   :settlement
   {:accepted
    {:learns [:command/recorded-success :gesso/command-id]
     :reacquire [:request/card [:request/id :arguments]]}
    :rejected
    {:learns [:command/typed-rejection :gesso/command-id]
     :reacquire [:request/card [:request/id :arguments]]}
    :unknown
    {:recover [:request/reconcile-claim
               {:request-id [:request/id :arguments]
```

```
               :command-id :gesso/command-id}]}}})
```

`:actor` describes a semantic participant position; it does not authenticate a browser. Schema names denote admitted domain interfaces. Their relationship fields are shown explicitly so that a same-shaped Membership or Organization cannot be substituted merely because it is active or has a permissive policy. An implementation may establish these bindings through checked acquisition contracts, explicit guards, or both; the obligation is not optional.

This declaration is a **semantic sketch**, not a parsed Part IX rule or an available Gesso constructor. In particular, its field-path forms, the `:no-conflicting-active-primary-assignment` condition, and `[:call ...]` are not operators in Part IX's closed reader-based rule grammar. To make a typed-rule claim, Gesso must use that grammar with a supported, checked elaboration from the source meaning. Otherwise those obligations remain explicit opaque contracts with their applicable implementation premises. Neither a familiar-looking vector nor arbitrary host evaluation can substitute for rule checking. The illustration does not assert that such an elaborator already exists.

The Organization policy call represents the contract defined above. Its interface and result category can be checked, but policy correctness and effect behavior remain conditional on its named implementation assertions unless stronger corresponding evidence is supplied.

This is the self-Claim variant: the trusted actor is also the intended helper. Reassign and manager-assisted Claim require separate actor/target-helper authorization and eligible Membership conditions. Real helper responsibility belongs to the production RequestAssignment model, not a fabricated `:request/helper` field. The transition sketch describes a Request status change and establishment of an active primary assignment under the appropriate relationship and absence/uniqueness guards. It does not define physical RequestAssignment fields or XTDB lowering. Those belong to the public production model and its qualified authoritative realization, with a complete frame and durable publication intent.

The guard belongs to new incorporation. It is not a precondition for recognizing an already incorporated command. Otherwise a successful retry would be rejected because its own first incorporation changed the Request from open to claimed. Disclosure of a recorded result has its own current authorization policy; it must not be confused with permission to perform a new claim.

Verification of the declaration produces claim-specific evidence. For example:

Proposition	Required evidence and scope
The new-claim guard is well typed	Dedicated semantic checking over its declared inpu
New incorporation requires the guard and excludes a competing primary assignment	A checked reachability/transition argument over th
Role projection does not create Claim authority	A checked distributed-admissibility/projection judg
Compatible replay does not incorporate again	A checked command-protocol argument, condition
Replay disclosure requires its own authority	Checked semantic requirement plus concrete runti
The declaration covers accepted, rejected, and unknown settlements	Exhaustive checking over that finite declared outco
The policy implementation realizes its contract	The named assertion unless stronger correspondin

These are separate judgments. A certificate may collect them only with their actual evidence, scope, and premises. Merely emitting the table as a map does not establish any row. A rule checker cannot decide whether Alice's future Membership is active, and a plan proof cannot by itself establish that an independently written handler enforces its declared guard.

A trusted invocation boundary binds the concrete caller and passes the command to the public authoritative model operation:

```
(defn invoke-claim!
  [{:keys [session now] :as ctx}
   {:request/keys [id] :gesso/keys [command-id]}]
  (let [principal (auth/current-principal! session)]
    ;; The admitted model binding validates the command envelope and owns
    ;; replay, current disclosure permission, and new-incorporation guards.
    (request-model/claim!
      (assoc ctx :biff.fx/now now :principal principal)
      {:request-id id :command-id command-id})))
```

The context is supplied through the admitted server binding. The principal does not come from the command body. The public model operation must establish the relationship between the principal and the Membership it uses, rather than trust a browser-supplied helper identifier.

Claim command identity, replay, and uncertainty

Before the command's first possible submission, its command boundary allocates a stable `:gesso/command-id`. Its allocation may be local or use an earlier trusted issuance protocol. Allocating the bytes is separate from establishing their trusted binding. At the authoritative boundary, authentication and command admission establish or validate the operation, canonical semantic arguments, principal, and authority scope to which the identity is bound. The binding must be compatible with the admitted operation version. Browser-carried bytes alone confer no authority.

A retry preserves that command identity. A new transport attempt has a new transport-request identity, and a new presentation execution has its own execution and resource-instance identities. Neither changes which command is being reconciled.

The public `request-model/claim!` operation checks for a compatible recorded command before interpreting the Request as a new claim. It validates the recorded binding and current permission to disclose the result. A permitted successful replay returns the recorded success without another transition, including when the Request is now claimed or has subsequently changed again. Incompatible identity reuse is rejected without disclosing another principal's record or outcome.

For new incorporation, the Request identity and state, the principal's active Membership, the Organization identity and policy, the absence of a conflicting active primary assignment, and every relevant relationship or predicate dependency are protected by the same atomic contract as the Request/assignment changes, successful command-result record, and required durable publication intent. A pre-read may prepare a candidate; it cannot replace commit-time protection of those facts.

Two attempts can both initially observe no command record. If one commits and the other's guard fails, the failing attempt must use the command protocol to determine whether the same command was incorporated concurrently. It must not report a fresh domain rejection merely because its attempt lost a race. Similarly, absence observed at one read basis does not establish that an outstanding attempt cannot later commit. Part IV owns the authoritative resolution, retry, retention, and terminal-rejection contracts.

Part IV distinguishes model outcomes `:committed`, `:rejected`, `:already-incorporated`, and `:commit-unknown`. This example presents `:accepted`, `:rejected`, or `:unknown` to the participant:

Model outcome	Presentation settlement
`:committed`	`:accepted` when permitted successful command evidence is delivered
`:rejected`	`:rejected` when an authoritative typed rejection for the applicable command scope is delivered
`:already-incorporated`	The permitted recorded result; recorded success maps to `:accepted` without a second transition
`:commit-unknown`	`:unknown` until authorized outcome evidence resolves it

A delivery failure may leave the participant uncertain even after a commit. Conversely, accepted command evidence establishes that the command succeeded at its recorded point; it does not establish that the Request is still owned by that helper at every later observation. Current presentation comes from an authorized acquisition with its own basis and scope. Reading a currently claimed Request does not by itself identify the command that claimed it.

The assembly checks these mappings and the required acquisition/reconciliation paths. Acquisitions still enforce current read permission and must represent denial, absence, or an unavailable dependency under their declared contracts. Neither accepted nor rejected settlement grants a new right to read the card.

An admitted Claim plan may have strong structural and transition guarantees while remaining conditional on a policy assertion, an authentication boundary, or a qualified backend contract. Production correspondence must establish which realization enforces those requirements. A certificate cannot manufacture the principal, Membership, Request state, Organization policy, or current authority. Sending it back from the browser adds no authority.

The two decisions remain separate:

Stage	Basis of the decision
Artifact admission	Appropriate proof, checking, and preflight evidence under the admission policy
Concrete execution	The admitted runtime semantics and current trusted premises
Authority decision	The applicable principal, resource, policy, scope, and current state

The following inference is forbidden:

> A certificate says ”valid”, so the runtime grants authority, accepts a current fact, or selects a semantic branch without establishing its required premises.

This applies more broadly than authorization. Assurance evidence must not secretly decide which role the current user has, which principal is authenticated, whether a current authoritative fact is true, whether a stale basis is acceptable, which settlement is canonical, or what transaction should commit.

A separately declared runtime credential, observation receipt, or other evidence protocol may supply a runtime premise only through its own trusted validation contract, with its subject, scope, applicability, and authorization meaning made explicit. That is a distinct runtime input category. It does not turn a proof sidecar, build report, or matching artifact hash into authority.

Proof and diagnostic sidecars are therefore runtime-nonsemantic. Running the same `ExecutablePlan` with or without a matching read-only sidecar must produce the same semantic transitions and abstract effects, modulo diagnostics. Evidence explains or justifies why an executable was accepted; it does not become a bearer capability or an alternate semantic control path.

The dependency direction is intentional:

- application business meaning belongs in application models;
- choreography coordinates use of public model semantics;
- projection derives local behavior from the global choreography;
- runtime code executes projected behavior;
- proof/checking artifacts explain why construction was accepted;
- physical browser, network, database, and process mechanisms realize declared effects under explicit assumptions.

A lower layer must not acquire higher-level meaning merely because doing so would make an implementation convenient. For example:

- a DOM node does not become a semantic Request merely because it has an id;
- a web node does not become Request authority merely because it handles the request;
- a browser-supplied principal id does not become authenticated identity because it has the expected shape;
- a map with `:verified? true` does not become a verified choreography;
- a matching hash does not establish the truth of the proposition represented by the hashed artifact;
- a Live invalidation does not establish the changed authoritative fact;
- a rendered button does not confer authorization to execute its operation.

Normative language

This design uses the following terms deliberately.

- **MUST** or **must** identifies a correctness requirement of the intended design.
- **MUST NOT** or **must not** identifies behavior that would violate the intended correctness boundary.
- **SHOULD** or **should** identifies the ordinary design path. Deviations require a concrete reason and must preserve the relevant invariants.
- **MAY** or **may** identifies an implementation freedom that does not change the semantic contract.
- **proved** is reserved for mechanically established properties under named assumptions.
- **verified** may also describe a mechanically checked closed construction whose result is not claimed as a theorem over an explicitly formalized semantic relation. The result classification must remain precise.
- **trusted** does not mean ”probably correct.” It means the proposition is outside the proof currently being claimed and is accepted as an explicit assumption.

A green test suite never silently upgrades a tested property into a proved property.

A successful schema validation never silently upgrades a value into trusted authority.

A valid certificate identity never silently upgrades the certificate's claims into truth unless an appropriate checker has accepted the certificate under the premises and evidence classification attached to those claims.

Interpretation of examples and implementation freedom

Examples illustrate the enclosing contract. Their concrete syntax is illustrative unless the owning section explicitly defines it as the required public or wire representation. Their stated required behavior must still obey the normative invariants.

Examples using symbolic operators must identify their semantic contract and whether they denote checked expressions, runtime premises, opaque calls, or inspection output. Unknown operators and unresolved references are construction errors; illustrative syntax is not permission to accept arbitrary forms.

An abbreviated diagram does not waive an authorization, admission, scope, ownership, or outcome-attribution condition merely because the diagram omits that step. Worked examples must identify consequential policy choices instead of silently selecting behavior that the specification leaves open.

The implementation may choose a representation or mechanism where the design allows alternatives. It must state which supported contract was chosen and provide evidence appropriate to the resulting claim. Implementation freedom does not permit silent weakening of a required capability or assurance class.

A contradiction between normative passages is a specification defect to be reconciled at the owning section. It is not permission to select whichever passage makes an implementation easier to accept.

A declared recovery path establishes available modeled behavior. It does not by itself establish that the environment will cause that path to execute or that every execution terminates. Liveness claims retain their named premises.

Correctness scopes

The system intentionally combines several correctness scopes. No one scope subsumes the others.

Domain correctness

Application models define domain meaning.

Examples include:

- whether a Request may be claimed;
- whether a Membership is active;
- whether an Organization permits an operation;
- what a ”reconciled” claim outcome means;
- which facts must be protected atomically at commit time.

Gesso may check that a rule expression is well typed, that a declared operation contract composes with its dependencies, and that the distributed execution respects that contract. Gesso does not infer whether the Organization chose the right business policy.

Choreographic correctness

Choreo defines and checks distributed coordination around public semantic operations.

It reasons about:

- participating roles;
- principal and authority requirements;
- knowledge;
- provenance;
- communication;
- authoritative observation;
- branch discriminators;
- provisional state;
- command and execution identity;
- settlement;
- resource ownership;
- convergence;
- failure and recovery;
- projection into role-local behavior.

Runtime correctness

Runtime code realizes the checked semantics.

It must preserve:

- semantic identities;
- execution-instance identity and declared resource-instance ownership;
- stale-work rejection;
- correlation;
- progression requirements;
- resource lifetimes;
- provisional-versus-authoritative distinctions;
- permitted machine transitions;
- transport and server realization.

Whole-application closure

Individually valid components can still form an impossible application.

The assembled application therefore has its own closure boundary. For an authoritative browser command, a rendered semantic affordance must have a coherent path through the applicable application boundaries:

Closure step	Required relationship
Affordance	Resolves to the declared semantic operation and valid arguments
Choreography	Supplies an admitted trajectory for that operation
Projection	Produces the required actor-local plan
Browser artifact	Realizes that plan under the admitted compatibility contract
Transport	Reaches the declared trusted server capability
Model binding	Invokes the public authoritative operation with bound inputs
Outcome handling	Preserves typed settlement and progression obligations
Acquisition	Obtains a permitted authoritative successor under current read authority
Presentation	Renders that successor through the admitted ownership and lifecycle rules

This is the path for the stated browser-command case. A read-only operation, subscription, background task, or workflow with no browser presentation has its own declared effects, outcomes, and lifecycle; it need not manufacture a browser plan, domain write, or card reacquisition. Closure checks follow the obligations of the declared operation kind.

Later parts of this design specify that closure in detail. Its inclusion here establishes an important foundational rule: local correctness does not imply whole-application correctness.

Three distinct application assurance claims

Whole-application assurance separates three claims that are related but not interchangeable.

Claim	What it establishes	What it does not establish
Declared closure	Required relationships among the application elements within the declared assembly scope are checked	That every intended product capability was declared
Production enforcement	Actual supported execution passes through the boundaries that make the declared checks effective	That every possible host-language escape hatch is impossible
Behavioral completeness	Required user capabilities and authoritative outcomes are present and usable	That their internal assembly is structurally closed unless closure is checked separately

A self-consistent assembly can omit intended functionality. Conversely, a behaviorally successful path can exist while bypassing a checker that the design claims is mandatory. The three claims therefore reinforce one another without collapsing into one another.

Assembly closure is always relative to declared scope. Gesso cannot infer an application author's missing business intention merely because the remaining graph is closed. Required product behavior therefore needs an acceptance basis independent of the assembly's own declaration set. That acceptance basis should state user capabilities and authoritative outcomes rather than duplicate route strings, operation registries, projected-plan identities, or other realization metadata already owned by the assembly.

Production enforcement is a separate obligation. A checker that correctly rejects an isolated malformed fixture provides no application guarantee when the real application path can bypass that checker. Strong claims about an assembled application must therefore identify the production boundary that makes the relevant check effective.

Inside trusted host-language code, deliberate escape hatches may remain possible. For example, trusted application code that intentionally obtains a lower-level transaction capability may bypass the model/Gesso path. The ordinary assurance goal inside that trusted boundary is misuse prevention: supported application construction should make accidental semantic bypass difficult, explicit, and diagnosable. A deliberate departure from that supported path requires an explicit boundary and assurance scope. A raw writer can invalidate dependent invariants for later operations over the affected state, not just the assurance of its own request. The dependency graph must represent that impact. Ordinary opaque computation inside a managed contract is not itself such a departure.

These distinctions are foundational for later HumanHelp acceptance:

```
assembly closed
    ≠
production path demonstrably enforced
    ≠
all required HumanHelp capabilities present
```

One Claim implementation can be used to see all three application-level claims without inventing three unrelated examples.

The author supplies operation declarations and independent realization choices. The following source is an excerpt of an application declaration; named view, acquisition, recovery, and artifact providers must be supplied by the surrounding application. Admission rejects unresolved providers.

```
(def humanhelp-application
  {:assembly/id :humanhelp/web
   :operations [claim-request]
   :views [:request/card]
   :realizations
   {:request/claim
    {:model-implementation #'request-model/claim!
     :transport {:kind :http/post :path "/requests/claim"}}}
   :providers
   {:request/card request-card-provider
    :request/reconcile-claim claim-outcome-provider
    :browser/runtime gesso-browser-artifact-provider}
   :assurance-policy :humanhelp/managed-application})

(def humanhelp-assembly
  (assembly/prepare humanhelp-application))
```

The operation supplies its semantic identity, input, settlement, and recovery requirements. The realization supplies a concrete model implementation and a transport choice. Framework-owned plan, route, and capability identities are derived and resolved; the author does not reproduce them in several registries. The policy reference resolves to the application's declared admission policy.

Inspection can expose the expanded graph without making that graph a second source of truth:

```
(assembly/inspect humanhelp-assembly)
;; illustrative derived output, not an author-maintained input:
{:assembly/id :humanhelp/web
 :operation :request/claim
 :edges
 [[:request/card :claim :request/claim]
  [:request/claim :plan :plan/request-claim]
  [:plan/request-claim :artifact :artifact/gesso-live]
  [:request/claim :transport :transport/trusted-post]
  [:transport/trusted-post :route :route/request-claim]
  [:route/request-claim :capability :server/request-claim]
  [:server/request-claim :model-operation :request/claim]
  [:request/claim :settlement :request/claim-settlement]
  [:request/claim-settlement :acquisition :request/card]
  [:request/claim-settlement :recovery :request/reconcile-claim]]}
```

A closure checker establishes relationships over the resolved sources:

```
(assembly/check humanhelp-assembly)
;; ⇒
{:result :accepted
 :evidence :assembly-verified
 :closed-edges
 [[:request/card :claim :request/claim]
  [:request/claim :choreography :request/claim]
  [:request/claim :plan/request-claim]
  [:plan/request-claim :artifact/gesso-live]
  [:request/claim :transport/trusted-post]
  [:transport/trusted-post :route/request-claim]
  [:request/claim :route/request-claim]
  [:route/request-claim :server/request-claim]
  [:server/request-claim :request/claim]
  [:request/claim :request/claim-settlement]
  [:request/claim-settlement :request/card]
  [:request/claim-settlement :request/reconcile-claim]]}
```

The checker must collect these entries from the actual admitted choreography, projected plan, artifact bytes, route table, server binding, model operation, settlement, and acquisition/recovery declarations. Copying plausible entries into a disconnected map would show only that the map agrees with itself. The accepted report is illustrative of a checker that resolves every referenced source and validates the recorded edges. In this example, `:result :accepted` reports only that the closure judgment succeeded for its checked subject. It is not an admission handle, evidence of current authorization, or proof that a serving host installed the checked realization. The managed issuer and actual host-adoption boundary have separate obligations. A copied inspection result can satisfy neither.

That still does not establish production enforcement. The actual startup and request path must make the admitted assembly unavoidable for the claimed supported path. Conceptually, production wiring might look like:

```
(defn start-humanhelp!
  [ctx]
  (let [prepared (assembly/prepare humanhelp-application ctx)
        admitted (assembly/admit! prepared)
        handler (routes/build admitted)]
    ;; The Biff integration owns installation and the before-serving gate.
    ;; Returning a handler in an unused map is not installation evidence.
    (biff-host/start!
      ctx
      {:biff.ring/handler handler
       :before-serving
       (fn [installed-runtime]
         (assembly/require-installed! admitted installed-runtime))})))

(defn claim-handler
  [ctx request]
  (let [binding
        (assembly/require-route-binding!
          (:admitted-assembly ctx)
          :route/request-claim)]
    (server/invoke-bound-operation! ctx binding request)))
```

Now the relevant malformed production fixture is not merely passed to an isolated checker. It is sent through the same startup boundary:

```
(deftest stale-artifact-prevents-startup
  (is
    (thrown-with-msg?
      clojure.lang.ExceptionInfo
      #"browser artifact does not realize admitted plan"
      (start-humanhelp!
        (assoc test-ctx
          :browser-artifact
          artifact-without-request-claim-plan)))))
```

The Biff-host calls are an illustrative integration contract, not additional public Biff APIs. The integration must own the actual module lifecycle, final handler installation, and ordering before requests are served. Checking a detached route table or accepting an unused handler does not satisfy it.

The controlled defect supplies tested evidence for production enforcement of this boundary. A corresponding valid startup and usable Claim path must also succeed. Broader production enforcement requires the other applicable boundaries to have their own evidence.

Behavioral completeness still needs an independent statement of what the user must actually be able to do:

```
(def required-humanhelp-capabilities-excerpt
  [{:capability :request/create
    :eligible-scenario :authenticated-customer
    :required-outcome :request/created-and-visible-to-creator}

   {:capability :request/claim
    :eligible-scenario
    :authenticated-active-helper-sees-open-request
    :required-outcome
    :request/claimed-with-active-primary-assignment}

   {:capability :board/refresh
    :eligible-scenario
    :authenticated-viewer-clicks-refresh-with-no-advisory
    :required-outcome
    :authorized-current-board-observed-and-installed}

   {:capability :board/refresh
    :eligible-scenario
    :authenticated-viewer-clicks-refresh-with-pending-advisory
    :required-outcome
    :authorized-current-board-adopted-with-only-covered-debt-discharged}])
```

These entries are an excerpt for the Claim illustration, not HumanHelp's complete required-capability basis. The two Refresh scenarios cover different obligations. Explicit user permission can authorize a new scoped acquisition without an advisory; a pending advisory does not itself authorize acquisition and can be discharged only by sufficient, permitted, actually installed observation. Mere response receipt does not satisfy either adoption outcome.

The Claim result means production Request claimed status **and** the appropriate active primary RequestAssignment, not an invented helper field on Request. The outcome keywords in these illustrative product-oracle entries are neither new persisted status values nor implemented Gesso API names. The complete independent acceptance basis and its operational milestone scope are specified in Part VIII's required capabilities.

If `:request/claim` is removed from both the view and the assembly, the remaining assembly may still be perfectly closed. The independent capability list is what prevents that self-consistent omission from being called complete.

Conversely, a hand-written `POST /claim` route may make a browser acceptance test appear green while bypassing `assembly/admit!` and the declared Claim operation. Behavior happened, but production enforcement of the claimed Gesso boundary was not established.

The point of keeping all three claims separate is therefore practical rather than terminological: each one catches a different way a plausible application can ”work” while failing the assurance statement the project intends to make.

Operational correctness

Formal semantics do not eliminate operational concerns.

Real-browser tests, fault injection, property tests, load tests, lifecycle tests, and application acceptance remain necessary for:

- browser primitive assumptions;
- HTMX and EventSource/SSE behavior;
- compiler/runtime implementation correspondence;
- scheduler and timer behavior;
- memory and CPU characteristics;
- dependency outages and reconnect behavior;
- application-specific semantics;
- human-visible interaction behavior.

Required capabilities are independent of exposed capabilities

A milestone's behavioral acceptance must name the required user capabilities and authoritative outcomes independently of the set currently exposed by the implementation. Tests derived only from existing affordances cannot establish that an intended affordance was not omitted.

For each required capability, acceptance must establish an eligible scenario in which the intended interaction is usable and reaches its required outcome. Relevant ineligible scenarios must establish the specified unavailability, rejection, or recovery behavior. Marking every occurrence unavailable does not satisfy a requirement to provide a usable capability.

An explicitly narrower milestone may defer a capability. Its scope must say so; its completion must not be reported as completion of a broader milestone that requires that capability. This specification distinguishes such deferral from unavailability caused by current production model policy.

The independent acceptance basis owns user capabilities and outcomes. It must not become a second registry of route strings, generated plan identities, transport metadata, or model implementation already owned by the assembly.

Assurance model

Gesso uses several kinds of evidence, and the design must not collapse them under a single word such as `verified`, `safe`, or `proved`.

The governing question for every assurance claim is:

> What proposition is being claimed, over what object, under what assumptions, and what evidence justifies accepting it?

Every claim has separately identifiable dimensions:

Dimension	Required meaning
Proposition	The statement, variable bindings, quantifiers, and observation scope
Subject	The semantic interface and, where relevant, concrete implementation or artifact
Mechanism	Derivation checking, dedicated checking, runtime enforcement, testing, or an explicit assertion
Premises	Assumptions, dependency guarantees, and concrete premises required for application
Coverage	Supported language, bounds, abstractions, and correspondence relation
Applicability	Versions, configuration, validity conditions, and evidence that the claim still applies
Status	Established at the stated scope, accepted as a premise, open, contradicted, or inapplicable

These dimensions need not appear as one literal map in every API, but they must be recoverable in the assurance explanation. Conditionality and boundedness qualify a claim; neither supplies an evidence mechanism. Several mechanisms can support different parts of the same argument. There is no single total ranking that turns all evidence into a universal confidence score.

A claim's **disposition**, its **evidence mechanism**, an individual check's **result**, and the selected application's **admission or release decision** are separate meanings. An established judgment states what its evidence actually established within its quantification and observation scope. Its evidence may be a proof, an exact finite check, or a completed test of expressly named scenarios; sharing a claim disposition does not make those mechanisms equal. A declared enforcement requirement is not evidence that the actual installed host enforces it. A named trusted premise remains a premise even if policy accepts it. Part XI's `:passed`, `:failed`, and other closed gate results describe whether required checks completed at their scopes, not the semantic strength of the propositions or the authority of their subjects.

The assurance model distinguishes:

1. properties established mechanically from explicit semantics or checked derivations;
2. premises enforced dynamically when a concrete execution meets the external world;
3. assumptions that remain outside the mechanically discharged core;
4. whole-application closure properties checked over assembled artifacts;
5. liveness claims whose validity depends on named environmental assumptions;
6. test evidence over trusted implementation boundaries and concrete systems.

These classifications are related, but they are not interchangeable.

For example:

```
proved semantic property
≠
exact finite-state check
≠
closed application assembly
≠
runtime-enforced premise
≠
trusted assumption
≠
green integration test
```

A complete assurance argument may require several of them simultaneously.

A store checkout provides a useful worked example because one ordinary button crosses nearly every assurance class without any one class subsuming the rest.

Suppose the semantic operation is declared approximately as:

```
(def place-order
  {:operation/id :checkout/place-order
   :actor :customer
   :input [:map [:cart/id uuid?]]
   :invocation
   {:requires [:authenticated-customer :admitted-command-binding]}
   :new-incorporation
   {:guard :cart/checkout-eligible?
    :requires [:bound-current-cart]
    :atomic-with :command/result-record}
   :repetition
   {:requires :command/result-disclosure-authority
    :result :permitted-recorded-outcome}
   :provisional
   {:view :checkout/submitting
    :owner :gesso/execution-id
    :correlate :gesso/command-id}
   :effect
   [:model/invoke :order/place
    {:cart-id [:cart/id :arguments]
     :customer :authenticated-customer
     :command-id :gesso/command-id}]
   :settlement
   {:accepted
    {:learns [:command/recorded-success :gesso/command-id]
     :reacquire [:order/by-command :gesso/command-id]}
    :rejected
    {:learns [:command/typed-rejection :gesso/command-id]
     :reacquire [:cart/current [:cart/id :arguments]]}
    :unknown
    {:recover [:order/reconcile-command :gesso/command-id]}}})
```

The command envelope supplies the stable command identity. Every presentation execution and managed resource has its own applicable ownership identity. Payment authorization, if required, is a separately declared external protocol; a database transaction must not pretend to make a payment provider atomic with an order write. The operation must define which provider outcome or durable obligation permits order placement and how unknown external outcomes recover.

An assurance report during assembly may include both accepted claims and an open progress obligation:

```
{:subject
 {:assembly :store/web
  :operation :checkout/place-order
  :plan :plan/checkout-place-order
  :artifact-digest "sha256:..."}

 :claims
 [{:id :checkout/rule-well-typed
   :proposition [:rule/well-typed :cart/checkout-eligible?]
   :status :established
   :evidence {:class :semantic-checked
              :mechanism :typed-rule-checker
              :result :check/cart-checkout-rule}}

  {:id :checkout/principal-required-at-runtime
   :proposition [:runtime/requires-current-principal
                 :checkout/place-order]
   :status :open-obligation
   :required-evidence
   {:class :runtime-enforced
    :boundary :server/authenticated-checkout-handler
```

```
   :obligation :actual-installed-boundary-correspondence}
  :premises #{:authentication/correct-principal-binding}}

 {:id :checkout/payment-provider-contract
  :proposition [:provider/validated-acceptance-means
                :payment-authorized]
  :status :accepted-as-premise
  :evidence {:class :trusted
             :owner :external/payment-provider
             :assumption :payment/provider-contract-v4}}

 {:id :checkout/application-path-closed
  :proposition [:application/closed-path :checkout/place-order]
  :status :established
  :evidence {:class :assembly-verified
             :report :assembly/store-web-A12}}

 {:id :checkout/browser-realization-tested
  :proposition [:implementation-corresponds-in-scenarios
                :checkout/place-order
                #{:accepted :rejected :unknown}]
  :status :established
  :coverage {:scenarios #{:accepted :rejected :unknown}}
  :evidence {:class :tested
             :suite :chromium/checkout-lifecycle}}

 {:id :checkout/retry-eventually-recovers
  :proposition [:eventually [:checkout/settled-or-reconciled]]
  :status :open-obligation
  :environment-premises
  #{:eventual-provider-availability
    :eventual-authoritative-read
    :continued-participant-execution
    :fair-recovery-scheduling}
  :required-evidence :checked-progress-argument
  :evidence []}]}
```

Names in this report resolve to source-bound evidence or accepted premises; they are not self-authenticating flags. In particular, the authentication row is an **open physical enforcement obligation**. Naming a server boundary does not establish that an actual running handler adopted it, and neither this declaration nor a test of an isolated checker supplies a principal for a future request. Actual adoption needs its own evidence under Part VII.

The browser row's `:established` disposition concerns only the expressly listed scenarios and their completed, source-bound test evidence. Its `:tested` evidence class does not prove universal browser correspondence, and a suite name alone is not evidence that a current test ran or passed. This row would remain open if the required run or artifact binding were unavailable. The example's status labels illustrate these distinctions; they do not introduce a rival schema for Part XI release-gate results.

The last row is deliberately open. Naming environmental premises is not a progress proof. A release whose acceptance policy requires that progress argument cannot claim completion from this report. If a suitable argument is later checked, its mechanism and scope replace the open status while its environmental premises remain attached.

The report is deliberately heterogeneous. Consider what happens if each claim is misread as another:

```
;; Wrong: a type-check result does not authenticate this request.
(when (checking/accepted? :check/cart-checkout-rule)
  (treat-request-as-authenticated! request))

;; Wrong: a green Chromium test does not prove the provider contract.
(when (tests/green? :chromium/checkout-lifecycle)
  (assume-provider-cannot-lie!))

;; Wrong: assembly closure does not imply eventual provider availability.
(when (assembly/closed? :store/web)
  (claim-unconditional-termination!))

;; Wrong: the trusted provider premise does not prove the browser adapter.
(when (trusted? :payment/provider-contract-v4)
  (claim-browser-correspondence-proved!))
```

The correct composition keeps the propositions separate. A downstream release claim may depend on all of them, but the resulting explanation should still be able to say exactly which part is mechanically checked, which is dynamically enforced, which remains trusted, which is assembly evidence, which is tested, and which is conditional on future environmental behavior.

That is what the assurance vocabulary is for. It is not a taxonomy applied after the fact; it prevents evidence from silently acquiring a stronger meaning as it moves through the application.

The classification belongs to the **claim and its evidence**, not merely to the source file, namespace, subsystem, or tool that produced it. A single component may therefore participate in several assurance classes.

For example, the browser adapter may have:

- machine-checked transition invariants;
- runtime-enforced event normalization and plan-identity checks;
- trusted assumptions about DOM/browser primitives;
- Chromium integration tests providing tested evidence of implementation correspondence;
- conditional liveness claims requiring eventual event delivery.

The root assurance report must preserve these distinctions rather than reducing them to a single boolean such as `verified?`.

Nominal or otherwise admitted semantic categories participate in this model but do not add a new assurance class by themselves. For example, an admitted `VerifiedChoreography` value may be the product emitted after verification, but its assurance comes from the verifier/checker and its stated premises, not from the runtime type name. Likewise, an `AuthoritativeObservation` representation is authoritative only when the trusted acquisition path establishes that provenance.

This distinction prevents the implementation convenience of nominal typing from being mistaken for proof:

Boundary	What acceptance establishes
Representation validation	The value has the permitted shape
Semantic-category admission	Supported code may treat the value as category C at this boundary
Checked judgment or certificate	The stated proposition follows under the recorded premises and mechanism

Those stages may be represented by different types, by checked constructors, or by other existing mechanisms. Their assurance meaning remains distinct even if an implementation combines some of the physical representations.

Admission policy and unresolved proof obligations

Admission is a decision under a declared policy, not a synonym for proving every application function. The policy identifies mandatory structural checks, required evidence for particular claims, and which explicitly owned contracts may remain asserted. The normal policy supports full-language implementations while retaining their assumption dependencies.

Construction derives the obligations relevant to its declared claims and runs the supported automated checks under an explicit resource policy. Results must identify unsupported reasoning and exhausted checking budgets rather than omit the obligation. An accepted assumption must not hide an available failed check.

Mandatory identity, interface, authority-source, effect-binding, and required recovery checks cannot be disabled by adding an assertion. A named opaque functional contract may be an accepted premise where policy permits it. The result remains conditional; the premise is not marked discharged.

The checker and assembly must distinguish these outcomes:

Outcome	Meaning and consequence
Established judgment	The stated proposition follows by the identified mechanism over its stated scope
Accepted assumption	A named premise is permitted by policy and remains in dependent explanations
Unsupported proof fragment	The reasoning mechanism lacks a rule or representation; this does not establish that the program is false
Inconclusive check	A limit, cancellation, or tool failure prevented a result; no success may be inferred
Counterexample	The checked model contains a violation; report its scope and whether concrete correspondence is establishe
Malformed declaration	Required semantic structure does not compose; reject construction
Failed runtime premise	A concrete execution cannot proceed on that premise; follow declared rejection or recovery
Inapplicable evidence	The subject, scope, dependency, or backend profile no longer matches; dependent acceptance requires re-es

An unsupported or inconclusive proof obligation may be retained as an explicit assumption only by a policy that permits that exact kind of premise and an independently identified, scoped semantic or implementation contract whose truth is being assumed. An exhausted proof budget, unavailable checker, or unsupported expression is a checking outcome, not itself an assumption asserting the original proposition. The policy must identify the original obligation, selected weaker basis, affected dependent claims, and still-missing evidence; a mandatory gate cannot be satisfied by renaming its failure. The system must not silently convert a checker failure into trust. A known counterexample cannot be erased by asserting the contradicted proposition under the same claim. A correction to the specification, abstraction, implementation, or claim scope must itself be explicit and checked as applicable.

The interface must make clear which obligations block admission and which remain accepted premises. A component may be structurally admitted yet lack a particular application theorem or release qualification. The root report must not confuse these states.

Machine-checked semantic properties

A machine-checked semantic property is established mechanically from an explicit semantic model, checked derivation, exhaustive semantic state space, proof kernel, model checker, theorem prover, or another precisely identified formal mechanism.

The important requirement is not the name of the tool. The important requirement is that the claimed proposition and the relationship between the checked object and the production semantic object are explicit.

Candidate framework properties include:

- well-typed checked rule expressions;
- knowledge entailment;
- provenance entailment;
- authority derivation and non-escalation;
- message payload closure and knowledge-transfer constraints;
- behavior-sensitive branch knowledge;
- projection preservation for the supported language subset;
- declared resource invariants;
- progression algebra invariants;
- browser-adapter transition invariants;
- retired execution/resource isolation;
- semantic dependency compatibility;
- executable-plan provenance;
- certificate dependency consistency.

The individual phrases above name different judgments rather than one generic notion of ”verification”:

- **Well-typed checked rule expressions.** The baseline Part IX checker admits only its closed, reader-based expression grammar. Referenced readers must be declared with compatible nominal types; operators require compatible operand types; enum values must belong to their declared domains; and optional or nullable values require the supported presence/non-nil refinements before operations that need them. Strict Boolean evaluation and short-circuit order are part of that grammar's semantics. An embedded arbitrary function call, relationship query, or case expression is not silently an operator in this baseline language.

  Richer semantic construction may use opaque calls, relational predicates, and closed case analysis through separately defined interfaces and checking judgments. Such a predicate must use valid endpoint types and a checked source-to-interface binding; closed case analysis must be exhaustive where required by its admitted language. Any elaboration into the closed rule language needs an explicitly checked translation. Otherwise the opaque implementation premises remain in every dependent claim. A named interface is not proof that its callable satisfies the declared policy or effect contract. These checks establish supported semantic well-formedness, not whether the product organization chose the intended rule. A coherent but undesirable policy remains representable.

- **Knowledge entailment.** The checker asks whether a role is justified in knowing a semantic fact at the point where that fact is consumed. Relevant sources include declared entry knowledge, communication, authoritative observation, checked deterministic derivation, and explicit trusted assertion. Knowledge does not arise merely because a value happens to exist in a host-language map, another role knows the fact, the browser physically received related data, or an undeclared field survived serialization. A static knowledge judgment assumes its named entry/acquisition/communication premises; it does not prove that an actual browser message was authenticated or that an authoritative observation was acquired for this execution.

- **Provenance entailment.** The checker asks whether the claimed source or epistemic strength of a fact follows from the path through which the fact was obtained. A browser-supplied value is not an authoritative observation merely because it has the same shape or value. A fact learned by communication does not automatically become an independent authoritative observation. Any strengthening of provenance must be an explicit semantic rule rather than an inference from physical message origin.

- **Authority derivation and non-escalation.** Every authority or capability consumed by a semantic step must arise from an explicit trusted/runtime premise or an allowed checked derivation. Untrusted input, mere knowledge, provenance labels supplied by the browser, possession of a rendered affordance, possession of a URL, or successful schema validation cannot silently manufacture stronger authority. Principal, resource, scope, operation, and authority identity remain part of the judgment, so authority over one resource cannot be reused for another merely because their representations have the same shape.

  The framework-level claim is not ”Alice is currently authorized to claim Request R.” That concrete fact normally depends on runtime authentication and current model state. The machine-checkable claim is closer to:

  > No admitted semantic execution may consume claim authority unless the required authority premise is present through a declared trusted/runtime source or permitted derivation, and no weaker or untrusted fact can silently manufacture stronger authority.

The following inference is invalid:

> The browser says that its role is helper, therefore it has helper authority.

A valid new-incorporation path instead establishes the relevant premises:

Establishing boundary	Required premise
Trusted authentication	This invocation represents principal P
Bound authoritative state	The Membership belongs to P and the applicable Organization
Atomic new-incorporation guard	Membership, Request, Organization policy, and their relationships permit this transition for P
Authoritative operation	Incorporation is permitted only under those current guarded premises

Every establishing edge must have the appropriate declared evidence and current runtime enforcement. An observation from an earlier basis does not silently satisfy a transaction-current premise.

- **Message payload closure and knowledge-transfer constraints.** A semantic message defines the information flow permitted by one choreography communication boundary. By default its semantic payload is closed: only fields declared by the message contract participate in the semantic message. Physical transport metadata may accompany the message, but it does not silently become choreography knowledge.

  The checker should establish that every semantic payload field is declared, the sender has justified knowledge of every semantic fact it communicates, receipt introduces only the semantic facts declared by the message contract, provenance introduced by receipt follows the declared communication rule, and undeclared transport or implementation fields cannot become branch discriminators, authority premises, or arbitrary additions to choreography context.

  Conceptually:

  ```
  receiver knowledge after receive
      =
  receiver knowledge before receive
      +
  declared semantic consequences of message
  ```

  rather than:

  ```
  receiver knowledge after receive
      =
  receiver context merged with arbitrary incoming map
  ```

  An explicitly extensible payload may be supported where required, but the semantic status of extension data must remain explicit. For example, extension data may be transported opaquely without automatically creating first-class choreography facts. Payload closure is therefore distinct from ordinary Malli shape validation: Malli may establish that a value has the expected structure, while choreography checking establishes which information that communication is semantically allowed to transfer.

- **Behavior-sensitive branch knowledge.** A role must know enough to distinguish branches before its observable behavior diverges between those branches. If a role behaves identically in both alternatives, it need not learn a choice solely because the condition occurs in the global choreography. If its behavior differs, the verifier must justify knowledge of the discriminator through an allowed path. The evidence must refer to the applicable subject, decision, scope, and continuation: observing a Request's current status does not by itself establish which historical command succeeded or which earlier branch was selected. This prevents both under-checking and needless communication introduced only to mirror global syntax.

- **Projection preservation.** Projection relates the accepted global choreography to generated role-local machines. Executions of those projected machines must remain within behavior admitted by the global choreography for the supported language subset. Projection must not invent knowledge or authority, erase communication required for justified behavior, lose required resource constraints, admit forbidden branch behavior, change terminal/failure meaning, or introduce executions absent from the accepted global semantics. The preservation/refinement claim must state the supported language subset, direction of refinement, observable behavior, and treatment of internal steps. Safety refinement does not by itself establish liveness when internal divergence or unfair scheduling is possible; a progress claim has separate fairness and transfer obligations. Nor does correctness of generated local semantics prove that an adversarial browser or physical shell obeys those semantics.

- **Declared resource invariants.** Where choreography semantics include resources, the checker may establish that a resource is used only after acquisition, exclusive ownership is not duplicated, transfers occur only through declared rules, release occurs according to the declared lifecycle, terminal paths do not retain resources that must be released, and stale or retired executions cannot retain semantic ownership. Physical cleanup debt does not confer continuing semantic ownership.
- **Progression algebra invariants.** Progression requirements form a semantic consistency algebra rather than an arbitrary collection of tokens. Canonicalization should be deterministic; composition must preserve every required authoritative constraint; weaker frontiers must not masquerade as satisfying stronger requirements; unrelated requirements must compose without destructive interference; contradictory or malformed requirements must be rejected; and normalization must not silently discard semantically relevant coordinates. Concrete XTDB tokens remain runtime values even when the algebra governing their requirements is checked independently.
- **Browser-adapter transition invariants.** The browser adapter is modeled as an explicit transition relation such as:

```
AdapterStep
  (ExecutablePlan,
   AdapterState,
   NormalizedEvent)
      →
  (AdapterState',
   AbstractEffects)
```

  Candidate invariants include execution-instance isolation, resource-instance ownership, prevention of obsolete presentation being restored after justified authoritative supersession within the applicable observation scope, separation of provisional and authoritative state, immediate semantic retirement of obsolete work, scope-preserving acquisition requirements for queued refreshes, and terminal ownership/cleanup conditions. The proof applies to the abstract transition semantics; separate implementation-correspondence evidence is required for the browser shell that normalizes concrete events and performs abstract effects.
- **Retired execution/resource isolation.** Once an execution instance or resource instance loses its relevant semantic ownership, a callback, timer, network result, settlement, continuation event, or delayed effect cannot use that retired ownership to resume execution or mutate a replacement owner's state. Admission identifies the event's declared owner and concrete resource instance; matching a reused logical target name is insufficient. Physical cleanup may occur later, and new logical ownership need not wait for every obsolete physical operation to disappear. Independently useful authoritative or command-outcome evidence may enter only through its separately declared admission path; it does not reactivate the retired owner.
- **Semantic dependency compatibility.** A certified downstream artifact may depend only on upstream semantic interfaces whose guarantees satisfy the assumptions recorded when the downstream artifact was checked. A same-shaped value, namespace, or source path is insufficient. A relevant upstream semantic change must invalidate stale downstream certification rather than silently compose.
- **Executable-plan provenance.** An accepted `ExecutablePlan` must remain traceable to the semantic inputs from which it was produced, including the accepted choreography, relevant semantic dependencies, projection target, and exact executable representation identity. Executable identity is not itself a proof; it binds the executable object to the checking history and semantic inputs that produced it.
- **Certificate dependency consistency.** A certificate may claim only guarantees justified by its checked derivation, certified dependencies, explicit assumptions/assertions, and the checker or proof-kernel rules applicable to its certificate class. Certificates may not self-certify, depend on undeclared semantic inputs, silently accept stale dependencies, form unsupported circular justification, or claim stronger conclusions than their derivations establish.

”Machine checked” is deliberately broader than ”formally proved.” Useful result classifications include:

- **Kernel-checked or theorem-proved:** a proposition is accepted by a proof kernel or theorem prover from explicit premises and checked derivations. Acceptance of a particular derivation is relative to the admitted checker and proposition encoding. A universal soundness theorem for that checker is a separate claim unless independently established and related to the actual production checker. A finite per-derivation audit cannot silently stand in for that quantified theorem.
- **Exact finite-state checked:** a finite semantic state space is exhaustively explored and the property holds for every state/transition in that exact model. This is stronger than sampled testing but is not automatically a theorem about an unbounded concrete system.
- **Abstraction-based machine proof:** an unbounded concrete claim may be justified through a finite or symbolic abstraction only when a stated abstraction/refinement argument shows that every semantically relevant concrete behavior is represented soundly by the abstraction.
- **Proof-producing elaboration:** a richer compiler or elaborator constructs derivations that are accepted by a smaller checker/kernel. The elaborator does not become trusted merely because it emits proof-looking data; a buggy elaborator must be unable to convince the checker of an invalid conclusion.

Where the design calls a property **proved**, the proof claim must identify:

- the exact proposition;
- the formal or executable semantic model over which it is established;
- the assumptions/premises;
- the supported language subset;
- the observable/refinement relation where one is involved;
- any abstraction used to reduce an unbounded concrete domain;
- why that abstraction is sound for the claimed property;
- whether the result is bounded or unbounded;
- the checker, kernel, model checker, theorem prover, or proof mechanism;
- the identity/version of the checked semantic object where relevant;
- the relationship between the checked object and the production object;
- any implementation-correspondence assumption remaining outside the proof.

Passing generated examples is not by itself a proof.

Property-based testing is not by itself a proof.

Exhaustively checking a finite concrete truncation is not by itself an unbounded proof.

A proof over an abstraction may support an unbounded claim only when the abstraction/refinement argument justifies mapping all semantically relevant concrete cases into that abstraction.

Bounding a count of requests, executions, or queue slots does not by itself bound the information retained inside them or establish a finite semantic state space. The claim must identify the state dimensions being bounded, the retained data that remains unbounded if any, and the abstraction used for the property.

Conformance vectors generated from a formal model retain their actual evidence class. Executing every generated vector is not proof that those vectors cover all production inputs or derivations. Rule coverage alone is not a general production/formal correspondence theorem. A stronger claim requires a stated exhaustiveness or sound correspondence argument for its claimed domain.

The unqualified word `proved` must never be used merely to mean:

- unit tested;
- property tested;
- preflight accepted;
- schema valid;
- assembly closed;
- observed to work in Chromium.

Those may all be important evidence. They are different evidence.

Runtime-enforced premises

Some semantic or proof premises can be discharged only when a particular execution meets concrete external state.

Examples include:

- Malli value shape at external boundaries;
- authenticated principal binding;
- operation-registry lookup;
- route and capability correspondence;
- execution-instance identity and relevant resource-instance identity;
- command/correlation identity;
- projected-plan compatibility;
- browser-artifact identity;
- progression token/header decoding;
- current server-side authorization;
- authoritative basis interpretation;
- protected read/subscription authorization;
- concrete effect results;
- transaction outcomes;
- current model/entity existence;
- current membership or relationship state.

A runtime-enforced premise is best understood as:

```
static semantic result:
    if premise P holds,
    transition T is admissible

concrete runtime:
    check P for this execution

if P succeeds:
    execute within proved/checked boundary

if P fails:
    reject or follow declared recovery semantics
```

Runtime enforcement connects the static result to a concrete execution only when it establishes the required premise for the relevant subject, basis, scope, and time. A pre-read followed by an unguarded write does not establish a commit-time premise. A postcondition check after an irreversible effect cannot be described as preventing that effect.

An instance check does not discharge a universally quantified implementation contract. Validating one returned candidate does not prove that the callback was pure, that it had no earlier hidden effect, or that it returns valid candidates on every input. The enforcement mechanism and its correspondence to the checked predicate are themselves part of the assurance argument.

For example, a static semantic result may establish:

> A new incorporation of `:request/claim` requires an authenticated principal and a transaction-current claim-authority premise. A replay requires the compatible command binding and current disclosure authority instead.

The runtime system must still establish which principal this HTTP request actually represents and whether the model currently authorizes that principal to perform the operation.

These are not second-class guarantees. They are explicit dynamic boundaries whose successful enforcement keeps concrete execution inside the premises of the semantic result.

If a runtime premise fails, execution must reject, reacquire authority, restart under a current plan, reauthorize, or follow another explicitly declared semantic recovery path. It must not silently weaken the operation.

In particular:

> A required choreography premise fails, but the runtime performs an ordinary weaker HTMX action anyway.

is forbidden.

Trusted assumptions

Some propositions remain outside the currently mechanically discharged core.

Examples include:

- authentication middleware establishes the named principal correctly;
- the qualified XTDB backend profile supplies the specific guarded transaction, outcome, and consistency semantics on which the claim relies;
- an application model operation satisfies a semantic contract not yet discharged by a stronger model certificate;
- HTMX follows the documented lifecycle subset Gesso depends on;
- browser primitives implement the operations assumed by the browser effect shell;
- Clojure, ClojureScript, JavaScript, JVM, and compiler/runtime behavior preserve the implementation semantics assumed by the checker;
- cryptographic/hash primitives used for artifact identity or authenticated evidence satisfy their required engineering assumptions;
- operating-system/network primitives behave according to the explicitly assumed contract.

A trusted assumption is part of the trusted computing base for every proposition that depends upon it.

An assumption is not a failure.

An invisible assumption is a failure of the assurance boundary.

Assumptions must therefore be named, classified, attached to the claims that rely on them, visible in explanation/root assurance output, and narrowed or discharged when stronger evidence becomes available.

Useful classifications include:

- `:authentication`;
- `:trusted-model-contract`;
- `:runtime-primitive`;
- `:external-library`;
- `:external-system`;
- `:compiler-runtime`;
- `:environment-liveness`.

Backend contracts and qualification

Gesso proves or checks its own semantic and realization properties relative to explicit backend assumptions. A proof of XTDB's implementation is not required to make such a conditional argument useful. The exact behavior relied upon must nevertheless be stated and actively challenged by qualification evidence.

A backend profile records:

- backend product, exact version or build, and relevant configuration;
- Gesso adapter/lowering identity and supported transaction forms;
- value encodings, schema/storage mapping, and relevant time/basis semantics;
- the named assumptions on which each dependent claim relies;
- conformance cases, adversarial schedules, results, and their coverage limits;
- conditions requiring requalification or invalidating applicability.

The profile is derived from resolved dependencies and actual deployment configuration. A manually entered version label or passing report for another configuration does not qualify the deployment.

The contract must distinguish at least these obligations where a claim uses them:

Obligation	Required backend behavior and challenge
Atomic guarded transition	Relevant assertions and writes form the declared atomic operation; a failed assertion cannot leave a p
Current dependency protection	Point reads, cross-document relationships, absence, and predicate dependencies used for the decision
Write and frame semantics	Replacement, update, deletion, metadata, and missing-field behavior match the compiler's interpretati
Command incorporation	Domain effects and the compatible command-result record cannot be committed independently where
Command exclusion and retention	Compatible concurrent attempts cannot incorporate twice; recorded binding, expiry, tombstones or ot
Required publication intent	Where the operation promises durable advisory work, that intent is incorporated with the authoritati
Outcome evidence	A reported committed or rejected result has the stated authoritative meaning; delivery failure remains
Observation basis	Read tokens, visibility, valid/system time where relevant, and progression requirements have the state

A generic claim of transactional support is insufficient. For example, two blind writes and two guarded writes may have different conflict behavior. A revision check protects a dependency only if all relevant changes advance the revision under the accepted writer contract. A per-document check does not automatically protect a predicate over a changing collection.

Qualification tests must be indexed by the assumption they challenge. Required cases include conflicting guarded attempts, aborted writes, ledger/domain agreement, missing and extra fields, concurrent changes to protected relationships, incompatible or repeated command identities, result expiry versus continued duplicate prevention, required outbox incorporation, outcome loss, and basis-sensitive reads where applicable. Independent expected results and deliberately weakened guards must demonstrate that the suite can expose the relevant violation. Merely replaying the same compiler logic as an oracle is insufficient independent evidence.

Finite testing cannot guarantee discovery of every possible backend violation. Successful qualification supports an explicitly accepted backend premise; it does not convert that premise into a theorem about all backend executions. Part X specifies the required evidence scope and correspondence argument.

A relevant backend, adapter, configuration, encoding, or transaction-form change requires requalification before the affected qualified claim is reused. Contradictory evidence marks the premise violated; a mismatched or expired applicability condition marks it unavailable. Both propagate to dependent deployment claims. The system must identify the affected operations and apply their declared admission or operational failure policy, rather than continue advertising the old guarantee. Existing uncertain commands remain subject to their outcome protocol; withdrawing qualification does not invent their result.

The conditional abstract theorem may remain valid when a backend violates a premise. What becomes unavailable is its application to that realization. Reports must distinguish theorem validity from deployment qualification.

An externally observable transaction-internal prestate is one possible evidence mechanism, not a universal prerequisite. A valid realization argument may rely on the specified atomic assertion/write semantics and a correct lowering under the qualified contract. The proof obligation is semantic correspondence, not possession of a particular internal snapshot token.

Application assertions

Application assertions remain distinct from framework/trusted implementation assumptions.

An application may need to state semantic facts that Gesso cannot derive. For example:

```
"this external identity provider is the authority for employee status"

"this role mapping is organizational policy"

"this third-party callback means payment has been accepted"
```

Such statements may be necessary premises. They must not be represented as though Gesso proved them.

An `:application-asserted` proposition remains an explicit premise until evidence establishes that same proposition over the required scope. A runtime check may discharge an invocation-specific premise; it must not be used to silently discharge a universal implementation contract. If another accepted assumption replaces a premise, the explanation retains that dependency rather than reporting an assumption-free result.

This distinction is useful because:

```
framework trusts browser primitive X
≠
application asserts business fact Y
```

Both are assumptions, but they have different owners and repair paths.

Assembly-verified properties

Some important propositions concern whether a finite collection of individually valid artifacts forms one closed executable application. These are naturally checked as graph/assembly properties.

Examples include:

- every rendered semantic affordance names a declared semantic operation;
- every browser-managed semantic operation has its required projected plan;
- every browser plan has a corresponding compatible browser artifact;
- every operation requiring transport has a declared transport realization;
- every such transport resolves to its declared trusted boundary;
- every trusted capability resolves to the declared authoritative operation;
- required execution capabilities are supplied;
- settlement/progression obligations have declared realization paths;
- every managed fragment has initial authoritative content or an allowed acquisition path;
- every operation successor can be reacquired through an authorized authoritative path;
- no ordinary anonymous transport occupies a route reserved for a semantic operation;
- generated browser artifacts correspond exactly to the current assembly;
- the declared startup lifecycle orders required preflight before managed serving and names the actual installation boundary that must enforce it.

Assembly verification is mechanical, but it is useful to classify the result separately from a semantic theorem. It establishes closure over a concrete application graph. A closed graph and an internally consistent lifecycle declaration do not prove that the running server installed that handler, that its captured dispatch path cannot bypass the gate, or that all relevant non-HTTP writers are covered. Those are distinct production-enforcement obligations under Parts VII and X, and their unresolved status remains visible.

For example:

```
choreography theorem:
    projected Claim respects knowledge requirements

assembly verification:
    this application's Claim button actually resolves to that plan,
    current browser artifact,
    route,
    server operation,
    settlement,
    and reacquisition path
```

Both can be true or false independently.

Assembly checks should therefore be classified with precise language such as:

- `:assembly-verified`;
- `:preflight-closed`;
- `:artifact-current`;

rather than being upgraded automatically to `proved`.

Where an assembly proposition is established by a formal proof mechanism, its evidence may state that stronger result with the exact proposition and scope. Ordinary graph checking does not acquire that classification by association.

Conditional liveness

Liveness is not an independent evidence mechanism. It is a qualification on propositions whose truth depends on future behavior of the environment.

A statement such as:

> Every provisional claim eventually reaches a resolving outcome.

is meaningful only relative to premises such as:

- required events are eventually delivered;
- authoritative dependencies remain reachable;
- required authoritative rereads eventually complete;
- participants continue executing;
- relevant clocks/timers eventually fire;
- retry policy continues where required;
- the application can recover from an incompatible stale projection;
- application-defined conflicts do not remain permanently unresolved;
- required external systems eventually respond where the claim depends upon them.

The claim should therefore have a form conceptually like:

```
assuming:
    eventual event delivery
    eventual authoritative availability
    continued participant execution

prove/check:
    every admitted provisional state eventually reaches
    a resolving semantic state
```

The assumptions are part of the proposition. Removing them changes the proposition.

For commands, recovery of an authoritative **current presentation** and recovery of the **historical command disposition** are separate progress claims. An authorized fresh Request/card read may repair provisional display without identifying which command incorporated. Eventual attribution requires the appropriate retained command evidence, compatible binding, continuing disclosure permission, and an available reconciliation path; those premises must not be smuggled into ”eventual authoritative read.” After expiry or lost disclosure authority, the declared system may preserve an explicit unresolved command status even when presentation converges. That is not evidence of settlement to accepted or rejected. Likewise, when the product requires a user-initiated Refresh, the user's eventual action is an environmental premise, not something the scheduler can compel or derive from a pending advisory.

The design must distinguish:

- **reachability:** there exists a path to a terminal/resolving state;
- **deadlock freedom:** every reachable nonterminal state has an enabled modeled continuation; this does not by itself require that the continuation is taken;
- **fairness-dependent progress:** enabled progress eventually occurs under a fairness assumption;
- **environment-dependent convergence:** the system converges if specified external events eventually occur;
- **unconditional termination:** every execution terminates without such assumptions.

These are materially different claims.

The design must not call ”there exists a path to terminal state” an unconditional termination proof, nor hide network recovery, browser resumption, user action, or external-system availability inside the word `eventually`.

Tested evidence

Tests remain a required assurance layer even where stronger semantic checking exists.

Testing is particularly important for:

- implementation correspondence between semantic core and runtime shell;
- real JVM/Node/Chromium execution;
- HTMX/SSE lifecycle behavior;
- XTDB integration;
- browser primitive assumptions;
- HTTP/session/authentication integration;
- generated-artifact loading;
- fault injection;
- race schedules;
- reconnect behavior;
- load/resource behavior;
- multi-client HumanHelp behavior;
- multi-node realization once it exists.

Useful test classes include unit tests, property/generated tests, differential tests, mutation tests, integration tests, adversarial tests, browser acceptance tests, and load/fault tests.

Tests may provide very strong practical evidence. They must nevertheless remain classified as `tested` unless the particular procedure genuinely exhausts the explicitly claimed semantic domain and is classified more precisely, for example `exact-finite-checked`.

Thus:

```
100,000 generated executions passed
```

does not mean:

```
all executions satisfy the theorem
```

unless a separate argument establishes that the tested set is exhaustive for the claimed semantic domain.

Composition of assurance claims

The most important property of the assurance model is that evidence composes without losing its classification.

A downstream artifact should be able to say conceptually:

Claim-plan dependency	Evidence and retained premises
Rule well typed	Dedicated typed semantic check
Helper knows the Request identity	Checked knowledge derivation
New-incorporation authority	Checked semantic requirement and concrete atomic runtime enforcement
Model implementation contract	Proof where supplied; otherwise an explicitly accepted implementation premise
Projection preservation	Appropriate checked argument; a finite abstraction requires a sound refinement argument for an unb
Browser realization closure	Assembly verification against actual admitted sources and artifacts
Browser shell correspondence	Stated implementation argument, trusted primitive contracts, and independently classified browser te

The resulting assurance claim must not erase those differences and simply say ”Claim is verified.”

A stronger downstream result cannot silently upgrade a weaker premise.

For example:

```
trusted model contract
    +
proved choreography rule
```

does not become:

```
proved model implementation
```

The conclusion remains conditional on the trusted model contract.

Likewise:

```
assembly verified
    +
browser test green
```

does not establish a projection theorem.

Each evidence class retains its identity through composition.

The root report should therefore expose what was proved, kernel-checked, exact-finite checked, assembly-verified, runtime-enforced, trusted, application-asserted, conditional on liveness assumptions, and supported by testing.

Failure and repair interpretation

The assurance classification should tell the developer where repair belongs.

Typical relationships include:

Failure	Owner of repair or interpretation
Rule typing	Rule or domain declaration
Knowledge entailment	Justified information path or the consuming behavior
Authority derivation	Authority premise, binding, or policy boundary
Message payload closure	Message contract or undeclared information transfer
Projection preservation	Choreography, projector, or semantic definitions
Assembly closure	Application construction and bindings
Runtime authorization	The attempted concrete operation is not presently admissible
Contradicted assumption	The trusted boundary or applicability of the assumption
Browser failure despite a valid abstract theorem	Runtime correspondence or an assumed primitive
Missing liveness premise	The dependent progress conclusion is unavailable

This is why precise assurance terminology is not cosmetic. It makes a failed claim point toward the layer that owns the repair.

Assurance-model invariant

The governing invariant is:

> Every meaningful Gesso guarantee identifies the proposition being claimed, the evidence supporting it, the premises on which it depends, and the boundary between mechanically discharged facts and facts that remain runtime-enforced, trusted, asserted, conditional, or tested.
>
> No stronger assurance classification may be inferred merely because several weaker forms of evidence happen to agree.

The intended result is not that every statement becomes a formal theorem.

The intended result is that Gesso can state, mechanically and honestly, **why** each accepted artifact or application is believed to be valid, what remains outside that claim, and exactly where construction or execution must stop when one of its premises fails.

Semantic evidence vocabulary

The following vocabulary is shared by the later compositional hardening and whole-application proof architecture.

It is defined here so later layers do not need to redefine what their evidence means.

Identity and instance terminology

An identity names a particular semantic object or concrete instance. Its meaning must be explicit wherever admission, ownership, compatibility, or evidence binding depends on it. The canonical distinctions include:

- **Command identity:** the logical intent, which may survive multiple attempts.
- **Execution identity:** one concrete semantic execution of a choreography or operation; its relation to a command is explicitly declared.
- **Resource-instance identity:** one concrete timer, subscription, transport request, continuity capture, or other managed resource, distinct from its reusable logical resource/target name.
- **Transport-request identity:** one transport attempt, distinct from the domain object being acted upon and from the command it may carry.
- **Submission-attempt identity:** one attempt to incorporate a command at an authoritative boundary; one transport request need not imply exactly one backend attempt.
- **Capability-instance identity:** a concrete grant to perform a scoped action under its lifetime and usage contract; it is not the command identifier.
- **Admitted assembly generation:** the immutable application realization admitted together, including its routes, providers, prepared operations, and artifacts.
- **Compatibility identity/version:** the declared protocol or plan compatibility class, distinct from exact executable-artifact identity and from deployment.

A deployment may host multiple admitted assembly generations during a supported transition. A deployment identifier, assembly generation, execution identity, and resource-instance identity are not interchangeable coordinates.

Use `generation` only when its owning subject and purpose are explicit, such as `assembly generation` or a declared `resource-instance generation counter`. A counter is one possible representation; opaque identity is sufficient where only equality and non-aliasing are needed. Do not imply an ordering, universal counter, or shared lifetime merely by calling a value a generation. A later reference may abbreviate a locally established subject only when it remains unambiguous.

For browser transport examples, `:transport-request-id` names the transport attempt. Domain Request arguments such as HumanHelp's `:request-id` retain their domain meaning. Different transport attempts must not be conflated because they carry the same command or concern the same domain Request.

Operation identity is **preserved** through realization: it cannot be silently erased or substituted. Reserve **monotonic** for a stated ordering or information- growth relation. In particular, preventing retired work from restoring obsolete presentation does not imply that all observations form one totally ordered history or that every valid domain transition increases a numeric value.

Executable semantic core, proof kernel, and dedicated checkers

The **executable semantic core** defines the production transition meaning for its supported semantic subset. It consumes the applicable executable rules, admitted state, and inputs/events, and yields semantic state and declared effects. A portable machine step or browser adapter transition function is an executable semantic core within its stated scope. There need not be one universal step function for all layers.

The **proof kernel** checks derivations against explicit inference rules and premises. It accepts or rejects the claimed judgment; it does not perform the application's runtime transitions or supply runtime authorization. It is separate from the richer elaborator that proposes terms and evidence. It must not trust the elaborator's conclusion merely because the elaborator supplied it.

A **dedicated semantic checker** establishes a particular judgment or property, such as exact finite-state exploration or structural application closure, through its declared checking mechanism. It need not be implemented as part of the proof kernel. Its result retains the evidence class and trust premises of that mechanism.

These components may share canonical semantic definitions. Sharing definitions neither merges their responsibilities nor establishes implementation soundness. The relationship between the checked subject and the production object remains an explicit correspondence obligation. Canonical terminology calls the runtime component the executable semantic core, reserving `proof kernel` for derivation checking; external proof-assistant kernels must be identified as external.

The trust base of a claim includes every component that establishes a judgment without supplying evidence checked by another identified mechanism. Depending on the claim, this includes parsing, typing, normalization, substitution, transition interpretation,

frame rules, initiality, invariant preservation, induction, certificate admission, and source-to-artifact binding.

A small propositional kernel does not make the whole assurance system equally small if other modules directly establish additional semantic judgments. Those modules must either produce evidence accepted by the appropriate checker or remain explicit parts of the trusted checker implementation. Part IX owns their rules; Part X owns the corresponding soundness and implementation arguments.

The architecture separates semantic definitions, elaboration, derivation checking, runtime realization, and observations/reports. Checking a derivation does not submit a database transaction. An integration runner may perform real effects to collect evidence, but it is then an effectful realization boundary, not part of the pure proof kernel. The runtime consumes admitted semantics and current premises, not a diagnostic report standing in for either.

These definitions fix responsibilities without requiring one monolithic namespace. Shared definitions reduce drift; independent correspondence checks and soundness arguments address mistakes shared by execution and checking.

Raw declaration

A **raw declaration** is user, application, or framework data that has not yet crossed the semantic checker appropriate to its role.

Examples include:

- a raw Malli or domain schema;
- a raw business-rule expression;
- a raw operation contract;
- a raw choreography;
- a raw authority declaration;
- a raw derivation;
- a raw dependency reference;
- a raw application assembly declaration.

Raw data is permitted at construction boundaries.

Raw data must not masquerade as a certified semantic artifact merely because it has the same map shape as one.

Admitted semantic value

An **admitted semantic value** is a value that has crossed the construction, verification, authoritative-acquisition, or other checking boundary required for the semantic category in which downstream code will consume it.

Admission answers a question such as:

> May this value be treated by the supported program as an instance of semantic category C at this boundary?

Admission does not automatically answer every stronger question about the value. In particular:

- admission into a nominal category is not automatically a proof of every proposition named by that category;
- admission does not create current authorization unless authorization is the proposition established by the trusted runtime boundary;
- admission does not make a client-carried value authoritative merely because it was once authoritative before serialization;
- an admitted artifact identity binds the relevant object to its construction history but does not establish semantic truth by identity alone.

A nominal type, opaque wrapper, private constructor, checked constructor, or registry-resolved object may help represent admission. The requirement is that unsupported construction or same-shaped raw data cannot silently receive the stronger semantic treatment through the normal API.

Semantic interface

A **semantic interface** is the normalized meaning exported by one checked component to its dependents.

It contains only propositions downstream reasoning is permitted to rely on.

Depending on the component, this may include:

- declared value types;
- variants;
- cardinality;
- optionality/refinement rules;
- relationships;
- operation input/output facts;
- preconditions;
- postconditions;
- authority requirements;
- provenance requirements;
- effects and resources;
- named assumptions;
- named guarantees;
- compatibility constraints.

An exported interface distinguishes guarantees from the assumptions under which those guarantees hold. Listing a proposition as a required premise does not export that proposition as already established. A dependent component must either inherit the exact applicable premise, discharge it using a checked justification, or decline the stronger claim. The interface's named obligations and dependencies must be resolved to their source-owned subjects; a convenient summary is not an independent source of semantic truth.

Implementation details irrelevant to downstream reasoning should not become semantic commitments merely because they are convenient to serialize.

Assumption

An **assumption** is a proposition accepted from outside the proof currently being established. It is identified by its proposition **and applicability**: subject, parameters and bindings, semantic or authority context, scope, origin, and the implementation/runtime boundary where relevant. Its human-readable name alone is not sufficient identity. Two claims about current Membership for different principals, Organizations, Requests, or invocations do not share one premise merely because both are called `:current-membership`.

Assumptions are explicit leaves in the selected proof justification or explanation graph. Acceptance under an admission policy keeps the dependent claim conditional; it does not discharge the assumption. Discharge requires a separately checked, scope-compatible justification that inherits its own premises, as specified in Part IX.

They are not inferred from trust in a namespace name, source location, familiar implementation, passing test, or matching hash.

Guarantee

A **guarantee** is a proposition established by the current checked component relative to its certified dependencies and named assumptions.

Obligation

An **obligation** is a proposition required for a particular judgment, admission, or assurance claim. A mandatory check must be established before its result is issued. Where admission policy permits an unproved contract as a premise, the resulting certificate states the dependent conditional judgment; it does not claim that the original contract obligation was discharged.

An unresolved obligation must remain visible.

It must not be replaced by a default success value or silently omitted from the result.

Derivation

A **derivation** is machine-readable proposed evidence describing how a guarantee follows from specified premises and inference rules. A raw derivation is not self-validating. Only a successful check under the applicable calculus and subject/premise bindings can establish the judgment it proposes; the interpretation and soundness of that checking mechanism remain separately classified claims.

Derivations are useful for:

- mechanical checking;
- diagnostics;
- explanation;
- repair;
- trusted-computing-base reduction through independently checked evidence.

A derivation is evidence, not runtime authority.

Certificate

A **certificate** is a closed evidence artifact stating enough information for an appropriate checker to decide the claims it represents. Closed means that its required references and dependencies are accounted for, including named assumption leaves. It does not mean that those assumptions have been proved or that future runtime premises already hold.

A certificate may include:

- the semantic object checked;
- its semantic identity;
- certified dependency identities;
- assumptions;
- guarantees;
- proof obligations;
- derivations or checked references to derivations;
- applicable semantic-checker or proof-kernel version;
- result classification.

A certificate is not a boolean.

A certificate is not accepted because an ordinary Clojure map contains a key such as `:verified? true`. A submitted certificate-shaped value remains raw evidence until its required checking/re-admission boundary succeeds.

A certificate is accepted only through the checker and issuer boundaries appropriate to that certificate class. Admission binds the checked subject, claims, required source-derived obligations, dependency closure, permitted premises, and applicability. A copied successful report or a matching digest does not supply that issuer binding. A partial explanation of failed admission is useful diagnostic material, not a certificate satisfying the missing claim.

Semantic identity

A **semantic identity** is a deterministic identity of the semantic material normalized under a declared encoding and normalization contract. It identifies that material; it is not a decision procedure for equivalence of arbitrary Clojure programs.

It is deliberately distinct from:

- source-file checksum;
- file path;
- namespace name;
- build timestamp;
- runtime locator;
- physical artifact location.

Whitespace, comments, local helper names, and implementation details outside the exported contract need not alter its semantic identity. A claim about the concrete implementation may nevertheless need a new implementation binding or new correspondence evidence after such a change.

Changes to relied-upon semantic commitments must invalidate dependent evidence unless a checked compatibility or implication argument justifies reuse. A matching contract identity alone does not authorize evidence reuse across different implementations.

Examples include changes to:

- value types;
- rule contracts;
- authority contracts;
- operation postconditions;
- provenance requirements;
- compatibility rules;
- resource semantics.

Semantic identity identifies normalized semantic material under the declared normalization and identity contract. A matching untrusted identifier is not self-authenticating evidence that the corresponding material was checked.

Semantic identity by itself does not prove the truth of that material. Evidence about a particular generated artifact or implementation must also remain bound to the concrete object to which that evidence applies. Equality of normalized contract material does not automatically transfer test results or physical artifact correspondence from an earlier build to a different one. For opaque functions, the exported contract, admitted implementation binding, and evidence applicability are separate identities. Gesso does not require deciding extensional equivalence of those functions.

Proof DAG

A **proof DAG** is the dependency graph from a root checked artifact through all certified semantic dependencies and named assumptions.

A root may depend on the following semantic layers; actual dependencies form a DAG, not necessarily one linear chain:

Semantic layer	Typical exported evidence
Schemas, domains, and relationships	Typed interfaces and relationship constraints
Business rules	Well-formedness, proved properties, and explicit opaque contracts
Model operations	Transition obligations, invariants, and realization premises
Choreography	Knowledge, authority, coordination, and resource judgments
Projection	The stated global-to-local behavior relation
Executable plans	Semantic and artifact identity bindings
Whole application	Closure, production enforcement, and independent acceptance evidence

The proof DAG records the justifications and inherited premises for the particular claims being established. Its acyclicity and internal completeness are necessary, but do not establish that all **required** application obligations were included. The constructor separately derives the mandatory obligation ledger and required dependency edges from the original source, actual semantic subject, and selected profile. It must check that the selected proof DAG accounts for those obligations as established, explicitly accepted premises where policy permits, or unresolved blockers. A caller-supplied graph with a conveniently omitted guard, operation, authority check, writer, or correspondence edge cannot certify the reduced subject as the complete application.

Alternative sound justifications may have different dependencies; only the selected justification's actual premises are inherited by its dependent claim. No premise can be discarded merely because another claim in a shared artifact is assumption-free. Required obligation coverage and per-claim dependency closure are separate checks, not a second author-maintained proof registry. Part IX owns their normative checking and issuance rules.

A later stage may rely only on propositions actually exported by its certified dependencies or named as assumptions.

No stage receives credit for a predecessor's intended meaning when that meaning was never certified.

The certificate dependency DAG must be acyclic. Recursive or fixed-point semantics may be supported only through a deliberately formalized rule with checked premises; introducing a dependency cycle does not supply that justification.

This requirement is distinct from ordinary recursive computation. In particular, a cycle among Clojure namespace dependencies is a load-time dependency error, not a mechanism for expressing logical recursion. Any recursive reasoning admitted by the certification system must therefore be represented explicitly in its semantics rather than encoded as a cyclic namespace or certificate dependency.

Construction principles

The following principles apply throughout the design.

They are not implementation-history advice. They are properties of the intended finished system.

Canonical semantic construction and controlled effects

The operation's canonical semantic source determines the obligations of its realization. For an authoritative state-changing operation, that source contains or references:

1. The typed state projection and input contract, including optionality, nominal distinctions, identity bindings, and supported value representations.
2. Its invocation, repetition, and new-incorporation protocols.
3. Its guard and declared dependencies on authoritative state or external facts.
4. Its permitted transition relation, writes, preserved fields, and other declared effects.
5. Its result meanings, command outcome protocol, and required acquisition or recovery successors.
6. Its opaque implementation contracts, assumptions, and evidence requirements.

State semantics distinguish absence, present nil where permitted, and false. An accessor, serializer, predicate, or physical database mapping must not erase that distinction when the declared semantics uses it. Numeric ranges, overflow, comparison, nominal enums, and time representations are equally explicit; host runtime defaults do not define portable semantics by accident.

For a guarded assignment, all right-hand sides refer to the same prestate and admitted inputs unless the operation explicitly declares sequential intermediate steps. Its frame identifies state required to preserve both presence and value. For supported expressions and assignments, construction derives the direct read and write footprints and the default unchanged-field frame relative to the declared state projection. This syntactic footprint is not automatically the complete **protected semantic dependency set**: relational predicates, absence/uniqueness conditions, opaque policy calls, and cross-model authorization may depend on state not named by a direct field read. Those dependencies are exported through their owning semantic interfaces and incorporated into the source-derived obligation set. Where Gesso can check their coverage and physical protection, it does; otherwise the precise unestablished relation remains an explicit premise or an admission blocker under policy. A caller-supplied footprint may provide a declared dependency contract but cannot override derived evidence, hide dependencies, or grant authorization by assertion.

An abstract frame over three declared fields does not prove that every other physical database field is preserved. The storage mapping must account for domain fields, framework revision/time metadata, replacement versus patch semantics, and any fields outside the projection. Mapped identities must not collide through key normalization or encoding. Part IV specifies that mapping and its authoritative effect contract.

An opaque computation may supply a candidate result or change plan. Admission can check its typed result, target binding, supported guard/assignment/frame relation, and effect footprint before those effects are submitted. Such a check can establish those instance properties without proving the computation for all inputs. It does not prove that the computation had no earlier hidden effect; effect containment still needs its own enforcement or retained assumption.

From an admitted candidate to an authoritative outcome

A candidate is neither a committed transition nor a receipt. The controlled realization has these distinct responsibilities:

Boundary	Required responsibility
Preparation	Obtain appropriately bound inputs and observations; construct the candidate using admitted semantic sources
Candidate admission	Check the actual proposed targets, changes, frame, declared effects, and command binding rather than a caller-sup
Lowering	Translate the admitted semantic operation to a physical transaction/effect plan under the applicable storage and b
Submission	Submit that admitted plan through the scoped capability; reject unnoticed substitution or additional effects
Incorporation	Enforce current dependencies atomically with the required domain and command-record changes
Outcome admission	Interpret authoritative result evidence at its actual scope; preserve unknown status where evidence is insufficient
Delivery and recovery	Resolve participant knowledge and declared propagation obligations without inventing a commit outcome

The boundaries may be combined in an implementation, but their obligations cannot disappear. A candidate validated against an earlier observation needs commit-time guards or an equivalent atomic realization. Independently writing the ledger after the domain change does not implement an atomic command protocol. Postcommit invalidation or external delivery requires its own durable or otherwise justified recovery contract; a successful domain write does not prove that an interested browser learned about it.

The physical transaction may involve several resources. HumanHelp self-Claim can depend on the current Request, Membership, Organization and relevant RequestAssignment facts, including absence of a conflicting active primary assignment. The admitted incorporation changes the Request and its production assignment relationship, and atomically records the command result and required durable publication intent under the operation's declared policy. Protecting only Request status or constructing a toy `:request/helper` field does not establish this composition. The model's own public contract determines which relationships, guards, and assignment writes are actually necessary; this example does not invent their physical schema. A supported backend profile may impose explicit limits, but prototype restrictions such as one document, literal-only assignments, or disabled notifications are not the general semantic model.

Invariant obligations and coverage

For a stated invariant `I`, a closed operation catalog supports induction only when the relevant obligations are established:

- every admitted initial state satisfies `I`;
- each admitted transition preserves `I` from a state satisfying `I`;
- all state-changing behavior relevant to the claim is represented by those transitions or by explicit environment assumptions;
- the concrete realization corresponds to the stated transition semantics under its retained premises.

Conceptually, the base and step obligations are:

```
For every s:
    Init(s) implies I(s)

For every admitted operation o and every s, x, s':
    I(s) and T_o(s, x, s') imply I(s')
```

`T_o` includes that operation's permitted guards, assignments, and effects at the stated abstraction. The proof scope names the state projection and operation catalog; it does not silently include every possible database writer. Feasible initialization and usable required operations need separate positive evidence, so an impossible initial condition or a guard rejecting every invocation cannot be used to satisfy product completeness.

Initializers, migrations, imports, administrative operations, scheduled work, and external writers must have an explicit place in the coverage argument. Their absence from a developer-maintained list is not evidence that they do not exist. A proof about the supported catalog remains a conditional catalog proof until production writer coverage and correspondence justify its application.

Managed capabilities and admitted realizations

The ordinary application assembly supplies the capabilities needed to realize declared effects. Code receives the scoped capabilities required by its role; it should not ordinarily obtain raw database submission authority merely to construct a candidate. Read-only and static routes do not need writer capabilities. Background and asynchronous work receive explicit lifetime and transfer rules rather than borrowing an HTTP request's identity indefinitely.

A one-use capability bounds one grant's use. It does not make a logical command idempotent across new grants or retries. That requires the authoritative command binding and record protocol. Conversely, a command record does not prove that every write occurred through the intended scoped capability.

An admitted assembly generation binds the semantic sources, implementations, providers, routes, artifact identities, qualification profiles, and policy used by its realization. A relevant replacement requires re-admission or a checked compatibility transition. Supported development reload and deployment overlap must make that transition explicit; repeatedly checking mutable Var identities is not a substitute for a declared lifecycle and dependency contract.

Managed capability construction prevents accidental bypass in trusted application code. It is not a sandbox against hostile code sharing the JVM. Any additional containment claim must identify the actual isolation mechanism and its assumptions. Neither opaque Clojure nor explicit assertions excuse undeclared interference with protected state.

State semantic intent once

The ordinary coherent formulation should state each semantic fact at its owning source, then derive or verify its uses. ”One canonical place” means one authoritative owner **per fact**, not one monolithic declaration for every domain rule, choreography, view, route, acquisition, and runtime adapter. The public model owns business/transition meaning; choreography owns declared coordination; views select operations and bind actual arguments; ingress and providers own genuine physical choices. The application constructor joins those exports and checks required correspondences. No layer gains authority to replace another owner's contract merely by generating plausible metadata.

If Gesso already knows:

- operation identity;
- participating role;
- projected browser plan;
- command schema;
- transport requirement;
- expected settlement;
- progression requirement;

then views, routes, browser registries, and server handlers should not all independently restate those facts unless an independent physical choice genuinely exists.

Prefer:

One semantic declaration supplies derived browser metadata, derived or checked transport bindings, diagnostics, and application closure edges.

over:

An operation, button, plan name, transport, route string, and bootstrap entry are independently maintained as if they were unrelated declarations.

When independently authored values must agree, correspondence must be checked explicitly.

A Claim card reuses the `claim-request` operation already defined. It does not introduce a second version of Claim's authority, replay, or recovery semantics. The view supplies only the operation reference, domain arguments, and local presentation choices:

```
(defn request-card
  [{:request/keys [id status title]}]
  [:article.request
   [:h3 title]
   [:p (name status)]
   (when (= status :open)
     (gesso/action
       {:operation :request/claim
        :args {:request/id id}}
       "Claim"))])
```

The example's status-only `when` is a presentation filter, not a complete Claim-eligibility rule. Actual HumanHelp availability may additionally depend on the selected actor, effective Membership, Organization policy, current assignment, and permitted view projection. Those policies remain model-owned; a view can consume a prepared permitted affordance without rewriting the authoritative predicate. An eligible positive path must still exist and be tested under the independent product acceptance contract.

Visibility is not authorization. The action constructor resolves the admitted operation, validates the exposed arguments under its construction contract, and obtains the command/execution protocol from that operation. A rendered action and browser-carried command ID do not establish the current principal, commit-time eligibility, or command incorporation. Those remain trusted invocation and authoritative-operation responsibilities. The author does not manually create an authority token or a command-outcome query.

From those semantic declarations, Gesso may derive or validate lower-level realization data:

```
(project :request/claim)
;; ⇒
{:plan/id :plan/request-claim
 :operation :request/claim
 :role :helper
 :argument-schema ClaimInput
 :command-envelope-schema
 [:map
  [:arguments ClaimInput]
  [:gesso/command-id uuid?]]
 :presentation-owner :gesso/execution-id
 :command-correlation :gesso/command-id
 :transport-requirement :trusted-semantic-post
 :settlement
 {:accepted :request/card
  :rejected :request/card
  :unknown :request/reconcile-claim}}

(realize humanhelp-assembly
  :request/claim)
;; ⇒
{:browser/binding
 {:operation :request/claim
  :plan :plan/request-claim}

 :transport/binding
 {:kind :trusted-semantic-post
  :route :route/request-claim}

 :server/binding
 {:capability :server/request-claim
  :model-op :request/claim}}
```

The exact projection and realization representations are open design choices. What matters is that the view author did not separately type:

```
;; This is the kind of manually synchronized authoring Gesso should avoid.

[:button
 {:hx-post "/requests/claim"
  :data-operation "claim-request"
  :data-plan "humanhelp-claim-plan-v7"
  :data-command-param "__gesso_live_optimistic_command"
  :data-server-handler "claim-handler"
  :data-model-op "request/claim"}
 "Claim"]
```

That second version contains several strings that all purport to describe one semantic intent. Each value can be locally plausible while the collection is globally contradictory. A typo can easily produce a path that appears to render correctly and even sends an HTTP request while no longer corresponding to the declared choreography.

The desirable API therefore pushes complexity inward. Application code states the semantic operation and domain arguments; Gesso derives what it owns and checks correspondence where an independent physical choice genuinely remains. The illustrative `gesso/action` form is not an API commitment. It is a usability constraint on the supported public API.

Derive rather than synchronize

Duplication creates programs whose pieces are individually plausible but mutually contradictory.

Whenever a fact can be derived safely from a stronger semantic source, derive it.

Do not create a second source of truth merely to make one layer easier to implement.

Fail at the earliest legitimate boundary

A contradiction should fail at the earliest layer that possesses enough information to establish it.

Examples of definition/construction-time failures include:

- malformed choreography;
- unknown role;
- impossible local transition;
- invalid message declaration;
- inconsistent operation metadata;
- invalid certified dependency reference.

Examples of build/application-preflight failures include:

- declared semantic affordance with no projected executable plan;
- stale generated artifact;
- missing transport;
- missing trusted route/capability;
- missing server execution capability;
- managed fragment with no permitted initial acquisition;
- authoritative successor with no permitted reacquisition path.

An affordance introduced only while constructing a concrete view is checked before that rendered surface is serialized or exposed through the managed renderer. The checked subject is the **effective** action in its admitted render context, including relevant ancestor form/HTMX inheritance, target and selector interpretation, operation binding, and current assembly compatibility. A fragment checked alone cannot certify inherited behavior that depends on an unknown outer context. That context must be supplied and checked, or the corresponding managed-output claim remains unsupported and the required boundary rejects the output.

Arbitrary Hiccup or raw HTML that bypasses this boundary does not inherit the rendered-surface assurance claim; supported semantic action construction must make the distinction explicit. Deferred, streamed, cached, and transformed responses require the declared checking boundary before their effective managed output is exposed. Static assembly preflight checks only affordances and operation bindings knowable from its declarations; it cannot claim to have inspected future arbitrary view data.

The same HumanHelp Claim slice can fail legitimately at three different times, depending on when enough information exists.

A static construction defect can be rejected before the application starts:

```
(def broken-assembly
  {:operations
   {:request/claim
    {:plan :plan/request-claim
     :route :route/request-claim}}

   ;; Accidentally generated from an older application assembly.
   :artifact
   {:plans #{:plan/request-create
             :plan/request-refresh}}})

(assembly/admit! broken-assembly)
;; throws:
{:error/type :assembly/missing-browser-plan
 :operation :request/claim
 :required-plan :plan/request-claim
 :artifact-plans #{:plan/request-create
                   :plan/request-refresh}}
```

Nothing about a concrete Request or principal is needed to establish that contradiction, so runtime is too late. This deliberately abbreviated `broken-assembly` is a controlled negative fixture, not the recommended public application-definition shape. Real construction derives its required edges from source-owned operations and actual realization bindings, and checks the real build artifact rather than accepting a caller-supplied `:artifact/plans` report as proof of compiled content. A passing detached fixture is not evidence that the installed server uses this admission gate.

A rendered-surface defect may depend on concrete view construction. Suppose a generic Request-card renderer allows applications to choose actions, but one particular rendering asks for `:request/reassign` without supplying the required target relationship:

```
(render
  configurable-request-card
  {:request request
   :actions
   [{:operation :request/reassign
     :args
     {:request/id (:request/id request)}
     ;; required :helper/id is absent
     }]})

;; render preflight rejects before serialization:
{:error/type :view/operation-arguments-incomplete
 :view :request/configurable-card
 :operation :request/reassign
 :missing #{:helper/id}
 :available #{:request/id}}
```

The assembly may contain a perfectly valid Reassign operation. The defect exists only in this attempted concrete rendering, so render-time preflight is the earliest legitimate boundary.

Finally, current authorization may change after a completely valid render:

```
;; T1: Alice is currently an active helper.
(html
  (request-card
    {:request/id R
     :request/status :open}))
;; ⇒ contains a valid bound :request/claim affordance

;; T2: Alice's Membership is withdrawn.

;; T3: Alice submits a new command C42 from the still-visible affordance.
;; The authoritative operation establishes this rejection and its reason.
(invoke-claim!
  current-server-context
  {:request/id R :gesso/command-id C42})
;; ⇒ {:outcome :rejected :reason :membership-inactive}
```

Static preflight should not have rejected the application merely because a helper can later lose authorization, and rendering should not claim that the authorization can never change. The runtime boundary owns that dynamic fact and must reject the concrete invocation using current authority.

The three failures therefore form one continuous construction story:

Information becomes available	Earliest legitimate rejection boundary
From declarations	Construction or assembly admission
From a concrete rendered surface	Before serialization and exposure
From current external or model state	The trusted runtime boundary with the required current-state protection

”Fail early” means the earliest **legitimate** boundary, not the earliest point at which the framework could guess. An established contradiction rejects at its owning boundary. An unsupported analysis, unavailable source, or exhausted checking budget is **not** an established contradiction or an acceptance result: the selected admission policy must either require more evidence or explicitly retain a permitted premise and its enforcing boundary. A mandatory check cannot be silently skipped or converted into a green result because the earlier checker lacked enough information.

A negative conformance case identifies the first expected enforcing boundary and the prohibited action or exposure. Its paired valid case must succeed through the supported path; universal rejection is not a usable admission policy. In particular, a late response/render failure cannot undo an earlier valid authoritative incorporation or be reported as a command rejection. Command outcome, response admission, and browser installation retain their separate evidence and recovery paths.

Runtime-only conditions and required recovery

Some facts relevant to an execution can be established only at runtime. Examples include:

- the current authenticated principal is not authorized for a concrete Request;
- the Request changed after it was rendered;
- a transaction guard fails;
- an external provider is unavailable;
- a browser or another required participant disconnects;
- an expected environment event has not occurred within an application-relevant lease, deadline, or interval.

Do not attempt to statically prove genuinely dynamic domain facts merely to increase preflight coverage.

Runtime-only establishment does not place these conditions outside the application's correctness model. The distinction is between knowing before execution whether a condition will occur and knowing before execution that the application has a coherent response if it does occur.

Where a declared operation or workflow can encounter a runtime condition that invalidates its premises, removes a participant required for progress, or would otherwise strand the workflow, the application must define a permitted path forward. Depending on the application, that path may involve rejection, authoritative reacquisition, retry, compensation, timeout, retirement, reassignment, cancellation, explicit degradation, or another declared successor state.

Static checking may therefore establish a conditional property such as:

```
if runtime condition F is established,
the admitted application has a declared path
from F to a valid recovery, successor,
or terminal state
```

without claiming to establish statically whether F will occur in a particular execution.

A Request changing after rendering is one example. Gesso cannot establish beforehand that the Request will remain unchanged until the user acts. It can require that an operation whose rendered premises have become stale is checked against current authority and model state and that failure of those premises leads through declared rejection, reacquisition, or recovery semantics rather than allowing execution to proceed from obsolete state.

A seat-reservation operation shows what that requirement can look like as a small executable contract rather than a prose-only promise.

```
(defoperation reserve-seat
  {:operation/id :flight/reserve-seat

   :actor :traveler

   :input
   [:map
    [:flight/id uuid?]
    [:seat/id uuid?]]

   ;; This observation justifies what the traveler saw. It does not reserve the
   ;; seat or guarantee that the observation remains current until invocation.
   :entry
   [[:authoritative-observation
     :seat/availability
     {:scope [:flight/id :seat/id]}]]

   :invocation
   {:requires [:authenticated-traveler :admitted-command-binding]}

   :new-incorporation
   {:requires [:current-seat-state :current-traveler-eligibility]
    :atomic-with :command/result-record}

   :repetition
   {:requires :command/result-disclosure-authority
    :result :permitted-recorded-outcome}

   :effect
   [:model/invoke :seat/reserve
    {:flight-id :flight/id
     :seat-id :seat/id
     :traveler :authenticated-traveler
     :command-id :gesso/command-id}]

   :settlement
   {:accepted
    {:learns [:command/reservation-recorded :gesso/command-id]
     :reacquire [:flight/seat-map :flight/id]}

    :rejected
    {:reason-set #{:seat/already-reserved
                   :seat/no-longer-offered
                   :traveler/not-eligible
                   :reservation/precondition-changed}
     :reacquire [:flight/seat-map :flight/id]}

    :unknown
    {:recover
     [:seat/reconcile-reservation
      {:flight-id :flight/id
       :seat-id :seat/id
       :command-id :gesso/command-id}]}}})
```

The admitted command boundary allocates `:gesso/command-id` once, before the first possible submission, and the authoritative boundary establishes or validates its trusted binding as described for Claim. The model guards eligibility, the flight/seat association, and availability under the same atomic contract that records the reservation and command result. A pre-read may aid presentation or preparation but cannot replace that guard or authoritative command reconciliation:

```clojure
(defn reserve-seat!
  [{:keys [db principal now]} {:keys [flight-id seat-id command-id]}]

  (seat-model/transact-reservation!
    {:db db :now now}
    {:flight-id flight-id
     :seat-id seat-id
     :traveler-id (:user/id principal)
     :command-id command-id

     ;; New incorporation protects these transaction-current predicates.
     ;; A compatible replay is handled before applying new-command guards.
     :commit-guards
     [[:traveler/eligible? (:user/id principal) flight-id]
      [:seat/belongs-to-flight? flight-id seat-id]
      [:seat/offered? flight-id seat-id]
      [:seat/unreserved? flight-id seat-id]]

     ;; The scoped identity binds principal, operation, flight, and seat.
     ;; A compatible repeat returns the permitted recorded outcome,
     ;; subject to current disclosure authority, without another reservation.
     :deduplicate-by
     [:flight/reserve-seat (:user/id principal) command-id]
     :bind-arguments
     {:flight-id flight-id :seat-id seat-id}}))
```

The transaction API is schematic. Its realization must establish the stated guard, relationship, frame, and atomic command-record properties under the qualified backend contract. If ordinary Clojure computes eligibility outside the transaction, all facts on which it depends must receive equivalent commit-time protection; pre-read correctness alone is insufficient.

An aggregate transaction-guard failure may not identify which predicate failed. The model can return `:reservation/precondition-changed` when that is the strongest supported reason. A more specific reason requires authoritative evidence of the stated scope. A later reread can describe current availability but must not be presented as proof of the historical cause of an earlier transaction failure.

Before interpreting a failed attempt as command rejection, the outcome protocol must account for a compatible command incorporated by a concurrent attempt. When incorporation or delivery cannot be established, the participant retains `:unknown` and reconciles by the bound command identity. A timeout, a missing record at an insufficient read basis, or an unavailable dependency is not authoritative rejection.

A valid browser interaction may therefore begin from an authoritative observation at `T1` and still receive a typed rejection because Bob reserved the seat at `T2`. That rejection does not mean the earlier observation was fraudulent; it means the dynamic premise changed before the authoritative operation committed.

Preflight can check that `:rejected` is represented and that its reacquisition path exists. It must also distinguish the declared `:unknown` reconciliation branch from a fresh new-command retry. It cannot decide in advance which traveler wins the race. The example therefore makes the static/runtime boundary operational: dynamic uncertainty remains dynamic, while the permitted response to that uncertainty is still part of the checked application design.

Loss of a participant is another example. A physical network disconnect does not necessarily establish that the participant is semantically unavailable; temporary network failure, browser suspension, or reconnect may be ordinary execution conditions. The application therefore owns the policy that determines when loss of participation becomes semantically relevant. That policy may use a lease, heartbeat, deadline, authoritative presence state, explicit withdrawal, or another declared mechanism.

Once a participant is established to be unavailable, however, a workflow whose progress depends on that participant must not simply remain indefinitely dependent on an actor that can no longer satisfy its role. HumanHelp may decide that the affected Request becomes stale, becomes reassignable, returns to an available pool, remains assigned for a bounded lease, is cancelled, or follows another coherent application-defined path. Gesso does not choose among those policies, but absence of any declared handling is not itself an acceptable policy when the condition can leave the admitted workflow permanently non-serviceable.

The same distinction applies to transaction conflicts and external dependencies. A transaction guard may fail only at commit time, and an external service may become unavailable only during execution. Those facts need not be statically knowable. Their

possible effect on an admitted workflow still requires declared semantics whenever failure can invalidate the current trajectory or prevent required progress.

Some workflows may deliberately rely on an environmental event eventually occurring rather than define a timeout or alternate path. In that case the corresponding liveness claim is explicitly conditional on that environmental assumption. Omitting the failure or non-participation path must not silently transform a conditional progress claim into an unconditional completion guarantee.

When a required premise is known not to hold, the dependent liveness conclusion is unavailable. A conditional eventual-recovery assumption must not be used to claim progress for a participant already established to be permanently unable to satisfy the required role. Safety obligations continue to apply while progress is unavailable.

Local timeout, retirement, cancellation of a browser resource, and physical disconnect must not invent an authoritative command outcome. The command's status remains participant-relative until the required outcome evidence is available. Application recovery and current authoritative eligibility remain model-owned even if a retired or disconnected browser later resumes.

The governing rule is:

> Dynamic facts need not be statically knowable, but dynamically possible conditions that materially affect correctness must have declared semantics sufficient to preserve the application's stated safety and liveness claims.

Fail closed rather than silently downgrade semantics

A semantically managed operation must never become an ordinary weaker operation merely because required semantic metadata is absent.

Forbidden examples include:

Missing requirement	Forbidden silent fallback
Required authoritative basis	Emit the semantic mutation without its basis
Required browser plan	Guess a route and send anyway
Admitted browser-artifact correspondence	Run the managed plan without establishing compatibility
Required command envelope	Perform an ordinary mutation instead

If a semantically managed realization cannot be constructed, the correct behavior is a precise failure or an explicitly modeled unavailable operation. The unavailable state must preserve operation identity and the reason its realization is unavailable. It does not satisfy a milestone that independently requires the operation to be usable in an eligible scenario.

This rule does not make ordinary HTMX illegal. Ordinary non-semantic HTMX remains valid where the application intentionally requested an ordinary HTMX interaction.

The prohibition is against silent loss of previously declared semantics. Failing closed is relative to the effect boundary: before a protected send or write, the unsatisfied requirement prevents that effect. After an irreversible submission or incorporation may already have occurred, a failed response, render, or browser installation must not claim that the command was rejected or rolled back. It preserves the actual known outcome, or the applicable command-relative uncertainty, and follows the declared reconciliation and presentation-recovery paths. Restricting new work is not permission to erase durable obligations or to silently serve an older assembly whose required premises have become invalid.

Production boundaries carry the promised checks

A construction check contributes to an application's assurance only when the supported production path actually invokes it before the corresponding action becomes possible. An isolated checker test does not establish that relationship.

For a claimed production-enforced obligation, evidence must identify the actual installed boundary that admits the relevant object or action and demonstrate rejection of a representative controlled defect **at that boundary**. The test must identify the expected first enforcing boundary and the prohibited effect or exposure; rejection by an unrelated earlier wrapper does not certify the intended gate. The corresponding valid fixture must remain usable, so a checker that rejects everything cannot satisfy the requirement. For an effect-preventing check, evidence covers nonoccurrence of the prohibited effect at its specified scope. For a post-effect check, the honest result may instead be a withheld response and retained command uncertainty, not a fictitious rollback.

The detailed startup, handler, generated-artifact, and rendered-surface boundaries belong to the whole-application construction part. Their assurance classification follows the three application assurance claims.

Keep authoring vocabulary small and canonical

The ordinary path should have one obvious representation.

If five public APIs express the same semantic operation binding, humans and LLMs gain four extra ways to build inconsistent applications.

Prefer:

```
explicit data
+ strong derivation
+ strong validation
+ strong explanation
```

over:

```
implicit magic
+ duplicated configuration
+ weak runtime discovery
```

Ordinary Clojure behind an admitted contract is part of the normal path. Escape hatches for crossing a managed effect or authority boundary must instead be explicit, identify affected guarantees, and preserve unrelated invariants. An assertion is a declared premise, not a generic switch disabling checks.

The same principle applies to semantic-category distinctions. Stronger internal admission should reduce downstream ambiguity rather than impose a manual wrapping tax. Where Gesso can mechanically derive or construct an admitted value from declarations it already owns, ordinary application authors should not be required to construct nominal wrappers, certificate maps, transitive proof objects, or authority tokens by hand.

The desirable developer experience is therefore asymmetric:

Authoring situation	Required developer experience
Correct ordinary construction	Little additional ceremony
Semantic-category mistake	An early, precise diagnostic
Crossing a managed boundary manually	A deliberate API and explicit assurance impact

Errors are part of the public correctness surface

Important failures should have stable structured representations.

A useful error should answer, where applicable:

1. What failed?
2. Which semantic object or operation was involved?
3. Where was the obligation introduced?
4. Which semantic/proof/application edge failed?
5. What was required?
6. What was actually available?
7. Which known alternatives are legal?
8. Which assurance class was expected?
9. Which assumption, if any, would be required to proceed?
10. Is the result a contradiction, unsupported proof fragment, inconclusive check, inapplicable evidence, or failed runtime premise?
11. Is the reported fact about this attempt, the logical command, or a later authoritative observation?
12. Which boundary actually observed or established the failure, and which protected effects are known absent, known incorporated, or still uncertain?
13. Which subject, version, premise, and installed/observed scope make this diagnostic applicable, and what would require rechecking?

These questions describe a structured explanation contract, not a requirement for one universal error map. A construction refusal, a proof counterexample, an authoritative command rejection, an unknown command outcome, and a failed browser installation are distinct results even when they concern one action. A diagnostic must not convert one into another or infer the historical cause of a transaction failure solely from a later authoritative read.

Developer diagnostics may include more detail than a participant response. Disclosure policy must prevent errors or reconciliation endpoints from leaking protected records, prior outcomes, principal data, or existence information. Diagnostic detail is not an alternate authority channel.

Fuzzy suggestions may help diagnostics but must not silently alter security-sensitive execution.

Proof/check state is inspectable

A developer or LLM should not need to reconstruct application closure by mentally executing dozens of namespaces.

The finished system should be able to expose stable summaries of:

- semantic operation registry;
- role and authority declarations;
- projected plans;
- executable artifact identity;
- view affordances;
- transport bindings;
- trusted server capabilities;
- settlement/progression bindings;
- authoritative acquisition paths;
- open obligations;
- named assumptions;
- result classification, subject, scope, mechanism, and applicability for each guarantee;
- exact implementation and backend profile bindings where relevant;
- accepted assertions distinguished from discharged obligations;
- dependencies affected by changed or contradicted premises;
- the actual admitted writer/effect coverage and its remaining assumptions.

Inspection is diagnostic. It distinguishes declared/source-derived closure, issued construction, installed host or artifact bindings, invocation-specific runtime observations, and still-unestablished claims; a ”covered” entry must name which of those subjects its evidence actually covers. A source-derived inventory does not establish the presence of an unobserved writer or the adoption of a gate by the running process.

Inspection reuses the admitted source closure and already collected evidence under their stated applicability. It must not invoke potentially effectful application callbacks again merely to produce an explanation, and a copied inspection report is never an admission credential or runtime capability. Protected values, principals, and command outcomes remain subject to the applicable diagnostic disclosure policy.

Inspection does not create a new semantic input to runtime.

Stronger proof machinery must pay for itself

Every increase in proof complexity should answer:

> Which concrete malformed application, unsafe behavior, or required application behavior becomes impossible or immediately recognizable because of this addition?

Soundness work on an existing claimed guarantee also has concrete value: it establishes why the checker may be relied upon, even when it rejects no new surface syntax. Formal machinery should justify its complexity through such a guarantee, a useful class of accepted programs, or a recognizable defect. Where the only benefit is theoretical elegance without a claimed correctness, admission, correspondence, or usability improvement, defer the complexity rather than enlarging the trusted or developer-facing machinery by default.

The proved fragment grows from real correctness requirements. The application language remains the full host language. Improving proof coverage must not force authors to abandon useful Clojure computation or silently reclassify accepted assertions as proofs.

Part II — Semantic model

This part defines the semantic distinctions shared by choreography, authoritative operations, browser realization, and application assembly. It uses the assurance and admission vocabulary established in Part I. The definitions do not require a universal persisted fact graph, a new runtime class for every concept, or a single global revision coordinate.

The model applies to full-language applications under the checked construction contract. An opaque Clojure computation has an explicit semantic interface and retained premises; it is not excluded from the application because its implementation is outside the proof fragment. This part defines what the surrounding claims mean. Part IX owns their checking judgments, and Part X owns soundness and concrete correspondence. The intended semantics does not inherit incidental limitations of one runtime representation.

Examples use illustrative contract and API names unless explicitly identified as executable fixtures. Symbols such as R, U, B, and X17 stand for values bound by the surrounding example. An inspection map shows information that an admitted boundary must retain; writing that map does not establish admission. Later parts specify concrete construction, checking, and realization contracts.

Values and evolving values

A value may be a singular immutable value:

```
42
:open
{:user/id user-id}
```

A value may also be understood as evolving across a time, revision, causal, or transformation-step dimension.

For example:

```
foo = :bar at T1
foo = :baz at T2
```

or:

```
foo = :bar at step S1
foo = :baz at step S2
```

The observations themselves may remain immutable facts.

The evolving value is the relationship among those observations and the rules used to select or interpret one observation relative to a particular point of view.

”Current value” is therefore not an eternal mutable fact.

It is a derived view relative to some combination of:

- valid/effective time;
- causal or transformation relation;
- authoritative basis;
- authoritative progression;
- actor knowledge frontier;
- application policy.

For a HumanHelp Request:

```
F1: Request R was unclaimed at valid time T1.
F2: Request R was claimed by user U at valid time T2.
```

These are **semantic descriptions**, not a claim that a physical Request stores a `helper-id` field. HumanHelp's actual helper relationship is modeled by RequestAssignment. A valid interpretation of F2 must account for that relationship and its authorized temporal/observation context.

Both statements can remain true under the model's declared valid-time and correction semantics. The historical fact that an observation was made is distinct from a claim that its interpretation of the world can never be corrected. Recording an earlier observation does not make an erroneous domain assertion eternally true.

The Request's current state depends on the point from which the Request is evaluated and on later facts that supersede, refine, reconcile, or otherwise relate to the earlier history.

The framework must not confuse:

```
fact existed
fact became authoritative
fact became visible at a database frontier
actor learned fact
actor derived another fact
actor displayed representation of fact
```

These are distinct events and relationships.

Conceptual fact graph versus required persistence

The semantic model may reason in terms of facts and relations among facts without requiring Gesso to persist a universal fact DAG.

The conceptual graph is useful because it allows the design to reason about:

- provenance;
- supersession;
- derivation;
- authority;
- agreement;
- causality;
- command/result relations;
- provisional versus authoritative state.

Actual persistence remains owned by application models and their chosen authoritative systems.

For HumanHelp, XTDB and public model semantics remain the primary persistence and authority boundary.

Gesso must not introduce a duplicate universal fact store merely because the proof vocabulary can describe facts.

Immutability and correction

The statement that observations are immutable does not prohibit an application from correcting its model of the world.

A later authoritative fact may state that an earlier interpretation was wrong, superseded, withdrawn, or effective over a different valid-time interval.

What must remain explicit is the relationship.

The system should avoid interpreting absence from a current projection as evidence that an earlier observation never occurred. This semantic distinction does not require indefinite storage of every observation, prohibit legitimate data deletion, or introduce an audit-history retention policy. Persistence and retention remain model-owned.

This distinction becomes important for:

- reconciliation;
- idempotency;
- audit;
- stale-command interpretation;
- provisional settlement;
- authoritative reread;
- domain-specific corrections.

Semantic value versus representation

A semantic value is not identical to a wire encoding, DOM representation, XTDB object, or Clojure map merely because one of those represents it.

Representations may change while semantic identity remains stable.

Conversely, two values with identical serialized shape may have different semantic meaning because their:

- provenance differs;
- authority differs;
- basis differs;
- principal differs;
- operation context differs;
- plan/protocol compatibility identity differs.

Shape equality alone is insufficient when the semantic judgment depends on distinctions not established by that shape. Ordinary immutable values may still use ordinary value equality when the enclosing contract makes it sufficient. Gesso must not mistake matching schemas or encodings for equality of authority, provenance, subject, observation scope, or admission status.

Typed values, absence, and portable interpretation

The semantic interpretation of a value includes the type and context under which it is used. A nominal enum, currency amount, principal identifier, and resource identifier must not become interchangeable because their physical encodings overlap.

The following cases are distinct:

Case	Meaning
Field omitted from projection	No value was supplied for the field; its physical presence or absence may be unknown at this scope
Field established absent	A complete, admitted schema/read/guard judgment explicitly establishes absence of the field at its stated
Field present with nil	A present value permitted only when the type admits nil
Field present with false	A Boolean value, not absence
Fact unknown	The actor lacks a justified proposition; this is not the proposition's negation
Observation denied	The observation contract withholds disclosure; this need not establish either presence or absence
Resource authoritatively absent	A justified negative result within a declared subject, basis, time, and query scope

Omission from a partial projection must not be interpreted as known absence; only an applicable completed observation or admitted representation contract can establish that negative fact. A read that was denied, timed out, incomplete, or performed at an inadequate basis does not establish absence by returning no object. A complete negative observation is limited to its declared query, authority, visibility, time and basis; it is not a global nonexistence claim. Likewise, an absent **optional** field in admitted state differs from a missing **required** reader or malformed input, which must fail the applicable validation. A defaulting accessor must not collapse absent, nil, and false when the operation's guard, frame, or outcome depends on their distinction. The schema and acquisition contract determine which omissions have semantic meaning.

Portable numeric and value profiles specify ranges, overflow, equality, ordering, strings, nominal categories, and serialization. A mathematically unbounded integer in a proof is not automatically a safely represented browser number. A canonical encoding can normalize `1` and `1N` as the same mathematical integer without proving that every host arithmetic operation preserves that meaning. Concrete interpretation has its own target and correspondence contract.

Unsupported values at a wire or checked-IR boundary produce a typed rejection or use an explicitly admitted codec. This does not prohibit using those values inside an ordinary host-language computation whose exported interface is valid.

Representation, semantic category, and admission

Representation validity, semantic category, semantic admission, and proof are separate questions.

A Clojure map, record, scalar, wire value, DOM node, or other representation may carry a semantic value without the representation itself establishing the stronger status in which later code intends to use it.

The design therefore distinguishes conceptually:

Question	What a positive answer establishes
Does the representation have the required shape?	Structural validity under the selected schema
Does it belong to the required semantic category?	The required meaning, such as command rather than execution identity
Has it crossed the required admission boundary?	The boundary's checked bindings and applicability
Which propositions have been proved?	Only the checked conclusions under their retained premises

A positive answer to an earlier question does not automatically answer the later ones.

This distinction matters most where two values may have the same or nearly the same representation while differing materially in provenance, authority, lifecycle, or assurance status. Examples include:

- `ObservedBasisClaim` versus an authoritative basis established by trusted acquisition;
- `CommandId` versus `ExecutionId`;
- provisional versus authoritative projections;
- raw versus checked choreography declarations;
- projected plans versus admitted executable plans;
- raw certificate data versus checker-accepted certificate evidence;
- authenticated observation evidence versus an operation capability;
- historical observation evidence versus current authorization.

The supported Gesso APIs must not silently treat one such category as another merely because the underlying values have equal map shape, equal scalar representation, matching discriminator keys, or compatible Malli schemas.

This is an admission rule rather than a requirement to turn every semantic concept into a new runtime class. Permitted realizations include, where they satisfy the intended boundary contract:

- nominal types or records;
- opaque wrappers;
- smart or checked constructors;
- values reachable only through a verifier, elaborator, authoritative acquisition path, or registry lookup;
- tagged representations whose tags are validated by the appropriate trusted boundary;
- representations whose construction and use make the invalid substitution impossible or immediately rejectable through the supported path.

The design requires the semantic distinction and its enforcement at the supported boundary. It does not prescribe a nominal-type rewrite when a chosen mechanism provides the required protection.

Nominality is especially useful as misuse prevention inside cooperating trusted application code. For example, if `CommandId` and `ExecutionId` are both physically UUIDs, a supported API should not make accidental interchange look ordinary merely because both values satisfy a UUID schema. Similarly, a raw map containing the fields of a checked certificate must not become accepted certificate evidence solely by carrying a discriminator such as `:certificate/type` or `:verified? true`.

A boundary must retain or establish the information required to distinguish the categories it promises to check. If two untagged UUIDs reach a boundary with no trustworthy category or correlation information, UUID shape validation cannot reconstruct whether one originated as a command or an execution.

The supported construction path must therefore preserve the needed distinction through checked construction, typed or opaque values, declared field bindings, trusted correlation, or another mechanism that actually enforces the contract. Merely assigning different variable names to interchangeable unchecked inputs does not establish that protection. Framework-owned construction should derive these bindings wherever it already possesses the required information.

Nominal or opaque representation alone is not a security proof. A value called `VerifiedChoreography` is trustworthy only to the extent that its construction or admission path actually establishes the claims downstream code relies on. Likewise, a Clojure record constructor is not treated as a cryptographic or process-isolation boundary.

Crossing an untrusted serialization boundary also strips any purely in-process admission status. Conceptually:

1. A trusted boundary admits a server-side value under its declared premises.
2. Encoding and transmission produce bytes that may pass through an untrusted participant. The original in-process admission status does not travel merely because those bytes encode a familiar representation.
3. Decoding reconstructs data. Verification or trusted reacquisition must establish the premises required to admit that data for its new use.

Deserializing data into the same nominal-looking representation must not by itself restore authority, trusted provenance, certification, or admission.

Ordinary downstream application authoring should not pay unnecessary ceremony for these distinctions. Gesso should preferentially construct admitted values behind verification, elaboration, acquisition, projection, model, and assembly boundaries. Routine application code should not be required to manually wrap every Request id, title, status, or other ordinary domain value merely to satisfy framework ceremony.

The practical acceptance criterion is:

> Where two same-shaped values have materially different semantic meanings, accidental substitution through the supported API is either structurally unavailable or rejected at the earliest legitimate boundary with a diagnostic that identifies the expected semantic category, the supplied category, and the missing derivation or admission step. If the supplied category cannot be established, the diagnostic identifies it as unknown or unadmitted rather than inventing a trustworthy origin from an untrusted tag.

An expense-approval path shows what this means without requiring authors to create three record wrappers and a registry entry by hand for every value. The application declares an observation contract and supplies its model-owned acquisition implementation. Gesso owns admission and downstream checks.

```clojure
(def EmployeeValue
  [:map
   [:employee/id uuid?]
   [:employee/department keyword?]
   [:employee/risk-band [:enum :ordinary :enhanced-review]]])

(def ExpenseValue
  [:map
   [:expense/id uuid?]
   [:expense/employee-id uuid?]
   [:expense/department keyword?]
   [:expense/amount Money]
   [:expense/status [:enum :draft :submitted :approved :rejected]]])

(def approval-inputs-contract
  {:observation/id :expense/approval-inputs
   :input [:map [:expense/id uuid?]]
   :projection {:expense ExpenseValue :employee EmployeeValue}
   :purpose :expense/approval-preparation
   :authorization :expense/observe-approval-inputs
   :bindings
   [[:= [:expense/id :expense] [:expense/id :arguments]]
    [:= [:employee/id :employee] [:expense/employee-id :expense]]]
   :consistency :compatible-authoritative-read-context
   :implementation #'expense-model/read-approval-inputs!})

(defn observe-approval-inputs!
  [ctx expense-id]
  ;; The contract is resolved from the admitted application. Authentication
  ;; and read authorization come from ctx, never from observation arguments.
  (observation/acquire!
    ctx
    :expense/approval-inputs
    {:expense/id expense-id}))

(defn approval-facts-eligible?
  [ctx expense-id observed]
  (let [{:keys [expense employee]}
        (observation/require-for!
          ctx observed
          {:contract :expense/approval-inputs
           :arguments {:expense/id expense-id}
           :purpose :expense/approval-preparation})]
    (and (= (:expense/id expense) expense-id)
         (= (:employee/id employee) (:expense/employee-id expense))
         (= :submitted (:expense/status expense))
         (= (:expense/department expense) (:employee/department employee))
         (= :ordinary (:employee/risk-band employee)))))
```

The `Money` reference uses Part I's illustrative nominal USD-cents profile. It is not automatically an admitted Part IX scalar merely because a Malli schema names it. An actual checked realization must bind the supported nominal representation/codec and reader interpretation, or carry a separate permitted opaque implementation premise. The `:bindings` field paths above are also **schematic semantic-interface constraints**, not direct syntax of Part IX's closed `[:read reader-id]` rule AST. If used for a checked-rule judgment they require a defined, checked elaboration; otherwise they remain explicit contracts with their applicable checking/enforcement premises. Unsupported elaboration cannot be reported as successful rule checking.

Schema Vars and the observation map are declarations, not admission credentials. The public API names above illustrate the required contracts rather than freeze a particular namespace layout.

The admitted observation establishes the facts returned for this preparation purpose. The sample eligibility function does not

establish that its caller is a manager or may approve an expense. That is a distinct authorization decision owned by the authoritative approval operation. If the function is opaque to the checker, its functional contract remains explicitly asserted even though its input admission and relationship checks can be enforced.

The acquisition/admission boundary must establish all of the following:

Obligation	Establishing boundary
Caller identity	Trusted authentication bound to the actual context
Permission to disclose	Model-owned read authorization before releasing protected data
Requested Expense	The acquired Expense id matches the admitted argument
Related Employee	The acquired Employee id matches that Expense's employee relationship
Representation	Non-optional validation of the declared result schemas
Observation meaning	Correct authority, projection contract, parameters, temporal context, and compatible read basis
Permitted use	Current applicability of the admitted observation for this caller and purpose
Realization fidelity	Proof, runtime enforcement, or retained assumptions connecting the acquisition implementation to its contract

A single compatible read context may be supplied by a model operation or by several reads under an admitted consistent-acquisition contract. In particular, reading an Expense from one basis and its Employee from a later incompatible basis cannot satisfy the combined relationship merely because the records each passed schema validation. Separate successful reads alone do not establish a coherent combined snapshot. If that premise cannot be established, the boundary rejects the stronger combined claim or exposes separately scoped observations for a contract that permits them.

An implementation may use an opaque object, trusted issuance registry, or other checked representation for the observation. Its admission binding includes the result material, contract identity, subjects, basis, temporal/query context, principal context, purpose, and applicability required by the contract. A record class or `:admitted?` tag alone is insufficient. A process-local handle may be valid for a local invocation; continuation across hosts requires a declared shared, revalidated, or server-verifiable mechanism. None of this requires a universal persistent observation store.

Now compare an observation with client-carried data containing equal values:

```
(def observed
  (observe-approval-inputs! ctx expense-id))

;; Illustration: the browser returns the same domain values it received.
;; Parsing and shape checking are allowed; they do not restore admission.
(def returned-values
  {:expense expense-value
   :employee employee-value})

(= (observation/value observed) returned-values)
;; ⇒ true in this fixture

(approval-facts-eligible? ctx expense-id returned-values)
;; ⇒ rejected: no admitted observation for the required contract and context

(approval-facts-eligible? ctx expense-id observed)
;; ⇒ evaluates the sample eligibility rule under its retained premises
```

Reconstructing a record with a familiar class or copied fields must fail the same boundary unless the actual admission binding is re-established. Conversely, there is no need to reject an ordinary scalar solely because it was serialized when its use requires only shape-checked value equality. Admission requirements follow the claimed meaning, not a rule that all decoded values are unusable.

A browser may also return an observation receipt when the protocol explicitly supports one. Verification can establish the receipt's exact historical issuance claim and applicability; it does not silently supply a new current observation or approval authority. Otherwise the supported path reacquires authorized facts.

Finally, preparation is not incorporation. The eventual approval command must recognize compatible replay and, for a new incorporation, protect the Expense, Employee relationship, eligibility facts, and current approval authority under the atomic contract. An earlier admitted observation is not a substitute for that protection. A successful old approval result likewise does not prove that the Expense's present state remains unchanged.

The practical authoring surface remains an acquisition contract and ordinary functions. The richer admission representation is framework-owned, inspectable, and proportionate to the claims made about its use.

Business rules and the auditable semantic surface

A correctness-relevant business rule should not exist only as an incidental condition buried inside arbitrary implementation code.

For Gesso's purposes, a **business rule** is a named semantic proposition, relation, precondition, postcondition, transition restriction, authorization condition, disclosure condition, or other application judgment whose truth or applicability affects a correctness-relevant operation.

Possible application rules include the following. This list illustrates the kind of policy that belongs on the surface; it does not assert that each rule has already been adopted by HumanHelp:

- a Request may be claimed only while it is open;
- the claimant must have an active Membership in the Request's Organization;
- a requester may not claim their own Request;
- a completed Request is terminal;
- a protected observation may be disclosed only to an authorized principal;
- a command may be incorporated only under the authority and argument binding established for that command.

The semantic rule is distinct from any one Clojure expression that happens to implement or test it. A condition such as:

```
(= :active (:membership/status membership))
```

may participate in the realization of a rule such as `:membership/active-for-request-operation`. By itself, however, this expression establishes only that the particular `membership` value supplied to it has status `:active`. It does not establish whose Membership it is, whether it belongs to the Request's Organization, whether the authenticated principal is the claimant, or whether the Membership is still active when Claim commits. Those bindings and timing requirements belong in the rule's meaning. The expression is one implementation step; the named rule is the semantic proposition whose meaning should remain stable across refactoring, generated realization, formal translation, diagnostics, and audit views.

A Gesso application therefore has an **auditable semantic surface**: the compact set of named rules, operations, authorities, lifecycle transitions, invariants, knowledge/agreement requirements, and explicit trusted premises that determine its correctness-relevant behavior.

That surface should normally be substantially smaller than the application's full implementation. A large application may contain tens or hundreds of thousands of lines concerned with rendering, formatting, queries, adapters, integration plumbing, logging, caching, and incidental computation while its business semantics consist of a much smaller collection of rules and operations. Gesso should preserve that compression rather than require an auditor to rediscover policy by rereading the entire implementation.

The requirement is not that every conditional expression become a business rule. Conditions concerned only with presentation mechanics, formatting, local optimization, or other behavior outside the claimed semantic boundary need not be promoted merely because they are written with `if`, `when`, or a predicate call.

A condition is a strong candidate for the semantic surface when it controls or materially contributes to matters such as:

- authorization or eligibility;
- protected disclosure or observation;
- ownership or responsibility;
- lifecycle admission or transition;
- authoritative mutation;
- transfer of money, value, quota, inventory, or another conserved resource;
- command acceptance, incorporation, settlement, or retry meaning;
- establishment or use of authoritative knowledge;
- a domain invariant or cross-operation consistency rule.

The application may represent such rules using ordinary Clojure data, pure functions, checked constructors, existing model contracts, thin macro sugar, or another representation that preserves their semantic identity. This design does not require a second policy language or a macro-heavy pseudo-language. What matters is that the supported Gesso path can identify the rule and the semantic judgment it contributes, rather than knowing only that some opaque branch occurred in host code.

A rule representation may conceptually carry information such as:

```
{:rule/id :request/claim-requires-active-membership

 ;; Meaning:
 ;;   The authenticated helper must have an active Membership in the
 ;;   Organization of the Request being claimed when Claim commits.
 ;;
 ;; Rationale:
 ;;   Claiming transfers operational responsibility for the Request.
 ;;
 ;; Correctness significance:
 ;;   Principal binding, Request/Organization association, Membership
 ;;   ownership, and active status must be mutually compatible and
 ;;   protected or re-established at the authoritative commit boundary.
 ;;   A stale browser or server pre-read is not sufficient.

 :proposition
 [:and
  [:= [:membership/user-id]
      [:authenticated-principal/user-id]]
  [:= [:membership/organization-id]
      [:request/organization-id]]
  [:= [:membership/status] :active]]

 :subjects
 {:principal :trusted-authenticated-principal
  :request :current-authoritative-request
  :membership :current-authoritative-membership}

 :required-at :request/claim-new-incorporation

 :applies-to
 #{:request/claim}}
```

The `:proposition` field paths in this sketch are **semantic-interface notation**, not expressions directly admitted by Part IX's closed, reader-based `[:read reader-id]` grammar. In that baseline, a comparable active-status conjunct would use a declared `:membership/status` reader, an explicit `[:read :membership/status]`, and `[:literal :active]` under its nominal status type. A supported frontend must check the elaboration of the source meaning, operand types, and bound subjects before it may report that this stronger rule is mechanically well typed or established. Without that translation, the interface may still identify an explicit opaque contract and its permitted premises, but it is not certified merely because the vector is recognizable. The corresponding principal, Request, Organization, and transaction-current relationship obligations remain separate from checking the status conjunct.

The exact representation above is illustrative; its `:subjects` and `:required-at` entries express obligations, not evidence that current facts were acquired or that a transaction guard was implemented. The three field tests must refer to the **same** Membership, the trusted current principal, and the Request selected by this Claim. Part IV determines how an authoritative realization protects those facts and their relationship at commit. The semantic requirements are that the rule has stable identity, that its checked meaning **where supported**, or its explicitly conditional interface where opaque, is available to the parts of Gesso that rely on it, and that enough correspondence is retained for later assembly, diagnostics, certification, and formal artifacts to refer back to the same rule. The business meaning and binding belong to their public model contract; choreography, views, and generated checking products must not redeclare subtly different eligibility policies. An opaque Clojure predicate with the same name does not make that proposition mechanically established. Its contract is an ordinary supported dependency whose accepted assertions must remain visible. The new-incorporation rule is not imposed on a compatible replay merely because the same public operation processes both cases.

Business rules should also be generously explained in ordinary source. Where practical, the declaration or function that defines a rule should be surrounded by retained explanatory metadata and source comments explaining at least:

- what the rule means in domain terms;
- why the rule exists;
- which subtle correctness boundary matters to its enforcement;
- any intentionally surprising exception or asymmetry.

Comments improve human and model review, but comments alone cannot be premises of a mechanical assurance claim because ordinary Clojure reading/compilation need not preserve them. Prose that must survive elaboration into generated semantic inventories, diagnostics, or assurance artifacts must therefore be retained as checked metadata, documentation strings, source metadata, or another explicit representation. The machine-checkable rule declaration remains distinct from its explanatory prose.

This produces two complementary views of the same rule:

```
machine-checkable rule declaration
    : request/claim requires a current active Membership held by the
      authenticated helper in this Request's Organization at commit

human-auditable explanation
    : what "active" means here, why it matters, and why the check must
      participate in the authoritative transition rather than rely on an
      earlier browser observation
```

Neither view should silently substitute for the other. Explanatory prose does not prove the rule, and a terse machine predicate is not sufficient by itself for an application intended to remain auditable by humans.

When a correctness-relevant decision necessarily depends on application code whose semantics Gesso cannot inspect or derive, that dependency should still have an explicit semantic name and boundary. It may be represented as an opaque or trusted premise rather than being allowed to disappear into an unnamed implementation branch. Later certification and whole-application assembly determine what assurance may be claimed from such a premise; this part requires only that the semantic dependency itself remain visible.

The completeness of an **audited** semantic surface is a separate, scoped correspondence claim. The source-owned operation, writer, authorization, disclosure, and decision inventory must be related to the actual paths covered by the selected checking or inspection profile. A closed generated subset may support complete coverage of that subset; an analyzer that did not inspect an arbitrary Clojure body cannot establish the absence of hidden policy branches by finding none. An intentionally opaque body is permitted under a named implementation/effect/dependency contract and admission policy, with its uninspected internals retained as premises rather than falsely classified as checked rules. Known consequential decisions outside every admitted rule, runtime-enforced premise, or approved opaque contract are semantic-policy escapes and must be reported. An uninspected inventory is **unknown**, not empty. Part IX owns the precise evidence and rejection rules, including source-derived mandatory obligations; this section does not create a second author-maintained policy registry.

The semantic-surface acceptance criterion is:

> A reviewer should be able to inspect the correctness-relevant business policy of a Gesso application as a compact set of named semantic rules and explicit premises without reconstructing that policy from unrelated implementation code. Refactoring may change the physical Clojure realization without erasing or silently changing the identity of the rule being realized. Any correctness-relevant dependency that cannot be represented or derived by the supported semantic path remains explicit rather than being mistaken for an assured rule merely because opaque host code returned a truthy value.

Rule identity, interpretation, and opaque implementation

A stable semantic name identifies a rule's place in the application. It is not by itself an identity of unchanged meaning. A change to relied-upon types, bindings, temporal requirements, effects, or guarantees changes the exported semantic interface even when its `:rule/id` remains the same.

The interface identity uses a declared versioned canonical representation. Map/set ordering and irrelevant formatting may normalize; vector order, meaningful string content, nominal types, presence, and other semantic material must not be silently erased. Normalization must reject unsupported material or collisions rather than treat different subjects as the same declaration.

Opaque Clojure has a separate implementation binding. Its contract identity does not prove that binding implements the contract, and Gesso does not require deciding equivalence of arbitrary functions. A concrete implementation change may preserve a theorem about the interface while requiring new correspondence evidence or assumption applicability for the realization.

The semantic surface must expose the inputs and effects on which a meaningful decision depends. A deterministic predicate over supplied authoritative values has different premises from a function that reads a clock, consults an external service, or writes state. All remain expressible; concealed dependencies do not inherit the guarantees of an effect-free contract. A result observation, a contract assertion, and a universal proof have different scopes.

Facts, time, and authoritative progression

The design distinguishes at least the following dimensions.

Valid/effective time

Valid or effective time describes when a domain fact applies in the modeled world.

Examples:

- Membership became active on one date;
- a Request was open during one interval;
- Organization policy changed at a domain-effective time;
- a correction applies retroactively to an earlier period.

Valid time belongs to domain meaning.

Recording time and correction history

Recording or system time identifies the authoritative history under which a fact was recorded, corrected, or withdrawn. It differs from the valid/effective time at which that fact applies in the domain. A correction recorded today may change the authoritative interpretation of an earlier valid-time interval.

An observation that depends on both dimensions must retain both, directly or through its admitted read contract. A read basis that bounds transaction visibility does not alone select the valid time or historical correction view. The database provider defines the supported relation between these coordinates under its qualified backend profile.

Domain or causal relation

Domain or causal relations describe how semantic facts relate.

Examples include:

- based on;
- produced by;
- superseded by;
- reconciled by;
- rejected by;
- confirmed by;
- corrected by;
- derived from.

These relations are not necessarily database transaction order.

Authoritative database progression

Authoritative database progression describes the consistency/basis frontier at which authoritative facts are visible.

For HumanHelp's primary persistent authority, this is supplied by the admitted XTDB provider under its qualified backend contract. The provider owns token interpretation and the evidence for a comparison or read guarantee. Generic choreography code does not decode an XTDB token and invent those semantics independently.

A basis is scoped to its authority/provider context and relevant database incarnation or equivalent identity. Equal raw tokens from different databases, tenants where applicable, or reset histories are not automatically comparable.

A provider's **minimum visibility requirement** and the **actual basis of an acquired observation** are different: a read may be required to include a confirmed effect at or beyond B1 and return an observation from B2. B1 is not thereby the exact snapshot used, and B2 does not establish that the Request's current value equals its immediate post-commit value. The adapter must expose which relation it established—an exact observation coordinate, an admitted lower bound, or only a requested minimum—and retain the query/temporal scope. A transaction submission, allocated log position, or claimed commit token is not by itself evidence that a particular domain command incorporated. Part IV owns the concrete commit-to-visibility contract and its backend qualification.

An authoritative progression value may advance because unrelated data changed.

Therefore:

```
database advanced
≠
this Request changed
```

Actor knowledge frontier

An actor knowledge frontier describes what authoritative information a particular runtime actor has acquired through its admitted observation paths. It is scoped by the facts, authority, projection, and relevant temporal context of those observations. It need not be one token covering all actor knowledge.

For the supported browser implementation, the frontier records semantic acquisition and incorporation by that implementation. It is not a server proof that an adversarial browser or the human using it retained or understood the information.

Two browser actors may observe the same authoritative database at different frontiers.

The same principal may therefore have two browser tabs with different knowledge.

The database's current state and an actor's current knowledge are distinct. Acquired evidence, content selected for adoption, and content successfully installed in a presentation may also be distinct. A protocol must identify which transition advances each claimed frontier; merely scheduling a read or allowing a DOM swap does not establish its completion.

Physical delivery time

Physical delivery time may matter operationally without becoming semantic authority.

An SSE notification may arrive after another message.

A callback may fire after an execution has been retired.

A browser may wake after a deployment.

The fact that an event arrived later in wall-clock time does not by itself make the event semantically newer or authorized to mutate current state.

Do not collapse dimensions into one revision integer

The framework must not collapse:

```
valid time
recording/correction time
domain fact identity/version
causal relation
XTDB progression
actor knowledge frontier
command identity
execution identity
artifact compatibility identity
```

into one global monotonically increasing application revision.

A global revision number would erase distinctions that matter to correctness.

For example, XTDB may advance while Request R is unchanged.

Likewise, Request R may have meaningful valid-time history independent of when a database transaction was recorded.

A browser may hold knowledge through frontier X17 while authority has progressed to X23 without any relevant Request fact changing.

That is ordinary distributed execution, not necessarily staleness requiring rejection.

Progression as provenance

When established by an **admitted authoritative acquisition**, a basis supplies provenance evidence approximately of the form:

> The acquired representation is justified by an authoritative read with the stated subject, query/visibility and temporal interpretation at this admitted basis or justified lower bound.

The actor may subsequently choose that representation for command preparation. A browser-carried `:observed-basis` field alone is instead a claim about its starting context; it does not establish that the browser actually acquired, installed, adopted, or used that representation. A protocol requiring evidence of prior observation needs its separately validated issuance/receipt contract as described in authoritative observation evidence. The distinction does not prohibit using a client-supplied basis as nonauthorizing protocol context for stale-input handling.

It does not mean:

- the entire database is still at that frontier;
- every fact the command depends on is unchanged;
- the client owns authority over that frontier;
- the client can prove merely by sending the token that it really performed the observation;
- the framework should reject every command formed from an older frontier.

The authoritative-operations and progression part defines the exact command, scope, and XTDB semantics, including scoped progression requirements.

At the ontology level, authoritative progression is one provenance coordinate among several and is not a universal domain revision.

Scoped progression decisions and requirement composition

A comparison result binds its exact operands, comparison context, and admitted provider contract. Generic code can validate the representation and binding of a witness such as ”B2 advances B1”; that check does not prove the assertion is true. The provider or another declared reasoning mechanism must establish its meaning, with any backend assumptions retained.

An explicit comparison distinguishes equivalence, advancement, precedence, known incomparability, and unavailable or invalid comparison evidence where the contract supports them. Equality of opaque bytes is not provider-verified equivalence across incompatible authority, incarnation, temporal interpretation, or relevant scope. Failure to establish an ordering is not proof of incomparability, and incomparable observations need not be individually invalid. A comparison does not automatically prove that a read covers a particular domain effect or that a selected query was complete; those are additional observation and commit-correlation obligations.

Pending acquisition requirements compose as a conjunction of scoped obligations. A conservative representation retains an orderless set of distinct requirements and removes exact duplicates. Replacing it with one basis, taking a numeric maximum, or discarding a requirement as subsumed needs an admitted coverage/implication argument. No join or total order is assumed merely because the provider can compare some pairs of bases.

An observation discharges a requirement only when its authority, subject, projection/query scope, temporal meaning, principal context where relevant, and progression satisfy that requirement. A later card read cannot by itself discharge a board-membership acquisition requirement. Part IV owns the concrete algebra and provider contract.

Bounds on retained requirements must preserve this meaning. Coalescing may produce a conservative stronger reacquisition obligation when the contract justifies it. Silently dropping debt to meet a memory bound is not permissible. When no sound bounded representation is available, the runtime follows its declared backpressure, retirement, or reacquisition policy.

Roles, principals, actors, hosts, and authorities

These concepts are distinct and must remain distinct throughout declaration, projection, runtime execution, proof, diagnostics, and application assembly.

Role

A **role** is a static position in a choreography.

Examples:

```
:helper
:request-authority
:observer
```

A role describes a position in the distributed protocol.

A role is not automatically:

- a user;
- a browser tab;
- a process;
- a web server;
- an authorization grant.

An actor may realize the projected behavior of a role only through an admitted binding. That binding is not itself a grant of business authority. The authenticated principal's permission to exercise that role for a subject and operation is determined at the applicable trusted boundary; dynamic business permissions must also be protected where the authoritative operation incorporates its effects. A browser-selected role label or actor ID cannot supply that permission.

Principal

A **principal** is the identity or entity whose authority is being exercised.

Examples:

```
[:user user-id]
[:service :humanhelp]
```

A principal is a semantic/security identity.

A principal is not identical to a browser actor.

One principal may have:

- multiple browser tabs;
- multiple devices;
- multiple protocol executions;
- retries through multiple hosts.

Those actors may hold different knowledge while acting for the same principal.

Actor

An **actor** is a concrete participant carrying a local projected machine, execution state, or knowledge state.

Examples:

```
[:client client-id]
[:service-instance logical-service-actor-id]
```

An actor gives semantic identity to a participant beyond one physical callback or request.

The exact runtime realization may be short-lived.

An actor identity does not require a permanently resident process.

Host

A **host** is the physical runtime that happens to execute a step.

Examples:

```
[:web-node node-1]
[:web-node node-3]
```

Hosts are replaceable physical realizations.

A retry may run on another host without changing:

- semantic command;
- principal;
- authority;
- logical operation;
- relevant actor identity, where the protocol preserves it.

A host does not acquire semantic authority merely because it executes authority code.

Authority

An **authority** is the semantic locus entitled to establish some class of authoritative fact.

An authority is named by an authority identifier such as:

```
[:authority :request]
[:authority :request request-id]
```

The identifier does not itself contain an authoritative fact, nor does it name a particular database record or server process. It names a locus in the application's semantic model. Application rules, together with Gesso's execution rules, determine which facts and transitions that locus is entitled to establish and under which premises.

For example:

```
[:authority :request R]
```

may govern authoritative facts concerning Request `R`, such as its current lifecycle state, ownership, or the incorporation of a command affecting it. The concrete facts may be represented by one or several XTDB documents and may depend on other authoritative observations, such as Membership or Organization state. Those storage objects help realize the authority; they are not themselves its semantic identity.

This distinction permits the physical representation to change without changing what the application means by:

```
[:authority :request R]
```

For example, the same authority could be realized using a different document layout, a different database, or a different web node while continuing to govern the same class of semantic facts.

Authority is therefore distinct from both **storage location** and **execution location**.

A physical web node may temporarily realize an authoritative operation without becoming the authority itself.

For a Request operation:

```
semantic authority
    [:authority :request R]

first host
    [:web-node A]

retry host
    [:web-node C]
```

Node `A` and node `C` may each exercise the same semantic authority while executing different attempts of the operation. Process-local identity or process-local memory does not determine the authority.

The authority's logical identity can survive host replacement. The realization must still establish continuity of authoritative state, command records, issuer bindings, and applicable backend semantics. Merely reusing the authority name after a database reset or migration does not establish that continuity. Old and new progression values need an admitted comparison or migration relation before they can be used together.

Consequently, any fact that must survive retries, failover, or migration cannot derive its semantic force merely from having been remembered by the same process. It must instead be recoverable from authoritative state, from evidence validly issued on behalf of the semantic authority, or from another mechanism whose semantics are explicitly admitted by the application and Gesso.

Server is not a universal semantic identity

”Server” is normally an implementation location or a role category, not one unique actor identity.

Multiple web nodes may realize the same logical authority while holding different process-local state.

The design therefore avoids proofs or runtime assumptions that rely on:

```
same server process
= same semantic authority
```

or:

```
different server process
= different semantic authority
```

Neither equivalence is generally valid.

Relationship among role, principal, actor, host, and authority

A useful conceptual example is:

```
role
    :helper

principal
    [:user U]

actor
    [:client C]

host of the browser-side actor
    user's browser/device

authoritative role
    :request-authority

semantic authority
    [:authority :request R]

host currently realizing authority
    [:web-node A]
```

A later retry may change the final line while leaving the semantic operation otherwise intact.

Verification and diagnostics should identify which category an identity belongs to instead of overloading one generic ”id” field wherever practical.

Authentication establishes the principal binding. The operation's trusted admission and model policy establish whether that principal may instantiate a particular role for a particular subject. Actor and host identity provide correlation or realization information; neither substitutes for those decisions. Changing a browser tab, transport connection, or physical host must not silently carry permissions into a different principal or authority scope.

An authentication or principal-context instance may also need identity distinct from the principal itself. Logout, account switching, credential replacement, or a declared revocation transition can invalidate pending work even when a later session represents the same principal. The applicable contract states which bindings survive and which must be re-established. No universal session epoch or global counter is required.

Identity correspondence evidence must name its actual issuer and trust boundary. A typed principal value, a matching command ID, or equality between caller-supplied maps does not authenticate a request, authorize an actor, or establish that the recorded semantic arguments were the ones admitted. The physical request path must derive the principal from trusted authentication, validate the command's operation, subject and arguments against the admitted binding, and disclose which authorization and backend premises have actually been established. Structural identity agreement alone is not a certificate of authority.

A collaborative editor makes the categories easier to follow when several change independently. Suppose Alice is permitted to edit document D. The following is inspection data, not a credential accepted from the browser. The fixture symbols U, D, C1, C2,

E19, E20, C77, and C88 stand for prebound values of their respective identity kinds. The Clojure expressions illustrate the identity relationships; they are not a standalone application, a credential, or a checked authorization proof.

```
(def editor-session
  {:semantic/role :editor
   :principal [:user U]
   :principal-context :session/alice-1
   :actor [:client C1]
   :actor-host [:browser-context :alice-tab-1]
   :authority [:authority :document D]
   :operation :document/save
   :command C77
   :execution E19
   :transport-request-id :http/204
   :authority-host [:web-node :A]})

(def retry-session
  (assoc editor-session
    :transport-request-id :http/205
    :authority-host [:web-node :C]))

(= (select-keys editor-session
     [:principal :principal-context :actor :authority
      :operation :command :execution])
   (select-keys retry-session
     [:principal :principal-context :actor :authority
      :operation :command :execution]))
;; ⇒ true
```

The operation's retry contract preserves command C77, including its admitted semantic arguments. The authority host and transport request change. The browser actor's own host does not thereby become node C, and neither physical location authenticates the principal.

Alice opens a second tab and forms a separate save intent:

```
(def second-tab
  (assoc retry-session
    :actor [:client C2]
    :actor-host [:browser-context :alice-tab-2]
    :command C88
    :execution E20
    :transport-request-id :http/206))

(= (:principal retry-session) (:principal second-tab))
;; ⇒ true
(= (:authority retry-session) (:authority second-tab))
;; ⇒ true
(= (:actor retry-session) (:actor second-tab))
;; ⇒ false
(= (:command retry-session) (:command second-tab))
;; ⇒ false
```

These maps isolate identity relationships; they do not implement the save operation's argument binding or admission checks. The new intent has its own command identity. If the second tab instead reconciles C77, it may retain C77 only through that command's binding and reconciliation contract. Copying a session map is not a command-transfer or authorization protocol.

The authoritative server path authenticates each invocation independently of the inspection fields. The following ‘auth/current-principal¡ and ‘document-model/save¡ functions denote application-owned contracts, not APIs that this section claims Gesso already provides:

```
(defn invoke-save!
  [{:keys [session] :as ctx}
   {:document/keys [id content] :gesso/keys [command-id]}]
  (let [principal (auth/current-principal! session)]
    (document-model/save!
      (assoc ctx :principal principal)
      {:document-id id
       :content content
       :command-id command-id})))
```

The admitted route binds this entry to the save operation. The public model operation validates command binding and current disclosure permission for replay, and current editing authority for new incorporation. It protects the relevant state dependencies atomically with a new save and its command record. An authorization check made earlier in a handler is insufficient if the relevant permission can change before incorporation.

If the Organization revokes Alice's editing permission, a new save intent is rejected under the current policy. An already incorporated C77 is handled by its replay protocol: the recorded result is returned only if current disclosure policy permits it. Revocation must not cause a successful old save to be misreported as never having happened, nor must historical success bypass the new disclosure decision. If outcome evidence is unavailable, the caller's knowledge may remain unknown.

The principal, actors, and semantic authority can remain the same while the authorization relation changes. Their continued existence does not imply continued editing rights. Category-specific diagnostics identify which binding or permission changed rather than reporting an unexplained generic id mismatch.

Identity-sensitive admission and replay counterexamples

The collaborative-editor scenario also has distinct negative cases that a complete application check must cover. They are not reducible to comparing identity fields in a handler or to checking that the projected role exists:

- **Revocation before new incorporation.** If Alice's current editing permission is revoked before a new save is admitted at the qualified authoritative incorporation boundary, the new effect is not admitted. A prior positive handler check cannot replace transaction-current protection of a dependency that can change concurrently. An unauthorized caller also must not be able to create a durable terminal rejection for somebody else's command.
- **Revocation racing with incorporation.** The outcome must follow the application's qualified atomicity and conflict semantics for the relevant authorization dependencies. A previously incorporated effect is not undone by a later revocation; neither a process-local ordering guess nor a browser timestamp may decide which authoritative transition won.
- **Previously incorporated C77 after revocation.** Compatible, authorized read-only resolution uses C77's original principal, operation, subject and semantic-argument binding, with a fresh current disclosure decision. It does not rerun the save, and a disclosure denial does not turn historical success into a claim that the command never happened.
- **Identity or argument substitution.** C77 cannot be transferred to another principal or silently rebound to new content, an operation, or an authority by changing browser-supplied fields. Incompatible or unauthorized resolution is not permission to disclose the recorded result or its existence.
- **Lost or indeterminate response.** Missing, expired, or inaccessible command evidence may leave incorporation unknown. It is not evidence of rejection, and neither an alternate host nor a fresh request grants an automatic resubmission right.
- **Session or host change.** A second browser tab, account switch, credential replacement, or retry through node C requires the applicable current trusted bindings. Stable command or authority identity cannot carry permissions across principal contexts or database incarnations without the declared compatibility and recovery contract.

These are normative acceptance and correspondence obligations, not a claim that a current Gesso gateway already authenticates principals, command arguments, session context, or backend causality. Application assembly and assurance reports must retain any such undischarged premises explicitly.

Logical identity and physical realization

Logical identity must not be inferred from incidental physical identity.

The following identities are distinct:

```
role identity
principal identity
actor identity
authority identity

semantic operation identity
semantic command identity
protocol execution identity

global choreography identity
projected-plan compatibility identity
exact executable artifact identity

logical fragment/target identity
logical resource/continuity-slot identity
concrete effect/resource-instance identity
concrete continuity-capture identity

transport-request identity (one transport attempt)
backend submission-attempt identity
scoped capability-instance identity
authentication/principal-context identity where required
physical DOM-node identity
web-node/host identity

admitted assembly generation identity
deployment identity
```

Some applications may not need every category in every operation.

The architecture must nevertheless avoid conflating categories merely because one implementation currently uses the same value for two of them. Resource- instance identity is the general category; a transport-request identity or continuity-capture identity may be a specialized instance of it. Listing these categories does not require multiple independent tokens for the same instance. The representation must preserve whichever distinctions the admission contract actually uses.

A checkout operation puts most of those identities into one compact lifecycle. The following structure is intentionally verbose so the independence is visible:

```
(def checkout-attempt
  {:operation/id
   :checkout/place-order

   :command/id
   #uuid "11111111-1111-1111-1111-111111111177"

   :execution/id
   #uuid "22222222-2222-2222-2222-222222222219"

   :plan/id
   :plan/checkout-place-order-v3

   :artifact/id
   "illustrative:browser-artifact-A12"

   :assembly/generation
   :assembly-A12

   :deployment/id
   :deploy-2026-09-21-1

   :cart/id
   #uuid "33333333-3333-3333-3333-333333333333"

   :order/id
   nil

   :transport
   {:transport-request-id
    #uuid "44444444-4444-4444-4444-444444444204"

    ;; Fetch callback ownership is distinct from HTTP attempt correlation.
    :resource-instance/id
    #uuid "55555555-5555-5555-5555-555555555051"

    :host
    [:web-node :A]}})
```

Suppose the request connection disappears before the browser knows whether the command committed. A permitted retry might preserve the semantic command while replacing the transport attempt and its resource instance:

```
(def retry
  (→ checkout-attempt
     (assoc-in
       [:transport :transport-request-id]
       #uuid "44444444-4444-4444-4444-444444444205")
     (assoc-in
       [:transport :resource-instance/id]
       #uuid "55555555-5555-5555-5555-555555555052")
     (assoc-in
       [:transport :host]
       [:web-node :C])))

(assert
  (= (:command/id checkout-attempt)
     (:command/id retry)))

(assert
  (= (:execution/id checkout-attempt)
     (:execution/id retry)))

(assert
  (not=
    (get-in checkout-attempt [:transport :transport-request-id])
    (get-in retry [:transport :transport-request-id])))

(assert
  (not=
    (get-in checkout-attempt [:transport :resource-instance/id])
    (get-in retry [:transport :resource-instance/id])))
```

A different recovery strategy might retire the original execution and begin a new execution that still tries to reconcile the same command:

```
(def reconciliation-execution
  (→ retry
     (assoc
       :execution/id
       #uuid "22222222-2222-2222-2222-222222222220")
     (assoc-in
       [:transport :transport-request-id]
       #uuid "44444444-4444-4444-4444-444444444206")
     (assoc-in
       [:transport :resource-instance/id]
       #uuid "55555555-5555-5555-5555-555555555053")))

(assert
  (= (:command/id retry)
     (:command/id reconciliation-execution)))

(assert
  (not=
    (:execution/id retry)
    (:execution/id reconciliation-execution)))
```

If authorized command-outcome evidence links this command to its created Order, a domain identity enters the picture without replacing the earlier identities. Finding a plausible Order by an unrelated query would not establish that link:

```
(def reconciled
  (assoc reconciliation-execution
    :order/id
    #uuid "66666666-6666-6666-6666-666666666483"))

{:command (:command/id reconciled)
 :execution (:execution/id reconciled)
 :order (:order/id reconciled)}
;; ⇒
;; three different semantic coordinates
```

The `assoc` above illustrates a domain-coordinate relationship only. It does not acquire evidence that this Order was created by command C77. Attribution requires an admitted command/outcome record or equivalent issuer-qualified correspondence, not a matching Order found by an independent query.

And a deployment may change the admitted assembly or artifact compatibility without saying anything about whether command 177 committed:

```
(def after-deploy
  (recovery/start-admitted-execution!
    current-context
    {:assembly admitted-assembly-A13
     :operation :checkout/place-order
     :command-reference
     (commands/require-bound-reference!
       current-context (:command/id reconciled))
     :purpose :command/reconciliation}))

;; A new admitted execution uses A13's actual artifact and bindings.
;; The command's identity and recorded incorporation do not change.
```

The recovery boundary checks compatibility, current principal/authority scope, and the command's recorded binding. It does not blindly associate a new artifact name into an old execution. This example chooses a new execution; a contract that permits continuation of an existing one must explicitly define that transition and the resource ownership it preserves or retires.

The checkout maps are also inspection fixtures. The `assoc` expressions show the intended identity relationships; they do not grant the ability to create a reconciliation execution or establish command-outcome evidence. The admitted runtime owns those transitions. The `assert` forms demonstrate fixture expectations, not optional production guards. The artifact label above is illustrative, not a valid physical SHA-256 artifact receipt. The `recovery/start-admitted-execution!` and `commands/require-bound-reference!` names are schematic admitted-runtime interfaces; this section does not claim that those Vars exist in Gesso. A runtime that exposes corresponding functionality must actually check the recorded binding, issuer, compatibility, current disclosure rights, and ownership transition.

The point is not that every runtime map must literally contain every field. The point is that a design using one generic `generation` or one generic `request-id` could not faithfully answer all of these questions:

- is this the same semantic intent?
- is this the same protocol trajectory?
- is this the same transport attempt?
- does this callback belong to the current physical resource?
- is this browser artifact compatible with the admitted assembly?
- which Order, if any, was authoritatively created?

The identities exist because those questions have different answers.

Semantic command identity

A semantic command identifies one logical intent to invoke an operation under its bound semantic arguments and authority context. It is distinct from each attempt to deliver or incorporate that intent.

It may survive:

- network retry;
- host replacement;
- a new protocol execution;
- another transport attempt;

where operation semantics permit that retry.

The command identity and uncertainty contract in the authoritative-operations part defines the relevant retry and attribution conditions. Possessing the same command identifier alone does not establish that a retry is permitted or that the command already committed.

The authoritative binding covers the operation/interface compatibility context, canonical semantic arguments, authenticated principal, and authority or tenant scope required by the operation. A new business intent receives a new command identity; retrying or reconciling an existing intent preserves its binding. Changing bytes in an identity-bearing argument is not a harmless transport retry unless the declared canonicalization treats the arguments as equivalent.

Command-record retention and identity reuse have a declared horizon. Forgetting a record must not silently make an old command eligible for a second incorporation under an unlimited deduplication claim. Expiry, tombstones, namespace retirement, or another explicit protocol may bound that guarantee. Absence of an outcome record is not, by itself, proof that no pending attempt can commit.

Protocol execution identity

A protocol execution identifies one realization of a distributed trajectory.

Two executions may relate to the same semantic command without being the same execution.

Their resources and callbacks must not be interchangeable merely because the semantic command matches. A protocol execution can span several local steps, transport requests, and backend attempts. Replacing or resuming it has a declared ownership and compatibility contract; a new HTTP request alone does not determine whether the execution is new.

Plan compatibility identity

A projected plan may remain in a browser across deployment.

Compatibility identity answers whether that suspended or retried projection is safe to continue under the current server/runtime contract.

Compatibility does not necessarily require exact artifact equality. It must nevertheless follow an explicit supported compatibility rule. A runtime must not infer compatibility from matching names, approximate version ordering, or the absence of a visible error.

Exact executable artifact identity

Diagnostic and proof correspondence may require a stronger identity than runtime compatibility.

The exact executable artifact identity answers:

> Does this diagnostic or proof metadata describe these exact emitted runtime coordinates and transitions?

A plan/protocol compatibility identity and an exact executable digest therefore serve different purposes. Evidence tied to exact emitted coordinates must not be attached to a merely compatible artifact as though exact correspondence held. Conversely, a compatible continuation need not be rejected solely because diagnostic-only material changed, if the declared compatibility contract permits it and any exact correspondence claim remains accurate.

A byte digest and matching build receipt establish only the stated physical correspondence, conditional on the trusted build and issuer boundary. A digest does not prove that the emitter compiled the claimed semantics, that the resulting JavaScript refines the admitted browser machine, or that a browser will execute it faithfully. Stronger executable correspondence requires the separately established build-source and implementation obligations. Neither a matching digest nor a certificate carried by an untrusted browser grants authoritative permission.

Admitted assembly generation and deployment identity

An admitted assembly generation identifies the immutable application realization accepted for use under the construction contract in Part VII. Its bindings include the relevant routes, prepared server operations, providers, rendering rules, executable plans, and artifact correspondence.

Deployment identity identifies a physical rollout or placement, if the implementation needs that distinction. One deployment may host more than one admitted assembly generation during a supported transition. Moving an admitted realization between hosts does not, by itself, change its semantic identity or establish a new compatibility relation.

Neither identity substitutes for command, execution, or resource-instance identity. Two concurrent executions using the same assembly generation still have separate ownership. Replacing a resource inside one execution need not change that execution's plan or the application's assembly generation.

Transport-request identity

A transport-request identity identifies one transport attempt. In browser/runtime examples its canonical field name is `:transport-request-id`. A new retry attempt has its own transport identity even when the command identity is retained under the operation's retry contract. The execution-to-attempt relationship is declared by the supported realization rather than inferred from shared field values.

HumanHelp's domain `:request-id` identifies a Request being acted upon. It does not identify an HTTP request. Domain arguments and route parameters retain that meaning; they must not be renamed as transport identities.

Event correlation identifies a candidate owner/attempt. Possessing a matching transport-request identifier alone does not establish semantic admission, current ownership, authorization, or command outcome.

Submission attempts and scoped capabilities

A submission attempt is one attempt by an authoritative realization to incorporate a command. Several attempts may correspond to one command, and an HTTP request may cause zero, one, or several attempts according to its declared retry contract. An attempt's rejection does not necessarily reject the command if another compatible attempt incorporated it.

A capability instance grants a scoped operation under an issuer, subject, principal or delegation binding, applicability period, and use policy. A one-shot writer capability can prevent repeated use of that grant without preventing a fresh grant for the same command. Durable command deduplication, capability consumption, and transport correlation are separate contracts.

Capabilities used by asynchronous work have explicit owners and lifetimes. Transfer to a background execution must be admitted; merely retaining a request-local value after its request ends does not extend its authority.

Exact process-local submission-return correlation plus a read of domain, command-record, and ledger facts at one pinned basis can establish bounded co-observation. Even when all those identities match, co-observation alone does not prove that the observed transaction caused those facts, authenticate the request-bound principal or arguments, authorize a retry, or settle the browser's optimistic state. Every stronger attribution claim must state the additional trusted issuer, submission, and transaction-causality premises it discharges. Failure to establish them is `not-established`, not a favorable command outcome.

Cross-identity counterexamples and diagnostic obligations

Application construction, runtime admission, and assurance reporting must distinguish these cases rather than accepting an apparently matching field:

- The same `:command/id` arrives with changed canonical cart contents, operation, principal, or authority scope: it is not a compatible retry of the recorded semantic intent.
- The command ID is familiar but its retained record is missing, expired, or inaccessible: absence is not a rejection or proof of nonincorporation. The declared retention/recovery contract determines whether safe reconciliation remains possible.
- A callback names the former execution or transport attempt after that owner has retired, even if the command is still live elsewhere: it cannot regain the retired owner's semantic rights. Part II's resource-instance and owner-relative admission rules govern the precise check.
- The plan label matches after deployment but executable bytes, assembly generation, or scoped binding differ: plan-name equality supplies neither exact artifact correspondence nor permission to resume an old execution.
- The same semantic authority name appears on a replacement host or newly initialized database: authority continuity requires qualified state and command-record lineage, not name or host equality.
- An Order with plausible attributes exists after an uncertain checkout: its existence is not command-attributed incorporation without admitted evidence linking that exact semantic command and outcome.

A checker should identify the mismatched identity category and the evidence boundary that could resolve it. It must not collapse `unknown`, `incompatible`, `unowned`, `unauthorized`, and `not-established` into a fabricated success or a claim that the command was authoritatively rejected.

Logical target identity

A logical UI target may survive replacement of its physical DOM node.

For example, a Request card may remain the same logical fragment after HTMX replaces the element representing it.

The runtime must not use physical-node identity as the sole semantic target identity when representation replacement is expected. A target also has an applicable actor, principal, projection, and ownership context. The same CSS id after navigation or account switching does not imply the same semantic target. Changing a query or projection meaning creates a new scope or requires an explicit compatibility transition.

Resource instance identity

A replaceable physical or runtime resource must be distinguishable from prior instances of the same logical resource when events from those prior instances could still arrive.

For example:

Event	Current instance for logical slot S	Permitted consequence
Resource created	R1	R1 may act within its admitted ownership
Resource replaced	R2	R1's rights over the replacement are retired
Callback from R1 arrives	R2	Reject mutation of R2-owned state through R1's retired ownership

Resource instance identity answers a narrow question:

> Does this event belong to the currently admitted instance of this replaceable resource?

An instance ID must be associated with the physical producer by the registration and callback-closure boundary. Merely finding an equal value inside event data does not prove that association. The owning runtime records which producer and lifecycle supplied the expected identity, while event correlation checks the claim against that registration. The trustworthiness of those registrations remains an explicit host/runtime premise, not a property established by equality of two untrusted maps.

A resource-instance generation counter, scoped epoch, opaque token, UUID, object identity, or another suitable mechanism may represent resource-instance identity. The representation must provide the properties required by the boundary; additional ordering or structure is unnecessary unless the contract uses it.

A resource-instance generation counter is appropriate when the contract uses monotonic ordering between successive instances within that resource scope. A distinct opaque identity is sufficient when the only required question is whether an event belongs to the current instance.

The distinction must remain valid for as long as callbacks from an earlier instance can still reach the boundary. Reusing a token, resetting a counter, or wrapping a resource-instance counter must not make a retired instance indistinguishable from its replacement. A reset is safe only when a separate scope or lifecycle boundary prevents old events from being admitted as current.

Resource instance identity must not be used as a compressed representation of independent semantic facts.

An event may be inadmissible because it belongs to a retired resource instance, but it may also be inadmissible because its command, execution, plan, artifact, authoritative basis, ownership, principal, authorization context, or another required semantic correspondence no longer matches the current operation.

Those distinctions remain separate even when several of them happen to change at the same time.

For example:

Coordinate	Before replacement	After replacement
Continuity slot	S	S
Resource instance	R7	R8
Execution	X12	X12
Authoritative basis	B40	B40

A late callback carrying R7 is rejected because that resource instance is retired, even though the execution and basis still match.

In this case the execution and authoritative basis may still correspond. The callback is rejected specifically because it belongs to the replaced physical resource instance.

Conversely:

Coordinate	Before authoritative advance	After authoritative advance
Continuity slot	S	S
Resource instance	R8	R8
Execution	X12	X12
Authoritative basis	B40	B41

The provider justifies the basis relation. An advance of authoritative state does not by itself replace the physical resource.

The resource instance has not changed, but an operation that requires basis B41 must not treat possession of R8 as evidence that B40 remains admissible.

This separation prevents a resource-instance counter, epoch, or opaque identity from becoming a universal proxy for semantic freshness. A numerical comparison is meaningful only within the declared counter scope and ordering contract; identities from different resource or execution scopes cannot be ordered merely because both happen to be integers.

A browser Refresh race shows how narrow that identity can be in executable code. Assume a logical board target remains `:request-board` while HTMX or the Gesso adapter replaces the physical fetch/resource used to refresh it.

The runtime might hold state resembling:

```
(def initial-board-runtime
  {:target
   :request-board

   :execution/id
   #uuid "70000000-0000-0000-0000-000000000012"

   :current-resource
   {:resource-instance/id
    #uuid "71000000-0000-0000-0000-000000000007"

    :kind :authoritative-acquisition

    :scope
    {:projection :request/board
     :organization-id
     #uuid "72000000-0000-0000-0000-000000000001"}

    :status :in-flight}

   :adopted-basis :B40})
```

Before Refresh A completes, the component is reconstructed and starts Refresh B:

```
(def after-replacement
  (→ initial-board-runtime
     (assoc
       :current-resource
       {:resource-instance/id
        #uuid "71000000-0000-0000-0000-000000000008"

        :kind :authoritative-acquisition

        :scope
        {:projection :request/board
         :organization-id
         #uuid "72000000-0000-0000-0000-000000000001"}

        :status :in-flight})))

(def refresh-a-result
  {:event/class :acquisition-complete
   :target :request-board
   :execution/id
   #uuid "70000000-0000-0000-0000-000000000012"
   :resource-instance/id
   #uuid "71000000-0000-0000-0000-000000000007"
   :basis :B41
   :html "<div>older acquisition</div>"})

(def refresh-b-result
  {:event/class :acquisition-complete
   :target :request-board
   :execution/id
   #uuid "70000000-0000-0000-0000-000000000012"
   :resource-instance/id
   #uuid "71000000-0000-0000-0000-000000000008"
   :basis :B42
   :html "<div>current acquisition</div>"})
```

These abbreviated completion maps expose only the fields needed for the owner race. A complete normalized response also carries, or is bound by registration to, the expected source and observation contract, query/temporal context, principal context, and projection material. Bare `:basis` and `:html` fields cannot establish those premises.

A narrow owner check can reject A for exactly the reason that matters, but it must not call that narrow result semantic admission:

```
(defn current-resource?
  [runtime event]
  (let [resource (get runtime :current-resource)]
    (and
      (= :in-flight (:status resource))
      (some? (:target runtime))
      (some? (:resource-instance/id resource))
      (= (:resource-instance/id resource)
         (:resource-instance/id event))
      (= (:target runtime) (:target event)))))

(defn same-execution?
  [runtime event]
  (and
    (some? (:execution/id runtime))
    (= (:execution/id runtime)
       (:execution/id event))))

(defn correlate-acquisition-owner
  [runtime event]
  (cond
    (not (same-execution? runtime event))
    {:result :owner-rejected
     :reason :wrong-execution}

    (not (current-resource? runtime event))
    {:result :owner-rejected
     :reason :retired-resource-instance}

    :else
    {:result :owner-correlated
     :event event}))

(correlate-acquisition-owner
  after-replacement
  refresh-a-result)
;; ⇒
;; {:result :owner-rejected
;;  :reason :retired-resource-instance}

(correlate-acquisition-owner
  after-replacement
  refresh-b-result)
;; ⇒
;; {:result :owner-correlated
;;  :event ...}
```

That result establishes only the displayed equality and lifecycle checks for this supplied runtime state and event. Their truthful source and correspondence to the executing callback remain separate premises. Full semantic admission is a distinct boundary. Conceptually it can consume the correlated completion only after the rest of the acquisition contract has also been established:

```
(defn admit-acquisition-completion
  [trusted-context runtime event]
  (let [{:keys [result] :as owner}
        (correlate-acquisition-owner runtime event)]
    (if (not= result :owner-correlated)
      owner
      (acquisition-admission/admit!
        {:trusted-context trusted-context
         :runtime runtime
         :event event

         ;; Resolved from the registered acquisition owner, never from
         ;; an event-supplied selector or a caller-selected check list.
         :contract
         (acquisition/require-registered-contract!
           trusted-context runtime)}))))
```

The resolved contract determines the mandatory checks. The illustrative `acquisition/require-registered-contract!` and `acquisition-admission/admit!` names describe trusted-runtime contracts, not public Gesso Vars guaranteed to exist. In particular, the runtime selects the contract from its previously admitted acquisition registration; the event cannot select or replace it. These checks include the expected acquisition plan, authorized principal context, source provenance, observation scope, compatible progression, current resource owner, and permitted installation right. Event fields and arbitrary callers cannot remove a check. Owner correlation alone cannot make received HTML semantically admissible for installation. Content escaping, active-content policy, and DOM effect safety remain additional realization obligations; provenance is not sanitization.

The final state transition checks the latest owner and applicability, rather than relying on a correlation result cached before an asynchronous gap. If physical installation happens later, its effect carries the admitted owner binding and the shell rechecks the required installation right at actuation. An allowed installation is not automatically a completed installation. A synchronous implementation may combine them under its correspondence contract; otherwise completion or failure is admitted as a separate event.

Notice what the owner guard does **not** infer. It does not claim that B42 is universally newer than every basis in the application. It does not prove that the returned HTML is authorized for some other user. It does not establish that the command which caused an invalidation succeeded. It establishes only that this completion belongs to the currently selected resource instance and execution. The later admission boundary establishes the additional premises required before downstream code may treat the completion as an admitted authoritative acquisition result.

A different event class can have a different semantic owner. For example, a shared advisory is not execution-private:

```
(def shared-advisory
  {:event/class :board-advisory
   :owner
   {:subscription/id
    #uuid "73000000-0000-0000-0000-000000000031"

    :source-instance/id
    #uuid "74000000-0000-0000-0000-000000000031"

    :scope
    {:organization-id
     #uuid "72000000-0000-0000-0000-000000000001"
     :projection :request/board}}

   :requirement
   {:kind :authority-may-have-advanced
    :scope
    {:organization-id
     #uuid "72000000-0000-0000-0000-000000000001"
     :projection :request/board}}})

(def advisory-runtime
```

```
  {:board-subscription
   {:subscription/id
    #uuid "73000000-0000-0000-0000-000000000031"

    :source-instance/id
    #uuid "74000000-0000-0000-0000-000000000031"

    :scope
    {:organization-id
     #uuid "72000000-0000-0000-0000-000000000001"
     :projection :request/board}

    :status :open}})

(defn admit-board-advisory
  [trusted-context runtime event]
  (let [subscription (:board-subscription runtime)
        owner (:owner event)]
    (if-not
      (and
        (= :open (:status subscription))
        (some? (:subscription/id subscription))
        (some? (:source-instance/id subscription))
        (= (:subscription/id subscription) (:subscription/id owner))
        (= (:source-instance/id subscription)
           (:source-instance/id owner))
        (= (:scope subscription) (:scope owner)))
      {:result :owner-rejected
       :reason :wrong-subscription-or-source}
      (advisory-admission/admit!
        {:trusted-context trusted-context
         :runtime runtime
         :event event
         ;; The registered subscription fixes the shared-event contract.
         :contract
         (subscription/require-registered-contract!
           trusted-context subscription)}))))
```

The subscription check is only owner correlation. Shared semantic admission also checks the declared event class, source provenance, full scoped requirement, and permitted advisory transition under the registered contract. A client-supplied `:owner` or `:event/class` does not by itself prove any of those premises. The illustrative `subscription/require-registered-contract!` and `advisory-admission/admit!` names likewise denote semantic boundaries, not established exported Gesso APIs.

This short example fixes one board projection with no additional query parameters. A query-specific board must retain those parameters and its visibility context. The tokenless requirement shown here does not invent an acquisition lower bound; its registered contract must define a sufficient read/source-fence protocol. The later board example uses an advisory carrying an authoritative lower bound.

Assigning that advisory to whichever command execution happens to be current would be a category error. Conversely, marking a private callback as `:shared? true` in untrusted event data must not bypass the execution/resource checks above.

This is the executable consequence of owner-relative admission: first determine which declared semantic owner governs the event class, then apply the correspondence checks that owner's transition actually needs.

Adversarial ownership and actuation cases

A whole-application checker and the browser refinement tests must be able to distinguish the following cases. These outcomes concern cooperating runtime semantics; security against a hostile browser remains a server responsibility.

Scenario	Required classifica
Refresh A's callback reaches the target after B replaces A's resource	Reject A's retired
Cancellation or cleanup for A finishes after B starts	Cleanup is best ef
The event repeats B's resource ID but comes from an unregistered producer, another principal, or another board scope	ID equality alone
B's own callback carries a view from an incompatible projection or an insufficient authoritative observation	Owner may correl
A swap is admitted, then its owner retires before a delayed DOM effect executes	Recheck the instal
A valid shared subscription advisory arrives while no command execution is active	Check the subscri
A private callback asserts that it is shared after its execution retires	Reject the untrust

The event's normalized class and registered producer determine which check is applicable. Cancellation cannot stand in for semantic retirement, a matched request ID cannot stand in for producer provenance, and a previously admitted effect cannot stand in for current permission when the physical action runs. A rejected late callback must not erase or roll back a newer owner's installed state. These obligations must be tested at the race boundaries, not only on isolated identity-value fixtures.

Required correspondence checks are semantic gates

At every runtime boundary, an event must satisfy the particular identity and correspondence conditions required by that boundary.

Depending on the operation, those conditions may include:

- current resource instance identity;
- command identity;
- execution identity;
- plan or artifact identity;
- authoritative basis;
- ownership;
- authenticated principal;
- current authorization;
- continuity or provisional ownership state.

A boundary need not check every category. It must check the categories on which its correctness actually depends.

These checks are not merely debug assertions. They determine whether a runtime event is semantically admissible. A matching identifier establishes only the specific correlation it was issued to establish. It cannot substitute for producer provenance, current owner rights, authoritative progression, or an operation-specific transition guard.

Failure of browser event admission does not establish that a corresponding authoritative command was rejected or failed to commit. A callback may be stale while durable command incorporation remains unknown or has already succeeded. The browser may retire that callback without manufacturing a business outcome; a separately admitted reconciliation path owns any later command resolution.

The design therefore does not require one universal generation value. It requires independent semantic distinctions to remain independently representable and requires each runtime event to satisfy the correspondence conditions appropriate to the operation it is attempting to affect.

Admission follows the declared semantic owner

Every asynchronous event capable of influencing Gesso semantic state passes the common semantic admission boundary. This describes a required semantic gate across supported paths, not a demand that every event class use the same function or owner record. The boundary first resolves the event's declared class and semantic owner from the admitted producer/registration, then applies the correspondences required by that class. A normalized event tag may describe a candidate event, but does not by itself attest the producer or its authority.

Conceptually:

1. Resolve the declared event class and owner from the registered producer and admitted contract.
2. Check owner-specific correlation and current applicability.
3. Check that the proposed semantic transition is permitted.
4. Admit that transition, or classify the rejection and any recovery path.

The admission contract fixes the required checks; no event field or arbitrary application callback can select a weaker set. A missing, retired, or incompatible registration cannot silently fall back to another owner class. Any allowed alternate recovery route is separately declared and checked.

The owner is not universally a command execution.

Event class	Relevant owner and checks
Execution-private callback	Admitted execution, current resource instance, ownership, correlation, and permitted execution state
Shared fragment advisory	Declared subscription or fragment coordination scope, admitted source/correlation, current source instance where required, and valid advisory requirement
Authoritative acquisition completion	Admitted acquisition owner, request/resource correlation, returned observation scope and progression, and applicable installation rules
Continuity completion	Declared continuity owner or slot, current resource instance, and restoration rights still held for the target
Transport or asynchronous local completion	Execution and effect-generation ownership, expected attempt or pending effect, and permitted continuation
Timer firing	Declared timer owner and timer generation, current execution where required, and an outstanding timer registration
Optimistic settlement or timeout	Admitted optimistic scope and execution/timeout ownership, compatible settlement evidence, and permitted disposition

This table identifies distinctions rather than prescribing one public event record or a closed enumeration of all future events. Each supported event class must define its own required correspondence contract.

An idle board can therefore admit a legitimate shared advisory without an unrelated command execution. A shared event must not be assigned to whichever execution happens to be current. An execution-private callback must not become shared merely because its execution was retired.

The declaration of an event as shared is part of the checked contract. An untrusted field such as `:shared? true` cannot select weaker admission rules. Shared admission permits only the declared shared transition; it does not restore a retired execution's private ownership or authorize installation of domain content carried by an advisory.

The browser-realization part defines the concrete normalization, transition, and effect contracts implementing this boundary. All supported paths obey this owner-relative rule; there is no alternate callback path that may mutate semantic state while claiming that admission was enforced.

The event's class is resolved from its registered producer and admitted contract, not selected by an untrusted `:event/class` field alone. Event payloads may carry correlation values; the callback/resource registration supplies the corresponding expected owner. Final applicability is checked against current state. Checking identity early and mutating a replacement owner later would leave a race. Event-level admission and physical effect actuation may be separated by an asynchronous interval; in that case the effect must carry the relevant admitted owner/correlation and be rechecked where its physical interpreter can still touch replaceable state. The existence of an earlier allow-effect is not a permanent installation or restoration capability.

Retirement removes the retired owner's semantic rights immediately, even if physical cancellation or cleanup is incomplete. Late command-outcome evidence may still be useful through a separately declared reconciliation path. Such a path does not revive the old execution or authorize its stale presentation effect.

Negative cases for owner-relative admission

The following are distinct counterexamples that construction coverage, browser-transition testing, and physical correspondence work must preserve:

Event or effect	Required treatment
A private callback reclassifies itself as shared after retirement	Do not accept the asserted owner class;
A timer or transport callback matches an execution ID but not its current effect/timer generation	Treat it as stale; do not resume the new
An advisory matches the board scope but has no admitted subscription/source registration	Do not infer producer provenance or a
An acquisition completion matches its resource but carries unqualified basis, scope, or content	Correlation alone cannot authorize autl
A continuity restore or swap is permitted before its owner is superseded and actuates afterward	Revalidate the physical installation/rest
A command's response is lost, then its browser execution retires	Retire browser effects without classifyi

Admission diagnostics must distinguish a stale or unowned callback, an unsupported or missing contract, insufficient observation evidence, and an actual authoritative operation outcome. These are not interchangeable typed results. The detailed error vocabulary belongs to the relevant supported transition and recovery contract; this section does not invent one universal error schema.

These browser-runtime claims concern the admitted, cooperating implementation. They are not a server-verifiable proof that an adversarial browser ran those checks. The authoritative server boundary independently enforces the security properties stated below.

Trust boundaries and adversarial browser actors

A browser is an untrusted participant.

This statement is stronger than saying browser input should be validated.

The user controls the machine and may modify:

```
gesso-live.js
projected choreography data
local machine state
DOM
hidden fields
command envelopes
role names
principal ids
observed basis values
provenance labels
provisional/authoritative markers
available buttons
bound capabilities
diagnostic/proof sidecars
runtime correlation fields
```

Therefore no browser-originated value may confer trusted authority merely because it has the expected representation.

What the browser may propose

A browser may propose values such as:

- a semantic operation;
- operation arguments;
- a semantic command identity;
- an observed-basis claim used as protocol context;
- correlation information whose validity is checked independently;
- provisional local state affecting only its own representation;
- a projected-plan identity whose compatibility is checked by trusted code;
- transport data required to correlate a request with a server-side operation.

Proposal is not authority.

What the browser may not establish by assertion

The browser may not establish by assertion:

- authenticated principal identity;
- entitlement to a choreography role;
- model authority;
- authorization to operate on a protected target;
- authoritative provenance;
- an authoritative fact;
- a privileged server-machine state;
- permission to read protected data;
- permission to subscribe to protected Live state;
- the truth of a proof certificate;
- that a stale execution is current;
- that a provisional projection is authoritative;
- that an observed-basis claim proves prior observation.

Trusted server code must derive or check those propositions through trusted boundaries. An identity-shaped value, successful browser preflight, valid projection, signed diagnostic, or previously acquired result does not establish current authorization. The server must distinguish trusted request identity, current operation/subject permissions, and disclosure rights for each protected result. An application-selected ‘trusted‘ flag or caller-owned proof record cannot discharge any of those obligations.

Trusted authoritative execution

For each protected authoritative operation, the supported trusted path must establish the obligations relevant to the branch it actually executes:

1. Authenticate the concrete caller from trusted context and resolve the operation through the admitted application binding.
2. Validate the command envelope, identity binding, arguments, and compatible operation interface. Client fields do not supply privileged continuation state or current authority.
3. Handle a compatible recorded command under its current disclosure policy, without imposing a new-incorporation guard on historical success.
4. For new incorporation, acquire the required facts and atomically protect current authorization, relationships, guard dependencies, and domain effects together with the command-result record.
5. Account for concurrent compatible incorporation when interpreting a failed attempt, and preserve uncertainty when authoritative evidence is insufficient.
6. Produce only the permitted typed outcomes and scoped progression evidence, with disclosure and recovery rules appropriate to the current caller.

These are branch-specific requirements, not instructions to repeat a mutating transaction for a recorded command. A compatible historical command is reconciled from qualified authoritative evidence under a current disclosure decision; a fresh incorporation must establish its applicable guards at the actual incorporation boundary. A failed attempt does not certify command rejection when another attempt might have committed, and missing or inaccessible command evidence does not authorize a blind retry.

A checked Ring route, issued writer handle, typed identity, or successful source-level guard report is not, by itself, proof that the deployed server used that route, the authoritative transaction caused the observed result, or every external writer was excluded. Those are separately scoped physical correspondence and environment obligations.

Preparation checks are protected or re-established at incorporation wherever intervening changes could invalidate the decision. Browser freshness does not supply that protection. Read-only operations and subscriptions follow their own acquisition/disclosure contracts rather than manufacturing a write protocol.

The server must never treat a client-supplied machine continuation such as:

```
{:next-state :request-authority/commit-admin-operation}
```

as evidence that the client is entitled to reach that state.

Instead the browser requests a semantic operation, conceptually:

```
{:gesso/command-id C42
 :operation :request/claim
 :arguments {:request/id R}
 :observed-basis X17}
```

and trusted server semantics decide what operation and continuation are permitted. Here ‘C42‘, ‘R‘, and ‘X17‘ are schematic fixture values, not authenticated credentials or an admitted continuation. In particular ‘:observed-basis‘ is protocol context, not proof that the browser observed that basis.

Security boundary

A malicious browser can always lie to itself.

It may render false information, suppress buttons, manufacture buttons, change labels, or locally pretend an operation succeeded.

Preventing a user from corrupting their own local representation is not the security target.

The security target is:

> Compromising or replacing a browser projection may corrupt that browser's local representation and protocol participation, but must not increase the authority of its authenticated principal, disclose data unavailable to that principal, or produce an authoritative model transition unavailable to that principal.

This is a verification target, not merely an application coding convention.

Read and subscription authorization

Hiding content in a view is not access control.

Not rendering a button is not access control.

Not advertising a Live interest is not access control.

Every protected authoritative observation and subscription must be authorized from trusted server context.

An opaque or signed token may protect integrity, correlation, or accidental mutation.

Possession of such a token is not automatically authorization unless an explicit capability design says that possession has that semantic meaning and its applicability and validity conditions are satisfied.

The protected-observation and subscription contracts must state when authorization is checked and what later permission changes imply for subsequent disclosure. Initial subscription admission must not silently become an indefinite permission grant. The protocol may use revalidation, bounded grants, revocation, or another explicit policy consistent with the claimed protection. No such policy can make an adversarial browser forget data already disclosed. Invalidation metadata, counts, existence hints, and command-outcome responses can themselves disclose protected information and need an appropriate scope. A hidden card does not make an otherwise revealing subscription harmless.

The check applies at each supported disclosure boundary, not only when the first page or subscription is created. Subsequent fragment reads, notification payloads, subscriptions, command reconciliation responses, and error messages must satisfy the applicable current or explicitly bounded authorization and information-disclosure policy. Where non-disclosure of resource existence is required, success/denial categories, counts, timing-dependent metadata, and reconciliation diagnostics must not become an unreviewed existence oracle. This does not promise secrecy against every side channel; the concrete claim must name its protected outputs, observation model, and environmental premises.

Proof artifacts are untrusted in the browser

A proof sidecar or diagnostic certificate shipped to a browser cannot become a security capability.

An adversarial browser may fabricate, modify, remove, or replace it.

Authoritative security therefore depends on trusted server-side state and checked protocol boundaries, not the browser's possession of proof metadata.

This follows the general rule:

```
evidence about why code was accepted
≠
runtime authority to perform operation
```

Adversarial boundary and evidence cases

The following cases are acceptance and correspondence obligations, not proof that a current implementation already prevents each attack.

Adversarial action or uncertain event	Required trusted treatment
Client changes principal ID, role, tenant, command operation, or subject in its envelope	Resolve authenticated identity and admi
Client reuses a recorded command ID with different semantic arguments, principal, or authority	Do not transfer the historical result or p
Editing permission is revoked after preparation but before a new guarded write	Protect or re-establish the changing autl
Client fabricates a newer basis, successful settlement, or privileged continuation	Do not turn browser assertions into autl
Client forges subscription scope or keeps using a Live stream after permission changes	Require trusted scoped admission and th
Client forges or replays a proof sidecar, browser artifact stamp, or capability-shaped map	Check trusted issuer, purpose, owner an
A write may have committed but the response or readback is missing	Preserve indeterminate incorporation ar
A second writer or old Ring handler bypasses the inspected application path	Record the uncovered writer/deploymen

The adversarial browser may falsify its **own** state; the trusted boundary must independently protect authoritative effects and allowed disclosures. Conversely, an intentionally bypassing trusted host program is part of the claimed environment or trusted computing base unless it is prevented by an actually enforced isolation mechanism. Static assembly checks, runtime checks, physical observation, and formal certificates must each report only the guarantees their own evidence establishes.

Trusted host code and accidental-misuse prevention

The adversarial-browser boundary must not be confused with a claim that Gesso sandboxes arbitrary Clojure code executing inside a trusted application process.

Code that deliberately obtains a lower-level transaction capability, database handle, model implementation namespace, or another authority-bearing resource may be able to bypass the supported Gesso/model path. If such code can mutate the same authoritative state directly, it belongs to the trusted computing base for every guarantee that depends on those model invariants unless a stronger isolation boundary prevents the bypass.

Within that trusted host-language boundary, Gesso is primarily concerned with preventing **accidental semantic misuse** by cooperating application code. The normal path should make incorrect substitutions, missing derivations, stale artifacts, undeclared authority paths, and accidental bypasses difficult to express and easy to diagnose.

The supported paths have these obligations:

Path	Required treatment
Ordinary supported code, including opaque Clojure	Narrow semantic/model interfaces, checked construction, explicit premises, and adm
Accidental mismatch	Early, specific rejection at the boundary that can establish the mismatch
Deliberate lower-level write	Explicit writer coverage or an environment assumption; reassess every dependent in

Checked managed construction is the ordinary path, including opaque Clojure behind declared contracts. The assembly supplies scoped effect capabilities and binds their realization to admitted operations. View and handler construction does not ordinarily receive raw transaction authority simply to compute a value. The whole-application contract specifies the actual enforcement and remaining host-process assumptions.

A deliberate lower-level writer may affect every dependent invariant over the state it changes, including guarantees used by later operations. Its impact is not confined to the assurance of the single bypassing invocation. Initializers, migrations, jobs, and external writers therefore participate in the declared writer coverage or remain explicit environment assumptions.

A source-derived writer inventory, locally observed Biff handler installation, or one-shot XTDB submission report must disclose its actual host and backend scope. None can silently upgrade itself into global writer completeness, authentication of deployed traffic, universal transaction causality, or proof that all writes refine admitted model operations. Any claim dependent on those properties keeps the corresponding premises open until the proper physical boundary establishes them.

These mechanisms are valuable even though a sufficiently determined trusted application author can rewrite the program or deliberately acquire a lower-level handle. The target is not to make trusted code physically incapable of cheating. The target is to make the coherent path obvious, accidental misuse hard, and deliberate escape from the claimed boundary conspicuous.

Knowledge

Knowledge is tracked with respect to actors or projected roles. In this specification, a knowledge judgment records what an admitted execution is justified in using under its premises. It is not an assertion that every browser-held proposition is true, that an adversarial participant ran the machine, or that a human understood the representation.

Knowing an input value and knowing a domain proposition about that value are different judgments. A server may know that a browser submitted Request id R without knowing that R exists, that its status is open, or that the caller may act on it. Similarly, knowing that a client asserts F does not establish F.

A fact or value may become known through several distinct provenance paths.

At minimum the semantic model recognizes:

- `:input`;
- `:communicated`;
- `:authoritative`;
- `:derived`;
- `:asserted`.

These labels describe justification, not merely transport. Their appearance in an incoming map does not establish the corresponding provenance. A knowledge judgment includes the subject and scope of the fact and the premises on which the permitted use depends; a provenance label alone is not the whole judgment.

The important verifier question is not:

> Which packet delivered this map key?

It is:

> Before this actor performs behavior that depends on fact F, what justifies that the actor knows F?

A knowledge judgment has a subject, scope, and support

A semantic fact is more than a map key. Its judgment binds the actor or role, the proposition and values, the subjects, applicable temporal/query context, and the justification on which its permitted use depends. The surrounding admitted contract may supply these bindings without repeating them in every runtime value.

For example, an inspection view might show:

```
{:actor [:client C]
 :proposition [:request/status R :open]
 :context
 {:authority [:authority :request R]
  :projection :request/card-v1
  :arguments {:request/id R}
  :principal-context :session/helper-1
  :basis B
  :valid-at V}
 :support
 {:kind :authoritative
  :acquisition :acquisition/A17
  :premises #{:request/projection-contract
              :storage/qualified-read-semantics}}}
```

This describes the status at V under the authoritative read context B. It does not assert perpetual openness or grant permission to claim. The map is inspection data; admission must resolve its acquisition and premises rather than trust caller-supplied labels. The symbols C, R, B, and V denote bound fixture values, not inferred identities or live Gesso runtime bindings. Neither the illustrative session label nor the inspection map authenticates the principal, qualifies the read, or issues a reusable knowledge credential.

A local key such as `:request/status` may index the fact only when the owning context fixes which Request and observation it denotes. Reusing the key for another subject must not merge their evidence. Equal values may have several independent justifications, but a justification for R1 cannot strengthen R2's claim merely because both values are `:open`.

Support is dependency-sensitive. A derived judgment retains the contracts, assertions, and scoped facts used to justify it. Replacement or correction of a current observation changes which dependent current claims are applicable; it does not retroactively

erase the event that an older value was observed. A failed or withdrawn assumption makes dependent unconditional use unavailable, while a conditional theorem may remain valid under its stated premise.

The implementation need not retain an unbounded runtime proof history. It may keep sufficient bindings, compact evidence, or a reference to recheckable sources. Eviction and retirement follow explicit scope/lifetime rules; loss of evidence must not be interpreted as successful discharge.

Input knowledge

Input knowledge consists of the values and propositions supplied under the operation's entry contract. The contract determines whether the actor knows only the supplied value, knows that another participant asserted a proposition, or may rely on the proposition itself under a named trusted premise.

Conceptually:

Premise	Meaning
A receives an input under the admitted entry contract	The input's subject, type, and context are bound
The entry contract justifies the proposition being used	It distinguishes a supplied value, a reported assertion, and a trusted premise
Conclusion	A may use that scoped input knowledge with `:input` provenance and the contract

Input knowledge is not equivalent to arbitrary function arguments.

The operation contract must say which incoming facts are accepted as justified entry knowledge and under what assumptions.

For example, a trusted server may supply an authenticated principal as input knowledge to a server projection.

A browser cannot create the same trusted input merely by serializing an identical principal id.

Communicated knowledge

Communication may establish knowledge constructively.

Conceptually:

Premise	Meaning
A's send is justified under the message contract	The sent value and permitted semantic claims are established or explicitly assumed
B admits the matching receive	Sender, execution, payload binding, and required validation satisfy that contract
Conclusion	B gains the declared knowledge of X with `:communicated` provenance and the retained m

An admitted choreography must justify the sender's knowledge of the value it transmits. At an adversarial transport boundary, a message's receipt does not prove that the sender satisfied that semantic premise; trusted validation or the explicitly stated assumption must supply the justification being claimed.

The receiver gains only the values and semantic consequences declared by the message contract. An authenticated sender can still communicate a mistaken claim; sender authenticity and the truth of that claim are separate premises. A delegated authoritative statement requires the declared issuer and validation contract, not merely a familiar sender name.

Arbitrary extra map keys must not silently become choreography context.

Communication establishes the fact that the declared value was communicated in that execution; it does not automatically establish that the value is authoritative domain truth.

Authoritative knowledge

An actor may acquire knowledge by observing a representation justified by authority.

Conceptually:

Premise	Meaning
Authority justifies F	The proposition has a declared subject, time, and authority scope
B acquires an admitted projection establishing F	The read, disclosure, projection, and basis contracts hold
Conclusion	B may use F with `:authoritative` provenance at that observation context

The physical representation may be:

- structured data;
- server-rendered HTML;
- another checked projection;
- an application-specific authoritative representation.

The representation format does not weaken the provenance if the trusted path from authority to observation is valid.

Derived knowledge

A role may derive a proposition from facts it already knows through a declared semantic rule. For a deterministic functional derivation, the justification includes the exact input bindings and the function contract on which the result depends.

Required premise	Meaning
Inputs available	The role has the required scoped values or propositions
Rule applicable	Its types, subjects, refinements, and other preconditions hold
Result justified	The supported evaluator/derivation establishes the result, or the admitted opaque contract remains an explicit pre
Dependencies retained	The conclusion preserves the assumptions and evidence scope used by the rule

A dependency inventory is not itself a derivation. Recording that F depends on X and Y, or accepting a caller-supplied result with the right shape, does not establish that F follows from X and Y. A provenance constructor may record this description, but a stronger checked judgment needs its own mechanism.

A checked judgment must retain which obligation was actually discharged: input availability, applicability of the semantic rule, derivation of its conclusion, or physical correspondence to the executable realization. Establishing one does not discharge the others. In particular a static role-local must-analysis over protocol keys, although valuable for rejecting unavailable values, is not by itself a proof of proposition truth, subject binding, temporal applicability, or correctness of opaque application code.

Ordinary Clojure remains available. If its functional behavior is opaque, the semantic surface names the contract, implementation binding, and accepted assertions. Input/output checks can establish instance-specific properties without claiming universal correctness, purity, determinism, or termination.

An effectful or nondeterministic computation can establish knowledge of its particular observed result through its declared effect/result boundary. It does not become a deterministic derivation from inputs that omit the clock, random choice, external read, or mutable state it used. Later reasoning may use that observed result as an explicitly bound input.

The system must not infer equal results across actors from identical function names or source text. Cross-actor derived agreement needs compatible semantic inputs, the required deterministic contract, and correspondence of the realizations to that contract. JVM/browser arithmetic, locale, time, and other environmental semantics must be included where relevant.

Canonicalization and structural evidence checking must not acquire hidden runtime effects by evaluating arbitrary application callbacks. Where execution is part of an evidence mechanism, that execution has an explicit trusted boundary, resource contract, and result scope.

Deriving a value from a client assertion does not make it authoritative. An inference can strengthen a conclusion only when its rule and actual premises justify that strengthening. Provenance labels are descriptions of support, not a total ranking in which adding `:derived` upgrades every earlier premise.

Asserted knowledge

An explicitly accepted assertion supplies a premise that the verifier does not derive. It is an ordinary supported mechanism for full-language and external-system contracts, subject to the admission policy from Part I.

Its identity binds the exact proposition, subject, parameters, scope, owner, implementation or external boundary, and applicability conditions required by the claim. Two assertions with the same friendly name but different subjects or meanings must not merge. Accepting a premise is not discharging it.

An untrusted actor's assertion remains a claim by that actor unless a trusted entry contract explicitly permits relying on its proposition. A browser cannot register itself as the trusted owner of an authorization premise by sending an `:asserted` tag or a familiar assertion id.

The assertion remains visible in every dependent explanation. It must not be relabeled as a proved derivation, a successful authoritative observation, or a runtime-enforced current premise merely because its result matches another value. Contradiction, expiry, or loss of applicability propagates to dependent uses according to their scope. Independent guarantees remain available when they do not depend on the affected premise.

Knowledge use

Knowledge determines whether a role may, depending on the operation:

- transmit a value;
- consume a value;
- use a value as an FX input;
- correlate a message;
- choose a branch;
- rely on a discriminator when its projected behavior diverges;
- derive another value;
- construct provisional state;
- invoke an operation whose entry contract requires the fact;
- claim agreement with another role;
- establish a checked rule precondition.

A role must not consume facts it cannot justifiably know.

Knowledge is semantic, not incidental context

The presence of a key in a runtime map is not evidence of knowledge.

For example:

```
{:user/id U
 :request/status :open}
```

does not answer:

- who knows those values;
- why they know them;
- whether they are authoritative;
- at what basis they were observed;
- whether they may be transmitted;
- whether they may justify a branch;
- whether they are merely local provisional data.

The semantic machinery must not infer provenance from incidental map membership.

Counterexamples for knowledge admission

The following failures are distinct obligations for the verifier, runtime boundary, or application-assurance layer that owns the affected claim:

Apparently available information	Missing justification or forbidden inference
The browser supplies Request R and a status value tagged `:authoritative`	Input possession and a provenance label do
A role receives a message with an undeclared extra map key	Open transport payload does not confer sen
An authenticated sender communicates that R is open	Sender identity and receipt establish commu
A branch uses a value only present on some incoming protocol paths	The required fact is not definitely available
Two different Requests have the same status value	Evidence about R1 cannot justify a claim ab
An actor observes one page of a filtered board	That result does not establish complete boa
A derived fact depends on an opaque callback or accepted assertion	The dependency remains an explicit premis
Two actors run a same-named function against different locale, time, or other implicit inputs	The function's name does not establish equa
A principal loses permission after a valid historical observation	Historical acquisition is not erased, but late

These cases must not all be collapsed into `unknown-value`. Some concern missing protocol availability; others concern insufficient authority, subject/observation scope, opaque proof premises, or current permission. An assurance report identifies the relevant boundary and leaves unresolved proof or enforcement obligations visible rather than laundering them into `:authoritative` or `:derived` knowledge.

Knowledge and privacy

Knowledge is scoped.

Correctness does not require every actor to learn every fact.

Privacy, authorization, role relevance, and protocol structure may deliberately prevent knowledge propagation.

A proof that ”eventually everybody knows everything” would be wrong for many applications.

The useful property is:

> Every admitted actor behavior has the scoped knowledge and premises its contract requires, with the claimed degree of justification made explicit.

Permission to disclose, retain, use, or forward a fact is separate from the historical fact that it was acquired. Revocation cannot make an adversarial recipient forget prior disclosure, but it can remove permission for a later read, subscription, operation, or transmission. Knowledge alone is not a capability.

Knowledge derivations as inspectable evidence

Where knowledge is machine checked, the checker should retain enough derivation information to explain why a fact is known.

A boolean such as:

```
{:knows? true}
```

is insufficient as the long-term proof product.

The derivation should be able to expose leaves such as:

- input;
- communication;
- authoritative observation;
- deterministic derivation;
- explicit assertion.

These derivations belong to verifier/checker artifacts. Runtime state retains the identities and premises actually needed for execution, not necessarily the entire explanatory graph.

An observed adapter transition may supply a diagnostic account of a role-local step, requested transport, provisional display, or authoritative installation. It must be derived from the actual semantic transition rather than a fabricated successor supplied by a caller. It is still not, by itself, evidence of a physical network delivery, a database commit, or universal host correspondence. The observation category determines which stronger claim could be supported.

Authoritative observation as a knowledge path

Authoritative observation is a first-class semantic provenance path even when it is not represented as a first-class portable machine opcode.

This distinction is important for ordinary web applications.

A HumanHelp browser may learn facts such as:

```
Request R is done
Request R is claimed
Membership U is inactive
Organization O is closed
```

by rereading an authoritative representation after Live indicates that relevant authority may have changed.

That knowledge is not weaker merely because it arrived as rendered HTML rather than as a dedicated choreography participant message.

The authority remains in the model/database.

The HTML is one representation from which the admitted actor may acquire the fact under the declared projection and normalization contract. Arbitrary DOM text, an undeclared data attribute, or a locally fabricated status does not become semantic knowledge by looking like the server's rendering.

Advisory invalidation versus authoritative observation

The normal causal relationship, when the application policy permits the acquisition, is:

1. A model operation incorporates an authoritative mutation under its storage contract.
2. Live supplies an admitted, scoped advisory about potentially relevant change.
3. Application policy or user action supplies a permitted acquisition trigger.
4. An authorized read satisfies the applicable scope and progression requirements.
5. The actor acquires the justified observation. Adoption and installation follow their declared policy rather than being implied by notification receipt.

A Live invalidation means approximately:

> Relevant authority may have changed; a sufficiently current authoritative observation may be required for later behavior or offered for explicit adoption.

It does not mean:

> This particular domain fact is now true.

Therefore invalidation is advisory knowledge about possible change, not authoritative knowledge of the changed fact.

Receipt of the advisory does not by itself choose when the application adopts new content. Acquisition may follow an explicit user action, a declared automatic policy, or another admitted trigger. Any convergence claim that relies on such a trigger must name it among its liveness premises.

Observation scope and selective incorporation

An authoritative observation establishes only the facts and coverage justified by its acquisition contract. Its semantic context includes the authority, subject or projection scope, relevant basis/progression, and applicable temporal or query meaning. Filters, search terms, ordering and pagination where they affect coverage, projection version, and principal-specific visibility may all be part of that meaning. These need not all be repeated as fields when they are unambiguously supplied by the admitted contract.

A later observation of one Request does not establish acquisition of all board membership at the same database progression. An actor may hold individually justified observations from different acquisitions without possessing a single consistent snapshot of their union.

Consequently, installation of new authoritative content must preserve each observation's justified scope. Claims spanning multiple observations require the compatibility or consistent-acquisition premise appropriate to that claim. The mere existence of separate authoritative observations is insufficient.

The authoritative-operations and progression part owns the detailed scope and progression contract and advisory discharge rules. Browser installation follows the scoped installation contract. HumanHelp's selective-adoption policy is a use of those generic contracts.

An observation contract distinguishes returned rows from coverage of the query. A complete authorized board result may establish membership or absence within that projection. A page, search subset, or single-card response does not establish membership of the entire board. Ordering and cursor context matter when they determine which portion was returned.

Result	Permitted semantic conclusion
Complete authorized scoped result	Returned facts and the stated coverage/absence conclusions at its read context
Partial or paged result	Facts in the returned scope; completeness only for explicitly established coverage
Authoritative not-found	Absence within the declared subject, visibility, temporal, and query contract
Denied or privacy-masked response	Only the permitted denial/opaque result; no stronger existence conclusion
Timeout, disconnect, or unavailable read	No successful observation of the missing result

Two different values at the same raw basis are not automatically an authority contradiction. First establish that the authority, subject, projection version, query parameters, temporal context, visibility context, and deterministic interpretation are the same. Different valid times or projections can produce different correct answers at one database basis. If the complete meaning is the same and incompatible facts remain, treat that as contradictory evidence; do not resolve it by last-arrival-wins.

An incomparable observation can remain valid evidence in its own scope without being eligible to replace the currently adopted projection. Replacement needs the declared supersession, progression, or reconciliation relation. Equal displayed values do not by themselves advance a knowledge or coverage frontier.

Render reuse, cache keys, and observation scope

Caching and concurrent singleflight sharing are physical render optimizations; neither creates a new authoritative observation, proof of a current permission, or progress beyond the source acquisition's justified basis and coverage. A reused result retains the original observation's qualified authority, subject, projection, temporal/query context, visibility, and evidence scope. Its reuse must be admissible under the **requesting** actor's current applicable read/disclosure contract, or under an expressly bounded grant that still applies. The fact that two callers request the same URL or logical fragment does not prove that either may receive the other's HTML.

A cache or in-flight deduplication key must distinguish every render or admission dependency relevant to safe sharing, or establish a checked equivalence covering those dependencies. Depending on the projection, these can include principal-specific visibility, tenant/subject, query/filter, ordering/pagination/cursor, projection version, temporal context, locale, and the required authoritative progression. Identical opaque basis bytes from incompatible authority/provider scopes do not create an equivalence. A shorter cache TTL or a familiar fragment ID cannot repair an underspecified sharing key.

A cache hit is allowed to represent deliberately retained older content under a declared selective-adoption policy. It must not be described as a new, more-current database read or used to discharge a stronger pending refresh requirement without evidence that its scope and actual acquisition coverage satisfy that requirement. Coalescing may combine compatible work but may not discard independent opaque requirements by assuming last arrival is newest.

The ordinary Gesso fragment-render helper can build scoped render keys, but render-key completeness, actual query/disclosure equivalence, and enforcement at the protected response boundary remain separate application and physical correspondence obligations. A helper's cache-key shape alone is not a certificate of those facts.

Acquisition, installation, and semantic adoption

Acquisition produces an observation under its declared source and read contract. Installation applies a representation to a concrete target. Adoption selects the observation or projection on which subsequent behavior relies. The contract states when these transitions coincide and when they are separate.

An authorized response may arrive without being installed because its owner was retired, the target changed, or a newer compatible result was already adopted. Its evidence may be usable through another declared admission path, but it must not mutate a replacement owner merely because it is authoritative.

Scheduling an installation or emitting an allow-swap decision is not proof that the representation was installed. A synchronous realization may establish both within one supported boundary; an asynchronous realization needs a completion or failure contract. The adopted frontier and advisory discharge must reflect the transition they claim, including the actual permitted installation when adoption requires it. Part VI owns the concrete shell/refinement relation.

An advisory received while an acquisition or installation is pending must be evaluated against the resulting observation's actual coverage. It cannot be discarded merely because it arrived before a callback or because the acquisition was called Refresh.

Observed basis is protocol context, not proof of prior observation

A naked browser-supplied `:observed-basis` value is a client claim about the context from which the browser believes it is acting. It is not, by itself, proof that the browser actually received or incorporated an authoritative observation at that basis.

For ordinary Gesso commands this is intentional. The basis is useful protocol context for consistency, reconciliation, provenance, stale-command handling, and authoritative reacquisition even when the browser is adversarial.

For example, a browser may say:

```
{:request-id R
 :observed-basis B}
```

Here `R` is the domain Request identity, not a transport-request identity. The server may safely use `B` to understand the command's claimed starting context without treating `B` as permission:

Validated comparison or context	Permitted treatment
B is compatible or sufficiently current for the declared use	Continue under the ordinary current model guards
B is stale or incompatible for that use	Follow the declared reconciliation, rejection, or reacquisition path
B is invalid, unsupported, or lacks required comparison evidence	Report or handle that condition explicitly; do not invent an ordering or

Current authenticated principal binding, model authorization, authoritative state, and transaction guards remain decisive. A client may lie about `B` without thereby gaining authority.

The provider still validates the token's representation and applicable authority/database context. A client cannot select a different backend, expand query scope, or demand an unbounded wait by supplying a fabricated future basis. Supported bounds and failure responses are part of the acquisition contract; failure to satisfy a required basis is reported rather than silently dropped.

The trust requirement changes if an application gives **prior observation itself** security significance. Examples include:

- an operation is allowed only after the principal was previously issued a particular authoritative representation;
- a continuation is valid only when it follows a specific server-issued observation;
- an acknowledgement must refer to an exact previously issued semantic version;
- an application distinguishes ”client claims to have seen X” from ”the server can verify that it issued X to this principal/session.”

In that case a forgeable client field is insufficient. The application needs an explicit server-verifiable mechanism such as:

- a server-issued capability;
- a MAC-authenticated observation receipt;
- trusted server-side session state;
- a signed server attestation;
- a nonce or continuation identifier bound to a prior response;
- another mechanism whose trust semantics are stated explicitly.

The framework must never silently promote:

```
ClientClaim<ObservedBasis>
```

into:

```
VerifiedObservationEvidence
```

merely because the value has the expected shape or happens to equal a real basis.

A Message Authentication Code (MAC), for example HMAC-SHA-256, is one possible way to let an adversarial browser carry integrity-protected server evidence, under the named cryptographic, key-management, encoding, and verification assumptions. This example specifies the trust relationship rather than a complete cryptographic implementation. The server authenticates a canonical payload using a secret key known only to trusted infrastructure:

```
receipt-payload
    =
{:receipt/type :gesso/observation-issuance
 :receipt/version 1
 :issuer :humanhelp/observation-service
 :audience :humanhelp/request-flow
 :key-id K
 :principal [:user P]
 :authority [:authority :request R]
 :projection :request/public-row-v1
 :scope S
 :basis B
 :temporal-context T
 :semantic-digest D
 :purpose :prior-observation-issuance
 :issued-event :authorized-response-constructed
 :issued-at I
 :expires-at E}

mac
    =
HMAC(server-secret, canonical-encoding(receipt-payload))
```

The browser may read, store, copy, and return the payload and MAC. Under the declared integrity assumptions it cannot feasibly change `P`, `R`, `B`, `purpose`, or another authenticated field and produce accepted replacement evidence. The MAC does not encrypt the payload; receipt contents must therefore be suitable for disclosure to the recipient. Scope S and temporal context T include the query, visibility, and provider/basis context required by the projection contract; they are not inferred from the token's familiar shape. The digest D identifies the canonical semantic observation material, not arbitrary HTML bytes or the truth of the statement by itself.

On receipt, the server recomputes and verifies the MAC and then checks the context in which the receipt is being used:

1. Parse the untrusted envelope under bounded, supported encoding rules.
2. Resolve the admitted receipt contract and its trusted verification key.
3. Verify the MAC over the exact canonical payload.
4. Check the required identity, scope, context, purpose, and lifecycle bindings.
5. Admit only the historical issuance proposition justified by those checks.

The verifier chooses the admitted receipt contract, trusted issuer/key mapping, canonical encoding, and verification mechanism. Client-supplied fields do not select an arbitrary algorithm or trusted issuer. Version, type, audience, principal or session binding, authority, subject/scope, projection, purpose, expiry, and semantic-material binding are checked as required by that contract. Key rotation, clock interpretation, and revocation/expiry behavior belong to the declared receipt lifecycle. Validation failure grants no stronger premise.

The MAC authenticates the statement represented by the payload; it does not make an unjustified statement true. If the server authenticates a bad or overstated proposition, the MAC faithfully preserves that mistake.

In particular, a receipt normally proves something like:

> Trusted server infrastructure issued an attestation for principal/session P about the declared semantic observation of authority A at basis B, under the receipt's stated issuance contract.

The issuance contract must identify the event actually attested: for example, construction of an authorized response or release of a response to a transport. A receipt accompanying a cached projection must attest the actual admitted issuance and semantic material; it cannot silently describe a cache hit as a fresh authority read, or inherit a different principal's issuance context. Issuance is not proof of successful delivery, browser installation, retention, or human acknowledgement. Returning the receipt proves possession of the authenticated bytes, not those stronger events by itself.

It does not by itself prove:

> The browser installed the observation, or the human read, understood, or consciously incorporated it.

If human acknowledgement is semantically relevant, that is a stronger protocol and needs its own explicit proposition and evidence.

Observation evidence must also remain distinct from an operation capability. A valid authenticated receipt concerning an old authoritative observation may remain historically valid even after current authority changes:

```
valid observation receipt
≠
current authorization
```

For example, another helper may have claimed Request R after the receipt was issued. The receipt can still prove that an earlier observation was issued, while current authoritative reread and model guards correctly reject a new claim.

Gesso should therefore distinguish at least these concepts:

Concept	Semantic meaning
ObservedBasisClaim	Client-supplied, forgeable consistency or reconciliation context
AuthenticatedObservationReceipt	Server-authenticated evidence of the stated prior issuance event
CurrentAuthoritativeObservation	Trusted acquisition satisfying its scope and read contract; current relative to that acquisition
OperationCapability	Scoped permission under an explicit grant and its validity conditions
CurrentAuthorization	A current trusted decision permitting the concrete principal's operation under the authoritative policy

None should implicitly coerce into another. In particular, historical observation evidence and possession of an operation capability do not silently become current authorization unless the declared capability semantics explicitly make possession sufficient and all of that capability's current validity conditions are satisfied.

Gesso may emit compact authenticated observation receipts alongside authoritative acquisitions when a supported protocol benefits from evidence of prior issuance. Routine issuance is an implementation option, not a baseline requirement justified solely by the cost of computing a MAC. Its key lifecycle, payload disclosure, expiry, and replay contract must be explicit. The authenticated object should be the **semantic observation** or a compact digest/identity of it, not arbitrary HTTP response bytes.

The receipt contract must identify exactly what its semantic digest binds. A digest of an untyped map without authority, projection, temporal, or query meaning is not enough when those distinctions matter. Verifying a receipt does not re-read the model or establish that the old facts remain current.

Even if such receipts become common, ordinary `:observed-basis` should remain a distinct weaker protocol concept. Requiring cryptographic evidence whenever a client reports its starting basis would unnecessarily turn useful consistency context into a security credential.

Replay semantics also remain explicit. A MAC makes a token difficult to forge or alter; it does not make the token single-use. A reusable observation receipt may legitimately be replayable because it attests to a historical fact. A one-shot capability must enforce its declared consumption or replay policy. Expiry limits validity duration but does not by itself prevent reuse within that duration. A nonce distinguishes issuances but does not by itself record consumption. A consumed-token record or another trusted mechanism may enforce single use; command identity and idempotency may instead enforce a permitted duplicate-use outcome. Those are different contracts and must not be presented as interchangeable guarantees.

The governing rule is:

> When prior observation becomes a premise of authorization or another security-sensitive conclusion, the mechanism establishing that premise must cross an explicit server-verifiable trust boundary. Client assertion remains client assertion until that boundary is crossed.

Observation need not imply a long-lived waiting opcode

Many authoritative observations form entry knowledge for a later short choreography.

For example:

Stage	Semantic consequence
Browser renders an admitted Request observation at X17	That execution may finish with its declared adopted knowledge
A later Live advisory arrives	It records advisory information; it need not resume the finished execution
Policy or user action permits a new acquisition	An authorized read obtains an observation satisfying the new requirement, her
A new short choreography begins	Its entry contract binds the observation at X24 as justified input knowledge

The portable choreography language therefore need not acquire a universal `:observe` or durable wait operation merely to model ordinary web rereads.

If an application genuinely needs one semantic execution to suspend on an authoritative condition and later resume, that facility should be designed deliberately with explicit persistence, compatibility, ownership, and recovery semantics.

It must not appear accidentally as a consequence of normal Live behavior.

Observation and representation independence

Authoritative observation is semantic.

Its correctness must not depend on one specific presentation format.

A browser may acquire justified knowledge from an authoritative HTML projection when the server-rendered representation and its acquisition path establish the required facts.

Another application may use structured data.

The important proof obligations concern:

- authority;
- authorization;
- basis/progression;
- projection semantics;
- correspondence between representation and declared fact.

They do not concern whether the bytes happened to be HTML or EDN unless the representation mechanism itself introduces a relevant assumption.

Observation and reuse counterexamples

These are acceptance and correspondence cases, not claims that every current Gesso rendering path already passes them:

Case	Permitt
A cached board for principal P is requested by Q with different visibility rights	Rechec
One singleflight render lacks an authoritative minimum required by another caller	Do not
A filtered page returns no Request R	Do not
A previous observation receipt validates after an ownership or permission change	Retain
An SSE advisory arrives while a fragment acquisition or installation is pending	Keep u
A response is admitted as authoritative but its swap is rejected after owner retirement	The sta
A cached response or receipt is served using the right basis but the wrong principal, query, projection or temporal interpretation	Reject t

The responsible checker or runtime must identify which premise failed: current disclosure, acquisition scope, query coverage, authority/progression, receipt issuance, resource ownership, or installation. Observation failure does not by itself establish that a separately submitted authoritative command was rejected.

Agreement

Agreement is stronger than two actors having values with the same shape.

Agreement means the protocol can justify that the actors share the relevant semantic value or fact under the declared relation, scope, and frontier.

This is a judgment about those actors' relevant observations or values at the specified points. It does not automatically mean that either actor knows the other actor received the value, that both know that both know it, or that they continue to share the same current value after later changes. Protocols needing acknowledgement or stronger knowledge about receipt must declare that additional obligation.

A checked agreement judgment must identify the participating actors or roles, the subject/proposition or semantic value, the agreed relation and relevant frontier or observation contexts, and the support for each side. These bindings may be supplied by an admitted contract rather than repeated in every runtime record. The checker must also retain any trusted issuer, derivation, transport, or physical-correspondence premises on which that agreement depends. A declaration or pair of equal values is not itself the evidence that those premises were discharged.

If A communicates X to B:

The message contract justifies A's send, B admits the matching receive, and the sent and received values are related by the contract's declared encoding and semantic interpretation. Under those premises, the protocol may establish `:communicated` agreement on X at those two events. It does not establish acknowledgement, current-state equality after later changes, or domain truth beyond what that message contract justifies. An outbound send request or single receiver-side observation cannot by itself establish the complete sender-emission, transport-delivery, and matched-receive correspondence. A concrete replay witness can support its stated schedule without establishing the relation for every possible distributed execution.

Agreement can arise through multiple provenance paths.

At minimum:

- `:communicated`;
- `:authoritative`;
- `:derived`;
- `:asserted`.

Communicated agreement

Communication can establish agreement on the communicated value.

It does not imply agreement on every fact either actor knows.

Message contracts therefore matter.

Closed payloads make the agreement boundary explicit.

Authoritative agreement

Two actors may independently observe the same authoritative fact and obtain justified agreement without receiving one identical network packet.

For example:

Actor	Physical read path	Required semantic support
A	Web node 1	An admitted observation supporting scoped proposition F
B	Web node 3	An admitted observation supporting the same proposition F under the required compatibility relation

The agreement judgment uses that compatibility evidence; matching host or packet identities are unnecessary.

The hosts differ.

The packets differ.

The required agreement is on the scoped proposition being used, not necessarily on an identical packet, exact read timestamp, or complete database snapshot. The relation must establish that the observations support that same proposition under compatible subject, temporal, projection, and authority contexts. Query filters, visibility and coverage also matter when they determine the

proposition being compared: two individually authorized pages need not support agreement about complete board membership. Equal progression tokens alone do not establish this; neither do equal status values at unrelated times.

For example, two observations that a Request is open can support agreement on a specified state/version or valid-time proposition when the authority contract justifies the relation. They do not prove that the Request never became claimed and reopened between those observations. Such a history claim needs its own evidence. A generic basis advance is not evidence that every fact was unchanged.

Derived agreement

Agreement may propagate through a declared deterministic derivation when the inputs and derivation justify doing so.

The system must not infer derived agreement merely because two actors happen to run source code with the same function name.

The derivation relation must be part of the admitted semantic contract. Where its realization is opaque, the conclusion retains the exact deterministic and implementation assumptions used. A nondeterministic computation may support agreement on an explicitly shared result, but independent invocations need not produce that same result.

Asserted agreement

Application code may explicitly assert agreement that Gesso cannot derive.

Such assertions are named, scoped premises under the admission policy. They remain visible in diagnostics and proof reports, including the particular actors, proposition, relation, and frontier to which they apply.

They cannot silently suppress a mandatory failed correspondence check or be introduced by an untrusted participant choosing an assertion id. An asserted agreement does not become mechanically derived agreement because the two displayed values happen to match.

Shape equality is not agreement

Two values can validate against the same Malli schema without being agreed semantic facts.

For example:

```
{:request/status :open}
```

in two browser contexts does not establish:

- same Request identity;
- same authoritative basis;
- same provenance;
- same valid time;
- same semantic observation;
- same authority.

Malli answers shape questions.

Agreement answers semantic relationship questions.

Agreement evidence and counterexamples

Agreement is justified only for the explicitly compared values or propositions and their stated relation. Its evidence need not be one monolithic runtime certificate, but an assurance report must not treat the following weaker observations as proof of the stronger conclusion:

Apparent match	Missing premise or prohibited conclusion
A issued a send and B reports an equal received value	Establish the matched emission/delivery/receive relati
A and B each display `:request/status :open`	Bind the same Request, authority, projection, valid-tim
Two nodes return the same basis token with different query filters or permissions	Token equality does not establish the same query cove
Two actors observed R open at distinct compatible points	Agreement on the stated observations does not prove
Two actors execute the same named derivation	Require compatible semantic inputs, a valid derivation
A caller supplies an agreement assertion or proof-looking sidecar	Retain the exact trusted assertion premise; an untruste
Both actors have once learned the same fact, but one actor's permission changes	Historical agreement does not imply current authoriza

An actual disagreement under an established common context is not repaired by calling one observation newer without qualified progression evidence. Where context compatibility or source authenticity is unavailable, the relevant agreement claim remains not established; this does not turn either individual historical observation into a fabricated error. Agreement may also be valid under explicit opaque premises, but those premises must remain visible and must not be reported as unconditional theorems.

Agreement is scoped

Actors need not agree on all application state.

Agreement may be scoped to:

- a Request;
- a fact;
- a command;
- an operation input;
- a branch discriminator;
- an authoritative frontier;
- another explicitly modeled semantic object.

Privacy and irrelevance are legitimate reasons not to establish agreement.

Agreement frontiers

For evolving values, actors may agree only through a common frontier for the relevant facts and relation. A frontier is not necessarily one globally ordered position for all actor knowledge.

For a single modeled chain whose prefix observations are retained and justify the claimed agreement, for example:

Participant	Justified retained prefix of the modeled chain
Actor A	F1, F2, F3, F4
Actor B	F1, F2, F3
Common prefix for this relation	F1, F2, F3

The example requires the stated prefix knowledge. Observing only the latest snapshot at F4 does not automatically establish that the actor knows every intermediate transition through F4. Snapshot agreement and agreement on an event-history prefix are different relations.

The application universe may change continually.

Complete instantaneous agreement among all relevant actors may never occur.

That is not itself a correctness failure.

Correctness instead requires:

- recorded observations retain their historical meaning, while domain corrections and the applicability of their interpretations remain explicit;
- relevant divergence is explicit rather than silently erased;
- actors possess enough justified knowledge for the behavior they perform;
- where convergence is required, the protocol provides a valid path toward a newer common authoritative frontier;
- under quiescence and the choreography's stated liveness assumptions, required agreement debt can fall to zero.

Frontier is relation, not necessarily arithmetic

A frontier need not be represented by an integer.

The browser must not invent arithmetic ordering over opaque progression values unless the authoritative system explicitly provides a supported ordering operation.

The relevant semantic questions are relational:

- is this observation at least as strong as the requirement that triggered it?
- are two authoritative observations compatible for the fact whose agreement is being claimed?
- has this actor incorporated the authoritative change it was required to observe?
- may this local projection continue under the current compatibility contract?

The concrete XTDB and scope-coverage mechanisms are specified in the authoritative-operations and progression part. An ordering relation must not be assumed to provide a join, a greatest common frontier, or complete observation coverage unless the supported contract actually establishes those properties.

Different actors may have different valid frontiers

For example:

Participant	Justified scope in this example
Authority	Modeled chain through F5
Actor A	Required prefix observations through F5
Actor B	Required prefix observations through F3
Actor C	Only its privacy-permitted subset through F4

There may be no meaningful statement that all three actors share one universal application frontier.

Agreement and knowledge are scoped to the relevant facts and authorities.

Advisory knowledge and adopted knowledge

An actor may learn that authority advanced without immediately adopting the new authoritative projection.

This distinction is important for application behaviors such as a multi-user board:

Transition	Advisory state	Adopted pro
Relevant advisory received	Records the justified scoped reacquisition requirement	Existing proj
Explicit Refresh succeeds under the installation policy	Discharges only requirements covered by the adopted observation	Selects the n

The advisory requirements and adopted authoritative observations are therefore distinct pieces of actor state. The word frontier is shorthand for the scoped knowledge relation; it does not require either state to be represented by one maximum scalar token.

An HTTP or HTMX request completing, a fragment response being accepted, a swap being permitted, and an observation actually being installed and adopted are distinct transitions. A requirement whose contract demands only a qualified authoritative read may be discharged by evidence of that read without installing its representation. A requirement whose contract demands adoption cannot be discharged solely by request completion. If a swap is cancelled, the target retires, or the result fails its scope or progression contract, any still-required adoption debt remains until a permitted transition actually satisfies it. A physical render manager may retire a completed request's in-flight bookkeeping without that retirement proving the actor's adopted frontier advanced.

A correct application may intentionally preserve that distinction.

HumanHelp's board can represent that distinction explicitly instead of overloading one current-revision slot. Suppose Alice creates Request R3 while Bob's adopted board contains R1 and R2. The examples below are abbreviated inspection and pure-transition fixtures; they are not browser assertions that the server must trust.

The selected board projection is complete for its declared search/filter query, not necessarily for every Request in the Organization. Each acquisition records its concrete temporal context under the current-state read contract. X17, X18, X19, and X20 are opaque basis labels; the admitted provider establishes the relations used in this example.

```
(def board-scope
  {:organization-id 0
   :projection :request/board-v1
   :query {:search "" :show-terminal? false :order :newest}
   :coverage-contract :complete-selected-query
   :time-policy :current-state})

(def bob-board
  {:actor [:client :bob-tab]
   :principal-context :session/bob-1
   :owner [:fragment :bob-request-board]
   :projection board-scope
   :adopted
   {:requests [{:request/id R1 :request/status :open :request/title "R1"}
               {:request/id R2 :request/status :open :request/title "R2"}]
    :observation
    {:authority [:authority :request-board 0]
     :scope board-scope
     :principal-context :session/bob-1
     :basis :X17
     :temporal-context T17}}
   :advisory {:requirements #{}}
   :acquisition nil})
```

For this example, the producing authority can attach a justified lower bound X18 to the advisory generated by its committed change. The advisory does not say that R3 is visible to Bob, and its issuance/disclosure follows the admitted subscription contract.

```
(def create-advisory
  {:event/class :board-advisory
   :owner
   {:subscription/id :subscription/bob-board
    :source-instance/id :source/bob-board-1
    :scope board-scope}
   :requirement
   {:scope board-scope
    :authority [:authority :request-board 0]
    :minimum-basis :X18}})

(def current-board-subscription
  {:board-subscription
   {:subscription/id :subscription/bob-board
    :source-instance/id :source/bob-board-1
    :scope board-scope
    :principal-context :session/bob-1
    :status :open}})

(defn record-advisory
  [board admitted-advisory]
  (let [requirement
        (advisory-admission/require-requirement-for!
          admitted-advisory
          {:scope (:projection board)
           :principal-context (:principal-context board)})]
    (update-in board [:advisory :requirements]
      requirements/conjoin requirement)))

(def admitted-create-advisory
  (admit-board-advisory
    admitted-board-source-context
    current-board-subscription
    create-advisory))

(def bob-after-advisory
  (record-advisory bob-board admitted-create-advisory))

(get-in bob-after-advisory [:adopted :requests])
;; ⇒ still R1 and R2

(get-in bob-after-advisory [:advisory :requirements])
;; ⇒ the scoped requirement for at least X18
```

`requirements/conjoin` retains the conjunction of obligations, conservatively as an orderless set unless admitted implication evidence permits simplification. It does not choose whichever token arrived last. The admission functions above resolve registered source/owner contracts; calling a map admitted does not supply that evidence. The local source context belongs to the cooperating browser realization and is not a server credential.

The symbols `0`, `R1`, `R2`, `R3`, `A2` and the example observation/advisory values are prebound inspection fixtures, not executable definitions. Functions under `advisory-admission`, `requirements` and `board-runtime` are schematic contracts for this worked example, not claims that those named public Gesso Vars currently exist. Their actual implementations must be checked against the admitted application, producer registration, progression comparison and browser effect/owner semantics before any resulting guarantee is reported.

A requirement saying only ”later than the browser's X17” would not necessarily cover the change at X18. The browser must not manufacture X18 from an advisory with no such bound. A tokenless notification needs its separately declared fresh-acquisition or source-fence protocol. Part IV defines how that protocol establishes sufficient coverage.

Bob has learned that reacquisition may be needed or available for the scoped board. He has not thereby learned R3's title, status, requester, or visibility. The application may show a toast and Refresh affordance without replacing the adopted rows.

The Refresh interaction separates a pure browser transition from the server acquisition and the subsequent installation:

```
;; A2 is a fresh resource instance issued by the managed runtime.
(def refresh-start
  (board-runtime/step
    bob-after-advisory
    {:event/class :board/refresh-requested
     :trigger :user
     :resource-instance/id A2}))

;; Illustrative effect emitted by that pure step:
[:acquisition/request
 {:owner [:fragment :bob-request-board]
  :resource-instance/id A2
  :principal-context :session/bob-1
  :scope board-scope
  :requirements
  (get-in bob-after-advisory [:advisory :requirements])}]

;; The host performs the authorized read outside the pure step. The trusted
;; server derives the real principal from authentication, not from this effect.
;; The returned observation is admitted under the request/response contract.
(def refresh-completion
  (board-runtime/step
    (:state refresh-start)
    {:event/class :acquisition-complete
     :owner [:fragment :bob-request-board]
     :resource-instance/id A2
     :acquisition authorized-board-observation-X19}))

;; For an asynchronous realization this requests installation; it does not
;; yet assert that the new projection was physically installed and adopted.
(:effects refresh-completion)
;; ⇒ a scoped installation effect for the admitted observation

;; Another relevant advisory arrives before installation completes.
(def bob-before-install
  (record-advisory
    (:state refresh-completion)
    admitted-board-advisory-X20))

(def installed
  (board-runtime/step
    bob-before-install
    {:event/class :presentation/installed
     :owner [:fragment :bob-request-board]
     :resource-instance/id A2
     :installation-result admitted-installation-result-X19}))

(def bob-after-refresh (:state installed))
;; ⇒ adopts the permitted projection at X19
;; ⇒ discharges the X18 requirement when its full scope is covered
;; ⇒ retains the X20 requirement and every unrelated uncovered requirement
```

This example's adoption policy permits installing the result that covers the captured X18 requirement while retaining a newer X20 requirement. A policy that requires all pending requirements to be satisfied before adoption must instead defer that installation and reacquire; the runtime must not silently choose between these policies.

The illustrated `board-runtime/step` has the intended pure transition shape: next state plus requested effects. The present Gesso browser adapter exposes a related general `AdapterState × NormalizedEvent → State × Effects` boundary, but that does not prove these particular HumanHelp fixtures or schematic function names are compiled or running in the current application. A

corresponding checked realization must bind callbacks to registered instances, enforce installation rights against the current owner, and report actual completion or failure. The final ownership check and target mutation must share a declared serialization boundary; a check followed by an unprotected asynchronous mutation leaves the same race. An abstract `:presentation/installed` event is not evidence that a hostile browser actually performed the physical DOM effect. An installation acknowledgement supplied as arbitrary client data is not accepted as evidence of this local execution. An adversarial browser can still falsify its own display; the server's security does not depend on this local report.

The transition uses the latest state, including intervening advisories and owner changes. Reusing the state captured when Refresh began could erase X20. If another acquisition already adopted a stronger result, the query changed, the account context changed, or A2 was retired, this completion cannot install through the old ownership. A separate admitted path may retain independently useful outcome or observation evidence.

A synchronous shell may combine installation and its successful semantic completion when its correspondence contract justifies doing so. In either realization, effects are not performed inside a retriable pure state update, such as a `swap!` function. Only effects from the accepted state transition are performed, under their declared identity and retry contract.

If Bob is authorized to see R3 and it belongs to the selected query, the new board may contain it. If privacy or filtering excludes it, a complete authorized acquisition can still discharge the covered board requirement without revealing the hidden Request. It establishes the authorized projection, not all Organization state.

Now consider agreement between Alice and Bob. They have separate admitted observations. An agreement check states the proposition and relation it needs:

```
(agreement/check
  {:actors [[:client :alice-tab] [:client :bob-tab]]
   :proposition [:request/status R3 :open]
   :relation :compatible-authoritative-fact
   :observations [alice-observation bob-observation]})
```

The result can establish agreement only if those observations justify the same scoped proposition under that relation. Equal `:open` values and larger-looking basis labels are insufficient. The observations may have different physical delivery paths and compatible rather than identical read contexts; the required compatibility is evidence, not an inference from their labels.

Even established agreement that R3 is open does not establish agreement that it belongs in Bob's authorized board. Alice may never have learned Bob's visibility policy, query, or membership. That is a separate proposition with separate premises. Nor does a shared observation of openness authorize either principal to claim the Request now.

Privacy can make the shared proposition strictly smaller than either actor's total knowledge:

```
(def alice-projection
  {:request/id R3
   :request/status :open
   :request/title "Need help moving a cabinet"
   :request/requester-phone "+1-555-0100"})

(def bob-projection
  {:request/id R3
   :request/status :open
   :request/title "Need help moving a cabinet"})
```

Given the appropriate admitted observations, they may agree on the shared Request fact while Bob has no permitted knowledge of the private phone number. The illustrative domain maps alone do not establish either provenance or agreement. No convergence rule requires disclosure of the omitted field.

Bob can therefore retain board-adoption debt while holding newer justified knowledge about an unrelated Request or account setting. The meaningful question is agreement about which proposition, under which authority, scope, relation, and observations.

Convergence does not require replay of every intermediate change

A browser offline from X17 through X900 need not replay every invalidation between those points.

If the application semantics permit it, the actor may converge by acquiring a current authoritative observation at X900 or later.

The correctness obligation is not ”deliver every historical notification.”

It is:

> Provide a valid path by which the actor can acquire sufficiently current authoritative knowledge for the behavior it needs to perform.

That path must address the interval between establishing an interest and the read/notification stream that covers it, as well as reconnect and source replacement. Missing an invalidation before registration cannot be repaired by assuming that all future invalidations will arrive. A supported contract may use an acquisition/subscription fence, durable cursor or replay, gap detection, or a declared periodic reacquisition trigger. The chosen mechanism and its coverage must be explicit; this part does not require a particular broker or multi-node routing topology.

The critical race is between the authorized baseline observation and the subscription's effective coverage. A read before subscription admission may miss a change that occurred before interests were registered; subscribing before the read may still leave ambiguity unless the producer and reader establish which changes the read covers. The supported protocol must establish a join/fence, durable continuation/replay, gap-detection-and-reacquisition, or other equivalent evidence that no **relevant, required** change disappears between those boundaries. A source/router replacement must similarly preserve or re-establish that coverage. Matching subscription IDs, event delivery, local interest maps, or a tokenless advisory do not themselves discharge the gap obligation. A valid recovery need not replay every historical event when an authorized read can re-establish the required current projection.

Eventual convergence also needs an eventual trigger. Dependency availability alone does not make a browser refresh if it never learns that its observation may be incomplete. Where the product deliberately permits indefinite manual deferral, the convergence claim remains conditional on the required user action or another declared trigger.

Agreement debt

Where the choreography requires actors to converge, the difference between their relevant justified knowledge may be understood as agreement debt.

The design does not require one universal numeric debt metric.

The useful property is qualitative:

- divergence is represented;
- required convergence paths exist;
- stale or provisional actors do not pretend agreement already exists;
- under stated liveness assumptions and quiescence, the required divergence can be eliminated.

The existence of a recovery path is distinct from its eventual execution. Where adoption requires a user to activate Refresh, required eventual adoption is conditional on that action or another permitted acquisition trigger. Where the product permits indefinite deferral, remaining advisory information is not by itself a violation of an unconditional convergence guarantee; such a guarantee was never established for that deferred adoption.

A liveness or debt-elimination proof must state its actual fairness and environment assumptions: eventual triggering when required, availability of an authorized acquisition, recovery from disconnection or lost events, sufficient read/progression coverage, and a period of relevant quiescence when convergence needs one. A mechanism that could refresh is a safety or reachability fact, not a proof that the actor eventually refreshes. Likewise the mere presence of an opaque later-looking basis cannot certify that all independent pending requirements were satisfied.

Privacy constrains convergence

Agreement is always scoped by authorization and relevance.

The system must not satisfy a convergence property by leaking protected facts to actors that are not entitled to observe them.

A correct convergence path may therefore involve:

- one actor learning the full authoritative fact;
- another actor learning only an authorized projection;
- both actors agreeing on a smaller shared semantic proposition.

For example:

```
authority knows:
    full Request details + private requester metadata

helper A may know:
    Request is available for claim

observer B may know:
    Request count increased

A and B need not agree on private requester metadata.
```

Frontier and convergence counterexamples

The following cases are normative acceptance and correspondence obligations, not claims that today's browser or multi-node runtime already satisfies them:

Case	Required treatment
Actor A sees snapshot F4 but has no event-history coverage	Do not infer knowledge of the full F1–F4 transitio
Two opaque basis tokens look ordered numerically or by arrival time	Require authority-qualified progression evidence;
A change commits between initial board read and effective subscription registration	Use the admitted coverage fence, replay, or gap-re
A tokenless advisory arrives without a trusted source fence or sufficient reacquisition	Do not manufacture a lower bound for an unseen t
A newer X20 requirement arrives while an X19 result is awaiting installation	Adopt X19 only if the declared policy permits it; re
A2's completion arrives after owner, query, or principal context replacement	Retire the stale installation right; do not mutate th
Bob receives a privacy-filtered board that does not show R3	Conclude only the authorized query's established
The browser is disconnected and the user indefinitely defers manual Refresh	Keep convergence conditional on an eventual trigg

A diagnostic or formal obligation should distinguish a missing observation premise, unqualified ordering, advisory coverage debt, stale owner, privacy restriction, and unsatisfied liveness assumption. None becomes a success by comparing raw timestamps or making a stale view look current.

Foundational invariant

The foundational agreement invariant is:

> An admitted Gesso execution never treats mere co-presence, shape equality, physical delivery, or coincidental local computation as semantic agreement. Agreement must be justified by a declared communication, compatible authoritative observation, checked derivation, or explicit trusted assertion, and any claim over evolving facts is relative to an explicitly justified temporal and observation relation. Assertions remain premises, and a claimed common frontier does not imply a universal order, complete history, or current authorization.

Part III — Choreography and projection

This part defines choreography as a global semantic specification, the operation contracts it exposes, and the obligations for deriving role-local behavior. It uses the evidence classifications of Part I and the identity, admission, knowledge, and authority model of Part II. Concrete model/FX integration follows in Part IV; browser realization and whole-application construction follow in Parts VI and VII.

The intended architecture supports ordinary full-language Clojure under the checked construction contract. A small portable choreography vocabulary is an interface for distributed coordination, not a replacement for the application's implementation language. Opaque code retains its named contracts and accepted premises through verification, projection, assembly, and execution.

Examples use illustrative declaration and API names. Their normative content is the specified semantic contract rather than the literal spelling of an example's helper names. Any public construction surface realizing the example must establish the corresponding checked bindings. Generated plan excerpts and state tables are inspection views, not a second handwritten workflow. Fixture symbols such as R, C, and E denote identities or values bound by the surrounding example.

What a choreography is

A choreography is a global description of a distributed state machine involving N roles.

The global choreography is projected into N role-local machines. At runtime, concrete actors instantiate those roles and execute the local machines independently. The entry/participation contract binds each role instance to its actual actor and principal or authority. A role name is not a credential, and two tabs for one principal do not share knowledge merely because that principal is the same.

The application may have many concurrent executions of the same choreography. Admission states which instances may participate, how their messages and resources are separated, and whether a logical role can have multiple members. An aggregate role is not silently treated as one omniscient actor. Projection of one execution also does not prove isolation from other operations; model invariants and atomic guards govern their shared authoritative effects.

There does not need to be a second centralized global machine executing beside them. The global choreography is the specification from which local machines are verified and projected.

Under the declared entry, runtime, and environmental premises, the interaction of admitted projected machines must refine behavior allowed by the global choreography. The refinement claim must identify its supported language subset, observable behaviors, and treatment of internal or physical steps.

The refinement relation is between admitted global semantic executions and admitted distributed executions of the independently projected machines under the specified environment, message-delivery, and fault contracts. A global `:communicate` occurrence is a semantic communication boundary; emission of a packet by a projected sender is not, by itself, proof that the receiver admitted the matching message or acquired its declared knowledge. Permitted hidden local steps and transport activity require an explicit trace-observation relation.

A successful structural projection check, role-local reachability result, or replay of one concrete schedule establishes only its stated checking property. It is not an all-schedules refinement theorem or a fairness/liveness proof. A stronger claim must quantify over the admitted schedules and retain its reordering, delivery, terminal-outcome, and environment premises. In particular, a projected role finishing does not mean it knows the authored global terminal result unless the choreography establishes that knowledge.

This claim about cooperating projected machines does not assume that an adversarial browser follows its projection. Trusted-side admission and model enforcement must still preserve the protected authority boundary for arbitrary client inputs, as specified in the trust model.

A choreography may describe:

- local computation;
- communication;
- authoritative establishment and observation requirements;
- environmental events;
- branching;
- authoritative operations;
- provisional projections;
- resource acquisition and release;
- failure paths;
- timeout paths;
- settlement and reconciliation;
- convergence requirements.

A choreography does not imply that all participants change state atomically. Distributed actors may temporarily hold different knowledge. Model guards protect authoritative effects at their own qualified transition boundaries; projection neither serializes every concurrent execution nor transfers one participant's unobserved authoritative result to another. Missing messages, indeterminate outcomes, and stalled environments remain explicit recovery or progress obligations rather than being erased by successful construction.

The important distinction is between:

```
authoritative state
provisional state
actor-local knowledge
```

and not a false claim that the entire distributed system changes simultaneously.

A choreography also does not imply one persistent runtime machine per connected client or one persistent server process per conceptual workflow.

The global description may be realized by short-lived projections that interact through durable authoritative facts.

A concrete coordination vocabulary

The canonical global representation is a versioned graph of typed semantic states. States have stable source identities for explanation, successors, and the contracts their transitions use. Semantic keys and options are closed for the selected representation version: an unknown opcode, misspelled required field, unsupported option, or unresolved contract reference produces a specific construction failure. An explicit nonsemantic metadata/extension area may be retained, but its contents cannot affect execution or strengthen a claim without a declared interpretation. The closure guarantee belongs to the **admitted semantic interface** after checked elaboration, not to the mere existence of a Clojure map or successful low-level structural normalization. A low-level normalizer may retain fields it does not interpret. The supported construction profile must distinguish meaningful semantic fields from inert, declared metadata and reject unknown or misspelled executable fields before claiming a closed contract. Silently discarding such a field could weaken an operation without a visible failure.

A supported baseline has six node kinds:

Global node	Meaning	Required contract
`:local`	Work at one role	Named implementation, placement, inputs, outputs, effe
`:authoritative`	Result of a public authoritative operation at its trusted owner	Operation interface, caller/authority binding, result alte
`:branch`	Selection using an already established local value	Owning role, typed selector, exhaustive supported alter
`:communicate`	One semantic transfer between roles	Sender, receiver, payload, correlation, disclosure, and ch
`:await`	A role admits an event from its environment	Registered source, event alternatives, owner, payload, a
`:return`	Declared global terminal disposition	Meaning of completion, remaining obligations, and any

The corresponding role-local vocabulary includes `:send` and `:receive` for the two sides of communication, alongside local work, authoritative work, branching, environment waits, and local return. A participant message cannot satisfy an environment wait merely because its event keyword matches. An environment event likewise cannot impersonate a participant receive.

Selectors distinguish missing bindings, present nil, and false according to the declared type. If nil is a supported member of that selector type, it needs an explicit alternative; it must not be conflated with an absent value. A target that cannot represent such a selector rejects the unsupported contract during construction rather than interpreting the value through generic truthiness.

A local node is not permission to conceal a database write, outbound message, protected disclosure, or required resource transition. Its interface exposes effects that matter to the claimed behavior. A computation may be hidden in a particular distributed trace only when the trace abstraction permits it; it remains subject to local safety, resource, and implementation obligations.

An authoritative result is not synonymous with a committed write. It may report rejection, a disclosed recorded outcome, or uncertainty under the operation's contract. The result's category and payload determine what the next state may conclude. Merely naming the state `:authoritative` establishes no runtime trust.

An acquisition can be realized as a declared effect followed by an environment completion carrying an admitted authoritative observation. It need not be a new portable observation opcode, and it must not be mistaken for a new model mutation. The observation contract binds the authority, subject, query, projection, principal context, basis, and temporal meaning required by Part II.

Higher-level constructs such as an operation invocation, authoritative acquisition, retry, resource scope, or a source-language conditional elaborate to these transitions or to an explicitly versioned extension with defined semantics. An invocation at a browser role does not relocate the authoritative operation into the browser: elaboration includes the declared request, trusted-side operation, and outcome-delivery boundary. A macro or helper must expose this expansion for inspection and checking.

For an invocation whose caller role and authority role are distinct, the elaborated program must account for three **different** propositions: the caller has issued a command under an admitted binding; the trusted authority has produced an admitted command-specific outcome; and the caller has actually acquired that outcome under its own knowledge, source, and correlation contract. The semantic `:authoritative` boundary belongs to the trusted authority, not the initiating browser or manager. A network request, HTTP response, or effect callback is not by itself a global `:authoritative` result or an admitted knowledge transfer.

A declared participant communication may carry the bound request or its result, producing the corresponding global `:communicate` occurrence and projected `:send` / `:receive`. Alternatively, an explicitly authored environment acquisition may admit a trusted outcome observation at the waiting role through `:await` and its registered source contract. Different admissible realizations need not add a peer message merely to model transport. They **must** preserve whatever communication or observation the authored protocol actually requires, and establish the result's command, principal, authority, subject, outcome, and causal relationship before a dependent branch. No compiler insertion of a synthetic environment event may make an otherwise unsupported global history appear to have occurred.

The final `:return` describes a global terminal judgment under the authored protocol. Individual projected roles may become locally `:completed` earlier; that fact neither supplies the global return outcome nor discharges unfinished required communication, external effects, recoverable delivery, or command uncertainty. Any terminal refinement claim relates the full admitted distributed execution to an actual global terminal trace under its named progress and environment assumptions.

For example, this small graph classifies an already acquired review input. It does not itself perform an approval:

```
(def classify-review-graph
  {:name :expense/classify-review
   :initial :classify
   :states
   {:classify
    {:op :local
     :role :manager
     :action :expense/classify-route
     :requires #{:review/inputs}
     :outputs #{:review/path}
     :next :choose}
    :choose
    {:op :branch
     :role :manager
     :on :review/path
     :cases {:finance :finance-required
             :manager :manager-route}}
    :finance-required
    {:op :return :outcome :review/finance-required}
    :manager-route
    {:op :return :outcome :review/manager-route}}})
```

The operation entry binds `:review/inputs` to the manager's admitted observation. The action interface promises an output of type `[:enum :finance :manager]` under its typed policy or retained opaque contract. The branch consumes that output; it does not compute the policy or invent the selector. If the action can fail or yield a third alternative, its actual interface and graph must expose that path or the admitted outer failure contract must handle it. The constructor cannot accept a partial success-only graph as a totality proof.

The displayed map is an illustrative semantic graph, not a verified standalone typed program. Its `:review/inputs` entry binding and the `:review/path` enum are supplied by external admitted entry/action contracts; neither is established merely by the keys in this map. Structural verification establishes the well-formed state, value-key and branch successor contracts; role-local definite-knowledge analysis establishes whether the selector is available at the required program point. Those checks do not by themselves prove that an opaque Clojure action always returns one of the two enum alternatives or that its implementation respects the intended policy. A checked nominal enum and complete cases require a separately admitted type/operation contract and correspondence evidence, or an explicit unmatched-value failure. The baseline branch-case contract does not admit `nil` as a case value. An explicitly admitted extension for **present nil** distinguishes it from an absent value and from `false`, and checks all corresponding case and failure continuations.

Vocabulary admission counterexamples

Apparent input	Required classification
A local state uses `:output` instead of `:outputs`	Reject an unknown executable field at the checked construction boundary; do
A two-case route action returns `:deferred`	Reject the unmatched selector or use an admitted third case; never silently cho
An `:await` event matches a participant message keyword	Enforce the environment-source boundary; it cannot satisfy projected `:receiv`
An open message contains an extra map key	The extension does not automatically become known semantic input or autho
A `:local` action hides a protected write or disclosure	The effect still requires an exposed admitted contract and trusted enforcement
One projected role completes while another still has work	Do not claim that the global outcome occurred or that every role knows it

These are requirements for selected construction and realization profiles. They are not assertions that the current raw graph normalizer, static verifier, or browser implementation already proves the whole closed, typed, provenance-qualified and physically enforced contract.

The sample spells out input/output sets to show the semantic representation. Authoring helpers derive them from inspectable expressions and operation interfaces where possible. An opaque action's declared boundary remains an explicit commitment. Source states such as `:classify` may become compact runtime locators; that encoding does not change these semantics.

A worked expense review

Consider a manager reviewing a previously submitted Expense. This short choreography starts with a submitted Expense selected from an authorized queue. The employee's earlier submission and Finance's possible later approval are separate short executions connected by durable state. This keeps an unused Finance role from waiting forever in a protocol that never needed its action.

The application owns the domain schemas and public operations:

```
(def Expense
  [:map {:closed true}
   [:expense/id uuid?]
   [:expense/employee-id uuid?]
   [:expense/department keyword?]
   [:expense/amount Money]
   [:expense/status
    [:enum :draft :submitted :awaiting-finance :approved :rejected]]])

(def ApprovalPolicy
  [:map {:closed true}
   [:policy/department keyword?]
   [:policy/finance-required-at Money]])

(def finance-route-rule
  {:rule/id :expense/requires-finance
   :inputs {:expense Expense :policy ApprovalPolicy}
   :preconditions
   [[:= [:expense/department :expense] [:policy/department :policy]]
    [:same-currency [:expense/amount :expense]
                    [:policy/finance-required-at :policy]]]
   :returns :boolean
   :expression
   [:money/≥ [:expense/amount :expense]
             [:policy/finance-required-at :policy]]})
```

`Money` and its operations use the explicit money profile from Part I. The rule does not compare unspecified host decimals or mix currencies. Its policy is deliberately simple: amounts at or above the threshold require Finance; smaller amounts may take the manager path, subject to all other model guards. A real application may supply a richer opaque Clojure policy under an explicit contract.

The review acquisition returns Expense and Policy under one compatible read context. It binds the Expense to the selected id and the Policy to that Expense's department. Knowing an id does not establish any of these facts. The manager principal and permission to read come from trusted acquisition context. Compatibility of independently acquired values must be established by the read/authority contract; merely putting them in the same return map does not prove they were read from one coherent basis.

A domain may also impose a manager approval ceiling. This simplified Finance-threshold example does not silently assert that a manager has unlimited approval authority: if the real domain applies a manager ceiling, that rule belongs to the admitted public model operation and its transaction-current guards (and, where used for routing, to the declared review policy). It must not survive merely as an unused schema field or an informal browser-side assumption.

The operation invocation contracts are:

Operation	New-incorporation requirement	Su
`:expense/approve-as-manager`	Current manager permission, submitted Expense, applicable Policy, and manager-route eligibility	E
`:expense/mark-awaiting-finance`	Current manager permission, submitted Expense, applicable Policy, and Finance-route eligibility	E

Each operation atomically protects its current dependencies, domain changes, and command-result record under Part IV. The preparation read determines which operation to try; it does not reserve eligibility or authorize incorporation. A compatible replay follows the recorded-outcome and current disclosure rules without reapplying a new-incorporation status guard. The authoritative incorporation's notification contract is chosen independently of the subsequent caller-visible send. If the business operation requires a **durable review notification**, C2 incorporation also establishes the relevant stable publication/work intent at that atomic boundary, or the operation supplies an equivalently qualified durable discovery path. A manager browser's later result-bearing send may prompt that work but cannot be the only recoverable record when the browser may disappear between commit and send. If the notice is deliberately best effort, the operation does not manufacture mandatory delivery debt; its accepted command and terminal claims explicitly retain that weaker notification guarantee.

Here is the choreography's control structure. The invocation and acquisition rows are authoring constructs whose elaboration exposes the boundaries described above; the table does not introduce additional hidden primitive semantics.

State	Owner and action	Successors
Acquire review inputs	Manager requests the admitted Expense/Policy observation	Success: classify; denied/not-found:
Classify	Manager applies the typed Finance-route rule to the bound observation	Finance required: invoke escalation
Invoke escalation	Trusted authority processes the bound escalation command	Settle the command-specific model
Invoke manager approval	Trusted authority processes the bound approval command	Settle the command-specific model
Settle	Manager admits the disclosed result for the actual command	Accepted: publish review notice; re
Publish review notice	Manager emits the permitted historical review result toward the notifier	Manager's sending boundary may f
Close without notice	Manager emits the declared no-notification disposition	Notifier must admit that closure me
Notify or close	Notifier receives the correlated disposition	Review notice: perform the declare

1. Admitted elaboration of the review protocol

   The table is an authoring view. It does not justify the manager learning a value merely because the authoritative operation appears earlier in the **global** graph. Checked elaboration retains the following boundaries and source-to-plan evidence:

Authored step	Admitted semantic/role-local obligation
Manager acquires Expense and Policy	The manager's declared acquisition effect and admitted `:await` observation establish sc
Manager classifies the route	A `:local` action or supported typed expression establishes the manager-known selector
Manager submits C2 to Expense authority	The bound command request reaches the declared trusted operation interface under the
Expense authority settles C2	The ‘=:authoritative=‘ semantic operation occurs at the trusted authority role with auth
Manager acquires C2's result	A declared authority-to-manager `:communicate` with admitted `:receive`, or a genuinely
Manager selects notification or closure	A role-local `:branch` uses the actually admitted outcome. Recorded success uses the res
Notifier receives the selected disposition	The chosen admitted `:communicate` establishes the notifier's permitted fields at actual re
Notifier performs the declared notification effect	The effect has a declared semantic footprint and a separately evidenced physical issue/c
The review session finishes	Local `:return` / completion may occur at different times for manager and notifier; the cl

   This is an **obligation mapping**, not a claim that the rows are literally nine global opcodes or that every deployment must use direct manager-to-authority messages. Different transport and observation choices may be valid only under their exact authored contracts and a checked refinement relation. The compact executable plans and their source/proof mappings must preserve causal order, role knowledge, observable operation identity, failure alternatives, and all terminal obligations. The notation does not assume that a physical provider callback is an authoritative database decision.

   The model outcome and the presentation settlement are distinct:

Model evidence	Permitted continuation
`:committed` with recorded successful approval/escalation	Accepted continuation for that exact command
`:already-incorporated` with a permitted recorded success	The same accepted continuation, without another incorporation
`:rejected` with an authoritative command rejection	Rejection handling; no accepted-success notice
`:commit-unknown` or indeterminate result delivery	Unknown handling until command-specific authoritative evidence resolves it

   If a model retains other terminal outcomes, the disclosed recorded disposition has its explicit mapping. A transport failure after a successful commit does not turn that commit into a rejection. An unrelated Expense now being approved does not establish that this command approved it.

   This worked review chooses a **result-bearing notifier protocol**: the recorded message deliberately discloses the permitted result of the bound manager command to the notifier. This is a different disclosure contract from the **neutral notifier protocol** in the later behavior-sensitive example. The outward notification text can be identical under both manager outcomes, but an internal recipient of `:review/result` still learns which result was reported. Matching text on screen is not a proof that the notifier cannot distinguish the branches.

   The message contains only the declared, disclosure-permitted review information. It does not contain the manager's private threshold or entire observation. The issuer or admitting trusted boundary justifies the command outcome before that result is disclosed. A small inspection view of this **result-bearing** contract is:

```
{:message/id :expense/review-disposition
 :from :manager
 :to :notifier
 :required #{:review/session-id :review/disposition}
```

```
:correlation #{:review/session-id}
:variants
{:recorded
 {:required #{:expense/id :gesso/command-id :review/result}
  :result-type [:enum :approved :awaiting-finance]
  :source-evidence :admitted-command-outcome}
 :no-notification
 {:required #{}}}
:delivery-contract :expense/review-notification}
```

The `:variants` entry in this map is **higher-level contract inspection data**, not a self-executing global `:communicate` operation or a second independent message language. Checked elaboration turns the alternatives into actual admissible communications and disjoint projected receive alternatives, for example with separately identified events or an admitted discriminating closed payload contract. Each resulting communication retains its required, optional, correlation, sender/receiver, and disclosure contracts. The constructor must reject unresolved overlaps rather than letting message arrival or an optional field arbitrarily choose a continuation. Role-local knowledge-flow checks alone are not proof of the application-level tagged result's meaning or trusted source.

The admitted review session supplies its correlation identity at entry. The declared `:review/disposition` discriminator must be checked as `:recorded` or `:no-notification`, with the recorded variant's `:review/result` bound to its closed approved/awaiting-Finance alternative and exact command-result evidence. Checked elaboration makes the common and variant-specific obligations explicit:

Admitted variant	Closed semantic fields and discriminator
`:recorded`	Common `:review/session-id` and literal `:review/disposition :recorded`, plus `:expense/id`, `:gesso/command-id` an
`:no-notification`	Common `:review/session-id` and literal `:review/disposition :no-notification`; no command result or failed-read

The variants' schemas, source/recipient admission, correlation and permitted disclosure are independent obligations; the `:variants` inspection map alone does not prove them. They may elaborate into distinct identified communication events or a common event with an admitted disjoint tagged-payload contract. A receiver may not select a variant by absent optional fields, message order, or a convenient fallback match. The receiver's actual consumed variant alone establishes its declared knowledge. Only the recorded variant carries a command outcome. The no-notification variant terminates participation without claiming the Expense is absent, the command failed, or no review was ever recorded. It does not carry a failed read's private subject. Both variants have closed payload contracts and declared recipient permission; timing and participation metadata also follow the declared disclosure policy. A required participant is not left waiting merely because an earlier acquisition or operation failed; unavailable delivery follows the session's explicit timeout/retirement or recovery contract.

This is contract inspection data, not a credential. The trusted notification boundary must validate the origin and permitted command outcome if an adversarial manager browser transports the notice. A manager-role label or plausible result string cannot establish that a review occurred. If the service instead derives the notice from an authoritative outbox, that is the declared realization and causal path; projection must not invent it silently.

The delivery contract also decides what failure means. The following milestones have **different** completion evidence:

Review/notification milestone	What it establishes, and what remains open
C2 incorporated	The bound Expense operation and command outcome are authoritative; if required, a pro
Publication work recorded	The model owns a stable scoped work/intent identity and recovery obligation; neither a p
Manager local `:send` completed	The manager's declared local emission succeeded under the send contract; the notifier ma
Notifier admitted `:receive`	The particular global semantic `:communicate` occurrence is realized and declared commun
Physical notification issued or accepted	The registered notifier/provider completed the **named** physical stage under its correlatio
Notification effect fulfilled, failed or handed off	Only the separately declared completion, failure, retry, retirement or durable continuatio

2. Notification realization profiles and recovery ownership

   The worked table above depicts a **manager-mediated** review session. It does not silently define the physical topology of every durable notice. The application selects and admits its actual realization, including the producer of each semantic message, recovery owner, and terminal claim:

Authored notification profile	Permitted behavior after C2; required proof boundary
Manager-mediated, best effort	After an admitted result, the manager may send the correlated disposition. Loss of that send is p
Model-owned durable publication	C2 creates or qualifies stable discoverable work. A separately admitted worker/notifier executio
Recoverable manager-mediated delivery	If the authored global protocol requires the manager-to-notifier occurrence, durable state alone

The chosen profile is a **global-plan choice**, not a mere transport choice for an unchanged manager-to-notifier graph:

Authored graph	Relevant terminal obligation
Manager-mediated communication	The projected manager actually sends, the notifier admits the correlated receive, and the global term
Separate durable publication	C2's guarded success and retained work may complete the review execution under a work-recorded
Checked continuation or delegation	A required original communication persists through an admitted owner/role transfer or semantic alt

The graphs may have the same business-level **work-recorded** postcondition while admitting different participant traces. The checker must validate the **selected** global specification, projected source and receiver roles, knowledge paths, and property-specific terminal evidence. A later worker effect cannot retrofit a missing manager message into the old graph. Equally, a work-recorded review terminal does not wait for future worker fulfillment when that stronger effect was not its postcondition; the worker's unfinished obligation remains live under its separately admitted owner.

A worker may continue the **business notification obligation** after the manager's browser retires without continuing the manager's old runtime execution. A model-derived outbox trigger is not by itself a peer `:communicate`, a consumed `:receive`, or the old manager's actuation. If the worker's protocol is a new short execution, its entry knowledge, current authority, consumed work identity and effect ownership must be established there. If a checked alternative replaces the manager send, the specification and projected plans expose that alternative rather than silently rewriting a manager-origin message as worker-origin.

The result-bearing disclosure contract is retained under each profile: an authority- or worker-origin notice may disclose an admitted command outcome only through its own trusted command binding and current recipient policy. The best-effort profile cannot interpret a failed or omitted post-commit send as an authoritative `:no-notification` result unless that exact closure path is authored and admitted. In particular, silence does not turn accepted C2 into a rejected C2.

A durable worker and an optional browser hint may coexist, but the contract must relate their publication identities, independently admitted occurrences and deduplication effects. A repeated hint is not a second business command, and an outbox retry is not proof the provider failed the earlier attempt. A global terminal for the manager session can promise C2's incorporation and recorded work independently of the later notification execution. A terminal promising actual external fulfillment still needs that later effect's qualified evidence and assumptions; a promise of eventual fulfillment additionally names the permitted discovery, availability, provider and fairness premises.

The stable delivery/work identity is not interchangeable with C2's semantic command identity, a transport attempt, or the root identity of one participant message. A compatible command replay does not create new required work. A retried delivery may create a new physical attempt without authorizing an additional externally significant notification; the provider and receiver contracts govern deduplication, reconciliation, and any legitimately repeated effect. An exactly-once physical effect claim requires evidence strong enough for the chosen failure model; a durable outbox by itself does not establish it.

An accepted `:send` completion establishes that the sender discharged its **local sending** boundary, not that the global `:communicate` occurred at the recipient or that the notification effect executed. The declared semantic communication is realized at admitted receiver consumption; if that cannot happen before local retirement, the global terminal path requires a distinct, explicitly admitted handoff or failure/timeout continuation with its own business meaning. Neither a missing receive nor a sender's local `:return` silently chooses `:no-notification`. For a required durable notification, the model transaction or another justified handoff must retain recoverable notification work. A direct browser send that can disappear after commit cannot establish eventual notification. For a best-effort notice, loss may be allowed but the assurance claim must say so. The normal global terminal for such a protocol may promise an **attempt** or admitted retirement, not guaranteed delivery. A durable-handoff terminal may promise that recoverable work exists, not that the future effect already happened; that stronger future-delivery claim remains subject to explicit scheduler, provider, and fairness premises. If the declared terminal promises actual physical fulfillment, the proof must also include the qualified effect-completion evidence and the physical observation/refinement contract, not only a coarse distributed trace. Replaying an accepted command result must not resend an externally significant notification unless its own deduplication or repeat policy permits it.

Worked-review counterexamples and correspondence duties

Case
Manager's Policy/Expense observation is inconsistent or no longer eligible by incorporation
A manager approval ceiling applies but is absent from the chosen operation's guarded policy
A recorded and no-notification disposition use the same event, and the receiver selects by whether optional result fields happen to be prese
Manager completes a transport send of the no-notification variant and the compiler records that the notifier returned without admitting it
A retry discloses a permitted recorded successful command outcome
A browser fabricates `:recorded` with a plausible Expense id or command id
A result-bearing message discloses `:review/result` to the notifier while a proof calls the notifier route-ignorant because its displayed notic
The global graph contains the authority's C2 result and the manager settles on it without a receive, trusted observation or other admitted k
A manager-side HTTP 200 response is lowered into `:authoritative` at the manager rather than at Expense authority
The manager finishes locally after sending a notifier message, and the compiler reports global success while the required notifier receive/ef
Two result variants can match the same sender/event/channel and payload
An accepted model commit is followed by notification delivery failure
C2 commits and a required notification is lost because the browser closes before sending, with no durable intent or discovery
A notifier admits the review message and the global terminal is reported as a delivered user notification before the provider effect complete
A notification worker retries after an unknown provider response using a new unbound work identity
The manager chooses an approval path without notifying Finance

These are example-level acceptance and correspondence obligations, not by themselves a certification of compiler, application, or notifier realization. An all-participant choreography remains permissible **when** its declared delivery and branch-knowledge obligations can be met; this short review intentionally uses a later Finance execution rather than requiring an idle Finance participant to wait indefinitely.

Projection produces complementary plans, not separately authored workflows:

Projection	Required behavior and knowledge
Manager	Knows selected Expense id; acquires scoped inputs; derives the route; invokes the selected operation; interprets its co
Expense authority	Admits each actual invocation, authenticates the caller, applies replay or atomic new-incorporation semantics, and dis
Notifier	Receives the declared disposition; performs the notification effect with its own outcome handling or returns without t

Under this result-bearing contract the notifier acquires the disclosed `:review/result` when the recorded variant is admitted, whether or not its physical notification text varies with that result. This contract cannot be used to prove that the notifier is ignorant of the manager's route. It does not require disclosure of the private threshold itself. An application requiring a route-neutral notifier instead authors the separate neutral contract discussed under behavior-sensitive choice; the projector cannot erase a required result-bearing message merely because the visible notification would look the same.

Finance's later queue observation establishes entry knowledge for a new execution; no unused role in this short choreography has to infer that a missing message means it may terminate.

A role's local return means that role has no remaining work under its contract. It is not proof that every other role completed, every message was delivered, or the whole business workflow finished. The global completion judgment and any durable recovery handoff retain those distinctions.

Choreography is specification, not a hidden coordinator

The global choreography is the semantic specification of allowed distributed behavior. It is not a requirement that Gesso run one omniscient global state machine alongside every projected role.

The implementation must therefore preserve two complementary truths:

- the global description is authoritative for what behaviors are admitted;
- projected actors execute locally and independently using only the knowledge, resources, and authority available to them.

A runtime architecture that requires every local step to consult a central choreography interpreter solely to rediscover projected control flow would defeat an important purpose of projection. This does not prohibit a declared authoritative operation from consulting its centralized model authority or checking a trusted runtime premise. The prohibition concerns an unnecessary global control interpreter, not model-owned coordination whose semantics the application actually requires.

Conversely, independently executing local machines do not become semantically independent programs. Their combined traces remain constrained by the global description.

The executable global semantics may still run as a reference oracle in a checker, simulator, or deterministic test harness. Such a harness may schedule several projected roles and compare a supplied execution with the global machine. This is not a second production coordinator, nor is one successful witness a proof that every admitted distributed schedule refines the global specification. Actual production participants must obtain their needed local knowledge and satisfy their own message, owner, environment, and authoritative operation contracts without consulting a hidden global control state.

The intended relationship is:

Stage	Product and obligation
Global specification	Declares the allowed distributed behavior
Verification and projection	Derives admitted role-local plans with explicit premises
Independent local execution	Produces concrete interactions under the realization contracts
Refinement judgment	Relates those interactions to the admitted global behavior and stated observation alphabet

The absence of a central runtime machine is therefore not a weakening of the global specification. It is the point of a successful projection.

The **observation alphabet matters** when the choreography carries externally significant effects. A coarse distributed semantic trace may omit ordinary `:local` steps; this does not authorize hiding a notifier's real external delivery, a charge, a protected disclosure, or an actual DOM installation from a claim whose postcondition includes it. A checked local action exposes its effect contract and the relevant issued/fulfilled/failed boundaries to its realization. An assurance claim about the physical effect uses a corresponding physical observation and owner/effect identity relation, even when a different semantic trace intentionally abstracts that action away.

The return condition is property-specific: ”command incorporated”, ”required publication intent persisted”, ”notifier consumed the message”, and ”recipient was actually notified” are distinct postconditions. A global terminal outcome cannot establish a stronger physical postcondition merely because a weaker semantic trace reached `:return`. Durable handoff can discharge a **work-recorded** obligation if the authored terminal permits it, while retaining a separate future fulfillment obligation and any named environmental progress premises.

The **claim subject** matters as much as the observation alphabet. An assurance result over one reviewed global plan does not directly certify a second worker execution, its installed effects, or the composed business workflow. Gesso derives each required judgment from the authored application and selected assurance policy, then joins only evidence whose subjects, identities, premises and scopes actually correspond:

Claimed property	Required evidence subject and boundary
Review choreography terminates	Selected global plan, exact terminal alternative, and its admitted participant/role-local tra
C2 incorporated	Bound command, qualified authoritative operation, protected state/command record and
Required work was durably recorded	C2's protected stable work identity and authoritative atomic/discovery contract, with an a
Worker can undertake the work	Its separately admitted plan and entry knowledge, trusted acquisition/authorization, activ
Physical effect was issued, accepted or fulfilled	Stage-specific source-bound effect/attempt/work identity, registered provider semantics a
Outstanding work eventually resolves	Appropriate safety/correspondence premises plus complete recovery, permitted discovery

The composed application claim preserves the **dependency edges** between those subjects: C2's publication record, worker acquisition, issued effect, and reported completion need trustworthy correlation and source-to-installed-realization correspondence. A certificate about the authored choreography may be sound while the deployed worker, route, scheduler, codec or provider binding is absent or substituted. Part VII requires those actual paths in the assembly; Parts IX–X own evidence admission and property-specific refinement; Part XI decides which claims and remaining premises a release profile requires. The semantic plan does not create an installed worker, transport route, physical effect or acceptance result by declaring the same identifier in multiple maps.

An admissible **work-recorded** terminal may be used by a caller without waiting for a worker that has not yet started, as long as the separate work and recovery obligation remains accounted for. If the requested release claim additionally promises physical fulfillment or conditional eventual delivery, it must include the relevant installed paths and stronger evidence rather than silently promoting the review's weaker terminal. Independent local and worker executions may compose into a justified business-level property without becoming one fictitious long-lived Choreo machine.

Semantic behavior versus physical realization

A choreography describes semantic behavior. Physical realization is permitted to vary when that variation does not change the semantic trace.

For example, the semantic statement:

```
:request-authority establishes the result of :request/claim
```

does not require:

```
one fixed Aleph process
one fixed HTTP connection
one fixed DOM node
one fixed browser tab
one fixed database connection
```

The authority may be realized through different physical resources at different times. A retry may reach another host. A fragment may be replaced by a new DOM node. A browser may reconnect. None of these physical substitutions creates a new semantic authority or silently changes the meaning of the operation. Neither does their mere occurrence establish that continuation is safe: admission must preserve or re-establish the relevant command binding, qualified authority/incarnation and basis, role/principal rights, execution or physical resource ownership, and permitted artifact/projection correspondence.

A host restart or changed transport attempt cannot manufacture command nonincorporation or authorize retry after an unknown outcome. A new DOM resource cannot inherit the old resource's pending effect permissions merely because its logical target name is equal. A database with the same authority label but a different unqualified incarnation cannot be treated as the same authoritative history. The exact checks are owner- and operation-relative, not a demand to attach every identity to every semantic event.

This end-state design therefore requires every layer that crosses a replaceable physical boundary to preserve the logical identities needed by the semantic layer rather than reconstruct them from physical coincidence.

Detailed multi-node routing belongs to its separately scoped realization contract. The semantic requirement established here is only the stronger and more durable one:

> A host may realize a role or authority for an execution, but physical hosting does not create semantic authority.

Physical realization correspondence cases

Proposed realization	Required evi
A deterministic test harness advances all roles together	Useful bound
A command is retried through a different HTTP host after a lost response	Preserve com
An HTMX target is replaced before a delayed effect completes	The retired r
A replacement authority reports the same semantic name with different database lineage	Re-establish
A projected executable plan is repackaged or deployed elsewhere	Check the ad
A coarse semantic trace hides the notifier's effectful local action, yet an assurance result claims successful external delivery	Require quali

These cases define realization and assurance obligations. Listing the cases is not a proof that physical transport and browser effects refine the abstract Choreo machine; that stronger judgment requires its own property, premises, and correspondence evidence.

Choreography may be partial with respect to the whole application

One choreography need not describe an application's entire lifetime.

A choreography may describe one distributed semantic operation or one coherent short-lived protocol. Durable model facts may connect many such protocols over hours, days, deployments, or browser lifetimes.

This means that the following are both legitimate:

A short Claim choreography invokes the operation, interprets the authoritative outcome or preserves uncertainty, settles its local obligations, and terminates or hands off recovery under the declared contract.

and:

Execution	Authoritative role in the workflow
E1	Create the approval request, establish pending state, and settle or hand off its command
E2, later	Acquire current approval premises, act on the request, and settle its own command

The durable relationship between E1 and E2 lives in authoritative model state unless the application explicitly adopts a durable workflow facility. The later execution must acquire its own current premises and authority, not inherit an expired browser execution's knowledge, resource rights, pending callbacks, or command identity. E1's command may still need separately owned reconciliation after E1's presentation retires. The model's durable relation supplies continuity of business state, not an implicit suspended local program or permission to replay E1.

This separation prevents Gesso from smuggling a universal workflow engine into ordinary Choreo merely because some business processes span human time.

Choreography validity and application realizability are distinct questions

A choreography can be internally valid while the surrounding application fails to realize a required use of it. An intentionally unexposed operation is not automatically an assembly defect: required exposure follows the declared realization scope and independent product acceptance.

For example, a verified operation may still be unusable because:

- no view binds it;
- no projected browser plan is packaged;
- the packaged artifact is stale;
- no transport binding reaches the trusted server;
- the server has no corresponding execution capability;
- no authoritative acquisition path can observe the successor state.

Missing required realization edges are whole-application assembly failures. Omitting an intended user capability can instead leave the remaining assembly closed while failing behavioral completeness. Those distinctions follow the three application assurance claims in Part I. Neither failure changes the meaning of choreography validity itself.

An assembly claim is therefore relative to an **admitted required-use scope**, not every operation merely present in the model. For a declared use, closure requires the specified correspondence edges across the projection, artifact, route, execution capability, model operation, settlement, and observation or recovery path. A route whose name happens to match does not prove the actual deployed handler or physical outcome; that stronger claim needs separately qualified evidence. An omitted optional affordance may be acceptable for assembly closure and nevertheless fail the product's independent capability acceptance. The checker must report which question it answered.

The distinction is important:

```
valid choreography
    ≠
complete application realization
```

and equally:

```
HTTP route exists
    ≠
valid choreography
```

The unified design therefore preserves Choreo as a strong semantic layer while also requiring whole-application checks of the joins between semantic and physical/application layers, production enforcement of those checks, and independent capability acceptance. Closure results retain their actual assurance classification rather than automatically becoming semantic theorems.

Construction must fail when the contradiction is already knowable

Choreography construction and verification must reject known contradictions at the earliest supported boundary possessing enough information to establish them. Unsupported constructs must be identified explicitly; they must not receive a successful result by omission from the checker.

Definition-time failures include, for example:

- unknown roles;
- malformed transitions;
- impossible branch references;
- invalid message declarations;
- inconsistent operation metadata;
- use of an undeclared authority;
- a local behavior that requires a fact that the role can never know.

These failures should not be deferred to browser execution merely because a runtime implementation could eventually notice them.

This does not mean Gesso should pretend to know dynamic domain facts in advance. A Request becoming claimed by another user after rendering remains a runtime/domain event. Static strictness applies only where the contradiction is already present in the declared semantic structure. Static admission must also require declared handling or recovery for dynamic conditions that can strand a required operation. It can check that an unknown-result path exists without predicting whether the next network delivery will be indeterminate.

A structural contradiction, unsupported proof obligation, and accepted opaque contract are different results. Full-language code is admitted through its declared boundary and assurance policy; a missing role, unresolved operation binding, or impossible continuation is not waived by calling it an assertion.

A declared recovery branch is not sufficient on its own when an application requires that branch to be usable: its owner, required knowledge, temporal and authority scope, executable plan, and necessary physical route or acquisition must also be checked at the layer that can inspect them. If a required edge is known missing, report an assembly failure rather than calling the branch 'conditionally proven'. If the edge exists only under an uninspected trusted-host premise, retain that premise explicitly rather than calling it a proven deployment guarantee. Runtime uncertainty remains possible even after a sound static check of the available paths.

Distinct diagnostic and acceptance cases

Situation	Correct judgment
An operation is well-formed but intentionally unpublished and not a required application use	Not an assembly closure def
A declared Claim affordance references a missing browser plan or command transport route	Missing required realization
A graph branch reads a value never definitely available to its owning role	Static choreography knowle
A correctly declared operation races with another actor's Claim	Dynamic authoritative outc
A `:commit-unknown` route names reconciliation but has no usable registered recovery capability	Reject required-path assemb
A preflight report agrees with supplied route declarations but no deployed-handler adoption evidence exists	Report bounded source/decl
E2 begins after E1 completed, using a current authorization read	Valid new execution with its

The checks can fail at definition, projection, application assembly, startup or authoritative runtime depending on which trusted facts are available there. An accepted opaque contract does not override a known contradiction. A finite local check must not be promoted to a global proof, and a structurally complete application must not be called behaviorally complete without its independent acceptance evidence.

Canonical authoring principle

Application authors should state semantic intent once whenever the system can derive the rest.

A choreography should not require synchronized copies of the same semantic fact in:

- operation metadata;
- view metadata;
- browser-plan names;
- transport declarations;
- route strings;
- server registries;
- bootstrap registries.

Where an independent physical choice genuinely exists, it may be declared separately and checked for correspondence. Otherwise the preferred direction is:

One semantic declaration supplies the derived local plan, binding metadata, transport requirements, and diagnostics. Independently chosen realization parameters are checked against that declaration.

This principle does not collapse all layers into one macro. It reduces duplicated sources of semantic truth while preserving explicit architectural boundaries.

”Declare once” is relative to the **fact being declared**, not a demand for one universal application map. The public model owns business invariants, authorization dependencies and the actual effect of its operations. The choreography owns role-relative coordination and admitted continuations. The view owns the authored presentation choice; the application realization owns genuinely independent physical routes, resource bindings and deployment choices. A generated plan, handler summary or action descriptor is a derived view of these sources, not a rival authority for any of them.

For each generated edge, the system must retain its source and applicable version/identity, distinguish a mechanically derived fact from an imported trusted contract, and report unknown or unsupported derivation separately from a failed check. Opaque Clojure business code cannot become a proven policy merely because its operation name was imported. A hand-copied declaration with matching strings is not an acceptable substitute for an established source-to-assembly correspondence.

HumanHelp's Claim action gives a useful target for that authoring principle because the same semantic intent must eventually appear in a view, browser plan, transport, trusted handler, model operation, settlement, and authoritative reacquisition path.

A declaration-first surface separates the helper's inputs from authoritative obligations and explicitly maps model outcomes into presentation behavior:

```
(def request-claim
  {:operation/id :request/claim
   :roles
   {:helper {:principal :user :actor :client}
    :request-authority {:authority :request}}
   :input [:map {:closed true} [:request/id uuid?]]
   :entry
   {:helper {:values #{:request/id}
             :observation-for-optimism :request/card}}
   :invocation
   {:authority-role :request-authority
    :model-operation :request/claim
    :arguments {:request-id [:input :request/id]}
    :principal-source :trusted-authentication
    :command-protocol :stable-bound-command}
   :provisional
   {:role :helper
    :projection :request/claiming
    :owner-contract :current-execution-target-and-resource}
   :settlement-contract :request/claim-settlement})
```

The model-operation reference imports its authoritative interface. Its current Request, Membership, Organization policy, and Request/Organization association are protected by the new-incorporation contract; those are not facts the browser is expected to establish. In particular, current Membership, Request, Organization, and Claim authority premises remain actual model-owned dependencies; keeping them behind an imported public operation does not remove their transaction-current guard obligations. The checked import must identify the source-owned operation and applicable contract revision, including which rules are inspected or still opaque. An authorized observed card supports only the declared optimistic presentation. A non-optimistic invocation can have a different entry contract without claiming that same provisional behavior.

The admitted command boundary allocates a stable `:gesso/command-id` before the first possible send and binds it to the semantic operation, arguments, principal, and authority scope. It preserves that identity through retries. A browser may carry the bytes, but command authority comes from the trusted binding and model protocol, not from choosing a UUID.

The settlement contract is inspectable as follows:

Admitted result	Presentation settlement	Required next path
Successful `:committed`	`:accepted`	Acquire the permitted authoritative successor
Permitted `:already-incorporated` with recorded success	`:accepted`	Same successor path, with no new incorporation
Authoritative `:rejected`	`:rejected`	Settle provisional ownership and permit authoritativ
`:commit-unknown` or indeterminate delivery	`:unknown`	Preserve command identity and reconcile against co

Denied disclosure, unavailable acquisition, and an expired or incompatible recovery context also have declared paths. They do not become fabricated success or rejection. A reconciliation response may resolve a command while a later read is still needed to determine the current card state.

The view refers to the operation and supplies only its domain arguments:

```
(defn request-card
  [{:request/keys [id title status]}]
  [:article.request-card
   {:data-request-id id}
   [:h3 title]
   [:p.status (name status)]
   (when (= status :open)
     (gesso/action
       {:operation :request/claim
        :args {:request/id id}}
       "Claim"))])
```

The rendering boundary supplies the admitted target and observation context; the raw domain map and the conditional button are not authorization. The action constructor resolves the operation's admitted binding and derives the necessary plan, transport, and runtime descriptors. User-visible absence of the button does not enforce the model guard.

A generated helper plan has these obligations:

Concern	Derived or imported requirement
Entry	Selected Request id and the observation required for this optimistic mode
Command	Stable command reference with the admitted operation/argument binding
Provisional display	Current execution, target, principal context, and concrete resource ownership
Result intake	Expected operation and command, admitted source, permitted result, current owner, and settlement mapping
Recovery	Unknown remains unknown until reconciliation; reacquisition observes the declared scope and progression

Command identity alone is not enough to own a provisional DOM effect. A later execution may reconcile the same command while the earlier execution's callback has lost all rights to mutate the current target.

Only independent physical choices need separate authoring:

```
(assembly/realize-operation
  humanhelp-assembly
  {:operation :request/claim
   :transport {:kind :http/post
               :route :route/request-claim}
   :model-binding request-model/claim-operation})
```

The assembly derives and checks the view/action, browser-plan, server-entry, decoder, model, and settlement joins from the declaration and these bindings. It must not require the author to copy the generated plan name back into a separate bootstrap registry. Repeated metadata may be an inspection view or an explicit correspondence constraint, but is not an independent source of truth.

These code blocks specify **authoring and realization contracts**. The names `gesso/action`, `assembly/realize-operation` and `assembly/require-dispatched-binding!` are illustrative; their spelling is not a separate normative requirement. A plan registry or preflight check that proves a narrow structural join cannot simply be renamed into an end-to-end source-derived executable binder. The admitted authoring surface either derives the promised edge from checked source material or exposes the independently authored binding and its explicit open correspondence premise.

A schematic trusted handler illustrates the authority handoff:

```
(defn request-claim-handler [ctx request]
  (let [binding (assembly/require-dispatched-binding!
                 ctx :route/request-claim)
        principal (auth/current-principal! (:session ctx))
        trusted-ctx (assoc ctx :principal principal)
        command (server/decode-bound-command!
                 binding trusted-ctx request)]
    (server/invoke-bound-model-operation!
      binding trusted-ctx command)))
```

The dispatched binding comes from the admitted route, not a route name supplied by the browser. Authentication precedes any binding that claims an authenticated principal. The decoder checks the envelope and command reference under the operation contract; the public model boundary independently validates the authoritative command binding and current permission to disclose a replay. For new incorporation it protects all current guard dependencies and effects atomically with the command record. Separate handler pre-reads cannot supply that atomicity or reject a compatible replay merely because Claim succeeded earlier and the Request is now claimed.

The handler also uses the admitted response and recovery mapping. A response rendering failure after incorporation leaves the recorded command result intact and may leave the browser uncertain. Part IV owns the concrete transaction; Parts VI and VII own presentation and application construction.

The application author should not have to synchronize:

```
;; Avoid this style of duplicated semantic naming.

{:button-operation "claim"
 :button-plan "claim-v9"
 :hx-post "/requests/claim"
 :wire-command "__gesso_live_optimistic_command"
 :handler-var 'humanhelp.routes/claim
 :server-capability "claim-handler"
 :model-op "request/claim"
 :settlement-fragment "request-card"}
```

when those values are all consequences of one declared semantic operation. Where the physical route or capability name genuinely remains independently authored, assembly construction should check its correspondence rather than hoping the strings stay synchronized.

This example also makes choreography validity and whole-application realization visibly different. The choreography elaborated from `request-claim` can be valid while the `:request/card` view omits the affordance, while the current browser artifact omits `:plan/request-claim`, or while the route is not bound into the current assembly. Where those edges are required by the declared realization scope, those are application-realization failures. If the operation is intentionally not exposed, its absence is not automatically a closure defect. Whether the intended product capability is present remains an independent acceptance question, not a reason to weaken the operation contract.

Authoring and assembly acceptance cases

Apparent construction	Required treatment
A view uses `:request/claim` but the corresponding required browser plan is absent or stale	Reject the required reali
A server route string matches the declared operation but the actual deployed handler is uninspected	Preserve installed-handl
A copied operation summary omits Membership or Organization claim guards	Reject an incompatible c
A browser selects a command id, actor role or claimed principal	Carry the bytes as input
A rendered card omits the optional Claim affordance	Do not invent an assembl
An optimistic callback completes after its DOM owner retires, while reconciliation still recognizes the command	Historical command ide
A static surface snapshot passes but another path returns unvalidated raw HTML	Report the actual preflig

These are target design obligations. Static semantic construction, an assembled PlanRegistry, validation of supplied rendered surfaces, physically installed Biff handlers, and real command/DOM effects have distinct evidence boundaries. The authoring system must not hide a missing boundary by inventing a derived registry, accepting unchecked metadata as proof, or silently downgrading the operation.

Choreographic operation types

A choreographic operation type is richer than an ordinary function type.

An ordinary function type is roughly:

```
A → B
```

A choreographic operation type describes an entry into a distributed protocol and the valid continuations from that entry.

It may include:

- ordinary value schemas;
- evolving values and their basis;
- roles participating in the operation;
- principal and authority requirements;
- facts known by each role on entry;
- provenance justifying those facts;
- facts on which roles agree;
- resources required by operations;
- facts and values produced;
- possible outcomes;
- provisional execution permissions;
- authority transitions;
- continuation requirements;
- compatibility/version requirements where a suspended projection may outlive a deployment.

The operation type should **not** duplicate the complete choreography by manually listing every branch already represented by the choreography.

The choreography is the source of truth for control flow.

The type declaration supplies external semantic commitments that cannot or should not be inferred, while analysis derives as much as possible from the actual choreography.

An admitted operation type therefore joins several different judgments, rather than representing one large schema:

- **value and presence**: input/output shapes, required versus optional values, and the distinction between absence, nil, and a present false;
- **knowledge and provenance**: which role can justify using each fact at entry and at each use, not merely which keys appear in the global store;
- **authority and resources**: the bound principal and public model-operation interface, plus the resource and ownership permissions needed for effects;
- **control and settlement**: permitted semantic results, role-local continuations, and required recovery when an outcome remains uncertain;
- **compatibility**: which source contract, projection, artifact, and version conditions allow an already-started execution to continue.

The choreography body supplies derivable branches and value flow. The public model operation owns its dynamic guard and authoritative effects. An external operation type may import the model's exposed contract, but may not fabricate proof of source Clojure predicates, transaction-current enforcement, or deployed handler correspondence from a matching operation keyword. A checked judgment identifies the exact supported language and which obligations were proved, runtime-enforced, asserted, or remain open.

For example, the declaration may state:

```
{::choreo/roles
 {:helper
  {:principal :user
   :actor :client}

  :request-authority
  {:authority :request}}

 ::choreo/inputs
 {:request-id
  {:schema :uuid
   :known-by #{:helper}
   :provenance :input}

  :observed-basis
  {:schema ObservedBasisClaim
   :known-by #{:helper}
   :provenance :input}}

 ::choreo/authority
 {:request-authority #{:request/claim}}

 ::choreo/optimism
 {:role :helper
  :projection :request/claim}}
```

This example declares role requirements, input knowledge, and the intended optimistic realization. It does not grant claim authority to whoever sends a matching role label. Trusted server admission must establish the concrete principal, subject, and operation premises. The helper's input knowledge of an `ObservedBasisClaim` is knowledge of the claimed token, not evidence that the helper acquired the observation it names. A schema check on that token can establish its shape, not its issuer, observation coverage, progression, or permission to use it as a trusted basis. If a later boundary needs an acquired `:request/card` observation for optimism, a browser-submitted token alone cannot satisfy that acquisition premise.

The names `ObservedBasisClaim` and `AuthoritativeBasis` are conceptual semantic categories. They do not require two unrelated physical token formats, nor do they require a particular Clojure nominal-type implementation. The browser may carry the same underlying XTDB token bytes that trusted code later compares or uses to construct a consistency requirement, while the semantic status of those values remains different.

XTDB-specific representation belongs in the XTDB/Live consistency adapter rather than being permanently baked into the generic Choreo type vocabulary.

The verifier derives knowledge changes, branch requirements, and continuation structure from the choreography body. Where a claim depends on an external model guard, opaque derivation, registered runtime producer, or physical effect, the checker must retain the corresponding obligation rather than inventing a derivation from control-flow structure.

Operation type admission counterexamples

Apparent well-typed input	Required classification
The browser supplies a valid UUID-shaped principal or command identifier	Shape is not authenticated princ
The browser supplies a well-formed observed-basis token	An input claim is not a qualifie
A role branches on a declared fact available only on another role's path	Reject unsupported role-local u
A projected outcome is `:accepted` while authoritative incorporation remains `:commit-unknown`	Preserve the unknown comman
A valid operation schema refers to an absent resource or retired DOM owner	A type-shape check does not au
A suspended execution resumes against a changed contract with the same operation keyword	Require admitted contract/artifa
A typed model operation declares a business invariant but its executable guard is opaque or uninspected	Retain an implementation/polic

A report should identify the failing boundary or open premise: malformed value, missing role knowledge, unauthenticated provenance, unauthorized operation, unsupported continuation, or unqualified physical realization. These categories must not collapse into one successful ”typed” verdict. Finite projection or typed-rule checks may discharge their named properties without being promoted into an end-to-end physical certification.

The exact Clojure syntax remains tentative. These semantics are the intended contract.

Operation identity is semantic, not transport identity

A choreographic operation has a stable semantic identity that is distinct from the mechanism used to invoke it.

For example:

```
:request/claim
```

is not synonymous with:

```
POST /requests/123/claim
```

nor with:

```
button id="claim-request"
```

nor with:

```
browser plan index 17
```

The transport, view, and executable-plan coordinates are realizations of the semantic operation. They may change across refactors or deployments without changing the operation's meaning, provided compatibility rules permit the change.

This distinction is required so that application closure can later verify correspondence instead of mistaking physical equality for semantic identity.

A stable operation name is also distinct from the normalized semantic identity of its contract. Keeping `:request/claim` unchanged while changing a relied-upon authority rule, input requirement, outcome, or postcondition does not preserve dependent validity merely because the name stayed the same. Conversely, a transport refactor need not change the semantic contract if its checked realization and compatibility obligations remain satisfied.

These identities have different continuity rules. The semantic operation name identifies the intended capability; the source-owned contract and its applicable version/fingerprint bind the dependencies on which a particular use relies. A `CommandId` denotes one bound intention across compatible retries, whereas an `ExecutionId` denotes one execution attempt, and a role, principal, actor, authority, and host each answer different questions. An executable-plan digest identifies exact artifact content, not the model rule it invokes or the deployment that actually installed it. The compatibility checker must compare the relevant **source contract, admitted realization and required continuation**, not merely equality of any one of these names or hashes.

A fingerprint or digest can support exact content comparison only within its specified canonicalization and source-binding contract. It does not establish semantic correctness, issuer authenticity, model authorization, or physical deployment adoption by itself. Keeping a command identifier stable while changing its principal, operation, arguments, authority scope or incompatible contract is not a compatible replay just because the raw identifier matches.

Semantic categories at operation boundaries

Operation APIs must preserve distinctions whose accidental substitution would violate a semantic invariant even when the underlying Clojure representations are identical or structurally compatible.

Representative distinctions include:

- `ObservedBasisClaim` versus trusted `AuthoritativeBasis`;
- semantic `CommandId` versus runtime `ExecutionId`;
- provisional versus authoritative observations;
- raw declarations versus checker-admitted declarations;
- raw certificate data versus checker-accepted certificate evidence.

The requirement is semantic non-interchangeability through the supported API. The design does not mandate that every such category be implemented as a `defrecord`, Java class, wrapper object, or other nominal runtime type. Permitted realizations include:

- nominal or opaque host-language values;
- smart constructors with inaccessible or conventionally private construction;
- checker-produced admitted values;
- explicit tagged data whose use is guarded by the appropriate admission path;
- another admitted representation that prevents accidental semantic-category substitution.

The choice should be the smallest mechanism that makes the wrong ordinary call difficult and the resulting diagnostic precise. A representation technique is not valuable merely because it looks more typed.

Ordinary downstream application code should normally author domain values and semantic declarations, not manually wrap every UUID, basis token, command, or projection in framework ceremony. Gesso should perform category establishment at elaboration, checking, authoritative acquisition, model, or runtime boundaries wherever it can derive the distinction itself.

For example, a useful public failure is conceptually:

```
boundary:               trusted claim preparation requiring an acquired basis
requires:               AuthoritativeBasis for the declared scope
received:               unadmitted ObservedBasisClaim
missing step:           trusted authoritative acquisition / admission
```

rather than accepting the same-shaped value and relying on a comment to explain why the caller should not have done so. The diagnostic applies at the boundary that requires the acquired basis. A command intake may legitimately accept an `ObservedBasisClaim` as weaker context; it must not demand that the browser manufacture server-trusted authority before submitting a command.

The category and admission rules are defined in Part II. When the supplied category cannot be established, diagnostics report an unknown or unadmitted value rather than trusting a client-supplied category label.

A nominal host-language value also does not retain trusted status merely because it was serialized and reconstructed. Crossing an adversarial wire boundary returns the value to an untrusted representation until the relevant verifier, authenticator, decoder, or trusted acquisition path re-establishes the stronger semantic category.

Category establishment is scoped to the evidence issuer, subject, operation, time/context, and consumer boundary that rely on it. For example, a cooperating JVM may hold a checker-issued, identity-bound structural admission handle; a copied description or a deserialized lookalike is neither that live handle nor proof of a theorem. Even the live structural handle attests only its named structural check, not production authorization. Likewise, a shape-valid tagged principal returned from an untrusted wire must be authenticated again, and an apparently valid authoritative progression witness requires the trusted authority's actual ordering judgment. API ergonomics should hide this work from ordinary application authors where Gesso can perform it automatically, without turning a raw or stale value into a trusted one.

Identity and admission counterexamples

Apparent correspondence	Required treatment
Two calls use the same `:request/claim` keyword after the model policy changed	Compare the relied-upon semanti
A retry keeps its `CommandId` but changes arguments or the bound principal	Reject incompatible binding; the i
An `ExecutionId` matches a command's raw UUID, or a host has the same name as an authority	Keep the categories distinct; neith
A browser sends a tagged `Principal` or an `AuthoritativeBasis` looking map	Validate shape separately from tru
A serialized copy of a structural checker handle or report carries the expected fingerprint	Do not grant live admission, theo
Two executable plans share an admitted content digest but one host runs an unverified handler	Exact artifact content is not instal
A once-qualified resource or basis witness is reused for another owner, scope, or authority incarnation	Require requalification under the

These cases distinguish structural mismatch, incompatible contract, missing trusted origin, stale scope, and unproved physical implementation. The end-state authoring API should report the missing establishment step and owning boundary, not force developers to fabricate wrapper values or allow an untrusted label to satisfy the requirement.

This is primarily an accidental-misuse boundary inside cooperating application code. It does not claim that deliberately hostile trusted host code is unable to bypass Gesso, import lower-level implementation namespaces, or directly use a database capability it already possesses.

Operation contracts export a semantic interface

An operation type should expose a normalized semantic interface containing only facts that downstream reasoning is allowed to rely on.

Depending on the operation, that interface may include:

- input value types;
- required entry knowledge;
- required provenance;
- role participation;
- principal/authority constraints;
- model operation identity;
- possible semantic outcomes;
- values/facts produced;
- authority changes;
- resource requirements;
- provisional permissions;
- settlement requirements;
- continuation compatibility;
- named assumptions and guarantees.

It should not include arbitrary implementation details merely because those details are easy to serialize.

A supported exported interface must bind each relied-upon commitment to its source-owned contract and exact semantic subject, including applicable revision or compatibility conditions. It distinguishes derived choreography facts from imported public-model rules and from explicit trusted or opaque assumptions. A caller must not treat a source name, matching fingerprint, structurally valid summary, or an uninspected guard as proof of physical implementation or of the original policy. Derived facts may be used only within the verifier's actual supported language and property scope.

For Claim, the normalized interface identifies the helper's actual entry knowledge, the trusted principal/authority handoff, the public model operation, and its result contract. It separates four **boundary-facing command-resolution categories**. Their epistemic status and incorporation implications differ:

Category	Interface meaning
`:committed`	Qualified authoritative evidence that this exact bound command incorporated and has a recorded outcome
`:rejected`	A justified rejection of this command under the operation contract, rather than an inference from missing delive
`:already-incorporated`	Compatible repetition of the same bound command with a permitted recorded outcome; no new incorporation
`:commit-unknown`	This observer cannot establish the command's disposition from currently admissible evidence; **not** a fourth dura

The payload states the command binding, disclosed disposition, and any scoped observation or progression evidence. A successful callback or HTTP response is not sufficient to construct one of these outcomes. `:commit-unknown` denotes uncertainty of the observing boundary, not a third database truth between committed and uncommitted. It may arise from indeterminate delivery even when the authoritative transaction has a definite result.

`:confirmed` and `:reconciled` may describe protocol events if explicitly defined; they are not substitute names for the canonical command-resolution categories. The presentation mapping to `:accepted`, `:rejected`, and `:unknown` remains an explicit checked contract. A recorded result does not imply permission to disclose it. In particular, compatible `:already-incorporated` resolution requires the original command binding, the authorized recorded disposition, and current disclosure permission; it must not re-run the business operation or be rejected merely because the previously successful operation changed the Request's status. `:commit-unknown` requires command-specific reconciliation or another declared recovery path, not an arbitrary fresh command ID or a fabricated `:rejected` outcome.

The semantic interface does not encode incidental realization details such as:

```
specific Ring handler var
specific Reitit route index
specific DOM node id
specific XTDB Java class
specific browser bundle offset
```

Those belong to later realization layers unless independently required by runtime semantics.

External commitments versus derived structure

Operation declarations should state only semantic commitments that cannot or should not be safely inferred.

The verifier/projector should derive:

- reachable continuations;
- behavior-sensitive branch obligations;
- knowledge propagation caused by declared communication;
- terminal paths;
- local plan structure;
- resource obligations that follow from the body.

It should not require the author to restate those facts manually in metadata.

Redundant declarations are dangerous because each copy may remain individually plausible while disagreeing with the others.

The preferred rule is:

> Declare external semantic commitments; derive internal control-flow facts from the choreography body.

For each exported fact, a checked interface should distinguish:

- `:derived`: obtained from the exact admitted choreography and the named structural, knowledge or projection check actually performed;
- `:imported`: supplied by the specified public-model or authority contract, retaining its source binding and any uninspected predicates;
- `:runtime-required`: requiring a concrete authenticated principal, current guarded model transition, actual message receipt, resource ownership or qualified authoritative acquisition;
- `:assumed/open`: a typed external premise or unsupported proof/physical correspondence, still visible to every downstream consumer that relies on it.

These are specification-level evidence classes, not a claim that current Gesso already exposes this exact record format. No category grants trust by being written into caller-controlled metadata. Where a composition includes several operations, each imported guard, invariant premise and read/write footprint remains attributable to its original source; possible shared state interference is not evidence that transitions commute or execute atomically together.

Exported-interface counterexamples

Apparent interface success	Required judgment
A role-local verifier proves that a branch selector is definitely available	Preserve that specific knowledge/availability r
A successful HTTP callback says Claim was accepted	Require command-bound authoritative disposi
A duplicate Claim presents the same command ID but a different principal or arguments	Reject incompatible binding rather than treatir
A prior successful command is replayed after its Request changed state	Resolve from a compatible recorded command
A network timeout has no readable command record	Retain `:commit-unknown` or a qualified recovery
Two operation interfaces have compatible types and overlapping read/write sets	Preserve interference and open concurrency as
A report repeats a model rule or invariant name but has not inspected its implementation	Retain the original source-owned opaque/phys

The interface must remain suitable for independently checked composition without converting unknown provenance, an assumed predicate, or a physical deployment gap into a guarantee. Downstream diagnostics should identify which specific imported or derived claim cannot be established.

Availability is not authorization

An operation may be present in a model or choreography registry without being currently executable by a particular principal.

Likewise, a prepared or bound operation may be rendered as an affordance without becoming durable authorization.

These distinctions must remain explicit:

Established fact	What it does not establish by itself
Operation exists	Its use is valid at this choreography state
Use is structurally valid at this state	The current principal is authorized now
View renders an affordance	The browser has authority over protected state

The trusted side re-establishes dynamic authorization at execution and protects the relevant model premises at the authoritative transition boundary where intervening changes matter. A rendered affordance, a successful earlier check, and a valid operation contract do not reserve future eligibility.

This separation lets Gesso reason strongly about operation identity and distributed protocol structure without pretending to statically prove current business facts that can change between render and commit. A valid source-level operation interface also does not attest that its named handler, current authenticated principal, or transactional guard was actually installed and enforced. An admission diagnostic must identify whether it checked choreography structure, current model authorization, or the physical join; an unavailable independent premise is not a successful authorization.

Malformed operation type versus wrong business policy

The unified design distinguishes structural/type malformedness from product correctness.

If a declaration says that `:request/status` is an enum and a rule attempts:

```
(< (:request/status request) 10)
```

the expression is malformed relative to its semantic types and should be rejected.

If the application states:

```
(= (:request/status request) :open)
```

when product policy actually intended another valid status test, the expression may be perfectly well formed while encoding the wrong policy.

Gesso should make the former difficult or impossible to certify **within the supported typed rule language**. It cannot infer the latter merely from syntax. The two Clojure-shaped examples illustrate the semantic distinction; they do not imply that arbitrary host-language Clojure expressions are automatically translated into a closed proof language. The admitted typed rule language has explicit reader types and operators, so a numeric ordering comparison against a nominal enum fails admission under the corresponding type discipline. Arbitrary Clojure policy functions instead retain an opaque typed/effect contract unless their relevant source is elaborated into an inspected language or checked by another admitted mechanism.

This distinction becomes more explicit in the later compositional-proof part, where typed business-rule expressions and certified semantic interfaces are defined in detail. It is already a language-design constraint here because Choreo must not confuse ”type checks” with ”is the policy humans intended.”

A valid rule may still be inadmissible at a distributed program point

Intrinsic rule validity is not enough.

For example:

```
rule:
    (= (:request/private-review request) :approved)

intrinsic type:
    Boolean
    VALID

browser use before browser knows :request/private-review:
    INVALID AT THIS PROGRAM POINT
```

A distributed use of a rule therefore creates obligations beyond ordinary type checking, including:

- the role knows every required fact;
- required provenance is available;
- trust or authority requirements are satisfied;
- optional values have been refined before use;
- authoritative-basis requirements are met where the rule depends on authoritative observation.

This is one of the central reasons Choreo needs a semantic type/knowledge layer richer than ordinary function typing. A global value store's containing a binding does not make it definitely available to the chosen role on every incoming path. Nor do a claimed authoritative basis and a correctly typed provenance record prove the observation was actually acquired or permits this use. A role may depend on an optional value only after its applicable presence refinement. A program point may therefore be rejected even when its predicate is intrinsically well typed and its runtime inputs would happen to be present in one favorable execution.

Operation declarations should remain inspectable data

The ordinary authoring surface may eventually include macros or helpers, but the semantic representation after elaboration should remain explicit enough to:

- inspect;
- normalize;
- verify;
- project;
- fingerprint semantically;
- explain on failure;
- compare against dependent declarations.

Convenient syntax must not create an opaque second semantics.

Normalization must preserve the information needed to identify the owning declaration and explain failed obligations without making source coordinates part of semantic identity when they are diagnostic-only. The compiler or elaborator must not synthesize an undeclared trusted assertion simply to make a knowledge, authority, or continuation obligation pass.

The desired direction is:

Author-facing syntax elaborates into explicit normalized semantic data. That same subject supports verification, projection, diagnostics, and certification; none may silently interpret a different policy hidden in the syntax layer. The checked normalization retains the owning source and the identity of the semantic contract, even if locations and pretty source snippets remain diagnostic-only rather than contributing to semantic identity. A generated plan, copied checker report, or friendly syntax cannot independently become an authoritative second definition of a business rule.

Operation-admission diagnostic classes

Classification	Evidence and response
`:invalid`	A contradiction is established in the admitted syntax, type, value-flow, role knowledge, or stated interface; reject at t
`:unsupported`	The construct is outside the checker's supported language or bounded capacity; do not report a successful check or s
`:open-premise`	A needed model/authority, external Clojure, runtime producer, or physical deployment correspondence was not estal
`:runtime-required`	A declared dynamic fact such as authenticated eligibility, current transaction state, or resource ownership must be es
`:runtime-rejected`	An actual guarded operation was rejected by its trusted boundary; do not reinterpret this as a static type failure
`:indeterminate`	Delivery or outcome evidence is insufficient; retain command-specific uncertainty and a permitted recovery path ins
`:policy-mismatch`	Product policy is allegedly wrong despite type-correct syntax; require a source-owned rule, product acceptance crite

These names describe required diagnostic distinctions; an API may expose them using an equivalent typed representation rather than these literal enum keywords. A useful report identifies the declaration and semantic subject, the role and program point where relevant, required versus available knowledge/provenance, the supported checker property, the failing or open premise, and the correct next construction, runtime, or product-acceptance boundary.

Admission counterexamples

Apparent success	Required outcome
A nominal status enum is passed to a numeric comparison in the admitted typed-rule AST	Reject the malformed type use, without claim
A well-typed predicate tests `:open` but product policy intended another permitted status	Type checking may succeed; the policy-inten
A browser branch reads a private fact known only to the server	Reject that role-local use; global value availab
An optional receive field was present on one path but omitted on another	Require path-specific presence refinement bef
An input claims an authoritative basis with the expected schema	Do not upgrade a claimed token to a trusted a
A valid choreography has a declared but physically uninstalled required command handler	Report the missing implementation/assembly
A command times out after being sent	Preserve the indeterminate outcome and reco

A failed supported check, an unsupported check, an open external obligation, and a genuine runtime rejection must remain separate. None can be laundered into success through a structural fingerprint, a synthesized assertion, or an exception path that swallows a checker limitation.

Full-language computations retain their contracts

An action reference binds a semantic interface and its concrete realization. The interface distinguishes the role that executes it, required scoped inputs, declared outputs, effects, failure/uncertainty alternatives, resource use, and applicable termination or progress assumptions. The implementation may be an ordinary Clojure function, a supported CLJC/CLJS realization, or a declared external effect. JVM code is not automatically executable in a browser; target placement and communication remain checked obligations.

For inspectable typed expressions, elaboration derives required inputs and produced bindings. For opaque code, the declared footprint and functional contract remain explicit premises unless another mechanism establishes them. The framework should derive ordinary plumbing while avoiding duplicate manual inventories; it must not claim to discover all effects of unrestricted Clojure.

The executable machine yields a declared effect request and later admits its completion or failure. It does not run arbitrary application functions while canonicalizing a declaration, checking its shape, or computing an artifact fingerprint. Callback execution belongs to an explicit realization boundary.

A realization can report completion only for the pending declared effect, through the machine's admitted event path and the applicable owner-relative execution or resource binding. It cannot directly edit a projected machine, select a different continuation, introduce undeclared outputs, or treat a previously retired callback as current. The receiver must enforce the operation's applicable output, identity, provenance and runtime guard contract; a correctly shaped callback map is not that enforcement. An uncertain physical effect retains its declared unknown/recovery alternative rather than becoming a fabricated failure or a second automatic invocation.

Validating an output and recording its dependency list does not prove that the implementation calculated the promised result. Runtime validation, instance observations, checked derivations, and asserted contracts retain the evidence classes established in Parts I and II. A named local result cannot become authoritative solely because the caller supplied an `:authoritative` label.

Nontermination, blocking, nondeterminism, and external effects are expressible but affect the available claims. If a required continuation depends on such an action finishing, the plan needs an applicable completion premise, enforceable bound, isolation/cancellation contract, or recovery path. A timeout callback in the same blocked event loop does not by itself enforce a bound on synchronous code. No universal totality proof is required for the host language.

Definite values, knowledge, and provenance are separate analyses

The abstract global value store describes values in the specification; it is not a shared runtime context and does not make those values known to every role. Entry contracts establish role-local facts with their actual provenance. A low-level test fixture that gives all roles the same input set is not the ordinary application entry policy.

The verifier computes at least these distinct judgments:

Analysis	Question
Value availability	Is the required binding defined on every applicable path?
Role-local knowledge	Does the consuming role have a permitted knowledge path for that binding?
Provenance and support	Do its subject, scope, observation, and retained premises justify this use?
Optional/presence refinement	Has this path established presence and the required variant before use?
Causal eligibility	Has the predecessor observation or result required by this continuation been established?

A forward must-analysis is a useful baseline. At a join, unconditional facts must be justified on every reachable incoming path. Each environment-event or receive alternative contributes only its own established fields. Optional fields are not definitely present merely because one alternative can supply them. Path-sensitive refinements may establish stronger facts on selected edges.

Agreement on a value at a join does not erase differing assumptions. The result retains the appropriate support for each path, or a conservative combined premise, without upgrading an asserted path to a proved one. Unreachable edges may be excluded only under the checker mechanism that establishes their unreachability; a branch being unlikely is insufficient.

Bindings have explicit version and scope semantics. An implementation may use fresh immutable binding identities or a checked replacement discipline. Reusing one context key for a new value invalidates support that depended on the old binding; it does not silently inherit the old value's authoritative status. Omitting an optional field on a later receive must not accidentally reuse an earlier message's value unless the contract deliberately preserves that binding. Equal status values for different Requests are still different scoped facts.

Required transmitted fields, and every optional field actually sent, must be known to the sender with the declared meaning. Static analysis establishes what it can; runtime construction checks the actual payload and remaining premises. An optional field is not an exception to knowledge or disclosure requirements.

Loops require analysis to a sound fixed point over the chosen abstract domain. A bounded analysis reports its bound and unresolved obligations; resource exhaustion is not successful verification. Graph reachability of a terminal establishes neither that all executions terminate nor that an environment event will occur. Recursion, parallel composition, or another extension must carry explicit semantics and a checking contract; unsupported syntax cannot become an unchecked alternate path. Full-language work remains available behind the declared action boundary without pretending the checker understands it all.

Executable plans and explanation artifacts

Projection emits one canonical executable plan per role and an optional explanation artifact tied to that exact plan. A plan may use compact, non-negative integer state locators. Those locators are local to that artifact, not semantic operation identities and not stable across rebuilds.

The executable plan retains every runtime-semantic guard, contract reference, binding, correlation requirement, event distinction, and outcome needed to run the admitted transition. Diagnostic source coordinates and proof derivations may live in a sidecar. Stripping a field merely because it was introduced by verification is invalid if runtime execution still needs its meaning. In particular, a guard on a delayed effect or message completion remains a runtime admission obligation even when the proof explaining that guard is compiler-only. Conversely, a diagnostic sidecar must not be read by the production machine as a source of new permissions, facts, or continuations.

The exact plan identity binds the versioned canonical representation and all relevant contract identities. The explanation artifact maps exact emitted coordinates to source states and judgments under that identity. A compatible but different plan cannot inherit an old coordinate map as though it were exact. Compatibility for continuation remains a separate judgment.

A plan-shaped map, a current version tag, and a dense locator range establish only their checked representation properties. They do not establish projection correctness, provenance, authorization, or host realization. Admission binds the generated artifact to the verified subject, selected assurance policy, admitted assembly, and concrete runtime contracts needed for the claimed use.

Acceptance counterexamples for full-language boundaries

These examples state construction-checking obligations under the selected supported assurance profile. Unrestricted Clojure does not become fully proved merely because an opaque action has a declared type or effect contract:

Apparent shortcut	Required chec
A test harness supplies every entry value to every role, and the browser later uses a server-only input	Test-fixture co
A receive branch omits optional `:approval` but later code reuses a value left under that key by an earlier receive	Require definit
Two Requests have equal `:status`, and a continuation borrows the first Request's authoritative provenance for the second	Reject unsupp
A sender adds an optional message field it does not know, although the payload schema accepts that field	Reject the actu
A host callback writes the projected machine's `:state` or jumps to a success continuation without a declared completion	Reject the alte
A late callback from a retired effect instance returns a shape-valid completion	Apply its decla
A transport timeout follows an authoritative effect that may already have committed	Preserve indet
A loop has an apparent terminal edge but no justified termination or progress assumption	Do not claim l
A plan-shaped map is paired with a certificate or coordinate map from another exact artifact	Reject the mis

Successful admission must preserve useful ordinary Clojure effects behind declared contracts, while requiring independent evidence for guarantees the chosen checker and physical realization do not establish.

Malli as the value-schema substrate

Malli provides the ordinary value-shape layer underneath choreographic types. Its supported schemas describe data and runtime value obligations; the model and Choreo remain the owners of business meaning, knowledge, authority, and legal transitions. A schema reference is not a separate semantic registry that can redefine the owning operation's contract.

A choreographic value may conceptually carry:

```
{:schema ClaimCommand
 :known-by #{:helper :request-authority}
 :agreement #{#{:helper :request-authority}}
 :provenance :communicated}
```

or:

```
{:schema RequestProjection
 :known-by #{:helper}
 :provenance :authoritative
 :basis AuthoritativeBasis}
```

These are conceptual interface sketches, not self-authenticating runtime envelopes or necessarily literal Malli schema expressions. The schema references and the `:known-by`, `:agreement`, `:provenance`, and `:basis` claims have different semantic owners. Malli can check their declared shape; trusted admission, derivation, communication, authoritative acquisition, and agreement must establish the corresponding claims. Copying the map, adding a tag, or passing Malli validation cannot upgrade any of them.

Malli is responsible for questions such as:

- is this a UUID?
- does this map contain the required keys?
- is this settlement one of the allowed values?
- does this intermediate machine context have the promised shape?

Malli is not responsible for questions such as:

- did this value come from a trusted actor rather than merely having a valid shape?
- may this helper claim this Request?
- is this Organization currently open?
- does this principal have authority?
- is this stale basis still semantically compatible with the operation?
- did communication actually establish knowledge?
- did an authoritative observation justify the actor's knowledge?
- does this branch satisfy the global choreography?

Those remain domain, FX, Choreo, or authority/progression concerns.

Runtime validators are compiled and reused for the admitted schema and effective registry definitions whose identities they reference. Their applicability includes the relevant schema form, registered references, closedness and refinement options, custom predicate contracts, and target runtime semantics when these affect validation. A stable schema name, an unchanged source filename, or a cached compiled function is not sufficient evidence that the same contract remains in force.

If those bindings belong to an admitted assembly generation, validator use remains coherent with that generation. A changed schema, registry entry, extension, or predicate realization requires an appropriately recompiled or independently established compatible validator before relying on it. A canonical description may name and compare known semantic dependencies; it does not pretend that arbitrary host-language functions can be proven equivalent by hashing their names or serializing them. JVM/CLJS or other runtime realizations may need separately qualified predicate semantics. Validator correspondence does not depend on a browser execution identity or resource-instance counter.

The admitted application validates important runtime boundaries in production, including:

- externally supplied choreography inputs;
- message payloads;
- authoritative observation/projection envelopes where they cross a runtime boundary;
- FX input/output boundaries;
- projected-machine state transitions;
- settlements;
- provisional/canonical protocol payloads.

The successful path uses compiled boolean validators where applicable. Rich Malli explanations are constructed on failure or explicit diagnostic request.

These validations remain enabled in production. Optimizations based on measurement may remove redundant work only where an equivalent check has already been established at a trusted admission boundary and the binding, value, schema contract, and consumer scope remain valid. No optimization may drop a required check where untrusted or dynamic values can enter.

Schema-boundary acceptance cases

Tempting shortcut	Required behavior
A browser sends a shape-valid principal and authoritative-basis map	Admit the shape only; authentication, obs
Two schema references have the same name but different registered enum sets	Use the admitted effective registry for the
A custom predicate changes behavior while retaining its registered keyword	Preserve a changed or unresolved predica
A JVM predicate is unavailable or behaves differently in CLJS	Require a supported target realization or
A closed command has an extra field that later code would interpret	Reject the undeclared semantic field or ap
A value was validated under an old assembly but enters a different operation after deployment	Revalidate under, or establish compatible
An implementation disables a required boundary validator after a benchmark	Preserve an equivalent trusted check and

These are finished-system acceptance obligations. They do not claim that plain Malli validation itself certifies source-to-runtime correspondence, trust, or the correctness of arbitrary custom predicates.

Shape validation is necessary but not semantic certification

Successful runtime validation establishes conformance to the particular schema under the validator and predicate contracts being relied upon. It is classified as runtime enforcement, not automatically as a formal theorem. A custom predicate's name does not establish an authority or provenance claim that its implementation never checks. Custom predicates are executable code: their determinism, side effects, exception behavior, and target portability are part of their contract when relied upon. A schema used for formal reasoning must have a supported semantic interpretation or explicit premises; serializing a function reference does not give the prover that interpretation.

It does not prove:

- that a browser supplied the value honestly;
- that the value has authoritative provenance;
- that the current principal may use it;
- that another actor knows the same value;
- that a model operation may be invoked;
- that a choreography branch is globally legal.

This is not a limitation to work around. It is a deliberate separation of correctness scopes.

The unified design therefore treats ordinary value schemas as one layer in a larger semantic interface:

Layer	Separate obligation
Value shape	The actual value satisfies its admitted schema
Semantic type/refinement	Its category and applicable refinements are established
Knowledge/provenance	This role has the required scoped justification
Authority/operation contract	The trusted boundary permits this operation under current premises
Choreographic admissibility	This local action and continuation obey the distributed specification

These are independent, composable judgments, not a mandatory sequence of runtime calls or a ladder in which passing one automatically proves the next. A supported static theorem about a schema may discharge an identified shape-related premise; a runtime validator enforces a shape-related premise for a particular admitted value. Neither conclusion entails the other semantic judgments. Each stronger claim needs its own applicable construction, proof, trusted admission, or explicit open premise.

A success at one layer cannot be silently promoted into a stronger proposition owned by a later layer.

The same rule applies when a stronger semantic category happens to use a nominal host-language representation. Malli may validate the fields inside an `AuthoritativeBasis` representation, for example, but shape validation does not establish that the value came through the trusted authoritative acquisition path. Conversely, an admitted value may still use ordinary immutable Clojure data internally. What matters is the checked construction/admission relation, not whether the physical carrier is a record or a map.

A discriminator such as `:type :verified` is therefore useful only as data unless the supported API also establishes why callers are entitled to rely on that category. A trusted semantic category is established by an admitted producer, derivation or authority-bound construction and is usable only under the subject, scope, issuer, version and consumer constraints of that judgment. A caller-created lookalike must not be promoted into the same judgment by deserialization, rewrapping or matching a registry key. This does not require a special physical wrapper for every value: ordinary maps remain valid carriers when the trusted construction and evidence remain enforceable.

Closed value boundaries are the default

Where data crosses a semantic boundary, the declared schema normally describes a closed payload. Closedness is an admission property of the effective boundary contract, not a comment attached to an example map. It applies to the semantic fields that the receiver is permitted to interpret, including nested declared structures where their contract is closed. The source-owned schema and its admitted version, optional fields, variants and extension points determine the accepted semantic vocabulary; the sender may not select a weaker schema by supplying its name.

Closedness is especially important for:

- client commands;
- choreography messages;
- provisional-state envelopes;
- settlement envelopes;
- authoritative projection metadata;
- FX inputs and outputs.

An open map is not harmless merely because consumers ignore unknown keys. Unexpected fields can become an accidental covert semantic channel when later code begins to read them.

When an application genuinely needs an open payload, that must be explicit for the particular boundary, and its semantic treatment must be visible to verification and diagnostics. Merely permitting extra physical fields does not extend the machine's semantic payload or give a receiver new knowledge. An explicitly open transport envelope may retain undeclared adapter data without entering it into role-local fact storage, deterministic semantic history, branch decisions, operation arguments, authorization, semantic correlation, or proof conclusions.

If an extension field is to acquire any such semantic role, its source-owned contract, required shape, knowledge/provenance path, disclosure restrictions, and compatibility with existing consumers must be admitted. A planned extension cannot retroactively reinterpret unknown bytes under an old closed semantic contract. Physical routing or telemetry may use separate admitted transport metadata rules; those rules do not grant domain authority.

Semantic payload closure is distinct from rejecting every byte of physical transport metadata. The decoder separates declared semantic payload from permitted transport metadata through an explicit, qualified contract. The consumer enforces the closedness of the **decoded semantic fields**, not merely the spelling of their wire representation. Decoding and normalization must not allow incompatible encodings, duplicate physical fields, alias collisions or coercions to create an ambiguous or attacker-chosen semantic binding; an ambiguity affecting the admitted contract is rejected rather than silently resolved into new authority.

At a closed semantic boundary, undeclared semantic fields are rejected. At an explicitly open physical boundary, permitted undeclared data stays outside semantic context unless separately admitted. This applies whether the data arrives through an HTTP command, participant message, event, FX callback, authoritative projection, settlement or reused artifact. Both admission and later consumers preserve the same division; a generic map merge, cache retrieval or serialization round trip cannot silently make excluded fields usable semantic inputs.

Shape-and-closure acceptance counterexamples

Apparent success	Required admission or re
A browser posts `{:type :verified :basis B}` matching a registered Malli schema	Validate its shape if appli
A different principal copies another actor's correctly shaped command envelope	Reject unauthorized use
A receive contract is closed, but the sender adds an undeclared `:admin?` field	Reject the semantic extra
A message expressly allows open physical data with an undeclared `:hint`	Preserve the allowed ada
A newer consumer starts branching on the old open `:hint` field without changing its admitted interface	Require a new checked s
A wire decoder maps both `"request-id"` and `:request/id` into one semantic key with conflicting values	Reject an ambiguous dec
An untrusted command includes a schema/registry selector that relaxes required or closed fields	Enforce the receiver's ad
A value passes Malli validation, but a custom predicate has no supported semantics for the chosen proof profile	Retain the predicate's pre
A shared cache or callback returns a map containing valid declared fields plus stale foreign-scope metadata	Recheck the consuming s

These cases constrain the finished system without prescribing a single transport decoder, container representation, or proof backend.

Runtime validation preserves static premises

Static reasoning depends on runtime values continuing to satisfy the contracts under which the reasoning was performed.

Consequently, runtime checks are not ”extra tests” after proof. They enforce the assumptions on which static reasoning depends wherever untrusted or dynamic values enter. The admitted checker identifies which premises require runtime enforcement; the physical application binds those checks to its actual input, effect, acquisition, and continuation paths. A validator invoked in an unrelated test, omitted route, or obsolete artifact does not discharge that deployed boundary obligation.

Representative boundaries include:

Runtime input	Required admission before use
Browser command bytes	Bound operation lookup, decoding/schema checks, command/caller binding, and trusted invocation contract
External read result	Projection-envelope validation; authorized acquisition, subject/scope/basis/provenance admission; then only th
FX implementation result	Declared output validation, effect/owner correlation, and admission of the next semantic step under its current
Settlement payload	Schema, trusted source, command binding, current owner and disposition-to-settlement checks before the adm

A successful shape check is not itself authorization, authoritative observation, sender knowledge, current resource ownership, or permission to install a browser result. The receiver establishes the separate semantic premises applicable to the particular operation, execution and consumer. Where an earlier trusted admission is reused, its subject, version, scope, lifetime and consuming boundary must still apply. The same rule covers cache hits, deferred effects, and cross-deployment requests.

If a runtime premise fails, execution rejects, defers or recovers according to the declared protocol without manufacturing a terminal business result. It must not continue on the theory that static verification made runtime validation unnecessary.

Validators should be reusable and diagnostic failure should be rich

The successful path should use precompiled validators where possible.

The failure path should preserve enough information to explain:

- which semantic value failed;
- which schema or refinement was expected;
- where the value entered;
- which operation or transition depended on it;
- whether failure concerns shape, provenance, authority, or another layer.

A raw Malli explanation may contribute to internal diagnostic evidence, but downstream error reporting must not force users to reconstruct semantic context from a low-level schema mismatch alone. Diagnostic construction, exception metadata, persistence, logging, traces, and responses must each respect the applicable disclosure boundary: credentials, raw command data, private payload fields, and protected model values cannot leak merely because validation failed. A low-level exception containing a full rejected value is not automatically safe to log or return. Stable semantic identifiers, authorized/redacted value summaries, checker/issuer scope, and the failed obligation are normally sufficient. Diagnostic denial must not itself be misreported as a rejected authoritative operation.

Schema changes participate in semantic compatibility

A schema is not merely runtime validation configuration when another semantic artifact relies upon its meaning.

Changes such as:

- removing an enum member;
- changing required versus optional;
- changing cardinality;
- changing a field's nominal semantic type;
- changing a closed variant set;

may invalidate dependent operation, rule, or choreography artifacts.

The canonical interfaces therefore have semantic identities independent of source filenames. Dependency analysis tracks the effective schema and registry, nominal/refinement meanings, closedness and extensions, runtime predicate contracts, and the actual consumers that rely upon them. Changes invalidate or require new correspondence evidence for affected validators, type judgments, operations, projected plans, browser artifacts, application assemblies and assurance products. A matching name, digest label, file path, or well-formed version tag alone cannot discharge that dependency.

Compatibility is directional and consumer-relative. An old producer accepted by a new reader, a new producer consumed by an old reader, continuation of an in-flight execution, and reuse of a proof may have different requirements. A schema widening or narrowing can affect required fields, accepted variants, trusted refinements, disclosure rules, and downstream branch meaning even when every observed test value still passes both validators. An admitted compatibility relation must identify the source and target interfaces, the relevant consuming semantics, and the qualification evidence; otherwise the dependent claim is invalidated or remains explicitly unresolved.

These obligations apply independently of when the proof machinery is implemented. The language/model boundary exposes semantic dependencies rather than hiding them inside unrestricted host-language callbacks.

Runtime-premise and compatibility counterexamples

Apparent success	Required outcome
A schema-valid browser command arrives on an unregistered or bypassing route	Reject or report the missing install
An authorized board read passes its schema but belongs to a previous principal or Request scope	Do not establish current authoritat
An FX completion matches its output schema after its resource instance was retired	Reject the stale completion or appl
A valid-looking settlement reports `:rejected` after the command may already have committed	Preserve unresolved incorporation
A failure diagnostic includes a protected field in an exception map destined for logs	Redact or withhold at the relevant
A new schema admits an extra enum variant used by a branch in an older executable plan	Check the actual direction and con
A predicate implementation changes but the schema identifier and file name remain unchanged	Requalify the predicate contract an
A compiled validator and proof sidecar come from different application generations	Reject mismatched dependency ap
An older writer emits a closed payload accepted by a newer reader, but new business guards changed	Shape compatibility may hold whil

These cases are part of the finished-system acceptance contract. They do not prescribe one implementation of the validator cache, proof backend, or deployment migration mechanism.

Projection and branch knowledge

Projection creates a local machine for each role.

Projection must preserve whatever mechanism is required to justify local behavior.

Sometimes that mechanism is communication.

Sometimes the actor already has justified entry knowledge.

Sometimes the relevant fact is learned from authority outside the current execution and enters a later choreography as authoritative input knowledge.

The projector/verifier must therefore reason in terms of knowledge requirements, not merely message presence. For each local use, the obligation identifies the role, the scoped fact or proposition, the required provenance and authority, and the program point before which the fact must be available. A globally defined value or a proof that another actor possesses it cannot serve as this role's local knowledge. An entry assumption is usable only under its declared role, source, scope and admission premises.

A role may branch only when it knows enough information to choose the branch.

The verifier should use behavior-sensitive choice analysis:

- if a role behaves differently between two branches, it must know the discriminator or an equivalent fact before its behavior diverges;
- the required local evidence may be supplied through declared communication, authorized authoritative observation, role-specific entry knowledge, supported derivation, or an explicit assertion within its admitted contract; assertion establishes that the source asserted a proposition, not the proposition's independent truth or authority;
- if its behavior is equivalent across the branches, it does not need to learn the choice merely because it is syntactically mentioned.

This is intentionally more precise than a simple ”every role mentioned in either branch must know the condition” rule.

For example:

If authority chooses an approved or rejected continuation and the helper's behavior differs, the helper needs evidence distinguishing those continuations before it performs that differing behavior.

That requirement does not imply a dedicated authority-to-helper choreography message if the helper can instead obtain the required approval fact through a declared, authorized authoritative observation that satisfies the same branch obligation.

An alternate knowledge path is valid only when it establishes the **specific proposition required by this continuation**, with its subject, scope, provenance, causal predecessor, and temporal/authority qualification. Reading a currently approved Request does not necessarily establish which earlier command was approved or which historical branch was selected. If later model changes can erase or supersede the needed fact, the protocol needs a suitable retained outcome, receipt, communication, or different continuation contract.

Knowledge-path equivalence is not automatically protocol equivalence. A required global communication cannot be silently replaced by a reread simply because both yield equal values: the replacement must be admitted by the authored choreography or by a proved, contract-preserving refinement that accounts for observable communication, correlation, participant waiting, control flow, failure and progress obligations. The projector must neither invent missing history nor add a different physical action to rescue an otherwise inadmissible declaration.

Messages should be closed by default.

A message contract declares the fields that may cross the boundary. Arbitrary extra fields should not be merged into choreography context.

An explicit `:open-payload? true` may exist for cases that genuinely require an open payload, but closed contracts are the default.

Within the admitted choreography semantics, a sender must have justified knowledge of every semantic fact it transmits. Receipt from an adversarial participant still requires the trusted-boundary treatment defined in the knowledge model.

Receiving a message establishes only the declared communicated values after the message's required source, channel, correlation, payload and owner-relative admission. The same-shaped bytes from a different sender, scope or execution do not establish the original sender's proposition. Required and actually transmitted optional semantic fields remain subject to sender knowledge and disclosure checks; undeclared fields in explicitly open physical payloads do not become semantic knowledge.

Projection must not erase communication that is genuinely required to justify knowledge.

Conversely, projection should not manufacture communication merely because the global choreography contains a choice when the target role's behavior does not depend on it or its knowledge is established through another valid path.

Projection is a semantic compilation boundary

Projection is not a convenience transformation that happens to produce smaller maps.

It is the boundary at which the global specification is turned into the local information and actions available to a role.

A correct projector must therefore preserve every fact required to justify local behavior, including when relevant:

- message schemas;
- correlation identity;
- entry knowledge;
- knowledge-producing communication;
- authoritative-observation requirements;
- branch discriminators;
- provenance requirements;
- resource ownership;
- failure/terminal structure;
- executable guards required by local semantics;
- the causal prerequisites that distinguish local authorization to act from mere presence of an identically shaped fact;
- scope and disclosure restrictions on what the role may actually receive;
- explicit alternatives for unavailable communication or observation, when the global contract requires recoverable continuation.

At the same time, it omits global facts the role neither knows nor needs. A globally available private discriminator is not permitted in a local artifact merely because it would make branch selection convenient. Preserving global semantics does not authorize revealing private premises or relying on information that the recipient is not allowed to acquire.

The target is neither ”copy the whole global machine to everyone” nor ”erase everything inconvenient.” It is a justified local program.

Projection cannot create knowledge

A projector may rearrange or encode information justified by the global choreography. It may not manufacture a fact merely because the local machine would be easier to implement if the fact were present.

Bad:

Only authority knows `:approved?`, but the browser projection contains it without any admitted input, communication, or observation. This projection manufactures knowledge and must be rejected.

Good:

Authority communicates the permitted command outcome, or the browser obtains an authorized observation establishing the required proposition. The local branch becomes admissible after that declared knowledge path succeeds.

The same principle applies to provenance. Projected data cannot silently upgrade:

Supplied support	Unjustified upgrade
Asserted premise	Mechanically derived proposition
Communicated value	Authoritative truth without a supporting authority contract
Client-supplied principal id	Trusted authenticated principal binding

Projection preserves or weakens claims according to explicit semantics; it does not strengthen them by serialization. Distinguish a valid local use of an **asserted** fact from a claim of authoritative truth, a known value from a definitely present optional binding, and a current observation from a retained command-specific decision.

Knowledge-preservation acceptance counterexamples

Apparent local justification	Required checker outc
The global state contains `:approved?`, but the browser has no entry fact, receive or observation path	Reject the local use; g
The server asserts that a Request is approved, and projection marks it authoritative	Preserve asserted prov
The helper sees the Request's current `:approved` status and resumes an earlier command-specific approved branch	Require a retained res
The helper reads a result for Request A but continues Request B, whose status value is equal	Reject subject/scope s
A participant sends an optional `:review/result` field without having established its value	Reject the actual trans
A received message carries the right payload from an unbound sender or old execution	Do not establish know
A compiler replaces a required global notification with an authoritative reread that yields the same status	Require an authored a
The browser knows a private discriminator only because projection copied the authority's local state	Reject the projection;
Two local branches look identical after ignoring an effect or timeout outcome	Do not erase the disti

These are finished-system obligations. Their enforcement may be distributed among construction-time knowledge verification, refinement checking, and trusted runtime admission; no one layer may silently assume the others have already discharged its premises.

Behavior-sensitive choice is the central branch rule

A role needs evidence sufficient to select its continuation when its observable local behavior depends on a choice. The required knowledge must exist before the first differing behavior, not merely become available eventually.

This avoids both extremes:

- projecting too little and leaving a role unable to decide its next behavior;
- overcommunicating every global discriminator to every mentioned role.

The analysis should compare continuations semantically rather than merely by source syntax.

If two global branches project to equivalent local behavior for role R, R need not learn the branch solely because R appears somewhere inside both branches.

If they diverge, the projector/verifier must establish a valid knowledge path before the divergence. Whether behavior differs is judged against the declared local semantic observations and obligations, including waiting, effects, outcomes, correlation, resources and relevant causal order. Equal final values or superficially equal message maps are not sufficient. A branch choice may remain private only when the projection also respects its disclosure and information-flow contract; equivalence under a coarse trace alphabet does not alone prove confidentiality.

A focused expense example separates private choice from public behavior. Suppose an accepted manager review has one of two recorded results:

Manager result	Notifier behavior	Finance behavior in an explicitly participating short sess
Awaiting Finance	Notify that a manager review was recorded	Receive review-required, acquire current permitted facts,
Approved by manager	The same notification with the same declared payload	Receive not-required and terminate local participation

This is a **separately authored neutral notifier protocol**, not a relabeling of the result-bearing `:expense/review-disposition` message in the earlier worked review. Its message says that a manager review was recorded for this Expense and admitted session, not which operation, result, or current Expense status it produced. The neutral contract correlates the common session and disclosed Expense subject; it does not transmit `:review/result`, a route-dependent command identity, or another route-specific field to the notifier by unnoticed metadata. The two branches need not create identical command identities in the authoritative model, but those command identities cannot implicitly leak through this neutral notification's admitted envelope.

The sender must still establish the coarse proposition **a manager review was recorded** through its actual admitted command result. A trusted outcome-derived event or a trusted notification boundary that verifies the recorded result may establish this proposition without disclosing the route-specific outcome to the notifier. An adversarial manager browser's assertion of review completion is not equivalent evidence. The notifier's effect and result-handling contracts are the same on both successful branches. It does not need the private threshold or route selector. If the application changes notification text, destination, or other relevant observable behavior by branch, that difference requires a justified, disclosure-permitted input. A neutral payload establishes no unconditional noninterference guarantee for timing, occurrence, transport metadata or other observations outside its expressly checked disclosure and observation contract.

The two notification paths must also have compatible causal prerequisites. Here each notice follows an authoritative result establishing a recorded review of the same Expense under the declared message contract. Merely finding equal payload shapes in two branches would not establish that meaning. If a notification occurs in only one branch, or if observable destination, cardinality, delay or resource behavior differs in a way the contract distinguishes, the notifier may learn something about the choice even from identical bytes. Any such distinction requires its own admitted knowledge and disclosure path rather than an unsupported equivalence assertion.

Finance is different when it is already a participant in the short session. The global contract must distinguish both continuations, for example through a declared disposition message:

```
{:message/id :expense/finance-disposition
 :from :manager
 :to :finance
 :required #{:expense/id :review/command-id :review/disposition}
 :correlation #{:expense/id :review/command-id}
 :disposition-type [:enum :review-required :not-required :review-unavailable]}
```

This is a schematic intended message interface, not an assertion that `:disposition-type` is itself an implemented or independently sufficient Choreo constructor option. Its admitted payload schema must bind the required `:review/disposition` to this closed variant set and establish disjoint received alternatives where the projected receiver needs them. The sender must know the actual disposition and its correlation to the original review command, and must be permitted to disclose it to Finance. A matching enum tag and command id supplied by a different actor cannot establish that binding.

The projected receive can branch on an admitted `:review/disposition` after trusted source, subject, session and command correlation are established. A `:review-required` message justifies entering a current observation/action path; it does not itself authorize Finance approval. A `:not-required` message permits local return without claiming that Finance acted. `:review-unavailable` closes this session's participation when the manager's **already bound** command could not produce a usable result. That variant requires the real `:review/command-id` referenced by this message contract; it must not fabricate one to describe failure before command allocation. If an earlier acquisition or entry admission fails before a command is allocated, the session needs an independently authored session-only closure/retirement alternative, or Finance must not have been admitted as a waiting participant. That closure has its own declared payload, causality and knowledge contract; it cannot masquerade as a command-specific disposition. `:review-unavailable` does not assert that the command never committed; unknown command resolution remains with its declared recovery owner.

These are additional messages deliberately chosen by this session protocol. They must not be inserted by the projector merely to rescue an otherwise unrealizable declaration. Their delivery, timeout, and session-retirement paths must also be defined. A role that simply waits for `:review-required` forever cannot claim completion of both branches; absence of a message is not knowledge that Finance was unnecessary.

The invalid alternative is to give Finance a branch on `:needs-finance?` simply because the projector knows the manager's selector. Finance has neither that selector nor its private policy inputs. The projector must report the missing knowledge rather than copy either into Finance's context.

The normal durable alternative avoids this participating short session. The manager commits `:awaiting-finance`, and a later Finance actor starts a new execution from an authorized queue observation. Selection from that queue is an explicit user/application input, not automatic approval of every visible Expense. The chosen operation rechecks current eligibility at incorporation.

That alternative establishes a new workflow entry condition; it is not proof that a particular old command or branch occurred unless the observation also establishes that command-specific link. Projection may use it only where the global contract declares that continuation semantics.

Behavior-sensitive projection therefore controls both how much knowledge is distributed and when it must exist. Equivalent local continuations can share a plan; different continuations require evidence before the first difference, including a difference between waiting, performing an effect, and terminating.

1. Supported equivalence and conservative rejection

   Behavior-sensitive checking must define the local equivalence relation used by its supported language subset. The observables include whichever messages, effects, authoritative invocations, resource actions, and terminal or failure outcomes the semantic contract distinguishes. Relevant ordering and timing constraints must not disappear merely because the final values happen to match.

   Permitted internal steps may be abstracted only under an explicit semantic relation. The checker is not required to decide equivalence of arbitrary Clojure functions or unrestricted programs.

A concrete conservative baseline compares normalized continuation graphs up to renaming of diagnostic state identities and local binding names. It preserves operation/action contract identities, bound inputs and results, message destinations and payload meanings, observable effects, owner scopes, and terminal/failure alternatives. Ignoring source coordinates does not ignore causal dependencies or substitute one implementation contract for another. Cycles require a checked graph relation rather than recursive unfolding without a termination bound. Stronger equivalence rules must identify their mechanism and premises; equality of final values or of function names is not such a rule.

A receiver may merge remote alternatives when admitted incoming messages distinguish the alternatives before their different behavior. The branch selector must be established at the receive boundary, rather than inferred from the absence of a message or from a value in a mutually overlapping optional field. If alternatives share a sender, event, channel, and correlation scope, their admitted payload contracts must be disjoint or an explicit discriminator must select them. Optional fields alone commonly overlap. A supported checker rejects a known ambiguity and reports an unresolved obligation when opaque schema predicates prevent establishing disjointness. It must not choose the first matching branch by map or registration order.

When the supported analysis cannot establish equivalence or a sufficient knowledge path, it must reject or report an unresolved obligation at the appropriate admission boundary. It must not silently assume equivalence, disclose a private discriminator, or introduce an undeclared message to make projection succeed. An explicit supported communication path or visible trusted assumption may be authored where permitted by the relevant contract. An asserted equivalence remains a premise of the resulting projection claim; it cannot silently create missing input data, discharge a mandatory failed owner check, or authorize private disclosure.

2. Trace preservation and progress are separate obligations

   Excluding forbidden local traces is a safety claim. It does not by itself prove that a projected program can perform every required interaction, avoid deadlock, or eventually complete. A projection that admits no useful behavior must not satisfy the required operation merely because its empty behavior is safe.

   Each claimed projection result must state the global/local behavior relation and the entry and environmental premises under which it applies. Required realizability, terminal behavior, and progress obligations must be discharged with the appropriate evidence. Liveness remains conditional on its named delivery, availability, fairness, or application-action assumptions.

   These obligations supplement, rather than replace, the independent application capability acceptance defined in Part I. A concrete witness that one schedule projects correctly is not a proof about every admitted schedule; a proof of trace inclusion is not necessarily trace coverage, absence of deadlock, liveness, authorization, or secrecy. Completion and acceptance use the specific theorem and environmental premises that their claims require.

Branch-choice acceptance counterexamples

Apparent shortcut
Both branch continuations emit the same neutral notification payload, but only one branch emits a notification
Finance reads a manager-only threshold from the global branch selector
Finance waits only for `:review-required`, while the other branch has no message or local terminal path
Finance receives a permitted `:review-required` variant from a different session or with a copied command id
A notifier's destination or effect changes by route while its payload remains the same
A supposedly neutral review message hides `:review/result` or route-specific command metadata in its envelope while showing identical n
Two receive alternatives have overlapping optional-field schemas and the runtime picks the first match
The manager's command is unresolved and a `:review-unavailable` message is treated as proof of rejection
Finance is already waiting, but the manager fails before C2 is allocated and sends `:review-unavailable` with a made-up command id
A finite structural check finds some terminal path and claims Finance always completes
A concrete replay succeeds and its result is labeled universal projection refinement

The cases express the finished language's admission requirements, without prescribing a particular projection algorithm or proof backend.

Communication, causality, and completion

Global communication and physical transport have different granularity. A global `:communicate` describes the semantic transfer under its message contract; the local plans expose sending and admitted receipt separately. The realization relation identifies which concrete event establishes receipt and receiver knowledge. A local `:complete-send` boundary may permit the sender to finish without waiting for the peer, but semantic communication has not thereby occurred: its realization witness is the admitted receiver consumption of that same message occurrence. A send callback may mean enqueued or accepted by a transport, not delivered to the peer. It must not establish the stronger event by implication. Nor does acceptance by a receiving transport alone establish that the correct role-local machine admitted and consumed the message. The realization relation identifies the admissible receiver event, its actual message origin, and the resulting knowledge transfer. Acknowledgment meaning is contract-specific: transport receipt, machine consumption, and business adoption establish different propositions.

The channel contract states relevant ordering, loss, duplication, replay, correlation, and disclosure semantics. Where loss is possible, a required receive needs an appropriate retry, timeout, durable handoff, or other recovery path, with any eventual-delivery claim stated as a premise. Each retry or handoff preserves the declared occurrence, command and session semantics, or explicitly establishes a new occurrence under the applicable admission rule. A timeout establishes that a deadline elapsed locally; it does not prove non-delivery, remote rejection, non-incorporation, or peer termination. The receiver's silence is not evidence that it chose an alternate branch. A sent closure/disposition message is likewise not an actual consumed closure. Where a participant terminates locally after sending, the global terminal claim still accounts for the receiver's required communication and subsequent effects, or an authored terminal/handoff alternative that explicitly owns the undelivered obligation. The terminal relation must not manufacture the missing receiver event, assume fair delivery without a premise, or erase an outstanding durable effect because its initiating process has retired.

Correlation binds the message occurrence to its admitted conversation, operation, participants, and semantic subject as required. Command, execution, transport, and message-occurrence identities are distinct unless the contract deliberately identifies particular instances. Correlation fields learned from the first message still need an admitted routing/source scope; accepting an unknown id does not let an arbitrary message choose which authority or session it belongs to. At first contact, an unbound correlation key may be learned only after trusted routing, origin and scope admission; the sender's claim of a session identifier cannot itself establish the authority to bind that session. The required relation concerns the intended occurrence and participants, not merely equality of supplied key strings.

Duplicate handling is explicit. Reaccepting a transport delivery must not repeat an already consumed local transition or authoritative effect unless the declared repeat semantics permit it. Rejecting a stale private callback does not erase independently usable command-outcome evidence; reconciliation can admit it under its own owner and disclosure contract. Duplicate suppression is qualified by the protocol's semantic occurrence identity and admitted replay scope; a repeated delivery of an already consumed occurrence cannot become a second command or effect through a new transport attempt. Conversely, independent legitimate occurrences with equal payloads must not be collapsed merely because their bytes match. Retrying an uncertain authoritative command uses the declared stable command identity and reconciliation contract, not a fresh incorporation disguised as message redelivery.

Projection also preserves causal dependencies across foreign authoritative steps. If role B may act only after command C has a recorded successful result, B needs a message or authoritative observation establishing that exact premise before acting. Merely

traversing a foreign authoritative node in the compiler does not establish runtime evidence at B. A message reporting transport acceptance or an attempted invocation is not proof that C committed. The authoritative result contract determines which outcomes establish the specified predecessor; an unresolved result retains reconciliation rather than satisfying success or definite failure by default.

A completion descriptor naming an authority and basis is insufficient by itself. An admitted read of an unrelated card, a read before C became visible, or a current state compatible with several commands cannot discharge C's dependency. The witness must bind the predecessor, subject, required result, scope, and relevant progression. Where the continuation requires only current eligibility rather than C's historical success, the global contract must state that weaker, different premise.

For a wait with several alternatives, dependency discharge is edge-specific. An ordinary timeout or cancellation alternative does not inherit the evidence provided by a successful authoritative-observation alternative. Nor does receipt on a different session edge establish the intended predecessor. A compact plan must retain that distinction or reject the unsupported projection, not discard the dependency because one edge could establish it.

This rule does not impose a global lockstep order on independent work. If an abstraction permits reordering independent local actions, its dependency and effect contracts must justify that commutation. If the global meaning really requires receipt before a sender's later effect, the protocol needs evidence such as a declared acknowledgement; local send completion is insufficient.

The refinement claim names its observation alphabet. A coarse distributed trace may hide local calculations or reads, but that does not prove local branch correctness, confidentiality, resource safety, or presentation behavior. Nor can it prove an externally significant notification simply because the receiver admitted the message that **requested** one. The notifier's issued/fulfilled/failed effect evidence belongs to its declared effect contract and the physical realization relation. Where an application promises a durable notification, the authoritative command's required publication/work intent and the notification worker's later physical attempts have distinct, stable, correlated identities and separate disposition milestones. An acknowledged message, a broker acceptance, and a provider fulfillment cannot be collapsed into one semantic event. Those properties require their own retained observations and judgments. Choosing an alphabet that hides all useful behavior cannot establish application acceptance.

A required durable publication may be realized by a separate model-work-driven notification execution after the initiating actor has retired. This is not a license to forge that actor's `:send` or the recipient's corresponding `:receive` in the original projected execution. The authored protocol either retains and recovers its required sender occurrence, admits a checked delegation/alternative with valid origin and knowledge, or uses a separately declared worker/receiver path whose observations are correlated to durable work. The global terminal and effect evidence must follow the chosen path; structural similarity of messages or work identities cannot establish refinement across the paths.

Message payload closure protects knowledge boundaries

Closed messages are not only a schema preference. They are a knowledge-flow boundary.

If a message declares:

```
{:request-id UUID
 :outcome ClaimOutcome}
```

Here `:request-id` identifies the domain Request involved in the Claim outcome; it is not `:transport-request-id`, which identifies a transport attempt. Receiving this message may establish only the declared communicated facts.

The receiver does not acquire arbitrary sender context, and the implementation must not merge unknown payload keys into machine context by convenience. Declared required fields are present on an admitted message; optional fields establish knowledge only when actually sent and received under their contract. A sender must know each transmitted semantic field and be entitled to disclose it. Receipt establishes **communicated** provenance, not independent authority of the transmitted assertion. A claim that the message carries a committed outcome also requires trusted source and command-result correlation.

An explicitly open payload must follow the extension-data contract from the value-schema boundary. Openness alone does not grant undeclared fields semantic knowledge status or permission to influence a branch. Nonsemantic transport extensions remain outside admitted fact storage, branch inputs, operation arguments and semantic history. Promoting any extension to a semantic field requires a new admitted contract and its knowledge, disclosure and compatibility obligations.

The example's `:request-id` is domain correlation, not a transport attempt identity. A command-specific Claim settlement also requires the applicable command/result and owner binding; matching only the Request id cannot justify reusing another Claim's disposition. Any weaker guarantee actually selected remains visible in the result.

Communication and causality acceptance counterexamples

Apparent shortcut	Required admissio
A send adapter reports successful enqueue and the sender records peer receipt	Preserve transport
A sender completes a required closure send and returns locally while the receiver remains waiting	Do not report the a
A proxy acknowledges receipt and the application treats the payload as installed in a browser	Require the separa
A global trace hides the notifier's local effect and a proof of message receipt is advertised as proof of external delivery	Reject the unsuppo
A message from untrusted first contact contains a plausible session id	Establish routing,
A dropped message times out and the sender declares the remote authoritative effect rejected	Preserve uncertain
Two deliveries of one semantic occurrence arrive through different transport attempts	Prevent a second s
Two independent occurrences have equal payload bytes	Do not suppress th
A notification refers to Request A and claims Command C committed, but the only trusted read is of Request B	Reject predecessor
An await timeout and an authoritative-observation event converge, and both inherit the observation's knowledge	Keep dependency
An open message includes undeclared `:admin?`, later read from a generic context merge	Exclude the field fr
A sender transmits optional `:outcome` but knows only the Request id	Reject the actual tr

These examples constrain the finished system's semantic admission and correspondence obligations without prescribing a particular wire format, queue implementation or distributed transport.

Projection preserves necessary communication but does not worship messages

Communication is one way to establish knowledge, not the definition of knowledge.

A projector preserves a declared communication when that occurrence is part of the global observable protocol, establishes a necessary causal predecessor, or is the knowledge source required for another role's behavior. It does not discard a required send/receive merely because an unrelated later read could return an equal value. Only an authored alternative or a checked refinement under a specified observation and recovery contract can change how a global communication obligation is realized.

Nor does the projector invent a redundant direct message merely because one actor's choice eventually affects another actor. If the **authored** continuation requires a proposition that the other actor may instead establish by another admitted path, the legitimate sources include:

- justified entry knowledge;
- authoritative observation;
- supported deterministic derivation from facts already known to that role;
- another declared and justified knowledge path.

These paths are not interchangeable merely because their values have the same shape. Each source must establish the proposition actually required by the continuation, including its subject, role, authority/provenance class, decision or command correlation when needed, temporal scope, and disclosure permission. The evidence must arrive before the role's first dependent action. A declared assertion is usable as an **assertion** premise when that is the contract; it does not automatically establish authoritative truth.

An authoritative reread can establish current permitted state, subject to its acquisition, freshness/basis, scope, and progression contract. It does not automatically prove that a particular earlier command committed or that a specific historical branch was taken. Where the continuation depends on that history, the design supplies a command-bound receipt or outcome, a retained authoritative fact with the necessary correlation, or a different explicitly authored continuation. A current observation and a command-specific decision may be compatible without being equivalent.

The choice of knowledge path also determines recovery. A durable authority rendezvous requires suitable initial acquisition and reconnect/missed-advisory repair so a dropped notification does not permanently hide a required fact. An advisory may justify starting or repeating an authorized acquisition; it is not itself the new authoritative data, proof of visibility, or proof of browser installation/adoption. By contrast, a genuinely ephemeral report may permit loss and expiry without a durable reread: reconstructing old transient intent from a later database value would change its semantics.

Neither optimization grants an unlimited liveness claim. If progress depends on eventual acquisition, fair delivery, an active subscription, or a user's Refresh action, the guarantee names those premises. A projector cannot solve a missing semantic producer by assuming that a future read or message will eventually appear.

This matters deeply for Gesso's web architecture because durable business facts often rendezvous naturally through model authority and reread rather than long-lived direct participant messaging.

HumanHelp Request publication and a collaborative typing indicator illustrate two different recovery contracts, detailed in the following sections. Their contrast is a construction requirement, not a heuristic that all durable data must use rereads or all transient data must use direct messages.

Required local meaning	Permissible coordination, subject to its contract
Current authorized Request projection	Scoped advisory or discovery followed by authoritative acquisition and justified a
Outcome of one particular Request creation command	Command-bound authoritative result/receipt or other admitted correlated evidenc
Recent admitted typing intention	Authorized ephemeral report with source ordering and expiry; loss may be accept
Required authored participant communication	Admitted receipt of that semantic occurrence, or an expressly authorized refineme

Necessary-communication acceptance counterexamples

Tempting substitution	Required result
A compiler removes an authored participant message because a later query may return the same field	Preserve the message or demonstra
A helper rereads `:approved` and claims its own earlier command was approved	Require command-specific correlat
A new Request is committed but the advisory is lost and no discovery/reconnect path exists	Reject the claimed required conver
A browser receives a board advisory and marks an unacquired card as adopted	Do not establish authoritative cont
A typing report is dropped, and a later database query is claimed to reconstruct that transient report	Preserve the declared ephemeral lo
A role has a valid fact from another principal's or Organization's scope	Reject unrelated knowledge even i

These are obligations of completed Gesso. An application may deliberately choose either communication or authority-mediated rendezvous when the chosen path satisfies the actual semantic, disclosure and recovery contract.

1. Durable Request publication and selective adoption

   For a newly created Request, the authoritative result belongs to the public Request model. The creator learns the permitted command outcome and reacquires the declared card or board projection. Other clients need not receive the whole Request from the creator's execution. A command-bound accepted result can also justify immediate presentation of the creator's newly created card when the result and membership-adoption contract supply the required content and scope. The framework does not force an additional board reread merely to display an already admitted result. Updating one card, introducing a card into a filtered or sorted board, and adopting complete board membership are different propositions; each requires the corresponding declaration and completion evidence. An unresolved command cannot be shown as a committed creation solely because an optimistic view was rendered.

```
{:operation/id :request/create
 :authority :request
 :caller-role :customer
 :command-protocol :stable-bound-command
 :accepted-continuation
 {:creator-acquisition :request/created-card
  :remote-coordination :request/board-advisory}
 :rejected-continuation :request/create-rejected
 :unknown-continuation :request/reconcile-create}
```

   This is an inspection view of imported contracts. The admitted create operation supplies the authorized caller and command-specific created Request identity. An arbitrary new row found by a query does not establish which Request this command created.

   Remote coordination is triggered from the authoritative incorporation or a recoverable publication handoff under its declared contract. Required invalidation cannot depend solely on the creator's browser receiving a response and deciding to notify peers. Publication failure after commit leaves a committed command plus delivery debt; it is not a rejected create command. Required publication is secured by an atomic authoritative change/publication intent or an equivalently qualified durable discovery mechanism. A best-effort post-commit notification alone cannot establish no-lost-change recovery when a process can fail between commit and publication. The recovery contract preserves work identity, scope and required progression across retries and router or subscriber replacement. Part IV owns the atomicity, propagation and repair obligations of that handoff.

   An advisory identifies a permitted scope and justified acquisition requirement. Before the browser records it, the shared-event admission establishes the current subscription or fragment owner, source registration and instance, source provenance,

permitted transition, and scope/requirement applicability. Matching a subscription identifier or accepting a transport event by itself does not establish those facts. Advisory payload data cannot relax the registered producer, recipient or disclosure contract. Its disclosure does not reveal a hidden Request by implication. A lower bound must be supplied by the admitted producer or a declared fence protocol, not invented from the receiver's old basis. Subscriptions and reacquisition cover initial registration, reconnect, and missed-notification gaps through a declared read/registration fence, durable cursor, buffered event interval, periodic authorized rediscovery with stated premises, or an equivalent protocol. Neither a new subscription alone nor an old snapshot plus best-effort events establishes gap-free awareness. If a producer lacks a minimum progression bound, a tokenless advisory needs another qualified fence or read rule; ”newer than the recipient's last displayed basis” can be too weak.

The browser uses the pure transition and effect-shell separation from Part II. The following calls illustrate the semantic steps, not a second independent business-rule authority or a requirement to use these exact helper names:

```
(def after-advisory
  (board-runtime/step current-board admitted-board-advisory))

(def refresh-start
  (board-runtime/step
    (:state after-advisory)
    admitted-user-refresh-event))

;; The shell executes the emitted acquisition effect. It retains its
;; registered owner, concrete instance, scope, and principal-context binding.
(:effects refresh-start)

;; The completion is admitted against the latest board state, not the state
;; captured before the request. A later installation completion may be needed.
(board-runtime/step latest-board admitted-acquisition-completion)
```

The server derives the caller from its actual authentication context. The browser effect cannot grant a principal identity by carrying a field. The returned observation is bound to the full query, projection, visibility, temporal context, authority, and basis required by the acquisition contract.

Acquisition, permitted installation, actual installation, and semantic adoption are separate where the realization makes them separate. An acquire-only obligation may be discharged by an admitted read, but an acquire-and-adopt obligation that requires physical installation is not discharged by a completed HTTP request, validated response, proposed swap or mere installation permission. A pure state update does not itself swap a DOM node. An asynchronous installation must report its admitted completion or failure, under the same owner, target instance, scope and observation that authorized the effect. The last installation-right check and the physical mutation must share an appropriate serialization or equivalent protection against target replacement. Adoption discharges only requirements that the installed/adopted observation actually covers, preserving later or unrelated requirements and respecting the current owner. Coverage is about the authorized query and its visibility, projection, basis and temporal semantics, not about the presence of a particular hidden row in the rendered HTML. An observation can correctly cover an advisory without exposing the Request when the current principal or active filters exclude it. A stronger admitted acquisition may discharge more than the requirement that initiated it, but not an unrelated scope or a requirement whose adoption milestone has not been reached. Physical effects run outside retriable state-update functions. The worked X18/X19/X20 race in Part II fixes the intended interpretation: a permitted X19 installation may satisfy X18 while retaining an intervening X20 requirement under partial-advance policy. Under an all-current-requirements policy, that X19 installation instead defers until sufficiently covering evidence is acquired. The policy is declarative and must not change according to which asynchronous callback runs first. An owner, principal, query, resource or target replacement invalidates the old installation right even if the response's basis is newer. The accepted pure transition is the only source of emitted physical effects.

Bob may therefore retain the old board while showing a Refresh affordance. The notification does not establish the new Request's title or visibility; authoritative reacquisition does. Eventual adoption remains conditional on the declared automatic or user trigger, not merely on the database being available. The creator's immediate accepted-command update and other viewers' selective Refresh policy can therefore coexist without giving the creator's browser authority to publish trusted facts for those viewers.

Durable-publication acceptance counterexamples

Apparent shortcut	Required constr
A Request commits, then the web node crashes before an unprotected best-effort advisory	Preserve require
A viewer subscribes after the initial read and misses a change in the registration gap	Require a fence,
A correctly shaped advisory arrives for a retired subscription instance	Reject the obsol
A remote advisory includes a hidden Request title that the recipient is not entitled to see	Reject prohibite
A tokenless advisory is satisfied by reading merely later than the viewer's old basis	Require qualifie
The creator's command has an unknown disposition and the optimistic card is treated as a committed new row	Reconcile the st
A correlated accepted create result updates one card, then code assumes the entire board's sorted membership is current	Check and admi
An X19 result is installed while X20 remains outstanding	Discharge only
An HTTP response completes but its HTMX swap is cancelled or its target retires	Retain acquire-a
A valid newer observation returns for an old principal context or replaced board owner	Reject obsolete i
A complete authorized board observation omits a Request hidden by access policy or selected filter	Permit coverage
An effect is executed inside a retriable pure state update	Reject duplicate

These are requirements of completed Gesso. They do not prescribe a single database, routing topology, browser shell, or selective-adoption policy.

1. Ephemeral typing reports

   A typing indicator has a different meaning. It reports that an admitted editor channel recently sent a typing intention; it does not prove that a human is physically typing and need not have a durable database fact to reread.

   The wire payload carries intent, not sender identity. An untrusted client cannot select the admitted actor, authenticated principal, source instance, recipient, or ordering epoch by adding fields to this payload:

```
(def TypingPayload
  [:map {:closed true}
   [:document/id uuid?]
   [:typing? boolean?]])

(def valid-typing-payload? (m/validator TypingPayload))

(def typing-contract
  {:message/id :document/typing
   :from :editor
   :to :peer-editor
   :payload TypingPayload
   :sender-binding :authenticated-registered-document-channel
   :receiver-binding :authorized-document-subscription
   :delivery {:durability :ephemeral
              :delivery-guarantee :best-effort
              :replay :none}
   :ordering {:source :registered-actor-stream
              :duplicates :ignore
              :older-sequence :ignore}
   :display {:meaning :recent-admitted-typing-report
             :expires-after-receipt-ms 3000}})
```

   The trusted boundary uses the registered channel and current document policy. The following functions are contract sketches: the complete implementation may elaborate them from admitted declarations, without requiring applications to write a second, unchecked authentication or schema language:

```
(defn admit-typing-message!
  [registered-channel decoded-message]
  (when-not (valid-typing-payload? decoded-message)
    (throw (ex-info "Invalid typing payload"
                    {:error/type :typing/invalid-payload})))
  (let [document-id (:document/id decoded-message)
        sender (channels/require-current-sender! registered-channel)
```

```
       permission
       (document-model/require-typing-permission!
         (:trusted-context sender) document-id)]
   (typing-boundary/issue!
     {:contract :document/typing
      :registered-source registered-channel
      :sender sender
      :permission permission
      :payload decoded-message})))
```

These API names specify boundary responsibilities. The channel supplies the authenticated principal, admitted actor, current source instance, and their binding; the payload supplies none of those. The model permission result binds that principal and document under the declared authorization timing policy. Issuance checks their correspondence and constructs the receiver envelope. Recipient routing separately checks each subscription's permission to receive this document's indicators. Source admission and recipient admission are distinct: a legitimate editor report is not automatically authorized for every peer, document view, or account context. Later revocation follows the explicit subscription policy; initial registration is not an indefinite disclosure grant. The admitted receiver envelope binds the registered sender/actor, document, source instance and applicable ordering evidence. These fields are issued or verified by the trusted boundary, not accepted as additional assertions from the adversarial wire payload.

The issuer assigns a sequence within the admitted actor-stream instance, or uses another admitted ordering mechanism with the same contract. The sequence is part of the trusted report envelope, not a client-chosen field in `TypingPayload`. For the sequence-based realization, the issuer supplies a strictly advancing value for accepted reports in one registered stream; the receiver compares it only against that stream's admitted high-water mark. Delivery remains best-effort and may be reordered; the sequence rule means older accepted reports cannot reverse newer display state, not that every issued report will be delivered.

Reconnection requires an admitted new source binding and a declared retirement or succession rule. Neither a random larger sequence nor an unfamiliar epoch may replace the current source merely by arriving. An old stream's delayed `:typing? false` or `true` must not change the replacement indicator. Different source instances have no implied numeric ordering, even for the same actor; their precedence comes from the source/owner admission contract. This provides ordering of admitted reports, not proof of when the sender actually typed.

The peer maintains one indicator owner for the document, admitted actor, local principal context, and source instance. The following table is its semantic transition contract:

Event	Required admission	I
New `typing? true` report	Correct source and owner; sequence exceeds last accepted within that stream	I
New `typing? false` report	Same checks	I
Duplicate or older report	Same source but non-advancing sequence	N
Expiry callback	Current timer instance still owns this indicator and its deadline has elapsed	I
Retired timer/source callback	No current ownership	I
Navigation, account change, or subscription retirement	Owner lifecycle transition	I

The sequence high-water mark remains while that registered stream can deliver messages, including after a stop or expiry. Otherwise a delayed duplicate could resurrect an indicator. A `typing? false` report changes presentation, not the fact that a higher-sequence report was accepted. Timer expiry likewise does not reset report ordering. Safe reclamation of a stream's ordering state requires its admitted retirement/fencing contract to exclude future delivery into the retired owner; mere invisibility or elapsed display expiry is not such a fence. Bounds and cleanup follow that lifecycle, and identities from unrelated streams are not numerically ordered.

The timer uses a local elapsed-time contract. The display claim is three seconds after the last accepted `typing? true` report, not three seconds after an unverifiable human action. The admitted receipt determines the local expiry deadline; a `typing? false` report retires the timer rather than scheduling a new display deadline. Newer admitted `true` reports replace both the deadline and its owning timer instance. A timer callback must pass current owner, instance and deadline admission before requesting the hide effect; checking a callback's timer id alone is insufficient if its target or subscription was replaced. A delayed first delivery can still show a recent-receipt report; if the product instead requires bounded age from issuance, it needs additional age and clock/delivery premises. A suspended browser rechecks expiry when it resumes rather than treating a late timer callback as evidence of continued typing. Timer scheduling alone is not a hard wall-clock guarantee. If the physical shell cannot run at the intended deadline, its eventual completion must still be admitted under the current

semantic state. Physical timer and DOM effects run only from accepted transitions, not inside retriable pure state updates or direct callback mutations of projected machine state.

Envelope consumers bind `:document/id` and `:actor/id` to distinct local names, such as `document-id` and `actor-id`. Their UUID-shaped representations must not erase the distinction between the document being discussed and the sender. This is the ordinary authoring consequence of Part II's identity categories.

If a report is lost, a database reread two minutes later cannot reconstruct it usefully. The indicator may simply remain absent or expire; there is no required durable replay. That differs from required acquisition of durable Request state:

Property	Request publication	Typing report
Durable semantic truth	Public Request model	None required for typing intent
Remote coordination	Scoped advisory and authoritative acquisition	Admitted ephemeral message
Loss handling	Declared repair/discovery path for required convergence	Absence or expiry is permitted
Local resource protection	Acquisition and installation owner checks	Source ordering and timer-instance checks

Projection preserves the selected knowledge and recovery path. It must neither replace genuinely ephemeral semantics with a later database query nor require direct peer delivery where a declared authoritative rendezvous is sufficient. It also must not turn a best-effort presence hint into a guaranteed delivery, proof of ongoing human activity, or an authoritative statement of editor membership. An indicator is a scoped presentation of an admitted recent report.

Ephemeral-report acceptance counterexamples

Apparent shortcut	Required admission or result
A sender adds `:actor/id` and `:principal/id` to the client payload	Reject undeclared fields and deriv
A valid document-scoped typing report is sent to a peer whose document access was revoked	Enforce the recipient subscription
A delayed `true` sequence 8 arrives after admitted `false` sequence 9 from the same stream	Ignore the older report without re
An old stream sends sequence 9999 after reconnection established a new admitted source	Reject the retired source; unrelate
A `false` report hides the indicator and its sequence high-water mark is cleared	Retain replay protection while tha
An expiry timer from the previous `true` report fires after a newer report replaced its resource instance	Reject the retired callback; it must
The browser suspends beyond expiry, then handles a delayed callback as proof of fresh typing	Check the local deadline and curr
A timer callback updates DOM directly from captured, obsolete document/account state	Require an admitted current-owne
A report is lost and a later Request-like database reread is treated as a replay of that typing intention	Preserve permissible ephemeral lo
A typing report passes the shape validator and is treated as proof that a human is actively typing	Retain only the declared recent ad

These are finished-system acceptance obligations, not claims that any particular backend or browser timer already realizes them.

Local executable guards belong in the executable projection

Proof derivations and global proof trees do not belong in the production browser.

However, a local predicate or guard whose result is required to execute the projected semantics is runtime-semantic data and must remain in the executable projection or in a trusted, admitted realization bound to that projection. The generated local program retains the guard's operational contract, required inputs, evaluation point, and failure continuation. A trusted server-only authorization or authoritative progression check must not be weakened into a browser-computed Boolean merely to make every guard portable. For each physical role, the assembled application binds the guard to the place with the authority and information needed to enforce it.

A compilation or proof artifact can establish that a guard is necessary or sufficient under stated assumptions. It cannot replace the actual check when the premises depend on the current runtime owner, resource, principal, observation, or target. A guard used to establish a proof premise is also part of the exact semantic contract on which that proof depends.

The distinction is:

Material	Required location
Guard checking that resource instance I still owns the right to resume execution E	Executable semantics where that transition needs it
Derivation explaining why that guard contributes to a property	Proof or diagnostic artifact, bound to the exact execut

The guard identifies the execution and resource instance required by this callback's admission contract; it is not a universal generation comparison. Shared fragment or subscription events use their own declared ownership checks. An effect completion may also require the effect invocation, target, current plan, scope, operation or command identity according to its contract. Conversely, a boundary that does not depend on a given identity category need not manufacture a corresponding universal counter.

Passing an ownership guard does not by itself establish the knowledge, provenance, or authority required for the resumed branch. The owning runtime also validates the normalized completion or failure, current transition, declared input/output schema, and applicable admission evidence. Guard evaluation and the state transition it protects must be serialized or otherwise protected against relevant owner replacement; a check followed by an unprotected asynchronous mutation reopens the race. The physical effect interpreter executes only effects of accepted transitions, with declared retry, idempotency, and resource-retirement behavior.

The executable semantic core evaluates runtime guards as part of the admitted transition semantics. A proof kernel or dedicated semantic checker establishes its stated judgment about those semantics; it does not replace the runtime admission check or supply missing runtime facts.

The production machine must never depend on an optional proof sidecar to decide a transition. A sidecar can carry semantic origins, property judgments, assumptions, and diagnostic paths when bound to the exact executable subject, but it cannot become a hidden production guard, authority token, or alternate state store. Removing optional proof/diagnostic material cannot alter the runtime transition relation. Conversely, removing an executable guard from a supposedly optimized plan changes the semantics and invalidates claims that depended on that guard.

Projection failure should identify the missing justification

When projection cannot produce a legal local machine, diagnostics should explain the actual semantic deficit.

Useful failure information includes:

- role;
- global state/transition;
- local behaviors that diverge;
- required discriminator;
- facts currently known;
- available provenance;
- missing knowledge path;
- candidate communication or authoritative-observation paths when derivable;
- the violated obligation and the admission stage that failed;
- whether the conclusion is proven invalid, unresolved under an opaque predicate or assumption, or blocked by an unsupported language construct;
- relevant source-to-projected locations and contract identities, where available.

A generic ”projection failed” error throws away precisely the information that the verifier already computed. Diagnostics must also distinguish a missing knowledge path from an actual failure of source authentication, guard ownership, resource correspondence, or authoritative realization. A private discriminator's absence may be named as an obligation without disclosing its value or the private policy itself.

These explanations are compiler/assurance evidence, not a permission to expose every source fact to a runtime user. Diagnostic rendering, structured errors, logs, and proof reports enforce their respective disclosure contracts. A redacted error retains useful obligation and location identifiers where permitted. If a checker cannot establish the property, it must reject or expose an unresolved obligation; it must not label it proved or quietly supply a fabricated guard or knowledge path.

Host-language callbacks may not bypass projected semantics

A local runtime may invoke host-language functions to realize declared local effects, but arbitrary callbacks must not become an alternate transition path.

The admitted interaction is:

```
projected semantic transition
    → declared effect with owner, resource and contract identity
    → registered host implementation
    → normalized completion or failure
    → owner-relative event admission
    → projected semantic transition and accepted effects
```

The forbidden alternative is:

```
host callback
    → mutate choreography state or protected DOM target directly
       using captured or unchecked authority
```

The latter reopens the proof and knowledge boundary. A host callback can produce a message or normalized event, but cannot unilaterally confer its acceptance. An ignored stale completion must not be reinterpreted as proof that the corresponding authoritative command rejected, never committed, or lost an independently recoverable result. Execution-private effects, shared advisories, and physical installation callbacks each retain their own actual owner and acceptance conditions.

The common boundary is owner-relative event admission in Part II. Part VI develops its browser implementation. An execution-private effect completion must not bypass that boundary, and a declared shared event must use its actual fragment or subscription owner rather than an invented execution.

Guard-and-diagnostic acceptance counterexamples

Apparent shortcut	Required completed-
A projected browser callback resumes after its resource instance was retired	Reject the stale comp
A fragment advisory is associated with the last active command execution because the event lacks an execution id	Apply the registered
A browser claims its local `:authorized?` guard passed and submits an authoritative mutation	Bind current authent
An effect callback checks a current target, yields, then installs into a replacement DOM target	Enforce owner check
The projected machine consumes a normalized failure that lacks the required effect invocation correlation	Reject or recover un
A production optimization omits a runtime guard because the proof sidecar once stated it was sufficient	Preserve the executa
A proof/diagnostic sidecar is absent, but the runtime attempts to load it to decide the next branch	Keep executable tran
Projection cannot justify a branch, and the compiler inserts a synthetic authority observation to make it type-check	Reject or expose the
A projection error includes a private threshold and an authorized user's role lacks permission to view it	Redact protected fac
A stale browser callback is ignored and the command status is recorded as rejected	Preserve independen

These are normative acceptance cases for finished Gesso. Runtime guard placement and implementation languages may vary, but every claimed projection and deployed realization must preserve the corresponding admission, resource, evidence, and disclosure obligations.

Long logical workflows and short runtime executions

A choreography may describe a logical workflow that spans human-scale time. Its business meaning can remain continuous while its participating runtime executions start and finish independently. Neither the global description nor the presence of an `:await` operation implies that one Aleph node, browser, or other process must retain a suspended machine in memory for hours or days.

An in-execution wait and a durable business wait have different contracts. An active role-local `:await` suspends that execution until an admitted event, retirement or recovery decision. A durable business wait instead records a model-owned pending condition or obligation from which later authorized actors can establish new entry facts. That distinction is explicit in the assembly; neither form silently acquires the guarantees of the other.

For example, a business flow may conceptually involve:

An approval workflow requests approval, waits under its domain policy, and continues when an admitted decision establishes the next condition.

A robust HumanHelp realization may instead be:

Period	Short execution and durable connection
Request submission	E1 incorporates pending approval, binds its command result, and records required durable follow-up under the aut
Human-scale delay	Pending domain state and any deadline/work obligation remain discoverable and recoverable under their declared
Approver action	E2 starts with its own authenticated principal, admitted plan, permitted current observation and explicit selection,
Later requester action	E3 separately acquires its authorized entry facts, if further work is needed, without inheriting E1's or E2's executio

The durable wait lives in model state, along with whatever command/outcome, pending-work, deadline or notification obligations the business contract requires. Durable existence alone does not guarantee that a later actor will discover or act on it. Where progress is required, the application declares the discoverable query, permitted recipients, wakeup or scheduling strategy, missed-notification repair, and assumptions about authorization, actor action, availability and fairness. A pending state without any permitted continuation is not a completed workflow.

Subject to the authoritative store's declared durability, recovery, retention and compatibility assumptions, the durable workflow fact can survive:

- web-node failure;
- deployment;
- load-balancer rerouting;
- client disconnect;
- browser suspension;
- long offline periods.

The ordinary Gesso choreography contract does not promise durable persistence, transparent relocation or automatic migration of arbitrary suspended server-local executions. A durable model record, an admitted command receipt, and a recoverable task are not snapshots of all in-flight runtime state.

Applications that require persisted workflow-machine continuations use a separately admitted, opt-in capability. Its contract covers stable checkpoint identity, atomicity relative to authoritative decisions and pending effects, recovery discovery, owner/fencing transfer, plan and data compatibility, replay and idempotency, uncertain effect disposition, resource reconstruction, expiry, cancellation, and eventual retirement. Restarting from a serialized machine map without these guarantees is not durable-continuation admission. The capability may be realized by a dedicated workflow engine or a Gesso-managed facility, provided the same semantic obligations hold. No such facility is imposed on ordinary short choreographies.

Durable waiting belongs in authoritative state by default

Human-scale waiting is normally represented as durable domain state rather than a suspended in-memory choreography execution.

This has several consequences.

First, subject to those persistence, retention and compatibility assumptions, the durable workflow meaning can survive:

- deployment;
- process loss;
- browser suspension;
- load-balancer rerouting;
- reconnect;
- long periods without any participant online.

Second, later actors can acquire the relevant facts by authorized authoritative observation rather than requiring the original process to resume. The observation establishes current, scoped eligibility; the actor's fresh execution still checks its own authorization and operation guards at incorporation. A queue entry or notification is not a grant to approve every visible item, and the later execution does not automatically inherit the historical outcome or transient authority of a prior command.

Third, a deadline belonging to business policy is durable authoritative meaning, not merely a browser or process-local timer. If the contract requires action after a deadline, a qualified durable scheduler, recovery scan or equivalent admitted mechanism owns that obligation. The scheduled attempt must recheck current eligibility and distinguish deadline occurrence, effect dispatch, and successful incorporation. A delayed callback cannot retroactively turn an expired authorization or retired resource into a valid operation.

Fourth, a durable operation with unfinished publication, external-provider work or command resolution retains that work's stable identity and recovery owner after its initiating execution retires. A process failure or timeout does not prove the corresponding effect never occurred. Reconciliation and retries follow the public operation's idempotency and disclosure rules; a new execution identity does not silently create a new command intent.

Finally, the framework avoids making one runtime's memory an accidental source of business truth. The later-execution and expense-history examples in the following section elaborate those identity and command boundaries.

Durable-wait acceptance counterexamples

Apparent shortcut	Required semantic outcor
E1 records `:pending` only in an in-memory Choreo machine and its web node crashes	Do not claim durable wor
A durable pending record exists but no actor, queue, schedule or discovery path can reach it	Do not claim required eve
E2 starts from an authorized pending queue but directly approves every row visible in it	Require explicit admitted
A user was authorized when E1 started, so E2 inherits that principal and capability days later	Reject borrowed transien
An E1 notification fails after commit, and the application reports E1's authoritative command rejected	Keep the committed outc
A process-local deadline timer fires after the workflow's authority, owner or policy changed	Recheck durable deadline
A worker times out on an external effect and starts a new command to perform the same business intent	Preserve stable command
A persisted continuation is loaded with an incompatible plan and reused effect capabilities from its old process	Require checked migratio
Two approvers concurrently observe pending state and both submit approval commands	Authoritative incorporati
The workflow has a graph path to an approved terminal but every eligible actor may remain offline forever	State the environmental f

These requirements describe completed Gesso independently of one particular process manager, workflow engine, database, transport, or scheduler.

A later choreography is not automatically the same execution

When durable state connects two periods of activity, the later protocol may be a new execution with a new execution identity.

For example:

Execution	Command	Durable effect when accepted
E1	C1	Submission becomes pending
E2, later	C2	Approval decision is recorded

The business process is continuous in domain meaning, but E1 and E2 need not be one persistent protocol execution. E2 must establish its own authenticated principal, current model eligibility, required knowledge, and applicable plan. The existence of E1's durable result does not carry E1's transient authority or resource ownership into E2. Its accepted result can establish durable facts about the domain and the command that caused them, within the model's authorized observation contract. It is not a reusable principal credential, an open effect handle, or evidence that the later actor may commit without rechecking current business rules. A new protocol execution may nevertheless continue the same logical workflow without a suspended E1 machine.

This distinction avoids conflating:

- semantic command identity;
- protocol execution identity;
- durable domain identity;
- workflow/business identity.

An operation's command identity also differs from each physical invocation, transport retry, authoritative basis and resource instance used to carry it. The assembly must preserve the distinctions relevant to each boundary, even when several belong to one domain workflow. A compatible retry can have a new transport/request identity while retaining its original semantic command. A later deliberate business action has its own command, even when it targets the same Expense and uses an equivalent-looking payload.

Where an application truly requires a durable suspended execution, that feature must define its own:

- persistence;
- ownership;
- versioning;
- migration;
- replay;
- resource cleanup;
- liveness;
- recovery semantics.

It is not silently inherited from ordinary Choreo.

The expense example extends across several days without keeping a suspended server process. Its three periods of activity have these contracts:

Period	Entry knowledge and action	Authoritativ
Monday, employee submission	Selected owned Expense and admitted submit intent	Guard curre
Thursday, manager review	Authorized queue observation, explicit selection, coherent Expense/Policy observation	Guard curre
Friday, Finance review, if escalated	Authorized pending-Finance observation and explicit selection	Guard curre

Each row expands to a short choreography with the same explicit model-result mapping used earlier. Queue selection is an input event that may cancel or become stale. A manager's authorized observation can justify proposing one route, but the authoritative model must enforce the selected operation's current manager permission, status and policy at incorporation. If the policy, expense status, actor or selected route becomes invalid, the operation follows its declared rejection or uncertainty semantics; a prior queue read is not a commit-time guard.

The manager-approved branch is already a completed business decision for this example. The escalation branch makes the Expense eligible for a distinct Finance execution later, subject to then-current authorization and state. No silent automatic Finance approval or queue-wide action is implied. Queue visibility does not authorize bulk approval, and a normal return from an invocation is not by itself proof of successful incorporation.

Public model calls expose domain arguments and the bound command identity:

```
(expense-model/submit!
  trusted-employee-context
  {:expense-id E :command-id C1})

;; These are mutually exclusive manager alternatives, not two writes.
;; The admitted decision chooses one semantic operation before C2 is bound.
(case admitted-manager-route
  :approve-as-manager
  (expense-model/approve-as-manager!
    trusted-manager-context
    {:expense-id E :command-id C2})

  :escalate-to-finance
  (expense-model/mark-awaiting-finance!
    trusted-manager-context
    {:expense-id E :command-id C2}))

;; Only an Expense with admitted awaiting-Finance state can enter this route.
(expense-model/approve-as-finance!
  trusted-finance-context
  {:expense-id E :command-id C3})
```

These calls illustrate public operation contracts, not a standalone workflow engine or a claim that these exact function Vars already exist. C2 is bound to the chosen semantic operation, its canonical arguments, the appropriate principal/authority scope and interface version before first submission. It cannot be repurposed to try the opposite manager branch after an uncertain result; doing so would be incompatible command reuse. The registered model realization owns its guards, principal interpretation, time source, and atomic command protocol. A policy may be evaluated from a coherent authoritative Expense/Policy observation to present the permitted choice, but the actual incorporation must protect every mutable dependency on which that decision depends. A compatible snapshot, version expectation, or atomic predicate/effect design may realize that contract; an unconstrained read

followed by a stale write may not. The business policy and operation meaning are owned by public model interfaces, not duplicated by the UI or the projected manager choice. Callers do not pass a mutable list of commit guards that can omit ownership or status checks. The trusted context comes from the admitted invocation boundary; constructing a map with a familiar `:principal` field is not authentication.

If submission time is generated by the server, it follows the operation's time contract. The model distinguishes a command's effective domain time, recording/transaction time, and incidental invocation time where they differ. A transport retry does not acquire a new semantic command merely because the wall clock advanced. The recorded outcome retains the time actually incorporated; compatibility of replay is checked against the intended semantic arguments, not a fresh incidental timestamp.

A compatible retry checks command binding, retained-result applicability and current disclosure permission before returning the recorded outcome. It does not require Monday's Expense to remain a draft on Thursday. The historical recorded result does not assert that the Expense is **currently** submitted or that an earlier actor's authorization remains current. A new command requires the current guards. If command evidence was pruned, restored from another authority incarnation, or migrated incompatibly, the operation applies its declared retention, identity and unavailable-result policy rather than guessing non-incorporation. Competing attempts of one command are interpreted against authoritative outcome evidence; one failed attempt does not by itself reject the whole command.

When a result is unknown, recovery retains enough authorized command identity and binding to reconcile later. Retiring E1's UI or process is allowed only under the declared resolution or handoff contract; it must not erase the unresolved obligation or create a new command to retry the same intent accidentally. Current Expense state may have advanced beyond submission and is not by itself an answer to whether C1 was incorporated. A pending reconciliation obligation belongs to an admitted durable owner or recoverable reference after local retirement; the UI's disappearance is not command resolution. A later compatible retry or authorized command-result lookup must preserve the original intent's operation and canonical argument binding.

An illustrative history and identity view is:

```
{:expense/id E
 :events
 [{:status :submitted
   :effective-at monday-time
   :recorded-at monday-recording-context
   :caused-by {:command C1 :operation :expense/submit}}
  {:status :awaiting-finance
   :effective-at thursday-time
   :recorded-at thursday-recording-context
   :caused-by {:command C2 :operation :expense/mark-awaiting-finance}}
  {:status :approved
   :effective-at friday-time
   :recorded-at friday-recording-context
   :caused-by {:command C3 :operation :expense/approve-as-finance}}]}

{:monday {:execution E1 :command C1 :principal employee-P
          :plan :plan/submit-expense-A12}
 :thursday {:execution E2 :command C2 :principal manager-M
            :plan :plan/review-expense-A13}
 :friday {:execution E3 :command C3 :principal finance-F
          :plan :plan/finance-review-A13}}
```

This is a permitted inspection history, not a requirement to store one growing vector or a universal event DAG. It shows the **escalation branch** of manager review; a manager-approved branch records its own approved outcome for C2 instead of inventing a Thursday escalation or a Friday Finance command. The model's actual persistence, retention, correction, and command-evidence contracts determine what can be reconstructed. Effective and recording time remain distinct as in Part II. A current `:approved` status alone is not evidence of which actor or command caused it; history and causal claims require the relevant admitted retained records, not inference from status equality.

The business process is continuous because the authoritative Expense model retains its meaning. The displayed escalation-history E1, E2 and E3 separately establish their current principal, knowledge, permissions, resources, and admitted plan. The business may be resumed under a newer compatible plan without pretending that a paused E1 artifact survived unchanged. A new plan can change permitted UI affordances or current policy under its declared migration/compatibility contracts; it cannot retroactively rewrite C1's recorded historical outcome. Historical workflow continuity does not transfer a retired execution's capability or resource rights.

A durable workflow facility is an additional opt-in contract if an application needs persisted continuations. It must state checkpoint atomicity relative to effects, ownership or fencing, replay and deduplication, effect uncertainty, schema/plan migration, recovery discovery, and resource reconstruction. Serializing an in-memory machine map is not enough to establish those properties.

Later-execution and command-history acceptance counterexamples

Apparent shortcut	Required ad
A manager opens a pending queue on Thursday and bulk-approves all displayed Expenses	Require dec
A manager selects `:approve-as-manager` but the runtime invokes `:mark-awaiting-finance` using the same C2	Reject opera
A manager's policy read once said Finance was unnecessary, but Expense or Policy changes before commit	Enforce the
The manager-approved Expense is later sent automatically to Finance because the global workflow describes a Friday period	Do not inve
Friday's Finance actor reuses Thursday's principal, plan capability or callback owner	Require fres
A compatible C1 retry arrives after Thursday's manager decision and fails because Expense is no longer draft	Resolve C1
An unauthorized caller guesses C1 and demands its stored result	Enforce cur
C1 had an unknown outcome; an abandoned Monday browser issues C1-new to repeat the intent	Retain origi
A server retry recalculates a later timestamp and changes the recorded submission outcome	Return the
The current Expense status is `:approved` and a report claims that this proves C2 took the manager-approved branch	Require cor
A deployment installs plan A14 and claims it retroactively changes the meaning of C1 recorded under A12	Preserve his
Two concurrent C2 attempts select conflicting manager operations for one command id	Reject incor

These are finished-system acceptance conditions. They do not require one universal event log representation, an always-resident workflow machine, a particular database engine, or these illustrative operation function names.

Durable model state is the normal rendezvous point

For long workflows, authoritative state provides a stable rendezvous point between actors who were never concurrently connected. A later short execution acquires permitted current facts through the model's declared observation contract; it does not need a live communication channel to the earlier participant or a preserved local machine from that participant's execution.

The durable subject, business workflow, command intent, protocol execution, actor, plan, and transport attempt have distinct identities. A later actor can continue work on the same durable subject without inheriting any earlier principal credential, resource instance, command capability or projected branch. A durable business fact establishes only the proposition its authoritative source and observation scope justify. It is not an unbounded certificate of every event in the subject's history.

A later actor may know what happened because authority establishes:

```
:approval/pending
:approval/approved
:request/claimed
:request/done
```

not because it receives a packet from the actor who caused that state. These keywords illustrate model-owned domain judgments, not privileged universal Gesso statuses. An accepted `:request/done` observation establishes the current status under its source, scope, visibility and temporal contract; it does not automatically establish which earlier command or actor made it so.

This follows Part II's authoritative observation model: observation is a first-class knowledge path. A subsequent execution receives only facts it is authorized to acquire. A scoped query, authorized projection and justified basis may establish current eligibility, but the public model operation still protects its current mutable business guards at incorporation. Reading a queue does not grant the right to act on every item in that queue.

Durable pending state is not itself a progress guarantee. The application declares the opportunity for an eligible actor to discover and act on it, or states the environmental premises on which progress depends. That contract covers relevant authoritative queries, recipient visibility, initial and reconnect acquisition, missed advisories, work/deadline scheduling if required, and retry or reassignment after an owner disappears. It identifies the owner of required unfinished work rather than assuming that some browser or process-local callback will remain alive indefinitely.

Where loss of a required participant can strand the workflow, the runtime-condition and recovery requirements from Part I apply. Any lease, reassignment, cancellation, escalation, expiration, or other successor policy remains model-owned and must preserve its current authorization, fencing, command binding and atomicity requirements. An expired lease is not a permission to repeat an uncertain effect under a new command. Lost notifications cannot erase required durable work; failure to discover a pending item cannot silently change it into a terminal business result.

A durable model can be available without satisfying liveness if all eligible actors remain offline or decline to act. Progress claims name their delivery, scheduler, availability, fairness and human-action premises. A proof of reachable terminal states does not by itself prove eventual completion.

Deployment compatibility remains explicit

A durable business fact may outlive the executable plan that originally produced it.

That is normal.

What cannot be assumed is that a suspended **runtime execution** from an older deployment remains compatible forever. The corresponding contracts have different persistence and compatibility boundaries:

Subject across deployment	Required independently checked relationship
Durable domain state	The current model still interprets its admitted historical and current meanings, or a pr
Retained command/outcome and unresolved work	Original command binding, disclosure, retention, disposition and durable recovery ow
Projected execution and resource ownership	Continuation is explicitly supported by the old-to-new plan/assembly relation, or old
Physical client/server pairing	Actual route, protocol, codec, asset, producer/consumer and permitted rolling-release
Assurance or diagnostic evidence	A claim remains applicable to its exact artifact, interfaces, assumptions and physical c

The later stale-plan/recovery design therefore treats durable model truth, command reconciliation, execution rights, deployed routing, and proof applicability as separate dimensions. A matching plan name, URL, source file, version number or copied digest label is not sufficient compatibility evidence.

A deployment may invalidate continuation of an old projection while leaving the authoritative business state entirely valid. A safe recovery may retire old execution-private callbacks and resources, then admit a **new** execution under a current plan after authenticating the principal, acquiring justified entry facts and checking command/observation compatibility. Where a particular suspended execution is allowed to continue, a checked directional continuation or migration contract establishes its state mapping, owner transfer, pending effect treatment, input/output interpretation and recovery premises. Neither path may silently reissue an unresolved authoritative command with a fresh command identity.

Compatibility is directional and conditional on the installed realization. An older browser talking to a newer server and a newer browser talking to an older server can require different evidence; unsupported combinations must be excluded by an enforced routing contract or receive typed retirement and recovery behavior. An offline browser cannot retain indefinite continuation rights simply because it once had a valid plan. Conversely, retiring the old execution does not erase committed business work or command-result records whose recovery lifetime outlasts that plan.

If deployment changes the meaning or representation of persisted domain state, the public model requires its own compatible interpretation or protected migration. Migration is an authoritative writer under the applicable invariants, concurrency and authorization rules, not an incidental side effect of running a new browser plan. Rolling upgrade and rollback must account for the actual old/new writers and durable state they can encounter; rollback is not an automatic undo of incorporated domain migrations. Diagnostic sidecars may require exact artifact identity even when a declared continuation compatibility relation permits a different runtime artifact.

Durable-rendezvous and deployment acceptance counterexamples

Apparent shortcut	Requ
A Request remains `:pending` after server restart, but no permitted query or recovery worker can rediscover it	Do n
A new approver reads an authorized queue and uses the previous approver's principal or resource grant	Requ
An observer sees `:request/done` and attributes it to a particular old command without a retained outcome	Requ
A required notification disappears during a router failover and the pending work becomes undiscoverable	Prese
A lease expires while an external effect might have committed, and a replacement worker creates a new command	Reco
A newer plan rejects the old runtime representation and the application deletes its durable domain object	Retir
A stale client presents a matching plan name to a newer server with incompatible command outcomes	Requ
A current client is routed to an older server that lacks its required decoder	Exclu
A proof sidecar for an old runtime artifact is attached to a merely compatible newer plan	Prese
The application retains a command ledger after old assets expire, then treats lookup failure as proof the command never committed	Appl
A release changes persisted domain status representation without validating old writer and rollback behavior	Enfo
A new plan automatically adopts all refreshed board membership, bypassing the application's explicit Refresh policy	Prese

These obligations specify completed Gesso independently of an individual deployment, particular browser, server framework, database, or routing topology.

Semantic acceptance obligations

The definitions above require executable judgments, constructive admission paths, and focused conformance cases. Part IX owns compositional certification; Part X owns soundness and implementation correspondence. The semantic definitions used by construction, static verification, projection, the portable machine, host realization, and assurance tools must agree on their shared meaning. They must not silently introduce alternative meanings for a role, value, message, transition, authority, or terminal result.

A case is evaluated against an identified semantic subject, application assembly where relevant, selected assurance profile, and declared environment and trust assumptions. The outcome distinguishes a justified admission, a specific rejected violation, an unresolved obligation, and a capability outside the selected supported profile. Calling a case ”accepted” never means that all stronger properties have been established. Rejection cannot be used to claim success when the system should have accepted an ordinary valid application under the same declared contract.

The following cases constrain Gesso's completed semantic language and its realizations. Each case remains a mandatory conformance obligation:

Case
Valid opaque Clojure action with a complete admitted interface
Missing operation binding or malformed continuation
Role uses a value known only to another role
Optional message field absent on one path
Optional field actually sent without sender knowledge
New asserted output overwrites an authoritative binding
Ordinary environment event matches a participant message name
Two receive alternatives accept the same correlated message with different behavior
Role has equivalent local continuations under the supported relation
A result-bearing notifier message contains an authorized `:review/result` but the proof treats it as a neutral message because the displayed
An authored invocation projects an authority-owned result directly into the caller without an admitted transfer or observation
A browser's successful HTTP response is recorded as a trusted authority operation solely because the helper named it `:authoritative`
A recorded/no-notification receiver chooses the variant from missing optional result keys instead of validating the declared tag
A command-bound `:review-unavailable` message names a fabricated command when pre-command acquisition failed
The sender locally completes a required no-notification disposition and the global outcome is marked complete before receiver consumptio
The first projected role completes, but a required receiver/effect remains outstanding and the proof reports global terminal completion
Role must wait in one branch and return in another
Unrelated authoritative read follows a foreign commit
Timeout after submission
Compatible replay returns permitted recorded success
Old resource completion matches the command but not current owner
Required notification is lost after commit
A required post-commit notification exists only as a browser's future send, and the browser closes after C2 commits
A durable worker completes its own valid notification, but an assurance report attaches its receipt to an old manager-to-notifier global plan
A manager browser vanishes after C2 commits; the outbox worker later notifies successfully, but projection claims the manager's required
An accepted C2 is followed by a lost best-effort browser send and the checker selects `:no-notification` as though it were the authoritative
A durable outbox and optional browser hint both request the same notification, but the provider is invoked twice under unrelated publicati
A notifier consumes a command outcome message, then the system reports external notification delivered before the effect provider admits
A provider times out after perhaps delivering a notification, and the worker creates a new unrelated work identity to retry
A formal checker establishes the manager's work-recorded terminal, but an application assurance report calls the notification delivered alth
The application proves C2 created durable work, but the deployed assembly has no qualified worker ingress, recovery scheduler or permitt
A worker execution reports a provider result for work identity W2, while the composed certificate joins it to C2's retained work W1 solely
A provider acknowledges notification dispatch, but a certificate asserts that a person saw the message
A coarse distributed trace omits an effectful `:local` node, yet a proof certificate claims the notification was physically delivered
Old typing timer or duplicate report arrives after replacement
New valid artifact has different runtime locators
Finite graph has a path to return but can loop forever
A Malli-valid command claims a privileged principal or authoritative basis
A cached validator or custom predicate is reused after its effective contract changes
An explicitly open transport payload contains an undeclared semantic-looking field
A projector removes an authored communication because an authorized reread might return the same value
A transport send succeeds, but the destination role never admits the corresponding receive
A semantic message occurrence is delivered twice through different physical attempts

Case
A foreign authoritative step is followed by an unrelated current read of the same domain object
A scoped advisory has a valid schema but belongs to a retired subscription or unauthorized recipient
An authorized Request acquisition completes but its required browser installation is cancelled
An earlier accepted typing `false` is followed by a delayed lower-sequence `true` on the same source
A pending durable workflow survives deployment but its original runtime execution is gone
Manager review admits direct approval, but the application schedules Finance as if escalation were recorded
The current Expense is approved and the application attributes approval to a specific historical command
An old client communicates with a new server and its plan-name matches
A proof sidecar matches an old artifact but the new artifact is only continuation-compatible
A proof of one concrete schedule is reported as universal refinement, liveness or confidentiality

Tests exercising these cases are conformance evidence for the tested subjects and profiles. They do not replace a proof of the general projection relation, prove arbitrary host callbacks, or establish every external-system premise. Negative and mutation cases must reject for the intended semantic reason, not merely because the fixture became malformed in an unrelated way.

Acceptance evidence and claim boundaries

Each exercised obligation records enough stable information to understand which judgment was tested or checked:

Evidence dimension	Required distinction
Subject	Authored normalized declaration, verified global machine, emitted role plan, assembled application, physical handle
Claim	Exact structural, knowledge, authority, projection, trace, safety, progress, privacy, deployment or model-operation p
Scope	Relevant actors, data, roles, operations, schema/predicate definitions, application generation, deployment pair and e
Result	Admitted, rejected with a relevant semantic counterexample, unresolved with named premises, or outside a declared
Origin	Semantic locations and exact artifacts/interfaces on which the evidence depends, with applicable trusted-issuer and
Coverage	Whether the evidence is a test fixture, bounded model check, finite structural theorem, concrete witness, conditiona

The generator and assembly checker expose these distinctions without making ordinary Gesso application authors handwrite a second formal-methods language. When a model checker, proof kernel, static analyzer, runtime validator, or physical test discharges an obligation, its actual property and premises remain visible in the assurance record. A stronger dependent judgment cannot be claimed merely because a weaker component returned success. A conditional proof remains conditional, and an unsupported assertion about arbitrary host-language behavior cannot be relabeled as verified implementation fact.

Cross-layer conformance includes both directions needed for the selected property: valid authored actions must have justified projected/runtime paths, and observable projected behavior must trace back to admitted semantics. A generated runtime cannot introduce a hidden authority, a new observable effect, or an alternate terminal result merely because no positive fixture exercised the difference. Tests for message loss, duplication, stale callbacks or deployment changes exercise the relevant admissible schedules and recovery premises; they do not assume fair delivery by choosing only successful schedules.

An accepted physical realization establishes no stronger claim than its covered installed entry points and writers, runtime artifacts, decoders, owners, effect implementations, persistence providers and trust bindings. A test of a mock operation or selected route cannot certify every production invocation path. The release-level proof/coverage obligations remain owned by Parts VII, IX, X and XI. The semantic requirements here are independent of a particular implementation revision.

Positive construction acceptance

Acceptance also includes positive cases. The framework must not achieve apparent soundness by rejecting every meaningful application or by requiring every application author to implement their own formal toolchain.

Valid construction
An ordinary Clojure/ClojureScript function has a complete admitted operation/effect contract but is opaque to the closed prover
Two remote branches have equivalent authorized local behavior
An application chooses either a result-bearing or a truly neutral notifier contract for the same manager decision
A manager's declared invocation elaborates through a trusted authority role and a correlated result acquisition
A tagged result-bearing or no-notification message uses a checked closed discriminator and sender/receiver source correlation
A sender finishes a local send while the recipient is still processing a required message

Valid construction
A successful Expense command records qualified durable notification work, and the notification worker processes it after the manager exec
A review notification is explicitly best effort and its declared terminal promises only the admitted attempt or retirement
Manager review returns after C2 atomically records durable notification work, while an authorized worker's independent execution is still
A durable C2 records stable notification work, then a separate authorized worker execution acquires the pending work and notifies after th
A browser-side review notice is deliberately best effort and is lost after C2 incorporation, with an admitted timeout or retirement alternativ
The selected assembly has an authenticated C2 operation, recoverable publication work, an independently admitted worker with installed e
An application promises a work-recorded review terminal and separately conditional notification progress, while the worker has not yet fu
A declared open extension carries nonsemantic adapter data
A trusted operation is retried compatibly after its state has advanced
A valid projected schedule reorders independent hidden steps
A scoped authoritative reread legitimately establishes a current fact
A creator has a correlated accepted Request result and the application allows immediate local update
A typing report is lost under an admitted ephemeral policy
A valid pending business task outlives E1 and an authorized E2 starts under a newer compatible plan
A plan changes only diagnostic-only material while its declared continuation relation remains valid

Paired admission and verdict-preservation gates

A useful acceptance suite must distinguish correct rejection from correct **admission** under the same semantic contract. For every supported case family, exercise a well-formed positive control and a minimal adverse change to its source, event or physical binding. The two sides share the declared semantic subject, application scope, requested assurance profile and relevant environmental assumptions, except for the specific condition under examination. If that condition legitimately changes the semantic subject, retain both exact identities and the stated relation rather than claiming byte-identical subjects. The required distinction is behavioral, not an insistence on identical test-file syntax.

Property and paired construction	Positive control and justified result
Definite role knowledge	A role consumes a declared required field from an admitted, source-correlated receive, then
Behavior-sensitive projection	Two legal branches have equivalent observable local behavior for a role; the local plan omits
Early session closure	A pre-command acquisition failure uses an authored, correctly correlated session-only termi
Opaque host-language effect	A typed Clojure function with declared operation, authority/effect interface and permitted o
Authoritative command replay	A disclosed, compatible repeat of a previously incorporated C2 returns its original result wit
Durable and weaker notification profiles	A work-recorded review ends after C2 and qualified durable intent, leaving a recoverable wo
Correlated worker and effect	An independently admitted worker processes the bound work identity and a provider report
Physical installation and adoption	An authorized scoped acquisition is actually installed into the current admitted target and ac
Concrete correspondence and general proof	A valid projected schedule is admitted as a **concrete witness** against its named global plan,
Finite capacity and progress	A fully explored supported finite domain may establish its exact finite property; a fair, recov
Intended policy versus valid typing	A well-typed, currently admissible business predicate is accepted as a **typed rule**, with prod

The admission result for a valid construction may still be **conditional**: an opaque host callback, provider promise, backend qualification or fairness premise can remain open. A supported static contradiction is **invalid**. A modeled runtime guard denying an actual operation is **runtime rejected**, not an invalid authored choreography. An unresolved submitted command remains **indeterminate** until its disposition is admissibly known. A checker outside its supported language or capacity reports **unsupported** or **incomplete**, not either a successful proof or a semantic counterexample. An alleged product-policy error requires independently named intent and cannot be manufactured by Gesso's type checker.

A paired fixture records the original source-owned declaration and mutated dependency, normalized subject, selected profile, stage, expected judgment, actual checked property, premises, observation/effect scope and intended rejection or uncertainty reason. For a positive case, evidence must identify what was actually admitted or executed, rather than merely counting the absence of exceptions. For a negative case, failure of an unrelated schema, parser, route, or fixture setup does not satisfy the intended semantic gate. Source metadata, copied reports and test-injected status flags cannot act as admission credentials.

Where a mandatory property is unestablished, the selected release/certification claim remains unsatisfied even if a weaker conditional application is legitimately admissible under another declared profile. A genuinely valid in-profile program must not be rejected just to inflate the number of ”sound” denials. Positive construction, intended rejection, unresolved premises, bounded counterexamples and actual runtime enforcement are separate acceptance dimensions; none substitutes for the others.

Success requires both the expected rejections and these permissible constructions within their named profiles. The acceptance matrix is a minimum cross-layer conformance basis, not a closed enumeration of every valid Gesso program or a substitute for

general correctness arguments. The finished design is accepted only with consistent semantics, property-specific justified claims, explicit boundary premises, and nonvacuous support for meaningful application construction.

Part IV — Authoritative operations and progression

This part defines the public model boundary, authoritative construction and execution, command disposition, durable propagation, and scoped acquisition of authoritative state. It connects the assurance classifications of Part I, the identity and evidence model of Part II, and the operation contracts of Part III. Browser coordination follows in Part VI; whole-application closure follows in Part VII.

The central requirement is that the operation which an application admits, the transition which it submits, and the outcome which it reports describe the same semantic act. A well-typed declaration alone cannot establish that relationship. Neither can a successful HTTP response, a copied certificate, a local callback return, or an independently observed database state.

Ordinary full-language Clojure remains available under the checked construction contract. Gesso proves what its admitted semantics and evidence support, identifies what remains an assertion or environmental premise, and preserves those distinctions through execution. An opaque implementation is not silently treated as proved, and a restricted proof mechanism does not define the limits of the application language.

The declarations and API names below are illustrative specifications. They do not claim that the exact displayed functions are already implemented. Inspection maps explain admitted objects; constructing a similar map does not create an execution capability or authoritative evidence. Example identifiers denote separately bound fixture values unless a literal is supplied.

Model integration

Application models own domain truth. HumanHelp supplies the running examples, but Membership, Organization policy, and Request lifecycle rules remain HumanHelp semantics rather than special cases in Gesso.

A model may organize its implementation as follows:

Namespace	Responsibility
`request.domain`	Business predicates and pure transitions.
`request.schema`	Public value schemas and private persistence representations.
`request.graph`	Relationships and declared graph/query semantics.
`request.fx`	Acquisition, preparation, and authoritative effect realization.
`request.core`	Narrow public semantic interface.
`request.choreo`	Distributed programs consuming public model interfaces.

The layout is conventional; the dependency rule is normative. Public `core` operations may use their model's internals. Choreography consumes public `core` interfaces and does not reach through them into private domain, graph, schema, or FX implementation. A model's `core` does not facade its choreography.

For example:

```
(ns net.humanhelp.site.model.request.choreo
  (:require
   [gesso.choreo.core :as choreo]
   [net.humanhelp.site.model.request.core :as request]))

;; The authoritative participant invokes a public model operation.
;; trusted-context is supplied by that participant's admitted runtime.
(choreo/at :request-authority
  (request/claim! trusted-context claim-command))
```

This is not permission for the browser to supply `trusted-context`, nor for the choreography to call `request.fx` directly.

Placement and semantic ownership

The placement test is whether a rule still matters when the model operation is called synchronously from one JVM, without a browser or distributed protocol. Claim eligibility, current authorization, valid state transitions, command deduplication promised by the model, and atomicity normally pass that test. Browser optimism, role participation, transport retries, projected execution lifetime, and presentation settlement normally belong above the model.

Some information crosses this boundary because it has an independent model meaning. A stable command ID can be a public model input because durable replay requires it. An expected domain version can be a public precondition. Neither requires the model to know a tab's execution ID or HTMX swap policy.

Choreography may consume a public business judgment; it must not reimplement a slightly different Claim policy. A view may suppress a button as a convenience, but that suppression does not establish permission to commit. Conversely, business policy must not exist only inside choreography while `core` remains a pass-through to unconstrained writes.

Cross-model calls use public semantic interfaces. When Request depends on Membership and Organization, their models expose the necessary facts, predicates, and composable authoritative obligations through those interfaces. They need not expose private storage paths. A coordinating operation may then protect the combined obligations in one transaction.

A cross-model public interface must make its **semantic commitment** inspectable without disclosing the consulted model's private storage or implementation. For a condition needed by another model's authoritative transition, this includes its logical subject and parameters, relevant trusted principal/authority, declared read dependencies, and the point at which the condition must hold. An admitted implementation may supply a data-level obligation, a qualified guard constructor, or a trusted callback bound to that public contract. Merely naming `:membership/active` in an operation manifest does not establish that its actual guard is installed.

A successful preliminary Membership or Organization read is evidence of that observation at its qualified basis, not automatically proof that its predicate still holds when a later Request Claim commits. Where Claim promises atomic enforcement of combined eligibility, the authoritative operation must compose the relevant model-owned conditions into the **actual protected decision** using qualified transaction-current guards, assertions, or an equivalently justified backend mechanism. A stability premise

may replace such a guard only when separately admitted for the precise interval and scope; an unchecked preflight Boolean or caller-supplied ‘authorized¿ cannot. If the chosen backend cannot protect the promised combined condition, report the missing enforcement/correspondence obligation or weaken the **declared** atomic guarantee explicitly; do not silently accept a race.

Cross-model construction
Request Claim composes Request state, current Membership eligibility, and Organization policy at one admitted authoritative incorporatio
A prior Membership read returned `:active`, but Membership was revoked before the Claim commit
A browser or choreography sends `{:claim-allowed? true}` and the transaction uses that value in place of current model guards
A coordinating operation uses admitted public constraints from two models without importing either model's private FX or XTDB schema
The backend cannot enforce the promised joint invariant, but an assembly report infers safety from two separately successful reads

The rule and its source owner remain the same in a single application process or across several web nodes sharing one qualified logical authority. Multi-node routing and propagation can change which host learns about a change or when it can observe a sufficient basis; they cannot introduce a second Claim policy or substitute a stale host-local authorization cache for transaction-current enforcement.

A workflow spanning peer models should live at the level that owns the workflow. An onboarding flow coordinating Invitation, Membership, Organization, and external provisioning need not belong to Invitation merely because it starts there. Semantic dependency also does not dictate a literal namespace load order. Contracts may be represented separately to avoid circular implementation loads.

Public interfaces expose commitments, not implementation privilege

A public operation interface must expose enough information for a caller and the assembly checker to establish:

- the semantic operation and interface version;
- input and outcome schemas, including closed semantic discriminants;
- required authority and the source of the trusted principal binding;
- semantic preconditions and the point at which they must hold;
- promised atomic effects and explicitly separate later effects;
- command binding, replay, disclosure, and uncertainty policies;
- authoritative observation or progression evidence supplied with outcomes;
- named assumptions and the assurance status of each claimed guarantee.

It must not make callers depend on private XTDB forms, FX scratch values, or helper Vars. A synchronous REPL invocation remains meaningful when supplied an admitted context and authorized principal; bypassing HTTP does not bypass the model's contract.

Ordinary application code should receive scoped semantic capabilities rather than a raw database mutation handle. Clojure namespace privacy, records, and nominal wrappers do not sandbox a determined author in the same trusted process. The assurance claim therefore states its trusted-code and writer boundaries.

A bypass writer can invalidate assumptions used by later supported operations, not merely forfeit assurance for its own invocation. Coverage must account for jobs, migrations, imports, administrative tools, other services, and direct database access wherever their writes can affect a protected invariant. An admission policy may accept a named external-writer premise; it must not report that premise as established by observing only Gesso-issued calls.

Well-formed rules and appropriate policy are separate judgments

Where semantic types are known, malformed rules must fail admission. Comparing a closed status enum as though it were a number cannot become a certified rule. For ordinary Clojure predicates outside the analyzable vocabulary, the interface records their types, contracts, implementation binding, and asserted meaning. Merely invoking such a predicate does not prove its contract.

Neither type checking nor invariant preservation decides whether a business policy is desirable. Gesso can establish that the declared Claim policy is consistently enforced under stated premises. HumanHelp still chooses that policy. The executable semantic core and any later proof kernel/checker have different responsibilities, as established in Part I.

Core, FX, and choreography

FX realizes the authoritative commitments of a public model operation. It may acquire data, compute candidates, construct a protected transition, invoke a backend, and translate evidence into a typed outcome. It must not invent a second business semantics or a second distributed truth model.

One semantic operation can span a public `core` function, domain predicates, a registered transition constructor, an FX pipeline, and a backend adapter. Those are implementation components of the operation, not automatically separate commands. Their contracts must agree on inputs, authority, permitted effects, and outcomes.

The public model operation and the choreography invoking it are not required to have identical callable signatures or host representations. A REPL, scheduled job, installed HTTP handler, or another admitted Choreo realization may invoke the same `:request/claim` meaning under its own authorized entry contract. A shared keyword, copied interface map, route name, or successful transport return is not evidence that the installed callable implements that meaning. Application assembly binds the source-owned operation, selected trusted executor, actual guard/write coverage, outcome interpretation, and permitted disclosure to the same qualified semantic contract. Every path claiming to implement the operation must satisfy its declared boundary; passing through choreography is not a prerequisite for ordinary model use.

Conversely, a single command may involve several internal FX callbacks, backend statements, and asynchronous publication attempts. Those do not acquire independent authority to rewrite the business result or allocate a second domain command identity merely because they execute separately. A later notification worker has its own work/attempt identity and effect contract, but its work remains correlated to the authoritative command and retained publication intent under the selected protocol.

Choreography adds coordination obligations: which role calls the operation, which participants learn its outcome, what they may provisionally display, and how they recover when delivery is uncertain. Those obligations do not authorize Choreo to manufacture model outcomes or bypass current model guards.

A concrete public Claim contract

For this example, Claim moves an eligible open Request to its production `:claimed` state and creates the corresponding active **primary RequestAssignment** for the authenticated helper. RequestAssignment owns the helper relationship and its uniqueness/history semantics; a convenient `:request/helper` field must not become a competing source of truth. Collaborator assignments are not mistaken for a second primary helper. The helper must have current active Membership in the Request's Organization, and current Organization policy must permit the claim.

This worked operation explicitly selects a **durable work-recorded publication profile**: successful new incorporation records the command result, the required durable notification intent, and the protected Request plus primary assignment transition together. Another operation may lawfully choose an admitted best-effort profile, but it must publish a correspondingly weaker contract rather than silently claiming work-recording or guaranteed delivery. The durable intent is not evidence that a notifier received a message or that an external effect completed.

The public value schema is small:

```
(def ClaimCommand
  [:map {:closed true}
   [:request-id uuid?]
   [:command-id uuid?]])

(def valid-claim-command? (m/validator ClaimCommand))

(defn claim! [trusted-context command]
  (when-not (valid-claim-command? command)
    (throw (ex-info "Invalid Claim input"
                    {:gesso/error :model/invalid-input
                     :operation :request/claim})))
  (model-runtime/invoke-admitted!
   trusted-context :request/claim command))
```

Validation is an unconditional boundary check, not an optional assertion that a production configuration can disable. The example does not log the entire command or trusted context in an error. A concrete application may use a typed error return instead of an exception if its public interface declares that choice.

`invoke-admitted!` resolves the exact registered operation and its capability binding. It does not accept an arbitrary caller-provided guard/write map as an equivalent operation. The browser's wire field `:gesso/command-id` may map to the public model field

`:command-id` through an admitted boundary mapping. `:request-id` here denotes the HumanHelp domain Request; transport correlation uses `:http-request-id` or `:transport-request-id`.

An inspection view of the contract might be:

```
{:operation :request/claim
 :interface-version 1
 :arguments ClaimCommand
 :principal-source :trusted-runtime-context
 :command-policy :claim-command-policy-v1
 :publication-profile :durable-work-recorded
 :new-incorporation
 {:requires [:request/argument-target-matches
             :request/open
             :request-assignment/no-active-primary
             :membership/principal-and-organization-match
             :membership/active
             :organization/claim-permitted]
  :establishes [:request/claimed
                :request-assignment/active-primary-recorded
                :request-assignment/primary-uniqueness-preserved
                :command/result-recorded
                :publication/intent-recorded]}
 :replay
 {:requires [:command/binding-compatible
             :command/current-disclosure-permitted]
  :effect :return-permitted-recorded-outcome}}
```

These keywords are schematic **inspection labels**, not a second registry of facts or an executable guard-language API. The public RequestAssignment model owns what `no-active-primary`, `active-primary-recorded` and `primary-uniqueness-preserved` mean, including which other writers may alter assignment state. The application derives or supplies their qualified transaction-current realization through its admitted model/authority interface; copying this map cannot establish that the operation, installed handler, backend guard or durable worker exists.

A collection of reassuring labels is not a proof. In particular, new-incorporation conditions do not apply unchanged to replay: after a successful Claim the Request is claimed and has an active primary assignment, but a compatible authorized repetition must still be able to recover the historical recorded success. A later Unclaim or Reassign may change the current Request and assignment without changing that historical command outcome. Result disclosure still depends on current authorized policy.

Public Claim construction
An eligible authenticated helper claims an open, unassigned Request using a newly admitted command
A different collaborator assignment already exists, but the primary slot is eligible and empty
A client supplies a matching `:request-id` and a forged helper identity in a transport wrapper
A previous Claim committed and is replayed compatibly after the Request became claimed or was later reassigned
An installed handler invokes a similarly named helper or FX callback that omits assignment uniqueness or actual durable work recording
A best-effort notice is selected by a separately admitted contract

Model outcomes and invocation errors

The operation interface uses the following model outcomes:

Outcome	Required meaning
`:committed`	Qualified affirmative physical evidence attributes incorporation of the bound command to the reported admitted
`:already-incorporated`	A compatible retained command record establishes that the command was incorporated; this resolution path ret
`:rejected`	The declared authoritative rejection/closure policy establishes that the bound command is terminally rejected w
`:commit-unknown`	Available evidence does not establish a permitted definitive command disposition. The command may or may no

Outcome variants have closed discriminants and variant-specific payload schemas. A successful variant carries command binding and result evidence; a rejection carries its permitted reason and disposition evidence; an unknown variant carries the admitted reconciliation reference and uncertainty classification. Fields are not made optional merely to allow every variant through one loose map schema.

A model outcome, a physical **submission-attempt** result, the historical semantic command disposition, and a participant's **permitted knowledge** of that disposition are different subjects. A positive, correlated backend acknowledgment can justify an invocation-specific `:committed` result under the qualified adapter contract. A later read of the durable command record can establish incorporation of the command while leaving open which of two attempts performed it; that read supports the compatible recovered-result path, not invented evidence that the immediately preceding submission succeeded. Physical submission receipt, transaction acknowledgment, actual authoritative incorporation, result disclosure, Live delivery, and browser installation are not one terminal event.

An invocation-specific `:committed` result requires qualified physical evidence tying the **actual complete submitted effect**, its command binding, trusted principal/authority, and affirmative outcome to the reported submission or invocation. Co-observing a transaction key and retained command record at one compatible pinned basis can establish narrower facts; it does not by itself identify the attempt that caused the record. Abstract transition proof does not supply that missing physical causality.

Conversely, a compatible retained command record can justify the historical `:already-incorporated` recovery path without naming the winning attempt. Qualified binding-scoped terminal closure, not a failed guard or absent row, is required for stable `:rejected`. Otherwise a physically failed attempt, timeout, incomplete result or unestablished backend premise leaves the command unknown or supplies only a narrower typed attempt/invocation failure. These classifications cannot be strengthened by exchanging report-shaped maps.

An authoritative readback must preserve its **own** observation scope, basis, completeness and trusted origin. If a command record is not observed after a timeout, absence at one insufficient or stale read is not evidence of non-incorporation. If a command record **is** observed, matching the stable ID alone is insufficient: its full bound operation, authority, principal, canonical arguments and compatible interface must be checked before returning a recorded result. When the present principal is no longer authorized to learn it, disclosure can be unavailable despite known incorporation. An observation of a Request's present `:claimed` state cannot replace this bound command evidence.

Rejection has a similarly exact boundary. A failed authorization precheck may prevent **this entry attempt** from proceeding; an explicit backend assertion failure may establish that **this transaction attempt** did not incorporate. Neither necessarily proves that a concurrent compatible attempt did not or will not incorporate the semantic command. A terminal `:rejected` outcome requires its promised authoritative exclusion/closure evidence, or a specifically narrower declared rejection policy that does not masquerade as permanent command closure. Otherwise use the appropriate typed invocation failure, physical attempt result, or unresolved command disposition. Do not manufacture a durable rejection record from a transaction which definitely aborted without storing one.

Malformed inputs, unsupported interface versions, incompatible reuse, invalid credentials, disclosure denial, and backend qualification failure require typed handling. They are not all domain rejections. For example, an unauthorized caller must not be

able to create a terminal rejection record for somebody else's command. A disclosure denial must not falsely say that the command was never incorporated. Privacy policy may require an intentionally indistinguishable unavailable response for several underlying conditions. Likewise, a caught exception, a failed post-commit advisory, or a delivery timeout cannot be reclassified as a domain rejection. A known incorporated command remains incorporated despite failure in a separately owned publication/response stage; if the physical outcome is genuinely indeterminate, preserve unknown until qualified evidence resolves it.

The outcome recorded for replay is a historical operation result. A Request projection freshly acquired alongside it has a different meaning: it describes state at that acquisition's scope and basis. The API must distinguish these values. It must not imply that a recorded successful Claim guarantees the Request is still assigned to that helper now.

This distinction is also a **role-local branch knowledge** obligation. A continuation about whether the historical command C42 incorporated may use qualified, currently disclosable C42 result evidence if the authored choreography admits that acquisition path. A continuation about who is currently primary helper may instead use a scope-complete authorized Request/RequestAssignment observation at its own basis. These are different propositions, even when their rendered values happen to agree. A role requiring one cannot substitute the other, nor infer that another actor received an approval message merely because the authority recorded C42.

An alternative authoritative observation may satisfy the **knowledge premise** of a differing branch without a dedicated message. It does not automatically replace a globally required send/receive action, its acknowledgment, causal ordering, waiting semantics or failure path. The authored protocol must declare the alternative or a qualified refinement must preserve those effects. The local actor obtains the evidence before its first differing behavior, with current owner, subject, scope, source, permission, temporal and command/decision correlation. This joins the projection knowledge rule without introducing a second operation/outcome type system.

Presentation mapping is explicit:

Model evidence	Presentation settlement
Permitted committed success	`:accepted` for the bound presentation operation.
Permitted already-incorporated success	The same accepted successor, without a new incorporation.
Authoritative command rejection	`:rejected` through the declared rejection branch.
Commit unknown	`:unknown` until admissible reconciliation evidence resolves it.

A model may define successful results other than Claim success, and rejection details may be undisclosable. Assembly must check the actual variant mapping, disclosure requirements, and permitted presentation effects. A new command ID is not an acceptable way to disguise an unresolved retry. Presentation settlement additionally requires that the evidence be admissible to the **current** bound execution and resource owner. A correct model outcome cannot resurrect a retired browser execution, justify unrelated remote adoption, or discharge a required physical installation that never happened.

An explicit invocation sequence

The following logical phases may be implemented by one FX pipeline or several cooperating components. The semantics must survive either arrangement.

Phase	Responsibility
Admit	Validate inputs, operation/interface identity, trusted principal, and capability.
Resolve or prepare	Recover a compatible prior result, or acquire data and compute a candidate.
Incorporate	Protect semantic dependencies and atomically establish the domain transition, command record, and required durable
Propagate	Attempt publication or arrange durable recovery according to the operation contract.
Respond	Return only the outcome and observation evidence the invocation can justify and disclose.

Replay does not blindly rerun notifications or other effects. A recorded durable intent may remain unfinished and be resumed by its declared recovery owner, but it retains its original work identity. A compatible replay is not a new reason to create a second notification or charge.

Each phase may run on the same host or on a later compatible application node. The semantic command binding, trusted model rules, durable disposition and presentation/adoption policies do not change with host placement. Cross-host reads still need qualified sufficient progression; neither local cache agreement nor the fact that both hosts use the same logical database automatically establishes this.

Outcome and invocation acceptance cases

Case
A fresh Claim receives a qualified affirmative transaction acknowledgment bound to the actual submitted attempt
A Claim submission loses its acknowledgment, then a compatible command record is acquired authoritatively
Two concurrent physical attempts share one compatible command ID, and one has a confirmed assertion failure while the other incorporat
An input is malformed or the caller lacks an authenticated principal before any submission
A terminal command rejection is promised, but an abort did not record a binding-scoped exclusion and another attempt may still commit
A known incorporation is followed by a failed Live publish or missing browser settlement
An authorized read finds a compatible command record and a newer RequestAssignment state after later Reassign
A helper continuation explicitly permits learning C42's historical disposition from a qualified command-record observation
A choreography requires an authority-to-helper approval message and no read alternative or qualifying refinement was declared
An old command record is found but the current principal lacks permission to disclose its outcome
A replacement browser execution receives valid evidence about an old command
A host retries an unresolved command on another web node using the original immutable binding
Concurrent same-binding attempts A1 and A2 target C42; A1 aborts while A2 commits, and a qualified read finds C42.
A completed transaction key and command record are co-observed but the full-submission causal bridge is not established.
A terminal rejection fence and a pending incorporation attempt contend under the qualified command namespace.

Atomic authoritative transitions

An authoritative transition establishes the invariant-bearing effects of one semantic command at a declared atomic boundary. Preparation may be extensive: queries, calculations, validation, composition of model constraints, and construction of proposed writes can occur before that boundary. They do not become authoritative merely because preparation succeeded.

If preparation itself reserves stock, creates a lease, or persists an intent, that change is a separate authoritative transition with its own valid state and recovery obligations. It must not be hidden inside something described as a pure pre-read.

For each transition, the contract identifies:

1. the authority and semantic state being changed;
2. the operation, command, canonical arguments, and trusted principal binding;
3. the conditions that must hold at incorporation;
4. the permitted state change and what must remain unchanged;
5. the command-result and durable-work effects included in the same boundary;
6. the backend mechanism protecting these commitments;
7. the evidence that distinguishes incorporation, rejection, and uncertainty.

The required protection concerns semantic dependencies, not automatically every field read during preparation. An unused display name need not become a commit guard. Conversely, absence, uniqueness, aggregate limits, and relationships may be semantic dependencies even when no existing document contains the fact to guard. Protecting only returned rows cannot establish that no matching row was inserted concurrently.

Atomic writes alone are insufficient. A backend can atomically apply writes derived from stale or incompatible facts. The qualification must establish that the required conditions hold at the operation's declared incorporation point, using an appropriate transaction isolation, conditional mutation, transaction function, constraint, or equivalent qualified mechanism.

Semantic rules become implementation obligations

Every rule used to justify admission or a safety claim must have an explicit path to its authoritative realization. The path identifies the semantic predicate, bound values, mutable dependencies, protection mechanism, and evidence status. It also identifies the frame: effects outside the declared change are forbidden unless separately specified.

For Claim, the obligation set is:

Semantic obligation	Binding or protection required
The requested Request is the acquired target.	Equality between the command's Request ID and the acquired a
The Request is open and has no active primary RequestAssignment.	Protect the current lifecycle state and the absence of an active p
Membership belongs to this helper and Organization.	Principal, Membership, Request, and Organization associations
Membership is active.	Current eligibility protected at incorporation under the policy's
Organization permits the claim.	The relevant policy and any inputs it depends on protected at in
Request state and primary assignment are the permitted domain change.	The helper is derived from the trusted principal binding; the tra
The command is not incorporated twice.	Atomic command-binding compatibility and uniqueness within
Success can be recovered and propagated.	The compatible command result and the work intent required b

Evidence may establish some of these obligations by construction, others by a checked derivation, others by runtime checks, and others only under accepted backend or opaque-code premises. The report must preserve those classifications. The word *checked* must identify what was checked; it must not silently include all unexamined behavior beneath that check.

The correspondence is **bidirectional**. Every correctness-relevant source-owned semantic rule has a traceable protection, construction, derivation or explicitly accepted premise in the effective authoritative operation. Conversely, an FX, model, backend or installed callback whose condition changes whether the command is admitted, which domain effect is written, who may cause it, or what protected result is disclosed must correspond to a declared rule or named opaque/physical premise. An incidental query optimization, retry, logging branch or diagnostic does not become business policy merely by existing. An unreported extra policy guard that silently rejects valid declared business operations is as much a correspondence problem as a missing required guard. Gesso may impose a separate **declared** platform safety/admission policy; it must not claim the narrower implementation is the complete realization of a more permissive business contract without identifying that policy.

The active-primary absence and uniqueness rules deserve explicit care. Checking a Request projection, an empty previously read assignment collection or a single row's version does not necessarily prevent another writer from inserting a primary assignment before incorporation. The actual backend mechanism must protect the **predicate over the relevant set**, with its qualified isolation, uniqueness/constraint, transaction assertion or other justified serialization contract. A reference-model search result or syntactically correct `ASSERT` is not, by itself, proof of that physical concurrency property. If the selected backend, writer inventory or precondition-coverage profile cannot establish it, retain the missing correspondence or supported-premise obligation rather than certifying primary uniqueness. A valid operation under a stronger backend/profile is not rejected merely because a narrower proof mechanism cannot analyze it.

Canonical construction and physical submission

The default managed path derives a transition from the admitted operation's canonical semantics and bound inputs. It does not ask application code to independently maintain three descriptions of the same effect: a business rule, a reassuring guard map, and an unconstrained transaction vector.

The derived or checked construction binds at least:

- the operation/interface and semantic-definition identities;
- the exact admitted implementation and relevant adapter/profile identities;
- typed argument, principal, authority, and target bindings;
- acquired input provenance and any required acquisition capabilities;
- protected predicates, permitted writes, and frame obligations;
- command-result and durable-work effects;
- the invocation and physical submission to which resulting evidence belongs.

The execution gate must account for the complete submitted transaction, not just a selected domain fragment. Command ledger writes, outbox entries, creates, updates, deletes, transaction options, and any effective backend-side hooks must be included in the checked effect boundary or covered by an explicit accepted premise. An extra write or a weakened guard cannot be ignored because the original domain fragment still matches a certificate.

For the chosen durable Claim contract, the Request lifecycle change, new primary RequestAssignment, binding-compatible command outcome, and retained publication work must form **one protected authoritative effect**. A later notification send is independently owned work, not a fifth in-transaction provider delivery. Neither the number of public model interfaces used nor the number of FX callbacks determines the number of transactions. An adapter must expose enough of its effective guard/read and mutation footprint for its selected assurance profile to judge the whole operation. An uninspected transaction hook or extra writer is an explicit open premise or a rejection under a closed coverage policy, not evidence silently erased from the frame.

An inspection report is not the executable capability. The runtime must prevent ordinary callers from substituting a new transaction, changing a target, replacing the executor, or attaching a report from an unrelated successful call. The result must be linked to the actual issued invocation, submitted material, and qualified backend return path. Structural equality with expected output is not evidence of that provenance.

This judgment covers the **effective** submission after FX composition, conditional branches, error handlers, backend option selection and installed executor dispatch, not only an earlier approved plan fragment. A check performed before a permitted dynamic branch must either cover every realizable effect variant or be enforced again at the actual submission boundary. A normal successful Claim path does not thereby credential a fallback handler that emits a Request-only update, nor an unrelated worker entry that accepts a caller-supplied transaction vector. For each declared variant, the checker distinguishes legitimate model effects, additional effectful operations with their own public contracts, and opaque implementation premises.

The **installed-path relationship** is separate from correctness of an individual submitted vector. Part VII owns whole-application closure: the served route or other real ingress must invoke the selected trusted executor with the admitted operation/authority/command binding. Part IV's stronger operation-family claim applies to that physical entry only under a checked installed binding or explicit accepted premise; the existence of a similarly named route, a valid standalone gateway or a copied report cannot establish it.

Different construction mechanisms can support different subsets. A proof for one scalar document update does not cover nested values, nullable fields, inserts, deletes, multi-document transitions, or arbitrary transaction programs unless its proposition says so. Those operations remain available through broader checked mechanisms or explicitly asserted contracts. They must not be quietly forced into the narrow proof's vocabulary.

Atomic-construction acceptance and counterexamples

Construction
An authorized Claim closes the current Request, Membership, Organization and active-primary predicates in a qualified atomic backend tra
A helper has a collaborator assignment but no active primary assignment, and the public model permits Claim
A candidate observes no active primary assignment, but another command inserts one before this Claim's unprotected write
Membership is revoked after a candidate read and before the actual transaction; the implementation submits only the prepared Request upd
An FX callback silently adds an otherwise legitimate-looking Organization eligibility condition absent from the declared product rule
A statement writes the Request and primary assignment, but the durable-work record promised by this selected profile is written only in a
The effective submitted transaction contains an unapproved write or omits a required command/result guard, while a domain-only fragmer
An installed Claim route invokes the admitted trusted executor and its complete guarded Request/RequestAssignment/command/work subi
A dynamic FX fallback submits a different command vector from the one checked before invocation
The selected physical adapter cannot establish isolation for the absence/uniqueness predicate, although the abstract operation is otherwise
A pure Clojure constructor with an admitted typed/effect interface produces the correct multi-document plan but its body is outside the sel

Representation and frame fidelity

The mapping between semantic fields and physical storage must be explicit and checked for the supported representation. Distinct semantic fields must not silently collide on one physical field. Required fields, system metadata, nested paths, deletion, and field ownership need declared treatment.

Presence matters. A missing field, a field with `nil`, a field with `false`, and an empty collection are not interchangeable unless the model explicitly makes them equivalent. A proof that depends on a complete acquired entity cannot use a partial projection or unqueried relationship whose absence is interpreted as unclaimed. HumanHelp's primary helper relation belongs to the public RequestAssignment model, not to a nullable Request helper field.

For RequestAssignment, the **absence of an active primary assignment** requires a scope-complete, authorized and provenance-qualified query over the relevant relationship. An empty page, a missing field, an incomplete query, a failed read, and a complete empty result are different. Even a complete observed absence does not itself prevent a concurrent insertion before incorporation. The selected backend must independently protect the primary-absence/uniqueness predicate at the authoritative transition under a qualified atomicity and writer-coverage contract.

Whole-document replacement must preserve the declared frame despite concurrent changes. Comparing a selected subset of fields and then replacing an entire stale document can overwrite unrelated updates. Valid solutions include a qualified full-state guard, an appropriately maintained revision guard, or a backend update whose semantics preserve untouched fields. The chosen solution must actually support the claimed frame.

A revision field is sufficient only under a premise that every relevant writer maintains it and that its comparison has the required semantics. An equal before/after value does not prove that no intermediate change occurred. History or non-repetition properties require stronger evidence than a state predicate; ordinary state invariants do not universally require an ABA-prevention mechanism.

Capability lifetime and command lifetime

Execution capabilities are scoped to the admitted runtime and operation. Their contract states whether they may be used once, reused, transferred to an asynchronous worker, or retained beyond a request. One-shot consumption must be atomic with respect to competing local users of that capability.

A one-shot capability prevents reuse of that capability instance. It does not prevent another request from obtaining a different capability for the same semantic command. Durable command deduplication still belongs to the command protocol. Similarly, a thread ID, request path, or callback identity does not replace the semantic command binding.

Retiring a request-local capability prevents future use according to its contract; it does not roll back an already submitted transaction. Transferring work to a durable worker requires an admitted transfer and a new appropriate capability, not accidental retention of a web request's ambient authority. The worker admission binds its retained work/intent identity, source-authorized operation or continuation, current policy and permitted effect class. A worker may be scheduled, retried, moved to another node or awakened by a provider callback without owning an HTTP route; it still needs the admitted runtime effect boundary and complete backend/writer-coverage classification. The fact that the original request once held a legitimate capability does not credential a replacement process, its installed worker handler or any additional transaction that worker might submit.

A copied inspection report, structural admission handle, command record, or caller-authored map is not a runtime mutation capability. The issued capability must bind its qualified issuer, authorized actor/authority, operation, allowed effects, target and

resource lifecycle as required by the runtime contract. A replacement web node may obtain a newly issued capability while preserving the **same semantic command** and current model policy; an in-process handle or retired request-local authority is not made transferable by serialization.

A complete Claim preparation example

The following pure function illustrates **candidate computation** rather than a required concrete Gesso API. It assumes that the principal, Request, Membership and Organization have passed their public contracts, and a public RequestAssignment acquisition supplied the separately validated ‘assignment-observation‘. This observation represents a complete, scoped collection of active **primary** assignments for the exact Request; it does not count collaborator assignments as primary owners. Its query meaning, source provenance, disclosure, compatible basis and complete coverage must already have been established independently by the model-owned acquisition contract.

The illustrative ‘:observation/*‘ keys expose these obligations, but their presence in arbitrary caller data does not prove them. The function neither commits, authorizes a backend write nor resolves durable command replay.

```
(defn claim-candidate-decision
  [command principal request membership organization assignment-observation]
  (let [active-primary (:observation/primary-assignments
                        assignment-observation)]
    (cond
      (not= (:request-id command) (:request/id request))
      {:candidate/status :invalid :reason :target-mismatch}

      (not (and (= :complete (:observation/coverage assignment-observation))
                (= (:request/id request)
                   (:observation/request-id assignment-observation))
                (vector? active-primary)))
      {:candidate/status :insufficient-observation
       :reason :primary-assignment-coverage-unestablished}

      (> (count active-primary) 1)
      {:candidate/status :invalid-observed-state
       :reason :multiple-active-primary-assignments}

      (not= :open (:request/status request))
      {:candidate/status :ineligible :reason :request-not-open}

      (seq active-primary)
      {:candidate/status :ineligible :reason :already-has-primary-assignment}

      (not= (:principal/id principal) (:membership/user membership))
      {:candidate/status :ineligible :reason :membership-principal-mismatch}

      (not= (:request/organization request)
            (:membership/organization membership)
            (:organization/id organization))
      {:candidate/status :ineligible :reason :organization-mismatch}

      (not= :active (:membership/status membership))
      {:candidate/status :ineligible :reason :inactive-membership}

      (not (true? (:organization/claims-enabled? organization)))
      {:candidate/status :ineligible :reason :claims-disabled}

      :else
      {:candidate/status :eligible
       :request-id (:request/id request)
       :helper-id (:principal/id principal)})))
```

This simplified policy is illustrative; a richer Organization predicate, Location relationship or separately permitted manager-

selected target-helper variant has its own declared dependencies. Trusted authentication establishes the actor, not current Membership or permission to select a different helper.

An `:insufficient-observation` result does not claim there is no helper; `:invalid-observed-state` identifies inconsistent acquired data, not a terminal command rejection. A complete collaborator-only collection is not an active-primary assignment. A terminal Request with no primary owner is not thereby reopened. Independently acquired model facts must also satisfy the coherence relation required by the chosen candidate policy; matching IDs alone is not a common snapshot.

The managed operation binds this candidate to canonical transition construction. The authoritative realization protects the target, eligibility, associations, absence/uniqueness of the active primary assignment, and full frame together with the Request lifecycle change, created primary RequestAssignment, bound command outcome and retained work required by this Claim's selected durable publication profile. It must protect competing relationship insertions, not merely compare the Request revision. A caller gains no authority by returning `{:candidate/status :eligible}` or by inventing `{:observation/coverage :complete}`.

A candidate rejection from a pre-read is not automatically the final command outcome. A concurrent attempt might already have incorporated the same command. The command protocol below resolves that case before it reports authoritative rejection. Conversely, a positive candidate must be revalidated or protected at incorporation; it cannot survive arbitrary Membership or policy changes merely because preparation succeeded.

Interleaving	Required result
Membership is deactivated after preparation, before incorporation.	The new Claim cannot
Request moves to another Organization before incorporation.	The stale association c
Another helper inserts an active primary RequestAssignment without updating the previously read Request field.	The protected current
A primary-assignment query returns an empty partial page or was scoped to another Request.	Preserve insufficient ac
A collaborator exists but there is no active primary assignment and the Request is open.	The collaborator is not
A Request is terminal but has no active primary assignment.	Lifecycle state indepen
The same command already succeeded but its response was lost.	A compatible authorize
This physical attempt aborts while another attempt for the same command commits.	The command is incor

API boundaries do not force transaction boundaries

Models may expose composable constraints and effect constructors through public interfaces, allowing one coordinator to establish a cross-model invariant in a single atomic transition. Combining those components must check shared target bindings, incompatible writes, frame overlap, and the conjunction of their requirements. Concatenating independently prepared write lists is insufficient.

If the participating authorities cannot share the required atomic boundary, the design must name valid intermediate states and a recovery protocol. A sequence of individually atomic calls is not an atomic cross-model operation merely because it appears in one FX vector or choreography step.

HumanHelp Claim's Request, RequestAssignment, command result and selected durable publication intent share the protected authoritative transition, including current Membership/Organization dependencies. The public models expose semantic constraints without handing Choreo their private XTDB forms. A backend that cannot establish this multi-entity boundary, including the absence of competing primary assignments, must disclose the open physical premise or reject the stronger claimed correspondence.

Representation and capability acceptance cases

Positive or adverse construction	Required
An authenticated eligible helper has a complete, trusted, scoped empty active-primary observation for an open Request	Candidate
An empty assignment page is incomplete or filtered by a different visibility/query scope	Do not cl
A collaborator exists but is not the active primary owner	Do not re
A Request projection lacks a helper field while a current active primary RequestAssignment exists	Public Re
Membership becomes inactive or another primary assignment is inserted after preparation	Recheck/
A full-document replacement preserves checked fields but erases another writer's unrelated edit	Reject fra
An old request-local capability or a copied structural admission map is sent to another host	It grants
Concurrent local users attempt to consume the same one-shot capability	At most o
A retained work item is processed on another host after its original request capability retired	Admit on
A background provider-result incorporation uses a source-bound worker entry and freshly authorized guarded local operation	Permit th

Command identity and authoritative disposition

A semantic command identifies one intended **semantic action occurrence** under a declared command policy. It survives physical retry and authorized reconciliation. It is distinct from the operation definition, any particular server invocation, choreography execution, backend submission attempt and transport exchange. One command can have several physical attempts without becoming several admitted business actions.

Identity	Meaning and lifetime
Operation identity	The public semantic operation, such as `:request/claim`.
Operation interface version	The input, result, and behavioral contract being invoked.
Semantic command ID	One bound intent whose disposition can be deduplicated and recovered.
Choreography execution ID	One distributed execution and its protocol-local resources.
Transport request ID	One HTTP exchange or other delivery attempt.
Backend submission-attempt ID	One physical attempt to submit or execute a transaction.
Capability instance ID	One issued execution or acquisition right.
Resource instance ID	One lifecycle instance of a managed resource or installation target.
Principal-context identity	The authenticated context relevant to authority and disclosure.

Other identities, including deployment and projected-plan compatibility, may be needed. They must be named for their purpose; a bare universal `generation` coordinate must not stand in for them all.

A nominal identity wrapper or distinct wire tag is useful for preventing accidental interchange of `:command-id`, `:execution-id` and `:transport-request-id`. Its bytes do not authenticate an actor or prove a durable command exists. A new transport exchange or compatible projected-plan execution can carry the same command ID; it cannot reuse a retired execution's resource/installation authority. Conversely, preserving an ID across a deployment does not automatically make an incompatible old projected plan safe to resume. Protocol continuation and stored command disposition are separately admitted.

The principal, role, actor, host, command and resource identities are not one interchangeable credential. Two tabs belonging to one authenticated principal may have distinct branch knowledge, board observations, acquisition grants and installation owners. A semantic command can survive host/actor replacement; the successor still needs its own trusted right and source-qualified result before it may continue. Matching command bytes, session, role or transport identifiers do not transfer a retired execution's capability or establish that a different actor received its message.

Command binding and canonical inputs

The admitted command boundary supplies or admits a stable `:gesso/command-id` before the first possible send. A browser may carry an identifier, but choosing its bytes confers no permission. The trusted boundary establishes the binding and must preserve it through retransmission and reconciliation.

The command policy defines the namespace in which IDs are unique and the immutable binding compared on repetition. That binding normally includes:

- authority/tenant and command namespace;
- semantic operation and compatible interface identity;
- canonical semantic arguments and their canonicalization version;
- the principal or explicitly declared delegated authority responsible for the intent;
- any domain preconditions that are part of the intent, such as an expected entity version.

Command compatibility is not equality of an arbitrary serialized map. Defaults, omitted fields, numeric representation, ordered versus unordered collections, and schema evolution require defined canonicalization. If a digest stands in for canonical arguments, the contract identifies its algorithm and accepted collision premise; a short convenience hash is not automatically an adequate binding.

The public command namespace and the physical lookup/uniqueness key must correspond. A model can choose globally unique IDs or IDs local to an authority/tenant namespace, but two distinct admitted command identities must not silently alias the same durable record. Conversely, a second tenant or principal must not gain a foreign outcome by guessing an ID. Record identity, full immutable binding, retained disposition and current disclosure policy have distinct jobs: the record key locates a candidate; it does not authenticate the caller or establish compatibility. A canonicalization or interface upgrade must either preserve the admitted old binding meaning through a declared compatibility relation or keep the old command unresolved/unsupported with its own recovery; reinterpreting a retained ID as a new intent is forbidden.

An observed read basis used only as a freshness hint need not be an immutable semantic argument. An expected version that the user intentionally conditions the command on is an argument. The interface must distinguish the two. Retry must not silently turn a conditional command into an unconditional one.

Server-generated timestamps, random identifiers, and other choices also need declared semantics. Values defining the user's intent or the identity of a created entity must be stabilized or recoverable as the policy requires. Transaction-local candidate calculations may be recomputed when permitted, but their variation cannot change the command binding or invalidate replay.

Command identity is not an idempotency policy

Attaching a UUID does not provide deduplication. An operation must declare its incorporation policy, storage boundary, compatible-repetition behavior, incompatible-reuse handling, retention horizon, and recovery mechanism.

For Claim, the intended policy is one incorporation for a compatible command binding. Repetition after incorporation returns a permitted recorded outcome. Distinct command IDs can still conflict on the same Request; command deduplication does not replace domain concurrency control.

A fresh Claim must protect **two independent exclusions**: the command namespace cannot incorporate the same bound intent twice, and the Request cannot acquire two concurrently active primary RequestAssignments. Command uniqueness does not prove primary-assignment uniqueness; checking only the Request's current revision or a node-local command cache does not protect either property in all implementations. The Request, admitted primary-assignment change, command record and required durable publication intent of the selected Claim profile are one protected effect. Every actual writer capable of violating a relied-upon rule must be included in the qualified writer/enforcement coverage or treated as an explicit premise.

An operation that intentionally permits repeated independent effects must say what constitutes a new semantic command. A new delivery attempt is not, by itself, a new intent. If the product supports repeating an action, the admitted entry path issues a fresh command for that deliberate action.

The command store need not contain an in-flight row for every implementation. Its protocol must nevertheless distinguish evidence of incorporation, evidence of terminal rejection or closure when promised, and the absence of a definitive result. A transient local registry may assist scheduling; it cannot replace a durable record where process failure recovery is promised.

The resolution boundary also needs a truthful **acquisition classification**. A qualified, complete read finding no command record is an observation of absence at its basis, not an exclusion of a concurrent or pending submission. An unavailable, stale, unqualified, wrong-namespace or undisclosable lookup is not such a read. It cannot safely authorize fresh incorporation or be recast as a public `not-found` existence oracle. Existing-record disclosure and new-command permission are distinct policy checks: a compatible replay need not satisfy the now-false preconditions for **new** incorporation, while any returned historical result still requires current permission to learn it.

Atomic resolve-or-incorporate protocol

The following is a semantic algorithm, not a requirement to implement one giant callback in the database:

1. Validate the admitted operation, interface, canonical arguments, authority scope, and trusted principal. Establish which command binding this caller is permitted to invoke or resolve.
2. If a retained command record is found, check binding compatibility and current permission to disclose the result. Return the permitted recorded disposition without applying the operation again. An incompatible record cannot be overwritten or treated as a new command.
3. If no definitive record is established, protect command uniqueness together with the conditions for new incorporation. An earlier non-transactional absence check is only a hint.
4. At the authoritative boundary, either establish the permitted domain transition, command result, and required durable intents together, or do not establish that transition. Conflicting concurrent attempts must not each incorporate the same command.
5. Interpret the backend result according to its qualified contract. A local return that means submitted or queued must not be translated into committed.
6. On contention, an abort, or an ambiguous return, resolve whether another attempt established a command disposition. Return only what the admitted evidence supports. If the result remains indeterminate, preserve unknown.

The implementation may use retries internally. Such retries retain the semantic command and must not execute unrepeatable external effects inside a callback that the backend may rerun.

The preliminary lookup in step 2 is an optimization, not an atomic absence guarantee. A preliminary record read or candidate ineligibility judgment can select an attempt's path, but does not settle command disposition on its own. The protected incorporation/exclusion mechanism and qualified backend outcome interpretation own that stronger judgment. Compatible

replay must resolve retained results without rerunning mutable eligibility guards for **new** incorporation. Concurrent compatible invocations may both observe no record and prepare candidate transitions. The qualified authoritative mechanism must ensure that at most the policy-permitted incorporation wins for the command namespace; a loser resolves the surviving record under its own trusted disclosure conditions, rather than reporting terminal business rejection solely because its physical transaction aborted. An incompatible competing binding must not overwrite the winner's record or enter the old command's authorized replay branch. A distinct command ID cannot avoid the independent RequestAssignment concurrency rules. A transaction that committed its domain effect without the required command record is not a valid realization of this policy.

The same algorithm applies to one or several application nodes sharing the qualified logical database. On another node, the command retains its canonical identity and public semantics even if its local execution, transport request and capability instances differ. A local registry, cached negative lookup or lagging reader is never the authority for deduplication; any replay or new submission must use a sufficient authoritative acquisition/protected incorporation path. Cross-node router placement and Live invalidation delivery do not define or strengthen the command's business identity.

Command identity and incorporation acceptance cases

Construction
An eligible helper sends one Claim and the transaction protects Request, primary RequestAssignment, command result and selected durabl
The same compatible command is retried after a lost response or on another web node
Two same-binding attempts both read an absent command record before one commits
A concurrent attempt reuses one command ID with different target, principal, tenant or canonical arguments
Two tenants use IDs valid in their separately declared namespaces
Two different command IDs attempt primary Claim for the same open Request
A stale follower or node-local cache sees no command record despite an in-flight or incorporated transaction
A successful Claim is replayed after its helper loses Membership or after a later Reassign
Two browser tabs of the same principal know C42 but have different observed board frontiers and owners.
An old record exists but the caller cannot disclose it or invokes an incompatible projected plan
A canonicalizer upgrade changes argument meaning while an old command record remains retained
A command is durably recorded only after its domain write, or a direct writer bypasses protected uniqueness

Rejection must name its scope

There are three distinct negative facts:

- preparation found a currently ineligible candidate;
- one physical transaction attempt definitely did not incorporate;
- the semantic command is authoritatively rejected under its declared policy.

Only the last justifies the model outcome `:rejected` as a definitive command settlement. A successful competing attempt can coexist with either of the first two facts.

If the operation promises a stable terminal rejection, its mechanism must close the command against pending and future compatible incorporation. One solution is an atomic terminal rejection record which contends with the incorporation record; another is a qualified equivalent closure or fencing protocol. A mere lookup that finds no record cannot provide this guarantee while a previously submitted attempt could still commit.

Such closure needs a **linearization or exclusion contract**: the operation must identify the authoritative command namespace/binding, the physical mechanism by which an incorporation and a terminal rejection mutually exclude each other, and the point at which the participant can justifiably learn closure. A rejection fence that blocks only later entries at one web node does not exclude an older pending submission at another node. Nor does an absent record read at basis B necessarily exclude an already submitted transaction that becomes visible at a later basis. If a model instead promises a weaker **attempt-local** or **observed-at-basis** denial, it must use a distinct typed outcome/diagnostic and not pretend to have closed the semantic command.

A permanently denied invocation may be invalid before any authoritative submission—for example, an untrusted caller with no permission to originate the command. That denial does not give the caller the authority to create a durable terminal rejection for another principal's otherwise valid command. A policy can intentionally settle a command as rejected, but the retained binding and exclusion mechanism must establish that stronger fact under the policy's specified lifetime.

A rejected database transaction generally cannot also be assumed to have persisted its own rejection record. If closure requires a subsequent transaction, that transaction has its own races and possible uncertainty. It must resolve against any competing incorporation before the model reports a definitive rejected command.

This does not require every backend failure or bad request to create a durable terminal record. Interfaces may expose typed invocation failures or unresolved attempt diagnostics without claiming terminal command rejection. An operation with a different rejection lifetime must make that scope explicit and provide a presentation mapping that does not falsely promise finality.

In particular, the rejection of a transaction because its RequestAssignment or Membership guard failed is evidence about **that attempted submission**. It need not be a retained command result. Another compatible attempt may have incorporated the command, or the application may legitimately allow a later new invocation under a different command ID after conditions change. A confirmed failed attempt must not manufacture a terminal rejected command row in the transaction that aborted, and an unknown return must not be reclassified as an explicit assertion failure by the caller.

A fresh intentional action after rejection may use a new command ID. Automatic retry after an unknown result must not do so merely to escape the unresolved record. An unauthorized or malformed invocation cannot close another party's otherwise valid command.

Retention, upgrades, and expiry

Deduplication guarantees have a declared horizon. Deleting a result while old messages can still be accepted may permit reintegration of a previously incorporated command. The design must choose a coherent policy: retain the necessary record or tombstone, reject commands outside an admitted validity window, retire the command namespace with a protected fence, or provide an equivalent mechanism.

Result disclosure retention can differ from incorporation-prevention retention. A compact tombstone may prevent duplicate effects without retaining sensitive response data. In that case the public resolver reports only the information the tombstone and disclosure policy justify; it does not fabricate the original payload.

The retention policy must cover **delayed already-submitted attempts** as well as new incoming retries. Expiring a result and deleting its prevention key does not make an in-flight callback, durable queue entry, offline browser command, or paused old worker harmless. To reuse or retire the namespace safely, the authority needs an enforceable validity window, epoch/fence, retained exclusion record, or an equivalently justified policy preventing an old attempt from incorporating after its original evidence expires. A wall-clock TTL checked only by a web handler is not sufficient if the authoritative backend can still accept an older prepared transaction.

At expiry, the API must distinguish **cannot disclose the previous outcome**, **cannot establish the previous disposition**, and **the old identity is authoritatively barred from incorporation**. The latter may be known even when the former outcome payload has been erased. When the command is still unknown and old attempts are not safely excluded, a new ID is a new semantic operation, not an innocuous retry or proof that the old operation failed. Data-minimizing result erasure and durable duplicate prevention must be reconciled without silently weakening either promise.

Interface upgrades must specify compatibility for retained commands. Changing argument canonicalization or the meaning of a policy version cannot silently rebind an old ID. A database restore, tenant migration, or authority fork must also define which command namespace and records remain valid. Backend position numbers alone do not preserve those identities.

A migration must preserve or explicitly retire the authoritative command exclusion **and** the meaning of its bound operation, principal, authority scope, canonicalization version, and result-disclosure rule. Merely copying the latest domain Request and RequestAssignment documents does not copy historical command disposition or outstanding publication work. A restored database missing newer command records cannot advertise those commands as safely absent unless its deployment/recovery protocol separately fences the previously accepted attempts. An incompatible new implementation may refuse a continuation, but it must not revive or duplicate an old effect by calling the retained command a fresh one.

Command disposition and participant knowledge

The command's authoritative disposition and a participant's knowledge are different state components. The backend may have committed while the caller still knows only that delivery failed. `:commit-unknown` describes this missing knowledge; it does not assert a mysterious third kind of database commit.

Unknown must be preserved when the available evidence cannot distinguish incorporation from non-incorporation. A timeout, disconnected SSE channel, retired browser execution, failed response serialization, or exhausted retry budget cannot turn that uncertainty into rejection.

Definitive knowledge is also subject to authority. A runtime may know that a record exists while the current participant is not permitted to learn its contents. The public interface must represent unavailable disclosure without inventing a domain failure or leaking the existence of protected information.

The authoritative domain state, bound command disposition, and participant knowledge/disclosure are separate coordinates. An allowed current-state read may repair a Request card without proving which command produced it. A permitted command record may prove historical Claim success after later Reassign without proving who is primary **now**. A nondisclosable command record remains an authoritative fact while the public participant receives only its permitted unavailable response. A browser's retired execution can neither reinterpret that unavailable result as rejected nor regain rights to settle the old presentation target.

Knowledge must cite its evidence subject and scope: command binding and namespace, authority and trusted read source, observation basis/coverage, winning attempt if attribution is claimed, retention or closure policy, and current disclosure entitlement. A response does not acquire a stronger judgment because it moved through several HTTP nodes, reconciliation callbacks or Live messages. Different hosts can have different knowledge while sharing one logical authority; they may not apply different business semantics for the command.

Rejection, retention, and knowledge acceptance cases

Construction	Re
A terminal rejection transaction and a compatible incorporation contend under one admitted command exclusion/closure mechanism	At
A preparation check finds Membership inactive, with no authoritative command attempt and no retained rejection	Re
One XTDB assertion fails, but another compatible physical attempt records the bound Claim	Pr
An explicit rejection record is written in a second transaction after the first transaction aborted	Re
A command result payload is erased but an admitted tombstone/fence still prevents the old ID from incorporating	Pe
A delayed offline or queued retry arrives after deletion of its record and before any enforceable expiry fence	Re
An authenticated, deliberately new command is admitted after an expired prior window that safely bars the old command	Pe
A database backup is restored without a command record for an already executed Claim while old attempts may still arrive	Do
An operation/interface upgrade changes canonical argument meaning while an old command remains unresolved	En
A caller may read the current Request but lacks permission to see the historical command record	Pe
A later web node has a stale negative lookup but another node has a qualified retained incorporation result	Tr
A retired browser execution receives late, valid command-specific success evidence	Pr

Worked example: Claim committed, response lost

Let `R` be the Request ID, not an HTTP request ID. Let `C42` be a stable semantic command ID, not a choreography execution, transport attempt, or database transaction ID. The domain argument and the wire command envelope are distinct:

```
(def C42 #uuid "f1bf42ee-7965-4a53-905b-c59038f75e42")

(def claim-command
  {:request-id R
   :command-id C42})

;; Wire representation at the admitted browser boundary.
(def claim-wire-arguments
  {:request-id R
   :gesso/command-id C42})

;; Alice's authenticated identity is NOT a client-authoritative field.
;; The public operation admits/binds it from trusted runtime context.
```

Alice's original choreography execution is `E1`. The original HTTP delivery is a distinct transport request `T1`, admitted at a server under Alice's trusted principal. The selected Claim contract is the durable work-recorded profile from section 04.02. Its one protected authoritative transition moves Request R to `:claimed`, creates Alice's **active primary RequestAssignment** (with primary uniqueness protected independently of command uniqueness), records C42's complete compatible binding and success result, and records the required durable publication intent. The transaction incorporates. The HTTP response conveying this fact is then lost.

The following table uses an omniscient account of the example to distinguish what **actually happened** from evidence available to each participant. Alice has not yet learned that incorporation succeeded.

Subject	Established fact or permitted knowledge
Authoritative command record	C42 incorporated under its bound principal, operation/interface, Request and canonical argun
Current domain at the incorporation point	Request R became `:claimed` and its permitted active primary RequestAssignment identified A
Durable publication work	The required work intent was recorded atomically; no notifier receipt, external delivery, brow
Physical transport T1	Its response was lost after incorporation; T1's delivery failure did not undo the command.
Original browser execution E1	It has no admissible success evidence; its command presentation must remain unknown until
Current browser owner	It may be E1 or a successor with distinct resource-instance rights; no old callback may claim a

An authorized current projection of Request R together with its active primary RequestAssignment may establish that Alice is the currently assigned primary helper at the read's actual basis and scope. It does **not** attribute that relationship to C42. A different command could have produced the same state, or C42 could have committed and then been followed by Unclaim, Take-over or Reassign. A Request-only projection is not the source of primary helper truth. The command resolver therefore uses retained **bound command-specific** outcome evidence, not a pattern match on Request status or RequestAssignment.

Even a correctly disclosed current projection and a command record may be read at different bases: their joint use must qualify any claimed temporal ordering or causal relation. A stale or incomplete read that finds no command record cannot prove non-incorporation. The operation may use one appropriately qualified snapshot or a declared consistency/progression relation; mere proximity of two reads, or their origin in a shared logical database, is not that relation.

The public resolution path is conceptually:

```
(request/resolve-claim!
 trusted-context
 {:request-id R :command-id C42})
```

This **schematic public operation**, not a claim that the named Clojure helper already exists, resolves the full immutable binding required by the contract, including authority/tenant namespace, operation/interface and canonicalization compatibility, canonical arguments, principal or declared delegation. It uses a qualified authoritative read, checks the current principal's permission to request resolution and separately to disclose the historical result. The request-supplied C42 bytes and `:request-id` are not credentials. Unauthorized callers must not gain an existence oracle through differences in error wording, timing-dependent disclosure, or returned binding details.

The resolver's receipt of a retained command record can establish the command's incorporation without proving that T1, rather than a concurrent compatible submission, caused it. Nor does a command's incorporation authorize the current browser owner to install a stale projection. This is not a public wrapper around unrestricted raw database lookup.

Resolution results distinguish:

Resolution classification	Meaning
Resolved incorporation	A compatible retained record justifies the permitted recorded result.
Resolved rejection	Qualified command-closure evidence justifies the permitted rejection.
Not established	This lookup did not establish a definitive disposition; pending incorporation or unavailable retained evidence ha
Unavailable	Acquisition or permitted disclosure could not supply a result. Privacy policy determines how much detail may b
Incompatible	The admitted binding conflicts with the retained binding, when this distinction may be disclosed.

These are resolver classifications, not additions to the canonical model outcome set. The interface declares how each maps to an operation outcome or typed invocation error. In particular, not established does not mean rejected. A stable terminal rejection requires the closure/exclusion evidence of 04.07, not a failed assertion or lookup that merely found no record. A command-record constructor that persists only incorporation results does not, by its existence alone, establish durable terminal rejection, expiry fencing, restored-command exclusion or an executable recovery owner. Each stronger guarantee needs its own admitted contract and qualified physical mechanism.

A retained tombstone may establish exclusion of duplicate incorporation without retaining a disclosable result. A record missing because of unqualified read, expiry, migration or restored-backup gap does not justify fabricating the prior outcome or sending a new command under a fresh ID.

A compatible invocation of `request/claim!` can alternatively recover the same record through its replay branch. It returns `:already-incorporated` and the permitted success even though the Request is now claimed. It must not rerun the new-incorporation guard first and reject solely on that changed state.

If another physical attempt finds a compatible, qualified retained success after its own ambiguous submission, that proves command incorporation under the record's admitted binding and read scope. It does not necessarily prove which physical attempt incorporated it. Such evidence may justify the historical `:already-incorporated` recovery result for an authorized resolver; an invocation-specific `:committed` needs additional admitted attempt-to- complete-submission causality. No such record can grant historical disclosure to an unauthorized principal, and a present Request projection is not a substitute command-existence oracle.

C42 recovery and presentation are independently admitted

The browser has two different useful recovery paths. One asks the public command resolver about C42 under its current principal and binding. The other reacquires a currently authorized Request/RequestAssignment projection at an appropriate basis, to repair the displayed card or board. The former resolves **historical command attribution** when permitted; the latter acquires **current business state**. Neither substitutes for the other when both properties are required. Local author feedback may be adopted under its admitted command/presentation mapping. Unrelated remote board changes remain subject to the application's selective-adoption policy; receipt of an advisory cannot force their installation.

Recovery observation or event	Required interpretation
Authorized C42 readback returns the exact compatible incorporation record	Disclose the historical Claim result to the per
Authorized current Request/RequestAssignment read shows Alice still primary	Permit current-state presentation under its ba
Current projection shows Bob as primary after a legitimate later Reassign	Keep C42's historical success separate from B
A scoped read reports no C42 record while an earlier T1 submission remains unresolved	Keep the command disposition unknown; abs
A server knows the command record exists but Alice's current authority forbids disclosure	Return only the permitted unavailable/indisti
A current reconciliation owner receives qualified C42 evidence after E1 retired	May settle surviving command knowledge via
A second Alice tab knows C42's ID but never received the first tab's result.	Reconcile through the second actor's authoriz
A Late T1 response or E1 timeout arrives after replacement execution E2 owns the target	Suppress retired owner effects; do not erase a
Durable notifier work is still pending after C42 resolution	Recover the original work identity through its

The successor's admission is not automatic merely because it knows C42. It must preserve exact operation/binding compatibility, current principal/disclosure policy, its own resource instance and presentation contract. Reconciliation evidence useful beyond E1 can be **transferred as knowledge**, not as E1's retired mutation or installation authority. This preserves owner-relative event admission.

Same-principal recovery requires an admitted knowledge path for the recipient actor. E2 knowing C42's identifier does not prove it received E1's branch message or authoritative result. E2 can learn the historical disposition through a currently permitted and correctly correlated recovery read or communication under its own owner/plan; that knowledge does not retroactively revive E1's retired installation, callback, or optimistic-overlay rights. The command may remain authoritatively settled while all surviving browser presentations still need reconciliation.

1. Branch knowledge is not just current-state observation

   Suppose C42's Claim incorporated, but a later Reassign made Bob the current primary helper. A trusted Request/RequestAssignment read may establish that Bob is primary **now**; it cannot establish that C42 was rejected, that Alice's earlier Claim did not succeed, or that the helper received the original approved-branch message. A qualified retained C42 result can establish the historical incorporation to an eligible successor actor, but does not restore a lost message as if it had arrived at its old program point.

   If the admitted local choreography branches on **current assignment**, an appropriately scoped and timely authoritative observation may justify that branch. If it branches on **C42's historical outcome**, it needs command-specific retained evidence or other authorized causally bound history. If the global choreography requires an approval notification or synchronized receive, replacing it with rereading is admissible only as an authored alternative or qualified contract-preserving refinement accounting for correlation, ordering, observable message effects, waiting and recovery. Neither a Live advisory nor a generic board Refresh is by itself that substitution.

   An unavailable or undisclosable historical record may leave the command-specific branch unestablished even when the present Request state is known. The declared recovery continuation must allow that uncertainty without inventing a different branch or a fresh command.

Reconciliation needs an owner, not only a lookup

A read-only resolver is useful but is not an executable recovery strategy by itself. An admitted unknown branch identifies who requests reconciliation, how the command reference and immutable binding survive local retirement, how a successor proves its current permission and resource ownership, which evidence sources are used, when retries are allowed, and what happens

when automatic recovery stops. A retained C42 reference without a current principal, authority, canonical argument binding and compatibility contract is insufficient for authorized reconciliation.

The recovery policy may include backoff, bounded automatic attempts, a durable queue, user-triggered refresh, or operator handling. The safety claim must hold when progress never occurs: no duplicate incorporation, invented rejection, privacy leak or retired installation follows from an exhausted retry budget. A liveness claim additionally names assumptions such as eventual backend availability, retained command/exclusion evidence, correct authoritative read visibility, scheduling fairness and continued authorization. A timeout budget can end automatic work without establishing non-incorporation.

The selected recovery owner must be able to survive the loss of the resource whose outcome is unknown when the application promises recovery beyond that resource's lifetime. The specific choice—current browser successor, durable server worker, operator intervention or another authorized owner—is a plan decision. An implementation need not run a permanent poller for every command. Whatever owner is chosen must retain or reacquire the required identity, backoff/retry state and trust context through a declared mechanism; a Clojure callback captured in a now-dead HTTP request is not a durable owner.

Static preflight need not predict dynamic conditions. It must nevertheless require declared handling or recovery paths for conditions that can strand an operation, including unknown commit, undisclosable replay, incompatible retained interfaces, and unavailable authoritative acquisition.

Retiring E1 revokes E1's installation and settlement rights according to its resource policy. It does not cancel C42 or erase its result. A later execution E2 may reconcile C42 through an admitted participation path. Late evidence must not revive E1 or install into a resource instance now owned by another execution.

Cancelling a local request is similarly distinct from an authoritative cancellation command. Whether an authoritative operation can be cancelled is a model question with its own valid transitions. Once incorporated, an effect may require a compensating operation rather than retrospective cancellation.

Lost-response acceptance cases

Case
C42 commits the complete Claim transition and durable work while T1's reply disappears
C42 readback is allowed and fully bound while Request R has since been reassigned
The helper must branch on C42's historical approval after a later Reassign; a current Request read now shows Bob primary.
A projected helper is waiting for an explicitly required approval message; a current authoritative read returns the same apparent `:approved`
A later admitted execution defines its branch solely by current RequestAssignment ownership.
A complete authorized Request/RequestAssignment projection shows Alice currently primary but has no command attribution
Two compatible physical attempts overlap, one commits and both replies are lost
The command record lookup is stale, out of retention, incomplete, or made on the wrong tenant
A legitimate current owner E2 reconciles C42 after E1 retires
A stale E1 callback fires after E2 installed a newer current projection
Alice may view Request R but may no longer disclose C42's historical result
C42 is authoritatively resolved while its original durable notification work remains undelivered
A recovery owner's automatic attempts expire while command incorporation remains unproved
Several web nodes share the same authority, but one read has not reached the necessary basis

External systems and valid intermediate states

One database transaction cannot generally atomically commit a database change and an arbitrary external provider effect. The model must represent valid intermediate states and durable recovery rather than describing a remote API call as if it were part of the database's atomicity guarantee.

The durable-workflow boundary of Part III applies: a process-local choreography execution is not automatically a durable owner of unfinished business work.

Worked example: placing an order with payment authorization

Suppose the business policy permits an Order to become placed once the provider has authorized the specified payment. Capture and settlement are separate effects unless the operation explicitly includes them. This example must not call an authorization receipt proof of a completed charge.

Money is an amount in a declared currency, with a supported precision and rounding rule. A bare floating-point number or unqualified decimal is not the complete payment argument. The immutable payment intent binds the Order, merchant/provider account, operation type, amount, currency, instrument reference, and relevant policy/interface versions.

The local public entry operation is conceptually:

```
(order/place-intent!
 trusted-context
 {:command-id order-command-id
  :cart-id cart-id
  :payment-instrument-ref instrument-ref})
```

The authoritative model validates ownership and eligibility of the Cart and instrument reference, computes or validates the authoritative price, and protects the inputs required by its pricing and inventory policy. It atomically creates a valid pending Order, records the command result, and records durable payment work. The pending Order is a committed domain state, not an optimistic browser fiction.

For this workflow, invoking the payment provider is a separately admitted physical effect **after** the immutable intent and durable work have committed. A callback that the local transaction mechanism may re-evaluate or retry must not make that external call. Neither a queued job nor a submitted HTTP request is provider acceptance, and the durable work's owner must survive its original worker, process, and browser execution.

The Order command's success means that this intent was established. If the UI wants to wait for final placement, choreography must distinguish intent acceptance from business completion. It must not display a final placed result merely because `place-intent!` committed.

Provider request identity and evidence

The durable payment intent has its own stable identity. Provider requests use an idempotency key derived or allocated under the provider's actual key scope and retention rules. The binding includes the merchant/account, provider operation, and immutable request meaning. A globally formatted string does not by itself establish global uniqueness or deduplication at the provider.

The qualification contract states whether repeated requests with that key are deduplicated, how conflicting payloads are handled, how long records remain, and how an indeterminate request can be resolved. Provider idempotency is an external premise; Gesso cannot manufacture it by storing a local key.

If the provider does not supply a sufficient deduplication or lookup contract, the application must use a recovery strategy appropriate to that limitation. An unknown effect may require manual reconciliation. Blindly sending a fresh key after a timeout can create a second charge or authorization.

Provider evidence must be authenticated or otherwise admitted by a qualified adapter and bound to the exact payment intent. Its interpretation checks the merchant/account, provider operation, Order or intent reference, key, amount, currency, and result status. Receiving an untrusted webhook map with a familiar status string is insufficient. Signature verification alone also does not prove that the signed event refers to this intent or establishes the required effect.

The adapter must distinguish authorization, capture, decline, pending status, and an indeterminate transport result. A provider lookup that finds no record does not establish non-occurrence while an earlier request may still be in flight, unless the provider contract supplies that stronger guarantee.

If provider deduplication records may expire while local work is unresolved, reusing the original key does not retain an expired deduplication guarantee. Automatic submission must stop or follow an independently qualified safe recovery protocol; allocating

a fresh key cannot silently discharge the old unknown effect.

Incorporating a provider result

A worker resolves or submits the provider operation under the durable intent. It then invokes a public model result-incorporation operation with the admitted provider evidence. That operation protects the payment intent and current Order state, records the accepted receipt, makes the permitted domain transition, and updates durable work in the same local atomic boundary.

This local result-incorporation operation is itself a new authoritative transition, distinct from the original place-intent command and the remote provider attempt. Its current guards, provider receipt identity, local command binding where required, and work-state changes are protected together. If the result changes an Order projection whose Live freshness is promised, it also records the corresponding recoverable **publication impact** or qualifies an equivalent discovery path. Completing the original payment-authorization work does not establish that the new domain transition's observers were notified. Conversely, a successfully delivered Live invalidation cannot establish that the payment provider authorized, captured or settled money. Provider work and publication work have separate identities, premises and completion criteria.

Current local state and admitted evidence	Required treatment
Payment pending; matching authorization succeeds.	Record authorization and advance according to the placement policy.
Payment pending; definitive matching decline.	Record the decline and move to the declared failure or alternative-paymen
Payment pending; response lost or provider result unresolved.	Keep a truthful pending/unknown condition and retained recovery work.
Matching receipt already incorporated.	Return a compatible recorded result without applying the effect twice.
Receipt conflicts with immutable intent or prior evidence.	Reject or quarantine the evidence through a declared diagnostic/recovery
Cancellation requested; a late authorization succeeds.	Record the real external outcome and enter the declared void, compensati

Worked failure window: provider succeeded, local outcome unrecorded

In the preceding example, the following sequence is a permitted physical history, not an atomic Order-placement transition:

1. The Order, bound payment intent, semantic command outcome, and payment-work obligation commit together. The Order is still `:payment-pending`.
2. A worker sends the admitted authorization request under the retained provider key. The provider authorizes the money, but the worker dies before the local result-incorporation operation commits; its HTTP reply may also disappear.
3. A successor recovers the same durable work and **first** reconciles the original provider intent by a qualified lookup or other admitted evidence. Verified authorization permits the guarded local incorporation; an inconclusive lookup leaves the intent unresolved, with its recovery owner intact. A different key is not an innocent retry.
4. A duplicate webhook or another worker's late result may race with this incorporation. Guarded, receipt-bound local transitions must prevent a second incompatible authoritative effect; they do not independently prove that the provider executed only one remote effect.

This history differs from a payment that never reached the provider and from one whose local authorization receipt was already incorporated. The model and its diagnostic result must not collapse the three cases merely because all three can appear as a pending Order to a particular browser.

There is another distinct crash window **after** a qualified provider receipt is incorporated locally but **before** the resulting Order-change invalidation is delivered. Recovery here must not call the provider again to regenerate the publication. The original provider attempt is already reconciled; the new work is to deliver or reconcile the authoritative local Order change under its own propagation contract. The application may avoid replaying individual advisories when a qualified complete authoritative reread covers the affected scope and frontier. A payment worker's completed flag and an SSE emitter's successful enqueue are not interchangeable settlement evidence.

Workers may use leases or fencing to prevent stale local writes. Such mechanisms do not retroactively stop a remote call already issued by an old worker. Late evidence must still be handled according to the actual external effect and current model state.

Cancellation therefore needs explicit states such as cancel requested, void pending, refund pending, or review required when those distinctions matter. A terminal cancelled label must not cause the system to discard evidence that money was authorized or captured. Compensation is a new effect with its own identity, uncertainty, and recovery obligations; it is not an atomic undo of history.

Safety and progress for external work

The system can promise that no Order is reported placed without the required admitted authorization evidence, under the provider and backend premises. It cannot promise that every payment finishes merely because a worker exists. Eventual resolution

additionally depends on provider availability, usable lookup/idempotency behavior, durable scheduling, and any required human action.

Required work must remain discoverable after process loss. Acknowledging a job because an HTTP call was sent, rather than because the contracted work state was durably incorporated, can lose the only recovery owner. Queue acknowledgment, lease expiry, and model work completion require a declared relationship.

Paired external-effect acceptance obligations

Admitted scenario	Adverse or unsuppor
The local pending intent and required work commit before the provider call.	A locally retryable tr
A matching qualified provider authorization is incorporated under current Order guards.	An unsigned or misn
The provider authorized before the worker crashed; a retained-key lookup supplies conclusive evidence.	A lost response is tre
Compatible provider retries fall within its established key scope and retention contract.	Retention expires or
Duplicate compatible provider callbacks race to incorporate one result.	Conflicting, reordere
Cancellation and late provider success follow declared void/compensation policy.	Cancellation erases a
Durable work is discoverable after worker or node loss.	Queue acknowledgm
Provider authorization is locally incorporated and its separate required Order-projection publication is recoverable.	The worker dies afte
Resolution is conditional on provider availability and admitted reconciliation.	The system asserts u

FX value flow and lifecycle

FX should make value dependencies and effects inspectable while remaining usable from ordinary Clojure. Pure calculations consume explicit values and return explicit values. Effectful steps declare their inputs, outputs, failure classes, and effect boundary. A context is a capability-bearing environment, not a hidden bus into which every step writes arbitrary intermediate state.

Context and values have different roles

The context may contain admitted database/acquisition capabilities, a trusted principal binding, clocks or randomness with declared semantics, tracing, and scoped backend interfaces. Intermediate Request data, candidate decisions, prepared transitions, and outcomes should normally flow as explicit arguments and results.

This permits the system to explain which value justified a step and to detect missing or mismatched inputs. It also reduces accidental use of stale ambient data. The machine's original invocation argument and its subsequent state outputs are distinct value categories. A later step must receive needed invocation values explicitly, through a declared retained-value binding or other admitted value path; it cannot assume that the ambient context silently accumulated them. The same applies to graph or multistate resolvers.

A capability's presence in context does not extend its lifetime or widen its authority. Authentication state captured at entry does not freeze mutable authorization unless that temporal policy is explicitly part of the contract. The trusted invocation boundary owns capability provenance: an untrusted input map may not overwrite a principal, clock, database handle, or other privileged context binding through an incidental map merge. Missing required capability bindings known at construction must be rejected before invocation; effective runtime authentication, availability, expiry and current policy still require their appropriate dynamic checks.

Graph or multistate implementations obey the same rule. They may provide a convenient evaluation strategy, but hidden mutable cells must not become an untracked second model of operation state. Pure dependency nodes and effectful nodes require distinct semantics, including recomputation and retry behavior. Passing a client value through a trusted FX step does not upgrade it into an authoritative fact. A query result passed to another step does not become knowledge of a remote choreography participant until its admitted communication or acquisition path actually establishes that knowledge.

Ordering and completion are explicit

An ordered effect description specifies what each step waits for, how values flow, and what happens after failure. Merely placing effects in a vector is not enough if the interpreter may run them concurrently or treat enqueue as completion.

An illustrative descriptor could distinguish:

```
{:fx/type :sequence
 :steps [:admit-command
         :resolve-or-prepare
         :incorporate-authoritative-transition
         :construct-response]
 :failure-policy :phase-aware}
```

The registered step definitions supply types, branches, capability requirements, and completion semantics. The map alone is not an executable operation. In particular, an effect's vector-valued argument is not implicitly an ordered list of effects. An effect-order edge only orders execution as specified; it cannot itself establish transaction atomicity, durable completion, or remote receipt. The public operation advertises typed inputs, outcomes, authority, pre/postconditions, effect classes and failure semantics without requiring Choreo to inspect its private FX state graph. Independent publication work may proceed separately once durable intent exists; it need not delay an otherwise justified commit response unless the public contract explicitly promises that stronger acknowledgment.

Exceptions are interpreted by phase and evidence. A failure before submission, an ambiguous failure during submission, a post-commit publication failure, and a response-rendering failure are different events. A generic catch that turns all four into `:rejected` violates the operation contract.

Phase and available evidence	Permitted conclusion
Preparation fails before any possible submission.	The attempt did not submit; this alone is not a durable termina
Submission or response is lost without conclusive incorporation evidence.	The command disposition remains unknown pending admitte
Commit is established, but publication or dispatch fails.	Preserve committed domain and command truth, classify the d
Commit is established, but rendering or reply delivery fails.	Preserve the command outcome; the participant may lack setl

A later compatible command replay, a delivery-work retry, and a browser's fresh acquisition are different operations. Retrying notification must never re-execute the original authoritative mutation merely to make an observer current.

Every submitted effect path—including an exceptional fallback, deferred FX continuation and worker retry—must be admitted for the **actual complete** transaction and its trusted caller/worker authority. A registered step graph does not establish which callbacks or transaction options were used by the installed runtime. The selected checker can derive the effect before issuing an executor, enforce a final gate immediately before backend submission, or admit a qualified opaque implementation premise. It cannot claim full transaction correspondence merely because the initial step order is valid. Likewise, a worker's legitimate recovery of a publication item authorizes the promised delivery effect, not an unrelated replay of the earlier domain write.

Retryable backend callbacks must not contain unprotected external effects. Clock and random values used in semantic decisions need declared sampling and retry semantics. A clock or randomness facility is an ambient capability; its particular sampled value is a semantically relevant result whose origin and reuse across retries must be explicit. Ordinary Clojure functions and macros may implement these steps, but any opaque effect footprint remains an explicit contract premise.

Post-commit delivery debt

Once a command commits, failure to notify, render, or deliver does not uncommit it. Required later work becomes explicit delivery or recovery debt. The operation contract identifies what must be durable, who owns it, how it is retried, and what counts as completion.

For a durable publication requirement, the domain transition and publication intent must share an atomic boundary or an equivalently qualified discovery mechanism. A commit followed by an unprotected best-effort publish can leave a permanent gap if the process dies between those actions.

The intent binds the authoritative effect, relevant subject/query impact, progression requirement, and stable delivery/work identity. It need not capture every currently connected subscriber. A durable change log or outbox can feed later routing and interest matching. The required impact must be derivable from admitted source-owned change meaning and the selected application contract, not solely from the subscribers known to one process at commit time. Matching later-installed interests to that retained impact, or instead closing the initial-read/subscription gap through a qualified acquisition fence, must satisfy the promised awareness and recovery scope. This does not require persisting a per-subscriber acknowledgment for every broadcast or retaining every unrelated database change forever. Delivery semantics must state whether duplicates are possible and how consumers recognize compatible repetition.

An in-memory list of local interests is not proof that all affected participants will learn a change. If publication is conditioned on known interests, the design must cover remote interests, subscription races, disconnects, and router failover to the extent that its convergence claim includes them. Otherwise that claim must be limited explicitly. The particular routing topology belongs to the transport design; the no-lost-required-work obligation belongs here.

In particular, losing the process between authoritative commit and the first in-memory publication must be covered whenever automatic notification is promised. A durable change log, work intent, qualified reconciling acquisition, or another declared mechanism may satisfy the relevant obligation; a live SSE connection alone cannot. On subscription inception or reconnect the observer must have an admitted way to acquire a sufficiently qualified state or recover missed interests. If the only repair is user-initiated Refresh, the convergence claim is explicitly conditional on that action.

Publication completion also has a scope: queued, durably recorded, accepted by a broker, delivered to a connection, admitted by a participant, and adopted by a projection are distinct milestones. The contract names the milestone it can observe and retains downstream obligations where required. A work item may be retired at a narrower milestone **only if** remaining promised obligations have qualified independent owners, coverage or recovery. Replaying a publication work item must not repeat the domain transaction whose impact it announces. The same rule applies to a provider-result incorporation that creates new publication impact: payment-work completion, local receipt incorporation and Order-view dissemination are distinct settlements.

Recovery failure may leave debt visible and unresolved. It must not manufacture success, discard the command record, or turn the committed domain operation into a rejection. If an effect is intentionally best effort, its omission and the resulting weaker guarantee must be part of the public contract.

Paired FX and recovery acceptance cases

Admitted construction or history
A generated FX machine receives the invocation input in its first step and passes declared results to later steps.
A trusted context supplies a declared principal and transaction capability.
A pure query step returns a typed value with source provenance.
Effect sequencing waits for the declared completion boundary.
Submission fails before any possible incorporation, or an ambiguous submission is reconciled.
Commit succeeds and a subsequent publication fails; the original command result is retained.
A required scoped change intent survives process loss before any in-memory invalidation emit, and a successor or qualified reader reconcil
A source-owned change impact is recoverable for a late or remote subscriber under the declared guarantee.
A disconnected participant returns through qualified acquisition or its declared Refresh path.

Authoritative basis and XTDB

An authoritative basis describes the database observation context justified by the backend adapter. It is not a domain entity version, command ID, execution ID, or universal time coordinate. A basis may advance because unrelated data changed. Equal entity content at different bases does not make the bases equal, and a changed basis does not by itself make a command's business preconditions false.

XTDB integration should use XTDB-specific consistency mechanisms where they provide useful guarantees. Gesso need not invent a backend-neutral global revision system merely for speculative portability. It must describe the selected mechanisms through a qualified adapter contract rather than inventing generic arithmetic on opaque tokens or claiming guarantees from a dependency's name alone.

The backend qualification contract of Part I applies. A Gesso proof may be conditional on specific XTDB premises. It need not prove the whole database implementation, but it must make visible exactly which database behavior would invalidate the proof.

Basis provenance and the untrusted command boundary

An `AuthoritativeBasis` is established by an admitted authoritative read or trusted runtime boundary. A `ProgressionRequirement` is a scoped obligation on a later acquisition. An `ObservedBasisClaim` is what an untrusted participant says it previously observed. They may carry identical XTDB token bytes, but they are not interchangeable semantic capabilities or a demand for three wrapper classes.

For example, a browser command may include `:observed-basis` to help request consistent reads or explain what it believed. Parsing the token, comparing it to a server-issued value, accepting its Malli shape, or trusting a client-added `:authoritative` label does not prove that the browser received, installed, or understood that observation. If this history matters, a separately admitted receipt or trusted server-side record must establish the precise historical proposition it supports; issuance still does not prove delivery, installation, human acknowledgement, or current business authorization.

An untrusted basis claim may only influence a read through its validated provider, authority/database incarnation, scope and permitted consistency mode. Invalid or unsatisfiable claims fail under declared typed handling rather than weakening the authoritative read. A command's current business guards are determined by source-owned operation policy, not by whether its browser token happens to match the most recent database frontier.

Backend assumptions are distinct obligations

The adapter/profile identifies the loaded backend version and relevant build, configuration, transaction API, query API, value codec, temporal options, and Gesso realization. A supported version range requires evidence of compatibility under its declared policy; it is not an invitation to ignore version changes.

Premise	Required meaning
Atomic publication	The protected domain changes, command record, and required durable intents are established together or not
Dependency protection	The semantic predicates, including cross-document, absent-row, or aggregate conditions where used, hold at th
Representation and frame	Encodings, field presence, update semantics, and untouched data correspond to the admitted transition.
Command exclusion	Competing compatible attempts cannot each incorporate the same command under the declared namespace ar
Outcome interpretation	The qualified return or later evidence actually establishes the claimed committed, rejected, or unresolved statu
Read visibility	The read mechanism honors the declared progression and temporal requirement, and its observation evidence
Durability and recovery	Retained command and work evidence survives the failures included in the promise.
Writer coverage	Every writer capable of invalidating a dependent invariant follows the required discipline or is covered by an e

These premises are not interchangeable. Atomic writes do not prove predicate protection. A row count does not prove target or principal binding. A committed transaction ID does not prove an exact immediate poststate unless the backend contract supplies that relationship. A source-bound local return does not prove all remote backend semantics. An external pinned pre-read plus a later pinned post-read does not establish the transaction's internal snapshot or that no unenumerated writer changed a protected dependency.

The adapter must specify the actual protected mechanism for each supported operation class. It must not imply that arbitrary Clojure callbacks execute inside every XTDB transaction, or that a particular SQL construction has a stronger isolation meaning than its qualified contract establishes.

Writer coverage is an independent **deployment-scoped** premise, not a conclusion from the number of registered model operations or successful local gateway submissions. The covered physical inventory includes effective web handlers, internal calls, imports, migration/repair entrypoints, scheduled jobs, provider callbacks and any external writer able to change protected state. Coverage is established by qualified installed binding and backend access restrictions, by sufficiently supported independent inventory evidence, or under an explicit accepted external-writer assumption. An unknown inventory cannot be treated as an empty set. A non-route worker does not need a fabricated HTTP handler, but its actual installed callback and backend mutation rights belong to the same coverage argument.

A reported check applies to the exact active application/deployment, backend profile, executor/adapter implementation and protected operation family. Replacing a Ring/Biff handler after the checked installation, registering a new raw writer or changing an effective backend hook invalidates the dependent closure claim until its relationship is re-established. This need not revoke unrelated established claims; it must not preserve a stale application-wide certificate. Part VII owns the comprehensive installation graph; Part X owns the stronger physical refinement proof. This XTDB premise alone neither discovers hidden writers nor proves actual deployment adoption.

Qualification and conformance evidence

Qualification combines the declared backend contract, implementation binding, checked Gesso-side correspondence, and targeted conformance evidence. Useful tests include positive cases, deliberately weakened guards, altered targets, extra submitted writes, representation edge cases, contention, and lost or ambiguous returns.

Finite tests can falsify assumptions or detect incompatibilities; they cannot prove that no current or future XTDB behavior violates a premise. Gesso must therefore avoid promising that qualification will detect every possible backend defect. Its defensible promise is that assumptions are explicit, evidence is bound to the actual supported profile, relevant changes invalidate or trigger requalification, and observed violations propagate to dependent assurance.

Qualification should target the behavior on which proof depends. A dependency version string check is useful configuration evidence, but is not a semantic conformance test. Likewise, a passing abstract transition model does not test the actual adapter's transaction construction or backend interpretation. Regression evidence must include the scope- and failure-sensitive boundary: read-after-write progression, snapshot-token propagation, relevant guards under contention, absence and aggregate dependencies, representation/field preservation, valid-time selection, and unknown submission or readback outcomes. A passing positive readback without paired mutated or competing-writer controls is insufficient to certify the protected operation family.

If a required premise is violated or its support becomes unavailable, the affected proof claim cannot remain unqualified. Admission policy may stop the affected operation, accept a named weaker assurance class, or require operator action. It must not silently

keep the old certified label. In-flight commands whose outcomes are uncertain still require resolution; qualification failure does not establish their rejection.

Acquired prestate and actual transaction state

An acquired prestate can support preparation and construction checks. It is not automatically the backend's internal prestate at the later incorporation point. The protection mechanism must establish the needed relationship despite that interval. If a prepared whole-document update can overwrite fields that were not in the acquired predicate set, the declared frame and dependency protection must cover those writes or explicitly narrow the proven operation class. A version check on one document is not automatically a guard on absent rows, related documents, aggregates, or unrelated writers.

The design does not universally require the backend to expose an internal prestate token. A sufficient qualified guard, isolation contract, or conditional transition may establish the required refinement without such a token. Where that relationship is only a backend premise, it must remain named as such.

Reading before and after a concurrent transaction also does not necessarily produce its exact state pair. Another writer may intervene. A read guaranteed to include a commit can legitimately include later effects. That is useful visibility evidence, but not an exact immediate poststate witness.

Commit evidence, visibility requirements, and observations

Submission acceptance, transaction completion, and query visibility may be different backend milestones. The adapter must distinguish them. A log position allocated to an aborted or merely submitted transaction must not be presented as evidence that the domain command incorporated. A returned submission ID, a completed transaction key, and a pinned read that observes a command record are different evidence subjects; their association must be justified rather than assumed from matching metadata.

An observation may establish that a bound command record exists, or that an identified transaction completed, without proving that this invocation's **complete submitted footprint** caused that record. The adapter must distinguish (a) historical bound command incorporation, (b) identified transaction completion, and (c) the physical association among the admitted invocation, full effective submission and resulting record. (a) supports an authorized compatible recovered-result branch; (c), under the declared backend completion guarantee, is needed for attempt-attributed `:committed`. Without (c) the narrower observations remain useful but must not be promoted into an unqualified invocation outcome by a copied gateway report.

After a confirmed commit, the adapter may issue a progression requirement that an authoritative read must satisfy to include the relevant effect. The read path must enforce that requirement or return a typed unavailable/unsupported result. It must not silently read from an older available snapshot and label it current enough.

The actual observation can be later than the minimum requested basis. The evidence must distinguish a minimum visibility guarantee from an exact observed basis. It must not relabel the requested minimum as the exact snapshot used. If the adapter can establish only a lower bound, the observation contract says so and permits only conclusions justified by that bound.

Temporal interpretation matters separately. A transaction can be visible in system time while a query at a different valid time returns a different domain state. Authority/database incarnation, tenant, valid-time selection, system-time selection, and relevant authorization context are part of scope compatibility.

A restore, fork, or migration may create an authority/database incarnation in which old tokens have no comparable meaning. The adapter must establish an explicit migration relation or reject comparison. Equal-looking numbers or token bytes from different incarnations are not evidence of continuity.

Worked example: database advancement versus business conflict

At an admitted basis X17, an authorized browser observes Request R unclaimed and constructs command C42. At X18 a different Membership changes; at X19 another Request changes. When C42 reaches authority at X23, that advancement alone is not a reason to reject Claim. The source-owned Claim operation must check R's current lifecycle, active primary RequestAssignment uniqueness, membership/authorization dependencies and the command's retained identity under its protected incorporation contract. If an intervening helper actually claimed R, the relevant domain facts differ and the operation applies its specified conflict or reconciliation policy. X17 and X23 are illustrative opaque basis symbols, not integers on which the browser may calculate.

The same X17 bytes returned by the browser remain an `ObservedBasisClaim`; they do not certify that its display was installed. A read satisfying an admitted minimum X23 requirement may observe a later frontier and perhaps later authoritative changes. It may justify visibility through X23 but cannot, solely by that fact, prove C42's exact immediate poststate or its incorporation.

Paired XTDB qualification and correspondence cases

Admitted case	Adverse or u
A trusted read establishes a scoped basis, and an acquired projection meets its minimum frontier.	A browser-su
An operation's current guarded dependencies remain valid despite unrelated database advancement.	Another writ
A compatible supported guard protects the relevant document, absence and aggregate conditions.	Only a famil
A transaction and bound command record are atomically incorporated under the qualified backend.	A submitted
An admitted read reaches the required scope and frontier at a compatible valid/system-time selection.	A newer but
One pinned snapshot establishes a consistent set of acquired targets.	Separate befo
A qualified adapter for a supported XTDB build and configuration passes relevant positive and mutated conformance cases.	A dependenc
One installed writer family demonstrably uses the admitted guarded gateway for its full effective transaction.	An import, p
A checked handler is the active deployed ingress using the admitted executor and backend profile.	A later appli
A migrated or restored database presents evidence under an admitted incarnation relation.	Identically sh

Scope-aware progression and authoritative acquisition

A progression requirement asks for an observation of a particular meaning at a sufficient authoritative frontier. It is not merely a token. A later Hotel observation cannot satisfy a Flight obligation simply because the database position appears newer.

An admitted acquisition declaration must make the following information mechanically available, directly or through stable references:

- authority, tenant, database incarnation, and qualified provider;
- entity or query meaning, schema/projection version, and parameters;
- selection, filters, ordering, pagination, and coverage requirements;
- principal/visibility context and the policy for current disclosure;
- valid-time and system-time interpretation;
- the required progression relation and its evidence source;
- the acquisition owner and resource-instance correlation;
- the requested completion milestone: acquisition alone, or acquisition with permitted installation/adoption.

An acquisition result includes evidence of its actual scope and coverage, not just the data. A familiar shape, a token supplied by the browser, or a tag saying `:authoritative` is not enough to admit it. Admission follows the trusted acquisition contract from Part II.

The evidence must relate the **executed** read to its effective query meaning, including parameters after trusted validation, authorization/visibility conditions, selected projection, actual returned coverage and read basis. A declared route, correct result shape, or successful HTTP response is not by itself proof that the installed handler ran the declared query or returned a complete authorized result. A reused cached/singleflight projection needs its own justified provenance, read-scope compatibility and current disclosure permission; reuse does not grant a broader scope or a newer frontier.

The actual acquired observation is not an unconstrained basis label: its provenance includes the executed query, selected projection, result coverage, visibility and authority scope, read frontier, temporal interpretation and acquisition instance. An installation candidate must derive from that same admitted observation or a qualified transformation preserving those claims. A cached response for another principal, substituted partial HTML or copied basis token does not inherit the original acquisition's completeness.

Acquisition also has a **proposition and consumer** dimension when used as choreography knowledge. A trusted Request/RequestAssignment observation can justify a local branch about current assignment only when its declared subject, actor, source, temporal meaning and availability-before-branch satisfy that role's knowledge obligation. It does not establish an earlier command's accepted/rejected branch or prove receipt of a required communication. A qualified retained command-result read can justify historical C42 disposition under the separate command-binding and disclosure rules, but it does not automatically make a required global message optional. The authored choice or its qualified refinement must permit that knowledge path; correct query coverage alone is insufficient to establish protocol equivalence.

Query meaning and completeness

Entity contents and aggregate membership are different scopes. A Request card can establish facts about one Request without establishing the complete board. A board acquisition must identify its authorized collection selection, effective **server-side** predicates, projection, page/cursor interpretation, and completeness claims. Where the server filters, searches, sorts or pages, those inputs are part of the executed query's declared coverage.

Changing a server-side search or filter changes the query meaning. Sorting or filtering an already adopted cohort in the browser changes only presentation; it cannot narrow or broaden the authoritative membership acquisition, advance its basis, or discharge collection-adoption debt. Neither kind of change silently grants permission to adopt unrelated new rows. For HumanHelp's selected board policy, Organization/Location and bounded cohort identify authoritative collection scope; text search, display order, mine-first, unclaimed-first and show-terminal are local presentation options, not server query inputs. A future remote search or page acquisition needs its own declared authoritative query and coverage contract.

An empty query result establishes absence only within the successfully evaluated authorized scope and coverage. A partial response, timeout, inaccessible page, or denied query must not become authoritative absence. Privacy policy may limit which distinctions can be disclosed, but it cannot make a failed acquisition evidence of a domain fact.

One page can satisfy a requirement for that page under a stable query/cursor contract. It cannot satisfy a complete-board requirement merely by carrying a sufficiently recent token. A set of separately acquired pages establishes a complete common snapshot only when the paging contract supplies that stronger relationship. In particular, page windows gathered under different query parameters, visibility permissions, cursor generations or incompatible bases cannot be combined into a complete result by counting rows. Changing sort order may change page membership, even when it does not change the underlying authorized relation. A valid empty page is not automatically an empty board.

Opaque progression and comparison judgments

The progression provider owns token interpretation. Generic Gesso code may preserve opaque values and ask the admitted provider for supported relations; it must not infer ordering from string comparison, integer-shaped encodings, or HTTP arrival order.

Comparison distinguishes at least equivalent, advances, precedes, established incomparable, and not established. Invalid or incompatible evidence is also a typed failure. Unavailable comparison is not established incomparability, and neither is equality.

For some providers, equality is the only available relation without additional evidence. Equality of admitted tokens in a compatible context can be useful; equality of untrusted bytes is still just equality of bytes. An advancement witness must be admitted from a provider capable of establishing that relation and bound to its endpoints, scope, incarnation, and relevant contract version. Validating the witness map's shape alone does not establish its truth.

An unmet minimum, unavailable provider comparison, incompatible incarnation, or unsupported temporal interpretation must not be reported as a satisfactory read. The boundary may give a typed unavailable/unsupported verdict or use a declared bounded retry/recovery policy; it must not silently drop the required frontier, infer a relation from arrival order, or wait forever without a contracted resource limit. A proof of progression between token values does not itself prove the acquisition's query scope or authorization.

Worked example: travel observations with different scopes

Consider two requirements for one authorized travel-planning session:

Requirement	Scope and coverage
F	Flight 17 availability, specified cabin and fare-class selection, projection v1, session visibility context, declared temporal pol
H	Hotel 9 availability for check-in D1 and check-out D2, specified room/occupancy selection, projection v1, the same permitt

The symbols PF20 and PH24 are illustrative opaque values. Their numeric-looking suffixes do not authorize arithmetic. Even if both are realized by one XTDB database, scope remains part of the requirement.

Suppose the provider and acquisition boundary establish the following evidence:

Observation	What it can establis
Full Flight 17 projection, correct cabin/fare selection, compatible time/context, admitted advancement through PF20.	Satisfies F.
Hotel 9 projection with advancement through PH24.	May satisfy H; it do
Flight 17 seat count alone at a newer compatible basis.	Satisfies only an app
Hotel 9 for different dates.	Does not satisfy H e
Correct hotel query, first page only, when H requires complete offered-room coverage.	Does not satisfy H's
Correct values with an unadmitted basis tag.	No authoritative sat
Correct selection at incompatible valid time or visibility context.	Requires a declared

F and H may be fulfilled by separate admitted acquisitions. That does not establish that the combined travel display is one atomic database snapshot, or that either offer remains reservable. Booking eligibility remains a current authoritative model obligation at the booking boundary.

Worked example: HumanHelp board membership versus one Request card

Suppose an authorized HumanHelp board query at admitted basis P17 establishes membership of the declared Location cohort, including active and terminal Requests within its bounded authorized scope. Local search, sort and show-terminal choices may hide or reorder the displayed members without changing that acquisition. A later correlated authoritative operation result or reread at P19 establishes a newer projection for Request R, with current lifecycle and authorized active primary RequestAssignment. The card may be displayed according to the declared local adoption policy while the board still has its P17 membership coverage. The actor cannot call this a complete board acquisition through P19; one card does not prove inclusion, exclusion, ordering, or absence of all other Requests.

A later P19 full-board acquisition can justify new membership only if the actual executed selection, visibility context, temporal interpretation, completeness and current admission evidence match that requirement. Even then, acquisition is not automatic visible adoption: an advisory Refresh policy may require the current owner or user to approve replacing the board. Likewise, a successful HTMX response whose target retired or whose swap was denied is not evidence that the P19 board was installed. A retained P17 board need not become false merely because some other scoped evidence advanced.

Paired acquisition and progression acceptance cases

Admitted observation or construction
A trusted acquisition executes the admitted query and returns the complete authorized scope at a sufficient basis.
A complete Flight 17 seat-map read satisfies its matching Flight requirement.
A local choreography branch concerns who currently holds the primary RequestAssignment, and an admitted read establishes it before div
A board query returns an admitted complete empty selection.
A page is acquired with a stable query/cursor/basis and its declared page-level coverage.
A provider establishes a direct, compatible progression witness for the observed read and required basis.
A read honors the requirement's tenant, principal, temporal scope and minimum basis.
Cached fragment reuse has a compatible current principal, parameters, scope, basis and disclosure policy.
An admitted later Request card updates only that card while older board membership remains separately justified.
An older complete board acquisition arrives after a newer authorized local Request S was selectively adopted.
A complete read is permitted and the current owner actually installs it under the declared milestone.

Requirement satisfaction and safe combination

The executable semantic core defines the following relations for each supported scope and progression contract. The notation does not prescribe API names.

- `satisfies(O, R)`: admitted observation O establishes R's scope, coverage, and consistency requirements.
- `entails(R2, R1)`: every admitted observation satisfying R2 also satisfies R1.
- `combine(R1, R2)`: a representation retaining both requirements, or an explicit unsupported result where the requested representation cannot do so.
- `fulfills(E, B)`: the admitted evidence collection E fulfills every obligation in bundle B at each obligation's declared completion milestone.

The default conservative combination is conjunction: retain both obligations. A provider may offer a sound compact representative or join. A successful single-observation combination must satisfy the law:

```
satisfies(O, combine(R1, R2))
  implies satisfies(O, R1) and satisfies(O, R2)
```

When no one observation can satisfy both, a bundle may allow separate acquisitions. Bundle fulfillment does not imply a common snapshot. It also does not erase the distinction between acquiring evidence and adopting it.

For a bundle B, `fulfills(E, B)` requires each currently outstanding obligation to have a specifically admitted witness in E meeting its **own** query scope, minimum frontier, authorized visibility and declared completion milestone. The witnesses need not be one read, but no witness can be counted twice for incompatible scopes merely because its token looks sufficiently new. An unsupported compact join does not make either original obligation disappear; the conservative bundle remains the safe alternative when it is representable.

A complete board observation at P18 and an independent locally admitted Request card at P19 fulfill their respective scopes, not a synthetic complete-board observation at P19. Evidence sufficient to satisfy a board acquisition requirement may still be ineligible to **install** a whole-board replacement over newer, narrower adopted facts. For acquire-and-adopt debt, the declared installation and semantic-adoption milestones remain independent requirements.

Dropping R1 because R2 supersedes it requires established entailment, including compatible scope and coverage. A later-looking token for another query is not such evidence. Unknown comparison does not authorize dropping either obligation. Entailment is a claim about **all admitted satisfying observations** under the applicable provider and semantic versions, not a guess drawn from a few equal payloads or previous request completions. Any proof or provider-qualified subsumption rule must account for scope, authorization, query version, valid/system-time selection, coverage and adoption milestones. Evidence that a particular observation satisfied two requirements is not by itself a proof that those requirements entail one another in every future state.

Normalization preserves semantic meaning and is deterministic where stable identity depends on it, under the declared semantic/provider versions. If the implementation coalesces duplicates, reorders, or regroups requirements, it must establish the corresponding idempotence, commutativity, or associativity of obligation meaning. These need not be claims of byte-identical representation.

A stronger combined requirement may preserve safety while making acquisition impossible. A liveness claim must therefore establish that the strengthened requirement is satisfiable by the admitted read path under its declared progress premises. Soundness is not permission to promise eventual acquisition of an impossible join. If no admitted read can satisfy the proposed common

scope or basis, Gesso must retain separately satisfiable obligations where supported or report a typed unsupported configuration, rather than replacing achievable requirements with an impossible one and continuing to promise progress.

Bounded representation and backpressure

One in-flight request does not imply bounded retained debt. A coordinator may accumulate incomparable scopes or requirements while that request is pending. Its bounds must distinguish request concurrency, obligation count, representation size, and any finite abstraction used in model checking.

For a finite scope domain with sound subsumption, one representative per scope may suffice. Otherwise the implementation may retain bundles, partition work, apply backpressure, request an authorized broader acquisition, or report an unsupported configuration. It must not silently discard obligations to make a counter stay below a limit.

Broader acquisition requires permission and a coverage contract. It does not automatically authorize broader visible adoption. A user who has not chosen to refresh board membership must not acquire a new visible board merely because the coordinator compacted its metadata.

Resource retirement may legitimately end obligations scoped solely to that resource, under an explicit retirement policy. It must not erase durable command uncertainty or work owned elsewhere. The reason for retiring an obligation is part of its lifecycle semantics, not a claim that it was satisfied.

Backpressure must classify what is being bounded: physical in-flight requests, queued wakeups, retained distinct requirements, serialized obligation size, resource owners or the abstract states used in checking. A one-request limit bounds only concurrency; infinitely many new incomparable query scopes can still accumulate. A finite verification model must declare an abstraction and why it preserves each checked safety or progress property. Queue eviction, timeout, failed reads, cancelled swaps and owner replacement require explicit retention, transfer, qualified retirement or an unsupported result; counters reaching zero are not evidence of fulfillment.

Mixed observations do not imply an aggregate snapshot

A board may retain membership justified at P17 while one Request card is updated through a later correlated operation result at P19. That can be a valid application state under its adoption policy. It is not a complete board snapshot at P19.

The representation must preserve enough provenance to support only the claims actually established. It need not use a universal persisted fact graph, one mutable object per fact, or a per-entity integer revision. It does need to distinguish the scope of the later card observation from the older board membership observation.

Replacing an adopted projection requires its declared supersession or reconciliation relation. An incomparable observation may remain useful evidence without being eligible to replace the current display. Equivalent bases also do not make different projection schemas, principals, or query meanings interchangeable.

A complete older collection cohort may coexist with newer selectively admitted local card or membership evidence. Installing the collection must preserve the newer evidence under an admitted scope-aware reconciliation or defer an incompatible replacement. The resulting mixed-provenance presentation is not one full collection snapshot at the later card frontier; the database basis of the older collection does not permit overwriting independent adopted facts.

Worked example: a later advisory during an in-flight Refresh

An authorized complete HumanHelp board starts a managed acquisition A1 with outstanding obligation O1: acquire and adopt the current board query through provider-admitted frontier P18. While A1 is in flight, another relevant advisory adds O2 for the **same** query and adoption milestone, through P20.

A1's originally captured O1 is its request-start justification; it does not freeze the set of outstanding obligations. On completion, the coordinator must evaluate the **current** obligation set against the actual admitted observation and its installation evidence. For the compatible board scope:

Completion evidence from A1	O1
Complete board acquired through P18 and actually installed.	Dischar
Complete board acquired through P21, with provider-established advancement through both P18 and P20, and actually installed.	Dischar
Complete board through P21, but the current browser target retired before installation.	Remain
HTTP completion without admitted query, basis or coverage evidence.	Remain

A `:acquire-only` obligation may be discharged by sufficient admitted acquisition even when installation is denied; `:acquire-and-adopt` requires actual permitted adoption. The names P18/P20/P21 are illustrative opaque frontiers, not comparable integers. If provider evidence cannot establish advancement, the coordinator retains the obligations despite numeric-looking suffixes.

This example does not dictate one browser queue layout. It requires lossless semantic accounting across the period in which new obligations arrive after the physical request begins. A failed or cancelled A1 does not automatically submit the original domain command again; any successor acquisition must retain the necessary scope and requirements without weakening authority.

Paired combination, queue and mixed-observation acceptance cases

Admitted behavior	Adverse o
Combine two distinct scopes as an explicit conjunctive bundle.	Keep only
A provider establishes sound entailment for compatible requirement scope, coverage and milestone.	Comparab
One admitted observation satisfies two obligations according to their full contracts.	This one v
A stronger combined requirement is supported by an admitted read path.	An impos
In-flight A1 returns admitted P18 board evidence after later O2 at P20 arrives.	Completin
The returned board observation is admitted through P21 and meets both milestones, including actual adoption when required.	A browser
One physical request coalesces compatible wakeups while retaining independently queued requirements.	A one-in-f
A finite-state abstraction soundly preserves the property being checked.	Arbitrary
An owner retirement follows a declared policy and transfers or ends only owner-scoped work.	Retiremen
One Request card at P19 is displayed alongside board membership from P17 with separate provenance.	The card a
An older complete P18 board is reconciled with newer correlated S/P19 local evidence.	A whole-b

Progression flows through propagation

When an authoritative change carries a source-declared impact whose freshness is promised to an eligible participant, the propagation contract must preserve enough information for a resulting authorized reread to include that change. Current connected interest is one possible routing input, not necessarily the source of the impact obligation. An invalidation is an advisory derived from a relevant authoritative change; not every database write has to become a broadcast. The logical sequence is:

1. Qualified outcome evidence establishes the authoritative change.
2. Recoverable change/publication evidence carries an admitted progression requirement and affected scope information. A durable record, qualified change frontier, replayable source or equivalent reconciliation path can establish this under the declared delivery guarantee. The mechanism explains how the impact remains discoverable if no compatible interest was registered on the writer's local router at the instant of incorporation.
3. Routing delivers an advisory to a relevant participant under its transport and recovery contract.
4. The participant's acquisition requests a scope and progression sufficient to cover the relevant obligation.
5. The acquisition boundary admits only evidence that actually establishes the promised coverage and visibility.

The advisory remains advisory: it can establish that an acquisition is needed without supplying the new authoritative contents. Preserving progression does not require every stage to repeat the complete token; a protected reference to the requirement is sufficient when it can be resolved under the contract. The origin of a required notification and its recovery owner must survive the failure windows covered by the application's promise. A process-local emit, router enqueue, or SSE send is not an independently durable incorporation record and does not prove delivery, participant receipt, authoritative reread or installation. The system may safely redeliver advisory invalidations; deduplicating or coalescing them must preserve outstanding scoped progression obligations, not depend on recording per-recipient acknowledgements for each broadcast.

A tokenless advisory needs an alternative qualified fence or acquisition rule that prevents a satisfying reread from preceding the triggering change. Merely reading something newer than the participant's old display may still miss that change. If the transport cannot supply such a rule, the weaker guarantee must be explicit. A router may broaden fanout or redeliver possible missed invalidations without tracking exact delivery completion, but the resulting read must still satisfy the affected scope and an admissible consistency fence; redundant wakeups do not manufacture a trusted frontier.

Initial acquisition and subscription races

Initial content for a managed projection participates in the same scope, authority, progression, and installation contract as refresh content. Being server-rendered does not exempt it. The owner/resource instance and any initial basis claim must be established through the admitted bootstrap path. A managed empty target with neither qualifying initial projection nor a permitted first acquisition is structurally incomplete even if its subscription can connect. A server-side render with no qualified source/coverage evidence also cannot assert that the browser actually installed the projected contents.

The relationship between initial read and subscription must avoid an uncovered interval when the application promises eventual awareness of changes. Suitable strategies include an admitted read/subscription fence, replay from a durable cursor, buffered changes across registration, or another qualified recovery mechanism. Each strategy states how it handles reconnect and owner replacement. Interest registration at a single router is not proof that other writers or routers knew about that interest in time. The protection must cover the actual interest-discovery, publication, routing and acquisition boundaries, including the uncertain handoff when a router or process fails. Single-node and multi-node realizations preserve the same application-level guarantees; replication, routing topology, health detection and selective redelivery are physical strategies that must be checked against those guarantees.

A periodic authoritative refresh may provide a weaker eventual-awareness strategy under explicit scheduling and availability assumptions. It does not prove delivery of every invalidation. A router's successful local subscription does not by itself prove that no relevant upstream change was lost before its interest became visible. A new subscription can be justified without replaying every earlier event if a post-registration authoritative acquisition is fenced to include all relevant changes through a qualified frontier. That mechanism can also cover router failover by conservatively triggering redundant invalidations or acquisitions. It cannot infer a covering frontier merely from the fact that all surviving routers report healthy or from a fresh connection's existence.

Worked gap: read before interest registration

A browser acquires Request board Q at admitted frontier P17, then requests a managed Live subscription. Before the subscription becomes effective for all relevant writes, another authority transaction changes Q at P18. The initial read excludes that change. If publication uses only the interests visible at write time, and the subscription starts after the change without replay or a new read fence, the browser can remain indefinitely at P17 despite a healthy SSE connection. Neither a later keepalive nor successful router registration repairs that history by itself. The failure exists even when all local broadcast attempts reported success: the relevant interest was not yet visible at the publisher's matching boundary. Recoverable source impact plus qualified replay or a new subscribed-read fence can close the gap; a durable per-recipient send ledger is one possible mechanism, not a necessary semantic requirement.

An admitted implementation may instead register and buffer the interest before the initial read, replay changes since a durable cursor, acquire a new qualified frontier after registration, or periodically reconcile Q under explicit fairness/availability premises. If registration or router failover loses its continuity evidence, the participant must retain an unresolved acquisition obligation until qualified recovery establishes Q at a covering frontier. An implementation does not have to deliver every intermediate invalidation when a fresh authorized acquisition suffices. But it cannot claim eventual awareness without a path from this missed-change history to that acquisition.

Observed basis is not authority to commit

A browser-supplied observed basis can be useful as an untrusted input to freshness handling. It does not prove that the browser acquired or adopted the corresponding data, and it does not establish current authorization. Stronger claims require admitted evidence bound to the relevant observation and actor. Even a server-signed observation receipt proves only the proposition it actually attests (for example issuance under a named scope and frontier), not that the DOM installation completed or that the same principal remains currently authorized.

An authorized Live interest is scoped to its principal, current query and visibility context, actor/owner instance and subscription lifetime. Initial registration is not unconditional permission to disclose protected content forever. Subsequent delivery must follow the declared trusted revalidation, bounded grant or revocation policy when Membership, Organization, principal or disclosure rights change. A shared SSE connection may carry separately owned interests without equating their knowledge or installation rights. Replayed advisories after reconnect may restore pending scoped work but cannot prove that a different actor installed the same basis or granted Refresh consent. Browser callback cleanup is a supported-client behavior, not a substitute for server-side security: an adversarial browser can retain already disclosed information.

The server must also bound and validate requested consistency behavior. A malformed, foreign-incarnation, or unattainable token must not force unbounded waiting or silently downgrade to an unconstrained read. A qualified server frontier, known issued basis and merely client-echoed claim remain different evidence sources even when their portable tokens have identical bytes.

Advancement alone does not invalidate a command. The model checks its actual business preconditions. When the operation explicitly requires an expected domain version, the server enforces that condition as part of command meaning; when it does not, unrelated database advancement is not a reason to reject.

Dependency upgrades must not silently change token interpretation, read visibility, temporal defaults, or command/result compatibility. The assembly and qualification artifacts bind the relevant provider contract, and persisted requirements need an explicit compatibility or migration policy.

Paired propagation and bootstrap acceptance cases

Admitted construction or history	Adv
A committed change produces recoverable scoped progression debt, and the resulting authorized reread covers its frontier.	A su
A token-bearing invalidation carries an admitted requirement or recoverable protected reference.	A to
A tokenless advisory triggers an admitted acquisition fenced against its originating change.	A re
Initial authorized projection is acquired or an explicit first-acquisition path exists.	Man
Subscription registration is covered by replay, buffer, fence or reconciled read across the initial gap.	Q is

Admitted construction or history	Adv
One router disappears and an admitted failover path reacquires or conservatively redelivers possibly lost invalidations.	The
A source-declared impact survives without a commit-time matching interest and a later subscriber's fenced read covers that change.	Only
A local provider-result incorporation creates Order-change propagation work and the notifier crashes before emit.	The
A disconnected owner reconnects with an admitted state or cursor and reestablishes the acquisition obligation.	A re
Two tabs of the same principal share an SSE connection but only A installed P20.	B tr
Membership changes after subscription admission, revoking access to the Request scope.	An
A signed or server-held observation record establishes precisely its declared historical proposition.	Mat
A malformed, unattainable or foreign-incarnation consistency claim is handled with a bounded typed failure or declared recovery.	Serv

Advisory obligations and adoption

An advisory obligation records what a participant must still acquire or adopt under the application's declared policy. It is scoped debt, not a Boolean global dirty flag. Its meaning includes the required observation, coverage, progression, owner/resource lifetime, and completion milestone. The owner and **authoritative** query selection are part of the obligation's meaning: two requirements with equivalent frontier tokens need not be coalescible across distinct principals, server-side collection predicates, projections, or adoption policies. HumanHelp's local text filter, local ordering and terminal visibility toggle do not by themselves define a different acquired board cohort; they remain current presentation state, not grounds to discard, reissue or discharge authoritative board debt. A debt-free status is a claim justified by admitted settlement evidence, not by a queue's emptiness.

Advisory knowledge and adopted authoritative knowledge remain separate. A participant can know that newer relevant authority exists while deliberately retaining its current visible board until the user chooses Refresh. A glow or toast may represent that debt without adopting new membership.

The obligation representation must distinguish:

- an obligation to acquire admissible evidence;
- an obligation to acquire and semantically adopt it;
- any required physical installation that realizes that adoption;
- retirement or transfer of an obligation under an explicit lifecycle policy.

These distinctions need not create four public APIs or four universal stores. They must be explicit wherever a completion claim depends on them.

Acquisition, permission, installation, and adoption

The following milestones cannot be collapsed:

Milestone	Established fact
Acquisition completed	An admitted observation has arrived with the required source, scope, and correlation.
Installation permitted	Current owner, resource, scope, progression, and presentation policy allow this proposed installation.
Installation completed	The permitted physical effect actually completed under the realization contract.
Adoption recorded	The semantic state now treats the justified contents or membership as adopted, using the required completion evi

For an acquire-only obligation, the first milestone may suffice. For an acquire-and-adopt obligation whose realization includes a DOM update, merely returning true from `can-install?` or emitting an HTMX allow-swap decision does not suffice. Nor does HTTP success, an effect-issued marker, or an untrusted client-reported completion prove that physical installation occurred. A completed-installation observation is admitted only through its qualified realization and exact effect/owner/candidate correlation. The semantic state must then perform the declared adoption transition using **current** obligations and authority/visibility policy; installation and adoption are not equivalent merely because both happen in one ordinary browser callback.

The final installation-right check and the physical mutation must be serialized against competing owner changes and relevant installations, or protected by an equivalent mechanism. Checking a right, yielding, and later mutating an arbitrary replacement target can install into the wrong resource instance. Permission is scoped to the particular acquisition generation, authorized query, observation, target instance and owner; it is not transferable to a newer DOM target or a replacement Choreo execution. A late, duplicate or stale completion may supply separately admitted evidence, but must not exercise the retired instance's installation or debt-discharge rights.

The physical completion or failure is correlated to the admitted installation effect, owner, target instance, and observation. Adoption and debt discharge use that evidence under the current semantic state. A synchronous implementation may establish these milestones together; an asynchronous shell needs explicit completion/failure handling. The coordinator must not restore an obligation snapshot captured before a newer advisory arrived. It settles each **current** obligation only if admitted coverage and the required milestone are satisfied. Conversely, if another admitted installation already fulfills that obligation, a rejected late candidate does not justify retaining debt forever.

Installation permission also accounts for the candidate's actual physical **write/replace footprint**. A whole-target HTMX swap can erase newer, independently adopted card, membership, or local confirmation surfaces even when its board query and basis are valid. The realization must preserve those newer scoped facts through an admitted selective reconciliation, or deny/defer the conflicting replacement. An observed after-swap callback does not prove semantic non-regression when the actual physical effect discarded such facts.

If a physical update can partially apply, the realization must define that case. It may require repair, authoritative reacquisition, or a representation-unknown state. It cannot report complete installation merely because the effect began. Similarly, loss of a

completion event requires a reconciliation path if the physical representation may already have changed. A physical effect whose completion is unknown cannot be classified as either definitely installed or definitely absent without additional evidence. The recovery contract must say whether it re-observes the actual target, replaces it from an admitted complete projection, or transfers the unresolved obligation to an eligible owner. No recovery route may silently promote speculative or incomplete DOM state into authoritative adoption.

Worked boundary: permission before owner replacement

A1 acquires a complete authorized board projection at P18 and receives permission to install it into target instance T1 owned by execution E1. Before the physical mutation completes, E1 retires and a replacement E2 owns T2. A late `installation-completed` event for A1/T1 is **not** permission to install into T2 or to declare E2's board adopted at P18. The old obligation may be retired or transferred only by its declared lifecycle policy; outstanding work for E2 is independent.

If A1's mutation already occurred but its completion event was lost, the physical representation may be unknown. The system must reconcile or reacquire before claiming an adopted P18 board. If a separate qualified installation of P21 for the current owner has completed, that evidence can satisfy compatible outstanding adoption obligations; the rejected P18 callback does not force a regression or perpetual pending debt.

Paired advisory/adoption acceptance obligations

Admitted history
An advisory creates scoped acquisition/adoption debt under the current principal, authoritative query and owner.
A HumanHelp user changes local search, order or show-terminal while board adoption debt remains.
An admitted query fulfills an acquire-only requirement without installation.
Installation permission is issued for the currently owned target and candidate.
The trusted physical realization reports completed installation for the exact effect and current owner.
A new advisory arrives while the original candidate is being installed.
A newer compatible P21 observation is installed by a separate qualified path.
P18 complete board evidence is installed while S/P19 creator-local card and confirmation remain adopted.
Owner E1 retires; its installation work is retired or transferred under the declared policy.
A physical mutation partially succeeds or throws, leaving representation uncertain.
A completed effect loses its callback; a later qualified target inspection or reacquisition establishes state.
HumanHelp gives an originating creator immediate correlated card evidence while other viewers retain board membership until Refresh.

Worked example: two advisories during one refresh

Let B denote one complete authorized HumanHelp board acquisition for the selected Organization and Location, declared collection bounds, projection version, visibility context and temporal policy. These authoritative query properties are fixed for this example; local text search, sorting, priority ordering and terminal visibility are **presentation** over the adopted cohort and may change independently while a read is in flight. The board owner and target are one admitted resource instance. The board has adopted an observation at P17.

The provider establishes that an observation through P18 includes the change announced by O1, an observation through P20 includes the change announced by O2, and P21 covers both requirements in this compatible scope. It also establishes the advancement relations used below. These are admitted provider judgments, not arithmetic inferred from the names.

Advisory O1 creates an acquire-and-adopt obligation for B through P18. The user chooses Refresh. That action grants one logical acquisition under HumanHelp's declared scope and lifetime, not indefinite permission to fetch or install later remote changes. The coordinator allocates a fresh acquisition resource A1 under the managed lifecycle and issues the read for O1's requirement. Its identity is not merely a reusable request number; reuse would need a separate admitted instance discriminator. The grant binds the current principal, board owner, authorized Location/Organization, selection, acquisition scope, progression requirements and permitted adoption transition.

While A1 is outstanding, advisory O2 creates a further obligation for B through P20. The coordinator must retain O2. Completing A1 must not restore an old copy of the obligation set captured before O2 arrived.

Assume the application permits adoption of a non-regressing partial advance while stronger advisory debt remains visible. The sequence is:

Event	Adopted board	Retained acquire-and-ad
Initial admitted state	P17	None
Receive O1	P17	O1 through P18
Start A1 for O1	P17	O1
Receive O2	P17	O1 and O2 through P20
A1 completes with admitted B observation at P18	P17	O1 and O2
Installation is allowed but has not completed	P17	O1 and O2
Installation of P18 completes and is admitted	P18	O2
Later permitted acquisition and installation covers P20	At least the required frontier under provider evidence	None of these two obliga

If A1 instead returns an admitted B observation at P21 and its installation successfully completes, that evidence can discharge both O1 and O2. The coordinator checks the latest obligation set at completion, not only the set that happened to start A1. The acquisition's actual evidence can be stronger than its requested minimum.

If installation fails, neither acquire-and-adopt obligation is discharged by that attempt. The acquired observation may remain available as evidence under its lifetime and disclosure contract, but the visible board has not thereby adopted it. An HTTP success response or HTMX swap permission is not completion; any partial physical update or lost completion requires qualified inspection, repair or reacquisition under the current owner.

If a different permitted path already adopted P21, the late P18 response must not roll the board backward. The existing adopted P21 evidence may already satisfy O1 and O2; that is a separate justification for discharge. Rejecting the late installation does not mean the coordinator must keep debt that independently admitted adoption evidence has already fulfilled.

An application may instead require an acquisition to satisfy all current board debt before adopting anything. Under that policy, P18 is retained or discarded as acquired evidence and the coordinator requests enough evidence for P20. Either policy is valid when declared; the framework must not switch between them accidentally according to response timing.

For the HumanHelp proving application, the selected policy permits the P18 non-regressing partial advance but keeps O2 visible. One user Refresh grants one logical board acquisition and does **not** silently grant an automatic successor read merely because O2 arrived in flight. A separate explicit Refresh/Retry, or an independently declared bounded repair grant, is required for the next physical read. Repeated user clicks while A1 is active can be coalesced only under the specified bounded successor-intent policy; they do not create an unbounded automatically draining queue. The generic browser adapter's physical scheduling strategy must be qualified against this policy rather than being treated as the policy itself.

The coordinator's semantic obligations

The example is implemented through admitted transitions, not direct mutation by arbitrary transport callbacks. A useful transition decomposition is:

1. `advisory-received`: admit source and scope, then add or soundly combine debt.
2. `acquisition-requested`: validate or allocate the current acquisition grant, allocate the managed owner/resource, and bind the authorized query, requested scope, progression, grant budget and lifetime to a declared effect.
3. `acquisition-completed`: admit the effect completion and observation, then determine which requirements its evidence can cover.
4. `installation-requested`: propose only the permitted effect for the current target and owner; preserve obligations still awaiting adoption.
5. `installation-completed`: admit actual completion, update justified adoption, and discharge only obligations fulfilled in the current state.
6. `acquisition-failed` or `installation-failed`: retain unresolved obligations, release or retire resources according to policy, and select the declared recovery continuation.

These are semantic phases, not a mandatory source-file layout. Part VI specifies their browser realization and the refinement between DOM effects and semantic adoption. The permanent executable semantic core owns the transition meaning; proof artifacts explain why the admitted realization preserves it.

Every completion checks current owner, resource-instance, acquisition-grant and target correlation. A response for an obsolete server-side query selection, principal context, Location, Organization, page/window, route or target instance must not install merely because its database basis is recent. Changing only HumanHelp's local search, sorting or terminal-visibility controls does **not** make an otherwise compatible full-cohort acquisition obsolete. The completed board may be installed under current ownership and rendered with the **current** options; response-time display options must not overwrite those newer local choices. A change to Organization, Location, declared window, principal or authority context does change the acquisition scope and requires

an appropriately new owner/grant. Evidence may be usable through another declared admission path, but cannot resurrect a retired execution's rights.

The same principal may operate multiple HumanHelp board actors, each with its own installed cohort, advisory debt, Refresh grants, search options and physical target. One tab's P20 installation or explicit Refresh does not become another tab's adopted knowledge or acquisition consent. A later command resolver may legitimately disclose a historical result to another authorized actor, but does not thereby transfer the original execution's branch knowledge, optimistic overlay or DOM rights.

Logout, principal/Organization switch or established loss of disclosure retire incompatible subscription, acquisition and pending installation rights under the supported browser policy. Alice's late HTTP/SSE response must not become Bob's current board content after account switch. Protected server reads, mutations and disclosures remain independently authorized; client cleanup cannot retract data already seen by a malicious or offline browser. Durable command and work records survive under their own binding and recovery policy without disclosure to the new principal.

The obligation set is updated from current state. Removing O1 must not remove a newer obligation with a superficially similar key, overwrite a concurrently added scope, or clear a global dirty flag representing several independent requirements. Sound coalescing may replace the representation, but must preserve the combined meaning.

HumanHelp's selective adoption policy

HumanHelp can use these generic contracts to show the creator's own committed Request immediately through a correlated admitted result, while other viewers receive an advisory and explicitly refresh board membership. Remote creation and remote lifecycle changes to already visible Requests both follow explicit Refresh for ordinary board adoption. A user's own confirmed Create, Claim, Unclaim, Reassign or other admitted operation may update only its declared correlated Request-card or selective membership scope, provided the operation and current trusted disclosure permit that result. The policy must state which scopes a correlated operation result can update and how that update relates to the existing board selection. An SSE advisory cannot carry authoritative Request status, current helper ownership or permission to silently install it.

A card outcome can update an already adopted Request without proving a complete new board query. The currently active primary RequestAssignment, not a cached helper field or provisional card label, establishes authoritative Claim ownership under the source-owned model. An operation that introduces or removes a visible row needs an explicit membership-adoption rule. Filters, sorting, retained selection, scroll continuity, and optimistic overlays must not cause that scope distinction to disappear. A card conflict reread may correct that one Request without adopting unrelated board membership or proving that an outcome-unknown command incorporated.

Interleaved Refresh and newer local Request evidence

Bob begins one explicitly granted board Refresh A1. Its complete Location cohort is read at P18. Before A1 installs, Bob's own confirmed Create S incorporates and its authorized correlated result installs at P19, selectively admitting S to the cohort. A1's P18 read may still cover its board **acquisition** requirement, but physically installing that older board must not erase S, its selectively adopted membership, or the accessible ”Your new Request” result. The combined P18 cohort and S/P19 card are not one complete P19 board snapshot, and S's later basis cannot clear O2/P20 or import remote R.

The authorized physical realization may install the P18 base cohort while retaining the independently adopted S/P19 evidence, or it must decline/defer an incompatible replacement and keep the relevant adoption debt. A later reacquisition needs its own explicit grant or declared bounded repair continuation: HumanHelp's one-Refresh policy is unchanged. A complete newer acquisition can supersede both when its current scope, provider progression, permissions and actual installed effect establish that relationship. The creator-result surface has its own declared lifecycle and cannot be silently retired by an unrelated older board swap.

The framework does not hard-code HumanHelp's particular choice. It provides checked acquisition, ownership, progression, installation, and obligation mechanisms from which the application declares that choice.

Worked HumanHelp sequence: remote R, local S, and an excluding filter

Bob has adopted authorized board cohort Q17 at P17. Alice creates remote Request R at P18; Bob receives a valid advisory for the board but does not adopt R. Bob then successfully creates Request S at P19. Bob's current local search filter matches R but excludes S.

The correlated **authorized** Create result for S can selectively admit S under Bob's current board-scope membership rule without importing R. Because the current filter excludes S, the UI presents S in an accessible ”Your new Request” result area with an explicit way to reveal it in the ordinary board. It does not silently clear Bob's search or introduce R. Local search, sorting, show-terminal and other presentation changes operate over the already adopted cohort; they cannot acquire remote R as a side effect. The outstanding advisory for R remains visible.

Only Bob's subsequent explicit Refresh, followed by a complete authorized board acquisition and qualified installation, can adopt R under this policy. An observation with a later basis for S is not evidence that the complete board was read at that basis, and its adoption does not discharge R's scoped obligation. If Bob has an unrelated outcome-unknown command, that command's attribution remains independently unresolved even after a board reread shows its present Request status.

Paired Refresh and HumanHelp acceptance obligations

Admitted behavior	Adverse or u
O1 starts a user-granted read through P18; later O2 through P20 remains outstanding during A1.	The old captu
A1 installs admitted full-board P18, while P20 remains unmet.	Partial discha
A1 acquires P21 with admitted matching query, provider progression and completed installation.	A server-ech
One explicit Refresh grants one logical acquisition; another user click may create one bounded authorized successor intent.	An O2 adviso
A local Create of S is installed by a correlated authorized result while R remains only advisory.	The newer S
S is hidden by the current local search and remains visible in a separate accessible result area.	The UI silent
A local Claim result updates its affected Request using production RequestAssignment evidence.	A provisiona
Remote Claim/Unclaim changes produce advisory for another viewer until explicit Refresh.	A pushed car
Board scope stays valid while Bob changes local presentation options mid-read.	Response-tim
Same-principal tabs A and B have boards P20 and P17 respectively; only A explicitly Refreshes.	B silently ado
A late Alice response arrives after the browser switches to Bob or another Organization.	The old respo
An unknown command remains resolvable through command-specific retained evidence.	A board rerea
A newer authorized local card is installed while an older whole-board read completes.	The old read
A1's P18 response arrives after Bob's own S/P19 has been installed.	Whole-target
A permitted full-board install is partial or fails.	A successful

Assurance obligations and acceptance cases

Part IV's assurance is a chain of bounded claims. The semantic transition must express the intended rule; construction must bind it to the actual inputs and implementation; the complete submission must realize the protected effect; qualified backend behavior must justify the outcome; propagation and acquisition must preserve scope and progression; installation must justify adoption.

A report uses Part I's separate claim dimensions: proposition, subject, mechanism, premises, coverage, applicability, and status. It distinguishes derivation or construction checking, runtime enforcement, testing, and explicit assertion. Named environmental premises qualify a claim; conditionality is not an additional proof mechanism. These descriptions are not interchangeable assurance grades. A strong proof of an abstract fragment cannot conceal an unexamined physical execution boundary. Status belongs to a precisely stated proposition and subject: an established abstract invariant, a source-derived conditional construction report, a trusted runtime enforcement requirement, a bounded finite search and an observed physical outcome are different judgments. Their agreement can support a stronger composed claim only through the declared composition and correspondence rules. No copied report, familiar status tag or passing test grants a certificate or application admission by itself.

Required invariant families

The supported construction and runtime must establish, or explicitly qualify, at least the following property families:

1. **Binding fidelity**: operation, command, canonical arguments, principal, authority, target, and realization cannot be silently substituted between admission, submission, outcome, and reconciliation.
2. **Authoritative safety**: each incorporated transition satisfies its protected business conditions and declared frame under the backend/writer premises.
3. **Command consistency**: compatible repetition does not incorporate again; incompatible reuse cannot overwrite the old binding; terminal disposition is reported only with evidence of its declared scope.
4. **Truthful uncertainty**: absence of outcome evidence, timeout, and local retirement do not manufacture non-incorporation.
5. **Recoverable required work**: committed operations retain the declared command/result and propagation or external-work evidence across promised failure modes.
6. **Scoped observation**: acquired data supports only its admitted scope, coverage, temporal interpretation, visibility context, and progression.
7. **Justified adoption**: only current permitted installation/adoption paths advance adopted knowledge and discharge adoption debt.
8. **Evidence fidelity**: a classification, certificate, or passing test is not promoted into a stronger proposition than it establishes.

Each family needs a named source-owned operation or application subject, its claimed boundary and the actual premises on which its status depends. An uninspected writer inventory is unknown, not empty; a zero count of detected violations is only about the inspected population. A missing required operation is not a proof that the remaining operations preserve every invariant. Likewise, an invariant true only because no valid transition is enabled is insufficient when the application promises a usable operation. Independently declared capability and positive reachability obligations prevent vacuous success.

Liveness claims are separate. They identify scheduling, delivery, backend, provider, retention, authority, and human-participation assumptions. Safety must not depend on pretending that those assumptions always produce timely success.

A structural mismatch is rejected at the owning admission boundary; a valid construction with a modeled counterexample does not receive the claimed invariant; an unestablished premise remains explicitly unestablished unless the chosen policy permits a named conditional assertion. A heuristic suspicion about otherwise admitted business policy is a review finding, not automatically a proof failure. Assertions cannot substitute for a missing required declaration, contradictory operation contract or fabricated physical evidence. The report must preserve both unsupported and contradicted outcomes without silently relabeling either as business rejection or environmental unavailability.

Executable conformance cases

The implementation must provide conformance cases for the supported mechanisms, including negative cases, reachable positive controls, and relevant interleavings. Each case names its original source and operation, the controlled mutation or fault, expected checking boundary, affected proposition, oracle, forbidden physical action, and permitted continuation. A positive case must pass through the **same** boundary as its adverse variant; rejecting every input is not conformance. The independent effect observer must establish when a forbidden mutation, disclosure, installation or retry did or did not occur; absence of a logged success message is not proof of absence. These cases complement semantic derivations; passing them does not prove arbitrary backend behavior.

Area	Case
Target binding	Command names R1; acquisition or submitted mutation targets R2.
Claim eligibility	Request is open but already has a helper.

Area	Case
Relationship	Membership belongs to another principal or Organization.
Concurrent authorization	Membership, Organization association, or relevant policy changes after preparation.
Same-command concurrency	Two compatible attempts race.
Different-command concurrency	Two helpers claim the same unclaimed Request.
Replay	Original Claim succeeded, response was lost, Request is now claimed.
Incompatible reuse	Same ID, changed canonical arguments or authority binding.
Terminal rejection	A pending attempt races with rejection closure.
Unknown outcome	Submission may have committed; local response is lost.
Disclosure	A former caller no longer has permission to read a recorded result.
Same-principal actor isolation	Tab A adopts P20 while tab B still has P17.
Role-local recovered outcome	E2 knows C42 but did not receive E1's branch result.
Branch discriminator substitution	A currently claimed Request card is substituted for a retained C42 approved/rejected resu
Required communication substituted by read	The projector removes a required authority approval notification because a current rerea
Revoked Live interest	Membership/Organization rights change after subscription registration.
Atomic footprint	Domain update succeeds but ledger/outbox write is removed or fails.
Submission fidelity	A checked fragment is followed by an extra write, changed option, altered target, or wea
Return provenance	A copied success report or unrelated callback return is substituted.
Concurrent attempt attribution	A1 aborts while compatible A2 incorporates C42 and qualified readback finds the retaine
Co-observed but causally unbound evidence	A completed transaction key and command record occur at one pinned basis without a ju
Attempt failure versus command closure	An ordinary failed guard or absent-record read is compared with a binding-scoped termi
Frame	Unrelated fields change concurrently before a whole-document replacement.
Representation	Missing, nil, false, nested values, and field-name collisions are exercised.
Predicate protection	A row is inserted that defeats an absence, uniqueness, or aggregate condition.
Writer coverage	An external writer bypasses a required revision or invariant discipline.
Installed executor substitution	A valid Claim operation and checked plan exist, but the effective Ring/Biff route invokes
Deferred FX fallback	A failure branch or asynchronous continuation submits a vector or backend option outsi
Background and migration writer closure	A provider callback, importer or migration writes protected RequestAssignment facts thr
Deployment drift	The installed handler, worker entry or backend adapter changes after the admitted assem
Capability lifetime	One capability is raced, reused, or retained beyond its admitted lifetime.
Opaque code	A full-language callback uses an asserted effect contract.
External intent	Provider evidence changes account, operation, amount, currency, or intent identity.
External uncertainty	Provider request times out and later succeeds.
Provider result versus publication	Authorization is locally incorporated, but the notifier crashes before its Order change car
Commit-to-publish crash	A qualified domain/command/work transition commits and its process dies before async
Late interest discovery	A relevant authoritative change commits while no interest is known to the writer's route
Cancellation race	Authorization arrives after cancellation was requested.
Scope	A newer Hotel or single-card observation is offered for a Flight or complete-board obliga
Temporal context	Token bytes match but incarnation, valid time, query, or visibility context differs.
Acquisition failure	An empty-looking result actually represents failure, denial, or partial coverage.
Adoption	Installation is allowed, then fails or targets a retired owner.
New advisory	O2 arrives while an acquisition for O1 is pending.
Late response	P18 completes after compatible P21 was adopted.
Mixed-scope installation	A P18 whole-board read arrives after a correlated S/P19 card was installed.
Same-principal tabs with separate board scopes	A installs a qualified P20 response, while B still has a separate P17 owner and outstandin
Acquired-query lineage	A legitimate provider basis is attached to substituted partial HTML or a different princip
Subscription race	A relevant change occurs between initial read and effective subscription.
Retention	A stale retry arrives after result eviction.
Dependency change	Backend or adapter behavior changes beneath a retained proof claim.

The semantic cases should be derived from the same canonical definitions used for construction wherever practical. Independent adversarial and mutation cases then challenge the correspondence boundary rather than merely restating the implementation. Backend integration cases exercise the actual qualified adapter and its relevant transaction/read paths.

Paired positive/adverse control families

The following pairs complement the preceding negative corpus. The valid side must be accepted under its declared premises; the adverse side must be rejected, remain unknown, or be qualified at the appropriate boundary. These cases cannot all be replaced by

a stub that returns `:unsupported`.

Admitted positive control
A permitted HumanHelp Claim establishes one active primary RequestAssignment under current guards.
A compatible C42 repeat resolves its historical result under current disclosure policy without another write.
An admitted helper continuation offers a qualified authoritative C42-result acquisition as an alternative to direct approval communication.
A distinct current-state helper continuation branches on an authorized complete RequestAssignment observation.
A complete submitted domain transition and command-result footprint is present under the qualified backend.
A command times out after possibly committing and is reconciled with authentic command-specific evidence.
A valid payment intent is submitted after durable work registration and recovered with matching provider evidence.
A provider result is recorded under current local Order guards with separate, recoverable change-publication work.
A required change is committed with retrievable source impact and its interested browser later acquires a fenced view.
A permitted full-board read has complete scope and admitted progression through its required frontier.
A managed initial projection has an admitted acquisition/subscription continuity fence.
O1/P18 acquisition installs while later O2/P20 remains, under one authorized HumanHelp Refresh grant.
A correlated permitted after-swap effect installs into the current owner and semantic adoption records it.
One complete P18 board is selectively installed beside newer S/P19 local card and membership with retained authorizations.
Remote R stays advisory while Bob's local S is visibly confirmed under an excluding filter.
A valid opaque Clojure realization is admitted with its explicitly accepted correspondence premise.
A complete supported operation family includes eligible transitions and explicit disabled/rejected paths.
A real installed Claim ingress invokes the bound operation through the actual checked FX/XTDB submission gate.
An authorized provider-result worker uses a fresh scoped worker capability and independently checked local-result incorporation.
The deployment controls all relevant state-changing backend entrypoints or names an explicit accepted external-writer premise.

Test fixtures must distinguish admitted evidence from illustrative maps. A fixture that creates an advancement witness by changing a token field does not test provider truth. A fixture that returns a successful predicate without performing installation does not test adoption. A diagnostic resolver that finds a retained record does not demonstrate a durable recovery driver.

A finite-state check reports the explored state/input universes, enabled operations, examined interleavings, exhaustion or incompleteness and proposition scope. Enumerating sample integers or principal IDs is not a general theorem. Inductive preservation requires qualified initialization and a step rule for every admitted operation under the declared assumptions; neither finite success nor preservation alone proves physical execution correspondence. Fault injection must distinguish an injected failure from a malformed baseline and test the actual recovery or prohibition boundary.

Cross-part capability closure and non-vacuous acceptance

An operation family's valid local semantics, passed safety checks and successful source closure do not establish that a required application capability exists or can actually be exercised. An independently specified product-capability inventory must remain separate from the set of operations, enabled branches, rendered controls or routes discovered by inspecting the application under test. The latter may be incomplete precisely because a promised capability was accidentally omitted.

For each independently required capability, the enclosing application assurance records its declared intent and eligibility conditions, an eligible actor/input/prestate witness, the intended effect or observation, its source-owned semantic operation or explicitly non-mutating interaction, the admitted construction, the actual installed invocation and physical result, the evidence or premise at each edge, and a nearby adverse case. One abstract enabled transition is not proof of an installed HTTP operation or browser interaction; one installed route is not proof that an eligible user can reach or complete it. Conversely, an honest conditional or unsupported proof status does not by itself make an otherwise permitted application capability unimplementable. The report classifies which claim is established, incomplete, contradicted, or dependent on a declared accepted premise.

For HumanHelp, a successful self-Claim cannot stand in for Reassign: the positive Reassign case requires an authorized manager, a distinct eligible selected target helper, the real target-discovery and confirmation path, and the corresponding protected RequestAssignment replacement/history. The adverse variant changes target eligibility or a required current guard and must fail at the appropriate protected boundary without falsely passing because Claim works. A separately promised response interaction likewise requires its own independently selected product meaning; it cannot be silently equated with Claim. An unselected product contract is an unresolved product decision, not proof of an implementation defect for a nonexistent selected operation and not a completed capability.

Positive witnesses must use the same qualified source and actual enforcement boundary as the paired adverse cases. Test-only stubs, unreachable all-false guards, an empty operation catalog, a permanently disabled control or a fabricated permissive fixture cannot establish promised capability completeness. Existence of one successful witness is distinct from universal safety, production-path correspondence, and fairness-dependent eventual success; the root report must preserve those distinct judgments.

Completion criteria for this part

An implementation satisfies this part for an admitted operation family when its public interface, canonical semantics, construction, physical submission, backend profile, outcome interpretation, command policy, propagation, and acquisition/adoption paths form one inspectable contract. Supported mechanisms must reject mismatches early and explain the exact unsatisfied obligation.

Unsupported proof cases do not disappear. They are either implemented through another supported mechanism, admitted with explicit accepted assertions under policy, or rejected with a usable explanation. Dynamic unavailability has an explicit continuation. Success means the claimed semantic effect and evidence were established, not merely that the application started without errors.

Completion is evaluated **per admitted operation family and assurance profile**. A root application report identifies exact source/certificate identities and versions, the enumerated operations and independent required capabilities, the inspected physical route/writer/worker/browser boundaries, conditional premises, uncovered or untrusted implementation bodies, admissible positive/negative cases, and unresolved correspondence. Changes to backend behavior, trusted adapters, installed handlers, asset bytes or relevant business policy invalidate only the dependent claims under the declared dependency rules, but cannot retain a stale root acceptance. One certified source fragment does not imply every deployed entry point exercised it; genuine installation/adoption evidence must justify that stronger conclusion. Operation-local effect proof, actual submitted transaction correspondence, installed ingress/worker coverage, backend writer completeness and response/adoption realization are separately established edges. A valid physical operation under a narrower evidence profile can retain that scoped claim while the broader deployed-application claim remains unresolved; an undocumented bypass must not be silently reclassified as a merely unsupported business rule.

An assertion cannot repair a missing operation binding, malformed schema, unresolved capability, or contradictory effect declaration. It supplies a named premise about otherwise well-formed behavior; structural admission obligations still have to be discharged.

Part V — Provisional state and executable presentation

This part defines provisional presentation, local continuity, prepared view bindings, portable execution, and the separation of executable plans from diagnostic evidence. It uses Part IV's command outcomes, scoped observations, and acquisition/installation distinctions. Part VI specifies browser coordination and physical realization; Part VII closes the assembled application path before a managed affordance is served.

The intended architecture supports the full application language through the checked construction contract. Declarative interfaces and a portable coordination vocabulary make behavior inspectable; they do not require every computation or renderer to be expressed in a small proof language. Opaque functions retain explicit contracts and premises wherever their behavior has not been mechanically established.

Examples use illustrative declaration and API names. They specify intended semantics, not a promise that the displayed syntax already exists. Inspection maps and tables describe admitted values; copying their fields does not create authority, provenance, resource ownership, or execution rights. Generated fragments explain one canonical construction, rather than additional plans application authors must maintain independently.

Evidence and ownership joins across Part V

This part composes distinct evidence dimensions rather than a single global `success` status. Each proposition has an establishing source, a current owner and a strictly limited consequence:

Proposition	Establishing evidence and owner
Historical command disposition	Qualified Part IV command-specific result and its current command-resolution owner
Scoped observation and absence	Admitted provider/operation observation with Request/RequestAssignment, collection, princip
Provisional representation	Declared local derivation, execution and target/presentation owner
Actual installed/adopted scope	Qualified host completion for a current physical target, plus current coordinator recheck of obs
Continuity and local drafts	Current capture, slot, target incarnation, local edit identity and disclosure scope
Prepared view capability	Public model preparation, admitted operation and complete entry-mode/route/asset assembly;
Portable role-local transition	One projected machine, source-qualified normalized event and adapter/host effect contract
Executable, proof and packaged identities	Exact role-plan contents, global source/checker claims, accepted premises, opaque implementa
Behavioral product completeness	Independently declared eligible HumanHelp operations and real user-visible/model results

A transition may settle one dimension and leave others pending, unknown, failed or in recovery. Retirement transfers or retains separately owed command-resolution, acquisition/adoption, Live/Refresh, physical cleanup and mandatory audit work under the declared current owners. Optional diagnostics never discharge that work. When the needed evidence cannot be established, the corresponding claim stays limited or unresolved.

Optimism

Optimism is an explicit provisional trajectory associated with a semantic command. It permits responsive local presentation while authority determines the command's result. It does not predict a future database state with certainty and does not grant permission to make that prediction authoritative.

A provisional projection means:

> Given these admitted observations and declared local inputs, this is the presentation permitted for this command while its authoritative disposition is unresolved. Its derivation, owner, affected scope, and resolution policy are known. The authoritative model remains free to produce any result allowed by the operation contract.

The runtime distinguishes at least:

Category	Meaning
Authoritative observation	An admitted acquisition or operation output establishes specified facts within its scope and evidence contract.
Provisional projection	A declared local derivation associated with a command; it is not evidence of incorporation.
Authoritative absence	A successful admitted observation establishes absence within its complete authorized scope.
Unknown or unavailable	Required evidence has not been established or cannot currently be acquired/disclosed.
Local draft	User input or interaction state which has not become an authoritative assertion.

A visually identical card can belong to different categories. The runtime retains the distinction even when rendering produces similar Hiccup or DOM. Representations may be ordinary immutable values or opaque wrappers; the requirement is correct admission and use, not one mandatory host-language type.

The evidence category, command disposition, and installation/adoption progress are **independent dimensions**. A permitted provisional value can be installed locally while its semantic command remains unresolved; that installation establishes only the local presentation effect, not incorporation or an advance in authoritative adopted knowledge. A valid command-specific success can also be known while the fresh card is unavailable or has not installed. Even visually identical provisional and authoritative renderings do not collapse these boundaries. Adoption requires a suitable admitted observation, the current authorized target and owner, and the applicable completed physical effect. A successful renderer call, a prior command outcome, or permission to perform a swap is not substitute evidence of completed installation. Unmet acquire-and-adopt obligations remain until independently discharged.

Provisional derivation contract

Each optimistic operation declares:

1. its semantic command binding and entry conditions;
2. the authoritative observations, local inputs, and explicit assumptions used to derive the provisional value;
3. affected entity, query-membership, field, and presentation scopes;
4. the derivation implementation and its input/output types;
5. owners and resource instances permitted to install, update, or remove it;
6. its concurrency and dependency policy relative to other local work;
7. settlement, supersession, timeout, cancellation, retirement, and recovery continuations;
8. the evidence and physical completion required before claiming adoption.

An optimistic update normally depends on an admitted authoritative observation. The contract expresses the actual dependency, rather than demanding an existing entity snapshot in every operation. Optimistic creation may depend on an admitted collection scope, policy observation, declared draft inputs, and a temporary local entity identity. It must not manufacture authoritative nonexistence or assume a client-chosen ID is available. The server enforces creation conditions.

Purely local drafts need not be mislabeled optimistic domain operations. If the application renders under weaker entry knowledge, its declared assumptions remain visible and the projection cannot imply guarantees requiring absent observations. Missing required knowledge is an admission failure; an assertion cannot silently supply that missing binding.

Several observations may justify one projection. Its basis is then a scoped set of premises, not necessarily one universal database token. The derivation cannot strengthen acquired coverage or turn mixed observations into a common snapshot. Part IV's progression and scope contracts apply. A permitted draft may display its author's local inputs, but those inputs do not establish authoritative facts, protected relationships, global collection membership, or another principal's disclosure rights. The derivation must respect the admitted visibility and source scope of each protected observation it actually uses.

Presentation state is not a new domain status

An optimistic Claim can display a pending helper and a Claiming label while the last qualified Request/RequestAssignment observation describes an `:open` Request with no active primary helper under that observation's coverage contract. That pre-read is not a transaction-current Claim eligibility guard. A pending presentation tag must not be inserted into a closed domain enum unless the model defines that state.

An inspection view might be:

```
{:projection/type :request/card
 :projection/status :provisional
 :gesso/command-id C42
 :execution/id E42
 :resource-instance/id card-owner-instance
 :premises {:observation-ref card-observation-ref}
 :subject {:request-id R}
 :presentation
 {:phase :claim/pending
  :display-helper-id Alice
  :label "Claiming..."
  :busy? true}}
```

The schematic `:display-helper-id` is a view-level cue, not HumanHelp's persisted primary `RequestAssignment` relationship. Neither it nor the pending label may be used as the current-model eligibility guard or submitted as an already-established assignment. Part IV's authoritative operation protects the real relationship and its current eligibility conditions.

This does not assert `:request/status :claiming` as authoritative domain data. The private representation may be smaller, but the categories remain recoverable. A renderer consumes admitted presentation state; it does not establish provenance by writing `:projection/status`.

The admitted command boundary supplies or admits C42 before the first possible send. Allocation happens once for the intent and is retained through retry. E42 names one local execution; `card-owner-instance` names one presentation resource. Neither substitutes for the command ID.

Provisional entry and presentation evidence cases

The following pairs exercise the boundary between an admitted local presentation and independently established authoritative effects. They specify finished-system acceptance, not a particular browser helper API.

Eligible positive case	Adversari
Alice has a trusted, scoped Request R observation at X17 and binds C42 before installing a pending Claim view.	A browse
A creator previews a new Request from admitted collection/policy context, explicit draft inputs and a temporary local identity.	The previ
C42's success is admitted while no canonical Request card has completed installation.	A settlem

A positive witness must not be made to pass by constructing a fabricated admission map. Its intended source and owner bindings are inputs to the admitted mechanism, while the adversarial variant changes exactly the relevant premise. The physical test must observe actual installation or its absence; a copied inspection envelope is not that evidence.

Optimism cannot confer authority

An adversarial browser can fabricate provisional values, labels, bindings, and purported machine state. The trusted model boundary independently establishes principal identity, command binding, and current authorization. It never accepts an optimistic assignment or browser authorized flag as permission.

This remains true for a legitimate binding rendered by the server earlier. Membership or policy may have changed. Historical issuance or observation evidence is not current authorization; an observed-basis claim is not proof the caller acquired or adopted the observation. Trusted authentication and transaction-current business guards protect **new incorporation**. Resolving a previously incorporated command is a different operation: its immutable command binding, current authorization to seek resolution, and permission to disclose a historical result are checked independently. Revocation of a current permission cannot rewrite a past incorporation; neither does a retained result grant perpetual disclosure or new execution authority.

The model chooses legal authoritative outcomes. Choreography and its admitted presentation contract choose provisional and resolution behavior. Views choose the visual treatment. These responsibilities must not collapse into a helper that changes the DOM and independently decides business policy.

Separate command knowledge from presentation progress

A participant tracks several questions:

Dimension	Examples of established state
Command knowledge	Qualified incorporation of the bound command, terminal rejection under the declared exclusion policy, or unresol
Physical attempt	Preparation, delivery, aborted/failed attempt, submitted attempt, or qualified invocation-specific result; one attemp
Acquired evidence	A particular admitted Request observation is available.
Installation progress	Not requested, pending, completed, failed, or indeterminate under its physical-effect contract.
Adopted presentation	A scoped observation or provisional projection owns the current visible representation.
Recovery ownership	This execution or another admitted owner is responsible for remaining work.

This does not require a separate mutable store for every row. It requires enough information to avoid false implications between them. A command can be known committed while a fresh card is unavailable. An authoritative card can be installed while attribution to the original command remains unknown.

Recording acquired evidence is not recording completed visual adoption. A semantic transition may request installation and enter a pending-installation state. It cannot discharge acquire-and-adopt obligations until the required physical completion has been admitted, as specified in Part IV's adoption contract. A terminal settlement may release the local execution while a provisional DOM remnant still waits for canonical replacement; release does not relabel the remnant authoritative or erase independent acquisition/adoption debt.

Worked example: HumanHelp Claim

Alice has an admitted Request-card observation for R at basis X17. Its scope includes the Request, projection/schema version, authority/database incarnation, temporal interpretation, and permitted visibility context. Its admitted projection says the Request is open and represents no active primary helper under the qualified RequestAssignment read/coverage contract. That is not a promise that a scalar `:request/helper-id` exists in HumanHelp's production model, or that a pre-read proves absence at incorporation. Preparation offers a Claim affordance using public model semantics.

This observation supports the example's provisional presentation. It does not prove Membership, policy, or Request state will still permit Claim when the server receives it. Part IV's model operation protects those conditions at incorporation.

Activation binds C42 to the Claim intent and creates E42 with its own resource instances. The admitted entry path records those identities before issuing any transport effect. A retry reuses the existing C42 binding rather than allocating a replacement intent.

The provisional overlay can show Alice as tentative helper and disable the conflicting Claim affordance. It does not replace the retained authoritative observation or authorize further domain actions solely from that tentative assignment.

Admitted event or evidence	Command knowledge	Presentation continuatio
Successful `:committed` result for C42.	Incorporation established.	End or transform the per
Successful permitted `:already-incorporated` result.	Prior incorporation established.	Use the accepted success
Authoritative `:rejected` result.	Rejection established under the command policy.	End the pending overlay
`:commit-unknown` or indeterminate delivery.	Outcome unknown.	Use a truthful unknown/
Current Request observation without command attribution.	C42 may remain unknown.	Supersede the provisiona

Model outcomes map explicitly to presentation settlements `:accepted`, `:rejected`, and `:unknown`. Labels such as confirmed, reconciled, and superseded may describe how provisional ownership ended. They are not extra model outcomes or substitutes for incorporation evidence.

1. Accepted command, card still pending

   Suppose the server establishes C42's incorporation but fresh card acquisition fails. The browser can know Claim succeeded while lacking an installable current card. It must not turn the acquisition failure into rejection or overwrite established incorporation with unknown.

   Presentation can say the action succeeded and current details await refresh. An old observation remains historical or awaiting refresh. Tentative values do not become a complete authoritative card merely because the command succeeded.

   A result projector may supply an admitted observation with adequate scope and progression. It then follows the installation contract: record installation pending, issue a scoped effect, and adopt only after actual completion is admitted. Focus-restoration failure afterward does not undo that adoption.

2. Rejection with a conflicting current state

   Suppose Bob's different command claims R first, and the qualified command-resolution mechanism also establishes terminal rejection of C42 under its declared exclusion/fencing policy. A subsequent admitted card may show Bob as primary helper. These are separate facts with separate evidence: rejection of C42 and the observed state of R. If all that is known is that one Alice submission failed its guard, the browser has evidence of that **attempt**, not automatically a terminal command rejection; another compatible C42 attempt may still have incorporated.

   The rejection reason comes from permitted model evidence, such as already claimed under this simplified policy. Presentation cannot infer that reason from the card. A rejected result need not always contain a disclosable card: authority may permit the outcome but not fresh details, or only a less detailed message.

   One aborted physical attempt is insufficient to establish command rejection; a competing compatible attempt may have incorporated C42. Part IV's command-disposition contract determines which evidence is definitive.

3. Timeout and two recovery tasks

   A settlement timer establishes a local deadline event, not command rejection. Admission checks its resource owner and relevance to the current phase. If definitive evidence was already recorded, a late timer cannot replace it with unknown.

   For an unresolved command, the runtime may retire E42's provisional ownership and enter or transfer recovery. Two tasks can remain:

   - acquire an authorized current card to recover presentation;
   - resolve C42 through permitted command-result evidence to recover attribution.

   The first does not automatically complete the second. A read-only resolver also does not supply an executable recovery driver. The contract names the owner, triggers, retry policy, evidence source, retention, and behavior when automatic progress ends.

   Retry exhaustion may stop automatic work while leaving the command unknown. The product can provide manual reconciliation or another declared handoff. It cannot silently replace C42 with a new command merely to resume a pending UI. A successor recovery owner receives newly admitted rights for its own resource instances; inheriting C42's immutable

intent does not inherit E42's retired timeout, DOM, transport, or role-local knowledge. Failure to disclose a retained historical result may leave C42's participant attribution unknown even if a separate authorized card read restores current presentation.

4. Authoritative supersession without attribution

   Suppose an authorized observation at X19 shows Alice as helper and is permitted to replace the provisional card. After installation completes, that observation justifies the display. C42 remains unknown if no command-specific evidence establishes its disposition; another cause could have produced the same state.

   If C42's success arrives later, it may resolve retained command knowledge through the current resolution owner. It cannot revive E42, regress the card to an older result snapshot, or repeat Claim. Installation and command-resolution rights are distinct.

   For a stronger temporal case, suppose C42 really incorporated Alice's Claim, then an authorized manager's later Reassign installed Bob as the new primary helper. A permitted compatible C42 replay still establishes Alice's **historical** success. A separately admitted complete current Request/ RequestAssignment observation establishes Bob's **current** responsibility. Neither can be substituted for the other: the result cannot reinstall Alice, and Bob's card cannot prove C42 failed. Scope, temporal context and qualified provider progression, not increasing-looking basis labels or callback arrival order, determine which observation may replace which presentation.

5. Conditional restoration instead of unconditional rollback

   An optimistic snapshot captures a previously justified presentation. It does not prove authority still has that state. Restoration is allowed only under the current owner, target instance, observation context, and intervening adoption or local-edit policy.

   The safe default removes the owned provisional overlay and renders from the current admitted base where available. If the implementation restores captured content, its guard is checked at the physical mutation boundary. An earlier Boolean check is insufficient if another installation or owner change can intervene.

   After X19 is adopted, a late E42 failure cannot reinstall X17. Even before supersession, restoring X17's visual shape retains recovery-pending or unknown classification. It does not assert that the model rolled back or C42 failed. The current physical owner check must be made at the actual mutation boundary; a formerly authorized snapshot is no longer a rollback right after owner, target instance, scope, principal visibility, or independently newer local state changes. If restoration fails or completes only partially, preserve the actual physical presentation uncertainty and repair debt. No presumed rollback success can close command resolution or unrelated Refresh obligations.

6. Paired command and presentation histories

   These paired cases preserve the R4 counterexamples while testing the stronger Part IV command and adoption contracts:

Positive history
A qualifying command-specific outcome establishes C42's incorporation, but current card installation is pending.
Qualified terminal closure excludes every compatible C42 incorporation attempt before presenting rejection.
The owner retires E42 after timeout, then a new authorized owner resolves C42 or independently reacquires the card.
A current scoped observation installs Bob after a later authorized Reassign while a compatible replay reports C42's earlier success for Alice
A guarded rollback restores a still-compatible local view without claiming authority.

   The positive controls must reach the same trusted, browser and physical boundaries whose adverse cases they challenge. A fixture that simply assigns `:committed`, `:rejected`, `:authoritative`, or `:installed` cannot issue evidence or satisfy an actual-operation acceptance claim.

Every provisional trajectory requires resolution semantics

The operation accounts for accepted incorporation, rejection, unknown outcomes, authoritative supersession, local cancellation, incompatible-plan retirement, and physical-effect failure wherever those cases can occur. Every terminal path ends or explicitly transfers provisional and resource ownership. Physical cleanup has its own completion/failure contract.

Not every operation needs every branch, but omitted branches need a supported reason they cannot occur. Static preflight need not predict dynamic facts; it requires declared handling for conditions capable of stranding the operation. Each transition identifies the current execution and effect owners, which command-attribution and acquisition/adoption debts survive, which physical resources are retired or transferred, and which effects must be retried or repaired. Semantic retirement takes effect without waiting for best-effort DOM cleanup; a still-visible provisional remnant remains visibly provisional. A recovered authoritative card can settle presentation while the original command's disposition remains unknown.

A recovery path is not proof it eventually succeeds. Liveness claims identify continued authority, backend/network availability, retention, scheduling, and user-action premises. Unknown and retired states must remain safe even when those progress premises fail.

Cancelling a local wait or dismissing an overlay does not cancel an authoritative command. A genuine cancellation operation has model-defined effects and can itself be too late, rejected, or uncertain. Wording preserves this distinction where it affects the user's decisions. The recovery owner must not reuse a retired execution's timer, callback, transport, DOM target rights, or authenticated principal authority merely because it retains a compatible semantic command identifier. A terminally failed submission attempt does not by itself establish a terminally rejected command.

Concurrent provisional work and application-defined resolution

Different clients can hold incompatible provisional projections without creating multiple authoritative histories. The model resolves commands under its policy. Within one browser, overlapping provisional effects also require a policy; untracked DOM replacement must not choose it accidentally.

An operation family declares one of these or another equally precise strategy. Its conflict relation covers the actual read, write, field, query-membership, target/installation, and separately retained local-draft scopes used by the provisional operations; equal or different DOM node IDs alone do not establish that semantic effects are disjoint. The selected policy states its arbitration and successor-owner rule at the point where overlapping work is admitted:

- serialize conflicting commands or disable conflicting activation;
- permit independent overlays on disjoint declared scopes;
- supersede local presentation ownership while retaining older command resolution responsibility;
- compose or rebase overlays under an admitted dependency/conflict rule.

Gesso does not assume all optimistic effects commute. If C2 depends on C1's provisional output, that dependency and its resolution are explicit. Rejection, unknown resolution, or changed authoritative results for C1 can invalidate C2's projection. C1's provisional output cannot silently become an authoritative premise for C2.

The continuation depends on whether C2 is only a local draft, has an admitted but unsent command, or may already have been submitted. A local draft may be rederived, suspended, discarded, or presented as needing review under the declared strategy. An already-issued C2 must retain its own immutable command binding and independent disposition/recovery; retracting its overlay does not retract its possible authoritative effects. A dependency failure is not automatically a terminal rejection of C2. Rebase, composition and supersession must preserve any independently newer adopted observation and outstanding command and installation obligations.

Rebasing an overlay does not authorize changing an already-issued command's immutable arguments. Changed intent follows a declared new-command or cancellation/replacement protocol, retaining uncertainty about any old command.

Application-defined strategies remain ordinary Clojure/ClojureScript behind typed registered contracts. Inputs identify admitted observations, scopes, owned overlays, command knowledge, current owners, relevant dependencies, and declared local inputs. Outputs specify retention, removal, recomputation, deferred admission, or acquisition, with explicit unresolved debt and resource handoffs. They do not write domain state, disclose a protected observation, transfer authority, or create evidence by returning a success tag. A stale result or callback from the superseded owner is rechecked against current owner and scope before any physical mutation.

Analyzable strategies can support derived properties. Opaque strategies retain asserted behavioral premises while structure and runtime bindings are checked. No strategy may suppress required recovery debt or relabel an unknown command as rejected merely to simplify the display.

Multi-target presentation and membership

One operation can affect a card, count, board membership, and notification surface. Its presentation contract declares scopes and required local atomicity. A collection of DOM swaps is not automatically one atomic installation.

Where partial installation is possible, the runtime records **per-target** completed, pending, failed, or indeterminate effects with their source observations, coverage, owners, and recovery paths. It cannot advance a whole-board adopted frontier for all targets because one card installed. A requirement for atomic multi-target presentation must name an actual all-or-nothing physical mechanism or a declared compensating protocol whose intermediate state and failure cases are admitted; ordinary sequential DOM writes cannot be called physically atomic. Retiring one card's provisional owner need not remove unrelated overlays or advisory debt. A later broad board refresh also cannot erase a separately newer creator-local card or membership without the selected adoption policy.

HumanHelp can permit immediate correlated self-action updates while other participants receive advisories. A card alone does not prove a complete new board selection. Search, sorting, terminal-item visibility, and local filters preserve that scope distinction.

No silent downgrade

Missing required optimism, command identity, scoped entry knowledge, or resolution support prevents an enabled incomplete realization. The framework cannot silently emit an ordinary `hx-post` and hope for equivalent behavior.

An explicitly declared non-optimistic alternative may be selected when its own realization is complete and is permitted by the authored operation and application contract. That is an explicit construction or invocation decision, not a catch-all fallback for missing plans or adapters. In a managed HTMX operation the required command/plan/correlation gate is enforced before the physical send becomes irreversible; a missing projector, wrong owner, or unready transport handoff fails that attempted path closed without a naked replacement request. Other supported transports may realize asynchronous provisional derivation under their declared buffering and failure contract: one particular HTMX bridge's current synchronous limitation is not a finished-system prohibition.

An unavailable affordance preserves operation identity and reason without an active weaker transport path; it does not count as a working capability in an eligible acceptance scenario. Raw HTMX remains legitimate for explicitly unmanaged interactions and for authored non-optimistic paths with their own complete authorization, effects, and recovery semantics.

Views remain free to use labels, badges, spinners, CSS, layout, and accessible treatment. They render state rather than choose business outcomes or redefine progression.

Paired resolution and capability cases

The finished application must admit useful eligible paths, not merely pass negative checks by declining all optimistic work.

Positive admitted history
E1 times out; successor E2 acquires a current card and later resolves C1 under its retained binding.
C2 is a not-yet-submitted provisional draft depending on C1; C1's rejection selects the declared suspend/rederive continuation.
Two independent local overlays use proven disjoint declared semantic and installation scopes.
An operation installs its card while a required board count remains pending, then follows a declared repair.
An authored non-optimistic operation uses its complete admitted HTTP path, and a managed optimistic operation issues C42 through its fu

These cases must exercise their claimed model, admission, browser and host boundaries. A diagnostic map or artificial rejection before the intended boundary is not proof of correct physical enforcement.

Continuity

Continuity preserves browser-local interaction state across representation replacement: focus, selection, scroll, open details, and explicitly selected local drafts. It does not establish domain facts, authoritative provenance, command validity, or agreement.

Ordinary continuity belongs behind the browser-effect boundary. Portable choreography observes the effect outcome defined by that boundary, not separate focus-restored or layout-finished events. An application can deliberately give a browser interaction semantic meaning through an explicit contract; such meaning is not inferred merely because continuity captured the interaction. A permitted authoritative installation remains completed even if an optional focus, selection, or scroll follow-up fails; a failure of physical installation itself retains its separate incomplete-realization and recovery contract. No continuity result may manufacture command acceptance, authoritative observation, installation evidence, or permission to reveal protected data.

Ownership and physical captures

A continuity capture belongs to an admitted replacement/effect scope and a concrete target instance. It need not invent a command execution for an ordinary fragment refresh. Its identity distinguishes the logical slot, capture instance, owner, and relevant target incarnation. Part II's resource-instance contract applies.

The imperative shell can retain opaque DOM references and capture resources. Portable semantic state retains the owner/resource references and lifecycle needed to control their use. A DOM node still existing does not prove its owner is current. A delayed restore must not consume or overwrite a newer capture. Restoration must check the current continuity slot, replacement owner, target incarnation, applicable rendering/disclosure context, and the local edit generation of each value it may overwrite **at its actual mutation point**. Admission when the capture was taken or when a deferred callback was scheduled is not a durable right to mutate. A user typing after capture can invalidate restoration even when no second board replacement occurred. If an owner is retired while a callback is queued, semantic permission ends immediately and the remaining physical handle is cleanup debt, not a renewed grant.

The default capture registry binds a capture to the restore implementation and capture format which produced it. Re-registering a box implementation must not reinterpret an older capture under incompatible code. An explicit versioned migration may be supported; otherwise incompatible captures are retired. This binding does not permit an old implementation to execute after its rights have been revoked by the current runtime policy. The bound implementation is an interpretation constraint, never an independent authority to restore; a compatible capture still requires current owner, target and edit checks.

Capture/restore hooks are ordinary host code. The framework can validate their interfaces, registered identity, declared footprint, and resource use; claims about arbitrary hook behavior remain explicit assertions unless established by another mechanism. A custom hook must not acquire semantic installation rights by calling an internal DOM helper.

Continuity does not preserve arbitrary DOM state

A continuity policy selects what may be captured and restored. It must not copy an old fragment wholesale or restore authoritative security/context fields such as credentials, anti-forgery tokens, operation bindings, principal identifiers, or hidden command arguments as if they were harmless input drafts. A generic capture of every input, including hidden fields, is not a sufficient security policy. Admission must classify and constrain permitted local edits independently of their HTML element type and reject or exclude protected fields before restoring into new server markup. Captures are local interaction data, not proof that a previously disclosed value may be disclosed again.

Restoration respects the newly admitted rendering. It may reopen a Request already present and permitted in the new board, but cannot recreate a removed or undisclosable Request. Duplicate or ambiguous element keys require a declared failure policy; selecting the first matching node can restore into the wrong control. A positional fallback is safe only where the author has established the relevant structural identity and compatibility across replacement; changing collection order or membership cannot justify restoration solely by equal list index. On missing or ambiguous identity, omit the optional restoration or follow its declared recovery path rather than guess a recipient.

Local draft preservation also needs a product policy. A draft can be retained as local user input without claiming it matches authoritative data. If replacing a form would lose important unsaved work, the application can declare a prompt or block-before-install policy. That is an explicit interaction requirement, separate from ordinary best-effort focus or scroll restoration. It cannot turn an already committed model operation into a rejection. Where the user edits a field after its capture, that field's newer local intent prevails over the old capture unless a separately declared and admitted user-confirmed conflict policy says otherwise. A selection or focus restore may be omitted when its intended editable target no longer exists; this local failure does not invalidate a completed authoritative installation.

Search text and query meaning must remain aligned

A search control's draft text is local input. The applied query and the membership of its result set have semantic meaning. Restoring one does not establish the other.

The author declares whether search is **local filtering of an already adopted authorized cohort** or a **server-side authoritative query selector**. HumanHelp's text search, local sort order, priority and terminal-visibility controls may filter and reorder the selected Organization/Location's adopted board without changing its authoritative acquisition scope. Applying `cabinet hinge` to a locally available `cabinet` display can be valid if both are view predicates over the same authorized complete cohort; it is not evidence that previously excluded Request records were acquired.

In contrast, if the board was acquired under a server-side search predicate `cabinet`, restoring the control to `cabinet hinge` must not silently label those results as satisfying the latter authoritative query. The application keeps draft versus applied query identity distinct, performs a permitted acquisition for the new selector, or follows another declared transition. The same applies to server-side sorting, filters, pagination and changes in principal, Organization, Location, temporal or disclosure scope. A continuity callback cannot grant another Refresh, widen the original observation, or discharge unrelated acquire-and-adopt debt.

Filtering a locally adopted set may be a legitimate view operation, but cannot prove complete membership beyond that adopted cohort. The query contract states whether a control is local presentation, an unapplied draft, or the selector of authoritative acquisition; the DOM's restored text alone is not a source of query provenance.

Paired continuity and query-ownership cases

These positive/adverse cases state what a finished Gesso application must admit and what the browser may not infer from a captured local value.

Eligible case
A current replacement restores Bob's permitted search draft, focus and open R17 details after installing a newly authorized board still conta
A capture restores an unchanged draft through the still-current slot and local edit instance.
A stable unique control key survives an admitted replacement.
HumanHelp filters its adopted authorized cohort locally while preserving a separately declared authoritative board query.
A new board installation completes and optional focus restoration fails.

A positive control requires the real current owner and physical completion where claimed, not an inspection map decorated with ownership fields. An adverse control must reach the intended stale-callback, query, or disclosure boundary rather than pass because an unrelated fixture failed first.

Worked example: replacing a HumanHelp board

Bob has locally filtered his adopted HumanHelp Request board by the text `cabinet`, opened Request R17, focused Refresh, and scrolled partway down. His **authoritative** board query identifies the admitted Organization/Location, principal/visibility policy, projection version, temporal scope and complete collection cohort, not that local text filter. A separately granted Refresh acquires a complete authorized board observation for that same authoritative query at X31. The example assumes a qualified provider comparison establishing the relevant progression; X31 is a symbolic basis, not an ordered number.

If the product instead implements `cabinet` as a **server-side** authoritative search selector, that selector belongs in the acquisition identity and the query must be revalidated when the user changes it. Continuity cannot silently reinterpret one fetched cohort as the result of another server query.

An inspection view of a continuity capture is:

```
{:slot :request-board
 :capture-instance/id I1
 :owner {:replacement/id O1}
 :target-instance/id B1
 :capture-format :board-continuity/v1
 :local
 {:focus-key :board/refresh
  :scroll-anchor {:request-id R17 :offset 12}
  :open-request-ids #{R17}
  :search-draft "cabinet"}
 :context
 {:applied-query-ref board-query-ref
  :captured-edit-instance edit-instance-1}}
```

The actual resource can be opaque and include implementation-specific physical handles. Here `board-query-ref` denotes the **authoritative cohort selector**; `:search-draft` records an independently mutable local filter. The capture's edit identity must be checked against a later user edit at the actual restore point, not merely compared once while preparing the swap. The context prevents a draft restore from misrepresenting the applied query or overwriting newer edits. Neither `board-query-ref` nor X31 is admitted merely by appearing in this inspection map. The capture cannot issue another Refresh grant, change Bob's authorized visibility or supply its own adoption evidence.

A replacement follows these phases:

Phase	Required behavior
Prepare	Admit the observation, authorized authoritative query and proposed target; establish the current replacement owner, t
Capture	Capture only locally permitted state under that owner; optional box failures remain local diagnostics, not authorizatio
Install	Recheck current owner, target instance, visibility, scoped progression and any independently newer adopted card/mem
Admit completion	Correlate the host's physical outcome to the issued installation effect; record completed, failed, partial or indeterminat
Restore	Restore only still-applicable local state after installation, checking owner, target instance, surviving controls and post-
Release	Revoke the exact capture's restoration rights before idempotent physical cleanup; never release a newer owner's captu

If installation succeeds but Refresh is absent, focus restoration may fail while the new board remains adopted. The result distinguishes installation from optional continuity:

```
{:installation
 {:status :completed
  :effect-id install-effect-1
  :observation-ref board-observation-X31}
 :continuity
 {:status :partial
  :restored #{:scroll :details}
  :skipped {:focus :target-no-longer-present}}}
```

This inspection result is not a browser-issued credential. The admitted shell completion supplies the evidence the semantic owner needs. It must not be converted into a generic failed swap which triggers stale rollback. The completed installation records only its justified target/scope and does not clear independent newer Live requirements or another actor's Refresh debt. If a separately newer self-action card or membership has already been adopted, the whole-board replacement must preserve it through a declared reconciliation or decline the conflicting footprint, rather than treating the broad board response as permission to overwrite it.

Now replacement O2 captures I2 while I1 still awaits layout. O2 owns the current board instance, and Bob has typed `cabinet hinge`. I1's delayed callback must not restore `cabinet`, consume I2, move focus back, or reset O2's scroll state. Checking ownership only when scheduling the callback is insufficient; it is checked again immediately before the delayed mutation. A callback that retrieves the same logical target or matches the same DOM key can nevertheless belong to an obsolete physical target incarnation. A new actor/tab, principal scope, query selector, or unrelated replacement cannot inherit I1's rights merely by using the same visible target name.

Even without O2, a user edit after I1's capture may make restoration stale. The policy compares the relevant local edit instance or equivalent ownership information before overwriting input or focus. The rule is not that a numerically larger capture always wins; it is that only the current permitted owner can perform the particular mutation.

Open details are intersected with the newly authorized rendering. A local capture can request R17 be open if R17 remains present; it cannot recreate R17 if the new scope excludes it. A scroll anchor that disappeared uses the declared fallback, such as the nearest surviving anchor or a bounded raw offset.

Failure and cleanup

Optional capture failure must not prevent an otherwise permitted installation. Optional restoration failure must not erase an installation already completed. Promises and deferred callbacks need the same failure isolation as synchronous hooks; catching only synchronous exceptions does not contain asynchronous failure.

If installation itself partially applies, the result must preserve that uncertainty and follow the repair contract. Neither the semantic state nor the UI may pretend the entire fragment is installed. Continuity success cannot compensate for missing installation evidence. If the host can prove complete installation for one independently scoped target but not another, only that target's required coverage is discharged. Where the declared operation promises all-or-nothing board replacement, it must provide a real physical atomicity mechanism or a stated compensation/recovery protocol; an HTMX callback sequence alone supplies no general multi-target atomicity.

The result of optional continuity failure after a completed installation cannot reclassify the prior model operation as rejected, revoke the installed observation, or consume another replacement's pending repair. A declared **mandatory** user interaction (such as a user-confirmed unsaved-draft barrier) belongs to its own preparation/permission contract; it may block a still unissued replacement where authorized, but does not retroactively cancel a completed authoritative command or installation.

Semantic retirement revokes restoration rights before best-effort physical cleanup. Failed cleanup is diagnostic/resource debt, not authority for an old callback to act again. Resource release should be idempotent and scoped to the resource instance it owns.

A style lock illustrates the requirement. Seeing the same min-height value that an old effect once wrote does not necessarily prove it still owns that value; a newer effect might have written the **same** value. Compare-and-restore based solely on CSS value equality is therefore not an ownership check. Cleanup must use the resource-instance ownership contract or a justified equivalent: release may restore a previous style only when it still owns the affected property/lock and the restoration cannot undo a later owner's write. A later owner's equal-value write must remain protected just like a distinct-value write. If ownership cannot be established, omit the risky local cleanup, report retained physical resource debt, and never regain semantic rights. An idempotent repeated release of a retired handle must not mutate any current owner's state.

Physical resource lifetime may outlast semantic ownership. In that case the remaining handle is powerless cleanup debt. Portable Choreo needs only the contracted installation/effect outcome; browser diagnostics may report the failed optional focus, scroll, or capture operation separately.

Board replacement and cleanup conformance pairs

The complete positive path is an authorized, adequately scoped and permitted X31 board observation, the real physical installation result, a separately current continuity restore, and the exact resource release. Tests must not skip the host mutation boundary by supplying a completed-looking result map.

Positive physical history
Bob filters the adopted authorized board locally by `cabinet`, Refresh installs the same authoritative cohort at qualified X31, and unchanged O1's board installation completes; its optional focus restore fails because Refresh is absent.
O2 supersedes O1 and I2 restores a still-current edit.
X31 board installation is permitted while a separately newer self-action card exists.
A continuity height lock L1 is released while no successor owns that style.
A fragment install succeeds and a later optional restore Promise rejects, followed by a permitted new refresh.

These are finished-system obligations. The scoped source inspection of a particular browser realization does not prove these cases for every custom continuity box, HTMX version, server query mode or deployment.

Views and UI annotation

Views declare semantic intent and render prepared presentation. They do not redefine choreography, duplicate domain authorization policy, or independently assemble transport/protocol parameters for a managed operation.

The ordinary path is:

1. a public semantic operation and choreography declare the behavior;
2. application assembly resolves its plan, transport, acquisition, settlement, server capabilities, and artifact requirements;
3. preparation binds per-render arguments and admitted observation context;
4. the view supplies element choice, labels, classes, layout, and accessibility;
5. the renderer derives a complete managed realization from the admitted binding.

A view may use a prepared binding directly or a concise declaration normalized through the same preparation boundary. Multiple convenient authoring forms must not create multiple independent meanings of Claim. The preparation result records both **which semantic operation is intended** and **which actual entry mode is offered**. An input-collection entry is not an unbound final mutation; a temporarily unavailable operation is not an incomplete executable action. The render-time decision expresses qualified, potentially stale presentation knowledge. It is not a durable grant to execute an operation after changes to Membership, Organization, RequestAssignment or other protected state.

Binding values versus operation-owned policy

The registered operation owns its choreography, supported realization modes, settlement mapping, provisional derivation, recovery policy, required resources, and compatibility constraints. Per-render binding supplies only admitted variable inputs: subject/arguments, observation references, and target context where the operation permits them.

Those binding fields have a closed schema. A view cannot override an operation's plan, timeout semantics, principal authority, rollback policy, or result mapping by inserting an extra key. Presentation options occupy a distinct checked area; unknown semantic keys fail with a precise error. Preparing a visible action cannot silently substitute a second semantic operation or drop the required choreography mode, route, browser artifact, acquisition or settlement contract because an available binding happens to contain insufficient metadata. Such a missing/incompatible construction edge is a typed defect, not a product policy denial. The plan and its trusted server execution route are derived from the assembled operation; a view-supplied identifier is never an independent authoritative registry entry.

For example:

```
(defn prepare-request-card [trusted-context admitted-card]
  (let [card (observation/value admitted-card)
        claim (actions/prepare
               trusted-context
               {:operation request-choreo/claim
                :arguments {:request-id (:request/id card)}
                :observations {:request-card admitted-card}})]
    {:request {:id (:request/id card)
               :title (:request/title card)
               :status (:request/status card)}
     :actions {:claim claim}}))
```

This illustrative preparation path verifies that the observation, argument, operation, principal context, and target scope agree. It obtains availability through public model/application interfaces. The renderer does not rerun a second Claim policy in Hiccup. This initial availability judgment is **qualified to the preparation observation and context**; an enabled button does not establish that the later transaction-current guard will succeed. The public model still owns the current guarded Claim and real RequestAssignment transition. The browser must not turn a stale visible Claim into either unauthorized execution or a fabricated terminal rejection.

The prepared affordance need not serialize principal credentials, server capabilities, or proof objects. When the browser returns binding data, the server re-establishes trusted principal identity and the required semantic bindings. Canonical command arguments use the same declared mapping as Part IV; a domain Request ID must not be confused with `:http-request-id` or another transport ID.

Availability, input collection, and construction defects

Prepared actions distinguish legitimate product states from broken realization:

Preparation result	Meaning and permitted rendering
Available	A complete admitted entry affordance can be enabled.
Policy unavailable	Current prepared semantics do not offer the operation; hide or disable under the declared disclosure
Needs input	A declared interaction must collect further user input before final invocation; the offered input-colle
Deferred or dynamically unavailable	A declared prerequisite cannot yet be established; show the contracted pending/unavailable state ar
Construction failure	Required operation, plan, binding, capability, route, or semantic input is missing or inconsistent; rej

These classifications have closed payload contracts. Missing or `nil` entries cannot silently stand for every kind of unavailability. A product may use a compact representation internally, but the preparation boundary must establish which meaning it has.

The categories distinguish **product intent** from **realization evidence**: policy-unavailable is a model/preparation decision under the specified disclosure context; needs-input is an admitted and complete entry **flow** whose final mutation requires additional bound input; deferred names the missing dynamic prerequisite and its recovery/refresh contract; construction failure identifies a missing or incompatible mandatory assembly/binding edge. The same absent value cannot be reclassified as whichever of these renders most conveniently. Legitimate product non-availability does not count as an implementation defect, but making an intended eligible capability permanently unavailable cannot satisfy positive whole-application acceptance. A renderer cannot downgrade a construction failure into a disabled button, nil action, ordinary HTMX send or reassuring diagnostic-only success.

A missing Reassign helper ID is not inherently a defect if the action is meant to open an admitted helper-selection form. That entry action is fully specified, and final Reassign remains unavailable until its required arguments are bound. If the application tries to emit the final mutation without a required helper ID, that is a construction failure. The declaration determines which case applies; the renderer must not guess.

For the eligible HumanHelp manager's genuine Reassign, the entry flow obtains a candidate helper list under the actual authorized Organization/Membership and target-selection contract, permits selection and confirmation, and then prepares the distinct final Reassign command. The selected helper ID is an untrusted **argument**, not proof of the helper's current eligibility. The trusted server reestablishes principal, manager permission, selected target identity and current target eligibility at incorporation; it guards the actual primary RequestAssignment replacement/history. A successful self-Claim or an empty, permanently disabled selector cannot serve as this positive product witness. When the product's separately promised response interaction has not yet been assigned a meaning, preparation may not simply invent Claim as that operation to satisfy the expected capability list.

An inspection action set could look like:

```
{:claim
 {:availability :available
  :binding-ref claim-binding-ref}
 :cancel
 {:availability :unavailable
  :reason :policy/not-offered}
 :reassign
 {:availability :needs-input
  :entry-binding-ref helper-selection-binding-ref
  :required-inputs #{:helper-id}}}
```

The references resolve to admitted objects. A plain map with these fields is not sufficient. Reasons must obey disclosure policy; a disabled button must not leak a private Membership or Organization fact. Public reasons may be intentionally coarser than private admission diagnostics; an unauthenticated or differently authorized observer cannot infer protected membership, role, account existence or Request visibility by comparing availability reasons across rendered views. A disabled or busy control is presentation, never the server's anti-replay, rate-limit, authorization or command deduplication mechanism.

Preparation validates the complete **declared required** action set for that rendered surface or product state, including actions the view will hide. This does not require every possible application operation to be offered on every page; the independent application capability inventory declares which operations and input-collection flows are expected in which eligible contexts. A hidden action cannot conceal a malformed **required** binding, and omitting an entire expected action from both the preparation code and rendered markup cannot make it disappear from the external acceptance oracle.

Application assembly must close operation identity, browser plan, compatible artifact, trusted server entry/route, acquisition, result mapping and effect realization for the enabled path. Rendered-surface inspection checks the actual dynamic Hiccup produced,

including branches that static enumeration misses when exercised; it cannot prove that every possible Clojure branch was enumerated. Separate eligible positive and nearby adverse witnesses test the complete path from independently declared product intent to real installed control, user input, trusted model mutation and visible outcome. Tests derived only from buttons that happened to render cannot detect a missing product action. Nor may a source-derived report promote its bounded rendered-surface check into a universal whole-application certificate.

Paired affordance and availability acceptance cases

These cases require independently declared product intent and real construction, not copied inspection maps or a catalog inferred solely from currently rendered buttons.

Eligible positive path
Claim is prepared from a qualified Request observation, rendered with its admitted operation/plan/binding, and later checked under trusted
A manager opens a complete helper-selection entry, chooses a distinct eligible helper and confirms final Reassign.
A legitimately policy-unavailable action is hidden under permitted disclosure rules, while required actions are still structurally checked.
A deferred action reports an admitted prerequisite and recovery path without pretending the user can invoke it yet.
The renderer exposes only disclosable availability and accessible busy/disabled status, while the server binds the actual current principal.

This table defines expected finished-system semantics, not a claim that the present Gesso view helpers already implement every availability category.

A small view API

Once preparation succeeds, view code can remain ordinary Hiccup:

```
(defn request-card-view [{:keys [request actions]}]
  (let [claim (actions/require-prepared! :request/claim (:claim actions))]
    [:article.request-card
     [:h3 (:title request)]
     [:p.status (name (:status request))]
     (case (actions/availability claim)
       :available
       (gesso/action
        {:binding (actions/binding claim)
         :presentation {:pending-label "Claiming..."
                        :class "gesso-action gesso-action--primary"}}
        "Claim")
       :unavailable
       nil
       :deferred
       [:span.action-status "Claim is temporarily unavailable."])]))
```

This example's Claim preparation contract admits available, policy-unavailable (as `:unavailable` in this illustrative view API), and deferred variants. A deferred result has a declared prerequisite and recovery path; it is not a missing plan disguised as a temporary UI state. The final Claim mutation has no input-collection phase; an unexpected needs-input result is a typed contract mismatch, not an implicit nil or enabled empty form. By contrast, an explicitly authored Reassign **entry** may have a complete helper-selection flow even while its final mutation still needs an eligible selected target. Construction failure is raised by preparation/validation before this view can serve an incomplete active control. A hidden required action remains subject to the declared preparation and application-acceptance checks.

The renderer derives routes, HTMX attributes, command-wire fields, plan and artifact references, correlation, and progression handling from the assembly. A view author does not repeat these details in strings. If required metadata is missing, the renderer reports the exact missing edge and rejects incomplete managed construction. An explicitly non-executable unavailable affordance is permitted only when the **declared semantic policy** selected that availability outcome with a complete non-executable rendering contract. It is not a catch-all fallback for missing plan, route, artifact, command binding, or browser runtime. The failing path cannot emit an otherwise active physical request.

Semantic identity is preserved through rendering. Binding, encoding, and transport selection may elaborate an operation but cannot silently substitute another operation or weaken its required protocol. A valid pre-serialization Hiccup annotation records the author's intended semantic identity, but cannot by itself prove that the **physical** browser request matches it. The checked

surface must join the actual method, concrete route, target/submitter behavior, plan, artifact and required transport to the assembled operation. Additional = hx-get=, `hx-put`, native form submission, form overrides, or conflicting string/keyword render attributes may create a second or ambiguous request even when a canonical button's semantic metadata remains intact. The complete inspected physical request coordinates must be compatible with the declared mode; one marked request does not exempt sibling or alternate requests.

The framework may retain in-process Hiccup metadata for pre-serialization inspection; that metadata does not automatically survive HTML serialization or authenticate anything returned by the browser. An opaque encoded binding is not necessarily authenticated, and authentication of its bytes would still not establish current model authorization.

Prepared affordance is not a security capability

The term *prepared affordance* is preferable for the object consumed by a view. If the implementation calls it an operation capability, diagnostics must qualify that meaning. It is not an irrevocable or unforgeable authorization credential merely because the server prepared it.

A trusted execution capability has a different contract: it permits a scoped server action under current admission rules. A view binding cannot supply the transaction handle, authenticated principal, or other trusted resource required by the model.

Browser-returned observation information is an observed-basis claim unless a trusted boundary establishes something stronger. An authenticated receipt can prove its issuer's stated historical fact under that receipt contract. It does not prove the human saw the card, the browser installed it, or permission still holds now.

Gesso constructs or admits stronger semantic values at the boundary with enough information. Web developers should not have to hand-assemble proof wrappers to render an ordinary button. In-process admission discipline prevents accidental misuse; it is not a sandbox against a determined author with arbitrary trusted host access. A received operation identifier, observed-basis claim, plan fingerprint, command identifier or encoded client value is input under its declared trust contract, not an execution credential. Trusted server-side operation resolution, authenticated principal, current domain guards and qualified command-outcome disclosure remain independent obligations.

Raw HTMX and presentation freedom

Ordinary HTMX remains a first-class mechanism for interactions deliberately outside Choreo's promises. The distinction is explicit: an ordinary affordance has its ordinary contract; a managed affordance has a complete managed realization or a precise non-executable failure/unavailability state.

An ordinary route still obeys server authentication and model safety. It must not occupy a route reserved for a managed operation as a way to bypass its protocol. This prohibition concerns the **physical** method and route, including a native form's actual action and submitter override, not only the presence of `:choreo/op` metadata. A view may intentionally use raw HTMX on a separate, legitimate route, or an explicitly admitted non-optimistic realization of an operation when its own contract is complete; it may not use either path to bypass an operation's required protected server-side model semantics. Declaring fewer client guarantees does not authorize direct unprotected writes. A malformed managed request must not silently become raw HTMX because its metadata was lost during a render or swap.

Views own text, element choice, semantic CSS, icons, layout, ARIA, pending labels, and how provisional or authoritative states look. They must not interpret a newer database token as command rejection or choose reconciliation policy. They display the decisions made by the admitted model/runtime contract.

Status must be understandable without relying only on color or motion. Keyboard activation, form submission, disabled/busy semantics, and duplicate activation handling must match the declared entry policy. Loading and provisional treatment must not be falsely announced as authoritative incorporation or adopted current state. An available action and a complete input-collection entry must remain operable using the declared keyboard, focus and accessible-status path; a policy-hidden, deferred or failed construction must not retain an accidental active native/HTMX submit path. Disabling a button alone is not the framework's command deduplication mechanism, nor does an `aria-disabled` or `aria-busy` marker provide server authentication, an actual pending-command lock or transaction-current permission.

Assembly obligations and diagnostics

An enabled managed control creates obligations for its complete path: operation, choreography, projected plan, loaded browser artifact, transport, trusted server entry, model realization, outcome mapping, acquisition, and presentation. Part VII checks these **before each relevant surface is served**, including dynamically generated surfaces. Its later position in this document does not permit serving incomplete controls until a future check runs.

Actual dynamically rendered Hiccup is checked against the admitted ApplicationAssembly at the serialization boundary, so a newly eligible branch cannot bypass preflight merely because it was absent from a sample snapshot. The result is scoped to that actual rendering and the trusted serving path that invokes the guard. It does not enumerate all possible Clojure branches, prove that every deployed server entry used that wrapper, or validate arbitrary pre-serialized HTML without an appropriate enforcement boundary. Subsequent asset, route, deployment or binding changes require fresh applicable admission; a prior successful check is not permanent authority.

Errors preserve source component, operation, role/subject when disclosable, expected binding/plan/capability, and the missing or incompatible edge, with physical request method and route where relevant. Missing plans should not collapse into generic malformed HTML errors. Diagnostics must preserve the difference between policy unavailability, input collection, deferral, and implementation failure. Public-facing errors disclose only permitted reasons; private repair reports may carry the fuller source and correspondence context.

Paired rendered-affordance and no-downgrade cases

The following cases distinguish a checked, useful view from a Hiccup-shaped certificate that has not yet been connected to an actual physical operation.

Eligible positive history
A prepared Claim binding renders a single managed button whose actual POST path, plan, artifact and trusted entry all match the assemble
An ordinary HTMX interaction uses its declared non-Choreo route, while a separate managed Claim uses its exact registered semantic rout
A prepared manager Reassign entry opens its complete input-collection form and, after selecting a distinct eligible helper, prepares the fina
An accessible Claim button displays provisional pending status while C42's incorporation or canonical installation remains unestablished.
A dynamically rendered, user-specific Claim branch is checked under the current assembled application before its Hiccup is serialized.

A successful positive case must exercise the intended browser and trusted server entry under the actual operation contract. A negative test must reach the physical collision, altered request, malformed construction, or missing-admission boundary it claims to test, not merely be rejected earlier by unrelated fixture data.

Clojure and ClojureScript execution

The permanent executable semantic core gives projected behavior one meaning across supported hosts. Shared `.cljc` definitions are the preferred realization where practical. JVM, Node, and Chromium must not carry independently handwritten versions of the same state machine merely because their callbacks differ.

This commitment does not make the whole application portable. Database access, trusted authentication, arbitrary server code, DOM manipulation, and network integration belong to declared host boundaries. Full-language actions remain available there. The core interprets their contracts and admitted completions; it does not need to understand or emulate all application code.

Portability is a **semantic** obligation, not merely the ability to compile the same namespace. Supported plan versions and inputs must give the same transition, role-local value, classification, effect request and ownership decision on JVM Clojure and supported ClojureScript hosts. Where a result depends on equality, numeric representation, map/set membership, ordering, serialization, exception classification or absent-versus-`nil` distinctions, the plan's portable value contract either fixes that meaning or rejects the unsupported value/operation at admission. Different printed diagnostics, source coordinates and physical handle representations are not differences in semantic behavior; differences in branch selection or effect authorization are. An opaque backend basis cannot be compared with host-local lexical or numeric order, regardless of whether both environments can print it.

The executable semantic core is distinct from the later proof kernel/checker. The former executes admitted transition meaning. The latter checks derivations or other stated judgments about that meaning. Compiler, verifier, and proof machinery need not ship to the production browser. The artifact executed by a production browser must be the compatible checked projection, not an independently handwritten approximation of the verified behavior. Build-time checks and tests may observe that semantic machine, but diagnostic sidecars, witnesses or optional proof metadata must not become hidden inputs that change its transitions. A same-source or same-schema claim is not itself a compiler, runtime, host-effect or global refinement proof.

One vocabulary from construction through execution

Part III's role-local vocabulary is canonical:

Node kind	Runtime responsibility
`:local`	Request the declared local action and consume its admitted completion.
`:authoritative`	At the appropriate trusted role, request a public model operation and consume its admitted typed result.
`:branch`	Select a declared successor using an established typed value.
`:send`	Issue a declared participant communication with the required known payload.
`:receive`	Await an admitted participant message matching one unambiguous alternative.
`:await`	Await a declared environment event from its registered source contract.
`:return`	Complete this role's local trajectory with its declared outcome.

Optimism does not introduce a separately handwritten transition-table language with different guard or outcome semantics. Provisional derivation, command handling, acquisition, and installation are declared actions/events coordinated through the same vocabulary and the browser adapter's explicit ownership contract.

A projected `:receive` is not satisfied by an environment event with the same keyword. A normalized map cannot select its own trust or ownership class. Participant communications, authoritative observations, local action completions, and environment events retain their distinct admission paths.

The plan format has closed semantic keys and versioned interpretation. Unknown opcodes, unresolved action references, malformed successor locations, overlapping receive alternatives, or unsupported event contracts are construction failures. An explicit extension area can carry nonsemantic metadata without changing execution. Semantic extensions require a registered interpretation and their own assurance contract.

An `:await` environment event does not satisfy an expected `:receive` merely because the two events share a keyword or payload shape. A role-local environment completion, a participant delivery, an authoritative operation result and a qualified authoritative observation have distinct admission provenance and effects on role knowledge. Admitting a decoded value must not assign its own source class, trusted principal or authoritative basis. Serialization across the browser/server boundary discards any authority attributed solely to in-process wrappers; the receiver re-establishes the relevant category using the declared trusted protocol and source.

State ownership and effects

The role-local machine owns its protocol state and established values. The shared browser coordinator owns cross-execution target arbitration, obligations, and resource lifetimes where required. These are composed responsibilities, not competing copies of one machine. Each semantic fact has one declared owner.

The imperative shell owns physical handles: DOM references, timers, transport objects, listeners, and opaque continuity captures. It interprets admitted effects and reports correlated completions/failures. It does not independently choose successors or reinterpret a domain outcome from a status string.

An abstract transition can update semantic intent immediately and request a physical effect. The facts it records must match that point in the lifecycle: request issued is not request completed; observation acquired is not observation installed; installation permitted is not adoption completed. A host may report immediate completion before the portable machine enters its formal await state; the declared registration/buffering or ordered handoff contract must preserve that completion without fabricating success or losing it. Effect completion is admitted only for the issued physical effect, current permitted owner and declared completion source, at the actual semantic boundary. Host exceptions are runtime faults with recovery/retirement obligations, not proof that a previously valid semantic state/event was impossible.

The runtime allocates fresh effect/resource instances through a declared mechanism. Nondeterministic choices such as UUIDs and sampled time enter as explicit admitted inputs or action results. A pure transition does not secretly consult a global clock or allocate different command IDs when replayed in a test. Before a command's first possible send, its stable binding is already recorded by the appropriate owner.

Physical effects carry the owner, target/resource instance, operation/command correlation where relevant, and typed payload/reference required for admission. A callback copies the correlation issued with that physical resource. It must not fetch a newly current execution ID and relabel an old callback as belonging to the replacement execution.

The transition semantics are total over their **supported** states, normalized event classes and declared contracts: an applicable event advances, stutters, is classified duplicate/stale, requests recovery, or is rejected with a typed fault. An unknown event, malformed payload, unsupported plan version or unadmitted source cannot silently fall through into browser-specific callback code. Where a state/event combination is excluded by construction, that exclusion needs an explicit invariant and appropriate check; an unexpected host exception must not be repackaged as a legitimate domain rejection.

A local execution's semantic retirement revokes its ownership immediately, whether or not physical timers, DOM nodes or XHR handles still exist. Remaining handles are cleanup debt. A successor may inherit declared command-resolution or acquisition obligations through an admitted handoff, not the retired execution's old timer callback, target incarnation or actor-local knowledge. A value copied into `NormalizedEvent` is data for admission, not proof the sender may revive that retired execution.

Cross-host portability and physical-boundary witnesses

The common conformance corpus executes the same admitted plan and **semantic transition functions** in JVM Clojure, Node ClojureScript and Chromium ClojureScript, comparing portable semantic results, typed rejection classes, effect intents and owner/obligation state. Browser-host tests additionally observe actual effect emission, callback correlation and physical completion. An equal JVM/Node result is evidence for those exercised cases only; it neither establishes all possible compiler behavior nor certifies Chromium or real host effect correspondence.

Eligible positive history
The same admitted branch selector, closed event payload and portable basis relation select the same successor in JVM, Node and Chromium
A role awaits a declared participant `:receive` and admits a source-qualified correlated message.
A physical installation request is registered, its immediate completion is buffered and later admitted for the current owner and target.
An execution retires E1 and its successor E2 takes separately admitted recovery obligations.
A supported but inapplicable event produces a specified typed stale/duplicate/fault outcome with no unauthorized effects.

The test corpus must retain both useful successful transitions and the adversarial cases above; a reject-all machine or a synthetic matching report cannot establish functional completeness. Actual provider, HTMX, DOM, XTDB and deployment obligations remain independently qualified.

A concrete installation segment

The following is an inspection of a generated local segment using the canonical node kinds. Entry already supplies an admitted observation reference and an admitted installation request. Registered action and event contracts supply schema, provenance, current-owner, and failure obligations not repeated in this compact payload.

```
{:gesso.choreo/type :gesso.choreo/executable-plan
 :gesso.choreo/version 1
 :role :browser
 :initial 0
 :states
 {0 {:op :local
     :action :presentation/request-installation
     :requires #{:installation/request}
     :outputs #{:installation/effect-ref}
     :next 1}
  1 {:op :await
     :events {:presentation/installation-completed 2
              :presentation/installation-failed 3}
     :event-contracts
     {:presentation/installation-completed
      {:required #{:installation/effect-ref :installation/completion-ref}
       :optional #{}
       :open-data? false}
      :presentation/installation-failed
      {:required #{:installation/effect-ref :installation/failure-ref}
       :optional #{}
       :open-data? false}}}
  2 {:op :local
     :action :presentation/adopt-completed
     :requires #{:installation/request :installation/completion-ref}
     :outputs #{:presentation/adoption-ref}
     :next 4}
  3 {:op :local
     :action :presentation/retain-and-recover
     :requires #{:installation/request :installation/failure-ref}
     :outputs #{:presentation/recovery-owner-ref}
     :next 5}
  4 {:op :return :outcome :presentation/ready}
  5 {:op :return :outcome :presentation/recovery-handed-off}}}
```

The representation version is illustrative. A concrete emitted segment must satisfy the actual selected schema and its registered interfaces; this excerpt is not a second author-maintained implementation. The source-level `:presentation/installation-completed` and `:presentation/installation-failed` names describe **distinct admitted source outcomes**, not two strings the browser may choose by examining a response code. The failure-reference contract includes failed, partially completed and indeterminate realization with an explicit affected-target/coverage classification where supported. If the host cannot determine what physically installed, it must retain that uncertainty rather than constructing a false completed or wholly failed event. An absent callback is neither event; interruption, deadline and owner-loss paths belong to the registered wait/recovery contract.

The request action completes when installation work has been registered and issued under the host contract. It does not claim that the DOM changed. The host must buffer or otherwise serialize an immediate completion so it cannot be lost before the machine reaches its await boundary.

Registration binds an effect instance, source observation, target incarnation, current owner, method of realization and completion/failure contract **before** the physical action can complete. An HTMX before-swap allowance, HTTP success, request-finished event, or provisional DOM render establishes neither the required physical replacement nor canonical adoption. If installation consists of multiple independent targets, completion is a scoped per-target fact rather than an unconditional whole-board success. A promised all-or-nothing installation requires an actual physical mechanism or an admitted compensation and recovery policy.

The completed event is admitted only from that issued effect, the declared host completion source and the current permitted target owner. The completion reference resolves to actual physical completion evidence with the matching effect instance, target incarnation, observation scope and installed footprint. It does not become authoritative because an environment payload calls

itself completed, nor can a late callback claim the generation of a successor.

The adopt action consumes admitted completion and rechecks the shared coordinator's **current** owner, scope, source observation, independently newer adopted facts, and obligations accumulated since installation was requested. Merely possessing a reference with a familiar shape is insufficient. A qualified installation may discharge precisely covered obligations; it does not erase later Live invalidations, a pending granted Refresh, a newer creator-local card/membership, or unrelated targets. If the source was valid but the completion owner was superseded, the old callback cannot promote or overwrite a newer installation.

The failed path preserves the failure's scope, including partial or indeterminate installation where applicable, and establishes the declared recovery owner before returning. It does not reject an already incorporated command. Already installed targets, remaining unsafe/intermediate DOM, unfulfilled acquire-and-adopt obligations and local continuity cleanup need their own typed physical evidence and repair disposition. A recovery handoff cannot inherit retired callbacks, timers, target grants or role-local knowledge merely because it inherits the pending requirement.

Owner retirement, source loss, or unbounded absence of a completion follows the segment's registered interruption/deadline and recovery contract. Assembly must reject a required physical effect with no such handling when it can strand this wait. An implementation-specific synchronous effect interpreter does not make asynchronous host completion impossible: the projected machine suspends at its declared `:await` and receives qualified completion through the adapter/host boundary.

The fragment does not claim that all browser state belongs to one execution. The adopt action communicates with the shared coordinator through its admitted interface. That coordinator updates current adoption and advisory debt from its latest state, preserving obligations which arrived after installation was requested.

Claim execution uses independent outcome and installation paths

A complete generated Claim trajectory composes segments like the one above with command handling. The following table describes semantic boundaries rather than an alternative executable format:

Boundary	Required successor behavior
Invocation admitted	Record stable intent and execution ownership; derive permitted provisional presentation; issu
Command submitted or delivery attempted	Retain pending/unknown knowledge until evidence establishes more; submission alone is no
Accepted command evidence	Record qualified historical incorporation knowledge independently of whether a current card
Rejected command evidence	Record terminal command rejection only under the qualified, binding-scoped exclusion/discl
Unknown delivery or local deadline	Preserve unresolved command attribution unless stronger qualified evidence already exists; r
Observation acquired	Validate scope and progression; propose installation without claiming adoption.
Installation completed	Admit completion under the current owner; update adoption and discharge only covered obli
Installation failed	Preserve command knowledge and unresolved presentation debt; follow repair or reacquisitio
Execution retired	Revoke its local rights; preserve or transfer unresolved command/work obligations according

A deadline racing with a participant response requires explicit arbitration between the projected machine and its owner-relative adapter. The timeout is not smuggled into `:receive` as a forged participant message. The supported waiting/interruption mechanism must preserve the declared race semantics; a projection backend lacking that mechanism rejects the unsupported construction rather than dropping the timeout branch. Both events retain the generation and physical resource correlation issued with their original owners; the losing callback is stale for that execution and cannot resume its terminal branch. If a late source-qualified outcome establishes historical C42 success, a separately admitted recovery owner may incorporate that knowledge without resurrecting the old E42 machine or target grant.

Whichever event is admitted first may change local ownership, but it cannot change what evidence means. A later success can still resolve a retained command through an admitted recovery owner. It cannot restore retired installation rights. A late timer cannot erase a known success. Nor can a successful C42 replay force Alice's historical Claim back into a card now reflecting Bob as primary helper after a later authorized Reassign. The qualified current Request/RequestAssignment observation and installed source decide current presentation; C42's retained result decides only its attributable history.

Paired installation and Claim execution witnesses

These cases exercise real runtime boundaries. A caller-supplied map containing `:installation/completion-ref` is not itself an admitted physical completion, and a green callback report is not independent product acceptance.

Eligible positive execution
A qualified scoped card is requested, its matching after-swap physical completion arrives for the current owner, and the coordinator adopts
Installation is registered before the host completes synchronously; the queued completion is delivered at the matching `:await`.
A current Request card installs while a newer board invalidation or separate creator-local card was registered after the request began.
One of two installation targets completes and the other is indeterminate; the recovery owner receives exactly the remaining repair work.
C42's server-qualified historical Claim outcome is incorporated under current disclosure while a later Reassign has installed Bob as helper.
A timer and a participant message race at one suspended execution; the first admitted event settles the local branch.

These positive cases are independent of reject-all safety checks: installation, Claim, recovery and user-visible current presentation must be reachable under eligible premises, and failures must be observed at the actual stated host, semantic or trusted command boundary.

Boundary shapes, knowledge, and provenance

Actions declare required values, optional inputs where supported, closed outputs, possible failures, effects, and completion behavior. Validators are compiled for the admitted schema/registry version and invoked unconditionally at the relevant boundaries. Optional assertions disabled in production are not boundary validation.

Missing, present nil, false, and absent optional output are distinct according to the schema. A later completion which omits an optional field must not inherit an old value accidentally. An unknown branch selector is a typed contract fault rather than permission to choose an arbitrary default. Validation binds the selected **issued effect instance** and its declared action/event contract, including its loaded interpretation version; passing a value check for a different action, completion generation or schema is not admission.

When an event omits an optional output, the resulting state follows its explicit retention/clearing rule instead of accidentally reusing a stale value from a prior completion. Duplicate, delayed and wrong-generation events are checked before they can change that state. A closed result schema cannot become open by inserting extra fields in an external callback.

Passing a value schema establishes shape. An opaque local function returning a map is not automatically an authoritative source. Its output retains the provenance and asserted properties justified by its declared action contract. Authoritative outputs and observations cross their dedicated admission boundary; ordinary event normalization does not upgrade them. Value shape, source provenance, current owner, qualified observation scope and authorized disclosure are **separate** premises. A callback cannot claim that its response came from a participant, trusted backend or current physical target merely by copying an in-process event tag or binding token.

Full-language implementations may access facilities beyond the portable core under their declared host contracts. Properties such as purity, effect footprint, termination, and semantic accuracy remain explicit premises when not derived or enforced. A managed local action with an asserted functional contract is valid under an accepting assurance policy; an unresolved action reference is not.

Normalization precedes admission

The browser boundary converts raw HTMX, timer, EventSource, DOM, and lifecycle callbacks into bounded normalized data. It may decode and copy primitive values and issued correlation. It does not derive authority from mutable attributes, HTML shape, event names, or a physical handle's continued existence.

Admission identifies the event class from its registered source path, resolves the declared owner, checks current applicability, validates payload and binding, and then delivers the event to the appropriate semantic transition. Untrusted fields cannot opt into a weaker ownership class. The issuer of a physical effect retains an immutable reference to the instance, generation, source, expected completion and target incarnation. An asynchronous callback carries that issued identity, rather than reading the currently active execution and relabelling itself. Admission is checked again at the semantic transition boundary; normalizing bytes is neither a replay grant nor a physical installation/completion witness.

A nominal record or opaque wrapper may prevent accidental substitution within cooperating code. After untrusted serialization, the receiving boundary must re-establish the required category through validation, authentication, acquisition, or verification. The wrapper's name is not transferable proof.

Explicit transition and failure results

For every supported state/event class, the runtime has an explicit result: advance, accepted no-op, stale/duplicate rejection, recovery transition, or runtime/contract fault. Unknown event classes and invalid plan versions are classified at admission. Unexpected exceptions are faults, not evidence the input combination was mathematically impossible.

Event-admission results use a separate vocabulary from presentation settlements and model outcomes. For example, `:step/applied` and `:step/ignored-stale` describe a runtime decision; they do not mean Claim was accepted or rejected.

Selection must be deterministic under the declared contract. Construction rejects overlapping alternatives unless their equivalence or explicit arbitration is established. A runtime must not simply take the first matching event keyword from several candidates and silently ignore the rest.

A total dispatcher over a supported event vocabulary does not prove arbitrary Clojure callbacks terminate. It also does not imply every event is valid. The scope of totality, supported value domain, and callback premises must be stated. Invalid states can produce typed invariant violations with a safe fault/recovery path rather than undefined mutation. A semantic step classification such as `:step/ignored-stale`, a host effect failure, a qualified historical command result and an installed authoritative card inhabit different namespaces and evidence domains. None may be used to infer another simply because a diagnostic report presents them together.

An admitted transition updates portable semantic state before interpretation of the effects it emits, so a synchronous physical callback cannot observe an obsolete pre-transition owner. This ordering does not certify the physical effect: synchronous exceptions and asynchronous rejection retain their declared failure/recovery classification without fabricating command rejection or completed installation.

Cross-host portability

The shared core and corpus run under JVM Clojure, Node ClojureScript, and Chromium ClojureScript. Tests compare normalized semantic transitions, emitted abstract effects, correlation, and fault classifications for the same admitted inputs, including stale and adversarial schedules.

The portable data contract specifies numeric range and precision, equality, collection semantics, missing/nil/false behavior, encoding, and errors. JavaScript numeric precision must not silently alter an identity or selector. Platform objects, mutable DOM references, and arbitrary functions do not appear in the portable plan payload. Registered action references bind host implementations through the assembly's separate implementation contract.

Source sharing reduces divergence but does not prove every compiler/runtime behavior equivalent. Cross-host conformance tests remain evidence for exercised cases. A stronger claim identifies its formal correspondence and external compiler/runtime premises.

Production tests execute the canonical emitted plan and the production semantic core. Pure tests can substitute declared effect interpreters; browser integration tests additionally exercise the actual shell and DOM/HTMX lifecycle. A mock that always returns installed does not establish physical installation correctness. Advanced compilation and actual packaged assets need their own relevant checks. Conformance spans the emitted canonical plan, the exact bound implementation and production asset rather than a test-only stand-in with the same source name. The check records precisely which JVM/Node/Chromium cases ran and which physical browser callbacks, deployment paths and provider effects remain unexamined. A matching diagnostic/source stamp is not proof that the physical asset contains that plan and implementation closure.

Compatibility and deployment

The runtime distinguishes plan representation version, operation/interface meaning, protocol compatibility, admitted assembly identity, and exact packaged artifact identity. Deployment identity records physical rollout where needed; it does not substitute for any of these contracts or give all resources one shared lifetime.

Parsing old bytes does not establish continuation compatibility. A suspended browser can retain an unresolved command while its old local plan is retired. The admitted compatibility/recovery policy must preserve that command reference and current disclosure requirements without resuming incompatible state.

On version or assembly change, compatibility is checked separately for (1) parsing the plan, (2) executable semantic operation and interface meaning, (3) registered host actions and physical protocols, and (4) permission to continue each currently suspended execution. A recognizable but incompatible plan is not malformed, yet must not be resumed. The old execution is semantically retired first; physical cancellation, listener removal and resource cleanup follow under their own fallible contract. A new compatible runtime may receive **declared handoff obligations** for command-specific reconciliation, scoped reacquisition, installation, and retained Live/Refresh requirements. It cannot inherit old generation/callback rights or role-local message knowledge by copying a state map.

If the physical cleanup or authoritative reconstruction fails, retain the unresolved obligations and report the failure under the appropriate scope. The original historical command's stable binding must not silently become a new command, and a current Request card cannot establish the disposition of that old command. Likewise, an already completed command outcome does not prove that a new browser asset has installed a compatible authoritative view.

Exact executable payload identity, compatibility judgment and packaged asset identity answer different questions. A rollout identifier or an equal source/digest label does not justify cross-version continuation, proof-sidecar attachment or physical entrypoint coverage without the requisite binding evidence.

The production browser executes admitted local plans; it does not reconstruct the global choreography, rerun projection, load proof DAGs, or solve obligations to decide an ordinary transition. Executable guards and necessary admission contracts remain executable. Their explanations remain separate.

Paired admission, compatibility and deployment cases

The positive corpus exercises real admitted transitions and compatible recovery, while adverse histories reach the protected boundary rather than passing because some unrelated fixture was invalid.

Eligible positive history
A registered local action completes with the current effect generation and an explicit optional-output clearing rule.
A participant message enters via its authenticated and correlated source path; an environment timer separately enters via its issued owner.
A new browser asset starts a compatible plan, while an older incompatible suspended execution retires and hands off C42 resolution and bo
The semantic state commits its transition before a synchronous effect immediately calls back; a failed host effect follows the declared repai
The exact built artifact, executable plan, handler closure and served entrypath are checked together in an exercised production-mode brows

These are obligations of the completed Gesso design. A current test, source-derived report or diagnostic sidecar does not itself establish trusted event provenance, general compiler correctness or deployment correspondence.

Executable projection and diagnostic sidecars

Construction emits one canonical `ExecutablePlan` per projected role and separate compiler/diagnostic evidence products. The executable plan is the semantic runtime payload used by production and tests. A `DiagnosticProofSidecar` may explain it but cannot complete or alter its meaning.

A build may use a rich internal representation while checking and projecting. The commitment is that production does not execute one machine while verification or proof-enabled tests quietly execute a different, richer client machine. Compiler transformations still need their stated correspondence; separating the sidecar does not eliminate the projection or packaging proof obligation. The build must distinguish an executable plan's **exact bytes/terms** from the verified global choreography and its observable outcomes. Multiple distinct global subjects may produce identical projected local plans, even for every role, while differing in a source-level terminal proposition or other unprojected semantic condition. Thus exact plan equality cannot by itself authenticate a claimed global theorem, the exact authored source, or a complete refinement. Source-specific structural and behavioral correspondence are separately checked against the actual admitted global subject.

What belongs in each product

Product	Contents
ExecutablePlan	Supported local node structure, executable selectors/guards where defined by the format, action/mess
Admitted runtime state and inputs	Concrete command/execution/resource bindings, established values, observations, outstanding effects
Assembly/implementation binding	Registered action realizations, trusted boundaries, loaded versions, supported host capabilities, and ap
Compiler correspondence data	Mapping from source semantics to actual runtime locators, including generated/synthetic states and c
DiagnosticProofSidecar	Explanations, source locations, checked derivations or references, named premises, and exact executa
Packaged artifact manifest	Relationship between the admitted plan set, implementation/runtime closure, and emitted assets.

These products need not be stored as six separate files. Their meanings and trust boundaries must remain distinct. In particular, application execution cannot depend on a source span or optional explanation object to discover which guard it was supposed to run.

One assembled application may contain several roles and independent operations, each with its own exact executable identity. The compiled plan-set identity and manifest must bind the complete intended **role-to-plan** inventory, including role absence/presence and relevant assembly generation. An exact match for one browser role cannot certify that a changed server role, missing supported operation, or replaced implementation is consistent. The plan set is an explicit construction subject, not the application's entire behavioral/physical certification.

The executable semantic core and browser coordinator do not accept a diagnostic sidecar as a transition input. The conceptual interface is:

```
(semantic/step executable-plan admitted-state admitted-event)
;; Returns the next semantic state, declared abstract effects,
;; an explicit step classification, and the actual transition locator.
```

Guards operate on admitted runtime premises and executable contracts. The proof kernel/checker may explain why those guards contribute to a property; it does not supply current runtime permission by having accepted a derivation earlier.

Exact identities and their domains

The design distinguishes:

Identity	Question it answers
Runtime locator	Which state, transition, or effect in this exact emitted plan was selected?
Source semantic name	Which authored or generated concept does that locator explain?
Normalized semantic dependency identity	Which exact semantic definitions and dependencies a claim or construction uses.
Proof-obligation identity	Which proposition/derivation is being checked or explained?
Exact executable identity	Does this sidecar describe this exact canonical runtime payload and coordinate layout?
Compatibility identity/judgment	May this runtime safely participate or continue under the selected server/assembly contract?
Packaged asset identity	Which actual emitted browser/server asset bytes are loaded?

Runtime locators need not be stable across compilations. Source names can remain useful while behavior changes, so equal names do not preserve proof applicability. Two plans can be compatible while having different exact payload identities. The same plan payload can also be bound to different opaque implementations; applicable assurance therefore includes the relevant implementation/assembly closure, not merely the plan digest.

A named semantic subject, the normalized transitive dependency identity described in Part I, a verified source-to-executable correspondence judgment, and an executable content digest answer different questions. A proof for one global terminal outcome cannot be transferred to a globally different subject whose projected executable happens to be the same. Neither can a proof tied to an old opaque host-action contract be transferred to a changed implementation under the same plan bytes. Runtime locators remain coordinates **of that exact emitted plan**. Diagnostic correlation obtains them from the selected transition and actual compiler mapping, not from a second execution of a reference machine or a nearest-name lookup.

Canonical payload identity

The exact identity recipe specifies:

- the object being identified: canonical plan terms, a plan set, or exact asset bytes;
- encoding and schema versions;
- type-preserving representation of supported scalars and collections;
- deterministic map/set ordering without discarding semantic distinctions;
- unsupported values and their rejection behavior;
- excluded self-identifying fields, or an external identity envelope;
- the digest algorithm and its accepted collision premise;
- interpretation dependencies bound separately when not included in the payload;
- an explicit domain tag distinguishing a single plan from a role-keyed plan set, a semantic subject, a proof obligation, or packaged asset bytes;
- injectivity of the supported canonical value encoding up to its declared equivalence relation, including unambiguous type tags, lengths and boundaries;
- the exact role inventory and ordering-independent role bindings for a plan set;
- the trusted source of the bytes or terms actually digested, not a caller's assertion that some independently recomputed object was the shipped one.

A printed EDN map is not automatically a canonical encoding. Nil versus absence, keyword versus string, list versus vector where meaningful, numeric forms, and map/set order require explicit treatment. Host functions and DOM objects cannot be silently printed into a supposedly portable canonical digest. The canonicalizer must define which scalar classes, Unicode/byte encodings and numeric forms belong to the supported domain, and must reject the remainder rather than fall back to host-specific `pr-str` output. Distinct admitted values must not collide merely because their printable forms, default stringification, comparison keys or container iteration order coincide. Map keys and set members must have deterministic, unambiguous encodings, including when a host compares dissimilar keys or scalars differently. The required portable digest should be reproducible on its specified platforms from the same canonical value domain; a current JVM-only hash implementation is not automatically a complete cross-host codec.

An identity may intentionally ignore metadata that is **not** executable, such as source coordinates. Such exclusion must be declared in the identity domain and must not discard a guard, event schema, runtime successor, semantic interpretation dependency or any other field that affects the actual execution. Equal digests mean equality of the explicitly identified canonical subject only, subject to the algorithm's collision premise; they do not mean semantic equivalence of arbitrary programs.

A convenient design keeps the payload free of its own digest:

```
(def executable-identity
  (artifact/digest
   {:algorithm :sha-256
    :encoding :gesso-plan-canonical-v1
    :value executable-plan}))

(def plan-manifest-entry
  {:role :browser
   :executable-identity executable-identity
   :plan-format-version 1
   :runtime-contract-ref runtime-contract-ref
   :implementation-closure-ref implementation-closure-ref})
```

The algorithm and format labels illustrate a declared recipe, not a new trust source. If identity is embedded in an envelope, the payload domain excludes that self-identifying field. It must not hash an object including the very digest being computed.

Hashing canonical plan terms is not hashing the emitted JavaScript bundle. Packaging must establish which plan set and runtime/implementation closure the actual asset contains. This requires a checkable relationship from the exact admitted plan/role set and current host-action contracts to the built output, the emitted asset bytes, and the asset actually served/loaded at the admitted entrypoint. Advanced compilation, bundling, stale caches, service-worker delivery, alternate script entrypoints, or swapped deployment files may invalidate that relationship without changing the original source plan digest. Where that physical provenance is unavailable, the correct status is unestablished correspondence, not a retroactive change to the executable's semantic identity.

A copied or stale generated stamp cannot substitute for that relationship. Exact equality of a stamp supplied by two untrusted locations is not construction evidence. An exact executable match is a necessary condition for sidecar coordinate correlation, not sufficient proof of the sidecar's authorship, checker result, assumptions, or installed application correctness. A diagnostic sidecar

mismatch disables that explanation; it must not disable an otherwise independently admitted executable runtime or authorize a best-effort locator match.

Paired artifact-identity and assembly witnesses

The following cases distinguish legitimate reuse, exact-match diagnostics and genuinely applicable proof/deployment evidence.

Eligible positive case
Two projections differ only in map insertion order or excluded nonsemantic source-location metadata, and the declared canonical recipe yi
A compiler emits an exact role-keyed set and independently binds its verified source semantics and structural correspondence.
A diagnostic sidecar names an actual runtime locator of the exact emitted plan and has separately checked issuer, proof claim and premises
Compatible plan versions permit a newly admitted runtime to recover command and presentation obligations while using fresh physical re
The manifest binds the emitted browser plan/roles, current handlers and the exact deployed bundle served at the checked route.

These are required finished-system positive and adversarial witnesses. A digest comparison alone is neither a model proof nor confirmation that the installed browser executed the matching plan.

Binding sidecars to the actual compilation

Compiler correspondence is generated alongside the executable plan from the same admitted semantic subject. It records the actual mapping, including synthetic states or compacted continuations, rather than reconstructing it later from similar state names. The compiler's source-to-runtime mapping includes direct and synthetic states, semantic continuations and every source alternative intentionally collapsed into one receive or runtime locator. One compact locator need not mean one unique authored predecessor; a diagnostic explanation may not silently discard other sources.

The sidecar, structural projection evidence and concrete correspondence witnesses are bound to the admitted global semantic subject and the actual complete role-to-ExecutablePlan set. Identical local plans may still arise from distinct global choreographies with different terminal outcomes. Exact plan identity cannot authenticate global source meaning or a claim about installed opaque implementations.

A sidecar inspection view can be:

```
{:sidecar/version 1
 :role :browser
 :executable-identity executable-identity
 :semantic-subject-ref semantic-subject-ref
 :checker-result-ref checker-result-ref
 :implementation-closure-ref implementation-closure-ref
 :locations
 {installation-complete-locator
  {:semantic-id :request-claim/presentation-installed
   :source-ref source-ref
   :claims
   [{:proposition-ref adoption-proposition-ref
     :mechanism :checked-derivation
     :evidence-ref derivation-ref
     :premises [shell-completion-premise
                current-owner-premise]
     :coverage-ref coverage-ref}]}}}
```

The locator comes from the actual emitted plan mapping. The symbols above denote bound values; replacing them with copied identifiers cannot authenticate the claims. The sidecar may contain checked and unverified claims with distinct status. Its existence does not prove every listed obligation was discharged. A structural projection certificate establishes its named structural property; a lockstep or weak concrete correspondence witness establishes only its replayed schedule and premise-relative observable trace. A correctly bound **failed** witness remains failed, not successfully proved merely because exact digest matching succeeded. Finite replay does not quantify over all fair schedules, nor certify actual browser installation, backend execution or HumanHelp product capability.

Attaching behavioral correspondence evidence requires the exact semantic and executable subject to agree with the artifact being explained. Evidence for choreography A cannot be composed with an artifact generated from choreography B merely because their operation names or state counts match.

Diagnostic admission first validates the sidecar format and exact executable identity. Where tooling labels claims checker-verified, it additionally validates provenance, the checker result, applicable premise set, and implementation binding. A fabricated sidecar can copy the digest. Exact correspondence and trusted claim acceptance are separate checks. A copied `:valid? true` or `:mechanism :checked-derivation` is a claim, not a trusted checker result. The admission mechanism must identify its proof/checker version, source inputs, witness, result, named assumptions and dependencies under an accepted trust boundary. A source-bound conditional theorem, a finite witness, an externally asserted action and a physically installed realization have distinct evidence statuses.

If construction requires a proof or preflight result, enforce that required gate at build/assembly admission, independently of optional runtime diagnostics. Omitting a sidecar from production neither revokes a previously admitted executable nor waives a required build failure. Attaching one cannot authorize a rejected construction.

If exact identity mismatches, diagnostics must refuse correspondence for that plan. They cannot guess by timestamps, similar names, or near-matching runtime coordinates. Ordinary execution may continue if its own admission and compatibility remain valid. An unavailable explanation is not an executable failure and is not evidence of a safe transition either.

Observe the selected transition, not a recomputation

A trace contains the locator selected by the actual semantic step, its admitted subject identity, and a bounded description of its result/effects. The observer uses that locator to retrieve correspondence. It must not rerun guards or transition selection independently to guess what happened. A stale, ignored, malformed or rejected-before-selection event may have no selected transition locator; do not invent a successful one for it. Where a generated locator has multiple source origins, preserve the actual admitted alternative and the complete compiler mapping. Host-effect failure after a valid semantic step is a distinct, correlated physical outcome, not a retroactive rewrite of its locator or a fabricated domain rejection.

Recomputation can diverge if callbacks are effectful, registries change, state is not identical, or the observer has different inputs. Even when the pure core would reproduce the same result, reporting the actual selected locator gives a clearer correspondence contract.

Optional diagnostic access is read-only. It does not receive mutable live objects or authority-bearing handles it can modify. Redacted immutable trace values are preferable to whole contexts, DOM captures, credentials, or unrestricted command payloads. Missing or redacted data is reported as such; an explanation must not invent an unavailable premise. The observer receives an immutable, bounded diagnostic projection rather than aliases to mutable owner registries, resource handles or privileged server facts. Redaction must precede disclosure to a less-trusted observer; copying a current scope token or executable digest into the trace does not grant its recipient authority or permission to read private data.

Diagnostic noninterference

For the same admitted initial state, canonical plan, runtime interpretation, and normalized event sequence, adding, removing, corrupting, or replacing a sidecar must not change semantic transitions or abstract effects, apart from permitted diagnostic output.

```
semantic-trace(run(plan, initial-state, events))
  = semantic-trace(run-with-diagnostics(plan, initial-state, events))
```

This is a conditional noninterference claim over fixed semantic inputs. It does not claim instrumentation has no wall-clock, memory, scheduling, or resource cost, or that every real browser receives the same event sequence with debugging enabled. The fixed-input comparison includes step classifications, portable semantic state, command and resource ownership, outstanding acquire-and-adopt obligations, and the ordered required abstract effects, modulo optional diagnostic observations. A stronger assertion about physical scheduling or fairness requires separately stated nonblocking and effect-delivery premises. Stripping diagnostic-only adapter effects from a pure test is not a proof that arbitrary same-process custom observers cannot block a real browser.

Excluding a sidecar parameter is necessary but insufficient. Ambient mutable registries, shared callbacks, exception propagation, hidden source lookups, or observer decisions fed back into admission can reintroduce a control path. Module/API boundaries and implementation contracts must prevent those paths.

Optional diagnostics must not gate required effects or completion delivery. The normal path records the actual transition and issues its required work according to the semantic contract. A bounded best-effort trace queue can feed observers independently; overflow drops diagnostic work under a visible diagnostic policy, not semantic work. Diagnostic capture, redaction and bounded queue admission must not delay or reject a required semantic effect merely because a trace sink is full or malformed. A required correctness or compliance audit instead belongs to a declared semantic effect with a real durability and failure/recovery contract, not an optionally dropped diagnostic queue.

Catching an observer exception does not contain a nonterminating or blocking observer. In-process custom observers require an explicit bounded/nonblocking premise, or execution isolation adequate for the promised containment. The framework cannot

prove arbitrary observer code harmless just because its return value is ignored. Redaction and diagnostic failure reporting must also be bounded and contained. A `try=/=catch` around user observer code does not stop a callback that never returns. The claim must either name an admitted bounded/nonblocking observer premise or provide isolation and enforceable resource budgets appropriate to the promised containment. Diagnostic failure may not be reclassified as command rejection, accepted incorporation or physical installation.

If an audit action is required for the operation's correctness or compliance contract, it is a declared semantic effect with its own failure policy. It must not hide inside an optional observer whose loss is advertised as irrelevant.

Paired sidecar and observer conformance cases

Eligible positive case	Adve
The actual selected runtime locator is reported with all merged receive alternatives and generated origins.	An ob
A structural certificate and a valid concrete witness are checked against the exact current role-plan set and admitted global source.	Anoth
The same admitted state/event corpus runs with no, matching, and corrupted sidecars.	A dia
A bounded observer reads a redacted immutable trace; overflow drops optional diagnostics alone.	A thr
A required audit is declared and executed as a protected semantic effect with recovery obligations.	A con

Positive tests must exercise the intended compiler, transition and host effect boundaries; neither a copied digest nor a reject-all observer can establish full correspondence or noninterference.

Sidecars carry no authority

A browser-side sidecar cannot establish principal identity, current model permission, authoritative state, subscription authorization, or permission to read protected data. It cannot authenticate a principal, authorize a protected Request/RequestAssignment, warrant a model transaction outcome, claim a current board observation, establish a Live subscription or extend the disclosure of a historical result. The server accepts commands under its trusted boundary, regardless of the browser's proof-looking material.

Nor may a digest-matching sidecar turn a reported compiler theorem into the browser's own permission to send, install, reread, settle, reveal or retry. The trust boundary must be the same with no sidecar, a valid sidecar, or a maliciously forged one. A user-visible diagnostic may truthfully explain why an action is unavailable, but never supply a missing trusted authorization or an executable workaround.

Sidecars can themselves reveal source details, policy names, or private values. Production packaging omits them by default; authorized diagnostic distribution follows the application's disclosure contract. Compiler source paths, policy names, proof premises, operation IDs, observation references, diagnostic traces and redacted values are each assessed for disclosure. The reader's identity, permitted Organization/Location, tenant scope, retention period and intended diagnostic purpose govern distribution independently of exact executable identity. A complete copied sidecar or trace from another authorized context is not implicitly disclosable to the current browser or user. Normal execution must remain complete when sidecars are absent; a mandatory semantic audit or build-time proof admission remains separately mandatory.

The implementation can enforce part of this separation by dependency direction and closed runtime formats, and test it with missing, malformed, mismatched, throwing, and resource-heavy observers. Such checks support their stated scope; they do not establish a sandbox for arbitrary same-process trusted code. An optional observer that requires unbounded time, possesses mutable owner/DOM handles, or can block required effects needs a different isolation contract; merely ignoring its return value does not ensure noninterference. Optional diagnostic failure is not equivalent to a failed command or installation.

Build output and assembly closure

Build output supplies machine-checkable correspondence among the current semantic subject, projected plan set, registered implementation closure, runtime contract, packaged assets, and optional exact sidecars. Test, diagnostic, and production modes execute the same admitted semantic payload for a given build. The construction evidence explicitly binds the complete role/operation inventory, trusted route and handler closure, physical transport, semantic implementation contracts, generated plan set, runtime interpretation, and actual asset bytes for every served entrypoint within the claim's coverage. Compiler output, an in-memory ApplicationAssembly report and the installation/serving environment remain distinct subjects.

Part VII's assembly admission phases verify that the served application loads the required artifacts and bindings. A source file containing a correct plan is insufficient when the browser actually loads an older bundle. A browser's claimed artifact identity is likewise not server authorization. A valid rendered-surface preflight report applies to the surface and handler path actually inspected; it does not prove that every possible dynamic Clojure branch, pre-serialized HTML, alternate Biff entrypoint or later

middleware/asset replacement was covered. Changes to the effective route, handler, compiled asset, semantic dependency or accepted backend premise require the applicable construction/recovery/requalification policy.

The build may legitimately transform or optimize code. Any transformation that changes the object covered by a claim needs the declared correspondence or requalification. Sidecar separation avoids a second proof-annotated runtime; it does not excuse testing only an unoptimized development realization while claiming the production shell has been exercised. Exact plan identity, accepted source-relative structural proof, bounded JVM/Node/Chromium semantic conformance, physical HTMX/DOM execution, trusted XTDB write/readback correspondence, and a served production bundle each require evidence at their own boundaries. A successful HTTP 200, sidecar match or copied build stamp cannot substitute for any missing physical or product-positive observation.

Conformance and acceptance obligations

The same canonical semantics must support construction checks, runtime interpretation, and focused conformance cases. Part IX supplies certification judgments; Part X supplies the stronger soundness and implementation correspondence arguments. This part requires the following cases at their appropriate boundaries.

The acceptance report maintains three independent judgments: (1) model/business-rule correctness under stated provider and writer premises; (2) closed application assembly and actually enforced rendered/served path coverage; and (3) behavioral completeness of an independently specified, usable HumanHelp product. None implies either other judgment. Each result records its exact source/version, premises, positive and adverse witnesses, coverage, scope and unresolved or inconclusive obligations. A failing or absent witness is not rewritten as an accepted assumption to produce a green overall status.

Area	Case
Provisional typing	Claiming label is rendered over an open Request.
Entry knowledge	Required observation is absent or belongs to another Request.
Creation	A local preview is created before authoritative entity existence.
Command identity	Transport retries or a new execution continue an unresolved intent.
Accepted outcome	Command success arrives without a fresh card.
Rejection	Current card resembles a conflicting outcome but no rejection evidence exists.
Timeout	Timer arrives after accepted evidence.
Supersession	A fresh card installs while original command attribution is unknown.
Installation	Permission is granted but the DOM effect fails.
Late rollback	Old failure follows newer adopted authority.
Local concurrency	One provisional command depends on another.
Rebase	New authority changes the local projection after a command was sent.
Multi-target effects	One of several required swaps completes.
Continuity failure	Focus restore throws after successful installation.
Deferred continuity	An old layout callback follows a new capture or user edit.
Query controls	Draft search differs from the query that produced the board.
Capture implementation	Restore implementation changes after capture.
Cleanup ownership	A newer style lock has the same physical value as an older lock.
Protected content	A Request disappears from the authorized board.
Availability	Reassign intentionally requires helper selection.
Construction defect	A required action is missing, even if hidden.
View binding	A renderer tries to override registered operation policy.
Event classes	An environment event copies a participant message name.
Event selection	Several alternatives overlap.
Callback correlation	An old physical callback reads the new current execution ID.
Immediate completion	Effect completes before its await state is reached.
Portability	Missing/nil/false or numeric representation varies across hosts.
Opaque action	Full-language implementation has an asserted contract.
Exact sidecar	Sidecar belongs to another emitted plan with similar names.
Forged proof metadata	A sidecar copies the correct executable digest.
Observer failure	Observer throws, blocks, or floods output.
Trace locator	Diagnostic code tries to recompute the selected transition.
Packaging	Current source plan differs from the loaded asset.
Missing product action	Rendered buttons are internally valid but an intended eligible action is absent.
Privacy of sidecar	A diagnostic from another tenant/Organization or principal contains otherwise valid source/trace details.
Local versus remote adoption	A remote Request advisory arrives while a creator-local Request is installed.
Command versus current card	A past Claim succeeds and a later Reassign selects a new primary helper.
Non-vacuous positive product	The implementation hides or disables every action, rejects every command, or never acquires a board.

Independent positive HumanHelp witnesses

The independent Part VIII product catalog determines the expected capabilities before inspecting what the view happened to render. The following witnesses must be reachable under eligible fixtures, with genuine source-owned model transitions and actual browser presentation:

Product path	Positive user action and observable result
Authenticate and Create	A real eligible user opens Create, enters valid content and commits one Request in the authorized Or
Claim and retry	An eligible helper claims an open Request through the installed control; the production model establi
Lifecycle and manager Reassign	Eligible actors can Unclaim, mark on the way, complete or cancel under their respective real model p
Local board and continuity	An authorized complete active-and-terminal Request cohort for the selected Location supports local s
Selective adoption and one Refresh	Bob receives an advisory for Alice's R but R stays absent; Bob creates S and sees his own correlated S
Uncertainty and recovery	The UI distinguishes qualified acceptance, terminal rejection and unknown delivery, and provides usa
Response interaction	The product exposes and exercises the separately selected response-flow contract, with its independe

For every promised capability, a complete positive witness exercises the actual admitted operation and physical result, not an inspection fixture that simply returns `:available` or `:installed`. The default authorized board cohort includes active and terminal Requests under its declared complete collection bounds; local Show terminal, text search and sorting cannot manufacture a fresh acquisition grant. The R/S history permits one explicitly granted Refresh cycle. An invalidation arriving during its installation remains an independent obligation unless the admitted completed observation actually covers it; a hidden second Refresh cannot make a defective test pass. Controls legitimately unavailable under product policy remain valid negative cases, but an eligible action permanently hidden or unconstructible is a behavioral-completeness failure even when assembly safety checks pass.

Evidence boundaries and acceptance disposition

For each conformance case the report identifies the operation, accepted premises, independently expected behavior, test entrypoint, exact version/build/subject, observed trustworthy source and physical effects, comparison oracle, and coverage exclusions. Pure plan tests prove only their scoped transition relation; Node or JVM parity covers sampled semantics, Chromium tests cover exercised host interactions, finite model exploration covers its declared bound, and trusted backend correspondence depends on qualified physical writer/readback evidence. A source-derived diagnostic or matching digest is not itself an application certificate.

A negative test must reach the particular rejecting boundary instead of failing earlier due to an unrelated malformed fixture. A positive control with the minimally changed valid premise must succeed at that same boundary. An unknown/unavailable provider, inconclusive proof budget or missing production deployment evidence remains an explicitly unresolved claim rather than an empty success.

Whole-Part-V acceptance additionally requires reconciliation of all thirteen reviewed source sections against R4, their internal joins and Parts I–IV/VI–X, plus independent inspection of the assembled publication and evidence limits. Completing the 05.13 sectional audit alone does not accept Part V or certify the framework implementation.

Negative tests must fail for the intended reason, not because an unrelated malformation prevents reaching the behavior under examination. Positive cases must retain ordinary raw HTMX, explicit non-optimistic modes, valid retries, full-language implementations, permitted partial observations, and useful application-defined resolution strategies under their contracts.

Pure tests compare admitted transitions and effects; shell tests exercise actual completion, ownership races, and failure handling; cross-host tests exercise the portable representation; asset checks bind the shipped realization. Qualified production model tests must also witness the protected RequestAssignment transition and applicable writer/invariant coverage. Passing these tests does not promote asserted callbacks, backend premises, or browser behavior into general proofs. Different test types may share a semantic plan without sharing an automatically proven implementation.

Completion means the declared presentation path can execute and recover with truthful state classifications. A page that starts, emits valid-looking markup, or returns HTTP 200 has not thereby satisfied this part.

Part VI — Browser realization and recovery

This part specifies how an admitted browser plan becomes physical requests, DOM changes, subscriptions, timers, and recoverable user interaction. It connects Part V's portable execution and presentation contracts to Part IV's authoritative operations, scoped observations, and unresolved commands. Part VII assembles these contracts before a managed affordance is served.

The browser is an execution environment with delays, duplicate callbacks, reentrant event handlers, disappearing targets, lost connections, and suspended pages. These conditions belong in the realization contract. A successful pure transition, HTTP response, or swap callback does not by itself establish that the intended application state has been adopted.

Examples use illustrative API names and inspection records. They specify the intended architecture; they do not assert that a displayed function already exists. Symbols such as `E42` and `observation-ref` denote values established by the surrounding construction. Copying their fields does not create an admitted observation, resource owner, installation permit, or server credential.

Illustrative names are scoped by their worked examples. In the Claim trajectory, `CardObs1` denotes the admitted card observation and `P1` denotes its installation permit; in the board Refresh trajectory, `O1` and `O2` denote advisory obligations, while `P17`, `P18`, and `P20` are illustrative provider positions, never numerically ordered here. A Request named `R` and a board representation labelled `R` are also different subjects when used in separate histories. Neither matching example labels nor nearby physical generations identify the same semantic instance across trajectories.

The full application language remains available under the checked construction contract. Opaque renderers, adapters, and other application functions have explicit input/output, effect, and completion contracts. Unverified behavior remains an assertion or a named environmental premise. It is not promoted to a theorem because the code passes through a small runtime interface.

Browser adapter refinement

One executable semantic core and a narrow physical shell

Browser coordination is defined by the permanent executable semantic core, including the browser adapter's transition semantics. A separate proof kernel, when used, checks evidence about that semantics. The proof kernel does not become a second runtime interpretation of browser behavior.

The projected Choreo machine owns the role-local trajectory. The browser adapter owns its admitted invocation, asynchronous admission, fragment coordination, resource ownership and the conditions under which that machine may advance. These are composed responsibilities within **one executable semantic interpretation**, not competing copies of the machine. The browser shell, HTMX integration, timer callbacks and view components do not keep independent semantic answers to which execution is current, whether a command committed, whether rollback is legal or whether a presentation is authoritatively adopted.

For an admitted plan and normalized event, the adapter computes a next semantic state, a selected transition identity, and abstract effect descriptions:

```
(adapter/step admitted-plan adapter-state normalized-event)
;; ⇒ {:state next-state
;;     :transition-ref selected-transition-ref
;;     :effects effect-descriptions
;;     :diagnostics diagnostic-values}
```

This is an **illustrative** inspection shape, not a required signature or claim about the current adapter API. The exact return representation is an implementation choice. A selected-transition reference, if supplied for inspection or evidence, must be derived from the actual transition and its admitted source, not from a parallel diagnostic simulation or an independently chosen branch. Its absence from a runtime API does not excuse missing transition/effect correspondence where the application claims that assurance.

The following boundaries are required:

- The transition does not perform HTTP requests, mutate DOM, register listeners, read wall-clock time, or invoke an uncontrolled effectful observer.
- Nondeterministic inputs enter as explicit normalized values with their required provenance. Reading a clock or generating an identity is an admitted boundary operation, not an invisible dependency of a supposedly pure step.
- Semantic state contains portable identities, ownership, knowledge, obligations, and effect lifecycle records. DOM nodes, XHR objects, functions with mutable closures, and physical cancellation handles remain in the shell.
- The shell interprets abstract effects and supplies their actual completion or failure events. It cannot silently decide a different semantic successor.
- Opaque local computations use the declared execution boundary from Part V. Their inputs, results, failure behavior, and premises remain inspectable even where the computation itself has not been proved.

The normalized event contract distinguishes the event's **reported fields** from its **admitted origin**. Correlation is attached when an admitted physical resource is created or registered, and carried through its lifecycle. Merely parsing an execution ID, source identifier, command ID or apparent basis from client bytes cannot authenticate the event, bind a current owner, establish a trusted authoritative observation or authorize a server mutation. The supported browser realization identifies the source, schema, owner, resource instance and generation admitted for each event class. Execution-private callbacks and shared fragment invalidations use different admission contracts: a shared invalidation is not rejected merely because it did not originate from the currently selected local command.

One logical dispatcher owns semantic state for a runtime instance. Applications may have several isolated runtime instances, but two mutable copies must not both claim to own the same execution, coordinator, or resource instance. Shared resources have an explicit shared owner and protocol. A later event observes one committed predecessor transition, not an arbitrary interleaving of two partially updated copies. Each accepted event has a linearized semantic disposition; physical effects and their callbacks occur after that state transition is committed. If a physical effect synchronously calls back into the dispatcher, the callback must see the newly committed semantic owner, generation and obligations. The interpreter cannot apply an obsolete pending effect from the prior transition to a newly owned resource merely because the effect was already emitted. Where multiple host tasks can dispatch concurrently, the implementation needs an admitted serialization/ordering mechanism; an unprotected read/modify/write cycle is not sufficient.

The required relationship is:

```
registered physical source
  → normalize and admit source/identity/event
  → compute semantic transition and abstract effects
  → commit next semantic state / retire obsolete rights
  → interpret admitted physical effects through narrow shell or HTMX gate
  → report actual completion/failure as a new normalized event
```

HTMX remains responsible for ordinary declared requests, parsing, supported swaps, OOB handling and settlement. Gesso may gate a lifecycle action **before** a protected physical mutation when its invariant requires intervention; it must not perform a forbidden mutation first and ask the adapter afterward. The exact supported HTMX lifecycle contract, including other physical mutation paths, is specified in the following sections, not presumed from the mere existence of a `beforeSwap` callback. The shell likewise must not reimplement HTMX's native request/swap machinery to create a second, unexamined execution path.

An effect request is not evidence of effect completion. Emitting `:htmx/allow-swap` does not certify that a target was installed; a successful request does not establish a committed Claim; and recording a tentative presentation does not establish current RequestAssignment state. The appropriate owner must admit actual, correctly scoped physical completion before advancing adopted presentation knowledge. Timer cancellation, aborted requests and listener cleanup may fail or race. Semantic retirement first revokes stale callback rights, while physical cleanup remains a separate, possibly incomplete obligation.

Optional diagnostics observe actual selected transitions without choosing different outcomes or supplying missing permission. Their resource and failure isolation must be established under declared bounds; a throwing or unbounded same-process observer is not magically harmless merely because its return value is ignored. Mandatory semantic checks and application admission are not optional sidecar observers.

The event/physical boundary also has a trust asymmetry. A hostile browser can skip the adapter, alter attributes and submit requests directly. Current principal, Organization/Membership and RequestAssignment eligibility, as well as the actual transaction and disclosure rules, remain with trusted server and model authorities. Browser admission protects browser-coordination guarantees, not server authorization.

Core/shell boundary acceptance histories

The following paired cases require actual semantic dispatch and, where physical behavior is claimed, instrumented host effects. An inspection map with the expected keys is not a substitute for the relevant event source.

Eligible history
An admitted execution installs a timer with its exact owner and generation. Its synchronous completion re-enters dispatch after semantic s
One execution owns its provisional card; a later scoped authoritative installation retires that right. Delayed physical cleanup then finishes.
A subscribed fragment receives an admitted shared invalidation while a different command is locally active.
A guarded HTMX request is permitted, followed by an independently observed completed installation and correctly scoped adoption.
An optional observer reads a bounded transition record without influencing the selected successor.

These cases establish the acceptance **obligations** for the finished design. Section 06.02 defines concrete identity lifetimes; 06.03–06.05 define HTMX gates, physical effect correspondence and adoption; 06.06–06.07 define scoped Live and Refresh coordination; 06.10–06.11 define managed effect and event admission details. Part VII must close the **actual** assembled and served entrypoints. No single pure test, callback log or claimed sidecar identity certifies those separate physical boundaries.

Distinct identities serve distinct purposes

The browser preserves identity through every boundary at which a callback, effect, recovery attempt or physical mutation might otherwise be attributed to the wrong work. Identity has a declared allocation authority, namespace, incarnation or generation, comparison relation, lifetime, and permissible transfer. An equal-looking string, a reusable DOM selector or a compatible plan version is not by itself evidence of the same current owner.

Identity	Meaning and lifetime
`:runtime-instance/id`	One admitted browser runtime lifetime in a page or worker context; a replaced, resumed or concurrently load
`:authority-context/ref`	Reference to the admitted principal, tenant/Organization and relevant policy context at the specified trust bou
`:execution/id`	One local execution of an admitted role-projected plan; an execution has its own generation and may retire w
`:gesso/command-id`	One semantic command under Part IV's immutable intent, actor/authority, operation, arguments and compatil
`:resource-instance/id`	The concrete incarnation of a timer, listener, subscription, continuity capture, transport handle or managed ph
`:effect-instance/id`	One issued effect and its completion, cancellation, failure or uncertainty protocol; issuance does not mean the
`:transport-request-id`	One attempted HTTP or other transport exchange; a retry of C42 may receive a new attempt identity while re
`:target-instance/id`	One physical DOM target incarnation and ownership generation, distinct from a reusable selector, logical viev
`:coordinator/id`	One scoped Live acquisition coordinator and its ownership lifetime; its in-flight requests have their own attem
`:assembly-deployment/id`	The pertinent installed application or deployment incarnation for artifact and continuation admission, not a s
`:executable-identity`	The exact projected executable identity from Part V, distinct from a semantic-compatibility judgment or a pro

This vocabulary does not require every host API to use these exact keys, allocate a separate UUID for each row, or expose implementation-only handles to applications. Where two identities share an underlying value, the admitted contract must establish that they identify the **same physical instance** for all operations relying on that alias; equality of their representation alone does not establish the claim. Generations belong to a named allocator and owner. Counters can be reused only when their full identity also prevents an old callback being confused with a new incarnation. A local generation never orders different actors' commands, deployments, or opaque XTDB bases.

A HumanHelp domain Request `R` remains a model-owned subject, such as `{:arguments {:request-id R}}`. It is not an HTTP request ID. The authoritative actor/Organization/Membership and current `RequestAssignment` eligibility are established by trusted server/model enforcement, not by a copied `:authority-context/ref`, plan digest or `:gesso/command-id` on a browser element. Browser-side admission may bind the same semantic intent C42 across H1 and H2, but it does not thereby prove either attempt incorporated, that E42 is still current, or that an observed card is physically installed.

Identity allocation, currentness and retirement

An admitted resource or effect record binds its issuer, semantic owner, runtime/plan context, generation, expected source and completion contract **before** physical issuance. The shell carries the resulting correlation through the physical handle or a registered mapping; later callbacks cannot reconstruct authority from today's DOM ancestry, URL, selector or ”active execution” variable. The runtime checks both the reported identity and the admitted origin and still-current record when the event arrives.

Distinct lifecycle subjects retain distinct end conditions:

- C42's command history and unresolved incorporation/attribution survive transport timeout or E42 retirement until Part IV's qualified resolver, retention and terminal-exclusion rules dispose of that obligation.
- E42's execution and its private timer, transport, rollback and target rights end or transfer through explicit semantic retirement; a new E43 does not receive E42's callback rights merely by continuing C42.
- H1 and H2 are different physical attempts. H1's completion cannot settle H2's pending transport effect, even if both refer to C42. Compatible command replay is determined by the authoritative boundary, not by the browser picking the response it prefers.
- A logical target may remain `:request-card R` while the actual DOM instance changes from B1 to B2. The old element's listeners, continuity snapshots and deferred mutations cannot act on B2 without newly admitted rights, even if both elements have the same `id` attribute.
- Shared Live invalidations follow their declared fragment/coordinator ownership rather than an unrelated execution-private identity. Revoking E42 must not accidentally erase a separately retained, legitimately scoped Refresh requirement or subscription obligation.
- A suspended tab or newly served deployment re-enters compatibility and source-admission checks before resuming old work. Semantic command recovery may be possible while old physical continuations remain irrevocably retired. Sections 06.08–06.09 own the detailed protocol.

Currentness is not solely a structural test on a copied map. A transition must establish **who issued** the resource, which runtime and generation presently own it, which host source actually emitted the callback, and what semantic event is permitted in the current machine state. Invalid or missing correlation fails closed for protected actions; an already late, otherwise well-formed callback is classified stale/duplicate where the admitted event vocabulary allows it. A malformed event, a stale event, an unsupported host profile and an unknown authoritative command outcome are not interchangeable diagnoses.

Commit semantic state before issuing physical work

The shell validates and commits the selected next state before interpreting effects. This includes the effect issuance records and retirement of any superseded local rights. An immediate completion callback must observe the effect's current issuance and owner rather than a physically existing resource absent from semantic state.

Committing first does not settle reentrancy. A handler can synchronously submit another event, retire E42, replace B1, or begin a newer effect before the outer transition's remaining effects run. The interpreter therefore checks each protected effect's **current** issuance, owner, generation, target and permitted mutation footprint immediately before the last controllable physical act. A previously emitted batch is not an irrevocable blanket permit. No delayed physical mutation may rely on a permit that has ceased to be applicable, even if its later completion would be correctly rejected as stale.

The check must distinguish effect kinds. A retired execution may still require best-effort cancellation of its own old physical handle, a non-mutating diagnostic, or transfer of durable recovery debt. Such cleanup is not permission to restore old presentation or cancel a successor's identically named resource. When a physical action has already partly occurred, the system records actual completed, failed and indeterminate outcomes and follows its declared compensation contract instead of pretending a pre-action check made it atomic.

A conforming dispatch sequence is:

1. normalize the raw event and admit its actual source, owner and legal state;
2. compute and validate the next transition without physical side effects;
3. commit the next state, issuance records, retirement and transfer obligations;
4. offer an immutable record of the **selected** transition to bounded diagnostics;
5. interpret each effect subject to its current actuation gate, with no unadmitted transport or DOM action between the check and controlled mutation;
6. admit actual host completion, partial failure or uncertainty as another event.

One logical dispatcher owns semantic state for an admitted runtime. Nested dispatch must see the newly committed state; the outer frame cannot overwrite changes made by the inner dispatch when it returns. A serialized event queue may be used for events that do not need a synchronous host decision. If multiple worker or host entrypoints can mutate one semantic state, their

admission/commit order requires a real linearization mechanism. A mere read-then-write or matching timestamp does not supply it.

Diagnostic observation is not a second semantic dispatcher. Observer failure may be isolated through bounded buffering or a suitable failure-isolation mechanism, but recording the selected event cannot block a mandatory effect, replace a transition, or leak protected observation data. Promises and callbacks initiated by supposedly observational code must also obey the declared noninterference contract. This does not assert zero diagnostic cost or a sandbox for arbitrary same-process code.

Synchronous lifecycle decisions need a synchronous gate

Certain supported HTMX/browser hooks require an allow/veto decision before returning to the host. The supported integration profile identifies the **actual last cancellable boundary** for each protected physical request, swap, secondary mutation and follow-on effect. A deferred Promise, event queue, later observer or eventual rollback cannot retroactively veto a request already sent or a DOM mutation already performed.

At such a gate the currently admitted semantic dispatcher computes a synchronous decision from the available admitted state, including its source/attempt/target generation and required prerequisites. If the source is uncorrelated, the applicable permit has expired, required knowledge is unavailable, the dispatcher cannot commit safely, or the handler fails, the protected operation is prevented before that act or is rejected as an unsupported realization. Fail closed is a **physical outcome**, not just a logged `:rejected` status while HTMX proceeds. A host profile that cannot actually suppress the protected action is not certified by calling `preventDefault` at an ineffective point.

The gate may direct a separate, explicit reacquisition or retry flow. That flow owns its own attempt identity, checks authority and required progression again, and retains C42 only if it is a compatible retry of the same still-admitted intent. An unknown result must not be silently relabelled rejected or replaced by a fresh Claim intent. A prepared `:needs-input` or deferred affordance is not evidence that a previously unadmitted physical send was allowed.

Synchronous decision-making does not imply that a handler is free of later physical consequences. Timers, Promises, custom-element reactions, animation callbacks, extension activity and queued work require their own correlation and last-actuation safety. Section 06.03 specifies the HTMX response and mutation surface; 06.05 and 06.10–06.11 specify actual installation, effects and async admission.

Paired identity and lifecycle histories

These cases require independently sourced identity/host evidence, not an inspection map fabricated to match the expected fields.

Eligible history	Adverse h
C42 remains one admitted Claim while H1 times out and a permitted H2 retries under the same immutable command binding.	H1's late r
E42 issues effect F1 to B1; the shell commits issuance before a synchronous callback re-enters dispatch.	A callback
A newer E43 owns B2 at the same DOM selector after E42 retires; an old queued DOM effect checks current target and stops.	The old ef
A supported HTMX before-request gate has an admitted current permit and allows the exact attempted send.	The sourc
A scoped Live invalidation arrives during E43 while a private E42 timer also fires late.	The runtir
A sleeping page re-admits compatible current assets and recovers C42 through a qualified resolver.	It resumes

The finished framework must support ordinary useful HTTP/HTMX and Clojure application paths alongside these gates. Missing mandatory correlation rejects the incomplete **managed** path, not all raw HTMX or every unrelated ordinary handler. Browser event admission guards local realization; trusted server/model policy still guards actual RequestAssignment mutations and protected reads.

HTMX remains the declarative transport and swap owner

For a declared HTMX integration, HTMX owns its supported ordinary request, response-processing, swap, out-of-band (OOB) and settle lifecycles. Gesso binds protected actions to admitted semantic plans, applies narrow allow/veto and correlation gates where necessary, and accounts for their actual effects. It does not reimplement HTMX's ordinary transport and DOM behavior or secretly issue a second physical request/swap while pretending the HTMX action was the one controlled. Explicit unmanaged HTMX remains legitimate under its own server-enforced policy; weaker transport behavior is never a silent fallback from a failed managed operation.

The supported **browser integration profile** identifies an actual running combination, not merely a dependency name. It includes:

- the installed HTMX and extension versions, configuration, response handling, custom swap strategies, history/navigation behavior and loaded application assets, with the applicable browser/host assumptions;
- the permitted request modes, form/submitter overrides, targets, swaps, response statuses, headers, OOB fragments, scripting and extension paths;
- documented lifecycle events and the **actual last controllable boundary** for each request send, response-driven action, target mutation and deferred effect, including which decisions are synchronous and cancellable;
- adapter-issued attempt, source, owner, request generation and physical target-incarnation correlation carried by supported host events;
- permitted action and target footprint, effect ordering/partial results, and the evidence required to establish real completion, failure, cancellation or an indeterminate physical result;
- positive and adverse conformance tests for that exact profile, with version-compatibility, escape and unsupported-mode dispositions.

A matching library version string does not prove an extension, alternate startup path or runtime configuration obeys the same gates. A custom HTMX extension is not required in principle. If a particular supported behavior cannot be safely realized through the ordinary documented hooks, the framework may provide a separately specified wrapper/extension/alternate realization with its own host correspondence evidence, or reject that combination. It may not claim protection from a hook that cannot veto the relevant physical action. Changing an external HTMX event name should not change Gesso's underlying Choreo/browser semantic requirements.

The integration must account for the request's effective method, URL, form/submitter parameters, headers, target, swap and entry binding, rather than trusting only a generated annotation or its initial DOM shape. An application may render a valid managed affordance and later replace it, inherit unexpected attributes or use an alternate submitter; the **actual admitted physical request** must still correspond to the protected operation and trusted route. Neither client-side checking nor a copied execution/command ID supplies server authorization.

For every managed response or host action, the profile answers:

Boundary	Required qualification
Before physical send	Which actual request, owner, correlation, binding and synchronous gate permit this irreversible attempt?
Before response-driven effects	Which result bytes and metadata are admissible, which secondary behaviors may run, and what happens
Before each protected mutation	Which target incarnation, scope, permitted representation and still-current physical actuation right cove
After the physical act	What actually completed, for which target/effect instance, and what remains failed, partial, indeterminat
Across replacement/recovery	How are old element owners, pending events, subscriptions, continuity and fragment coordinator obligat

The request gate and mutation gates are not interchangeable. A successful `:htmx/before-request` decision permits at most the corresponding attempt; it cannot authorize arbitrary response headers or later targets. A `:htmx/before-swap` decision is not evidence that every OOB, script, history or extension effect has already been controlled. A callback named `:htmx/after-swap` is an observation whose correlation and actual completed scope must still be checked; it cannot establish a command's incorporation or a whole-board adoption by its name alone.

For a supported cancellable lifecycle hook, a failing normalization, missing source identity, expired permit or throwing semantic gate must physically prevent the protected action. An asynchronous decision returned after the request or mutation starts does not provide the required veto. No application can claim a stronger browser safety invariant when an unexamined response path can mutate that same protected state outside its admitted footprint.

Response data, response effects, and content safety

A response may convey a command-specific historical result, an authoritative observation, replacement HTML, response headers, OOB fragments, browser events, navigation/history instructions or active content. These are different **subjects of admission**. Authorizing one does not authorize the others. A browser-visible HTTP status or response body is neither a trusted business-rule decision nor evidence that the user's display was installed.

For HTMX 2's ordinary `hx-swap`"none"= behavior, suppressing the main target insertion does **not** imply a response without browser effects: OOB content and applicable response headers can still be processed. A managed command-response endpoint using that mode must explicitly admit, isolate or exclude such effects. For example, a response that reports C42's result may still carry an OOB replacement of a Request card or a response-triggered event; neither gets permission from the mere fact that C42's command outcome was admitted. A registered side effect must have its own owner, disclosure scope, target and completion contract. A malformed or unauthorized additional effect is rejected or quarantined **before** its protected mutation. A typed error recorded after unintended DOM replacement is too late.

The profile inventories the applicable response and extension mechanisms, not just the primary HTML swap. Depending on the qualified HTMX version and configuration, these include OOB swaps, response-specified retargeting and swap overrides, client-side events, redirects/location changes, history updates, script processing, native form behavior, and extension-defined mutations. Response-triggered events can themselves cause further requests or application actions; a declared effect chain must either be admitted through its own entry gate or excluded. A configured `HX-Trigger` header cannot silently become an unexamined mutation capability. Likewise a response carrying `HX-Retarget` or `HX-Reswap` cannot enlarge a previously admitted mutation footprint without a new effective check.

A protected main-target permit does not grant all OOB targets. Each secondary target requires a valid current instance, authorized content/disclosure scope and mutation permission. One multi-target permit is possible if the complete footprint and intermediate physical behavior are genuinely covered; otherwise individually scoped permits and completion records are needed. Sequential DOM writes are not automatically physically atomic. A later failed target does not undo earlier physical changes, and observed partial completion must retain the required repair/adoption debt. Section 06.05 owns the detailed scoped installation contract.

An application may safely choose a restricted response profile (e.g., permitting only one known inert fragment and prohibiting unexpected response effects), but restriction has to be enforced at the **actual response-processing boundary**. Merely expecting the server to return a harmless shape is insufficient if any other reachable path, cache, extension or response override can produce an unreviewed mutation.

Authoritative provenance and HTML/content safety are separate. A qualified XTDB observation does not make arbitrary user-supplied HTML, JavaScript, `hx-*` attributes or event handlers safe for insertion. Rendering declares escaping, sanitization where untrusted markup is intentionally supported, allowed active content, script/CSP policy, HTMX attribute processing and extension behavior. The trusted construction and serving path must prevent client-supplied content from manufacturing managed actions or protected subscription/command bindings. Rendering into an inert container can be a useful preparation step, but later activation and execution are separate physical effects.

The actual checked fragment must preserve the relationship among its protected data, operation binding, target/coordinator instance, authoritative observation and accepted protocol version. Copying an old `:choreo/op` annotation onto a newly generated button does not provide admission. Replacing a stable Live/HTMX behavior-owning root without re-establishing its declared

connection, trigger, current-owner and continuity hooks can silently end all subsequent refreshes. An admitted replaceable inner fragment may change while its owner remains stable; if the root itself is replaced, the new owner and subscription lifecycle must be explicitly reconstructed and checked.

Transport and response conformance histories

Each positive case must exercise the real supported HTMX processing path and the expected source/owner bindings; negative cases must reach the relevant physical boundary rather than fail on an unrelated malformed fixture. The table specifies finished-system tests for the actual qualified host profile and its explicitly recorded assumptions.

Eligible positive path
An admitted managed Claim H1 receives a qualified C42 settlement through a command-response with `hx-swap`"none"= and no unadmitte
A protected acquisition swaps the current target B2 under an admitted permit and reports actual completed installation.
A supported cancellable pre-send hook binds actual form method, route, arguments and owner to the admitted operation.
A permitted HTML fragment escapes untrusted Request text and preserves its validated data/operation bindings after replacement.
A complete authorized refresh replaces the inner fragment while stable root owners, Live subscriptions and continuity resources survive o
A scoped installed result and a separate current command readback establish their own qualified propositions.

Ordinary raw HTMX interactions continue to work on legitimate routes under ordinary trusted server authorization. Managed stronger promises apply only when the exact selected rendering, transport, response, target and deployed integration paths have been admitted. Part VII must verify the actual application closure for those paths; Part VIII requires nonvacuous eligible HumanHelp Create/Claim/Reassign and one-Refresh product behavior, not merely passing negative fixtures.

Worked example: Claim followed by protected acquisition

Alice sees HumanHelp Request R in a qualified, authorized observation of the currently selected Organization/Location. The rendered Claim affordance binds one public model-owned operation and typed arguments. It does not mint a trusted principal, establish current Membership, or authorize the write merely because the browser displays `Claim`. The observed pre-read may report an `:open` Request with no active primary `RequestAssignment` under its stated coverage; that is not an atomic, transaction-current eligibility decision.

The example uses these distinct subjects:

Subject	Meaning
R	HumanHelp's model-owned Request identity; not an HTTP request.
C42	Stable semantic Claim command intent and protected binding, allocated before the first possible submission.
E42	One local browser Choreo/optimism execution with separately revocable physical rights.
H1, H2	Distinct physical transport attempts that may carry a compatible retry of C42.
B1, B2	Physical incarnations of the Request-card target, not merely the same DOM selector.
CardObs1	An admitted card observation, with its own source, covered scope and qualified progression.
P1	A runtime-issued installation permit for an exact representation, target and effect instance.

The values are illustrative. A later execution may continue permitted C42 reconciliation, but does not inherit E42's retired transport, timer, target or actor-local branch knowledge. Two requests that reuse C42 are not automatically interchangeable in their transport-result correlations. The command's immutable semantic intent is bound to its operation, arguments, actor/authority context and compatible replay contract under Part IV. A copied ID in a request header is not that admission.

The following trajectory is a supported **design**, not a claim that these names are already the public browser API:

Stage	Required semantic/physical correspondence
Prepare	Admit the current view binding and relevant scoped observation. Establish a stable C42 intent and local E42/target
Provisional	Derive and install only the locally admitted provisional presentation; failed physical installation is reported as such
Request construction	Bind an exact H1 physical attempt, current execution/generation and effective HTMX method, route, inputs and se
Send admission	At the supported synchronous hook, verify the projected send state, still-current attempt/target owner, complete p
Trusted Claim	Authenticate the actual principal and guard current Request, Organization/Membership/actor and active primary R
Outcome observation	Admit binding-scoped C42 command evidence from the qualified response or permitted readback. Distinguish inco
Acquisition	Independently admit permission to read the current authorized card or board cohort, its required query/scope and
Installation	Derive a prepared representation from the admitted CardObs1 observation and acquire a current P1 covering its ac
Completion/adoption	Admit actual completed physical effect/postcondition for the matching CardObs1/P1/target. Discharge only its cov

Part IV owns the **authoritative** Claim transaction and historical command knowledge, Part V the local provisional derivation/settlement treatment, and Part VI the browser's actual effect and installation boundary. These are composed responsibilities, not competing interpreters.

A permitted `:committed` result, or a qualified `:already-incorporated` result for C42, establishes the specified historical incorporation, subject to the authoritative source and binding contract. It may permit an `:accepted` presentation status; it is not an assertion that today's primary helper is Alice, that a particular HTML fragment installed, or that the board cohort changed. A terminal `:rejected` command outcome requires its own qualified exclusion evidence. `:commit-unknown` remains unresolved until an admitted resolver establishes more. The browser's generic optimism `:confirmed`, `:reconciled`, `:failed` and timeout dispositions are not substitutes for the public model outcome vocabulary.

Two independent histories make this distinction unavoidable:

- H1 may have incorporated C42, then the network response was lost. The browser can retire H1/E42's physical resources, recover the current card through CardObs1, and still lack C42-specific historical attribution. It must retain or transfer the command-resolution obligation rather than asserting that the new card proves H1 succeeded or failed. Retrying C42, if permitted, must preserve immutable command binding without executing a second incorporation.
- C42 may have successfully installed Alice's primary assignment, followed by a manager's authorized Reassign to Bob. A historical C42 result and a newer qualified Request/RequestAssignment observation are simultaneously true. An old settlement callback cannot reinstall Alice over Bob; a current Bob card does not turn C42's earlier success into rejection.

Claim's response and card acquisition may use separate physical HTMX requests, which is a particularly clear realization. A combined result and observation is also possible, but each proposition must pass its own source, disclosure, scope, and completion gates. An `hx-swap`”none”= command response must still satisfy section 06.03's OOB/header/secondary-effect contract. Its success cannot be counted as a card swap. A raw `afterSwap` callback is not automatically physical proof of installation in an unrelated or retired target.

When provisional derivation requires asynchronous computation but the chosen HTMX gate requires a synchronous allow/veto, that derivation must complete **before** entering the gate or be deferred to an admitted re-entry/reacquisition path with a new physical attempt. The gate cannot allow the send on hope of eventual projection. Failed, missing or incompatible managed realization remains an explicit typed failure; authored raw or non-optimistic alternatives keep their independently complete authorization and recovery contracts.

HumanHelp's board has an additional adoption distinction. Alice's correlated local Claim may update her own card promptly after the relevant qualified outcome/observation and actual installation. Other actors may receive only a scoped Live advisory until their explicit Refresh grant. Local text search, sorting, selected card, open details and scroll state remain continuity of an already adopted authorized cohort, not proof of a new server query. A creator-local newer S must not be overwritten by an older whole-board R snapshot; one explicitly granted remote Refresh must satisfy its stated scope without silently acquiring an extra cohort. The detail of these policies belongs to Parts IV–V and 06.05–06.07, not to an ad hoc Claim listener.

Browser FFI contracts are small but substantive

The browser FFI is the physical interpreter of **adapter-issued** effects, not another source of command meaning, business rules or Choreo branch selection. HTMX owns its native request/response and supported swap behavior; a Gesso-owned effect handler may intervene at admitted HTMX gates or perform distinct Gesso-owned timer, provisional DOM, continuity, subscription and cleanup work. It may not quietly duplicate an HTMX request or swap.

Every supported physical effect handler has a registered contract:

- **Input and binding:** typed effect kind, implementation and protocol/asset identity, closed required inputs and admitted normalized source; unsupported inputs fail explicitly.
- **Ownership:** exact runtime, semantic effect instance, resource incarnation, current owner/generation, target incarnation, representation and allowed physical footprint.
- **Actuation:** synchronous/async classification, actual last-controllable permit check, required serialization with protected mutation, and any possible host reentrancy.
- **Outcomes:** actual completion postcondition, correlated effect and target, failure, cancellation, partial/indeterminate mutation, duplicate delivery, and unknown physical result.
- **Cleanup/recovery:** registration before issue, handling of immediate callbacks and handles returned after retirement, exact-resource release, stale callback no-op, observable residual debt and any necessary compensating/reacquisition path.
- **Premises/evidence:** supported host/HTMX version, external behavior relied on, test/proof scope, and unproved opaque or third-party effect assumptions.

A handler returning a syntactically successful envelope does not establish a physical action. The completion contract must identify what the host actually accomplished, or classify its result as unknown/partial. An absent, throwing or misbound mandatory handler must not silently satisfy the projected semantic obligation. A sidecar or observer callback cannot supply the missing effect.

The shell's physical registry maps adapter-issued references to actual opaque host objects. Correlation is established at issue or registration and carried through callbacks, not rediscovered by whatever element currently matches a selector. A timer cancellation cannot remove the successor owner's identically named timer. A delayed DOM write cannot act on B2 using a revoked B1 permit merely because the eventual callback would be rejected. If the host supports a guarded last-actuation section, its check and mutation must preserve current ownership under the declared serialization/reentrancy contract. If it does not, the selected physical realization is unsupported for that strong guarantee.

The current semantic transition is installed before issued effects are interpreted. Immediate physical callbacks must therefore see committed ownership and effect state. A nested callback may retire the outer owner before its remaining effects run; every remaining protected effect must recheck current permission. Concurrent dispatchers need a genuine serialization mechanism rather than two mutable competing copies of the same semantic state. The FFI may retain physical handles, but cannot choose a semantic successor or transfer authority as a shortcut around that protocol.

Ordinary application code and third-party browser libraries can mutate the same DOM and trigger network work. For a protected footprint, those actions must participate in the admitted host profile, be prevented from interfering, or remain an explicit noninterference/trust premise with corresponding limits on the claim. Full-language Clojure application development does not eliminate physical or opaque effect obligations.

Paired Claim and FFI correspondence histories

These cases state nonvacuous finished-system acceptance conditions; each successful path must reach the actual intended source and physical effect, and each adverse control must fail at its target boundary rather than at an unrelated invalid fixture.

Eligible positive history
Alice has authorized Request/assignment pre-read; C42/H1 executes against current trusted Claim guards and creates one valid active prim
H1 has unknown delivery and a compatible H2 replay of C42 obtains qualified historical incorporation evidence.
C42's permitted historical result is admitted, and a separately authorized CardObs1 card is physically installed at current target B2.
A new primary helper Bob is authoritatively reassigned after Alice's Claim, and a qualified newer card installs.
An effect is registered before issue and its immediate callback re-enters dispatch; mandatory handler reports a checked real postcondition.
A correlated self-action updates Alice's card; a remote actor receives advisory R, while an independently newer local S survives the actor's
A legitimate third-party host operation uses a declared and qualified protected mutation footprint.

Parts VII–X retain independent installed route/asset coverage, backend/physical correspondence, HumanHelp positive capability and proof-kernel obligations. An accepted design example, test fixture or emitted plan digest does not certify the actual deployed application.

Scoped authoritative installation

Part IV distinguishes an authorized observation from the act of acquiring it and from a participant's subsequent adoption. Part V distinguishes authoritative presentation from provisional overlays, continuity, and historical command outcomes. The browser must preserve those separations across the **actual physical effects** of HTMX and the Gesso-owned shell. A result about one card, field, or complete query cohort cannot be promoted to a claim about every other visible or required scope.

Stage	What is actually established
Acquisition permitted or requested	The current owner may start, or has started, a specified read within a declared grant.
Observation acquired	An admitted source establishes particular data, coverage, authority context and qualified progression
Installation prepared	A proposed representation and target/mutation footprint have been derived from that observation.
Installation permitted	The semantic runtime has issued a currently valid, precisely scoped right to attempt that physical act
Installation reserved or in progress	The host has entered the specified guarded actuation protocol.
Installation completed	A correlated physical postcondition is established for each covered target/effect instance under the su
Observation adopted	The participant's semantic state records qualified completed installation for the precise covered scope

These are semantic distinctions, not a requirement for seven different public callbacks or an extra persisted record per DOM node. An implementation may combine stages whose premises are established together. It may not erase the stage or invent its evidence. An HTTP 200, response parsing, `htmx:beforeSwap`, permitted swap, generic `htmx:afterSwap` callback, or matching plan digest is not by itself a whole-application or whole-board adoption witness. The selected supported host profile must explain what each callback actually establishes, including its limitations for OOB, nested, deferred, missing-target and extension effects.

For example, a complete authorized Request-board observation with membership scope Q may establish information about the board cohort while a correlated newer Request-card observation establishes the current presentation of S independently. Conversely an acquired card alone does not establish the absence of R from an open-request collection, or that every card in Q came from one snapshot. The browser must represent both qualified facts and the obligations each one has or has not satisfied. Neither the identity of a database basis token nor the act of swapping a DOM subtree alone settles the comparison.

1. Installation permits bind the intended physical act

   An installation permit is a **runtime-issued semantic right for a physical operation**, never a credential for a server-side domain write. It must bind the decision to exactly the realization it permits, including:

   - the admitted runtime/assembly and relevant authority-context references, with their freshness and disclosure premises;
   - the current execution, coordinator or declared shared owner, its generation and the actual effect instance;
   - the observed data's source, semantic scope, query/cohort and qualified progression or coverage references;
   - the prepared representation's content-sensitive identity, render/binding contract, version and supported host profile;
   - the actual target incarnation(s), intended actions and complete mutation footprint, including secondary/OOB effects if admitted;
   - the physical actuation/serialization and completion postconditions, partial-failure disposition, cleanup and recovery obligations.

   One illustrative inspection shape is:

```
{:permit/ref installation-permit-ref
 :runtime-instance/id runtime-id
 :assembly-deployment/id installed-assembly-ref
 :authority-context/ref context-ref
 :owner/ref board-coordinator-ref
 :effect-instance/id installation-effect-id
 :observation/ref qualified-observation-ref
 :scope/ref covered-query-and-projection-ref
 :representation/ref prepared-content-ref
 :targets [current-board-target-instance]
 :action :dom/replace-children
 :host-profile/ref qualified-htmx-profile-ref
 :completion-contract/ref checked-postcondition-ref}
```

   The record illustrates a permit **already selected and registered by the semantic runtime**. A matching client map, repeated digest, copied DOM attribute or proof sidecar does not mint one. Neither a browser permit nor a completed DOM effect grants authority to call a protected server model operation.

Permits have explicit issuance, reservation/physical commitment, completion, failure, revocation and retirement semantics. An issued but unconsumed permit can be revoked. A physical act that has crossed its defined commitment point cannot be treated as unexecuted merely because semantic ownership subsequently changes. A duplicate completion cannot replay the physical act, adopt the same evidence twice or consume a successor's permission. A successor target at the same CSS selector or HTML `id` is not the same physical incarnation.

2. Final admission covers delayed, reentrant and secondary effects

   Expensive or arbitrary application rendering occurs under its declared full-language/effect contract before the last protected actuation where possible. The runtime verifies that its result still matches the observation, permitted representation and current target. It must not install a once-authorized stale render merely because rendering completed successfully.

   Immediately before every protected physical mutation, the shell or admitted HTMX integration verifies the still-current permit, owner, effect instance, target incarnation, representation and scope. Checking a captured `allowed?` bit at request time and then rejecting only the **late callback** does not protect a later DOM act. The final check and the physical commitment point require a stated serialization protocol; if user code, custom elements or the host can reenter during mutation, the protocol must identify reservation, suspended conflicting transitions, partial effects and exact failure recovery. One JavaScript event loop does not make a multi-target swap an atomic semantic transaction.

   An old E42 effect may have been admitted for B1 and scheduled for later execution. In the meantime, another authorized installation replaces B1 with B2 and revokes E42's permit. The queued effect must not mutate B2 even though B2 has the same selector and Request R. Its late physical completion or failure may be observed for cleanup, but cannot restore B1's obsolete representation. A recognized stale permit is suppressed at actuation, not mislabeled as successful install.

   For HTMX-owned effects, section 06.03's version/configuration profile must supply an actually effective pre-mutation gate for the entire protected footprint. A main-target `:htmx/allow-swap` does not license an unexamined OOB target, response header, extension mutation or reentrant follow-on request. If a supported host profile cannot control the protected act at the required boundary, it must use a separately qualified realization or fail admission before the unsafe effect, not declare success from a later callback.

3. Actual completion determines adoption

   The completion evidence identifies the issued permit and effect, actual resource/target instances, admitted source, realized representation and physical postcondition. The host profile may accept a sufficiently correlated lifecycle event, a checked DOM predicate, or a stronger instrumented mechanism, but must say which concrete mutation paths it covers. A generic event with the right name is not proof that an unrelated target's contents, owner or required accessibility/interaction bindings were installed.

   Admission of a completion event checks its current semantic owner, target incarnation and source as well as any intervening installations and observed physical effects. A physically completed **older** act may remain historical diagnostic evidence without advancing today's adopted frontier. If completion reporting is lost or ambiguous, the browser cannot manufacture adoption from matching text in the DOM. It retains an explicit reconciliation obligation: inspect under a qualified physical predicate, reacquire/reinstall an allowed scope, or use another declared compensation or recovery mechanism.

   An installation can succeed in one target and fail in another. The runtime records completed, failed, pending and indeterminate per-target outcomes with their respective observation, effect and owner bindings. A completed card does not justify marking the board's membership cohort fully adopted; a completed board shell does not establish all internal card/action bindings merely because the outer element exists. If the product requires all-or-nothing presentation, the physical mechanism must actually provide that atomicity or the declared compensation protocol must specify its observable intermediate states. Ordinary sequential DOM mutations do not satisfy that promise.

   A partial or inconsistent realization must not expose enabled semantic affordances whose guarded bindings no longer correspond to their visible content. The runtime may quarantine the affected target, preserve valid independent scopes, or perform declared reconstruction. It cannot quietly downgrade a managed action into unprotected HTMX or use an old provisional snapshot as a universal ”undo”. Actual model incorporation is not reversed by a local DOM failure.

   Optional focus, scroll or disclosure restoration is a separate local continuity operation. Failure to restore such optional state after a successfully admitted authoritative install does not annul the command or that installation. If a mandatory user-draft barrier was declared before replacement, its own preparation contract determines whether replacement was authorized; it cannot retroactively veto an already completed authoritative operation. Physical resource cleanup remains owner- and instance-relative even when an old owner and its successor set the **same** CSS property value.

4. Progression, coverage and selective supersession

   An installation can supersede provisional presentation or an older authoritative projection only where its scope and qualified progression/provider judgment authorize that replacement. Opaque XTDB bases do not acquire a total order

through numeric-looking labels. The browser must preserve actor/authority context, projection, selected query, collection membership, fields, temporal interpretation, target ownership and observation source.

The satisfaction relation is obligation-specific. An acquired entity card may satisfy an entity-information requirement but not a complete filtered board query. An observed collection absence requires the proper authorized cohort coverage; it is not proved by a missing node or a locally hidden item. Advisory obligations whose required discharge stage is **adoption** persist after fetching the response until the corresponding physical scope actually installs. Obligations that arrive while a read is in flight remain if that read does not cover them.

A whole-board R response can overlap a separately newer, correlated creator-local S card. The correct result may adopt the board membership from R while preserving S's more advanced Request-card representation. That requires a **qualified scope-aware merge** or other admitted policy, not a fabricated claim that the mixed view is one XTDB snapshot. If a safe merge cannot be justified, the runtime rejects/reacquires the conflicting scope, retains independently valid presentation and its unsatisfied obligations, or follows an explicit recovery strategy. It must not erase S merely because the whole-board response was initiated later.

Likewise a remote actor's advisory may establish new Refresh debt without granting a new read. One explicitly granted Refresh binds the applicable query/context and read budget, and may discharge only the obligations satisfied by its one qualified acquisition and actual installation. An implementation may offer a different declared automatic-drain policy, but one granted read cannot silently become unlimited successor acquisition. Sections 06.06–06.07 own that coordinator protocol.

5. Paired physical-installation and adoption histories

   Each positive history needs an eligible authorized source, actual host execution and a checked target-specific postcondition. Each nearby negative case must reach the intended rejecting boundary rather than fail earlier on an unrelated malformed fixture.

Eligible positive history	Adv
A current coordinator obtains CardObs1 for card R, reserves P1 for B1 and physically installs the admitted representation.	The
P1 is issued for B1; a newer authorized install creates B2 and retires P1 before a deferred effect acts.	The
A two-target operation completes card B1 and count T2 under individually tracked permits and matching physical postconditions.	Car
A valid R board result is adopted under its membership scope while independently newer local S remains displayed in its card scope.	A b
An explicit one-Refresh grant reads the admitted board query, installs covered scopes and discharges the obligations it satisfies.	An
A replacement succeeds and optional continuity restores focus/scroll when its owner remains current.	Res
A supported HTMX host reports a correlated after-swap completion for the specific tracked footprint under its qualified contract.	A n

These are finished-framework conformance obligations. Part VII must close actual served routes, renderer/asset and physical writer paths; Part VIII requires nonvacuous HumanHelp Claim/Reassign, creator-local presentation and remote one-Refresh behavior; Parts IX–X require explicit proof subjects, trust assumptions and implementation/physical correspondence. A finite adapter test or source-derived diagnostic does not certify those systems.

Fragment coordination

A fragment coordinator is the single semantic owner of acquisition, installation, advisory debt and relevant scheduling decisions for one **admitted logical scope during one owner-instance lifetime**. That scope binds the authoritative applied query, tenant/Organization and actor disclosure context, projection/cohort contract, required progression and relevant deployment/target ownership. It is not a global mutex for every view of the same domain Request; nor may two coordinators independently claim the same current physical target or acquired/adopted obligation without an explicit sharing protocol.

An invalidation is information that a relevant authoritative change **may require** acquisition or adoption. It is not itself a database observation, a trusted command outcome, permission to read protected data, or proof that another participant has adopted that change. Advisory delivery and user-visible Refresh availability can coexist with independently newer local authoritative presentation. The coordinator, not an arbitrary SSE callback, decides how each requirement enters its scoped obligation state.

The coordinator distinguishes these independently owned subjects:

1. **Obligations:** source-qualified advisory or gap requirements, their scope, origin, coverage relation, and required discharge stage (acquisition, actual installation/adoption, or recovery).
2. **Policy permission:** whether, by whom, for what query and how many times an acquisition or installation may be initiated; it is distinct from trusted server read authorization.
3. **Active cycle:** one admitted logical acquisition/installation lifetime, with its captured requirements, current request generation, physical attempts, target/permit identities and qualified completion or repair state.

4. **Successor scheduling:** at most one coalesced scheduling slot under the default policy, containing a **possible next cycle**, not an unconstrained right to submit more requests.
5. **Evidence:** independently qualified acquired observations, actual installed/adopted scopes, progression judgments and unresolved comparison/repair obligations.
6. **Recovery/resources:** failed or indeterminate effects, subscription continuity, retained debt, semantic retirement, pending physical cancellation and bounded cleanup policy.

The default coordinator promises at most one active **logical** acquisition/installation cycle and one coalesced successor slot for its admitted owner and scope. A cycle ordinarily lasts through physical installation completion or a terminal cycle disposition, not merely until the HTTP response arrives. If a realization separates the network-read slot from installation, it must state and check separate bounds on outstanding prepared and physical installations. Bounding in-flight network requests alone does not establish one coherent current DOM mutation.

```
active logical cycles for one admitted coordinator ⩽ 1
successor scheduling slots for that coordinator ⩽ 1
```

This is a **logical scheduling bound**, not a claim that an aborted request ceases to exist physically, that all callbacks have returned, or that retained debt uses constant memory. Physical remnants from earlier generations can coexist with the active request. They have no installation authority and must have their own release, instrumentation, backpressure and boundedness contract. A single successor slot may represent arbitrarily many incomparable requirements unless the supported requirement domain supplies a sound finite abstraction or an explicit overflow/recovery policy.

1. Advisory debt is not permission to acquire

   A new invalidation first passes source, subscription/interest, current owner and scope admission. It then adds an obligation or produces a **justified** combination with existing obligations. An advisory that says ”newer information may exist” but gives no authoritative minimum-read token still represents debt when the product's contract requires future reconciliation. Absence of a token is not permission to discard it. Conversely a duplicate of an **exact already-covered** requirement need not create new work if a qualified coverage judgment establishes that the active read or adopted result already satisfies it.

   The obligation-to-observation satisfaction relation comes from Part IV's declared progression/coverage provider. The coordinator does not infer ordering of opaque XTDB tokens, timestamps, SSE sequence labels or request-generation integers. Combining distinct requirements by keeping only the numerically largest-looking basis is not an admitted implementation of that relation. Where requirements are incomparable, each independent condition must survive or be summarized by an explicitly sound provider-supplied representation.

   A product may select one of several **declared** policies:

   - **Explicit user Refresh:** remote advisories update debt and presentation (for example a Refresh glow) but do not initiate acquisition. A qualified user action grants an exact read and adoption policy for the applied query and captured obligations.
   - **Bounded automatic refresh:** a particular declared scope permits one acquisition on an admitted invalidation, possibly with bounded coalesced successors and explicit backpressure.
   - **Deferred or gated refresh:** work waits for edit completion, reauthorization, connection recovery, foreground activity, or another declared prerequisite, without erasing the debt.

   The product must not silently switch among these policies because an HTMX trigger happened, a queue slot became free, or a callback saw a nonempty obligation set. A managed automatic-fetch policy can be useful, but it is not the universal semantics of Live. A disabled or pending managed affordance remains honest about its retained obligation; an ordinary raw HTMX path may have a different, explicitly admitted contract.

   An explicit one-Refresh grant binds its current owner, principal/ authority context, applied query and authorized cohort, captured requirement snapshot, allowed physical acquisition count, installation/adoption policy, validity interval and expiry or handoff conditions. For example:

```
{:grant/ref refresh-grant-ref
 :owner/ref board-coordinator-ref
 :authority-context/ref context-ref
 :query/ref applied-query-ref
 :requirement-snapshot/ref captured-debt-ref
 :reads {:initial 1 :successors 0}
 :observation-policy :current-complete-query
 :adoption-policy :install-if-current
 :expiry-policy/ref current-expiry-ref}
```

This record illustrates an already admitted grant; a fabricated map from client code cannot create an authorization. A permit to **schedule** a read is not the server's permission to disclose Request or RequestAssignment data. Each actual request still passes trusted principal, tenant and current read policy.

The read budget is consumed on the defined admission of the **actual physical acquisition**, not by setting a boolean in a completed handler. An unstarted queued successor cannot spend an expired grant later. Whether a failed attempt consumes the read budget, and whether a permitted retry is the same attempt or an additional read, are explicit grant-policy questions; they are not silently decided by HTTP error timing. A scheduled successor with no remaining permission is blocked, not dispatched; its requirements remain. The provider may return a qualified observation stronger than the captured minimum, satisfying additional debt without consuming another read. Satisfaction does not retroactively authorize a second acquisition.

Grant expiration and page suspension are interpreted under a declared clock and lifecycle source. Delayed timers do not extend permission. A resumed page must revalidate current authority, applied query, deployment and owner as required; holding the old grant reference is insufficient.

2. Coordinator state and transition obligations

The following is an **inspection model**, not another mutable store that application authors maintain next to Gesso's semantic adapter:

```
{:coordinator/id board-coordinator-id
 :owner-instance/ref board-owner-ref
 :scope/ref applied-authoritative-query-ref
 :authority-context/ref context-ref
 :obligations outstanding-scoped-debt
 :permission current-grant-or-none
 :active-cycle active-cycle-or-none
 :successor-slot successor-intent-or-none
 :acquired-evidence acquired-scoped-evidence
 :adopted-evidence adopted-scoped-evidence
 :continuity subscription-and-gap-state
 :physical-remnants bounded-cleanup-records}
```

Each obligation identifies what was learned, by which admitted source/interest or continuity recovery event, for which authority/query/projection, and what stage qualifies as discharge. Coalescing is permitted only if the replacement representation preserves all required facts or records what remains unknown. For incomparable provider requirements, a bounded scheduling slot cannot itself guarantee bounded retained-data size. Limits and overflow behavior must be specified separately, including whether progress can block, degrade to a conservative reacquisition, or require a new user grant.

Event or boundary	Required semantic result
Authorized scoped invalidation while idle	Record or soundly combine its obligation. Start a cycle only if the currently selected
Invalidation while an acquisition is active	Retain any requirement not proven covered by the active/captured read; coalesce sch
Duplicate or already satisfied requirement	Suppress redundant scheduling only under a valid exact/scope-aware coverage judgr
Explicit user Refresh	Admit the current applied query, authority context, requirement snapshot and read b
Physical request constructed/issued	Register the logical cycle and its exact attempt/generation, source, query, captured o
Authorized observation acquired	Admit the actual provider's coverage and progression. Clear only those obligations v
Installation proposed/permitted	Bind current representation and physical target permit; retain adoption-stage debt u
Installation completed or partially failed	Advance only physically completed qualified scopes; retain per-target failure, ambig
Request failed or transport outcome unknown	Retire the appropriate attempt without claiming its covered facts; preserve unmet de
Active cycle finishes	Re-evaluate current obligations and current permission; start at most the admitted su
Applied query, actor, owner or deployment changes	Revoke obsolete private resource rights; transfer only proven-compatible scope/debt
Subscription or connection continuity breaks	Record continuity as unestablished and the required fence/reacquisition obligation, e
Coordinator retires	Revoke scheduling/installation rights before physical cleanup; retain/transfer surviv

A coordinator must not conflate physical completion with a response having passed a `:htmx/after-request` callback. An acquired, but uninstalled, board is not an adopted board. An advisory received while one read is in flight is discharged only by an actually qualified observation and its required adoption stage, not because the response happened to arrive later.

Consider O1 requiring a complete authorized board observation and adoption under its applied Organization/query. Bob presses Refresh, granting exactly one read A1. While A1 is in flight, another relevant advisory O2 arrives. If the actual result

satisfies only O1, O2 remains even when A1 installs correctly. If the result independently satisfies both requirements and its required scopes really install, both may clear without another request. If A1 fails, both relevant unsatisfied requirements remain; no ungranted successor may start. If A1's observation is valid but its DOM installation fails, acquired evidence and outstanding installation debt coexist. Section 06.07 gives the expanded same-case table and subscription-gap recovery.

A failed cycle with no queued successor is not proof that its debt disappeared. The implementation must preserve unmet requirements independently of its queue slot, even when it has exhausted automatic retries or cannot currently schedule work. Conversely, installing another admitted observation that provably satisfies the debt may discharge it without initiating a redundant read. Logical success, unsatisfied obligations and retained physical cleanup are different state dimensions.

A late A1 response after A1's owner retires cannot install into A2's target just because its basis looks stronger. The physical attempt, current source, effect and target incarnation must still be admitted. A valid old observation may be reused only under a separately admitted current read/installation policy with qualified progression and disclosure. It cannot revive A1's expired actuation permit.

3. Boundedness, fairness and correspondence obligations

   The default one-active/one-successor invariant must be checked on the **entire** semantic coordination cycle, including outstanding installation, not only on the number of in-flight HTTP requests. A stronger application may use a declared parallel strategy with explicit independent target ownership and equivalent safety/ resource bounds, but it must not silently claim the default single-flight theorem.

   A sound checker or exact-finite abstraction must cover admitted/stale owner generations, source-qualified advisory arrival, incomparable obligations, grants and budget consumption, actual acquisition and installation success/ failure, page suspension, late physical requests, and retirement/handoff. Tests that merely call a local :fragment/refresh effect and see one HTTP request prove neither non-loss of obligations nor eventual convergence.

   Safety claims require that no obsolete request can install, no unqualified requirement can be discharged, no read begins without the selected permission, and no independently newer adopted scope is regressed by an older board. Liveness claims additionally depend on user action where explicit Refresh is required, provider availability, fairness, retry budgets, storage retention and a working subscription-gap recovery path. A finite state diagram with a terminal arrow is not proof of termination or convergence.

   Multi-node deployments must realize the same scope, interest, obligation, authorization and recovery semantics as single-node ones. The coordinator cannot infer that silence from a distributed router proves no relevant changes occurred. Interest installation, router failover and missed-delivery reconciliation require named physical guarantees or a qualified reacquisition/fence. This section does not prescribe a prematurely proved router algorithm; 06.07 and the deployment/correspondence parts own those details.

4. Paired fragment-coordinator acceptance histories

   The finished design requires both a real eligible successful path and a nearby adverse/ambiguous case that reaches the particular boundary under test.

Eligible positive history	Adverse or ambiguous histo
Bob receives O1, retains advisory debt and presses one Refresh to admit A1 under the applied board query.	The same advisory automat
O2 arrives while A1 is in flight; A1 installs a qualified observation satisfying only O1.	A1 completion clears O2 m
A1's real qualified complete query observation covers both O1 and O2 and installs all required scopes.	The adapter assumes arbitra
A1 fails or its physical installation is partial; no successor read is permitted.	Failure ends the cycle and e
A1 logically retires, A2 is admitted later, and a late A1 response arrives after an unsuccessful abort.	The late response mutates A
One full-cycle coordinator finishes acquisition and admitted installation before starting its next logical cycle.	It counts only open HTTP c
The user changes the authoritative applied Organization/query while retaining a local text-search draft.	The old owner installs its ea
A subscription disconnect is detected and a recovery fence/read is admitted under its policy.	No more SSE messages arri

   No valid path is accepted merely because all managed actions were disabled. The product acceptance oracle comes from independently required HumanHelp behavior, including active Claim/Reassign, creator-local results, remote advisory without premature adoption and one explicitly granted Refresh where that policy is selected. A fabricated basis map, self-asserted source tag or matching plan digest is not evidence of completed provider acquisition or physical board installation.

5. Worked example: two obligations during one Refresh

   Bob has adopted a qualified, complete HumanHelp Request-board observation for applied query Q and authority context K at provider position P17. An admitted Live advisory O1 establishes an outstanding requirement for compatible Q coverage

satisfying provider condition P18. Under this product's **explicit-user-Refresh** policy, O1 creates advisory debt and a visible Refresh affordance, not automatic permission to fetch or adopt another board. Alice may independently create or Claim a Request while Bob is viewing Q.

Bob presses Refresh once. The admitted action creates grant G1, capturing the applied Q/K scope, O1 and a budget of one physical acquisition with no automatic successor. A1 is the admitted logical acquisition/installation cycle; its physical HTMX attempt, installed target and resource generation have distinct identities. During A1, a separate admitted advisory O2 arrives with a requirement associated with P20. No ordering or satisfaction relation follows merely from the labels P17/P18/P20. The relevant progression provider must establish it.

```
;; Illustrative already-admitted inspection values, not authority grants
;; created by supplying these maps from an arbitrary browser.
{:grant/ref G1
 :owner/ref board-owner-B1
 :authority-context/ref K
 :query/ref Q
 :captured-requirements [O1]
 :reads {:initial 1 :successors 0}
 :adoption-policy :qualified-scoped-install}

{:cycle/ref A1
 :grant/ref G1
 :query/ref Q
 :captured-requirements [O1]
 :request-instance/ref H1
 :owner/ref board-owner-B1
 :status :acquiring}

{:obligation/ref O2
 :source/ref admitted-live-interest
 :query/ref Q
 :required-stage :adopt
 :progression/ref P20}
```

These maps display distinctions; they do not implement :advisory/satisfied-by?, create a trusted provider observation, or justify any physical DOM installation. In particular, a field named `:progression/ref` is not an assertion that P20 is comparable to, or later than, P18.

When O2 arrives, the coordinator adds it to the current obligation set. It does not modify the historical snapshot captured by G1, silently expand the read budget, or assume that A1's already constructed request includes O2's minimum-read requirement. The **actual returned observation** may nonetheless independently satisfy O2. Discharge is calculated against the current outstanding obligations and their required stages, not against only G1's originally captured list.

Qualified A1 result and actual permission
A1 returns a complete authorized Q observation O18, provider-proved to satisfy O1 only; the current target installs it successfully.
A1 returns O18 satisfying O1, but HTML or DOM installation fails or remains indeterminate.
A1 returns an independently admitted O21 for Q and the provider establishes that it satisfies both O1 and O2; all required physical scopes i
A1 fails before obtaining admitted current evidence.
A1 returns an observation that covers O1 but has no comparable judgment for O2.
G1 authorizes exactly one acquisition; a queued successor intent remains after A1.
A different declared policy explicitly grants one bounded drain-debt successor.
Another qualified physical installation already satisfies O2 while A1 is active.

If A1's response has a provider progression satisfying both O1 and O2, **but** its actual physical footprint covers only one card, adoption-stage board obligations remain until the declared board cohort postcondition is established. Conversely a separately qualified newer local card S may remain adopted while Q's board membership is adopted from an older compatible response R. That selective merge cannot be described as one atomic backend snapshot; it must preserve each source, coverage and physical representation relationship under the policy of 06.05.

The original R4 examples used X17/X18/X20/X21 to make the two possible outcomes concrete. These labels remain useful examples, but editing a fixture's `:basis` field from X18 to X21 does not issue a new admitted observation or prove the

provider's satisfaction relation. Test witnesses must come from the actual qualified source, an explicitly modeled provider or a sound fixture generator, with separate installation completion evidence. No unconditional comparison such as `X21 > X20` is permitted.

A1 may be retired while an attempted physical abort fails. A later response carrying an apparently stronger provider token cannot write into the new A2 owner/target or discharge its obligations using A1's revoked permit. If that observation is still useful, its source/authority and progression must be readmitted under a **new** applicable owner and permitted installation path. A later physical callback cannot resurrect a retired logical acquisition.

An explicit one-Refresh test needs an actual authorized eligible board and a counted physical request budget. It must positively show the first permitted request, then prove no ungranted successor was sent, while remaining debt stays visible. A test that merely disables the button or intercepts all fetches does not establish the required positive product behavior.

6. Applied query identity, local drafts and owner changes

   The coordinator's authoritative scope is the **applied** server query and its authorized cohort. It binds tenant/Organization, principal/actor and current disclosure context, projection, relevant server-side filters, selection criteria, temporal interpretation, and required coverage/progression semantics. A sort order is included when it changes the server query or meaning of the returned representation; pure local rearrangement of the same acquired cohort remains a presentation choice.

   Typing in a local live-search field need not initiate a new server query: it may filter an already acquired authorized cohort while retaining the same applied Q. Local search draft, sort preference, selected card, open details, scroll and compatible input drafts are presentation/continuity data. They cannot create trusted collection membership or rewrite the acquisition's authorized scope. Conversely, if a filter **is applied** to the server query, a previously in-flight response for Q1 may not be installed as if it answered Q2.

   A normal applied-query change admits a new coordinator owner and target generation, revokes the old owner's private resource and installation permits, and creates or transfers the newly applicable obligations under a checked coverage and authority judgment. Compatible independent observations may be retained, but a merely copied query key, DOM selector or subscription ID is not evidence of such a transfer. The old subscription may briefly coexist physically with the new one during handoff without remaining semantically authorized to install into its successor.

   Authority-context transitions require the same explicit re-admission. A logout, Organization switch, Membership change, or altered disclosure rule invalidates assumptions attached to the earlier principal/context. The browser must not retain protected old content, subscription interests or acquisition permissions as if they belonged to the new context. The trusted server independently checks every read, stream subscription and write, including those submitted by a hostile client that bypasses the browser adapter.

   Continuity restoration may preserve an authorized local draft, focus or scroll under its owner policy. It may not reinstate an old **applied** server filter, older Request card, outdated operation annotation, unapproved user draft, or subscription owner as though those were harmless UI state. An optional failed focus restoration after an admitted installation is a local fault; it does not revoke the authoritative observation or authorize obsolete rollback.

   HumanHelp's remote actor who has O1 sees an advisory and Refresh glow, but no hidden acquisition under the explicit-Refresh policy. Alice's own correlated new Request S can be installed locally after its qualified operation and card observation, without requiring every remote participant to adopt it. When Bob finally grants one Refresh, it must preserve a separately newer locally installed S in a compatible scope rather than blindly overwriting S with older board R. The product's positive behavior includes actual enabled Claim, Unclaim, Done, Cancel and manager Reassign where eligible, not only correct rejection of deliberately invalid commands.

7. Initial load and subscription recovery close the same gap

   Initial server rendering is also a qualified acquisition and physical installation. Its projection, applied query, trusted disclosure scope, coverage, installed asset/plan compatibility and physical owner must be related to the subsequently admitted Live subscription. An HTML fragment's mere presence on page load is not proof that its authoritative baseline and subscription are coherent.

   Consider an initial authoritative board observation B taken at provider position P17. The stream/interest that should report later changes may not be effective until after B. A relevant write W can occur in that interval. If no subsequent event is delivered for W and no other fence is established, the browser cannot infer that its initial B remains current. Establishing the stream after rendering is not, by itself, a coverage proof.

   A supported provider/transport realization closes that hand-off gap by an explicitly justified mechanism, such as:

   - registering the durable interest/stream before capturing the baseline, buffering or replaying admitted invalidations, then reconciling against a proved overlap/fence;

- an atomic or otherwise qualified subscription/read fence that establishes all changes up to a declared provider position are reflected in the baseline or replay set;
- durable cursor/replay with explicit retention, delivery, source, ordering and gap detection semantics;
- an admissible post-interest current-state read whose provider/stream protocol proves coverage of the relevant gap, including authorized query and applied-scope identity;
- bounded periodic reacquisition with an explicit staleness/ liveness contract, where continuous no-gap knowledge is **not** claimed without stronger evidence.

The same question arises when a router first learns of a node's interest, when a browser reconnects, or when a standby router takes over. An invalidation that no router formed or retained cannot be reconstructed merely by resending the messages still available to a recovering router. Idempotent redelivery is useful for messages that **were** generated and may have been lost; it is not a proof that every write inside an interest-registration race was ever represented. A qualified baseline/fence, replay contract or reacquisition is still required to establish the promised current-state coverage.

If the system provides only best-effort SSE notifications, the safe contract retains continuity as **unestablished** across the gap. It may warn that the view may be stale, offer an explicit Refresh, or use a separately authorized reacquisition policy; it must not assert a successful subscription handoff because no more messages arrived. The ordinary refresh may legitimately happen after an advisory without providing a precise global stream-order witness; the assurance claim must name what the provider actually establishes.

Disconnect, `online`, restored `pageshow`, foreground `visibilitychange`, server restart, subscription replacement, tab suspension, live-root replacement and router failover can all create or expose a continuity gap. These triggers are **opportunities to detect or repair**, not authoritative progression evidence or unconditional permission to fetch. A lifecycle wakeup with no progression token still creates the required reconciliation/unestablished-continuity state when the selected contract demands it.

For a product selecting explicit user Refresh, a recovery gap may create visible debt without consuming an ungranted read. If a strong subscription invariant requires an immediate fence/read, application construction must either explicitly authorize that bounded bootstrap or recovery acquisition, declare a weaker live guarantee, or reject the incompatible combination. It may not quietly bypass the user's one-Refresh grant while claiming that the same grant covered two reads. Mandatory safety or privacy revocation may require separate immediate local invalidation under its security policy; that is not permission to fetch new protected data without trusted authorization.

Stream event IDs, XTDB basis values, HTTP request generations and distributed router sequence numbers serve different purposes. They are not interchangeable, and their numeric-looking forms establish no universal order or durable replay property. Every admitted identifier needs a declared source, scope, epoch/ incarnation and provider relation. Duplicate, late and reordered messages are ordinary protocol cases; the coordinator preserves independently valid debt and ignores stale **installation rights** without discarding a current, genuinely scoped advisory.

Current-state views can recover from a missed sequence of invalidations by obtaining a qualifying current observation, without replaying every intermediate write, provided the provider's read and authorization semantics establish the promised resulting state. A workflow promising delivery or execution of **every** event requires a separate durable event-processing contract: refreshing a board cannot prove that each intermediate event was consumed.

Interests themselves have semantic and physical lifetimes. A shared SSE connection may serve several fragments and survive replacement of one card, but a retired card interest cannot continue authorizing callbacks or disclosures for that target. Owner replacement and new stream registration preserve or deliberately reestablish live trigger bindings and cleanup resources by exact instance identity. A physically lingering old connection is cleanup debt, not a second semantic owner.

8. Paired Refresh and subscription-gap acceptance histories

   These are obligations for the **finished** system and its declared provider/host profile. A positive path must use an eligible authorized read and actual installed result; a nearby negative must exercise the relevant boundary without being rejected for an unrelated malformed fixture.

Eligible positive history
Bob has O1 debt, grants exactly one G1 Refresh, and A1 installs qualified Q evidence covering only O1 while O2 arrived in flight.
A1's real authorized observation is provider-proved to satisfy O1 and O2 and physically installs all required scopes.
A1 fails or installation is partial while O1/O2 remain, and Bob may later take a new explicit action.
Bob applies Q2 while Q1 is in flight; a new coordinator owner admits the current Q2 read and preserves eligible local draft state.
A current local correlated S card remains adopted while an older permitted R board membership response installs under a qualified selectiv
The baseline B is read with a qualifying interest/read fence, or a registered replay source covers W between B and subscription activation.
A disconnected tab or failed router recovers through a named durable replay or qualified current-state reacquisition.

Eligible positive history
A best-effort stream reconnect creates an explicit stale warning and awaits Bob's grant under the selected explicit policy. A shared SSE connection outlives card replacement and a new interest is registered for the current owner.

The complete HumanHelp product criterion remains independent: eligible Claim, Reassign, local search, creator-local immediate presentation, remote advisory, one granted Refresh and physical continuity must be observed through real model and browser paths. Source-level state-map equality or reject-all construction cannot satisfy those positive requirements.

Projected-plan identity, deployment, and stale continuation recovery

A browser may keep a page open while Gesso's executable plans, application operations, public-model policy, served assets, wire contracts, or trusted server deployment change. A suspended page may later be restored from browser history, the back/forward cache, a service worker, a local persisted record, or a long-running tab. The successful decoding or presentation of that old state does not authorize its continuation.

The finished application admits a resumed execution only for the **actual subject and boundary** for which required compatibility is established. A version label, shared source name, matching DOM attribute, copied plan digest or still-available JavaScript object is not itself a compatibility judgment. The implementation must preserve the difference between identifying the exact object installed, deciding whether two subjects may interoperate, and establishing the current permission for a protected act.

Parts IV–V own historical command incorporation, immutable intent, executable subject identity, model authorization and provisional presentation. Sections 06.01–06.07 establish physical owner/target rights, actual installation, advisory debt and subscription continuity. This section joins them at startup, resume, deployment and compatibility boundaries. Section 06.09 handles the subsequent retirement, obligation transfer and recovery protocol; it is not a license to keep old browser effects alive.

Compatibility is a collection of scoped judgments

An admitted application construction identifies separately:

Subject	Required question at the applicable boundary
Executable interpretation	Can this **actual loaded** browser runtime execute the exact projected-plan payload and all required bou
Local continuation	Can this particular execution state, pending effects, generations, ownership and provisional work safely
New operation invocation	Are the current generated binding, operation meaning, argument schema, route and application authori
Existing command resolution	Can a trusted resolver still interpret a previously submitted command's immutable operation/actor/argu
Compatible command repetition	Is another transport attempt for the **same** still-unresolved command admitted by its original deduplicat
Observation and rendering	May the current authenticated reader acquire the required scope and may the installed renderer/host pr
Wire and transport	Can the deployed peer decode and interpret the precise request, settlement, progression and subscriptio
Physical realization	Do the actually loaded HTMX, SSE extension, Gesso adapter, shell, browser handlers, scripts and respon
Authority and disclosure	Do current principal, tenant, Membership, protected scope, session and policy conditions permit the pro
Deployment/assembly closure	Does the actual served and loaded artifact set correspond to the admitted assembled application and the

These judgments can have different outcomes. A new deployment may refuse to resume an old browser machine while still servicing a qualified resolver for commands that the old machine submitted. A freshly loaded plan may be safe for new Claim commands while an older C42 needs a retained operation schema and readback contract. A byte-identical old script may nonetheless lack current permission to expose a previous user's data. An accepted semantic compatibility relation never confers current server authority.

Choreography representation, plan payload, semantic operation meaning, wire protocol, command identity, local execution, effect generation, assembly/release generation and deployment placement are separate coordinates. No single global monotonically increasing version integer is required. Deployment rollout identity also does not imply that all servers, cache layers, service workers and currently connected browsers are serving the same asset set. Mixed-version compatibility requires explicitly supported intersections and a defined safe behavior when a request reaches an incompatible peer.

Equality is sufficient only where the stated subject actually demands exact identity **and** that equality is established from its admitted source. Interoperation and controlled migration may use a weaker equivalence relation if it states which invariants, observation interpretations, commands and effects remain valid. Conversely a blanket `compatible=true` or successfully decoded EDN value is not such evidence. A claimed digest must name its canonicalization and equivalence domain, provenance, collision premise, and the actual bytes/implementations to which it binds. A proof/sidecar digest does not replace the mandatory runtime or authorization checks.

Resume admission precedes protected physical work

An application states where it obtains current compatibility evidence and which boundary events require revalidation: initial page startup, browser-history or bfcache restoration, tab wake, foreground transition, service-worker takeover, a long-lived socket reconnect, application/asset replacement, and server or policy change where observable. The mere receipt of `pageshow`, `online` or an SSE open event is **advisory**; it does not prove that old code is compatible with the current server or that missed changes have been replayed. Conversely a tab cannot assume that no change occurred because no such callback ran during suspension.

Before any protected subsequent physical send, DOM effect, subscription update or machine continuation, the supported realization has to possess current, applicable admission for that operation's identity, owner and effect generation. If compatibility cannot be established at the supported boundary, the protected act is blocked and the declared recovery path begins. A stale async continuation cannot acquire new rights by reading today's global ”current plan” or a successor DOM element with the same selector. A queued effect must check its current permit at the last controllable physical act, even if its origin was compatible when issued.

A realization may cache compatible judgments with a scoped lease, trusted invalidation or renewal policy instead of re-proving its entire application on every keystroke. It must qualify expiry, observation of policy changes, counterparty version selection and what happens during network isolation. A stale local lease cannot replace trusted server-side current principal, Membership, RequestAssignment or other protected policy checks. Failing to reach the server cannot automatically mean an old grant remains valid forever.

The re-admission protocol distinguishes:

1. **Known current compatible continuation:** the required interpretation, runtime resources, authority context, and progression/continuity conditions remain valid, so the exact current execution may advance.

2. **Recognizable but incompatible continuation:** old private execution and physical actuation rights are semantically retired before a fresh, admitted plan is instantiated; its durable command obligations survive under a separate recovery owner where promised.
3. **Malformed or untrusted retained state:** decoding or source admission fails. Do not reinterpret malformed bytes as a compatible plan or grant a stale callback permission; record the declared recovery/error state.
4. **Compatibility indeterminate or temporarily unavailable:** fail closed for protected old effects, preserve unresolved commitments and required display/subscription obligations, and follow bounded retry, reload, reauthorization or manual recovery rather than guessing a branch.

These classifications do not collapse `unknown command outcome` into `invalid plan` or `authoritative rejection`. The current user may recover a card or an authorized board without recovering the history of a particular old C42, and vice versa. A renderer becoming usable does not prove the earlier write incorporated.

On account logout or protected context change, the runtime removes or quarantines old locally displayed protected state and revokes old subscription/transport/owner permissions according to the selected privacy/display contract. This is a requirement on the honest application's future presentation, not a claim that already-seen information can be erased from an adversarial browser or a person's memory. Trusted handlers still authorize each read, subscription and write independently of any browser-side plan stamp.

Served assets and physical version correspondence

The installed browser execution is determined by more than the nominal plan file. It includes the **actual** fetched/executed scripts and modules, registered handler implementations and their closures, compiled build configuration, HTMX/SSE versions and extensions, served fragments, service-worker caches, startup paths, response modes, and browser host behavior on which protected effects depend.

An application may choose a content-addressed bundle, build manifest, compatible artifact set, capability negotiation, or a similarly strong scheme. Whatever scheme is chosen must relate the inspected or certified semantic artifact set to the real resources subsequently served and loaded. Merely including an assembly digest in generated HTML is not a proof that the host executed those bytes or that a stale service worker and new server agree on the protocol.

A release or rolling deployment can deliberately support more than one version. Its advertised compatibility is then a scoped matrix of allowed client/server/route/projection combinations and migration rules, rather than a vague ”both versions supported.” Old pending commands require a resolver horizon independent of how long their old browser JavaScript is allowed to execute. An offline user whose browser crosses that horizon requires a truthful unknown or manual disposition, never a fabricated terminal rejection.

When asset, profile or protocol admission fails, a current authorized reload/reconstruction is the conservative default. The application may preserve independently authorized local input or draft state through a declared migration, but it need not preserve arbitrary suspended machine locators, timers, DOM snapshots, closures or untrusted persisted permission maps. A scoped migration is admissible only with explicit semantic correspondence, identity and resource transfer rules, noninterference constraints and actual post-migration validation.

Recover command bindings without resuming stale machinery

A historical command's recovery contract retains or permits trusted lookup of at least its immutable semantic command identity, original operation and schema, actor/authority binding, canonical arguments, outcome-evidence provenance, deduplication/replay relation, retention horizon, resolver owner and allowed disclosure. Sensitive arguments may remain only server-side behind an authenticated opaque lookup reference; persisting them in browser storage is not required. All client-persisted records are untrusted on return.

C42 is not a bearer credential and its ID is not sufficient to establish the original command's actor, inputs or permitted result. Changing its arguments or applying new operation meaning while retaining the ID is not compatible replay. Assigning a new ID to escape an indeterminate result can cause duplicate incorporation. Where the original write may already have occurred, a qualified read-only resolver or safe deduplicated repetition is used under Part IV; each path retains the current server's authorization and disclosure checks.

The recovery contract separates **survival of semantic obligations** from the **retirement of private physical rights**. An incompatible E42 loses timer, provisional rollback, target and transport callbacks. An admitted recovery owner may resolve C42 and acquire a new current Request card under its own rights. It cannot execute E42's old pending DOM effect, reuse its physical request instance, or pretend that a visible card is sufficient historical attribution. Section 06.09 specifies the exact transfer and durable recovery lifecycle; this subsection specifies what must be compatible before the transfer is attempted.

Paired compatibility and resume acceptance histories

Each positive witness must exercise the actual selected runtime and trusted boundary where the guarantee is claimed; each adverse case must reach the intended incompatibility boundary rather than fail solely because of a malformed, unrelated fixture.

Eligible positive history
A suspended P7 execution resumes on a peer proved compatible with its exact plan, host and current authority-context contract.
A P7 command C77 was sent before deployment D8. Its admitted historical resolver remains available while P7's browser execution is retir
An offline browser restores from bfcache and obtains current compatible executable, resource, authority and subscription admissions befor
A rolling deployment declares and tests compatible old/new client-server protocol combinations, including loaded HTMX and registered ef
A principal/Organization change re-admits any allowed current data under the new context and retires old protected subscriptions.
Compatible command recovery is available but current result disclosure is denied; the resolver returns the declared nondisclosing status.
An application explicitly proves a specific supported retained-draft migration across P7⟶P8 and validates its new binding.
A suspended client cannot reach a compatibility authority, so it preserves unresolved C42 and enters bounded blocked/recovery state.

The finished design does not require the application author to manually write an external proof language or reconstruct all the internal artifact relationships. Gesso's application assembly and assurance pipeline generates the needed correspondence obligations from the declared operations, plans and deployment profiles. Its guarantees must still name the trust premises, actual installed subjects and boundaries of any opaque Clojure or browser behavior.

Retirement transfers surviving obligations before cleanup

A stale or incompatible browser execution must lose its private semantic rights before any of its physical resources can affect the current view or operation. Semantic retirement and **survival of unresolved obligations** are separate properties: revoking E7's timer, send, rollback and target authority does not establish that C77 was rejected, that a server transaction stopped, or that the application may forget how to resolve a possibly incorporated command.

The retiring owner and a current recovery owner participate in one declared handoff contract. For obligations promised to survive, the handoff must establish both their continued ownership and the old execution's revocation. An implementation may establish the durable recovery record before revocation or use an admitted atomic handoff. It may not first erase the only recoverable command binding, then claim that a later best-effort callback will recreate it. If an atomic handoff cannot be performed, the runtime fails closed on old private effects, retains or reconstructs the obligation from a separately qualified durable source where available, and exposes any unresolved recovery failure. It cannot keep an unsafe old execution running simply to wait for successful physical cleanup.

Subject	What retirement changes
E7 local execution and projected machine	Revokes machine progression, private send, rollback, timer, target and restoration rights.
H1 physical HTTP attempt and old timers	Requests best-effort abort/cancellation and rejects callbacks under the retired owner.
C77 immutable command intent	Does not rewrite C77's operation, actor, arguments or historical evidence.
Current authoritative entity/collection	Does not change trusted model state.
Continuity and provisional presentation	Expires obsolete restore/rollback permits and old target ownership.
Live and fragment coordination	Revokes retired private interest/target rights, retaining any owned shared connection accordir
Deployment/runtime identity	Refuses to continue unsupported machine or effect implementations.

Logical retirement must precede or be committed independently of physical cleanup. The adapter invalidates execution/resource generations and exact target ownership; the shell attempts cancellation and release of its **captured** physical handles afterward. A failed abort, late XHR response or thrown cleanup hook is physical debt, not a resurrection of E7. A later callback may be admitted only as cleanup evidence, or under a separate surviving owner with independently justified semantics. Its old execution-private rights never transfer by implication.

This ordering also constrains recovery failure. If the new page or resolver cannot be loaded, E7 remains retired. A handler that throws, a storage quota failure, an unavailable browser location, or a failed timer cancellation must produce a typed visible/diagnostic state at the applicable boundary, not re-enable old effects or interpret the last command as rejected. A newer owner may begin safe independent work only while runtime-wide resource and isolation premises remain satisfied.

Durable command recovery has an explicit failure domain

The assembly declares which interruptions its recovery promise actually survives: a local execution replacement, page reload, tab process loss, device restart, offline interval, deployment rollover, account change, backend failover, or retention expiry are different failure domains. A recovery promise across page destruction cannot rely solely on an in-memory JavaScript closure or a pending Promise. It requires an appropriate durable client record, trusted server discovery/intent ledger, durable command-idempotency record or another specified surviving mechanism. These mechanisms have different privacy, availability and retention properties.

At the command boundary, the application must preserve or permit trusted reconstruction of:

- the exact C77 semantic command identity and original operation/schema;
- authenticated actor/Organization context and immutable arguments or a protected resolver reference sufficient to check the original binding;
- the authoritative write/deduplication semantics under which another attempt could be compatible, including the possible-incorporation interval;
- existing qualified settlement, retry and readback evidence with its source, scope and temporal interpretation;
- a designated current recovery owner and trusted resolver entrypoint;
- which retention/expiry conditions permit a strong answer, leave uncertainty, or require manual reconciliation;
- present disclosure authority and the current authorized projection, which are re-established independently of the original actor.

A semantic command ID alone is not a credential, an original binding, or proof that the operation was accepted. Persisted browser bytes remain attacker-controllable input. They must not carry unconditional read permission or override the current principal, Membership, tenant or transaction rules. Sensitive canonical arguments may remain server-side; a browser may retain only an opaque lookup key whose use is authenticated and appropriately scoped.

If retention is bounded, the framework must specify what evidence survives expiry and whether the original authoritative effect may still be pending. An expired deduplication window cannot silently be treated as permission to repeat an unknown non-idempotent operation. A missing resolver record is not necessarily a terminal rejected command: it can mean `:commit-unknown`, denied disclosure, unsupported old schema, expired history, or an actually proven non-incorporation, which have distinct consequences. A promise of eventual automatic resolution requires realistic fairness, provider availability, retention and scheduling premises; otherwise a truthful blocked/manual state is valid.

A selected plan may be retired while the recovery ledger remains useful for multiple later deployments. The current plan's operation interface does not automatically define the old command's meaning. A surviving resolver must understand the exact original command and its committed transaction/evidence scope; compatible replay requires a named authoritative deduplication/repetition contract. A new command ID cannot be allocated just to evade an unknown C77 result and produce a duplicate Order or assignment.

Worked example: a sleeping checkout crosses deployment

An authorized browser running executable plan P7 prepares and submits the `place-intent!` operation with semantic command C77 through one physical attempt H1. The trusted transaction may have created Order O900 in `payment-pending` state and

atomically admitted durable payment work. That is not the same as asserting that the later payment completed. The network response is lost. The browser knows only that the historical C77 disposition is unresolved, while E7 may still hold a provisional ”submitting” presentation and a pending retry timer.

Deployment D8 makes P7's continuation incompatible with the installed plan P8. Alice's tab wakes after a long suspension. The supported runtime checks current plan/host compatibility and present authority context before allowing another protected send, timer continuation or DOM act. An `online` or `pageshow` callback is a wakeup opportunity, not compatibility or authorization evidence.

The default safe sequence is:

```
recoverable C77 binding/lookup is already durable,
or its survival is established by an admitted handoff
        |
P7 continuation found incompatible
        |
retire E7 and revoke H1/timer/rollback/target rights
        |
attempt best-effort physical cleanup
        |                  (physical failure may remain debt)
admit current P8/runtime + current principal/authority
        |
resolve historical C77 under its old operation contract
        |                 separately
acquire authorized current Order 0900 observation
        |
install the qualified current presentation
        |
continue durable payment/workflow monitoring independently
```

The handoff of recovery obligations and retirement of unsafe local rights form a safe semantic transition; physical cleanup need not finish before authoritative recovery can proceed. The diagram's steps need not impose an artificial total order between independent readback and current-state acquisition. A current Order may be known when C77 attribution remains unresolved, or C77 may be historically incorporated before a current Order card can be read or installed. Each result retains its own owner/source evidence.

The trusted resolver identifies the original operation/actor and deduplication context, checks the current requester's permission to learn the result, and returns only the proposition its evidence supports. For illustration:

Trusted evidence and current read	Allowed presentation/recovery
C77 is qualified as incorporated; an authorized current read shows O900 still `payment-pending`.	Show current pending Order and separat
C77 is incorporated and later independently qualified payment completion is observed.	Show the authorized completed state; the
The old command was authoritatively rejected and disclosure is currently permitted.	Present its qualified rejection and the adn
An authorized read finds O900, but no qualified causal attribution to C77 exists.	Display O900 if permitted while preservi
Neither incorporation nor rejection is established after H1's lost response.	Retain `:commit-unknown` and the declared
The principal changed or disclosure is denied.	Quarantine old protected display and give
The original operation resolver/schema is unavailable or its retention horizon elapsed.	Block incompatible repetition, report the
P8 cannot be loaded or current resources/transport are temporarily unavailable.	Leave E7 retired; use a bounded reload/re

This example is deliberately about a durable business process. A payment-pending Order can advance while the browser is closed because the trusted workflow owns it; an old browser timer does not. The same principle applies to HumanHelp's Claim: a protected RequestAssignment may have incorporated even if the browser lost the response and its old provisional presentation was retired. A subsequent manager Reassign does not retroactively reject the original Claim. Trusted historical command resolution and permitted current card acquisition remain distinct.

No part of this example creates a universal runtime `order-model/resolve-command` function or a concrete public Clojure API by implication. Actual operation admission, resolver schema, durable payment progression and evidence must be supplied by the declared application model and the assembled Gesso interface.

Recovery is bounded, observable, and non-privileged

A branch described as recoverable must state its recovery owner, required inputs/evidence, current authorizations, executable next actions, retry/expiry policy, progress conditions and terminal or manual alternatives. It cannot be satisfied merely by labeling a catch block `:recoverable`. Recovery must have a real path for eligible positive users as well as explicit failure states.

The policy covers incompatible executable assets or protocol schemas, failed authenticated readback, revoked Membership, subscription/interest gaps, missing or stale target instances, partially completed DOM effects, exhausted retention, browser storage loss, server-side resolver unavailability, and network partitions. Each branch preserves the distinction between:

- physical resource cleanup and logical execution retirement;
- a historical command result and the currently acquired entity;
- acquired data and physically adopted representation;
- a still-pending durable business workflow and its prior admission/command result;
- uncertain command delivery and proven terminal exclusion;
- a current recovery plan and an obsolete suspended machine;
- permission to initiate recovery acquisition and permission to disclose its protected result.

Restart/reload, readback, reacquisition and reinstallation do not automatically erase outstanding Live interest/Refresh debt. An initial-load or reconnect gap must be covered by the applicable 06.06–06.07 source/fence/admission contract. A fresh initial authoritative read may be separately permitted as a new-page bootstrap under its declared product policy; it may not be billed invisibly against an exhausted prior one-Refresh grant.

Failure to acquire a current artifact or observation may stop safely in `:blocked`, `:unknown`, `:read-only`, or `:manual-reconciliation` with an explicit reason and next admitted action. Automatic retries require bounded budgets, backoff, cancellation and fairness assumptions. A failed reload cannot simply schedule another unbounded reload, and expiry of a retry timer cannot reauthorize the stale execution. If the resource budget for lingering physical effects is exhausted, the runtime may require a stronger reset while retaining truth about unresolved commands.

Current-state views need not replay every missed UI event after a long offline interval when a qualified current observation establishes the promised state. Systems that promise durable per-event processing must separately retain and process those event obligations. The browser cannot manufacture completed payment, Claim attribution, or an entire event history from one freshly rendered card.

Paired retirement and recovery acceptance histories

Each positive witness must exercise an eligible real application path and the relevant current trusted authority/physical boundary. An adverse case must reach the intended failure/recovery point rather than being rejected for an unrelated malformed fixture.

Eligible positive history
C77's immutable binding survives before or atomically with E7's incompatible-plan retirement; its resolver remains available under a declar
E7 is semantically retired, its timer/transport/target rights revoked, and physical cleanup succeeds.
A resolver proves C77 created pending Order O900; another qualified source later proves payment completion.
P8's current authorized Order read succeeds while historical C77 attribution remains unknown.
A compatible replay of C77 is admitted through its original deduplication/operation schema during the retention horizon.
Alice's tab wakes as Bob; the current trusted resolver denies disclosure of Alice's old protected command.
A full reload reconstructs current plan and an authorized board/read without an ungranted second remote Refresh.
A resolver is unavailable and an authorized user follows bounded retry or manual reconciliation.
A supported target migration preserves permitted local draft state and rebinds its current owner.

Durable command storage, server idempotency, plan retention and recovery UI are specified by the supported failure-domain and operation contracts. The assembly and certification evidence must establish their actual realization or explicitly reject a profile lacking required recovery.

Browser managed effects

A managed browser effect is physical work whose ownership, allowed footprint, completion, failure, retention and retirement are part of the admitted application's semantic contract. It may start a timer, register a listener, invoke transport, install a provisional or authorized representation, capture continuity, persist a permitted local value or release a resource. The shell implements the physical work; it does not independently decide authoritative business results, Choreo branches, command attribution or which superseded execution may regain presentation ownership.

The finished framework does not need to own every unrelated browser resource. It must, however, close **every actual mutation and callback path** that can affect a protected managed scope. An opaque application effect or third-party library is admitted under a declared contract and noninterference premise, not excluded from the guarantee by simply calling it ”unmanaged”. Server-side authorization and physical browser effect admission remain independent obligations.

Effect kinds have multiple independent properties

Resource, reversible, authoritative, asynchronous and ephemeral are useful descriptions, not mutually exclusive state-machine cases. For each admitted effect, the assembled application records the independently applicable dimensions:

Property	Required distinction
Kind and permitted footprint	Timer, listener, transport, provisional/authorized DOM target, continuity slot, local storage, subscription in
Semantic purpose	Execution progression, provisional presentation, authoritative installation, read/advisory coordination, com
Ownership	Exact runtime, root/child effect scope, owner generation and resource instance; shared ownership is an exp
Issue and completion	Issued, physically committed, completed, failed, partly realized, unknown, or cancelled under an effect-spec
Timing and reentrancy	Synchronous host gate, immediate callback, Promise, deferred callback, host reentry and any required final
Repetition	Repeatable read or cleanup, once-per-physical-resource mutation, command-bound transport retry, or nonr
Revocation and release	Semantic removal of rights, best-effort physical cancellation, verified release, retained remnant and transfe
Compensation	No compensation, legal local restoration, declared repair, or a distinct authorized server command; no univ
Evidence and assumptions	Proven adapter consequence, tested physical handler, external host/HTMX premise, or as-yet-unproved opa

One local DOM installation can be synchronous, reversible only until a stronger authoritative projection arrives, and capable of allocating listeners whose lifetime exceeds the installation step. A timer can be an asynchronous callback source and a resource requiring release. The actual authoritative observation remains an authority-source fact; the browser's physical installation is not a new trusted model mutation.

One root effect scope per execution, with explicit owned children

An active local execution has one admitted root effect scope, with child scopes for resources such as provisional presentation, its restoration permit, settlement timer, transport attempts, continuity capture and local asynchronous work. Fragment coordinators, shared SSE connections and durable command-recovery owners have separately declared root scopes. All current managed effects must be reachable through their admitted owner graph. A physical registry is a mapping from semantic issued resource identities to opaque host handles, not a second mutable model of which execution ought to win.

```
execution E42 / generation G7
  → provisional card target B1 / install effect F1
  → settlement timer T1 / issued resource R1
  → physical attempt H1 / issued transport F2
  → local continuity slot L1 / physical capture F3

board coordinator B3 / distinct owner
  → Live interest S9 / shared connection with explicit references
  → applied query Q / advisory and adoption obligations

command recovery owner K8 / distinct lifetime
  → unresolved C42 historical attribution and resolver obligations
```

This ownership graph is illustrative, not a second runtime API. Sharing the command ID C42 does not make an old execution's timer or card current. A coordinator may continue after E42 retires, and K8 may retain C42 while E42's provisional resources are revoked. A current subscription invalidation is admitted by its own registered source, not by E42's private callback.

Each physical resource identity includes an issuing runtime, owner, generation and concrete incarnation sufficient to distinguish recycled timer IDs, DOM selectors, register slots, transport IDs and reused handle values. Equality of a CSS selector, numeric timer ID or current style value does not prove ownership. An alias between identities is permitted only under an explicit same-resource contract. Resource ownership transfer is a semantic transition with old rights revoked and a current receiving owner established; it is not two uncoordinated registry writes that leave a gap or dual owners.

A shared connection has an explicit reference/interest lifecycle. Detaching one fragment interest must not close a connection still required by another. Duplicate attachment or release must not grant a new interest or decrement another owner's reference. Reference counting can realize the policy but is not itself evidence of correctly admitted subscription scopes or protected server disclosure.

Issue registration precedes host work; late handles remain owned

The adapter records an **issued effect and resource instance** in committed semantic state before calling the physical handler. An immediate completion may then be correlated with the issued work, and a nested event sees the newly committed owner. That semantic order does not by itself close the physical-handle race.

Some host APIs return a cancellation handle only after initiating work. An injected scheduler may invoke its callback synchronously; a registered listener may trigger a callback during registration; a Promise may settle while the outer effect batch still holds other work; retirement may occur before a host call finally returns a handle. The physical registry therefore implements a protocol equivalent to:

Physical state	Required treatment
Issued, awaiting handle	The semantic owner and issue instance are recorded before host invocation; callback admission is a
Callback/completion before handle	Completion or retirement is correlated to the same issue without inventing a second live resource c
Handle returned for current issue	The opaque handle is registered only if the issue and owner are still current and the resource remai
Handle returned after retirement	The handle is never published as current; release/cancel it through the retired issue's exact cleanup
Issue throws after partial registration	Record the actual uncertain physical footprint, attempt permitted cleanup, and preserve the origina
Duplicate/late completion	Apply the declared duplicate/stale admission disposition; do not settle a successor resource.
Release completes or fails	Mark only this resource instance released or report retained physical remnant debt; no fabricated s

A late host return is not allowed to recreate a resource that a synchronous callback or nested dispatch already completed. Checking only whether the registry key exists **before** invoking the scheduler, then unconditionally associating the returned handle afterward, fails this contract. The shell may use an issuance cell, retired tombstone, effect-specific latch or another bounded mechanism with an equivalent checked transition. Tombstone retention must have explicit generation/reuse and resource bounds; retaining all old handles forever is not the intended solution.

Registration also precedes every irreversible protected **act**, with a still-current permit checked at the last controllable mutation boundary. An outer dispatch's effect list is not a perpetual license: an immediate handler callback can retire its owner before a later listed effect runs. A deferred effect must not write to a newer target solely because its completion event would later be rejected as stale. A physical action that crossed its declared commitment point may be partial or irreversible and requires actual completion/failure/repair evidence rather than fictional rollback.

Local once-only physical actuation is distinct from remote command incorporation. H1 and H2 can be different HTTP attempts for the same immutable C42 under the admitted deduplication/repetition contract. ”Do not run this DOM effect twice” is not evidence that the server received exactly one request or that C42 incorporated exactly once.

Semantic retirement precedes best-effort physical cleanup

Retirement first revokes the execution's semantic progression, callback, rollback, target and pending-send rights. The host then attempts abort, timer clear, listener detach, temporary-node removal and continuity/resource release through the appropriate exact physical handles. A failed cancellation does not restore the old owner's rights, and a late callback cannot mutate the newer owner simply because it carries a stronger-looking observation or has the same physical selector.

A cleanup operation is idempotent **for the same issued resource instance** and safe in the presence of repeated retirement, timer/settlement races, failed aborts and nested dispatch. Cleanup order follows declared resource dependencies, normally reversing acquisition order where that matters. Independent cleanup is best effort: if the first release throws, other permitted releases must still be attempted. Cleanup failures are separately recorded against the original owner and resource.

Deleting a temporary DOM node by re-querying today's ID is unsafe if that ID now belongs to a successor. Resetting an applied CSS property only when its current value matches the old value is also insufficient: a newer owner can intentionally write the **same** value. Resource release must compare ownership/incarnation or use an exclusive physical handle/lease capable of distinguishing the writers. If a host API cannot provide that relationship, the application must explicitly limit the guarantee, serialize ownership or choose another safe realization; coincidentally equal values are not evidence of unchanged ownership.

Physical-remnant records describe attempted, completed, failed and indeterminate release separately from semantic retirement. The system must specify finite or bounded retention, instrumentation, backpressure, compaction and a safe degraded/unavailable state if physical remnants accumulate. Logical single-flight does not bound lingering XHRs, timers or native listeners after abort failure. An exhausted resource budget may stop new work without granting old callbacks permission or erasing unresolved command and subscription debts.

The runtime may recover current authority and install a new target after semantic retirement, without waiting indefinitely for the last physical cleanup. That independence depends on the still-retained old effects being unable to act on or interfere with the new scope under the declared host noninterference premises.

Restoration and continuity have narrow, expiring permits

A local provisional ”undo” is a scoped, guarded physical action. It may restore particular previously captured presentation only while the same current owner retains the applicable target and no stronger admitted authoritative or competing presentation has superseded it. A rollback cannot reverse server incorporation, reassign a RequestAssignment or overwrite a newly installed canonical card. Expiration occurs on owner retirement, target replacement, admitted authoritative installation, conflict-policy revocation or another specified semantic event.

Continuity captures are opaque physical resources whose implementation and restore contracts are bound at capture. A changed registry of restore functions cannot reinterpret a stale capture as an authorized new operation. The supported capture may preserve focus, input draft, caret, scroll, details-open state, or other declared local state. It must not promote obsolete server data, a previous applied query, old operation bindings, privileged HTML snapshots or expired disclosure rights to authoritative state.

Continuity preparation required to protect a mandatory user draft must run before the protected swap is allowed. Optional restoration, after the authoritative installation has completed, is a separate best-effort local effect: a failed scroll/focus restore cannot retroactively reject the command or undo the current observation. Delayed animation-frame callbacks must recheck that their capture and target owner remain current before touching DOM. A resource's physical released latch can prevent an old callback from acting, but does not replace the semantic owner and target-generation check.

A provisional presentation timeout and a command-history resolver deadline may have different owners. Settlement or authoritative supersession must immediately revoke the former's ability to roll back, while the latter may persist after the local execution retires. Blindly cancelling **every** timer with the same command ID would lose a required unresolved-command recovery obligation. Conversely retaining the presentation timer because the command is still unresolved would risk stale physical mutation.

Fault containment preserves original causality and honest uncertainty

Failure before a pure semantic transition is committed and failure after physical actuation begins have different consequences. If an adapter step fails before commitment, its emitted effects have not yet been interpreted, and the last valid committed state is the relevant diagnostic reference. The declared failure path retires unsafe owner rights rather than inventing a successful step or blindly rewinding an already committed successor.

If a physical handler throws or returns an indeterminate result after some actions have taken place, the runtime preserves the committed semantic transition and records actual attempted, completed, partial or unknown effects. It revokes unsafe rights, continues permitted independent cleanup and repair, and retains the original failure/command uncertainty. It must not blindly reissue an entire effect batch, restore a superseded DOM snapshot or translate a network failure into an authoritative `:rejected` outcome.

The primary failure F and cleanup failures CA/CC have different provenance and ordering:

```
primary failure F
   → semantic owner retired
   → cleanup resource A: failed CA
   → cleanup resource B: released
   → cleanup resource C: failed CC

report primary F with subordinate cleanup evidence [CA CC]
retained resource debt for A and C; no invented release
```

Reporting, observer callbacks and diagnostics are not allowed to select a different semantic successor. Optional observers require bounded, declared failure isolation; a throwing/reentrant or unbounded same-process diagnostic can still interfere unless the implementation supplies that protection. A corrupt runtime-wide physical registry may require retiring the entire runtime rather than assuming each child owner remains isolated. Independent owners may proceed only while their stated shared-resource and noninterference contracts remain true.

Recovery keeps actual historical command incorporation, local provisional presentation, authorized current observation and completed DOM adoption as distinct evidence subjects. A failed physical cleanup does not undo C42's server Claim; a failed DOM installation does not prove that C42 was rejected; an expired recovery window does not prove the original command never ran.

Paired managed-effect and cleanup acceptance histories

The finished system must provide eligible positive cases that exercise real admitted host effects and nearby adverse cases that reach the relevant physical lifecycle. A fabricated success map, disabled operation or purely symbolic timer is not by itself a conformance witness for the deployed browser.

Eligible positive history
E42 issues T1 with a recorded effect instance; a synchronous test scheduler fires before returning its handle and T1 retires in the nested dis
E42 starts provisional B1; its callback reenters and lets a newer E43 own B2 before the outer effect batch finishes.
An exact old timer/resource handle is cancelled or released twice safely while the successor timer shares a label or registry slot.
A continuity owner installs a height lock and releases it while no successor owns that style.
A newer authoritative card installs while E42's provisional rollback is semantically retired and a separate C42 resolver remains active.
One shared SSE connection has two admitted current fragment interests; one owner retires.
A mandatory effect fails after partial host work; the runtime records its stage, retires the unsafe owner and attempts remaining cleanup.
A current target's admitted swap finishes, then optional scroll restoration fails or a stale animation-frame callback arrives.
Several failed native aborts leave physical remnants while logical E42 work is retired.

Part VII must account for the **actual** registered handlers, rendered routes, third-party writers, startup paths and served assets. Part VIII requires positive HumanHelp Claim/Reassign and one-Refresh behavior through those physical paths. Parts IX–X distinguish finite host tests, proof-kernel claims, source correspondence and the real browser/transport assumptions.

Async event safety

Every asynchronous event that can advance browser semantic state or cause a protected physical effect passes the declared normalization and owner-relative admission boundary. Timers, HTMX callbacks, SSE invalidations, Promise and worker completions, cross-tab messages, deferred continuity work, command resolver results, browser lifecycle events and delayed DOM effects are all subject to this rule. No callback may independently choose which execution ”probably” owns the event or resume a Choreo machine outside the admitted adapter.

An admitted event is a proposal for one semantic transition. It is not a server credential, a trusted domain fact or permission to perform all subsequent physical effects. Its **registered origin**, event class, payload, owner, current resource incarnation, legality and claimed evidence are separately established. A permitted semantic transition may issue physical effects; those effects still require current actuation permits under 06.02, 06.05 and 06.10.

The conceptual gate is common across event types, but its requirements depend on **owner class**. Requiring E42 on every event rejects genuinely shared work; accepting all events without an execution ID allows private callbacks to evade retirement. The general rule remains Part II's owner-relative event admission.

The event class follows the registered source

The assembled application's admitted source registration establishes which event kinds an actual physical callback may deliver, the allowed owner class and scope, the required correlation, accepted schema and resource lifetime. Event metadata supplied in the payload does not select a more permissive class. An old transport callback cannot become current shared command evidence merely by adding `:event/class :command-evidence` to its body.

Registered source class	Admitted owner and purpose
Execution-private	A current execution and issued local/send, timer, transport or provisional effect generation.
Fragment coordinator	A current applied query, managed root and physical HTMX request/target generation for acquisition and installa
Shared subscription	A current subscription/interest resource and authorized scope, independent of the active local command.
Command recovery	A surviving resolver owner and qualified C42 historical command-result binding, possibly after E42 retirement.
Runtime lifecycle	A registered `online`, `pageshow`, visibility, reconnect or deployment wakeup requesting further checks, not claimin
Optional observer	A bounded, nonauthoritative inspection of an actually selected transition/effect, without permission to forge sem

A source registration has a runtime/owner incarnation, physical callback identity, permitted event types, schema, effect or resource instance, subscription/authority scope and retirement contract. It must be established by the actual integration, not reconstructed from today's DOM selector, mutable global or client-reported source label.

A registered SSE listener can establish that a message entered through a particular local subscription path. It does not by itself prove that the message's claimed XTDB basis or RequestAssignment facts were issued by the trusted authority. EDN decoding, equality of duplicated wire fields and correct DOM listener ancestry are structural/source checks, not universal provider provenance. Worker and cross-tab APIs likewise identify transport channels without making arbitrary message contents authoritative. Server and model policy remains independently enforced against hostile browser traffic.

A schematic normalized inspection event could be:

```
{:event :transport/succeeded
 :source-registration/ref registered-transport-source
 :runtime-instance/id runtime-instance
 :owner/ref [:execution E42]
 :resource-instance/id issued-resource-instance
 :execution-generation current-generation
 :effect-instance/id pending-send-effect
 :transport-request-id H1
 :gesso/command-id C42
 :payload decoded-result}
```

This is not a claim that the current adapter exposes those exact fields. Some identities may be carried in trusted local registrations rather than serialized in the event. Constructing a matching map is not admission. An actual source/owner binding and the correct semantic evidence must exist independently.

Admission checks owner, issued work, legal state and evidence

For a declared source class, the gate establishes all relevant conditions:

1. The physical callback is associated with a current source registration that permits the claimed event kind, scope and host lifecycle.
2. Its runtime, owner, resource instance and generation are current for that kind of work; reused labels, selectors and counters do not establish instance identity.
3. Where relevant, correlation matches a real issued local effect, H1 transport attempt, timer, subscription interest, fragment request or command resolver. A shared invalidation need not name E42.
4. The payload obeys the source-bound schema; evidence of command outcome, current observation, progression, absence or physical completion has the **required independent provenance**.
5. The current semantic machine/coordinator/resolver state legally accepts the event. A completion cannot settle an unissued effect; a timeout cannot reverse an already resolved presentation branch.
6. Duplicate/replay rules apply to the **correct layer** of identity and bind any repeated ID to its source, owner and equivalent payload.
7. Any subsequent protected transport, disclosure or DOM mutation obtains its own current permit. Event admission does not revive retired physical rights.

A single logical dispatcher commits its selected semantic state before interpreting effects. An immediate nested callback therefore sees the new owner and effect issuance records; the outer handler cannot overwrite the nested successor state on return. Where multiple host tasks can dispatch against one owner, their commits require real serialization. Event arrival order does not establish authoritative progression or server causality.

Diagnosis	Meaning and required response
Unknown source/correlation	No admitted issued work matches; never guess the newest execution.
Malformed payload	Declared schema or source-bound identity is violated; fail the required physical gate closed.
Stale owner/resource	Old generation/instance has retired; no old callback or physical mutation rights.
Duplicate delivery	Apply the class's admitted idempotence/consumption rule; do not rerun one-shot effects.
Conflicting duplicate	Same ID has incompatible binding/payload; classify and reconcile as a protocol/evidence fault.
Illegal current state	Source is known but its event cannot advance this machine or coordinator state.
Incompatible plan/context	Suspend/retire old private work under the explicit recovery/compatibility contract.
Unestablished authority	Source does not justify claimed command incorporation, current scope, basis, adoption or disclosure.
Physical failure or uncertainty	Preserve partial, failed or unknown effect results; do not fabricate domain rejection.

These are browser-level classifications, **not** new public-model outcomes. In particular, `:committed`, `:rejected`, `:already-incorporated` and `:commit-unknown` require admitted authoritative operation evidence. Expected stale callbacks may produce bounded diagnostics without changing state; conflicting current evidence or a broken invariant needs its declared typed fault and recovery path, not universal silent ignoring.

For a synchronous HTMX gate, a typed error is not enough if the physical request/swap still proceeds. A rejected protected action must be stopped at an actually cancellable supported boundary. A generic `afterSwap` callback cannot certify an unexamined OOB, extension or successor-target mutation.

An early physical delivery can be consumed before it is legal

Physical delivery identity is distinct from command identity and machine reception readiness. Some protocols consume a delivery ID when that source **first observes it**, even if the current projected machine cannot yet accept its envelope. If the ID remained unspent, the same early message could be replayed once the machine reached the appropriate receive state and become incorrectly fresh.

For such a protocol, E42 receives physical message M1 when its machine does not accept that envelope. Admission records M1 as consumed under its source and execution generation, but does not advance the machine. When E42 later becomes ready to receive, the same physical M1 is still a duplicate. That consumption is a negative capability preventing a later unauthorized replay, not proof that an earlier command incorporated or that some new trusted resolver result must also be ignored.

The exact consumption point is defined by event class. A one-shot timer may be spent upon firing. A shared advisory may be coalesced under qualified scope/coverage, while a trusted historical command result can be delivered repeatedly under an immutable authenticated resolver contract. The application must not use one global ”deduplicate by C42” rule across transport attempts, physical messages, command readbacks and current observations.

An observed-but-unaccepted delivery cannot be resurrected by replaying its ID after that machine's state changes. A changed runtime/resource incarnation or explicitly justified durable replay fence can permit bounded compaction without allowing an old callback to masquerade as newly issued work.

Independent owners can accept useful late information

Suppose E42 submitted HumanHelp Claim C42, but its provisional presentation and execution-private resources have retired. The current board subscription S9 is owned by coordinator B3. Surviving command resolver K8 retains an unresolved C42 obligation. E43 may later make a separately admitted compatible C42 retry with its own H2 attempt.

Arriving event	Required treatment
E42's old settlement timeout	Stale private resource; cannot reject C42 or restore an old card.
E42's deferred provisional callback	Expired target/rollback permit; cannot write to B2 even if it finds the same selector.
Current S9 advisory	B3 may record scoped debt under its interest and Refresh policy without an E42 command.
Retired predecessor S8 advisory	Stale resource/interest; matching fragment names cannot grant S9's rights.
K8's qualified C42 resolver response	May establish historical command incorporation/rejection/unknown according to authentica
Old E42 transport callback relabelled shared	Still from the retired private source; no second-chance shared admission.
H1 callback while E43 owns H2	H1 cannot settle H2; common C42 does not merge transport-effect generations.
Browser reconnect wakeup	May establish a continuity-gap obligation, not that every missed invalidation was replayed.

This rule permits **separate qualified late evidence**, not a loophole to adopt an obsolete physical continuation. A current Request card may install while C42 attribution remains unknown; C42 historical incorporation may become known without proving the current RequestAssignment helper or card adoption. Part IV owns the trusted business result and Parts V/VI own local presentation and physical realization.

Duplicate, conflicting and missing evidence have different lifetimes

A physical transport delivery, semantic C42 command, trusted result, authorized observation and actual target installation answer different questions. A duplicate at one layer is not automatically a duplicate at the others. If a stable delivery or result ID is repeated, the adapter checks the same admitted source, operation/argument binding, scope and equivalent payload. Conflicting payloads under the same ID require a typed evidence or protocol fault; ”last message wins” is not progression.

Known historical incorporation does not become unknown because a later H2 transport attempt times out. A success-looking HTTP response likewise cannot establish incorporation on its own. A newer authorized Request/card observation may supersede presentation within its qualified scope without retroactively rewriting previously valid historical command evidence.

The runtime declares consumed-message, duplicate-result and retired-resource retention or compaction. Unbounded tombstones are not required, but deleting an old record while permitting its callback to collide with a newly allocated incarnation is unsafe. Epochs, fresh resource identities, bounded replay horizons and qualified provider coverage can establish the nonconfusion premise.

Shared invalidations may be duplicated, delayed or reordered. The fragment coordinator preserves incomparable obligations until an actual provider-covered acquisition or adoption stage discharges them. A repeated advisory does not authorize an extra one-Refresh read. Conversely stream silence does not prove that an interest-registration or router-failover gap was covered; the 06.06–06.07 fence/recovery contract still applies. Liveness needs stated fairness, availability and retention premises; stale-event safety alone does not prove eventual convergence.

Fault handling, timeout retirement and adversarial scheduling

If a pure semantic step fails before state commitment, the runtime must not manufacture effects it never issued or restore an old machine and pretend the transition succeeded. It retains the last committed state for diagnosis and retires the unsafe owner under its fault contract. If an effect already started, actual partial physical changes and unresolved remote incorporation require independent recording, compensation or readback. An entire previously issued batch is not blindly rerun after a host exception.

Winning settlement or authoritative supersession revokes competing **presentation timeout authority** before slower continuity or DOM follow-up. Physical timer cancellation may fail; the callback is still stale. A different command-resolution deadline may survive under its own recovery owner. Cleanup errors remain subordinate to the original semantic or host failure.

Tests and sound abstract models explore source/owner relations as well as adversarial ordering: response versus timeout, early message versus later receive, H1 completion versus H2 retry, authoritative installation versus provisional rollback, shared advisory while no command runs, reconnect versus retired subscription, duplicate/conflicting result, nested callback before remaining physical effects, new principal context versus old source, and invalidation during one-Refresh acquisition. Checking a handful of particular generation integers does not prove the required equivalence classes.

Paired asynchronous-admission acceptance histories

Each positive witness must execute an eligible admitted source and legal machine path; the nearby adverse case must reach the relevant source, owner, trust or physical boundary rather than be rejected for an unrelated malformed fixture.

Eligible positive history	Adverse or incomplete history
E42 receives completion for an actually issued current local effect generation.	The same completion belongs to a r
M1 is observed early, consumed and leaves the projected machine waiting.	The identical M1 is replayed after th
E42 accepts qualified settlement and revokes its presentation timeout.	A duplicate result ID carries conflict
K8 resolves C42 and B3/S9 accepts a legitimate scoped board advisory after E42 retires.	E42's old transport relabels itself sha
H1's correlated managed HTMX request completes its admitted specific target effect.	Stale H1 callback or unexamined OO
H2 retries C42 with compatible immutable command binding and receives trusted resolver evidence.	H1 completion settles H2 or H1 time
O1 and O2 arrive on a current shared interest during one granted A1 Refresh.	Duplicates erase incomparable O2 o
A lifecycle wakeup detects a subscription gap and follows an admitted recovery policy.	`online` or `sseOpen` is treated as proo
An immediate physical callback reenters after semantic state commit and retires a pending effect.	The outer batch acts on its revoked
A current authenticated user receives a properly scoped callback.	A cross-tab payload copies source ID

A fake source map, diagnostic trace or always-disabled affordance cannot establish conformance. Part VII must check the actual registered handlers/entrypoints; Part VIII requires independently positive eligible HumanHelp Claim/Reassign and remote one-Refresh behavior. Parts IX–X separate finite test/model evidence, proof subject and actual browser/backend correspondence.

Authoritative absence

An authorized observation may establish that a domain entity is absent under a qualified entity lookup, or that a previously displayed entity does not belong to a **particular covered projection**. These are positive authoritative statements with specified provenance and scope. They are not consequences of an empty HTML element, a missing response, local filtering, a timeout, a missing DOM target or a rejected read.

The browser treats the following as distinct knowledge and physical states, even when they produce similar-looking empty presentation:

Observation or realization state	What it actually establishes
Present	An admitted authoritative value or membership under its declared actor, query, field and temporal
Authoritatively absent	A qualified negative entity or membership fact under an admitted coverage and progression contra
Not yet acquired	No applicable source observation has established the requested presence or absence.
Acquisition failed or indeterminate	The attempted read did not establish the requested fact; earlier justified knowledge and outstandin
Not disclosed or no longer authorized	The participant may not currently learn or retain protected content; the policy may deliberately hi
Physically not displayed	The actual current DOM/host does not show the entity, possibly because of filtering, retirement or

An authoritative absence may therefore be installed and adopted just as an authoritative presence may. The **evidence** that the entity is excluded and the **physical completion** of removing its representation are separate propositions. Adoption of negative knowledge is not a browser-only tombstone that will forever override future authorized presence. It remains subject to the same source, projection, progression, local continuity and ownership contracts as presence.

Absence requires actual covered authority, not an omitted row

The admitted source identifies the principal and Organization/tenant disclosure context, applied query identity, server-side predicate and arguments, entity or membership subject, projection/field coverage, temporal and consistency interpretation, relevant authoritative basis/progression, and the meaning of its negative result. A complete membership observation can justify the omission of a member only within the universe whose membership it actually closes.

A successful empty list is not automatically a complete query. A paginated page, truncated or top-N result, projection missing the membership field, filtered subset, cached partial response, redacted record, tenant-limited view, or speculative client-side search result does not certify exclusion from a larger universe. Even a complete **authorized** query says nothing about entities that policy deliberately does not disclose. A failed or denied read must not be converted into ”no Request exists” or expose protected existence through an overprecise error.

An explicit trusted exclusion response may justify a narrower negative membership fact without loading every card. It must bind the exact entity, applied query/predicate and authorized context, provider coverage/temporal semantics, and the exclusion reason it claims. For example, ”R17 is not in Bob's open-Requests board” need not establish whether R17 still exists, was completed, was reassigned, became inaccessible or was deleted. When the reason itself would reveal protected state, only the permitted nondisclosing proposition can be returned.

The provider's negative fact is admitted through a real acquisition or qualified persisted observation. A client-written `:provenance :authoritative` tag, a matching plan digest or a structurally valid basis token cannot certify its truth. The generic browser may preserve an opaque source/evidence reference and scope; the trusted consistency/model boundary owns the provider proof or expressly named premise. Absence need not allocate a permanent tombstone for every theoretically missing entity: a complete cohort observation or checked compact exclusion summary can represent justified negatives within its declared retention.

The membership scope is distinct from the domain entity's existence and from its fields. A Request can be excluded from an ”open for helper” board while remaining accessible in an authorized completed Requests projection. An omitted optional field may reflect a projection contract rather than a declaration that its domain value is absent. These forms of negative knowledge have different queries and cannot silently substitute for each other.

Worked example: R17 leaves Bob's open-Requests board

Bob has an admitted and physically adopted open-Requests board observation B100 for applied query Qopen, Organization O and his current authority context K. It includes Request R17. A subsequent trusted operation changes R17's state so that it no longer matches the predicate for Qopen. Depending on authorization, R17 might still be present under an independently queried completed-Requests projection Qdone; R17's absence from Qopen alone does not establish its deletion or a different helper's Membership.

The next authorized acquisition is evaluated against these possibilities:

Candidate evidence	Valid conclusion
An admitted complete Qopen membership observation B105 under K, with qualified progression and R17 excluded.	R17 is absent fro
An admitted explicit R17 exclusion witness for Qopen/K with the necessary coverage and temporal meaning.	R17 is excluded f
R17 omitted from page two, while Qopen covers multiple pages and no complete membership/exclusion witness exists.	No negative mem
An admitted Qdone observation contains R17 as completed.	R17 is present in
Bob's permitted board changes from Qopen to Qdone or he toggles a local `show-terminal?` view option.	Distinguish a nev
The server denies R17 disclosure after Bob's Membership changes.	Apply the require
Qopen acquisition fails, is cancelled, or produces incompatible/indeterminate content.	Preserve the qual

For the eligible complete B105 case the semantic path is:

```
qualified source establishes R17 excluded from Qopen at B105
        |
confirm K/Qopen coverage and provider progression
        |
prepare current membership outcome :absent for R17
        |
issue current scoped removal/installation permit
        |
check exact board owner, target incarnation and representation
        |
perform permitted physical removal or checked no-op
        |
admit real matching physical postcondition
        |
record adopted absence for the covered membership
```

A prepared negative result is not yet physically adopted. The permit binds the current runtime/assembly, actual Qopen coordinator, B105 source and covered membership, current board/card instances, target representation, mutation footprint and effect generation. A stale prior request or an OOB/extension mutation cannot bypass the permit by targeting the same HTML `id` attribute. If a multi-target update removes a card but fails to install a new count or status control, its actual per-target outcomes and repair obligations remain distinct.

The physical postcondition must establish that the **current owned representation** for R17's Qopen membership no longer appears or grants obsolete managed actions. A generic ”target not found” may indicate an unmounted board, wrong owner, selector mismatch or an already-correct current target. A no-op removal can count as completed only if the qualified host contract verifies the intended absence under the current scope and owner. A generic `afterSwap` event does not by itself establish that predicate.

Once a qualified absence is adopted, it retires obsolete provisional/continuity restoration rights for the corresponding presentation scope. A late stale callback must not recreate the removed card from an old capture. A current authorized later presence can legitimately restore R17 to Qopen through its **new** qualified observation and physical install. Absence does not represent an irreversible global tombstone.

The local filter distinction is especially important: Bob may hide an already acquired terminal Request through `show-terminal?` or text-search without starting a new read. Hiding the item is a local presentation choice, not new authoritative membership evidence. Re-showing it needs retained still-justified authorized content and current presentation ownership. If intervening authority has excluded it or revoked its disclosure, the local filter cannot resurrect stale markup or grant a new subscription/operation binding.

Provisional presence and negative authority remain separate

Suppose E99 provisionally inserts Request R30 into a local creator view while Create command C99 is unresolved. A complete board acquisition omits R30. The system must answer three **different** questions before changing presentation:

1. Does an admitted source actually establish R30's exclusion from this exact applied query, actor/context and coverage horizon?
2. Does that negative evidence, its qualified progression and the selected presentation policy permit superseding E99's provisional presence in the current physical target?
3. Does a separately qualified command-result source establish whether C99 incorporated, was rejected, or remains `:commit-unknown`?

The acquisition may have occurred before C99's possibly pending transaction incorporated, or under a different query/actor scope. In that case R30's omission is not automatically rejection or proof of nonexistence. Even sufficiently progressed **current board** absence does not by itself establish C99's historical attribution. A subsequent C99 incorporation, if still possible under the model's temporal/deduplication contract, requires its separately authorized reconciliation and potential later read.

The application may keep a visibly labeled pending overlay outside adopted membership, remove the provisional card upon an admitted negative-authority supersession, or use a declared conflict-resolution/reacquisition strategy. It must state the scope, owner, command-outcome and temporal premises for that choice; it cannot silently make an unresolved Create a rejected command, or misrepresent the overlay as an adopted current Request.

If stronger authorized absence supersedes E99's provisional representation, revoke the old DOM/rollback/timer ownership before physical cleanup. The C99 resolver obligation survives under its proper owner where promised. Removing an obsolete card does not justify cancelling all C99 history, repeating an unknown command with a fresh identity, or treating a missing physical target as a successful server operation.

A manager's later Reassign and the affected RequestAssignment facts illustrate the same distinction: a helper's card may no longer belong in one projection while its historical Claim was genuinely incorporated. Entity/query membership, current assignment and historical command disposition remain separate trusted questions.

Absence participates in qualified progression and adoption

Presence and absence are both qualified observation outcomes. The runtime compares them using the source provider's declared scope/coverage and progression relation, not the lexical/numeric order of basis labels, SSE arrival sequence or browser request generation. Two distinct opaque bases are not ordered merely because one was fetched later.

For the **same** applicable membership scope, a later provider-proved exclusion may supersede an earlier present card; a subsequent provider-proved presence may supersede that absence. When basis values are incomparable, the coordinator retains the competing obligations or invokes the declared read/merge/conflict policy. A complete board membership from R can coexist with an independently newer creator-local card S only if the qualified scope-aware merge preserves both provenances and does not assert a single atomic backend snapshot.

A scoped negative fact may satisfy an advisory requirement only if its source establishes the required query, authority context, coverage, progression and discharge stage. Acquisition of a negative result does not clear adoption-stage debt before the actual current DOM removal/no-op postcondition has been admitted. Failing to install a negative membership result preserves repair debt without falsifying the underlying acquired fact.

Absence from an open-Requests board does not discharge a complete-query requirement for a different Organization, a partial page or a card-specific historical command resolution. One explicit user Refresh continues to obey its original read budget: discovering an omission cannot silently authorize another fetch to ”check whether it was really deleted.” If more evidence is necessary, obtain a new permitted read or enter an honest blocked/ unknown/reconciliation state.

Security and privacy revocation is a different actuation contract. A current principal losing access may require immediate removal or quarantine of protected presentation even without evidence that the entity is absent from the model. That revocation does not permit a new protected read and must not reveal nondisclosable existence. No browser-side cleanup can erase data an adversarial client already recorded; the trusted server continues to enforce authorization at every read, stream and write.

The initial baseline/subscription and multi-node router gaps from 06.06–06.07 apply equally to negative knowledge. Missing invalidations must not leave a newly excluded Request permanently displayed under a guarantee of current-state convergence; the promised replay/fence/reacquisition and user-action assumptions must actually support the claim.

Paired authoritative-absence acceptance histories

The finished design requires real eligible trusted sources, complete coverage or qualified exclusion and actual host completion. Negative fixtures must reach the relevant absence, scope or physical boundary rather than pass solely by producing an empty element.

Eligible positive history	Ad
Complete authorized Qopen B105 excludes previously present R17 and is provider-proved compatible with the installed membership.	R17
Trusted explicit R17/Qopen exclusion witness has narrower sufficient coverage.	Bro
R17 is absent from Qopen while a qualified Qdone projection still contains it.	The
A current authorized removal permit operates on the matching Qopen card and checks its physical postcondition.	A g
New qualified B105 exclusion retires R17's old provisional/rollback rights; later provider-proved presence may reintroduce it.	A s
C99 Create remains unknown and R30 is omitted from a board read that predates possible incorporation.	Om
Bob hides a retained authorized R17 under a local text/sort/terminal filter and restores it while membership still justifies the content.	Loc
A complete authorized removal of one card and matching count update both physically finish under their distinct permits.	Car
An acquired negative result satisfies O1 but installation fails and one-Refresh grant is exhausted.	The
Access is revoked and the honest client clears protected presentation under a current privacy policy.	The

These cases specify the required negative-knowledge and physically verified removal contracts for every profile claiming authoritative-absence adoption. Part VII must inspect actual model, writer, renderer and browser entrypoints; Part VIII requires nonvacuous HumanHelp Claim/Reassign/Done, creator-local presentation and remote one-Refresh behavior; Parts IX–X distinguish finite tests, trusted model premises and real physical host refinement.

Failure, liveness, and convergence

Browser safety, eventual progress, bounded resource use, and recovery are different claims. The finished framework must express each with the correct subject, quantification, source and environmental premises. A runtime that retains an unresolved command, a failed installation or unmet advisory debt while reporting that condition truthfully can remain **safe** without being currently **live**. A system that eventually renders the right card after first issuing an unauthorized Claim was not safe merely because it converged later.

The application construction and assurance pipeline must distinguish proved semantic obligations, checked finite reference cases, tested actual implementations, qualified physical host correspondence, runtime-enforced admission and still-trusted external assumptions. No successful diagnostic, model fixture or optional proof artifact may become the permission by which production chooses a semantic transition. Parts IX–X retain the separate proof and certification contracts.

Failures preserve authoritative truth and its observer-relative limits

A failure classification answers **what happened at which boundary** and **what this participant is justified in knowing**. It cannot derive domain truth from a generic network or browser error.

Failure or disposition	Required semantic and physical treatment
Rejected local input/operation admission before protected send	Report local nonadmission; do not manufacture a server-side `:reje`
Qualified trusted operation rejection	Preserve original command/actor/operation binding and permitted
Qualified incorporation or compatible already-incorporated outcome	Retain historical incorporation even if later notification, DOM insta
Transport loss after possible physical send	Preserve `:commit-unknown` until a qualified resolver/deduplication c
Trusted commit followed by invalidation/stream failure	Commit remains true; retain missing-delivery/interest-gap and app
Failed acquisition, denied disclosure, missing provider witness	Do not invent presence, absence, collection membership or progres
Prepared installation, host mutation or OOB effect fails	Distinguish acquired observation from completed per-target physic
Stale or incompatible continuation	Revoke obsolete semantic/actuation rights before best-effort physic
Resource cancellation/release fails	Preserve physical remnant information and enforce resource budge
Adapter invariant or source-normalization failure	Contain the affected semantic owner/runtime, retain original causa
Lost compatibility, grant, storage or recovery evidence	Report blocked/unknown/expired/manual-reconciliation as applica

A UI timeout belongs to its actual owner: it may expire a provisional spinner, cancel a physical waiting policy, trigger a qualified resolver attempt, or mark a local presentation deadline. None of those facts alone says whether C42's trusted RequestAssignment Claim committed. A timer that makes E42's presentation stop waiting need not retire K8's distinct command-history recovery deadline. A domain `:rejected`, `:already-incorporated`, conflict disposition or current ”not applicable” result is not synonymous with a broken HTMX/FFI integration, malformed response or impossible adapter state.

The runtime must retain source-relative evidence and primary fault causality. If physical effect F partly executed and cleanup CA/CC failed, report F with subordinate release failures and the actual remaining obligations. It may safely quarantine, reacquire or use declared compensation, but cannot blindly repeat a nonrepeatable effect batch or restore an obsolete provisional snapshot. A current authorized Order/Request card can be reconstructed while the historical command that may have created it remains unresolved.

Safety properties quantify over all admitted relevant transitions

Under the named semantic contracts and actual implementation/host premises, safety requires at least:

- **No unadmitted physical act:** a protected HTTP send, DOM mutation, OOB/response-header effect, timer-triggered restoration or disclosure operates only under its current relevant registered source, issuance/owner identity and last-controllable permit.
- **No stale authority revival:** retirement removes E42's private semantic/physical rights independently of whether abort, listener detachment or timer cancellation succeeds. A late callback cannot settle E43 or mutate B2 using B1's old selector.
- **No fabricated business outcome:** local timeout, transport failure, successful parsing, plan compatibility and physical installation cannot manufacture incorporation, rejection or current RequestAssignment facts.
- **No false authoritative adoption:** a current authorized acquisition supplies only its actual covered positive/negative observation, while adopted presentation requires the matching physical target-specific completion and permitted scope.
- **No invented progression/absence:** opaque XTDB bases, duplicated metadata, SSE arrival order, partial query omission and browser generations are not trusted provider ordering/exclusion witnesses.
- **No obligation loss through scheduling:** advisory, subscription-gap, command-resolution, partial physical-effect and adoption debts survive until their own declared discharge conditions hold, regardless of queued-slot or logical-cycle cleanup.
- **No permission bypass:** automatic versus one-user-Refresh policies, server read/write authorization, tenant disclosure and compatibility are separately enforced at real physical boundaries.
- **No incorrect restoration:** local continuity or provisional rollback does not overwrite newer qualified canonical state or transfer physical resource ownership because two values or CSS selectors happen to match.
- **No false assurance provenance:** observed adapter effects, physical host actions, trusted model results, finite model checks and full application certificates are not collapsed into one success label.

These are families of invariants indexed by runtime, principal, scope, owner, resource instance, command, attempt and effect identity—not just assertions about one convenient concrete Request ID. A safety proof must quantify over the declared transition/environment relation, or limit its claim to a faithfully checked finite subject. A test of E42/E43 with two chosen integer generations cannot by itself prove noncollision for arbitrary reuse or server deployment.

An application with opaque Clojure side effects, a third-party DOM writer, an unverified HTMX extension or a progression provider not yet related to actual XTDB executions can still be useful, but guarantees depending on those components retain the named trust/physical correspondence premises. Failing to establish a required premise means the stronger claim is **not established**, not that the proof silently assumes every opaque function did the right thing.

Terminal reachability and liveness are not interchangeable

Finding a path from each reachable state to some terminal state does not establish that every applicable execution will reach it. A machine can remain forever at `waiting` if the environment never supplies settlement, timer, allowed user action or authoritative read:

```
waiting --qualified settlement--> resolved
        --timer fires----------> local recovery / C42 still unknown
        --nothing enabled------> still waiting
```

Safety excludes invalid steps; liveness adds explicitly quantified progress premises. A claim that ”every submitted command eventually reaches a trusted result” needs more than a browser timeout: it depends on a resolver that can distinguish incorporation from nonincorporation within its retention horizon, eventually available authoritative services, relevant fairness/retry scheduling, surviving command binding and present disclosure permission. Where the resolver cannot decide after history expires, the supported progress claim may instead reach an honest manual or unknown recovery terminal; it may **not** claim a proven committed/rejected outcome.

The assembly and checker distinguish:

Progress property	Essential premises or limitation
Eventual local retirement	The relevant local transition/callback is eventually scheduled, or a separately safe shutdown/admissi
Eventual command attribution	A qualified durable resolver/deduplication record survives and eventually responds with permitted e
Eventual authoritative acquisition	A current authenticated read remains permitted and a sufficiently covering provider response can ev
Eventual physical adoption	Current compatible target/host permits persist sufficiently, mandatory installation can finish or reco
Eventual Live agreement	Relevant state/query/authority eventually stabilizes or has a suitable convergence contract; gap dete
Eventual cleanup/resource recovery	Physical handles can eventually be released or the implementation can safely reset/degrade within a

Weak/strong fairness assumptions must attach to the actual enabled transition and scheduler, not merely appear as a generic label. An environment that keeps a callback disabled, repeatedly revokes permission, partitions the network or changes authority forever may prevent eventual progress without violating the safety invariant. An application cannot promise unconditional convergence while allowing permanent denial of current data or indefinite suspension of its only authorized user action.

Liveness proof obligations also avoid vacuity. If every real operation is disabled, no invalid command reaches the server, but the product still fails its required positive Claim, Reassign or Refresh behavior. An acceptance test must establish that eligible paths actually execute through trusted model, transport and physical display stages, then challenge the corresponding failure and adverse scheduling cases.

Agreement debt may converge under scoped quiescence

Different participants may legitimately adopt different authorized views while writes continue. A participant should know which observations it has adopted, their authority/source, query/field coverage, provider progression and any unfulfilled advisory/gap debts. The strongest generally useful promise is **scoped conditional convergence**, not instantaneous equality of every browser's DOM or knowledge.

A representative contract is:

```
Assume:
  relevant model state and applied query/authority eventually stabilize;
  a sufficiently covering authorized observation is eventually obtainable;
  the interest gap is fenced, replayed, or repaired by reacquisition;
  the selected acquisition grant is eventually available and actually used;
  the required host/transport/renderer and fair scheduler eventually succeed;
  retained obligations are not lost and no incompatible owner blocks forever.

Then:
  the participant eventually acquires and physically adopts the
  promised stabilized covered projection, or meets its explicitly
  declared weaker recovery/blocked terminal where the liveness
  contract permits that outcome.
```

To claim **actual adopted convergence** rather than a weaker recovery alternative, the assumptions must exclude permanent failure and grant absence. It is invalid to place a ”manual help available” state in the conclusion and then claim every user eventually observes the latest board. The current model truth, a participant's acquired evidence, installed view, unresolved C42 history and advisory/Refresh debt are separate components, not one monotonic counter named current.

HumanHelp's remote one-Refresh policy intentionally allows an interval in which Bob knows an advisory exists but has not authorized acquisition or adoption. Its liveness target is conditional on Bob's qualifying user action. The creator's correlated local S card may be more advanced than an older R whole-board membership result; a correct convergence merge retains the separate qualified scope evidence instead of inventing a common XTDB snapshot. One Refresh may satisfy multiple accrued O1/O2 obligations if its actual acquired and installed observation covers them, but never silently issues A2 when G1 permitted only A1.

A browser that missed changes from X17 through X900 may recover a **current-state** view directly through a sufficiently qualified X900-or-later observation rather than replaying every intermediate SSE packet. That result requires real query/authority/progression coverage and permitted physical adoption. It does not establish that every intermediate business event or side effect was processed. Applications requiring durable per-event processing need a separately declared event/log and exactly applicable effect contract.

The initial baseline-to-interest gap, late registration and multi-node router failover retain the same semantics as single-node: a missed event not generated under any interest cannot be restored by redelivering the known events alone. A valid fence, durable

replay or qualified current-state reacquisition is required to make the promised continuity and convergence conclusion. SSE silence or an `online` event does not prove currentness.

Boundedness is not supplied by single flight

The application declares resource budgets, overflow policy, retention horizon and recovery for distinct categories:

Budgeted resource	What the bound must account for
Logical coordinator cycles	Active acquisition/installation cycles and their qualified owner identities.
Queued successor scheduling	Number of successor slots, without confusing the slot with acquisition permission.
Pending advisory/requirement information	Potentially incomparable obligations, provider-sound compaction and retained gap/adoption
Command resolution and deduplication	Original binding/records, durable retention, expiry, replay fences and still-unknown results.
Physical attempts and host remnants	Requests that cannot be aborted, native listeners, timers, DOM/continuity captures, OOB effe
Delivery replay/identity history	Consumed message IDs, retired resources, allocator epochs and safe compaction windows.
Diagnostics and evidence buffers	Observer throughput, trace redaction, retained failure provenance and bounded storage.
Current adopted/retained projection	Permitted board/card data, privacy/disclosure expiry, presentation resources and restoration
Retry and recovery work	Attempts, backoff, reloads, resolver reads, grant consumption and manual escalation.

The default one-active/one-coalesced-successor coordinator bound is a bound on **logical scheduling**, not a theorem that all HTTP, DOM or resource costs are bounded. A single queue slot can stand for unboundedly many incomparable requirements; a failed abort can leave physical work alive even when semantic state declares it retired.

Coalescing is safe only where the provider's coverage/progression contract proves that a sound summary retains all required discharge conditions. A stronger complete read may collapse several obligations where admitted; choosing a maximum-looking opaque basis or dropping the oldest request because the queue is full does not. If no sound bounded summary exists, the application must choose a declared backpressure, resynchronization, storage, or visible capacity-failure contract. It cannot simultaneously promise unbounded distinct obligations, bounded memory and no durable external storage without a sound representation theorem.

Resource exhaustion must preserve safety: block new protected sends or physical effects, revoke/retire unsafe owners, report actual missing work and allow only permitted reset/recovery. Silent downgrade to unprotected raw HTMX, erasure of pending C42, unreported drop of O2 or fabrication of a complete installation is not an overflow policy.

Even where a resource budget is finite, liveness may fail because capacity remains exhausted. The target assurance must separately state whether cleanup, durable compaction, user intervention or process reset eventually restores capacity, and which commitments survive that failure domain.

Layered assurance distinguishes checks from assumptions

The following evidence classes support different, noninterchangeable claims:

Evidence or check	What may be established, and what is not implied
Portable executable semantics and invariant checker	Safety for the declared semantic subject and admitted transition relation under prov
Exact finite Recife/TLC check	Obligations on the selected finite reference model and its enumerated assumptions;
Actual adapter-step observation	A transition selected by the current executable adapter for those admitted inputs; n
Executor-bound source/physical observation	Precisely observed bounded execution evidence under its registered executor, extrac
Browser/Chromium conformance trace	Actual tested host effects and callbacks on the specific HTMX/asset/extension profil
Trusted backend operation or authorized readback	Qualified incorporation/negative result or current observation under the exact mod
Matched physical installation postcondition	Completed target-specific effect and possible scoped adoption; not an unrelated wh
Application assembly and deployment certificate	Only the explicitly checked full inventory, semantic/proof subject, trust premises, s

A proof kernel is distinct from the portable executable semantic core. The former checks precisely defined evidence or derivations; the latter drives production execution. Neither diagnostic observer nor proof report is allowed to alter the chosen runtime branch. Gesso application authors declare business rules, operations and Gesso construction; they are not required to manually write Recife, Quint or TLA+ specifications. Generated proofs and checks require explicit correspondence from application declaration to model and from model to the running implementation/host.

When a finite checker rejects, times out, is malformed or becomes unavailable, its obligation remains unestablished; the system does not attach a success label or conflate the checker failure with a domain-model rejection. An optional diagnostic may be omitted under a declared weaker assurance policy, but an application requiring that specific assurance cannot be certified by executing without it.

The specified assurance level determines which construction, provider and physical correspondence obligations must be discharged. A runtime or deployment unable to establish a required premise reports that limit rather than silently weakening the normative guarantee.

Paired failure, liveness and boundedness histories

Each positive witness must use an eligible application path and check its actual command/read/physical result. Each adversarial witness must reach the intended failure, fairness or capacity boundary without being rejected for an unrelated malformed input.

Eligible positive history
A qualified C42 Claim incorporates; its response is lost but authorized readback later proves the original command result.
An authoritative Request card is acquired and its permitted physical target install completes.
E42 is retired, its stale callbacks fail admission and exact cleanup eventually releases old resources.
Bob sees O1, grants exactly one A1 Refresh, and installs a covering result that also satisfies O2.
Under quiescence, available authority, grant, fair scheduling and working fence/reacquisition, Bob eventually adopts the promised Q view.
A current read at X900 establishes the promised board state after missing earlier invalidations.
O1/O2 are provider-justifiably compressed into one covered obligation; resource use stays within declared limits.
A trusted excluded-member result removes R17 from its current board target while retaining its possible presence in another query.
An eligible Claim/Reassign and remote Refresh complete through real model, transport and browser callbacks.
A finite reference check reports bounded assumptions and nonclaims for its exact derived execution subject.

Part VII must certify the actual installed operation, route, subscription, renderer and handler inventory. Part VIII must exercise real HumanHelp Claim/Reassign, creator-local updates, remote one-Refresh and continuity behavior. Parts IX–XII must preserve the correct proof, physical/backend correspondence, deployment and multi-node trust boundaries.

Refinement, correspondence, and conformance evidence

The browser refinement claim relates one admitted executable semantic plan and its source-normalized transition/effect history to the actual physical behavior of the selected browser, HTMX/SSE integration, backend, renderers and other installed effects. These are **different subjects**. A valid role-local adapter transition is not a trusted server commit; a requested send is not actual delivery; an observed browser receive does not by itself prove a corresponding sender emitted it; a local machine finishing does not prove the global choreography terminated.

Gesso's finished assurance pipeline makes these distinctions useful: the application author declares domain rules, Gesso construction and the supported physical profile, while the assembly and evidence pipeline generates, checks and relates its semantic witnesses. Authors need not manually write Recife, TLA+ or Quint. Generated evidence does not confer semantic authority on diagnostics or let an unsupported host action pass a physical gate.

The **object being certified** is always explicit: exact choreography and projected executable artifact, admitted application/operation assembly, actual source event or host trace, specified backend execution, selected release/profile and the claimed safety or liveness property. A pass over an attractive surrogate does not establish correspondence of the application that ran.

Observe actual adapter transitions and independent physical occurrences

A diagnostic about the browser's internal machine starts from the **actual** adapter step, not an externally proposed successor that a report happens to describe. The recorded next state and ordered abstract effects come from one execution of the admitted portable transition function against the exact before-state and normalized event. The observation carries the current execution, owner, effect and applicable artifact identity and cannot itself create an installation or command result.

The physical shell and qualified external sources additionally record or attest the concrete actions to which the guarantee applies. For each claimed physical step, the evidence names its own producer, registration, environment/asset profile, observation point, occurrence identity, effect/target incarnation, and real disposition. A report that merely restates the adapter's intended effect is not physical evidence that the effect happened.

Evidence layer	Proposition it can establish
Selected portable adapter step	The real semantic adapter selected this next state and ordered abstract effects for these admitt
Role-local projected machine history	This role locally performed or accepted the recorded step under its executable plan.
Abstract `:transport/send` effect	The semantic runtime requested a correlated transport attempt.
Correlated physical host issue/return	The selected host performed the reported issue/completion boundary under its qualified profi
Trusted domain command/result witness	The authenticated model operation or qualified resolver established a particular C42 historica
Qualified authoritative acquisition	The trusted provider established stated query/entity/absence, scope, context and progression.
Real scoped physical installation witness	The current permitted effect established its declared target-specific postcondition, including s
Participant adoption transition	The participant admitted the actual covered installation into its current scoped presentation s
Checked concrete correspondence witness	A specific independently sourced and bound trace satisfies the stated relation and checker pre

For the actual host trace, issue, send, send-failure, receive, response processing, native swap, OOB target mutation, timer callback, resource release and completion may be distinct occurrences. The selected profile says which observations are available and sufficient for each protected contract. If that profile does not expose a controllable or observable protected mutation, the application must reject the strong realization, use another qualified host contract or explicitly retain the unproved premise; it cannot obtain a universal witness by giving an unobserved step the same label as a known one.

An event-source registry and an adapter-created correlation value help bind an actual callback to issued work. They do not authenticate arbitrary client-payload claims. Trusted backend/XTDB and command evidence must be produced at their own authoritative boundary. An actual physical browser observation cannot be replaced by a synthetic event sequence merely because the same adapter steps appear when replayed.

Refinement relates concrete traces, not similar-looking outputs

For an admitted initial relation, one may state a conditional forward-simulation/refinement obligation as follows:

```
Assume the named executable, source, backend and host contracts.
Let R(c, s) relate a concrete running state c to semantic state s.

For each actual concrete protected transition c → c':
  either:
    there is a matching permitted semantic transition s → s'
    with its required identity/evidence/physical bindings, and R(c', s');
  or:
    the concrete transition is an admitted internal stuttering step
    with R(c', s) and no hidden protected effect or lost obligation.

For every claimed semantic physical completion:
  an appropriately sourced concrete occurrence establishes its
  exact permitted target/effect postcondition.

No concrete protected effect is omitted from the correspondence.
```

This is a **conditional proof obligation** whose discharge requires evidence for each supported browser/HTMX profile and deployment. The relation has to cover source-normalized event admission, current owner and resource instance, physical commitment points, partial mutations, runtime faults, recovery obligations, qualified progression, and observations/authority context. A model containing only successful send/swap histories cannot refine an implementation with failed abort, duplicate settlement, OOB changes and stale callbacks.

A concrete step can correspond to zero semantic progress only if its ”internal” status has been justified. A DOM replacement, unadmitted second request, cross-principal disclosure, resource-generation change that revives old rights, or loss of pending O2 debt cannot disappear into an unspecified stutter. Conversely purely local observer bookkeeping, bounded trace buffer maintenance or a harmless ignored stale event may stutter when its noninterference contract really holds.

Safety refinement and liveness are separate. Infinitely many internal steps can satisfy a naive safety stuttering relation while preventing an enabled authoritative installation forever. A stronger liveness conclusion requires an explicit divergence condition and the fairness, provider, grant, recovery and physical scheduling premises of 06.13.

The relation need not impose one artificial total order on independent concurrent operations. A declared weak or commuting correspondence may account for causally independent steps and permitted internal actions. It must preserve the actual happens-before edges, per-resource and per-command order, server transaction constraints, physical target serialization and every externally

observable effect whose order matters. Commutation cannot swap a settlement ahead of its issuing send, a DOM completion before its current permit, or an old callback past the revocation that makes it powerless.

Trace alignment preserves actual occurrence identities and provenance

A concrete trace matcher checks occurrences, not merely similar maps or eventual output values. It establishes a declared mapping from each relevant real occurrence to its role-local/semantic counterpart and verifies that neither side supplies invented send, delivery, incorporation, completion or adoption events.

For a supported exact or weak correspondence, the matcher:

1. Binds the exact admitted choreography, projection, executable plan and application/assembly subject; checks the artifact relation rather than relying on a friendly source name or copied plan digest.
2. Binds the concrete trace to its actual executor/ physical source, runtime/deployment instance and declared extraction point. A caller-supplied list of plausible observations is not evidence that those events happened.
3. Preserves relevant individual occurrence IDs and their issued effect, owner generation, command, transport attempt, target, observation and resource incarnation.
4. Requires every **protected concrete** action to be accounted for exactly under its permitted semantic or independently justified internal classification.
5. Never duplicates one physical send, receive, target mutation or completion to satisfy two unrelated obligations; never drops the inconvenient occurrence and calls the remaining sequence aligned.
6. Checks the declared causal and ordering relation, allowing only proven independent commuting steps.
7. Distinguishes absent physical evidence, a reported physical failure, an explicitly checked no-op and actual successful completion.
8. Carries source/provider trust premises forward; a structurally validated `:advances` or `:committed` carrier is not a self-authenticating authoritative witness.
9. Exposes ambiguous matches and their assumptions rather than guessing the desired witness from the final state.
10. Rejects traces whose extraction, version binding, loss reporting or coverage cannot support the requested property.

An effect request and its completion may share a semantic effect identity but remain **different occurrences**. Multiple compatible HTTP attempts H1/H2 can share C42 yet retain distinct physical IDs and possible delivery outcomes. A trusted command resolver may prove C42's historical incorporation while the actual H1 receive still lacks a transport witness. A successful DOM replacement can be physically observed while an unrelated board advisory remains outstanding.

The trace extractor is part of the assurance boundary. It must be sufficiently independent of the semantic claim it is checking that it cannot silently fabricate the required fact. For a physical host claim, a reporter that merely copies `:htmx/allow-swap` into `:authoritative/installed` is not a witness to a real swap. For source/backend claims, submitting a model result as an opaque request's expected value does not prove the actual transaction followed that model. The emitted record must identify who/what observed the actual execution and at which point.

Completeness is relative to the claimed footprint. An instrumented real Chromium test can establish one concrete tested lifecycle and admit the remaining uncovered extension/browser paths as explicit assumptions or unsupported. It cannot claim complete physical correspondence if the instrumented hook does not see mutations from permitted scripts, OOB processing or another DOM writer.

A digest can bind evidence to exact bytes under its canonicalization and collision premises; equality of digests is not admission of the evidence itself. A diagnostic/proof sidecar is read-only: runtime operation/branch selection depends on the executable semantic plan and current admitted inputs, not on whether a favorable certificate happens to be attached or can be forged.

Global communication and termination need independent witnesses

A browser role-local `:send` is normally hidden from the distributed-observation alphabet until actual communication requirements are established. A role receiving an envelope can produce a **candidate** distributed communication shape, but cannot prove from its receive history alone that the expected sender emitted the same event, that transport delivery corresponds to that send, or that a global admissible schedule exists.

The corresponding independent witnesses include:

Candidate global claim	Required additional evidence
Browser role-local receive corresponds to distributed communication	Sender emission, delivery/correlation and global schedule ad
Browser role-local terminal outcome corresponds to global return	Global terminal outcome, all-role terminal compatibility and
Browser local authoritative observation means trusted model step occurred	Authenticated operation/transaction evidence with exact sch
Browser adapter requested transport and therefore remote effect occurred	Actual physical send/delivery plus qualified receiver and ope
Browser installed a view and therefore application reached agreement	Current scoped physical/adopted observation and independe

Importing an already-advanced browser machine at `:execution/start` cannot attest the steps that occurred before import. Likewise machine history erased on retirement cannot be reconstructed as observed merely because a terminal effect was emitted. The evidence pipeline records these unwitnessed intervals and refuses to promote candidate observations into certified global facts.

A local execution becoming terminal is not the global choreography reaching its specified return outcome; an outer role-local success envelope is not the trusted final model result. To establish a global claim, the independent role/sender/authority evidence is related to the actual observed schedule with the declared correspondence checker. This avoids a self-confirming loop in which the browser emits both the assertion and the supposed proof of its truth.

The portable semantic implementation may have independently checked canonical choreography-to-projection correspondence for a selected concrete witness, including weak/commuting alignment. That evidence is valuable but still witness-scoped until a quantified refinement argument or separately justified complete abstraction has been established. Linking to the actual browser host and backend execution is an additional obligation, not automatic from projection agreement.

Execution isolation is local except for declared shared effects

Execution-private state and effects belong to their exact owner generation. E42's timer, transport completion, provisional rollback, local output and target lease cannot advance E43 or mutate E43's private physical target simply because both executions reference the same Request or historical command C42. An old callback may remain physically present after logical retirement, but it has lost actuation rights.

The model also admits **intended shared interactions** through their declared semantic owners. A current fragment coordinator may receive a scoped SSE advisory, adopt an authorized RequestAssignment observation, revoke old provisional presentation or apply a declared resource-pressure policy while several local executions are active. Those operations affect multiple executions in ways the semantic relation must represent rather than exclude as unmodeled interference.

Interaction	Required isolation/correspondence treatment
E42 private callback after E43 starts	Deny E42's progression and stale physical target rights; optiona
Current B3 shared advisory during E42 work	Admit under B3's current subscription/interest and applied scop
Shared authoritative card supersedes E42 provisional display	Revoke only the applicable weaker presentation rights under a
E42 and E43 attempt to mutate the same DOM target	A current serialization/merge/supersession relation determines
An old owner and new owner share a timer ID or same CSS height value	Exact resource/owner incarnation controls release; old cleanup
Current whole-board R result overlaps newer creator-local S	Preserve separately qualified card/cohort representation or reac
One coordinator retires while a shared SSE connection serves another	Detach only its exact interest and revoke its callbacks; preserve
Deployment/runtime fails and shared capacity is exhausted	Apply declared runtime-wide containment/backpressure and re

An isolation theorem therefore names the protected state/physical footprint, allowed cross-owner transitions and the environmental noninterference premises. It cannot mean ”no execution ever affects another” when the application intentionally coordinates shared authoritative views. Nor can two execution names make a common DOM target or native browser side effect thread-safe by declaration.

Where one effect interferes with another, the assembly supplies an admitted serialization, supersession, transactional physical primitive or scope-aware merge. If a host reenters during a protected multi-target action, the protocol names actual intermediate states and partial failure disposition. JavaScript's event loop alone is not proof that an arbitrary multi-target HTML swap is physically atomic.

An actor/tenant change, permission revocation or incompatible deployment can justifiably retire several owners at once. Each affected resource, advisory requirement and command resolution obligation still requires a current scoped disposition. The server independently authorizes reads, subscriptions and writes against a hostile client that may ignore all browser isolation gates.

Diagnostics and witness extraction do not alter execution

Observing the execution must not be a second executable semantic interpreter. For the same admitted plan, event sequence and actual physical input behavior, attaching or removing a read-only diagnostic observer must not add, remove, reorder or rewrite semantic transitions, abstract effects, command sends, owner state or authorized completion.

A throwing, malformed, slow or disabled observer follows an explicit bounded failure-isolation policy. An ordinary in-process callback without scheduling/ resource isolation can still block or reenter execution; calling it ”optional” is not proof of noninterference. Instrumentation effects and buffer capacity need a real contract. A production certificate cannot depend on collecting complete traces from a buffer that silently drops relevant mutations without loss markers.

The actual source boundary and physical capture must be audited for completeness and trust assumptions. The strongest runtime conformance check may require dedicated instrumentation or an external observer outside the semantic decision path. The application's semantic branch never switches because a proof report is present, absent or favorable.

Paired refinement and isolation acceptance histories

Every positive test exercises a real admitted transition, source, physical effect or concrete independent witness as appropriate. Its adversarial counterpart reaches the relevant correspondence boundary, rather than trivially rejecting an unrelated malformed fixture.

Eligible positive history
A real adapter step emits a specific ordered effect vector and the observer rederives it from the same state/event.
E42's role-local receive has matching independent sender, delivery and global-schedule witnesses.
Two roles finish under a globally admissible terminal schedule with an independent global outcome witness.
H1's actual issue and host completion are observed with registered correlation, and trusted C42 evidence is separately acquired.
A permitted real HTMX card swap installs B2 and a qualified target-specific postcondition is observed.
A weak/commuting witness reorders only proved independent private steps while preserving required causal edges.
A current B3 shared invalidation coexists with private E42/E43 work and affects only declared scopes.
E42 and E43 compete for one target; a qualified supersession policy revokes old rights and preserves newer installed content.
The same plan and inputs produce identical semantic/effect traces with and without a bounded read-only observer.
An artifact-bound concrete Choreo witness passes its real checker, and a separately sourced host/backend trace supplies the required physi

Part VII must close the deployed application inventory and installed entrypoints; Part VIII requires positive HumanHelp Create/Claim/Reassign, creator-local and one-Refresh behaviors through actual model/browser effects. Parts IX–X specify the exact

checker, proof-kernel, source and physical correspondence premises. This section defines their joins; the correspondence checks and trust assumptions must be discharged for each claimed supported realization.

Browser realization invariants and conformance

Authoritative installation and resource invariants

Part VI closes the browser realization contract by stating what the **actual admitted implementation** must preserve under every relevant event, source, owner, target and failure class. An invariant about one portable transition is not automatically an invariant of an external HTMX callback, an opaque effect handler, XTDB transaction, third-party DOM writer or installed deployment. Each stronger statement requires the corresponding realization or independently accepted assumption.

The executable browser semantics owns current coordination, local Choreo progression, abstract effect issuance and admission. The shell interprets those effects through registered physical handlers and normalizes actual host events back into the same semantic state owner. The actual protected actions remain subject to their supported physical gates. Trusted business operations and authoritative observations remain separate model/backend concerns. Diagnostics and assurance artifacts may explain this process, but do not decide which transition production executes.

1. Installation and adoption invariants

   For every admitted authoritative **presence or absence** installation, the following propositions must be maintained under the declared observation, host and authority contracts:

Invariant family	Required property
Source and disclosure	The observation is qualified for the actual principal/authority context, applied query, entity or collection, fi
Exact representation	The prepared content and generated operation bindings correspond to that observation, the permitted rend
Permit and current target	The runtime issued a current permit for the exact effect, owner, physical target incarnation(s) and entire mu
Last-actuation control	Each protected physical mutation remains permitted at the last controllable act; deferred or reentrant work
Physical result	Completion records an actual correlated target-specific postcondition; partial, failed, absent or unknown ph
Scoped adoption	Acquired, installed and adopted facts are recorded only for actually covered and completed scopes; an entity
Obligation discharge	Only advisory, gap, acquisition, installation or recovery obligations whose separately declared satisfaction c
Nonregression	Older board R cannot silently erase a separately newer justified card S; an old provisional snapshot cannot c
Negative knowledge	A removal follows a complete covered membership observation or qualified exclusion, not a missing page, s
Independent command truth	Neither browser installation nor local timeout changes a historical command's trusted incorporation/rejecti

   An acquired fact can be valid while its installation is pending or failed. A physical card can be correctly removed while an unrelated counter/OOB target fails, leaving only its own completed scope adopted. A current authorized RequestAssignment observation can coexist with a previous historically successful Claim and later manager Reassign; the current card does not rewrite the earlier command outcome. A checked no-op removal counts as physical completion only if a current qualified postcondition covers that exact representation and owner.

   The provider's progression relation, not basis-string arithmetic, determines whether an observation supersedes previously adopted data. Different authorized cohorts and independently installed cards may coexist only under a declared scope-aware merge/reacquisition policy. A mixed view is not evidence of a fictitious single XTDB snapshot.

2. Resource, retirement and recovery invariants

   Every live semantic resource is owned by one current execution/coordinator/recovery scope or an explicit shared-ownership class. Physical handles may remain opaque, but the semantic issue ID, generation and exact owner binding are established before host issue and retained through callbacks. A late host handle returned after retirement is cleaned up under its retired issue; it cannot be installed into the successor's live registry.

   For an execution E that has **semantically retired**, the central R4 property remains:

   > No subsequently delivered execution-private callback or deferred effect may use E's revoked rights to advance current semantic state, alter current protected presentation or assume ownership of a successor's physical resource, regardless of whether E's physical cancellation succeeded. Separately admitted shared coordinator and historical command-resolution work retains only its own rights.

   This property does not mean physical resource cleanup is instantaneous, perfect or free. Failed aborts, retained listeners, delayed timer callbacks, stale layout captures and partial DOM effects are bounded/observable cleanup and repair obligations. Physical cancellation is an optimization and resource-management mechanism, while semantic revocation plus last-actuation prevention supplies the protected safety property.

   A resource cleanup operation is idempotent for **its issued resource instance**, not for whatever now uses the same registry key, timer number, DOM selector or CSS value. A newer owner writing the same `min-height` value is not proof that the old

owner may restore the style. Where the host lacks an adequate isolation primitive, the realization must supply owner-aware arbitration, a genuine exclusive lease, an admitted weak guarantee or an unsupported profile rejection.

An unresolved command's lifetime can exceed the execution that submitted it. Its immutable command binding, trusted resolver, deduplication contract, retention and disclosure rules must survive the specified failure domain where recovery is promised. The system cannot infer `:rejected` from timeout, browser retirement or absent current card; it cannot infer `:committed` from HTTP 200 or a similar-looking current entity. Unknown incorporation and currently acquired/adopted model state remain separate.

The fragment coordinator has at most the declared number of active logical cycles and coalesced scheduling slots. That bound does not cap unbounded incomparable requirements, pending physical attempts, stale handles or retained command history. Every category has its own sound compaction, retention, backpressure, reset or explicit unavailable policy.

3. Event/source and runtime safety invariants

   Every semantic event is normalized through its actual registered source and admitted for its declared owner class, current resource, issued effect, generation, legal machine state and duplicate/replay contract. A stale private callback cannot relabel itself as shared. A legitimate current SSE advisory may affect a shared coordinator after the initiating execution retires; the event must still respect subscription interest and authority scope.

   A physical delivery ID consumed upon source observation stays consumed if an initially illegal receive later becomes legal. Conversely an independently qualified historical command resolver result is not discarded just because an old transport attempt used the same semantic command ID. Transport H1, command C42, effect/target incarnation, subscription interest and trusted model observation are not interchangeable identity classes.

   The runtime installs selected next semantic state before interpreting effects. Immediate nested callbacks see current state, and all later effects in the outer batch check their own still-current physical permit. An error or missing mandatory handler fails at its actual boundary; neither observer nor optional diagnostic is permitted to fabricate a successful semantic effect.

   The supported HTMX/asset profile must identify genuinely effective synchronous gates for protected sends and mutations. A handler that logs failure after HTMX has already performed an unauthorized swap is not conformance. If no supported gate exists, choose another qualified realization or reject that managed combination.

Adversarial-browser authority boundary

The protected server is exposed to arbitrary client behavior. An adversary may bypass the generated application, alter `hx-*` attributes and hidden inputs, forge plan/permit maps, replay command IDs, invoke endpoints directly, change its DOM, retain earlier received content, emit syntactically valid request bytes or choose unexpected ordering and multiplicity. A correct honest browser adapter is an implementation-quality guarantee; it is not a security boundary that the server may trust.

Under the named authentication, model, XTDB, cryptographic and deployment premises, the trusted boundary must enforce at least:

- **Principal derivation:** identify the authenticated actor using trusted credentials/session state, not browser-supplied role, Organization, Membership, permission or plan annotations.
- **Current guarded transitions:** validate typed operation/argument bindings and required current Request, active primary RequestAssignment, Membership and Organization facts at the transaction's authoritative conflict boundary. A pre-read is not an atomic guard.
- **Original-command binding:** qualify immutable command IDs, actor/operation/schema and canonical arguments, deduplication, compatible replay and allowed retention; a copied ID cannot confer command rights or mask changed inputs.
- **Read and Live disclosure:** authenticate every protected entity/collection read and subscription, with current scope, tenant and source/interest admission. Result readback may require **current** disclosure permission even for historically incorporated commands.
- **Provider evidence:** issue or qualify the command, observation, collection absence, scope and progression assertions actually justified by trusted model/backend operation.
- **Failure and privacy:** distinguish inaccessible from absent, and unknown from rejected, without leaking nondisclosable existence or claiming secrecy can be retroactively restored after an adversarial client has saved data.
- **Physical interpretation boundary:** client certificates, rendered affordances and DOM installation permits never replace trusted operation authorization, regardless of any browser-side assurance success.

The strong theorem, for its admitted server transition relation, is that no arbitrary syntactically representable client message can cause an unauthorized domain transition, protected disclosure or subscription merely by inventing browser-side proof/correlation metadata. The precise theorem must name authentication, model authorization, transaction atomicity, consistency, request schema and output-disclosure assumptions. A framework cannot certify arbitrary opaque model side effects merely by testing its HTTP dispatcher.

Client content safety and browser permissions are separate: even an authenticated authoritative observation may contain unsafe interpolated user content. Escaping, script/extension handling, response headers, OOB targets and active HTML policy follow the qualified 06.03 rendering/transport contract. An adversarial client may modify its own DOM, so the server security argument does not assert that hostile-client presentation stays honest.

An invalid endpoint check should not prevent the **positive** authorized operation from working. HumanHelp's independent acceptance therefore exercises real eligible Create, Claim, Unclaim, Done, Cancel and manager Reassign as their actual production model rules allow, plus negative competing-Claim, revoked-Membership and cross-tenant attempts. Rejecting all submissions, including authorized ones, is not product conformance.

Required conformance scenarios

The finished conformance program combines portable transition/property checks, application construction/ preflight checks, exact finite or stronger semantic proofs where applicable, trace/source correspondence, real browser/HTMX physical tests, trusted backend operation tests, and multi-node/recovery tests for supported profiles. Each layer states its actual test subject, assumptions and nonclaims.

For each required safety property, the suite uses an **eligible positive control** that demonstrably reaches the tested physical/model boundary and a nearby adverse case that reaches the same boundary with the decisive premise changed. A test that rejects every action because no handler was installed, no authenticated user exists, or no network response can be obtained is not a valid positive conformance test.

Test area	Eligible positive witness
Synchronous HTMX gate	An authorized bound request/swap executes once using a qualified cancellable profi
Main, OOB and response-driven effects	Authorized complete multi-target footprint installs with checked postconditions.
Physical issue and callback reentrancy	An issued timer/local effect can complete synchronously, correlated to committed s
Wrong-target and partial DOM effects	Current B2/observation permit physically installs its prepared representation.
Optional continuity and ownership	Valid focus/draft/height restoration runs for its current capture and target.
Valid command and timeout	Eligible C42 Claim reaches trusted RequestAssignment write and correct bound resu

Test area	Eligible positive witness
Independent model authority	Current authorized Request, actor/Organization and active primary assignment gua
Current card versus command history	A historical successful Claim and newer Reassign both have qualified evidence; the
Qualified negative membership	Complete Qopen or explicit exclusion proves R17 absent, and current target remova
One explicitly granted Refresh	Bob's current Q read occurs exactly once, installs covered O1/O2 state and retains s
Initial subscription and reconnect	Baseline/interest fence or qualified replay/current reacquisition covers missed W an
Query, principal or plan change	Current Q2 and supported P8/asset/host combination admit new owner with autho
Registration and resource cleanup	Current effect/interest registrations are exact and cleanup releases only their own p
Duplicates, early messages and conflicting outcomes	A legal delivery is consumed once and an identical qualified repeat follows declared
Authoritative acquisition and installation	Qualified provider response and real target postcondition advance only their actual
Actual trace and global refinement	Role-local, sender/delivery/global schedule and independently sourced physical/mo
Diagnostics and proof noninterference	Same canonical plan and inputs execute identically with a bounded observer presen
Capacity and liveness	Under stated quiescence, grant, fairness and availability, an eligible user acquires an
Hostile client	An eligible authenticated operation succeeds with correct model result and authoriz
Release and application closure	Every actual admitted route, browser asset, handler, worker, subscription and positi

Coverage generation varies message order, request and callback delay, nested completion, physical cancellation failure, OOB partial mutation, owner/resource reuse, read and write races, provider incomparability, query/tenant transitions, duplicate delivery, network partition and durable recovery expiry. It must not assume a single convenient concrete integer instance for an unbounded identity property. No test fixture may simply return the **desired** progression token, attest nonexistent physical completion or self-report a trusted backend result as its own evidence.

A good finite test suite can establish a range of actual executions under a precise host profile and fail on real mismatches. It cannot turn tested traces into a quantified inductive theorem. An exact finite model checker can establish its enumerated subject and assumptions, while a source/physical correspondence check must additionally bind those model events to the **real** installed code and external transaction/host behavior. A stronger claim requires an independent proof or justified abstraction and the identified trust premises.

Conformance evidence identifies the exact loaded runtime/library/configuration, application/deployment assembly, backend provider, renderer and physical handlers, observed source/extractor, relevant tests, known failures, unsupported paths and assumptions. Replacing a dependency or enabling a new HTMX extension may invalidate the profile: nominal version compatibility alone does not qualify a newly exposed DOM writer. Unsupported protected paths are rejected or explicitly accepted only under a weaker named guarantee, never quietly included in a strong certificate.

Assembly obligations and the browser realization invariant

The application constructor must account for the actual managed paths it emits and serves, not just a sample Hiccup function or one favorable demonstration route. A closed application inspection relates:

Assembly obligation	Required closure
Semantic operations	Model-owned operation definitions, roles, typed arguments, current guards, authoritative transaction/readbac
Executable projection	The exact Choreo plan/operation/observation mapping, installed interpreter, semantic preflight and known dy
HTTP and rendering	All relevant Ring/Biff/HTMX routes, response modes, alternate writers, templates, fragment roots, target/ope
Browser startup and assets	Actual entrypoints, compiled CLJS/JS modules, handler registrations, HTMX/extensions, service workers, cac
Physical resources	Timer/transport/continuity/optimism handlers, source registrations, exact target/owner incarnation, last-actu
Live/coordinator	Interest/subscription lifecycle, baseline/gap fence, applied query/grant policy, obligation retention, progressio
Recovery	Incompatible plan retirement, surviving command intent/resolver, unknown-result handling, current disclosu
Assurance subject	Invariants, exact finite model subject or stronger proof, actual source/executor/physical evidence, kernel/chec

The construction pipeline may use Clojure's full language and opaque application effects, but it must require the admitted contract for the effects and relationships that its guarantees rely upon. It need not statically predict future XTDB values, network delivery, DOM disappearance, which actor wins a competing Claim, or whether a target becomes inaccessible. It must, however, reject a missing mandatory operation/effect realization, inconsistent source or route binding, known unsupported protected host gate, invalid plan/protocol relation, or dynamic condition with no declared safe failure/recovery path when that path is required by the selected guarantee.

Preflight and deployed runtime have different jobs. Static admission establishes the complete declared construction, supported profile and required recovery contracts. Runtime admission checks the current principal, context, owner, source, applicable progression, target and effect at the actual boundary. Physical tests and trusted provider witnesses establish remaining observable facts where the stated assurance requires them. A successful static report alone does not justify executing against a changed runtime, or assert that a browser's actual DOM changed.

No silent downgrade is admitted for a protected managed affordance. If the selected projection, preflight, handler registration, qualified HTMX hook or trusted operation route is unavailable, the application must fail at the appropriate construction, startup or runtime boundary, not submit an unprotected command while reporting that the managed flow was enforced. Ordinary explicitly declared raw HTMX interactions remain supported under their own complete server authorization and rendering contracts.

1. Unified Part VI invariant

   > For every admitted managed browser action, the actual application has one selected executable semantic meaning and a current qualified owner, source, plan, effect and authority/observation scope. Its physical realization is declared for the installed host profile, and every protected act is prevented unless the applicable current permit remains valid at its controllable mutation/send boundary. All reported consequences are attributed to the actual issued work and suitably independent source/physical evidence. A request or trusted command result is not by itself completed DOM installation; acquired authority is adopted only for its genuinely covered and physically completed scope. Stale owner and provisional restoration rights expire before best-effort cleanup, without cancelling independently surviving command-resolution or Live obligations. Failed, partial, absent and unknown evidence remains distinct from trusted incorporation or rejection, and neither opaque basis arithmetic nor uncontrolled response effects may invent progression. Current trusted authorization remains mandatory even if an adversarial client ignores the browser entirely. Logical concurrency, physical resources, obligation retention and conditional liveness obey their own declared bounds and environmental premises.

   This is a **normative invariant of the finished supported realization**. Its applicability is established through independently checked obligations at the semantic, construction, trusted model, physical host, subscription and deployment layers. A real deployment claim must report which ones are discharged and which remain conditional, unknown or unsupported.

2. Whole-part reconciliation and independent acceptance remain distinct

   Completing the fifteen numbered Part VI sectional reviews is a necessary editorial milestone, not automatically accepted whole-part architecture or proof. The next reconciliation explicitly checks:

   - consistent identity and terminology for command C42, execution E42, physical H1/H2 attempts, effect, representation, target incarnation, coordinator, grant, authority context and deployed artifact;
   - consistent transitions from source admission to physical last-actuation, qualified acquisition, installation, adoption and unmet debt;
   - absence of contradictory HTMX control, OOB/response-effect, refresh/single-flight or shared-subscription claims;

- actual recovery/retirement ownership across 06.08–06.11, including durable command promises and resource bounds;
- R4 preservation coverage without accidentally restoring superseded illustrative APIs;
- Parts IV–V and VII–XII joins, especially HumanHelp positive product behavior, real installed application closure, provider correspondence and precise formal proof subjects;
- source/publication assembly, cross-part Org references, stable anchors, examples, trust-premise consistency, conformance matrix and explicitly unfinished implementation requirements.

An independent candidate-acceptance review then evaluates the **whole** reconciled Part VI, including whether the specified abstractions are compatible and sufficient for the finished-system target. Only that later decision should mark Part VI accepted. It is not obtained merely by counting reviewed files or successfully concatenating them.

Part VII — Whole-application construction

This part specifies how one source-owned application definition becomes an admitted construction and an actual serving application. It joins executable Choreo projections, trusted model operations, Clojure/ClojureScript handlers, server routes, generated browser assets, rendered controls, Live interests, authoritative acquisition, recovery, and the physical lifecycle. Parts I–VI define the meaning and ownership of those components; Part VII establishes that the required **connecting edges** are present and are enforced in the realization that actually runs. Parts IX–X provide the stronger proof and implementation-correspondence arguments and preserve their evidence scope.

Gesso applications retain ordinary Clojure, ClojureScript, server rendering, HTMX, and application-specific effects. The framework does not require all business logic to be written in a formal language or every local operation to use Choreo. Instead, construction identifies the interfaces, effects, producers, consumers, runtime dependencies, trusted premises, and observable boundaries on which each selected guarantee relies. A sound declaration of an opaque effect remains a named contract or assumption until its relevant implementation correspondence is established; calling it application code does not make its effects disappear.

Examples use illustrative operation and assembly names. A valid-looking inspection map, plan digest, source label or success report does not issue a runtime permission or trusted transaction credential. Present execution conditions, including the authenticated actor, current authorization, model facts, resource incarnation and installation permit, are checked at their proper runtime/model boundaries rather than inferred from construction.

Whole-application realization and preflight closure

A valid choreography, model transaction, compiled browser plan or Ring route can be correct in isolation while the combined application is broken. A rendered Claim button may not emit its declared operation; its shipped plan may be absent; a correct browser send may reach a different physical route; the trusted route may lack its required execution provider; the command may commit while no permitted authoritative acquisition or repair can establish the resulting display. Each local component can pass its own tests.

Whole-application closure requires every effect and observation edge needed by the selected application construction and assurance contract to have a compatible, source-owned realization. This checks the composition of the actual submitted application, not merely whether several independently constructed registries look plausible. The supported construction is an inspectable semantic/physical artifact with an identity, selected capabilities, trusted premises, supported host/deployment profiles, and specific closure and enforcement obligations.

Closure is **not** a second choreography. Choreo states which distributed behaviors are admitted; projection derives executable role-local behavior; application assembly establishes that each required concrete/browser/server realization is supplied. A route cannot legalize an invalid Choreo trajectory, and a valid plan cannot manufacture a missing route, model operation, subscription or generated asset.

The required edges depend on the kind of work:

Admitted application behavior	Realization that must be closed
Managed browser command such as Request Claim	Rendered affordance, projected plan, loaded browser/HTMX host, issued send, truste
Server-owned operation or background task	Registered event/worker, trusted operation and providers, applicable publication, dur
Ordinary read-only HTMX or search route	Actual HTTP/read source, current disclosure authorization, declared query/coverage
Local draft/filter/focus interaction	Current local state owner, presentation and continuity permissions, effective physica
Explicit ordinary mutating endpoint	Its declared model writer, authenticated input and current transaction guard, output/

No semantic metadata is required on irrelevant static assets. However, a managed route or physical writer whose effects can invalidate an invariant must belong to that invariant's coverage argument or remain an explicitly trusted and bounded environment premise. The application cannot claim whole-writer safety while a reachable unaccounted raw route changes the same RequestAssignment state behind its guarded Claim.

Three application claims remain independent

The authoritative Part I application-assurance claims retain distinct evidence subjects:

Claim	Required positive evidence
Declared structural closure	Every required operation, route, source, artifact, capability, observation, effect and recovery edge i
Actual production enforcement	Real installed handler/startup paths, issued transport and renderer/HTML/HTMX boundaries perf
Independent behavioral completeness	Eligible users can perform the required product operations using actual controls, trusted model eff

The three claims can be simultaneously true, false or not yet established in different combinations. A closed application can omit a required manager Reassign control. Production may bypass its checked response wrapper even though the assembly was valid. A test may manufacture C42 by calling a helper directly even when the real Claim control emits no command. These are distinct failures and cannot be flattened into one `:verified?` flag.

Application construction always requires its selected mandatory structural and real admission boundaries. Stronger proofs and finite-model campaigns are selected by an explicit assurance policy, not silently necessary for every ordinary use. Omitting an optional theorem cannot disable required route, actor, artifact, source or physical checks. Selecting an ambitious proof does not make a disconnected checker an enforcing gate; only an actual installed, correctly scoped boundary supplies enforcement.

Ownership of whole-Part-VII cross-section obligations

The Claim, Live and XTDB examples recur because they cross independently checked boundaries, not because each section declares another model, registry or policy. These sections own different **relations**:

Section(s)	Primary whole-application obligation
07.01–07.03	Application scope, source-owned assembly, typed closure and evidence graph.
07.04	Assurance policy, operation/exercise and physical writer coverage.
07.05–07.07	Actual trusted route/executor, built and served browser assets, effective rendered HTML/HTMX.
07.08–07.09	Command-specific settlement, Live/progression and required recovery handling.
07.10–07.11	Installed Biff lifecycle, actual serving adoption and physical fault-injection evidence.
07.12–07.13	Source-owned semantic audit, non-authoritative diagnostics and source-local repair.
07.14	Independent positive product acceptance and backend/evidence change qualification.
07.15	Qualified end-state security invariant and trusted authority boundary.

Part I owns the application-assurance claims, Parts II–III model/Choreo semantics, Part IV authoritative guarded transactions and command truth, Parts V–VI browser realization and physical continuation. Part VIII owns HumanHelp positive user acceptance; Parts IX–XII own stronger proof, physical refinement, multi-node deployment and release obligations. Part VII does not create duplicate semantic or proof authorities.

Complete **declared** operation coverage, independent promised product capabilities and actual protected writer/deployment completeness are three different inventories. The absence of an unknown raw writer cannot be proved by an empty declared registry; a selected weaker assurance profile cannot silently delete a known protected effect or manager Reassign promised by the independent product profile.

Coverage follows effects, not the presence of Choreo metadata

Coverage is relative to the properties, security and product claims made. The assembly identifies the admitted ingress, egress and state-changing effects through actual HTTP routes, model functions, background work, queues, scheduled tasks, administration, imports, initialization, migrations, Live publication, external systems and browser/local writers that may affect those claims.

An ordinary search endpoint can be the declared current authoritative acquisition for a board without being a command choreography. A preference update can use a simple guarded model operation. An explicitly raw HTMX endpoint may remain legitimate under its own complete authorization and effects contract. None obtains a silent right to bypass a selected whole-application invariant because it lacks `:choreo/op` metadata.

A sound assembly represents **known writers**, their model-affecting footprints, operation and route identities, required providers, publication and observation obligations, and any explicitly scoped external writers. Declared coverage is not a proof that a dynamically loaded plugin, unobserved scheduler, hidden route or unbounded external system is absent. Where discovery is incomplete, the report must distinguish an **unknown** inventory from an actually enumerated empty one. A stronger closed-world

claim may require controlled registration, an enforced runtime boundary, independent deployment inspection or an explicitly qualified assumption.

The closure graph has positive obligations for commands that succeed: trusted incorporation must have the declared settlement, affected-scope, publication, acquisition, adoption or repair path. Live interest without a permitted first acquisition cannot establish a current initial view; Live invalidation without a supported registration-gap recovery cannot promise complete continuity. A command path that is perfectly guarded yet always lacks its provider, or an entire screen that can never adopt its observation, is not a realized product capability.

Construction-time knowledge and current-world authority

The finished assembly validates every requirement whose meaning and inputs are available at construction, build, preflight or installation. For example, it can check a declared missing plan, absent trusted route, incompatible method/template, unsupplied required provider, stale browser artifact/receipt, ambiguous physical realization, uncovered output, unregistered source or undeclared recovery branch without waiting for a real user to encounter the defect.

Some properties cannot be decided when the application is assembled: the eventual Claim winner, the current principal, RequestAssignment eligibility at transaction time, the outcome of an uncertain send, the arrival of an SSE advisory, or the physical existence of a target after another execution supersedes it. Each belongs to its current trusted or physical admission boundary. A dynamic condition is acceptable only where a specified enforcement and recovery/rejection path handles it; it is not permission to replace a protected action with raw HTMX.

A production path is considered enforced only while the checked construction is still the one being used. The checked callable must be the installed callable, the verified asset must be served/loaded, and the actual response must pass the required pre-serialization or protected physical gate. Rebuild, handler replacement, route override, alternate response encoding, stale service worker and third-party extension paths must be closed by the qualified deployment or remain named unsupported/assumed profiles. Successful compilation or a copy of the expected report is not proof of these installed facts.

An application with arbitrary Clojure rendering cannot claim that a finite catalog of sample Hiccup values enumerates every future conditional control. It can still admit a current route/operation backbone and enforce the required Choreo/physical request check at every **actual canonical render**. Other response producers need their own guard, an enforced shared serving boundary or a weaker explicitly scoped claim. The absence of an observed unauthorized control in a small snapshot is not completeness evidence.

Assurance vocabulary describes the checked proposition

The report retains what was checked, for whom, where, under which version/coverage and what remains conditional. Useful distinctions include:

Evidence state	Interpretation
`preflight-closed` or `assembly-verified`	A specified declarative graph/construction was checked for its listed obligations at an identifie
`production-enforced`	Evidence ties the actual installed handler, physical effect, route or output boundary to the selec
`runtime/model-enforced`	Present authentication, authorization and guarded model transitions are checked by their trust
`behaviorally-demonstrated`	An eligible end-to-end product case exercised real user, backend, transport and presentation pa
`formally-proved` or `finite-checked`	Only the precisely defined theorem or finite model subject and its assumptions are discharged;
`inconclusive`, `unsupported` or `open`	A required premise is not established, the selected realization is unsupported or more evidence
`contradicted` (evidence)	A grounded observation contradicts the exact proposition; it is not automatically a rejected dor
`rejected` (admission)	The candidate or invocation is blocked at its applicable construction/enforcement gate; distinct

For instance, one application report may state that Choreo semantics are proved for a named finite/symbolic subject, the browser artifact correspondence is assembly-verified, the authenticated principal is runtime-enforced, and actual HTMX behavior is covered by tested host assumptions. It must not summarize those subjects as a single unqualified ”verified Gesso application.” A diagnostic label, digest or report has no power to authorize a business transition.

R4 preservation: broken compositions and positive controls

The following adjacent histories describe eligibility **and** the failure mechanism. They are not merely tests that all requests can be rejected:

Eligible application history	Broken co
A rendered authorized Request Claim control has a bound plan, shipped current runtime and trusted POST route.	The view
The current generated browser asset and receipt match the installed assembly and selected host profile.	A stale bro
A guarded Claim route has the required authenticated principal, clock/transaction provider and model-owned operation.	A missing
A qualified C42 incorporation produces covered publication/acquisition and physically adopted card or honest repair.	The comm
An initialized Live interest has a permitted baseline and reconnect/interest-fence recovery.	A board su
A canonical guarded Biff/HTML path validates the actual rendered control before serialization.	A later ha
An ordinary read/search HTMX endpoint is correctly authorized and responds as an ordinary route.	The same
A manager uses an eligible real Reassign control; another helper's competing Claim is rejected at the trusted model boundary.	Tests build
The assembly accounts for named background, scheduled, migration and administration writers affecting Request state.	A known
An application enables stronger finite/proof assurance with its own checked subject and runtime gates intact.	The check

Part VII's later sections specify the ApplicationAssembly shape, closure graph, concrete routes, renderer, artifact manifest, Live and recovery mechanics, lifecycle and diagnostic surfaces. This opening section establishes the admission **scope** and its three independent success conditions; it does not provide a second implementation of those later checkers.

Part VIII must exercise nonvacuous positive HumanHelp Create, Claim, Reassign, Done/Cancel, creator-local presentation, remote advisory with exactly one explicitly granted Refresh, and local search/selection/ scroll continuity as appropriate to the selected product contract. Parts IX–X must relate admitted declarative edges and physical occurrences to their actual formal and implementation evidence.

Application definition and ApplicationAssembly

One source-owned application definition composes the declarations that their real owners export. Its normalized realization is the conceptual `ApplicationAssembly`. It is a graph of typed subjects, bindings, obligations, and evidence, not a collection of unrelated success flags.

The public authoring surface should remain small. Source owners declare genuine choices once; construction derives repeated identities and verifies independently chosen physical correspondences. Applications should not maintain separate operation registries for browser compilation, preflight, startup, rendering, model checking, and tests.

Source ownership and derivation

Source owner	Authored facts
Domain/model module	Typed operation meaning, guards, dependencies, transitions, outcomes, command/recovery contract, opaque p
Choreography module	Distributed roles, permitted knowledge flow, coordination, local recovery choices.
View/component	Selected operation, concrete arguments/observations, presentation, availability choice.
Route/ingress module	Supported physical dispatch and input mapping.
Trusted execution adapter	Public model delegation and actual implementation/provider binding.
Acquisition/Live module	Projection/query scope, observation contract, subscription and refresh policy.
Runtime/build profile	Compiler/runtime, physical handlers, assets, supported host behavior.
Application composition	Selected modules, configuration, assurance policy, lifecycle/deployment profile.

An exported declaration is not accepted solely because it came from a namespace with the right name. Construction validates its semantic category and required source/implementation correspondence. A raw declaration, an admitted operation, an issued executor, and a copyable report can use similar host representations without being interchangeable.

The application definition must also identify the **coverage of its source inventory**: which declarations exhaust the selected deployment's known routes, workers, scheduled writers, renderer producers, external effects, and extension points; which are only sampled or discovered; and which remain unknown. A map with no registered writer is not evidence that no relevant physical writer exists. If a strong assurance policy depends on complete source or effect coverage, incomplete inventory is an open or blocking obligation, not success.

Model semantics, physical route selection, provider selection, rendered affordance selection and build/runtime installation are independently authored choices. Construction derives facts that have one semantic owner, checks each independent choice against that owner, and retains source provenance for diagnostics and change invalidation. Matching strings or plausible metadata must never convert a physical selection into semantic authority.

A schematic composition might be:

```
(defn application-definition [config]
  {:application/id :humanhelp/example
   :domains [request/definition
             organization/definition]
   :flows [request-flow/definition]
   :surfaces [request-card/definition request-board/definition]
   :ingress (routes/definitions config)
   :execution (server/execution-bindings config)
   :acquisitions (live/acquisition-definitions config)
   :providers (server/provider-bindings config)
   :browser-profile browser/profile
   :lifecycle (system/lifecycle-definition config)
   :assurance-policy assurance/managed-application-policy})
```

Each referenced module exports its actual owned declarations; the example does not stand in for omitted definitions. The composition does not retype operation IDs, argument schemas, settlement variants, or plan digests already determined by those declarations. Independent physical route selection and provider selection remain real choices that must be checked.

Tests instantiate this constructor with declared fixture configuration. They can replace a clock or storage provider for a scoped test, but must identify that substitution and avoid transferring its evidence to an unexamined production provider. A hand-built shadow assembly is not production-closure evidence.

Worked Claim source-owner join

The following is schematic application composition. The referenced Vars and the precise constructor spelling are illustrative, not claims about the current public Gesso API. Each module exports its own actual declarations:

```
(defn claim-application [config]
  {:domains [request-model/definition organization-model/definition]
   :flows [request-choreo/definition]
   :surfaces [request-card/definition]
   :ingress (request-routes/definitions config)
   :execution (request-server/operation-bindings config)
   :acquisitions (request-live/definitions config)
   :providers (request-server/provider-bindings config)
   :browser-profile browser/profile
   :assurance-policy assurance/managed-application-policy})
```

For the `:request/claim` slice, construction checks the following distinct facts instead of recopying one operation ID into six registries:

Owner	Required relation and rejected near miss
Request model/Choreo	The admitted Claim targets the current Request and RequestAssignment under trusted principal, Membership an
Request-card view	A real eligible control supplies the operation, typed Request argument and required observation/command bindi
HTTP route	Actual POST method, path template and decoded path Request match the command's Request target; a matching
Trusted executor	Installed callable implements the model-owned command and recovery contract; an entry naming a different fun
Providers	The actual clock and principal have the value shapes required by the executor, including `:biff.fx/now` when that
Browser and Live	The loaded generated plan/physical handlers, authorized reread, progression and subscription/Refresh policy join

One command identity persists through permitted delivery uncertainty and retry; a later current card observation is not by itself historical incorporation evidence. Current authorization, atomic execution and actual DOM installation require their own runtime evidence. The Claim illustration cannot stand in for separately required Create, Unclaim, Cancel or manager Reassign paths.

Checked construction supports opaque application code

An ordinary Clojure function can implement a rule, renderer, projection, candidate builder, or input conversion behind a declared interface. Construction establishes what can be checked and retains a named premise for the rest, under the admission policy.

For a candidate builder, the actual returned targets, assignments, frame, command binding, and supported effect footprint can be checked before submission. That instance check does not prove that the builder had no hidden earlier effect or that it produces valid candidates for every input. Effect containment and functional conformance remain separate enforcement or assumption obligations.

Normalization must not silently execute arbitrary application work to discover its meaning. Functions invoked during construction have an explicit purity, determinism, totality/failure, and resource contract. Compilation, dependency resolution, and model exploration use their declared budgets. An effectful builder cannot be called repeatedly by inspection or diagnostics as though it were inert data.

The boundary between a checked **instance** and a function's general behavior is retained in the report. A pure bounded export may be evaluated under its admitted inputs; an uncontained effectful callback cannot be probed for inspection as though it were data. Exception, timeout, unknown dependency, nondeterminism or undeclared side effects produce explicit dispositions rather than an empty declaration. Agreement between a callback's own generated assumptions and its reported result is not independent evidence.

A canonical admitted construction should derive execution and checking products from the same semantic subject wherever possible. An independently supplied reference model remains useful, but its relation to the production operation is an explicit obligation. Agreement between two copies of the same mistaken policy is not evidence that the intended business rule was correct.

Closed registries and explicit extension boundaries

Registries declared closed reject unknown, duplicate, ambiguous, or incompatible members before use. Extension points identify how new modules, routes, operations, or providers become a newly admitted construction. Dynamically loading a plugin must not mutate an admitted registry while retaining old closure evidence.

Closure is scoped by registration source, deployment profile, effect family and inventory boundary. A closed declared route table does not imply an exhaustive inventory of background writers. A validated collection of rendered Hiccup snapshots does not prove all future conditional responses. Where a stronger claim needs that coverage, the system needs independent inventory evidence, a real producer/effect-boundary guard, or a truthful weaker disposition.

Adding or replacing a plugin creates a candidate new generation with new admission and compatibility decisions. During rolling installation, older requests and workers retain their previously admitted generation; loading new code cannot silently grant them the new generation's rights.

Unused declarations can be available for composition without being exposed or active. They are labeled accordingly. A generated executable or installed writer that claims a role in the active application must have a corresponding source and coverage classification; it cannot disappear from the graph as unused merely because no UI currently invokes it.

Construction phases and admitted assembly identity

Construction has an acyclic dependency order. It may expose several internal products without asking application authors to maintain several definitions.

Phase	Product and admission responsibility
Normalize	Resolve source-owned semantic declarations, physical choices, configuration, and explicit opaque contracts.
Derive	Produce required role plans, interfaces, closure graph, proof obligations, and the browser build manifest.
Build/check	Generate assets and correspondence products; run applicable semantic, closure, qualification, and assurance checks.
Admit candidate	Bind accepted evidence and retained premises to an immutable candidate realization under the selected policy.
Install	Construct actual router, wrapped handler, providers, worker entrypoints, and asset serving from that candidate.
Activate	Verify the required installation relationships and permit external application service.
Retire	Revoke or transfer owned rights/work, drain permitted users, and release resources under the lifetime contract.

Some installation facts cannot exist before installation. The lifecycle therefore allows private preparation and a final activation gate, while preventing unchecked application operations from becoming externally available. A report saying a candidate is structurally closed is not a report that its server was activated.

The activation gate verifies what was actually installed or bound for serving: the dispatcher and checked handler, executable and served assets, provider and worker registrations, required acquisition paths, and the selected deployment's physical applicability premises. A constructed router value alone does not establish that the serving host adopted it. An unsuccessful build, partial installation or rejected activation must not expose unchecked operations; already acquired resources retain identified cleanup, cancellation and surviving physical-remnant obligations. Repeated startup must not silently duplicate workers or reuse another generation's installation receipt.

The identities used in these phases identify different objects:

- normalized semantic/realization input identity;
- concrete implementation closure and qualification-profile identity;
- exact executable plan identity;
- browser manifest identity;
- emitted asset content identity;
- checker result/evidence identity with its applicability;
- admitted assembly instance, optionally described as an assembly generation;
- deployment instance/placement identity.

An assembly generation is a scoped application-construction lifetime. It is not a universal freshness number, browser execution ID, resource incarnation, command ID, or database progression token. Examples use `:assembly-instance/id` when referring to that admitted lifetime.

A digest recipe specifies the object, canonical encoding, schema, relevant compiler/provider dependencies, excluded self-identifying fields, and collision premise. Formatting-only source changes need not alter semantic identity, but a changed opaque implementation can invalidate implementation evidence even when its public contract is unchanged. Part V's exact plan and asset distinctions apply here.

Identity comparisons are typed relations. Equal semantic identities may coexist with different handler implementations; equal artifact bytes can be installed under different execution permissions; distinct deployment instances can use one generation. Evidence is reusable only when its **complete** dependencies, mechanism and applicability premises remain valid. A digest cannot issue a trusted executor, prove artifact provenance, or establish physical serving. The normalization contract must detect omitted dependency fields and ambiguous encodings, not merely select a digest algorithm.

The final asset digest cannot be an input to the same bytes containing that digest without an explicit cycle-breaking construction. Detached receipts and content-addressed asset locations provide a straightforward solution.

Immutable admitted bindings and scoped revalidation

An admitted assembly binds the actual implementations, providers, policy, qualification profiles, route semantics, rendering contract, and artifact set on which its claims depend. A request captures that assembly instance at entry. Its later callbacks, rendering, and response completion preserve that reference.

Capturing a generation never freezes changing principal authority, live RequestAssignment state, transaction progression or physical target/resource ownership. Those are re-established at the relevant current effects. An incompatible deployment may permit an old request to finish only under its explicit retirement policy; otherwise its obsolete physical actuation rights must be revoked and any surviving durable command resolved through compatible recovery. A callback must not splice a new route, plan or provider into an old continuation merely because names still match. Subscriptions and background workers require corresponding generation and resource lifetimes.

An immutable map containing mutable Vars or registries is not automatically an immutable realization. The supported construction captures applicable callable bindings, uses assembly-owned registries, or detects and re-admits relevant replacement before dependent use. An ordinary reload is a declared construction transition, not an arbitrary mutation excused by development mode.

Construction evidence may be cached by its complete dependency identity and applicability conditions. There is no requirement to recompile plans, explore a finite model, or hash every asset on every response. The runtime must preserve or re-establish the relevant binding at the boundary where it is used. A mutable asset path or live registry needs a concrete stability/invalidation mechanism; a cached green report cannot supply one.

Admission handles can be opaque process-local values; explanatory reports remain copyable data. A private issuance registry can prevent accidental substitution of a copied map for an issued executor. Object identity alone is not durable proof, remote attestation, or a sandbox against hostile code sharing the JVM. Process restart reconstructs local admissions from their actual sources and any independently checkable durable evidence.

Positive and adversarial construction histories

History
An eligible Claim has source-owned model, control, POST, issued executor, correct clock, built-and-served browser asset and authorized ca
A route and command use the same operation name but select different Request IDs.
A provider is named `:clock` but the executor never receives the required `:biff.fx/now` value.
Sampled Hiccup cards are valid, but a conditional render emits an unannotated POST to a protected command route.
An unlisted background writer can mutate protected RequestAssignment state.
A plugin adds a route or a mutable Var points to a new handler after admission.
A delayed callback belongs to a retired generation or superseded browser target.
A fixture constructor passes with test providers but production uses another actual handler.
A candidate is checked but the serving router never adopts its guarded handler.
A finite checker exhausts its budget or cannot express the selected obligation.

These are scoped construction requirements, not a demand to prove every ordinary Clojure function. Plain Clojure and explicit ordinary HTMX remain legitimate while their real effects must respect any application-wide safety property being claimed. Later sections define the detailed closure edges, serving enforcement, diagnostics and positive acceptance criteria.

Whole-application closure graph and evidence

The graph represents semantic interfaces and actual realization relationships. It must expose enough structure to locate a failure without forcing a reviewer to reverse-engineer many namespaces. The runtime need not carry the entire graph on every request; it uses admitted bindings derived from it.

The graph has **typed nodes** for semantic sources and obligations, selected application realizations, installed physical bindings, and the propositions whose evidence can support admission. These are not interchangeable merely because all happen to be represented by Clojure maps. Each node identifies its source owner, semantic category, scoped identity, admitted generation where applicable, and the implementation or deployment profile against which it is being checked. A generated browser plan and the bytes that were actually served are distinct nodes with a correspondence edge between them.

Edges identify a **relation and its justification**, not only the existence of two endpoints. Construction derives required edges from each admitted operation's interface, the independently specified application capabilities, and the selected assurance/deployment policy. An explicit source declaration and an observed physical binding can disagree; neither may silently rewrite the other to make the graph close.

The realization graph may contain legitimate feedback paths such as command settlement, Live advisory and authoritative reacquisition. Its **evidence dependency argument**, however, must be well-founded: a report cannot certify its own source binding, and two success summaries cannot mutually establish one another without an independently admitted base. The proof-DAG layer of Parts IX–X records these dependencies; Part VII requires their ownership and non-circular applicability already at assembly time.

Operation-specific edges

Edge or obligation	Required correspondence
Affordance to operation	Concrete arguments, observations, selected realization, and availability implement the named o
Operation to choreography/projection	Required role-local behavior follows the admitted coordination and knowledge contract.
Plan to executable artifact	The generated runtime contains the intended admitted plans and bound implementation/profile
Executable plan to physical handlers	Every reachable required effect has a compatible handler and completion/failure protocol.
Transport to ingress	Method, route semantics, encoding, selectors, correlation, and protocol agree.
Installed ingress to executor	Actual dispatch invokes the selected issued/admitted callable, not a metadata-equivalent substit
Executor to public model operation	The trusted operation binding implements the declared model interface and preserves its comm
Operation to providers	Required typed resources resolve to compatible providers with the right origin and lifetime.
Candidate to authoritative realization	Checked targets, guards, assignments, frame, and effects reach the admitted lowering/submissio
Model outcome to participant settlement	Each supported outcome maps explicitly without collapsing incorporation, rejection, repetition,
Required propagation to recovery	Publication/durable work, progress requirements, and missed-delivery recovery have actual ow
Observation to presentation	Scope, progression, authorization, representation, target ownership, and physical completion su
Application policy to enforcement	Mandatory checks are invoked at the actual construction, installation, request, effect, and outpu
Product capability to acceptance	Independent promised behavior is exercised through the admitted application; closure alone do

Not every operation needs every edge. The operation's admitted interface derives its obligations. A read cannot require a fictional mutation just to fit a universal pipeline, and a mutation cannot omit recovery because it looks like an ordinary POST.

Graph coverage, orphan nodes and independent capabilities

The assembly checks **both directions**: every required semantic effect has the appropriate realization, and every installed realization that claims a semantic/effect role is connected to an admitted contract and classified coverage. A generated plan with no source operation, a semantic route with no registered executor, a Live acquisition without its managed projection, or an installed state writer outside the admitted effect inventory cannot be hidden by reporting only successful edges. A declared but unused component may be classified `available-not-active`; an installed writer that changes protected state cannot use that classification to escape global invariant obligations.

A useful closure explanation distinguishes at least:

Subject	Required graph disposition
Exposed managed operation	All reachable semantic, physical, settlement and recovery edges requi
Ordinary read-only HTMX route	Its real transport, authorization, data/projection and response contrac
Ordinary state-changing route or non-HTTP writer	Its physical effects are classified and checked for interaction with prot
Installed worker, provider, browser asset or subscription	Its source ownership, effect scope, lifecycle and actual deployment bir
Available but inactive declaration	Not counted as a deployed behavior or as positive product acceptance
Unknown source, unclassified writer or incomplete host inventory	Remains an explicitly scoped coverage gap; silence or an empty suppl

Closure is always relative to a stated inventory, effect footprint, deployment generation and policy. An operation whose **required** edge is missing fails the corresponding admission gate; where the environment cannot establish exhaustive physical inventory, the stronger all-writer or production-enforcement claim remains open rather than being inferred from the discovered subset. Section 07.04 owns the complete application assurance policy and its writer-coverage obligations.

Evidence is attached to a proposition, not a prestige label

A checked edge records its subject, expected relation, actual binding, source ownership, mechanism, premises, coverage, applicability, and result. These are the assurance dimensions from Part I.

An illustrative inspection row is:

```
{:obligation/id :claim/installed-route-to-executor
 :subject {:assembly-instance/id assembly-id
           :route/id :route/request-claim
           :operation/id :request/claim}
 :expected {:executor/ref admitted-claim-executor-ref}
 :observed {:installation/ref route-installation-ref}
 :mechanism :issued-dispatch-construction-check
 :evidence/ref checked-binding-evidence-ref
 :status :established
 :scope :this-installed-dispatcher
 :premises [server-adopts-dispatcher-premise
            implementation-closure-premise]}
```

The checker derives this result from the relevant construction/installation. Writing `:status :established` into a map does not satisfy the obligation. A handler's operation name proves neither its body nor adoption by the real server.

The illustrative row is a **diagnostic view** of a checked proposition, not a public constructor for trusted evidence. The original sources, selected implementations, independent observations and actual checker mechanism remain separately identifiable. For example, a route-to-executor report can establish that the issued dispatcher closes over a particular callable in the current construction; it cannot also assert that the deployed web server uses that dispatcher, that the handler has no additional side effects, or that the current caller is authorized. Those require separate evidence or named premises at their actual boundaries.

For each edge the report records whether the relation was required, how its expected and observed values were obtained, the exact identity and lifetime of each physical participant where relevant, the evidence's validation mechanism, the checker and input versions, and the original source/dependency subject. A copied success report or forged discriminator does not reproduce a live issued binding. A source-derived explanation can be retained after admission without conferring the permissions of the admitted executor or current physical target.

A report distinguishes unresolved structural edges from retained runtime or external premises. Structurally arranging a trusted principal provider can close an edge; the actual principal's current authorization remains a runtime/model condition. The report must not list every future authorization decision as an unfinished build task.

The same temporal discipline applies to command outcomes and presentation: an issued command, a submitted XTDB transaction, a pinned historical readback, a current authorized Request-card observation, and completed physical DOM adoption answer **different propositions**. A branch receipt, callback label, HTTP 200 or current-looking card cannot certify all five. Application closure requires a lawful path to obtain each required fact; the actual fact remains scoped to its source, time, authorization and physical observation.

Checked, unsupported, inconclusive, and contradicted are distinct

The admission policy identifies which obligations are mandatory, which require a particular evidence mechanism, and which may remain named assertions. A missing route, mismatched executor, invalid schema, wrong authority source, or missing required recovery cannot be excused by adding a trusted flag.

Unsupported proof fragments and exhausted checking budgets do not establish falsehood or success. They receive explicit dispositions under the policy. A known counterexample to the same scoped claim cannot be relabeled an accepted assumption to make admission green. Correcting an abstraction, narrowing a claim, or fixing implementation requires a new explicit check of the changed subject.

Claims such as assembly-verified, runtime-enforced, exact-finite-checked, and proof-kernel-checked describe different evidence at different scopes. They do not combine into an unexplained percentage or one universal verified Boolean. The permanent executable semantic core remains distinct from the proof kernel and from dedicated correspondence/conformance checkers.

Evidence dispositions and safe composition

For a proposition and its exact scope, the evaluator distinguishes: `established` with retained assumptions and mechanism, `open` when required evidence has not been obtained, `unsupported` when the selected checker cannot express the obligation, `inconclusive` when an attempted check did not finish or exhausted its declared budget, and `contradicted` when grounded evidence establishes a counterexample to the proposition. The vocabulary in a concrete report may be more specific, but it must preserve these semantic distinctions. An unsupported theorem cannot be made established by an application-side label; an explored counterexample cannot be discarded merely by lowering the assurance policy silently.

Evidence dispositions, assembly-admission results, Part IV model command outcomes, browser participant settlements and HTTP statuses are distinct type domains. An evidence `contradicted` or assembly `rejected` is not an authoritative `:rejected` Claim. A correctly rejected Claim may instead demonstrate a working runtime authorization guard. Browser `:accepted`, `:rejected` and `:unknown` remain participant-relative presentations under 07.08's explicit evidence/disclosure mapping.

Evidence composition uses **matching subjects and applicability**, not just matching successful statuses. A finite model pass supports its bounded state/transition proposition; conditional induction supports the exact declared abstract catalog and retained premises; an in-process handle supports its specified source/issuance identity; and observed XTDB or browser behavior supports only the checked physical occurrence and profile. The composite claim records every essential dependency, the unproved bridge between independently authored model and implementation, and any unknown writer, deployment adoption or host-trust assumption. There is no transitive promotion from structural closure to universal runtime correctness.

A sound failure report points to the minimal broken relation and its owning source without presenting an inferred repair as a completed one. A missing plan is not repaired by changing a server-side operation name. A wrong HTTP target cannot be repaired by adding a browser success marker. A model execution with an unknown outcome cannot be relabeled rejected because the current card no longer shows the assignment.

A Claim realization closes more than its happy path

For HumanHelp Claim, the source-owned model operation protects current Request state, helper Membership, Organization policy, and their association under the Part IV command protocol. Assembly must connect that operation to all required browser and trusted-server paths.

Concrete requirement	Failure it prevents
Generated Claim binding includes typed Request arguments and the admitted observation context.	A visually enabled control lacks the i
The actual browser asset loads the matching plan and physical handlers.	Startup succeeds but clicking the butt
The real request carries the selected command envelope and stable `:gesso/command-id`.	A working `hx-post` reaches the route
Path Request selector and decoded command argument are checked for agreement.	The URL selects one Request while th
Installed route invokes the bound public operation through its admitted executor.	A similarly named handler performs a
Provider binding supplies sampled time in the representation the executor consumes.	A declared clock exists but the requir
Model outcomes have explicit participant mappings and disclosure checks.	Repetition after a successful commit i
Unknown has a current reconciliation owner and compatible resolver path.	Lost response strands the command c
Scoped acquisition and physical installation close the presentation path.	HTTP 200 is counted as a current car
Required Live propagation and reconnect recovery close remote participant behavior.	The initiating user succeeds while oth

The explanation derives these rows from the admitted construction and its evidence. A shorter example that omits the resolver, physical completion, or publication recovery must identify its narrower scope instead of being labeled a complete application proof.

Required positive and adversarial graph histories

Each case is evaluated against the same source-owned assembly, not a manually constructed substitute graph. The evidence mechanism and negative case are independently identified; supplying a prefilled report is not a test.

Observed history or construction
An eligible helper can render and execute Claim through the actual admitted browser plan, trusted request, guarded transaction, authorized
The generated artifact lacks the selected Claim plan, despite a valid choreography.
The POST route exists but selects Request R2 while the command and card select R1.
A server realization advertises `:clock` but never supplies the required `:biff.fx/now` value.
The model commits Claim, but no authorized Request-card acquisition or compatible unknown-command resolver is installed.
A Live advisory arrives but the remote user's one-Refresh grant has already been consumed.
The current card no longer displays the helper after a later Reassign, while an earlier Claim remains historically incorporated.
An apparently unused generated plan or installed write route has no admitted semantic owner, or a scheduled writer is missing from the de
A static render sample passes, but a conditional response later emits a protected unannotated HTMX action.
A finite checker times out, or a known source-bound counterexample contradicts a claimed invariant.
A production server replaces the checked handler after preflight or serves an older JavaScript artifact.
An ordinary authorized read or a server-only workflow has no browser command plan.

These histories preserve the R4 requirement that the same Claim can fail at different edges for different reasons. They also prevent an error-free but incomplete graph, a reject-all construction, or a certificate-shaped report from being mistaken for a useful and fully realized application. The formal policy and physical enforcement of these edges are developed in 07.04–07.15.

Application assurance policy and operation coverage

The application definition contains one explicit assurance policy used by construction, installation, and activation. Stronger finite, inductive, or correspondence checks may be optional under one policy and mandatory under another. Their selection is visible in the admitted subject and explanation.

A low-level compatibility API may preserve an unassured legacy path, but that path cannot inherit the managed application's enforcement or theorem claims. The ordinary managed startup path does not silently treat an omitted policy as permission to skip required checks. It applies the declared default policy or rejects a missing required selection.

Policy decisions are bound to subjects and effect boundaries

The policy is a source-owned selection of required **propositions**, evidence mechanisms, qualification profiles, and failure dispositions. It is not an unstructured `:strict?` boolean or a list of checker names that might run. Each mandatory proposition identifies the assembly generation, operation and effect scope, relevant implementation/source identity, proof assumptions, and the lifecycle boundary where its evidence must be current.

Boundary	Policy responsibility
Source/assembly admission	Reject unavailable mandatory semantics, missing declared owners or an incomplete required catalog; reta
Model checking	Require the selected finite, inductive or other proof obligations with stated bounds, soundness premises,
Installation and activation	Verify that the required checked dispatcher, assets, handlers, providers and writer restrictions are actuall
Invocation and protected effect	Revalidate dynamic authority, current provider/command binding, relevant physical target and commit-t
Post-execution observation	Attribute results, record physical/model correspondence or contradiction, and trigger declared repair wit
Positive application acceptance	Verify independently required real capabilities and recovery histories; a closed reject-all application does

A selected strict profile cannot request a guarantee for which its implementation has no supporting mechanism and then quietly lower that requirement. It rejects admission or makes the limitation explicit under a separately selected weaker profile. Unsupported, inconclusive and contradicted mandatory obligations remain distinct. A compatibility/unmanaged path may be explicitly supported, but must advertise its narrower assurance and must not inherit managed operation credentials, protected effects or certification labels by resemblance.

One construction owns the semantic catalog and its checking products

A useful construction binds:

- the admitted application assembly;
- the exact relevant operation catalog and semantic definitions;
- rule/invariant declarations with their source ownership;
- actual executor/handler implementations;
- finite model families or inductive evidence where requested;
- the comparator and observation contracts used for implementation evidence;
- required exercise/coverage gates and their budgets;
- the installation and startup path consuming that construction.

Later comparison cannot accept replacement model callbacks or an unrelated catalog while retaining the old admission handle. Diagnostic inspection reports the already checked subject and results; it does not rerun user callbacks to invent a fresh explanation.

This unified construction is an ownership discipline, not a requirement for one large function or public record type. Layered internal constructors can provide separate products while preserving their common sources and dependency bindings.

The admitted catalog is an **identified closed subject**, not whatever a test chooses to enumerate. Each rule records its domain and source owner; every operation's abstract meaning and required authoritative writer correspondence are distinct from the executable handler body and its physical write capability. An independent reference model can check a mistake in the production rule, but a pair of identically wrong rules does not establish the intended business policy. Unproved equivalence between a modeled callback and an issued executor is retained as an obligation, not silently inferred from matching IDs.

Construction retains a dependency argument joining admitted initializers, operations, effects, authorizations, rule/invariant subjects, checker implementation and deployment qualifications. Later tests use that subject. Substituting different families, predicates or host registrations after a certificate/report has been produced cannot reuse its admission. Reports are not capability-bearing replacements for the checked construction.

Finite operation families and non-vacuous coverage

A finite exploration may expand registered operations over declared principals, arguments, states, and choices. The expansion must preserve the mapping from each modeled occurrence to its actual operation contract. Synthetic model IDs are diagnostic coordinates, not production command or execution identities.

The report records the declared bounds, resource budget, visited states or state occurrences as appropriate, attempted transitions, remaining frontier, and whether the requested exploration completed. A depth bound can define the intended finite problem; exhausting a computation budget before exploring that problem is inconclusive. Completing a depth-limited exploration does not establish unbounded invariant preservation.

Coverage dimensions remain separate:

Coverage question	What satisfying it establishes
Every relevant registered operation has a family.	No catalog member was omitted from that family mapping.
A rule matches a source in an explored state.	The rule's modeled domain was exercised at least there.
An operation has an enabled attempt.	The model's enablement relation selected an attempt; its incorporation guard may st
A required committed/rejected branch occurs.	That modeled outcome was exercised within the declared exploration.
A modeled path preserves an invariant.	The checked path/model satisfies the stated proposition at its scope.
A real observed invocation matches a modeled step.	The admitted comparison establishes its stated local correspondence, with retained p

A policy can require particular rules, operations, and outcomes to be exercised. These gates prevent vacuous green reports, but do not establish product usability, all predicate branches, representative domain completeness, or production execution. A rule with no matching modeled source is reported as unexercised; it cannot be counted as a substantively tested business policy.

Nonvacuity is a vector of independent checks, not one coverage percentage. For every named operation, the explanation can retain counts of disablement, enabled attempts and committed/rejected modeled outcomes, with each observation joined to the **original** registered operation and its finite principal/argument choice. A disabled action is not a rejected transaction. A rejected attempt is an attempted operation, but not a committed transition. A finite model that visits only the empty initial state may be exhaustively explored within its stated bound and still fail required Claim-positive exercise.

The complete report also identifies which chosen principals, arguments, guards, branches and initialized states were **not** covered, when that is knowable. It must never infer that the chosen fixture domain represents all possible users, Requests, Organizations or membership races. A bounded model tree is not an observed production execution trace, even where its terminal state resembles a physical document. If an exercise gate is required and its operation or outcome was not reached, an independent successful invariant check cannot waive the failed gate.

Application authors should not write separate theorem-prover programs simply to use these checks. Gesso derives supported model/checking inputs from its semantic construction. Explicit reference callbacks or external models remain possible, with their translation and correspondence premises visible.

Induction is scoped to a catalog and its realization

A conditional application invariant requires an admitted initial-state argument and preservation by every relevant transition in the scoped catalog. Adding a new operation or writer can change that obligation even if no existing operation body changed. The evidence dependency graph records the actual catalog, rule, backend, and implementation subjects.

An abstract catalog proof is not automatically a physical database proof. Applying it to production also requires the stated initialization, writer coverage, input/state mapping, atomicity, frame, and transition-correspondence premises. Part IV defines the authoritative realization obligations; Parts IX and X define how stronger evidence composes.

The target inductive safety argument has the form: every admitted initial state satisfies `I`, and for each relevant operation, `I(s)` together with its admitted transition premises implies `I(s')`. Its quantification is over the **complete claimed transition catalog**, not over only the operations that happened to be enabled in one finite exploration. A guard that is always false makes preservation vacuously true; such a result does not establish the positive behavior required by HumanHelp. Fairness, liveness, progress, resource boundedness and eventual user Refresh/adoption require separate named assumptions and arguments.

Applying the abstract invariant to actual XTDB state demands a justified mapping for initial state, principal/arguments, transaction-internal prestate, guarded commit, all relevant physical writers and observed poststate. Concurrent transactions and multi-step external effects do not become one serial transition merely because an abstract step is atomic. A counterexample, unclassified operation or invalidated catalog requires repair or a separately justified narrower claim, not deletion from the proof subject to recover a green report.

A selected strict assurance policy must be invoked at the real candidate admission and installation boundaries. Calling a checker in a test or exposing an optional function named require-policy does not enforce that policy in production. The installation result identifies which gates actually ran and which checks remain required at invocation time.

Evidence must retain original subjects and provenance

An issued executor can be bound to a checked semantic source and its actual implementation. An issued handler can then be constructed to call exactly that executor after trusted request conversion. This provides stronger structural correspondence than attaching an operation name to an arbitrary function.

The binding does not prove the correctness of opaque request conversion, output rendering, backend behavior, or arbitrary extra effects. Those contracts remain explicit. A wrapper around an issued executor does not automatically inherit its issuance identity or all its guarantees; supported composition records the new wrapper's role and effects.

A report copied from a genuine check is still a report. Admission consumes the required original source/issued product or independently verifies a durable certificate under its declared checker. It does not accept a caller's success map as a substitute. Evidence about one callable cannot be transferred to another because their names or displayed metadata agree.

Writer coverage includes routes and non-route work

The coverage graph joins the semantic catalog, admitted executors, installed ingress, and actual effect providers. It distinguishes at least:

1. declared operations and their permitted effects;
2. registered/issued executor implementations;
3. installed HTTP and non-HTTP entrypoints;
4. observed invocation paths at the stated scope;
5. physical submission capabilities and any independent writers.

Matching operation ID sets is useful but does not prove semantic equivalence. A host-provided inventory is evidence about what it enumerated, not proof that it omitted nothing. Independent observations and effect containment can strengthen coverage; the remaining exhaustiveness premise stays explicit.

The graph must not require every writer to be used by an HTTP route. Queue consumers, scheduled jobs, migrations, and initializers have legitimate entrypoints and different lifetimes. Each relevant writer has an admitted owner/path or an explicitly classified external role. Conversely, a route can invoke several admitted operations if its choreography/effect contract permits that composition. The operation graph is not a universal one-route/one-writer bijection.

An unclassified writer blocks a claim requiring closed writer coverage. Under a policy that permits a limited claim with external writers, their permitted interference and its effect on dependent invariants are named premises. A generic label saying external must not conceal a known contradictory mutation.

Protected-state closure covers physical write capability

The protected write domain is expressed in terms of the actual entities, tables, fields and invariants the application claims to preserve. An accurate operation-ID registry is still insufficient if a Biff handler, queue worker, migration, initializer, raw XTDB call, external service or retry can reach the same protected state through a different writer. An installed capability inventory describes which paths can **potentially** mutate protected targets, including broad full-document replacement, erasure or SQL effects; a claimed write footprint does not itself enforce a restriction on the physical sink.

For a stronger all-writer claim, the system must either restrict the real write capability at a boundary that every relevant path actually uses, or establish correspondingly strong independent evidence of exhaustive writer inventory and permitted interference. An instrumentation callback that observes only the operations developers registered cannot certify absence of an unregistered writer. Source-relative host enumeration, path/handler identity, target-scope declarations and operation-ID equality are useful intermediate evidence, but their scope and all unverified completeness premises remain visible.

A selected invariant can survive **some** additional writers if their actual effects are shown to preserve it under the declared interference model. Neither a global raw-writer prohibition nor an assumption that every route is managed is universal. The application instead declares its supported writer policy and refuses to attribute the stronger invariant to effects outside that policy. No trusted report map can grant itself physical submission rights or prove a real XTDB gateway has excluded bypasses.

Request-scoped and work-scoped submission

The normal managed path gives candidate code the resources needed for preparation and gives the admitted submission boundary the scoped effect capability. A raw XTDB client need not be available to every handler merely because the application uses XTDB.

A request has its own invocation identity, current principal binding, selected assembly instance, issued effect resources, and completion/retirement protocol. Fresh grants are issued for subsequent requests. A one-use observation or writer handle must not become a production route that works once and then rejects every later valid invocation.

A process-wide mutable slot that installs one request's callback and rejects unrelated concurrent requests is not the general concurrency architecture. The normal design uses request/work-local contexts or an equivalent isolated registry with explicit ownership. A qualified serial controller can remain a supported limited profile where the limitation is intentional and visible.

Two simultaneous valid Claim attempts must be allowed to reach independently owned admitted request contexts without stealing each other's callable, principal, source observation or one-use writer right. Their eventual guarded transaction results may differ because only one can incorporate a conflicting assignment; that is model concurrency, not a reason for the second request to fail with an unrelated stale global-resource error. An application policy may choose an explicit serialized deployment but may not present it as unrestricted parallel progress.

An effect capability is issued to the correct owner and checked at its actual use, not as a process-wide permanent flag. Reentrant callbacks, cross-thread continuation, provider retries and cancellation must not amplify that capability. The source of time, current principal, guarded prestate and command attribution remain separately bound; completion of an HTTP response does not by itself mean a durable submitted command has committed or failed.

Asynchronous handlers, worker tasks, and durable workflows need explicit context propagation or ownership transfer. Merely retaining a thread-local binding does not establish their lifetime. Work that survives an HTTP response acquires its own admitted owner; it cannot borrow the retired request's capability indefinitely.

Cancellation and cleanup first revoke further local use. They do not undo a transaction already submitted or prove its rejection. Cleanup must not overwrite an intervening registry change, and failure after dispatch retains command/effect uncertainty and the required recovery owner.

One-use physical grants and semantic command deduplication remain separate. Compatible retries preserve `:gesso/command-id` and its binding while receiving new invocation/transport resources. Backend retries or repeated transaction function evaluation obey the provider's declared contract and cannot silently multiply external effects.

Observation after execution is not a pre-effect guard

A correspondence observer can detect that an installed route invoked the wrong writer or that an actual result differs from its model. If that check runs after dispatch, it is evidence and diagnosis; throwing from it cannot prevent or undo writes that already happened.

Mandatory safety checks run before the protected effect, with commit-time conditions enforced by the authoritative boundary. Post-execution observation may add evidence, flag a broken premise, or trigger containment and recovery. The explanation states which role each check plays.

A sequence of observed results matching a model path does not by itself establish physical chronology or uninterrupted state continuity. Occurrence identities, ordering evidence, captured state scope, and unobserved interference remain part of the correspondence argument. Equal before/after documents do not prove that no intervening writer ran.

A protected operation therefore has two deliberately separate questions: **may this effect occur now?** and **what can later be concluded from observing its execution?** A post-commit comparator cannot be offered as the former. An observed mismatch may invalidate subsequent admission, demand operator intervention or create a recovery obligation. It may not retroactively convert a committed Claim into a rejected command, erase an already visible physical mutation, or authorize a compensating write without a separately admitted operation.

Policy acceptance and mutation histories

A compliant completed application must demonstrate both meaningful positive execution and targeted rejection/failure, under the **same real selected assembly** and evidence profile. The following histories are independent acceptance requirements, not prefilled mock reports:

History
A current eligible helper clicks actual Claim and the issued command incorporates, then authoritative acquisition and installation complete
Two eligible helpers concurrently attempt Claim on one open Request.
A finite model reaches a Claim guard but every configured outcome is rejected.
Every abstract operation preserves `I`, but the finite model never enables required manager Reassign.
A registered operation has no matching finite family or its family uses a different abstract preparation callback than the physical executor.
A rule exists and has been checked over some states, but no explored state matches its declared source predicate.

History
A scheduler or migration can write protected RequestAssignment through an undeclared XTDB path.
A writer declares a narrow field footprint but can physically replace or erase the complete RequestAssignment document.
A copied green assurance report is supplied at startup instead of a live checked construction and bound installation.
Two parallel requests share one one-use process slot and the second is rejected before its own valid execution.
An unauthorized Claim is detected only by a comparator after the write has already committed.
An HTTP response is lost after submission and the client retries with the original command identity.
A check exhausts its time/memory budget or a required theorem lies outside the supported checker language.
A strict policy requires complete physical writer coverage but the selected installation can only enumerate host-supplied registrations.

The expected diagnostics identify original declaration, physical source, required proposition, applicable policy, checker/host boundary, remaining premises and missing or contradictory evidence. A negative-only test fixture cannot establish the product's real positive capabilities. Nor does a successful modeled or observed transition establish checker soundness, deployed correspondence or future execution. Sections 07.05–07.15 own the specific route, physical asset, acquisition, startup, output and acceptance enforcement that makes these requirements operational.

Transport, installed routes, and trusted execution

Preserve operation identity through physical realization

Once an action is declared to mean a semantic operation, later bindings preserve that identity. Missing observation basis, plan, transport, or handler causes a construction/rendering rejection or an explicitly unavailable action under its UI contract. The system cannot erase the semantic tag and emit an ordinary POST to keep the control clickable.

Alternate realizations are legal when each is independently declared and checked for the same operation, and the selection rule is explicit. An optimistic path, a non-optimistic command path, and a server-only path may have different requirements. Choosing one is not permission to invent a weaker interpretation at runtime.

An unavailable rendering preserves the operation and reason while suppressing its effective executable paths, including form-level and inherited actions. Styling alone does not disable a transport. A permanently unavailable required capability still fails independent product acceptance.

Ordinary HTMX and rendered semantic-route collisions

A plain HTMX request remains a legitimate route realization when its effect and authorization contract are ordinary. Introducing Choreo must not make read-only filters, search, navigation, or independently declared ordinary server actions illegal. The inverse rule is equally important: an action targeting an assembled semantic-operation route may not become an anonymous ordinary request merely by losing its `:choreo/op` binding, including when a component or inherited form supplies the method and URL.

The rendered checker evaluates the **effective** supported action. This includes native form `action`=/=`method`, form-level `hx-post`, relevant inherited attributes, a submitter's `formaction`=/=`formmethod` overrides and conflicting HTMX request attributes. A disabled-looking child still has an executable form or submitter unless all supported physical activation routes are actually suppressed. Semantic-route collision is evaluated using the method and concrete selector under the **same** routing semantics that dispatch will use. A GET and POST sharing a path are not automatically the same endpoint.

The full client DOM may change after checked server serialization. A rendered surface checker cannot establish that arbitrary JavaScript will never forge or mutate a request. Trusted ingress must independently reject malformed, ambiguous and unauthorized operation envelopes, including direct HTTP clients that never used an admitted browser control. Section 07.07 specifies the complete dynamic-render enforcement and its raw-HTML escape boundaries.

Dispatch and checking share route semantics

The route constructor supplies both the installed dispatcher and the checker projection, or establishes their correspondence through a qualified integration. The checker must use the actual method/path interpretation, parameter decoding, base path, route precedence, and discriminator rules that select the handler. A guessed regular expression over a URL is insufficient if it disagrees with real dispatch.

A supported routing profile declares its analyzable subset. Literal paths and simple segment parameters are a useful initial subset. Unsupported patterns, rewrites, wildcard dispatch, or content-based selection are explicitly rejected for that profile or admitted through another qualified contract. Prototype restrictions do not define the whole framework's eventual route language.

Overlapping routes are either rejected or have a deterministic precedence relation that both checking and dispatch use. Merely forbidding every overlap is one possible narrow profile, not a universal requirement. Routes serving several operations use an admitted discriminator and closed registry; they do not execute browser-supplied machine state or select by fuzzy names.

Method/path matching uses the effective request after the declared trusted normalization chain. Proxy rewriting, host/base-URL rules, query selectors, method overrides, and content negotiation are included where relevant. Unknown or conflicting selectors fail explicitly rather than allowing one layer to silently override another.

For example, a rendered `POST /app/requests/R1/claim` is not resolved by comparing text with the route template `/app/requests/:request-id/claim`. The same route matcher and codec that dispatches the request must establish the match and decode R1. If the envelope separately names Request R2, neither the path nor envelope can silently win: the trusted ingress rejects the contradiction before invoking Claim. This remains true if the request came directly from curl rather than the checked page.

The checked source is the installed router, or a constructor from which both the actual router and its checking projection are derived. A supplied snapshot of declared route metadata is weaker than independently checking the actual installed method/template/callable and the deployed request path. Host rewrites, middleware and multiple route alternatives belong to the selected route profile. If an integration cannot inspect a custom router, it requires a named adaptation/qualification premise rather than silently claiming that a simplistic matcher covers all deployments.

A multi-operation endpoint needs a verified discriminator and a trusted operation-to-executor mapping. Ambiguity, duplicate matches, a missing discriminator, or a discriminator supplied only by untrusted metadata cannot establish semantic dispatch correspondence.

The installed callable is the checked callable

The construction should issue or compose the actual route handlers and dispatcher, then install those products. A separate map claiming that route R uses executor E is weaker evidence than constructing R to invoke E and checking the actual installation's binding.

The installation check distinguishes:

- a declaration naming a handler;
- a constructed dispatcher bound to that callable;
- a system map holding the wrapped dispatcher;
- the actual server instance accepting requests through that wrapped dispatcher.

Each is a different evidence boundary. Inspecting a caller-selected system map does not prove what a running server captured earlier. The supported server constructor must consume the admitted handler directly or establish its actual adoption through an equivalent qualified mechanism.

Ordinary middleware remains possible. Its composition preserves the required routing, principal, request, response, and effect contracts. A middleware stage that rewrites an operation selector or replaces the handler changes the relevant binding and cannot inherit the previous report without checking.

Actual host adoption and invocation lifetime

The source-issued route/executor, the installed Biff handler slot, the network-facing server's captured handler and the physical effect invocation are distinct evidence subjects. An opaque local handle or a fresh comparison against a Biff system map can support the first two without proving the server actually serves that map. A later module, hot reload, retained old dispatcher, independent raw XTDB client or alternate HTTP entrypoint can break the claimed relation after a report was issued.

The supported managed application constructor must either enforce the admitted handler selection at the serving boundary, or carry explicit deployment-adoption and bypass premises consistent with its stated assurance profile. A protected route must not preserve the strong production-enforcement label when its actual installation is unknown.

Each accepted HTTP invocation obtains a current request-owned context, correct generation and appropriate scoped write capability; it is not authorized by an old request's one-use handle or a copied result map. Asynchronous completion, callbacks and durable work have separately declared owners and retirement rules. A bounded-serial execution profile may be qualified as such, but does not claim unrestricted parallel request admission or execution.

Trusted command admission

The transport contract defines the real wire field/encoding, supported protocol, operation/interface identity, command binding, argument codecs, and outcome channel. The browser transports the stable command ID before any possible send. `:transport-request-id` identifies an attempt and is not the domain Request ID.

At the trusted ingress boundary:

1. decode and validate the actual supported envelope and typed arguments;
2. resolve the operation through the admitted server registry;
3. require agreement between operation and subject selectors in the route, envelope, query, and trusted dispatch context where the contract duplicates them;
4. obtain principal/tenant context from its trusted provider;
5. bind the current invocation to the admitted public model operation;
6. enforce command repetition, disclosure, and atomic current-state guards as specified in Part IV;
7. encode only the permitted result and observation evidence.

The admitted server registry selects and invokes the actual issued executable delegation to the **public model operation**. The route owns protocol decoding, trusted subject selection, principal binding and outcome encoding; it must not reimplement Request/RequestAssignment business policy in a second handler. A route's successful response is not evidence that its declared operation executed: the claimed outcome must be causally joined to the exact issued invocation and its authoritative transaction or typed rejection.

The wire contract states where `:gesso/command-id`, the semantic operation, argument schema, supported protocol revision and applicable observation context actually appear in an HTMX/form/request transport. Unrecognized versions and inconsistent duplicate fields fail before protected effect submission. `:http-request-id` and `:transport-request-id`, where used, identify request/attempt scopes rather than the durable domain command, physical Request entity or browser execution. Compatible retries may use a new transport attempt while preserving the original command binding; they must not manufacture a second logical Claim merely because the first response disappeared. An existing command result and a current entity read answer different questions.

For a protected mutation, trusted request conversion and domain-model validation are independently specified. The route must not accept browser-owned principal, tenant, role, plan digest, or purported admission certificate as proof of current authority. Trusted authorization and transaction guards evaluate the appropriate **current** authoritative frame, including current membership and Request state; a previously displayed allowed affordance remains only a view of older knowledge.

Browser-supplied principal IDs, role labels, basis tokens, plan references, certificates, or assembly IDs do not grant current authority. A basis carried by a browser is protocol context unless its particular provenance/use contract establishes more. Part II's observation-evidence distinction applies.

Where cookie-based or comparable ambient credentials make cross-site invocation relevant, the selected transport/authentication profile includes the required origin/request-integrity defenses. Structural route closure and typed decoding do not substitute for that security contract.

The selected transport must also preserve disclosure and correlation policy: a response for command C1 must not be accepted as C2 or reveal another principal's protected result. HTTP acceptance, server-side settlement, historical incorporation, authorized readback and physical client adoption remain different evidence propositions. Their distinction survives network retry, duplicate transport delivery, redirect, and loss after commit.

Required execution providers are typed bindings

A requirement is more precise than a member of a set such as `#{:clock :db}`. It identifies the input/output contract, origin, invocation scope, lifetime, ordering/dependency constraints, and supported failure outcomes of the provider.

For example, a Claim executor may consume a sampled instant under `:biff.fx/now`. A zero-argument clock function under another key does not satisfy that requirement until a declared adapter samples and binds it at the intended boundary.

```
{:requirement/id :claim/commit-time-input
 :consumer {:operation/id :request/claim
            :input-path [:execution-context :biff.fx/now]}
 :value-contract :time/instant
 :provider/ref trusted-clock-provider-ref
 :binding-adapter/ref sampled-invocation-time-ref
 :lifetime :invocation
 :failure-continuation/ref time-unavailable-ref}
```

The example is a construction binding, not proof that a clock is accurate or that the value is transaction time. The operation names the temporal meaning it requires. Retry semantics state whether a sampled value is preserved or replaced and how that affects command binding and guards.

If Request Claim requires `:biff.fx/now` as a sampled `Instant` at the guarded model boundary, an application supplying only a function under `:clock` has not closed that edge. The admitted adapter must actually sample and place the value at the consumer's expected input path. Validation of type, source, temporal meaning, and lifetime is separate from verifying clock accuracy. On retry, the contract must distinguish an original command-bound timestamp from a newly sampled invocation time; changing a timestamp must not silently change the logical command's meaning.

When an execution provider becomes unavailable after structural admission, the operation follows its declared typed failure, uncertainty or recovery path. In particular, an external side effect or XTDB submission that might have occurred is not rejected merely because its return path or subsequent publication failed. A fallback clock, database or publication provider is permitted only if its semantic and trust correspondence has been explicitly admitted.

Providers include authentication, transaction execution, authoritative reads, progression comparison, clocks, identity generation, publication, and external services as needed. Missing, ambiguous, incompatible, or wrongly sourced supply is rejected where statically knowable. Runtime unavailability follows its required declared handling path. Having a transaction capability does not authorize the transaction's business operation.

Positive and adversarial transport histories

The application must make real eligible operations usable and reject individual invalid realizations at their earliest legitimate boundaries. A schematic status map is not a test of the actual installed request path.

History
An eligible helper uses the real annotated Claim control; its decoded Request selector, authenticated identity, command and trusted executo
A parameterized Claim POST omits Choreo identity after a component strips its metadata, though its physical URL still resolves to a seman
A form posts to a protected Claim route after the child button was visually disabled, or a submitter overrides its target.
POST and GET use the same URL, but only POST is a semantic Claim route.
A browser sends operation Claim for Request R1 while the decoded URL selects R2.
Two routes overlap and checking chooses route A while the running router chooses B.
A current issued Claim handler is checked, but Biff or the deployed server instead installs another callable under the same route ID.
The same physical endpoint admits Claim and Cancel, but no validated discriminator binds the selected operation to its issued executor.
The incoming command claims its own principal or organization role, which conflicts with trusted session/tenant context.
Claim needs a sampled `:biff.fx/now` Instant, but the provider exposes only a differently shaped unsampled clock.
A Claim commits, its HTTP response is lost, and the browser retries.
An asynchronous handler or worker continues after the invocation or assembly generation is retired.
A post-commit observer reports a wrong handler while the domain write is already durable.
A normal read-only HTMX search uses a route without any semantic-operation annotation.

These histories require source-derived routing, provider and executor identities, real physical installation and appropriately qualified runtime evidence. The next sections separately close browser artifact, generated HTML, acquisition and installation joins. No transport checker, HTTP 200 or copied certificate grants current model authority.

Browser assembly manifest and physical artifacts

The browser build consumes the admitted semantic/build inputs of the same application definition. Its manifest identifies the exact admitted plan set, operation-to-plan correspondence, executable identities, interpretation/profile contracts, required physical handlers, and relevant implementation/toolchain inputs. It does not contain a digest of bytes that do not yet exist.

The admitted plan inventory must cover every required operation realization and identify any deliberately included shared or currently unused plans. Missing, duplicate, incompatible, or undeclared executable entries fail the applicable inventory contract. Bundling an explicitly admitted extra plan is not intrinsically wrong; letting an unexamined executable enter the claimed closed registry is.

Source-derived inventory and independent build choices

The required manifest is derived from the same admitted operation and role sources as rendered actions, server transport and application preflight. It names each executable plan's exact semantic digest and supported role, features, protocol, source/projection version and compatible host runtime. Derivation cannot start from a second manually maintained browser registry whose members happen to agree today. A newly required Claim/Reassign plan therefore changes the admitted build input even before any JavaScript exists.

Inventory class	Required treatment
Required executable plans	Exactly matched to the admitted operation/role projection, with no missing, duplicate or misr
Shared or deliberately inactive plans	Explicitly classified; they cannot be treated as active product capabilities or silently escape th
Runtime/host effects	Enumerate physical handlers, event sources, transports, acquisition and supported completio
Compiler, runtime and physical realization	Bind the implementations and options capable of changing emitted behavior, not merely frie
Dynamically imported code and workers	Account for the transitive loaded resource set and the declared runtime loading/qualification

Source-owned semantic identity, exact projected payload identity, build environment identity and emitted asset bytes identify distinct objects. The manifest must not infer trusted source provenance from a filename or self-advertised digest, and its graph must remain acyclic: the digest of an output cannot be included in the very same unqualified bytes it digests.

A concrete generated-entrypoint construction

The normal build uses a Gesso-owned generated bootstrap source. It embeds or imports the exact admitted manifest/plan payloads and invokes the canonical runtime bootstrap. Application code supplies typed physical realization options; it cannot replace the manifest-owned operation registry or plan resolver through those options.

A concrete build sequence is:

1. Resolve the application definition once into the candidate semantic inputs and selected browser realization.
2. Derive the manifest and exact executable payloads from that construction.
3. Generate the bootstrap entrypoint and explicit imports for the admitted physical realization implementations.
4. Compile with the declared toolchain, dependency versions, options, and inputs.
5. Record the generated-input correspondence and digest the actual emitted assets.
6. Verify the staged manifest/receipt/asset set before publication.
7. Admit an immutable asset location set and bind it to the candidate assembly.

A low-level emitter accepting arbitrary JavaScript plus a copied manifest is not equivalent to this construction. It may remain a testing or integration API with a narrower guarantee.

The default builder must actually generate the entrypoint from the admitted manifest and resolve its physical realization options through a constrained, typed interface. A physical options value may select concrete DOM/HTMX adapters and integration callbacks, but may not inject a second plan resolver, rewrite admitted operation IDs, replace semantic transitions or bypass required effect handlers. The options Var's identity, its resolved implementation, compiler classpath and deployment profile participate in evidence invalidation.

The exact build result is tied to the **source-generated input** and the compiler/toolchain actually invoked. A successfully staged JavaScript file does not prove that the compiler preserved Gesso's intended semantics; that remains a named fidelity/correspondence premise until stronger checking exists. Likewise the existence of a generated entrypoint in source does not prove the intended runtime bundle was emitted or later loaded.

There is no requirement for a magical parser that extracts all semantic plans from arbitrary optimized JavaScript. Source-generation correspondence, compiler fidelity under a named premise, and exact emitted-byte binding provide a concrete argument. Stronger

independently checked compiler/artifact correspondence can replace the relevant premise when available. A runtime registry probe can add conformance evidence, but a self-reported registry is not independent proof of all code executed by the bundle.

A supported producer may expose an independently checked emitted-plan inventory, but the method of obtaining it must be declared. It cannot be satisfied by merely copying the expected plan digests into a receipt beside unrelated bytes. If the compiler does not support a trustworthy registry extraction, the build can instead retain its exact source-generation witness and a scoped compiler-fidelity premise, with real-browser startup/plan execution supplying additional behavioral evidence. Neither method licenses a claim of universal equivalence for arbitrary optimized JavaScript, third-party scripts or dynamically loaded unknown chunks.

Receipts bind provenance and bytes separately

A detached build receipt records the manifest identity, generated input/plan identities, compiler/runtime/implementation closure, relevant build options, and the exact digest/size/location of each emitted asset. Its evidentiary value depends on provenance from the admitted build/checking boundary, not on the receipt's own claims. It conveys no permission to perform a domain operation.

Verifying a digest establishes equality with recorded bytes under the digest premise. It does not establish that those bytes implement the intended semantics. Conversely, trusted compilation from the correct source does not establish that the server later serves those same bytes. Both relationships belong in closure.

The receipt and build manifest have **different provenance**. A copied receipt with matching expected labels cannot authenticate either the emitting compiler or the served file. The receipt issuer must be the admitted build/checking boundary or another independently qualified source, and the checker must read the actual bytes being admitted at the relevant storage boundary. Exact byte digest matching is a content-identity assertion under the collision premise, not a proof of inclusion, runtime behavior or permission to execute a protected domain operation.

If the selected output is more than one file, each member's digest, relative location, dependency edge, load policy and serving identity contribute to the retained asset-set evidence. Content/security headers, cross-origin loads, service-worker caches and import maps can alter which code executes even when the entrypoint file itself is correct; the selected physical profile accounts for these paths or leaves their implications explicitly unestablished.

The asset set includes dynamic chunks, workers, imported runtime libraries, and other resources whose behavior affects the claim. Relevant CSS or configuration that can suppress or redirect an affordance also has a declared dependency or qualification role. Diagnostic-only sidecars retain their separate noninterference contract from Part V; they do not become executable semantic inputs.

Build inputs include opaque physical implementation changes where their behavior matters. Reusing the same named `current` toolchain/runtime label across changed implementations cannot preserve evidence applicability. Deterministic compilation is useful but not a prerequisite for naming actual emitted bytes: a receipt can bind one actual output even when another valid build yields different bytes.

Publication and serving preserve the checked binding

Candidate builds use isolated staging locations. A failed build or rejected candidate cannot overwrite assets still served by an admitted assembly. Atomic replacement of one file is insufficient if readers can observe a new bundle with an old receipt or a mixture of old and new chunks. Publish a verified immutable asset set, then switch the admitted serving reference.

The concrete serving strategy must prevent or detect substitution between check and use. Content-addressed/versioned immutable assets are a useful default. An open immutable resource handle, qualified store, or equivalent serving contract can also work. Reading a mutable file once at startup and trusting its path forever is not exact serving correspondence.

A page generated under assembly A must reference assets bound to A even if assembly B activates before the browser requests them. A mutable logical URL that now serves B can break that relationship. Versioned URLs or a qualified assembly-specific asset selection protocol close it. Cache, CDN, service-worker, and old-client behavior follow the selected deployment/profile contract.

Atomic set activation and failure recovery

Staging one JavaScript file and separately writing its receipt with atomic per-file renames is **not** atomic publication of the pair, much less of a multi-file bundle. If the artifact is replaced but receipt publication then fails, the failed candidate cannot be considered admitted even when a subsequent reader notices the mismatch. More importantly, changing a mutable live asset path may already have disturbed the previously served generation. A valid end-state design stages a complete candidate set in isolation, verifies it, and activates an immutable set reference atomically with the serving-generation selection or provides an equivalently qualified mechanism.

The previous admitted generation must remain serveable until its references and permitted old-client lifetimes have been retired. Candidate failure, cancelled compilation or a partial publish does not overwrite or delete its live assets. A publish pointer switch may be rolled back only under an explicitly safe deployment/compatibility policy; restoring asset bytes after clients have fetched the new set does not undo those clients' execution.

A successful build receipt establishes only the candidate artifact relation. Activation additionally needs actual route/static-handler adoption and the asset set available at the URLs generated for that assembly. A page rendered under A is bound to A's immutable asset coordinates even if B becomes current before the browser fetches them. The storage, CDN and cache policy must either preserve that binding or detect a mismatched response before accepting it. Same-name shared URLs are safe only with a qualified generation-aware selection or compatibility contract.

The currentness check asks whether the assets intended for this admitted assembly match its required build. It does not demand that every sleeping browser already runs the newest deployment. Older continuations follow Part VI's compatibility and recovery rules, including retained command resolution.

Browser plan compatibility is weaker than build/serve exact currentness: an older client may legitimately continue or recover under Part VI even while a new deployment must reject its stale build receipt. Conversely, byte-equal current assets do not automatically make a retired execution legal, confer principal authority, preserve a stale physical target or resolve a submitted command. Both generation/host identity and exact resource/command lifetimes remain necessary.

A build that adds a required manager Reassign control and plan to assembly B cannot reuse A's browser bundle merely because the old Claim plan is still present or both bundles advertise a compatible transport version. An application that starts with B's semantic registry but serves A's runtime has not closed its protected operations. It must refuse activation or explicitly choose and admit a compatible, honestly described lesser realization that still meets independent product requirements.

Tests of startup/currentness use the same verification mechanism and actual artifact path as production. Synthetic stamped files can test a narrow checker, but cannot stand in for the real generated runtime in application acceptance.

Positive and adversarial artifact histories

Tests and deployment qualifications must connect the **real** source-generated browser bundle to the admitted application. A unit test with arbitrary JavaScript and a copied stamp has a narrower subject.

History
The canonical application constructor derives Claim and Reassign plans, builds the generated CLJS bootstrap, verifies actual staged bytes ar
R5.1 application B requires Reassign but the browser artifact and receipt still belong to A.
An extra unregistered executable plan appears in a supposedly closed runtime bundle.
A caller supplies arbitrary JavaScript and manually stamps it with the correct plan digests.
A build updates the exact JavaScript bytes but leaves the previous receipt or tampers with its digest.
A generated entrypoint embeds the right plans while its declared physical realization Var points to a changed implementation.
The JavaScript entrypoint is current, but an old dynamically imported chunk or service-worker cache provides incompatible runtime behav
Candidate B's compilation fails before publication, while A is still serving.
Candidate B replaces the shared JavaScript filename, then receipt publication fails.
A is rendered before B activates, and the browser fetches an unversioned URL that now serves B's incompatible bundle.
A current asset receipt matches locally, but the network server or CDN sends different bytes.
The new B bundle is exact, but a sleeping A browser still owns a durable unknown Claim.
A unit checker passes on a fake stamped bundle, but the actual shipped browser cannot register or execute Claim.
A build's compiler or physical handler is outside a declared proof/qualification profile.

These cases do not require every build to be bitwise reproducible. They require that **the actual admitted bytes** and their required dependencies are identified and correctly served with provenance sufficient for the selected profile. The source-generated pathway, exact content check, physical activation and real-browser behavior are complementary claims, not substitute green flags.

Rendered affordances and concrete response closure

Minimal semantic intent in views

The view selects an admitted operation, supplies concrete arguments and required observations, and chooses presentation/availability. Prepared bindings from Part V derive the plan, transport, encoding, and other realization details from the assembly. A view should not manually copy browser registries, authority roles, command envelope formats, or artifact digests.

A prepared action is not current authorization. The trusted model boundary still checks the principal, Request, Membership, Organization policy, and other commit-time dependencies. Rendering an action establishes neither a permanent permission nor a promise that a later attempt will succeed.

An action can be deliberately unavailable under its UI contract. Its semantic identity and reason remain inspectable, and no effective transport survives that unavailability. A missing required basis cannot be repaired by discarding semantic metadata or forging a browser observation record.

The authored view may be a Request card rendered for an eligible helper, a manager with an admitted Reassign target, or a participant who lacks the required authoritative basis. Those produce different concrete controls, but the underlying semantic operation does not change to satisfy a template. A rendered enabled Claim/Reassign action is **structurally usable** only if its selected operation, typed arguments, basis/observation obligation, actual browser plan, required physical transport and return path close together. That observation need not promise current authorization: a competing Claim or Organization policy change can invalidate the action before commit.

Positive product acceptance remains independent: a perfectly well-formed permanently disabled Reassign button does not demonstrate that a manager can reassign an eligible Request. A rendered affordance is also not an authorization token; client-supplied metadata cannot substitute for the trusted principal and current guarded model operation.

Check the effective action, not one attribute

The render profile defines the HTML/HTMX subset whose effective behavior can be checked. It accounts for all supported request sources and their composition, including:

- generated semantic bindings and ordinary explicit request attributes;
- form action/method and submitter overrides;
- form association, including detached controls;
- applicable inherited attributes and explicit inheritance controls;
- relevant defaults, relative URLs, document/base context, and current path;
- supported boosting, extensions, response triggers, and navigation behavior;
- string/keyword or other attribute spellings that the serializer treats alike;
- conflicts between authored and generated transport, target, and protocol fields.

The profile must follow the actual selected browser/library behavior. It cannot assume every attribute inherits, ignore a supported default action, or accept silent last-write-wins merging where effective dispatch becomes ambiguous. Duplicate IDs, invalid nested forms, conflicting attribute spellings, and browser repair of malformed markup are rejected or explicitly handled where they affect the claim.

Derive the effective request under the selected HTML/HTMX profile

The checker and actual browser must agree on the action of the **activated control**, not merely on attribute names present in one Hiccup vector. A detached submitter's `form` owner may be elsewhere in the document; a button may default to submit; its `formaction` and `formmethod` can override the parent form; an ordinary form may default to GET. Explicit `:hx-post` and a native submit path can coexist and may have different effective actions. Duplicate keyword/string spellings can serialize as two physical attributes, so deterministic selection of one Hiccup map key does not establish unambiguous HTML behavior.

A checker for the current selected profile must enumerate **all** supported actions that can cause relevant semantic transport, including the activation path of each submitter, not just the primary button or one handpicked `hx-*` attribute. If the same DOM node offers a valid managed Claim `hx-post` **and** an additional unadmitted protected native form submission, the valid action cannot exempt the second one. Conflicting transport, argument, target and observation selectors reject unless the profile supplies an explicit, independently checked resolution.

The physical route match uses the same supported method, path decoding, base URI, precedence and normalization as actual installed dispatch, or a qualified correspondence to it. A plain GET search that happens to share the path text of a semantic POST is not silently transformed into Claim. Conversely a concrete `/requests/R/claim` POST does not become ordinary because its metadata disappeared. Rewriting and inheritance outside the supported profile must be classified as unsupported or explicitly constrained; they cannot be counted as checked merely because the scanner did not find them.

A fragment can depend on a form or inherited context outside the returned tree. Its assembly/rendering contract supplies the necessary host context or constrains it to a self-contained composition. If required context is unknown, the checker reports unsupported/unestablished behavior; it does not declare the fragment complete because the missing ancestor was outside its scan.

The target's scoped identity and composition environment are first-class inputs where needed: document/base URL, HTMX inheritance rules, owning form, submitter, current route coordinates and authorized target/fragment ownership. A standalone fragment can be preflight-closed **relative to its declared self-contained host contract** without claiming that a future uncontrolled parent element will preserve that contract. If an out-of-band replacement, boosted navigation or cache insertion can change those coordinates, the selected output/installation integration must check the changed effective context or retain the corresponding external premise.

An ordinary effective action targeting an assembled semantic route must carry that route's required semantic realization. Moving the action to an ancestor or substituting concrete path parameters does not make an anonymous semantic request legal. Conversely, an ordinary read/search route remains ordinary when it meets its declared acquisition and effect contracts without a command choreography.

Arbitrary client JavaScript can alter the DOM or send requests directly. Render checking establishes the trusted output's supported realization, not honesty of a hostile browser. Server admission remains independent. Effects of ordinary scripts within protected browser footprints follow Part VI's ownership and noninterference contract.

The final structured value is the checked value

Dynamic rendering is checked after the application has produced the concrete structured output and before prohibited output is delivered. Checking a prepared view model and then running an unchecked renderer is insufficient: the renderer can introduce a malformed control after the check.

The normal response path is:

1. retain the request's admitted assembly and response-context binding;
2. perform view preparation and component expansion under their declared contracts;
3. materialize the concrete structured tree and effective-action context;
4. validate semantic bindings and all supported effective actions against that assembly;
5. serialize that same checked value through the admitted serializer;
6. apply only response transformations whose correspondence contract preserves the checked properties, or recheck after a relevant transformation;
7. release the response through the admitted delivery boundary.

Lazy values, component callbacks, mutable references, and serializer-time code must not be evaluated once during checking and again with different results during serialization. A supported implementation can materialize a bounded immutable tree first, or use a serializer with an equivalent checked occurrence protocol. It must state the actual mechanism and resource/failure limits.

A pre-serialization guard that receives the **same top-level Hiccup reference** does not by that fact ensure that lazy descendants, custom components or a mutable reference produce the same expanded HTML twice. The supported render profile must identify which values are inert, which must be fully realized before checking, how custom renderer callbacks are constrained, and whether serializer-time transformation invalidates the check. A materialized exact occurrence is preferable when feasible; where a paired inspect/emit protocol is used, its semantics, effect containment, failure behavior and implementation assumptions are explicit. A counterexample discovered after some output is already streamed can block further delivery and demand repair, but cannot retract sent bytes.

When validation fails, the response producer must not silently invoke a different unchecked serializer, return a pre-rendered copy, or use a privileged raw HTML fallback to keep serving the control. The fallback is itself within a qualified minimal-output protocol, which must fail safely without unbounded recursion or accidental disclosure.

This is structural/effective-action checking, not automatic HTML sanitization or proof of arbitrary renderer purity. Content safety and active-content policy are separate contracts, as in Part VI. A renderer that has already performed an external side effect cannot have that effect undone by rejecting its output.

Systemic enforcement covers deferred and alternate responses

The canonical application handler installs the guard once for all supported structured responses. Nested guards compose without disabling the outer guard. Application authors need not remember a manual validation call at every render.

A dynamically scoped guard can implement the synchronous path, but an asynchronous response may be completed after that dynamic scope has ended or on another thread. The response completion carries an explicit assembly/guard context or uses

another qualified propagation mechanism. Absence of the required guard at a managed serialization boundary is a failure, not a silent return to legacy unchecked behavior.

The ordinary Gesso/Biff synchronous handler integration can use a dynamically bound guard around `gesso.http/html-response`. That mechanism does not automatically cover a response serialized after the handler returns, a future completed on another thread, a raw pre-serialized Ring body, a server module replacing the checked handler, or a server that never uses the wrapped `:biff.ring/handler`. Such a path is not magically checked by the existence of a valid ApplicationAssembly or by an earlier test of `check-rendered-surface`.

The finished managed application contract identifies how the guard reaches each admitted producer and final response emission: captured generation and currentness validation, explicit propagation to deferred continuations, and a startup/installation gate that actually adopts the protected serialization boundary. The policy may explicitly exclude an ordinary unmanaged response whose effects are disjoint from its managed claims, but it must refuse a whole-managed-surface guarantee if the omitted response can emit protected operations. An opt-in unit checker cannot prove the checked production path was used.

The supported response profile covers or explicitly classifies:

Response path	Required treatment
Ordinary structured page/fragment	Check the actual final structured occurrence before serialization/delivery.
Deferred/asynchronous response	Preserve the admitted assembly and guard through completion, timeout, cancellation, and
Streaming output	Check each delivered managed portion under a compositional context or buffer the requi
Cached structured fragment	Preserve semantic binding and applicability; check the concrete occurrence in its current
Cached/pre-serialized HTML	Establish a separate admitted output correspondence and context/authority policy; a prev
Error response after handler failure	Use the same supported output boundary or an explicitly admitted minimal error protoco
Redirect/navigation/secondary-target response	Check its declared effect and target/protocol behavior, not only the main HTML body.
Raw HTML/custom renderer	Admit a separate realization contract or exclude it from the specific structured-output cla

Naming an escape hatch does not satisfy a requirement that all application managed surfaces are covered. A policy requiring that coverage rejects an uncovered managed path. Static assets or truly unmanaged output can be explicitly outside that claim, provided their effects do not invalidate its premises.

A response cache key is not just a URL when its contents depend on tenant, authorization, query, semantic contract, or assembly instance. The cache contract states the partition, freshness, revocation, and revalidation behavior needed for its claims. Returning a cached checked card to an unauthorized user is still a disclosure failure.

Cached output has at least two independent questions: **was the stored structured occurrence checked against the generation that produced it?**, and **is disclosing and executing it valid for this recipient, context and current generation?** Cached HTML cannot inherit indefinite permission from an earlier checked render. Invalidation and reacquisition are separate from permission to disclose a cached body; an old card can be structurally coherent yet expose an operation no longer available to the current principal. A cache hit that bypasses the admitted checking or authorization boundary must be classified as such.

Worked example: a render-time defect after a valid Claim

Startup admits the Claim operation and its return path. For a particular request, the trusted model incorporates command C42 successfully. While preparing the resulting card, a bug creates a Cancel affordance without its required Request argument.

The output guard rejects that concrete card before delivery. It does not change the authoritative Claim outcome to rejected. The trusted request boundary retains its known result and follows the admitted response-failure protocol.

Evidence available to the participant	Required treatment
A permitted, correlated committed outcome is delivered through the supported error/settlement channel.	Preserve known incorporation;
The server knows incorporation, but its response is lost or the browser cannot admit the payload.	The browser remains unknown
A later card read shows the current claimant without command attribution.	It can establish current state at
The malformed response concerns an operation rejected before submission.	Preserve that actual disposition

The error protocol defines the actual HTTP/transport representation and the browser hook that can decode it, including non-success status handling where used. A map containing settlement metadata is not enough if the selected HTMX profile never exposes that metadata to the runtime.

The fallback response itself must be valid and bounded. It must not recursively invoke the same failing renderer forever or expose privileged diagnostics. A minimal typed response or a later authenticated resolver may be more reliable than trying to render the full card again inside the error handler.

The error path must preserve the distinction between the **server's known authoritative result** and **the browser's participant-relative knowledge**. For example, the server may know that Claim C42 committed and that the Cancel action in its response is malformed. If a correlated, authorized settlement is actually delivered and admitted by the selected browser HTTP/HTMX error protocol, the client may learn the incorporation and separately report presentation failure. If the error response is lost, the client retains `:unknown` until the durable command resolver supplies admissible attribution. A current Request card is insufficient proof of which earlier Claim command won.

A bad result renderer may also have run **after** the protected transaction, and its failure cannot retroactively veto that transaction. Conversely, a rejected pre-effect command remains rejected when rendering its error body fails. The fallback must not opportunistically invent another domain write or refresh authorization under the pretext of recovery. The next section owns the complete authoritative outcome, acquisition and propagation mappings.

Tests distinguish no malformed output was delivered from no protected mutation occurred. The former is the render guard's responsibility; the latter requires an earlier command/model gate. Both need a positive control showing that valid requests remain usable.

Positive and adversarial rendered-response histories

Each test uses the selected source-owned assembly, **actual** relevant production response boundary, and where applicable the real browser control. Supplying an already successful report or constructing a valid protocol command in place of a broken control is insufficient.

History	Re
An eligible helper renders a well-formed Claim button, activates it, and an admitted plan submits the command to the protected route.	Th
A manager sees an eligible Reassign affordance with typed Request and target Helper IDs.	Pr
The Reassign renderer omits `:helper/id` on one conditional card while startup's registry is correct.	Re
A Claim button lacks basis and is marked visually disabled, but its enclosing form retains an active semantic POST.	Re
A marked Claim button emits the certified `hx-post` and an additional unadmitted protected `hx-put`.	Re
A detached submit button overrides a search form to the Claim POST route.	Re
A keyword and string Hiccup key both serialize a relevant physical request attribute.	Re
A form uses the browser's implicit action/current document URL, which the selected analyzer cannot reconstruct.	De
A fragment is valid in isolation but gains a protected inherited action after installation in another parent.	Cl
A lazy child renders safely when inspected but emits a different protected action on the serializer's second evaluation.	Re
An asynchronous task serializes Hiccup after the dynamically scoped handler guard has ended.	Re
A cache serves a previously valid Request card to a different principal or retired generation.	En
A valid Claim C42 commits, but a malformed Cancel affordance is discovered while building its response.	Bl
C42 commits but its render-failure settlement is lost or ignored by the selected HTMX error lifecycle.	Pr
A renderer fails before a prohibited protected write would be initiated, and the guard catches the malformed page.	Th
A handler returns raw pre-serialized HTML while the canonical Hiccup wrapper would otherwise be installed.	De

History	R
A real valid ordinary HTMX search form uses a nonsemantic GET route.	A

Controlled-defect tests distinguish source mismatch, effective physical request, response producer, delivery and postcommit recovery. An application that disables every action or turns every server result into HTTP 500 does not satisfy independent positive HumanHelp product acceptance. The fact that Gesso can check one current Hiccup occurrence is not by itself a claim that the deployed server checks **every** possible rendering path or that the browser interprets it correctly.

Authoritative outcome, acquisition, and propagation closure

Model outcomes and presentation settlements are explicit mappings

The authoritative command protocol uses Part IV's `:committed`, `:rejected`, `:already-incorporated`, and `:commit-unknown`. Domain result payloads and browser presentation settlements are separate types.

A Claim presentation may use `:accepted`, `:rejected`, and `:unknown`. Assembly checks the explicit mapping, including a permitted recorded success from `:already-incorporated`, incompatible command reuse, nondisclosable prior results, and unresolved evidence. It does not substitute an assortment of transport labels such as confirmed or superseded for the model outcome contract.

The trusted boundary checks permission to disclose a prior outcome. A successful replay lookup does not confer disclosure authority on the caller. A local timeout, HTTP 500, missing command row under an insufficient lookup, or present-state card cannot be wired as unconditional proof of command rejection.

A direct response, durable resolver, and current-state acquisition have different claims. Assembly closes the paths required by the operation rather than assuming one route serves all three purposes.

Historical command truth, presentation, and disclosure

The four authoritative outcomes are not four interchangeable HTTP statuses. The supported command protocol has an explicit, provenance-aware mapping to participant-facing settlement and to the particular observation/recovery path needed after ambiguous transport delivery.

Authoritative outcome or evidence	Permitted participant interpretation
`:committed` with admissible command binding and disclosure	`:accepted` for that command, with the required progressi
`:already-incorporated` with exact recorded binding and authorized disclosure	A permitted replay of recorded incorporation without an
`:rejected` from the authoritative operation for this command	`:rejected` with only the disclosed reason; not an inferenc
`:commit-unknown` or uncorrelated/lost outcome	`:unknown` with a retained command-resolution owner; ne
A current authorized Request-card acquisition	Current projection at a stated scope and basis, which may

A transient retry and a durable command are separately identified. Retries preserve the original `:gesso/command-id`, principal/operation/argument binding and the versioned protocol required to interpret it, while a new HTTP or transport attempt may have its own correlation identity. A command ID is not authority or permission to disclose another user's recorded outcome. Reconciliation must validate the current disclosure predicate and the **original** binding before revealing replay or enabling any further action. An absent record in one read, even a pinned read, is not evidence of nonincorporation unless the selected storage/consistency model establishes the stronger negative proposition.

When a domain update and durable command record are claimed to be atomic, the qualified physical transaction must actually write both in the same guarded commit. An unrelated single-update writer plus a later diagnostic lookup cannot establish that stronger command-record claim. The source-owned assembly must retain the distinction in its assurance report.

Recovery includes an owner, trigger, and retained binding

A declaration that names a resolver endpoint is incomplete if nothing can invoke it or the command binding is lost when the local execution retires. Required recovery states identify:

- the work/command and sufficient retained binding or protected recovery reference;
- the current owner and lifetime, including transfer on retirement;
- the evidence needed to resolve the condition;
- current authorization and disclosure requirements;
- the actual trigger and permission for the next action;
- bounded automatic retry/backoff where permitted;
- behavior when plans, contracts, providers, or retained records are unavailable;
- the explicit terminal, blocked, deferred, or manual-reconciliation state.

Part VI's cross-deployment command recovery applies. A new plan cannot reinterpret old arguments under changed operation semantics while keeping the old command ID. The assembly identifies retained resolver/compatibility support independently of new invocation support.

A recovery endpoint's mere existence is not a recovery **behavior**. For a possibly submitted Claim, the resolver must preserve typed uncertainty until it obtains admissible command-specific evidence or reaches a separately defined blocked/manual state. Its invocation permission, authorized principal, record retention, operation-version decoder and outcome-disclosure policy are independently checked. A response renderer failure, timeout, disconnected tab, browser refresh, retired generation or process restart must not silently discard ownership of the outstanding resolution obligation.

Where a retry is admitted after uncertainty, the system establishes the original operation/argument binding and declared deduplication guarantee rather than treating an unobserved outcome as a fresh operation. A read-only command-record lookup is not a retry permit or an implementation of the authoritative operation. Loss of command retention or an incompatible replacement plan produces an explicit defer, block, transfer or manual reconciliation condition, never a made-up successful or rejected settlement.

Progression remains scoped through the return path

The mutation, command record, publication/durable intent, acquisition, renderer, and browser coordinator must agree on the meaning of their progression and coverage requirements. Generic assembly code validates compatible provider contracts; it does not calculate a universal maximum over unrelated tokens.

Acquisition requests carry the required scope and lower bound under the provider's semantics. Returned evidence reports the actual established observation. An installation uses that observation and the browser's current target/owner rights. Adoption and debt discharge require the applicable physical completion, as in Part IV and Part VI. HTTP completion is not a substitute.

The progression contract is typed by its provider and queried subject. An XTDB transaction key, snapshot token, read-consistency requirement, fragment advisory and physical DOM installation receipt may all contain opaque values without sharing an order. Requirements compose only under the declared scope and combination relation; generic maximum, string comparison or receipt equality must not fabricate an advancement witness. Where a mutation's returned basis is needed for a later projection, the selected acquisition route must establish the required lower bound and expose its **actual** satisfied observation, not simply echo the requested token.

The browser may mark an observation adopted only after authorized projection acceptance and the applicable physical installation/target completion, under the selected progression and owner-generation policy. A 200 response, swap attempt or Live advisory is insufficient. If continuity restoration fails after a successful authoritative DOM installation, that local failure must be represented separately; it does not roll back the installed observation or the server's transaction.

A query membership obligation may require different evidence from a card-content obligation. Initial server-rendered content is also an acquisition path with provenance and coverage. An empty managed fragment needs a permitted acquisition path and a truthful loading/unavailable state; absence of both initial knowledge and acquisition is a structural defect.

Subscription establishment cannot open a silent knowledge gap

An initial server-rendered card and the subsequently active interest subscription have a **relative** consistency problem: a relevant commit may occur after the initial read but before the subscription is registered. Likewise a subscription may become active first and then a later read may cover already published changes. The selected strategy identifies the baseline observation, interest-registration fence/receipt, replay or buffering horizon, and the provider's consistency comparison by which one phase covers the other. It may use re-acquisition instead of replaying every invalidation, because an invalidation requests fresh authority rather than carrying authoritative domain content.

A Live assembly that validates only a configured SSE route and GET endpoint has not thereby established the no-gap property. Startup, new subscription, reconnect, host change, authorization-context change, interest evolution and router failover must each have a qualified continuity/reconciliation path. The registration and baseline need not be performed in one atomic database transaction if a suitable fence and recovery argument closes the interval. Without one, the application retains advisory/acquisition debt rather than claiming an interest was up-to-date merely because no advisory arrived.

Required publication and durable work have recovery semantics

When operation semantics or product convergence requires publication, it belongs in application closure. Naming a publisher function does not establish that it runs after commit or survives a crash between commit and publication.

The assembly binds the chosen Part IV recovery strategy: durable publication/work intent with a worker, a qualified replay stream, periodic reacquisition, explicit Refresh under its conditions, or another declared mechanism. It identifies the failure domain and liveness premises. There is no universal promise of event delivery merely because an SSE connection exists.

A lost settlement or publication leaves incorporation intact and creates or preserves the relevant delivery/recovery debt. Worker acknowledgment and retries must agree with the durable work contract. External effects such as payment or notification are not undone by browser retirement or response failure.

The commit-to-publication failure interval is a real failure boundary. A completed XTDB mutation followed by process failure before an in-memory SSE dispatch is not repaired by adding another function named `publish!` to the assembly. An application requiring durable convergence chooses an actual mechanism: e.g. transactionally bound outbox/change intent, replayable write log with stated ordering and retention, or a qualified independent periodic/permitted authoritative reread. These strategies differ in latency, permissions and liveness, and the assembly records which one closes the selected product promise.

A durable work processor acknowledges only after the relevant delivery, handoff or recovery contract is satisfied; retry after crash must be idempotent or have an admitted duplicate-safe effect protocol. A publication may be redelivered because its meaning is advisory, but repeated delivery must not issue repeated one-use Refresh grants, new command IDs or unbounded acquisitions. Delivery loss, worker failure and compaction/retention expiry lead to honest recovery debt or an explicitly bounded unavailability, not an assertion that the original guarded mutation was rolled back.

Live closure includes subscription continuity and adoption policy

The Live declaration binds the authoritative query/projection, relevant interests, initial observation, subscription owner, progression provider, acquisition route, and coordinator policy. The stable logical owner and replaceable physical target have distinct identities and reconstruction rules.

The application closes the baseline/subscription gap and reconnect gaps using a qualified fence, replay, buffering, polling, or reacquisition strategy. A process that publishes changes only for interests known locally must justify how remote or newly registered interests avoid missed updates. Absence of a locally known interest is not proof that no other participant needs the change.

Shared subscriptions retain per-interest ownership and authorization. Connection replacement, authorization-context changes, and lost continuity produce the required recovery work even when no new advisory arrives. Coarse routing/filtering can optimize delivery only under a contract that preserves the required coverage or has a compensating recovery path.

The semantic interest is owned by its logical participant/fragment scope, not by the accidentally selected SSE connection or router node. One physical stream may carry multiple authorized interests, and a router may hold coarse or hierarchical filters as a performance optimization. Neither the absence of an interest in one process's memory nor failure of a particular router establishes that no remote participant needs a relevant refresh.

For distributed deployment, the same single-node semantic rule holds: a change relevant to a registered or newly established interest must eventually become recoverable by a permitted authoritative acquisition under the declared fairness/availability assumptions. Interest advertisement, invalidation routing, router replacement, replay horizon, duplicate-safe redelivery and authorization are separate contracts. A router that learns an interest after an invalidation was forwarded requires a fence/replay/reacquisition argument; copying a coarse filter to another node does not prove that the intervening change was preserved. A failed router can be handled by conservative re-delivery of a qualifying range when the protocol is idempotent and bounded, but the availability of such a replay source is an actual premise, not an implied fact about SSE.

The design does not require storing every successfully delivered advisory for each client. It requires sufficient provenance, retention, scope and failure detection to repair possible loss without assuming exactly-once messaging. A single-node implementation and a multi-node implementation realize the same interest/progression/acquisition obligations; extra routers introduce deployment evidence, not a different semantic model.

Advisory debt, acquisition permission, and adoption permission remain distinct. HumanHelp may announce remote changes and wait for explicit Refresh. A newly arriving advisory does not silently inherit an expired one-read grant. Closure checks that the declared policy has a real trigger, retained debt, and a viable path to the promised scoped adoption.

Positive and adversarial end-to-end settlement histories

The following cases must be exercised against the selected source-owned application, qualified transport and relevant browser/Live boundaries. A model trace or a synthetic SSE frame does not establish real production-path acceptance by itself.

History
An eligible helper Claims an open Request; the guarded command incorporates and the initiating card is authoritatively reread and physica
Another helper receives an advisory about that Request, but HumanHelp policy requires explicit Refresh.
The remote user exercises one permitted Refresh, then a second advisory arrives after that grant is consumed.
A Claim commits, but its HTTP response disappears.
A compatible retry locates the same disclosed durable command record.
A pinned command-record read returns no row, or prior disclosure is not permitted.
A source-observed domain update exists but the selected physical writer did not atomically create the canonical command record.
A later manager Reassigns a Request after a helper's earlier Claim incorporated.
The acquisition route returns a card whose snapshot is older than the required mutation progression.
The authoritative HTML is installed, but browser continuity restoration then fails.
A fragment has neither initial authoritative content nor an admissible acquisition path.
A change occurs between initial page observation and SSE interest registration.
The transaction commits and the server crashes before it sends the declared invalidation.
One advisory is duplicated during failover or replay.
Router A loses state while Router B learns an interest after the relevant invalidation.
A Live stream reconnects with a changed principal or lost authorization.
A publication topic is declared but the actual model executor never emits it.
The browser receives an advisory but its target was replaced before the response completes.

Successful acceptance requires usable Create/Claim/Unclaim/Reassign histories where promised, including the remote advisory and user Refresh policy, not just a structurally valid but inert graph. No green assembly report establishes eventual progress without named fairness, network, replay-retention, provider and user-action premises. Parts VIII–X define stronger product, compositional and physical correspondence evidence separately.

Static preflight and required dynamic handling

Static preflight is mandatory construction/startup work for the managed application. It consumes the actual canonical construction and artifact products, rather than a manually reconstructed preflight-only registry. It runs at the earliest stage with enough information and produces structured obligations/results.

The supported managed constructor owns the admission sequence. Application authors must not remember to invoke a separate optional checker before starting an otherwise unchecked server. A narrower low-level or legacy startup path may exist, but it does not inherit the checked-managed policy, its issued executor rights, or production enforcement claims.

A successful **structural** preflight records the exact application generation, source-owned operation and failure catalog, selected evidence policy, constructed executable/asset products, and still-unmet installation, invocation, runtime and external premises. It cannot claim that the network listener adopted the guarded handler merely because a candidate assembly was checked. Installation, activation and serving adoption are distinct mandatory gates where the policy requires them.

The preflight checker rejects **known missing** mandatory edges, while preserving separate classifications for unsupported, inconclusive and externally assumed obligations. An opaque `:recovery-exists? true` field, copied success report or abstract terminal state is not a checked continuation. A proof checker that cannot model a required recovery transition must not relabel its inability as structural success.

Failure belongs at the earliest legitimate boundary

Boundary	Representative checks
Semantic construction	Malformed interfaces, contradictory roles/knowledge, invalid transition terms, unsupported declaration stru
Assembly/build	Missing operation realization, incompatible provider, wrong plan/route binding, incomplete required recover
Installation/activation	Wrong actual handler/executor, missing policy gate, unsupported lifecycle ordering, incorrect server/asset a
Concrete rendering	Missing bound argument, anonymous semantic route, unavailable action retaining transport, unsupported e
Request/effect admission	Malformed envelope, contradictory selectors, stale assembly/resource binding, invalid actual candidate or ef
Authoritative incorporation	Current permissions and dependencies, atomic guard failure, command binding/replay/disclosure rules.
Completion/recovery	Actual outcome, acquisition/installation evidence, unresolved delivery, partial effects, cleanup and retry con

A valid assembly can receive a rejected Claim because Membership changed after render. That is not assembly corruption. Conversely, a known missing clock binding should not be deferred until the first Claim throws a missing-input exception.

A different failure layer can become relevant **after** the previous layer succeeded. For example, a Claim route and its required sampled `:biff.fx/now` provider can be structurally closed at admission, yet the trusted principal may lose Membership before incorporation. The earlier check remains valid for its actual proposition; the runtime must still reject the now-unauthorized command under the model's current-state guard and preserve its real typed outcome.

A malformed Claim command envelope can be rejected at trusted request admission even if its rendered control previously passed Hiccup checks. An actual DOM target can retire during acquisition even if the response was correct. A post-commit response-render failure blocks the invalid output but never proves the earlier authoritative write failed. The diagnostic identifies the earliest **legitimate** boundary with the needed information without confusing a later detection with prevention at an earlier boundary.

Required handling coverage

Preflight need not predict dynamic facts. It must check declared handling/recovery paths for supported dynamic conditions that could strand an operation. A happy-path provider or reachable terminal node is insufficient.

The obligation identifies the condition, source contract, continuation input, owner, required resources/evidence, trigger, and permitted outcome. Construction checks the corresponding graph edges and type/binding compatibility. An opaque statement saying recovery exists cannot replace an available structural check.

Derive recovery obligations from the operation, not a second catalog

The source owner declares the selected operation's failure/outcome classes, delivery and recovery expectations, and admissible continuation semantics. Application construction derives a **coverage relation** for those classes through its installed route, physical providers, browser projection, Live and acquisition sources. A separate hand-authored recovery registry that repeats operation names is not independent coverage evidence.

Each required continuation is inspectable as a typed edge with:

Evidence dimension	Required content
Original condition	Which operation phase/failure/outcome class creates the obligation, and its source/version.
Trigger and permission	The real event, timer, explicit user action or retained work that may initiate continuation, including whether it h
Current owner	Command/work identity, assembly generation, principal or authorization scope, resource lifetime and transfer/r
Inputs and evidence	Typed prior result, original command binding, current authorized observation, progression lower bound or erro
Physical realization	An actually constructed and installed resolver, route, worker, provider, browser effect or explicitly qualified exte
Continuation contract	Which outcomes may be reported, what can be retried, what remains unknown, which effects or targets are forb
Applicability and failure	The preflight evidence mechanism, required premises, unsupported opaque behavior, boundedness and fallback

An edge is not covered merely because a function with a matching name exists. The expected result type must match the actual continuation input, and its owner must be able to invoke that continuation through a permitted live path. A required `:commit-unknown` resolver that can never retain the original command ID has no usable input; a valid GET acquisition endpoint without an authorized trigger cannot satisfy a user-mediated one-Refresh contract.

For opaque Clojure callbacks, a checked interface can establish the declared input/output and provenance relation only to the extent of the actual mechanism. It does not prove every callback return value, side effect or eventual completion. Required stronger properties need their own contract checks or named assumptions. A declaration whose supported checker cannot establish a mandatory relation remains unsupported/open under the selected policy, rather than green by unchecked assertion.

Condition	Required declaration and realization
Provider fails before submission	A typed local failure/retry path that does not claim authoritative incorporation o
Delivery is lost after possible submission	Unknown command disposition, retained binding, and a valid reconciliation ow
Prior result cannot be disclosed	A permitted nondisclosing continuation without false rejection or information l
Acquisition cannot establish required scope/progression	Retained debt and a declared stronger read, retry, defer, or blocked state.
DOM installation is partial or completion is lost	Repair/reconciliation without premature adoption.
Continuity restore fails after adoption	Local continuity failure without undoing authoritative installation.
Subscription misses changes	Gap recovery independent of receiving a fresh advisory.
Plan/operation version becomes incompatible	Retirement/transfer and retained command resolution under a compatible contr
Recovery budget or retention expires	Truthful deferred/blocked/manual handling with no fabricated authoritative disp

These checks cover the declared supported conditions. They do not prove that arbitrary external software has no additional failure modes or that recovery will eventually succeed. Unexpected failures enter an explicit containment boundary that preserves evidence, uncertainty, and surviving obligations.

Recovery states must preserve the kind of uncertainty

Some operational outcomes can be distinguished at one boundary but not at another. Before an authoritative attempt is initiated, a trusted provider failure may be a local no-submission result under the contract. After submission might have occurred, a network timeout, callback exception or publication loss cannot be converted into the same no-submission result unless actual command-specific evidence establishes it. An existing command-record resolver may prove a disclosed replay without proving current permission to issue a new write. Current-state reacquisition may show a new claimant without identifying an earlier Claim command's incorporation.

Each continuation therefore states whether it is: a guard that **prevents** an effect; a settlement based on authoritative evidence; a read-only reconciliation; permitted acquisition; a participant-local presentation/continuity repair; or a compensating **new** admitted operation. These cannot substitute merely because they share an exception handler. Failure after successful DOM installation cannot retract the installed observation; a failure of continuity restoration keeps a separate continuity obligation.

For an optimistic Claim, a structurally sound required unknown-delivery path retains the command's original argument/version binding, records a durable or explicitly qualified recovery owner, checks disclosure before returning prior results, and allows

a separately granted readback only when its progression and browser-target rights hold. Neither a generic HTTP 500 nor an optimistic `:rejected` toast is sufficient handling for unknown incorporation.

A path to a terminal node is not a termination theorem. Liveness depends on the stated availability, scheduler, permission, resource, and user-action premises. The application explanation shows those premises next to its progress claims.

The application need not promise that an unavailable provider, uncooperative user or partitioned router will inevitably recover. It must instead make its liveness contract conditional and testably explicit: which fairness or availability premises it needs, how long recoverable command/evidence material is retained, which obligations remain when those premises fail, and the authorized manual/deferred disposition. A timeout cannot be used to assert proof of termination, to discard a prior durable submission, or to make a permanently disabled required product capability count as completed acceptance.

Controlled-defect and positive-path qualification

Construction-time controlled defects deliberately remove **one** required edge from the actual source-owned assembly and expect a stable failure at the earliest applicable boundary. A corresponding valid assembly and fault-injection scenario must exercise the real continuation and the preservation of authority, uncertainty or physical installation. Tests that invoke only a helper checker cannot claim that the deployed managed startup or returned browser response actually enforces it.

History
A valid Claim model, plan, route, sampled time provider and authorized resolver are assembled and successfully exercised.
Claim requires `:biff.fx/now` but only a differently shaped clock provider is registered.
The browser plan requires an unknown-command resolver, but the selected operation assembly has no resolver or retained original bindin
A resolver exists but has no real authorized trigger, available owner, or compatible continuation input.
A valid rendered Claim is attempted after the helper's Membership is revoked.
A Claim submission might have committed but HTTP settlement is lost.
The durable command record is nondisclosable to the current principal.
A granted Refresh was consumed, and another Live advisory arrives.
The acquisition route exists but returns an observation below the required scoped progression.
A fragment subscription is established after a relevant missed change, but nothing new is published.
The DOM adoption finishes, then continuity restoration fails.
A valid Claim commits but its response renderer generates a malformed Cancel action.
One required recovery handler is replaced after installation without new admission.
Recovery queue budget is exhausted while a command may remain incorporated.
A checker hits its resource limit without completing a mandatory structural/proof obligation.
All invalid commands are rejected, but a valid eligible Claim or Reassign is permanently unavailable.

Negative cases must identify the exact source-declaration change, expected error/obligation ID, actual production entrypoint or qualified unit-check boundary, observed absence of a prohibited effect/output, and recovery/blocked behavior. A rejected build may still own cleanup of already prepared resources, but it cannot activate an unchecked candidate.

Resource policies are part of admission

Construction and runtime declare limits for model exploration, graph size, render materialization, response buffering, active executions, recovery queues, physical remnants, and evidence retention where those resources matter. Limits have typed outcomes and handling paths.

A resource limit cannot turn an unfinished proof into a successful check, a dropped advisory into discharged debt, or a lost command record into a rejected command. Static checking validates the declared policies; runtime enforces their actual budgets and preserves the appropriate scope of uncertainty.

Resource policy also owns work after an HTTP response or assembly generation retires. The contract specifies whether a command resolver, Live interest, deferred structured response, worker and physical target is cancelled, transferred, retained for bounded recovery or reported as unavailable. A process-local handle cannot silently survive restart as a durable certificate. Resource exhaustion must never leak an unbounded queue, promote a stale grant, retry a possibly committed external effect without an idempotence contract, or expose partially installed application operations.

A managed profile may select conservative limits to remain usable in a small deployment and more capable limits for a larger host. It must not convert a resource-limited checker into a different theorem, omit required operation/failure classes, or silently weaken the user's chosen application assurance policy.

Lifecycle, installation, and supported startup

The supported application startup path composes native lifecycle modules under a qualified integration contract. For a Biff application, that contract names the selected Biff module/system behavior and the server's handler adoption behavior. The design does not depend on an unspecified future release automatically performing Gesso's checks.

Every lifecycle owner has a stable qualified identity, dependencies, start/init semantics, owned resources, and cleanup behavior. Duplicate IDs, unknown required IDs, incompatible shared ownership, and required dependency cycles are rejected. A module is not accepted merely because it uses the expected ID while replacing the required implementation with a different function.

Lifecycle identity, resources and independent host choices

The construction retains a **module provenance** relation, not merely an ordered list of keyword identifiers. Each actual lifecycle module has a source owner, selected implementation, generation/profile, required dependency edges, side-effect footprint, resource ownership, initialization contract, start/stop behavior and compatibility scope. A built module with the correct `:biff.core/id` but another `:biff.core/start` implementation is not the admitted module. A shared XTDB provider, listener, subscription or browser asset reference needs explicit shared ownership and retirement rather than a duplicate cleanup callback.

The application derives mandatory Gesso preflight and guarded-output module requirements from its canonical assembly, while independently selected Biff and host-specific modules are checked against that selection. Duplicate IDs, unknown ordering targets, mismatched implementations, inconsistent provider owners and cycles affecting required ordering fail before application exposure. A valid module graph alone is not evidence that the host started the named modules, installed the anticipated handler, or serves their outputs.

Preflight and installation precede application exposure

The required partial order is:

1. obtain and validate the source/configuration and private resources needed for construction;
2. close semantic, build, provider, and selected assurance obligations;
3. construct/install the admitted router, response guard, execution/provider bindings, workers, and asset references;
4. check their actual required installation relationships;
5. activate the application-serving boundary.

A particular framework may have separate init and start phases. All phases must respect the exposure rule. Starting an application listener during an earlier init callback cannot be excused because the named preflight module later appears first in a start-order list.

There is no universal requirement that preflight be the first module in every system. Its real dependencies may need private initialization. There is a universal requirement that no managed application operation is externally available before its required admission and enforcement bindings are ready.

Health or administration endpoints can have separately admitted scope. They must not expose unchecked application behavior under a health-check exception. A listener bound privately or returning only a separately admitted unavailable response is different from a ready application listener; the profile states its observable behavior precisely.

Init-before-start and external-effect fencing

The selected host lifecycle matters. In released Biff 2, all module `:biff.core/init` functions run before the ordered `:biff.core/start` phase. Placing the Gesso preflight module first in the **start** order therefore cannot prove that an initializer did not already open a network listener, schedule protected background work, mutate protected XTDB state, or publish a browser resource under an unchecked configuration.

Initialization required for preflight may allocate private, non-exposed resources under admitted contracts. An initializer that creates an effecting resource must have a fence, disconnected capability, private binding or other independent guarantee that prevents protected external application behavior until activation. This applies to scheduled workers, callbacks, subscriptions and background tasks as much as to HTTP traffic. An initializer that must perform a durable effect needs its own authorized transaction/work contract; preflight cannot retroactively make it private merely by being sequenced later.

The externally observable activation condition is that **every** mandatory semantic, asset, route, provider, writer, render and policy obligation needed by the selected managed service has passed at its respective legitimate boundary. An early open listener is allowed only if it cannot serve the managed application before that condition, under a separately checked unavailable/administrative protocol. A health endpoint is not an exception permitting unchecked Claim or arbitrary model effects.

The ordinary constructor installs the guarded handler

The application start function derives or enforces the required modules and ordering. Application authors do not manually remember a preflight ID, wrap one test handler, or maintain separate checked-dev and unchecked-production entrypoints.

A schematic API sequence is:

```
(let [definition (application-definition config)
      candidate (application/prepare! definition)
      installation (application/install-private! candidate host-profile)]
  (application/activate! installation))
```

The contracts of these calls are substantive:

- `prepare!` admits the semantic/build candidate under its policy; it does not report that a server has started.
- `install-private!` consumes that candidate, constructs the actual bindings, records owned resources, and runs the relevant installation checks without exposing unchecked application operations.
- `activate!` consumes the genuine current installation and completes the declared exposure transition; it does not trust a caller-edited accepted flag.

Implementations can combine these calls behind a smaller public interface. Their phase obligations still hold. A constructor that merely returns an accepted map without installing the required path does not satisfy the contract.

The final server receives the exact admitted wrapped handler, or a qualified wrapper composition preserving its contracts. The application can inspect the actual server/installation relationship to the extent promised by the profile. Local system-map adoption, observed invocation, and deployed-service adoption remain separately scoped evidence.

Concrete Biff installation and serving adoption

The ordinary supported Biff 2 integration must identify its actual `:biff.ring/handler` owner and the handler consumed by the real Aleph or equivalent serving module. A canonical module can check the current `ApplicationAssembly`, require a callable handler, install a wrapped handler and remove an internal temporary assembly key. Those actions establish a concrete local system-map binding, but they do **not** independently establish that an already-started listener or a different server module adopted that handler.

Later Biff modules and mutable Vars can replace or invoke another callable after the checked module ran. An externally facing server can ignore the conventional handler system key, and a handler can return raw pre-serialized HTML bypassing the structured Gesso guard. The finished managed path must prevent or detect these substitutions within its claimed coverage, or retain explicit disqualifying deployment premises rather than reporting unconditional production enforcement.

An actual adoption receipt or qualification may include the issued handler/route identity, server instance/endpoint, admitted assembly generation, guard and policy version, asset serving reference and observed guarded invocation. A source-issued system map, a caller-provided receipt, or a single successful test request does not by itself prove perpetual adoption or completeness of every serving path. Controlled faults at the real boundary are required for the claimed operational scope. Section 07.11 owns that independent production-path evidence.

Request lifetime spans asynchronous completion

At entry, an invocation captures its assembly instance and concrete context. It resolves route, executor, providers, outcome encoding, render guard, and assets through those compatible bindings. Asynchronous completion carries this context explicitly rather than consulting whichever registry became current meanwhile.

An invocation's completion, cancellation, timeout, and transferred work determine when its assembly/resource references can be released. A handler returning a future is not necessarily a completed invocation. Streams and workers have separate declared lifetimes.

Old assembly bindings do not freeze current business authority. Each operation still checks the current trusted model/authentication premises required by its contract. If an old assembly or provider profile is revoked, its continued use requires the declared retirement/compatibility policy, not indefinite reliance on the fact that a request once acquired a reference.

An asynchronous Ring handler's returned value and its completed response are distinct lifecycle events. Closing the handler's lexical/dynamic scope when it returns a future, stream or callback must not release the request's generation, change the installed guard to a different one, or authorize use of a new executor/provider accidentally looked up through a global Var. The request's retained context supplies the checked response boundary through completion, rejection and cancellation.

Background jobs, Live streams and durable command-resolution work have their own admission and retirement scopes. Some obligations transfer across generations or process restarts using retained durable state; a process-local admitted handle does not become a durable certificate by serialization. A retired Request card target cannot be reanimated by an old callback, and current trusted model authorization is re-evaluated where an effect requires it. HTTP response completion alone does not discharge uncertain command incorporation, post-commit publication or physical DOM adoption debt.

Reload builds a candidate before activation

Development reload and deployment use the same coherence rule. A candidate is constructed and checked separately from the current admitted assembly. Activation switches the appropriate new-entry bindings as one admitted transition, using a compare-and-activate or equivalent mechanism that prevents concurrent reloads from replacing a newer instance with an older candidate.

The transition covers routes, issued executors, providers, assurance policy, render checking, asset references, and other relevant realization bindings. It cannot expose new routes with stale browser assets and an unrelated old report.

In-flight work remains bound to its still-valid assembly or follows an explicit checked transfer. New invocations use the new instance. Old assets and resolver contracts remain available for the lifetimes promised by the compatibility and recovery policy; reference counts alone do not account for sleeping browsers that are no longer represented by a live server request.

A rejected candidate may leave the prior instance serving only while that instance remains valid. If a security, backend, or semantic premise has invalidated the old instance, failed replacement is not permission to keep using it. The application suspends affected work or follows a declared restricted mode.

Relevant change invalidates dependent evidence. An incremental implementation can reuse unchanged checked subgraphs and artifacts, but must recheck their joins into the new root. Cache reuse is justified by complete dependency/applicability bindings; it is not an assertion that the source looks similar.

Generation-coherent activation and draining

An activation transition has a single authoritative choice of the new-entry generation for the selected deployment scope. It may switch an immutable dispatch/asset/provider composite by atomic pointer exchange, a short quiescence barrier, or an equivalent qualified host protocol. Updating several mutable Vars in sequence without isolating new requests is not coherent activation. A compare-and-activate check also prevents an older candidate that finished compiling late from superseding a newer admitted generation.

The transition records which old requests, streams, workers and browser clients may continue, which may only resolve previous commands, and which must be cancelled or migrated through an explicit compatibility contract. A browser page rendered under A can fetch its A-specific assets after B is current; it must not receive an incompatible B bundle at the same unchecked mutable URL. The retained old serving reference and command decoder lifetime cannot be inferred solely from active server request reference counts.

An invalidated security or model premise may require revocation of an old generation's **future effect rights** even while preserving its durable command outcome and authorized reconciliation. Keeping old code running because B failed preflight is permitted only while A's own actual predicates and deployment authorization remain satisfied. The shutdown of A does not authorize a premature deletion of resources still owned by B.

Cleanup preserves ownership and primary failure

Each lifecycle module owns the resources it acquires and any explicitly shared references. Failed startup retires completed private installations and accounts for partial acquisition, in dependency-safe order. Cleanup errors are retained as secondary diagnostics; they do not replace the primary admission failure.

No failed candidate leaves an unchecked application listener or worker active. Work already transferred to a durable admitted owner remains governed by its contract. Cleanup cannot erase a committed transaction, acknowledge unfinished durable work as complete, or cancel a replacement assembly's shared resources.

Shutdown and replacement preserve uncertain command/work obligations through admitted transfer or durable recovery. Best-effort physical stop follows semantic retirement; failure to close a socket or release a registry handle does not retain the old owner's authority to initiate work.

A failure is not necessarily all-or-nothing at the physical level: a module may open a socket and then throw without returning a completed start result, or a worker may publish an advisory before an activation attempt is rejected. The lifecycle contract records partial acquisitions and side effects as they occur, not only successful module return values. Failed admission prevents new

protected exposure but must retain cleanup failures, surviving external effects and transferred durable debt as distinct diagnostic facts. An already committed domain mutation cannot be undone by simply calling a stop handler.

Resource teardown is idempotent to the degree declared by its owner and ordered against shared dependents. The primary failure cause remains visible, with secondary cleanup errors and unreleased physical resources attached. A failed cleanup may demand process fencing, restart or operator action; it never silently upgrades a failed candidate into admitted service.

Positive and adversarial lifecycle histories

These examples apply to the **real managed constructor** and selected Biff/Aleph/Live profile. A fake startup map or a successful stand-alone preflight unit test cannot replace observed host/module execution where that evidence is required.

History
Canonical startup validates a real source-derived Claim/Reassign assembly, builds current browser assets, installs the wrapped handler and
A malformed browser artifact is detected before normal application service.
The Biff module uses the expected preflight ID but substitutes a different `:biff.core/start` callable.
Preflight is first in the ordered Biff start list, but an earlier `:init` function already starts a protected Aleph listener.
A module initializer starts a scheduled raw XTDB writer before admission.
The checked handler is installed in `:biff.ring/handler`, but the actual serving module captures another callable.
A later module or hot reload replaces the guarded handler while keeping the original report.
A valid handler begins under A, then returns an asynchronous response after B becomes current.
Candidate B installs new routes but still references A's incompatible browser assets or provider set.
Two reload candidates race and the older finishes after the newer activates.
Candidate B fails validation while a still-valid A is serving.
Candidate B fails, but A's authorization or security premise has been revoked.
A previously served page uses an A-specific asset reference after B activates.
A module acquires a socket and then fails before returning its startup result.
A cleanup callback throws while the original preflight failure is being reported.
An old request is retired while its Claim might have committed and its response or Live publication was lost.
The canonical HTTP guard blocks a malformed Reassign response, while a raw Ring HTML endpoint in the same process bypasses it.
A deployment starts healthy and serves an actual valid Claim with authorized reread and browser adoption.

Part VII's lifecycle contract applies to a single-node and to multi-node deployments with the same application-generation semantics. Additional network nodes, router failover and distributed activation require separately stated traffic placement, epoch/fencing, recovery and compatibility premises. There is no claim that Biff module ordering alone serializes an entire distributed deployment.

Production-path enforcement evidence

Checker correctness and production integration require different tests. A checker can reject malformed input perfectly while the actual application never calls it. The supported construction therefore provides observable seams for controlled faults at the real startup, route, effect, and response boundaries.

Evidence is an observed boundary, not the availability of a checker

A production-enforcement claim identifies the **specific installed application generation and deployment profile**, its selected assurance policy and every relevant physical path carrying that claim. It links:

```
source-owned semantic application
  → admitted plan, artifact and trusted route/executor
  → actual constructed Biff/Aleph handler and provider
  → network-facing request/effect/output boundary
  → observed guarded transaction, response and client adoption
```

These are different evidence subjects. A valid application report does not attest the selected server module. A checked handler in `:biff.ring/handler` does not prove a deployed listener consumed it. A successful Ring invocation does not prove a remote proxy uses the same route or serves the same asset bytes. A 200 response does not prove the authoritative model committed. A correct browser snapshot does not prove the required command was emitted from a real control, nor that an earlier command incorporated.

Each test report names its original source, concrete generation, entrypoint and binding identities, installed dependency/handler versions, checker or observer mechanism, physical observation and unresolved host premises. Tests may use isolation and controlled fixtures, but must state which real layers they replace. A report containing green flags without source-to- physical causality cannot become an issued executor, deployment attestation or transferable certificate.

Each enforcement case records:

- the admitted source/configuration and actual entrypoint under test;
- the controlled defect and the subject/edge it breaks;
- the earliest boundary with enough information to reject it;
- the expected structured diagnostic or typed disposition;
- the prohibited output or physical effect whose absence is observed;
- any already-performed effect or surviving uncertainty that must be preserved;
- a corresponding valid case showing the path remains usable.

A controlled mutation starts from a valid source-owned candidate and changes one specified dependency or installed realization; the expected failure must be attributable to that mutation. Changing an unrelated test oracle to reject every request is not enforcement evidence. Mutation selection includes **removing the guard itself**: tests which still pass when the real production enforcement connection is absent have failed to establish its presence.

Failure evidence is stage-specific. A prevented protected write requires an actual pre-effect guard and a checked absence of that effect under the stated physical observation scope. A mismatch detected after XTDB incorporation or response delivery is a **post-effect contradiction**, with retained incorporation/disclosure/recovery obligations, not prevention. A checker timeout, unavailable observation or incompletely instrumented host produces an explicit unestablished result, not an assertion that no effect happened.

A test can inspect an issued handler's identity and observed local Biff-system invocation, or independently verify a real network listener/route, or observe a guarded XTDB mutation. It describes exactly which of those links it established. Satisfying all of them for one fixture remains bounded physical correspondence under stated principal, environment, input and writer assumptions; it does not prove every arbitrary Clojure handler or all deployment traffic.

A test-only wrapper that rejects before the real entrypoint does not establish production enforcement. Removing the actual enforcement connection must make the appropriate integration test fail. The test must distinguish prevented effect, detected violation after effect, and inability to establish either.

Controlled defects exercise actual construction and installation

Controlled defect	Boundary and required result
Required plan omitted from the source-derived browser build	Build/admission rejects the missing realization.
Correct receipt paired with different asset bytes	Asset admission rejects; the candidate does not serve that pair.
Correctly named route installed with a different executor	Installation/invocation gate rejects the mismatched binding.
Required provider name exists but returns/binds the wrong shape	Static contract mismatch or concrete provider admission rejects
Required assurance policy is selected but its gate is omitted	Managed installation/activation fails; a standalone checker test
Render guard removed or an outer guard shadowed	Concrete malformed output cannot pass the managed delivery
Deferred response loses assembly/guard context	Completion rejects or uses the declared safe failure path; it can
Final renderer introduces an anonymous semantic action	Final structured-output checking detects it even if preparation
Post-check middleware changes the protected action	The transformation is rejected, rechecked, or lacks the claimed
Undeclared writer appears in the actual covered inventory	Required closed-coverage admission fails; otherwise the affecte
Candidate activation changes only routes and leaves old assets/providers	Coherence check prevents partial activation.
Post-commit rendering fails	Malformed output is withheld while incorporation/participant

These defects use isolated fixtures and the supported production constructor. They do not alter a real user's live data to test failure. The corresponding assertions observe the actual server/effect/output seam relevant to the claim.

The production seam must actually fail the negative control

Controlled faults are submitted through the normal managed startup or actual qualified runtime path, not by calling only a test-side validator with fabricated route/manifest data. In particular:

- A source-derived browser build test uses the real generated entrypoint and actual emitted bytes. A forged receipt tests receipt rejection, not correct compiler semantics or served network assets.
- A route/executor test verifies the issued callable and **actual** selected method, decoded subject and trusted principal. A string-equal route table supplied only to a checker does not establish installed dispatch.
- An operation/physical-write test captures the original command, guard and transaction identities and reads back authorized authoritative state. If the physical submitter can be replaced with a function returning a fake committed map, the test must not report authenticated incorporation without a stronger independent write observation.
- A render test exercises the final checked serialization path, including malformed **dynamic** Hiccup and relevant HTML/HTMX effective actions. The alternate raw or deferred output path is separately qualified or outside the claim.
- A lifetime test distinguishes early Biff module `:init` side effects, module `:start` ordering, actual host handler selection, a deferred completion, and a reload replacing the generation. Preflight's appearance first in the start list does not fence earlier init work.
- A writer-coverage test challenges the real effect boundary or independently qualified host inventory. An operation-ID list checked against another declaration list does not prove absence of unregistered raw writers.
- A command/Live test distinguishes actual model commit, durable command attribution, post-commit publication, baseline/subscription coverage, acquisition permission and physical browser adoption. A mocked SSE frame establishes neither an emitted invalidation nor delivery.

Every test's failure interpretation states whether it established rejection of an invalid candidate, rejection of an invalid invocation, blocked output, observed contradiction, or an unresolved external premise. Cleanup and preservation of prior committed work are asserted where relevant. An invalid candidate must not activate protected service, but a failed response check cannot retroactively reject a valid previously committed Claim.

Real browser acceptance starts from real controls

Top-level browser acceptance loads the actual generated runtime and operates the application's rendered controls. It observes the emitted protocol, selected route, trusted model result, returned evidence, physical installation, and subsequent authoritative read as required by the capability.

A genuine browser case starts with a server-rendered, source-owned HumanHelp control and its actual compiled JavaScript, real HTMX events and physical target. It records a causal chain from the control activation to the command envelope, selected trusted HTTP route, source-owned model operation, durable result, authorized acquisition and installation. A direct HTTP fixture sending an idealized command remains useful, but is only a server/protocol test.

The browser qualification includes expected loaded artifact identity, actual URL and asset bytes under the selected serving profile. The initial HTML and SSE connection provide their required authoritative baseline/interest relationship. Multi-client tests exercise remote advisory without granting automatic Refresh, then an explicit one-use authorized Refresh that establishes and installs a sufficient current observation. A lost response or duplicated advisory must retain command/Refresh debt without inventing command outcomes or extra acquisition permission.

Direct, same-process Ring tests and a real Chromium/browser over loopback HTTP/SSE are complementary. A loopback integration verifies its socket/bridge and observed client behavior, but does not by itself prove a public deployment's load balancer, CDN, credential boundary, all writers or inter-node router failover. The report retains that qualification rather than attaching one generic `:runtime-enforced` label to all environments.

It must not repair a missing command field by injecting a valid envelope or manually dispatching the event the button failed to produce. Such setup is useful in a lower-level protocol test, whose claim is deliberately narrower.

Acceptance includes valid repetition and concurrent activity, not just the first successful request. Request-scoped issuance must allow later requests and the admitted degree of concurrency. Out-of-order completions must preserve separate invocation/command/target ownership. A one-shot test harness cannot establish a reusable production route.

Independent positive product completeness

The tests separately declare the minimum required **real capabilities** that constitute HumanHelp acceptance: an eligible creator can create and immediately see a Request; an eligible helper can Claim; authorized users can Unclaim, complete and cancel as their models permit; a manager can Reassign; participants can observe initial state and later advisory/Refresh behavior under the prescribed ownership policy. Each capability identifies a positive reachable scenario, authoritative result, valid control and, where applicable, physical presentation.

A reduced application can be internally consistent, reject every malformed operation and display a coherent empty page while entirely omitting Reassign or Claim. Structural and negative enforcement tests may pass; the product- completeness gate must fail. Positive acceptance cannot be satisfied with a hidden or permanently disabled button, manually injected protocol message, test-only model callback or fabricated server response. A test may isolate a trusted external provider with a qualified fixture, but it must record exactly which effects and assumptions that fixture replaced.

Tests also exercise valid **subsequent** operations after one successful command and at least the concurrency policy advertised by the application. Two legitimate concurrent Claim attempts are not supposed to both acquire an incompatible RequestAssignment. They should instead reach independent request-local admission, then receive truthful guarded incorporation, rejection or recovery under the declared atomicity model. Rejecting one attempt because a global single-use test slot was already occupied is not evidence of normal concurrent application usability.

Expected fault diagnostics are asserted specifically. Broadly ignoring console, page, or request errors would hide the integration failure this layer exists to catch. Conversely, a deliberate network-loss test need not pretend no diagnostic should occur.

Positive and adversarial enforcement histories

The following histories illustrate independent evidence propositions, not a universal single test fixture:

History
The installed managed application serves a real eligible Claim control, which commits and is followed by authorized card acquisition and p
Removing the actual managed startup/preflight module still lets the test's standalone validator pass.
An early Biff `:init` callback starts an unchecked protected worker before preflight's first `:start`.
A correct checked Biff system map is built, but the actual server captures an older handler.
A locally issued Biff host-adoption handle matches a copied returned system map.
The browser receives a correctly named JavaScript asset but its emitted bytes differ from the checked receipt.
Two valid, independently constructed products exchange an execution handler, Live fragment, or asset while preserving familiar operation
A malformed semantic POST loses `:choreo/op` only after final conditional Hiccup rendering.
A native detached submitter or second `hx-*` action bypasses a checked button's main transport.
The selected server route decodes Request R2 while the browser command binds Claim to R1.
A request's selected provider advertises `:clock` but does not supply the consumed `:biff.fx/now` shape.
A physical XTDB submitter is replaced by a fake `:committed` result that performs no write.
A valid Claim incorporates but the response's Cancel affordance is malformed.
A failed settlement leaves the Claim command uncertain while a later card happens to show a helper.
The Live baseline precedes interest registration and a relevant update occurs between them.
A remote client receives duplicated advisories after using its one permitted Refresh.
A deferred response retains an old generation while a new handler and assets activate.
A scheduler or raw XTDB path changes protected state without appearing in the registered operation inventory.
Two consecutive valid Claims on independent Requests are rejected by a shared consumed invocation slot.
An application passes all negative checks but omits the required Reassign control and executor.
A Chromium test injects a valid Claim envelope instead of clicking the broken actual button.
A network-loss scenario produces its declared command-unknown diagnostic while all unexpected browser errors remain visible.

Tests exercise the original admission and effect boundaries with isolated disposable state and explicit failure injection; they do not corrupt real users' records. The report separates **prevented effect**, **observed post-effect violation**, **physical completion**, **current-state read** and **historical command incorporation**. Known open assumptions, unsupported probes and test-environment substitutions remain in the provenance of the reported claim.

Sections 07.12–07.15 must expose these same distinctions through explanation, diagnostics, acceptance qualification and the security invariant. Parts VIII–XII retain independent positive product proofs, formal/model correspondence and deployment/release assurance. An implementation may make substantial progress on one layer without being entitled to the stronger adjacent layer's certificate.

Application explanation and semantic audit

The application exposes a structured, read-only semantic audit derived from its actual admitted sources, construction products, and evidence. It supports review of the correctness-relevant policy without treating a large implementation as an undifferentiated body of code.

The audit is not a second editable policy registry. Business-rule prose, operation cards, diagrams, and evidence summaries are projections of the same admitted subject. Prose explains meaning and rationale; it does not gain mechanical force by appearing beside a checked predicate.

Inspection scope and source lineage are part of the artifact

An audit has an identified **subject**: the admitted application generation, original semantic/source versions, selected operation catalog, assembled realization/deployment profile, checker versions and evidence dependencies. Views are derived from those same objects, not from a second registry, a stale report-shaped map or a source scan that merely found familiar names. A current, successful assembly report can authorize **describing that assembly**; it is not itself authorization to execute operations or a proof that an uninspected model callback implements its descriptive policy.

The root view states, at minimum, which declarations and real sources were inspected, the admission/checking mechanism, which evidence is only source-relative, the completeness of each inventory, and the claims not established. Counts always have an inventory qualifier:

Count or scope	Permitted interpretation
Declared operations and relational rules	Exact members of the selected admitted source catalog, not proof that no other business r
Checked finite state/operation/rule instances	Exercise over the stated initializers, principals, arguments and reachable-state bounds, not
Observed routes, writers and browser artifacts	Physical observations under the named host/serving scope, not all unobserved entrypoint
Open declared structural edges	Failed or unmet edges within the known closed subgraph, not an inventory of human inte
Opaque model callbacks and external effects	Preserved named premises and unsurveyed implementation surface; unknown never mean

An omitted semantic rule that never entered the declared source cannot be proven absent by successfully checking its smaller catalog. The source-owned rule catalog, application/operation cards, proof dependencies and runtime observations therefore retain both their positive **included** inventory and the relevant completeness/opaque qualifications.

Required audit views

View	Information and required limits
Business-rule catalog	Stable rule identity, meaning/rationale, formal or opaque contract, source, affected operations, authority, enf
Operation card	Typed inputs, principal/authority dependencies, guards, transition/frame, outcomes, command repetition/di
Authorization/disclosure	Current authority sources and policies for invocation, reads, subscription, and prior-result resolution.
Domain lifecycle	Actual admitted states and transition relation, generated from semantic definitions rather than guessed from
Realization path	Exposed surfaces, plans, assets, routes, actual handlers/executors, providers, acquisition/installation, and ser
Rule/evidence dependencies	Claims affected by a changed rule, operation, implementation, backend profile, or accepted premise.
Writer/entrypoint coverage	Declared, installed, observed, classified, excluded, and still-unestablished paths at their actual scope.
Opaque-premise report	Each retained premise, its owner/reason, applicability, available tests, and dependent claims.
Assurance and exercise	Exact finite bounds, rule/operation/outcome exercise, conditional induction, runtime gates, conformance res
Recovery/continuity	Owners and triggers for unknown commands, failed adoption, missed publication, subscription gaps, and ve

A domain lifecycle view uses the operation model's actual enum. It must not rename HumanHelp's states to superficially similar completed/cancelled labels or infer terminality from a UI caption.

Business rules, operations and actual authority

The rule catalog is a normative future capability whenever the application expresses relevant semantics. For each named rule it records the source-owned identity, exact proposition or relational terms (when machine-readable), domain meaning and rationale, affected operations, authoritative owner, required enforcement site, source-to-implementation correspondence, and independent evidence or retained premise. A rule that has a formally stated predicate but no demonstrated XTDB transaction guard remains a source declaration with an **unestablished physical conformance** edge, not a production-proved invariant.

For example, HumanHelp Claim's active Membership condition must identify that the authenticated helper's current Membership in the Request's Organization is checked at authoritative Claim incorporation. The audit must distinguish that declared obligation from evidence about the actual guarded transaction, actor/Request/Organization scope and interference. A matching rule name, handler name or passing finite fixture cannot establish current live Membership protection.

Operation cards preserve an executable contract rather than a marketing summary: typed Request and principal/argument sources, authority, guards/prestate, transition and frame, command identity and duplicate handling, authoritative outcomes and disclosure, provider/effect requirements, physical route/plan/asset, and the permitted acquisition and recovery paths. A current participant's authorization is determined by the trusted model and observation context, not by reading a previously generated audit card. Rendering an eligible or unavailable button does not change what Claim is allowed to do.

The authorization/disclosure view distinguishes at least **invocation**, **current Request read**, **subscription/Live interest**, **prior command result disclosure**, and manager Reassign authority. Sharing a role label does not establish equivalent permissions. Current user policy and tenant/Organization context are dynamic model facts, even when the application has structurally established the existence of their trusted enforcement bindings.

Where model states and transitions are declared, the lifecycle view reports their exact internal identifiers and transition relation. It cannot infer an unexpressed ‘:completed‘ or ‘:cancelled‘ state from a UI label, or mark a terminal state because a component hides its actions. A model exposing no inspectable state relation yields an explicit unknown/opaque lifecycle view, not an invented diagram.

Unknown inventory is not an empty inventory

If a model adapter exposes only a public operation name, the audit cannot infer its internal predicates or atomic dependency protection. It reports those bodies as not inspected with their explicit conformance premise. An empty business-rule list would misleadingly suggest there were no relevant rules.

The same distinction applies to writers, side effects, publication, and policy branches. Zero unresolved declared edges is not proof of complete human intent or absence of undiscovered behavior in arbitrary host code.

The audit must distinguish three situations that are often mistaken for one another: (a) a **known complete** catalog with zero entries of a particular class; (b) a partial catalog containing some inspected declarations; and (c) an unavailable source whose cardinality is unknown. A zero count is valid for (a) only within its explicit closure scope. An empty vector shown alongside `:count :unknown` in (c) means **no enumerated rules were available to this projection**, not that the actual application has no rules.

Opaque boundaries remain explicit when registered model functions can branch on Membership, perform unreported reads, mutate protected state or publish topics outside the admitted declarative surface. Where the supported semantic language has precise relational or state-transition terms, the same audit can enrich those rule cards with meaningful checked facts. It must not infer that arbitrary adjacent Clojure was consequently proved equivalent.

Enriching the application with source-owned relational rules or checked transition terms adds concrete audit coverage. It does not retroactively establish that uninspected callbacks contain no extra policy. The view distinguishes complete coverage of a supported construction subset from registered opaque boundaries and unexamined external behavior.

Declared Live overlap is not a transaction dependency proof

Part IV's guard and footprint contracts determine authoritative dependency meaning; shared Live topics establish presentation impact only.

A **dependency** view has typed edges and provenance. A rule-to-operation edge may be source-declared, derived from the admitted transition relation, or observed from a specific physical guard. A Live topic overlap is instead a source-relative **presentation impact** edge. Operations that touch different protected records can publish one shared board topic, while two operations that share one guarded RequestAssignment dependency can legitimately publish different topics. Reclassifying either relationship as complete transactional conflict information would mislead application assurance and incremental invalidation.

For a given changed predicate, model operation, opaque callback, provider, route, generated plan, artifact or host deployment premise, the audit can identify **known downstream dependent claims** and which admission/checker/production evidence must be revisited. A missing source-ownership edge or an opaque implementation with unknown side effects creates an open impact boundary; the tool cannot promise that the listed dependents are complete. Reusing evidence requires matching original subject, source and implementation versions, proof/certificate premises, physical profile and checker semantics, not just equal operation IDs.

Explain one operation end to end

A useful explanation of Claim identifies:

1. the exact semantic source, rule dependencies, and operation interface;
2. exposed controls and their selected realization alternatives;
3. projected roles, executable identities, physical handlers, and asset bindings;
4. route/input correspondence and the actual installed executor;
5. provider contracts, invocation gates, and current authoritative guard obligations;
6. model-to-presentation outcome mapping, including repetition and unknown;
7. acquisition, propagation, installation, and recovery paths;
8. assembly/installation identity, policy, evidence applicability, retained premises, and unresolved or excluded claims.

The output may show a local issued-handler binding as established while leaving backend transition refinement conditional on its qualified provider. It may show finite rule exercise without claiming that every production predicate branch was executed. Those distinctions are useful information, not deficiencies to hide behind a single green label.

Assurance classifications are proposition-specific

A rule's **declared guard binding**, an abstract invariant, a finite exploration result, conditional induction, the physical XTDB realization of that guard, the deployed writer-coverage boundary, and real browser adoption are different claims. Each report row names its proposition, source/evidence class, checking mechanism, exact scope, assumptions, dependencies, proof or observation status, open contradictions and validity lifetime.

In particular, ‘assembly-verified‘ can describe correct source-owned route/plan/provider structure, while a model-implementation-conformance proposition remains ‘trusted‘ under an explicitly named premise. A finite model checking every selected principal/argument pair does not prove those pairs cover all humans or requests. A conditional inductive argument does not automatically establish physical XTDB refinement, all-writer coverage, kernel soundness or no-gap SSE delivery. A bounded physical observation cannot be promoted to universal deployment correctness.

The explanatory surface retains unsupported, inconclusive, contradicted, open and established evidence at their individual subjects. Contradiction to one physical realization cannot be hidden by a successful structural check of another. Nor can a checked rule be advertised as exercised when the finite exploration visited no state in which its source predicate applied.

Inspection does not execute or authorize

Explanation reads admitted results and immutable/redacted trace values. It does not invoke operation callbacks, submit transactions, rerun model builders, mutate runtime registries, or repair missing edges. Expensive fresh checking is an explicit separate action whose result has its own identity and scope.

No diagnostic output needs to expose live executor handles, credentials, principal-sensitive data, or unrestricted command arguments. Redaction reports what is unavailable rather than fabricating a replacement explanation. Internal repair detail and client-facing errors can have different disclosure policies.

The inspection API must not convert an arbitrary supplied report into freshly admitted truth. The normal public path re-derives or verifies the current application assembly and its source dependencies before producing an audit. A lower-level projection accepting an already checked report has a strict caller precondition; it is not an authenticated report verifier in isolation. A private issued executor, proof/checker handle or live DOM/Request object may be inspected only through its qualified boundary and is never copied into public audit output.

A source location may be important for repair but still sensitive. The selected diagnostic consumer and privilege determine which source paths, rule literals, principal scopes, commands, tokens and observations may be disclosed. A redacted field is explicitly marked redacted/unavailable and cannot be used as evidence that the hidden dependency is absent. Deterministic view ordering, stable semantic identity and bounded generation make comparison and review practical; these are UX properties, not additional execution authority.

The tool can suggest which **source owner** needs correction, but it must not automatically rewrite a rule, select a near-matching operation for dispatch, change a provider binding, or use a report as a new command. A separate source correction and admission is required to change the application's meaning. Sections 07.13–07.14 own structured diagnostics and full acceptance qualification.

The explanatory surface can show an incomplete candidate's rejected/open obligations without making that candidate executable. A stale sidecar or trace mapping must not explain a different executable plan by matching similar names; Part V's exact correspondence rules apply.

Positive and adversarial semantic-audit histories

Acceptance exercises the audit through its source-owned application explanation path. It distinguishes a view of an admitted candidate from a deployed or behaviorally accepted system; tests of individual projections have a correspondingly narrower claim.

History
A source-owned Claim rule explicitly requires active helper Membership at commit and is connected to an admitted model transition and c
A Claim operation is exposed through a browser plan, HTTP POST, issued executor, typed provider and authorized reread.
A legacy model adapter exports only a public Claim name and opaque callback.
A checked finite relational rule has never matched a source entity in any explored state.
An induction proof conditionally preserves an abstract single-owner invariant but host writer inventory is incomplete.
Claim and Cancel both publish `:request/change`, but only their Live topics overlap in the observed source.
A raw scheduled writer modifies RequestAssignment outside the declared operation registry.
The selected manager Reassign operation has correct semantic structure but is absent from the installed controls.
An application reports zero declared structural obligations while its opaque model code has uninspected authorization branches.
A report-shaped caller map asserts a fully checked rule catalog without a fresh trusted source/assembly.
A stale browser source map or artifact sidecar names the right operation but belongs to another generated bundle.
A candidate changes a provider Var, guarded callback or source rule but reuses last generation's green coverage report.
A command has an authenticated historical incorporation while a later Reassign changes the current card.
A limited-privilege caller requests an audit containing private principal or command information.
A UI repair agent sees a nearby valid operation name or provider adapter in the report.
A real HumanHelp implementation passes structural proof/report checks but cannot execute an eligible Claim or manager Reassign.

These cases require concrete original sources and honest evidence for the selected application and audit level, not an independently maintained idealized rule document. Part VIII's real browser product tests and Parts IX–X's formal/physical proofs can **add** evidence to the same source-bound audit when justified; a copied or self-issued report cannot substitute for them.

Structured diagnostics and repair

Diagnostics identify the failed obligation and its actual owner. Human-readable messages are rendered from stable structured data; tools should not parse prose to discover whether the missing edge was a route, provider, asset, or recovery continuation.

Diagnose the original proposition and its legitimate detection stage

A diagnostic is a source-bound **explanation of a failed or unfinished proposition**. It is not a new application semantic fact, a replacement operation, a trusted executable or a certificate. The ordinary diagnostic surface derives from the current canonical assembly and actual checker result; it cannot accept an arbitrary successful caller map as proof that the inspected source or installed handler is current.

The framework distinguishes a candidate that could not be admitted, an admitted but not installed construction, a failed actual invocation, a known post-effect contradiction, and a participant-facing outcome that remains uncertain. A single generic `:error` or HTTP 500 cannot safely express all of them. In particular, a browser's reported `:unknown` after Claim settlement loss is not a failed preflight, an authoritative `:rejected`, or evidence that the transaction was rolled back.

A stable diagnostic identity includes a **kind** and a scoped obligation/subject identity. Its generation, semantic source, route/plan/asset, original command or observed resource identity, evidence and checker version, and relevant deployment profile are separately retained when applicable. A copied diagnostic is valid information about its former subject, not automatically about a freshly changed generation. Diagnostic vocabulary is versioned so tools can distinguish renamed public keys from actually changed failure semantics.

A diagnostic includes, where applicable:

- the error kind and detecting phase;
- operation, projection, route, resource, or claim subject;
- assembly/installation and relevant executable/artifact identities;
- source location that introduced the obligation and source responsible for supply;
- expected relation and actual observed/bound values, within disclosure limits;
- rejected, unsupported, inconclusive, contradicted, or inapplicable status;
- remaining premises and whether any protected effect already occurred;
- mechanically known repair alternatives without automatic substitution.

A report also states **which relationship was actually checked**, which original source supplied each expected condition, which independent observation (if any) supplied the actual value, the checker or host trust boundary, and whether the failure disposition is `rejected`, `open`, `unsupported`, `inconclusive` or `contradicted` at the specific proposition. Unobserved means unobserved, not false; copied source declarations cannot authenticate their own physical adoption.

When a checker emits multiple issues, a deterministic `first-blocker` is a navigation aid with an explicit priority rule, not a claim that no other obligations are open. The full explanation retains the applicable open/assumed/dependent obligations, subject identity, and whether discovering another fault requires repairing the first. Stable sort order and machine-readable paths make diffing repeated diagnosis possible without creating executable authority.

An illustrative provider error is:

```
{:error/kind :incompatible-capability-provider
 :phase :assembly/admission
 :assembly-instance/id candidate-id
 :operation/id :request/claim
 :obligation/id :claim/commit-time-input
 :introduced-by claim-source-ref
 :supplied-by server-binding-source-ref
 :expected {:input-path [:execution-context :biff.fx/now]
            :value-contract :time/instant}
 :actual {:input-path [:execution-context :clock]
          :value-contract :clock/zero-argument-function}
 :suggestions [{:kind :explicit-binding-adapter
                :contract/ref sampled-invocation-time-ref}]}
```

The suggestion does not install that adapter. Source correction and admission establish the new binding. Likewise, an unknown `:request/claime` may suggest `:request/claim` diagnostically but must never be corrected automatically for execution.

The provider example must be interpreted at the **consumer's input path**. A named `:clock` capability that returns a zero-argument function is not the `Instant` required at `:biff.fx/now`. A diagnostic can propose a source-owned sampled-time adapter when the

adapter contract is mechanically known; it must not invent an implementation, assume the clock is trustworthy, silently provide a default time, or modify Claim's transaction semantics. If the provider genuinely failed only at invocation, the diagnostic identifies that runtime fact rather than retroactively claiming the builder proved its behavior.

Source-local repair is a separate, re-admitted action

Diagnostics describe **candidate actions**, not repairs that have occurred. Every suggested source correction names the owner of the declaration that must change, the exact broken relation, the known alternative interface (if any), the verification and deployment stages that would need rerunning, and the extent of evidence that the suggestion itself has. A human or coding agent may separately choose to edit the source under normal authorization and version-control rules. A rebuilt assembly, browser artifact, route and assurance report then require fresh checking of the changed dependencies.

A repair tool must not treat a near-match operation string as permission to dispatch it; rewrite a protected role, tenant or Request selector; add an anonymous HTMX fallback; substitute a new executable merely because a report names it; or automatically retry a possibly incorporated command. Where a domain **compensating operation** is legitimately needed, it is separately admitted and authenticated under its own current guards, command identity and effect contract. No diagnostic callback gains the runtime rights needed to perform that operation.

A proposed fix can have multiple legal implementations. The checker explains what is established and what is unknown rather than guessing a unique implementation from a plausible name. After source change, it rechecks the canonical assembly and all dependent generated, installed and runtime evidence; it cannot simply edit the old closure report's `:status` field to green.

Stable diagnostic families

Family	Representative distinctions
Semantic construction	Unknown operation, duplicate identity, invalid interface, contradictory transition, incompatible realization.
Plan/build	Missing plan, wrong plan/operation, stale manifest, asset-byte mismatch, incomplete asset set, unestablished build
Dispatch/execution	Missing/ambiguous route, selector conflict, wrong issued executor, unsupported route semantics, unclassified writ
Provider/effect	Missing provider, incompatible value or origin, invalid actual candidate, wrong target, unsupported lifetime.
Rendering	Missing arguments/basis, anonymous semantic route, implicit downgrade, conflicting attributes, unknown effectiv
Recovery	Missing continuation, incompatible outcome mapping, absent owner/trigger, unsupported retained command con
Installation/lifecycle	Missing gate, wrong actual handler, premature exposure, assembly-instance mismatch, invalid dependency, failed
Assurance	Unsupported fragment, inconclusive budget, counterexample, inapplicable evidence, unaccepted premise, insuffici

These families need stable public identities without requiring this document's exact keyword spellings. A missing resource and an existing incompatible resource should not be indistinguishable. Dynamic authorization denial retains its model outcome and privacy policy; it is not reported as a corrupt assembly.

Concrete R4 error distinctions remain mandatory

A conforming diagnostic surface can explain the following independent facts, without requiring these illustrative keyword spellings as its public API:

Diagnostic subject	Required distinction and useful evidence
Unknown semantic operation	Original rendered view/source and unknown identifier; admitted operation IDs and safe near ma
Missing browser plan	Admitted operation and its exposing role/view exist; the required projected plan is absent. Show
Stale generated artifact	Current source-derived manifest versus actual emitted bytes and current receipt/sidecar. Report
Transport/route mismatch	Differentiate no route, method/protocol mismatch, overlapping/ambiguous route, wrong decoded
Anonymous protected HTMX action	Effective concrete method/path, matched semantic route, originating Hiccup/control, and missing
Missing or wrong capability	Source operation and consumer input contract (including `:biff.fx/now`), selected actual provide
Missing initial/current acquisition	Logical fragment/owner, required authoritative projection, progression lower bound, permitted t
Missing recovery continuation	Original uncertain command or failure class, required owner/trigger/input, exact resolver/acquis
Lifecycle or deployment mismatch	Candidate versus installed generation, actual Biff module/handler, served asset binding, failed ga
Unsupported or inconclusive assurance	Exact obligation, checker/budget or unsupported language, retained premises and known witnes

An unavailable, wrongly typed or ambiguous resource must not be silently classified as merely absent. A known physical violation must not be relabeled an open proof obligation. Likewise, an authorization denial for a current principal and Request remains a typed model outcome, not evidence that the application graph is corrupt.

A diagnostic must not claim semantic verification succeeded merely because it reached a later code path. It reports only established evidence. Source summaries, artifact stamps, and provider declarations can be useful repair facts without being proofs of their implementations.

Disclosure, sidecars, and diagnostic noninterference

Source-level internal diagnostics and user-visible response errors may contain different details under distinct disclosure policies. A public failure must not leak another principal's command history, Membership/Organization contents, private Request identifiers, session material, tokens, executable handles or unrestricted host source paths. Redaction preserves explicit `unavailable`≠`redacted` markers and the non-sensitive identity needed to correlate a permitted repair; it cannot infer that a hidden obligation is absent.

Browser diagnostic sidecars must correspond to the **exact** generated executable plan and artifact before a source location or trace is treated as authoritative diagnostic provenance. An old sidecar with matching operation names is not a valid map for a changed bundle. Browser debugging observers are non-authoritative: enabling, disabling or throwing from the observer cannot change the semantic trace, adapter effects, resource ownership, settlement, execution continuation or permission to perform physical work.

Optional diagnostic probes may be bounded and opt-in where they execute expensive model checks. Their failure or timeout is not permission to weaken a mandatory startup gate, abort an unrelated admitted command after incorporation, fabricate a passed proof or suppress a critical runtime error. A diagnostic system may fail safely while the application's underlying admitted behavior and obligations remain as specified.

Positive and adversarial diagnostic histories

Acceptance checks stable machine-readable failures against the actual relevant source/physical boundaries. A correct looking test report by itself is not sufficient evidence of installed enforcement.

History	Required di
A source-owned Claim has all required plan, HTTP route, typed provider, issued executor and authorized acquisition edges.	Explanatio
A rendered control references `:request/claime` while the catalog contains `:request/claim`.	Reject the u
A plan exists for Claim but the current source-derived manifest omits it.	Report mis
A receipt belongs to the previous generated bundle, though its filename and operation name match the current asset.	Report exa
A route is installed with the wrong method or matches two operations under the actual router's precedence rules.	Distinguish
A native form submits anonymously to the protected Claim POST route.	Report effe
A Claim provider lists `:clock` but the handler requires sampled `:biff.fx/now`.	Report sou
A managed fragment has no initial observation or permitted authoritative reread.	Report acq
An unknown command has a read-only resolver but no retained ID, authorized trigger or disclosure contract.	Identify the
A current authenticated helper loses Membership immediately before Claim.	Preserve su
The real server ignores the checked Biff handler and serves an older raw renderer.	Retain loca

History	Required di
One checker times out while another finds a concrete counterexample.	Preserve th
A first-blocker summary names a missing clock, while a second plan/acquisition problem also exists.	Show deter
A copied green report claims physical write coverage and an agent proposes reusing it after provider reload.	Refuse repo
A browser observer throws while a valid local execution is running.	Preserve ex
A low-privilege browser receives a diagnostic whose internal version mentions another user's protected command record.	Redact the
A Claim has already committed, but response rendering fails on a malformed Cancel action.	Distinguish
A remediation suggestion proposes a replacement handler, while a required external dependency or callable remains opaque.	Show the li

These histories retain R4's requirement that structured errors are genuinely actionable. They also reject ”helpful” automatic changes that could silently alter authorization, operation identity or command repetition. A source-derived explanation can support both human and coding-agent repair, but the subsequent edit, build, admission, deployment and tests remain ordinary independently authorized work.

Acceptance, qualification, and change impact

The application is usable as well as closed

The three claims in 07.01 are assessed at different boundaries. This section owns **independent acceptance and qualification**, not a second Choreo or Request business model. A closed application may still lack eligible Claim, manager Reassign, initial board knowledge or one-Refresh adoption. Disabled controls or negative tests cannot satisfy the positive product profile.

The oracle describes intended observable behavior, not duplicate executable rules. Disagreements require review rather than silently rewriting either specification or implementation.

Three independent gates and a stable product oracle

Acceptance reports separately: (1) structural assembly closure, (2) enforcement through actual deployed request/effect/output paths, and (3) the application's promised **positive** behavioral completeness. Success at one gate cannot mask failure or unsupported scope at another.

The product acceptance profile describes user-visible capabilities and authoritative outcomes independently of the **current** registry. It is not a second implementation of the business model. HumanHelp's required profile includes creator Request creation with immediate visibility; helper Claim and Unclaim; permitted completion and cancellation; manager Reassign; relevant board search/sort/filter behavior; initial authoritative acquisition; and another participant's advisory followed by explicitly authorized one-use Refresh. Each promised capability has a positive reachable scenario, real rendered control, actual model result, and required physical observation, alongside rejection and unknown-outcome recovery cases.

Deleting Reassign can leave a smaller internally closed application; it must fail the independently selected product-completeness gate. A permanently disabled Claim, hidden manager action, synthetic protocol envelope or reject-all server does not substitute for usable positive behavior. A disagreement between the independent product oracle and source-owned execution creates a specification/ implementation review obligation, never automatic rewriting of the oracle to match the current route table or vice versa.

Tests name exactly which providers, authority, browser, Live, backend or deployment relationships their fixtures replaced. A direct trusted-route fixture is useful but cannot satisfy the real-browser end-to-end requirement by manufacturing the missing rendered-control-to-command edge.

Conformance is bound to actual subjects

Evidence records the source/implementation identity, assembly/configuration, provider and browser profiles, test/checker version, exercised paths, bounds, and known limitations. A passing fixture with mocked providers is not evidence that an unexamined production transaction or subscription implementation conforms.

Each backend profile names the assumptions required by dependent claims, including atomic dependency protection, command-record incorporation, outcome interpretation, read progression, retention, and failure/retry semantics where used. A current or future backend version is qualified against those same assumptions. Detecting a violation invalidates or blocks the dependent claim; it must not be hidden by a version update or a stale successful receipt.

Backend qualification checks actual behavior, not labels

The selected backend profile identifies its exact query, basis, write, transaction and retry interfaces and expected behavior: guarded dependency evaluation, transaction-internal prestate, atomic commit and conflict handling, canonical command-record co-incorporation where promised, outcome interpretation, authorized read progression, retention, and failure uncertainty. Loaded backend/adapter identity, deployment and option surfaces, tested schedules, checker version, assumptions and independently unobserved facts are separately recorded.

For XTDB, successfully checking a reviewed 2.2.0-beta1 API, query/write option keys and one-shot gateway fixture is a meaningful **interface prerequisite**, but is not proof of actual internal transaction state, authenticated physical causality, all production writers or universal abstract-to-XTDB refinement. A compatibility test may pass while a stronger atomicity or progression premise remains unproved or is observed to fail. The corresponding propositions must retain different results.

Qualification can layer API/interface checks, bounded real transaction tests with disposable state, guarded source-to- physical correspondence checks, independently established host/deployment facts, and stronger future proofs. An unsupported or changed backend feature does not acquire the older proof's applicability merely by returning the same operation IDs or a familiar version label. A strict policy rejects use of required unqualified behavior before the protected effect; weaker scopes must be separately selected and accurately disclosed.

Conformance tests can expose an assumption violation without proving the entire backend. That is a useful and intended contract. Unknown qualification after a relevant upgrade remains unknown until the selected admission policy establishes sufficient applicability; a changed version number is neither proof of failure nor proof of continued correctness.

Dependency-specific change impact and evidence reuse

An admitted assembly retains typed dependencies from semantic rule, initializers, operation families, principal/authorization contracts, executor and provider closure, routes and actual writers, browser source and emitted assets, Live acquisition, backend interface/behavior, deployment and checker/proof policy to the particular proposition each supports.

Change	Dependent evidence or behavior to revisit
Business rule, guard, operation semantics or arguments	Source-model checks, nonvacuity, executor correspondence, role/browser pr
Handler, callback, provider or mutable Var	Exact issued implementation, typed input/effect, installation and physical er
ClojureScript/compiler, plans, assets or cache/serving policy	Exact built and served bytes, runtime conformance, compatible old-client op
XTDB backend, adapter, query/write options or semantics	Selected interface, atomicity, physical observation, command history, progre
Biff routes, middleware, server, worker or raw writer	Actual handler adoption, protected writer inventory, effect containment, pro
Live interests, routing, retention or access rules	Initial/subscription gap, reconnect recovery, authorized acquisition and one-
Checker, proof kernel, policy or product profile	Validity of corresponding proofs and assumptions, required checks, and inde

Reuse is permitted only where **all** original source, checker, implementation, resource, environment and deployment assumptions relevant to the particular evidence remain applicable. A stable semantic digest does not freeze an opaque handler's implementation, an unchanged JS filename does not prove the emitted bytes, and a passing previous XTDB compatibility test does not certify a changed backend.

Reverification follows known dependency edges and marks opaque impact as open rather than pretending a partial dependency catalog is exhaustive. Incremental checks may reuse untouched subgraphs but must recheck their joins into the new admitted generation. A copied green report, process-local handle or old module/asset receipt cannot promote a newly changed deployment into the prior generation's authority.

Required verification scenarios

The verification strategy covers semantic construction, production enforcement, and product behavior with appropriately scoped evidence. At minimum, relevant application profiles exercise:

Scenario	Evidence sought
Same canonical definition feeds build, startup, render, and tests	Removing or changing an owned declaration affects the act
An opaque builder returns an invalid candidate	The actual candidate is rejected before submission without
Finite exploration reaches its declared bound or exhausts its resource budget	Scope-complete finite results remain distinct from inconclus
Rule/operation/outcome exercise is vacuous	Required exercise gates reject; the report does not inflate at
A copied report or metadata-equivalent function replaces an issued product	Admission rejects the substitution or explicitly establishes a
The server captures an earlier handler while the system map later changes	Installation evidence detects or limits the server-adoption cl
A worker writes state outside the HTTP catalog	Its path is classified and covered, or the dependent closed-w
Two valid requests overlap and complete in reverse order	Their scoped capabilities, outcomes, and rendering contexts
A deferred/streamed/cached response bypasses the synchronous guard	The selected response contract prevents unchecked manage
A fragment's effective form/action depends on outer context	Checking uses admitted context or rejects the unestablished
Old HTML requests assets after a new assembly activates	It receives the bound compatible asset set or follows the dec
Artifact, handler, policy, provider, or backend premise changes	Dependent evidence is invalidated/rechecked without muta
Startup or activation fails after acquiring resources	No unchecked application exposure remains; cleanup and su
The response fails after authoritative incorporation	Server knowledge and participant uncertainty remain distin
A disconnected subscriber misses relevant changes	Initial/reconnect acquisition closes the required gap withou
A required user capability is removed while the remaining graph closes	Independent product acceptance fails.

Formal checks, controlled-defect tests, observed execution correspondence, and real-browser acceptance are complementary. A unit test of a report shape cannot replace evidence that the actual handler or browser edge is used. A successful request cannot establish every route or every possible schedule.

Qualification matrix and controlled positive/negative tests

Evidence level	Required positive control and adversarial limitation
Source and finite model	Original rule/operation/outcome coverage with nonvacuous positive and rejected branches, plus source-muta
Inductive and formal	Exact abstract invariant, transition catalog, proof/checker version and retained premises; unmodeled writers c
Source-to-implementation	Real issued executor, route, typed provider, browser plan and generated asset joined to admitted sources; equa
Physical backend	Actual guarded XTDB write, command record, current observation and progression under selected profile; cha
Production serving	Biff/module/handler, network traffic, emitted HTML and served JS actually use checked paths; removing the r
Browser and Live	Real rendered control and compiled JS/HTMX submit, authorized settlement/reread, Live advisory/Refresh an
Product completeness	Explicit required Create, Claim, Unclaim, completion, Cancel and Reassign capability success; deleting one fai

Each test states its scope, original sources, concrete checker and backend/host, selected actor/argument/schedule, applied fault, observed results, bounds and unestablished premises. It checks a corresponding **valid** behavior. An intercepted fake committed map cannot prove physical XTDB incorporation; a direct injected protocol command cannot prove a button worked; a local Biff system map does not attest a separately deployed listener.

Additional positive and adversarial histories

History	Required classifica
Real eligible Claim and manager Reassign controls trigger admitted guarded operations and authorized card adoption.	Positive product c
The operation registry deletes Reassign and all remaining preflight checks pass.	The independent p
A disabled Claim is accompanied by a manually injected ideal protocol command.	Route fixture may
Two clients receive remote advisory after Claim; the second uses one explicit Refresh.	Test initial knowle
Reviewed XTDB API/option checks pass but a transaction atomicity premise fails.	Preserve interface
A new backend version changes query/write options after preparation.	Refuse affected op
A source report is edited to assert verified, or a private issuance handle is copied.	Neither issues pro
A rule changes but retains operation ID `:request/claim`.	Recheck dependen
New JavaScript bytes are served under an unchanged filename or a CDN returns stale bytes.	Fail exact serving-
A scheduled raw XTDB writer is missing from the registered operation catalog.	Complete writer c
Static Hiccup passes while dynamic rendering emits a malformed semantic POST.	Actual output gua
Claim commits, then its response fails and the client retries.	Preserve historical
A subscriber misses a change between initial read and interest registration.	Require fence/repl
Two independent requests overlap and finish in reverse order.	Preserve separate
A later module replaces the checked Biff handler after an earlier success receipt.	Invalidate deploye
An abstract model passes safety, but manager Reassign never becomes enabled.	Record abstract sa
A model checker times out or an external physical witness is unauthenticated.	Report inconclusiv
A feature passes negative and structural checks but the real browser cannot activate it.	Fail real-browser/p

Application qualification grants no new business authority. Current principal and Membership, RequestAssignment state, guarded transactions and disclosure of recorded commands remain with their trusted model/runtime owners.

The suite includes positive controls for each rejection gate. Rejecting all requests, never exposing a control, or disabling concurrency universally is not an acceptable way to satisfy the intended application's behavioral requirements.

Whole-application security and construction invariant

Preflight establishes the stated structural relationships and selected assurance conditions. It does not confer a principal, disclose a protected result, authorize a transaction, or establish that a mutable world still satisfies a prior guard. Those premises are checked at the trusted boundary that owns them.

Trusted authority is an explicit boundary, not metadata

The canonical construction may establish that an operation has one admitted name, typed arguments, generated plan, physical transport and trusted server executor. It must not replace the authenticated principal or current model authorization with a browser argument, role label, stored card, copied proof report, or prior successful preflight. The server resolves every protected operation through its own installed and current trusted registry.

Source of information	Authority it may establish
Rendered control, plan digest, basis, command metadata or browser certificate	A request for a particular semantic action or a participant-
Trusted request ingress	Valid method/route/envelope, decoded subject, authentica
Public authoritative model operation and guarded transaction	Current domain guard, permissible effect and authoritativ
Durable command record and resolver	Command-specific historical incorporation or an explicitly
Application assembly, artifact receipt or host installation evidence	Structural/build/serving correspondence at its qualified su
Formal proof, finite-model report, checker diagnosis or runtime observation	The exact proved, bounded, observed, open or assumed pr

Untrusted values may guide selection only after they are decoded, normalized, validated and matched to the trusted route/operation/subject contract. Inconsistent Request IDs in path and command arguments, a forged principal, a wrong tenant, an invalidly replayed command ID, or a purported assembly certificate supplied by the client cannot win a conflict with trusted context. Protected result disclosure, including history of a command that has already incorporated, is evaluated under its own current authorization policy, not the policy used when the command was originally issued.

The browser can forge local metadata, operation names, command bytes, certificates, and assembly IDs. Trusted ingress and model operations remain secure without assuming that the browser follows the admitted implementation. Conversely, server construction controls accidental misuse by trusted host code; a hostile JVM with reflection or arbitrary bytecode replacement is outside that claim unless an additional isolation mechanism is explicitly supplied and qualified.

Threat model and effect containment

Gesso's structural contracts are designed to prevent accidental and misconfigured bypass in cooperating application/server code and to remain safe against a malicious or stale browser request. They do not by themselves sandbox arbitrary trusted application code, a compromised JVM, reflection, hostile classloading, administrator-level access to the XTDB process, or an independent writer with unrestricted database credentials. Claims covering such adversaries require separately qualified process isolation, physical write capability restriction, deployment authentication or independent complete writer evidence.

The protected write domain and its allowed writer policy are named. A route registry that lists every declared Choreo operation cannot establish that no scheduler, initializer, migration, worker, raw XTDB client or external writer can mutate the same protected state. A host-supplied writer inventory is useful conditional evidence, but its completeness and ability to constrain actual physical effects remain separate propositions. Stronger all-writer invariants must be enforced at the actual capability boundary or depend on explicitly justified complete deployment/physical-writer premises. Hidden writers are not excluded by matching operation IDs or a successful local proof.

The same principle governs model authorization: an application-supplied callback that claims to enforce active Membership is not automatically equivalent to a transaction-internal guard. Correctness of the declared business rule, the admitted abstract transition, the real executable guard, the physical XTDB prestate and commit, and every interfering writer require separately applicable evidence. A finite example of Claim safety or conditional induction can establish an important abstract fact without proving all deployed transactions respect it.

A structural report, artifact receipt, process-local handle, proof certificate, and runtime observation each has a defined role. None silently acquires the others' authority. Production evidence is applied only to its actual subject, scope, assumptions, and current applicability.

No credential promotion across lifecycle, evidence or time

Each issued effect right has a checked owner, operation, principal scope, source generation, physical target, permitted use and retirement contract. Source/assembly closure cannot mint that right merely by returning a copyable report. An issued request-local

callable cannot be copied as a portable credential or reused by another invocation without its own admission. A browser-issued one-Refresh permission is deliberately distinct from the existence of an advisory or the ability to connect to an SSE stream; a consumed grant cannot be revived by replay.

Construction, installation, actual network serving, current invocation authorization, successful authoritative incorporation, permitted readback, physical DOM installation and subsequent participant continuity remain independent statements. When a Claim commits but its response is lost, its incorporation does not disappear: the participant remains `:unknown` until command-specific evidence authorizes a different settlement. When the card is updated and a later Reassign changes current state, that observation does not rewrite historical incorporation. A failed response renderer cannot turn a prior committed mutation into rejection; a later observer cannot serve as a retroactive pre-effect guard.

Generation changes preserve the original binding of already issued commands, qualified old-client asset/plan compatibility and declared recovery work. They do not preserve an obsolete principal's right to initiate new protected effects. An old callback cannot silently borrow a new handler/provider or a retired browser target. An initializer that opened a listener or writer before preflight cannot be excused by placing the checker first in a later Biff `:start` ordering. New protected traffic is admitted only through the selected, actually adopted generation and its applicable physical/effect/security gates.

Single-node and distributed realizations are constrained by the same security and semantic invariants. Multi-node routing, interest advertisement, failover, replay and invalidation duplication require additional consistency, isolation and liveness premises; a router's locally missing interest is not authority to omit a remote participant's permissible recovery. Advisory Live delivery alone never authorizes a protected data read, and missing messages never prove the participant is up to date.

The whole-application invariant is:

> One source-owned definition yields an admitted semantic and physical realization under an explicit assurance policy. Every required operation, effect, output, observation, and recovery edge has a compatible current binding, and the actual installed paths enforce the required checks before their protected actions. Construction, artifact, installation, invocation, and physical-completion facts remain distinct. Missing structural edges reject rather than silently weaken meaning; opaque application logic retains explicit contracts and premises. Admitted assembly instances remain coherent through build, serving, asynchronous work, reload, and retirement. Declared closure, production enforcement, and independent product completeness are separately evidenced, and authority remains with the trusted runtime/model boundaries that own the relevant current facts.

Conditions for interpreting the invariant

The quotation is a **normative end-state property**, not a certificate that a particular Gesso commit or arbitrary Clojure application currently satisfies it. Its ”every required” quantification applies to the **truthfully covered** operation, effect, writer, product and deployment scope under the selected assurance policy. It cannot prove unknown writers absent or make a smaller operation catalog satisfy broader HumanHelp capabilities. To assert it of an admitted application generation `A` and its specified protected effects, the evidence must account for all the following conditions:

1. The source-owned catalog and its independently selected positive application requirements are identified; required operations, rules and recovery cases are nonvacuously covered. Unknown or opaque business rules remain declared premises, not implicitly absent.
2. Semantic operations, browser controls/plan, exact generated artifacts, actual routes, trusted model calls, physical providers, protected writers, final output, observation and recovery all have appropriately checked bindings. A structurally known missing edge prevents admission, rather than downgrading a managed operation to anonymous HTMX.
3. The installed startup, actual handler, guarded transaction path and relevant response/asset serving mechanisms enforce those checks in the qualified deployment. Local system-map equality and source-relative diagnostics do not establish physical serving or writer completeness.
4. Each attempted effect is checked against **current** trusted authentication, authorization, model guards, command binding and transaction assumptions at its legitimate authoritative boundary. The invariant must continue to hold under all covered relevant concurrent writers, not only the operation that the user clicked.
5. Protected output, prior-command disclosure, Live interest, one-Refresh acquisition, physical adoption and old command resolution respect their own permissions, progression and lifetime, including ambiguous delivery, postcommit failures and generation turnover.
6. Relevant changed sources, providers, checkers, plans, backend assumptions or deployed wiring invalidate dependent evidence. Bounded finite checking, conditional induction, host observations, compiler premises, actual backend behavior and production deployment are never collapsed into one green claim.
7. Independent real user acceptance demonstrates required positive Create/Claim/Unclaim/Cancel/complete/Reassign and advisory/Refresh behaviors as applicable; a structurally closed, successfully rejecting but functionally empty application does not qualify.

A qualified invariant is about the actions and physical deployment scopes it actually covers. Fairness, continuous connectivity, backend correctness, correct application business rules, distributed recovery and hostile trusted processes require separately named assumptions or proofs. A finite model checker can prove an invariant for its admitted finite model; the complete real application guarantee additionally needs correct mappings and coverage for all claimed physical effects.

Security and construction falsification histories

Tests select the original source-owned application, intentionally change one structural or physical edge, and observe its legitimate real boundary. Valid positive cases remain independently required.

History	I
An authorized helper uses the real Claim control, and the current model incorporates the guarded transaction with permitted readback.	S
The browser forges `:principal` or Membership role in a Claim payload.	T
A Request R1 Claim envelope is sent to the installed R2 Claim route.	I
The browser sends a copied ApplicationAssembly, valid artifact receipt or alleged proof certificate to skip server checks.	I
A command ID that previously incorporated is replayed with a different operation, arguments or principal.	I
A correctly bound old command result is requested by a principal lacking current disclosure permission.	I
Claim's declared active-Membership rule passes a finite check but the issued XTDB callback omits the real current guard.	I
A worker using raw XTDB privileges alters RequestAssignment outside the registered route catalog.	I
A current generated browser asset is checked locally but a different bundle is served to an old page.	I
Biff's preflight starts first in ordered `:start`, while a previous `:init` has already exposed protected work.	I
A valid Claim commits, but its HTML response is malformed and never delivered.	I
An SSE advisory arrives twice after a one-use Refresh grant was consumed.	I
A Live router fails while a new subscriber's interest was not yet known to its replacement.	I
A server-side diagnostic report is copied from generation A and submitted as generation B's admission proof.	I
The managed application has no usable manager Reassign, although its reduced semantic graph and guards pass.	S
A runtime checker is unsupported or times out while all route/asset declarations look correct.	I

These histories are requirements for future qualified implementations and tests, not assertions of a freshly completed Gesso test suite. Their scope and independent acceptance must remain visible in the application's semantic audit and explanation surface.

Part VII completion is not automatic acceptance

Finishing the fifteen sectional R5.1 source reviews establishes only editorial progress. Whole-Part-VII reconciliation must separately inspect internal duplication and contradictions, cross-part interfaces, the complete R4 preservation inventory, and consistency of the revised final invariant with its construction, browser, model and deployment premises.

Independent reviewed-candidate acceptance then evaluates the resulting published part and its original sources. Neither step grants whole-application certification to Gesso or HumanHelp. Parts VIII–XII and full-plan review retain their separate outstanding requirements.

Part VIII — HumanHelp proving application

This part specifies a complete proving application for the contracts in Parts I–VII. HumanHelp supplies real domain rules, authenticated users, persistent Requests and assignments, a useful board, concurrent browsers, and recoverable failures. It must demonstrate that the assembled path works, rather than merely that its components can each return a successful check.

This is a normative application design. Requirements describe the system to be built; they are not assertions that the corresponding implementation or proof already exists. The selected product policies are stated here so that browser behavior has an independent acceptance oracle. A remaining unresolved product contract is identified explicitly and cannot be counted as completed.

Examples use illustrative construction and inspection notation. They do not introduce a second public API alongside production model and Gesso contracts. In particular, a plain map containing an operation, evidence reference, or successful verdict is not an issued executor, admitted observation, or permit.

Purpose, scope, and independent claims

The fifteen reviewed units specify one application, not fifteen independent implementations. 08.01–08.03 own its production semantics, application identity and independently required user capabilities; 08.04–08.07 own adopted board knowledge, local Create, remote advisory/one-use Refresh and user interaction; 08.08–08.10 join authenticated authority, Live, command recovery, concrete routes, assets and host lifecycle; 08.11–08.14 own assurance/test oracles, structural enforcement, connected evidence and resource limits; 08.15 owns joint acceptance. Parts IV–VI own generic command, progression and physical-effect semantics, and Part VII generic application assembly. None of these overlapping views creates a second Request machine or operation registry.

The immediate application is the HumanHelp example UI, backed by the production `net.humanhelp.site.model.*` models. The eventual production UI may consume the same contracts. Its completion is a separate product milestone; the example must not depend on that unfinished interface to supply essential behavior.

The example owns integration and presentation. It does not own a parallel Request store, a simplified Membership policy, a substitute assignment model, or a browser-specific interpretation of Request states. Development fixtures create valid production entities and enter through declared initialization paths.

Three claims must be reported separately:

Claim	HumanHelp evidence
Declared closure	One application definition connects required operations, plans, routes, providers, acquisitions, rendered actions
Production enforcement	Actual serving, dispatch, model invocation, transaction submission, and browser effect boundaries enforce the a
Behavioral completeness	Independent user scenarios succeed through real controls and persisted production semantics.

An application can be closed after accidentally deleting Reassign. A model can be correct while the browser never sends its command. A browser can display success while no authoritative assignment exists. Each must fail the claim it violates; none is repaired by a green result in another column.

HumanHelp is a broad reference consumer. It exercises authentication, creation, all required lifecycle operations, selective adoption, search, sort, visibility, details, continuity, response interaction, disconnect/reconnect, and concurrent use. A single Claim demonstration is a necessary vertical slice, not the whole application.

Generic mechanisms and application policy

Gesso supplies operation identity, projection, semantic admission, managed effect ownership, authority and observation boundaries, scoped progression, application closure, evidence classification, and recovery composition. HumanHelp supplies Request eligibility, Organization and Membership policy, assignment meaning, board ordering, and the choice to defer remote changes until Refresh.

HumanHelp failures can justify a generic improvement when they expose a reusable missing obligation or enforcement boundary. A missing way to preserve unknown command outcomes is such a gap. A wrong rule about who may reassign a Request is a HumanHelp model defect. Gesso must not acquire HumanHelp business rules to make a reference application pass.

The proving UI is the selected real application, not a parallel demo

During this integration milestone, the global `/app` entrypoint actually selects `net.humanhelp.example.app` through the application's normal routing and startup path. The incomplete `net.humanhelp.site` UI is **not** another acceptance implementation and need not be finished to demonstrate this reference application. Eventually it may consume the same production model and Gesso contracts; its completion is independently scheduled.

The boundary is strict in both directions. Production `net.humanhelp.site.model.request.core` and the relevant public Organization, Membership and User model APIs own their persistent facts, eligibility and guarded transitions. The model-owned `net.humanhelp.site.model.request.choreo` coordinates those **public** operations and exports semantic browser capabilities. The example provides board/query composition, views, routes and user interactions, but cannot invent an easier RequestAssignment schema, mutate an example-only Request store or implement authorization in a card component. A broken production transition is repaired in its production owner; replacing it with a sample mutation to make Chromium green is an invalid migration.

The physical browser participant `:helper` in the current Request choreography is not evidence of actual helper Membership: the same signed-in participant can display requestor and manager affordances. Its trusted authority role, the authenticated principal, selected target helper, physical server host and current model guard remain distinct. Example request inputs and generated browser operation metadata cannot supply the principal or bypass current Organization and Membership checks.

Development fixtures bootstrap genuine production User, Organization, Location and relevant Membership/assignment relationships through declared model/initialization paths. A fixed example Organization or Location used by a selected fixture is **not**

a substitute for tenant-scoped authorization or complete production capability coverage. Tests identify their fixture and actor assumptions, and exercise the real public production model rather than a mock that merely returns a plausible Request card.

One application definition and actual installation

HumanHelp has one source-owned application definition. It composes production model operations and choreography with concrete browser and server realizations, physical routes, acquisition scopes, views, assets, capabilities, and the chosen assurance policy. The constructor returns the admitted application used by startup, rendering, dispatch, and inspection.

Tests call that constructor with explicit fixture dependencies. They do not maintain a second operation registry or a shadow assembly. Product acceptance remains independent: its list of required capabilities cannot be generated solely from whatever operations happen to remain in the application definition.

Product scope must not be confused with a closed lifecycle slice

The application has multiple overlapping but nonidentical inventories. The model-owned operation catalog identifies what the production model can execute. The browser-plan registry identifies the lifecycle operations currently exposed through one client projection. The application's complete protected effect/writer scope includes other sources such as Create, initialization and background work. The independently selected HumanHelp product profile defines what a **complete proving application** must make usable.

A structural checker can validly close a six-operation browser lifecycle slice — Claim, Unclaim, mark on the way, Complete, Cancel and Reassign — without thereby having checked Request Create, authentication, read authorization, search, details, selectable response interaction, all model writers or positive end-to-end product behavior. The finished application must **compose** its distinct Create operation and every other required protected capability into appropriate route, trusted model, effect, rendering and acceptance evidence. Its report states what was included, intentionally separate or still unqualified, never advertising a partial fragment as a fully admitted product.

The acceptance oracle is independently selected and includes the full required-capabilities contract in 08.03, not merely the set of current browser plan keys. Removing manager Reassign from a plan may leave a smaller graph that passes its own checks; that must still fail the independent product gate. Likewise a green lifecycle preflight is not evidence that a real Create dialog produces a persistent Request with immediate creator-local visibility.

Source responsibilities

The following names locate responsibilities; they do not require that every application use exactly this file decomposition.

Owner	Responsibility
`net.humanhelp.site.model.request.*`	Request and RequestAssignment meaning, lifecycle rules, public reads and write
Production Organization, Membership, and User models	Their own entities, association and eligibility rules, permitted public projections
Production Request choreography	Distributed operation contracts, projected roles, semantic operation identities, o
`net.humanhelp.example.board`	Authorized read composition and pure presentation over adopted observations;
Example Request-card component	Render prepared production affordances and adopted facts; provide stable card i
Example Refresh component	Render pending, refreshing, failed, and disconnected states; issue an explicit adc
`net.humanhelp.example.live`	Scope routing, subscription policy, query/projection contracts, acquisition and i
Example optimistic integration	Application presentation choices bound to generic projected plans and effect ow
Example routes and app composition	Physical dispatch, trusted context construction, issued executor bindings, startu
Example application-preflight integration	Invoke and explain the canonical constructor and selected admission policy; no
Example views	Page composition, dialogs, local options, accessible feedback, details, and declar

Late renderer resolution can break a namespace dependency cycle. It cannot allow a renderer to escape admission or be silently replaced after its binding has been checked. Reload creates a new candidate assembly and adopts it coherently under Part VII.

Source ownership also distinguishes **public-model** calls from private implementation reuse. Cross-model decisions use the owning model's `*.core` API and required guarded dependencies; the example must not reach into another model's `domain`, `fx` or `graph` internals to create a more convenient eligibility path. The Request Choreo coordinates the same public operations; it cannot embed a competing set of Organization or Membership rules.

A concrete route declares which trusted callable it actually uses, which semantic operation it exposes, what principal and Request selector it receives, and which physical model/ transaction and authorized acquisition follow. A view that displays a semantic capability cannot silently downgrade to an anonymous HTMX POST because its optimistic basis is missing. A raw writer or alternate Ring route outside the plan inventory remains relevant to the protected-effect scope; a report of zero declared uncovered writers cannot prove that actual writers are absent.

A concrete construction path

The supported path performs these steps:

1. Load production definitions and explicit configuration, including the runtime profile, board policy, trusted capabilities, and assurance requirements.
2. Resolve the operation catalog, semantic declarations, opaque host contracts, choreography projections, and required acquisition/recovery continuations.
3. Derive browser plans and the manifest from that source; build the generated bootstrap and exact asset set under the qualified compiler/build profile.
4. Bind physical dispatch to the issued server realizations and the trusted context/provider mapping. Include relevant non-HTTP writers in the scope of the application claims.
5. Admit concrete view preparation and render boundaries, query scopes, Live declarations, effect implementations, and product-specific adoption policy.
6. Apply the selected structural and assurance gates, preserving unresolved premises and the actual scope of each result.
7. Install the admitted handler and runtime resources into the serving host; expose ordinary service only after the required initialization succeeds.

The serving host must actually consume the checked handler. Merely returning one from a helper or storing it in a system map does not establish adoption by a running server. Tests observe the installed entrypoint and exercise it.

The example application route selected globally, the canonical application constructor, emitted browser artifact, guarded HTML output and physical server startup are **joined deployment facts**. A build script and a runtime may share a source declaration yet load different JavaScript bytes, or a Gesso module may install a checked `:biff.ring/handler` while an alternate server captures a different callable. Those are distinct failed/unsupported evidence claims, not successful application installation.

On Biff 2, all module `:init` functions can run before ordered `:start` functions. Mandatory preflight must therefore fence protected listener, scheduler, writer and public operation effects even when private initialization is allowed. The actual Aleph server must consume the admitted handler before ordinary service; a later handler/asset/provider replacement invalidates the affected evidence. No startup policy may pretend a successful check rolls back a mutation that already committed or resolves a command whose response disappeared.

Test setup and the running application use the **same** construction owner and original production model definitions. Controlled defects remove a real required operation, route, provider, plan, artifact or acquisition edge and must fail the corresponding gate. A test-only duplicate assembly remaining green after the production constructor breaks is not valid acceptance evidence.

Assurance is explicit and useful

HumanHelp uses the full host language. Pure functions, FX composition, ordinary Clojure and ClojureScript, and opaque integration code remain available. The application identifies the contracts on which its claims depend and proves the parts supported by the selected semantic fragment.

The admission profile must require structural closure and the applicable issued callable/plan bindings. For operations represented by the source-owned relational construction, it also requires their declared invariant checks, rule exercise, and operation/outcome coverage. Conditional induction is required where the application claims that stronger result; a finite exploration result is not renamed an induction theorem.

Unsupported operation semantics must be declared as opaque premises with an owner, effect/dependency interface, and conformance obligations. The admission policy explicitly accepts or rejects those premises for the claimed application invariant. It must not make an incomplete model look total by omitting the opaque operation from writer coverage.

For example, a proof of unique active primary assignment may depend on the Request command implementation, initialization, collaborator/reassignment writers, and the qualified transaction adapter. A theorem about the finite abstract operation catalog alone does not establish the corresponding physical database invariant.

The permanent executable semantic core governs admitted transitions. A proof checker or later small proof kernel checks a specified class of proof evidence. Neither replaces HumanHelp's transaction boundary or current authorization.

Positive and adversarial assembly histories

The following scenarios exercise the same production-backed example selected by `/app`. Structural checks, actual production enforcement and independent product outcomes are reported as different propositions.

History
The selected example creates a Request through the production model; its creator immediately sees the committed Request under a qualifie
A valid helper uses the actual Claim card and the public production model incorporates the guarded assignment.
The same browser is a requestor on one Request and an authorized manager on another.
An example Claim handler writes an example-only RequestAssignment or directly imports another model's private FX helper.
A fixture supplies a valid Organization ID but no applicable active Membership for the actor.
An assembly checker includes all six lifecycle browser plans but omits Create, an actual raw writer or another promised product operation.
A lifecycle operation ID is removed from the real production browser declaration while an independently rebuilt test registry still includes
The named route and operation exist but the deployed Aleph handler uses a different callable from the checked system-map slot.
A Biff `:init` starts a protected writer/listener before the checked startup's ordered `:start`.
A valid operation commits, then its response renderer fails or network settlement disappears.
A test injects a correct Claim protocol directly while the actual button never submits.
The source/asset/handler generation changes while a pending card response or command resolver still refers to the prior generation.

An eligible Create and Claim scenario must be demonstrated in addition to controlled invalid cases. Subsequent 08.xx sections own the full Request lifecycle, selective board adoption, one-Refresh policy, response-flow choice and concurrency matrices; this initial assembly section does not redefine their domain policy or pretend those end-to-end capabilities have passed independent acceptance.

Production model truth and public operation boundaries

A Request belongs to its production Organization and Location. Its helper relationship is represented by production RequestAssignment semantics. The UI must not replace those facts with a convenient `helper-id` field or infer the whole lifecycle from whether one optional field is present.

The canonical Request states for this application are `:open`, `:claimed`, `:on-the-way`, `:done`, and `:cancelled`. Display text such as "Completed" may label `:done`; it must not introduce a competing persisted `:completed` state. Public predicates and prepared projections provide state meaning to presentation.

Production aggregate, state vocabulary, and source authority

The production Request and RequestAssignment are separate persisted entities forming one model-owned Request aggregate. A Request owns Organization, Location, requestor, content and lifecycle identity. A primary or collaborator RequestAssignment owns its helper, role, active/ended state, provenance and history. An active primary assignment is not interchangeable with a convenient `:request/helper-id` view field, a provisional label or the browser's selected target.

The public `request.core` predicates and snapshots, not example component conditionals, determine `:open`, `:claimed`, `:on-the-way`, `:done` and `:cancelled` and the assignment relationships each permits. Terminal completion and cancellation close the assignments required by the production operation; Reassign preserves a lawful historical end of the previous primary and creation of the new primary in one protected operation. Collaborator writes, Request editing, initialization and Organization/ Membership changes remain within the relevant invariant/writer scope even when this proving UI does not expose each as a dedicated control.

Use public `net.humanhelp.site.model.request.core`, `membership.core`, `organization.core` and `user.core` semantics for cross-model decisions. The production `request.choreo` layer coordinates these public operations; neither it nor the example may independently reimplement private `domain`, `fx` or `graph` policy to win a race against the public model.

Atomic dependencies, not a sequence of reassuring reads

A lifecycle handler may parse inputs, authenticate, prepare context, and perform advisory reads. The public production operation remains responsible for the resolve-or-incorporate protocol in #req-atomic-authoritative-transition.

For a new Claim, the protected dependency set includes the targeted Request, its Organization/Location association as required by policy, the actor and selected helper where distinct, effective Membership, Organization policy, relevant assignment facts, and command exclusion. The ownership change, assignment changes, command result, and required durable publication intent are established at the declared atomic boundary.

The actual set follows the production rule. If eligibility depends on a related entity or on the absence of another active assignment, that dependency must be protected too. Guarding only the Request document is insufficient. Conversely, the example should not invent unrelated dependencies merely to make a diagram appear comprehensive.

Separate reads of Request, Membership, and Organization followed by an unguarded write leave a race. Rechecking only the rendered card's revision leaves a race. A transaction plan must preserve every relevant dependency through the qualified submission mechanism, including absence or query dependencies where required.

Guard construction and physical correspondence are different claims

The admitted Claim plan has a source-relative obligation to protect the aggregate snapshot, current operational Organization/Location scope, selected target's active eligible Membership and relevant role/skill assignment, and (for manager-assisted Claim) the actor's current supervisor/administrator authority. It additionally excludes active assignments as a **cardinality/absence** condition. The exact dependency footprint is determined by the real model policy, not by a hand-authored audit approximation.

Gesso's generic transaction representation may combine `:commands` (Request/RequestAssignment mutations), `:guards` (versioned dependency evidence), `:assertions` (including set-level absence/cardinality) and `:changes` (semantic changes for Live). These are different kinds of obligations: guarding each observed assignment is insufficient by itself when a new matching assignment could appear. The Request aggregate concurrency root and required relevant absence assertions must be honored by the selected XTDB transaction semantics.

The **model planner** may have a well-formed protected plan while the **physical backend** still requires independent qualification for transaction-internal evaluation, assertion atomicity, contending writers, post-commit progression and actual submission/observation provenance. An optional finite proof of the source-owned rule does not authenticate every XTDB attempt. Likewise, calling `model.tx/transact!` is not proof that unregistered maintenance writers cannot bypass Request invariants. The application can qualify the bounded real transaction behavior it has observed without calling that evidence universal linearizability or a complete-writer certificate.

The browser's previously adopted basis `X17` is an authoritative **scoped observation frontier**, not the Request's revision, a global snapshot of all HumanHelp state, or an automatic stale-command veto. Unrelated commits may advance the database while Claim remains valid; a relevant Membership/assignment change must be reconsidered by the model and guarded at incorporation.

Actor, target helper, and command identity

The authenticated principal comes from the trusted request/runtime context. Browser fields, rendered capabilities, certificates, selected personas, and knowledge of a UUID do not establish that principal.

A target helper can be a legitimate domain argument. Reassign requires target selection; a production Claim variant may permit a manager to claim for another eligible helper. That argument does not replace the actor. The model checks the actor's authority to select the target and the target's eligibility atomically. The basic Claim control in this proving UI means "claim for myself"; any separate manager-assisted variant is labelled and tested as such.

Before the first possible send, the admitted command boundary supplies a stable `:gesso/command-id` and binds it to the operation/interface, canonical arguments, authority domain, and principal. A declared mapping may expose that value as `:command-id` at the public model boundary. Retries preserve the binding.

The HumanHelp domain argument `:request-id` denotes a Request. Transport correlation uses `:http-request-id` or `:transport-request-id`. Execution, resource instance, board owner, assembly/deployment, and command identities stay distinct. One generic "generation" field cannot stand in for all of them.

Request actor, target, scope, and trust separation

For the ordinary `:request/claim` control, **claim for myself** binds target helper to the currently authenticated actor within the trusted operation, not from a posted `:helper-id`. A distinct manager-assisted Claim may legitimately supply a different `:helper-id`, but the authoritative model must protect both the actor's **exact** supervisor or administrator role at the applicable Location and the chosen target's separate helper eligibility. HumanHelp does not grant manager authority merely by a role-name hierarchy or by the browser using the one physical `:helper` choreography participant.

Reassign likewise must not equate an actor's permission to choose a target with the target's eligibility to hold the new assignment. Relevant Organization scope, Location, current Membership, active role assignments and declared skills are model dependencies. A target-helper picker can narrow options for UX; it cannot authorize a protected transaction.

A route must reconcile the normalized path Request ID with the semantic command's Request argument. A separately chosen target/helper/skill cannot replace the trusted principal or ownership context. A valid operation capability or browser-chosen principal identifier is inert presentation data. Public model authorization and its current protected dependencies determine the actual transition.

Resolve an existing command before applying new-command guards

A compatible repetition checks the retained command record's binding and current permission to disclose its outcome. It returns the permitted historical result without incorporating again. The fact that the Request is now claimed does not justify rejecting a retry of the successful command that claimed it.

A new command must pass the current commit guard. Concurrent compatible attempts must not both establish a new incorporation. Incompatible reuse is a typed protocol failure. A failed physical attempt is not automatically a definitive command rejection when another delivery of that command may still succeed.

Command retention, exclusion after result expiry, recovery across interface versions, and disclosure after role changes follow Part IV. The browser may not create a new identifier merely to make an unresolved operation appear retryable. A command-recovery record and a current Request read answer different questions.

Historical incorporation requires an actual selected implementation

The complete target protocol is **resolve or incorporate**: a trusted server obtains an authoritative record for the original typed command binding (principal, operation, canonical argument set, interface/authority domain and stable command ID), verifies current disclosure rights, and returns the recorded result when repetition is compatible. An unresolved original command never becomes permission to mint a fresh ID merely because a timeout occurred or the current Request looks claimed.

For new incorporation, the command record/exclusion and Request/RequestAssignment guarded mutations must share the qualified atomic boundary. An existing record must be effectively immutable for the selected retention contract. Record expiry must not permit duplicate logical incorporation while an old retry can still arrive; the operation must define exclusion, compatibility or manual reconciliation through that lifetime. Retried transport attempts can reach another Aleph node without changing their

semantic identity. A later current-policy or principal-role change may forbid disclosure of a previously successful result without reversing the historical commit.

This is a **required finished-system protocol**, not an assertion that every current ‘request.core‘ call already persists command ID and outcome. Current production public Request operations expose transaction `:commit/status` and progression; their model transaction success is not, by itself, a durable same-command replay certificate. Gesso's source-level command-record helper is experimental construction with a trusted disclosure/lookup assumption, not an installed HumanHelp exactly-once service. The application must connect, qualify and test that mechanism before claiming the stronger recovery property.

Outcome and presentation contracts

Model evidence	Presentation and recovery obligation
`:committed` with permitted success evidence	Settle the bound operation as `:accepted`; install only the ob
`:already-incorporated` with a permitted successful recorded outcome	Use the same accepted successor without another mutation
`:rejected` with authoritative command disposition	Follow the rejection branch and permit the declared scoped
`:commit-unknown`	Keep command disposition unknown and invoke the bound
Malformed input, incompatible reuse, invalid credentials, or disclosure denial	Use its declared typed handling; do not invent a domain rej

A replayed success is historical. It does not prove that the helper still owns the Request after later Unclaim or Reassign. The UI may say that the command succeeded while displaying a newer permitted card observation with a different current assignment.

A timeout does not mean rollback. A publication failure after a known commit does not erase that commit. The server can know `:committed` while a disconnected browser knows only `:unknown`. A lost response is resolved with authoritative command-specific evidence, not by guessing from a fresh card that another actor could have produced.

Distinguish the current model API from the intended protocol layers

The present public Request operation reports `:commit/status :committed` and, when available, a composable authoritative `:progression`. The current Request choreography checks that committed result and constructs its own protocol-v3 `:confirmed` authority projection. These are different **interfaces** from the Part IV `:committed` / `:already-incorporated` / `:rejected` / `:commit-unknown` command disposition taxonomy and from the browser's `:accepted` / `:rejected` / `:unknown` presentation. A correct adapter preserves the evidence required for each mapping; matching words or a generic 200/500 response do not supply the missing durable record.

For the initiator, a direct correlated success can justify an explicitly scoped card acquisition or adoption if it meets the protocol's trust and progression requirements. A remote browser may receive only an advisory and is not automatically entitled to reread the board; one-user-Refresh remains a separate grant. Validating a current card can supersede provisional display even while command-specific historical attribution remains unknown.

A post-commit error in HTML generation, publication or response delivery cannot reinterpret the actual Request/RequestAssignment commit as rollback. If physical storage reports an uncertain write, neither the model nor participant may invent a definitive rejection until admissible authoritative evidence settles the bound command. Recovery has a real retained owner, trigger, authorization, progressive read, expiration and explicit blocked/manual state when automatic resolution is not possible.

Positive and adversarial model-boundary histories

Acceptance exercises the production source/physical path and the relevant trust boundary. It does not substitute copied maps or idealized isolated model fixtures for a real deployed operation.

History	Req
An authorized helper claims a genuinely open Request through the real public model.	Req
Two distinct eligible helpers submit different Claims from the same observed open state.	At r
A missing active assignment appears between Request snapshot read and commit.	The
A helper's Membership or Location role becomes invalid after an advisory read but before incorporation.	The
A manager selects another eligible helper under the same Request Location.	Gua
A browser forges `:principal`, `:helper-id`, selected role or a browser certificate.	Do r
A route targets Request R2 while the bound semantic command specifies R1.	Reje
A permitted current Claim arrives with observed basis X17 after unrelated XTDB commits to X23.	Do r
Claim modifies Request but the primary assignment assertion or insert fails.	The

History	Req
The public production Claim returns committed Request and assignment with progression, but no durable record for command C42.	Clas
C42 is delivered twice with identical principal, arguments and operation, with an authoritative disclosed record retained.	Reso
Another principal reuses C42 or a later retry changes C42's Request/target/helper arguments.	Reje
C42 is incorporated, then Reassign changes the current helper and C42 is retried.	A pe
The command record is absent at a lookup basis that cannot rule out pending C42 incorporation.	Rem
A previously authorized principal loses permission to see C42's result.	Enfo
A successful Claim commits and HTML rendering or Live publication fails.	Pres
A physically unregistered writer bypasses the Request lifecycle API to change active assignment rows.	Reta
A browser shows “Claimed” provisionally but the real model rejects or no persisted active primary exists.	Prov

These histories are required behavioral targets, not claims that HumanHelp's current ‘dev‘ branch passes the complete durable-command or real-browser matrix. Section 08.03 owns the worked Claim control and positive product oracle; 08.04–08.09 own the cohort, adoption, continuation and recovery details. Parts IV and VII govern protocol outcome types and real assembly/production enforcement, and Parts IX–XII retain independent formal/physical proof.

Required user capabilities

The following obligations are independent of the assembled operation list. Conditional unavailability under production policy is legitimate; permanent unavailability or omission of a promised capability is not.

Capability	Positive user interaction	Independent authoritative
Authentication	Sign in as a real eligible fixture user.	Reads and writes bind to t
Create	Enter valid Request content in the real dialog and submit.	One Request in the selecte
Claim	Eligible helper clicks Claim on an open Request.	Production claimed state a
Unclaim	Eligible current helper releases a claimed Request.	Production open state and
Mark on the way	Eligible current helper acts on a claimed Request.	Production `:on-the-way` s
Complete	Eligible helper completes a claimed or on-the-way Request.	Production `:done` state an
Cancel	Eligible requestor cancels an allowed active Request.	Production `:cancelled` sta
Reassign	Authorized manager chooses a legitimate different eligible helper and confirms.	Production replacement o
Refresh	User explicitly adopts current board information.	A qualified scoped observ
Search and options	User changes search, creation order, mine-first, unclaimed-first, or terminal visibility.	Deterministic presentatio
Details and continuity	User opens details, edits allowed local input, and survives permitted updates.	Correct disclosed informa
Response interaction	User completes the separately selected response-flow contract.	Its independent product o
Recovery	User encounters rejection, uncertainty, disconnect, or stale application state.	An honest status and a us

The production model owns exact eligibility. These descriptions do not authorize an actor who lacks current Membership or whose Request state changed. Positive fixtures satisfy policy through real production relationships; negative fixtures change one relevant premise and assert the corresponding outcome.

Independent user-capability oracle and positive controls

This table is a **separately selected product requirement**, not a census of the current browser plans or a generated registry of admitted operations. Every promised capability has its own positive reachable scenario using the real selected `/app` surface and public production models. Its fixture establishes legitimate current User, Membership, Organization, Location, Request and RequestAssignment relationships without bypassing a guarded operation. The expected model effect and observable UI are independently specified; the test does not rewrite the oracle when a handler or button is removed.

A capability may be hidden, disabled, or rejected in a legitimately ineligible state. The same operation must remain **usable** in a separate eligible state. Structural closure, refusal of invalid requests, and absence of console exceptions cannot satisfy a product capability whose button never submits or whose state never commits.

The product oracle separates (a) rendered affordance and browser input, (b) trusted transport and production model operation, (c) committed Request/assignment and permitted command-outcome evidence, (d) authoritative acquisition and physical installation, and (e) relevant local/remote presentation continuity. An isolated source, server, or browser fixture may establish some of these; it must report its narrower scope.

The full product acceptance also covers the broader response-flow decision, real authentication, creator-local Create, multi-client advisory and one user-Refresh, search/order/filter, details/continuity, and unknown/reconnect recovery. Passing a six-operation lifecycle gate does not qualify those other capabilities. The explicitly selected response-flow contract remains a genuine pending product decision until chosen and independently tested; the evaluator may not silently accept either absence or a substitute unrelated workflow.

Reassign is not a hidden model function with a permanently disabled button. Its UI must provide target discovery/selection, visible validation, confirmation, and a tested eligible path. Target search reveals only permitted User projections and is revalidated at commit. “Take over” is not silently treated as a synonym for Reassign; if exposed, its actor and target policy must be explicitly bound.

Additional model operations, including Edit and collaborator management, remain inside the relevant writer/invariant coverage even when this stage does not promise dedicated example controls for them. Excluding a control is different from pretending that the writer cannot affect the shared database.

Lifecycle operations share one user-to-model path

For Claim, Unclaim, Mark on the way, Complete, Cancel and Reassign, the positive case begins at the **actual rendered enabled control**. The source-owned semantic operation must retain its identity across the generated plan, loaded browser artifact, HTMX request, trusted route, public model operation, guarded outcome, settlement, scoped acquisition and final DOM. A missing

optimistic basis, plan or route rejects the managed action rather than silently converting it to an anonymous POST. A terminal removal follows authoritative membership/absence semantics, not a speculative view-side decision.

A present target-helper selector is necessary but not sufficient for Reassign. Target candidates are permitted User projections with an explicit target choice and confirmation. Trusted model authority must separately recheck the current actor's manager role, the target's Membership/helper/skill eligibility and the protected Request's current assignment at commit. A dropdown or physical `:helper` browser role does not grant model permission.

The canonical real Chromium scenario may exercise Claim, Unclaim, Claim again, Mark on the way, Complete, owner Cancel and manager Reassign with distinct legitimate fixture Requests. Positive acceptance additionally checks each operation's negative/concurrent case and fresh persisted result. One all-green happy trajectory alone cannot establish the behavior under stale Membership, competing commands, lost settlement or a permanently unavailable network. Section 08.12 owns the full independent test architecture.

Worked vertical slice: Claim

Alice is an authenticated helper with effective Membership in the Request's Organization. The board has adopted an authorized observation of Request R17 as open. The prepared Claim control carries its unconditional semantic operation identity and the concrete realization data its plan needs.

Admission and observation before Alice clicks

Alice's card has a Request identity `R17`, an adopted authoritative Request/assignment projection and a scoped XTDB observation frontier `X17`. The latter is neither the Request's revision nor a guarantee of database currentness. The page also has a live executable browser-plan/artifact binding, rendered action and target/owner identity; a server-issued certificate or metadata copied into the page does not grant authority.

The test locates the actual enabled Claim control in rendered `GET /app` and follows the generated script/HTMX path. It must not manually create, inject or edit `__gesso_live_optimistic_command`, silently synthesize a browser command in the HTTP fixture, or use an example-only Request store. A separately named lower-level HTTP fixture **may** manufacture the wire command to inspect server/model behavior, but it is never promoted into proof of this browser-to-command edge.

The real route binds its normalized Request selector to the command arguments, rejects a mismatch, and obtains Alice's authenticated User identity from trusted context. The physical browser participant `:helper` is not evidence that Alice has effective helper Membership; the current production model guard establishes that independently.

1. Alice clicks the actual control. The admitted browser plan binds command C42, its operation/arguments, execution owner, and presentation target. The bridge supplies the wire protocol; the test or page author does not handcraft it.
2. A provisional “Claiming…” state may appear. It is owned by that execution and does not establish authoritative assignment or disable another user's model operation.
3. The installed route resolves the exact admitted server realization. The trusted boundary authenticates Alice, validates the command envelope and its interface mapping, and invokes the public production operation.
4. Resolve-or-incorporate recovers an existing compatible result or protects the complete current guard and atomically incorporates C42. A new Claim cannot succeed if the relevant current policy denies it.
5. The outcome mapping settles accepted, rejected, or unknown as justified. An accepted result authorizes only its declared card/result acquisition and installation, not a fresh whole-board membership query.
6. Physical installation completion records adoption and retires superseded provisional rights. Late callbacks from the old target/execution cannot restore its speculative card or apply a second settlement.

Distinguish authority, delivery, and a successful browser test

A positive browser Claim test observes the generated command's exact operation, Request argument, stable logical command identity, execution/physical target, route and supported observed basis. It then correlates the trusted production result and any authoritative progression-bearing response with a permitted card acquisition and actual installed DOM. A fresh authorized reread or reload verifies that the Request and **primary** RequestAssignment persist. A provisional badge alone does not qualify success; neither does seeing a 200 response whose body was never installed.

The first eligible claim transition changes the persisted Request and primary assignment together. A rival new Claim with another command cannot create a second active primary; an open-state Request guard without the active-assignment absence/cardinality assertion is insufficient. A model-defined rejection for invalid current Membership or an already claimed Request is legitimate **runtime behavior**, not application assembly corruption.

There are three distinct settlement contexts: the public `Request core` result with `:commit/status` and transaction progression; the trusted current protocol-v3 optimistic server's `:confirmed` result with a justified projection; and the finished-system Part IV command taxonomy `:committed`, `:already-incorporated`, `:rejected`, `:commit-unknown` with browser `:accepted`, `:rejected`, `:unknown`. Their mappings are explicit, not inferred from an HTTP status or equal-sounding strings. In particular, the present source-backed Claim route does not establish durable historical replay of C42 merely by generating a protocol command ID. The Part IV same-transaction command-record/retention and disclosure mechanism remains a separate required implementation and qualification.

If Bob concurrently claims R17 with a different command, at most one new Claim can establish primary ownership from the eligible open state. In a fault-free run with both outcomes delivered, the other command receives the appropriate authoritative rejection. When outcome delivery is deliberately lost, unknown is a valid intermediate participant state and reconciliation is required.

If C42 commits but its response is lost, Alice's retry preserves C42 and its binding. It does not fail solely because R17 is now claimed. If a later Reassign has changed R17, a permitted replay can still report C42's historical success; the fresh card must reflect the distinct current observation.

If the result response fails render validation after commit, the invalid output is blocked. The commit remains real. Diagnostics and recovery must keep those facts separate rather than pretending preflight rolled back the transaction.

A checked semantic certificate can justify admitting the Claim plan under its premises. Separate evidence establishes installed dispatch and current atomic enforcement. Returning that certificate from the browser creates no Membership, principal, current Request state, or permission to disclose the command result.

Positive and adversarial capability histories

The following histories are acceptance **obligations**, not assertions that every scenario has passed in the running application. Each retains the source-owned business policy and selected product profile.

History
An eligible signed-in helper clicks the actual Claim button on an adopted open Request.
A lower-level integration test posts a manually encoded valid command while the actual Claim button is disabled or broken.
Alice's card lacks a justified XTDB observed basis though Claim is still a semantic operation.
Two eligible helpers race to claim the same open Request using distinct commands.
An unrelated XTDB transaction advances X17 to X23 before Alice clicks.
Alice's active Membership is revoked after a valid card was rendered.
Claim commits but its response or canonical Hiccup rendering fails.
C42's response is lost; Alice retries with the same binding after the Request became claimed.
C42 succeeds and a later legitimate Reassign replaces the current helper.
The browser reports a claimed card optimistically but no authoritative active primary assignment exists.
Alice Unclaims after a successful Claim and follows the rendered Unclaim control.
A helper clicks Mark on the way and later Complete from legal current states.
An eligible requestor Cancels a separate Request.
A manager selects a different eligible helper in the real Reassign menu.
A nonmanager sees a forged Reassign action or a stale selector includes an ineligible helper.
A full lifecycle browser test passes while Create, login, local search, Refresh or details never work.
A remote connected browser receives advisory after Alice's Claim.
A refresh/acquisition returns an older card after a newer justified installation.
An already consumed one-use Refresh grant is paired with a duplicated SSE advisory.
Browser session for the helper is absent or forged, despite a syntactically valid C42.
A real Chromium test exercises Claim, Unclaim, On-the-way, Complete, Cancel and Reassign with proper browser-generated commands.

These scenarios keep the R4 user-facing test architecture: unit model checks, Choreo projections, structural preflight, effective rendered affordance, server integration and real Chromium/HTMX product acceptance are distinct gates. No test may create the real browser wire command for it or treat a report map as authentication. Positive cases must stay reachable while all negative cases are correctly rejected or reconciled.

Board projection and selective adoption

The board separates authoritative truth from the information this browser has adopted. A database change does not automatically change a user's visible board. An advisory reports that the view may need attention; it is neither domain content nor permission to fetch and install everything currently available.

The selected policy is prompt adoption of the user's own correlated results and explicit Refresh for remote changes. Search, sorting, visibility options, and ordinary details interaction operate on already adopted information. These choices must be represented in application policy, not inferred from incidental HTMX route or callback behavior.

Scope, cohort, and presentation

A board scope includes Organization, Location, authority context, projection contract, temporal interpretation, and any declared collection bounds. Its adopted cohort is the set of Request identities admitted into that scope, with scoped observations and permitted display projections.

The default proving scope is one authorized Location and includes active and terminal Requests within its declared collection bounds. Terminal visibility is a presentation option. Reading only active Requests and later making “Show terminal” issue a fresh collection query would violate that option's contract.

The profile declares finite response/retention limits. If a complete result exceeds those limits, the server must return a typed scope-too-large result or use an explicitly declared paging/window contract. Silent truncation cannot justify complete membership or authoritative absence. A page covers its stated window, not the whole Organization. Moving to another page or Location is a new explicit acquisition intent, not a local sort operation.

Local text search and sort options do not change this authoritative collection scope. They derive a visible subset and order from the cohort. A future remote search feature must be labelled and specified as a different acquisition policy; it cannot borrow the “local presentation only” guarantee.

The complete-board acquisition composes Request, assignment, and permitted User display projections under the same qualified observation context. Repeated reads against an advancing database are not silently treated as one snapshot. If an integration deliberately combines observations, it preserves their separate provenance and proves only the relationships that their evidence supports. Current disclosure checks also remain distinct from the historical observation being displayed.

Initial acquisition, truthful membership and counts

The first rendered `GET /app` obtains an admitted initial Request cohort and permitted Request, RequestAssignment and User projection. The toolbar/list may be server-rendered, but a fragment's token does not by itself prove the other fragment has matching membership or counts. Any relevant change between initial read and subscription registration requires a source-owned fence, replay or authorized reacquisition. Merely connecting SSE does not establish a no-gap baseline.

A complete cohort result has an exact authenticated viewer, Organization/Location, collection and terminal inclusion, projection schema, temporal/query interpretation, read basis, finite bounds and completeness claim. Querying Requests, assignments and User display fields through successive reads against an advancing database does not prove one atomic snapshot. Either qualify their common context or retain the distinct provenance of every subread.

Displayed `:total-count`, `:active-count` and `:terminal-count` refer to this adopted cohort (or explicitly narrower adopted visible subset), not to the current global database. Unsupported, incomplete, unauthorized or truncated collections must be marked as such: empty results do not imply complete authoritative absence. Disclosing an old card still requires current read permission, independent of the basis at which it was first acquired.

State with distinct meanings

An inspection view of the board distinguishes the following records:

Record	Meaning and owner
Board owner and scope	One active coordinator lifetime, authority context, and applied acquisition scope.
Adopted membership	Requests admitted by a complete scoped observation or a permitted selective membership rule.
Adopted observations	Per-card and collection evidence actually installed, with provenance and scope.
Advisory obligations	Information still requiring acquisition/adoption, including unresolved subscription continuity gaps.
Acquisition grant	User or policy permission for particular reads and installations, with scope, budget, and lifetime.
Active cycle	Registered request and installation resources with current ownership.
Local options and drafts	Search, ordering, visibility, expansion, focus, and bounded unsubmitted input.
Provisional overlays	Execution-owned speculative presentation, not authoritative observations.
Command recovery records	Unresolved command bindings and permitted reconciliation references that can outlive a card.

These are not eight unrelated mutable registries. The executable semantic core owns their transitions; physical adapters report observations and completions. A presentation cache may store allowed fields, but it is not a second domain model or a source of write authority.

Resource identity and admission of physical installation

The board owner, scoped cohort, fragment resource instance, local option state, each card's physical DOM instance, issued acquisition, optimistic execution and unresolved command each have distinct identity and lifetime. The same Request UUID does not make a replaced DOM card the same active instance; a delayed callback cannot borrow the new instance's rights or reapply a retired provisional marker.

For an observation to become adopted, its trusted principal, permitted scope and corresponding Request, admitted progression/coverage, current board/execution owner and physical installation must all agree. The physical swap must complete; an HTTP 200 without installation is not evidence of adoption. A response prepared before a newer justified local card update cannot overwrite that card simply because it arrived afterward. Failed/cancelled swaps retain their unfulfilled read/adoption obligation.

Local filter changes may preserve focus, caret, scroll and compatible open `<details>` state across a managed HTMX replacement. Continuity does not resurrect an expired command, stale card owner or old acquisition grant. A local Create may admit exactly its correlated new Request, while a local Claim may update only its own qualified card. Neither can claim whole-board membership advancement through the new result's later basis.

A schematic inspection value is:

```
{:board/owner-ref board-owner
 :board/scope-ref board-scope
 :board/authority-context-ref authority-context
 :membership/adopted-ref membership-evidence
 :cards/adopted-refs card-evidence-by-request
 :advisory/obligations pending-obligations
 :acquisition/grant-ref current-grant
 :acquisition/cycle-ref current-cycle
 :presentation/options {:search ""
                        :created-order :newest
                        :mine-first? false
                        :unclaimed-first? false
                        :show-terminal? false}
 :presentation/result-ref local-result
 :commands/unresolved-refs unresolved-commands}
```

The references name admitted runtime objects or inspection subjects. Constructing this map cannot grant admission. In particular, replacing `membership-evidence` with a newer card's basis does not make the cohort a newer complete snapshot.

A concrete HTMX-compatible realization

The primary realization keeps server-rendered cards and a small browser presentation projection. An admitted observation supplies the card content and the permitted fields needed for local search, stable sorting, terminal filtering, and continuity. Local option changes select, order, or hide those already adopted cards without issuing an authoritative collection acquisition.

The browser projection contains only data the viewer may receive. Search indexes must not smuggle undisclosed email addresses, phone numbers, or other hidden fields into page attributes. Client-side display metadata remains untrusted as input to future server operations.

Search metadata is itself a disclosure surface. A match on an undisclosed email, phone or other restricted identifier can reveal private information even if it is never printed on a card. Indexes therefore include only permitted display/search fields and are retired or redacted when current access is lost. The current source's index contents are documented in the preservation audit.

HTMX remains the qualified transport and installation mechanism for authoritative updates. The application-specific local presenter uses declared managed effects for its own DOM changes; it cannot re-create retired cards, copy old owner annotations, or make an obsolete callback current. Stable Request identity is separate from the current physical card instance.

A server-rendered local-options alternative is valid only if it renders a retained, scope-bound projection without silently acquiring newer domain facts. Any retained projection handle needs principal/context binding, expiry, resource limits, and current disclosure checks. A browser-supplied list of Request IDs neither proves membership nor authorizes reads. The primary browser presenter avoids requiring this additional server snapshot service for ordinary options.

Search and options preserve admitted membership

The chosen realization may present an admitted browser projection or a retained, owner/principal/scope-bound server snapshot. A GET that requeries current collection membership is an authoritative acquisition, not a harmless local search. It must have a separately authorized grant; it cannot silently adopt remote Requests or discharge advisory debt. If retained data cannot be used, the renderer reports unavailable or offers explicit acquisition, never an unqualified latest-database fallback. This is a cohort-adoption rule, not a prohibition on Clojure or HTMX rendering. The existing requery path is analyzed in the audit.

Mixed observations are explicit

A board may retain its cohort from observation Q17, adopt a local result for S at another qualified observation, and keep an older card for R. It must preserve those distinctions. There is no scalar board “current revision” that proves all these cards and membership were read in one database snapshot.

Collection counts are counts of the specified adopted cohort or visible subset. A separately acquired current count needs its own scope and disclosure contract. Neither a pending-obligation count nor a progression token is a count of new Requests. The default advisory label is “Updates available”, not an invented number of unseen Requests.

Positive and adversarial board histories

The independent product oracle uses actual adopted membership, provider- scoped read evidence and two browser contexts where remote actions matter. A test must observe the remote advisory **before** asserting that its Request stayed absent; otherwise the assertion may pass vacuously.

History	Required result
The initially served `/app` includes a permitted terminal Request and a live interest is registered afterward.	Record authorized membership
`show-terminal?` changes from false to true.	Reveal terminal members alrea
Alice creates R remotely while Bob's board retains Q17.	Bob receives advisory; R and a
Bob searches for R's exact title after receiving its advisory.	Search only the admitted cohor
Bob changes mine-first, unclaimed-first and newest/oldest after R arrives remotely.	Deterministic order of existing
Bob creates S locally at P19 while remote R at P18 still waits.	Adopt only the correlated eligi
Bob's current filter excludes newly created S.	The correlated result remains v
Bob Claims an already adopted T while R is remote.	Install a justified T observation
S's card has basis P19 but board membership is still Q17.	Preserve different scope/progr
Search/options invokes a fresh `requests-for-location` query without Refresh grant.	Classify it as unauthorized add
A retained server projection expires or belongs to another principal or deployment generation.	Reject reuse or require a newly
Request and User display rows were read at different physical positions.	Keep independent provenance
A result exceeds configured full-cohort limits.	Return typed incomplete/scope
An older Refresh response arrives after a newer local authoritative card was installed.	Keep the newer applicable card
New advisory O2 arrives while Refresh covers only captured O1.	Discharge O1 only after author
A local options rerender replaces an expanded card while search is focused.	Restore compatible `<details>`,
A late callback addresses a Request with the same UUID but an obsolete card DOM instance.	Refuse it as a current owner; n
Viewer loses current Membership to previously adopted R.	Withhold further disclosure an
Search indexes an unauthorized User email/phone value.	Prevent disclosure through ma
Advisory UI displays “3 new Requests” based solely on notification count.	Reject unsupported numeric cl

History	Required result
User switches Location or explicit collection page.	Require a new authorized scop

The existing `example.board` unit tests exercise source-level presentation semantics on supplied production rows and distinguish Live/XTDB observed basis from Request revision. The existing Chromium lifecycle test exercises real browser operations and persisted outcomes. Neither automatically establishes this complete selective- adoption and two-client board matrix.

Creation and immediate local visibility

Create uses the production public operation, including trusted principal, authorized Organization/Location, content validation, command binding, atomic incorporation, and post-commit recovery. Client validation improves feedback but does not replace public-boundary or transaction checks.

Trusted Create authority and preflight before the form is used

The rendered dialog collects editable Request content. The trusted server resolves the authenticated production User and selected authorized Organization/Location, not a posted requestor ID or a fixed fixture value promoted into authority. The public `net.humanhelp.site.model.request.core/create` operation owns content normalization, active-user and operational-Location dependencies, command construction and atomic Request incorporation. The reference application may set a fixed example Location as a declared **fixture scope**, but it must not infer tenant or Membership permission from that constant.

Model planning needs real current-time, generated-ID/seed, transaction and XTDB capability inputs with the correct trust and lifetime. The canonical Biff/Gesso assembly checks their required provider contracts and actual handler adoption where knowable. A missing `:biff.fx/now`, `:biff.fx/seed` or production commit dependency that can be established before protected service is an admission defect, not a surprise error after a user has filled the dialog. The exact Biff FX provider mechanism is a physical host contract, not a browser-supplied clock or a stand-in demo function.

Validation failure is **pre-effect**: preserve the user's submitted title/details/location-detail, render source-owned field errors without creating a Request, and preserve the current board and its advisory/option state. A submit interrupted after the effect may have occurred is **not** a validation failure. It follows the stable command and unknown-outcome recovery contract.

On known successful creation, the user's own Request is acquired and installed through a correlated result path. That path can adopt the new Request into the current cohort only when it establishes the selected scope's membership predicate and permitted display projection. An entity identity or success flag alone is insufficient.

The response must not issue an unrestricted board query and replace the whole list as a convenience. Such a query can reveal unrelated remote Requests that this user has not chosen to adopt. A committed command's progression requirement is a minimum read condition, not authorization for every observation at that position.

Success while a local filter excludes the new Request

The user must receive visible confirmation without an implicit filter reset. This design provides a “Your new Request” result area adjacent to the board when the new card would otherwise be hidden by local options. It shows the permitted authoritative card and explains that the current filters exclude it.

The result area offers a visible action to reveal the Request in the ordinary list by explicitly changing the relevant local options. It does not offer an unlabelled whole-board Refresh as if that were required to see the user's own result. When the Request already matches the filters, it appears in the list and the confirmation can reference that card without creating a second active operation surface for the same logical target.

The area retains the latest completed local creation result; replacing or closing it affects only that presentation. Prior creations remain in the adopted cohort when eligible. A pending or unknown creation retains its independent command recovery state and is not discarded by a later successful creation.

The result area has a stable, permitted installation owner and accessible success/uncertainty message. Its “Reveal in board” action changes only local search, ordering or terminal visibility when that is sufficient; it does not issue an implicit new domain acquisition. When S is actually outside the selected Location, page/window or authorization scope, the area labels that difference and requires an explicit navigation/new acquisition. Closing a success result cannot silently discard S's admitted cohort membership, and rendering a second success must not erase an earlier unresolved command's recovery reference.

The result area is a normative finished-product design choice. When the applied search excludes S, the confirmed creator-local S remains independently visible and actionable without resetting options or admitting unrelated remote R. Acceptance establishes the excluding filter **before** Create and verifies authoritative incorporation and a usable visible result, not merely a toast.

If creation succeeds outside the current cohort's declared bounds, the result area can still show the permitted result under its own scope. It must label the scope difference and offer an explicit navigation/acquisition action rather than pretending the board's membership rule admitted it.

Prompt visibility means that the confirmed result is usable after the declared result installation completes. A provisional card before confirmation is allowed but is visibly pending and owned by the creation execution. No toast, modal closure, or speculative row by itself proves successful incorporation.

Commit, response, advisory and historical recovery are distinct

The production model's `:commit/status :committed` and scoped `:progression` justify an authoritative incorporation/read claim within their proven physical meaning. The stronger Part IV same-command replay result, permitted historical disclosure and browser `:accepted` presentation need their own typed evidence. A generated Request UUID, a visible toast or a current postcommit card does not by itself prove the historical outcome for original command C19.

The creator response and other-client notification are distinct postcommit responsibilities. Correlated creator settlement may succeed independently of advisory delivery. An advisory failure does not reverse a committed Request or authorize blind retry.

There are additional histories to qualify. If Request incorporation succeeds but creator-response rendering throws before the advisory helper is called, the mutation is still committed, neither creator installation nor remote advisory delivery is established, and the participant needs correlated recovery. If the response was constructed but transport or OOB installation fails, the server cannot infer that the creator saw S. If advisory delivery fails after response construction, the creator's success need not be revoked, but the other participants' notification/reconnect obligations must have a declared durable or otherwise honestly weaker recovery policy. A best-effort error log does not establish durable or unconditional remote delivery.

An application with reliable postcommit delivery records the necessary publication intent and command recovery debt under the selected atomicity/ retention premise. It must not claim two-phase commit or cross-client exactly-once effects merely because a single Node/XTDB transaction was committed. Retried Create with the original stable binding is resolved before re-incorporation; an unqualified implementation without such deduplication keeps the outcome uncertain and does not blindly regenerate S.

Worked example: remote R and local S

Bob has adopted board observation Q17. Alice creates Request R. Bob then creates Request S. Both are eligible for the same board collection; Bob's text filter matches R but not S. P18 and P19 below name provider-issued positions with the specified coverage judgments, not numbers the browser is allowed to compare.

Step	Bob's adopted membership and presentation
Baseline Q17 is installed.	The baseline cohort only.
Alice commits R and a relevant advisory reaches Bob.	R is absent; “Updates available” appears.
Bob searches or changes sort options.	Only the baseline cohort is presented differently; R
Bob commits S and installs its correlated permitted result at P19.	S is selectively admitted; its result area is visible de
Bob changes local options to show S in the ordinary list.	S is visible there; no new membership is acquired.
Bob presses Refresh and installs a complete authorized observation covering R and S.	Both are in the adopted cohort, subject to current l

The test must assert S's visibility before changing the excluding filter. Clearing search first would hide the product failure this scenario is intended to detect. It must also wait for observable advisory delivery before asserting continued absence of R, so that an assertion made before the remote event cannot pass vacuously.

The test also checks Bob's pre-Refresh cohort identity and current local options, one permitted Create result for S, requestor identity, production persisted Request state, retained pending R obligation, and physical installation of the dedicated S result area. A card for R appearing merely because the Create success response queried all new Requests is a failure, even if R's contents are accurate. If the R advisory arrives after S's result, the absence requirement is still meaningful; the test explicitly synchronizes that notification and verifies that S's local result did not consume it retroactively.

The R4 example is a **logical history**, not an instruction to compare the decimal portions of P18 and P19. The provider defines which basis/coverage statements may be composed. Local search and sort must not silently query the newest database as a workaround.

Correlation is narrower than global adoption

A command result can justify updating its affected card and, where declared, selectively adding/removing a member. It does not adopt every Request mentioned in an unrelated response, every row from an incidental reread, or every change before its progression position.

A correlated result is narrower than a recomputed board

For an admitted creator-local insertion, the settled result identifies the newly committed Request and its original command/execution binding. The trusted authority establishes its requestor and scope, the relevant transaction/progression and a permitted Request/assignment/User display projection. A successful HTTP status or submitted title alone does not justify that new member or its physical DOM.

The implementation may acquire a single affected Request/card with sufficient progression, or install the admitted complete result of a separately granted board acquisition. It may **not** reuse Create success as permission for an unrestricted `requests-for-location` reread whose resulting toolbar/list quietly imports remote Requests that the user had not chosen to adopt. A committed basis P19 can justify the creator's S read while membership for unrelated R remains Q17.

The creator's accepted operation permits a scoped affected-Request result; it does not authorize a whole-board reread that imports unrelated remote R. A full-board result is permitted only under its own admitted query and adoption grant. The view must preserve prior adopted members and outstanding remote debt. The present whole-board OOB source behavior remains documented in the audit.

A board fragment and toolbar may need different membership/count evidence. A newer card for S does not make the toolbar's total, open count or other cards current. The UI reports a mixed cohort truthfully and preserves any pending remote advisory.

Advisory debt is discharged by the relevant observation and installation scope. The application cannot clear “Updates available” simply because it received a newer local result. Two obligations with different scopes or required stages may need different evidence even when their provider positions are comparable.

Positive and adversarial Create acceptance histories

The actual ‘/app‘ Create dialog, public production Request model, source-bound server response, managed browser assets, Live advisory and physical DOM have distinct test witnesses. Required eligible success must be observed, not inferred from negative-case checks or a fabricated test-only Create result.

History	Required result and scope
An eligible signed-in user submits valid Request content.	Production Create incorporates one Reques
Client-side validation passes but trusted Request content validation rejects the title.	No protected effect; keep submitted values a
Mandatory Biff `:now` or `:seed` provider is structurally missing.	Fail application/operation admission before
A forged form posts another requestor or an unauthorized Location.	Trusted context/model determines actor and
S commits with a matching filter and valid cohort scope.	Install its permitted authoritative card into t
S commits but the active search excludes it.	Keep the filter and visibly show “Your new
S commits outside the current board's window or Location.	Show a separately scoped permitted result a
Bob has Q17, Alice creates remote R, then Bob creates S locally at P19.	Bob sees S and retains R's advisory/nonmem
Bob searches for remote R after its advisory but before explicit Refresh.	Do not acquire R by changing the local que
S is successfully read at P19 while the board membership and toolbar counts remain Q17.	Preserve scoped evidence and truthful distin
The success response includes a full, newly queried toolbar/list that now contains remote R.	Classify as an unintended board acquisition
The Create response builds correctly, then the advisory fanout throws.	Keep committed creator result and log/retai
Model commit succeeds but creator-response rendering fails before advisory dispatch.	Preserve committed authority and both unr
Server writes a 200/HTML success, but HTMX OOB fragment installation is rejected or fails.	Participant still lacks adopted S until physic
Duplicate transport retry carries original Create command C19 after unknown response.	Resolve disclosed historical incorporation w
An unrelated remote advisory arrives while S's local result installs.	Retain that obligation, even if the provider-
A second successful local Create T replaces the result area's display of S.	Keep eligible S in adopted membership and
A stale callback from S's retired result target arrives after current board owner changed.	Refuse old resource installation rights; requ
A current permission change makes S's details inaccessible to the creator.	Do not continue unauthorized disclosure m
A test starts with search matching remote R, then clears it before creating S.	Do not count this as the filtered-local-result
Remote Browser B has no advisory yet because fanout was lost.	Do not claim no-gap eventual notification f

Section 08.04 owns the baseline cohort and disclosure scope; 08.06 owns remote lifecycle advisories, Refresh grants and reconnect rules; 08.07 owns details and option interaction. Create does not acquire an extra broad authorization merely because the current example route happens to redraw the full board.

Remote lifecycle adoption policy

Selected policy: remote creation and remote lifecycle changes both require explicit board Refresh for ordinary domain-content adoption. This includes changes to Requests already visible. An advisory may update the pending indicator and explanatory status; it does not silently replace a card or its cohort.

This policy is intentionally separate from two permitted local continuations:

- The user's own accepted command may install its declared affected-card/result observation and selective membership effect.
- A rejected or uncertain local command may invoke its declared scoped recovery read. That read can reveal current state of the affected Request, even when another actor caused it. It cannot import unrelated collection members or automatically resolve the uncertain command's attribution.

An old displayed Claim button can therefore remain visible after another helper claims the Request. Clicking it is safe only because the public model enforces current authority. The resulting rejection and affected-card acquisition explain the conflict. An advisory may say the view has updates, but must not infer the winner or current domain state from advisory payload alone.

Authentication loss, explicit scope change, and an established loss of disclosure rights are separate context transitions. They retire incompatible execution and installation rights and remove protected projections where the supported client can do so. They are not delayed merely to preserve a board Refresh preference. No application can retract information already disclosed to an untrusted or offline browser; the enforceable guarantee is current checks at subsequent trusted disclosures and mutations, plus the declared supported-client behavior.

Remote lifecycle remains advisory-only

The selected end-state rule requires explicit Refresh for both remote Create and remote lifecycle changes, even to already-visible Requests. Managed Live invalidations must not bypass this user-grant policy through automatic collection rereads. Local affected-Request recovery and trusted revocation are separate. The history of R4's choice and the current routing mismatch are recorded in the 08.06 preservation audit.

A remote change to an already visible T must not reveal its new helper, terminal status or Request content through a supposedly advisory payload. A stale Claim button remains subject to current Request guards. A rejected or uncertain local command may independently acquire the affected T to explain conflict, without silently importing other Requests. Authentication loss or established revoked read rights are different trusted safety transitions: they retire forbidden projections and mutation permissions rather than waiting for optional Refresh. Previously disclosed bytes in an untrusted/offline browser cannot be retrospectively erased.

Explicit Refresh, coverage, and races

A Refresh click creates a grant for one logical acquisition of the current board scope and authority context. It captures the outstanding requirements and the current non-regression obligations. Its lifetime ends on completion, failure, expiry, owner retirement, or incompatible scope/context change.

The default has one acquisition and no automatic successor read. It can obtain a complete current query result that includes changes beyond the captured requirements. Installation can discharge any current obligation covered by that actual result within the grant's scope. It does not authorize a second read just because another advisory arrived.

A new explicit Refresh or Retry can create a new grant. While a cycle is active, additional clicks are coalesced into at most one user-authorized successor intent with current captured requirements; they do not create an unbounded queue. A pending indicator alone never creates that intent. The UI reports whether it is refreshing, waiting for another explicit action, or unable to refresh.

Refresh permission and issued read scope

A single user gesture creates a bounded grant with an authenticated principal, current Location/collection scope, board owner and deployment generation, captured requirements, physical target, expiry and permitted **one logical** query. The trusted server checks its authority independently of browser-supplied scope, basis, client stamp or form values. A malformed/impossible progression token must be rejected or boundedly deferred, not force an indefinite query wait. Advisory arrival is not a new read permission.

A second real click while a query runs can authorize at most one successor intent; neither an automatic retry nor a duplicate wakeup may create an unlimited queue. Expired, incompatible or cancelled grants preserve unresolved advisory debt and allow an explicitly admitted retry.

A rendered Refresh control and successful HTTP/OOB response do not alone establish a one-use grant, captured requirement ledger, correct response owner or qualified physical adoption. Each stage needs its own source and observable acceptance evidence.

A deployment may choose a bounded automatic repair/drain policy instead, but that is an explicit product-policy change with its own budgets, expiry, and acceptance cases. It is not an adapter optimization invisible to the user contract.

Acquisition and adoption are separate stages

1. Register the cycle, transport identity, scope, target owner, and requirements before physical dispatch.
2. Authenticate the actual read and validate the query/progression input under the qualified provider contract. A hostile or impossible token must not force an unbounded wait.
3. Acquire a projection with evidence of actual observation, completeness, and applicable authority context. Echoing the requested minimum does not prove the read met it.
4. Admit installation against the current owner, target instances, grant, scope, compatibility, and non-regression requirements.
5. Perform the qualified physical replacement/merge and report completion.
6. Record adoption and discharge only obligations satisfied at their required stage. Render current local options over the newly adopted cohort.

HTTP completion, response receipt, a proposed swap, or a backend read alone does not complete stage 6. If installation fails, acquired evidence may remain useful but the adoption/repair obligation remains visible.

Toolbar counts, list membership, cards, and board evidence must refer to the same declared installation or explicitly identified partial result. The primary full-board path renders one coherent board projection and coordinates its affected surfaces as one adoption batch. Where HTMX performs several physical mutations, admission is checked before the protected work and completion tracks each required effect; multi-fragment delivery is not claimed to be DOM-atomic. A partial failure leaves the batch incomplete, prevents a false “up to date” claim, and exposes repair without rolling back a newer independently admitted card. Re-fetching toolbar and list independently cannot manufacture a coherent board observation merely because both responses arrived successfully.

HTMX OOB output has distinct completion stages

With `hx-swap`”none”=, a response may still change toolbar, list, board-state and advisory targets through individual out-of-band effects. The coordinator must validate the planned complete board and current resource ownership before the effects, observe each required physical installation, preserve compatible current search/options and only then mark the batch adopted.

If the toolbar swaps but the request-list target is absent, HTTP receipt is not a coherent refresh. The application records a partial batch, retains unsatisfied advisory/repair requirements, and does not discard a newer locally installed S card. If one component needs a separately scoped acquisition, the result is labelled **partial** rather than falsely presenting a common snapshot.

Toolbar `:pending-open-count` and `:stale?` are presentation fields, not semantic coverage evidence. Replacing advisory markup cannot discharge incomplete, later or uninstalled Refresh obligations or create a new query grant.

Race rules

The default allows a non-regressing complete-query advance even when a stronger remote advisory remains outstanding. This is partial satisfaction of current debt, not partial or fabricated completeness of the query result. The remaining advisory stays visible for a later user action.

Race or failure	Required result
A later advisory arrives during a Refresh.	Preserve it; clear it only if the actual installed observation co
The read covers the captured obligation but not the later one.	Adopt the non-regressing result and retain the later obligati
A qualified read also covers the later obligation.	Discharge both after successful installation within the perm
A local Create/Claim result is installed while an older board read is in flight.	Recheck the incoming board result against the newer local c
An older result cannot establish safe replacement.	Decline that installation, preserve the local result and debt, a
A stronger result arrives from a retired request or board owner.	Reject its installation rights; stronger progression does not r
A read succeeds but physical installation fails.	Preserve remaining adoption debt and a usable retry path.
Local search/options change during the read.	Install into the same acquisition scope if still valid, then app
Organization, Location, page/window, or authority context changes.	Retire or explicitly transition the owner; the old response ca
A command remains unresolved when a board refresh completes.	Update permitted presentation without claiming the comma

A full-board replacement must not regress any retained observation whose scope it supersedes. If proof of safe replacement is unavailable, the default rejects that replacement and requires a new acquisition. It does not silently union an old complete result with arbitrary newer rows and call the union a complete snapshot. A mixed selective projection can be valid, but must retain its mixed provenance and must not claim snapshot-wide properties.

Opaque progression and notification delivery premises

Generic Gesso progression bases are opaque. A trusted XTDB/provider relation, not integer comparison, HTTP arrival order or the latest visible token, establishes whether a read actually covers a captured or subsequently arriving requirement. An affected-card observation may advance one subject while the board cohort remains older. The same user action can legitimately satisfy more than one obligation if the **one** authorized read and completed adoption cover each; a later advisory never creates a second read grant.

HumanHelp's separate app-owned creation advisory uses `client-plumbing` to queue non-adopting Refresh OOB/toast presentation to connected clients other than the creator. Gesso `live.client` owns finite ephemeral pending mailboxes, including disconnected-client overflow and drop policies. This mechanism does not prove durable eventual delivery through disconnection, queue overflow, multi-node failover or a missed subscription. A stronger product guarantee needs source-owned reconnect/fence/replay or authorized reacquisition. Without it the system must expose the weaker contract, not report a permanently silent stream as current board knowledge.

The notification is a wakeup, not the authoritative current Request state, a sufficient current disclosure capability, or a count of unseen new Requests. A durable command record and a board Refresh are separate recovery subjects.

Membership removal and authoritative absence

An authoritative query result can remove a cohort member only with the required coverage and current installation rights. A filtered-out row, a failed read, missing markup, or an empty partial page is not evidence of domain deletion. Terminal filtering hides `:done` and `:cancelled` in the default view without removing their admitted cohort membership.

If an affected-card read establishes that a Request has left the authorized scope, the UI follows the declared non-disclosing absence/unavailability branch. It must not reveal whether an inaccessible entity was deleted, moved, or made private unless policy permits that distinction. Such a read still does not resolve a prior command's attribution without command evidence.

Pending provisional presence has a resolution owner. It cannot survive every complete authoritative observation indefinitely, and it cannot disappear with its command-recovery record merely because a DOM row was removed. Part VI's separate rules for presentation supersession and command uncertainty apply.

Positive and adversarial Refresh acceptance histories

These are required finished-product scenarios, not assertions that the currently checked Chromium lifecycle source already passes the complete two-client Refresh matrix.

History	Required result
Alice creates R while Bob has adopted an older board.	Bob sees advisory, not R; Alice's correlat
Alice claims T already visible to Bob.	Bob's T remains stale with advisory unti
A manager reassigns Bob's visible T to another helper.	No changed-helper card is silently insta
Another user completes or cancels an adopted Request.	Bob's card and terminal visibility do not
Bob clicks the stale Claim button.	Current model guards reject or commit
Bob loses read permission to a previously adopted Request.	Current trusted reads/mutations refuse
An incoming `:updated` invalidation tries to schedule a managed list GET.	The final application converts it to advis
A creation OOB advisory arrives without canonical Request HTML.	No remote Request membership is adop
Bob clicks Refresh once with obligation O1.	One authorized coherent board read ma
O2 arrives after the O1 Refresh began.	Preserve O2 unless actual installed read
Bob clicks Refresh again while O1 is running.	Allow at most one separately user-autho
The Refresh caller sends an impossible basis or forges Location.	Reject/qualify input at trusted boundary
Bob installs local Create S while an older full-board read is in flight.	Decline regressing full replacement or u
A returned Refresh belongs to a retired owner or old Location.	Do not install, even if the provider basis
Toolbar OOB succeeds but Request list target is detached.	Keep the batch incomplete and advisory
A response is delivered but the browser fails to install it.	HTTP success does not discharge pendi
A coherent board query cannot prove compatibility of separately read Request/User projections.	Retain honest mixed provenance rather
Search/options change while Refresh is in flight.	Keep current compatible options, not th
A terminal Request is absent because `show-terminal?` is false.	Hiding does not prove authoritative dele
Bob sees a changed helper after Refresh while C42 remains uncertain.	Current card knowledge does not identi
A notification is dropped after ephemeral client-mailbox overflow.	Qualify delivery as incomplete; reconne
Advisory O2 is duplicated after a one-use Refresh grant was consumed.	Do not mint another read grant or erase

The real HumanHelp Chromium acceptance source covers browser-generated Claim, Unclaim, On-the-way, Complete, Cancel and Reassign with persisted reloads. It does not by itself establish all these two-client advisory-only remote updates, one-use grants, multi-target installation, loss and failover cases.

Search, ordering, visibility, and local state

The ordinary board presenter is a pure function of the adopted permitted projection, normalized local options, and the current authenticated-viewer projection. Its DOM realization is an admitted effect. Presentation does not re-evaluate production write authorization and does not manufacture observations.

Search uses a declared shared normalization contract: trim the input, apply the selected case normalization, and perform substring matching against the prepared search text. The contract specifies the supported character behavior and is checked on both host runtimes where both implement it. Search text comes from already disclosed Request title, details, location description, and explicitly permitted displayed labels. New private fields cannot become searchable merely because they are present in a production document.

The empty search matches the adopted cohort. No match means no visible adopted row matches; it does not assert that the database has no matching Request.

The adopted-cohort search and disclosure contract

The search index is an **already authorized projection**, not a second Request model and not a backdoor query. For each current viewer the index may contain only approved display/search fields from the admitted Request, RequestAssignment and User projections. Whether a field is drawn visibly or only used to decide matches does not change its disclosure classification: a match against an otherwise undisclosed email or phone can reveal that information. Identity, scope and current authorization changes revoke the applicable retained search projection, not just visible card markup.

A selected portable normalizer fixes whitespace trimming, case folding and Unicode/locale behavior on every host that implements the transformation. A ClojureScript search and Clojure server renderer must produce the same membership and order for the same qualified rows and options; JS and JVM case conversion are not assumed universally identical for all possible Unicode inputs without a tested contract. Typed unsupported characters or normalization differences must be reported/handled by that contract rather than silently expanding a match.

Search and option changes transform the viewer's currently adopted, authorized cohort rather than silently making a newest-database collection query. Server-rendered HTML/HTMX remains valid when it operates on a retained qualified cohort. This invariant needs real-browser acceptance, including outstanding remote advisory.

Each search edit has a presentation generation. If an earlier server response or scheduled client effect arrives after a later edit, it cannot restore the old search or install an older filtered result. An empty result means ”no matches among the adopted permitted Requests”; when membership is incomplete or its continuity unknown, the message makes that limit clear rather than claiming that no matching Request exists in XTDB.

Ordering is a single explicit key

“Mine” means requested by the authenticated viewer or assigned to that viewer as active primary helper. It is a presentation category, not permission to act. The prepared projection supplies these facts from the production models. A missing viewer ID must not make two absent identifiers compare as “mine”.

When both priority options are enabled, mine-first is the primary grouping and unclaimed-first is the secondary grouping. Creation time then uses the selected newest/oldest direction, with canonical Request identity as the deterministic tie breaker. The UI makes this priority order apparent.

An illustrative key over prepared display rows is:

```
(defn board-sort-key [row options]
  [(if (and (:mine-first? options) (:viewer-related? row)) 0 1)
   (if (and (:unclaimed-first? options) (:open-unassigned? row)) 0 1)
   (case (:created-order options)
     :oldest (:created-order-key row)
     :newest (:reverse-created-order-key row))
   (:canonical-request-order-key row)])
```

The temporal order keys are prepared under the declared portable ordering contract; the example does not assume arbitrary host timestamps are negatable JavaScript numbers. Unknown order options are rejected or normalized through the explicit option schema before this function runs. A malformed creation time is a projection error, not silently the Unix epoch.

Malformed or absent timestamps are typed projection errors, not valid epoch-zero values. Temporal sort keys must be portable across selected hosts with deterministic identity tie breakers and no unqualified overflow or representation assumptions. The JVM algorithm currently inspected belongs in the section audit.

“Unclaimed” is a model-derived open/unassigned category. A terminal Request with no active assignment does not become unclaimed. Multiple independent sort passes must not accidentally change which option is primary.

Sorting acceptance covers simultaneous priority flags, ties, missing authenticated viewer and the distinction between open/unassigned and terminal Requests. Missing viewer IDs must not compare as ”mine”; a viewer-related grouping is never business authorization for Claim or protected disclosure.

Order of transformations is declared: authorized adopted cohort, terminal visibility, local search, primary/ secondary sort groups, temporal key, canonical identity. A filter does not mutate adopted cohort membership; a changed order does not grant new authoritative read permission.

Defaults, drafts, and updates

The default options are empty search, newest first, mine-first off, unclaimed-first off, and terminal visibility off. Search changes apply locally as the user types using a bounded scheduled effect; stale callbacks cannot restore earlier search text. Options-dialog drafts become applied options only when the user confirms, and Cancel leaves applied options unchanged.

Authoritative Refresh preserves the current compatible local options. It does not restore options captured when a slow request started. Initial loading, explicit scope navigation, and user-requested reset have their own documented initialization behavior.

Form and asynchronous presentation ownership

The options dialog has a **draft** distinct from the applied values. Done commits the whole compatible normalized draft as one local presentation action; Cancel, Escape and dismissal must not leak partial option changes. The selection remains visible, keyboard-operable and persistently reflected in the controls after permitted rerenders. Unknown sort variants can normalize to documented defaults only when the accepted option schema allows that; a hostile authorization, Request or progression field submitted with the form is not a presentation option.

Current `example.views/board-options-form` renders newest/oldest, mine-first, unclaimed-first and terminal controls through a server-side HTMX POST, and preserves search via `hx-include` of the stable form. Its Done action is real. These are valuable UI building blocks, but a round-trip that computes fresh board state does not automatically meet the selected local-only adoption or draft/Cancel behavior under concurrent updates. The source-level `board_test.clj` exercises isolated filter/sort cases, not every browser interaction.

Typing search is a bounded, cancelable presentation effect (the current example declares a 250 ms HTMX keyup/search trigger). Late callbacks from old form/card resource instances are excluded by the current owner. Refreshing after a search begins must reapply the **current** search to newly and legitimately adopted board facts, not overwrite it with the request's captured text.

Local state may preserve which Request details are open, focus, caret, scroll anchor, and bounded unsubmitted text. It may not preserve old authoritative status, helper assignment, removed command capabilities, or obsolete target and owner annotations as if those were harmless continuity.

When a card disappears, restoration falls back to an existing safe target such as the board heading or the next logical control. A local draft tied to that Request must be labelled as no longer attached to an available target or retained in its separate permitted draft surface. It cannot cause the old card to be resurrected.

Continuity is physical restoration, not authority

Real-browser continuity tests must observe physical scroll, focus/caret and native `<details>` restoration for valid same-Request replacement and safe retirement. Declaring a key or listener is not proof of completed restoration.

Restoration is admitted only when the same authorized Request and compatible new card/fragment owner survive. A terminal filter may hide a card without deleting its admitted cohort membership; a Location change, loss of read authority or authoritative absence may retire the card entirely. In that case details and old action bindings are not restored, and focus moves to a safe existing control. Captured scroll or selected text cannot bring hidden protected content back into the DOM. An open accordion alone is not permission for a fresh model read or for a provisional command to continue.

Details, response interaction, and accessible feedback

Opening Request details reveals the permitted information already adopted for that card. The default details interaction is local and non-mutating. It does not implicitly update the card from the latest database state.

A separate explicitly labelled “Get current details” interaction may acquire a new single-Request projection under its own grant. Its implementation must state which card/details observations it adopts and how it preserves board membership and outstanding board obligations. It is not an unrestricted collection refresh.

Details projection, disclosure and scoped current reads

The default details panel uses the authorized Request projection already owned by the card; opening or closing it does not call `request.core`, advance the cohort, change status, or make a domain submission. It does not implicitly disclose email/phone or private Location data that the signed-in viewer was not allowed to receive. Any separately labeled ”Get current details” affordance requires a trusted, current single-Request acquisition grant, read-time authorization and progression/owner checks. It may supersede that Request's details observation while leaving unrelated board membership and pending remote advisories untouched.

An inaccessible Request need not reveal whether it was deleted, moved or made private. The application follows the selected non-disclosing failure branch unless the viewer has permission to distinguish those states. A previously expanded card remains subject to current disclosure and resource retirement when a new response is served.

Controls must be reachable by keyboard, have meaningful labels, and expose pending, accepted, rejected, and unknown states without requiring color alone. Updates-available and completion announcements are coalesced so repeated advisory delivery does not repeatedly interrupt the user. A failed refresh remains retryable; a disabled control explains its relevant availability condition where disclosure permits.

Technical identities and proof vocabulary belong in diagnostics, not ordinary product instructions. A user should see “We couldn't confirm whether your claim succeeded; check again” with a recovery action, rather than a raw progression requirement or resource-instance error.

Accessibility acceptance includes native form/dialog focus order, keyboard opening/closing of `<details>`, visible focus, discernible search/order labels, logical focus fallback on removed cards, state text and permitted announcements for advisory, pending, accepted, rejected and unknown outcomes. Screen-reader notifications are coalesced or limited by the selected accessibility policy; repeated SSE wakes must not become repeated disruptive announcements. The ordinary user gets a next action, not a theorem, command ID or internal XTDB basis. A timed or disconnected response retains a visible retry/recovery action rather than a spinner that can never settle.

Real-browser accessibility tests exercise labels, keyboard interaction, dialogs, accordion state, focus fallback and announced pending/unknown/repair outcomes. ARIA attributes or a declared continuity contract alone cannot establish that a real browser or assistive interface behaves correctly.

Independent response-flow contract

Status: unresolved product contract. “Response flow” is a required capability name, but that name alone does not specify a user outcome. It cannot honestly be implemented or accepted by choosing whichever existing handler is most convenient.

Before claiming full rich-board completion, HumanHelp must select one explicit meaning and record its independent contract. It may mean responding by taking responsibility through the lifecycle, contacting a requestor through a permitted channel, or another defined interaction. Those alternatives have different persistence, privacy, authority, and recovery obligations. This specification does not invent a message entity, messaging provider, or contact-disclosure rule.

The product decision and assurance boundary

The response flow must be selected by an independent HumanHelp product decision, with a versioned acceptance profile and a designated model/UI owner. This section does not infer a requirement to create messages, send email/SMS, reveal contact information or acquire a new Request domain role merely from the name ”response flow”. In particular, an existing `Claim` button does **not** automatically resolve this decision. The product may select ”respond by claiming” only if its independently authored user-intent and outcome contract explicitly says that this is the full promised response experience.

The selected flow's permission, storage and external-effect assumptions must be checked against production `request.core` and the relevant public Organization, Membership and User contracts. If it is non-mutating, an authorized read or local disclosed interaction may satisfy it under a separately testable observable outcome. If it mutates Request or emits external communication, it needs actual source-owned semantic identity, guarded model or qualified external effect, durable/uncertain outcome semantics and valid real-browser execution. The example cannot introduce a parallel response store just to satisfy this milestone.

The selected contract must define:

- the user's intent, eligible actor, Request preconditions, and actual affordance;
- input and validation, including what information may be disclosed to whom;
- the visible success result and any durable domain or external effect;
- the production operation or explicitly non-mutating interaction that realizes it;
- its relation to Claim, details, and other existing lifecycle capabilities;
- reload/reacquisition behavior and any relevant duplicate, concurrent, rejected, uncertain, cancelled, or abandoned interaction;
- an eligible real-browser completion scenario and the applicable negative case.

If the product chooses “respond by claiming”, acceptance must exercise that complete labelled user journey and explicitly establish that no separate capability is promised. If it chooses a different action, Claim tests cannot stand in for it. An unchosen contract is reported as unresolved, not passed, not silently dropped, and not counted as a framework failure.

Independent product acceptance for the selected response

Before accepting the full HumanHelp rich-board milestone, record the selected response-flow option, eligible actor, observable user intent, source/model owner, specific control and permitted outputs, whether and what changes, recovery/deduplication and privacy/disclosure assumptions. Then execute at least one eligible real-browser response with fresh authorized observation of the declared outcome, plus an ineligible/ concurrent/uncertain path appropriate to the selected flow. If selected behavior contains a communication effect, its recipient and delivery evidence must be explicit; a toast is not a delivered message.

An unresolved product decision is **not** a Gesso type failure or a license to remove the independent requirement. It is a named HumanHelp product acceptance blocker. The application's other capabilities can receive their narrower individual passing evidence, while the complete product profile remains unaccepted until selection and real outcome verification.

Positive and adversarial interaction histories

The following cases exercise real production-backed adopted rows and the physical board, not a test-only copied array presented as the entire product. Source unit evidence and real browser evidence remain separate.

History	Required result
The viewer types a trimmed, mixed-case search that matches an adopted Request title.	Filter the admitted cohort with the sele
Alice remotely Creates R, Bob has not pressed Refresh, and Bob types R's exact title.	Do not reveal R through search; retain
The current server search route performs a fresh Request collection query.	Count it as a new acquisition under the
A search term matches a private User email not disclosed to Bob.	Never reveal the match via result inclu
Search text changes from 'hel' to 'help' while the first HTMX request is still running.	Discard the earlier response on arrival;
Empty search on a complete admitted cohort.	Show the whole permitted current coh
Empty search on an incomplete/unknown cohort.	Do not claim no Requests exist or coun
Newest, then oldest are selected over Requests with distinct valid creation timestamps.	Display the exact inverse temporal ord
Both mine-first and unclaimed-first are enabled.	Mine is the primary category, unclaime
A viewer ID is missing while requestor/assignment IDs are absent.	Do not classify absent identifiers as 'mi
A terminal Request has no active primary assignment.	Do not classify it as open/unclaimed; h
A creation timestamp is missing or invalid.	Report a source/projection error; never
The user changes options in the dialog and presses Cancel or Escape.	Applied search/order/terminal options
The user confirms a valid draft while another remote advisory remains pending.	Apply options locally without acquirin
The current Refresh completes after the user has typed another search term.	Reapply the latest compatible local sea
A details accordion is open when a same-Request valid fragment replacement occurs.	Preserve compatible selected/open stat
A Request disappears from the allowed scope while its details were expanded.	Retire old card and effects, choose safe
The user opens details of a currently adopted Request.	Reveal only existing permitted projecti
The user explicitly selects 'Get current details' for an older card.	Authorize and adopt a scoped read of t
Repeated duplicate advisories arrive while the page is focused.	Provide bounded, accessible updates w
An unresolved Claim response yields unknown outcome.	Show comprehensible next action and
HumanHelp has not selected the response-flow contract but Claim happens to work.	Report response-flow product acceptan
The product explicitly selects 'respond by claiming' and documents the complete user experience.	Exercise its actual labeled real-browser
The product instead selects a separate authorized contact interaction.	Require its selected privacy, effect/deliv

The rich-board gate requires positive and adversarial interaction coverage, not a permanently hidden controls panel or a test that checks only the selected option labels. Section 08.08 owns authentication, read authority and Live reconnect; section 08.09 owns command-specific optimistic recovery. Parts VI and VII own physical continuity and managed output.

Authentication, read authority, and context changes

The example uses genuine authenticated sessions. Development persona selection, if available, is a declared development authentication facility excluded from ordinary deployment or protected by the intended development-only access policy. A browser cannot become another user by selecting a User UUID in a command.

Every authoritative read, command invocation, outcome lookup, target-helper query, and subscription registration performs its applicable trusted access checks. Being the fixed demonstration Location is a selection rule, not read authorization. Previously adopted Request IDs and signed/opaque projection handles do not exempt a subsequent disclosure from current policy.

Authority context includes principal and the relevant authorization domain and profile binding. Mutable Membership and Organization facts still require current checks at their actual boundaries; a context identifier does not freeze them as valid for an entire page lifetime.

Logout or a switch of principal/Organization retires incompatible board owners, command presentation rights, subscriptions, and outstanding installations. An old response cannot disclose one user's content into another user's page. The command recovery service retains only the independently authorized records needed for later recovery; it cannot disclose them to the newly active principal merely because the same browser is used.

Read and command errors use the application's explicit non-disclosure policy. Tests must not require a reason such as “Request exists in another Organization” when the correct response intentionally hides that distinction.

Trusted session provenance and actor consistency

Authenticated principal evidence includes the configured login issuer, protected cookie, expiry/revocation, middleware order and current production User. All request-context actor slots and connected-client recipient identities derive from trusted middleware or are cross-checked against the session. Conflicting slots must fail unless a separate authenticated impersonation capability was explicitly admitted. Development persona fixtures are restricted to their declared test profile. UUID shape, first nonnil context value and a browser role name are never authentication proofs. Current middleware and slot precedence remain documented in the 08.08 audit.

Gesso's trusted operation role may represent Request authority, and one physical `:helper` browser participant may display requestor/helper/manager controls; neither changes the **authenticated** principal. Production Request, Organization, Location and Membership APIs determine per-Request read, modification and selected-target permissions from the current subject/actor context. The fixed example Location and app-wide notification scope are not evidence of tenant-wide disclosure rights. An unauthorized existence test, recovery lookup, details read, search index or advisory recipient must follow the same explicit non-disclosure contract as the actual Request projection.

Public HTTP and browser boundaries

The application validates command and acquisition input before use, including interface versions, allowed fields, limits, domain identifiers, and protocol encoding. Authentication, CSRF protection for applicable session-based mutations, and safe content rendering remain required host obligations. A valid semantic operation ID does not replace those protections.

The selected command wire schema rejects undeclared actor/principal authority fields. A legitimate target-helper field is separately declared and policy checked. A tampering test must assert that exact typed rejection or the exact permitted domain outcome of the declared field; “anything below HTTP 500” is not an acceptable oracle.

User-controlled title, details, and location text are escaped through the selected rendering contract. Managed HTML must not execute undeclared scripts, install foreign operation metadata, or cause a response from one target to issue arbitrary out-of-band effects. The qualified browser profile states the permitted headers, redirects, events, scripts, and swap forms and rejects unsupported behavior before its protected effect.

Identity, CSRF, decoding and recipient-safe output as joined gates

The current HumanHelp site middleware wires session, CSRF, parameter decoding and application routing, while example routes apply `wrap-signed-in`. The finished source/physical claim checks that their **actual installed order** covers every managed POST, HTMX request, fragment, stream, pending OOB drain, command-result lookup and reconnect entrypoint as applicable. CSRF validation is a session-bound mutation defense; it cannot authenticate an actor named in a JSON/EDN or browser command, nor is it equivalent to read authorization for an SSE stream or target-helper query.

Decode the wire once according to the admitted protocol/schema with limits on nesting, size, keys, operation identity, method/path, Request selector, helper/target, projection and content. Reject undeclared actor/principal fields rather than treating them as a second identity source. An authorized manager's chosen target helper is a domain argument, not an alternate principal. The same authenticated actor must own the server command, any durable record binding, outcome disclosure and permitted direct

acquisition. Different authorized actors may independently receive the **same current Request state** without obtaining another person's historical command record.

Physical OOB payloads and stream messages are rendered under recipient-specific current disclosure rights. An app-wide send target does not authorize embedding private Request contents, other Users' contacts, numeric new-Request counts, a copied CSRF token or executable operation authority. A receiver-rendered pending fragment also revalidates authenticated recipient, current connection/ client ownership, projection generation and target scope at its **drain** boundary; a client ID or SSE URL parameter is only an opaque selector. The host must not accept another user's stale pending mailbox as a source of executable OOB markup or identity. An expired, logged-out or switched participant retires both stream and queued presentation rights.

Live publication, advisory routing, and recovery

Production model incorporation is the source of semantic changes. The example routes them; it does not synthesize an independent success event from a toast or HTTP status. Relevant RequestAssignment, Membership, Organization, and other dependency changes need appropriate routing even when the Request document itself did not change.

Live topics express observation interests. They are not a substitute for the transaction's guarded dependency set. Likewise, overlap between topics does not prove that two authoritative commands conflict or that all conflicting writers have been found.

The production change graph and the missing writers question

Production Request transitions issue source-owned semantic change intent under the selected publication profile. Managed Live invalidation cannot convert a remote lifecycle advisory into an ungranted current-collection reread. Relevant RequestAssignment, Organization, Location, Membership and user-display/disclosure dependencies belong in the source-owned interest/writer graph.

Full application interest completeness is not proved by routing only `:request` document changes. Active RequestAssignments, Organization operational status, Location selection, Membership eligibility and User display/disclosure can change which Requests belong to an admitted cohort or whether an actor may see a card or receive a notification **without** mutating that Request document. The source-owned change/interest inventory must include every relevant production writer and derived dependency, or make its unsupported physical/writer scope explicit. An extra notification event is not a transaction guard, and a version guard is not a substitute for a delivered notification.

An interest subscription records its permission and binding to the actual viewer, scope, publication generation and physical client resource. A global ‘[:app/all]‘ connected scope can be useful for a one-Location proving fixture; it cannot by itself prove that every connected User is authorized to learn the same future multi-tenant Request existence facts. Sending a generic non-identifying “updates available” cue can have a different disclosure contract from sending which Request, helper or Location changed.

Durable publication strategy

The primary application profile records publication intent atomically with the command's authoritative transition and result. A durable worker publishes that intent and records its progress under a declared at-least-once policy. Its work identity remains stable across retry; receivers tolerate duplicate advisories.

A crash after commit and before publication therefore leaves recoverable work. The worker's authority, ownership/lease, restart behavior, retry limits and backoff, dead-letter/escalation state, and retention are explicit profile contracts. A retryable delivery failure cannot create another domain Claim. If the worker is unavailable, the UI can still recover current state through an authorized Refresh under the declared liveness assumptions.

A deployment that instead relies on periodic reconciliation or user-initiated Refresh must declare that weaker publication recovery policy and test it. Merely registering a post-commit callback does not establish recovery after process loss. The application must not claim durable delivery if its selected backend/profile cannot atomically retain the required intent.

Exactly what must be atomic and what may repeat

The **selected strong deployment profile** records the command's authoritative incorporation, its stable result identity and the required publication intent in a physically qualified atomic transaction. A separate outbox/work document is not a competing Request truth store: it records the intent to notify interests, not an independently mutable Request status. A worker may publish the same intent more than once under at-least-once delivery, but may not execute another Claim, assign a different helper or manufacture a new command ID while doing so.

Work identity, lease, ownership transfer, attempted destinations, progress, bounded retry, escalation and retention are defined for crash points both before and after advisory send/acknowledgment. A “sent” flag can be evidence of handoff to a transport without proving reception, participant permission, qualified authoritative read or DOM adoption. A replay may re-deliver already observed advisory requirements; idempotent merge is safe only where requirement identity and scope genuinely match.

The strong publication profile requires real transaction-coupled intent and qualified recovery, not merely a postcommit callback. Present Gesso model-notify and HumanHelp best-effort Create fanout behavior is separately recorded in the 08.08 audit.

Some deployments may instead select a declared weaker reconciliation profile. Its pending interest gap, periodic or user-requested reacquisition, liveness conditions and availability/reporting are explicit. A weaker profile cannot claim exactly-once deliveries, unbounded retention or guaranteed prompt notification simply because periodic reads can eventually discover current state.

Publication evidence and board adoption evidence remain separate. Successfully publishing an advisory does not mean a browser received it; receiving it does not mean the browser acquired or installed current authority.

Advisory-only streams

The HumanHelp board subscription turns ordinary relevant events into pending obligations and product feedback. It does not wire every event directly to the adapter's automatic fragment-refresh path. The explicit-grant policy in this part must be enforced before a physical acquisition is issued.

Advisory content is minimal and authorized. It may carry an opaque requirement or a permitted scope reference. It must not disclose private Request content, unauthorized identities, or exact counts merely because it is not used as canonical domain content.

Repeated delivery of an obligation is idempotent where the provider establishes the same obligation. Distinct scopes or incompatible requirements are not merged by choosing a larger-looking token. When compact representation is unavailable, the coordinator uses the declared bounded debt/backpressure strategy and retains honest "updates/continuity unknown" state rather than dropping information.

Separating connected-client OOB from authoritative interest

`client-plumbing` supplies a user-scoped plus app-wide connected-browser adapter to `gesso.live.client`. Gesso's client mailboxes are bounded ephemeral presentation queues, including disconnected client limits/overflow. Their wakeups and request-for-pending operation are distinct from the production Request interest graph, qualified replay cursor, actual XTDB read and client adoption.

An OOB advisory targeted to another browser is recipient-sensitive; a callback that can render on drain from receiver context must be admitted **again** under that context. A disconnected client with an old opaque client ID cannot use a later session to drain another User's pending material, even when the original SSE listener was previously accepted. A send attempt, queue entry and message drained through an authenticated HTTP route are three different observable events, none of which proves the participant installed the advisory target.

The HumanHelp product's selected 08.06 policy routes all remote lifecycle changes to pending advisory/debt rather than automatic board rereads. Duplicate wakes cannot grant additional one-use Refresh acquisitions. An authorized initial or command-recovery read has a separate provenance; it must not be incorrectly counted as a user Refresh.

Initial loading and reconnection

The application establishes a baseline observation and subscription continuity through a qualified read/subscription fence, replay cursor, or equivalent declared provider strategy. The initial-page acquisition grant permits the baseline installation; it is not a standing grant for later remote updates.

If the interval between baseline read and subscription registration is not covered, the board must show unestablished continuity and require the declared repair. Seeing zero events cannot prove that no event was missed.

After stream loss, server restart, page suspension, or subscription replacement, the application marks the gap even when no new event arrives. Reconnection may restore authorized subscription delivery or replay advisory requirements. Under the selected board policy it does not silently adopt remote domain changes. An explicit Refresh can establish current-state coverage for the board; a fresh subscription/fence is also needed for the claimed ongoing continuity.

A replayable stream ID and a database progression token are distinct unless the provider contract explicitly relates them. A current-state view need not replay every intermediate event if qualified current observation suffices. A workflow that promises processing every event needs a different durable contract.

Stale events from a retired subscription are rejected by its resource-instance binding. A shared connection can serve multiple admitted interests, but releasing one card/board interest revokes that interest's rights without accidentally closing resources still owned by others.

Subscription fences, disconnect races and honest liveness

At the initial `/app` load, a source-owned interest definition establishes which authorized Request and dependency changes matter. Its first authoritative cohort read and stream registration must close the interval during which a relevant transaction might commit but be known to neither baseline nor replay delivery. Valid approaches include a provider-qualified subscribe-before-read fence, retained log/cursor with replay, or another controlled acquisition with evidence of equivalent coverage.

Gesso's SSE stream, client mailbox and XTDB basis are independent mechanisms: a reconnect token cannot be assumed equivalent to a transaction log position. A router that learns an interest **after** processing a change cannot infer that no missed invalidation exists from current in-memory subscriptions. Multi-node failover must re-establish interest/coverage and re-send or reacquire sufficient requirements under the same single-node semantic contract. Extra duplicate invalidation is harmless only when

recipients deduplicate and every read still has its proper permission. A permanently offline browser cannot be promised bounded visible convergence.

After an observed disconnect, mark continuity unsettled even when no event arrives. Reconnect may restore stream delivery and source- qualified advisory requirements without adopting remote Request content. Any fresh domain reread or full board reconstruction needs its own declared baseline/repair or user-Refresh grant and must report which rights and local presentation survive. A partial authorized read may resolve an individual command's affected card without proving complete cohort coverage. If none of the supported repair paths can close the gap, display a typed stale/unknown state rather than quietly marking the board current.

The actual HumanHelp ‘live$_{test}$.clj‘ verifies production scope, invalidation expansion, model progression forwarding and query delegation. It does not establish a crash-tested durable outbox, two-client subscription gap closure or cross-router repair. Those remain separate product and distributed assurance gates.

Positive and adversarial trust, publication and recovery histories

The complete proving application must pass both eligible authentic use and controlled hostile/ fault cases. These histories use actual selected routes, source-owned Request model and browser subjects; stubs establish only their explicitly bounded evidence.

History	Required boundar
Alice logs in through the selected production session and loads `/app`.	The session issue
An anonymous visitor fetches `/app`, Request fragments, stream or pending OOB endpoints.	Deny protected a
A test supplies a signed-in session for Alice but an untrusted `:current-user/id` for Bob.	Detect incompatib
Alice changes the `:user/id` or principal in the browser command envelope to Bob.	Trusted session re
Alice selects another eligible helper for an authorized manager Reassign.	Treat selected hel
Alice has a valid signed-in session but current Membership does not permit R17 details.	Reject/redact the
Two browsers have separate signed-in sessions for Alice and Bob.	Distinct principal
Bob changes Organization or logs out while an old authenticated fragment read is in flight.	Retire old owner/
A valid but copied client-ID or pending OOB URL is presented under another principal.	Verify current rec
A cross-tenant client is included in the broad `[:app/all]` notification scope.	Do not disclose u
A forged HTMX lifecycle POST carries syntactically valid Choreo metadata but lacks CSRF/session authentication.	Reject at the actu
A malformed encoded command, incompatible operation path or extra actor field is submitted.	Enforce declared
A signed-in user stores untrusted markup in Request details.	Escape or sanitize
A RequestAssignment or Membership change affects permitted board membership without a Request document write.	Source-owned de
Create commits and process crashes before best-effort advisory runs.	Persisted Request
Outbox worker sends an advisory then crashes before checkpointing success.	Stable work ID al
A durable notification record reports published while the browser is offline.	Do not infer recei
Multiple equivalent advisories reach Bob and unrelated newer obligations arrive between them.	Coalesce only ge
Alice's current read authority is revoked before her old pending OOB is drained.	Rerender/refuse u
An SSE connection drops between initial board read and interest registration while another user Creates R.	Catch the gap wit
Bob reconnects after missing one or more remote lifecycle updates.	Restore authorize
A router fails after learning Bob's interest but before a peer sees it.	Recover/replay in
A late invalidation from a retired session arrives after switching to another User.	Reject the old ow
A valid command settles but Bob can see only its newly affected card under an authorized recovery read.	Keep complete-bo
The response-flow product decision is still open although all Live tests pass.	Keep full Human

Gesso's generic client transport and HumanHelp's current Live/source integration can satisfy narrower positive checks without claiming unbounded eventual-delivery, a complete tenant-aware interest graph, installed durable outbox, or fully proven multi-node replay. The finished design requires those guarantees where selected, with bounded and explicit availability assumptions.

Optimistic presentation and command recovery

HumanHelp's Claim, Unclaim, Mark on the way, Complete, Cancel, and Reassign remain source-owned production Request operations whether their presentation is optimistic, explicitly non-optimistic, or unavailable. A provisional card is participant knowledge, never an authorization decision, transaction witness, command verdict, or new authoritative Request fact. The production model owns its guarded transition; Gesso owns portable command and execution semantics and physical effect admission; the assembled HumanHelp application owns available user-facing presentations and recovery paths.

Prepared operation, authority, and provisional basis

Each enabled managed lifecycle control requires its real semantic operation, typed arguments, Request identity, applicable plan/artifact, qualified observed basis, physical target/transport, and matching trusted server route. When its provisional trajectory cannot be prepared, the application selects only an explicitly admitted non-optimistic realization, a disabled/unavailable control with acquisition/explanation, or checked rendering failure. Missing optimistic basis never causes an anonymous ordinary `hx-post` to the same protected route or erases the operation identity.

An available Claim control is not authoritative permission. The trusted executor authenticates the current principal and rechecks production Request/RequestAssignment, Membership, Organization and other relevant guards. A browser-provided command, target ID, CSS class, authorization-looking card, wall-clock timestamp or document revision cannot fabricate a qualified XTDB observation or grant authority. An observed basis is scoped to what was actually established; it does not imply a universal order of all facts.

Distinct command, execution, transport and physical ownership

One user intention has stable semantic command identity C42 and exact binding to its operation, arguments, subject and trusted principal. Local execution E42, HTMX attempt H1, target instance T1, provisional generation G1, callback, timer, listener, and recovery resource are distinct identities. Compatible retransmission H2 of C42 must not silently become a new C43. A genuinely new user intention may have a new command ID only through its own admission, not merely because the previous request timed out.

The browser admits semantic state before issuing effects. The physical shell registers each provisional installation, timer, network continuation, rollback handle and callback to its current owning execution, target instance and generation. It checks effect admission before actuation, not merely at promise completion. HTMX event and settlement correlation uses the actual issued execution/transport, not changing DOM position or a copied command string. Retirement removes the obsolete effect's authority even if the original HTTP request subsequently completes.

A decoded response, HTTP 200, generic `afterSwap`, OOB update, or completed provisional DOM installation is not command-specific authoritative incorporation. Real target-specific physical completion establishes that an effect was installed; its provisional nature persists until qualified authoritative adoption or safe compensating rollback.

Pending state, rollback and supersession

Pending transport, awaiting authoritative settlement, installed provisional presentation, command-outcome unknown, known rejection, historical incorporation, and adopted current Request truth are distinct statuses. While an execution owns pending presentation, the card exposes an accessible nonterminal pending indicator and guards against duplicate user submission. It does not claim terminal success before its promised authoritative confirmation. Semantic retirement removes the old spinner's ownership; an independent visible unknown-command recovery affordance may persist.

A rollback may restore only the exact legitimate pre-provisional structural baseline while its semantic right, physical target identity, installation generation and continuity ownership remain current. It may not replace a newer authorized card, undo another successful action, reintroduce a retired provisional owner, or restore a Request no longer disclosed to the viewer. Restoring the provisional DOM of E1 as if it were canonical after E2 fails is forbidden. A snapshot of such DOM may be used solely to compensate the immediately failed physical installation, not as a terminal semantic rollback baseline.

If rollback is unsafe, missing, or superseded, the runtime preserves honest provisional/uncertain status and requests permitted authoritative acquisition or exposes a repair action. It cannot silently strip a provisional marker to make physically orphaned DOM appear authoritative. A later justified installation of T2 revokes the old T1/G1 restore rights; a late E1 failure cannot overwrite T2. A stale callback can be ignored or diagnosed without falsely claiming the old semantic command was rejected.

Independent operations may overlap under an application-declared commutativity/dependency policy. Dependent operations are ordered, deferred, or rejected under their admitted semantic plan. Retirement or supersession of one presentation owner does not automatically settle the historical command of another. Local card adoption also does not erase unrelated remote advisory evidence or one-use Refresh debt under the HumanHelp selective-adoption policy.

Command-specific outcome versus current Request truth

The trusted server distinguishes what happened to C42 from what is currently true of R17. A selected durable-command profile records the exact command binding and admitted historical outcome under its declared transaction, unique-ID, consistency and retention requirements, coupled atomically with the domain transition when the profile promises atomic incorporation. Lookup and replay require authenticated identity, current permission to disclose the record, exact binding comparison, and a qualified authoritative read. No browser command ID, protocol shape, or archived receipt grants that permission by itself.

The recovery decision distinguishes:

- disclosed, binding-exact historical incorporation and its recorded outcome;
- properly supported authoritative rejection and its actual provenance;
- incompatible or undisclosable binding/result;
- no matching record under a precisely qualified lookup/retention scope;
- still-unknown transport, transaction or observation outcome.

An absent record from a partial, lagging, unauthorized or expired lookup does not prove the command did not commit. Even a complete read is interpreted against record lifetime and any possible in-flight original submission. The client cannot convert an unknown C42 into a new C43 merely to retry it. Reissuing C42 requires the declared idempotent, exclusion-checked protocol and current admission; otherwise the participant reconciles or truthfully reports uncertainty. A duplicate exact C42 may return the historical authorized outcome without executing Claim again; a changed operation, principal, Request or arguments is incompatible.

A fresh permitted Request read can correct the visible R17 card even while C42's history remains unresolved. Conversely, a disclosed incorporated C42 does not say R17 is **still** assigned to the same helper: a later authorized operation may have changed it. Historical attribution and current card presentation therefore have separate acquisition and adoption obligations. A newer locally adopted card does not imply permission to adopt unrelated remote Request membership or dismiss outstanding Refresh debt.

Recovery across retirement, reload and evidence expiry

An uncertain command transfers to a declared recovery owner before its provisional physical owner is retired. The recovery contract states how a minimally sufficient command receipt or authenticated server-side recovery index survives navigation, reload, crash, compatible deployment and loss of the original target. No suspended old execution, callback or browser artifact resumes merely because C42 remains recoverable. The recovery owner is distinct from the original card and may continue after the card is filtered out or physically replaced.

Browser storage, if used, contains only limited, context-bound recovery references; it is not a second source of business truth or a credential. Sensitive command arguments, historical outcome and even tenant-scoped Request existence remain subject to current trusted disclosure checks. Logout, principal change, policy revocation or missing local storage does not imply rejection. Where the profile cannot recover a lost receipt, it reports that limit rather than inventing a historical answer.

Automatic retries/reconciliation obey a declared bound on attempts, time, work, rate and retention; after exhausting the budget the user receives an actionable, accessible recovery explanation. Beyond the supported record/discovery horizon the correct answer may be ”historical outcome no longer determinable”, not ”rejected”. Any promised eventual resolution identifies the durable storage, backend qualification, fairness, availability and disclosure premises on which its liveness depends.

Worked HumanHelp histories

```
Successful lost-response Claim:
  Helper A invokes rendered Claim(R17), constructing C42 / E42 / H1.
  The trusted model atomically commits Request + assignment and C42
  incorporation under the chosen durable-command profile.
  H1 response is lost; the browser shows outcome unknown.
  Exact disclosed reconciliation returns C42's historical result.
  A compatible H2 does not execute Claim a second time.
  Fresh authorized acquisition determines R17's current card.
```

```
Late rollback after a newer installation:
  E1 installs provisional R17 in target T1.
  An independent qualified authoritative card installs in T2.
  A delayed E1 network failure attempts to restore its snapshot.
  E1 owns neither T2 nor its generation: no old DOM is restored.
  Unresolved C42 attribution remains separately recoverable.
```

```
Current state cannot attribute historical command:
  C42 times out and another helper B claims R17.
  The browser reads "claimed by B" from current authority.
  Without command-specific evidence, it cannot infer whether
  C42 was rejected, once incorporated, or never delivered.
```

```
Nested provisional and forged retry:
  E1 leaves provisional DOM. E2 captures it during installation.
  E2 fails or its outcome becomes unknown.
  E1's provisional snapshot may compensate immediate install
  failure but cannot become E2's terminal canonical baseline.
  A fresh C43 presented as a retry of unresolved C42 cannot
  silently bypass exact-identity reconciliation.
```

HumanHelp acceptance exercises each required lifecycle operation through a genuinely rendered eligible control, the generated browser command, real HTMX transport, trusted selected route, production model outcome, physical settlement and fresh authorized acquisition. Top-level browser acceptance must not hand-inject the command wire envelope. Paired negative tests cover missing basis/plan, malformed or mismatched protocol, unknown delivery, duplicate/incompatible C42, permission revocation, late E1 callbacks, nested provisionals, reload, incompatible deployment and retention expiry. Source tests or a DOM-only success cannot replace a real browser/XTDB evidence path.

Routes, concrete rendering, assets, and host lifecycle

The HumanHelp proving application closes one installed application, not four mutually agreeing diagrams. Its real production Request semantics, trusted executor, Reitit/Biff routes, browser plan, generated assets, rendered controls, Live/acquisition paths and running Aleph handler must correspond to one admitted application generation. Part VII owns generic assembly and enforcement; HumanHelp supplies its actual production facts, real user capabilities and independent operational evidence.

One route authority, semantic protocol and writer inventory

`example.routes` owns route identity and method/path facts. The active Reitit router, `example.app` handler selection and the canonical `example.application-preflight` constructor consume those same facts. The example may use separate route IDs for Claim, Unclaim, Mark on the way, Complete, Cancel, Reassign, Create, board reads, fragment acquisition, Refresh and SSE, but must not maintain a shadow route catalog for tests or preflight. Normalized path parameters and effective HTTP method must select the exact declared operation, subject and trusted executor.

A protected lifecycle route accepts the required semantic protocol only when the decoded command, bound Request, declared operation and installed server realization agree. `POST /requests/R17/claim` cannot execute a Cancel command, apply to R18, select an arbitrary Clojure function from a URL, or treat a client-provided User ID as the trusted principal. An absent/malformed protocol, operation mismatch, unknown Request or incompatible command binding rejects before protected mutation. URL and form selectors are untrusted input, not authorities.

A semantic operation may be implemented by a trusted route before a particular user can see its eligible control. Supported operation, physically installed handler, user-facing affordance and current model permission are separate claims. HumanHelp's complete product contract requires eligible positive controls for all six lifecycle operations; the current renderer's inventory cannot define that requirement away.

Ordinary GETs, ordinary non-mutating HTMX, forms and deliberately admitted non-Choreo operations remain legal. A Create Request or other route that writes invariant-bearing state is nevertheless a real authoritative writer and belongs in operation/writer coverage, whether or not it uses optimistic Choreo. So do imports, fixtures, startup initialization, administrative work, scheduled jobs, provider callbacks and other physically reachable mutation paths. Absence of a declared writer is not proof that none can run. Each reachable writer is either covered by an explicit rule and interference/effect contract or reported as an uncovered or unknown boundary, never silently renamed read-only.

Live fragment, initial acquisition, advisory-stream and authoritative reread routes also share their actual route specifications with the corresponding source-owned interest, result and adoption contracts. The serving route may not quietly diverge from the assembled Live route or publish a transport-success label as authoritative model knowledge.

Actual rendered affordances and request coordinates

An admitted assembly is not sufficient if the actual emitted view selects a different request. The finished HumanHelp serving boundary checks each concrete enabled rendered occurrence at the last supported structured-response point before serialization, after component expansion and preparation. It checks semantic operation identity, required observed basis, browser plan/artifact compatibility, current binding, effective request method/path and route correspondence, and the permitted target/transport/effect shape. An inspected test fixture does not certify a different branch rendered for a real viewer.

The supported transport profile includes real HTMX `hx-get`, `hx-post`, `hx-put`, `hx-patch` and `hx-delete` requests, applicable native forms and submitters, explicit `action`=/=`method`, form owners, `formaction` and `formmethod` overrides, disabled controls, inherited attributes, OOB fragment destinations and semantic metadata needed by the plan. The final user-visible and physically executable request coordinates matter; a decorative parent button or omitted `type` must not hide a native submission path. A disabled-looking button retaining an active enabled request path is not accepted.

A concrete enabled Claim cannot omit its required command or observed basis, and it cannot downgrade into an anonymous ordinary HTMX POST to a reserved lifecycle route. If a target helper or current observation is unavailable, the renderer exposes the declared acquisition/input/unavailable state or rejects the invalid render. It must not invent authority or silently change the operation. An explicitly admitted ordinary mutation uses its own checked contract, not a false claim to semantic Choreo coverage.

The check is scoped to the actual supported render/transport language. String HTML bypassing structured rendering, user-provided raw markup, dynamic client-generated requests, opaque callbacks, unbounded JavaScript, and transport features not examined by the selected profile require an independently enforced boundary or explicit exclusion from a narrower claim. A scan of canonical Hiccup alone cannot certify all possible DOM or network behavior. The trust boundary additionally requires correct escaping, CSRF/session policy, permitted disclosure, and rejection of unauthorized script or request-capable markup; a passing route scan does not prove these.

Deferred, cached and multi-target response completion

Checking only the synchronous handler return is insufficient for deferred Hiccup evaluation, streaming HTML, callbacks, cache reads, alternate response helpers, error views, redirects and OOB payloads. Each path that can emit a managed affordance carries its admitted application and current output-check context to the physical serialization/serving boundary, or is explicitly disallowed by the qualified profile. A cached fragment is revalidated against the currently served assembly and recipient policy when required; a stored ”checked” marker from a different generation is not an unrestricted permit.

Out-of-band updates and multi-target installations have actual target identities and completion reports. A valid response before serialization does not prove every requested DOM effect occurred. Partial OOB completion retains its specific recovery obligations. A committed Request mutation is not reversed by a later failed render or incomplete browser installation; the participant must perform authorized result acquisition or truthful repair.

For ordinary routes returning legitimate raw HTML or other formats, HumanHelp declares the separate check/escape/serving contract. For protected managed routes the output boundary must be connected to the actual installed handler and cannot be bypassed by replacing the handler after preflight. Application preflight rejects a missing or incompatible enforcement connection where the selected deployment profile requires it.

Generated browser entrypoint and physical asset identity

The browser declaration and role-local plans come from the same admitted production Choreo as the server assembly. The supported build path generates a browser entrypoint embedding the exact admitted BrowserAssemblyManifest, compiles it with the selected ClojureScript/compiler configuration and physical realization options, and records a receipt for the resulting artifact bytes. Startup verifies the current manifest-to-receipt correspondence and the receipt-to-bytes correspondence before serving managed HTML. A matching filename, hand-copied stamp, or receipt for unrelated JavaScript is not evidence that the browser implements the admitted plans.

The build claim is conditional on its actual producer, compiler, linked runtime, dependencies and tested bootstrap contract. A hash does not prove arbitrary minified JavaScript semantics. The assembled publication inventory includes the script, its transitive required assets, styles and any separately supplied HTMX/SSE/browser runtime whose behavior the plan assumes. Their actual served URLs, bytes, cache headers, integrity/correspondence mechanism and client load order are qualified as part of the selected deployment, not inferred from their existence on disk.

A page admitted with generation A must load assets compatible with A throughout its supported lifetime, including after a rollout of B. Identical URLs cannot switch to incompatible bytes under an A page's feet. Use immutable versioned URLs and coherent publication sets, or a specified generation-pinning/reload/retirement protocol with an equivalent correctness argument. A CDN, filesystem, classpath resource and the active HTTP server may each expose different bytes; verifying a local build path alone does not verify what the client actually retrieved.

Generation transitions, reload and old clients

Reload and deployment first construct, validate and stage a coherent candidate B; activation switches the installed route graph, trusted executor, browser artifact, current assembly and Live/acquisition interfaces according to an explicit atomicity or compatibility contract. It may not serve A's routes with B's command plan and a stale A script as though those independent objects formed a valid new application. Old pages either continue against a supported compatible A boundary or receive a specified reload/repair response.

An already issued C42 can be recovered across a deployment only through a qualified compatible command/disclosure contract. Its retired local E42 execution, old target instance, pending listener, HTTP callback or generated browser plan does not resume merely because C42 remains historically meaningful. Background work, connected SSE subscriptions, one-use Refresh permissions and advisory debt are transferred, fenced, or retired under their separate ownership/recovery rules. A reconnect is not automatic authorization to adopt remote Request state.

A candidate that fails preflight never becomes an active half-B generation. A crash between staged assets and application activation must either leave a valid A available or enter the declared unavailable recovery state; it must not expose an accidental mixture while claiming successful service.

Qualified Biff, Aleph, HTMX and XTDB host contract

The finished application names an actual supported Biff release and native lifecycle semantics, Aleph serving integration, HTMX and SSE extension versions, browser/CLJS compiler and runtime, and XTDB adapter/configuration. ”Final Biff 2” or ”the current database” is not an executable compatibility contract. Changed versions, configuration or physical topology require requalification of the assumptions each dependent proof and integration gate uses.

At startup the trusted host initializes configuration, schema and model transaction providers, database connection and required FX capabilities, browser artifact, Live/advisory machinery and the canonical application assembly in the declared dependency order. It installs the checked actual Ring handler before Aleph admits traffic. The canonical Gesso application-handler module appears exactly once and at the required lifecycle position for a profile using that native startup path. The server must use that resulting handler rather than a different unwrapped Var or manually bypassed entrypoint. No unchecked normal production path remains enabled.

Biff explicit FX input/previous-result flow supplies time, transaction, authenticated principal and other required capabilities through trusted providers. One request's context is not silently merged into another's or replaced with browser-provided authority. Any startup seed or fixture write is an authoritative ingress with the same coverage obligations. Shutdown and startup failure release database, Live, thread pools, subscriptions and HTTP resources in the required order and preserve durable in-flight recovery promises.

Distinct authenticated clients must operate concurrently according to request- and execution-scoped ownership. A global one-use lease that serializes every HumanHelp write is not a valid substitute for per-command exclusion, per-Request conflict resolution or qualified XTDB atomicity. An intentionally narrow single-request test profile does not certify the real multi-client service.

Backend qualification follows #req-backend-assumption-qualification. HumanHelp must establish the required atomic guard/write and RequestAssignment uniqueness behavior, exact-binding command exclusion/observation, query completeness and authoritative absence where claimed, post-commit outcome and visibility guarantees, and the effect of concurrent writers. The evidence has precise version/configuration, source and physical execution scope; it is not a proof of all XTDB behavior. A changed backend assumption invalidates dependent certification until it is requalified.

Positive and adversarial installed-application histories

```
Positive — checked Claim through the actual service:
  A production Request operation defines Claim(R17).
  The installed Reitit route selects Claim and trusted Request R17.
  Real rendered Hiccup prepares a valid managed control for helper A.
  The generated current browser artifact forms C42 and HTMX H1.
  The trusted Biff/Aleph handler validates the exact command binding.
  Production Request/assignment guard and transaction commit.
  Qualified settlement and current acquisition install the result.
  A fresh reload sees the authoritative claimed Request.
```

```
Adversarial — valid startup label but wrong installed handler:
  Application preflight verifies an assembly and build receipt.
  An Aleph route starts with a separately wrapped or replaced handler.
  A protected response bypasses canonical render preflight.
  Admission fails for this deployment; a successful static fixture
  does not certify the physical server or its emitted controls.
```

```
Adversarial — stale A page after deploying B:
  The A page still references /js/gesso-live.js.
  B replaces that same URL with incompatible script bytes.
  The page invokes an A plan against B's route/asset context.
  Deployment must pin matching A assets or force qualified reload;
  it cannot silently treat the mixed execution as admitted.
```

```
Adversarial — deferred fragment and native form:
  A cached/deferred fragment materializes after the handler returns.
  A submitter inherits a POST action to the protected Claim route.
  The actual form lacks required semantic command binding.
  The final serving boundary rejects it rather than delivering a
  control that static render snapshots never inspected.
```

HumanHelp connected acceptance includes paired valid and defective production-assembly fixtures: missing handler/module, route operation or Request mismatch, absent plan, stale or forged build receipt, wrong generated bytes, omitted browser dependency, incorrect served asset URL, anonymous native form/HTMX collision, deferred unchecked HTML, rejected B startup, resource leak on failure, writer-inventory gap, stale A callbacks after B activation and simultaneous authorized clients. Each fixture identifies the first real enforcing boundary, asserts no prohibited effect or unauthorized disclosure, and preserves the independent positive Create/Claim/lifecycle/browser outcomes. This section describes the finished-system test obligation, not a claim that present

Gesso or HumanHelp already passes the full matrix.

Semantic model, proof obligations, and conformance evidence

The HumanHelp proving application must make its actual production Request rules precise enough for Gesso to assemble, evaluate, challenge and, for supported propositions, prove or check. This is not a second HumanHelp domain language maintained in `example.*` and not a requirement that an application author write TLA+, Quint, Recife or proof scripts. The source-owned Request model defines the business meaning; Gesso derives admissible semantic subjects and checking obligations from it and from explicit, typed dependency and effect contracts.

A correct model of the wrong policy is still a wrong application. The independent HumanHelp product contract therefore remains an external acceptance oracle for required operations, permissions, assignment semantics, user-visible behavior and adverse outcomes. Neither model generation nor executor agreement with that same source can prove the source expresses the intended HumanHelp rule. Conversely, a product test cannot by itself prove all legal traces or complete physical writer correspondence.

Production ownership and the checked semantic subject

The protected aggregate comprises Request and its separate RequestAssignment history, scoped to Organization and Location and dependent on User, Membership and Organization-owned authority. Request's public model boundary owns domain transitions and outcomes; its graph/FX implementations own the actual relevant dependency acquisition, guarded planning and authoritative transaction assembly. Request Choreo binds the **public** operations and their distributed presentations without reimplementing Claim, Unclaim, Mark on the way, Complete, Cancel or Reassign eligibility in the browser or example UI.

The Gesso semantic subject identifies the original production source and version, declared state and relationship types, operations, actor/target arguments, transition constructors, frames, observable outcomes, initial-state predicates, invariant propositions, external dependencies and explicitly opaque host effects. A projection of these declarations into a finite transition system is not a second editable operation registry. A changed source, effect footprint, model policy, schema or dependency invalidates the applicability of evidence derived from its former identity.

There are independent closure questions:

Question	Required evidence
Is the declared model well formed?	Typed, closed source and effect/role/transition construction for the selected la
Does the model capture intended HumanHelp behavior?	Independently supplied positive and negative product requirements, including
Does the abstract machine preserve its claims?	Exact finite exploration or a separately checked initialization/preservation der
Does issued Clojure execution correspond?	Original-source-to-executor binding, admitted input/frame/guard corresponde
Does actual XTDB/host execution refine that model?	Qualified transaction, readback, actual writer/guard and state-representation e
Does the running browser/app exhibit required behavior?	Installed route, generated artifact, real HTMX/DOM and authoritative outcom

No one row is silently inferred from another. Structural admission, finite model checking, conditional induction, runtime guard enforcement, observed physical agreement and end-to-end conformance are distinct judgments, not a numeric ladder whose higher rung automatically proves all lower ones.

HumanHelp invariant and obligation families

The admitted Request application declares at least these property families, with ownership, scope, assumptions and counterexample conditions visible:

Property family	Precise intended contract
Entity and relationship validity	The public Request, RequestAssignment, Organization/Location, Membership and User referen
Primary assignment uniqueness	For a Request there is **at most one** active primary RequestAssignment under the declared aggr
State/assignment correspondence	A Request in the relevant assigned states has the required active primary assignment; Unclaim,
Command eligibility	A new Claim/Reassign/other command passes the current trusted actor, Request state, Organiza
Guarded aggregate atomicity	Request and affected assignment writes, exclusion/absence checks and dependency versions eit
Transition/frame safety	Each operation changes only the admitted Request/assignment and declared associated fields/e
Command exclusion and historical result	Exact-binding replay cannot reincorporate; incompatible reuse cannot mutate or disclose anoth
Read, absence and disclosure	A read's scope and completeness, authoritative basis, access policy and interpretation of missin
Local and remote knowledge	Creator-local correlated adoption and remote advisory/one-use Refresh obey their distinct pern
Browser/physical ownership	Retired E42, target T1, subscriptions and provisional generations cannot actuate protected effec
Source and writer coverage	Every declared model writer and physically reachable protected writer is classified and either c

The **at most one** primary assignment rule does not by itself prove that every claimed Request has one; the latter is a separate existence and state-correspondence obligation. The existence/uniqueness of an active primary must also be distinguished from historic or collaborator assignments. An action's protected prestate may contain Request R17 and a complete guarded absence of active primary assignment; a snapshot of one returned assignment alone is not such an absence proof unless the query and exclusion contract establish completeness.

A helper's current Membership eligibility is normally a **commit-time precondition**, not automatically a permanent invariant. If Membership expires after a valid Claim, the policy must choose whether the old assignment remains historically valid, becomes operationally ineffective, or triggers a model-owned reassignment or revocation. The proof must not assert that every existing assignment is presently eligible while omitting Membership revocation, Organization policy changes, Location moves, User deactivation, administrative edits and external writers. Such operations either preserve the claimed invariant, perform a specified repair transition, or remain explicit premises of a properly narrower result. Live publication cannot repair a violated database invariant.

Closure, induction and cross-model interference

For an invariant I and the named admitted transition relation T, the ordinary abstract safety obligation has the shape:

```
Initial(s) ⇒ I(s)

I(s) and T(s, operation, s') ⇒ I(s')
  for every admitted operation and its required environment step
```

The initial condition includes more than ”the test database was seeded without errors”: it identifies permitted initial Request, assignment and dependency configurations, including legal empty states, and why real initialization/legacy import reaches them. An induction argument must show preservation by **every member** of the complete relevant transition catalog; checking Claim alone cannot certify Unclaim, Reassign, policy changes or a background assignment writer. For a nonclosed environment the claim remains conditional on an explicit interference assumption. A source-bound catalog report is not proof that no undeclared physical mutation path exists.

Guarded transition generation is required to expose actual read dependencies and protected write footprints. If an Organization policy or Membership predicate is implemented by an opaque Clojure function or backed by a query with an unproven exclusion guarantee, the abstract argument records that premise. It must not replace the function with a hand-authored conveniently matching predicate and describe the agreement as implementation correspondence.

Application-defined transition preservation includes concurrent interference: two helpers competing for the same R17; manager Reassign racing Membership revocation; Complete/Cancel racing an assignment change; initialization/import racing ordinary claims; and non-Choreo mutations affecting the same protected relation. The chosen qualified XTDB guard/transaction semantics must exclude forbidden joint outcomes at the physical boundary. A proof of serial atomic **reference** steps does not establish that the deployed backend executes only those steps.

A conditional inductive result identifies its original source, proposition, initial-state coverage, per-operation proof dependencies, explicit environmental assumptions and independent checking mechanism. A report copied from another assembly, a digest with matching text, or an in-process admission handle alone does not discharge the higher-level physical/assumption obligations.

Finite state families and non-vacuous exploration

Finite exploration exercises a deliberately bounded subset of the HumanHelp state and command space. Required example families include:

- two eligible helpers contending for one open Request and a third authorized actor attempting a later Reassign;
- two distinct Organizations/Locations, permitted and forbidden cross-scope reads, and incompatible actor/target-helper identities;
- active, expired, absent and role/skill-ineligible Membership, including changed policy between preparation and submission;
- Request states `:open`, `:claimed`, `:on-the-way`, `:done` and `:cancelled` with active, ended and historic primary and collaborator assignments;
- Claim, Unclaim, Mark on the way, Complete, Cancel, Reassign and Create, including invalid attempts and valid eligible positives;
- unique C42/C43 commands, compatible duplicate C42, incompatible command rebinding, lost settlement and post-commit uncertainty;
- local Create S after remote unadopted R, with local selective adoption and outstanding Refresh debt;
- source and host changes that alter dependencies, writer sets, application generation and read/disclosure permission.

Not all families must form one giant Cartesian product. Small independently checked families are legitimate when each obligation identifies its own state universe, command/actor/argument domains, initializers, number of attempts, bounds, checker configuration, transition-preparation strategy, coverage and omitted interference. If a stronger joined claim requires interaction between families, that composition itself needs a checked or explicitly assumed argument. A prepared-plan permutation explores precomputed choices against a selected initial state; it is not equivalent to rebuilding every operation after each successor state. A state-dependent reachable checker must cover changing enablement where the obligation relies on it.

Non-vacuity is an admission requirement. For each required independent capability there is an actual eligible source state, a successful attempt, a meaningful denied/current-policy attempt and the applicable retry/concurrency branch under the selected scope. A universally true invariant over a model in which no Request can be created or no Claim is ever enabled does not meet the HumanHelp capability contract. A source rule never matched by any selected state is reported unexercised, not silently checked. Removing Reassign from both the production declaration and the derived formal subject still fails the independently specified HumanHelp positive capability gate.

Where finite search finds a counterexample, the report exposes the smallest available typed, reproducible operation/state trace under the selected model and its assumptions, without disclosing protected tenant facts to a browser. Finiteness and lack of a counterexample mean only ”none in this checked subject”; they do not establish an unbounded theorem, a sound compiler, or an XTDB/HTMX physical result.

Host functions, proof premises and policy adequacy

Supported pure Clojure rules, typed operations and finite data constructors may be interpreted or translated for model checking, provided the transformation's semantic-preservation argument is visible. Opaque predicates, callbacks, time providers, database queries, serializers and side effects participate only through declared contracts. Sample tests of a function and its printable name do not prove its behavior for all admissible states.

Host/FX capabilities are explicit inputs, not hidden powers. Current authenticated principal, current time, transaction identity, random/seed generation, external provider results and authoritative observation are separate premises. Trusted runtime validation can establish concrete input/output schemas, exact command binding, observed version, permitted read or verified effect call at a named boundary; it cannot prove the function unconditionally pure, the backend fully serializable or the external world compliant. If the selected assurance policy requires a stronger contract than the chosen host can supply, the protected operation is not certified/admitted under that policy; it is not granted a fictitious weaker proof.

Independent policy adequacy is illustrated by Reassign: a generated transition and its executor may agree while omitting the essential authorized target-helper selection, or while misinterpreting collaborator history as primary ownership. The separate product oracle must reject that deficient design. A model with only the presently rendered Claim action also fails independent Unclaim, Mark on the way, Complete, Cancel and Reassign requirements even if the reduced machine is internally consistent.

Actual execution and refinement evidence

An abstract successor and a trusted XTDB transition are different objects. Source-to-physical correspondence requires a declared representation relation between abstract Request/assignment/ dependency state and actual stored documents, plus evidence for the guard/prestate seen at the **transaction boundary**. A read taken before submission and a post-commit document read are not automatically the transaction's internal prestate and successor. Command-record readback, transaction ID, externally supplied status and displayed outcome must retain their precise provenance and trust scopes.

A meaningful observed-step report correlates at least:

- the admitted source, operation, typed arguments and exact issued executor/route/actor binding;
- the selected initial/abstract state and qualified physical prestate, including relevant checked absences and dependency versions;
- the actual submitted guard/write footprint and backend transaction identity or a clearly delimited observation gap;
- qualified incorporation, rejection or unresolved submission outcome with separately disclosed command-specific evidence;
- the relevant complete/partial poststate read, basis and observed representation relation;
- the actual independently observed physical writer and installed host/asset generation where the claimed scope includes them.

These are not automatically one adjacent execution history. A caller may supply a sequence of individually matching states from unrelated XTDB snapshots. The checker must establish the needed chronology, exclusivity/interference frame, correlated transition lineage and observation completeness before claiming a particular path refinement. A successful read of a complete canonical command record following one submitted write is useful bounded co-observation; it does not, without stronger origin proof, establish the identity of every writer or that this command caused the observed state.

Protected physical writer coverage includes real HTTP handlers, public model calls, raw XTDB access, initialization, repair, imports, background work, external processes and privileged administrative paths. A host-observed list of registered writers is evidence about that **enumerator** and its inspected runtime; it is not by itself evidence that the enumerator saw every process, that all write capabilities are restricted, or that the deployed router uses the checked handler. Unknown inventory is an open coverage obligation, never an empty set by default.

A qualified backend can establish particular guard/write, unique-primary, command-exclusion, query/absence, progression and visibility properties for its named XTDB version and configuration. Version or configuration changes reopen affected obligations. This is not a claim that one integration test proves all physical XTDB executions or every database behavior.

Assurance report and joined acceptance

The HumanHelp assurance explanation reports each precise proposition with its subject, source identities, checked transition families, observed operation coverage, writer inventory scope, evidence mechanism, proof dependencies, versioned physical assumptions, untested cases, applicability and status. It separates a verified finite statement, a conditional abstract theorem, an enforced runtime guard, a bounded concrete observation, an opaque host assertion and an independently exercised product requirement. An unadmitted candidate, copyable ledger or structural handle is never itself a release certificate or grant of transaction authority.

```
Positive — competing Claim under a qualified guard:
  A and B are eligible for open R17 with no primary helper.
  Both submit through the real guarded production operation.
  The finite semantic subject forbids two new active primaries.
  A qualified backend test forces an actual conflicting
  guarded submission; at most one is incorporated.
  The other receives its actual supported outcome.
  Fresh Request/assignment reads agree with that result.
```

```
Adversarial — missing Membership revocation writer:
  The abstract Claim-only catalog preserves a predicate
  that every active primary helper is currently eligible.
  Membership is revoked through a real administrative path.
  The property no longer follows. The plan must cover that
  writer and repair transition, narrow the proposition or
  retain an explicit external assumption; "checked" is not
  a valid full-application answer.
```

```
Adversarial — false physical trace stitching:
  A test reads prestate P, executes another actor's command,
  then supplies a later unrelated snapshot Q and a copied
  C42 result label to a comparison checker.
  Agreement of isolated values is not transaction causality
  or a contiguous trace without the required lineage.
```

```
Adversarial — vacuous operation catalog:
  An implementation deletes Reassign and its formal plan.
  Every remaining checked transition preserves uniqueness.
  The independent product oracle still requires an eligible
  Reassign and rejects the deficient application.
```

Concrete HumanHelp acceptance includes (1) positive Request creation and each required lifecycle operation through public model semantics, (2) invalid actor/target and competing-state rejections, (3) relevant Membership/Organization/Location and other writer interference, (4) compatible/incompatible command replay and truthful unknown delivery, (5) qualified real XTDB guard/assignment/readback evidence, (6) application and browser installation/continuity/advisory evidence, and (7) explicit assumption and coverage-failure rejection cases. Part VIII's following test architecture states how these are exercised. Part IX owns compositional certificate policy, Part X the stronger proof-kernel/refinement and physical correspondence arguments. Their open work is not preemptively declared done by this sectional review.

Test architecture and oracles

HumanHelp is a proving application only if its tests exercise the real production Request models and the actual Gesso/HTMX/Biff/XTDB realization. The test architecture is a collection of independently meaningful judgments, not one green build, a second hand-authored application, or an unqualified claim that a rendered page proves a database invariant. The accepted requirements describe the finished Gesso/HumanHelp system without treating a narrower implemented test subset as complete evidence.

Independent test layers and evidence owners

Layer	What it establishes
Production domain and model	Request/RequestAssignment schemas, current actors and targets, Organization/Membersh
Projected choreography and semantic machine	Typed role-local plans, knowledge/continuations, provisional/status ownership, finite or i
Qualified XTDB/FX integration	Actual guarded Request/assignment transaction, relevant dependency/absence assertions
Application assembly and rendered output	Real source-owned operation/route/Live/browser closure; concrete effective HTMX/nativ
Physical adapter and browser conformance	Real HTMX/SSE/DOM event behavior, effect ownership, continuity, output/swap and fail
Installed host and publication	Actual Biff/Aleph startup, checked Ring handler, current asset bytes and dependency/pro
Real-browser product acceptance	A genuine logged-in user can complete required HumanHelp tasks through rendered con

A useful test names the exact operation, source/application generation, actor and target, provider/version assumptions, reachable starting state, injected fault if any, real enforcement boundary, authoritative postcondition and expected diagnostics. Results retain which parts were genuinely observed, constructed, assumed or untested. The same test should not use a private mock to produce both the behavior under test and the supposedly independent oracle.

Valid fixtures and independent product requirements

Normal fixtures establish real public User, Organization, Location, Membership, Request and RequestAssignment facts through the selected public production model and admitted initialization path. They bind distinct actor, requestor, helper, candidate target and manager identities to genuine authentication/session contexts; generated UUIDs, a crafted `:session :uid` map, or browser identity fields alone do not demonstrate authentication. Fixtures preserve actual progression/transaction receipts and test each authorized read at its intended scope. Deliberately malformed database state may be introduced only for an explicitly labeled corruption/rejection test; it must not become the conveniently seeded positive case.

The independent product oracle requires Create, Claim, Unclaim, Mark on the way, Complete, Cancel and Reassign, with a genuinely eligible positive for each and meaningful negative/changed-policy cases. Search, board sorting and terminal options, remote advisory-only presentation, one-use explicit Refresh, selective creator-local adoption, details/continuity, reconnect and the independently selected response-flow behavior are separate requirements. Removing a required capability from both the production catalog and its generated tests does not make the product complete. Required positives must not be satisfied only by constructing a disabled action or by a mock executor that always reports rejection.

A test can use isolated public-model seams to investigate a narrow Request aggregate, provided it calls the result a **seam-qualified** test and separately tests real Organization/Membership persistence, concurrency and disclosure at the relevant connected boundary. One invalid foreign-model stub cannot be used to certify the entire authenticated HumanHelp application.

Browser-originated Claim and lifecycle acceptance

The installed product test launches the supported generated Gesso browser artifact, authenticates a real test principal, navigates to the actual served `/app` page and locates the real eligible Claim control. It clicks the control as the user would. The actual browser runtime, not test code, constructs its protocol-v3 command, local execution identity and HTMX request; the trusted route binds that command to Claim and the selected Request. Production Request model/FX commits the real guarded transition, the response carries truthful settlement/progression, and current authoritative acquisition and physical completion install the settled card. A fresh reload and independent model read establish persisted Request and primary assignment truth.

```
Claim positive oracle:
  authenticated helper A → served Claim(R17) control
  real browser → semantic command C42, transport H1
  actual checked HTTP route → public Request Claim
  qualified XTDB guards + Request/assignment writes
  authoritative outcome + permitted readback
  physically installed settled card → page reload
  public Request/RequestAssignment query agrees with the result
```

A server unit test may deliberately manufacture a well-formed wire command to test trusted decoding or Request routing; it cannot be counted as this browser-originated positive. Likewise page evaluation must not call the private lifecycle executor, inject `__gesso_live_optimistic_command`, swap a prebuilt success body, or silently replace the generated client with a test implementation that already knows the expected outcome. Observation-only browser instrumentation and controlled network faults are legitimate when they do not supply the missing action, settlement or authority.

Every exposed lifecycle operation repeats the appropriate installed control-to-model-to-physical-result path with the real operation identity and necessary user input. Reassign must include a legitimate target-helper choice and must preserve the production transition's actual Request state and assignment history; it is not another Claim with a different label. Unclaim returns to the model-defined open state and ends the selected primary participation; Mark on the way establishes `:on-the-way`; Complete and Cancel establish `:done` and `:cancelled` respectively with their specified assignment closure and terminal visibility. Each eligible positive is accompanied by appropriate policy denial, malformed-binding, stale-state and replay cases. A terminal Request cannot regain an enabled Claim merely because its display row has no active helper.

The oracle is not ”DOM contains Claimed” or ”HTTP was 200”: it checks the transaction-related Request/assignment facts from a qualified fresh read, exact current actor/target, the committed operation's semantic outcome, authorized user-visible settlement, absence of obsolete provisional presentation, and persistence after reload. A response-render error after a valid commit is not retroactive domain rejection. Unexpected page/console errors, failed HTTP requests, missing command, absent model mutation, unhandled asynchronous errors and mismatched artifacts fail the scenario rather than being swallowed by the green status.

Deterministic concurrency, causality and negative knowledge

Two Playwright clicks at nearly the same time are a useful user experience test, not proof that two XTDB transitions competed over the same protected prestate. The backend test uses a controlled barrier after both real contenders have acquired the relevant preparation/guard snapshot and before their submitted guarded effects become mutually ordered, under the named backend profile. It confirms both reached the intended contested boundary and that at most one incompatible new primary is incorporated. It checks the losing request's actually delivered model outcome, rather than assuming all losing transports must report a particular HTTP code. If the barrier changes the guard mechanism or prevents the tested real submission, its result cannot qualify the physical concurrence claim.

An authorized manager Reassign racing target Membership revocation, a simultaneous Complete and reassignment, and compatible/incompatible C42 replay are separate concurrency or command tests. Atomicity and non-partial result must be read from the actual persisted Request and assignment set, not from an in-memory precomputed transaction plan or the last browser card alone.

Negative assertions need a completed, causal observation window. For example, to establish that remote R was not automatically adopted, the test first observes R's committed authority and that the intended already-connected B received the relevant advisory (or qualified gap/debt). It then records B's adopted membership and monitors the actual forbidden acquisition and installation paths until a defined synchronization milestone. A momentary count of zero before B receives any event cannot show the policy was respected. A single sleep cannot exhaust all possible late callbacks; deterministic adapter exploration and instrumented physical events cover the specified schedules. Claims beyond the observed window retain a liveness or fairness premise, never an invented absence proof.

Positive outcome delivery and unknown delivery have distinct test branches. A competing Claim with healthy confirmed transport must produce the supported definitive delivered outcome; a controlled lost response after submission may legitimately leave C42 unknown. A current claimed card cannot attribute the exact historical command. Each fault case documents precisely which transaction outcome is observed, which is uncertain, and which subsequent authorized command-specific recovery is permitted.

Lifecycle, model and command acceptance matrix

Scenario	Independent decisive observation
Self-Claim	The authenticated helper, not a forged client actor, becomes the only active primary
Claim under competing helpers	At most one new primary survives the real guarded conflict; delivered loser status a
Preparation followed by policy/Membership change	Commit-time dependency guard prevents the obsolete transition; no partial Request
Helper claim on behalf of another target	Only explicitly authorized actor/target variants succeed; target identity never substi
Compatible C42 replay	One incorporated command; same binding yields only the authorized historical resu
Incompatible C42 reuse	Changed Request, arguments, principal or operation rejected without mutation or u
Replay after Unclaim or Reassign	Historical incorporation of C42 does not falsely describe the current primary
Unclaim	Eligible actor returns Request to `:open` with correct ended assignment and preserved
Mark on the way	Correct `:on-the-way` state and active assignment; disallowed starting states reject
Complete	Model-defined permitted starting states yield `:done` and correct ended responsibilitie
Cancel	Authorized requestor/current policy yields `:cancelled` and correct assignment closu
Reassign	Legitimate current target selector, policy check, ended old primary and one new prir
Changed Reassign target policy	Commit-time guard rejects stale/ineligible replacement without partial ownership cl
Duplicate Create/lost Create response	Where the selected command profile promises it, one incorporated new Request and
Missing operation capability	Removing a required positive from both Choreo and model cannot make independer

Board, Live and presentation acceptance matrix

The selected HumanHelp policy is **remote advisory without automatic adoption**, including changes to already-visible Requests. Remote notifications may alert, but an unpermitted remote change does not replace current adopted fields; explicit one-use Refresh or a separately authorized local result is required for adoption.

Scenario	Required user and physical outcome
Initial page	Genuine authenticated principal, authorized cohort/scope, meaningful defaults, correct gen
Create ordinary	Creator's confirmed R appears promptly; a separate already-connected viewer gets advisor
Create under excluding filter	Creator sees authorized local result/confirmation while search/options and prior adopted c
Remote R, then creator-local S	S is selectively adopted for the actor; remote R remains outside that actor's adopted cohort
One-use Refresh	An explicit permitted click triggers one corresponding authoritative acquisition/adoption;
Advisory during Refresh	A later uncovered advisory remains outstanding after earlier captured requirement returns
Local result races older Refresh	Old board/fragment response cannot remove or regress a newer justified card or newer loc
Read succeeds but swap fails	Adoption and notification debt are not falsely cleared; visible repair remains available
Scope-limited/empty/error response	No invented complete absence, removed authorized rows or resolved unknown command
Search/options after remote advisory	Only adopted entries change local presentation; no implicit server-side collection requery
Newest/oldest, mine-first and unclaimed-first	Actual stable order: mine-first primary, unclaimed-first secondary, creation order then iden
Terminal visibility	`:done` and `:cancelled` are hidden/shown within adopted membership without changing da
Details/focus/scroll	Compatible same-identity replacement preserves declared continuity; target retirement or
Rapid input, stale callbacks	Later local edits and justified installations dominate stale search/option/continuity callback
Remote lifecycle change to visible R	Existing adopted R fields remain until the specified user-approved Refresh or authorized ac
SSE disconnect/reconnect	Lost interest/coverage is reported and recovered within the selected mechanism; reconnect
Duplicate/retired events	No duplicate protected effect, obsolete target resurrection or foreign recipient disclosure
Response flow	The independent response interaction product contract must be selected and passed; an un

All remote/local cases use at least two independently authenticated browser contexts, preferably distinct principals, with independently verified sessions, permission scopes and actual network connections. A pair of pages sharing one mutable auth fixture is not enough. Cross-Organization and cross-Location tests challenge both payload disclosure and metadata/count/interest leakage. The test distinguishes a user-local option from a server-side query change: where a new authoritative collection query is actually required, it needs its own authorization and scoped read/adoption permission; it cannot be smuggled in as mere presentation search.

Fault, recovery and environmental qualification

Fault or changed premise	Required evidence and safe result
Failure before actual submission	No new authoritative effect; typed local failure and permitted new admission
Submission with lost result	C42 remains unknown until exact authorized command-specific evidence; current card rea
Known commit, failed rendering/transport	Persistent transition remains; invalid output blocked and actor has qualified acquisition/re
Crash after commit before Live publish	Transaction-coupled durable publication intent or the explicitly chosen tested alternative
Live delivery gap or reconnect	Interest/grant fencing, scope and basis recovery; no accidental automatic adoption
Result lookup denied/expired	No protected disclosure; denial/retention expiry is not evidence of rejection
Fragment read timeout/partial/error	No invented complete authoritative absence, and no false Refresh-debt discharge
Partially installed OOB/multi-target response	Per-target physical completion and repair rights; no claimed atomic multi-target adoption
Older network/optimistic callback	Retired E42/T1 cannot overwrite newer justified installation or restart old plan
Deployment A to B while client A persists	Matching assets/compatible protocol or declared reload/retirement; no mixed-generation a
Wrong installed Biff handler or receipt	No protected service with unsupported assembly; paired valid startup still succeeds
Membership/Organization change in flight	Proper current guards, read/disclosure revocation and usable permitted recovery
Unknown/extra physical writer	Coverage obligation remains open; it cannot borrow the declared operation's proof
Changed XTDB/HTMX/compiler qualification	Dependent proposition requalified, blocked or honestly reported unsupported, never silen

Fault injection records the exact boundary and ordering: before send, after send/before transaction, after commit/before response, after commit/before publication, read-success/swap-failure, owner retirement and browser reload. It must not replace a real backend/handler with a fake transition while claiming physical refinement. Fault-free tests must not tolerate synthetic unknown outcomes; fault tests must not convert unknown to rejection. Automatic retry budgets, retention horizons, user recovery affordances and resource cleanup are separately observed.

Unexpected console/page/network failures and unrelated non-2xx responses fail the test. The fixture may accept an **expected** typed fault only in the case that injected it, with exact operation/subject, first enforcing boundary, permitted outcome and current poststate. Blanket suppression of 4xx/5xx, all timeouts, or all exceptions would conceal a broken product path. Negative cases pair with a valid, eligible positive so ”reject everything” cannot pass.

Evidence ledger, suite composition and completion

The finished HumanHelp acceptance report connects the real production assembly, generated browser manifest/build receipt, physically served asset, authenticated sessions, rendered controls, browser-created commands, installed Ring route/handler, XTDB transaction/readback evidence, actual SSE/HTMX/OOB effect events, current adopted frontier and independent product outcomes. Each reference has its source, version, trust boundary, observation scope and limitations. It must not use a copy of one green preflight result as evidence of all these independent edges.

The suite reports distinct statuses for domain/model, bounded semantic checking, backend qualification, declared and actual writer coverage, structural/render rejection, installed-host adoption, real browser lifecycle, full two-client board/Refresh, failure/reconnect, and independent response-flow requirements. A partial pass is reported as partial. A missing positive capability, unqualified real writer, untested physical join, or still-unselected response-flow contract prevents full HumanHelp product acceptance, even when every available test passes.

A future strengthened Gesso checker can produce more of the required oracles automatically from one source-owned application, but it cannot silently generate the independent HumanHelp business intent or prove physical causality from unchecked report-shaped maps. Part IX owns compositional certification; Part X owns the proof/refinement correspondence conditions. Section 08.13 details deterministic structural rejection, while 08.14–08.15 join deployment evidence and final acceptance gates. Their work remains open until demonstrated, not declared complete by this section.

Structural rejection and production enforcement

The HumanHelp proving application must demonstrate **selective**, source-derived rejection: an invalid operation, construction, rendered action, runtime invocation or physical continuation is rejected at the first boundary that actually has enough trusted evidence to reject it, while a nearby legitimate HumanHelp action remains executable. A clean report is not an execution credential. Passing a test by rejecting every operation is a failure of the independent positive product contract.

The finished system does not solve this by maintaining a second example-owned operation registry, a hand-copied route inventory, or checker fixtures that silently substitute for the production application. The production Request model, Request Choreo, concrete HumanHelp routes, actual server realization, Live acquisition graph and generated browser artifact are the sources from which the canonical assembly and its checks are derived. Tests intentionally damage **one** of those real sources or one installed boundary, then exercise the same checker, renderer or dispatcher used in service.

Admission stages and first enforcing boundaries

The selected assurance policy assigns each check to the earliest boundary at which its actual evidence is available. Static construction may decide a missing typed operation or route; it cannot invent the future authenticated principal or the current Request/assignment prestate. Concrete rendering can inspect the supported actual Hiccup/HTMX/native form coordinates; it cannot prove that an arbitrary future script emits no other request. The trusted route can bind untrusted input to the selected operation and real actor; the production model and guarded XTDB transaction decide commit-time business eligibility. A physical adapter can deny an obsolete target's effect only while it owns the actual admission/installation boundary.

Stage	Evidence and responsible rejection
Source/semantic construction	Unknown operation, invalid role or impossible plan, undeclared required recovery, or unclosed op
Application assembly	Missing required browser plan, trusted executor, route, provider, publication or authoritative acqui
Build and deployment admission	Invalid source⟶generated-plan⟶artifact receipt/bytes relation, unsupported compiler/runtime pr
Actual structured-render/serialization	Enabled but unprepared action, missing required observation, anonymous collision with a reserve
Trusted HTTP/command ingress	Malformed protocol, method/path/subject/operation mismatch, forged identity, incompatible comm
Current model and transaction	Changed Membership/Organization/Request eligibility, conflicting primary assignment, failed curr
Browser effect and acquisition	Unauthorized collection expansion, ungranted Refresh, invalid progression, retired subscription/ta
Host installation and execution	Invalid or replaced checked handler, unsupported Biff start order, unmediated writer, mismatched
Postcommit observation/recovery	Qualify the actual outcome, preserve unknown and repair obligations, never reclassify an already

A checker must not move a failure earlier by assuming facts it has not observed. Nor may the application defer a statically detectable missing route until the first user clicks Claim. If an error appears after a real commit, the responsible boundary blocks invalid output or later effects and initiates recovery; it cannot retroactively deny the successful transaction.

Controlled defects in one actual production construction

Each defect fixture starts with the ordinary real HumanHelp application constructor and its source-owned production Request Choreo/route/Live declarations. The test removes, substitutes or changes one declared edge **before** the relevant check, or exercises a specified post-admission physical replacement. The remaining sources stay real so failure is attributable to the selected defect. The test records the original and changed source/generation, first rejecting boundary, typed diagnostic and prohibited effect; the paired valid case is run through the same boundary.

Controlled defect	Required result and positive counterpa
Unknown `:request/claime` or valid operation with incompatible role	Source or preparation reports unknow
Missing Claim browser plan or incompatible plan/role	Assembly reports the exact required re
Missing route, method mismatch, duplicate ambiguous route or route points to Cancel	Assembly/installed dispatch rejects th
Missing current-time, trusted actor, transaction, seed or qualified read provider	Statically absent capability fails before
Missing initial board acquisition, fragment/stream route, settlement or denied-disclosure recovery	Admission reports the missing require
Missing exact browser artifact, stale plan stamp, wrong bytes or forged receipt	Supported build/start/serve check reje
Enabled action missing required `:observed-basis` or canonical operation identity	Real preparation/rendering rejects or p
Ordinary HTMX/native form submits to protected Claim route	Actual supported output scanner catch
Form owner/submitter overrides or keyword/string duplicated request attributes	Actual effective method/path and amb
OOB/error/deferred/cached rendering bypasses the guard	Serving boundary rejects unsupported
Forged browser principal, Request retargeting or valid-plan command on wrong lifecycle route	Trusted route ignores client identity as
Incompatible C42 reuse or lost C42 result	Current permitted command resolutio
Unqualified advisory used to adopt remote R or repeated one-use Refresh	Live/acquisition effect admission refus
Stale E1/T1 callback after newer T2 or removed target	Final physical effect gate refuses instal
Correct preflight report but different Ring handler actually served	Startup/host adoption blocks the selec
Omitted/false read-only background writer or unrestricted XTDB capability	Mandatory writer-coverage claim stay
Source removes Reassign from model, Choreo and generated checks	Independent product acceptance fails

A defect that changes several facts at once is not an oracle for which one should have failed first. The test either isolates them or reports their causal ordering and the evidence available to each boundary. A failure at an earlier supported gate can make later gates inapplicable; it does not demonstrate those later gates would work or reject correctly. The positive counterpart is necessary to exclude a permanently disabled or reject-everything application.

Physical rendering and both directions of protection

For HumanHelp managed lifecycle routes, the **producer** and **consumer** protections are complementary. Supported actual pre-serialization structured rendering cannot emit a live anonymous action to a protected route, even when its HTML looks like an ordinary button. The trusted receiving handler cannot execute such an action even if a malicious client manufactures it, the served markup was not produced by HumanHelp, or a later JavaScript sends it. Neither guard is permission for the other to be absent from the declared protection profile.

Rendering checks the actual expanded concrete occurrence, including conditional affordance selection, target identity, effective HTMX inheritance, native form owner/action/method and submitter overrides, supported OOB destinations and plan/basis preparation. A disabled-looking control with a still-active request route is not a valid disabled control. Missing observed basis is not silently converted to a guessed Request revision, wall-clock time, anonymous normal POST or inferred authorization. The application selects only an expressly admitted unavailable/alternative state or rejects the render.

The selected Hiccup/serialization mechanism has a finite language and a physical boundary. An inspector that receives canonical Hiccup cannot certify pre-serialized HTML, arbitrary JavaScript-produced requests, future middleware substitution, deferred work outside its guard, or bytes served by a different handler. Those paths require a connected enforcement mechanism, explicit disallowance under the admitted profile, or an honestly narrower claim. A passing snapshot render does not establish universal dynamic-output coverage. Unchecked raw HTML is not cured by adding a success-shaped value to the assembly report.

If an output check rejects **before** sending a protected command, the positive oracle establishes that no protected mutation was submitted. If it rejects a response after a previously committed Claim, the mutation remains authoritative: the correct oracle is blocked invalid output, preserved command/outcome evidence and permitted authoritative repair. It is wrong to assert Request remains open merely because a postcommit response-render guard failed.

Trusted actor binding and command disposition

For a routed Claim(R17), the trusted boundary uses the authenticated current principal and the concrete route's operation/Request selection. The browser-supplied `:principal`, target-helper string, role name, client ID, plan stamp, command identity and reported basis are data to verify, not an authorization token. Route-selected Claim cannot perform Cancel, reach R18, invoke an arbitrary Clojure callable or bypass the current production Request/RequestAssignment/Organization/ Membership/Location guards.

A malformed command, invalid transport and incompatible binding can be rejected **before domain invocation**. A well-formed authenticated command may instead fail a model-defined current eligibility rule after trusted execution begins. That is a legitimate domain rejection, not necessarily an application-assembly fault. A commit followed by lost outcome remains uncertain to the participant until qualified command-specific reconciliation; timeout, denied disclosure and missing historical lookup are not evidence of model rejection. A duplicate exact binding is resolved under the selected durable-command protocol; an incompatible reuse must not obtain another user's historical result.

The test checks separately for absence of prohibited model invocation, absence of new transaction incorporation, absence of unauthorized disclosure, and absence of stale physical installation. These are different postconditions. A checked route that refuses Claim before invocation should not be credited with proving every conceivable concurrent database writer obeys the same invariant.

Runtime installation, source identity and report integrity

Gesso's canonical managed Biff/Aleph path must install the actual checked Ring handler **before** admitting protected HTTP requests. The exact `application-handler-module` object is part of the native module collection once; its required qualified module ID is first in the declared Biff start order. The startup checker rejects a missing/duplicated module, a forged module carrying the correct ID, a non-ID component-function start sequence, a mismatched current assembly or noncallable Ring handler. The selected Aleph adapter must serve the resulting handler, not an older Var or different unwrapped function.

The host admission evidence is invalidated by later handler/module/route/asset replacement unless the replacement is admitted and checked under the new generation. If a server intentionally uses raw Ring responses, off-process writers or a separate HTTP entrypoint, the production policy must enforce or exclude them explicitly rather than inherit the canonical wrapper's limited guarantee. A current in-memory report does not prove which callable the deployed server actually installed.

ApplicationAssembly and checking reports are derived from the original source objects, verified identity and current underlying artifact; arbitrary Clojure maps are not capabilities. A caller must not gain authority by copying a passing report and changing `:valid?`, `:errors`, `:analysis` or an assembly field, swapping an operation registry, adding a favorable proof badge or using a stale content-identical replacement provider. Genuine reconstruction/rechecking is required when the selected source, host callable, compiler receipt, actual served asset or admission generation changes. Even a well-formed checked report is descriptive of its stated scope and never by itself permission to perform a protected effect.

A structural preflight can check known supplied writers and enforcement edges; it cannot discover every process with direct XTDB access by enumerating the registered Biff modules. The assurance report distinguishes declared, enumerated, physically observed and proven restricted paths. An unclassified writer remains an open mandatory condition or blocks installation under the selected policy. It cannot be hidden by relabeling a descriptor as read-only or reporting an uninspected inventory as empty.

Dynamic failure completeness and recoverability

Static admission does not guess future transaction results or assume every GET, SSE stream and subscription will succeed. It must nevertheless establish an allowed continuation or recovery policy for each dynamic condition that can strand a managed command or adopted board obligation: missing/expired observation, authoritative rejection, submitted-but-unknown transaction, unavailable disclosure, lost subscription, failed fragment reread, partial DOM/OOB installation, obsolete plan or target, exhausted retry budget, and unavailable trusted provider.

Each continuation identifies its owner, permissible current authority/read scope, bounded retry or user action, release of obsolete physical rights, and what evidence can terminate uncertainty. An invalid assembly is rejected if it has no declared legal recovery route for a required uncertain branch, even when a particular test happens to succeed. A permitted explicit degraded path is named and reflected in the assurance claim; the system must never silently turn a semantically protected lifecycle command into anonymous HTMX or turn participant unknown into rejected.

Recovery and static safety are reported separately: correctly preventing an unsafe stale swap is a safety success, while eventual convergence depends on declared provider availability, scheduling fairness, retention and (for explicit Refresh) permitted user action. Runtime denial may therefore preserve visible pending/advisory debt rather than fabricate completion.

Structured diagnostics and acceptance

Each rejection supplies stable machine-readable category, precise proposition, original source or physical occurrence, application generation where relevant, operation/route/target subject, expected versus actual relation where safe, first responsible enforcement boundary, and whether the effect was already submitted or committed. Diagnostics may report unknown rather than an invented yes/no; errors from domain guards and from assembly incompatibility retain distinct types and recovery contracts.

The diagnostic audience determines allowed detail. Redact authenticated credentials, full command bodies, private Request content, unverified tenant identifiers and protected read/query results. Useful developer-local explanations may show source paths, selected operation names, mismatch categories and safe route templates; they must not leak cross-Organization information to a browser or untrusted requestor.

```
Positive and adverse — one real production boundary:
  Good Claim(R17): canonical constructor, installed handler,
    enabled bound control, trusted principal, guarded commit,
    authorized readback and browser settlement all succeed.
  Missing Claim plan: same source construction with only
    required plan removed → preflight rejection, no fallback.
  Paired valid case still succeeds after restoring the plan.
```

```
Adversarial — anonymous request and forged principal:
  A real Hiccup/native form inherits the protected Claim POST,
    but lacks managed operation/protocol metadata.
  Supported final render check rejects its emission.
  Independently a forged external POST supplies :principal Mallory
    and incompatible Claim/R17 wire values.
  Trusted ingress does not execute as Mallory or mutate R18.
```

```
Adversarial — postcommit output failure:
  C42 validly commits Request R17 + primary assignment.
  A later output renderer fails structural enforcement.
  The invalid response is not delivered, but XTDB commit persists.
  Recovery obtains qualified current state and historical
    C42 outcome separately; the server never reports "not committed"
    merely because its HTML response was blocked.
```

```
Adversarial — startup identity and escaped writer:
  The browser receipt and ApplicationAssembly check successfully.
  HumanHelp supplies a noncanonical Biff module/incorrect order,
    or Aleph invokes a different unwrapped Ring handler.
  Service admission refuses the installed-boundary claim.
  A background raw XTDB writer outside the checked module catalog
    leaves global writer closure explicitly unproved.
```

HumanHelp's structural rejection gate exercises the production preflight constructor, actual rendered responses and installed host path against controlled defects: unknown operation; missing/mismatched plan, route or server; missing provider and initial acquisition; anonymous native form and HTMX action; forged/mismatched command; stale asset; handler/module mismatch; unauthorized advisory/Refresh; retired effect; hidden writer; and required capability removal. Each case has a paired usable positive through the same enforcing boundary, a precise typed diagnostic and an appropriately scoped no-prohibited-effect oracle.

A negative fixture that passes only because the app never starts, no browser control exists, all requests are rejected, or a fake checker is used is insufficient. The complete product still requires the real HumanHelp Chromium lifecycle and two-client board/Refresh behavior of 08.12. Section 08.14 joins connected evidence and operational limits; this section specifies enforcement; independent acceptance must establish whether the selected build actually passed the tests.

Connected evidence and operational limits

HumanHelp is useful as evidence for Gesso only when its real production model, assembled semantic operation, browser command, installed host, authoritative effects, Live advisory and adopted presentation can be related without inventing unobserved edges. The completed application exposes connected evidence for at least Claim, Create with selective adoption, Reassign with genuine target selection, and explicit one-use Refresh. It does not construct a parallel example-owned witness registry that could agree with itself while the running application uses different sources.

Connected evidence does not mean that one large success map, identical operation names, a common digest or passing tests automatically prove causality. Every link states which source-owned relation, verified construction, qualified physical observation, authenticated receipt, or still-open premise supports it. The claim remains no stronger than its weakest required, unestablished connection.

Claim-specific evidence and its ownership

Each end-to-end report identifies its **subject**: exact HumanHelp source and ApplicationAssembly generation, Request/actor scope, selected operation and assurance policy, physical runtime and backend qualification, observation interval, and the independently required product outcome. An audit view is derived from those subjects; it is not a credential, an execution grant, or a surrogate for verifying the present caller and protected effect.

Link in the chain	Required provenance and boundary
Production source	Exact public Request/RequestAssignment operation, relevant Organization/Membership dependency declar
Admitted application	Choreo/plan, checked operation/route/acquisition construction, known missing or opaque edges, and appli
Actual served control	Authenticated viewer and permitted scope, concrete rendered command identity, effective action/target and
Browser execution	Generated/served asset identity, C42 user intention, local execution E42, transport attempt H1, target T1 an
Trusted ingress	Actual installed Ring/Reitit handler and trusted principal, selected Request/operation, protocol/binding vali
Domain incorporation	Guarded Request/assignment transition, qualified XTDB transaction evidence, exact command record wher
Publication and delivery	Source-derived invalidation or transaction-coupled intent, authorized scope, publish attempt, recipient subs
Acquired authority	Scoped read permission, actual basis and completeness, operation-specific observation and any qualified ab
Physical settlement/adoption	Issued install permission, current target/generation, actual swap/continuity completion, adopted membersh
Assurance conclusion	Each established proposition, evidence mechanism, physical/test scope, accepted premises, untested bounds

This report is a directed, typed chain, not a chronology guessed from log timestamp order. A browser HTTP 200 cannot authenticate a model transaction. An XTDB readback at a later basis cannot by itself identify which command caused its values. A matched command-record value does not establish that every outside writer was observed; a returned command outcome does not prove the browser installed it. A Live publish receipt cannot prove recipient delivery or adoption. Independent evidence must actually join across the selected edges.

When a test or diagnostic sees only an in-process callback or caller-reported observation, the report says so and preserves the original trust boundary. In particular, a checked Gesso report/fingerprint is not a transferable proof certificate, and an observation attached to a copied Request/actor map does not become authenticated by the map's shape. Claims involving uninspected external writers, compiler semantics, hidden data sources or opaque policy functions remain conditional or open under the selected assurance policy.

Reports for developers and operators retain useful diagnostic context but redact sessions, private Request content, credential/command bodies and tenant-scoped identities unless their audience is specifically authorized. Correlators and structural fingerprints do not themselves grant result disclosure. A principal or access-policy change may invalidate further acquisition even when the historical command remains incorporated.

Connected HumanHelp journeys

Claim establishes a positive route from a real eligible button through browser-generated C42 and trusted route to production Request and primary assignment incorporation. It separately establishes permitted current reread and a physically installed settled card after provisional ownership retires. A lost H1 response is not negative transaction evidence: its participant branch remains command-outcome unknown until exact authorized historical correlation, while a fresh Request read may independently repair visible state.

Create/selective adoption establishes that local creator S receives its own correlated authoritative result while already-connected viewer B learns of remote S only as advisory debt. If B has an earlier remote R pending and creates its own local T, T may be adopted without R or R's pending debt being silently consumed. The evidence records the actual creator-specific transaction and reread, each viewer's independent session, authorized advisory scope, and the before/after adopted cohort. It does not call a broadcast or toast an authoritative card installation.

Reassign establishes legitimate distinct actor and target-helper identities, current relevant Membership, Organization/Location and Request guards, model-defined ending of the previous primary assignment and creation of exactly one new primary. It then connects the real target selector, browser command, trusted route, transaction, qualified readback and displayed result. An old candidate who loses eligibility between display and commit must not be counted as a valid successful Reassign.

Explicit Refresh establishes the one-use action/grant that permits a scoped remote acquisition, the captured required advisory obligations, the actual read result, qualification of the returned progression/scope and physical installation of the admitted projection. Only requirements actually covered by that successfully adopted result are discharged. A newer advisory, uncovered scope, failed read, partial OOB installation, obsolete response or lost permission retains its specific obligation or justified unknown status. No self-perpetuating successor read or passive SSE event inherits a fresh grant merely from a previous Refresh attempt.

```
Positive — four connected journeys:
  Helper A clicks rendered Claim(R17) → browser C42 / E42 / H1.
  Trusted Request model commits guarded Request+assignment.
  Actual authoritative reread and installation settle R17.
  Creator A adds S; viewer B receives advisory but not S's card.
  Manager M selects eligible target H2 and reassigns R17.
  B explicitly Refreshes; only the qualified acquired scope
    installs and only actually covered debt is discharged.
  Each edge retains its distinct source and physical witness.
```

These histories are independently testable but not one universal serial execution. Joined claims require actual identity, scope, predecessor/successor and temporal relations; four individually plausible snapshots do not automatically form a contiguous physical history.

Safety, uncertainty and progress are distinct claims

The following are safety properties: no double primary assignment under the specified guarded writer set; no unauthorized command or scope disclosure; no anonymous lifecycle route downgrade; no unpermitted automatic adoption; no retired E42/T1 continuation overwriting a newer installation; no loss of uncovered Refresh obligations through a successful-looking HTTP response. Their evidence can be finite, conditional inductive, runtime enforcement or bounded observed physical behavior, but those mechanisms are named separately.

Progress propositions have different conditions. If Live transport is available with the declared fairness and recovery strategy, relevant advisory may eventually reach an interested active browser. If the authoritative provider is available, permitted Refresh may acquire current data. If the user never takes the explicitly required Refresh action, a stale adopted board is allowed indefinitely; the UI must still explain the pending condition and expose the action. A transient unavailable provider, denied disclosure, expired command record or permanent network partition does not falsify a promise whose **premises** explicitly exclude that situation; it does prohibit claiming unconditional eventual convergence.

A publication failure after committed Request mutation does not reverse the mutation. The primary durable publication profile requires transaction-coupled intent recorded with the business transition and an at-least-once worker/recovery protocol with stable work identity, duplicate-safe delivery, restart, backoff, retention and a visible escalation policy. A simpler selected user-Refresh/periodic-reconciliation profile makes a narrower liveness claim and tests its actual recovery conditions. An in-memory callback, successfully registered subscription or best-effort postcommit push cannot claim the stronger durable crash-to-publication guarantee.

Similarly a stable C42 identity is not guaranteed historical exactly-once behavior unless the chosen backend profile provides qualified atomic command-record/domain-effect incorporation and exact-binding replay. Current Request state and C42 historical attribution are different acquisitions. Transport timeout, missing lookup, expired retention and unauthorized disclosure remain unknown or undisclosable where appropriate. A cleanup function cannot turn that uncertainty into rejection.

Resource limits, backpressure and debt accounting

The installed application publishes actual resource budgets and owners as part of its named deployment profile rather than implying unbounded queues or infinitely retryable work. At minimum its limits cover:

Resource	Required capacity and overload semantics
In-flight fragment acquisitions	Per logical fragment/request-generation bound; allowed coalescing and obsolete-transport cancellation;
Pending successor requirements	Bounded representation retaining scopes, covered/unknown knowledge and joinability of actual progres
Advisory debt	Per-viewer/scope bounded storage or safe compaction to a coarser ”updates/continuity unknown” oblig
Publication intent and delivery	Queue size, worker ownership, retry/backoff, retention, failed-work escalation and durable handoff whe
Command outcome recovery	Receipt/index retention, context/disclosure restrictions, retry rate/work horizon and honest expired/und
Browser-owned effects	Bounded live timers, listeners, transport handles, installed optimistic generations, retained continuity sn
Local presentation	Board cohort/page size, number of visible cards, detail/focus/scroll captures, drafts and permitted in-me
Diagnostic evidence	Bounded, redactable and access-controlled records with useful provenance retained long enough for the

Resource limits are not solely performance goals. An exhausted acquisition slot must not silently authorize an extra read; an overflowing advisory queue must not discard the only evidence of remote change; and evicting an unresolved command receipt must not invent a known terminal outcome. Safe coalescing may combine equivalent obligations or use a qualified coverage join. Where requirements cannot be safely joined, the runtime retains a bounded coarse unknown/debt state with a permitted full-scope repair path and visible explanation. One queue slot bounds stored work, not the semantic number of unfulfilled obligations it represents.

Overflow handling identifies its effect on safety, fairness and progress. Repeated hot scopes cannot indefinitely starve an unrelated permitted Refresh or a queued authoritatively correlated local result under a claimed fair profile. Dropped jobs report their loss/unknown coverage through the designated owner. If the selected profile has no durable queue, the evidence must not pretend pending in-memory work survives process restart.

```
Adversarial — advisory overload:
  Distinct remote R1 and R2 requirements accumulate while
    viewer B is offline; the fine-grained budget fills.
  Coalescing may retain a scope-wide "possibly changed" debt
    and require explicit authorized complete Refresh.
  It may not erase R1, declare B current after R2 delivery,
    or install remote cards without an explicit grant.
```

Resource retirement and ownership transfer

Cleanup first **revokes semantic permission** for the old execution, request, target, subscription, grant or continuity slot; it then attempts physical abort, listener removal, timer clearing, XHR/SSE release and continuity snapshot disposal. Physical abort is best effort. An old asynchronous callback that arrives after cleanup cannot regain rights because its generation has been retired, even if the browser or network library cannot cancel it.

Release is idempotent under duplicate completion, nested removal, aborted transport, unsuccessful cleanup and application shutdown. Exact owners are registered, so removing one card's interest does not accidentally close a shared SSE connection still serving another authorized subscriber. Partial cleanup failure is diagnosed and bounded; a detached DOM node or callback is not silently treated as harmless if it retains a reachable protected effect capability.

An uncertain C42 result, pending durable publish intent or unfulfilled Refresh/advisory debt may outlive its original physical card/target. Before disposing the old owner, the runtime transfers the still-valid semantic obligation to a declared recovery owner (possibly an authorized server-side index). That transfer preserves only the minimal qualified command reference and scope; it does not resume E42, re-grant a one-use Refresh, preserve disallowed private content, or override a new principal's access restrictions. Expired retention produces an honest recovery limit.

Startup, shutdown, rolling generation replacement, logout, permission revocation, browser navigation and disconnect all have declared retirement and handoff behavior. Cleanup must be testable both after a normal successful operation and after each partial failure path. A clean empty pending resource map is insufficient evidence of no leaked native listener or wrong-installed physical target; the test checks the actual supported shell/host resource boundary.

```
Adversarial — late physical completion:
  E1 uses T1 and an in-flight H1; then a newer
    authoritative card installs into T2.
  Semantic retirement revokes E1/T1 installation rights.
  H1's abort fails and callback eventually completes.
  The runtime drops E1's now-unauthorized effect; it
    cannot restore T1, change T2 or clear unrelated debt.
  If C42 remains unknown, the recovery owner retains
    its qualified historical inquiry independently.
```

Connected fault evidence and failure attribution

The application exercises at least these ordered negative histories against its actual boundaries:

History	Safe conclusion and observation
Invalid pre-submit operation	No protected command submitted; construction/route diagnostic names the first real reject
Submitted C42, lost HTTP result	Historical outcome unknown to browser; no fabricated rollback or new C43 replay; author
Commit, then response-render failure	Request/assignment truth persists; invalid output rejected; permitted reread and historical
Commit, then publication crash	Durable intent replays under the selected primary profile or an explicitly weaker alternate
Advisory received but no permission to adopt	User sees pending indicator; no unauthorized remote collection acquisition/installation
Refresh returns partial or stale coverage	Only actually installed, qualified covered obligations discharge; newer/uncovered debt rem
Reassign with policy change in flight	Current guard prevents obsolete target incorporation; no partial primary-assignment trans
Installed T2, late E1 rollback	No stale physical restoration even if transport cancellation fails
Restart with old A page and new B	Incompatible effects retire or request qualified reload; old C42 may be reconciled independ
Unenumerated raw XTDB writer	Physical-writer coverage remains unknown/open or fails selected admission; declared cata

The report distinguishes ”expected rejection occurred”, ”no forbidden effect was observed through the qualified mechanism”, ”qualified absence established”, and ”not observed or still unknown”. Lack of a log line, a passing timeout assertion or a successful build receipt is not a proof of non-execution. A bad browser response cannot undo a previously successful transaction. Fault injection exposes the precise point before submission, after submission, after commit, before publication, during acquisition or during physical install; it does not mock away the real effect and then claim physical correspondence.

Retiring obsolete migration state and classifying defects

The final HumanHelp proving surface uses production `net.humanhelp.site.model.*` semantics and the single admitted example construction. It has no parallel demo Request store, competing example authorization state machine, alternate ”visible revision” treated as XTDB authority, duplicate route/operation catalog, anonymous lifecycle fallback, stale CSS producer, or compatibility code that silently changes the supported Choreo/Biff/HTMX contract. Legitimate presentation-local option/draft state remains; it is not a competing source of Request truth.

Removing obsolete files, tests and resources happens only after the replacement path and required positives are established. Repository history and a short reason/replacement record make the removal auditable without making a temporary cleanup checklist part of the timeless product semantics.

When HumanHelp testing discovers a problem, the evidence identifies its ownership: a missing reusable representation/closure property is a **generic Gesso gap**; wrong example routes, assets, or application composition are **HumanHelp integration defects**; wrong lifecycle/Organization policy is a **production HumanHelp model defect**; changed XTDB, HTMX, Biff, browser or network behavior is an **external/runtime premise failure** requiring requalification. A generic framework rule is introduced only when the discovered need is reusable, not merely because the example exposed a bug.

Completion evidence is generation- and profile-specific. The HumanHelp app's positive behaviors, negative enforcement, resource bounds, durable or weaker recovery profile, live publication, physical writer coverage and open premises are reported separately. Part VIII §08.15 specifies their joint completion conditions. Parts IX and X own the broader compositional proof and implementation-refinement claims; their open premises cannot be silently discharged by a connected HumanHelp trace.

Completion gates

HumanHelp is the proving application for the finished Gesso design, not a permanently privileged demonstration or a convenient subset of the currently working UI. Completion requires a coherent installed production-backed application and independent evidence that its promised user capabilities work under the selected, qualified substrate. The gates are normative acceptance obligations; neither their presence in this document nor a green source-derived assembly means that the current implementation has already met them.

An acceptance claim names its exact application source and admitted generation, code/build/artifact identity, deployed handler and browser assets, authentication and disclosure profile, relevant XTDB/Biff/HTMX/CLJS versions and configuration, tested actor/Request/Organization scope, operational assumptions, known opaque boundaries, and observed evidence. It is invalidated or narrowed when any required binding, source, runtime, writer coverage or provider contract changes without applicable requalification. A partial result is reported as partial, with the independent failed, missing, conditional or untested gates still visible.

Non-vacuous gate set and independent acceptance

The fourteen gates below are distinct obligations, not fourteen names for a single passing source check. Each has at least one eligible positive and its applicable negative/fault case; completed checks must exercise the **real** enforcing or user-facing boundary rather than an uninstalled fixture or a copied result.

Gate	Acceptance obligation
1. Production model authority	Request creation and lifecycle mutations use public production Request and related User, M
2. One admitted application source	Choreo, role-local plans, routes, server operations, trusted providers, Live/acquisition, conc
3. Installed host and writer adoption	The actual serving Biff/Aleph handler and protected mutation paths use the admitted const
4. Current browser and asset correspondence	Generated plans, build manifest/receipt, actual served JavaScript and dependent assets, clie
5. Claim end to end	A genuine rendered enabled Claim control generates its own command in the actual brows
6. All lifecycle capabilities	Claim, Unclaim, Mark on the way, Complete, Cancel and Reassign each have an independe
7. Rich-board product behavior	Real Create, creator-local visibility, adopted-cohort search and sort, both priority toggles, t
8. Selective multi-client adoption	Distinct authenticated users/contexts demonstrate that remote creation and remote lifecycl
9. Explicit Refresh correctness	A user action creates a bounded scoped one-use read/adoption grant; actual authoritative c
10. Fault and recovery behavior	Lost/unknown command outcomes, already-incorporated replay, failed acquisition/installa
11. Structural and runtime enforcement	Controlled malformed operation, assembly, plan, route, provider, render, native-form/HTM
12. Semantic and assurance evidence	Application/business invariants, finite/conditional proof scopes, initial states, complete dec
13. Qualified physical substrate	Selected Biff lifecycle, HTMX/SSE browser adapter, CLJS/compiler/artifact supply, XTDB g
14. Independent real-browser acceptance	Chromium/Playwright uses the installed service and generated runtime with genuine cont

R4's eleven original completion gates are preserved in this more explicit partition: semantic-route exclusion, runtime adoption, artifact currentness, real Claim, the full lifecycle, rich board, multi-client behavior, production-constructor tests, malformed fixtures, Chromium tests and supported Biff/Gesso substrate. Extra gates make recovery, explicit Refresh, source/physical proof and dependency qualification visible rather than assuming they follow from a green browser test. The original gates do not become optional simply because they now have different numbering and clearer ownership.

Product contract decisions, profiles and exclusions

The required user capabilities are independent of the current implementation's registered operation list. Removing Reassign from Request Choreo and from the derived finite machine does not make the reduced application acceptable if HumanHelp still promises Reassign. Permanently disabling all Claim actions is likewise not an acceptable way to satisfy negative tests. A particular Request may legitimately deny a command under current model policy while another eligible fixture satisfies the positive capability.

The selected remote-lifecycle adoption decision at #decision-humanhelp-remote-lifecycle-adoption requires explicit Refresh for ordinary remote creation **and** lifecycle changes, including a Request that was previously visible. That choice is normative. A locally correlated settlement or permitted affected-Request recovery read has its own narrower grant; neither silently grants remote collection adoption. Loss of authenticated authority is a separate safety transition: protected projection and future trusted access are revoked according to the supported client contract, rather than delayed to honor a user's Refresh preference.

The independent response-flow requirement at #decision-humanhelp-response-flow must be selected by HumanHelp product authority, with actor eligibility, intent, source/model owner, input, disclosure, observable success, persistence/external effects and adverse recovery behavior. A working Claim is not automatically proof of the intended response interaction. If HumanHelp expressly chooses ”respond by claiming”, its own versioned product contract and real user acceptance must establish that this fulfills

the complete promise; otherwise the response-flow gate remains open. No new messaging model, provider or contact-disclosure policy is invented solely to close a checklist.

The primary strong publication/recovery profile at 08.08 requires a qualified transaction-coupled publication intent and its durable at-least-once recovery worker. A selected weaker periodic-reconciliation or user-Refresh profile has different liveness and crash guarantees and must be accepted as an explicit product/deployment choice with its own tested behavior and disclosed limitations. It cannot silently borrow the primary profile's durable delivery or timely notification claim. Exactly the same discipline applies to command history retention, backend writer coverage, Live continuity, application generation pinning and external effects.

An unsupported extension may be excluded only by a precise declared boundary that does not undercut an independent required HumanHelp capability. For example, a profile excluding arbitrary JavaScript requests still protects actual managed HTML and trusted lifecycle ingress; the exclusion does not justify bypassing the real Claim button or its server route. Unknown or unsupported is not ”passed”.

Whole-path safety, evidence and causality

A completed HumanHelp Claim has a real actor and Request, source-owned semantic operation and browser plan, eligible rendered affordance, actual compatible served runtime, browser-generated command, route-selected trusted executor, qualified guarded transaction, truthful outcome, scoped current read and physically settled adopted result. The evidence chain identifies each of these as an actual binding, observed effect or explicitly accepted premise. A successful UI animation alone cannot establish authoritative mutation.

Create with selective adoption, a legitimate target-helper Reassign and the one-use Refresh journey likewise need connected positive outcomes. Multiple plausible snapshots do not prove one adjacent execution history; the report must connect command, transaction, actor, read and physical installation with adequate correlation and trusted provenance. A matching model abstraction does not establish that every physical XTDB writer refines it. An exactly matching browser manifest cannot prove arbitrary compiled JavaScript correct.

A meaningful negative acceptance states whether the forbidden request was never emitted, denied at trusted ingress, rejected by current model policy, excluded atomically by XTDB, or prevented from physically installing. These are distinct claims. A valid commit followed by failed HTML/OOB delivery remains committed and requires authorized recovery; postcommit output failure cannot be credited as a precommit domain rejection. A lost command response may leave the historical outcome unknown even if the current Request state is freshly observed.

The desired final invariant includes both safety and qualified progress. A retired callback cannot reinstall obsolete DOM or consume one-use Refresh permission. Unknown C42 must have its declared bounded recovery/disclosure path; if the evidence horizon expires, honest indeterminacy is permitted. An already-connected viewer whose remote board has changed may remain on older adopted content indefinitely when the chosen policy requires an explicit Refresh the user never performs. The pending advisory, safe action and scope of its coverage remain visible as the user-facing progress contract; automatic remote adoption is not introduced to manufacture convergence.

HumanHelp evidence and release decisions

For each gate the independent acceptance record gives a state such as satisfied under a named profile, conditional on enumerated premises, open/untested or failed; it does not collapse these into one green ”complete” label. It records the source, code/asset and host identity, test suite and actual execution results, adversarial coverage, responsible owner, assumptions and expiration or requalification triggers. A passing test **definition** is not proof it was executed, and a caller-reported observation is not authenticated by a report digest.

The final installed HumanHelp application must have real independently authenticated multi-client browser contexts, matching source-to-host and source-to-asset evidence, genuine production model transactions, actual target/adoption observations and fault outcomes at their claimed boundaries. The real host's ability to serve unrelated ordinary routes does not excuse missing managed-operation enforcement. All proposed positive and adverse capabilities must be tested with the actual qualified backend/adapter when the gate requires physical evidence. The suite must not ignore unexpected console errors, failed HTTP requests, domain exceptions or missing assets by an overbroad diagnostic waiver.

No earlier or later proof-certificate claim is created merely by satisfying these HumanHelp product gates. Parts IX and X own compositional certification and stronger implementation correspondence; Parts XI and XII own release and roadmap coherence. HumanHelp's function is to exercise those reusable Gesso interfaces and surface counterexamples, not to introduce HumanHelp-specific semantics into the proof kernel. The source-derived assembly may be called closed for its admitted scope while wider writer coverage, physical refinement or independent business correctness remain open. Such a result is useful but is not full HumanHelp completion.

```
Positive — eligible joined acceptance:
  Two distinct authenticated HumanHelp clients start
    with a qualified production Request cohort.
  A's real Claim reaches guarded Request/assignment commit;
    a fresh reload confirms actual model truth.
  A creates S, and B receives an authorized advisory
    without automatically adopting S.
  Manager M reassigns R17 to an eligible target helper,
    with exactly one active primary after commit.
  B explicitly Refreshes and installs the scoped authority;
    only covered advisory requirements are discharged.
  Independently, every required lifecycle and selected
    response-flow capability passes its positive case.
  The installed source, asset, host, model and physical
    evidence support each claim under the named profile.
```

```
Adversarial — green reduced application is incomplete:
  Remove Reassign and its matching plan from the source.
  All remaining declared structural and finite checks pass.
  Independent HumanHelp capability acceptance fails:
    a promised eligible Reassign is unavailable.
  A copied assurance report cannot change that result.
```

```
Adversarial — partial success and unqualified recovery:
  Genuine Claim commits with a correct RequestAssignment.
  Response delivery fails; C42 history is still unknown
    to the browser, although current Request truth can be read.
  Publication intent is only an in-memory callback.
  The application may report committed Request truth and
    bounded best-effort Live, but not durable crash recovery.
  If response flow is unselected, full product acceptance
    remains open independently of Claim's success.
```

HumanHelp proving-application invariant

> The finished HumanHelp proving application consumes the authoritative `net.humanhelp.site.model.*` public production semantics for User, Membership, Organization, Request and RequestAssignment. One canonical, source-owned ApplicationAssembly binds Request Choreo, its actual browser plans/assets, concrete trusted routes and server operations, required capabilities, Live/acquisition and rendered affordances to the installed Biff/Aleph application generation. It does not maintain a competing example Request store, business-rule machine, revision frontier or operation registry.
>
> Every enabled lifecycle action retains its exact source-owned semantic identity and is executable only through an admitted generated browser and trusted production model boundary. Missing optimism, observation, route, plan or authorized recovery never silently degrades to an anonymous lifecycle POST. Actual Claim, Unclaim, Mark on the way, Complete, Cancel and Reassign are available in independently eligible cases and remain subject to current authenticated actor/target, Organization/Membership, Request state and guarded Request/RequestAssignment transaction semantics. Historical incorporation, authoritative rejection, uncertain delivery and currently displayed Request truth are separate propositions.
>
> One user's correlated confirmed Create or lifecycle result may promptly update only the permitted local Request/card and membership scope. Remote creation and remote lifecycle changes, including those to an already-visible Request, remain advisory until a separate explicitly permitted Refresh or affected- Request recovery acquisition. Search, sort, terminal options, drafts, response interaction, details and continuity operate on justified adopted content without inventing authoritative collection membership. A one-use Refresh establishes the actual scoped observation, installation and coverage needed to discharge its corresponding debt; a stale response, failed swap, unrelated local result or reconnect cannot erase uncovered or later obligations.
>
> Each protected execution, target, transport, subscription, grant and recovery owner has bounded resources and current physical effect rights. Retirement prevents obsolete callbacks from installing or rolling back newer justified state. Actual model commits are not undone by failed notification or rendering, and unknown outcomes and disconnected interest gaps follow their declared qualified recovery rules.
>
> Independent HumanHelp product intent and real multi-client Chromium/browser, public model, XTDB/host, negative-boundary and physical observation evidence establish only their properly scoped conclusions. Formal finite checks, conditional abstract proofs, source admission, operational qualification and deployed correspondence never impersonate one another. Full acceptance requires every promised capability—including the separately selected and tested response-flow experience— and the applicable safety, recovery, evidence and substrate gates to be satisfied for the identified application and runtime profile.

This is the Part VIII **target-system invariant**, not an assertion that current HumanHelp or Gesso source, existing test definitions or an R5.1 editorial review has already certified the physical application. Further whole-Part VIII and whole-plan reconciliation may sharpen cross-part terminology, assumptions and proof obligations without changing the independent product acceptance principle.

Part IX — Compositional certification

This part specifies how Gesso checks, composes, and preserves assurance about the semantic artifacts and applications defined in Parts I–VIII. It makes the certification boundary concrete: which judgments are checked, what evidence can establish them, how dependencies are derived, and what remains conditional on runtime enforcement or an explicitly accepted contract.

The specification describes the intended system from its foundations. It does not assert that every judgment or correspondence mechanism is implemented. A supported certification profile has an executable contract and conformance suite; an unsupported judgment cannot be made available by giving it a reassuring name. Operational admission, compositional certification, external formal assurance, and multi-node deployment retain their separate scopes.

A certification profile names the exact propositions it promises, its source and implementation subjects, authorized assumption classes, concrete admission stage, required evidence and observable rejection behavior. A lower profile can remain useful when a stronger judgment is unavailable; it is not an implicit substitute for a requested stronger profile. Full application assurance additionally needs the independent product, physical writer, authenticated execution and deployment obligations owned by later parts. A checked abstract theorem about an issued plan never authorizes a database commit or proves the current browser has installed that plan.

Examples use ordinary data notation. Unless explicitly described as executable source for a named language, their purpose is to expose a contract. Writing a certificate-shaped map, matching a fingerprint, or displaying an accepted report does not cross an admission boundary.

Governing compositional contract

A downstream claim may rely only on propositions actually established by its checked dependencies and on assumptions explicitly admitted by its selected policy. Every premise retains its subject, scope, applicability, and evidence class. Neither a namespace's reputation nor an upstream artifact's general “verified” label supplies a missing proposition.

In particular, the downstream consumer must identify the **specific** required proposition, the upstream exported guarantee, its type and subject, and any still-open premise. A matching proof object is insufficient when its schema, rule source, declared operation catalog, application generation, actor/subject binding, executable lowering or selected runtime contract has changed. Exact source identity is a change detector; compatible evolution must have its own checked relation and cannot be assumed from equal names.

Judgment results distinguish rejected construction, an accepted unconditional judgment within a stated formal model, a checked conditional derivation, a finite bounded result, a trusted/opaque assertion, and a runtime-observed fact. A passed conditional abstract checker never becomes an unconditional proof merely because all required source files were present. A composed judgment inherits the applicable open premises and trust of its required dependencies; it does not average them into a stronger green status.

Certification concerns a claim about a subject. It is not a universal property of a namespace, function, application, or certificate object. A rule can be well typed while its business meaning is wrong. A choreography can have a checked projection while its server realization violates the model contract. A finite model can preserve an invariant while an omitted physical writer breaks it.

The certified construction path must make these distinctions inspectable and make unsupported substitutions fail before the protected downstream construction or effect. It must also remain useful: declarations and public model contracts are authored once, and Gesso derives the routine identities, obligations, dependency edges, proof candidates, and diagnostics.

The checked construction path must begin from the real source-owned declarations and the selected application/host graph, not an auditor's second hand-maintained list of ”approved” operations. Every exported judgment has a specified coverage obligation over its relevant language: terms, branches, required operations, relationships, message contracts, effects, initial states and declared termination/recovery cases as needed. A checker may establish closure for its **inspected** subset; it cannot infer that arbitrary Clojure bodies, raw XTDB capabilities or external processes contain no uninspected business-policy writer.

Source completeness and intended product meaning are separate obligations. An internally consistent machine that has removed HumanHelp Reassign may pass the remaining derivations while failing the independent product capability oracle. An omitted rule, dependency or protected writer cannot be excused by a proof DAG that correctly checks every submitted node. The scope of a failed or unknown coverage check must remain visible in both the certification decision and the next downstream diagnostic.

Full-language applications remain first-class

Ordinary Clojure and ClojureScript remain available under #req-full-language-construction. A closed proof language is a tractable reasoning fragment, not a replacement application language. Arbitrary algorithms, libraries, recursion, effects, and opaque policy functions can participate through declared contracts on the same managed construction path.

An opaque component identifies its inputs, results, effects, relevant state and authority dependencies, implementation binding, and the assertions on which dependent claims rely. Purity, determinism, termination, or bounded resource use are required only when the claimed property needs them. A callback's type or name cannot establish those properties.

The framework proves supported surrounding properties even when one component is opaque. For example, command identity preservation, required result variants, unchanged frame fields, and retired-effect exclusion need not all depend on the truth of an Organization policy predicate. A conclusion that does depend on that predicate's implementation remains explicitly conditional on its contract.

Unsupported proof syntax is rejected by that proof profile. It is not silently executed as host code, silently replaced by an assumption, or used to downgrade a requested strong build. The author may deliberately bind a supported opaque contract or select another profile; the resulting claim and accepted premises must be visible.

A policy predicate that decides permission, disclosure, ownership, lifecycle transition or command incorporation must be accounted for in the semantic subject, through a checked dependency, a runtime-enforced condition or a named opaque contract. If Gesso discovers an unmodeled decision in the implementation subset it actually inspects, it reports a semantic-policy escape with the affected proposition and repair choices. Silence while examining only a structured rule AST is not evidence that hidden policies do not exist in an uninspected Clojure body. Human-facing policy rationale and independent tests can challenge whether the **right** rule was declared; neither automatically becomes a theorem in the proof kernel.

Opaque contracts remain useful at the edge of a narrower theorem: Gesso may still establish command identity, result-variant coverage or frame safety that do not depend on the opaque implementation. Claims that **do** depend on the predicate's truth, purity, termination, current actor or database consistency inherit its specified proof or trust premise. The requested assurance profile determines whether that open premise is permissible.

Three semantic responsibilities

Responsibility	Contract
Executable semantic core	Defines and executes admitted application transitions and abstract effects from current state and input/even
Proof kernel	Checks derivations in a small, closed inference language under explicit premises. It creates no runtime fact c
Dedicated semantic checker	Establishes a specified judgment such as rule typing, dependency closure, projection admissibility, or exact f

These responsibilities support one **canonical semantic vocabulary** but expose three different acceptance paths. The executable core interprets admitted transitions and effects, with runtime enforcement for current principal, transaction and physical resource rights. The proof kernel checks closed derivations over named propositions and premises; a derivation supplied by the compiler does not become true because the compiler generated it. A dedicated checker can establish a well-defined alternative judgment without pretending it was checked by the proof kernel. Each result records which checker/source version and trusted mathematical or interpreter assumptions it depends on. Testing the kernel against itself is not an independent soundness oracle.

```
One source-owned HumanHelp Claim:
  Request model declares current actor, Request, assignment and
    Organization/Membership dependency contracts.
  Gesso derives a guarded operation, Choreo projection and the
    applicable proposition/coverage obligations.
  A small proof checker may establish a conditional abstract
    invariant for the declared transition subject.
  A concrete XTDB/host boundary separately checks the current
    guarded write and independently permitted disclosure.
  Browser installation establishes its own target/generation effect.
  A certificate-shaped map cannot substitute for any of those steps.
```

These responsibilities can share canonical terms and versioned primitive semantics. They must not develop competing meanings for the same construct. Sharing code does not prove correctness, however, and it is not independence when a test oracle simply invokes the checker being tested.

The canonical semantic definitions are executable specifications: accepted term schemas, interpretation, judgment inputs and outputs, side conditions, and negative cases. They may be implemented in focused namespaces such as `gesso.assurance.rule*` and `gesso.assurance.proof.*` and the Choreo semantic modules. A logical `gesso.semantic` surface, if exposed, organizes those contracts; it must not create a parallel interpreter or require a wholesale namespace move.

Certified admission, composition, and failure behavior

Certified construction is a checked **boundary**, not a shape convention. After a stage requires an admitted semantic interface, its supported public consumer must not accept an unexamined map with copied `:verified?`, `:proof-status` or subject digest as a replacement. Appropriate implementations include private issued handles, checked constructors and freshly validated certificate references; the design does not require nominalizing every ordinary Clojure value. Ordinary cooperating use must not accidentally cross a required certification gate.

The construction sequence is conceptually raw declaration, normalization, elaboration, complete obligation generation, appropriate checking, qualified certificate/admission, downstream composition and installed realization. Not every profile requires a proof-kernel certificate, but every claimed stage must have its own actual checked admission. A failing required obligation yields no value of that **certified** artifact class; an intentionally partial/conditional diagnostic is a different artifact whose status cannot masquerade as fully admitted. The first boundary with enough evidence reports an exact typed failure and the downstream artifacts not issued.

A dependency graph may have recursive executable behavior, but its justification relation cannot use a cycle of certificates that establish each other's premises. A recursive theorem needs a specifically checked inductive, guarded or fixed-point rule. A certificate's exported semantic interface contains only the guarantees permitted for downstream use, together with its assumptions, evidence class and applicability scope. It is not permission to inspect arbitrary private implementation data and silently promote it to a proposition.

Proof sidecars, checked reports and trace diagnostics are allowed to decide **build-time admission**. Once the executable meaning has been admitted, optional proof data cannot mutate its actual transition relation or grant new runtime authority. Source/plan identity accompanies the issued artifact, but that identity alone is not proof of matching Clojure, XTDB, browser or transport execution. A proof kernel may validate a conditional derivation while a separate source-to-executor or deployed-host correspondence obligation remains open.

The accepted subject becomes stale when a required declaration, implementation binding, checker contract, dependency, browser build, installed handler or runtime premise changes beyond its declared compatibility relation. A trusted consumer must revalidate the relevant current facts and invalidate dependent claims; it must not accept a saved green report from a previous assembly. Conditional reuse may avoid redoing unrelated checks when identities and all required applicability premises remain genuinely valid.

Non-vacuous boundaries and honest failure reports

Positive controls accompany every rejection family. A certified-path test must admit a genuinely well-formed Claim, a supported opaque contract and an appropriate narrow-profile construction, while rejecting a malformed rule, absent required guard/capability, forged proof, false source identity and an unjustified stronger certificate. Rejecting every operation or granting every request is not evidence of correct composition.

The report distinguishes malformed syntax, unsatisfied semantic judgment, unsupported requested profile, absent obligation coverage, open but permitted assumption, stale application binding, failed trusted runtime premise and unknown physical correspondence. It identifies the exact proposition, original source, first applicable check, dependents not built, safe remedy and confidential facts withheld from the diagnostic audience. An unobserved backend writer or incomplete independent product requirement does not become success because its name is absent from the graph.

Claims, judgments, evidence, and applicability

The following distinctions are mandatory throughout construction and reporting.

Concept	Meaning
Raw declaration	Unadmitted source data, even when its representation passes a shape check.
Semantic subject	The exact normalized object about which a claim is made, with its interpretation and dependency ide
Proposition	A well-formed statement in an identified logic or contract vocabulary. Well-formedness does not esta
Judgment	The result of a particular checking relation over explicit subjects, environments, premises, and evider
Obligation	A required judgment derived for a subject and requested claim/profile.
Derivation	A proposed justification using recognized inference rules and bound premises.
Assumption	A scoped proposition accepted without having been proved by the current construction.
Certificate candidate	Serializable source/evidence offered for checking; its claimed status is untrusted.
Admitted certificate	The result of the required checking and admission boundary for a stated claim and profile.
Report	Copyable explanation of judgments and dependencies; not an admission or execution credential.
Runtime evidence	Evidence about a particular invocation, observation, resource, or effect at its actual boundary.
Semantic category / admitted value	A meaning whose actual checked-construction or trusted acquisition boundary was crossed; same sh
Assertion	Intended policy or proposed fact supplied by source; declaration alone does not establish truth.
Guarantee	A checked proposition under named premises and an explicit evidence class.
Semantic identity	Versioned normalized interface identity; distinct from proof, source location, and implementation fid
Applicability	The identities and conditions under which a checked conclusion remains usable in a particular comp

A guarantee is a checked conclusion together with its remaining premises. Calling it a guarantee does not remove those premises.

Semantic category, admission and truth remain distinct

An EDN map that looks like a Request observation is still raw data until the applicable trusted read, constructor or checker actually admits it. The semantic category describes its intended meaning; admission establishes that the required construction boundary was crossed, not that all facts represented by the value are proven. A concrete authoritative observation also retains current principal, read scope/completeness, basis, disclosure and installed-acquisition premises.

```
same-shaped value
  ≠ admitted semantic category
  ≠ established proposition about the subject
  ≠ presently authorized runtime effect
```

Gesso constructors, model operations, checked projections and acquisition boundaries ordinarily produce these categories automatically; application authors do not manually wrap every ordinary value. A checked downstream API rejects copied `:valid?` flags, fabricated nominal values, old handles or a serialized report where an admitted value is required.

The certificate candidate remains untrusted even if its structure matches an admitted one. The admitted certificate establishes only its exact checked proposition, context, assumptions and profile. Separate judgments still govern proof-kernel soundness, completeness of generated obligations, source-to-executor correspondence and actual physical effects.

A judgment identifies more than a Boolean result

Every judgment must identify:

- its kind and semantic interpretation version;
- the exact subject and relevant input/type/context bindings;
- the proposition or obligation addressed;
- the checker, proof rules, or interpretation procedure used;
- the checked evidence and its scope, including finite bounds where applicable;
- inherited premises and any checked discharge/substitution;
- the result and structured reason when the requested judgment is unavailable.

The result vocabulary distinguishes established, conditional, counterexample, invalid, unsupported, missing evidence, incomplete/resource-exhausted, and inapplicable/stale cases as appropriate to the judgment. These need not all be one public enum, but they cannot collapse into one success/failure flag.

A malformed proof is not a counterexample to its conclusion. A timed-out search is not a proof that the property is false. A sound conditional theorem can remain valid while its premises are not established for this deployment.

An exact finite result concerns the enumerated state/operation space and chosen initializers; it is not an unbounded result. A supported unbounded theorem from an abstraction needs the separately checked or admitted abstraction/refinement relation. Static typing, finite exploration, conditional abstract theorem, trusted implementation premise and runtime-observed effect never become interchangeable outcomes.

A trusted consumer must know whether ”conditional” names a **checked** implication with open premises, merely an asserted contract, or a copyable candidate result. Accepting an explicit assumption under the chosen policy is not discharging it. A timeout cannot establish falsity, and a malformed proof cannot refute a well-formed underlying proposition.

Evidence classes remain attached to propositions

Use Part I's assurance classifications. In particular, distinguish checked derivations, exact finite results, assembly verification, runtime-enforced instance facts, trusted or application-asserted contracts, and test evidence.

A report should express classification, assumptions, and applicability as separate axes. “Conditional” is not an excuse to omit which checker established the conditional implication. “Runtime enforced” must say whether the evidence is a contract requiring enforcement, an installed enforcing mechanism, or an observed successful check for one invocation. Those are different propositions.

A static certificate can establish that an admitted receive transition preserves a knowledge invariant if its runtime premises hold. It cannot establish that a future browser has received a particular observation. A contract for a Claim executor can require current authorization; its certificate cannot supply Alice's current Membership.

A particular physical read and its trusted read boundary establish only their scoped facts. A postcommit Request snapshot does not prove the transaction-internal prestate, historical attribution to C42, complete writer inventory, or a recipient's DOM adoption. An authenticated receipt may attest to origin/bytes under its issuer trust contract; it does not prove that the issuer's business assertion is correct.

Evidence classifications are not a single numerical strength ranking: a conditional proof may establish a general abstract relationship while a particular guarded XTDB read proves narrower but indispensable current eligibility. A root claim preserves the distinct propositions it requires rather than announcing one ”most verified” status.

An immutable obligation ledger with derived status

Keep original obligations and supplied evidence distinct. The source-derived ledger records what the claim requires. Checked evidence records how particular obligations are established or which premises remain. A report computes status from those bindings instead of deleting inconvenient obligations from the ledger.

Adding a certificate may establish certificate-admission or dependency-accounting obligations. It does not automatically discharge implementation correspondence, physical writer coverage, or checker soundness. Unknown new obligation categories remain visible and unresolved until a profile explicitly classifies them.

The selected policy may accept a named premise, but acceptance is not logical discharge. An unresolved mandatory obligation cannot be converted into an accepted premise by an elaborator's failure handler.

The ledger is derived from the original declared construction and any independent capability/product obligations. It verifies coverage of the relevant operation catalog, initial conditions, transition cases, relationship/guard dependencies, effects, and

required negative/recovery branches; physical writer coverage and correspondence are added when the selected claim needs them. Valid derivations for every **submitted** obligation do not certify omitted obligations or a deleted product capability. A model with Claim but no required Reassign fails the independent HumanHelp oracle even if its closed Claim theorem succeeds.

Each entry retains original obligation identity, semantic subject, current source/dependency version, checker/host boundary, required proposition, result/evidence class, open or discharged premises, applicability and dependent conclusions. Rechecking a changed source re-derives the current ledger, preserving historical evidence as history while invalidating affected live conclusions. Unchanged unrelated propositions may be reused only when their own current identity and applicability really remain valid.

```
For invariant "at most one active primary per Request":
  initial configuration preserves uniqueness
  Claim preserves uniqueness
  Unclaim, Reassign and other declared writers preserve uniqueness
  physical writer scope and abstract/XTDB relation are qualified

Checking Claim alone never discharges omitted Reassign, a hidden raw
  XTDB writer, or complete guard/absence semantics at commit.
```

Minimal semantic interfaces and ownership

A semantic interface exports the commitments downstream reasoning may use: types and variants, relationships, preconditions and postconditions, relevant state/effect footprints, outcomes, authority and disclosure requirements, observation contracts, and named premises. It does not expose every private implementation detail or serialize a namespace for convenience.

A minimal semantic interface is the only contract surface downstream reasoning may use. It names its public owner, normalized terms and versions, nominal type/presence/nullable meaning, input/output, effect/authority/disclosure scope, assumptions and justified guarantees. A downstream proof depends on the exported **required** commitments, not arbitrary incidental implementation fields or filesystem layout.

Exact semantic dependency identity is the conservative default. A later checked compatibility relation may allow changed interfaces without violating a dependent's closed-world assumptions. A digest or same operation name alone does not prove that relation. Source reformatting need not invalidate meaning-level evidence, while a changed implementation function, current provider, handler, or browser binary may independently invalidate concrete correspondence even if its abstract public contract did not change.

The hierarchy follows semantic ownership, not file layout:

Interface or evidence subject	Typical checked commitment
Schema/type	Fields, optionality, nominal categories, variants, representation constraints.
Domain/relationship	Well-formed relationships, typed paths, refinements, selected invariants.
Rule	Supported evaluation semantics, Boolean result, safe reads/refinements, declared dependencies.
Public authoritative operation	Inputs, outcomes, guard/write dependencies, command policy, effect and recovery contract.
Choreography	Knowledge, provenance, authority placement, communication, branches, continuations, and resource oblig
Projection	The named global/local correspondence property over specified observables.
Executable plan	Checked lowering/binding to the admitted projection and runtime format.
Application assembly	Closure of required declarations and compatible realization bindings.
Browser build/served asset	Correspondence under the generated-entrypoint and qualified build/serving contract.
Concrete render/install	Checks on the actual representation and current physical target/effect.
Root application claim	Proposition-specific closure of public model, Choreo, projection, generated runtime and application realiza

Public model interfaces remain the cross-model boundary. The certificate graph must not reach into Organization's private FX code to obtain authority that its public contract does not export. Inspection of implementation source for correspondence is permitted, but does not turn private internals into an implicit public dependency.

Schemas containing arbitrary Malli predicates do not all become a closed theorem language. The supported schema fragment can be translated and checked. Opaque predicates retain named validator/implementation contracts. Concrete validation of one value proves only the relevant instance result under that validator's contract, not universal data validity or the intended business meaning.

Implementation correspondence, current dependencies and reuse

HumanHelp Request Claim consumes the **public** Organization, Membership, User and Location semantic contracts. Gesso may check the rule subset and explicitly assume the opaque Organization implementation's conformance where a stronger profile permits it; it may not reach into private Organization FX data and treat implementation details as a public authority guarantee.

A relationship interface can establish path typing and cardinality declarations but not prove the current existence of every referenced record, current authorization, or transaction-guarded absence. Trusted acquisition and current model/XTDB guards must establish the needed concrete facts at their physical boundaries.

Likewise a checked projection is not automatically correct compiled JavaScript; a source-closed ApplicationAssembly is not automatically the installed Ring/Biff handler; a matched browser stamp does not prove HTMX behavior; and a current card does not prove a historical command result. Reuse of abstract contracts and reuse of concrete execution evidence have different identities and invalidation rules.

A copyable report, unsigned cached certificate or purported signed claim re-enters as untrusted data unless the applicable rechecking, issuer-authentication and dependency boundary is satisfied. A signature authenticates the named issuer/bytes when its key contract holds; it does not itself establish the proposition. In-process opaque handles help prevent accidental substitution by ordinary Clojure consumers but do not confine equally privileged JVM code or transfer admission across serialization.

Unqualified opaque callbacks, raw XTDB writers and outside host effects remain explicit trust or open coverage premises. The framework can still establish unaffected propositions without promoting a partial root report into a full release certificate.

Paired positive and adversarial interface judgments

The acceptance matrix requires a genuine positive and a nearby negative at the **same** relevant admission boundary: checked public Claim interface versus equally shaped raw map; current Organization identity versus stale dependency; full Claim/Reassign obligation catalog versus omitted Reassign; valid conditional opaque premise versus forged `:discharged` flag; issuer-checked proof versus copied certificate description; properly qualified transaction read versus a caller-crafted report; and a supported narrow profile versus unsupported strong-profile downgrade.

Each rejection identifies the first responsible checker, the exact affected subject/obligation, expected and actual category, which downstream values were not constructed, and an actionable safe repair. No result may claim that a hidden writer does not exist merely because the selected enumerator did not inspect it. A checker that rejects every candidate fails the positive gate.

Closed typed rule language

The baseline rule language is a small data AST with explicit reader declarations, nominal type information, strict Boolean behavior, and no embedded host evaluation. It is independently useful for guard fragments, invariants, model checks, and atoms inside the proof language.

An application may use ordinary functions outside this fragment. The fragment's closed syntax exists to make the claims about it precise, not to make unsupported application logic impossible.

This bounded **certifiable** fragment is not the only language in which application authors may write rules. Gesso can elaborate supported Clojure source or accept explicit typed rule data, provided the generated AST binds the original source, public types, dependencies, translation semantics and applicable preservation evidence. A manually copied predicate with an agreeing example result is not original-source correspondence.

R4's broader target includes typed field/relationship access, nominal variant predicates, deterministic derivations, explicit capability/role facts and exhaustive closed-variant match. These are **extensions**, not unacknowledged baseline operators. Each new construct must register closed syntax, typing, evaluation, failure, effect/reader footprint, normalization and checking obligations. An extension that invokes arbitrary Clojure remains an explicit opaque contract; Section 09.04 owns its detailed bridge.

Three different rule judgments

Judgment	Established conclusion	Still unestablished
Syntax	Supported data AST, exact operators/arity/readers and resource bounds.	Nominal type correctness or policy tr
Intrinsic type	Strict Boolean result under named types and safe binding refinements.	Available knowledge, current authorit
Concrete interpretation	Boolean value for one admitted immutable binding environment.	Universal theorem, correct human po

A separate Choreo-point admissibility judgment requires actual participant knowledge, provenance, authority and observation-basis scope (Parts II–III). Operation contract and actual XTDB guards (Part IV), finite/inductive properties (Part IX), and implementation correspondence (Part X) remain separately checkable. A valid rule evaluated to true cannot silently discharge any of them.

Source shape and baseline operators

A rule declaration has an exact reader set and expression. For example, this is only the open-state conjunct of Claim eligibility:

```
(def open-request-rule
  {:readers #{:request/status}
   :expression [:= [:read :request/status] [:literal :open]]})

(def open-request-types
  {:request/status
   {:type :enum
    :name :humanhelp/RequestStatus
    :values #{:open :claimed :on-the-way :done :cancelled}}})
```

The actual type declaration is derived from the production public schema. This example's `:done` is the HumanHelp terminal state; “completed” is display language, not another persisted variant. Checking this conjunct does not check the complete Claim policy.

Real Claim additionally needs trusted actor/target identity, Request and assignment state, current Membership/Organization/Location policy, guarded absence, command binding and permitted authoritative effect. Those may be represented by further source-owned typed rules, public model contracts or named opaque dependencies. A theorem about this one conjunct must not certify a complete Claim policy or a catalog that omits a separately promised HumanHelp operation.

The baseline grammar is:

Form	Meaning and constraints
`[:literal value]`	An admitted scalar or finite homogeneous scalar set under the literal/type profile.
`[:read reader-id]`	Read one declared binding from the same immutable evaluation environment.
`[:present [:read reader-id]]`	Test binding presence without evaluating an absent value.
`[:not e]`	Strict Boolean negation.
`[:and e1 e2 ...]`	At least two Boolean operands, evaluated left to right with short circuit.
`[:or e1 e2 ...]`	At least two Boolean operands, evaluated left to right with short circuit.
`[:implies p q]`	Rule-level conditional evaluation: evaluate q only when p is true.
`[: a b]`= and `[:not a b]`=	Equality/inequality under compatible declared semantic types.
`[:< a b]`, `[:< a b]`=, `[:> a b]`, `[:> a b]`=	Ordering for the explicitly supported ordered type; the baseline numeric domain is integers.
`[:in item finite-set]`	Membership with compatible element type and enum-domain constraints.

The initial language permits plain bounded vectors, qualified reader identifiers and literals consisting of nil, Boolean, mathematical integer, string, keyword or a nonempty finite homogeneous set of such scalars. A future empty-set expression must specify its element type; an empty untyped literal is not inferred from host collection behavior. Expression ASTs admit no host functions, arbitrary object, custom comparator, executable metadata, macro invocation or ambient Var. External input is decoded by a non-evaluating EDN reader and validated before use; in particular no reader-eval or arbitrary tagged constructor is admitted by this certified syntax.

The declared grammar checks unary `:literal`, `:read`, `:present`, `:not`; binary comparisons, equality, `:in` and `:implies`; and two-or-more-operand `:and` and `:or`. Its selected version specifies finite node, depth, arity, readers, set and literal budgets. Exceeding one of them is a typed incomplete/rejected result, not a Boolean value or invitation to run unrestricted Clojure. Current numeric limit constants in the reference implementation are not timeless design requirements.

The reader set equals the actual syntactic footprint. Missing and unused reader declarations are rejected in this baseline. A declared read footprint is still only a footprint over the semantic source; it does not prove the absence of hidden reads in an unrelated opaque implementation.

Reader type declarations have exactly the same keys as syntactic readers; no implicit reader acquisition is permitted. A nominal enum has a qualified identity and explicit nonempty finite domain. A raw keyword resembling a variant is not automatically a value of an unrelated nominal enum. Typechecking rejects unknown variants, mismatched nominal equality, ordering over unordered types, heterogeneous membership and non-Boolean connective operands.

Optionality and nullability are independent: absent, present nil and present false differ. A read of an optional binding requires a path that established presence. A non-null-required operator additionally needs a path that established non-nil when the field is nullable. Section 09.04 owns the detailed refinement, normalization and path-join calculus; this section establishes that type checking and interpretation share the same reader identity and left-to-right semantics.

A rule's exact syntactic reader footprint does not certify that a separate Clojure implementation has no hidden database, clock, network, random or authentication dependencies. Such dependencies are either source-derived and covered, explicitly opaque, or remain open/unsupported.

Baseline value types are Boolean, integer, string, keyword, and named closed enum, with independent optional and nullable attributes. Relationships, records, UUIDs, money, dates, and additional nominal types enter through explicitly specified schemas/codecs or later registered extensions. They cannot be silently reduced to interchangeable strings/keywords when a proof relies on their distinct meaning.

The language and its literal/set limits are versioned. Empty sets or broader collection expressions require defined typing behavior before admission; they are not inferred from host coercion. There is no implicit recursion, arbitrary function call, query, clock read, random value, or ambient principal lookup.

A program may be syntactically valid but intrinsically ill typed: for example, `[:< [:read :request/status] [:literal 10]]` cannot compare RequestStatus enum to an integer. Presence and non-nil checks must dominate their dependent reads in the specified short-circuit path. A valid rule may still be inadmissible where a browser has not learned an authoritative Request state or where an operation requires a different principal and trust boundary.

Evaluation and totality

The evaluator consumes a validated binding snapshot containing required readers and any present optional readers, with no undeclared bindings. Present nil and absent are different. Boolean operators require Boolean values; ordinary Clojure truthiness is not the rule language's semantics.

A well-typed rule in this total baseline must evaluate to a Boolean for every valid binding in its declared domain, subject to the interpreter/primitive correctness premise. Input rejection is separate from evaluating a valid rule to false. Resource limits may stop checking/evaluation but cannot yield a successful truth result.

`:and` evaluates operands left-to-right until false; `:or` until true; and `:implies` checks its consequent only when the antecedent is true. These are observable order and error semantics, not merely Boolean identities. A optimizer, normalizer or CLJS lowering must preserve guarded-read order unless its stronger rewrite is separately proved or qualified.

The interpreter validates the entire binding snapshot: all required keys present, no undeclared keys, correctly typed values, accepted enum members, nonnullable rules, and valid optional readers. A missing required binding or nil for a nonnullable field rejects; present false is not absent, and present nil cannot be silently interpreted as false. It returns a typed observation about one provided snapshot, never an assertion that the snapshot came from a trustworthy database or authenticated session.

The baseline integer meaning is mathematical integer comparison. It does not implicitly certify JVM fixed-width arithmetic or JavaScript Number behavior. Portable execution chooses a checked representation/range/codec and rejects unrepresentable values, or uses a separately qualified exact representation. A JVM/CLJS agreement test over small integers cannot prove all integer behavior.

A portable profile binds representation, codec, valid range and rejection at the boundary. Mathematical integer comparison can use separately qualified BigInt/exact arithmetic or a proven safe-range restriction. JavaScript Number rounding, JVM fixed-width overflow, numeric string coercion, and silently changing integer into floating point are forbidden when the checked rule requires exact integers. String, keyword, nominal enum and optional/nullable serialization also require explicit correspondence where JVM and CLJS representations or coercions differ.

Evaluation to true is a concrete value result. It is neither a universal theorem nor authority to commit. The acquisition and transaction boundaries must establish where the bindings came from and whether every dependency used by the guard is protected at the actual commit.

Source coverage, admission and adversarial acceptance

An intrinsically checked syntax/type/interpreter result is copyable descriptive evidence, not a certified theorem or an admission credential. A normalized rule fingerprint names its interpreted subject; neither it nor a proof-status label grants authority. Stronger requested profiles must establish their own original-source and proof/physical correspondence, rather than reclassifying this report.

The acceptance corpus pairs an eligible valid open-state Claim conjunct with an incompatible enum/integer ordering, unknown variant, omitted/unused reader, wrong enum identity, optional access without presence, nullable comparison without a non-nil guard, literal set type mismatch, unknown input key, absent/nil/false binding, overflowing or unrepresentable numeric transport, unsupported host form and incomplete resource budget. Each failure records the first responsible syntax/type/interpreter or trusted acquisition boundary and expected diagnostic.

Independently, a complete Claim certificate that only models `:open` must be rejected if the promised policy also depends on actor, Membership or assignment guards. Deleting Reassign from both the formal language inputs and the available HumanHelp implementation also fails the independent product capability oracle, even if every remaining rule is intrinsically valid.

```
Valid intrinsic rule for R17:
  status is :open          → evaluated conjunct true

Wrong rule type:
  status < 10              → rejected at intrinsic typing

Wrong distributed placement:
  browser branches on unobserved authoritative status
                           → rejected at Choreo admission

Wrong physical authority:
  stale helper issues Claim → current model/XTDB guard rejects
```

A reject-everything implementation cannot pass the valid positive tests. An unsupported rule is reported unsupported or bound to a separately admitted opaque contract, never silently executed by host Clojure and then labeled a verified rule.

Path-sensitive presence and null refinement

A certified rule's type judgment is indexed by its exact reader-type environment, binding/observation identity, and the evaluation path on which its refinements hold. A refinement is a justified **conditional fact about one observed binding**, not a reusable current fact about the underlying database entity or another participant. It cannot migrate between Request IDs, actor sessions, changed provider instances, basis/progression epochs, an older read and a replacement read, or precommit and postcommit states without a separately established compatibility/correspondence judgment.

The baseline uses two distinct refinements for a reader `r`: `:present` (the supplied immutable binding snapshot contains `r`), and `:nonnull` (that same present binding is not nil). Required fields may be present by their validated type contract; optional fields require an established presence fact before `:read` can be evaluated. Nullable values may be present nil; ordering and any other non-null-only operation additionally require a non-nil fact. Present false and integer zero remain present values, not Clojure-falsey absence. A typed premise about a reader must use the same nominal type and declared source as the dependent expression.

Short-circuit path transfer and joins

For Boolean expression `e`, the checker derives conservative facts valid **if the same expression evaluates true** and facts valid **if it evaluates false**. These are conditional judgments, not claims that either outcome occurred. The evaluator and proof/lowering interpretation must agree on the exact short-circuit order:

Form	Path where later child is evaluated	Facts available there
`[:and e1 e2 ...]`	Each preceding child evaluated true.	Facts established by those true evaluati
`[:or e1 e2 ...]`	Each preceding child evaluated false.	Facts established by those false evaluati
`[:implies p q]`	Consequent `q` evaluated only if `p` is true.	Facts established by true `p`.
`[:not p]`	Evaluation of `p` precedes Boolean negation.	True/false outcome facts are exchanged
Ordinary equality/inequality	Only when both operands are safe under this exact environment.	A checked nil comparison may establish

At a point reached through more than one possible predecessor, retain only facts valid on **every** incoming reachable path, unless a separate checked disjunctive/refinement rule justifies more. A false `:and` may have stopped at any one of several operands; it does not generally establish that **every** conjunct is false. A true `:or` similarly does not make every disjunct true. Unreachable or contradictory branches must be recognized by an explicit valid judgment, not used to infer arbitrary policy truth or to erase reachable side conditions.

For an optional nullable score, this order is safe:

```
{:readers #{:case/score}
 :expression
 [:and
  [:present [:read :case/score]]
  [:not= [:read :case/score] [:literal nil]]
  [:≥ [:read :case/score] [:literal 10]]]}
```

The first condition establishes only presence; the second rules out present nil. Reordering the comparison before either guard is a type error, not a false business-rule result. A valid alternative uses the false-path refinements of disjunction:

```
{:readers #{:case/score}
 :expression
 [:or
  [:not [:present [:read :case/score]]]
  [:= [:read :case/score] [:literal nil]]
  [:≥ [:read :case/score] [:literal 10]]]}
```

The disjunction returns true for absent or present-nil score and compares only when the score is present and non-nil. This is a **different business policy** from the conjunction; successful typechecking of both is not a license to substitute one for the other. Its exact typing and interpretation must be checked against absent, nil, false, zero and valid integer inputs. In particular, a Boolean truthiness test cannot act as a presence or non-nil guard.

R4's future richer refinements—closed variant discrimination, path cardinality, actor/subject facts, relationship existence, current capability and trusted observation basis—require dedicated sound transfer and join rules. A path-sensitive rule that is intrinsically safe to evaluate may still be inadmissible to a browser that does not know the premise or to a server that has lost the premise's current trusted source.

Conservative normalization

Canonicalization preserves every distinction relevant to the **typed interpreter's actual meaning**, including failure and defined evaluation paths, not merely the mathematical Boolean truth table of some idealized expression. The baseline may normalize plain map/set insertion order, equivalent exact mathematical integer representations, and omitted type flags that explicitly default to false. It preserves operator and operand order, nominal enum names/domains, reader IDs, optional versus nullable, missing versus present nil, and source/interpretation context where identity requires them.

Do not commute, sort, flatten, fold, eliminate, distribute or rewrite short-circuit terms merely because a Boolean algebra identity would preserve a truth table. Such transformations can change safe read order, the first error, resource use, whether an optional binding is accessed, branch-refinement facts or source obligation coverage. The permitted baseline equivalences have a specified finite meaning; stronger rewrite rules require an independently checked semantics-preservation theorem, or an explicitly admitted trusted transformation premise with its exact assurance class.

Normalization of author syntax and normalization of an already-typed, closed term are different boundaries. Original source must first be validated and elaborated under a named translation contract; accepting a cached report or arbitrary value bearing a fingerprint is not equivalent to rechecking original syntax and nominal types. If a certificate refers only to a normalized term, correctness of the source-to-term elaboration remains a separate checked or stated trusted premise. The normalizer must not issue its own proof of its soundness.

A canonical identity binds at least the rule language, nominal reader-type declarations, interpreter semantics, normalization/encoding profile and the semantic subject. It need not include whitespace or diagnostic source locations when those are semantically irrelevant. Conversely, changing an enum domain, optional or nullable flag, reader name, operator, argument order or encoding version cannot be hidden by stable source-path text. Structural identity is not a logical equivalence decision, a certificate of policy truth, or an issuer's authority to execute an operation.

A dependent abstract proof may be reused only when the required current normalized semantic interface or an explicitly checked compatibility relation still holds. A changed Clojure callback, model provider, deployed function body, installed Ring handler or browser asset can invalidate implementation correspondence even when a pure abstract rule fingerprint remains identical.

Opaque policy and relational extensions

An application may have a genuine correctness-relevant policy outside the closed rule AST. A production Organization eligibility function, external service or ordinary Clojure predicate can participate through a named contract owned by its public model/extension interface. It identifies the semantic subject and implementation boundary, input and output types, result variants, effect and disclosure scope, termination/cancellation expectations, current actor/Request/Organization dependencies, and the propositions whose checking depends on that opaque implementation. Its actual trust class and any open conformance premises remain explicit.

Checking a supplied output Boolean proves at most that a **provided** value satisfies the declared typed interface under the checker's interpretation. It does not establish who supplied it, whether the producer was the declared function, whether the function was deterministic, pure, terminating, confined to declared effects, or correct about the current business policy. A timeout request is not proof that arbitrary JVM code or network I/O can be forcibly interrupted. Declaring an effects inventory is not proof no extra effects occurred, and a claimed pure function is not automatically an enforceably pure one.

A successful opaque policy result may enter a closed rule as a typed reader only under an explicitly qualified **provenance and freshness** contract. It must be bound to the trusted actor, actual input/subject identities, issuer/implementation, relevant observation basis or transaction snapshot, authorized disclosure, and known authoritative dependency set. For a policy that determines whether Claim may commit, either the policy is evaluated (or safely validated) inside the same guarded transaction/linearization boundary, or a checked equivalence/validation protocol establishes that its cached result remains correct at that boundary. Merely seeing `:organization/eligible? true` in browser data, an earlier request, or an unauthenticated cache does not satisfy the guard.

Where the opaque policy depends on mutable Membership, assignment, Organization or Request facts, each dependency is protected by the appropriate current read/guard/transaction contract. A result may be stale even if its boolean type and original source fingerprint remain valid. A remote result cannot declassify protected information or grant authority by being marked `:runtime-enforced` in a caller-produced map.

Semantic-policy escape and coverage

The finished assurance system makes correctness-relevant policy visible inside the source and implementation scope it actually checks. A decision that determines authorization, ownership, disclosure, lifecycle permission, command incorporation, protected domain effects or terminal-state rights must correspond to an admitted rule, a public certified dependency, a current runtime-enforced premise or a named opaque implementation-conformance assumption.

If Gesso detects a policy-changing branch in a source subset it claims to inspect, but finds no corresponding semantic rule or named trust boundary, it reports a **semantic-policy escape** with the affected operation, proposition, source location, dependency and possible repairs. An explicit opaque premise is not a detected unresolved escape, but its implementation conformance remains open or trusted under the chosen profile.

Conversely, not observing an escape in an **uninspected** arbitrary Clojure function does not prove that no hidden policy branch or effect exists. No whole-program completeness claim follows from inspecting only rule ASTs, a declared reader set, or a closed operation registry. The checker identifies the exact program surface for which it has established coverage, and records external bodies and writers that could change the proposition. An independent HumanHelp requirement such as Reassign cannot disappear just because every remaining closed predicate has a correct derivation.

A business rationale, human-readable example or heuristic review finding is useful policy-audit evidence, not a kernel premise without explicit declaration and checking. Changing a business rule after such review is a visible semantic decision, not an implementation-only repair that can retain old certificates.

Rich relational/transition judgments

Relational schemas, quantified relationship constraints, finite-family rules, guarded multi-record transitions and richer closed variant matches may use dedicated interpreters and checkers rather than inflating the minimal Boolean interpreter. Each extension supplies the closed grammar, nominal types, data/access/authority scope, finite or unbounded domain, initial/transition meaning, side conditions, normalization, proof obligations and source-to-executor correspondence needed by its conclusion.

A nominally valid relationship path does not prove that its endpoint currently exists, that a reference is visible to this actor, that the query is complete, or that no concurrent writer can change it. A universal relational theorem over a modeled domain does not establish all deployed XTDB writer/absence behavior without the corresponding physical refinement and writer-coverage premises. A typed keyword such as `:relationship/valid` is not an inference rule merely because an application uses it in a guard.

The final composition keeps independent categories of evidence: intrinsic rule validity, checked abstract theorem, named opaque assumption, current guarded execution, and installed browser/host correspondence. No one of these silently promotes the others or becomes the business owner's chosen policy.

Positive, negative and changed-source acceptance

The conformance corpus runs genuine accepted typed optional/nullable rules beside variants whose presence/non-nil guards are reordered, whose `:or` false path is unsound, or whose refinement belongs to a different Request/snapshot. It exercises opposite policy results of the two valid score examples, including missing, nil, zero and boundary integer scores.

Normalization tests admit only the declared equivalences (map/set order, exact integer representation and default-false descriptor flags). They reject commutation of guarded conjunction, changed enum domains, optional/nullable flags, reader renaming, altered operator/threshold, unsupported host metadata and cache/report substitution. A matching pure rule identity paired with a changed execution implementation must still invalidate relevant concrete correspondence.

For opaque policies, tests distinguish valid named contract + current protected issuer/inputs from forged browser eligibility, stale Membership or Request, wrong implementation, undisclosed effect, withheld output, unavailable service, uninterruptible work and a merely requested timeout. A legitimate opaque execution may succeed under an expressly weaker profile without acquiring a strong proof or blanket authorization.

A hidden policy branch in the **inspected** source fragment is reported as an escape; an arbitrary body outside inspection is an open conformance premise, not a falsely passed exhaustive search. A true relational conclusion in an abstract finite model is separately checked against actual guarded database absence, cardinality and writer interference. Every negative has a nearby legitimate positive at the same first enforcing boundary; rejecting all rule expressions or every Claim is not acceptance.

Executable judgment contracts

The design requires executable definitions for the following judgment families. The notation is descriptive; production terms use closed, versioned data schemas. Their implementation may be split among focused checkers, but their meaning and admission result cannot be left to each caller's interpretation.

A judgment is a versioned, typed relation between named **original** semantic sources, its input context and the exact proposition it establishes. For each admitted family the standard defines: accepted source and elaboration form, nominal variable/subject identities, required context and dependencies, the deterministic meaning of the conclusion, finite or unbounded scope, checking and inference rules, coverage obligations, permitted assumptions, statuses, resource limits, and its current applicability. Implementation namespaces may divide these responsibilities, but a caller cannot supply a preferred truth/status field and thereby select the result.

The finished Gesso API retains a distinction between a raw declaration, a parsed or typed **description**, a checked derivation or dedicated checker result, a source-bound admitted certificate, and a runtime observation/effect. Each stage needs its own authority boundary. An immutable Clojure map, matching fingerprint or old proof report is not a substitute for an issued current checked result. Application authors normally express ordinary Clojure/Gesso rules; proof terms and ledger bookkeeping are generated and checked by Gesso rather than separately hand-maintained by HumanHelp authors.

Judgment	Inputs and conclusion
Rule typing	Reader types and path refinements establish a Boolean expression with safe reads.
Rule evaluation	A valid typed rule and admitted binding snapshot produce its Boolean value.
Propositional derivation	A fixed subject/context and named premises entail the exact goal.
Guarded-transition consequence	The declared relation matching pre/input/post entails the exact transition conclusion.
Invariant preservation/induction	Initialization and every declared transition establish and preserve the invariant.
Distributed admissibility	A rule/action is permitted at the specified role and program point.
Projection/lowering correspondence	The actual projected/lowered subject satisfies the named correspondence relation.
Certificate composition	Checked evidence establishes required claims for the actual dependency closure and selected profile

Each judgment also declares the scope it **does not** establish. An intrinsic Boolean rule does not imply distributed role knowledge; a propositional derivation cannot prove the truth of an atomic business fact from its type; abstract guarded consequences cannot assert that a specific XTDB transaction ran; and a matching generated plan does not prove its deployed browser artifact is the matching physical program. Required bridge judgments connect those distinct subjects, preserving their open assumptions instead of laundering them into one ”verified” label.

Obligation generation is independently checked

For each claimed artifact class, a source-derived checker must establish that the required obligation ledger is complete for the original normalized subject and requested profile. Validating every derivation the elaborator volunteered is insufficient. The ledger covers every declared operation, relevant reachable branch, declared reader/effect, message and continuation case, initial state, protected resource exit, public semantic dependency, and independently required product capability **within the selected guarantee**. Requiring a dependency to be covered does not imply that the checker has inspected an arbitrary Clojure implementation body or all physical writers; the latter remain separate coverage/conformance premises.

A missing branch cannot make a theorem easier. If a source retains both an approved and not-approved Claim branch but the proposed obligation list contains only the approved branch, coverage checking rejects the stronger artifact before certificate issuance. A proof about a Claim-only operation catalog also does not satisfy HumanHelp's independently required Reassign capability. The accepting boundary derives or checks required coverage against the original source and public capability contract, not merely against the elaborator's own submitted proof tree.

A judgment over a truly restricted model may legitimately pass while a larger required application proof remains open. Such a result records its actual subset, justified exclusions, opaque external calls, relevant writer set, completeness assumptions, and whether the expected next stage may consume it. In particular, ”unreachable branch” requires an appropriate checked unreachability argument, not a missing test or a hard-coded assumption that a UI button is disabled.

Results, failed proofs and assurance scope

Judgment outcomes distinguish: checked with no logical premises under the specified calculus; checked conditionally with named premises; a model counterexample to a stated property (where the dedicated checker can establish one); invalid syntax or derivation; unsupported language/assurance request; incompletely checked resource-limited exploration; mismatched or stale source/dependency; and unavailable physical/trusted evidence. A checked conditional conclusion remains conditional even if an operator has accepted the premises for a particular deployment profile.

A counterexample to a **claimed model invariant** is not the same thing as a malformed proof term, an incomplete search, or a misstated HumanHelp business policy. Likewise the absence of a finite counterexample is not evidence about unenumerated states, external writers or the actual deployed browser. Diagnostics identify the exact proposition, original source, first responsible checker, relevant context, status, dependencies, and what must be repaired or qualified; they redact protected data for the recipient. A result accepted only under a named opaque premise does not get reported as a discharged unconditional theorem.

Each judgment has positive, negative, unsupported, and resource-limit conformance cases as applicable. Parser rejection, a checker report, certificate admission, and runtime authorization remain separate products. The rest of this part fixes the baseline proof rules and the composition obligations rather than relying on an indefinitely deferred universal judgment calculus.

Small proof kernel and dedicated proof checkers

The proof kernel checks proposed derivations independently of the elaborator's claimed success. It accepts original source terms under a closed rule catalog, revalidates their subject/type bindings, and computes the conclusion and premise dependencies. It does not accept an alleged earlier checking report as a proof.

“Small” refers to a bounded logical responsibility and an inspectable trusted implementation. It does not mean one universal checker must contain rule typing, all model exploration, database refinement, browser effects, and every future theory. Dedicated checkers can establish different judgments and participate through explicit bridges.

The kernel's trusted surface includes the chosen logical interpretation, its parser and typed subject representation, every primitive inference rule and side condition, the checked implementation and relevant runtime/representation assumptions. Keeping the kernel small makes review possible, not automatically sound. Checker meta-soundness and, where stronger assurance is requested, an independent mathematical or mechanized argument and adversarial conformance evidence are separate obligations. A proof-producing elaborator and a checker that simply calls back into that same elaborator cannot count as independent fault containment.

For proof admission, the kernel recomputes the expected conclusion and inherited premises from the actual original proof steps. It does not trust user-provided `:conclusion`, `:valid?`, `:proof-status`, a compiled-report hash, or a previously checked but changed derivation. The certificate issuer subsequently checks required dependency/current-source bindings and the requested assurance scope; the kernel does not issue an application execution credential.

Baseline propositional language

The propositional layer has `true`, `false`, typed Boolean atoms, negation, conjunction, disjunction, and implication. Each atom contains an independently well-typed rule declaration. The proposition binds one exact semantic subject and one nominal variable context; declared variables equal the actual union of atom readers in the accepted source.

All proposition constructors have closed grammar and exact arity: `[:true]`, `[:false]`, a typed `[:atom rule]`, unary `[:not p]`, binary `[:implies p q]`, and at-least-binary `[:and ...]` and `[:or ...]`. Every atomic rule is rechecked against its declared nominal variable context, reader types and original source; undeclared and unused variables are rejected. The finite source and proof budget is part of the checker profile. An unsupported term cannot execute arbitrary Clojure or be interpreted as an implicit theorem.

An atomic rule denotes a Boolean proposition only under its specified interpretation. Its typecheck does not supply the atom's truth; logical constants remain distinct from runtime evidence that an Organization policy actually holds. In particular, introducing a **premise** `[:false]` produces a conditional derivation dependent on that premise, not an unconditional contradiction or proof that the system's current state is inconsistent.

Propositional connectives express logic. They are not the rule interpreter's control-flow operators. In particular, a presence atom in the antecedent of a propositional implication does not make an otherwise ill-typed consequent atom safe. Put that guarded evaluation inside one well-typed rule atom, or use a separately specified refinement-aware proof judgment.

This independence rules out a subtle unsoundness: the proposition `[:implies [:atom present?] [:atom unsafe-read]]` is not well typed if the second atom's reader is optional and has not been safely refined **inside that atom**. A logical implication can reason about two already well-typed Boolean propositions, but does not import the rule interpreter's short-circuit presence facts across separate atom boundaries. Later refinement-aware proof constructors must make the stronger relation explicit and prove its typing/evaluation meaning.

Logic here permits ordinary reuse of established propositions; it does not imply consumption or duplication of runtime resource leases. Section 09.06 owns resource-sensitive transition consequences and the proper treatment of actual one-use authority, and Part V owns physical effect/ownership lifetimes.

The minimal inference catalog is:

Rule	Premises	Conclusion and side conditions
Premise	Named P in the declared premise environment.	P, retaining that exact scoped premise as open.
Truth introduction	None.	True.
Implication reflexivity	None.	P implies P, with exactly matching well-formed P.
Conjunction introduction	P1 through Pn.	Their ordered conjunction, with at least two operands.
Conjunction elimination	Conjunction P1 through Pn.	The child at a checked index.
Implication elimination	P implies Q, and P.	Q, with exact antecedent matching.
Disjunction introduction	P.	A well-formed disjunction whose checked indexed child is e

Each rule is a **constructor-specific checked inference**, not a generic ”trust this consequence” function. Premise introduction preserves the exact named premise dependency; truth introduction uses no premises; implication reflexivity proves only `[:implies P P]` for the exact same well-formed P. Conjunction introduction maintains the declared ordered conjuncts; elimination uses a checked valid index. Modus ponens requires the exact antecedent match and ordered implication/antecedent inputs; disjunction introduction checks the requested child position and well-formed whole disjunction. These rules do not assert independent truth of an atomic `P` and contain no magic closure under semantic or arithmetic equivalence.

This catalog intentionally does not include an unchecked ”assert” axiom, arbitrary atomic truth, a parser shortcut for falsehood elimination, natural-deduction hypothetical discharge, untrusted rewriting or a Clojure callback that can register additional inference tags. A further rule is admitted only after a separate rule semantics/soundness obligation, source schema, exact subject/variable matching, conformance corpus and checker-version upgrade.

Proof steps have unique identities and can reference only earlier steps. The final checked step must establish the exact goal. Premise dependencies are computed through the used steps. The baseline rejects unused proof steps and unused declared premises, making an allegedly relevant premise or hidden dead proof fragment visible rather than silently carrying it along.

Each step has a unique identity, a recognized rule-specific exact source shape, only references to earlier steps, an explicit conclusion bound to the same proposition subject/context, and verified side conditions. The final step must equal the stated goal under the checking relation. The checker computes all **reachable** step dependencies from the final goal and rejects unused declared steps or premises rather than letting dead proof fragments conceal a forged assumption or apparently discharged obligation. This also prevents simple cyclic, self or forward references from appearing as an admissible finite proof DAG.

A valid propositional derivation with no named premises is a no-premise judgment under this stated logical calculus. It is **not** automatically a theorem about the executable state machine or the physical database, and it does not establish the kernel's own meta-soundness. A proof with named premises retains each exact one even when the premise is a Boolean constant or an opaque business-policy statement. A dependency graph of certificates may itself not close a circular logical justification without an explicitly checked induction or fixed-point rule.

This catalog has no arbitrary atomic-truth rule, no arithmetic solver, no hypothetical-context discharge, and no rewriting by an unproved equivalence. Additional rules require explicit source schemas, interpretation, side conditions, soundness obligations, and conformance cases. They cannot be installed through an application-supplied “trusted inference” callback.

What a checked derivation actually certifies

The checked conclusion has a versioned logical subject and nominal variable context. It is logically about those propositions under the accumulated premise environment. The framework separately checks whether the original Gesso model/Choreo/operation, the generated abstract projection, and the actual application runtime are the **same relevant semantic subject**. Source-to-proof elaboration, proof-to-operation relation, and operation-to-deployed XTDB/browser behavior are distinct bridges. If any required bridge is absent, the derivative claim remains conditional/open.

A trusted application consumer validates the current certificate's issuing checker, source dependencies, selected profile and applicability. A map reconstructed from serialized EDN with a copied certificate ID is not an admitted checked value. Even a valid local proof certificate grants neither an authenticated session principal, current database permission nor a physical browser installation permit.

Fault containment against a buggy elaborator

The architecture is tested with defects injected at the **actual proposed** boundary, not fake isolated checker data invented to look like the application. A large elaborator may generate an invalid derivation, an incomplete obligation set, or valid proof data for the wrong normalized source. These are three different failures with different responsible checkers.

```
1. Invalid derivation:
   Helper Claim branch asks for private-review fact.
   Elaborator emits :knowledge-from-global-syntax.
   Small kernel: unknown rule; no knowledge theorem.

2. Omitted required obligation:
   Normalized source retains Claim and Deny branches.
   Elaborator proposes evidence only for Claim.
   Source-derived coverage checker: missing Deny/continuation.
   No certificate for the requested closed construction.

3. Valid proof for wrong executable:
   Certified subject S1 and proof are legitimate.
   Lowering actually emits S2 but copies S1's stamp.
   Source/projection/executable correspondence rejects.
   The proof kernel did not inspect the emitted plan.
```

These cases must not be credited to a kernel that does not inspect the relevant defect. In particular, all volunteered proof steps can be correct while the required branch/operation ledger is incomplete; an entirely valid kernel theorem about S1 can coexist with a faulty or substituted executable S2. A stronger deployed correspondence claim must also check the actual served browser assets, installed Ring/Biff handler, guarded XTDB writer boundary and authorized observation where those are required.

Positive and adverse proof-kernel acceptance

At minimum acceptance exercises the concrete conditional derivation below and a truly premise-free truth/reflexivity proof; correct conjunction/ disjunction introduction/elimination and exact implication elimination; false as an **open** named premise; two distinct nominal contexts; a malformed or omitted rule argument; an unknown rule; a forward/self reference; unused step/premise; a copied source report or forged certificate; a changed checker version; oversized input/terminated checking; and a well-typed rule atom improperly relying on refinement from a different proposition.

Each negative names the first enforcing boundary and preserves a nearby admitted positive. An unsupported arithmetic relation is not ”proved false” when the kernel lacks an arithmetic rule; it is an unsupported judgment until a dedicated checker provides a separately qualified bridge.

A concrete conditional derivation

The following source states a deliberately small conditional claim. The variables are abstract Boolean inputs; they are not authoritative HumanHelp facts.

```
(let [p [:atom {:readers #{:example/eligible?}
                :expression [:read :example/eligible?]}]
      q [:atom {:readers #{:example/allowed?}
                :expression [:read :example/allowed?]}]]
  {:subject {:kind :operation
             :id :example/conditional-rule
             :interface {:operation :example/conditional-rule}}
   :variables {:example/eligible? {:type :boolean}
               :example/allowed? {:type :boolean}}
   :premises {:example/eligibility p
              :example/policy [:implies p q]}
   :steps [{:id :step/policy
            :rule :premise
            :premise :example/policy
            :conclusion [:implies p q]}
           {:id :step/eligibility
            :rule :premise
            :premise :example/eligibility
            :conclusion p}
           {:id :step/result
            :rule :implication-elimination
            :from [:step/policy :step/eligibility]
            :conclusion q}]
   :goal q})
```

The conclusion is Q under the two named premises. The derivation does not prove that eligibility holds, that the policy implication is the right product rule, or that either is satisfied by a particular request. Removing the eligibility step, changing its subject, changing the policy antecedent, or relabelling the goal must fail the appropriate check.

This is an authentic **conditional** proof of Q, not proof of either assumption's actual validity, correctness of the business-owner's policy, the original application elaboration, execution of an appropriate Request transition, or browser knowledge. The separate qualification path must still establish required premises at their true model/transaction/acquisition boundaries and prove or explicitly assume the applicable implementation correspondence.

An atom whose rule compares an integer with 7 remains an atom to this kernel. The kernel cannot infer that `x > 7` entails `x >7=` merely because a human sees the arithmetic relationship. A dedicated arithmetic/typed-semantic judgment can establish that implication under its own explicit scope and bridge contract.

Facts are not linear capabilities

Logical facts and concrete resources have different structural rules. A reusable assertion can be cited twice in a proof. A current one-use Refresh grant, command-attempt authorization, browser target lease, transaction capability or cleanup obligation cannot be copied, widened, consumed or transferred by an inference rule that mentions its name. The actual resource has a trusted issuer, owner, scope, generation, live instance and permitted effect set. The execution layer checks all those conditions at use.

A resource-sensitive checker may employ separately typed linear/affine/ownership contexts with explicit rules for issuance, transfer, split, join, consumption and retirement. An affine resource may permit discard where a linear one requires accounting; neither is freely duplicable. An abandoned request may transfer a recovery obligation without preserving permission for its retired callback to install DOM. The proof's claim about such a rule does not issue any token or establish its physical use.

```
Logical fact: "G7 permits at most one scoped Refresh read."
Actual G7: issued to Alice, cohort C, execution E2
  first qualified read consumes its read authorization
  later SSE advisory cannot reuse G7
  retired E2 cannot reinstall a stale response
  any remaining repair obligation has an explicit owner
```

The logical kernel can establish conditional propositions about resources; actual authority issuance, one-use enforcement and physical effect cleanup are separate runtime judgments.

Ordinary propositional facts can be reused in derivations. A runtime authority capability, effect lease, resource ownership token, or one-use installation permit cannot therefore be duplicated by conjunction introduction or repeated premise use. Logical statements about such resources are distinct from the resources.

Resource judgments carry their own ownership context and consumption/retirement rules. A bridge from a checked resource proposition to a permitted runtime act must still pass the actual authority and lifecycle boundary. A proof of "the plan requires a current permit" does not produce that permit.

Guarded-transition consequence language

This is an explicitly versioned abstract relation, not a second production interpreter. Its exact subject includes nominal state/input schemas, guard, read/write sets, assignment source and frame complement. The abstract state is a selected **projection** of application data; it is not automatically the whole database or all active writers. A transition descriptor checked for shape is not yet a universally valid consequence theorem, much less an admitted physical XTDB operation.

The baseline abstract transition has a typed state projection, disjoint typed inputs, a Boolean guard, exact read/write declarations, and simultaneous assignments. Each assignment initially supports a scalar literal or a required prestate/input reader. Unwritten declared state fields are framed unchanged, including their presence as well as their value.

Its meaning is a relation T(s, x, s'):

- s and x satisfy the declared domains;
- the guard holds in s and x;
- every assigned poststate field has its specified value computed from s and x;
- every framed field preserves its declared value and presence;
- s' satisfies the declared state domain.

There is no implicit sequencing between assignments. Reading a field on an assignment's right side reads the prestate, not an earlier assignment's output. Deletion, fresh allocation, relational updates, arithmetic expressions, external effects, and opaque transforms need their own specified transition constructors. The baseline must reject them rather than pretending a scalar assignment model covers them.

The consequence checker can derive:

Rule family	Established conclusion under the matching-relation premise
Guard	The transition's exact guard held in the protected abstract prestate/input.
Assignment	The specified poststate field equals its literal or prestate/input source.
Frame	An unwritten declared field preserves value and presence.
Conjunction introduction/elimination	Compose or select conclusions for the same relation and bound context.
Equality substitution	Replace a whole equality operand using an earlier checked equality with compatible types and pre

A consequence is conditional on T matching. It does not establish that an actual XTDB transaction executed T. Optional prestate/input fields are not total value terms without the required refinement. A written optional poststate field can be a total term when the assignment establishes its presence. Framing an absent optional field must preserve absence, not coerce it into present nil.

The frame covers only the declared projection. It says nothing about unmodeled database fields until a separate physical footprint/correspondence argument establishes that relationship.

Original-source, total-term and physical boundaries

A consequence judgment has the form:

```
For all compatible s,x,s':
  T(s,x,s') ⇒ Consequence(s,x,s')
```

An accepted `:guard` fact asserts the abstract guard only **assuming** the triple satisfies T. It does not prove that the current Request is claimable, the actor is authorized or the real transaction committed. An assignment conclusion refers to an explicitly written poststate field; a frame conclusion preserves both presence and value for an unwritten state field. Equality substitution uses a prior checked same-type, same-phase whole-operand equality, not arbitrary predicate rewriting, numeric solving or a fresh authoritative observation.

Optional prestate/input fields are partial terms without an applicable checked presence refinement. Written optional poststate fields may be total when the assignment establishes their presence. An unwritten optional field retains absent versus present-nil identity under the frame, even when both might appear as nil after an unsafe Clojure lookup. Readers cannot borrow refinements from a different Request, actor, basis or transaction.

The checker re-parses original source, verifies actual source-bound proof steps and final goal, and retains the matching-relation premise. Proof reports and structural fingerprints remain copyable descriptions; stronger certificate admission independently rechecks its source and premises. A valid finite witness for one transition is not a universal proof.

The closed scalar baseline does not model deletion, allocation, relational cardinality, arithmetic state updates, multi-document writes or external effects. Adding these requires distinct typed transition constructors, sound consequence rules, declared read/write/frame relationships and source-to-executor correspondence. Actual HumanHelp Claim/Reassign can involve guarded Request **and** RequestAssignment documents; a proof over RequestStatus alone cannot establish unique active primary assignments.

Positive cases for guard, literal assignment, prestate copy, frame equality and same-field optional absence accompany negative cases for forged guard, wrong post literal, a written field called framed, absent input treated as a total equality, mixed nominal phases, unsupported proof rule and stale source. Rejection occurs at the first responsible checker and cannot be credited as a proven unsafe real database state.

Invariant preservation and conditional induction

Let `Inv(s)` denote the conjunction of all declared invariants `I_j`. The per-operation preservation judgment for `I_j` may rely on **every** `I_k` holding in the common prestate:

```
Initial(s)            ⇒ I_j(s)       for each j
Inv(s) and T_i(s,x,s') ⇒ I_j(s')     for every i,j
```

This is a **simultaneous inductive invariant**: it is not automatically a proof that each clause preserves itself in isolation. The checker derives the required base cases and full operation-by-invariant cross-product from the original common catalog rather than trusting a volunteered list. A catalog's closedness is relative to the declared source, not evidence that every production writer or required HumanHelp capability was included.

An operation catalog declares one state interpretation, one initial condition, one invariant set, and an exact set of abstract operations. It does not duplicate the invariant independently in each operation's proof input. The checker derives the required base and preservation obligations from the catalog.

For each invariant I, the argument requires:

```
Base:          every declared initial state satisfies I.
Preservation:  for each declared operation T,
               I(s) and T(s, x, s') imply I(s').
Conclusion:    I holds in every state reached by a finite sequence
               of those declared transitions from a declared initial state.
```

Every invariant and every operation must be accounted for. An unsupported base case or one unsupported preservation case leaves that invariant's induction claim open. A conjunction of unrelated successes is not a whole-invariant proof.

Useful preservation constructors include unchanged-frame reasoning, literal postcondition reasoning, typed conjunction, and same-origin copy equality. Evidence from different constructors may be combined only by an explicit checked composition rule over the exact same transition, invariant clauses, variables, and remaining premises. Unioning two success sets is not such a rule. Conversely, there is no architectural reason to forbid a sound conjunction proof merely because its children were established by different dedicated checkers.

To establish a mixed invariant from different families, a new checked composition rule must bind all constituent judgments to the same transition, invariant clauses, typed pre/input/post identities and remaining premises. Merely unioning source-family success sets is unsound. Likewise a checker cannot make a missing operation disappear from the independent required-capability ledger. A supported narrow family may succeed while the wider invariant stays open.

The original source-derived obligation ledger remains intact: conditional proof checking does not rewrite open initial-state existence, complete operation coverage, source correctness, checker soundness, authorization or physical refinement obligations into discharged facts.

The induction conclusion is about the declared abstract machine. Applying it to production additionally requires initialization coverage, relevant physical writer coverage, and a correspondence relation for actual transitions. Those premises remain explicit. Membership revocation, imports, administrative writes, and background jobs cannot be omitted because they lack a browser control.

The mathematical induction argument does not by itself require an inhabited initial predicate, but an empty initial state set yields a vacuous result. Reports must expose satisfiability/existence evidence separately, and useful application completion requires appropriate positive initialization and enabled-operation scenarios. Safety induction also does not establish termination, fairness, or eventual user Refresh.

Non-vacuity, physical writer correspondence and product scope

An empty initial-state class makes a universal base implication formally vacuous. The report must separately expose initial-state satisfiability, at least one legitimate initializer, actual production initialization coverage and eligible enabled-operation histories. A checker that recognizes literal false may reject that obvious case; failing to detect a subtler contradiction is not evidence of existence. A mathematically correct safety proof does not itself provide user-visible progress or live resources.

For transfer to real Gesso/HumanHelp executions, the application must qualify an abstraction from actual Request, RequestAssignment, Membership/Organization and other relevant data into the declared abstract state. The selected XTDB version's transaction isolation, guard, absence, atomicity and current prestate conditions, along with every writer that could affect the invariant, must match a modeled transition or be independently proved irrelevant. Background repair, imports, administrative writes and raw external XTDB access matter even without a browser control. An unknown/unobserved writer is an open coverage premise, not permission to certify the entire production application.

“At most one active primary assignment per Request” is a different invariant from “exactly one when the Request is claimed.” The latter additionally requires conditional existence and status-to- assignment consistency. Declaring an enum invariant over RequestStatus proves neither active primary uniqueness nor absence of concurrently committed competing assignments. A meaningful dual-Claim test challenges the real guarded transaction; a Reassign test separates authorized actor from eligible target. Those physical tests support correspondence but cannot prove unbounded abstract induction.

A completed abstract safety induction also does not imply termination, fairness, eventual notification or inevitable user Refresh. Under the selected advisory-only remote adoption policy, a user who never Refreshes may safely retain older adopted content. Claims of eventual command reconciliation, outbox work or resource cleanup need their own qualified availability, retention, scheduling and recovery premises.

```
Admitted abstract conclusion:
  all declared reachable states satisfy I

Still separate:
  a valid initial state actually exists
  all required user operations are declared
  all relevant physical writers are covered
  concrete XTDB updates refine T
  implementation/checker rules are sound
  browser effects and recovery are current
  user eventually invokes Refresh
```

Positive and adversarial checks

Genuine accepted cases include frame preservation, literal postcondition, same-origin equality, one supported mixed-clause invariant, a valid initial consequence, and complete declared conditional induction. Malformed or open cases include missing base, one unproved operation, changed shared invariant, shrunk operation catalog, impossible initial class, forged conditional status, mixed-family proof-ID union, invalid optional frame, wrong state subject, unmodeled primary-assignment writer, and a Request-only status proof asserted as a RequestAssignment uniqueness certificate. Every rejection records its actual first boundary

and keeps a nearby valid positive. Unsupported proof constructors remain open rather than falsely proving or refuting the claim; reject-all is not certification.

Exact finite checking and bounded diagnostics

An exact finite result is a judgment about an **explicit finite transition system**, not about every state that happens to fit a convenient Clojure map. The checked source identifies nominal state/input domains, initialization, every admitted transition and nondeterministic choice, state comparison, invariant or temporal property, and observation semantics. The checker records whether its exploration was exhaustive relative to **that** finite subject. It cannot silently substitute a finite sample for a declared infinite type.

The result classification is not a Boolean:

Result	Supported interpretation
Exhaustive safe	Every reachable state/transition of the specified finite model was explored under the stated semantics and
Genuine counterexample	A valid finite-model initial state and enabled transition trace violate the property, with a replayable source
Incomplete	A state/case/depth/time/memory budget was reached, or the language/transition coverage could not be es
Invalid or unsupported source	A syntax/type/domain/interpretation requirement fails.

For safety, a complete algorithm enumerates all declared initial states and enabled transition alternatives, adding new states until the reachable set reaches a fixed point without dropped successors. If an enabled successor falls outside the enumerated state universe, either the model's declared transition/closure contract rejects it or a checked abstraction maps it to a covered state. It cannot silently be discarded and still report exhaustive success. Visited-state deduplication itself uses the correct nominal and presence-sensitive state equality. A bounded trace-prefix enumeration is not closed-world exploration unless an independent completeness argument justifies the bound.

The finite result also names its exact property and quantifier. Safety assertions about every reachable state, trace-invariant assertions, deadlock freedom, fairness, termination and leads-to are different judgments. Temporal conclusions need the appropriate finite-state temporal semantics and scheduling/ fairness assumptions; merely checking an invariant on visited states does not prove eventual delivery, user Refresh or progress after a process failure.

A completed exact finite exploration establishes its stated property over that finite model. Its evidence includes domains, initial states, transition relation, property, exploration algorithm/profile, completion status, and counterexample when one exists. A fixed trace-depth bound is a bound on explored prefixes, not proof of all paths unless a separate completeness argument establishes it.

Domains, closure, non-vacuity and model relevance

A finite enum or Boolean domain can be covered exhaustively when its nominal declared values and interpretation are exact. A bounded set of integer, string, Request ID, timestamp, actor or database-state representatives is generally a **sampled subset**. Even when every selected sample is explored, the report must identify the restriction and avoid claiming correctness for unrepresented values. Numerical boundary choices, nil/absent values, role/tenant scope, competing RequestAssignment rows, retry attempts and multiple client generations are meaningful dimensions; omitting them can hide an intended counterexample.

An initialized finite model can be empty or can disable every transition. Formal safety may then hold vacuously. Stronger product acceptance requires separately checked or witnessed eligible initial states, meaningful enabled operations and, where relevant, contention, recovery or failure paths. An invariant that says ”no two active primary assignments” cannot be certified for HumanHelp by a finite state made only of one RequestStatus field; the abstract property needs the assignment relation or a justified mapping to it.

The finite checker distinguishes **relative model completeness** from **application-source coverage** and **physical correspondence**. The source-owned operation catalog must include every required declared transition/branch for the selected claim; production coverage additionally needs all relevant initializer and writer paths, including administrative/import/background/raw writes, and a justified state abstraction. A closed catalog that intentionally or accidentally omits Reassign can be model-checked exactly while still failing an independently required HumanHelp capability.

To transfer safety from a finite abstraction to a larger concrete system, a checked relation must explain concrete initial-state inclusion and how each relevant concrete step is simulated by an abstract allowed step (or invariant-preserving stutter), and preserve the observed property. Under-approximate samples cannot establish this relation by themselves. The state-space reduction, symmetry or partial-order method must expose its own soundness premises. An abstract counterexample requires concretization or a separately checked feasibility argument before it becomes evidence of an actual XTDB/browser exploit.

A finite Boolean truth-table check can establish the validity of one instantiated propositional implication for all valuations of its distinct atoms. That does not prove a universally quantified rule schema over all formulas/arity or prove the production checker's implementation correct.

Truth tables, typed atoms and replayable witnesses

For a **fixed well-formed propositional formula**, complete enumeration of every Boolean valuation can decide its truth-table validity under the specified propositional semantics. Treating typed atoms as independent Boolean variables is appropriate for checking a tautology: validity for all valuations implies validity for any typed interpretation that maps atoms to Booleans. It does not show that all independently assigned valuation combinations are reachable from a real typed Gesso source or current Request snapshot.

For a candidate **counterexample**, a Boolean valuation in which two related typed atoms disagree might be impossible for any common input state. A typed-atom bridge therefore checks real nominal domains, reader presence/nullability, shared subject bindings and the simultaneous interpretation of correlated atoms. A sound abstract violating valuation is not automatically a feasible typed counterexample; the report labels the valuation as propositional-only until a typed binding witness or concretization is established.

Conversely, a typed counterexample must include the actual source data, qualified variable environment, initial/input bindings, evaluated rule terms, step sequence and property failure so that an **independent** interpreter can reproduce the violation. A compact counterexample with no original-source provenance or one that was merely emitted by a checker being tested is not yet independent semantic evidence. Counterexample minimization may improve diagnosis, but must preserve the original violation and replay relation rather than substituting a different subject.

A typed-atom bridge instead enumerates actual binding domains, evaluates atoms under their typed semantics, and derives the corresponding Boolean valuations. It must not fabricate impossible combinations of correlated atoms and call them reachable application states. All-Boolean-valuation validity can establish a stronger propositional tautology; a counterexample with impossible typed atom values is not automatically a concrete typed/application counterexample.

The checker must preserve distinctions among: an invalid syntactic derivation, a true proposition for a bounded finite model, a discovered model counterexample, an inconclusive timeout, and a proposition proved conditionally under named unverified premises. A counterexample does not necessarily refute the formal **inference rule schema**; it may instead reveal a false model premise or a source-to-concrete mismatch. The actual first accountable checker and source are reported with the result.

Independent bounded interpreters, differential JVM/CLJS tests, rule-instance truth tables, and mutation corpora are valuable evidence. They remain precisely scoped. Increasing the sample count or giving a module “soundness” in its name does not create an unbounded meta-theorem.

Independent oracles and evidence of rule soundness

Independence is **semantic and implementation-specific**, not a claim that two different namespaces have different names. An oracle that invokes the production rule evaluator, its normalizer, the production proof checker, or a wrapper around the same captured implementation shares those assumptions and bugs. That may still detect substituted public results, but it is not independent support for the reused rule.

The assurance inventory identifies which code and semantics each oracle shares: parser, typechecker, optional/null codec, integer comparison, transition interpreter, proof rules, step-closure algorithm, source catalog and mutation harness. A separately implemented finite oracle or test-local truth-table interpreter can independently challenge some of those layers. It must not be promoted into a complete meta-soundness result merely because it agrees on every enumerated test input. Deliberately injected unsound proof rules and transition mutants should be rejected by the specific independent oracle at the expected boundary, while an admitted positive source still passes.

A meaningful property claim about the **proof kernel** needs a different kind of evidence from checking one proof instance. The checker-soundness goal is: for every accepted well-formed derivation under the specified semantics, the conclusion follows from its computed premises. Establishing this universally requires a mathematical/mechanized rule-by-rule argument or a suitably trusted proof, together with a verified relationship to the actual parser, JVM implementation and runtime primitives. An independent finite audit may falsify an unsound rule schema with one example; surviving a finite mutation corpus does not discharge unbounded checker meta-soundness.

External theorem tools (Recife, TLA+/TLC, Quint, or a proof assistant) are **implementation aids for Gesso**, not an obligation for ordinary application authors to maintain a separate formal model. An external result is imported only with the source transition/typed semantics, exact theorem, model and version, assumptions, checked execution status, tool trust and correspondence bridge. A successful TLC exploration of a two-operation finite Request machine is valuable bounded regression evidence, not a proof about all XTDB writers or production browser/Live behavior.

Trusted checking base and fault containment

The trusted checking base is assessed **per judgment**. A proof-kernel certificate, exact finite-state result, typed atom interpretation, source/plan fingerprint and physically acquired XTDB observation may rely on different primitives and trusted execution paths. A single ”fully verified” flag erases essential distinctions and is forbidden as a substitute for the root proposition/assumption ledger. Reports remain source-bound, versioned and tamper-aware, but self-reported status or hash equality alone never crosses a certified admission boundary.

Proof search, budget scheduling and an elaborator's optimization heuristics need not live inside the small trusted kernel. Its actual admission parser, inference rule verification and primitive meaning still belong to the trusted surface. The complete application certificate additionally names the source-coverage, original-code elaboration, projection/lowering, physical writer, served browser and installed host bridges that this kernel does not check.

The checking base includes parser/decoder behavior, canonical representations, primitive interpretations, inference/checker code, dependency resolution, and the admission policy implementing the claimed judgment. A smaller kernel reduces that base; it does not certify its own correctness by being small.

The elaborator may propose wrong terms, omit obligations, or emit the wrong plan. Different boundaries catch those failures:

Failure	Required boundary
Invalid inference or changed premise	Proof checker validates the actual derivation.
Required obligation omitted	Artifact checker derives/checks complete coverage from the subject and profile.
Source normalized into a different program	Source-to-normalized correspondence or its explicit trusted translation contract.
Correct proof paired with another projection/plan	Projection/lowering correspondence over the actual payload.
Correct plan paired with wrong served bytes	Qualified build and serving correspondence.
Correct declarations paired with a bypassing handler	Installed implementation/effect-boundary evidence.

Recomputing with a captured reference implementation can reject substituted public checker results. It is useful accidental-tampering protection in the trusted process, but it is not an independent oracle for bugs in that same implementation. Multiple wrappers around the same procedure do not multiply its logical assurance.

Reproducibility, failure modes and host portability

For every finite or independently checked result, record the original source identity and canonical semantics, named domain enumeration and whether it is complete or representative, initial states, operations, property and quantifier, exploration strategy/algorithm, stopping reason, budgets, tool/checker version, trust and external assumptions, and a replayable counterexample or certified completion witness as appropriate. A duration, test count, fingerprint or green exit code without this scope cannot substitute for a claim-specific evidence record.

An exceeded case/state/time/memory budget and interrupted search must return incomplete, never safe or false by default. Invalid AST/typed domain, unsupported external operation, unresolved source correspondence and a legitimate model counterexample have distinct typed failure classes. A failed reference interpreter, missing Node process, unavailable TLC/Recife runner or different JVM/CLJS integer codec is an uncompleted check, not evidence that the application policy was disproved.

JVM-versus-CLJS portability requires actual independent host executions and comparison of the same normalized result/failure corpus; running the JVM half of a shared test file is not host agreement. Mathematical integer ranges, nil versus absent, nominal enums, short-circuit evaluation, Unicode/keyword encoding and nondeterministic iteration must be qualified for the chosen host/runtime. A finite cross-host regression test can expose divergence and support a selected portable profile; it is not proof of all host computations.

The root application accepts a requested strong assurance profile only when all **required** judgment types and obligations are discharged under their stated trust contracts. A finite diagnostic report can support testing and conditional decisions without becoming a runtime execution permit, a cryptographic certificate, an authenticated read, or proof of product correctness.

External formal evidence may establish rule-schema soundness or a checker theorem under named semantics. Its bridge must bind the theorem's statement, definitions, versions, and assumptions to the exact claim consumed by Gesso. A theorem about a logical calculus alone does not prove the parser, JVM implementation, database, or browser adapter.

Non-vacuous positive and adversarial checks

Acceptance includes a genuinely closed finite Boolean/enum model with valid reachable initial states and a proved safety invariant; a nearby source mutation producing an actual replayable counterexample; and an otherwise legitimate model whose exploration exceeds a small budget and is therefore **incomplete**.

A complete fixed-formula truth table checks an accepted tautology and a non-tautology. The latter's Boolean counterexample must not be presented as a typed application trace until correlated atoms have a satisfiable shared binding. A typed bridge must also accept a genuine correlated typed witness and reject inconsistent nominal, optional or nullable bindings.

The regression corpus independently exercises guarded frame/assignment cases, finite multi-operation orderings, abstract induction cases, propositional rule mutations, and the exact result of each declared checker rather than its display label. A deliberately faulty proof elaborator, an omitted operation or branch, altered original source and modified executable/served bytes must fail the appropriate **distinct** coverage, kernel, normalization, lowering or host correspondence boundary. A small finite checker cannot be credited for observing external XTDB writers it never enumerated.

A valid conditional theorem can retain open physical correspondence and checker meta-soundness premises without being discarded as useless. A requested stronger certificate must fail closed if any of its required premises remains unresolved. A malicious copied proof report, mismatched checker version, incomplete partial trace or all-rejecting test implementation is not acceptable evidence of completed certification. Successful local proofs and exact finite cases remain useful, precisely scoped evidence for future composition.

Distributed admissibility and projection judgments

Intrinsic rule validity is necessary but insufficient at a distributed program point. A role may evaluate a valid predicate only with the knowledge and provenance required by that use, and may perform an effect only through its applicable authority and resource boundaries.

The distributed environment includes the choreography subject, role, program point, typed value/observation bindings, knowledge, provenance, scope requirements, capability requirements, and relevant ownership state. It is generated from the admitted protocol and verified dependencies, not accepted as a caller's list of facts that happen to make the branch check pass.

Exact role-local judgment and conditional knowledge

The checked proposition identifies a role, original Choreo source, reachable program point, branch and role-local fact, with its nominal type, observation/snapshot identity, issuer, actor/tenant scope, basis/progression, resource/generation, and intended use. This is a **point-indexed** judgment, not a claim that every role can read every well-typed global fact. The verifier derives the environment from admitted construction and original producer paths; an application-supplied list of facts cannot manufacture the stronger knowledge or authority it happens to need.

A useful decomposition separates (1) definite value availability, (2) role-local knowledge of the particular observation, (3) justified provenance and disclosure strength, (4) authority for an effect, (5) required current basis/transaction compatibility, and (6) actual resource ownership. A well-typed Boolean rule, globally declared fact, historical command outcome, or copied certificate cannot discharge the other judgments by itself. A symbolic statement "after authorized O the browser knows R.status" remains conditional until O actually occurs at its trusted boundary.

The definite-facts analysis computes a **must** result: at a join, retain only facts available with compatible provenance on every reachable incoming path, unless a checked conditional/disjunctive representation explicitly retains the per-branch qualifications. A fact produced on one branch does not become unconditional at the join. Replacing an authoritative observation with a later local/asserted/communicated value does not preserve authority merely because the fact key is spelled the same. Refined presence, nullable fields and observation generation are tracked together; a refinement cannot migrate to a different Request, recipient, basis or client session without a new qualified relation.

Required knowledge and provenance constructors

The baseline distributed checker needs explicit cases for:

- entry inputs admitted under the role's entry contract;
- communication through a matching admitted send/receive contract;
- permitted authoritative observation at the declared acquisition/receive point;
- deterministic derivation from already available inputs under its checked or explicitly assumed contract;
- conditional branch refinements and conservative joins;
- explicitly named assertions whose weaker provenance remains visible.

Each constructor binds the exact subject and value observation, role, program point, source path, and any scope/basis requirements. A rule occurrence in global syntax does not give every participant knowledge of its operands. Receiving a command ID does not give a browser authority over its recorded outcome.

Consider a browser branch on a private review field. The field can be well typed and the rule Boolean, but the use must fail when the browser has no permitted knowledge path. The repair is not automatically to send that private field. The protocol may keep the branch server-side or disclose only an authorized derived outcome. A diagnosis must respect that choice and the disclosure policy.

For a conditional observation step, the checked statement is: if the declared runtime boundary admits observation O covering this subject/field, the successor role state has the corresponding knowledge with that provenance. It does not assert that O already exists, that the whole board is current, or that a future mutation is authorized.

Transport, disclosure, and authoritative observations

Participant communication and environment delivery are different constructors. A sender must actually own/know each required field and obey its disclosure contract. A receiver gains knowledge only from the matching admitted send/receive occurrence, subject, correlation, payload scope and acceptance path. Optional message fields remain conditional, and open-envelope extras do not silently become typed authoritative facts. Duplicate, delayed, dropped or replayed envelopes cannot create new model authority or a new trusted observation.

A declared `:await` alternative does not establish that an environment message arrived. A declared authoritative reread grants conditional knowledge only after a real qualified acquisition, with the exact issuer, authenticated recipient, authorized Request/field

subset, observation identity and basis/progression contract. Opaque basis tokens are compared only by their declared provider relations, not as client-controlled timestamps or universally ordered numeric counters. Source-level provenance metadata is not an authenticated witness of actual transport delivery.

The private-review example has an important positive repair: retain the protected predicate server-side, then communicate only an independently authorized public derived outcome under its own disclosure and provenance contract. Sending the private review field merely to make browser typing pass is not an admissible repair. An asserted Boolean, or a declassification rule not approved by the public model policy, cannot create recipient access to protected inputs.

Every branch, receive and environment alternative needed by the original Choreo, including rejection, no response, unknown command outcome and explicit recovery, requires a checked continuation. Unreachable exclusion needs an actual checked reachability argument; eliminating a negative branch from the proposed proof ledger is not a proof of branch closure.

Authority and transactional use

Authority placement distinguishes a browser role from a trusted server role. Claims about allowed operation invocation retain the trusted principal/capability premises. Claims about successful new incorporation additionally retain the current atomic guard and dependency obligations from Part IV.

A proof of an old observation's authenticity cannot promote it into a current commit guard. A fact that an actor knew at one point may remain known historically while no longer establishing current eligibility. Freshness, command binding, and protected prestate are separate judgments.

For outcomes, the certified model interface uses `:committed`, `:rejected`, `:already-incorporated`, and `:commit-unknown` with variant-specific payloads and permitted disclosure. Choreography and presentation separately map those results to `:accepted`, `:rejected`, and `:unknown`. A total mapping includes replayed historical success and unknown delivery; a card read cannot substitute for command-specific attribution.

Invocation, incorporation, adoption, and selective Refresh

The client may be authorized to **submit** a Request operation while still lacking current authority to commit it. The trusted server re-derives principal/role, Membership, Organization and Request scope, command identity, and guarded transaction prestate at the real model/XTDB boundary. A successful abstract proof about an earlier observation cannot establish current Claim eligibility.

Submission, authoritative disposition, and browser effect installation are distinct stages. The committed, rejected, already-incorporated and commit-unknown outcomes retain historical command attribution and variant-specific disclosure. A later Request card read does not prove that an earlier C42 succeeded, nor may a transport-lost commit be reported rejected solely because the browser never received a response. The presentation mapping also tracks accepted/rejected/unknown independently from authoritative incorporation.

HumanHelp's selected advisory-only remote lifecycle contract is a direct compositional test: receiving an SSE invalidation may increase advisory debt without adopting remote R or automatically requerying the board. A creator's correlated local Create S may be installed without importing that remote R. Collection Refresh requires its separately issued one-logical-read permission, current query/scope, physical installation and coverage evidence. A proof of conditional future knowledge cannot mint a Refresh permission or erase uncovered advisory obligations. A narrow affected- Request command recovery is not a general right to adopt a new collection cohort.

Projection and executable correspondence

A projection claim identifies global and local semantics, observables, internal steps, participants, message/correlation rules, transport assumptions, and whether it concerns trace inclusion, safety preservation, behavioral equivalence, or progress. These relations are not interchangeable.

For example, checking that each local step matches an allowed global step does not by itself prove that the composed participants can agree on branches, avoid deadlock, or make progress under failures. The composition theorem must account for the actual communication and scheduling model. Liveness claims retain their fairness, availability, timeout, and user-participation premises.

The artifact checker covers every relevant branch, receive, outcome, continuation, and terminal/resource case. It may exclude unreachable cases only under the supported reachability judgment. Omitting a troublesome branch from the proposed obligation list does not remove it from the subject.

Structural, concrete-witness, and quantified relations

These are distinct typed projection judgments, not incremental versions of one green result:

Judgment	What it can establish
Structural source/projection preservation	Every required reachable authored boundary, message, continuation, origin and terminal contr
Concrete lockstep witness	One specified global execution matches one projected multi-role schedule under exact observat
Concrete weak witness	One specified pair matches after permitted hidden/stuttering steps, with causal origin and term
Trace inclusion/refinement	Every admitted projected observable trace is matched by a permitted global trace under a state
Behavioral equivalence	The named relation holds both ways under its explicit assumptions.
Temporal/progress	The chosen leads-to, termination or fairness property under qualified scheduling and recovery

Global semantic communication may project into separate sender, queue and receiver steps. Message send, enqueue, deliver, drop, duplicate and retry are modeled as distinct physical/distributed actions when the theorem depends on them. An environment event cannot impersonate a participant receive. Role-local return is not the same as a completed global outcome; terminal compatibility must account for all roles, pending messages, admitted outcomes and resource retirement. Permitted hidden local steps can commute where independent, but a checker cannot arbitrarily reorder authoritative commits or pretend a dropped message was delivered.

For a universal result, obligations quantify over every admitted local scheduler/transport choice, or use a proved finite abstraction/simulation covering them. One successful witness proves neither every schedule safe nor that deadlock is impossible. Likewise proving each local step has some global origin does not by itself prove the independently composed participants agree on branching and termination. Fairness, network availability, user participation and crash recovery are explicit temporal assumptions.

The coverage checker derives from actual original branch/await/message/operation/continuation/terminal sources, including negative, timeout, retired-generation and recovery cases required by the claim. Correct volunteered local proofs cannot certify an omitted reachable branch or an independently required HumanHelp Reassign/Cancel/Refresh capability.

Lowering binds the admitted projection to the actual executable plan payload. Reconstructing expected output under a deterministic lowering contract and checking payload equality is a useful strategy. It still depends on that lowering contract; a shared emitter/checker bug requires independent correspondence evidence or remains an explicit premise.

Optional diagnostic proof sidecars never alter runtime transitions after plan admission. Required application-policy checks still occur at construction, installation, and the specified effect boundaries. That admission role does not turn a sidecar into runtime authorization.

Physical projection correspondence and acceptance cases

Source-checked projection, issued ExecutablePlan and actual served browser bytes have different identities and enforcement boundaries. The proof binds original Choreo subject, role-local plan, compatible runtime format, generated manifest and emitted payload. Equality with a deterministic recompilation can establish a qualified emission property, but an emitter/checker common-mode bug remains an open or independently checked premise. A copied plan stamp does not prove the installed browser runs its corresponding program.

Actual installation needs qualified Ring/Biff handler and trusted route, current session, physical HTMX/DOM adapter behavior, true transport origins, resource instance/generation and model writer boundaries. A browser trace with the same shape/order as an abstract witness is not an authenticated observation of real messages, XTDB incorporation or actual terminal state. Compiler-side proof sidecars do not authorize additional runtime effects.

The positive/adversarial corpus exercises a valid entry fact, matched required message field, proper authoritative observation and permitted server-derived public decision. Its nearby negatives omit communication, forge recipient knowledge, assert current authority from an untrusted Boolean, use private fields in a browser branch, substitute wrong actor/tenant, lose optional-field refinement, reuse an old basis/generation or forge a model outcome.

Projection cases include matched sender/receiver correlation, legal hidden-local commutations, concrete weak and lockstep witnesses, rejected send-origin substitution, overlapping receive alternatives, omitted terminal/continuation, unsupported message reordering, duplicate and retired-message delivery. The report names the first responsible static, source-coverage, dynamic witness, physical-host or authoritative enforcement boundary; a structural check is not credited with catching physical defects.

An actual valid Claim and a permitted selective Create/Refresh sequence are independent positive product controls. Rejecting all Choreographies, all Claims or all transport input is not an acceptable proof of admission correctness. A strong all-schedules result remains open until its separate simulation/abstraction and trusted transport assumptions are established.

Certified construction and certificate admission

Certified construction is a checked semantic boundary, not a declaration with an approval flag. Its subject is an exact source closure, requested proposition and assurance profile. The issuer derives required judgments from that subject, checks every mandatory local one, accounts for permitted open premises, and issues only the artifact category actually earned. A theorem about a selected abstract model must never silently become a theorem about deployed Clojure, XTDB, HTMX or browser execution.

Original-source authority and coherent construction

The supported application constructor obtains source-owned schemas, rules, operations, choreography, views, routes, dependencies and installed artifacts from the application's selected construction authority. It must not let a caller-selected digest, proof graph, report, or declared inventory redefine what source is actually assembled. A standalone proof constructor may accept a caller-selected abstract source; the resulting claim is explicitly relative to that abstract source, not the installed application.

Source admission records original declarations and diagnostic locations, semantic category, the chosen checker/profile semantics, imported interfaces, trust boundaries, callable bindings and versioned interpretation. It is necessary to distinguish a source declaration from its normalized term, its compiled implementation and the physical artifact it may eventually emit. For opaque Clojure functions, dynamic Var resolution, external effects and otherwise undiscoverable dependencies, the constructor requires an explicit implementation/coverage premise or rejects a stronger profile; an asserted source inventory is never independent proof of its own completeness.

A coherent admission owns a stable source and dependency generation. Internal checking can use stages, parallel work and caches; however, replacement of a model writer, route, rule or required proof dependency between acquisition and issuance must force consistent rechecking or fail closed. There is no legitimate issued product consisting of an old proof, a new handler and a differently versioned browser plan merely because their separate stamps look valid.

Checked pipeline and obligation ownership

Stage	Required checked result
Source admission	Identify selected original source, semantic categories, origin, trust and immutable construction scope; do not execu
Normalization	Obtain an exact canonical term under a versioned interpretation/preservation contract, keeping meaningful order,
Elaboration	Resolve names, roles, types and proposed derivations; treat elaborator outputs as candidates rather than proof.
Obligation derivation	Independently regenerate all judgments required by the original source, its claim and requested profile.
Evidence checking	Check mandatory inference steps with the appropriate small proof kernel or dedicated semantic checker, subject to
Dependency closure	Check the justification DAG and the source-derived completeness of required nodes, edges, original producers, clai
Premise accounting	Bind every inherited/open premise to exact source, scope, proposition and allowed profile, recognizing only checke
Issuance	Construct the typed admitted product only after its entire mandatory local set and accepted premise policy hold fo
Consumption	Verify the product against the consumer's own current subject, dependencies, implementation, installation and pol

This table specifies logical responsibility, not a mandatory nine-pass build algorithm. An implementation can fuse stages or reuse immutable verified subresults while preserving independent checking and precise failure reasons.

The elaborator cannot choose the list of obligations it wants to pass. The accepting checker must independently establish the required case/edge set relative to the **original** source and proposition. A structurally valid DAG which omits a difficult operation, proof branch, invariant, reachable outcome, required model policy or source-derived writer does not establish complete certification. A check confined to a declared abstract catalog must say so: real physical writer and installed-source completeness need separate inventory and conformance evidence under Parts IV, VII and XII. Source-derived obligation coverage and physical coverage are different evidence subjects.

Proposition scope, premise discipline and non-vacuity

An admissible claim binds (1) its exact proposition and subject; (2) a complete set of mandatory local judgments; (3) the checked inference/semantic rules used to discharge them; (4) required dependency guarantees and their scope; (5) every open premise permitted by the selected profile; (6) any required non-vacuous positive product control; and (7) the actual correspondence or implementation assumption where a stronger realization is claimed.

Checking an implication over no initial states may satisfy a formal formula without demonstrating a usable application. A checker that rejects every operation can be safe by vacuity without implementing Claim or Create. Where the requested claim requires inhabited initialization, an enabled positive path, all declared operations, or complete outcome variants, those are separate admission obligations. The profile must not conceal their absence inside a generic `:certified` label.

A trusted assertion is a visible assumption, not an inference of domain truth. An abstract rule, exact finite model check, conditional induction or structure check establishes only its actual scope. In particular, no such check by itself proves checker meta-soundness, arbitrary Clojure implementation equality, physical XTDB writer coverage, browser effect correspondence, or that the served program is the admitted one.

No self-certification or report-to-certificate promotion

An issuer does not accept `:valid?`, `:checked-result`, `:verified?`, `:discharged?`, a copied semantic ID, a developer-authored successful proof report or an unverified graph as authoritative checking evidence. It reruns the checker on original source or consumes an authenticated checker-issued internal result bound to the same source, claim, rule versions, dependency generation and profile. It independently checks both the derivation and whether that derivation addresses the full required obligation family.

Readable proof reports, diagnostic sidecars and fingerprints are copyable descriptions, not admission handles. The checker must distinguish claim, evidence, inference rule and checked conclusion. An author cannot establish an assumption solely by declaring its desired guarantee, nor can two certificates discharge each other's premises through a cyclic proof graph. A checked induction/fixed-point rule inside a component is different from cyclic justification; runtime retry loops are different again.

The ordinary public API may be conceptually as small as:

```
(certify! {:source application-owned-source
           :requested-profile selected-profile
           :dependencies admitted-dependencies})
```

This is a desired contract, not the name of a currently guaranteed exported function. Gesso derives normalized subjects, required dependencies, proofs and reports mechanically. Application authors write normal Clojure/Gesso domain rules and construction; they need not maintain a second model in Recife, Quint or TLA+, copy fingerprints, or hand-build proof DAGs. Where the truth

is application-owned or opaque they may declare an explicit scoped trusted premise, which remains open unless separately discharged.

Issuance classes and exact conditional guarantees

The result is typed, not Boolean. A raw declaration, admitted structural identity, checked dependency DAG, unadmitted proof report, checked abstract derivation, conditional certificate, verified application assembly, issued ExecutablePlan and installed/runtime-authorized effect are different products. A public consumer requiring one class cannot silently accept a weaker, same-shaped map or a weaker handle. The failure diagnostic explains the missing category, proof, source, dependency or enforcement obligation.

A conditional certificate may be fully admitted **as a conditional certificate** when all mandatory local checks succeed and the requested profile expressly permits each remaining premise. It cannot promise the same proposition with those premises deleted. A failed mandatory check, missing required dependency, stale source or unknown physical gate causes no stronger artifact to be issued. If a different, weaker category is useful it is returned and named separately, not smuggled through an API expecting the strong product.

The issuer's report maps each guarantee to subject, inference/checker, proof scope, assumptions, remaining obligations, evidence owner and applicable source/dependency generation. Partial diagnostics are useful but are not partial implementations of an otherwise required full certificate. Untrusted model contracts and implementation assumptions never become authoritative model facts by virtue of nominal packaging.

Process-local handles and explicit lifecycle

One supported JVM representation is an opaque handle registered by object identity after all relevant checks succeed. Registry state binds the source, exact claim/profile, dependencies, premise set and applicable checker/context. A manually constructed instance, same-shaped record, serialized EDN map or copied success report cannot create an admitted entry. A previously checked structural handle is not automatically a proof certificate.

Such a handle is local to its process and trust boundary. It is not a signature, a durable credential, a proof of arbitrary JVM function behavior, browser authorization or portable theorem. Weak references can avoid retaining dead objects; they do not establish revocation, freshness, semantic retirement, resource cleanup or concurrent-use safety. The issuer and consuming boundary must enforce synchronization, generation and invalidation rules explicitly.

This protects cooperating Clojure code against accidental category confusion and specified forged data through supported APIs. It does **not** claim to defend against hostile reflection, code/Var replacement or equivalent privileges inside the trusted JVM. A stronger attacker requires a stronger isolation or independently authenticated boundary.

Durable packages, data decoding and rechecking

A persistent package can contain original/normalized source, derivations, assumption/claim interfaces, dependency references, checker and interpretation versions, provenance and integrity material. It cannot persist an in-process handle's registration by printing it. Imported data must first be parsed as inert, bounded data; arbitrary reader evaluation, unknown tagged readers, custom executable accessors and uncontrolled graph traversal are excluded from the trusted checking boundary.

On import, validate the exact supported format and semantic/rule versions, actual source and claim, all required local judgments, transitive dependencies, premise scopes, evidence identity and the requesting consumer's policy. Reject missing, stale or cyclic references, fabricated result fields, invalid steps, unsupported tags, and incomplete finite checking represented as exhaustive. Depth, node, CPU/time and other resource budgets are part of a specified admission contract; exhaustion is not successful certification.

A signature authenticates bytes or a claimed issuer under its key/trust policy; it does not establish a theorem. Locally cached unsigned data may be usable after sufficiently complete rechecking. External packages may need authentication **and** proof checking. Neither a signature nor a fingerprint makes yesterday's implementation the current installed one.

Reuse, currentness and atomic publication

Correctness binds checks to immutable subjects, not reparsing the full proof DAG after every browser action. Reusable checked subresults are keyed by their exact proposition, source/semantic identities, dependencies, inference rules, checker/profile policy, premise bindings and implementation context required by the particular claim. Reuse and cache invalidation follow their claim-specific scope; an unchanged abstract interface may reuse its abstract proof without reusing an unverified changed implementation binding.

Currentness must be supplied by the **consumer's trusted** assembly/deployment context. An attacker can supply an old source and old graph that agree with each other; this establishes only their old abstract subject, not currentness. Changed callable

bindings, route/asset manifests, policy or checker versions, dependency guarantees and revoked profiles cause re-admission or separately checked compatibility. Runtime instance retirement is not reversed by static semantic identity.

Hot reload checks one new coherent candidate before switching the admitted application generation. A failed candidate cannot partially replace live source/proof/handler/asset generations. The selected deployment policy decides whether to retain a last compatible generation, refuse new operations or stop safely. A proof-sidecar may inform build admission but never alter runtime transitions after plan issuance.

Separate checks occur at the stages that actually own their facts: per-build semantic claims; per-installation route, served asset and physical source binding; per-invocation actor, authorization, current Request/Membership and command context; and per-effect target/resource/transaction authority. A cached proof that a rule is well typed does not cache Alice's current authorization, a new XTDB basis or a successfully installed HTMX response.

Product and adversarial acceptance boundaries

For HumanHelp Claim, a legitimate checked abstract rule still requires the real Request model writer, current principal/Membership/Organization, atomic guard and stable historical command identity with distinct `:committed`, `:rejected`, `:already-incorporated` and `:commit-unknown` outcomes. HTTP 200, a current Request card or a copied browser stamp cannot independently establish historical command incorporation. A checker claiming unconditional Claim correctness must produce the corresponding actual enforcement evidence, not point to an abstract rule certificate.

For Create, the creator-local selective installation and remote advisory-only path remain separate, with one explicitly authorized remote Refresh read and qualified scoped adoption. An admitted plan does not authorize a second read, erase newer local state or prove physical target installation by assertion. The independent positive HumanHelp acceptance requirements of Part VIII remain gates even when abstract proofs are sound: rejecting all operations does not implement the application.

Acceptance cases include genuine conditional admission and authorized use, alongside adversarial copied reports, forged fields, invented handles, missing operation/invariant/branch edges, changed dependency or model writer, old/current generation mixing, proof cycles, missing inference steps, silent assumption discharge, vacuous initial states, partial finite results, untrusted package readers, exhausted resource budgets, substituted installed assets, omitted dynamic writers, unsupported profile downgrade and reused historical command evidence. Each rejected case names the first responsible semantic, proof, source-closure, admission, deployment or physical gate. A positive equivalent semantic edit and separately checked compatible replacement must also be tested to prevent unnecessary false rejection.

Dependency composition and semantic identity

Compositional assurance is a judgment about a **consumer's required propositions** and the actual certified interfaces and source dependencies that justify them. An upstream artifact is not acceptable merely because it has the expected kind, friendly name, cached object identity, valid fingerprint, or an unrelated successful proof. The default composition rule is exact checked semantic identity; controlled evolution requires an independently checked, direction-sensitive compatibility argument.

Composition must preserve the distinction between an abstract semantic contract, evidence that the contract is satisfied, the actual production implementation, the installed application assembly, and the particular runtime authority or resource instance. It must neither demand complete rechecking of unaffected abstract contracts nor reuse an old physical correspondence claim after the implementation changes.

Actual exported commitments and required closure

A certified semantic interface exposes only the claims downstream consumers are permitted to rely upon: normalized subject and type commitments, operations, outcome and branch contracts, authority/ knowledge requirements, checked guarantees, scoped open premises and any declared relevant implementation correspondence. Private implementation fields, diagnostic source spans and unrelated guarantees do not silently enter the downstream proof contract.

For each requested downstream claim the constructor derives the **required dependency manifest** from its original normalized source, operation/Choreo declarations, checked judgment rules and selected assurance profile. A graph supplied by an elaborator is candidate evidence, not the authority that defines its own required edges. Composition checks both:

- **Structural validity:** every node/reference has a recognized checked category and exact identity; roots and edge kinds match; required proof-justification edges are acyclic, reachable and well formed.
- **Required source closure:** every needed upstream proposition, producer, type/branch condition, premise, resource policy and operation case occurs in the admitted source-derived manifest, including the edges the elaborator or cache did not volunteer.

A graph can pass the first and fail the second. For conditional induction the required closure includes the original operation catalog, shared initialization, every declared invariant, every operation, every base and preservation case, and their explicit open premises. Removing the most difficult `Request/Unclaim` case while leaving a self-consistent DAG is a failure of source-derived coverage, not a successful proof of the smaller application. A checked proof about a **deliberately smaller declared catalog** is legitimate only when it names that reduced subject honestly.

This completeness is relative to the checked source universe. A declared application catalog does not by itself prove every real route, model writer, startup hook, dynamically selected handler, background worker or installed browser source was discovered. Part VII's assembly closure and the qualified physical writer/ host evidence in Parts IV, VI and X must supply that independent coverage when a stronger installed-product claim depends on it. Opaque Clojure bodies contribute declared effect/dependency contracts and explicit conformance premises where mechanical discovery stops; those premises are not automatically discharged by a valid graph.

Distinct edge kinds and legal cycles

The assurance structure includes dependency edges that say different things: `requires-guarantee`, `uses-premise`, `justifies`, `implements`, `realizes`, `installed-with` and observation/correlation links. The selected **proof-justification projection** must be a well-founded DAG with correct premise/claim dependencies. These other edges cannot be silently counted as proof justifications.

Recursive rules, transition systems, subscription loops, retries and self-sustaining Choreo protocols can be well formed while their runtime graph contains cycles. An induction, fixed-point or coinduction step may justify a recursive semantic construction only under its admitted inference rule and premises. Two certificates whose only grounds are each other's promised conclusions do not become valid through ordinary graph cycle detection or by calling the cycle a runtime loop.

At a branch join, a dependency on a role-local fact is valid only with the exact knowledge and provenance required on all reachable incoming paths, or through an explicitly checked conditional/ disjunctive qualification. A communicated fact, old snapshot, opaque policy assertion and current authoritative observation are not interchangeable merely because they share a field name. Part III's distributed semantics and Part IX's 09.08 judgment remain controlling at this use site.

Six identities answering different questions

Identity	Subject and allowable reuse
Semantic interface identity	The normalized exported subject contract under named schema, nominal-type, interpretation and canon
Claim identity	One exact proposition with subject, model/scope, parameters, context and its conditional-premise mean
Evidence/check identity	The actual derivation or checker result, proof rules, trusted checker version, profile, dependency closure
Implementation binding	The specific callable, closure, generated executor, compiler/product, provider adapter or model writer w
Application/installation identity	One coherent source-to-route-to-asset-to-host configuration, with its selected assembly/deployment ge
Runtime instance identity	A particular command, role execution, actor/session, transport attempt, observation, physical target, lea

These identifiers do not collapse into one digest. Two artifacts can export the same abstract semantic interface and have different proof evidence. An unchanged interface is not proof that a new implementation conforms. The same installed application can serve different command instances with different current authorization. A later state snapshot is not the same observation or historical incorporation event as an earlier command result.

Canonical identity is exact only relative to a versioned normalization/encoding contract. Unordered maps/sets may normalize to one term, while ordered vectors, optional versus missing values, nominal identifiers, branch alternatives, closed versus extensible variants, and integer semantics preserve their declared distinctions. Where a normalization is claimed meaning-preserving, its own source-to-term argument is checked or retained as a named premise. Textual/diagnostic metadata is separate unless the proposition explicitly refers to source location or provenance.

A digest is evidence of term identity under its encoding and collision-resistance assumptions, not evidence of typing, truth, implementation fidelity or currentness. When a claim requires collision-free equality rather than assumed hash uniqueness, admission compares canonical terms or uses an independently qualified equality mechanism. Forged copied identity fields never create an issuer-issued handle.

Implementation binding requires more than a source filename or Var symbol. Captured closure state, late binding/dynamic Var resolution, transitive library versions, build configuration, provider options, generated bytes and actual selected handler can change behavior without changing the public name. Strong correspondence must bind the relevant realized dependency footprint or retain an explicit, testable stability/conformance premise where the host cannot prove that binding.

Dependency failure and transitive invalidation

Composition is consumer-relative. If a required upstream guarantee is absent, or its semantic identity, premise scope, nominal type, model observation, operation variant or trust domain differs, the consumer's claim cannot be accepted through the old edge. The failure propagates along **dependent** edges that rely upon the changed commitment; independently established unrelated claims need not be invalidated merely because they share a namespace.

A valid checker result with missing required source nodes is insufficient. A complete graph with a changed producer body can still support an abstract interface claim while losing production realization evidence. A copied original certificate cannot keep its old installation or resource rights alive after an incompatible reload. Cache presence, prior successful tests or an unchanged digest of a downstream file do not establish current source closure.

For a concrete HumanHelp evolution, let Organization domain O14 export `organization.claiming-enabled?` as a required Boolean and let Claim rule C21 depend on it. Suppose O15 instead exports a closed `organization.claim-policy` enum:

```
O14: claiming-enabled? : Boolean
  → C21: Request Claim rule
  → O31: Claim operation
  → H12: Claim choreography
  → P7: helper projection
  → E5: browser executable plan

O15: claim-policy : {open, helpers-only, closed}
```

O15 may be a valid new product interface. It is not a valid replacement for C21's **old required Boolean field**. The composition of cached C21 against O15 must reject with the missing or incompatible source commitment and a dependency path, regardless of whether cached O31/H12/P7/E5 still have their original bytes. On re-elaboration the old Claim rule must fail its unresolved field access, rather than accidentally interpreting `helpers-only` as unrestricted acceptance.

A repair changes the rule and issues a new semantic identity. The repaired rule explicitly checks current Request status, Membership and Organization association, trusted authenticated principal/role evidence, and the enum branch: `open` permits the policy conjunct,

`helpers-only` requires the admitted helper-role predicate, and `closed` rejects. The enum example describes one policy conjunct; it is not a complete production Claim authorization proof. Production writer and current-model conformance remain distinct premises and actual runtime checks.

This illustrates that **coherent upstream evolution** and **stale downstream composition** can coexist. The diagnostic should not mislabel O15 as malformed or say that a changed policy field necessarily makes every independent application artifact invalid.

Checked compatibility instead of optimistic substitution

Exact identity is the default. A proposed compatibility bridge is a checked proposition:

```
old dependent D requires interface Iold under premises Pold
new upstream U exports interface Inew under premises Pnew

bridge establishes, for D's actual use and scope:
  accepted guarantees(Inew, Pnew) entail required guarantees(Iold, Pold)
  without weakening required authority, knowledge, closed-world
  constraints, lifetime or physical implementation bindings
```

Compatibility is **directional** and **consumer-specific**, not an equivalence relation inferred from field counts or a label like "backward compatible." A compatible upstream for one consumer may be incompatible for another; transitive reuse requires a checked composition of bridges, not an unchecked assumption that each pair looked plausible.

The bridge must account for actual variance of typed inputs/ outputs, optional/null presence, field removal/renaming, closed-enum exhaustiveness, newly possible errors/outcomes, previously assumed impossibility, and temporal/authority scope. Adding an enum variant can invalidate exhaustive matching. Extending an operation from `:committed` and `:rejected` to `:already-incorporated` and `:commit-unknown` cannot preserve an old total-settlement Choreo that has no unknown/recovery branch. Adding a field cannot repair an earlier reliance on its absence or a confidentiality promise that the new field makes observable.

The bridge must preserve the relied-on **premises**, including issuer, principal/tenant, input scope and provider qualification, or re-establish them through a separate valid premise-discharge judgment (09.11). It may not silently upgrade `:asserted` to `:derived`, an abstract model to a physical observation, a past authenticated command result to current-state authority, or a checked contract to its unverified new implementation. Incompatibility is a typed rejection naming the first missing judgment. Undecidable opaque equivalence is a separately reported premise/unsupported-strong-profile outcome, not a silent success.

Cache keys, impact analysis and safe reuse

The reuse key is claim-specific: normalized subject, exact required interface/dependency identities, proof/checker rules, profile, proposition and premise bindings, and any relevant implementation, deployment or runtime applicability envelope. An implementation may maintain separate abstract-contract, proof-evidence, compiled-plan and installed-realization caches. One cache's successful hit never promotes an entry into a different assurance category.

A change propagates through the typed reverse-dependency closure used by affected claims. Precise impact analysis can allow an unaffected abstract rule theorem to survive a renderer CSS edit while requiring requalification of the browser artifact and physical installation claim affected by that edit. Conversely, a changed closed enum or a strengthened guard can force rechecking every consumer that relies on its former meaning, even when the dependent source bytes are unchanged. Source spans/diagnostics may need refresh even when semantic proof remains reusable.

A cache miss may reconstruct and recertify; this is not a failure of soundness. An old handle or successful cached report is not authoritative for an altered source, changed dependency graph, selected profile or installed executable. Tests must exercise both direct composition and the **real consumer's cache path**: a unit function that rejects stale EDN is insufficient if production loading bypasses it.

Change	Scope of required response
Formatting/source locations only	Retain eligible semantic claims if canonical meaning is identical;
New irrelevant implementation detail	Abstract proofs may survive; dependent implementation/physica
Nominal type, required field, outcome, guard or assumption changed	Recheck every relying proposition and dependent judgment, or v
Checker, kernel, normalization or policy version changed	Revalidate affected evidence under current admitted rules; old su
Changed opaque callable, provider/configuration or generated program	Re-establish applicable binding and correspondence or preserve a
Route/asset/host assembly generation replaced	Admit one coherent installation closure; reject mixed old/new ce
Individual runtime resource or command instance retires	Remove that instance's effects/grants; static interface identity do

Fresh source, coherent deployment and temporal claims

A dependency graph can be internally consistent while entirely obsolete. The consumer or responsible assembler must acquire its **current** source, dependency generation, policy and selected realizations from the trusted application/installation authority, not from a caller's mutually matching old source plus old graph. A source-issued handle proves only its named checked boundary until this currentness and any required installed correspondence are separately established.

For reload, construct and check a replacement closure before atomic publication of the new admitted assembly identity. New routes/handlers, browser plans/assets, model operation contracts and authorization policy cannot be mixed with old evidence merely because each component passed some check at a different time. A failed candidate leaves the previous compatible generation under its stated policy or prevents new admission; it never creates partially certified service. Per-invocation and per-effect authority remain current physical/runtime obligations, not consequences of this static source composition check.

Runtime continuity and durable command recovery are separate from static graph reuse. A known Claim command C42 may retain its historical durable outcome binding across a compatible reconnect, but neither that command ID nor a reissued plan stamp authorizes a new attempt, current readback or retired browser lease. Under Parts IV–VI, the applicable historical outcome, actor, session and current source/provider/plan binding must be checked at the actual trusted boundary.

Positive and adversarial acceptance

The final system needs both genuine supported progress and targeted rejection. At minimum, the test/report corpus covers:

- a complete original Claim source/induction DAG and successful consumer admission, versus a structurally valid pruned DAG;
- same-name new Organization policy O15 correctly rejecting stale C21 and all dependent old uses, followed by successful checked C22/new-operation/new-Choreo rebuilding;
- actual semantic equivalence after formatting or map-order changes being reusable, while an ordered-vector or nominal-type change is not silently equated;
- an irrelevant renderer style edit preserving rule-level proof reuse but requiring affected browser-asset correspondence;
- a closed-enum variant addition, missing-branch default, optional-field weakening, or altered operation outcome rejected before total-settlement/knowledge claims;
- a correctly scoped, checked asymmetric compatibility bridge accepted for one consumer, versus unjustified bridge reuse for another consumer, premise or implementation;
- injected extra proof node, omitted required edge, forged fingerprint and cycle of mutual proof justification rejected at their actual structural or source-derived boundary;
- copied proof report/handle and old source+matching old dependency graph rejected as current installed-app evidence;
- changed callable, provider, route or served asset detected for the corresponding implementation/installation claim;
- unrelated source change preserving reusable unaffected claims, while transitive changed dependencies are detected through the production cache and assembly path;
- positive eligible Claim and selective Create/Refresh continuing to satisfy independent product gates, with incorrect tenant/role or second unauthorised Refresh rejected by the real enforcement boundary.

The expected disposition and first responsible checker, source-coverage, composition, construction, installation or physical gate are recorded before exercising each positive/adversarial case. Formal or structural checks are not credited with observing actual browser/XTDB behavior unless the required authenticated physical evidence exists.

Assumption propagation and genuine discharge

An assumption is a scoped proposition whose truth is not established by the current proof chain. It is not an arbitrary text label or an implicit escape hatch. Its identity includes the exact subject, proposition, parameters, nominal types, model/observation context, source and dependency generation, issuer/origin, principal and authority domain where relevant, time/basis, implementation boundary, and permitted use. Which of these are applicable is determined by the proposition's semantic category, not by a universal requirement to attach an unrelated browser session to every abstract lemma.

The proof system distinguishes (a) premises **used** to derive a specific exported claim, (b) declared but unused or unrelated premises, (c) evidence that establishes a particular premise, and (d) still-open premises admitted conditionally by the selected profile. A syntactically matching assertion, success report, nominal type, checked graph or provider name does not make an open premise true. A certified conditional theorem is valid **with its conditions**, not a license to drop those conditions downstream.

Assumption identity, origin and provenance

Each premise records its exact proposition and its original source in the declared semantic interface or imported dependency. It also identifies any authority/knowledge preconditions, checking-rule interpretation, opaque Clojure binding, external provider/configuration envelope, application policy owner or physical observation obligation on which it relies.

Two phrases such as “Membership is current” can denote different subjects: Alice versus Bob, Request R17 versus R18, Organization O1 versus O2, transaction prestate versus an earlier candidate read, and a particular command attempt versus all future commands. Equal names, hashes, maps or equivalent-looking field values do not authorize premise unification across those boundaries. A qualified provider basis is ordered only by that provider's admitted scope/order contract; an arbitrary timestamp or later-looking token does not strengthen an observation.

The source catalog must distinguish at least domain assertions, opaque implementation conformance, external-service behavior, authenticated runtime acquisition/authorization, resource/instance rights, distributed knowledge and progress/fairness premises. These are not one linear ranking of “trust.” An assumption may be a legitimate explicit contract for one profile while a stronger claim requires independent evidence or refuses construction. In all cases an application-asserted proposition remains an assertion until a checking boundary establishes its own exact proposition.

Per-claim propagation through admitted dependencies

For each exported guarantee G, its accepted justification determines a reconstructible dependency and premise footprint, not merely a flat union of every premise in its containing namespace or certificate. If an imported interface offers G1 under P1 and G2 under P2, a consumer relying only on G1 inherits P1, not automatically P2. Conversely, consuming both guarantees cannot lose either used premise because a root report coalesced their labels.

Alternative accepted derivations may exist. The issuer records the specific proof path used, its premises and every attempted or checked substitution; it may report other viable paths without claiming that the union, intersection, or unrecorded “best” path is a single proof. If a different path is selected, its full premise footprint must be checked again for the requested profile and current dependencies.

Composition preserves source, dependency, premise and evidence identities. Transitive assumptions remain linked to the original producer, and the final report can answer which conditional claims, operations, role-local uses and installation boundaries depend on each premise. A dependency's checked guarantee cannot be promoted past an inherited open premise by renaming it or by flattening a root certificate's summary.

The proof-dependency graph is acyclic in its justification projection. Mutual claims that each “discharge” the other's premise have no well-founded justification. Runtime retries and recursive choreographies are different graphs and do not confer proof permission. Scope-sensitive sharing is permitted only for reusable facts with compatible subject, issuer and applicability; a one-use authorization, browser Refresh grant or command-specific outcome cannot be copied or spent as though it were a reusable lemma.

Genuine discharge as a checked substitution judgment

For an original accepted derivation of G under open premise P, replacing P with evidence E requires the composition checker to establish all the following:

1. **Evidence category:** E is itself admitted at the correct trusted checking, authenticated observation or enforcement boundary for the kind of proposition P. A copied report or structurally matched premise declaration is insufficient.
2. **Logical entailment:** E establishes precisely P, or an accepted inference/refinement bridge proves that E's proposition implies P. The checker performs type-correct, capture-avoiding substitution under the appropriate rule/context rather than treating copied identifiers as proof.

3. **Scope and applicability:** The checked relation binds original subject, parameters, principal/tenant, authority and disclosure scopes, source/semantic versions, instance/generation, and relevant temporal/basis conditions. No old observation is silently promoted into current transaction-state authority.
4. **Inherited conditions:** All assumptions, trust boundaries, dependencies and implementation qualifications of E are carried into the new derivation. An evidence chain conditional on Q replaces P with Q where applicable; it does not produce an unconditional G unless all relevant remaining premises are genuinely discharged.
5. **Complete checked construction:** The resulting proof/justification remains well founded, source- and claim-complete, and admitted under the consumer's selected profile. A changed premise set cannot bypass mandatory local rules, application coverage or positive non-vacuity obligations.

One useful schematic judgment is:

```
Gamma, P |- G           Delta |- P
---------------------------------  checked composition under same context
Gamma, Delta |- G
```

This is a **desired composition rule**, not a declaration that the current Gesso propositional kernel implements hypothetical-context discharge. A real checker must specify which entailment rules, substitutions, contexts, assumption multiplicities and evidence types it supports. It must reject unsupported generalization or silently preserved free variables. The original premise and the evidence that replaces it remain in the audit ledger even when P no longer belongs to the **open** set of the resulting claim.

A source checker must not accept a caller's `:discharged?`, `:verified?` or `:assumption-discharge` field as a verdict. Structural equality of two assumptions only establishes that their declarations have the same identity. Likewise, exact finite entailment over uninterpreted Boolean atoms is a result about that finite propositional abstraction, not evidence that an opaque business-policy atom is true of real Request data. If a proof uses additional premises, the resulting conclusion remains conditional on them.

Static model contracts and current domain facts

Consider HumanHelp Claim. A source-checked rule may rely on "the selected Organization policy callable implements the declared typed predicate for these inputs without undeclared relevant effects." This is an implementation-conformance premise, not the value of the policy for a particular Organization today. The constructor binds the premise to the selected callable/closure or generated implementation, its dependencies and qualification envelope.

A direct unit test can exercise the callable but does not prove universal conformance. A separately checked implementation correspondence argument can discharge the premise for the exact implementation, typed input domain and covered effects it establishes. Its remaining compiler, runtime, library and provider assumptions propagate. A changed function body, captured state, dynamically resolved Var or relevant library can invalidate that physical correspondence without invalidating the abstract policy theorem.

Actual Claim incorporation separately requires the current authenticated principal, Request, Membership, Organization association, eligibility and atomic guarded transaction. A prior browser view or one earlier server read is not the authoritative transaction prestate. A trusted invocation-bound guard or actual verified transaction observation may establish an **instance-specific** premise under its qualified writer/basis contract. It cannot be reused as authority for another user, Request, command, transaction, or later retry without fresh applicability checking.

Historical command incorporation and current state are different propositions. If C42 incorporated and the response was lost, command-specific trusted readback may establish its historical disposition, while later Reassign means the current Request state is different. Neither a generic current card nor HTTP 200 discharges the historical attribution premise; an old command result does not grant new write authority.

Conditioned progress, resource rights and disclosure

An environment promise such as "the network eventually recovers," "a provider eventually incorporates work," "the scheduler is fair," or "the user eventually chooses Refresh" is a progress assumption with a specified trace and failure domain. It is not a fact created by a timeout declaration, an SSE event, or an automatic retry. The corresponding liveness theorem is conditional on precisely the required fairness, quiescence, retention, service, client lifetime and user-action premises. Failure to establish progress does not automatically imply a safety violation; proven safety does not manufacture liveness.

In the HumanHelp remote board, an advisory about an updated Request may create refresh **debt** but cannot establish that the user authorized acquisition. The one-use Refresh grant is a current scoped resource right with a specified acquisition budget and installation obligation. Actual authorized invocation and completed physical adoption establish their own instance-specific propositions. An old grant cannot discharge another Refresh permission, and a local newer card observation cannot discharge unrelated whole-board coverage. Lost or coalesced invalidations may require an explicitly permitted reconnect/repair read or a weaker continuity promise, not fictional permission from prior SSE delivery.

Distributed knowledge premises also require the correct role, issuer, recipient and disclosure boundary. Communicating a typed value does not establish current model authority; withholding a private Organization fact cannot be repaired by pretending the browser knows it. The system may communicate an authorized server-derived public decision instead, under that decision's own issuer and scope contract. Information release, effect authority and historical/progression knowledge remain separate checks.

Profiles, contradictions and unsatisfied premises

Every assurance profile states which local obligations are mandatory, which premise categories may remain open, what corroboration they require, and where dynamic instances must be checked. A stronger profile cannot be achieved by silently reclassifying unsupported syntax, proof-rule failure, missing writers, escaped policy, unknown code or unavailable checker as application truth. A weaker conditional profile is a distinct requested and reported result, with the open-premise closure preserved.

Within its supported logic the verifier detects contradictory checked declarations, incompatible semantic commitments and inconsistent premise uses where feasible. It is not a decision procedure for arbitrary opaque Clojure code or third-party business policy. A propositional proof from inconsistent premises may be formally valid yet vacuous for a realizable application. The root assurance report therefore separates conditional logical derivation, satisfiability/non-vacuity under its declared abstraction, actual enabled positive product paths, and physical execution evidence. None substitutes for another.

An assumption's applicability can expire or be withdrawn without making the underlying abstract conditional theorem false. A provider upgrade outside an accepted envelope, revoked service credential, changed implementation binding, retired execution, expired basis/lease, or observed assumption violation invalidates the **dependent deployment or instance claim** until requalified. The dependency impact analysis propagates that change to exactly the relied-on claims; it does not automatically falsify unrelated semantic results.

Qualified external providers and failure envelopes

A premise such as XTDB “atomic guarded incorporation” must identify the exact operation and transaction semantics, actual gateway/writer, provider and adapter versions, relevant configuration, observation/correlation method, safety envelope and independently testable failure behavior being relied on. “XTDB is correct” is not a useful indivisible discharged proposition.

The same requirement applies to Ring/Biff authentication, HTMX event/lifecycle/adoption behavior, browser APIs and source/asset installation, compiler and language runtime, message delivery and other external services. Trusted origins and observation provenance must be established by their actual boundary. Successful qualification tests can support a bounded trust decision and detect regressions; they do not establish universal provider semantics for untested executions or versions.

A premise-qualification ledger records both the allowed realization envelope and what is actually observed/established. On a provider/host change the consumer rechecks applicability before reusing evidence. Missing qualifications yield explicit open obligations, a rejected strong profile, or a justified different weaker profile—not an automatic upgrade to a theorem. The report names which claims now lack an applicable premise, their dependencies and the earliest responsible requalification or physical enforcement boundary.

Positive and adversarial acceptance obligations

The finished framework's conformance corpus must contain successful conditional and genuinely discharged cases, alongside negative near-neighbors. At minimum:

- One claim G uses P1 but not another exported claim's P2; the report attributes exactly P1. An unrelated premise must neither leak into G nor suppress a premise actually used by G.
- A checked accepted E proving P under Q produces G conditional on Q; copying E's success map or setting P's `:discharged?` does not produce G.
- A real direct proof of P with no open premises removes P from G's **open** list while retaining the original P-to-E audit trail. Reciprocal certificate justifications are rejected.
- Same-label Membership premises for Alice and Bob, different Organizations or transaction bases remain distinct; cross-use cannot be admitted by matching names.
- An unissued assumption description that matches the required structure still fails as purported proof. A well-formed supported propositional derivation retains all its used premise IDs rather than dropping them.
- A legitimate current Claim under an authorized principal and guarded XTDB incorporation has qualified runtime evidence; a stale pre-read or later current card does not establish historical commit attribution.
- An unchanged Organization interface with a replaced policy callable requires renewed implementation correspondence; unaffected abstract rule proof may remain reusable.
- A valid remote advisory and single user-Refresh grant support exactly their authorized reads; a queued second read or wrong-scope whole-board adoption fails at the physical permission or coverage boundary.
- An admitted positive progression argument states fair scheduling, provider availability, retention and user action when needed; a timeout or absent user action does not falsely establish eventual completion.
- Satisfiable abstract initialization and actual eligible Create/Claim paths pass independent non-vacuity/product gates; inconsistent assumptions and reject-all construction cannot be counted as successful product proof.
- Qualified provider version and conformance evidence are accepted within their envelope; unqualified upgrade or observed provider contract violation invalidates affected deployment claims.
- A permitted public, server-derived decision reaches the browser with matching provenance; a private field or mere schema-typed payload is not silently granted disclosure authority.
- A valid weaker conditional profile reports all its open premises. A stronger mandatory discharge cannot silently fall back to a trusted assertion or lower guarantee class.

Each test oracle must identify the proposition, expected disposition, subject, use-time scope, required checking/physical evidence and **first responsible enforcing boundary** in advance. A source-level checker is not credited with catching a real HTMX or XTDB defect without qualified observation. The positive controls must continue to admit genuinely useful Create, Claim and declared recovery/Refresh behavior; a checker that rejects all cases does not satisfy this design.

Binding proof claims to production realization

A checked theorem about an abstract source is not automatically a theorem about the program running in XTDB, Ring/Biff, HTMX or the browser. Certification must cross the construction and installation boundaries of #req-application-assurance-claims with an explicit, directional correspondence argument. The finished Gesso framework must make the strongest supported relationship between source, checked semantics, executable artifacts, installed entrypoints and observed physical effects mechanically inspectable, without pretending that the existence of a certificate, matching name or stamp establishes their equality.

The source-relative semantic judgment, the implementation-binding judgment, the installed application/asset judgment and the actual occurrence judgment have different owners. An admitted proof may be reused after its concrete implementation correspondence expires, but no consumer can silently strengthen its scope. Failure to establish one edge leaves that edge open, rejects a profile requiring it, or yields a separately requested weaker claim; it cannot be supplied by the apparent success of adjacent edges.

The correspondence ladder and its evidence subjects

For each required path, the assembly records a typed chain of claims:

Edge	Required subject and boundary
Original declarations to checked semantics	Source-owned schemas, rules, operations, writer catalog and choreography match the normal
Checked semantics to issued realization	A checked constructor binds original operation, generated plan, exact implementation callabl
Issued realization to installed host	Trusted application assembly binds actual route, handler, worker, initializer, source registratio
Installed host to physical occurrence	The actual invocation binds request/principal, selected implementation, one command/transa
Physical occurrence to abstract operation	A stated abstraction/refinement relation matches actual relevant prestate, guarded transition,
Emitted browser plan to loaded execution	Generated manifest, compiled artifact bytes, serving origin, selected plan and installed brows
Browser effect to adopted state	Real source, scope, permission, target/resource generation, effect admission and observed phy

Every edge names its source, target, abstraction map if any, identity/provenance requirements, observable events, allowed hidden steps, failure cases, checked or assumed implementation semantics and expiry/requalification conditions. An origin-preserving structural join and one successfully replayed trace are legitimate **bounded evidence**. Neither automatically establishes all-path refinement, liveness, physical safety or complete coverage of the deployed application.

Proof justification remains acyclic even where actual protocols, retries and subscriptions legitimately cycle. A success report returned by the implementation under test is not independently authenticated evidence of that implementation. The checker records the trust and independence of each observation path, not simply a sequence of optimistic agreement booleans.

Source-owned construction and issued callables

A supported construction derives typed model/operation declarations, choreography, obligations, compiled plans and compatible issued executors from one admitted source and dependency generation. The resulting callable has a checked original subject, immutable operation/plan binding, source and profile identity and a declared effect/response contract. The public API should make it difficult to replace the checked executor with a same-shaped Clojure map or similarly named unverified function. An issued handle is a checked category within its trusted JVM boundary, not a proof that an arbitrary server process installed or invoked it.

An installed Ring/Biff handler must actually delegate to the issued callable through checked, current request-to-input and result-to-response mappings. That means checking the true HTTP method/path, route dispatch, authenticated session/principal source, canonical argument construction, tenant/scope association, anti-forgery and other mandatory host gates, operation and command IDs, model dependencies, and the disclosure/response mapping. Replacing a trusted principal with a request parameter or turning an unknown backend outcome into success is not made safe by an otherwise correctly issued handler.

Request-local ownership is essential. An operation's command identity, invocation context, transaction/XTDB node, guarded source, selected executor and outcome evidence must be associated with the **same real execution**, even under overlapping and reentrant requests. An optional diagnostic that runs a second transaction after the real request cannot be combined with the first request's result to claim same-invocation refinement. A single global mutable controller, reentrant callback or stale dynamic Var must not allow one user's context to provide another request's authority or transaction receipt.

Coverage follows **possible effects**, not merely route names. Startup initializers, import/migration paths, administrative commands, background jobs, scheduled work, external writers, direct XTDB access and other non-HTTP operations may alter protected state. Required physical-writer and initializer inventories must include or explicitly exclude each admitted effect-capable entrypoint under independently qualified source/host coverage. Not every route writes, and not every writer has a route. A host-enumerated

list is evidence relative to its own actual enumeration boundary; it cannot prove it enumerated every external process, direct writer or dynamically selected callback merely by comparing with itself.

Guarded incorporation and exact transaction causality

A protected incorporation claim is about a specific authoritative operation, actual canonical input, current actor and protected dependencies, permitted writer/plan, stable command identity, atomic guard and physically incorporated effects. These bindings must survive from the checked declaration to the same actual transaction or remain explicit open premises. A hypothetical prestate or caller-supplied before/after map is useful for testing an abstract relation but does not authenticate XTDB's internal transaction prestate.

The physical guard must operate at, or be qualified as equivalent to, the actual atomic read/write boundary. A preflight check on a candidate pre-read can establish only that **the candidate** would have been eligible. Between that read and a separate write, Request or Membership state can change. An admitted conditional guard/CAS implementation must bind required current authorization and state/frame conditions to the transaction's actual serialization and conflict behavior. Where the backend cannot expose or prove its internal prestate, the guarantee names the exact guarded-writer assumption and the observable failures that can test it; no unconditional transaction-level theorem is issued.

A submitted transaction, returned TransactionKey, a ledger row, pinned-basis poststate and a canonical durable command record are distinct facts. They can provide progressively stronger **bounded co-observation** when joined at one exact execution and source. However, even a matching ledger, command record and domain poststate can be produced through unrelated writers unless causal provenance, node/transaction binding and writer completeness are independently established. A copied transaction key, caller-supplied node or callback report does not authenticate the physical submission. The root report states which causal step is genuinely observed, which is an implementation trust premise, and which is unestablished.

A post hoc discrepancy can reject an assurance claim, diagnose a defect or quarantine a future deployment. It cannot undo an unauthorized physical write that already occurred. If the selected strong profile requires prevention, a genuinely installed and synchronous enforcement gate must run before the relevant effect and must fail closed on unknown principal, missing authority, changed generation, unverified target or incompatible operation. Checking only after submission cannot substitute.

Historical command identity, retries and outcomes

A stable command C42 and physical requests H1/H2 are not interchangeable identities. Compatible replay after a lost response must use the original command binding, deduplication/retention and actual historical outcome evidence, with a fresh disclosure check for the receiving principal. It must not regenerate a new domain write because HTTP delivery became uncertain.

A replayed incorporated Claim may now find Request in a different state, including after Reassign. A new-Claim guard such as “Request is open” must not be reapplied as a prerequisite for reporting a genuinely existing C42 outcome. Conversely, a present Request card does not tell whether C42 incorporated historically. Backend outcomes `:committed`, `:rejected`, `:already-incorporated` and `:commit-unknown` remain separate from presentation `:accepted`, `:rejected` and `:unknown`. One physically observed submission must not be silently retried on an exception; exception after submission means this caller may still lack attribution.

Durable command record creation, domain mutation and publication intent require the actual declared atomicity or compensating contract, not inferred success from similar subsequent data. Outbox publication, invalidation fanout, provider progression, browser notification and physical adoption add separate later events. No report may equate queued/outbox-ready work, dispatched notification, received SSE message, current fact acquisition and completed fragment installation.

Browser build, response and loaded-plan correspondence

The browser build contract follows original Choreo source through its checked projection, ExecutablePlan, manifest, emitted bootstrap and actual compiled asset bytes. Source-derived reconstruction or comparison of a deterministic emitter's output may establish a bounded build correspondence under its compiler/encoding assumptions. Copying the expected stamp to an unrelated bundle does not establish that the real asset embeds or executes the plan. Shared emitter/checker bugs require a separate independence argument or remain a named premise.

The consuming assembly checks actual serving routes, assets and compatible host/plan/runtime versions, then establishes which bytes the browser loaded and which plan/source generation they activated. A generated manifest does not prove the production server uses it, and a response carrying correct metadata does not prove the browser installed the correct program. Compatibility with suspended tabs, retired plans and per-session resource rights requires fresh runtime admission.

Rendered-affordance evidence must bind the concrete expanded Hiccup, operation identity, effective HTMX attributes/request target, response closure and final serialization. Checks of a supplied render are not universal coverage of every possible data-dependent, deferred, streaming, cached, exception or out-of-band response. The selected output forms must either be checked at

their actual emission boundary or be explicitly outside the strong profile. A raw ordinary HTMX control is legitimate as such; it cannot silently occupy a protected semantic operation route or inherit the strong managed claim without assembly and effect checking.

Actual DOM adoption is a physical event: an authenticated source/observation and authorized acquisition precede target- and scope-specific effect admission, followed by completion or honest partial failure. An `htmx:afterSwap` event or HTTP 200 without matching current target and installed content is not automatically completed multi-target or OOB adoption. An old resource generation cannot restore or remove a newer owner's DOM, continuity or transport effect merely because it once held a valid plan.

The HumanHelp creator-local Create path can adopt a single authorized new card without importing an unrelated remote board revision. Remote notification creates advisory debt, not permission to fetch. A one-Refresh user grant admits only its scoped acquisition, and physical completion/disclosure must be shown separately. A newer local card does not establish that an older whole-board query or missing item list is complete.

Independent proof, physical observations and refinement

The report classifies conclusions by evidence owner and quantifier: checked source syntax/ typing, structural admission, abstract transition proof, exact finite-model coverage, bounded simulated trace, controlled physical diagnostic, authenticated production occurrence, installed-protocol correspondence, or universal refinement under explicit environment assumptions. These classes are not ordered by a single "more verified" bit. A valid observation about one controlled node is not the same subject as a live production handler.

For an operation-to-physical refinement the checker specifies the simulation relation between abstract and concrete states, the allowed initial relation, observable operations/outcomes, stuttering/internal effects, partial-order/scheduling assumptions, what happens on rejection and unknown delivery, and how source/target traces are authenticated. The direction matters: a physical trace explained by **some** abstract trace is a safety/refinement question; existence of one executable example does not show that **every** permitted physical trace is explainable. A progress/correspondence theorem needs additional enabledness, fairness, delivery/retention and environment assumptions; no amount of finite examples alone establishes it for unbounded deployments.

Real source/host observations must name their failure domains and limits. Instrumentation may miss unobserved bypasses, external writers, reordered callbacks, truncated ledger windows, provider-specific snapshot semantics or physical events that escaped the collector. Where only a process-local handle ties an issued executor to a controlled invocation, the report states that scope and does not claim live router adoption. Where the proven object is merely an abstract source graph, it does not borrow the producer's host permissions or causal evidence.

Assembly, deployment and positive conformance gates

The entire root source and application closure must be checked under Part VII: model writers and initialization, installed routes and workers, issued executors, physical host provider bindings, rendered controls, actual loaded client assets and named recovery/acquisition paths. Current source and one coherent deployment generation come from the trusted assembler, not a client-supplied old source plus self-consistent old report. The plan remains timeless about intended guarantees; Gesso source evidence belongs in the preservation audit and is not promoted into normative claims about which module is currently implemented.

Single-node and multi-node deployments instantiate the **same** application semantics, authorization, interest, operation and adoption contracts. Distribution does not create a new class of trusted command, grant a remote board read or change what a Claim means. It does add topology-specific physical evidence: installed host/router generation, interest-registration and observation frontiers, durable publication and replay/gap recovery, correct source/tenant disclosure, failover and node-loss behavior. A correct single-node construction or one controlled two-host trace cannot prove all distributed failure schedules. Part X owns the concrete multi-node realization and its fault-qualified acceptance; Part IX requires these additional realization edges and premises to remain visible when the claim depends on them.

An accepted conditional root may contain outstanding external semantics, compiler, host, opaque model-policy or coverage premises when its profile explicitly allows them. A strong realization claim cannot accept unproven physical writer completeness, missing current enforcement, forged execution origin or absent production adoption as if they had been checked. A source-level failure cannot quietly downgrade the installed action to a different semantic category.

The independent HumanHelp proving application remains a positive control: a genuinely authorized Claim/Unclaim/Reassign, Create, creator-local visibility, remote advisory and one granted Refresh, live search/selection continuity and declared response interaction must work through their real production model/route/browser boundaries. Nearby wrong-principal, wrong-Organization, lost-response, stale-plan, stale-guard, duplicate submission, unexpected writer and unauthorized-refresh cases must fail at their actual trusted enforcement points.

A conformance corpus records the expected claim, source and selected profile, trusted observation boundary and exact first enforcing gate **before** running each test. Important paired cases include:

- a valid source-issued handler used by the current installed route versus a copied handler label or replaced callable;
- a valid Request principal and current guarded writer versus actor data substituted from request parameters or stale independent pre-read;
- one actual submission whose ledger, pinned command record, domain poststate and reported outcome agree versus unrelated co-written data or a second diagnostic transaction;
- successful known C42 replay with fresh disclosure versus new incorporation, stale principal or invented commit;
- an independently qualified writer/initializer inventory versus hidden background, migration or raw XTDB path;
- real emitted and loaded plan bytes matched to the admitted Choreo versus copied manifest stamp or old suspended tab;
- a valid concrete affordance/ response/target installation versus an unobserved deferred stream, wrong OOB target or superseded effect owner;
- one authorized remote Refresh and scoped adoption versus a second ungranted acquisition or claimed whole-board convergence;
- a controlled one-execution trace admitted at its actual observation scope versus a universal refinement conclusion inferred from it;
- a genuine eligible positive Claim/Create path versus reject-all admission with superficially green proof.

A passing abstract proof, green source test or successful controlled XTDB/Chromium scenario does not erase an unobserved production-correspondence edge. Conversely, production failures must be attributed to their real model, host, source, effect or environment boundary rather than credited to a checker that did not have that evidence.

HumanHelp compositional certification pilot

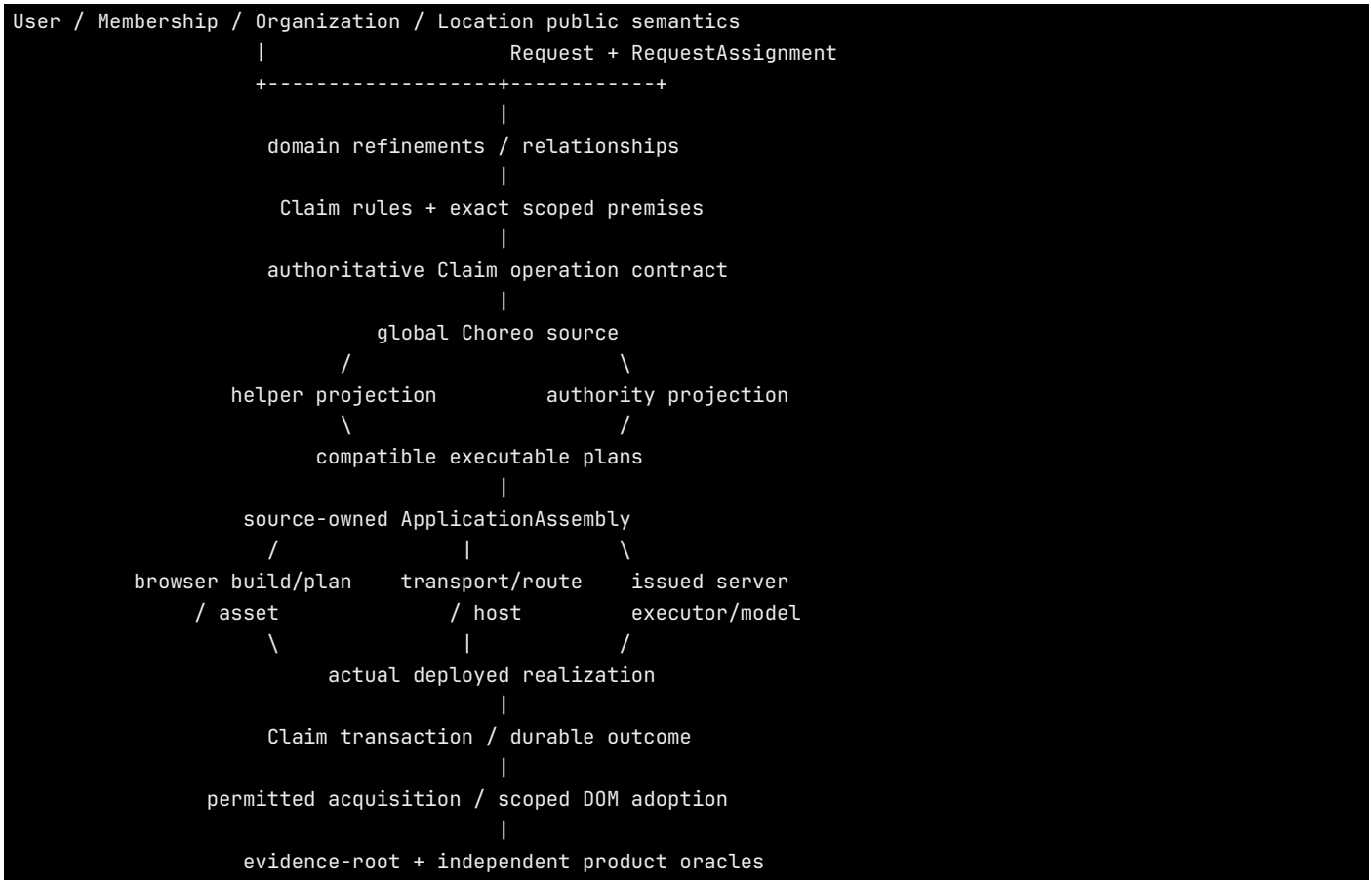
HumanHelp Request Claim is Gesso's first required substantial end-to-end **compositional assurance pilot**. Its purpose is to demonstrate inspectable inheritance of real public model commitments through rules, authoritative operations, Choreo, projections, executable plans, whole-application assembly, installed host/browser realizations and a qualified physical user journey. A toy scalar transition, an isolated checked graph, or a browser test with a handmade protocol envelope is useful subordinate evidence, not this pilot.

The pilot is built from the same source-owned HumanHelp production model and example application required by Part VIII, rather than a second, convenient proof-only Request store, operation catalog or business-rule machine. It uses the actual public User, Membership, Organization, Location, Request and RequestAssignment interfaces **where Claim genuinely depends on them**. The model's private domain, graph, FX and persistence code do not become application-owned semantic authority merely to simplify extraction. Cross-model commitments pass through the owning public `*.core` interfaces and admitted exported contracts.

The pilot must not invent a persisted `:request/helper-id` field, a `:completed` lifecycle state, or an `organization/claiming-enabled?` Boolean solely because an example proof is easier with them. Part VIII's canonical Request lifecycle is `:open`, `:claimed`, `:on-the-way`, `:done` and `:cancelled`; an active primary RequestAssignment is a separate authoritative entity with its own history and constraints. A derived view may offer an equivalent display projection only if its source mapping, eligibility, scope, provenance and coverage are themselves qualified. Illustrative schema symbols and sample operation names are not declarations of production facts.

The compositional chain and edge categories

The required root is a **real application** proof/evidence graph. Its conceptual shape is:



This diagram contains **different kinds of edges**. Admitted justification/guarantee dependencies must be well-founded. An `implements` or `installed-with` edge needs source and physical correspondence evidence; a runtime event edge needs actual observation, actor and execution lineage. A matching node or proof-DAG hash cannot establish those other kinds of relationship. Retry, reassign, reconnect and subscription loops belong to the runtime graph, not circular proof justification.

Every source-relative proof node identifies its normalized subject, original producing declaration, selected checker/profile semantics, required exported guarantee, actual dependencies, used premises and evidence class. Every realization node additionally identifies the selected callable/route/provider/build and the installation generation to which its claim applies. A later observation node

identifies the specific command, principal, operation target, physical attempt, source and scope of its observed result. Copyable root reports explain these links; they are not capabilities or certificates in their own right.

Independent Claim obligation manifest

The required obligation set is derived from the **original** source and requested assurance profile, then checked against the supplied evidence. A well-formed graph whose developer omitted Reassign, a writer, a required Claim premise or an outcome branch must not pass because the remaining subset happens to be consistent.

Subject	Required source/realization obligation
Public model truth	Actual Request, RequestAssignment, User, Membership, Organization and Location public interfaces are source
Input and identity	Exact Request target, operation and canonical arguments, authenticated principal, target helper when distinct,
New-command eligibility	Current Request lifecycle, actor's effective Membership, relevant Organization/Location policy and relationship
Abstract state machine	Reachable initial states, every declared Claim-related invariant, all relevant operations and guarded/frame tran
Atomic incorporation	Qualified same-transaction guard on actual protected prestate/absence conditions; Request/assignment change
Rejection/uncertainty	Malformed and unauthorized invocation remain distinguished from a model's justified new-command rejection
Compatible replay	Original command/target/actor/arguments and historic disposition are resolved without a second incorporation
Operation interface	All four `:committed`, `:rejected`, `:already-incorporated` and `:commit-unknown` variants and their exact proven
Global Choreo	Roles, branch discriminator knowledge, protected communication, causality, non-escalation, outcome/unknown
Projections/plan	Helper and authority projections preserve their qualified global contract; issued runtime plans cover actual adr
Installed application	Real rendered enabled Claim control, generated browser payload, actually served/loaded assets, trusted route a
Physical writer/host	Biff/Ring authentication, request mappings, actual XTDB gateway, other effect-capable routes, background/adr
Historical outcome	Actual command/transaction provenance, pinned compatible readback, durable retention and truthful reconcili
Browser adoption	Permitted authoritative card acquisition, source/target-specific HTMX completion, provisional retirement, curr
Assumptions/release	Every opaque policy, provider/atomicity, compiler, host, writer coverage, progress/retention and proof-kernel p
Independent product	A genuine eligible browser-originated Claim positive and relevant negative/concurrent/recovery histories work

No blank or fabricated item may be marked discharged merely to populate the manifest. A checked catalog is a claim about its specified abstract operations; physical writer completeness needs independent qualified inventory/correspondence, and a root must not borrow broad installed coverage from a caller-selected set of writer names. An inherited open premise remains linked to the **particular guarantee** using it, rather than disappearing into one undifferentiated list of trusted dependencies.

Worked Claim history: actor, assignment and atomic boundary

Let Alice be an authenticated, currently eligible helper, and R17 an authorized Request whose public model says `:open` with no active primary assignment. Alice's adopted card carries the correct Request identity and a qualified observation frontier X17. X17 is not proof that the Request, Membership or Organization state is still eligible at a later transaction. The actual rendered Claim affordance binds the Choreo operation, compatible executable plan, concrete DOM target and current asset/assembly generation, not a browser-derived authorization token.

1. The real browser control generates logical command C42 and an independent physical execution/transport attempt. The test does not fabricate `__gesso_live_optimistic_command` or the wire envelope on behalf of a disabled or broken control. A lower-level API fixture that constructs such input is a separately labelled **route-only** test.
2. The installed server route selects the admitted operation and obtains Alice's authenticated principal from trusted ingress. It checks canonical Request/command arguments and source compatibility; the browser role `:helper` is not evidence of effective helper Membership or authority to act for a different person.
3. A compatible prior C42 binding is reconciled first under its retention and disclosure rules. Otherwise the real public Request operation applies the full current guard to actual Request, RequestAssignment, Membership, Organization/Location and command exclusion dependencies at the qualified atomic boundary. A pre-read, rendered card or post hoc comparison is not that guard.
4. At most one previously open R17 can acquire a newly active primary assignment from competing different commands. An authorized new Claim establishes the required Request/assignment, command disposition and publication-intent facts under its actual selected atomicity and writer contract. A rejected competitor must not leave a partial primary assignment.
5. The participant settles only from the reported authoritative outcome and justified acquisition. The backend's four dispositions map explicitly to its separate `:accepted`, `:rejected` and `:unknown` presentation statuses. HTTP 200, optimistic “Claiming…”, copied certificate or SSE delivery does not mean incorporated.
6. The installed target/card read and physical DOM completion give actual scoped adoption under the permitted acquisition and current owner/target identities. A newly committed Request/assignment confirmed by authorized fresh read/reload is independent evidence; a visible provisional badge is not.

If Bob, also eligible, races for R17 using a different command, the qualified model guard must preserve one-primary safety under the actual backend concurrency semantics. Concurrent clicks or success-shaped responses without contested guarded incorporation are not a useful physical race oracle. If Alice's Membership is revoked after the card was rendered, a new Claim must be rejected at the real trusted model boundary; the old client affordance does not grant authority.

If C42 incorporated but its response was lost, Alice can have an unknown presentation while the historical transaction is already committed. A compatible retry retains C42 and its original binding, resolves its **historical** result where retention/qualification permits, and must not perform a second Request assignment. If a later legitimate Reassign changes the current helper, C42 may still have a true past incorporation and a different currently read Request card. An expired command horizon can leave honest indeterminacy; it cannot be silently converted to rejection or a fresh command.

A committed transaction followed by broken HTML, failed OOB install, missing Live delivery or rendering rejection stays committed. Recovery requires separate current disclosure, acquisition and physical installation rights. Neither browser cleanup nor a later mismatch report can retroactively undo the domain write.

Other real writers, Reassign and invariant preservation

The pilot's main safety proposition may be expressed as “at most one active primary RequestAssignment for a Request in each relevant authoritative state,” under exact Request/assignment model semantics. Its source-derived induction covers shared initialization and **every** operation that can change the protected aggregate in its selected catalog. That includes new Claim, Unclaim, lifecycle completion or cancellation, manager Reassign, collaborator/assignment changes, Request edits where relevant, administrative repair and any additional registered protected mutation. The source checker must not quietly shrink the catalog to Claim to make the induction pass.

Physical correspondence is more demanding. Effective Membership and Organization policy may be changed by writers outside the Request operation's own namespace; an invariant about **current helper eligibility** must either include those transitions and repair/rule semantics, narrow the invariant to what the operation actually enforces, or retain the external interference premise explicitly. The actual host's discovered writer inventory is not a proof that all processes, raw XTDB paths and external writers were found.

For Reassign, the manager actor, selected replacement helper and existing primary helper are distinct. A real approved target selection and current helper/manager Membership are revalidated at incorporation, and the old primary's historical end and new primary's activation are represented by one admitted operation under its actual atomic guard. “Take over” is not automatically the same semantic action. The pilot's Claim success does not establish these independent Reassign positive and negative gates.

Any abstract state-machine model checked by Recife, Quint/TLA+ or another checker is generated or linked from admitted Gesso semantics with original-source coverage and checker trust premises. Application developers are not required to maintain a second hand-written specification or copy proof identities. An exact finite witness over the stated bounds and a conditional induction over the declared catalog retain those scopes; neither alone proves the real XTDB prestate, all physical writers or host execution.

Browser and selective multi-client realization

The vertical Claim slice must traverse the real issued browser plan, compiled/served asset and generated command, installed Ring/Biff route, public model call, justified outcome and card installation. The strong profile additionally needs qualified evidence of the **actual installed** handler and loaded browser runtime. A copied manifest stamp, an in-process issued callable not installed in the server, a deterministic source replay or a synthetic successful request cannot substitute for these occurrence-level facts.

The proving application also exercises a separately correlated Create under its real production model: the creator may immediately install a permitted local new Request/card S without importing unrelated remote collection revision R. For another participant, remote creation and lifecycle changes, including changes to an already-visible Request, remain advisory. The user explicitly grants a scoped one-Refresh acquisition; actual provider coverage and completed target installation discharge only the obligations it covers. A new advisory, pending refresh debt, a newer-looking basis token or a single card is not an implied whole-board read grant.

Search, sort, mine-first/unclaimed-first, terminal visibility, details, scroll/focus and local draft continuity operate on justified adopted cohort state, never as excuses to silently pull remote changes. Unknown/rejected Claim and refresh failures retain usable bounded recovery and correct owner retirement. The pilot may test a selected Claim slice without declaring that the full rich board or response-flow product requirements have been accepted.

Three independent conclusions and final product gates

The pilot reports three independently evaluated conclusions:

1. **Semantic certification:** the model/rule/operation/Choreo/ projection/plan construction establishes its exact source-relative propositions with scoped proof evidence, checked complete dependencies and still-open assumptions.
2. **Production realization:** issued functions and actual installed host, physical writers and transaction/outcome, generated and loaded assets, real browser affordances and adoption relate to that construction with authenticated, correctly scoped correspondence or explicit open premises.
3. **Independent product acceptance:** real eligible browser-originated interactions and current production Request/assignment truth satisfy the independently selected product behaviors, adversarial controls and fault/recovery scenarios.

Each conclusion has different failure modes and evidence owners. A green browser test cannot fill a missing abstract induction or projection proof; a complete abstract theorem cannot install a handler or prove the Claim button works. A successful isolated Claim pilot is a milestone, not the completion of HumanHelp.

Part VIII independently requires eligible positive cases for Create, Claim, Unclaim, Mark on the way, Complete, Cancel and manager-target Reassign, plus authentication, rich-board continuity, two-client advisory/Refresh, recovery, actual generation/host and assurance qualification. Its fourteen separately specified completion gates remain product requirements irrespective of the number of operations the generated catalog currently contains. Deleting a required operation from both program and proof cannot turn a missing feature into a success.

The HumanHelp response-flow decision remains a genuine open product decision under #decision-humanhelp-response-flow. The pilot must not invent a messaging/contact policy or equate "response" with Claim simply to make the root report green. If a product authority later selects a precise response interaction, that separately versioned choice and its eligible positive, negative, privacy and recovery tests enter the original-source closure and independent product acceptance.

Generated root report and certificate boundaries

One readable **illustrative** root report is:

```
{:root {:application :humanhelp/example
        :operation :request/claim
        :assembly/ref assembly-ref
        :profile/ref profile-ref
        :source/ref original-application-source}
 :claims [{:claim/ref typed-claim-rule
           :evidence/ref checked-rule-evidence
           :premises []}
          {:claim/ref unique-primary-preservation
           :evidence/ref conditional-induction-evidence
           :premises [catalog-coverage-premise
                      implementation-conformance-premise
                      physical-writer-premise
                      backend-guard-premise]}
          {:claim/ref issued-to-installed-claim-path
           :evidence/ref scoped-host-correspondence
           :premises [router-adoption-premise]}
          {:claim/ref real-browser-claim-scenario
           :evidence/class :tested
           :evidence/ref actual-two-helper-run}]
 :dependency-closure/ref original-source-derived-closure
 :obligation-ledger/ref complete-claim-obligations
 :selected-derivations/ref checked-proof-choices
 :open-premises/ref exact-claim-scoped-premises
 :application-evidence/ref installed-host-and-asset-evidence
 :product-gates/ref independent-humanhelp-capabilities
 :unresolved-obligations/ref actual-open-items
 :applicability/ref current-build-host-provider-scope
 :decision/ref profile-admission-decision}
```

These maps depict **readable output**, not user-authored certificate input or proof of existing implementation behavior. References must resolve to trusted subject-bound records with claim, checker version, source/provenance, scope, open assumptions, evidence mechanism and actual result. A label like `:tested` must identify the real executed scenario and its complete allowed assertion, not merely a test name or passing definition. A successful typing claim with no domain premises does not eliminate its proof-kernel interpretation and trust assumptions.

The root reports source/type/operation evidence, projection and plan guarantees, implementation and host bindings, real command/transaction/read/DOM evidence, policy and provider premises, untested product capabilities, and every unresolved required obligation separately. Admission is a typed decision under the explicitly selected profile, not a global Boolean. An accepted conditional root carries its premises and does not grant runtime authorization. An unfinished source-to-host link or unselected product response cannot be hidden behind an artifact-wide `:accepted` stamp.

Paired acceptance and counterexample corpus

The pilot must admit genuinely valid source/physical/user scenarios and reject nearby defects at their **actual first** enforcing boundary. The corpus includes at least:

- valid public Request/RequestAssignment schema and Claim rule; missing/renamed status, invented helper field, incompatible relationship cardinality and wrong model namespace must reject or become explicit unsupported scope;
- a complete original operation/invariant catalog with Claim/Unclaim/Reassign cases; a pruned but structurally valid proof DAG cannot pass source-derived coverage;
- valid Organization/Membership/Location guarded Claim; stale Membership, wrong tenant, fabricated browser helper status and a manager acting for an ineligible target cannot pass the protected production boundary;
- a qualifying two-helper same-Request race with at most one new primary; two independent pre-read-and-write paths or a partial Request/assignment mutation fail;
- actual C42 incorporation, lost reply and compatible historical reconciliation; new-command retry, unsupported record retention and post-Reassign current-card substitution cannot manufacture history;
- source-issued and actually installed executable routing and browser bytes; uninstalled handler, copied certificate, wrong Choreo projection, changed asset or anonymous HTMX fallback cannot meet the strong profile;
- real completed target/card adoption with proper stale-target cleanup; HTTP success or provisional badge cannot count as installed state;
- legitimate creator-local S and remote advisory-only R with one eligible user-granted Refresh; an automatic remote whole-board adoption, second ungranted read or local S erased by older R fails the declared policy;
- conditional proof preserving opaque model/provider assumptions and actual writer limitations; a forged status map, dropped premise or fabricated production observation cannot issue a stronger certificate;
- independent positive Unclaim, Mark-on-the-way, Complete, Cancel, Reassign, board interaction and eventually selected response flow; a reject-all checker, disabled controls or a reduced authored catalog cannot satisfy the missing product gates.

The expected disposition and first semantic, model, source-coverage, construction, installation, transaction, physical effect or independent product-test boundary are specified **before** each mutant. An abstract counterexample is not an authenticated production history, and one real successful browser trace is not universal refinement over all deployed executions.

Profiles, diagnostics, and policy review

A certification profile is a **versioned admission policy** over exact claim/proposition families, mandatory judgments, permitted evidence mechanisms, open premise classes, implementation and installation applicability, and positive acceptance requirements. It is enforced at the real construction and relevant installation/effect boundaries. A profile is neither a percent of the application's Clojure lines nor a global toggle between "proved" and "unchecked." The same operation may have kernel-checked typing, finite-model evidence for one invariant, a conditional abstract transition proof, a runtime guard obligation, an opaque policy implementation premise and separately tested browser behavior. None of those classifications silently absorbs the others.

The selected profile cannot disable the baseline semantic-route, application-assembly, trusted-ingress and rendered-affordance closure required by Part VII. When a stronger profile asks for unavailable proof or physical correspondence, the result is a typed failure or explicitly requested weaker product. The constructor does not automatically downgrade a managed operation to an ordinary HTMX POST, interpret a trusted assertion as checked derivation, or call a timed-out verification successful.

Executable policy, claim-specific admission and applicability

An admitted profile specifies a stable profile identity/version and, for each supported proposition category, at least:

- the original-source and required-obligation inventory owner, semantic/checker rules and accepted evidence class;
- which judgments are **mandatory** and where all of their cases, premises, dependencies and non-vacuity witnesses come from;
- which well-typed, precisely scoped opaque or environmental assumptions may remain open and which must be discharged;
- the required source, checker, compiler, model, host, provider, browser and writer correspondence, when the claim requires it;
- the applicable build, deployment, principal, resource and observation-generation boundaries, including when rechecking or requalification is required;
- how incompleteness, unsupported statements, resource exhaustion, contradictory premises and physical uncertainty affect issuance;
- any independently selected product capability, privacy, safety and progress gates that cannot be inferred from a proof catalog.

A baseline compositional profile checks closed source categories, types and interpretation, source-derived complete obligation and dependency closure, soundly bounded admitted derivations for claims described as checked, exact original subject/proof identity, and allowed premise accounting. It permits an **explicit** opaque implementation or provider premise only for the specific claim and scope its policy allows. Proof reports and supplied claim descriptions are never accepted as issuer authority.

A stronger conditional-induction profile requires an inhabited initial-condition argument where applicable, every invariant base case and every operation/invariant preservation judgment of the selected original catalog, the correct frame/guard semantics, and precisely inherited open premises. A valid but unsupported abstract effect requires a qualified dedicated checker, supported refinement, or rejection of that requested strong profile; neither eliding the operation nor calling its preservation "trusted" yields an induction theorem. Coverage of a declared operation catalog is not complete coverage of physical non-HTTP writers or initializers.

A production-correspondence profile additionally requires the specified trusted source-to-issued executor, actually installed handler, writer/guard provenance, source-to-asset binding and physical effect/observation evidence. It must say whether its conclusion is an abstract model judgment, a bounded test observation, one authenticated production occurrence or a universal refinement under named simulation and external premises. Merely selecting a profile with the word "production" cannot authenticate a caller-supplied route inventory, ledger, basis, browser stamp or purported transaction outcome.

The profile's interpretation, accepted external envelopes and checking rules are themselves dependencies. New code, a changed checker/kernel semantic version, withdrawn assumption qualification, changed installed assets or changed policy requirements invalidate **affected** consumer admission as specified by 09.10–09.12. An old certificate cannot veto a new policy or retain revoked source/physical applicability. Immutable unrelated abstract proofs may be reused when their actual claims and trusted dependencies remain valid.

A checked semantic source and its profile may be edited or revised by application authority, but a consumer must use the selected, currently admitted profile and the appropriate application/deployment generation. Source-selected compatible profiles must not be replaced by an untrusted request parameter or report value. A disabled or omitted optional checking gate is still different from a passed strict gate; the report must make that distinction apparent to release reviewers.

First-blocker diagnostics with evidence and repair context

A rejected or incomplete construction produces a stable structured diagnostic rather than only an exception string, unqualified green badge or cascade of dozens of dependent errors. Each primary diagnostic identifies the original subject, required proposition,

source span/path when available, selected checker/profile, required versus supplied types/facts/bindings, attempted use site and role, evidence status, first **actually enforcing** boundary, dependency chain, and artifacts/effects not admitted. Additional independent failures may be reported, but downstream objects that were never constructed are consequences rather than new defects. Lexical iteration order of an unordered map must not randomize a first-blocker selection.

A diagnostic's **source authority** is as important as its code. A copied report cannot establish that a check ran; an unauthenticated physical snapshot cannot be reported as a transaction-level witness. Source-derived diagnostics may be copyable and useful without granting any proof certificate, model-write permission or browser capability. Sensitive counterexamples, principal/tenant facts and private policy inputs must be redacted or represented by authorized projections; a useful explanation must not become an unintended disclosure channel.

The principal classification distinguishes:

Category	Meaning and first relevant boundary
Malformed syntax or type	Invalid reader, unknown source reference, nominal mismatch, illegal comparison, cardinality or unsu
Refinement/presence unavailable	Reading an optional/nil/absent field or using a value before the required branch-local refinement hol
Knowledge/provenance unavailable	The correct rule is used at a role/program point without justified source, disclosure, branch knowled
Authority/resource unavailable	Required trusted placement, current actor/resource rights or physical permit is absent at the enforcir
Proof/derivation invalid	Unknown rule, missing premise, bad inference/substitution, wrong subject, forged status or mismatc
Required coverage missing	Original invariant/operation/branch/source dependency or independently required product capabilit
Dependency/version stale	Source semantic identity, checked interface, checker profile, callable, provider, artifact or installation
Counterexample	The admitted model and explicitly checked assumptions permit a trace violating a declared invariant
Unsupported judgment	Valid-looking construction asks for a checker/fragment, physical inference or guarantee unavailable
Incomplete checking	Timeout, resource bound, cancelled or failed checker, unqualified observation, unavailable dependen
Profile/premise not admitted	The required assumption, checker category, release policy or product guarantee is disallowed even tl
Advisory intent finding	A coherent checked policy appears suspicious or inconsistent with independently recorded business

A failed writer/authorization check after the effect is **detected post hoc**, not evidence of pre-effect prevention. A nominal proof counterexample is not an authenticated deployed history. An unsupported tool is not a failed proof of the intended statement, and a reject-all application does not satisfy positive product requirements just because no forbidden transition was observed. The response reports those distinctions with the original source, missing obligation and trust boundary.

Diagnostics should be actionable: for example, if `:request/status` is a closed nominal enum, the expression `(< request.status 10)` is malformed because no ordering exists between `RequestStatus` and `Integer`. The report names that operator, actual types, source rule, blocked `VerifiedRule`, and downstream Claim/Choreo/plan products that were **not** issued. It may recommend using an admitted status-equality/membership predicate, but must not silently rewrite the author's eligibility policy.

If `request.rules` was checked against Request schema S17 and S18 replaces `:claimed` with `:assigned`, the new schema may be valid; **composition** rejects the old dependent as stale. The diagnostic identifies both interfaces and the relied-on changed guarantee rather than calling S18 malformed or a copied cached certificate current. Source and implementation changes have distinct invalidation paths.

If an otherwise well-typed `approved?` predicate reads private `request.private-review` in a helper-browser branch before authoritative disclosure/knowledge, its source typing remains established while its particular distributed use is rejected. The report names the role/state/required provenance, and proposes a permitted server-derived public decision or correctly scoped acquisition **if** its authority and privacy preconditions can actually be met; it does not suggest leaking the private field.

Mechanically invalid, formally counterexampled, product-wrong

The semantic checker proves/checks what the application **actually declared**, under its recognized semantics. It cannot infer the business owner's desired policy from prose, an interface's friendly name or a neighboring rule. In a typed age policy example:

```
;; Schematic authoring syntax; not an asserted existing API.
(rule adult? (> user.age 18))
```

This is a coherent rule when `user.age` is an integer, even if product authority meant "18 or older." In contrast, comparing `email` (an email/string nominal type) with `18` using `=<` = is a semantic type error. A source-level checker should reject the type error and accept the well-typed rule unless some **independently declared** invariant or property is contradicted.

Where a well-formed rule allows a trace violating an admitted property, the checker can report a reproducible bounded/abstract counterexample together with its model, state, input, transition and assumed scope. It must not infer physical XTDB execution from a synthetic trace or invent a new business rule in the process.

A reviewer might regard the well-typed age rule, an asymmetrically permissive Claim, a missing helper-role condition, an unexpectedly reopening terminal Request or a weaker disclosure rule as suspicious. That is an **advisory intent finding**, not an erroneous proof-kernel rejection. Application ownership determines whether the policy changes. If accepted, the new intent becomes a source-owned explicit rule, invariant, operation contract or product acceptance gate, and normal source derivation/checking runs again. It is never quietly patched into a checker or browser adapter to conceal a policy change.

Keep human-readable meaning, rationale, security/lifecycle correctness notes, examples and counterexamples adjacent to the source semantic declarations, and carry them through audit views where feasible. These descriptions are review evidence, not hidden axioms or silently executable authorization tests.

Semantic-policy inventory and escape ownership

Gesso compresses correctness-relevant application decisions into inspectable named rules, interfaces, operations, invariants and explicitly scoped implementation/host premises. The goal is **semantic compression**, not whole-program proof of arbitrary Clojure. A condition belongs to the assured surface when it changes a claimed correctness decision, including actor/target authorization, capability, protected disclosure, operation eligibility, lifecycle, assignment/ownership, authoritative outcome, protected data movement or a claimed finality/non-regression invariant. Incidental styling, logging, collection formatting and other implementation branching do not automatically become business rules merely because they use `if`.

For every **inspected** correctness- relevant policy dependency, the report identifies one of:

- a named admitted rule/judgment with original source and evidence;
- an imported checked semantic guarantee and its scoped premises;
- a runtime-enforced fact with actual trusted observation/effect boundary and instance scope;
- a registered opaque implementation contract with exact callable, dependencies and accepted profile;
- an unresolved detected semantic- policy escape that cannot yet support the requested strong correspondence claim.

The constructor derives coverage from declared source, and checking of generated/closed bounded code may establish a complete policy footprint for that **specified** subset. An actual registered execution may reveal an undeclared guard, special case or writer. When observable behavior influences a claimed protected decision but is not accounted for by admitted rules or an explicit, allowed opaque contract, the result is a detected semantic-policy escape. The report identifies its actual inspection/observation boundary, affected proposition, source and needed repair. Moving that dependency into a named model rule, deriving it from an admitted interface, binding it to a qualified runtime condition or explicitly accepting an opaque implementation premise are distinct choices with different assurance strengths.

Ordinary arbitrary Clojure code can contain closures, macros, dynamic Vars, reflection, third-party code and side effects that a bounded analyzer cannot enumerate. Silence cannot establish the **absence** of hidden policy within those uninspected bodies. "Zero detected escapes" must say exactly which code, paths, source versions and effects were inspected; uninspected code is `unknown`, not an empty rule catalog, a negative test result or a complete host writer inventory. An opaque contract can be legitimate under a selected conditional profile, but naming it does not prove its realization implements the promised domain predicate.

The policy audit must not maintain a competing source of business rules. It derives available views from the admitted source/operation and trusted inspection reports, and clearly marks where they fail to expose the original model's private implementation or unknown physical entrypoints. Discovering a potentially relevant condition in host code is an explicit review question; the generic framework must not silently declare it part of the product's intended policy.

Coverage metrics, review authority and promotion

The assurance report includes scope, source/canonical subject, dependency closure, relevant evidence class, accepted open premises, actual inspected implementation subset, uninspected bodies, known policy escapes, first unresolved obligations, required independent product gates, deployment applicability and any review findings. Each required claim can carry multiple evidence facts and applicability conditions. The numerical counts of checked, finite-checked, runtime-enforced, trusted and untested propositions are not necessarily disjoint. No single "percent of correctness proved" can be obtained by adding them.

For example, a policy view might show 173 registered rule identities, 80 kernel-checked propositions, 61 exact-finite-checked propositions, 18 runtime-enforced premises and 14 trusted/opaque premises—numbers that may overlap, are illustrative, and make no claim about every rule in the actual codebase. It could simultaneously report four uninspected opaque model bodies, zero **detected** escapes in a closed checked subset and three unresolved advisory policy findings. This is neither "zero unknown business decisions" nor a measured 100% proof coverage. The report always names the actual subjects and provenance behind its figures.

An advisory reviewer, including an LLM or static heuristic, can use this compressed semantic view to flag suspicious asymmetry, contradictory rationale, permissive guards, missing actor constraints, unreachable real capabilities or untested privacy cases. The reviewer must not issue certificates, change current state, discharge assumptions, suppress valid implementation paths or rewrite model policy. Application owners decide whether a finding is a requirement and authorize the semantic source change; a checked constructor then derives and validates the revised obligations. This is a one-way authority distinction: suspicion -> policy decision -> explicit source contract -> mechanical checking.

Product acceptance is separate from semantic admission. In HumanHelp, the real Claim/Unclaim/ Reassign controls, valid actor and target, selective creator-local Create, remote advisory and one authorized Refresh, real two-client browser outcomes and the separately selected response interaction have independent positive gates in Part VIII. A checker that excludes every admitted action can be safe under a narrow abstract model yet cannot satisfy these product requirements. The root report must show the actual status of each gate; a missing response-flow decision is not solved by renaming Claim.

Positive and adversarial acceptance cases

For the finished framework, profile, diagnostics and review behavior must be tested at the actual specified boundary with paired valid and invalid cases:

- a valid typed original Claim source and chosen conditional profile passes its mandatory checks while preserving open provider premises; a missing mandatory invariant/operation case cannot be relabelled trusted to pass;
- a real requested strict induction with all original invariants/ operations passes; removing one while supplying a valid smaller proof DAG fails at source-derived coverage;
- a valid source uses a separately qualified opaque implementation; an unregistered implementation or silent "assume true" downgrade fails;
- exact source/checker/ profile/assembly scope permits applicable cached proofs; changed kernel policy, host asset or live callable invalidates the affected claim;
- valid `RequestStatus` equality passes; enum numeric ordering and wrong nominal endpoint fail with correct typed source location;
- a well-typed `adult? (> age 18)` remains accepted absent an explicit policy contrary obligation; the equivalent `email < 18` expression fails source typing;
- a real well-typed rule that violates an independently declared invariant yields a model-bounded counterexample, not arbitrary business intent correction;
- a helper-browser branch with justified authorized knowledge passes while the same rule without private observation/provenance fails use-site admissibility;
- an old Claim certificate depending on schema S17 fails composition under incompatible S18 while independent unaffected claims can remain reusable;
- a truly checked source provides reproducible first-blocker diagnostics, whereas copied results, changed map iteration and spoofed statuses cannot alter admission or diagnostic attribution;
- a closed generated guard with complete declared read footprint may establish scoped zero unresolved detected escapes; an opaque arbitrary model cannot claim the absence of undiscovered policy decisions;
- a detected authoritative exception branch is either source- linked or reported as an escape; simply registering a whole body as trusted does not prove its inner policy;
- genuine authorized review promotion changes the source contract and triggers checking; an LLM comment alone cannot issue proof or rewrite application policy;
- a real eligible Claim/ Reassign and scoped remote Refresh remain usable under selected product gates; rejecting every operation or silently adopting remote content cannot qualify;
- resource exhaustion, provider/host unavailability and unsupported proof tools yield typed incomplete/ unsupported status instead of success or a fabricated counterexample.

The case oracle records input provenance, the required claim, expected result, first enforcing source/checker/ assembly/physical boundary and independent valid control before test. The test corpus in 09.15 owns mutation injection and coverage execution. It must distinguish an actual physical observation from a merely source- level report; no copyable diagnostic can become a runtime authorization receipt.

Systematic mutation and conformance corpus

The mutation corpus is an **executable adversarial specification** of Gesso's admission, semantic interpretation, proof, source-closure, application realization and physical-effect boundaries. It must establish that a supported invalid change is rejected by its actual responsible gate **and** that an unchanged valid program remains constructible and usable under the same declared profile. It is not a percentage of code covered, a replacement for the independent HumanHelp product oracle, or a proof of soundness of all Clojure, XTDB, HTMX or the browser.

The corpus contains several independently meaningful positive baselines: small closed source terms for exact semantic checking; valid proof and conditional-premise constructions; real admitted operation, Choreo and ApplicationAssembly sources; and the production-backed HumanHelp Request/RequestAssignment Claim slice from Part VIII and 09.13. A mutation that only changes a checksum before the claimed semantic decision is reached exercises identity rejection, not the more interesting purported invariant/authorization checker. Both are useful, but their conclusions must be named separately.

Predeclared case contract and expected dispositions

Each case is data describing a precise **claim**, not an opaque "this test should fail." Before execution the corpus fixes:

- immutable baseline/source identity, model/runtime profile, selected application/deployment generation and relevant provider versions;
- isolated mutation operator and its actual introduction point: original unadmitted source, proposed elaboration, stored untrusted package bytes, admitted dependency **candidate**, installed artifact, physical effect boundary, or an external environment/failure seam;
- affected proposition and checked judgment, assumed source coverage, the specific forbidden artifact or runtime effect, and expected valid behavior of an independent positive control;
- expected result, first enforcing boundary, stable structured classification and permissible alternative **early** failures only where their cause is actually prior in that path;
- authoritative checker, observation instrument and physical evidence needed to decide whether an unsafe effect occurred, including the test's own assumption/inspection limitations.

A case may be **correctly rejected**, **accepted by design** because it is well typed and coherent under the selected rules, or **correctly admitted with precisely stated conditional premises**. These are not interchangeable pass labels. An unexpected acceptance, wrong reason, broken valid control, unauthorized effect, forged evidence promotion, or unjustified release disposition fails the case. Resource exhaustion, checker crash, test harness interruption, missing instrumentation and an unsupported checking mechanism are **incomplete test outcomes**, even when the requested strong profile properly refuses admission. A negative test whose specifically expected disposition is `:unsupported` may establish that unsupported-feature boundary works, but not that the requested feature is implemented.

Each runner records the **actual** first cause and observation. If the direct composition API rejects stale dependency C21 against O15, its expected result may be `:semantic-dependency-mismatch`. The normal production cache may instead miss, re-elaborate and reject unchanged C21 for unknown old policy field; that is acceptable only for a distinct predeclared production-path oracle ensuring no stale executable Claim is admitted. It must not change a direct composition test's expected failure after the fact.

Required mutation families and protected meanings

Family	Perturbation and first relevant guarantee
Schema and nominal identity	Missing/retyped field, optional to required or reverse, nullable versus absent, different nominal e
Rule interpretation and evaluation	Boolean truthiness versus presence, nil versus missing, short-circuit or branch refinement reorde
Propositional derivation	Forged premise, shifted conclusion, changed antecedent, forward reference, incorrect conjunctio
Guarded transition and frame	Candidate prestate used as atomic prestate, wrong guard, omitted simultaneous assignment, wro
Inductive obligations and finite search	Omitted base/invariant/operation, changed shared initial state, incomplete reachability, contradi
Required source/dependency closure	Source-required graph edge/operation removed while the residual DAG remains structurally vali
Premise propagation and discharge	Copied assertion presented as proof, inherited premise dropped, evidence from different actor/R
Choreo and role-local admissibility	Unknown discriminator, hidden private branch, unsupported receive overlap, lost actor/source k
Outcome and durable command history	HTTP status masquerading as commit, compatible C42 replay written twice, incompatible comm
Browser scope and resource ownership	Wrong target, OOB partial install, superseded execution/lease, stale plan, duplicate one-use Refr
Certificates and untrusted admission	Success map, forged fingerprint, checker-status field, manually created opaque handle, stale/cycl
Issued implementation and actual host	Changed closure/Var, substituted handler, uninstalled issued executor, forged route/source inven
Physical effect and observation	Caller-supplied principal/node/prestate, second diagnostic transaction, mismatched real invocati
Completeness and checker budgets	Truncated model/trace, weakened type domain, omitted mutant path, timeout, unusable externa
Product policy and user capability	Accepted coherent but surprising rule, original HumanHelp Claim/Unclaim/Reassign positive re

These families are **minimum semantic coverage**, not a fixed list of particular file names. Every class requires a meaningful positive neighbor. A type checker can correctly reject `emailed < 18` while accepting a well-typed `age > 18` policy even if the owner intended `age > 18=`. If the latter violates an independently declared invariant, the expected result changes to a property counterexample, not retroactive classification as malformed syntax. Human intent is not an implicit rule generated by the mutation harness.

Typed semantics also require accepted and rejected conformance cases for optional `absent` versus present `nil`, nominal equality, ordered-vector versus unordered-map normalization, short-circuit presence guarding, branch-local knowledge, simultaneous writes, reader footprints and actual source-derived operation and invariant completeness. A changed rule can be syntactically valid yet fail its **use-site** authority, disclosure or temporal provenance obligation.

Injection points and immutable category discipline

Source mutations operate on fresh **unadmitted** production declarations, before their source construction/checker pipeline. A mutation of a purported elaborated proof is candidate input to an independent original-source checker. Serialized-certificate forgery mutates inert package bytes before safe decoding and rechecking. It must never be modeled by mutating a private, issuer-registered live handle as if such a handle were an ordinary writable application map.

An implementation mutation may replace an unadmitted callable or dynamically selected host dependency before issuance; a later deployment substitution must be injected at the actual trusted installation or runtime boundary. The two test different claims. An already issued certificate may describe an unchanged abstract theorem while its **physical applicability** fails against the changed handler, provider, served asset, or owner generation. It is not sound to credit a semantic checker with detecting a post-installation substitution that it could not observe.

All mutation operators declare their allowed edit footprint. Accidental changes to the baseline source, unrelated fixture users, provider trust assumptions or execution order can invalidate the test rather than exercise the intended claim. Corpus generation is deterministic from a recorded seed and original inputs, or reports the chosen randomized seed, shrinking/minimization history and replayable schedule. Isolation includes fresh checker registration, caches, XTDB test node/session, browser state and app generation where appropriate. An order-dependent pass is a test bug until its environmental assumptions have been stated and controlled.

For physical mutations, the harness uses an actual permissible effect seam: model guarded submit, real route/authentication dispatch, host-selected writer, generated browser-origin command, emitted and served client asset or real DOM target/adoption. A browser test that manually inserts `__gesso_live_optimistic_command` exercises the server protocol only; it is not the real browser-to-command acceptance oracle. Injecting a fake submitted transaction receipt does not establish an actual XTDB observation. Separate tests may deliberately exercise synthetic payload validation, but they report that narrower boundary.

Oracle ownership and prohibition of post-hoc credit

The mutant's returned report, exception or output value is **not** its own independent success oracle. A checker-origin verification result can justify its stated internal judgment; whether a forbidden artifact was issued or a physical effect occurred requires an independent collector at the relevant trusted boundary. For example, an instrumented writer sink must observe whether a RequestAssignment was mutated; an unrelated test-local boolean `:effect-observed? false` supplied by the mutant is not evidence.

The runner must distinguish all of:

- **Prevented:** admission or effect gate rejected before prohibited construction, submission, disclosure, acquisition or adoption, and a qualified independent observer establishes its absence within the declared observation window.
- **Detected after effect:** a checker accurately reported an inconsistency after an actual forbidden effect. This is useful diagnostics, but fails a policy whose obligation was prevention.
- **Unknown/incomplete:** the collector cannot establish whether a write, browser update, transport command or physical submission occurred, or the evidence retention horizon expired. Absence of a report or `nil` is not proof of non-occurrence.
- **Correct positive:** the legitimate original source was admitted and executed through the same relevant path with an independently observed permitted result.

The first enforcing gate can be rule typing, proof kernel, required-source closure, dependency composition, assembly/preflight, trusted ingress, atomic model writer, installed plan or browser effect resource. A negative test must exercise **the earliest actual applicable one** under the declared profile, not force every failure into a generic proof error. Conversely, a properly rejected abstract mutation cannot claim that the XTDB writer was tested at all.

A corpus-specific exception from the test harness is an infrastructure event unless the expected check genuinely comes from the production boundary being exercised. Unexpected errors and console failures are not suppressed to make a browser test green. The harness proves no absent behavior for physical paths that its instrumentation did not cover.

Representative case data and runner obligation

A finished framework can expose a declarative corpus runner without making application developers write proof terms. The following is a **conceptual fixture schema**, not an existing Gesso API or a set of guaranteed exported namespace functions:

```
{:case/id :organization/stale-claim-certificate
 :baseline/ref :humanhelp/request-claim
 :profile/ref :humanhelp/checked-conditional
 :injection {:at :certificate/compose-candidate
             :change :organization/014-to-015
             :candidate :old-claim-rules-C21}
 :claim/ref :request/claim-eligible
 :required-boundary :source-dependency-composition
 :expected {:status :rejected
            :reason :semantic-dependency-mismatch}
 :must-not {:admit :old-Claim-executor
            :perform :request/claim-incorporation}
 :independent-oracles
 [:fresh-original-source :issued-handle-registry
  :qualified-writer-sink]
 :positive-control/ref :humanhelp/repaired-claim-C22}
```

The corresponding production-path case can instead exercise a cache miss and fresh elaboration of unchanged C21, expecting unknown removed field or another predeclared earliest genuine source failure. Both must establish that old C21 is not installed. An unchanged source/name with a changed opaque implementation must have a **different** strong-realization mutant: valid abstract proof reuse does not authorize the new physical writer without qualified binding.

R4's conceptual runner and `:mutant/id`, `:baseline`, `:mutate`, `:exercise` and `:expect` declarations remain a useful design precedent. The implementation may choose different public data names. Required behavior is not dependent on those exact identifiers. A faithful runner reconstructs fresh baseline source for each case, verifies its valid control first, exercises the mutated source at the declared boundary, and collects separate checker and independent artifact/effect evidence. It compares the result against the **predeclared** oracle and emits a stable witness of where the mutant was rejected or deliberately accepted.

For readability, one schematic outcome record is:

```
{:case/id :projection/missing-helper-knowledge
 :baseline {:status :accepted
            :evidence/ref baseline-evidence}
 :actual {:status :rejected
          :boundary :choreo/use-site
          :reason :knowledge-not-established}
 :independent {:issued-invalid-plan? false
               :observed-unauthorized-effect? false
               :scope/ref inspected-boundary}
 :disposition :correct-rejection}
```

Such a record only warrants the disposition if independent observation is genuine and the scope is appropriate. An arbitrary map containing the same fields is not proof that the runner executed.

Positive, accepted and conditional controls

An adversarial suite is incomplete without successful constructions. For each injected defect, the baseline is independently rerun through the same supported path. The corpus also requires:

- semantically unchanged source formatting or unordered-map entry order that may reuse a qualified abstract proof while refreshing relevant diagnostics;
- a legitimate new Organization closed-enum policy and fully repaired Claim C22 whose checked dependency closure is accepted rather than rejecting all interface evolution;
- a coherent application-owned age/approval policy change correctly classified as **accepted by design**, separate from independent product policy review and release acceptance;
- a trusted opaque model implementation permitted explicitly under one conditional profile with its premises intact, contrasted with an inadmissible strong profile requiring evidence that is not present;
- non-vacuous initial states, enabled Claim/Unclaim/Reassign, real eligible Create and the chosen positive browser paths under current model authority;
- actual command C42 replay, rightfully rejected new incompatible command, truthful submitted-unknown handling, creator-local S, remote advisory R and one scoped explicit Refresh;
- benign incidental code/asset changes that do not require rebuilding unrelated abstract theorems, alongside invalidation of affected installation correspondence.

Positive product gates remain independent from proof/construction. Removing Reassign from both the source catalog and generated model may leave a safe reduced machine while failing HumanHelp's required user capability. A checker that refuses all Claim attempts cannot satisfy a positive Claim oracle. A successful Claim pilot cannot complete Part VIII's unselected response-flow product decision or other independent gates.

Independent semantic and physical conformance

Differential conformance compares truly separate semantic interpretations where practical, rather than two wrappers calling the same shared buggy evaluator. For finite propositional checking, a separately implemented Boolean valuation oracle can distinguish a semantically invalid accepted step; it cannot establish theorem truth of the opaque business predicates represented by atoms. For typed rules, the independent denotation needs to preserve presence, nil, nominal types, short-circuit order and exact reader footprints. Tests of normalization must actually execute the source and normalized forms over discriminating input states, not merely compare their pretty printing.

Where a typed/rule mutation changes the proof statement, the test must show a witness that activates its altered meaning or demonstrate the specific static rejection. A mutation that never changes the evaluated predicate is not counted as a meaningful semantic disagreement. A proof checker may reject semantically valid but unprovable statements under its selected limited rules; that is an honest unsupported/unproved status, not evidence that the formula is false.

Transition and induction oracles need complete declared initializations, relevant state assignments and operation cases. Finite search reports actual bounds, visited states, truncation and whether a counterexample is reachable under the exact abstract model. A model counterexample gives a reproducible abstract trace, not a physical XTDB history. A solver that times out or sees no states must not report invariant preservation as an unconditional theorem.

For installed host/XTDB and browser tests, the corpus records a qualified environment version, source/asset/plan and service generation, actor, command, application declaration, expected source-to-physical binding and actual independent observation. Tests should cover concurrent guarded RequestAssignment Claims, changed Membership, adversarial routing/writer bypasses, valid durable replay and lost result, real generated browser command, settled and unknown UI, and actual target-specific OOB

completion or failed installation. Causality and source completeness remain conditional until qualified evidence actually establishes them; matching transaction keys, ledger rows and cards can be mere co-observation.

Physical safety and qualified progress must have separate oracles. A lost SSE invalidation, missing outbox worker, retired browser lease or refresh-denied remote viewer can satisfy one safety proposition while leaving a bounded recovery or environment-progress obligation open. The expected positive behavior follows the **selected** HumanHelp Live/Refresh policy, not automatic whole-board convergence invented by the harness.

Reporting, audit and completion boundary

A reproducible corpus report preserves case ID, original and mutated source identity, profile, injection point, expected/actual structured disposition, checker and independent observation origin, first enforcing boundary, positive control, tested semantic/provider versions and relevant source/test evidence. It distinguishes missing case coverage, cases unsupported by a checker, incomplete executions, deliberately accepted application policy, correctly caught malformed programs, property counterexamples and physical effect failures.

A published mutant rejection score may help track regression in this specific corpus. It must not call an unsupported feature success, infer checker soundness, prove that every possible policy branch was examined, or convert a collection of bounded physical diagnostics into a universal refinement theorem. The independent root report ties the particular claim and evidence to the source/host generation, premises and product gate that they actually qualify.

The corpus can be incrementally expanded as Gesso grows. Its acceptance criterion is *meaningful discrimination at actual trusted boundaries*: correct negative cases reject, correct positive cases remain usable, and conditional/unsupported/ untested cases retain honest scopes. No future source addition, test refactor or cached summary may silently weaken a predeclared case's expected guarantee.

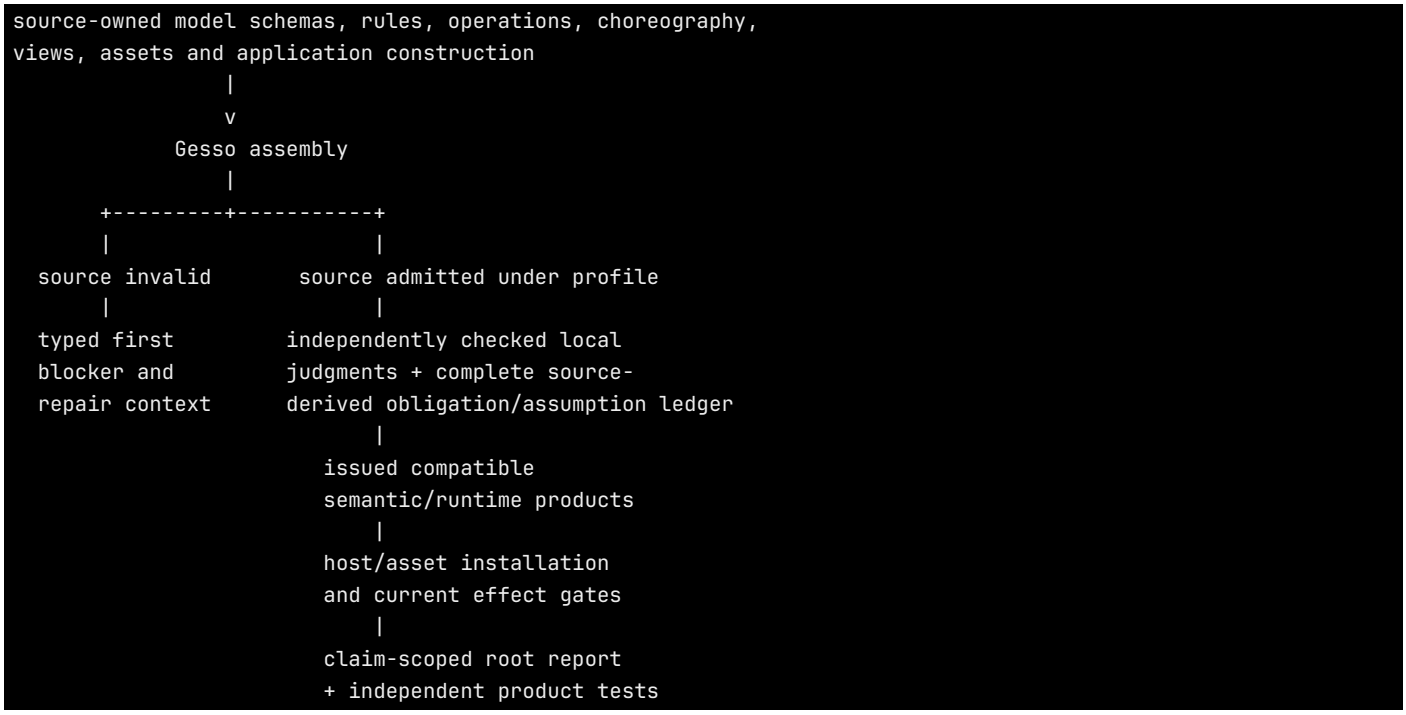
Developer workflow and completion criteria

Gesso's assurance architecture succeeds only when ordinary application development remains **source-first, declaration-first and repairable**. The developer owns the intended business policy, public model contracts, authoritative operations, Choreo, views and application assembly. Gesso owns the mechanical elaboration, scoped proof terms, required obligation inventory, checker/issuer bookkeeping, dependency identities, projection, derived plans and intelligible root assurance reports wherever its declared semantics can actually derive them. A developer must not maintain an independent TLA+/Quint/Recife/Lean program merely to use ordinary Gesso guarantees, copy transitive fingerprints into models, manually assemble proof DAGs or mint checker-issued handles.

The full Clojure language remains available for ordinary implementation. The closed, typed reasoning fragments are deliberately narrower than arbitrary computation and carry executable semantic interpretations. Where full-language code is opaque, its explicitly scoped implementation contract enters the same ordinary assembly path without pretending that Gesso has proved all of the function body. A user may choose a stronger profile requiring more independent correspondence; the system reports exactly what evidence is missing rather than silently granting it.

Developer-facing construction, feedback and ownership

The normal successful experience is conceptually:



This is a sequence of **semantic responsibilities**, not a required single compiler pass or a promise that every step is statically provable. The source-to-host and effect boundaries have independent evidence owners; a green source report alone does not install code. Likewise, browser and transaction execution can succeed while a stronger proof or source-coverage claim remains unestablished. Optional diagnostics may be computed and cached, but cannot alter issued runtime transitions or grant authority after plan admission.

For an ordinary user, routine IDs, strings, maps and Clojure values should not need hand-wrapped nominal/admission records at every expression. Typed wrappers and opaque issuer-registered products belong at the relatively few boundaries where a same-shaped raw value would break a real semantic or authorization invariant. The framework constructs those wrappers when it can; the developer receives stable diagnostics when it cannot. Manual creation of a similar-looking map is not a substitute for an admitted result.

The public workflow accepts explicit contracts for business facts only an application can own. It distinguishes a model assertion, a checked proposition, a conditional theorem with open premises, a physical authorization/observation, and an installed application capability. Declaring “this Membership is eligible” does not prove that a present Request writer checked it; qualified production authority still belongs to the model and installed trusted host.

Readable valid, invalid and conditional examples

The following is **schematic Clojure authoring notation**; it illustrates the desired contract without freezing exact macro spellings or asserting currently exported functions.

```
(rule claimable?
  (and (= request.status :open)
       (has-role? current-user :helper)))
```

With admitted RequestStatus and role sources, this can be an intrinsically well-typed rule. It is not, by itself, the complete HumanHelp Claim policy: current Membership, RequestAssignment cardinality, Organization/Location association and actual writer guards remain required by the governing public model contract.

A malformed use is:

```
(rule claimable?
  (< request.status 10))
```

No numerical ordering exists merely because RequestStatus is represented using a host-language keyword. The source checker reports the operator, actual nominal types, selected rule, required versus supplied facts, and the earliest blocked downstream artifact, without inventing a different policy.

A well-typed rule can also be **inadmissible at its use site**. A helper browser may not branch on a private authoritative `Request` review fact until the required role-local knowledge, provenance, disclosure and program-point conditions hold. Passing intrinsic type checking does not create a browser observation or an authorization to disclose the fact.

If upstream Request schema S18 removes a status variant required by rules certified against S17, the new schema may be coherent while the **old dependent** proof/operation/ Choreo/plan is stale. Construction rejects the mismatch at the first supported source/dependency boundary. A checked compatibility bridge or a re-elaborated valid rule may repair it; copied old signatures, names and fingerprints cannot.

A same-shaped `{:valid? true}` or `{:checked-result true}` map is never an issued rule or application certificate. A fabricated derivation with a missing premise fails its actual independent proof checker. A genuinely accepted conditional Claim certificate may still carry a named opaque Organization callable, XTDB atomic-guard, external provider or deployed-route correspondence premise; those are not silently upgraded to truths or hidden from the root report.

The developer's repair options for an unsupported or unestablished strong goal are specific: correct malformed source, provide a supported derivation/evidence mechanism, give a properly qualified explicit premise under a profile that permits it, change the requested **named** profile/claim with visible consequences, or revise the actual business rule with application-owner authorization. The diagnostic does not advise simply adding an assertion of the failed theorem, swapping a trusted principal for a browser parameter, or ignoring an unresolved physical writer.

Source ownership and derived artifacts

Each stage has an accountable producer/consumer. Public domain/model contracts and policy decisions belong to application model authorities; source normalization and required-judgment derivation belong to the specified Gesso semantics; proof checking belongs to the small kernel or dedicated checker with named assumptions; issued products belong to the admitted source/profile and dependency generation; actual routing and asset adoption belong to the trusted installation context; transaction and browser effects belong to their actual enforcing physical boundaries.

An application builder derives the complete **declared** closure, does not require the user to reconstruct transitive dependency hashes, and differentiates source-side completeness from independent physical writer/host inventory and production source binding. When source or policy changes, affected abstract proofs, runtime plans, implementation correspondence and installed resources are invalidated according to their own scopes rather than conflated into one namespace-wide success bit.

An unsupported opaque Clojure branch may be a valid explicit implementation premise for one profile and fail a stronger checked-causality or all-writer profile. A module exposing the same named public interface but a different callable cannot inherit unverified previous implementation evidence. The developer sees both which proof is still valid and which physical assurance requires requalification. A policy-intent finding from a human/heuristic/LLM reviewer remains advisory until the application owner deliberately changes the source-owned rule.

Compositional milestone acceptance contract

The framework's compositional-certification milestone requires **all** of the following independent gates. A gate is marked complete only with a declared positive control, adversarial near-neighbor, actual checker/host evidence and a source/profile scope that the test exercised.

1. **Precise source semantics.** Supported rule, typed value/reader/effect, relationship, operation, Choreo and proof fragments have executable, versioned syntax, interpretation and checking judgments. Optional absent versus present nil, nominal enums, short-circuit branch refinement, reader footprints, frames, integer/codec limits and source normalization have discriminating conformance witnesses.
2. **Complete original-source obligations.** Construction independently regenerates required initialization, invariant, operation/base/step, branch, knowledge, dependency and positive-path obligations. A structurally valid reduced proof DAG missing a difficult Unclaim/Reassign case cannot establish the original claim. Declared source coverage and actual physical writer coverage remain separate.
3. **Real checked issuance.** Kernel/dedicated checker conclusions are bound to their exact original subject, inference semantics, checked dependencies and evidence class. Forged report/maps, constructed handles, incomplete proof packages and fake rule tags cannot issue the requested product.
4. **Conditional premise accounting.** Premises propagate per **used claim**, with issuer, actor, scope, source, knowledge, time and implementation applicability. Genuine discharge uses checked evidence/typed substitution and inherits the replacement's remaining conditions; no trusted assertion is promoted by matching labels.
5. **Opaque full-language integration.** Ordinary Clojure implementation remains usable through declared contracts in the same authoring path. The source audit identifies checked closed subsets, explicit opaque implementations, actual detected semantic-policy escapes and uninspected decision paths, without assuming absent policy where no inspection occurred.
6. **Correct composition and reuse.** Exact semantic/proposition, implementation, installation and runtime identities remain distinct. Changed upstream commitments invalidate affected consumers; permissible bridges are checked and directional. Currentness comes from the trusted consuming assembler, not a caller's old mutually matching graph and source.
7. **Distributed use correctness.** Intrinsic well-typedness is separately checked from point-indexed knowledge, provenance, authority, optional refinements, observation/basis and correct role-local Choreo projections, with explicit unknown/ recovery alternatives.
8. **Production realization and authority.** Checked source connects to the actual issued executor, trusted server route, model writer, installed source/asset and browser plan with named qualified premises or authentic physical correspondence evidence. Pre-effect enforcement, post-effect detection, co-observed records and actual causality retain different claims. Provider assumptions remain versioned and scoped.
9. **Real HumanHelp Claim pilot.** Its root includes public production Request, RequestAssignment, Membership/Organization/ Location, Claim rule, required invariants, operation/Choreo, helper/authority projections, plan, assembly, installed handler/browser, atomic C42 result, selective card adoption and independent positive user outcome. Two-helper contention, Membership revocation, lost response, historical replay and one-primary safety have qualifying positive/negative evidence. A toy proof fixture cannot pass this gate.
10. **Independent product demands.** The full HumanHelp proving application keeps all Part VIII user-capability and fourteen completion gates, including real eligible Unclaim, Mark-on-the-way, Complete, Cancel, manager Reassign selection, creator-local Create, remote advisory, one explicit Refresh, continuity and the separately selected response interaction. A completed Claim proof milestone does not silently complete this independent product acceptance program.
11. **Adversarial conformance.** The mutation corpus has fresh valid baselines, isolated source/evidence/ installation/runtime perturbations, predeclared expected results, independent prohibited- artifact/effect oracles, actual first boundary, stable diagnostics and unambiguous incomplete/ unsupported statuses. Reject-all checking, infra failures and unobserved effects cannot be scored as success.
12. **Usable developer output.** Ordinary source derives the checked artifacts without proof-assistant bookkeeping. Error reports are repairable, privacy-conscious, source-relative and correctly distinguish malformed construction, model counterexample, stale dependency, unsupported analysis and coherent but suspicious product intent. The generated root report exposes every relied-on guarantee, checking boundary, open assumption, actual tested/physical evidence and incomplete independent product gate.

These are intended final-framework acceptance **obligations**, not an assertion that a present Gesso release already passes them. Different milestones can complete at different times; the report distinguishes their result classes and cannot advertise total completion on the strength of a subset or a successful single browser scenario.

Root report, release decision and open results

The root report is a readable source-derived inspection of an admitted construction, not a caller-writable admission token. It must let a reviewer answer:

- What exact application/ source/claim/profile and current generation were selected?
- Which original producer declarations and checked exported semantic interfaces justify each relied-on guarantee?
- Which mandatory proof/ source-coverage cases passed, failed, remain unsupported or were not attempted?
- What conditional premises were inherited, which were genuinely discharged, which remain trusted, and who owns their qualification/expiry?
- Which plans, handlers, model writers, physical source/assets, transaction observations, browser effects and deployments are actually bound or only asserted?
- Which independent positive product capabilities, races, retries, errors, recoveries and remote- acquisition rights were tested, missing or unavailable?
- What first blocked a stronger requested claim, what dependent artifacts were not admitted, and what legitimate repair/verification options remain?

The root does not turn ordinary checked schematic rules into instance-specific Membership authorization. It cannot treat HTTP success, SSE notification, a pinned domain record or a current Request card as a theorem of command incorporation. Nor may a successful real browser acceptance test forge a missing proof kernel theorem. Source, physical and product evidence are separate columns of the same inspectable construction, not a single `certified?` value.

The milestone does **not** demand a universal proof of arbitrary Clojure bodies, XTDB internals, HTMX, the JavaScript engine, the JVM or the unbounded real network. For strong claims that rely on them, it demands precise qualified premises, source-to-physical relations, checked or tested scopes and the appropriate runtime enforcement. External formal metatheorems about proof rules and universal refinement results may strengthen the root later; until then the report cannot invent them.

A release process checks that every selected mandatory gate has its applicable issued evidence and that permitted open premises are honestly accepted under the chosen profile. When a needed guarantee is missing, the stronger named release claim fails closed. A separately requested weaker claim may be usable without inheriting the stronger label, physical rights or false product completion.

Compositional certification invariant

A certified Gesso construction exports **only** the propositions warranted by the admitted source, their checked semantic judgments, independently required obligation/dependency closure, actual proofs or dedicated checker results, and explicitly attributed remaining premises. A raw declaration, normalizing digest, source filename, copied `valid?` field, test report, same-shaped Clojure map or uncorroborated host artifact never becomes an issued certified category by resemblance.

The full Clojure host language remains available. Closed typed semantic fragments have precise executable meanings; opaque functions enter through explicit, source- and implementation- scoped contracts with their assumptions visible. Policy correctness is not inferred from a well-typed expression: a coherent business choice can still be wrong by product intent. Mechanical invalidity, counterexample, unsupported checking and advisory review therefore remain different conclusions.

Intrinsic typing, source-derived obligation completeness, guarded transition/induction, finite-model checking, distributed knowledge/provenance, valid Choreo projection, issuer admission, source-to-implementation/installation correspondence, runtime actor/transaction authority, observed browser adoption and product acceptance are distinct judgments. They compose through checked, directional dependency and evidence interfaces without silently upgrading one into another or erasing open premises.

Justification dependencies are well founded; implementation binding, installation and runtime occurrence have their own independently qualified edges. Source changes invalidate affected dependents and may require checked compatibility; implementation or resource changes can invalidate physical evidence while leaving an abstract theorem intact. Cached reports, previous success, physical stamps and conveniently similar outputs cannot establish currentness or extend a one-use effect right.

The Gesso proof kernel checks only its admitted derivation language and named semantic assumptions. Dedicated checkers establish only their expressly stated judgments. Exact finite checking, conditional induction, controlled physical experiments and runtime effect enforcement each retain their actual subjects and quantifiers. No complete-looking submitted DAG can omit source-required cases; no successful readback automatically proves transaction causality or universal physical refinement.

HumanHelp Request Claim is the first substantial compositional proving application. Its typed public model and assignment semantics must connect through Claim operation, Choreo/projections, ExecutablePlan, ApplicationAssembly, installed route and asset, guarded model writer, trustworthy command-specific outcome and permitted browser acquisition to genuine user-visible adoption. An upstream semantic mismatch prevents stale dependent construction. An unverified writer, unknown command, stale browser target or disallowed assumption cannot borrow proof authority from a neighboring layer.

The independent HumanHelp product contract—including eligible operations, selective local Create, advisory remote updates, one user-permitted Refresh, recovery, continuity and an explicitly chosen response interaction— is not replaced by the Claim proof chain. A meaningful positive program must remain usable even as malformed and unsafe programs are rejected. The root report exposes all admitted claims, unresolved boundaries and independent product evidence so that developers, release reviewers and downstream checkers can rely on **exactly** the guarantees earned and no others.

Part X — Formal correspondence and system boundaries

Part X specifies when an assurance statement about Gesso's admitted semantics can also be asserted about **the application that actually runs**. Part IX defines claim categories, checked derivations, issuer-controlled construction and assumption propagation; this part specifies the additional semantic-transfer, host, transaction, browser, provider and deployment relationships that give such claims a physical interpretation. One must not equate successful model checking, correct assembly, real test execution and complete production refinement.

Gesso uses ordinary Clojure/ClojureScript construction and implementation, including opaque full-language functionality. Its supported closed semantic fragments admit stronger mechanized judgments; opaque implementations contribute named, scoped conformance premises. The application and its current trusted models still own business policy. A generated formal subject may strengthen a source-owned judgment, but cannot substitute a newly handwritten, more convenient application for the source or prove the current host installed it.

Single- and multi-node deployments implement the **same** application semantics, protected operation and participant permissions. Host placement and routing affect what correspondence, delivery, replay and failover evidence is necessary, not what a Claim means. The specific distributed architecture and acceptance cases appear in 10.16–10.19.

This is a timeless finished-system contract rather than a claim about the implementation at any particular revision. Parts XI–XII own independent completion and realization ordering. Every assurance conclusion retains the claim dimensions and evidence classifications of Part I, the compositional issuance boundaries of Part IX and the separate structural, physical and product categories of #req-application-assurance-claims.

Semantic ownership and the scope of assurance

One application meaning, several checking mechanisms

The source-owned **executable semantic core** specifies those supported state, operation, proof-relevant value and Choreo transition meanings that Gesso can inspect. Abstract model subjects, emitted plans, checked obligations and formal exploration derive from those admitted definitions under versioned normalization and interpretation contracts. The model need not use the same representation as a runtime program: where the representations differ, their **directional correspondence** is an independent proof/checking or explicitly qualified trust obligation.

An external formal model may be generated from the semantics, or a separate implementation of the semantics may serve as an independent cross-check. Neither receives an exemption from that obligation simply because it bears the original operation name. A more independent interpretation can detect common bugs, but then must show which exact source meaning it interprets; reusing implementation logic can help avoid drift, but creates common-mode risks. A trustworthy argument cannot be built by freely choosing whichever interpretation passes.

The rejected pattern is:

```
handwritten abstract model A → proof of A
handwritten production machine B → deployed application
(no checked relation between A and B)
```

The intended source-derived pattern is:

```
original Gesso/application declarations and contracts
                    |
            checked interpretation
                    |
        admitted executable semantics S
          /           |           \
  issued runtime    generated      checked formal
  plans/operations  obligations    subject/proof
          \           |           /
          explicit correspondence and assumptions
                    |
    installed host, actual physical execution, observations
```

The diagram is a dependency picture, not one proof that every link is already established. The issuer and trusted assembler reconstruct the required declarations and proof obligations from the **original** source. They may not accept an elaborator's conveniently reduced operation/invariant catalog as a complete application theorem. Part IX owns those original-source checks; Part VII separately owns installed application closure and its physical limitations.

Component	Governing responsibility
Source-owned executable semantic core	Defines supported transition and value/operation meanings
Normalizer and elaborator	Derive typed subjects, obligations, projections, plans and proof candidates from original decl
Dedicated semantic checker	Establishes an expressly stated typing, projection, finite or guarded-transition judgment
Small proof kernel	Rechecks admitted derivations from original proposition/subject and retained premises
External proof/model checker	Establishes the specific formal property under its selected semantics, bounds and logic
Trusted application assembler/host	Checks compatible issued products, routes, writers, assets and selected generation
Runtime authorization and effect boundary	Enforces or authentically observes the particular principal, guard, transaction and browser ef
Independent conformance harness	Discriminates semantic implementations and exercises identified actual failure seams

These responsibilities can share implementation modules without becoming a single evidence class. Running the same evaluator in two wrappers may establish reproducibility, but does not independently validate the evaluator. Conversely, a second independent evaluator may discover a mismatch while proving neither version correct outside the reference interpretation and tested domain.

A proof that the semantic Claim operation preserves one-primary RequestAssignment cardinality is conditional on its initial state, entire source-derived operation catalog, guard/frame interpretation and other relevant premises. Applying it to HumanHelp additionally requires correspondence to the **actual** protected public model operation, its current transaction guard, other writers and current installed caller. The rule does not become true for real XTDB merely because the semantic operation was called `Claim`.

Proof transfer is a separate, quantified obligation

The claim about a formal subject, a checked derivation and the claim about a runtime implementation have distinct quantifiers. For example:

```
S: admitted abstract states, events and transition relation
I: a selected implemented machine under a declared environment E
alpha: a checked abstraction of relevant implementation state/events

Abstract safety: every admitted S trace satisfies invariant P.
Realization safety: every relevant I trace allowed by E is
  related through alpha to an admitted S trace, including failure,
  concurrency, rejection and recovery observations.
Physical safety: P transfers to I only where the relation
  and all required implementation/environment premises hold.
```

This is a schematic proof obligation, not an existing Gesso user API or a claim that the current physical relation has been proved. The chosen simulation/refinement direction matters. A single passing execution, bounded finite exploration, identical source hashes or matching after-the-fact ledger fields cannot establish the universal realization condition. Unobservable internal steps, real operation outcomes, actor and command identities, installation generation and failure cases need an explicit relation and applicability envelope. Unknown effects and omitted bypass writers remain genuine open obligations, not vacuous proof successes.

Progress/liveness requires separate fairness, availability, retention, scheduling, continued participant and permitted user-action premises. Proving that a safe terminal state is reachable does not establish its eventual reachability for every fair run. Physical safety must not depend on fictional eventual delivery; a lost response may preserve a committed transaction while the browser honestly remains unknown.

A checked external theorem can be reused under its exact semantic assumptions while the compiled artifact, host route, provider adapter or browser bundle changes. Such reuse does **not** preserve a previous implementation-correspondence or runtime-authority judgment. The consumer independently requalifies each affected physical edge and retains the same source-derived meaning across single- or multi-host placements.

Evidence belongs to individual claims, not system labels

An assurance claim records at least its proposition and quantifiers, normalized subject, proof/observation scope, relevant source and generation identities, inference or enforcement mechanism, open premises, dependency closure and applicability conditions. A root report aggregates heterogeneous claims without assigning one global `:proved`, `:certified?` or confidence percentage.

The assurance classifications established in Part I remain distinct:

Evidence class	What it can establish when qualified
Machine-proved / externally machine-checked	The specific formal conclusion under the formal interpretation, inference rules and assum
Kernel-checked	Validity of the submitted closed derivation under the stated proposition language and ope
Exact-finite checked	The stated finite subject and bounds were explored as declared, including non-vacuity and
Assembly-verified	The actual checked source/assembly and artifact relation at its named inspection/installat
Runtime-enforced	A named concrete premise was enforced at the actual trusted boundary for the relevant in
Trusted	A specified external/opaque implementation or environmental contract is assumed within
Asserted	An application-owned proposition was declared but not established by that declaration
Tested	Actual qualified scenarios/observations support the named behavior over their exercised s

The words **conditional**, **bounded**, **historical**, **current**, **installed** and **progress** qualify the proposition or its applicability; they are not additional evidence mechanisms. An issued nominal semantic product and a copied report shape are not independent proof categories. A conditional liveness theorem remains conditional; a test of a network partition does not establish eventual fairness in untested schedules.

Each claim must separately expose:

- the source-derived obligation and original semantic subject;
- evidence offered and whether the responsible checker/physical gate genuinely accepted it, including independent provenance;
- inherited premises, which remain open, which were discharged through an admitted judgment, and which are accepted as trust;
- relevant finite bounds, physical trace/writer coverage and model-to-implementation abstraction assumptions;
- unsupported syntax, exhausted budgets, unavailable evidence, counterexamples, stale generations and their distinct outcomes;
- the selected profile's **admission decision** and any separate release/product completion requirements.

An unsupported proof fragment means the checker has not established its claim; it does not mean the statement is false. An actual model counterexample refutes the claimed property **under the examined model and premises**; it is not automatically a production XTDB trace. A source-checked report may be accurate but still not be issuer-controlled admission, runtime authorization or a durable transferable certificate. The selected admission policy cannot turn a failed mandatory proof into a trusted statement without an explicit permitted change of claim/profile and reconstruction of the resulting obligations.

A model operation's four outcomes `:committed`, `:rejected`, `:already-incorporated` and `:commit-unknown` are not these checking dispositions. Nor are participant presentation `:accepted`, `:rejected`, `:unknown` or Part XI release-gate states interchangeable with evidence classes.

Opaque host code remains useful and visible

Gesso must remain usable with arbitrary Clojure and ClojureScript. Developers write policies and real model implementations with normal host-language facilities; they do not maintain a second handwritten specification in TLA+, Quint, Recife or Lean. When such implementation behavior is relevant to an assurance claim but outside the closed language, it supplies an explicit **source- and implementation-bound opaque contract** and any checked or runtime-qualified premises the selected profile requires.

At an opaque boundary the constructed application records the declared typed input/output and failures, effect/authority dependencies, read and write footprints, required current actor and disclosure context, possible external/provider interactions, callable/closure and library/build/configuration applicability, and which exported claims rely on the contract. Actual effects and runtime guards are checked at the appropriate enforcing boundary when the claim requires them; a structural contract by itself proves neither the code implements the policy nor that every relevant branch or writer was discovered.

An independent theorem about pure command-identity binding need not be discarded when the renderer contains opaque formatting code unrelated to it. An opaque Organization-policy predicate, however, prevents claiming **that predicate's real truth** from the small rule calculus without a qualified bridge or visible premise. A raw XTDB writer outside the admitted operation catalog can invalidate a whole-domain invariant even if all declared transitions preserve it. The source completeness gate and actual writer coverage are therefore separate evidence subjects, not two names for one manifest.

An externally checked implementation theorem, an actual authenticated runtime observation or a sufficiently qualified provider contract may justify a **particular** opaque premise at its exact scope. Substitution/discharge follows 09.11's actual checked entailment and premise-inheritance rules; a successful test alone cannot silently become a general proof. A changed callable, dynamically bound Var, provider version or resource generation requires rechecking applicability even if the abstract interface remains textually unchanged.

Semantic-policy review is a separate owner: Gesso checks the rules that the application declares and detects unaccounted implementation policy only where inspection can establish it. HumanHelp decides whether its real Membership, Organization, Request and RequestAssignment policy is desirable. The formal checker cannot invent a missing helper-role business restriction from a friendly function name. A genuinely eligible Claim and its adversarial wrong-actor neighbor are both required positive/negative controls against a vacuous reject-all policy.

Acceptance and counterexample controls at this boundary

The finished framework must discriminate the following paired cases without changing their responsible evidence class:

- a source-generated abstract model and its corresponding production semantic evaluator agree on a discriminating operation; a separately handwritten model that drops an original Unclaim case cannot establish the original production claim;
- a genuine checked propositional derivation preserves its open premises; a copied `:kernel-checked` report, unsupported atom truth or circular assumption discharge cannot create a theorem;
- a bounded finite invariant check with nonempty valid initialization is reported under its exact scope; truncated exploration, empty/vacuous state space or a fabricated proof label is not universal assurance;
- a public Request/RequestAssignment Claim transition satisfying a source-derived one-primary invariant is accepted at its abstract scope; a hidden alternative writer or stale pre-read cannot inherit the corresponding physical safety proof;
- a source-issued handler, currently installed route and real generated browser plan have separately qualified correspondence; a matching stale source/asset stamp does not authenticate their deployment;
- a current authenticated helper performs an eligible Claim, while a browser-supplied helper flag, wrong-Organization principal or revoked Membership is rejected at the **actual** protected model/host boundary;
- a known historical command C42 is reconciled without new incorporation after lost delivery; a present Request card alone cannot establish that particular past result;
- a permitted creator-local Create and one user-granted remote Refresh produce scoped physical adoption; SSE advisory delivery cannot mint another read or a completed DOM effect;
- a correct conditional progress claim exposes its fairness/retention/user-action premises; a timeout or one positive retry is not an unconditional liveness theorem;
- a physically safe single-node and multi-node execution preserve the same operation semantics; a lost router/interest registration and unqualified replay/gap failover cannot be called universally safe or convergent by reusing a static certificate.

Each oracle names its subject, positive control, adversarial perturbation, expected first checking/enforcing boundary and needed independent observation before execution. A source checker cannot claim to have prevented a physical effect it cannot observe or gate, and a physical diagnostic is not retroactively an abstract proof.

System-wide assurance targets

The system-wide target matrix specifies **what would have to be true** of a finished Gesso application, not an inventory of guarantees already proved by any particular implementation. Each family has (1) an exact semantic property over its admitted source subject, (2) a directional source-to-runtime realization or trusted enforcement condition, (3) an evidence method and quantification scope, and (4) at least one independent positive and adversarial acceptance control. A theorem that establishes only the first is still useful but must not be promoted into the second by a common label or shared digest.

Part IX governs source-derived obligation completeness, issued evidence, checked premise inheritance and application-certificate categories; 10.01 governs ownership and proof transfer. This section states whole-system proof targets while leaving their detailed theorem statements to 10.03–10.04, actual transaction/writer refinements to 10.05–10.09, external proof/abstraction boundaries to 10.10–10.12, trusted host contracts to 10.13–10.15 and multi-node realization to 10.16–10.19. No parallel application-authored formal source is required.

System-wide claim/evidence matrix

Claim family	Source-relative semantic target
Choreography and projection	Admitted global steps, role projections, permitted local traces and branch discriminators sati
Knowledge, provenance and agreement	Each role knows and may use exactly its justified sources, observations and communicated f
Values, schemas and typed rules	Supported operators, nominal values, optional/nullable states, guard refinements and closed
Authoritative operations and invariants	All required initializations and original source operations preserve every claimed invariant u
Progression, observation and consistency	Requirements join without weakening; only sufficient, provenance-correct evidence may dis
Optimism and command outcomes	Provisional state has explicit ownership, command identity, settlement and honest unknown
Browser adapter and effect resources	Portable transition rules preserve execution/resource ownership, stale callback exclusion, re
Portable plans and generated assets	Same supported initial state, plan and admitted events yield equivalent semantic results with
Application closure and capability coverage	Declared affordances have compatible source-to-success/rejection/unknown/recovery paths;
Authority and disclosure	Browser-origin proposals never become trusted principals, current role rights, command inc
Conditional progress and selective adoption	Named command/observation/advisory/settlement obligations resolve only under specified f

Each source theorem/verification result has its own actor, operation, state/trace domain, quantifiers, abstraction, open premises and applicable source generation. Each physical row has a separate installation, invocation, actual effect and observation scope. A table entry describing a desired property does **not** discharge the corresponding physical obligation. Part I's evidence classification and Part IX's accepted conditionality must remain attached to the individual proposition when a root report joins rows.

Choreography, knowledge and actual role realization

The global language has a closed inspectable transition system and explicit declared effects. A checked projection preserves the specified observable global/local trace relation for its supported constructs, including branch knowledge and permitted hidden local steps. It must not invent communication, consume a nonexistent message, omit a necessary discriminator-producing event or allow a role to act on a protected fact it could not know. Where a global and local trace relation is many-to-one, the chosen simulation direction, observation alphabet and acceptable stuttering must be stated; replaying one convenient schedule cannot prove all-schedules projection preservation.

A sender may transmit only data justified by its current knowledge and authority. A receiver learns only the payload and provenance actually delivered and admitted at its execution point. Local deterministic computations add only justified results. An asserted role or copied authoritative-basis stamp cannot upgrade provenance. Correctness of an abstract communication step cannot authenticate that actual transport bytes came from the selected installed projected machine.

Agreement concerns specified facts, scopes and observation frontiers, not equality of arbitrary timestamps, DOM shapes or host placement. Two roles may have **different** valid observations because access and privacy differ. A claim that their authorized projections agree states which common facts they can legitimately compare and what compatible authority/basis evidence supports the comparison. An opaque remote provider or network delivery assumption stays visible when required.

Representative controls: a helper projection branches after receiving an admitted public decision, while a projection branching on an uncommunicated private Organization fact is rejected. A valid send/delivery witness preserves exact command/message occurrence, while a copied source ID or reordered receipt does not satisfy the same schedule. The strong production claim additionally tests the **real** emitted and installed plan, not merely an in-memory projection.

Types, rule use and portable value semantics

Schema validation, intrinsic typed-rule checking, distributed use-site knowledge and runtime model-policy truth are different judgments. A syntactically accepted field is not automatically present; a present nil is different from absent; a role with a typed `RequestStatus` value does not necessarily know it at this program point or have permission to disclose it. Short-circuit branch refinements and reader footprints must preserve source evaluation order. A closed variant's exhaustiveness claim applies only to the actually declared versioned variant set, not all future or opaque states.

The portable subset explicitly specifies signed integer precision/range, booleans, strings, nominal enums, absence, nil, maps/sets/sequences, encoding order where meaningful, branch exceptions and error classifications. A CLJ operation returning an exact integer outside JavaScript's safe numeric range does not become portable by silently rounding or coercing it. The framework may reject such a portable plan or use a specifically supported lossless representation; it must not report JVM correctness as evidence that compiled CLJS ran it identically. Arbitrary Clojure model implementation can remain server-only with typed/opaque boundaries.

Positive controls exercise both present-nil and absent fields, branch-sensitive rule evaluation and matching JVM/Node/Chromium results over a discriminating supported corpus. Negative neighbors reorder a reader across its guarding predicate, substitute numerical enum ordering, change an integer at the exact portability limit or claim agreement after a CLJS path never ran. A finite differential test is evidence for the sampled cases; universal portability requires a separately qualified semantic/correspondence argument.

Authoritative incorporation, outcomes and progression

A public model operation owns the business transition. The source-relative guarded-transition or induction claim covers its original initial-state condition, all relevant operations/invariants, simultaneous writes, frame effects and closure premises. Its production counterpart requires actual current guard evaluation and the relevant protected Request/RequestAssignment/Membership/Organization/ Location facts at the selected atomic XTDB boundary. A pre-read from an earlier snapshot, a browser-supplied role or matching transaction-shaped map does not establish the transaction's internal prestate or current authorization.

A real successful incorporation must satisfy the qualified atomicity contract for domain changes, canonical durable command result and publication intent where the operation requires them. Actual command C42 identity remains distinct from physical attempts H1/H2, transaction receipt, current Request state, DOM resource, provider basis and application generation. A failure of H2 after possible H1 incorporation does not show C42 never committed. Model outcomes `:committed`, `:rejected`, `:already-incorporated` and `:commit-unknown` must retain their history and meaning, separately from presentation `:accepted`, `:rejected` and `:unknown`. Replay under retention and current disclosure rules resolves historical incorporation without silently making a new Claim.

Progression algebra is conservative over genuinely opaque provider bases. It may merge identical requirements, retain incomparable ones and discharge an obligation only with sufficiently scoped, authority-qualified witness and actual projection coverage. Arrival order, increasing-looking integer, SSE notification or client-selected timestamp is not proof of basis advancement. A read permission, read execution, query result, DOM install, user awareness and command-specific settlement are separate events; no source proof can collapse them.

Positive controls include two real eligible helpers racing to Claim the same Request with at most one new active primary assignment, a durable C42 reconciliation after lost delivery, and a read whose actual provider evidence discharges exactly the named requirement. Adversaries include stale Membership, two independent pre-read writes, a fabricated TransactionKey or basis-advance witness, and a current card misattributed to C42 history. Bounded matching ledger/readback is useful but not sufficient causal proof or a physical-writer completeness theorem.

Optimism, real browser effects and resource ownership

Optimistic state represents a declared local intent and is never itself authoritative Request or assignment truth. Each provisional trajectory has a stable logical command where appropriate, distinct execution/physical attempt and resource identity, actual observation premises where required, and a typed settlement/recovery policy for commit, rejection, indeterminacy and newer current state. A Create intent for an entity not yet created need not invent a nonexistent entity basis; its operation may require a distinct parent/collection observation instead. Current authoritative supersession does not retroactively establish whether the particular command committed.

The browser semantic adapter is the source of supported transition and effect admission rules. Each active asynchronous effect is owned by a specific execution/resource or a declared shared owner class. An old execution cannot act merely because the same logical target identifier was reused. Retirement revokes the effect right before best-effort transport cancellation or cleanup. Real abort success is not necessary for safety if the physical shell correctly rejects stale callbacks.

The shell normalizes actual events and must enforce current permission not only when a callback is received, but again **at the instant** of managed DOM, HTMX/OOB, continuity or transport effect. A response HTTP 200 or `htmx:afterSwap` for one target does not certify completion of a different target, multi-target batch or interrupted installation. If an admitted operation permits

a partly applied physical batch, its explicit repair contract must preserve safety and report which parts actually adopted; one semantic instruction does not make multiple DOM mutations atomic.

Fragment admission/coordination requires a declared upper bound for active and queued **logical** slots under supported scheduling, with separately specified handling for retired physical requests and metadata retention. A disconnected transport, deferred callback, shared fragment notification or stale continuity owner cannot be reclassified as current command execution to evade the bounds. Incompatible stale plans retire and recover under the application's source, acquisition and permission contract; reconnect never grants automatic remote board refresh.

Positive controls cover valid target-specific installation, late callback rejection after owner retirement, duplicate transport resolution, real partial-OOB failure, and one authorized user-Refresh acquisition. Adversaries reuse a DOM ID under an old resource instance, defer the real mutation until after admission expires, or silently refresh the remote board on an SSE advisory.

Portable plans, build artifacts and runtime noninterference

A normalized `ExecutablePlan` and its supported initial state/event stream have a declared portable semantic interpretation. JVM, compiled Node/CLJS and Chromium may require distinct host adapters, but within the supported subset their semantic state transitions and declared outcomes must correspond. The claim does not require equal wall-clock ordering, browser layout, physical network schedules or provider availability. An executed Node harness does not substitute for the real Chromium DOM/HTMX effect path.

Executable artifact identity is distinct from compatibility identity and from proof-sidecar identity. Build and startup must relate source-owned manifest, emitted executable bootstrap, exact compiled bytes and installed serving generation. A deterministic manifest or matching stamp does not alone show which bytes the real browser loaded or that those bytes implement the admitted semantics. Source/build checking, actual served/loaded artifact observation and semantic correspondence retain different evidence owners.

Proof, diagnostics and external formal sidecars are **runtime-nonsemantic**. The same admitted input, state, plan and event should not change its domain or browser transition because a proof report was enabled, absent, cached or stale. A sidecar can affect whether a stronger **admission policy** allows construction but may not silently steer an already admitted machine through a different semantic path. Exact sidecar lookup requires exact artifact identity; a merely compatible plan does not borrow evidence without its own checked compatibility rule.

Useful controls compare admitted execution with and without optional diagnostics and verify no changed transition outcome; reject a copied manifest stamp with mismatched compiled bytes; verify a real loaded browser runtime matches the current authorized assembly and that an old suspended tab cannot use a retired resource.

Whole-application closure, capability inventory and authority

The required whole-application closure joins **real** rendered affordances, source operations, global Choreo, role projections, verified trajectories, portable plans, emitted/served assets, transport envelopes, trusted routes and server executors, public model operations, outcome mappings, progression, authorized acquisition and a renderable successor for success, rejection, unknown and recovery paths.

Static application preflight rejects an unknown operation, missing/incompatible plan, route or transport, stale artifact, wrong role or capability, unbound server operation, absent model/acquisition path, incompatible identity or undeclared dynamic recovery. It does not claim to know which dynamic Membership will exist tomorrow; instead it requires the necessary runtime guards and typed failure/recovery branches under #req-preflight-dynamic-handling-coverage. The actual installed host and generated physical response must enforce the admitted path; a caller's successful preflight map does not attest which route now runs.

Three checks remain independent: closure of **declared** affordances, an original application inventory of **required** product capabilities (so omitted Create/ Unclaim/Reassign does not pass vacuously), and real installed host/render/asset adoption. A raw HTMX control is legitimate in its explicitly ordinary category, but cannot impersonate a protected Choreo operation or be silently substituted for a failed semantic plan.

The browser may propose operation, canonical arguments, logical command ID, claimed observation and correlation metadata. It does not provide trusted authenticated principal, current Membership/Organization rights, actual model prestate, command incorporation or protected read/ subscription permission. A source certificate changes none of these. The server must bind the real principal, tenant and operation at the actual trust boundary; the public model enforces current protected policy and output disclosure. Cross-layer capability or assertion inflation must fail before its privileged effect.

The HumanHelp positive acceptance requires real eligible Create, Claim, Unclaim, Mark on the way, Complete, Cancel, manager Reassign, two-client board continuity, creator- local selective adoption and user-granted remote Refresh. The still-unselected response-flow product choice cannot be filled in by Gesso's proof checker. The independent Part VIII gates remain outstanding until demonstrated, even if every declared operation proof passes.

Conditional progress, fairness and multi-node scope

A progress or convergence claim states **which** obligation resolves: a provisional command, historical outcome, queued advisory, required read, compatible basis agreement or authoritative DOM installation. It names the environment premises used: provider reachability, eventual permitted read, scheduler/ participant fairness, required command/ outbox retention, continued execution, user action, sufficient quiescence and correct replay/failover where relevant.

A reachable terminal state is a possibility, not eventual completion under every fair execution. The existence of a timeout, Refresh route, outbox entry or reconnect is not a progress theorem. Under HumanHelp's selected explicit- Refresh policy, remote creation or lifecycle changes are advisory until the viewer grants acquisition. A theorem conditional on user action must not pretend the viewer is obliged to act. A qualified recovery system may coalesce or skip intermediate advisory events only when authoritative coverage, acquisition permission and adoption contract justify that behavior.

One- and multi-node deployments share these semantic obligations. Topology adds physical registration frontiers, node health/failover, durable invalidation creation and routing, lost/duplicate delivery and gap-recovery premises. A completed single-node safe Claim does not prove every distributed schedule, and a dropped advisory does not authorize a background collection read. The selected multi-node implementation must preserve the same authority and participant contract rather than force application authors to maintain different single-/multi- node semantics.

Safety is not contingent on fairness or eventual delivery: on loss or provider failure, the operation remains typed unknown, unresolved or rejected for an actually established reason. It cannot invent command rejection, provenance, permission or whole-board convergence.

Acceptance evidence and root-assurance reporting

A root report for each family identifies its original source, semantic subject, named quantifiers, actual checker/proof mechanism, finite/test coverage, open premises, qualified installed host/provider scope, source- to-physical correspondence, first enforcing boundary and independent positive/product oracle. A report describing a source theorem is not an issued runtime authorization receipt.

A completed assurance target requires the stronger selected profile's real mandatory evidence; it may retain explicit premises where that profile allows them. A source proof may coexist with an open physical obligation, a working runtime guard may coexist with an unproven kernel meta-theorem, and a passing browser scenario may coexist with a missing product action. The report must not reduce these to a Boolean `:certified?` or convert timeouts, unsupported fragments and uninspected writers into green verification.

The targeted evidence includes static and semantic-model checks, independent kernel/dedicated checker validation, exact finite/ formal theorem artifacts, source-to-issued and source-to- host correspondence, actual guarded XTDB and provider observations, JVM/Node/Chromium adversarial tests, source- owned build/ runtime assurance, and positive HumanHelp operation/product acceptance. Each remains scoped to what the instrument actually saw. The detailed proof/realization plans appear in 10.03–10.19, not in a second authored proof calculus here.

Framework theorem contracts

A Gesso theorem family is defined by its original source subject, a closed executable judgment and interpretation, its domain and quantifiers, conclusion, dependencies, admissible evidence, and named open premises. Theorems are not inferred from a source file's name, a checked-looking map, a compiled artifact, an observed happy path or the existence of a checker. Part IX owns the canonical source language, proof judgments, issuance, obligation closure and assumption-substitution rules. This part states their **theorem targets and production transfer conditions**, not a second proof calculus that application authors must maintain.

A family may have (1) a syntactic or semantic judgment checked for one original subject, (2) a theorem about all judgments of that supported language, (3) a source-relative conditional application invariant, and (4) a further implementation/physical refinement claim. These conclusions have different universes. Passing (1) does not prove (2), and (2) and (3) do not prove (4). An implementation can produce true finite witnesses without a general metatheorem; conversely, a proved checker metatheorem does not establish that the deployed route invokes the checked executable.

The finished framework should derive these obligations from ordinary Gesso model and Choreo construction, discharge only those supported by named checking mechanisms, and retain every remaining premise in the exact consumer claim. A partial/unsupported theorem can be useful conditional evidence, but cannot silently satisfy a stronger selected profile. Each family has an independent meaningful positive control alongside a nearby counterexample, including cases that prevent a reject-all implementation from qualifying.

Theorem inventory and the transfer frontier

Theorem family	Source-level conclusion
Certified composition soundness	Provider guarantee entails precisely the consumer premise substituted at the admitted subject/s
Business-rule type and evaluation safety	Supported operations apply only to values admitted by their typed, presence-aware reader cont
Rule-use knowledge safety	A role's branch/action uses facts justified at that exact Choreo program point
Provenance safety	Source/authority labels cannot be strengthened without an admitted inference or observation
Authority non-escalation	Untrusted browser or projected semantic artifacts cannot derive additional protected privileges
Projection preservation	Relevant projected local behaviors satisfy the stated global trace/refinement relation
Executable-plan provenance	Each issued managed plan is derived from a compatible admitted projection under exact source/
Stale certificate safety	Incompatible original semantic dependencies cannot justify new certified consumers
Progression preservation	Canonical requirements compose conservatively; only sufficient scoped evidence discharges the
Browser semantic adapter safety	Permitted normalized transitions preserve execution/resource/effect admission and retirement i

The evidence owner, domain, quantification and independent physical requirements of a row must not be replaced by its neighbor's successful report. The corpus in 09.15 and 10.02 supplies paired conformance families; this section makes the theorem's antecedents, obligations and nonclaims explicit. The stronger source-to-host refinement records and physical writer arguments belong to 10.04–10.09.

Typed rules, presence, evaluation and atomic propositions

For every admitted **supported** typed rule R and environment E satisfying R's exact reader and nominal-type contract, evaluation obeys the specified operator semantics and either returns the declared result type or one of the explicitly modeled failures. In notation, the target is:

```
WellTyped(R, Gamma) and SatisfiesReaders(E, Gamma)
  ⇒ Eval(R, E) is a result allowed by the typed rule semantics
```

This is a family theorem only when quantified over all R and E in the selected bounded/closed language. Checking a single R,E is an instance result. Optional absence, present nil, short-circuit reader access, branch-local refinements, integer/codec restrictions, nominal enums, and exception/failure behavior are all part of that semantic domain. An implementation that evaluates a guarded optional read eagerly is not justified by its final Boolean value agreeing on other inputs.

Rule well-typedness does not establish business-policy truth. An abstract typed atom may mean an application-owned predicate under an explicit valuation; the small propositional kernel may reason from that atom without proving the predicate's actual model implementation. The **typed-proposition bridge** must separately relate each admitted atom to the actual typed rule interpretation under the same original source, binding environment, reader footprint and applicable assumptions. Finite enumeration establishes agreement on its exact projected domain, not universally for arbitrary input maps or real server snapshots.

The actual public Request and Membership/Organization models own Claim eligibility. If a model's policy predicate is opaque Clojure, its specified contract remains an explicit relevant premise until qualified by an appropriate implementation theorem or runtime fact. An application should not rewrite that policy in hand-authored TLA+, Quint or Recife merely to get a checked result. Source-derived translation or independent oracle checking must retain the real public model's semantic subject rather than replace it with a test-friendly policy.

Kernel entailment and proof-checker soundness

For a closed admitted propositional derivation D, named premises Gamma, and conclusion P, the intended **logical soundness** statement is:

```
CheckKernel(D, Gamma, P) = accepted
  ⇒ for every valuation v of admitted atoms:
      (v satisfies all premises in Gamma) implies v satisfies P
```

The statement assumes a versioned interpretation of the connective language, well-formed source normalization, correctly scoped premise references, and exact rule meanings. A checker must reject malformed rule tags, forward references, substituted premises, incorrect conclusions, incomplete source declarations, unsupported constructs and circular proof justification before it can issue the relevant derived category. Open premises remain open; an asserted atom is not an axiom establishing that atom is true in production.

There are three separate evidence goals:

1. **One derivation:** reconstruct the original-source judgment and validate each step and its exact used premises. A truth table may independently check the claimed implication for that particular finite set of Boolean atoms.
2. **Inference-system soundness:** prove every admitted rule preserves semantic entailment, including all supported operands, parameter values, substitutions and source/normalization cases. Testing a few examples of each rule does not satisfy this quantifier.
3. **Concrete checker correctness:** establish that the actual issued checker, parser, normalization and evaluator implement the rule semantics the metatheorem concerns. A theorem about mathematical inference rules does not by itself prove the Clojure implementation.

No automatic hypothetical-context discharge, arbitrary typed-atom truth, SMT arithmetic or classical inference may be presumed unless that operation is in the separately admitted and checked language. A later proof kernel extension must update its exact theorem and versioned correspondence; the proof does not silently widen to all Clojure expressions. Valid checked conditional arguments retain their original Gamma rather than being converted to unconditional P.

Guarded transitions, frames and source-complete induction

For a normalized supported guarded operation T, the relevant theorem connects an admitted before-state, input and after-state through the actual typed guard, **simultaneous** assignment expressions and unchanged unwritten fields (including optional presence). A checked transition fact therefore has a specified relation, not an arbitrary pair of maps:

```
T(s, x, s') :=
  Guard(s, x)
  and for each written field f: s'[f] = RHS_f(s, x)
  and for each unwritten field f: PresenceAndValue(s'[f], s[f])
```

This relation is defined over the selected source state projection. It does not represent every XTDB document or every application writer unless that additional relation and coverage are established. For a selected invariant I, a full conditional induction target has both a source-correct base and an operation-complete step argument:

```
For every s: Init(s) ⇒ I(s)
For every original declared operation op and s,x,s':
    I(s) and T_op(s,x,s') ⇒ I(s')
Therefore: all reachable finite abstract states satisfy I,
  under the stated initial-state, catalog and semantic premises
```

The exact original initialization, operation and invariant inventory must be independently derived from the source being certified. A caller-supplied smaller catalog or structurally valid pruned DAG cannot establish the source's full theorem. Satisfiable initialization, enabled positive operations and relevant closedness tests are separate non-vacuity/product obligations: a false Init or reject-all operation set must not count as a successful HumanHelp application.

The theorem is abstract and conditional. Applying it to real RequestAssignment one-primary safety additionally requires the selected actual initial-state boundary, representation of real Request/assignment status and relationships, transaction-local current guard and atomic write semantics, coverage of every relevant physical writer/initializer, preservation of its unwritten fields, and execution refinement for real rejected, committed, unknown and recovery outcomes. One captured XTDB gateway submission or pinned readback can supply bounded physical evidence while leaving universal causality, physical writer completeness and backend assumptions open.

A real concurrent two-helper Claim with at most one new primary is a necessary positive/adversarial physical test. It cannot be replaced by two sequential model-checking inputs, nor can one positive experiment prove the all-schedules invariant.

Certified composition, premise inheritance and stale dependence

If consumer certificate B uses producer certificate A to justify a premise Q, sound composition requires a **checked** type/context/scope substitution showing that A's exported guarantee entails exactly the Q required at B's subject and observation/actor context. The resulting conclusion inherits all still-open premises actually used from A and B. A provider's trusted assumptions may be accepted by a selected profile but cannot be reclassified as unconditional guarantees to the consumer.

Proof justification dependencies must be well-founded; runtime subscription, retry and recovery cycles do not authorize circular proof references. Original-source-required obligations are separate from a DAG's ordinary structural validity. Compatibility may allow an unchanged abstract proposition under a revised interface only through a directional checked bridge with its own premises and scope; the new implementation or deployed asset still needs current correspondence evidence.

Stale certificate safety follows from exact source and required dependency compatibility, **at the current trusted consumer**. A matching old certificate and old graph cannot attest current source. An S18 Request schema may be intrinsically valid while a

Claim rule built against S17 cannot compose. Changed theorem, checker semantics or obligation inventory invalidates relevant proof applicability; a changed deployed callable, route, asset or resource can invalidate physical applicability while leaving a reusable abstract theorem intact. Mere string names and hash equality are not either proof or live host authorization.

Rule-use knowledge, provenance and protected authority

Knowledge safety states that for every supported projected role, at each admitted action or branch that uses fact F, the local state justifies F's value, source, provenance and relevant observation frontier at that **exact point**, subject to named disclosure assumptions. A global Choreo rule's intrinsic typing does not give every local role knowledge of every fact in its global transition. Required communication, receipt, source derivation and server authority are separate judgments.

Provenance safety forbids promoting client-supplied role labels, a copied basis, generic message shape or an asserted `:authoritative` field into a trusted model fact without an admitted and actually satisfied inference/enforcement rule. The receiver's learned knowledge is scoped to what was sent, received, authenticated and authorized; privacy-aware agreement does not require disclosing all facts to all participants.

Protected non-escalation says that a browser-origin operation proposal, argument, command identifier or observed revision cannot **by itself** increase the authenticated principal's capability to claim a Request, read a protected collection or install authoritative state. The installed server and public model must bind actual principal, Request, Membership, Organization/Location policy and permitted target at the real trusted enforcement boundary. Even an admitted abstract authority rule does not grant a physical permission in the absence of that enforcement or its qualifying trusted premise.

A pure reusable proof premise is not a linear/one-use runtime capability. A one-Refresh acquisition grant, browser execution lease, physical command identity or active model authorization cannot be cloned by substituting the same proposition in multiple proof obligations. The theorem of logical non-escalation and the actual effect-level capability check have distinct owners.

Projection, lowering and real browser refinement

Global/local Choreo projection states a precise relation between allowed observable global traces and admitted role-local traces, with the declared observation alphabet, causal partial order, hidden local/environment steps and refinement direction. For example, a global transition may correspond to several projected internal steps; their valid stuttering does not authorize reordering protected message delivery, inventing a branch discriminator or omitting an effect visible to an observer. Both soundness and completeness of projection are independent properties when claimed; checking one realization witness proves neither universally.

Executable-plan provenance then binds each issued managed plan to the **admitted** projection and exact semantic sources, lowering version, declared data/command shape and source-required dependencies. Semantic compatibility differs from exact emitted executable-artifact identity. A proof/diagnostic sidecar cannot steer the already admitted runtime transitions, and a valid projected plan need not be physically loaded in a browser.

Browser refinement connects the portable semantic adapter's state/event/effect transition to the **actual** shell, HTMX, transport, DOM and completion observations. It must cover stale and duplicate callbacks, resource-instance retirement, reused DOM IDs, transport cancellation failure, deferred physical mutations, partial OOB installation, failed render and physical effects occurring after a callback was admitted. The physical shell rechecks the current right at the **effect point**, and may declare repair for partial batches; it cannot assume several DOM mutations are one atomic operation.

A source-level adapter invariant transfers to real execution only under the appropriate complete simulation/refinement relation, current installed generated bytes and physical permission/effect premises. One tested Chromium scenario is a valuable real witness, not proof of every possible browser schedule.

Progression preservation, stale observation and conditional progress

The source-level progression algebra represents opaque authority bases as obligations, not inherently ordered timestamps or revisions. Canonical requirement composition is conservative: exact equal bases may coalesce; distinct incomparable bases remain required absent trusted authority-specific progression evidence. A source-side `:advances` shape does not prove that a provider actually advanced the basis or that the returned projection covers the required scope.

Discharge of observation requirement R by evidence E requires checked E provenance, actual basis, authorized collection scope, coverage and real acquisition/adoption where the consumer claim requires them. A recent- looking SSE advisory, browser assertion, equal entity card or faster transport is not a sufficient general witness. Creator-local Create S and remote advisory R are different observations; one user-initiated Refresh permits its selected remote acquisition only.

Safety is independently required when progress premises fail. A command may be committed yet its participant legitimately remain `:unknown` after lost delivery. Liveness/progress theorems must state which debt or provisional state resolves, the actual fair scheduling, availability, continued execution, retention, authorized acquisition, quiescence and any necessary user action. A trace

refinement allowing unlimited internal stuttering does not establish that anything eventually finishes. The existence of a timeout, retry route or reachable terminal state is not a fairness argument.

The same semantic safety and conditional progress contracts apply to single- and multi-node deployment. Router placement, interest registration frontier, outbox durability, replay/gap recovery, node failure and duplicate delivery introduce additional **physical** refinement and environment obligations, not new model authority or a second application meaning.

Claim-specific theorem report and acceptance controls

A source-relative root report should make each theorem family inspectable: original subject, conclusion and quantifiers, exact domain/abstract relation, proof method and checker version, source- derived obligation ledger, supported premises, actual evidence origin, stale dependencies, implementation/ physical transfer assumptions and independent positive/product tests. A report is not itself an opaque proof handle or authorization receipt.

At minimum, a relevant corpus must distinguish:

- **Typing:** guarded optional/present-nil evaluation succeeds; eager unguarded reader, illegal enum ordering or unsupported integer form fails;
- **Kernel:** correct derived P from Gamma passes; forged inference, omitted premise, wrong source or copied `:valid?` report fails;
- **Typed atom:** a finite typed-domain agreement has its actual bounds; unexamined Clojure predicate is not declared universally true;
- **Induction:** complete source catalog and nonempty initial model preserve I; pruned Unclaim, vacuous Init or modified frame/guard does not certify the original application;
- **Composition:** a valid checked conditional guarantee substitutes with all remaining premises; a trusted assertion or circular evidence chain cannot discharge a theorem;
- **Knowledge:** a helper branch on an admitted public decision passes; an undisclosed private Organization field fails at the use site;
- **Authority:** a real eligible Claim passes current model enforcement; client role flag or a stale Membership cannot authorize a write;
- **Projection:** a valid communication schedule preserves its trace relation; a missing discriminator, reordered delivery or fabricated sender origin fails at the correct scoped boundary;
- **Plan:** a current issued and installed plan uses exact admitted sources; a copied artifact stamp or silently downgraded ordinary HTMX route fails;
- **Browser:** a current target owner completes a justified effect; retired callback or half-installed OOB batch cannot mark authoritative adoption;
- **Progression:** a qualified basis and permitted Refresh discharge their exact obligations; SSE arrival does not grant a whole-board read;
- **History:** committed C42 replay is reconciled under actual retention; current Request status or H2 failure does not invent historical rejection;
- **Progress:** a specified fair recovery path may establish named conditional liveness; an unqualified timeout or unrequested user action cannot.

Each case records a valid positive neighbor, an actual negative mutation, its expected first checking/enforcement boundary and independent evidence of any claimed physical effect before the case runs. An incomplete or unsupported strong theorem is not a proof success; a correct negative source rejection does not imply every untested physical bypass was prevented.

Formal proof strategy and production correspondence

Formalization removes a **named** trust gap in the executable Gesso semantics. The source-owned semantic core, original-source obligation derivation, small proof kernel, specialized semantic checkers, generated formal subjects, proof-producing elaboration and external proof/model checkers participate in one correspondence architecture. They are not alternative applications, proof certificates for arbitrary Clojure, or a request that application developers maintain a second handwritten TLA+, Quint, Recife or Lean system. A successful formal result applies to installed software only through the separate relation and premises actually justified at that boundary.

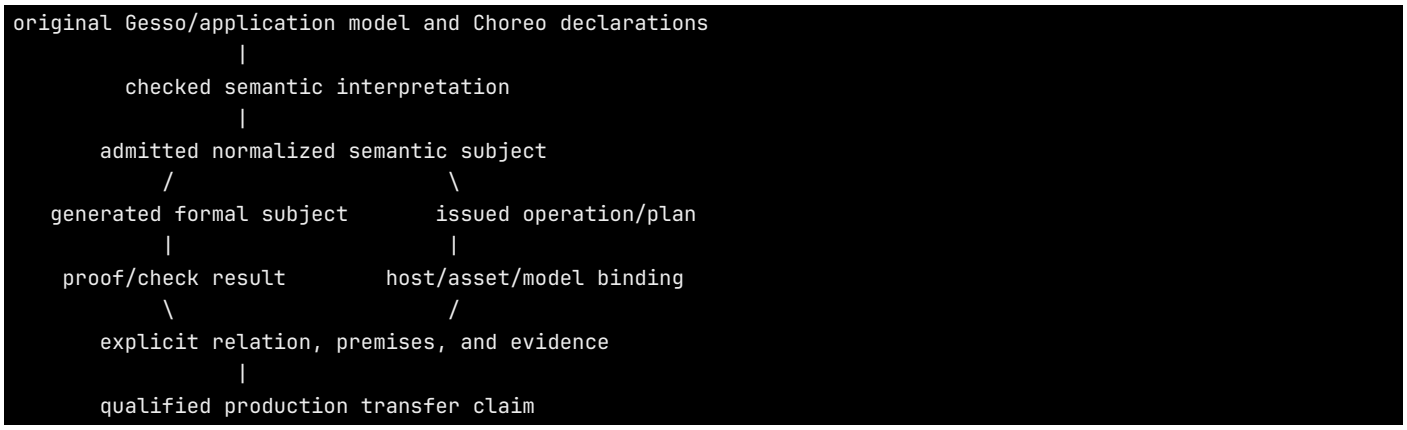
Part IX governs canonical judgments, source-complete derivation, issuer-controlled admission, dependency evolution and premise accounting. Sections 10.01–10.03 specify the one-source semantic ownership and theorem quantifiers. This section defines the **bidirectional correspondence and evidence-transfer discipline**. 10.05–10.09 own the selected transaction, guard, writer and HumanHelp-specific realization obligations; they are not preemptively proved by a formal correspondence report.

Source-derived correspondence record and two directions of traceability

For every formal claim whose guarantee is to be transferred, the original-source authority and trusted consumer derive an inspectable correspondence record with these distinct components:

1. The original admitted model/rule/operation/Choreo source, normalized semantic subject, required original initialization, operations and invariants, and exact versioned interpretation.
2. The formal encoding and proof obligation, quantification and environment domain, proof or model-check result, checker and translator identities, required evidence class and all remaining formal premises.
3. The proposed executable realization: issued operation and plan, callable/closure, route and model binding, compiled/browser artifact, assembly and selected installation generation.
4. An explicit representation/abstraction relation with **direction**, event observation alphabet, guards, state frames, permitted internal steps, failure and concurrency semantics, and the initial-state relation to which it applies.
5. For each transfer premise, its owner, original-source dependency, checked derivation, independent physical evidence or allowed opaque/trusted qualification, scope, invalidation conditions and current consumer policy decision.
6. Source locations, reproducible generation and checking instructions, privacy-appropriate counterexample translation, unsupported status and the first unestablished enforcing boundary.

The record supports **forward semantic derivation**:



It also supports **reverse traceability**:

```
formal obligation, premise, or counterexample
                  |
   exact source/translation/correspondence identity
                  |
   originating model rule, operation, or Choreo step
                  |
   production binding, effect boundary, source location
                  |
   evidence class and still-open implementation obligations
```

Reverse traceability makes a proof result explainable and a counterexample repairable. It does **not** prove the converse inclusion of all abstract traces in real execution, nor automatically prove that every production execution is represented by the abstract model. A cached record, matching digest, correct diagram or copied `:checked` map cannot issue a certificate, prove an installed route, or authenticate an XTDB transaction. Required closure is freshly derived from the trusted **current original source**, not a successful submitted subset or yesterday's matching graph.

Admission soundness, completeness, execution refinement and progress

The theorem claims have different directions and quantifiers. Let S be the supported formal source semantics, A the actual admission mechanism, I the installed executable under explicit environment E, and alpha an abstraction from relevant I states and observations into S:

```
Admission soundness:
  For all supported x, A(current-source, x) = admitted
    ⇒ FormalJudgment(S, x, admitted-premises).

Admission completeness (separate, where promised):
  For all supported x satisfying the required judgment and
  independent admission conditions, A can admit x.

Execution safety transfer:
  For every relevant I/E execution starting in the related initial
  state, alpha maps its relevant steps/observations to a valid S
  execution, including the represented failures and concurrency.

Conditional progress transfer:
  Every relevant I/E execution satisfying named fairness, delivery,
  retention, continuing-execution and user-action premises meets
  the selected progress obligation, without forbidden divergence.
```

These are schematic logical statements, not Gesso API calls. They leave the actual relation, allowed silent/stuttering steps, observation projection and premise qualifications to the admitted source and checker. Agreement over rejected examples does not establish admission soundness. Refusing every user operation may avoid some unsafe transitions but fails required positive capabilities, and is not admission completeness. Proof of an admitted source does not mean the installed server invokes that source.

A safety simulation may allow unbounded internal stuttering while transferring a safety invariant. It cannot thereby establish liveness; a progress theorem must exclude the relevant divergent executions under explicitly qualified fairness and recovery assumptions. Failure of those assumptions preserves safety and honest unknown outcomes. Neither a timeout nor a reachable terminal state is an unconditional eventual-settlement proof.

Worked Claim slice: same source, separate formal and physical obligations

The first proof-transfer pilot uses HumanHelp's actual public production Request, RequestAssignment, Membership, Organization and Location model interfaces, not a substitute toy single-map Request with an invented helper field. The domain owns the authorization and one-primary-assignment policy. The browser supplies a Request argument, operation proposal, stable logical command identity and perhaps untrusted observed basis; the server supplies the authenticated principal and current authoritative context.

A schematic correspondence **fixture** may have this shape. Its keys illustrate record semantics rather than prescribing a public Clojure API; the sample itself is not a checked certificate:

```
{:source
 {:operation :request/claim
  :domain-models
  #{:humanhelp/Request :humanhelp/RequestAssignment
    :humanhelp/Membership :humanhelp/Organization
    :humanhelp/Location}
  :semantic-subject :request/claim-031
  :rules :request/claim-rules-C21}
 :client-proposal
 {:request/id :request-R17
  :command/id :command-C42
  :observed-basis :untrusted-client-observation}
 :trusted-invocation
 {:principal :authenticated-server-principal
  :request :current-authoritative-request
  :membership :current-authoritative-membership
  :organization :current-authoritative-organization
  :location :current-authoritative-location}
 :formal-claim
 {:subject :formal/request-claim-031
  :proposition :one-active-primary-assignment
  :direction :source-induction-and-execution-refinement}
 :production-obligations
 #{:bind-command-to-principal-operation-and-request
   :check-current-request-and-membership-and-organization
   :enforce-one-primary-assignment-at-atomic-write
   :atomically-record-command-result
   :include-all-relevant-physical-writers
   :bind-issued-executor-to-installed-route}
 :evidence
 {:source-check :required
  :abstract-induction :required
  :installed-model-correspondence :open
  :atomic-tx-implementation :open
  :physical-writer-completeness :open}}
```

The fixture separates **admission correspondence** (the particular source-issued Claim plan admitted by production satisfies the intended checked semantic judgment) from **execution refinement** (every relevant execution of the actual installed Claim writer respects the specified atomic RequestAssignment transition, its command record and actual failure outcomes). A theorem for the first does not imply the second. The formal tool cannot authenticate caller-supplied model Vars, route IDs, production node or a handwritten `:installed-model-correspondence :checked` status.

A protected Claim transition may atomically depend on:

```
current Request is eligible and still open
current Membership is active for the authenticated actor
Request, Membership, Organization and Location associations
  meet the real public model policy
logical command is bound to operation, Request and principal
one active primary assignment is preserved
domain changes and durable command result share the
  required authoritative transaction/commit contract
```

The allowed physical mechanism may be an appropriately guarded XTDB transaction, a qualified conditional write, or another independently justified atomic implementation. The theorem needs the **same** relevant atomic boundary, not identical source syntax. A pair of true pre-reads followed by unguarded writes fails that obligation. A command ledger/readback matching the post-state can corroborate one observed history but does not alone prove causation, the transaction-internal prestate, all writer coverage, current actor validity or all future execution traces. Missing physical evidence remains an open premise or a failed strong profile; it is never silently discharged by the abstract proof.

An adversarial implementation may add an unmodeled priority restriction or take the helper identity from browser-provided fields while its formal model still shows the right atomic transition. These are different correspondence failures: the former can omit concrete behavior or required positive eligibility, while the latter violates principal authority. Shared names, hashes or tests of a happy path repair neither. A real product acceptance control must demonstrate the eligible Claim and correctly reject wrong-actor, revoked-Membership and incompatible-command attempts.

Atomic formal actions create specific production obligations

Formal atomicity does not mandate a single thread or node but does mandate a production correctness boundary adequate for the claimed property. If a transition treats current authority checks, domain state changes and command-result recording as one atomic operation, the implementation must justify that correspondence and all external premises used. A process-local one-shot submission handle alone cannot establish durable exclusion among independently issued retries or separately running nodes.

The relevant effect ledger distinguishes:

- source-checked normalization and restricted transaction lowering of the declared operation;
- trusted request-local principal/tenant/operation and command identity binding, and current relevant public model policy;
- the invocation-specific immutable transaction submitted by its actual selected gateway/client;
- backend-defined actual receipt, conditional commit/conflict and failure/unknown outcomes;
- physical domain and command-result atomicity, transaction-local guard and write frame;
- durable historical reconciliation after H1/H2 response uncertainty, without a second Claim;
- all relevant initializer, job, raw and external physical writers that could invalidate the protected invariant;
- source-to-deployed route/asset binding and browser permission/adoption where claimed.

An API return, a local receipt object, an opaque TransactionKey, a matching command row and a pinned readback may be related observations with distinct evidence origins. A report identifies exactly which relation is established, and which still requires the provider's contract or independent causal proof. If an exception occurs after submission, the public outcome cannot be manufactured as `:rejected`. No post hoc diagnostic check can turn a completed unsafe write into pre-effect prevention.

The authoritative submission architecture, guard footprints and physical writer completeness belong to 10.05–10.09. This section makes their proof-transfer dependencies unavoidable without specifying a second competing transaction API.

Evidence mechanism, transfer limits and qualified authenticity

Evidence mechanism	Warranted statement
External machine-checked theorem	Formal conclusion under exactly the encoded definitions, proof rules and assumptions
Source-issued checked derivation	Original declared subject satisfies its supported checker judgment and retains open premis
Generated/extracted checker or compiler	Artifact derives from the formal or semantic subject by a selected process
Proved representation or simulation	Specified concrete-to-abstract relation holds over its quantified domain
Exhaustive finite comparison	Compared behaviors agree over the enumerated state/input space with recorded coverage
Independent conformance/fault test	Selected actual executions agree with their expected behavior at observed boundaries
Controlled process-local gateway observation	Captured local call inputs/result and related observations at that owned call site
Inspected deployed host/asset report	Matching source/route/callable/asset facts in the inspected host scope
Human/code review of handwritten relation	Reviewer judged specified correspondence under recorded assumptions

A translator can be left inside an explicitly named trusted computing base or justified by a separately checked translation/refinement argument. Generated outputs are **derivatives**, not proof of their own correctness. An untrusted elaborator may propose proof terms; a smaller checker must validate the original-source judgment. Likewise, generated formal models must not be hand-edited to evade a counterexample and then presented as still representing the actual application.

Authenticity is relative to the chosen threat model. A private JVM registry can establish one process-local handle origin under a trusted JVM premise without providing a cryptographic remote attestation. A controlled captured XTDB client can justify a local call-site observation without authenticating a caller-supplied node as production. A copied report loses process-local identity warrant unless a supported durable rechecking or attestation contract restores it. Automatic signature demands do not replace stating which boundary is authoritative and who can substitute it.

Bind evidence and preserve currentness before substitution

The producer and consumer retain original-source identity, required obligation coverage, exact proposition, proof rules/checker profile, used premises, source dependencies, semantic interface, selected runtime plan, callable, install/asset and resource/instance identities as **different** objects. A certified consumer checks the current trusted source and relevant dependency closure before accepting a producer's purported guarantee. A successful report or matching old source plus old graph is not currentness.

An abstract proof and its exact immutable source closure may be reused without running its checker on every click if the actual dependencies and trusted lifecycle remain valid. Reuse does not suppress fresh model authorization, command replay/retention, actor and tenant context, browser effect ownership or current installation validity. A changed opaque callable, regenerated asset, replaced route, provider or runtime generation may leave an abstract theorem intact while invalidating the stronger source-to-production claim.

When a selected strong profile requires external checking, timeout, unsupported syntax, stale subject, failed checker or missing model correspondence must yield a typed unestablished/rejected admission, not a cached green success or invisible fallback to trusted code. A subsequent checker outage does not automatically invalidate a still-valid independent proof accepted under its lifecycle, but a refuted required premise or changed current binding cannot be hidden behind it. Weaker conditional use requires an **explicit separately selected profile** with its genuine remaining assumptions.

Counterexamples return to source rather than creating a second model

For a modeled violation, Gesso retains correspondence metadata to display the named invariant, original source/operation, actor/authority context, smallest useful trace prefix, failing abstract guard/frame step, formal translation and abstraction assumptions, related production binding and the evidence needed to decide whether a real implementation shares the defect. Sensitive Request/actor details are redacted to the authorized reviewer.

For example, a **candidate defective semantic model** that checked open status only at a prior read might allow:

```
A observes Request R as open
B observes Request R as open
A incorporates Claim for R
B incorporates a second primary Claim without an intervening release
one-primary-assignment invariant fails in the candidate model
```

This is a valid abstract counterexample **only if the formal candidate actually allows that unguarded commit action**. It is not a counterexample to a correctly atomically guarded formal transition, and it is not proof that the installed XTDB writer executed the trace. Conversely, a correct formal guard can coexist with an unguarded deployed writer; that is an open or falsified **production refinement** obligation, not an abstract invariant failure. Model approximation may also admit a spurious trace; concrete realizability must be independently assessed before blaming an actual producer.

A developer-facing diagnostic might identify a failing source guard and a required `net.humanhelp.site.model.request.core/claim` implementation check, marking the actual physical conclusion `:not-established`. The illustrative model binding is not evidence that a particular runtime route invoked it. Where a trace does not uniquely identify the defect, source-model, translation, assumption and production possibilities remain distinct.

The normal repair loop is:

```
Clojure/Gesso source and approved model policies
         → checked semantic construction
         → generated formal subject
         → proof/model checking
         → scoped counterexample or unsupported obligation
         → interpretation at the original source and physical boundary
         → human- or assistant-proposed source/translation/binding repair
         → full regeneration, admission and independent conformance checks
```

Ordinary application developers should not need to author or patch formal-language files. Framework maintainers may repair a faulty lowering, proof checker or external backend as their own source changes. Product policy remains owned by the application. An automated assistant may propose repairs but cannot silently weaken a named invariant, mark an opaque premise discharged, modify the generated specification just to pass, or issue a certificate. Every approved repair is re-derived and rechecked under the appropriate profile and independent positive tests.

Acceptance criteria for the correspondence strategy

At a minimum the finished system must demonstrate paired cases for:

- **One source:** correct generated formal Claim and issued plan correspond to the original operation; a separate handwritten formal F and independently executed G without a justified relation cannot pass.
- **Admission direction:** accepted source has the required formal judgment and checked obligation closure; an admitted malformed rule or pruned original catalog violates admission soundness.
- **Positive completeness:** eligible real Claim and required lifecycle operations remain constructible; reject-all construction cannot qualify even if its abstract safety is vacuous.
- **Execution relation:** a checked supported production trace refines the original abstract operation; copied equal final snapshots with unmodeled intermediate unauthorized effects cannot establish the same claim.
- **Actor binding:** trusted current Membership/ Organization policy authorizes an eligible Claim; browser-supplied actor identity cannot authorize the protected write.
- **Atomicity:** a guarded concurrent two-helper Claim preserves one active primary and durable command attribution; competing stale pre-reads and unguarded writes fail the claimed correspondence.
- **Command history:** C42 replay after lost H1 delivery recovers its true durable outcome; failed H2 or a later Request card cannot invent rejection or a second incorporation.
- **Physical evidence:** one owned gateway call, actual return and pinned readback support their bounded local witness; matching records are not accepted as universally causal or as complete writer inventory.

- **Provenance:** source, deployed operation and current plan bindings have genuine issuer/ host evidence; a raw copied proof report, node ID or stamp cannot authenticate them.
- **Currentness:** an unaffected pure theorem can be reused under checked source compatibility; a swapped callable, served asset or revoked provider assumption requires renewed corresponding physical evidence.
- **Counterexample interpretation:** the defective candidate formal action returns a specific source-relative trace; the report does not automatically assert the installed writer did it.
- **Repair boundary:** repairing the true source/ translation and rechecking succeeds; editing only a generated invariant cannot change production semantics or qualify the original claim.
- **Liveness:** the chosen fair recovery/user action premises justify a specific debt resolution; a safe stuttering trace or timeout alone does not establish eventual completion.
- **Topology:** one- and multi-node executions use the same Claim semantics; extra routing, registration-frontier, replay/gap and failover premises stay open until qualified.
- **Privacy:** authorized reviewers can trace a failure to its source; copyable reports cannot leak private principals or gain transaction authority.

Each case predeclares the original source, positive control, adversarial perturbation, expected first source/checker/physical boundary, independent effect observation and actual open premises. The acceptance decision must be claim-specific, not one global `:certified?` flag. A successful formal proof, source checker, physical test and product capability are different required pieces of the strongest end-to-end acceptance.

The selected transaction realization

The preferred protected-operation realization is a **source- and invocation-bound transaction gateway**: the trusted application derives one physical submission from the currently admitted operation, checks its supported lowering and protected effects, and permits the selected immutable vector to be submitted through exactly one invocation-scoped capability. The gateway is a **mechanism** for establishing specified aspects of source-to-physical correspondence. Its presence is not proof of current model authority, universal XTDB transaction semantics, actual command incorporation, complete writer mediation, or the existence of an installed production route.

The canonical operation remains the public model's guarded transition, not an independently authored transaction program or a browser command-shaped map. The framework may issue a constrained gateway for operations that fit the supported physical subset. Other model implementations remain legal through explicit opaque contracts and the selected assurance profile. They do not inherit the constrained gateway's correctness or physical writer claims without evidence.

Transaction authority, issuance and one actual submission

The end-to-end accepted path is a dependency chain, not one success-shaped status field:

1. **Trusted ingress:** The serving route or explicitly authorized worker binds the authenticated principal or service identity, tenant, admitted operation, canonical arguments, target and stable logical command identity from actual trusted context. Browser-supplied role, model prestate, claimed source/basis or apparent target is only a proposal and must not authenticate those facts.

2. **Source-current assembly:** The selected handler, public operation contract, original-source mandatory obligations, source-issued executable plan, model policy and target-binding interface are compatible at the current trusted consumer. A past certificate, copied route report or matching source name does not establish the handler actually deployed.

3. **Physical plan admission:** The checked lowering chooses a supported bounded operation family and retains exact guard and write dependencies, read/absence/uniqueness requirements, field presence/frame contract, protected document set, expected/current-state provenance, durable command-result write and publication intent when applicable.

4. **Gateway preparation:** The gateway recomputes or independently checks the **entire** canonical immutable transaction vector from the admitted source and rejects additional, reordered, substituted or unsupported physical operations. The caller cannot append raw SQL or XTDB operations, substitute a backend client, redirect the target, or supply its own submit callback inside this claimed constrained path.

5. **One-shot invocation:** The prepared attempt capability is issued for a particular physical invocation and consumed atomically with respect to local competing users before any second submission through that capability. The selected gateway owns the actual call of `xtdb.api/execute-tx` or an equivalent explicitly qualified authoritative backend API; the caller does not execute the vector separately.

6. **Outcome and external observation:** The gateway records what its own call attempted and returned, but hands command settlement to the qualified backend/provider and durable command-resolution contract. The public response distinguishes known rejection, known incorporation, known historical incorporation and unknown incorporation. Any post-submission

diagnostic read is separate evidence; it may not manufacture a retry permit, commit conclusion, principal authority or Live/browser settlement.

This path never turns a process-local issuance handle into a transferable client credential. Safe persistence of a proof or source artifact requires its own admitted rechecking procedure. A controlled test collector or locally captured transaction receipt does not authenticate the submitted node, current HTTP server or deployed public model.

Restricted lowering has one documented physical meaning

The supported lowering vocabulary is versioned and closed. It specifies the exact backend-facing semantics of every admitted insertion, asserted existence/absence predicate, conditional update, guarded replacement, allowed delete, reference/cardinality constraint, typed encoding, null versus absence and canonical command-result/outbox operation. Unknown forms reject **before submission** from the constrained path rather than becoming opaque extra statements appended after checking.

When an admitted plan contains several physically atomic operations, the gateway checks the exact ordered or semantically constrained vector of **all** those operations. It must say whether order matters, which expected values came from external reads versus internal transaction state, how backend conflicts are reported, and how write/update/delete semantics preserve the abstract frame. No claim about a complete transaction follows from matching only one emitted SQL string or only the nominal Request table.

The guard footprint and write footprint are distinct. For Claim, current Membership, Request, Organization, Location and assignment-absence/uniqueness may be guard dependencies even when only Request and RequestAssignment, canonical command record and publication intent are written. A whole-document replacement can overwrite an unguarded field. A positive-row CAS does not necessarily protect a relationship absence predicate. The exact lost-field, ABA and revision discipline obligations belong to 10.06; the admitted gateway must preserve the physical footprint needed to implement that later proof.

A **same-node pinned pre-read** can help prepare a conditional transaction, but it is not the transaction-internal prestate. A backend's conditional assertions and atomicity/serializability contract must justify the state on which the protected transition actually depends. If a revision is used to exclude ABA, every relevant writer must participate and wraparound, delete/recreate and prior-history assumptions must be specified. An independent physical writer that evades the revision can invalidate the guarantee.

Logical command identity is not a one-shot attempt capability

A stable logical command such as C42 can have several physical delivery or submission attempts, H1, H2, and so on. The one-shot gateway prevents duplicate submission **through the same invocation capability**. It does not prevent two separately issued attempt capabilities from incorporating the same logical command, nor does it establish that a lost response means the command was rejected.

The durable authoritative path therefore requires a command binding containing the permitted operation, principal, target and canonical arguments; guarded exclusion or idempotent replay; an atomic relationship between domain incorporation and its canonical command result where promised; retention and disclosure rules; and an explicit compatible-versus-incompatible replay decision. A retry can receive a fresh attempt capability **for the same logical command** after the chosen command-resolution policy allows it. An incompatible binding must be rejected rather than relabelled as a valid retry.

The four backend/model outcomes `:committed`, `:rejected`, `:already-incorporated` and `:commit-unknown` are distinct from HTTP success, an XTDB API's returned map, provisional browser presentation, and a later matching Request card. A post-submit exception may occur after incorporation. Unless actual qualified evidence rules incorporation out, the result is unknown; a second fresh attempt must respect durable command exclusion and the possibility that H1 already committed.

The durable command-result record, domain write and any required outbox/publication-intent record need the **specified common authoritative commit contract**. If outbox intent is committed atomically while external publication is asynchronous, the theorem requires durable eventual retry, gap/replay/duplicate handling under named liveness premises, not falsely asserting that SSE notification occurred in the same database transaction. Duplicated invalidations are permitted when safe, but cannot create ungranted collection reads or duplicate domain incorporation.

Request-local parallelism, resource lifetime and retirement

An application is not required to serialize every logical command through one process-global writer or a global HTTP request lock. Distinct requests, tenants and independent commands may execute concurrently when their real model semantics, isolation and capacity policies permit it. Per-attempt one-shot issuance is compatible with parallel requests; durable conflict protection for the same Request/command comes from the qualified model/backend mechanism, not global thread serialization.

A request-local gateway binding records an explicit owner, original checked operation, source/handler generation, authenticated invocation context, permitted target, bounded lifetime, and the authoritative submit function. The binder permits at most the expected single submission through that capability, and rejects duplicate or retired use before submission. After dispatch it retires the specific capability, even on an exception, without erasing another request's newly issued or current writer.

A thread-local binding alone is insufficient for a callback, deferred task, subscription, future, async write, response completion or worker that executes after its original thread/request scope ends. Such execution must possess an explicit admissible context with source identity, actor/service authority, lifetime and transfer rules, and must recheck the current state and rights at the actual effect boundary. A long-lived authorized job is not a retained, expired HTTP request capability.

Bounded queues, semaphores and concurrency limits control operational capacity. They cannot create authorization, prove atomicity, or change the logical command state machine. A queue-full or admission-timeout result is a distinct operational failure from business rejection, known transaction nonincorporation, or post-submit uncertainty. Once a physical attempt has begun, cleanup or cancellation cannot retroactively prove it did not commit.

Where Biff/Ring routes participate, the actual running server must use the **final** checked handler rather than an earlier module slot. Static route-to-writer declarations, process-local registry observations and per-request callback issuance are different checks. Dynamic route parameters, callbacks, jobs and non-HTTP ingress demand real source/target/actor binding; an enumerated static literal route does not prove every possible request target or future handler.

Physical observation and provenance hierarchy

The gateway's strongest report is only as strong as the trusted physical observation actually performed. Maintain these categories separately:

Observation	Warranted scoped fact
Canonical lowering accepted	Original admitted source/plan has the selected vector and supporte
Issued process-local one-shot consumed	Registered gateway entered one owned submission path
Captured API call and return	Exact local vector and result/exception observed at that call site
Public transaction key/ledger correlation	Identified transaction metadata agrees with a qualified provider vi
Pinned domain, command and ledger readback	Those observations agree at the named single basis and projection
Installed route/registered callback observation	The chosen checked local handler path invoked the expected regis
Qualified backend transaction proof/contract	A specified atomicity, failure, guard or isolation premise holds for
Independently closed physical writer set and current runtime binding	The specified physical writer paths are constrained within the pro

A physical observation made **after** incorporation may identify a defect but cannot guarantee prevention. For a safety claim that requires a pre-effect guard, the actual enforcing boundary must reject the invalid request **before** the forbidden write. A copied report, caller-supplied node or stale process-local handle cannot stand in for independently collected physical evidence.

An exact TransactionKey and same-basis readback may be excellent diagnostic evidence without showing that the selected attempt caused the final RequestAssignment state. A transaction ledger may be corroborative without proving business guards or command identity. For full production refinement, the claimed execution includes actual route adoption, principal authentication, transaction-local prestate/frame and write causality, relevant writer/initializer coverage, command history and installed/browser observations where the theorem depends on them.

HumanHelp Claim and adversarial production acceptance

A representative protected Claim path binds the current authenticated helper, Request, Membership, Organization and Location through the real public models, uses the admitted Claim transition and physical RequestAssignment one-primary constraint, and writes the canonical command result under its stipulated atomicity and authority contract. The browser does not supply the trusted helper ID or authorize a read merely because its planned UI was formally admitted.

Acceptance requires a positive eligible Claim and genuine concurrent two-helper contention, not only a proof of the abstract transition. It must also discriminate these nearby cases:

- a source builder changes target or operation after independent binding; no one-shot execution capability may issue;
- an attacker inserts extra SQL, swaps an XTDB node or supplies a different callback; the constrained gateway rejects that substitution before protected effects;
- a second local caller invokes an already consumed one-shot; no second submission occurs through that capability;
- a duplicate physical attempt uses a fresh one-shot for C42; durable command identity and replay policy, not one-shot memory, prevent duplicate incorporation;
- H1 commits and its response is lost; H2's exception or a subsequent Request snapshot does not manufacture an original rejection or new Claim;
- a handler is replaced after source checking, or another Biff controller/direct XTDB writer bypasses the selected registry; the unverified scope is reported rather than certified as physically complete;
- two unrelated requests use independent request-local issuers concurrently; a process-global one-shot must not become an accidental product limitation;
- a deferred handler continues after its request owner has retired; it must re-authorize and acquire a valid explicit execution context rather than borrow stale thread state;
- a valid guarded Claim changes RequestAssignment while an unrelated writer changes an unguarded field; the physical frame contract must prevent destructive stale replacement;
- the real backend cannot supply an authoritative outcome after a post-submit failure; the public result remains unknown and recovery obeys permission, command history and retention;
- an outbox invalidation is lost or delivered twice across nodes; retry/replay preserves command safety and keeps remote board acquisition user-authorized;
- a purportedly complete transaction has no genuine command-result or protected absence/uniqueness guard; its narrow source/host check cannot be promoted to end-to-end Claim refinement.

Each case names the selected source, route or worker, current principal/tenant, logical command, attempt, original and mutated transaction/handler, expected first enforcing boundary, and independently inspected physical effects. Unsupported forms or unknown observation scopes fail the claimed strong admission rather than disappearing from the report. The valid Claim and independent HumanHelp product positives remain required alongside adversarial rejection.

The restricted gateway is therefore a **preferred enforceable physical seam** for supported authoritative operations, not a theorem that arbitrary host Clojure, every possible XTDB caller, HTTP routing, multi-node delivery, or browser effect has already been certified.

Guard dependencies and write footprints are different

A protected operation has at least four related but **non-interchangeable** footprints:

1. Its **semantic guard dependencies** are every state fact, relationship, presence/absence predicate, authority input and observation whose current truth or prior history controls whether a transition is permitted.
2. Its **abstract write/frame footprint** identifies the fields or modeled entities assigned and those whose presence and value must remain unchanged.
3. Its **physical mutation footprint** identifies the entire persisted documents/rows, indexes, command records and other state that the actual backend operation can change, including implicit replacement effects.
4. Its **conflict/protection footprint** identifies the physical guards, versions, absence/uniqueness constraints and relevant competing writers whose behavior must be controlled so the abstract guard and frame remain valid at the actual linearization/commit boundary.

These sets can overlap without being equal. A Claim may depend on current Membership, Organization, Location, Request status, an absent active-primary RequestAssignment and command identity, yet physically change Request, RequestAssignment, canonical command outcome and outbox intent. This does not mean Membership should be rewritten merely because its validity is a guard. Conversely, a whole-document `:put-docs` can overwrite fields that do **not** occur in the abstract declared delta. All four footprints must remain source-derived or explicitly qualified, scoped, and reviewable; passing an operation-set proof on a smaller abstract projection does not silently close the physical footprint.

Frame preservation concerns presence, not merely named values

For a supported abstract guarded transition, all assigned expressions use the stated pre-state and input with **simultaneous assignment**, and the frame preserves all unassigned relevant fields, including their presence versus absence. A declared optional absent value is not an existing field containing nil. Physical equality must respect the admitted encoding and comparison semantics: SQL `NULL` comparisons, missing fields, nested values, timestamps and cross-language coercions cannot be treated as Clojure map equality without a qualified correspondence.

The physical promise depends on the mutation form. An update that truly changes only the declared fields can have a narrower physical write footprint than a complete document replacement. A backend `:put-docs` or equivalent complete replacement modifies the **entire persisted document**, including possible removal of a field never named by the source delta. A before/after map of two business fields does not make it a two-field physical patch. The protected domain must name the whole document for a whole replacement, including presence and deletion; claiming to protect every currently known key is insufficient when another key could be added.

The following executable Clojure example from the preferred R5 section illustrates the lost-field problem. It models immutable maps only; it is **not** XTDB code, a transaction-internal snapshot or a certified operation. A complete-map comparison includes key presence by Clojure map equality. The real backend must independently justify an equivalent representation and comparison policy.

```
(def prepared-before
  {:request/status :open
   :request/revision 7
   :request/summary "Original summary"})

(def prepared-after
  (assoc prepared-before
         :request/status :claimed
         :request/revision 8))

;; A writer outside the revision discipline changes an unguarded field.
(def current-before-commit
  (assoc prepared-before :request/summary "Corrected summary"))

(defn checked-field-equal? [expected actual field]
  (and (= (contains? expected field) (contains? actual field))
       (= (get expected field) (get actual field))))

(defn partial-guard? [expected actual]
  (every? #(checked-field-equal? expected actual %)
          [:request/status :request/revision]))

(defn complete-map-guard? [expected actual]
  (= expected actual))

(assert (partial-guard? prepared-before current-before-commit))
(assert (not (complete-map-guard? prepared-before current-before-commit)))
(assert (= "Original summary" (:request/summary prepared-after)))
(assert (not= (:request/summary prepared-after)
              (:request/summary current-before-commit)))
```

The partial guard accepts the stale prepared replacement, which would restore `:request/summary` to the original value. The complete-map guard detects that conflict in this **illustrative map model**. Neither assertion tests an actual XTDB transaction. A real guarded physical writer must reject the stale replacement or avoid losing the concurrent field by another justified implementation strategy.

Qualified mechanisms for guarded replacements

For a whole-document replacement, at least one of the following must be **actually sufficient under the selected backend and writer universe**, not merely declared in a source report:

- **Whole relevant pre-state compare:** The backend tests the complete covered pre-image—including field presence, absence and supported nested representation—at the atomic conflict point before replacing the document. Comparing previously observed fields alone is insufficient if a new key can be inserted. A top-level field-count assertion may close that particular insertion gap when combined with correct all-field equality, supported non-null semantics, stable field-name mapping and a qualified atomic `ASSERT=/=PUT` contract. It is not automatically a nested-frame, multi-document or all-writer theorem.
- **Closed monotonic revision discipline:** Every physical writer capable of changing the protected state advances the exact same bounded, non-reused revision/epoch and atomic comparison checks the predecessor value. The claim specifies authorized revision constructor, overflow/nonwrapping policy, deletion/recreation identity, migrations and operational exceptions. A revision increment by **this** command alone does not prove the all-writer discipline.
- **Transaction-local derive-and-preserve:** The backend computes the replacement from the actual guarded current state inside the justified atomic transaction, preserving the abstract frame and all required failure behavior. An external read followed by client-side merge is not transaction-local derivation.
- **Genuinely narrow physical patch:** The backend's supported atomic operation modifies exactly admitted fields with protected guard and absence/uniqueness semantics. A library operation called ”update” that serializes a full replacement is not this strategy.

A mixture of mechanisms may be required for the full domain. Their exact scope includes all protected documents and relationships, not just the row with a Request status field. If the selected lowering cannot express or independently justify the necessary guard/frame relation, the constrained gateway fails its stronger profile **before submission**. An expressly opaque alternative retains its own proof/implementation premises without inheriting a stronger checked-frame label.

The proof is directional. If a physical action is admitted, then under named backend, serialization and writer-coverage premises it must refine the declared abstract transition; a match between reported field counts does not entail that every relevant physical state is covered. The root report states which fields, documents, mutation forms, conflict mechanisms, source version, current handler and backend semantics were actually qualified.

External observation, transaction-time guards and actual causality

A pinned external read establishes a particular observed state at an identified provider basis. It may supply input to a prepared guarded plan, or a bounded diagnostic comparing the declared pre-image with observed data. It does not establish the transaction-internal prestate or guarantee no conflicting write occurs between the read and submission.

A qualified XTDB `ASSERT=/=ASSERT=/=PUT` implementation can enforce particular field/count predicates against the state seen by those assertions at its atomic execution boundary **if** the reviewed backend version, operation order, snapshot/isolation, conflict and failure semantics support that interpretation. A checked SQL string, caller-supplied candidate state or accepted plan shape does not establish those backend premises or authenticate the actual executed transaction vector. A postsubmission pinned read likewise does not prove that **this** submitted operation caused its state: another writer can intervene or restore identical bytes.

Separate source/model facts, prepared lowering, selected actual gateway submission, transaction receipt, ledger/domain readback, real backend atomicity, installed model route, principal authorization, and full physical writer coverage. A copied receipt and a direct XTDB bypass cannot become an admitted operation by matching diagnostic identities. A post hoc frame violation may be detected and reported; it cannot retroactively prevent a forbidden write.

Absence, uniqueness and write-skew require predicate protection

Positive comparison of documents that already exist does not prove **absence** of a matching row or **uniqueness** of a relationship. The source guard may say there is no active primary RequestAssignment for R17. Two different helper transactions can both insert distinct assignment documents while leaving every previously read Request field unchanged. A CAS on the Request status, by itself, protects the relationship only if the actual model/writer protocol proves that both inserts must conflict through that CAS or another shared linearization mechanism.

A physical absence/uniqueness invariant therefore requires a qualified atomic predicate check, exclusive key, constraint, conflict/write dependency or protocol shown to serialize all competing relevant writers. The proof states its target relation and model's active/terminal semantics, not merely ”no duplicate document ID.” Deleting and recreating assignment records, changing their active status, switching Organization, running migrations or bypass writers must participate in the appropriate invariant/concurrency argument. A database's documented unique-index or isolation behavior is a **specific qualified backend premise** for a specific schema and operation order, not a general Gesso theorem.

For HumanHelp Claim, the source catalog must include real public RequestAssignment transitions, current Request and helper Membership/Organization/Location policy and the actual writer domain. An abstract one-primary induction theorem remains valid under its declared operation catalog even if some real admin writer bypasses it; what fails is the stronger physical-coverage/refinement claim. This boundary is owned jointly with 10.07 and 10.08, not erased by a source proof.

ABA, monotonic revisions, identity and history

A complete value compare can admit an ABA history: a value changes and later returns to the same value. This is **not automatically** a safety failure if the policy depends only on the actual current state. A ”no intervening change” theorem, historical command exclusion, lease generation or long-lived observed transition may instead depend on history, and therefore needs a history-sensitive token or protocol whose scope actually includes those changes.

A strictly increasing, nonwrapping revision provides a possible ABA defense only if the following all hold:

1. The revision has an exact total-order representation with no narrowing, coercion, arithmetic overflow or reuse.
2. The original transaction requires the expected revision and advances it in the correct atomic write; a copied revision or arbitrary new value is rejected.
3. Every relevant physical writer, including jobs, migration tools, admin operations and other nodes, participates in the revision discipline.
4. Identity evolution through deletion/recreation, reset and reinitialization cannot resurrect an old revision inside a logically new object instance.
5. The selected database actually checks the revision predicate at the required atomic boundary, with explicit conflict and unknown outcomes.

A source checker may verify this command's exact predecessor/successor arithmetic while leaving the all-writer/history premise open. That is meaningful conditional evidence, not the full physical theorem. A timestamp, increasing SSE counter, command ID or XTDB basis token must not silently substitute for the revision property: each has different provenance and ordering rules. Immutable semantic identity is a preservation relation, not automatically a monotonic progression value.

Required conformance and independent effect oracles

The checked source, physical lowering and real implementation must be tested with positive and adversarial neighbors. At minimum, the following controls are required for their selected claim scopes:

- A complete document pre-image accepts a genuinely unchanged current state; an intervening summary edit must not be overwritten by a stale PUT.
- The original map example's partial guard accepts the conflicting summary; the full presence-aware guard rejects it. This is **model evidence**, not an XTDB result.
- A new field inserted between external read and commit defeats old-field-only checks; the selected whole-document protection rejects that conflict or uses a safe narrow patch.
- Absent, present nil, scalar, nested and deleted values are represented according to supported backend codecs; unsupported SQL equality or missing/NULL coercion must reject or remain explicit.
- Two concurrent eligible helpers can attempt Claim while preserving one active primary; separate-row insertion without shared predicate conflict cannot pass.
- A valid revision successor and current expected value may admit a transaction; unchanged revision, skipped successor, wraparound and stale expected value must fail the supported monotonic contract.
- A documented ABA restoration may satisfy a current-only predicate; it must **not** satisfy a declared no-intervening- change requirement without history evidence.
- A direct admin or external writer lacking revision discipline makes the deployment-wide monotonic guarantee open or false; one bounded writer catalog cannot assert completeness.
- Actual guarded submission, backend result and readback are attributed within their observed scope; matching poststate after a separate writer is not physical transaction causality.
- Public model Claim and RequestAssignment invariants have meaningful eligible positive paths; a gateway that rejects all physical attempts cannot pass the independent product gates.

Each case must predeclare the original source and guarded policy, tested backend subset, positive control, mutated input or interleaving, first expected enforcing boundary and independent pre-effect or post-effect physical observer. An invalid attempt rejected by the source checker is not proof that a distinct external writer was prevented. A timed-out backend or incomplete causal observation must be reported as incomplete or unknown, not a checked frame guarantee.

The goal is exact protection of the *actual claimed state and history*, with only those transactional and deployment premises that have truly been qualified.

Writer coverage reaches beyond HTTP routes

The original operation catalog supplies the **abstract operation set**. The deployed writer inventory supplies the claimed **physical coverage**. Those are joined by checked operation identity, relevant guard/frame footprints and actual effects, not inferred from matching source names. A source-complete induction proof can be correct for its admitted machine while a direct XTDB caller invalidates the corresponding physical invariant. The missing fact is production correspondence or enforcement, not a flaw that may be fixed by silently deleting the physical writer from the report.

For every protected claim, Gesso must identify the state, collection, relation, tenant, authority and history it intends to cover and the potential mutations that can invalidate that property. The inventory includes ordinary routes, non-HTTP background jobs, imports, service consumers, scheduled work, administrative and maintenance tools, migrations, initializers, other processes and external services that can write through independently held backend credentials. A handler need not be an HTTP handler and every writer need not have a route. Requiring a fabricated route for a scheduled operation would confuse placement with semantic ownership.

A host-supplied enumerator establishes only what that enumerator actually observed at the named process, registry and generation. “No inventory available” is **unknown**, never an empty closed inventory. A source-relative exact operation-ID join is useful but does not establish that differently named or undiscovered physical writers are absent, that a callable has its declared behavior, or that the running service uses the checked registration.

1. A scoped physical writer universe

   A strong whole-application claim declares its **protected physical universe**, including the exact backend/provider and deployment assumptions, not merely a list of approved Clojure Vars. The record identifies at least:

   - protected domain and property: real Request, RequestAssignment, command ledger and any other relevant persisted state, relationship, absence/uniqueness predicate, revision/history or effect;
   - authority domain: tenant/Organization, allowed principals and service identities, current policy, and the intended set of admitted operations and explicitly permitted operational transitions;

- effect-capable entrypoint classes: HTTP, worker, scheduler, queue consumer, system bootstrap/initializer, migration, maintenance CLI, administrator, other service and direct provider credential holders;
- actual capability path: credentials, connection pools, clients, mutable dependency injection, route and callback/worker bindings, executable/handler generations and provider-side grants that permit mutation even when no Gesso operation is invoked;
- mutation and conflict footprints: PUT/patch/delete/erase/SQL and transaction/vector calls, entire replaced documents, protected predicates, expected revision and any domain-specific constraints;
- observation/coverage: the independent inventory source, installation scope, last successful reconciliation, open unclassified or inaccessible paths, and evidence that the enforcing boundary actually covers every relevant mutation;
- lifetime and change control: deployment/credential generation, startup and shutdown, dynamic worker creation, route replacement, job promotion, provider rotation, external operational access and the conditions that invalidate the coverage judgment.

The universe is relative to a stated threat and deployment model. A capability restriction can exclude a path only when actual backend, process or credential enforcement prevents that path from writing the protected domain. A value in a host-declared capability table `:restriction :host-declared` is not such enforcement. A user who can import an unrestricted XTDB client, a service retaining database write credentials, or an operator with direct update access must be accounted for by real confinement, a separately controlled exception contract or an explicit still-open premise.

An inventory may be **complete relative to the inspected host registry** yet incomplete relative to all processes or privileged writers. These are different claims. The consumer must not infer universal closed-world coverage by taking the union of several individually incomplete enumerators. Instead it must justify the relevant universe and its partition into observed/mediated, constrained external, and specifically qualified open paths.

2. Three independently established closure judgments

   The framework keeps three closure questions distinct.

Closure subject	What sufficient evidence establishes
Original abstract operation closure	Every original required initializer, operation, invariant and semantic depender
Observed deployed entrypoint and writer correspondence	Each inventoried current host route, worker or other entrypoint is matched to
Enforced physical writer closure	Every actual mutation capable of invalidating the selected property is mediate

   An original semantic catalog may be complete while an installed application has an additional writer. An observed inventory may contain every registered route while an initializer or service principal writes directly. A backend-enforced physical closure claim can use a different implementation from the preferred Gesso gateway, but it must demonstrate the same protected source-to-effect refinement at the boundary it actually owns. The strongest claim cannot be issued by concatenating a source-side proof and a copyable inventory summary without that missing enforcement argument.

   The source-derived catalog should independently account for operation and initializer obligations; an installed-application inspection should independently account for route, worker, dynamic target, server startup and rendered affordance identities. Source closure and product capability completeness also differ: an internally closed application missing required HumanHelp Unclaim or Reassign cannot pass the positive product gate. A registered job that writes protected state must have its own corresponding operation/initializer or explicit privileged maintenance contract; it is not required to masquerade as a route.

3. Mediation, enforcement and operational exceptions

   The preferred supported path constrains physical writers through the admitted source-bound gateway and the qualified backend guards from 10.05–10.06. A production-wide assurance claim additionally needs evidence about **access to the backend**:

   (a) Only specified service principals and operator roles can acquire protected mutation authority; grants, network routes and backend capabilities are checked at the actual deployment boundary.
   (b) Normal application ingress paths receive only the appropriate source-bound executor/callback or least-privileged backend capability. The raw client used by a gateway must not be casually exposed through unrelated application closures.
   (c) A mutation through any class of privileged writer must satisfy the matching operation's authenticated authority, transaction- internal guard, protected frame, command/result and revision discipline, or its explicitly reviewed operational exception.
   (d) Known bypass-capable credentials or direct clients are either technically barred from the covered domain, included under an independently enforced and checked contract, or retained as open trusted premises that prevent the stronger claim.

(e) Current deployed handler/worker/callback and credential generation must still match the inspected/issued assembly at the time of use. A previously complete startup snapshot is not proof of later runtime stability.

An operational exception is not a general "administrator bypass" with automatically preserved invariants. It names who can invoke it, its permitted domain and mutation, the required approval and audit, the protected before/after or migration relation, whether the invariant is maintained or temporarily suspended, and the conditions under which stronger dependent certification resumes. A break-glass write can be necessary, but its presence changes the scope of any unqualified whole-domain proof.

A release migration can change schemas, rebuild assignments, reset revisions or create initial records. If those effects are outside the proved operation catalog, the deployment must either prove that the migration refines an admitted transition, or establish a new valid initial state and requalify the affected invariant and physical realization afterward. Checking that an abstract invariant held **before** migration does not prove it holds afterward. A startup initializer has the same physical responsibility even if it runs only once.

If an external provider enforces a relevant uniqueness, transaction isolation or credential restriction, the specific provider version, configuration, scope and failure semantics enter the claim as checked evidence or a named external trust premise. Merely using XTDB, Biff, a container sandbox or a particular route framework does not automatically provide those guarantees.

4. Dynamic entrypoints, multi-process identity and TOCTOU

   Real deployments may change handlers, start workers, restart nodes, rotate database credentials or replace callbacks while the application continues serving. Inventory freshness, currentness and the order of checking/activation matter. A process-local private registry can strengthen one in-process caller-origin check without authenticating the actual serving process, a remote node, the physical database client or a later deployment generation.

   The selected assurance profile must specify how the actual writer-enforcing boundary remains valid after startup. A correctly checked route-to-writer table whose handler is later replaced with a raw XTDB client no longer establishes current coverage. A positive request that passed an earlier registry snapshot does not prove all concurrent or future requests went through that registered writer.

   A robust design may use generation-scoped capability issuance, immutable checked routing, runtime enforcement plus ongoing qualified revalidation, provider-level permission separation, or another demonstrably sufficient mechanism. The chosen mechanism must close the relevant check-to-use gap rather than simply run a second enumerator after the write. A post-effect discrepancy is useful detection, never retroactive prevention.

   Multiple nodes do not require a second model of Claim or a new category of participant authority. They do require qualification of **all** node, service, scheduler, administrative and external writer paths in the protected universe. A failed host may lose process-local registry state while its previously submitted transaction can still incorporate; retry and command history must remain durable and appropriately scoped. Router interest propagation, outbox publication and node-loss replay are different from database write authority. An invalidation re-delivered safely may be harmless, but it cannot create an unmodeled domain write or a forbidden automatic remote board read.

5. HumanHelp one-primary and command-history coverage

   For the real Claim application, enumerate actual Request and RequestAssignment mutation entrypoints, plus relevant Membership, Organization and Location policy dependencies. An actor can be eligible under the abstract Claim contract while a raw migration or manager operation produces a second active primary outside the constrained model. The abstract theorem still describes its declared transitions; the whole-domain physical theorem does not follow.

   A stronger end-to-end Claim invariant requires: the source-derived original Claim/Unclaim/ Reassign and lifecycle catalog; protected concurrent assignments including absence and uniqueness; real public model guards; all relevant normal and operational writers; transaction-local current-state/frame correspondence; durable command identity and result; and actual installed call sites. A changed Membership or Organization may alter authorization before the Claim commits even if that model is not written by Claim itself. The physical guard dependency must therefore be protected under the qualified isolation contract, not added spuriously to the write footprint just to make a catalog look complete.

   A direct maintenance writer might still be permitted under a separately approved operation. It cannot be used to claim the original all-operations one-primary theorem without matching induction/transition coverage or an explicit reestablished initial state. A copyable `complete?` report does not grant administrative authority or turn an unknown writer universe into an empty one.

6. Adversarial coverage and honest reporting

Coverage tests require a valid source/installed operation and a deliberately selected negative neighboring writer, not a reject-all system. At minimum, demonstrate:

- A registered HTTP Claim and separately registered authorized scheduled operation are both accounted for under their correct actual entrypoint classes; the job is not assigned a fake route.
- An original source-complete operation catalog joined with a current route/ issued-writer table is reported as exactly that; inserting a raw writer the enumerator did not report cannot be considered disproved.
- A declared table-wide `:put-docs` or `:sql-dml` capability touching the protected domain cannot be reduced to only the currently listed documents.
- A genuine enforced read-only role cannot mutate the protected domain; a host map claiming `:read-only` while the same credential can write fails the strong closure claim.
- A live admin/migration writer that changes RequestAssignment must carry an appropriate source/ operational transition contract; a hidden backdoor breaks unqualified one-primary physical safety.
- A startup initializer and later maintenance phase satisfy their declared initial-state and requalification obligations; a migration that silently resets revision/ABA history does not.
- A current checked Biff handler actually invokes its registered writer during one admitted request; source-matched registration alone cannot certify the real deployed server or other handler generations.
- An async callback or worker uses explicitly scoped service principal and execution capability; it cannot borrow a retired request's thread-local writer lease.
- A privileged direct XTDB client is excluded by real provider/ capability enforcement or retained as a named premise; a missing enumerator entry cannot close the universe.
- A new deployment generation changes an actual handler, callable or credential; affected physical correspondence is invalidated, while unrelated abstract proofs can remain valid where qualified.
- Two real helpers race for a Request while another permitted writer mutates its assignment/revision; the protected one-primary invariant remains enforced or the additional writer is shown outside the claim.
- A failed transaction and dropped/duplicated outbox notice preserve actual command history and permitted advisory recovery; a router cannot turn a notification into a new Claim or unauthorized collection read.

Each test names the original source, protected domain, capability path, required source or operational transition, selected host/credential generation, first enforcing boundary and an independent effect observer appropriate to its scope. A static mismatch can reject a claim before execution; a controlled post-dispatch observer can detect a difference **after** an effect. Neither can prove that all other physical writers are absent without the separate closure mechanism.

Root assurance reports separate the following conclusions:

Report result	Meaning
`:source-catalog-closed`	Every original required abstract operation/initializer obligation in the inspected source is repre
`:registered-paths-classified`	Every writer actually returned by the scoped host enumerator is classified under the inspected
`:declared-capabilities-covered`	Supplied writer target/mutation declarations correspond to the protected-domain declaration i
`:installed-mediation-observed`	One or more actual selected installed call sites/requests were observed with identified source a
`:physical-writer-closure-established`	An independently qualified enforcing boundary excludes or constrains **every** relevant writer w
`:physical-writer-closure-unknown`	The actual universe or its enforcement has not been established; an empty inventory is not suf

The labels above are **normative report concepts**, not a claim that today's Gesso exports these keywords or that a caller can assert `:physical-writer-closure-established` in an untrusted map. The issuer applies the selected profile and the actual supporting evidence. A checked source theorem, matched operation-ID lists and a convenient inventory do not produce the last result by arithmetic.

The final requirement is deliberately strong: every real effect capable of invalidating the claimed invariant must be included, restricted, or explicitly assumed under a scope that the claim truthfully discloses.

HumanHelp Claim as a correspondence slice

Claim's public semantic contract, admitted plan, physical transaction, and participant observation have one connected meaning. The example is the self-claim case. A separately authorized operation that assigns another helper must bind and guard that target helper's Membership as well as the actor's permission; it cannot reuse self-claim assumptions unchanged.

1. Source-owned identities, actor and helper distinction

   The **source** of the Claim decision is the current public HumanHelp production model and the application-selected Claim operation, not a proof-only Request entity. The correspondence ledger binds the source generation, actual public model

interface versions, normalized Claim operation, current actor and authority domain, complete original operation/invariant catalog, role-projected executable plan and installed executor. Each identity has its own lifecycle: neither command ID, HTTP attempt, XTDB receipt, model operation ID, browser resource lease nor proof fingerprint can substitute for another.

The self-Claim principal, helper receiving the assignment and Request target refer to their own typed identities. They happen to denote the same person for a genuine self-Claim, but the relation is proved at the admitted operation, not assumed globally. Manager Reassign or a separately authorized claim-for-another operation has a distinct **actor**, **target helper** and existing-primary context. Its guard includes the actor's authority to select another helper and the target's independent effective Membership and eligibility. A source check or UI gesture cannot turn a self-Claim operation into an authorized manager assignment by swapping a helper argument.

The selected proof projection must name the **representation relation** from the public Request, RequestAssignment and effective Membership, Organization and Location records to the abstract model's state and roles. Derived display labels and cached board cards may stand for those facts only when the producer, disclosure scope and observation frontier are known. A single conveniently nullable helper field is not a second authoritative assignment store; a proof over that field must account for every primary assignment that contributes to its meaning. Assertions about actual schema keys must come from the current public model interfaces, not from illustrative Clojure data in Part IV.

2. The command envelope and trusted invocation

   The browser sends the domain Request identity and stable `:gesso/command-id` through the admitted command boundary. A transport may add `:http-request-id` for this delivery. The latter identifies neither the domain Request nor the command across retries. The server binds the principal from trusted context and validates the operation, interface, canonical arguments, and authority-domain binding of the command.

   The formal state projection includes the Request and the relationships relevant to its primary RequestAssignment, Membership, Organization, and durable command record. Primary ownership is not modeled by inventing a `:request/helper-id` field. A projection may summarize these relationships, but its representation relation must explain how the actual records establish that summary.

   The simplified candidate in Part IV illustrates a decision over supplied values. Using a nullable helper field in such a projection does not change HumanHelp's storage or authority model. Its production correspondence must still account for the primary assignment and every guarded relationship on which that value rests.

   The authority-domain binding must include the Request and selected Organization/tenant context at the trusted server/model boundary. An observed basis can justify some **client presentation** obligations but it does not establish current backend Request status, Membership or organization policy. The route maps exact typed operation, canonical arguments, actor and command binding to the selected public model contract; it does not promote a client-supplied actor, role, provenance flag, one-Refresh permission or `:authorized?` field into trusted authority.

   The source/installed correspondence must show that this browser control actually emits the admitted command envelope and that the deployed Ring/Biff route, authenticated principal, selected model callable and gateway share the compatible assembly/host generation. A manually inserted command attribute or synthetic successful request is permitted as a **route-level fixture**, but cannot count as an actual browser-originated positive product witness. A stale served plan or replaced handler reopens the relevant installed-realization edge even when the original abstract proof remains valid.

3. Historical command decision precedes new eligibility

   The transition distinguishes these branches:

Branch	Required decision	Au
Existing compatible command	Validate its full binding and current permission to disclose its recorded result	Re
Existing incompatible command	Detect disagreement in operation, arguments, principal, or authority binding	Re
New command with satisfied current guard	Enforce the complete current predicate and command exclusion atomically	Inc
New command with failed guard	Establish the typed nonincorporation outcome under the command-resolution protocol	No
Outcome not established for participant	Preserve uncertainty and the command binding	Re

This decision ordering is a **logical semantic requirement**. The implementation may combine checks in one protected transaction or route them through a separate authoritative read, but the observed behavior must be equivalent to compatible-history reconciliation before treating the request as a new command. A command key is **not** a security capability: its result may be disclosed only under current authorized disclosure policy, and a mismatched binding must not reveal another actor's command existence or stored outcome.

An old command can truthfully have incorporated while current Request state has been changed by a later Unclaim, Complete or Reassign. Conversely, a current claimed card does not attribute incorporation to C42. A not-found command record at one external pinned basis does not establish permanent nonincorporation while a prior attempt may still be pending. Retention expiration and inaccessible history must preserve an explicit unresolved/unknown branch unless stronger authoritative evidence establishes a different outcome.

The existing-command branch is logically prior to applying the new-command business guard. A successful retry must not fail merely because the first incorporation made the Request claimed. It still requires compatible binding and permitted disclosure. A forbidden lookup must not leak whether a protected command record exists.

For a new self-claim, the commit guard covers at least:

- the Request exists in the permitted authority scope and is currently open;
- the actor is the authenticated helper and has the required active Membership;
- Membership, Request, and the guarded Organization refer to the same Organization through the actual association paths;
- the current Organization policy permits this Claim by this principal;
- the primary-assignment conditions and any absence or uniqueness predicates required by the public model contract hold;
- the bound command has not already been incorporated incompatibly or won by a concurrent delivery;
- any other declared current policy dependencies are guarded as part of the same atomic decision.

4. Real guarded transition, physical race and frame

   The original-source invariant is stated against the public model's actual active-primary RequestAssignment definition. A useful abstract form is that each permitted Request has **at most one** currently active primary, but the actual source must define active, historical/terminal, collaborator versus primary, and any allowed transition of a Request to and from terminal states. A proof does not establish **exactly one** primary in every state: an open or unclaimed Request can legitimately have none. Source-derived induction must include relevant initializers, Unclaim, lifecycle transitions, manager Reassign and other assignment writers, not only the Claim step that is easy to prove.

   A new self-Claim must check both the public Request lifecycle and eligibility/relationship guard **at the actual transaction conflict boundary**. An external source snapshot, card, client basis or server pre-read may help prepare the attempt, but cannot authenticate the transaction-internal prestate. Protect the absence/uniqueness of another active primary even when competing helpers create **different assignment document IDs**; comparing only an existing Request status field is sufficient only if a qualified shared conflict protocol demonstrably excludes both incorporations.

   The abstract frame and physical footprint are separately specified. A complete-document PUT can overwrite an unrelated summary; an assignment status or deletion changes primary cardinality even without a Request row mutation. Document replacement, field presence/nil, expected revision, ABA history, command record, durable outbox intent, and backend guard/commit semantics retain the exact 10.05–10.07 obligations. A controlled two-helper race must demonstrate the correct positive winner and actual nonincorporation of the competing **different** command, without accidentally eliminating unrelated legitimate concurrent requests.

   Request lifecycle change, primary assignment change, durable command result, and required publication intent share the promised atomic boundary. An abstract single-record induction example does not establish this multi-record Claim contract. If the selected physical adapter supports only one-document updates, Claim's stronger claim remains unavailable until the complete operation has a qualified realization or an explicitly admitted implementation premise under a profile that permits it.

   The selected atomicity contract also determines how a guard failure is **known**. A backend conflict or post-submit exception may not prove which command incorporated, and request-local transport failure must not be reported as a public-model business rejection. A rejection record, if created, has its own typed authority and historical disclosure requirements. A gateway's process-local one-shot capability prevents a second use of that attempt, not two separate H1/H2 submissions of the same logical C42. Durable command binding and exclusion remain independent model/backend facts.

5. Model, participant and observation outcomes

   The outcome mapping is explicit:

Model outcome	Presentation treatment	Evidence retained
`:committed`	`:accepted` for a successful Claim	Command-specific incorpora
`:rejected`	`:rejected`	Typed rejection justified by t
`:already-incorporated`	Follow the permitted recorded outcome; a recorded successful Claim is accepted	Historical command attributi

Model outcome	Presentation treatment	Evidence retained
`:commit-unknown`	`:unknown`	Original command binding a

An `:already-incorporated` result is **historical** attribution of the original command to its recorded disposition, not a new successful domain mutation. If the recorded outcome was a rejection, the permitted replay presents that rejection rather than treating every compatible record as successful Claim. An indeterminate attempt continues to be `:commit-unknown` until a qualified authority establishes one of the other outcomes. The browser's `:accepted` is a projection/presentation status, not an independent XTDB commit certificate.

The real participant trajectory is separately identified: the originating button/DOM resource, emitted logical C42, H1/H2 network requests, settled or unknown adapter state, current owner/target, permitted authoritative acquisition, actual target-specific HTMX/OOB completion and rendered card. An HTTP 200, SSE advisory, optimistic ”Claiming…” badge, event handler call, source-compatible manifest or readback report does not establish all of those. A completed domain transaction remains incorporated when rendering or DOM installation fails; safe recovery cannot rewrite that historical fact.

A card reacquisition can restore a current authorized presentation. It cannot by itself prove which command produced that state. Command-result reconciliation is a separate protected acquisition. Lost disclosure permission or expired outcome retention may prevent attribution even while a permitted card read can supersede the old provisional view. The liveness claim must include the needed retention and disclosure premises.

6. Correspondence edges and independent physical witnesses

The formalization and assurance apparatus produces a **dependency record**, not a synthetic proof that every stage occurred. Claim semantics are owned by the public model, transition and Choreo. Each physical edge uses the actual source of its authority:

- Original source to abstract proof: exact public interfaces, admitted source operation, current normalization, original operation/initialization and invariant closure, typed rule/projection checks, and checked premise inheritance.
- Proof to issued handler: accepted compatible plan, exact model callable and source-owned route, current assembly artifacts and required trust contracts.
- Handler to real transaction: current authenticated principal and permitted operation/tenant, full atomic guard, physical assignment and command-result footprint, complete relevant writers and the qualified backend isolation/failure semantics.
- Transaction to historical outcome: actual canonical command binding and stored result under its retention and disclosure contract; H1/H2 receipt and later poststate observation are distinct evidence.
- Outcome to browser presentation: actual delivered response or allowed authoritative reconciliation, current execution/target ownership, scoped acquisition, physical DOM completion and admitted remote advisory/Refresh policy.

For this slice the correspondence ledger must separately show:

Obligation	Suitable evidence or explicit premise
Abstract Claim preserves the declared invariant	Checked guarded-transition or induction result over the exact projection
Physical records represent that projection	Checked representation definition or scoped trusted mapping
Current policy dependencies enter one atomic guard	Constrained lowering plus qualified backend guard semantics, or another justified
Physical writes implement only the admitted effects	Complete immutable transaction footprint and frame argument
Retry cannot incorporate twice	Atomic command exclusion and durable binding/result contract, including retenti
Actual invocation uses the admitted plan	Installed entrypoint and gateway mediation evidence
Commit and outcome are not confused with delivery	Provider outcome qualification and command-resolution enforcement
Required remote notice survives the promised failures	Atomic outbox intent plus the selected dispatch/recovery contract
Browser effect follows its permitted result	Explicit outcome mapping and final effect/adoption gates

The ledger's mandatory evidence categories cannot be discharged by one generic success flag. It should separately expose a checked abstract-invariant theorem, admitted operation/projection/plan, conditional model/public-interface mapping, installed route and loaded asset identity, physical guarded multi-record write, durable command/outcome and all-writer qualifications, actual browser installation and independent eligible user/product acceptance. Unknown/unsupported checks remain open at their first responsible boundary; missing positive application capabilities remain product failures even when source checks pass.

An opt-in command-resolution and XTDB readback mechanism may relate a source-bound plan, process-local return, pinned command record, and caller-reported outcome within a declared scope. Such physical **co-observation** does not by itself authenticate an installed production route, the current principal, the submitted transaction's causal effect, the full physical writer set, or actual browser command settlement. Its results never grant permission to retry, acquire a protected collection, or install a current DOM target.

7. Independent vertical acceptance and failures

   The Claim pilot requires a real eligible browser-originated positive path and discriminating adversarial neighbors:

   (a) **Eligible self-Claim:** real authenticated Alice sees an authorized open Request R17, the issued button generates C42 and H1, the trusted server/model accepts and atomically incorporates a new active primary, durable C42 result and required publication intent, and the permitted real card is physically adopted.
   (b) **Current authority:** after a rendered card but before incorporation Alice loses the effective Membership, moves outside permitted Organization/Location policy, or Request becomes non-open; the actual protected write is rejected, not merely the UI disabled.
   (c) **Two-helper contention:** Alice/C42 and Bob/C43 have genuinely concurrent protected attempts for the same R17; at most one **new** primary is incorporated, the loser has a justified typed result, and neither leaves a partial assignment.
   (d) **Historical replay:** C42 incorporates and H1's response is lost; a compatible H2 resolves C42 without a second incorporation or mistakenly rejecting it because R17 is now claimed.
   (e) **Binding/disclosure:** reusing C42 with different Request, actor, operation or arguments rejects; an unauthorized history query neither reveals another principal's outcome nor supplies a retry capability.
   (f) **Distinct actor/target:** a genuine manager-target Reassign guards both the manager's and replacement helper's effective rights, and does not borrow self-Claim assumptions or silently classify a takeover as the same operation.
   (g) **Current state versus history:** an authorized later Unclaim/Reassign changes the current primary but preserves C42's recorded past; current card state cannot stand in for historical attribution.
   (h) **Partial rendering:** domain incorporation succeeds but response markup or one OOB target fails; presentation stays honestly unresolved/partly installed and uses appropriate scoped recovery without issuing another Claim.
   (i) **Deferred/stale browser effect:** an old execution receives a delayed response after the resource/target is replaced; physical effect-time rights reject its DOM mutation without erasing a real committed command.
   (j) **Remote advisory/Refresh:** Alice's creator-local new Request S can appear immediately while Bob's remote change R is advisory. Bob's one user-authorized Refresh acquires and adopts only its permitted current scope; SSE delivery alone cannot force a whole-board read.
   (k) **Writer bypass:** a raw admin, background, migration or other external XTDB path capable of creating a second primary is constrained, modeled under an approved transition, or left as an explicit failed/open physical-closure premise—not silently omitted.
   (l) **Evidence provenance:** a bounded pinned readback, matching ledger or copied proof report is accepted at its actual observation scope but cannot assert physical causation, universal refinement, installed browser identity or product completion without the independent witnesses those claims require.
   (m) **Positive product coverage:** legitimate Create, Unclaim, Mark-on-the-way, Complete, Cancel, manager Reassign, board continuity and recovery remain independently required; a Claim-only proof cannot erase their absence.

   Each acceptance case names the original semantic source, current authority/host generation, selected physical actor, operation, command, transport attempt, target/asset, expected first enforcing boundary, positive neighbor, adversarial mutation and independently observed domain/history/browser effect. A source-level rejection does not certify an untested physical bypass was prevented. A post-effect diagnostic may identify a violation but cannot retroactively undo a committed transaction. A timed-out, uninspectable or privacy-forbidden observation is incomplete, not a passed physical refinement.

   There is no generic call that proves arbitrary Clojure execution refinement because an implementation Var was named in a map. Each row must identify the mechanism that actually establishes it, or retain the precise premise. Passing one Claim scenario is useful integration evidence; it is not a proof of all concurrent executions.

Formal obligations must return to executable production boundaries

A formal statement about Gesso's admitted semantics is useful only when the corresponding implementation responsibility can be found, qualified and checked at the boundary that can actually make the statement true or false. Every correctness-relevant **formal assumption**, operation, initial condition, resource, message, observation and permitted failure retains a traceable origin in the original semantic source and a named production realization obligation. This is **reverse traceability**, not an assertion of reverse simulation, executable completeness or proof that every formal trace can occur in XTDB, a real browser or a deployed multi-node application.

Part IX owns original-source proof obligation completeness, checked derivations, assumption inheritance and issuer-controlled admission. Sections 10.01–10.04 distinguish the formal theorem, the checked source and the concrete-to-abstract execution/refine-

ment relation; 10.05–10.08 spell out the actual gateway, guard/frame, writer-universe and HumanHelp Claim obligations. This section connects them without inventing a second proof language or accepting hand-authored formal machinery as a replacement for the application. Sections 10.10–10.12 later own external checker metatheory, finite abstraction and reproducible proof artifacts.

A formal conclusion with an unestablished physical edge can still be a genuine abstract theorem. Its **production** consequence is conditional, unsupported or not admitted under the selected strong profile. A report that says ”source checked” or ”formal proof passed” cannot silently promote that edge to current runtime authority.

1. Production-obligation record and first enforcing boundary

   For every transferred claim, the trusted source-derived ledger records the exact following facts, with no empty slot interpreted as success:

Obligation dimension	Required interpretation
Original source	Model public interface, typed rule, operation, Choreo declaration, initializer, invariant or selected product pol
Formal subject	Generated normalized transition/proposition, selected abstraction and observation alphabet, quantifiers, chec
Proposition and conditions	Assumptions and premises used, failure/unknown semantics, source-derived coverage and nonvacuity requir
Production subject	Actual model callable, installed handler/worker, plan, route, generated/browser artifact, backend client and d
Concrete representation	State/data and event mapping, actor/tenant/command identity, field presence, association, abstract frame and
Responsible first boundary	Source type/preflight checker, trusted ingress, protected model policy, transactional commit/conflict check, ba
Qualifying evidence	Genuine issuer-controlled check or independent physical observation, owner, provenance, freshness and supp
Remaining obligations	Open implementation/provider/opaque assumptions, unsupported checks, incomplete observation and indepe
Change and privacy	Exact invalidation/requalification rule, safe source span and redacted witness appropriate to the authorized re

   This is normative **record content**, not a prescribed public map API. Source locations and normalized IDs help developers navigate the report, but neither a Clojure Var name nor a source fingerprint proves the installed handler uses the checked operation. A source transaction target may depend on Request, RequestAssignment, Membership, Organization, Location and command history; flattening all those to an unqualified ”current state” destroys the information needed to prove atomicity and policy correspondence.

   The boundary's role must be classified accurately. A source/type checker can reject a malformed operation before issuance; an authenticated model guard can prevent an unauthorized protected effect only if it truly executes before the effect; a transaction ledger/readback can diagnose one completed event, but cannot retroactively prevent an invalid commit. A browser effect-time permission check can prevent a stale DOM mutation but cannot undo an already committed domain operation. Distinguish **prevention**, **post-effect detection**, **bounded observed correspondence** and **unestablished universal assurance**.

2. Formal assumptions create independently checkable production debts

   Every modeled capability or favorable environment hypothesis creates a matching physical obligation. Representative cases include:

Formal construct or premise	Required implementation/enforcement edge
Authenticated principal or tenant	Trusted server/session/service identity, current Membership and permitted authori
Atomic authoritative operation	Real guarded transaction or independently justified atomic implementation, curren
Absent active primary	Protected RequestAssignment cardinality/absence predicate across competing distin
Stable logical command and once-only incorporation	Durable C42 operation/actor/target/arguments binding, atomic command result/exe
Trusted authoritative observation	Provider-qualified basis, actual scoped read/projection, appropriate disclosure and a
Fresh browser resource and permitted effect	Real resource/owner generation, retirement, effect-time authorization, actual target
Eventual command/recovery/notice	Qualified retention, delivery/replay/gap repair, scheduler/provider/fairness and nec
Closed original operation catalog	Independently rederived initializations, all relevant operations/invariants and proof
Closed installed writer set	Qualified provider/credential/host enforcement across routes, workers, migration/a
Portable semantic execution	Supported JVM/CLJS/codec/plan and loaded artifact relation under its tested or pro

   Such edges may be justified by a dedicated checked mechanism, a verified compilation/refinement theorem, an actual enforcing boundary, a carefully delimited third-party/backend contract, or an explicit profile-permitted opaque premise. The evidence classes and their strength are **claim specific**. An open premise may remain in a useful conditional theorem; it cannot be marked discharged by attaching a shape-compatible success report. A strict profile rejects unavailable mandatory evidence while an explicitly requested weaker profile may admit a different, honestly conditional claim. Neither changes original business policy without source-owner approval.

An atomic formal Claim action in particular creates an atomic **production** obligation. A sequence of correctly observed pre-reads followed by an unconditional write does not satisfy that obligation, because Claim's current Request status, helper eligibility, relationship guard, active-primary absence and command exclusion must be qualified at the actual protected conflict boundary. The real backend may use a guarded XTDB transaction, an equivalent conditional mechanism or a separately proved implementation; identical source syntax is not required, but the same relevant observable transition and rejected/unknown outcomes are.

3. Correspondence direction, coverage and non-vacuity

   The strong safety-transfer claim is directional: **every relevant actual admitted production execution** under its stated environment and initial-state relation must abstract to an allowed semantic transition or trace. The chosen abstraction identifies which real states, events, stuttering/internal steps, failed/unknown outcomes, concurrent writers, side effects and browser observations count. A successful generated formal trace, matched operation name or one correct concrete execution is not the required universal quantification.

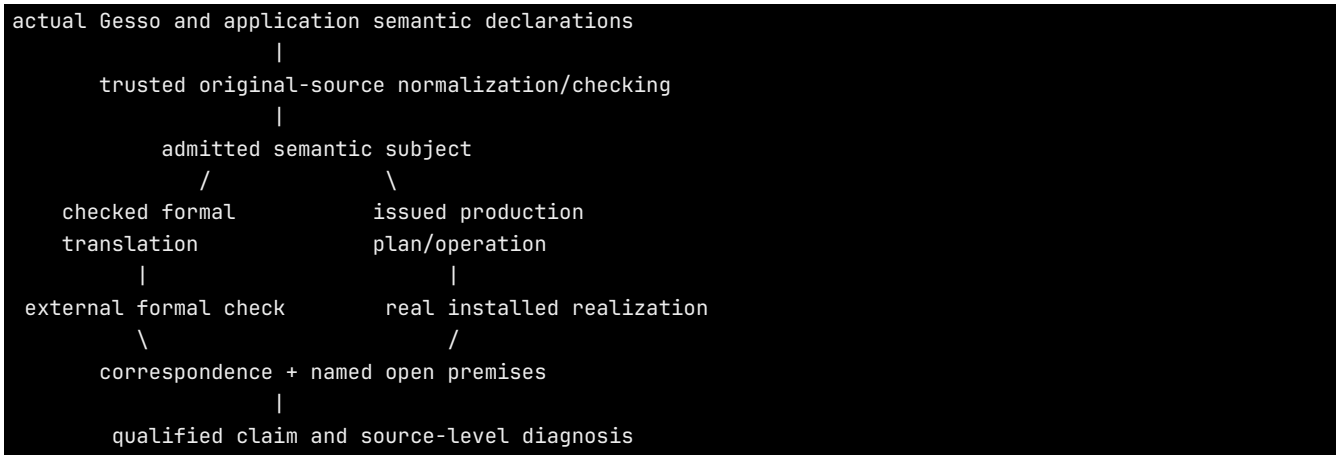
   A converse inclusion—every formal trace is physically realizable— is a distinct completeness claim if the selected profile needs it. A correct source-level counterexample can be physically unrealizable because of a deliberate abstraction over-approximation, a stronger deployed guard or excluded environment schedule. The **formal** safety claim still fails in that selected model, unless a justified refinement of the model or property removes the spurious behavior. The production claim cannot be declared false until the concrete relation and realizability are examined.

   A different danger is a model that omits a real operation or declares Init empty, so its invariant is trivially true. Source-derived original catalog closure, satisfiable/representative initial state, real eligible Claim and independent HumanHelp capability positives prevent such vacuous success from satisfying the claimed application. A universal proof over a carefully restricted subset is not a proof that the subset describes all actual source or physical behavior. A complete submitted DAG is not necessarily a complete **original-source** obligation ledger.

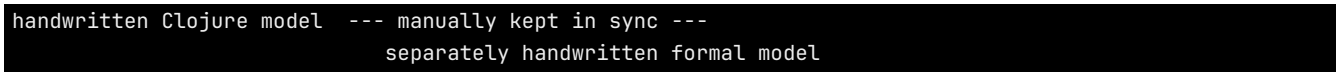
   Progress transfer has additional fairness, execution-continuation, retention, availability, delivery, quiescence and authorized participant-action conditions. A safety simulation that allows indefinite stuttering need not provide progress. Under HumanHelp's explicit Refresh policy the browser may legitimately remain unaware of remote changes until its user grants one scoped acquisition. Modeling the user as eventually refreshing without a stated premise changes product semantics and cannot support an unconditional liveness theorem.

4. Generated formal artifacts derive from source; repair the owner

   The preferred generation path is:



   Do not replace it with:



   A framework-maintained translation to Recife, Quint/TLA+, Lean or another supported backend is a derivative of the admitted semantic source. Backend version, translation rules, checker inputs, selected bounds/abstractions and actual generated bytes remain identified and reproducible. The backend can have its own implementation defects or soundness premises. A proof about the generated model is not a proof that the **translator** preserved the original meaning or that production executed that meaning.

   Application developers must not have to author, debug or hand-synchronize such a second formal specification. They may inspect or request explanations of emitted formal code. Maintainers may directly repair the framework's translation as a

source-controlled framework change. Changing a generated invariant in place so it becomes weaker and a checker turns green is not an application repair and cannot establish the previously claimed property. An intentional weakening of the actual business invariant is a **visible owner-selected policy change**, requiring corresponding source, proof, application and product reacceptance.

5. Counterexamples identify the modeled failure, not an invented physical event

   Suppose a **candidate defective** Claim model checks Request `:open` at an earlier observation and fails to require that status at its commit action. A formal checker may produce this bounded trace:

```
COUNTEREXAMPLE IN THE FORMAL SUBJECT
Property: Claim requires an eligible, open Request at incorporation.
A observes Request R open.
B observes Request R open.
A incorporates Claim for R.
B incorporates another active primary Claim for R without an intervening release.
Scope: the selected candidate model, initial states and exploration bounds.
Physical XTDB execution of this history: not established.
```

   That is a counterexample to the defective **formal candidate** if its admitted transitions really allow those incorporates. It is not a counterexample to a correctly guarded formal Claim action, nor evidence that the real XTDB model actually performed both writes. If the generated model purported to include the current-open guard, inspect source normalization, translation, property and abstraction before attributing the counterexample to a production bug.

   Conversely, a correct abstract guarded model may pass while an installed writer uses an outdated Request pre-read and unconditional PUT. Then the source-level theorem may be valid while the real refinement obligation is open or refuted. A matching pinned poststate, command ledger or local transaction receipt can be bounded diagnostic evidence without authenticating causation or excluding other physical writers.

   A good counterexample/first-blocker diagnostic retains:

   - original named property, its semantic subject and current source location; the actual original model and operation catalog against which completeness was checked;
   - the smallest **reproducible and meaning-preserving** trace prefix or typed-rule valuation, including actor/command, Request/assignment and observation identities only where authorized and genuinely relevant;
   - selected formal encoding, checker version, abstraction, finite bounds, fair/unfair schedule assumptions, source-to-generated relationship and exact failure classification (counterexample, unsupported, exhausted, vacuous, stale or incomplete);
   - the exact failing semantic guard, frame, knowledge or premise, if supported by the evidence; otherwise the plausible alternative model/translation/physical owners still under examination;
   - the production callable/route/worker, transaction/ writer or browser effect and its current generation, and the first actual enforcement/observation boundary to be checked separately;
   - which checked propositions remain valid, which specific physical bridge is absent, and whether an independent **actual** implementation trace concretizes the modeled failure.

   This is a data and explanation contract, not permission to expose raw private Request, Membership, command record, XTDB query or actor identities in a copyable report. Diagnostics use stable structured reasons and authorized redacted source witnesses, with correlation handles interpreted only at trusted boundaries. A failing downstream artifact blocked by one missing root premise is not reported as a collection of independent semantic bugs.

   When reducing a counterexample, an automated shrinker must retain the discrepancy and the declared source semantics, not merely find a shorter trace with a different failure. A minimized finite counterexample is bounded by the generated/tested language and domains; it does not become a universal proof or a real production incident by being reproducible.

6. Source-level and production repair are distinct ownership decisions

   The supported correction loop is:

```
original Clojure/Gesso source and actual product policy
        → source-checkable admitted semantics
        → regenerated formal subject and executable plan
        → independently classified proof/model-checking result
        → source-located counterexample or missing obligation
        → identify semantic, translation, provider, or physical repair owner
```

```
        → owner-approved source/implementation/assumption repair
        → rerun source, proof, assembler, production and product gates
```

The possible owners are separate:

Finding	Correct owner and repair
Defective admitted model guard	Application/public model owner corrects the actual rule/transition; rebuild proof an
Incorrect formal lowering	Framework maintainer fixes translation and rechecks semantic correspondence, che
Abstract overapproximation	Review abstraction and property against original semantics; refine within supported
Production lacks qualified atomic guard	Model/host/backend owner repairs the actual guarded write and tests concurrent ef
Missing original-source operation or physical writer	Restore source completeness or real writer confinement/qualified exception; rechecl
Current business policy differs from intention	HumanHelp owner explicitly changes and independently accepts model policy and
Unsupported checker/proof fragment	Add a compatible admitted proof mechanism or retain an allowed precise premise/o
Browser effect occurs after retirement	Repair current owner/effect-time permission and real shell behavior; exercise actual

An assistant or automated tool can propose and explain these repairs, but it does not acquire business-policy authority, trusted source issuance, deployed writer access or checker acceptance. A proposal is not a certificate. Repairs must preserve the same original-source requirement or explicitly announce a changed policy, applicability or assumption. Re-run the relevant negative mutation and a **real eligible positive control**; a repair that simply rejects all Claim attempts is not HumanHelp acceptance.

7. Acceptance: both directions of explanation and independent evidence

   For each production-transferred formal claim, Gesso must answer two questions without requiring developers to manually reconcile two source languages:

   (a) *Which original semantic object, executable production boundary and current host/asset generation does this formal subject constrain?*
   (b) *Which actual evidence discharges each correctness-relevant assumption used by the formal claim, and which remain open?*

   The following positive/adversarial pairs are required at their applicable scope:

   (a) **Source origin:** a generated Claim model, exact original source and emitted plan share checked semantics; a hand-modified formal model or stale source-ID pair does not pass.
   (b) **Original completeness:** all declared operations/initializers and invariant dependencies appear; a pruned Unclaim catalog is rejected as complete proof.
   (c) **Abstract counterexample:** the defective pre-read-only Claim candidate yields a scoped source-located trace; a genuinely guarded candidate is not falsely blamed.
   (d) **Correct model/incorrect implementation:** formal atomic guard succeeds; an unguarded deployed writer fails execution/refinement rather than the correct abstract theorem.
   (e) **Trusted actor:** current authenticated helper/Organization policy authorizes eligible Claim; browser role assertion does not discharge actor authority.
   (f) **Absence and uniqueness:** real concurrent different-command Claims preserve one active primary; two stale pre-reads followed by distinct assignment inserts cannot count as guarded incorporation.
   (g) **C42 historical result:** compatible H1/H2 recovery returns the qualified recorded result; a current Request card or missing row at one basis cannot invent historical rejection.
   (h) **Physical witness scope:** one selected actual gateway call and pinned readback support their reported local facts; copied receipt, wrong node or third-party intervening writer cannot gain universal causal authority.
   (i) **Stale resource:** current browser owner adopts only permitted target content; retired callback cannot mutate a reused DOM ID even if HTTP 200 arrived.
   (j) **Notification and permission:** a durable advisory with qualified eventual delivery/replay may support the named progress claim; SSE arrival alone cannot grant remote Refresh.
   (k) **Tool failure:** a finished supported proof check can be distinguished from timeout, unsupported translation, empty finite Init or lost external checker; none is success by default.
   (l) **Counterexample provenance:** a finite model trace maps back to exact source and explicit open physical question; source-located report does not invent an actual XTDB incident.
   (m) **Correct source repair:** changed original guard or verified translator regenerates all dependencies and passes relevant checks; editing only the emitted model is insufficient.
   (n) **Physical repair:** an actually guarded deployed writer prevents the competing invalid effect; a post hoc ledger discrepancy cannot be called pre-effect prevention.

(o) **Product non-vacuity:** an eligible self-Claim and required Unclaim/Reassign/other Part VIII capabilities remain exercised; reject-all is not completeness.
(p) **Topology:** single-/multi-node deployments preserve the same abstract Claim meaning, with separate routing, credential, writer and failure premises; node loss is not an authority grant or unconditional convergence.

Each case states the original source, profile, positive control, adversarial mutation, formal abstraction and bound, first checker/physical enforcing boundary, and independent relevant observation. The report distinguishes prevention, scoped diagnostic detection, proof counterexample and unobserved uncertainty. An uninspected route or external database writer remains open, not conveniently absent.

The final invariant is that formal assurance **constrains the implementation Gesso actually installs and operates**. It cannot become an independent story that is true only of generated models, nor can a real passing test fabricate a theorem that has not been checked.

External formalization has a concrete first target

The first external theorem target is the **closed propositional proof kernel** and its acceptance relation, because its language and trusted judgments are small enough to specify independently of arbitrary Clojure, XTDB transactions, the browser and the full Choreo machine. The purpose is to remove a specific trust assumption from Part IX's proof issuer, **not** to assert that having a proof assistant has already proved the framework or every application using it.

Ordinary application authors define rules, model operations and Choreo constructions in Gesso's admitted Clojure/data interfaces. The framework derives the formal subjects and proof obligations. They do not maintain a second handwritten HumanHelp specification or write Lean tactics, Quint, TLA+ or Recife models to use an ordinary admitted operation. Framework maintainers may choose a proof assistant such as Lean, another proof-producing checker or a mechanized semantics with a suitably small trusted checker. The architectural requirement is the theorem and evidence-import contract, not a permanent commitment to one tool vendor or syntax.

Part IX supplies the original subject, typed proposition source, small proof rules, source-derived dependency closure, premise inheritance, checker admission and explicit open assumptions. Sections 10.01–10.04 specify formal/theorem versus runtime correspondence; 10.09 names the independent production discharge obligations. External formalization must preserve these boundaries rather than add an unrelated parallel assurance scheme. 10.11 owns exact-finite exploration/abstraction and 10.12 owns the broader reproducible evidence package.

1. Specify the closed inference system before proving it

   The initial formal calculus covers the same **seven** inference rules as the admitted Gesso propositional kernel, with their exact well-formedness, arity, ordering, reference and conclusion restrictions:

Admitted rule	Semantic consequence and required shape
`:premise`	A declared named premise P remains open and entails P
`:truth-introduction`	Truth is derivable without taking an unrecorded atom as true
`:implication-reflexivity`	For the same well-formed P, P implies P
`:conjunction-introduction`	From admitted P1 … Pn, n at least two, derive the matching conjunction in the selected source order
`:conjunction-elimination`	From a checked conjunction derive exactly its selected indexed conjunct
`:implication-elimination`	From P implies Q and P, in the required reference order, derive Q
`:disjunction-introduction`	From P derive a disjunction containing exactly that P at the declared supported position

   A calculus statement quantifies over well-formed finite formula trees, finite derivation sequences, all valid rule parameter values and all Boolean valuations of the distinct admitted **opaque atoms**. The atom declarations are inert for this propositional theorem: the theorem does **not** decide whether an embedded age, Membership or Request policy predicate is true of real application state. Formula syntax admits its defined truth connectives; the seven **inference rules** do not thereby acquire additional proof operations. Neither the presence of `:not` or `:false` in a proposition nor a valid semantic tautology licenses an unimplemented classical proof rule, arithmetic solver, arbitrary rewriting, atomic-truth axiom or hypothetical-context discharge.

   Let `Gamma` be the named open source premises, `D` a well-formed source derivation, `P` its exact goal and `v` a total Boolean valuation of the opaque atoms in the admitted formulas. The initial external soundness target is:

```
For every supported original source Gamma, D, P:
  If the defined small-kernel derivation relation accepts D
  from Gamma with conclusion P, then
```

```
for every Boolean valuation v:
  (every premise in Gamma that D actually depends on
   is true under v) implies P is true under v.
```

The antecedent must use the **defined inference rules**, not an opaque call to Gesso's own implementation checker—otherwise the proof could accidentally assume precisely the correctness it is meant to establish. The formal theorem states how used premises are computed by rule references and what happens to declarations not used by a particular step. The source admission path may additionally require all declared premises/steps to participate; that stricter input discipline must not be confused with a logical rule that discharges unused assumptions. A finite closed derivation with `Gamma` empty is unconditional **only for the admitted opaque-Boolean interpretation**.

Proofs may be vacuous under inconsistent premises; a valid logical entailment does not prove that the application can ever reach a state satisfying those premises. Nonempty initial state, reachable eligible operations, authenticated actor context and real product capability have different independent obligations. A theorem proving `Gamma | P=` cannot silently produce `P` unless `Gamma` is actually discharged or separately justified.

2. Five distinct theorem and implementation claims

   The formalization has at least five separately reported objectives. A later result may depend on earlier ones, but a single passing artifact does not collapse them.

   (a) **S0 — Inference-rule soundness.** Every admitted rule schema preserves Boolean entailment for all legal formulas, arities, indices and reference patterns. Formalize the rules independently and prove their preservation lemmas, including the premise rule's open-assumption effect.

   (b) **S1 — Derivation-system soundness.** Induct on the finite ordered derivation. Each source step inherits exactly the used named open premises of its valid predecessor steps, and the final step entails the declared goal. Forward references, cycles, wrong-conclusion substitution, undeclared premise use and unsupported rules are outside the admitted derivation relation rather than trusted exceptions. A mathematical induction proof is not evidence that a particular Clojure parser enforces it.

   (c) **S2 — Actual Clojure checker correspondence.** For all supported concrete inputs accepted by the **production** parser, normalizer, proof checker and current admission path, establish a semantics- preserving relation to the formal derivation relation of S1. Include original-source normalization, nominal variable/subject identity, premise and step references, source rejection, malformed encodings, resource limits and implementation/version binding. This can be a proven implementation/refinement, verified extraction, an independently checked translation or a separately qualified trust premise. Simply invoking the production checker from the formal model is not independence.

   (d) **S3 — Typed atomic-proposition correspondence.** Relate opaque Boolean atom valuations to the actual supported typed-rule evaluator under its original source, reader footprint, presence/nil semantics, runtime variable bindings and evaluation/error contract. The conditional propositional theorem cannot establish any arbitrary Clojure implementation predicate, model eligibility or current server input. A bounded typed-atom oracle establishes only its enumerated original bindings unless an additional abstraction/correspondence theorem justifies wider scope.

   (e) **S4 — Issuer/composition and source-to-production transfer.** Bind the exact source derivation and checked theorem to the trusted current source, original-source-required obligation ledger, certificate issuer, selected admission profile, premise inheritance, compatible consuming operation and installed execution boundaries. The actual server model guard, XTDB write, loaded browser asset or protected read needs its own qualification under 10.04–10.09. A correct calculus theorem does not mint runtime authorization or a durable transferable certificate.

   A final application claim must state exactly which S0–S4 edges were established and which remain conditional, tested, source-checked or unknown. It is legitimate to prove S0–S1 before S2–S4. That yields a useful formal result but **not** an externally proven Clojure checker, sound whole-application composition or formally verified HumanHelp deployment.

3. Checker semantics, resource bounds and fail-closed admission

   The actual acceptance relation includes more than the names of seven inference rules. It must define which original source forms are admissible: metadata- free/plain structures where required, qualified IDs, exact keys, typed nominal contexts, formula depth/arity, atom encoding, duplicate or unused steps, strictly prior references, all required dependency/assumption bindings and the exact goal.

   A bounded production checker can soundly reject inputs because of a configured depth, formula-node, proof-step, premise, atom or computation budget. Such conservative rejection does not refute **soundness** of what it does admit. Completeness—for example, admission of all otherwise valid derivations in the advertised supported/bounded domain—is a separate

usability/property target. The strong product acceptance must not be satisfied by a checker or constructed application that rejects every legitimate Claim.

A successful proof result, a genuine semantic counterexample, a malformed-source rejection, an unsupported rule/judgment, exhausted resource budget, external tool failure and unavailable checker are distinct dispositions. If a selected profile requires S2 or another external theorem at construction, an unavailable, timed-out or mismatched check fails that **new** admission. The framework may continue serving a previously admitted still-current assembly under its actual lifetime contract; it cannot silently downgrade the profile, invent a checked theorem or grant an unauthorized protected operation.

Preprocessing and normalization belong inside the correspondence boundary. An independently sound rule calculus cannot compensate for a normalizer that drops a premise, changes the formula tree, coalesces distinct typed atoms incorrectly or maps source proposition A to theorem subject B. Revalidating just the resulting normalized formula is not necessarily validation of the **original** source claim. A checked trusted source-to- normal-form semantics relation is needed.

4. External backend and theorem evidence import

   The external backend must produce evidence whose meaning is independent of source-level report labels. The evidence package identifies at least:

   - the immutable formal theorem statement, explicitly admitted axioms or trusted primitives, proposition and proof-rule definitions, theorem/checker version, backend/toolchain configuration and claim category;
   - the canonical original-source semantic subject, normalized representation, dependency/obligation closure, source-to-formal encoding and all versions required to interpret them;
   - a proof term or independently checkable result where the backend supports one, together with the exact trusted checker/admission mechanism;
   - finite-domain or generated conformance coverage, resource/execution budgets and any unsupported language features when these are supplementary **tests**, not the external universal theorem;
   - current exact issuer/consumer binding, selected semantic source, required open/used premises, installed asset/plan identity if the claim transfers to production, and all unresolved S2–S4 implementation/environment obligations;
   - a reproducible invocation and observed disposition with privacy-appropriate diagnostics, source correspondence and invalidation/rechecking triggers.

   A Lean proof might be one implementation of the S0–S1 evidence. A separate kernel/checker or supported proof-producing tactic may be used, but the trusted assumption set must state the actual proof-checking kernel and any imported axioms. A successful CLI exit code, copyable EDN report or saved ”proved” label is not itself trusted proof origin. A checker-based import should recheck the actual proof/evidence at the selected current consumer, or rely on a properly qualified persistent attestation/lifecycle contract rather than raw purported hashes.

   The external checker does not need physical XTDB/network/Chromium access to prove S0. That is precisely why the abstract theorem and its independently qualified runtime transfer are separate. Proof/diagnostic sidecars must not change an already admitted runtime state transition or become a principal, one-Refresh grant, command retry authorization or browser effect lease.

   The tool-independent exchange need not be a single universal serialization format, but every backend must preserve the exact canonical judgment and evidence class. A backend may reject unsupported constructs honestly; it cannot silently translate them to axioms, omit difficult cases or assert all application obligations discharged.

5. Independent oracles, mutations and common-mode bugs

   Gesso's own independent rule-interpretation and finite audits are valuable **falsifiers** of claimed correspondence. They can expose a changed inference index, missing premise, malformed step, normalized atom mismatch or deliberate invalid rule. Running the same Clojure normalizer in two wrappers does not independently validate the normalizer; an independently written reference can discover more bugs but requires its own domain/soundness qualifications.

   An exhaustive truth table over one finite set of opaque atoms proves only the particular local entailment within that declared domain. Checking one representative for each of the seven rule tags does not quantify over every formula tree, premise combination, conjunction arity, conjunct index, depth or malformed source form. A compositional source-level induction over a finite derivation is stronger for **that derivation** than checking only its final Boolean valuation sample, but its local rule and interpreter premises must remain explicit. Neither result automatically establishes the universal mathematical metatheorem or production Clojure implementation.

   A required acceptance corpus includes paired positive/adversarial controls:

   (a) A valid derivation of a source premise is accepted **with that premise open**; the same source with the premise omitted must not yield an unconditional theorem.

(b) Premise-free truth introduction and implication reflexivity are checked for arbitrary supported well-formed formulas; an invented arbitrary typed-atom truth axiom is rejected.
(c) Correct two-or-more conjunction introduction and indexed elimination preserve their source order; reversed premise order, invalid arity or out-of-range index is rejected.
(d) Implication elimination requires its exact implication and antecedent; swapped references or changed consequent fails the rule.
(e) Disjunction introduction requires the indexed embedded source conjunct; a different injected atom or unsupported position cannot be inferred.
(f) Valid references use previously established derivation steps; forward/circular references and fabricated prior-step conclusion are rejected.
(g) The complete original source and normalized formula denote the same supported proposition; a dropped premise, altered atom or forged typed context cannot inherit the proof.
(h) A finite independent truth valuation matches a supported source step; truncation or over-budget enumeration is incomplete, not proof of all valuations.
(i) Formal S0/S1 applies to every supported rule/formula shape under its premises; one checked instance or sample catalog is not promoted to universal soundness.
(j) The source-level Clojure checker accepts an admitted derivation under its exact grammar; a deliberately faulty parser or normalizer admits the wrong original subject and fails S2.
(k) The typed-atom bridge agrees over its qualified source-binding domain; an opaque Membership or Organization-policy atom is not marked universally true from finite examples.
(l) The trusted issuer rechecks current original-source obligations and the genuine proof; a copied successful report or stale assembly/certificate cannot mint a new current proof.
(m) An external proof artifact with exact statement, axioms, backend and checker is reverified in the declared admissible scope; changed axioms, malformed proof, unsupported syntax or exit-code-only success cannot create the claim.
(n) A valid abstract Claim proof may coexist with an independently qualified physical implementation witness; an unguarded actual XTDB writer is not made safe by the kernel theorem.
(o) Retired browser owner/resource cannot mutate a reused target; proof-term reuse or matching artifact ID is not effect-time permission.
(p) Genuinely eligible HumanHelp Claim and the other independently required lifecycle/product positives remain executable; a reject-all source machine is not accepted as a completed product.

Each case names the original source, selected proof checker and quantified domain, exact positive control and mutant, first rejecting or enforcing boundary, actual evidence class, remaining premises and independently observed effect where physical correctness is claimed. A theorem that is still open or a tool run that was not performed must be reported as such.

6. Order later formal targets by the trust gap they eliminate

   After the propositional theorem and its concrete checker-binding work, expand only where a specific original-source judgment and transfer edge can be defined. Appropriate later subjects include:

   - typed-rule evaluation and semantic preservation of proposition normalization/atom meaning;
   - guarded-transition consequence and simultaneous assignment/ presence-aware frame rules;
   - source-complete conditional induction and exact certificate premise substitution;
   - role-local knowledge/provenance and protected authority non-escalation;
   - conservative progression/ observation algebra and requirement discharge;
   - global-to-local Choreo projection and portable executable-plan lowering;
   - browser adapter/resource refinement and selected physically enforced authorization/XTDB transitions.

   Theorem selection is driven by the original operation language, the size of the remaining trusted base, independent counteroracles and real correspondence needs. A formal theorem for one closed calculus cannot automatically cover arbitrary host-language conditionals or all multi-node schedules. The aspiration is high assurance of the original source and deployed application, without forcing developers to author formal syntax or pretending the full general-purpose Clojure runtime has been proved.

Finite exploration and abstraction

Finite exploration can expose a genuine semantic counterexample, establish a property **exactly for one complete finite model**, or supply a bounded conformance check of an implementation. These are three different evidence classes. To transfer a finite-model theorem beyond its enumerated domain, Gesso must provide a separately justified abstraction/refinement relation from the original admitted source to that model, and a further implementation relation for the actual production execution if the claim concerns XTDB, transport or the browser. A finite test does not become an unbounded proof because the software is deterministic or its sampled observations match.

The finite subject is derived from the canonical Gesso model, Choreo or browser-adapter semantics admitted under Parts I–IX. It retains exact source-generation, operation/initialization and invariant obligations, policy choices, source-derived reader/guard/frame semantics and open premises. Developers should not have to construct a second handwritten formal app with rules that merely resemble the runtime. An intentionally simplified independent test model may be useful, but its results are explicitly test-relative until the original-source correspondence is justified.

1. Exact finite claims: closure, totality and vacuity

   An exact-finite result records the following as the **complete, closed model under examination**:

Required element	Exact claim boundary
Initial states	Original admitted initial-condition/initializer relation, all selected starting states and their eligibility/inhabitatio
State and input domains	Each field's type, absence/present-nil interpretation, enum values and representative or exhaustive status, includ
Operations	Independently source-derived complete original operation IDs, guards, typed inputs, simultaneous writes, frame
Transition semantics	Admitted enabled steps and successor relation, stuttering, scheduling, terminal/deadlock treatment and whether
Property	Exact invariant, deadlock-freedom, eventuality or trace obligation, its quantifiers, scope and any fairness or envi
Exploration	Reachable-state/edge coverage, generated-vs-pruned states, cutoff, visited-state equivalence, trace/history handl
Evidence	Checker/translator version, canonical input, actual outcome/counterexample, original source location, independ

   An exhaustive result requires the whole reachable graph of **that** finite model under its declared transition relation, not merely all states up to a search depth or all choices returned by an unverified event generator. A BFS/DFS exploration may use state deduplication only when its equality or quotient is sound for the specific state/trace property. In particular, merging distinct histories because their current map happens to match can discard command-history, fairness, previous assignment or in-flight physical callback distinctions relevant to the theorem.

   A depth limit, state cap, edge cap, missing choice, unexpected exception, unsupported operation, unavailable dependency or timeout makes the selected exhaustiveness claim **incomplete**. It does not become a proof of absence of counterexamples. An actual reproduced counterexample to an adequately specified finite model can still be reported even if exploration did not finish; it must carry the actual trace and scope, not a fabricated ”complete” status. A potential deadlock is distinguished from a declared terminal state, a blocked environmental step and a liveness violation. An empty initialization set makes a universal invariant trivially true; that is not sufficient positive evidence that the application works.

   For source-relative induction, the finite oracle can enumerate selected prestates and operation inputs and check base and step preservation without exploring only reachable states. It still must name whether each domain contains **all** values of the declared type or only representatives. Enums and Booleans may have complete finite domains under the given versioned interface; integers, strings, UUIDs, timestamps, dependency depths and arbitrary sequences usually do not. Checking all input combinations over a representative set is exact for **that set**, not proof that no omitted value changes an operator's semantics. A finite model with every guarded operation disabled can look safe while missing required positive Claim, Unclaim or manager Reassign behavior.

2. Safety transfer requires the right simulation relation

   Let C denote the original/concrete semantic transition system, A the finite abstract system, R a relation from relevant C states to A states, and P_C/P_A the corresponding safety predicates. The intended **concrete-to-abstract** safety transfer needs:

```
Initial coverage:
  for every admitted concrete initial state c0,
    there exists abstract initial a0 with R(c0, a0).

Step simulation:
  whenever R(c, a) and a relevant concrete step c --e--> c',
    there is a permitted finite abstract trace a ==e==> a'
    (possibly with justified hidden/stuttering steps),
    with R(c', a') and preserved required observations.

Property reflection:
  for every related (c, a), P_A(a) implies P_C(c).

Then an A invariant over all its reachable states transfers to
related C reachable states, under the stated relation and premises.
```

The quantifiers and direction matter. Showing that each abstract step can be mimicked by one convenient concrete execution does **not** establish that every concrete execution is represented. Likewise, mapping each concrete state to some abstract state without checking its successors does not establish safety transfer. A proper abstraction may **overapproximate** concrete behavior and therefore admit spurious abstract counterexamples. A finite abstract violation is an actual failure of the chosen abstract property, but need not be realizable in the original semantics or in a deployed browser/XTDB execution; concretization and relation inspection decide the stronger attribution.

For **production** safety, a second relation must qualify the actual installed code, original source version, authorized backend prestate, all relevant writers, transport/DOM effects and failure semantics. An abstract source theorem does not authenticate the route, model caller, physical transaction or browser mutation. A single real-world event can correspond to multiple abstract steps, but that stuttering/partial-order permission must not hide an unauthorized write, stale DOM adoption or additional active primary assignment.

3. A bounded slot is not a bounded history

   Logical browser active/queued slots, request concurrency and selected finite state projections do not bound command identifiers, resource or effect instance IDs, history length, attempt count, provider bases, user inputs, dependency depth, retired metadata, in-flight network requests or physical DOM side effects. Gesso must either prove a state/data bound **under actual scheduling and retention premises** or choose a sound abstraction for each unbounded dimension; it cannot infer finite-state completeness from one bounded queue setting.

   Equality-class or symmetry reduction of opaque IDs is often preferable to arbitrarily choosing UUID 1–4. The abstraction must retain every equality and inequality relation relevant to guards, callback ownership, command replay, identity freshness, reused DOM IDs and noninterference between owners. An old resource A and active B may be represented by classes `:old` and `:current`, but two outstanding callbacks to **different old identities** cannot always be merged without proving the property ignores their difference. Fresh allocation can change the class partition. Recycling an old ID or aliasing two logical commands changes the transition relation unless ruled out by a source/protocol premise.

   An unbounded collection may admit a cutoff or data-independence proof, but that theorem must justify the cutoff for the particular property and all supported transitions. A finite representative Request/RequestAssignment model with two helpers can expose a one-primary race; it does not prove arbitrary number of helpers or all admin/migration writers without the required symmetry, cardinality, operation-catalog and physical-refinement arguments.

4. Resource ownership: a local abstraction with an explicit limit

   The following executable example isolates one ownership predicate. It uses opaque resource and effect identities and an active phase. Identity equality and phase are checked again at the point of mutation. It is not the complete Part VI adapter: execution ownership, plan compatibility, authority scope, observation coverage, and batch completion remain additional conditions there.

```
(defn owned-effect? [state callback]
  (and (= :active (:resource-phase state))
       (some? (:resource-instance state))
       (= (:resource-instance state) (:resource-instance callback))
       (some? (:effect-id state))
       (= (:effect-id state) (:effect-id callback))
       (= :awaiting-effect (:effect-phase state))))

(defn ownership-observation [state callback]
  {:active? (= :active (:resource-phase state))
   :same-resource? (and (some? (:resource-instance state))
                        (= (:resource-instance state)
                           (:resource-instance callback)))
   :same-effect? (and (some? (:effect-id state))
                      (= (:effect-id state) (:effect-id callback)))
   :awaiting? (= :awaiting-effect (:effect-phase state))})

(defn abstract-owned-effect? [observation]
  (and (:active? observation)
       (:same-resource? observation)
       (:same-effect? observation)
       (:awaiting? observation)))
```

```clojure
(def resource-a
  {:resource-phase :active
   :resource-instance :fixture/resource-a
   :effect-id :fixture/effect-a
   :effect-phase :awaiting-effect})

(def callback-a
  {:resource-instance :fixture/resource-a
   :effect-id :fixture/effect-a})

(def resource-b
  (assoc resource-a
         :resource-instance :fixture/resource-b
         :effect-id :fixture/effect-b))

(assert (owned-effect? resource-a callback-a))
(assert (not (owned-effect? resource-b callback-a)))
(assert (not (owned-effect? (assoc resource-a :resource-phase :retired)
                            callback-a)))
(assert (not (owned-effect? (assoc resource-a :effect-phase :completed)
                            callback-a)))
(assert (= (owned-effect? resource-b callback-a)
           (abstract-owned-effect?
            (ownership-observation resource-b callback-a))))
```

The four Boolean observations support a local predicate comparison. They are not by themselves a finite transition system. A finite abstraction for the whole adapter must also specify allocation, identity freshness, replacement, retirement, queues, and adversarial late events while preserving the necessary equality relationships. It must account for two callbacks that alias the same identity and callbacks that do not. Collapsing all old identities into one name without preserving the relevant relations can change the modeled behavior.

The final-mutation requirement follows even before a universal abstraction proof exists: callback A may pass its first check, be deferred, and arrive at mutation after resource B has replaced A. The shell must evaluate ownership against B at that moment. Retaining the earlier Boolean `true` is not permission. The same distinction applies when a completed effect has consumed its phase but a duplicate callback arrives.

5. From four Boolean observations to a complete browser machine

   The preserved Clojure example above has four **local predicate** observations: active resource, equal resource instance, equal effect ID and awaiting-effect phase. They demonstrate the Boolean equivalence of `owned-effect?` and `abstract-owned-effect?` for the specific constructed inputs; they do not enumerate all reachable adapter states or prove the concrete-to-abstract relation. In particular the four Booleans do not encode how an execution acquires ownership, what happens when targets are replaced, how effects are scheduled, or whether the shell rechecks permission at the physical mutation.

   A full finite adapter subject would also model, as needed by the selected claim:

   - creation/allocation and freshness of resource, execution and effect identities; permitted alias relationships and changes across target reuse, plan replacement and owner retirement;
   - normalized callback delivery, duplicates, delay, loss, cancellation failure and physical requests remaining after semantic retirement;
   - current plan and application generation, installed asset/route and target compatibility, active/queued logical slots, effect phases and bounded or abstracted retained metadata;
   - authoritative observation frontiers, provisional/confirmed command ownership, permission to acquire, and the different physical events for transport completion versus DOM install;
   - multi-target HTMX/OOB partial completion, repair, abort, continuity ownership and current effect-time revalidation;
   - environment/adversary choices that must be included to warrant a universal claim over the declared scheduling scope.

   The finite model should retain a stable **logical** command C42 independently of physical attempts H1/H2 and current resource A/B. Conflating those identities could make a late response look like a valid result of a new command. An optimistic current card is also not an authenticated statement of C42's durable incorporation. The transition relation must preserve these distinctions rather than assuming every successful HTTP response is a current authorized DOM adoption.

The physical browser requirement can be falsified by a precise interleaving:

```
A is the current resource; effect F from A is pending.
A callback for F arrives and passes callback-entry checks.
The callback is deferred before its managed DOM mutation.
A retires; new resource B adopts the same target DOM ID.
The deferred A callback resumes.
At the physical mutation boundary A no longer owns the target.
The shell must reject A's mutation even if its earlier check passed.
```

The originally preserved predicate rejects this case **only when evaluated against the current B-owned state at the mutation point**. A cached earlier `true` is not a permission lease. The browser adapter can model that admission; a theorem about the real shell additionally needs actual code/HTMX/DOM effect-time correspondence, physical scheduling and installed artifact currentness. This also applies to a completed effect that receives a duplicate callback after its awaiting phase is consumed. For a batch, successful installation of one target does not prove every other target adopted.

6. Liveness needs divergence, fairness and permission premises

   Finite safety and finite liveness require different claims. An abstract safety simulation can hide arbitrarily many internal steps while preserving protected invariants. A progress/termination theorem additionally needs to show that the relation does not admit an infinite concrete divergence that the abstract model silently skips. Fairness assumptions must refer to the actual relevant scheduler, participant, provider, delivery or recovery mechanism; a bounded test depth does not establish them.

   A fairness-sensitive model must identify which actions are enabled, which remain continuously or repeatedly enabled, what strong/weak fairness applies and whether environment or user actions are assumed. The declared property is specific—for example, a retained qualified command result becomes observable when authorized recovery is repeatedly attempted under sufficient provider availability. That is not the same as claiming every network partition heals, every user presses Refresh or every provisional UI state settles automatically. Under HumanHelp's selected remote-advisory policy, a remote board's actual acquisition is conditional on **user-granted** scoped Refresh; SSE delivery does not authorize background collection reading.

   Finite cycle detection can identify a lasso violating an explicitly encoded fairness/liveness obligation in the selected model. If the model is cut at depth N, a finite sequence with no terminal state is not in itself proof of infinite divergence. Conversely, a reachable terminal path does not prove all fair runs eventually reach it. For a concrete liveness transfer the abstraction must preserve applicable fairness, enabledness and non-divergence, not only a Boolean ownership invariant.

7. Evidence categories and reproducible adversarial acceptance

   An assurance report should explicitly identify: the original source and selected abstraction generation, the exact finite domains and whether they are exhaustive, initial and transition catalogs, property/temporal meaning, complete-vs-bounded exploration status, checker/tool options, evidence origin, reachability/ deadlock/fairness interpretation, original source locations of violations and open simulation, concretization and production-enforcement premises. A copied finite result is not issuer-controlled proof or a current deployment authorization. Section 10.12 owns the reusable evidence-package lifecycle.

   The following paired checks distinguish the strength of each conclusion:

   (a) A complete declared Boolean/enum domain is explored; one omitted enum variant invalidates exhaustive coverage.
   (b) Nonempty valid initial states and a genuinely enabled positive Claim exist; empty Init or reject-all operations cannot pass whole-product acceptance.
   (c) Every original operation and selected environment step appears; removing Unclaim, Reassign or a relevant competing writer invalidates the corresponding source/physical completeness claim.
   (d) All reachable states/edges in a finite model are exhausted; a depth/node/edge cap is labelled incomplete rather than counterexample-free.
   (e) Source base/step enumeration respects absent versus present nil and simultaneous writes; conflation or sequential assignment produces a discriminating independent-oracle mismatch.
   (f) A valid symmetry/equality abstraction preserves current-vs-old callback relationships; collapsing distinct alias classes or fresh allocation incorrectly changes an observable outcome.
   (g) A complete source/abstract initial and transition simulation implies the named concrete safety property; state mapping without step simulation is rejected.
   (h) A deliberate abstract overapproximation produces a scoped counterexample; it cannot be reported as a physical XTDB or Chromium incident without concretization.

(i) Competing helper C42/C43 commands are represented distinctly and at most one primary can incorporate under the claimed atomic model; merged command IDs falsely erase the race.
(j) A valid A callback mutates a current target when fully authorized; a deferred A callback after B replaces it must fail at **effect time**.
(k) Duplicate callback and completed effect cannot reuse a consumed permission; a cached initial `true` cannot grant a second DOM mutation.
(l) Real multi-target partial installation retains incomplete/repairable outcome; one target HTTP 200 cannot qualify full-batch authoritative adoption.
(m) A finite lasso under justified fairness can refute selected progress; a depth-limited prefix is not an infinite liveness counterexample.
(n) A fair authorized recovery is conditional on retention, availability and appropriate user action; it cannot silently force remote Refresh or assume network healing.
(o) A finite source-level transition result is reported as bounded; copied diagnostic or controlled physical observation cannot be promoted to original-source theorem or actual host authority.
(p) A selected abstraction backed by a checked step/property bridge can justify the corresponding quantified safety domain; a finite sample without the bridge retains its finite-only evidence status.
(q) Real browser execution agrees on the exercised supported resource/effect mutation case; a CLJC adapter-model pass does not itself prove that Chromium and HTMX executed the same physical schedule.
(r) Positive HumanHelp creator-local and explicitly user-authorized remote adoption remain possible; a model that hides these required cases is not a complete application witness.

Each case specifies source identities, finite/abstract universe, exact bounds and declared fairness, positive execution, adversarial change, first checking or physical boundary, independent observable result and still-open assumptions. Correct source rejection is not evidence that an unmediated physical writer was prevented. A finite checker timeout remains incomplete; an unknown physical outcome remains unknown; neither may be marked as an exhaustive safe run.

The final standard is **finite proof with honest scope** and **generalization only through a justified relation**. A small abstract state machine is valuable when it covers the right behaviors, not because its state count is small.

Reproducible formal evidence and maintenance

A formal or source-checked result is usable only when its **exact claim, scope, origin, assumptions, inputs and applicability** can be reconstructed. The evidence package is a versioned, inspectable record of what was actually checked, not an independent source of application semantics, an executable permission, or a transferable certificate merely because it contains a fingerprint or successful status. Gesso derives it from the original admitted source and trusted checker/issuer boundary; application developers do not hand-maintain parallel formal declarations.

Two enduring questions must be answerable separately: **Is this theorem still valid for the same semantic subject under the same premises?** and *Does the currently installed application still satisfy the correspondence and environment premises needed to use it?* A source-level proof can survive a route replacement while its production applicability fails. A physical observation can become stale without disproving the underlying theorem. No evidence record, regeneration operation or failed tool run silently upgrades one kind of claim to another.

Part IX governs the actual proof judgments, original-source requirement closure, premise substitution and issuer-controlled admission. Sections 10.01–10.11 identify the formal theorem, physical realization and exact-finite abstraction obligations. This section defines their reproducibility, dependency/revocation and deployment-applicability lifecycle; 10.13 separately owns trust-base and backend-assumption qualification details.

1. Evidence package: identify the proposition and the actual result

   An evidence package has a closed, versioned schema appropriate to its selected claim category. For every applicable entry it records:

Element	Required binding and interpretation
Claim	Exact proposition/temporal property, quantifiers, domain, observation alphabet, selected assurance profile and e
Original source	Actual admitted model/rule/operation/Choreo subject, normalization and identity versions, complete original op
Formal subject	Generated translation/extraction version, normalized definitions, declared axioms, explicit abstraction and proof
Method	Kernel/derivation checker, model exploration, independent oracle, external proof system, controlled physical obs
Reproducibility	Exact checker/translator/toolchain/dependency versions, meaningful flags, solver options, bounds, fairness, seed
Result	Observed success, counterexample, rejection, inconclusive, unsupported, timeout, environment failure or not-ru
Premises	Used and still-open assumptions, trusted external contracts, discharged source obligations, provenance of discha
Dependency closure	Exact semantic/interface and proof dependencies, checker/issuer dependencies, currentness and invalidation rela

Element	Required binding and interpretation
Production applicability	Installed callable, route/worker, model, assets, runtime/resource generation, backend/credential domain and actu
Evidence custody	Trusted issuer/checker origin, how inputs/results were acquired, immutable artifact identity, validation/import r
Recheck lifecycle	Owner, triggering changes/contradictions, failed or stale disposition, dependent claim invalidation and permitte

A theorem proof and a model-checking run have different semantics even when both finish successfully. An external backend result that exhausted its state space, a finite typed-rule sample, a source-rechecked kernel derivation, a process-local transaction witness and an observed Chromium DOM effect each establish only their own named claim. A report must not compress their distinct conclusions into a common `:certified?` boolean or make several incomplete witnesses add up automatically to universal soundness.

For exhaustive finite evidence, include the **entire declared finite machine**: types and optional presence, initial states, complete operation/environment relation, enabledness, abstract identity quotient, state/edge bounds, reachability and deadlock treatment. Record whether actual exploration was complete. For unbounded claims, include the separately checked abstraction, simulation, induction or external metatheorem required for that conclusion. Timeout, budget exhaustion, failed external checker and an unrun gate do not mean "no counterexample." An independently observed model counterexample can remain a valid **scoped failure** even if wider exploration terminates early.

2. Reproducibility means rechecking the right original source

   A replay starts from the authenticated original semantic subject and its required proof/profile obligations, regenerates or independently validates the formal subject, and checks the exact claim under the recorded definitions and assumptions. It does not trust a cached positive report as input to its own validation. Canonicalization must represent all meaning-changing parts: type, field presence, ordered expression and proof-step operands, nominal atom/role/operation IDs, interface, dependencies, abstraction, admitted primitives, relevant versions and assumptions.

   Structural identity is not denotational equivalence. A hash of a normalized EDN interface establishes an identity for those canonical bytes under the stated encoder and collision assumptions; it does not establish that an opaque Clojure function implements the interface, that an XTDB node executed a particular transaction, or that the served browser bytes came from the claimed build. Conversely, an irrelevant comment, formatter change or metadata field omitted under a **proved/defined canonicalization contract** need not invalidate an unchanged semantic theorem. The consumer must not choose arbitrary fields to omit merely to preserve a stale cache hit.

   The replay procedure must distinguish three reproducibility targets:

   (a) **Semantic replay:** same admitted source meaning, exact proof statement, valid abstract evidence and open premises, even when nonsemantic build details change.
   (b) **Artifact replay:** deterministic or explicitly qualified reproducible generation of formal/compiled artifacts, with relevant compiler, codec, dependency, target/runtime, optimization and asset-manifest inputs.
   (c) **Physical replay/observation:** a newly acquired, qualified observation at the actual running model, backend, authenticated handler/worker and browser effect boundary. This need not reproduce the same physical history; it establishes only the stated new physical applicability/behavior.

   A bitwise identical formal artifact need not result from every valid rerun if the selected backend has nondeterministic output. In that case a small trusted checker may validate the same theorem independently and the package records the actual seed, ordering, checker version and evidence identity. If the claim requires bitwise reproducibility of emitted assets or provenance of the exact deployed executable, then nondeterministic rebuilding is not an excuse to accept different unverified bytes. A stored digest is useful as an integrity index only when the artifact was acquired from a trusted source and the expected digest is authenticated at the consuming boundary.

   External theorem-checking code, formal definitions, axioms, imports, normalization, backend version and proof rules are part of the relevant trusted base unless independently justified. A successful CLI exit status is not by itself a proof. A generated checker result may carry a checked proof term, a genuinely re-executed finite trace or a named conditional provider assumption, but the package must say which.

3. Evidence origin, portability and proof-authority boundaries

   Gesso distinguishes **descriptions** of evidence from **issuer-controlled admission** of evidence. Ordinary maps, saved logs, Git objects, labels, summaries, copies of proof reports and matching fingerprints remain descriptive unless a trusted current checker/consumer validates their actual contents and binding. A process-local, registry-backed proof handle can be authoritative for the particular source-relative local check under its trusted-JVM threat model; serializing its printed representation does not make it a portable certificate or cross-process security token.

A durable proof artifact may be retained, moved between processes or reused across releases only under a separately specified authenticated revalidation, signed-attestation or trusted publication/import contract. If signatures are used, the key, signer, trust/revocation, validity scope, payload schema and bytes covered must be defined. Signing that a map was written does **not** prove its theorem is valid, its model source is current or its claimed backend execution occurred. Rechecking a sound proof term against current definitions can be stronger than trusting the signature of a fallible checker, but only within the actually established trust base.

A qualified theorem can be imported without rerunning its expensive derivation for every HTTP request when the exact immutable premise and dependency lifecycle supports reuse. Runtime authorization, actor/tenant membership, command replay/retention, protected reads, transaction prestate and effect-time browser ownership are **not** permanently discharged by that reuse. Logical premise substitution is also not a way to clone a one-Refresh user grant, one-shot transaction attempt, resource lease or actual principal capability.

Source- and production-based reports should be inspectable without publishing protected raw Request, Membership, Organization, command ledger, SQL parameters, document contents, session credentials or private browser state. The evidence store can retain authorized access-controlled artifacts while a copyable diagnostic exposes redacted subject handles, claim categories, bounded counters, stable failure reasons and permitted source spans. A privacy-safe redaction must not change the semantics of a supposedly exact checked subject; the trusted checker uses the protected original and the viewer receives a separate nonauthoritative explanation.

4. Dependency-aware validity, applicability and revocation

   The dependency graph separates semantic claims, checked proof rules, conditional assumptions and **physical applicability** rather than containing one undifferentiated whole-release checksum.

Dependency change or observation	Appropriate consequence
Meaning-changing original rule, model interface, operation set, initializer, invariant or proposition	Invalidate affected original-s
Checker rule, parser, normalizer, proof kernel, formal axioms or translation semantics changes	Revalidate/reprove affected c
Codec, target platform, compiler/lowering or generated executable changes	Requalify dependent executa
Installed route, callable/Var, worker, browser asset, resource generation or backend client changes	Reestablish current deploym
Backend version/configuration/isolation, writer credentials, topology or external provider contract changes	Requalify the affected extern
Accepted trusted assumption is revoked, contradicted or expires	Retract the dependent claim'
New observed counterexample contradicts a claimed formal theorem or qualified implementation relation	Classify and investigate the s
No relevant dependency changes under verified canonicalization and lifetime	Reuse only the unaffected cla

   Dependencies include the actual **original-source-required** obligations, not merely a structurally consistent user-submitted proof DAG. The graph must distinguish a theorem's used named premises, an application's required but still open capability/acceptance conditions, and the provider facts whose continued truth the consumer needs. If a provider's proposition only implies a narrower consumer assumption, the substitution is checked directionally and the provider's still-open premises propagate.

   Revocation and replacement operate on applicability, not history revision. A once-correct theorem does not become a false historical claim simply because the deployed XTDB version or Biff handler changed. Its **use** for that current system is suspended until the new correspondence is qualified. If the theorem's own proof rule or source meaning was invalid, the theorem's evidence must instead be retracted or rechecked at that level, with dependent results appropriately invalidated. A stored C42 command's actual committed outcome likewise is historical application data; invalidating a proof cache does not rewrite it.

   A compatible semantic interface may allow a pure proof to be reused through a checked directional equivalence/refinement bridge, including its assumptions. A mere matching friendly operation name, same shape, old certificate, copied fingerprint or source file path does not supply such a bridge. Changed opaque implementation can leave a source proof intact while its implementation obligation reopens. For source or runtime changes outside a theorem's proven dependency closure, avoid needless rechecking when that lack of dependency is actually justified, not simply inferred from a filename or build cache.

5. Admission, deployment and failure-aware requalification

   For a selected strong construction profile, required evidence must be present, correctly issued, current and within its qualified scope **before** the protected application/operation is admitted or an effect requiring it is permitted. Unavailable external prover, unsupported subject, invalid import, lost proof artifact, stale premise, failed backend qualification or incomplete evidence yields a typed failure/open status; it does not silently downgrade to an opaque rule, mark an assumption discharged or issue a certificate from cached descriptive metadata.

An outage of the external theorem tool need not retroactively invalidate a still-current independently checked immutable theorem under an explicitly defined lifecycle. It can prevent **new** strong-profile admissions and required rechecks. Whether a running deployment may continue serving depends on its own previously admitted generation, currentness, assumed environment and runtime enforcement policy. A change that revokes a needed current model/server authorization still prevents the relevant protected effect at the actual effect boundary, regardless of theorem-cache health.

A deployment rollout can temporarily serve different generations or route a participant to a host with different assets. Each claim identifies the actual process/host and loaded artifact generation for which it applies, or a proven compatibility relation covering the mixed versions. A prior successful startup check does not guarantee subsequent route/worker replacement, remote-node credentials, browser cache or direct XTDB writer remains constrained. Multi-node deployments use the **same** application semantics as one node, with additional physical topology, routing, writer, outbox replay, registration-frontier and failover premises rather than a new proof language.

A maintenance operation may be explicitly privileged, but it must state its protected effects, source/operational contract, effect on invariants and the requalification required before stronger dependent evidence resumes. Migrations and initializers are not outside the model solely because they do not arrive through HTTP. An incomplete/unknown physical writer census cannot be replaced by a green source proof report.

For each failed/retracted claim, preserve a useful typed diagnostic: the specific first blocker, original source/proof subject, evidence class, premise or physical boundary owner, current versus stale generations, safe reproduction instructions and an actionable requalification requirement. Do not turn a timeout into an observed counterexample, a failed post-submit read into a known rejection, or a post-effect mismatch into prevention of an already occurred effect.

6. Paired acceptance for reproducibility, reuse and staleness

   The following controls must be fixed before the acceptance campaign, including original source, actual evidence issuer, selected profile, positive case, adversarial mutation, first enforcing boundary and independently observable result where a physical effect is claimed:

   (a) The same canonical semantic source replays to a qualified exact theorem; a changed original guard cannot reuse the old semantic cache entry.
   (b) Source reordering that is explicitly nonsemantic retains defined canonical identity; reordering meaningful formula operands or proof steps must change the affected subject.
   (c) A real independent checker validates an admitted proof under its actual stated axioms; a “proved” flag or exit-code-only copy is rejected.
   (d) A complete finite-domain oracle records true exhaustive coverage; a timed-out, truncated or unsupported run remains incomplete, not counterexample-free.
   (e) A valid source-relative certificate can be reused while source and graph remain current; copied EDN, forged process-local handle and stale dependency graph cannot gain admission.
   (f) A declared assumption that is still needed propagates to the consumer; an apparently matching provider report cannot silently discharge it.
   (g) Revoking a qualified provider or current backend premise blocks dependent production applicability; an unaffected pure theorem need not be declared mathematically false.
   (h) A changed checker rule or normalizer forces affected independent proof validation even when the generated theorem bytes happen to be identical.
   (i) A changed Biff handler/worker or loaded browser asset reopens its physical binding; a source proof remains scoped to its actual original semantic subject.
   (j) A controlled installed gateway/XTDB observation supports only that acquired call/result/readback scope; copied node identity or matching ledger cannot prove full write causality and all writers.
   (k) A new legitimate helper with current Membership may Claim under the actual public model; an old proof artifact cannot authorize a revoked helper.
   (l) A compatible H2 replay preserves durable historical C42 incorporation; re-running the formal checker does not grant a fresh command write or rewrite a known outcome.
   (m) An authorized browser resource may perform its currently permitted DOM effect; an old certificate or resource-instance ID cannot grant an effect after owner retirement.
   (n) A staged multi-node rollout uses qualified exact host/asset/provider identities; merely matching the source hash on incompatible served executable generations is insufficient.
   (o) A redacted public diagnostic exposes the correct claim and first blocker without private Request/command data; a raw credential or sensitive trace is not published as evidence.
   (p) An actual semantic contradiction invalidates relevant affected claims under the chosen owner; a different external solver outage is classified as unavailable, not a falsified theorem.
   (q) A changed migration or direct external writer is either formally/physically included, constrained or retained as open;

a returned empty inventory does not establish complete coverage.

(r) Eligible Claim, Unclaim, Reassign and selective user-authorized Refresh remain independent product positives; a fail-closed proof gate cannot claim whole-app acceptance by rejecting all operations.

Each case records whether evidence was actually obtained, whether the result proves a finite/abstract theorem or only selected physical behavior, all remaining premises and its exact invalidation conditions. A copied report, stale fingerprint or successful explanation is never a permission to commit an XTDB transaction, disclose a protected document, replay a command or mutate a retired DOM target.

The maintenance invariant is **exact proposition, honest evidence, current applicability**. Changed systems should not invalidate everything indiscriminately, but no change that affects a claim's meaning or enforcing boundary may be hidden behind a convenient cache hit.

Trusted computing base, security boundaries, and failure modes

Every claim identifies its trusted base. The logical trusted base, physical execution dependencies, security boundary, and availability premises overlap but are not identical. Listing every library without indicating which claim depends on it is insufficient.

The assurance ledger

The root ledger identifies the executable semantic core, proof kernel, semantic checkers, normalizers, elaborators, lowerings, runtime gates, build process, external checkers, and implementation or dependency versions in scope. It joins these to original obligations, accepted evidence, application assertions, external assumptions, and enforcing boundaries.

For a physical claim, include the actual serving process's assembly identity, installed handler binding, artifact manifest, backend connection identity, and relevant runtime configuration under their stated trust contracts. An identity is recorded only where it has been acquired or verified appropriately. A value copied from caller metadata is labeled as claimed, not observed.

The ledger exposes which claims are conditional, unsupported, stale, refuted, or blocked by profile requirements. It preserves the reason and first boundary of failure without leaking protected inputs, command records, document contents, credentials, or raw provider exceptions into public diagnostics.

Source size and namespace inventories help focus review. They are not proof of soundness. A small explicitly described kernel is valuable because it reduces the surface requiring trust and analysis, not because short code is inherently correct.

1. Claim-indexed trusted base and independent observation

   The root ledger is a dependency view over individual exported guarantees, not one repository-wide ”verified” flag. Each entry names its original semantic subject and source obligation, precise proposition, checker or physical witness, version and issuer, open premises, affected application operations, enforcing boundary and currently applicable deployment scope. An inventory that cannot enumerate its claimed original-source or physical writer universe reports an open completeness obligation, not an empty set.

   The ledger makes five different categories of dependency inspectable:

   - **Logical**: executable semantics, proof rules, source-to-obligation derivation, elaborator, normalizer, proof checker and independent verifier.
   - **Execution**: the actually issued executor, lowering, installed handler, browser adapter/effect interpreter, build and loaded output correspondence.
   - **Security**: principal authentication and session binding, protected read/write/subscription admission, disclosure, credentials and capability origin where relied upon.
   - **Environment**: backend isolation, atomicity, read consistency, code execution, transport, process failure, persistence, clocks and any fairness/liveness assumptions used by that particular theorem.
   - **Evidence**: proof object or checked derivation, exact finite exploration, installed-assembly check, runtime enforcement, independently observed physical result, test, application assertion or still-trusted premise.

   Reports distinguish `machine-proved`, `kernel-checked`, `exact-finite checked`, `assembly-verified`, `runtime-enforced`, `tested`, `asserted` and `trusted` rather than upgrading them by aggregation. They also expose `unknown`, `unsupported`, `incomplete`, `stale` and `refuted` at the relevant scope. A source fingerprint or serialized assurance report has no authority to mint its own proof. Conditional root guarantees retain every relevant unproved premise transitively; one central opaque premise may outweigh many checked derivations. Evidence-class counts and namespace sizes are review aids, not a confidence score or theorem.

2. External-boundary inventory assigned to actual claims

   The following are candidate trust premises, not blanket assumed truths. For each applicable family the ledger names the exact behavior depended upon, version/configuration, checking or observation method and owner:

Boundary family	Behavior the particular claim may rely on
JVM, Clojure and third-party libraries	Primitive values, evaluation, concurrency, exception and memory behavior
CLJS, JavaScript engine and browser	Portable values, DOM and effect timing, resource lifecycle and stale-callback rejection
HTMX, SSE, Aleph/Ring and network	Actual route and response emission, session/request origin, reordering, loss, duplication and rec
Authentication and protected disclosure	Trusted principal binding, command arguments, authorization and guarded read/subscription re
XTDB and production adapter	Transaction guards, conflicts, atomic write footprints, query basis, codecs and provider time
Host, OS, storage and clocks	Persistent state, restart, failure, process replacement, delivery and time/fairness assumptions
External proof tools	Checker/kernel, input language, proof import, axioms, options and sound interpretation
Semantic/build pipeline	Normalization, compilation, lowering, asset manifest, emitted plan and loaded-byte corresponde

Boundary family	Behavior the particular claim may rely on
Cryptographic and identity mechanisms	Canonicalization, collision assumptions, signer origin, key lifecycle and credential secrecy

An external proof kernel may be a logical trusted premise; the browser runtime is a separate physical premise. Conversely, a mathematical theorem can remain valid when the deployed browser or backend becomes unqualified. An opaque Clojure callback remains outside the closed logic until its specified conformance is independently established. A malicious browser is untrusted in the modeled web contract; hostile code already holding unrestricted server JVM/database authority is a different threat model requiring separate isolation. Observation by a caller-supplied host, node or registry enumeration does not attest the deployed physical boundary. Identify separately the trusted issuer, acquired evidence and what is merely copied metadata.

The release-level ledger is the union of claim-specific dependency closures and the original-source coverage obligations. It can report rejected, stale and unobserved claims without exposing document values, command records, credentials, private principal details or raw exceptions through public diagnostics. Subsequent admission and runtime gates enforce the requisite premises; the ledger by itself does not issue a certificate or authorize an effect.

Backend assumptions are qualified contracts

The goal is not to prove all of XTDB. The goal is to state exactly which backend behaviors the application's claims use, qualify the concrete backend and adapter combination against those behaviors, and invalidate dependent applicability when that qualification no longer holds. This follows Part I.

Each assumption record contains:

- a stable proposition and the scope of the operations relying on it;
- the backend product and exact version or justified compatible version range;
- the adapter, client, query or transaction subset, codecs, topology, and relevant configuration;
- its source of justification and evidence classification;
- the executable qualification cases and their expected semantic outcomes;
- the recorded observations and known limits of those cases;
- invalidation triggers, responsible owner, and the admission/runtime response.

1. Qualification attaches to a selected lowering, not an entire product

   The qualification relation connects four independently justified links: (1) the original Gesso operation/invariant/writer catalog and its backend-dependent propositions; (2) the selected production lowering, guard/predicate and declared physical write/read footprint; (3) a test against the actual selected backend and adapter with an independent acceptance oracle; (4) evidence that the claimed installed process uses that qualified lowering. Completing only one link cannot discharge the others. A controlled test node is not automatically the deployed node, and observing equal records at one read basis is not proof of transaction-internal prestate or causality.

   The qualification fixture must record the positive legal operation, the adversarial mutation or interleaving, expected semantic result, physical observations and the backend API behavior being challenged. Independence is relative to the property: reusing the very same faulty guard or adapter to generate both expected and observed results is a shared-bug risk. Active write tests run on an isolated test backend; after a potentially submitted transaction, failures of diagnostic acquisition cannot authorize automatic retry.

   A timeout, environment refusal, unsupported query construct, missing witness, insufficient contention or ambiguous provider return is `incomplete` or `inconclusive` at the required scope, not a passing qualification. A reproduced contradiction is `refuted`. Every observation identifies which phase, basis, issuer and environment actually supplied it, plus the limits of that observation.

   Qualification tests must use the production lowering and actual backend boundary for the claim in question. A fake adapter that returns expected statuses tests the fake, not backend semantics. Conversely, backend tests do not establish that all production writers use the tested lowering.

   The qualification contract covers the behaviors actually used:

Assumption	Required challenge cases
Guard and mutation share the promised atomic semantics	Concurrent conflicting changes; guard failure; no partial promised writes
Physical replacement preserves the declared frame	Concurrent edits outside business fields; added/absent/null fields; unsupported
Relationship and absence constraints hold	Competing inserts, reassignment, association changes, duplicate command del

Assumption	Required challenge cases
Provider outcome means what the adapter reports	Submission/return/exception distinctions; abort; lost reply; restart and reconci
Read basis and coverage have the declared meaning	Lagging readers; pinned multi-query projections; insufficient progression; reco
Required intent survives commit failure windows	Crash before commit, after commit before dispatch, during retry, and after ack
Revision discipline supports a history claim	Competing writers, delete/recreate, restoration, and numeric exhaustion

2. Real-backend positive and adversarial qualification pairs

   The selected qualification profile must include relevant paired tests rather than merely collecting successful unit-test counts:

Positive boundary observation	Adversarial challenge and required outcome
Eligible guarded Claim changes authoritative ownership	Competing assignment or Membership change cannot be defeated by
Command and domain writes incorporate together	Duplicate command ID or a failed assertion cannot produce a half-co
Qualified full-document update preserves declared frame	Concurrent unrelated insert/remove, absent/present-nil or nested fiel
Protected absence/uniqueness transition admits only one primary	Two competing inserts or association changes must not both satisfy a
Result and physical query have a documented relation	ID-only submit, execute return, failed assertion, exception and lost re
Pinned observation matches tracked domain and command records	Independent pre/post snapshots and co-observed records never autor
Qualified read reaches the requested basis	Lagging reader or echoed caller token must not be reported as observ
Compatible command resolution is disclosure-authorized	Revocation, switched principal or arguments must not leak past resul
Required publication intent survives committed failure windows	Crashes before commit, before dispatch, during retry and after ackno
All covered writers preserve a version discipline	Raw writer, delete/recreate ABA, restoration and numeric overflow c
Supported codec preserves exact value meaning	Absent/nil, unsupported values and schema changes may not silently
Selected backend/adapter configuration is qualified	Changed provider, driver, topology, codec or transaction form cannot

The oracle must distinguish **pre-effect enforcement** from detecting an already committed violation after the fact. An observed transaction key, command record and document at a common basis do not by themselves prove which submitted command caused the state. One tested schedule does not establish universal isolation, all-writer confinement or an unbounded refinement theorem. The bounded tested result and every still-open transfer premise remain visible.

Provider time semantics are explicit. Valid time, system or transaction time, application domain revision, and a basis sufficient for progression are not interchangeable. A transaction receipt that establishes one does not fabricate the others. A read option must be qualified for the actual provider version; returning the caller's requested token in response metadata is not evidence that the read satisfied it.

The tests must distinguish a property violation from an unavailable environment or inconclusive run. A passed finite corpus is tested evidence under the named external premise, not proof that every backend defect will be detected. No finite suite can guarantee discovery of every future incompatible behavior. The operational guarantee is that relevant changes and observed contradictions invalidate qualification and trigger its declared response; the evidence report must not promise omniscient detection.

Qualification is enforced, not merely documented

Build/startup checks the selected backend profile and required qualification identity. Changing the backend, adapter, transaction shape, relevant codec, configuration, or topology requires the applicable requalification. A compatibility range is allowed only with an explicit justification; a floating dependency version cannot silently inherit the old result.

When a required premise is known to be violated, the affected guarantee is no longer applicable. The selected strict profile blocks new affected mutations or admission and exposes the failure. It may preserve safe reads or unaffected operations where their premises still hold. It must not silently downgrade the same requested claim to “tested.”

1. Qualification state, claim scope, and the first enforcing gate

   The selected application assurance profile identifies the **required** backend, original-source, runtime and installed-artifact premises for each operation and each exported guarantee. Qualification is not a permanent attribute of a dependency name: it is evidence applicable to a versioned backend, selected lowering, operation and installed assembly under explicitly scoped conditions.

   At build, startup and the actual protected execution boundary, the affected claim is classified from current evidence and current known premises. A required qualification that is unavailable, unsupported, stale, inconclusive or known to be contradicted cannot be substituted with a test receipt, copied success flag, historical certificate or a weaker profile while

continuing to claim the same guarantee. A strict profile must reject admission or a **new** dependent physical effect before the boundary it claims to protect. Unaffected operations or safe disclosures may remain available only when their own dependency closures and authorizations are satisfied. The decision explains the first failed premise and permitted recovery path without disclosing protected model state.

The precise gate is part of the proof obligation: a startup report that is no longer current at request time, a check after an irreversible write, or a handler wrapper that is bypassed does not show prevention by the real enforcing path. Relevant configuration, handler, backend qualification, installed plan or credential changes require the corresponding revalidation and coherent replacement/retirement of dependent issued execution capabilities. Source fingerprints and version stamps detect changes within their declared scope; they do not discover unobserved external mutations or authenticate the live host. A deliberate lower-assurance application mode may be offered as a different requested profile with its own explicit claims and owner consent. It is never a silent fallback from mandatory certification.

An operational incident may require reconciliation of already submitted commands. Stopping new submissions does not undo past commits or turn in-flight outcomes into rejections. Command evidence and admitted recovery paths remain available under their own authorization and safety contracts. A pre-existing theorem can remain valid as a conditional statement while its failed production premise is reported as unsatisfied.

Runtime authority survives every formal boundary

The browser is permitted to send arbitrary bytes. It may forge command-looking maps, DOM, headers, semantic IDs, certificate-looking objects, roles, principal fields, basis tokens, and resource references. Server authority still comes from trusted context and current public model policy.

1. An untrusted request cannot carry authority across boundaries

   For Claim, the browser may submit a believable command ID, existing Request ID, old principal ID, a cached permit, a current-looking progression token, and a well-formed proof report. Each remains untrusted request data. The actual server derives the current authenticated principal from its trusted session boundary; resolves the named operation and target through admitted registry and model paths; validates semantic argument and command identity bindings; and protects dynamic eligibility dependencies in the **actual** authoritative incorporation boundary, not only a preparatory read.

   Historical command resolution is a separate branch: a compatible recorded command may yield a permitted result only after current disclosure authorization. Revocation after an already incorporated effect does not erase history, but neither does historic success authorize another incorporation or another principal's disclosure. An inaccessible, absent or ambiguous historical record may leave the effect unknown. A copied old permit, a browser-supplied principal, or a changed request handler cannot transform that uncertainty into authority.

   The threat model records which runtime code, modules and database credentials are trusted. Neither compiled JavaScript, HTML attributes, client validation, a stamped plan, nor a certificate-like map is authority over protected server writes or reads. Public interfaces must reject same-shaped self-declared “certificates”; persisted evidence is inert until independently checked and admitted for the current source, premise, issuer and assembly. A matching checked route/affordance only proves its bounded construction relation, not that every direct entrypoint and privileged writer is mediated.

   Trusted registries resolve operations; protected reads, command lookups, and subscriptions are authorized at their actual boundaries. Artifact signing may establish origin or integrity under its key assumptions. It does not establish semantic correctness or authorize a principal. A checked rendering path does not replace endpoint authorization when the rendering is bypassed.

   The transport contract also owns request-origin controls, input limits, authentication/session handling, and protected-data disclosure. A theorem about modeled non-escalation is not a proof of the entire web stack. Hostile code with arbitrary JVM introspection or direct database credentials is outside an ordinary cooperating-process capability model unless a separate isolation mechanism is part of the stated threat model.

   Local opaque handles and captured function identities prevent common accidental substitution and forged report use. They are not a JVM sandbox, persistent revocation service, or defense against arbitrary modification of trusted code. A weak registry's garbage collection is not semantic retirement. Persisted data re-enters through validation; it does not regain issuance merely by preserving a printed identifier.

Hashes, compatibility, and installed artifacts

A recomputed digest binds specified bytes or normalized content under its canonicalization and collision assumptions. A copied digest string does not. Neither proves the proposition associated with the bytes.

An unchanged browser asset can still have stale evidence if a required semantic dependency changed. Conversely, a newly built compatible artifact need not invalidate every interface theorem. Exact artifact identity, semantic interface compatibility, and assembly applicability are checked separately according to Parts VII and IX.

1. A stale-but-byte-equal example

   The following is an illustrative **pure Clojure data comparison**, not an artifact validator, cryptographic check, trusted certificate API or proposed callable interface:

```
(let [cached {:artifact/id "sha256:91b..."
              :membership :semantic/membership-M18
              :assembly :assembly/H42}
      current {:artifact/id "sha256:91b..."
               :membership :semantic/membership-M19
               :assembly :assembly/H43}]
  [(= (:artifact/id cached) (:artifact/id current))
   (= (select-keys cached [:membership :assembly])
      (select-keys current [:membership :assembly]))])
;; ⇒ [true false]
```

   Even if the bytes were independently verified and the old browser asset genuinely still matched, its proof about M18 cannot simply serve a changed M19-dependent claim in assembly H43. The checked semantic dependency closure and the current installed assembly are separate obligations. Conversely, a newly emitted asset may preserve a semantically compatible interface without requiring every unaffected abstract theorem to be proved again. Both directions require a **checked** compatibility decision, not string equality.

   The browser build receipt binds canonical manifest input, trusted compiler identity, emitted plan registry, and actual output bytes. Startup verifies the asset that will be served. A manifest string embedded by an arbitrary emitter is not proof that its JavaScript implements the plan. The compiler and runtime remain named premises unless separately justified by correspondence evidence.

   The serving process must adopt the final checked handler, not a retained earlier closure. A local system-map assignment or router certificate is evidence about that inspected binding; the deployed-handler claim requires the actual server handoff. Deferred response bodies and streams retain the appropriate assembly and output-validation scope through emission. Versioned assets and protocol compatibility follow the coherent activation and retirement rules of Part VII.

Circular trust, shared bugs, and assertion inflation

An operation cannot prove its own implementation by declaring its desired postcondition. A certificate cannot count itself as its dependency. Recursive semantics require an explicit sound fixed-point or induction rule, not a cycle of mutually approving reports.

1. Issuer discipline, forged reports, and proof-dependency cycles

   A map such as the following is a counterexample to unsound public admission, not a valid certificate:

```
{:semantic-id :example/request-claim
 :valid? true}
```

   A public checked operation must not be issuable merely by matching these field names. Issuance checks the original admitted source, independent mandatory obligations and declared proof derivation through the relevant checker under its stated trust assumptions. An opaque process-local handle can enforce ordinary cooperating-code identity and lifetime boundaries, but is not a signature, JVM sandbox, durable revocation service, or protection against privileged runtime tampering. A copied serialized report, retained weak-reference key, or restored printed handle does not reissue authority. A durable package is imported as inert input and rechecked for its claimed source, type, version, premises and admissibility before use.

   Dependency graphs must have source-complete required obligations and an acyclic **justification** relation. A certificate cannot be its own evidence, and two certificates cannot discharge each other by a cycle of asserted equivalences. Legitimate recursion requires a separately checked fixed-point or induction rule with its soundness premise made explicit. A proof of projection cannot assume precisely the projection-preservation judgment it is supposed to establish. A theorem under open source/authorization/backend premises remains conditional even when the internal derivation is perfectly valid.

   Source-rechecking, independent negative examples, mutation tests, independently implemented checks, external formalization, and actual-boundary integration challenge different failures. They should remain separate mechanisms. Two wrappers

around the same faulty predicate do not become independent evidence because they produce different report types.

Original obligations remain visible. A report must show which root guarantees depend on each asserted or trusted premise. One central opaque premise may matter more than dozens of small checked facts. Required evidence cannot be removed from the ledger or made optional merely because the current checker cannot establish it.

Semantic fingerprints need both negative and positive controls: meaning-changing mutations must invalidate the applicable claims, while harmless source or formatting changes should preserve compatible interfaces. Under-approximation admits stale meaning; over-approximation encourages unnecessary proof churn.

Sidecar noninterference and required gates

Optional proof trees, theorem metadata, debugging traces, and diagnostic sidecars must not become inputs to production transition decisions. For the same admitted plan, initial state, and event stream, their presence or absence must not change the semantic result. Failure of optional reporting must not invent a commit, rejection, or effect permission.

Required admission checks are different: they determine whether construction or execution is permitted under the selected profile. Their failure may prevent admission. Runtime authorization and currentness checks likewise enforce real premises. Noninterference does not require bypassing either kind of gate.

1. Observable noninterference and enforceable failure semantics

   For identical admitted source, accepted inputs and environmental event history, switching optional diagnostic/proof-tree/trace sidecars on or off must preserve semantic transitions, emitted authoritative commands, execution identity, and the resulting authoritative outcome. Diagnostic code may record redacted observations but cannot inject an extra transition, alter a guard, modify an effect payload, perform an additional submission, or reinterpret an already-started transaction's uncertainty as rejection. If an observer fails **after** an effect might have been submitted, the authoritative status may remain unknown to that observer; neither an automatic retry nor a false rejection follows.

   An intentionally required admission checker is not an optional sidecar. Its failure must block the relevant protected action at the specified pre-effect gate. Likewise, a current authorization or read/disclosure check is a real execution premise, not mere diagnostic metadata. Optional reports may fail independently without claiming that required checks were skipped or that authoritative work was undone.

2. Paired enforcement, forgery, and noninterference controls

   The finished system must make each relevant boundary independently challengeable, with its real positive case and forbidden alternative:

Admitted positive case	Adversarial case and required boundary
Qualified selected XTDB backend admits eligible guarded Claim	Changed/failed mandatory backend premise blocks new deper
Current authorized principal uses an admitted operation	Forged principal, role, token, command binding or certificate r
Compatible historical command is disclosed by current policy	Revoked disclosure or changed argument cannot leak record o
Original source and valid issued proof are freshly checked	Copied report or same-shaped map cannot mint an issued valu
Acyclic source-complete proof dependencies justify a conclusion	Self-reference, mutually circular certificates and omitted requi
Versioned artifact is verified against the serving bytes and current closure	Same digest claim with changed semantic dependency/assemb
A fresh checked handler is actually installed and invoked	Retained old handler, raw route or direct writer is not silently
Rechecked semantic fingerprints preserve valid compatible changes	Relevant rule/premise mutation invalidates; irrelevant formatt
Optional diagnostic sidecar reports one underlying operation	Disabled, throwing or reordered diagnostic cannot create a sec
Required audit/publication is part of the operation contract	Treating it as droppable optional telemetry fails the operation-

   Positive controls establish nonvacuous permitted work, not merely successful rejection. These are prospective verification contracts; runtime/XTDB/browser tests in Part X 10.15 must reach the claimed physical boundary with independent effect observations and preserve unsupported, timeout and unknown classifications distinctly.

   If durable audit or publication is required for the operation's semantics, it is an explicit operation obligation with a failure contract, not an optional sidecar. Required effects cannot be hidden as diagnostics and then discarded under a noninterference rule.

Validation through the real enforcing boundaries

Formal and concrete evidence are complementary. Retain unit and property tests, exact finite checks, portable-runtime conformance, real browser integration, backend qualification, installed-application acceptance, and multi-node fault scenarios where their claim depends on physical deployment.

The validation ladder has independently owned claims

The validation ladder comprises complementary mechanisms, not a single regression-test success state. A complete release record identifies which obligation each layer challenges and the original subject from which that obligation was derived:

- **Unit and property tests** examine value, state transition, elaboration, normalization and invalid-input behavior; they cannot authenticate the production shell merely by invoking a pure function.
- **Proof checker and model/finite-state runs** establish their precise source, inference, finite-domain, abstraction or bounded-counterexample result. An incomplete exploration and a genuinely exhaustive finite result are different, even if both find no counterexample in visited states.
- **JVM/Node and CLJS portability tests** check actual shared semantics, codecs and error classification across the supported runtimes. Matching results are tested correspondence for the selected inputs, not universal source/physical equivalence.
- **Real Chromium integration** exercises the emitted, verified production browser asset, HTMX extension, EventSource, actual HTTP/SSE and DOM effect timing; a mocked adapter or synthetic browser event has narrower scope.
- **Actual Biff/XTDB integration** exercises the checked handler, issued operation, selected guarded lowering, real provider, authoritative writes, durable command record and acquired readback. A locally checked Ring object does not establish deployment through the serving socket.
- **Application acceptance** uses the actual HumanHelp public models and user-visible Claim, recovery, notification and selective Refresh actions; a toy Request, injected browser command or successful HTTP status is not a substitute for that vertical composition.
- **Multi-process acceptance** adds independent processes, shared authoritative persistence, command-resolution portability, interest routing and the selected failure topology. Part X 10.16–10.19 own its concrete design; a two-label single JVM is not two physical hosts.

Evidence may be reused only for the exact propositions, versions and qualified premises it establishes. The reported release outcome separates admitted semantic theorem, tested implementation correspondence, actual installed effect enforcement and current backend/host qualification. A failed test identifies the likely owner of the repair—original rules, semantic implementation, lowering, build/shell, policy, backend assumption or application assembly—without altering a theorem to hide an implementation or environmental defect.

An acceptance case records its admitted assembly and artifact, input and fault schedule, expected semantic result, first enforcing boundary, actual observation origin, and remaining assumptions. A negative test must reach the boundary it claims to test. A test-only wrapper that rejects earlier is not evidence that the production gateway, endpoint, or browser mutation gate enforced the rule.

Evidence fixture, independent oracle, and classification

For each required guarantee the acceptance fixture records the original Gesso source location and operation/invariant ID; expected state and allowed outcomes; exact issued assembly and loaded asset; principal, command and transaction/attempt bindings as permitted by disclosure; provider and adapter versions; controlled input/fault schedule; first **real** enforcing boundary; independent observations of both the authoritative state and any forbidden physical effect; and every open trust, coverage or fairness premise.

The expected result must come from the declared semantic obligation, not be reverse-engineered from the same adapter under test. Independent observations may use a second read path, a controlled database query, receipt/ledger inspection, forbidden-effect counter, or a browser user-visible oracle, with the independence and provenance limitations recorded. A test that asserts a wrapper's earlier rejection cannot claim the later production gateway rejected; a report computed only from supplied expected values cannot authenticate the physical transaction. When a source mutation changes semantics, record the **first** mandatory boundary expected to refuse it and challenge that boundary directly.

Run a valid positive control through the same tested boundary, not merely a separate fixture. A reject-all implementation can pass many negative cases; it cannot pass the matching positive control. Likewise a ”no forbidden write observed” verdict requires credible coverage of the actual write surface and observation interval, not an empty test-local counter with uninspected direct writers.

Outcomes are typed: `passed` under a precise scope, `counterexample` or `refuted` with an actual witness, `rejected-as-designed`, `unsupported`, `incomplete`, `unavailable`, `timed-out` and `commit-unknown` when submission may already have occurred. A deliberately rejected malformed declaration is not a counterexample to an admitted model; a finite search cutoff is not a positive theorem;

an unrelated console/network error is not an expected injected failure. Keep exact counterexample event order, identity, basis, observation origin and the point where semantic/physical evidence diverged, while redacting protected inputs and credentials from public reports.

Required mutation and positive-control pairs

Mutation or failure	Expected enforcing boundary	Positive c
Change a proof premise or dependency after issuance	Certificate currentness and required-closure admission	Unchange
Copy a successful report into a new map	Issuer or persistence revalidation	Original s
Append a physical write after canonical lowering	Owned gateway payload and footprint check	Exact adm
Change an unguarded field before a replacement write	Qualified frame/guard contract	Independe
Supply a stale principal or Organization association	Trusted invocation and atomic current-policy guard	Authorize
Retry an incorporated Claim after Request becomes claimed	Binding and permitted recorded-result branch	Same com
Lose one attempt while another delivery commits	Command-resolution boundary	Outcome
Replace a resource after callback admission but before mutation	Final physical effect gate	Current o
Break actual server adoption while retaining a checked router object	Installed production-entrypoint validation	Real requ
Change provider version or relevant adapter semantics	Qualification applicability gate	Qualified
Exhaust the finite checker before exploration completes	Evidence result classifier	Complete
Supply an unsupported host function to a closed rule checker	Closed-language admission	Explicit o

The mutation corpus covers source obligations and their actual derivatives independently. Examples include omitted operation or invariant, mutated guard/read dependency, dropped frame or uniqueness predicate, stale certificate or source closure, forged issued handle, changed plan/compiler asset, missing route/writer adoption, and an unchecked direct or deferred writer. Rejecting an invalid **declared** proof graph is not proof that a later physical raw XTDB write cannot bypass a production admission gate.

Where a positive result depends on a callback, do not assume an opaque Clojure function behaves as its declarative summary. Challenge the callback at its supported use-site or retain a named conformance premise. Where an observed result depends on backend behavior, recheck the selected qualified provider boundary, not a fake return map. An accepted source claim is not a universal security/availability claim, and a tested production slice need not refute a valid abstract theorem when its host implementation fails.

Positive controls establish useful acceptance, not just rejection. The test suite must also preserve a known formal counterexample, an incomplete exploration, and an unsupported feature as distinct statuses. “Nothing unsafe happened” is not enough if the intended operation never executed.

Physical concurrency and failure windows

Backend scenarios must include competing commands, repeated delivery of one command, policy changes between preparation and commit, changed relationship members, and concurrent writes outside the nominal business fields. The transaction tests inspect authoritative outcomes and state under the declared provider contract, not only HTTP status codes.

Physical Claim and command failure-window matrix

Tests involving authoritative mutations use concurrent requests that genuinely reach the selected physical gateway. Force contention with a barrier **after each request acquires its own prestate and issued capability** where that is the scenario. Observe what each submission actually did, which transaction committed or asserted, whether the document frame survived, and whether the command record and domain write share the promised all-or-nothing outcome. A post hoc comparison of two HTTP 200 responses is insufficient.

Required schedules distinguish:

- guard failure and successful same-version conflict control;
- competing Claim and Membership/Organization association changes;
- interleaved unguarded changes outside business fields, including absent/nil, whole-record replacement, relationship/absence and writer-footprint differences;
- duplicate delivery of **one logical command** versus two genuinely different commands and separate submission-attempt identities;
- crash/exception before physical submission, after a successful transaction but before the Ring response, and during later readback or command-resolution observation;
- already-incorporated compatible replay under current disclosure versus new incorporation, revoked permission or incompatible argument/target binding;
- a fresh **authorized** request after a resolved conflict versus prohibited reuse of the consumed one-shot writer; and
- outbox publication before/after commit, dispatch, receipt and retry under the promised delivery/gap-recovery contract.

A lost response after successful submission is not a rejected transaction; an observation failure cannot request an automatic second submission. The test independently inspects the durable physical record to distinguish the **actual test oracle's knowledge** from what the browser or request caller is entitled to know. A currently claimed Request does not establish which command caused the claim, and a matching transaction ledger/readback may be co-observation without proven causal correspondence. Time/basis, writer completeness and actual installed-handler scope must be qualified separately. Even a successful controlled embedded-XTDB contention test does not certify all backend schedules or external direct writers.

Browser scenarios include reordered callbacks, duplicate completion, failed abort, resource replacement, deferred mutation, insufficient observations, stale plans, and partial installation repair. Expected network or console failures in fault injection are asserted precisely; unrelated failures still fail the case. A successful portable test does not substitute for the real HTMX or browser lifecycle event that the shell normalizes.

Browser and user-level delivery schedules

A real-browser fixture must load the same checked runtime bytes and HTMX/SSE versions selected by the installation being claimed. It drives actual user actions and physical event delivery instead of injecting only already-normalized adapter events. Adversarial cases include SSE duplication, reconnect/loss, delayed or reordered fragment response, overlapping POST/GET, aborted request that still completes, stale callback after resource retirement, DOM root replacement, optimistic settlement after rejection or uncertainty, a deferred side effect whose owner changes before mutation, and partial installer failure followed by allowed repair.

Use both semantic adapter traces and observed DOM/network state: selected item, focus/scroll continuity, pending marker, refresh affordance, stable behavior-owning root, current/advisory/canonical progression and actual server-refetched HTML. A browser trace that retains a stale pending state, silently adopts a remote advisory revision, sends a second command, or installs an effect under a retired resource fails the appropriate obligation. Deliberately expected network/console errors must be identified by type and fault schedule; new or unrelated failures remain failures. Portable CLJS tests and mocked HTTP fixtures are useful separate layers, not evidence that native HTMX lifecycle events were normalized correctly.

For HumanHelp, complete acceptance starts from real production model operations, routes and protected public policy, covers the actual Claim/unclaim/take-over/done/cancel life cycle and independent multi-client visible state, and preserves the distinction between a creator's immediate result and another viewer's advisory update plus explicit Refresh. A browser-only demo, a fabricated positive command, or a reachable-looking checked route does not close this application-wide guarantee.

Host scenarios include concurrent independent requests and any supported deferred execution. They must demonstrate that one request's identity, writer capability, result, or admission limit cannot be silently inherited by another. Thread-bound APIs that intentionally exclude deferred work must reject it clearly rather than claiming it was mediated.

Independent host isolation and closure limits

Concurrent checked Biff requests must not share request-local writer issuance, principal context, one-shot permit, current target, result or admission budget. Test controlled overlap, queue saturation, timeout/exception cleanup and sibling-request isolation. If a

scoped writer is deliberately synchronous/thread-bound, deferred callbacks and off-thread use reject before an effect rather than inheriting a previous request's permit. A post-dispatch host report may detect a mismatch after irreversible work; it cannot certify that the write was prevented.

Production adoption is checked separately from an in-process system map: show that the actual listener dispatches through the final checked handler, that real HTTP invocations reach their intended operation and guarded writer, and that the serving JS/fragment/stream corresponds to the admitted assembly. Deliberate bypass tests should challenge raw Ring responses, retained pre-mediation handlers, background/worker/migration/admin writers and separate XTDB credentials under the application's declared supported universe. The result may correctly be `open` for uninspectable external writers; absence from the surveyed host registry is not proof of absence from the deployment.

Run completion also records negative controls for the test harness itself: missing backend, never-executed action, empty generator, untriggered race, absent browser asset, unsupported physical interface, and timed-out test. These do not pass due to vacuity. A scope-qualified failed oracle may block production admission if it is required by the selected assurance profile, without conflating that gate with optional logging or undoing an already committed effect.

No single passing history establishes universal refinement. These tests make premises falsifiable, exercise real boundaries, and provide reproducible regressions while the report retains their scope.

Multi-node realization and authoritative rendezvous

Multi-node deployment changes physical placement. It does not create a second authority model, a second command meaning, or automatic permission to adopt remote state.

One semantic machine under physical placement

Single-process and multi-process Gesso are **realizations of the same** admitted application. The original model rules, public operations, command-resolution outcomes, Choreo roles, knowledge obligations, browser progression and adoption policy do not acquire a separate distributed meaning. What changes with placement is which physical process, transport, backend, interest router and resource instance realizes each part of that same contract.

The distributable implementation therefore has two independently checked relationships:

- **Semantic preservation**: every selected distributed operation, settlement, invalidation, acquisition and effect corresponds to an admitted single-machine behavior, including rejection, unknown outcomes, stuttering and permitted nondeterminism.
- **Physical qualification**: the actual installed hosts, writer paths, authoritative backend, request/session bindings, transport and routing/recovery mechanisms satisfy the named assumptions required for that correspondence under the declared topology and failures.

The semantic machine does not become correct merely because two nodes agree with one another. A local reporter's assertion that it used the shared database, a matching application fingerprint or a successful load-balanced request is not evidence of all relevant physical entrypoints, current authorization, writer confinement or a general distributed refinement theorem.

Roles, principals, authorities, and hosts remain distinct

A role is a choreography role. An actor realizes that role under a principal and context. An authority owns specified authoritative facts or operations. A host is a physical process that may realize an authority projection for an invocation. None of these identities is interchangeable.

The identity contrast is:

```
choreography role:   :request-authority
original host:       web-node A
replacement host:    web-node C
semantic authority:  [:authority :request R]
```

These symbols illustrate different identity categories, not a special topology API or an assertion that node A owns authority. The same principal may have different actor instances in separate tabs, and separate HTTP requests may realize the same logical command. A host/process identity and an admitted assembly generation likewise must not be substituted for an execution, principal, authority or resource-instance identity.

A Claim sent through host A and retried through host C retains its bound `:gesso/command-id` and public operation meaning. Each delivery may have a new `:http-request-id` and attempt identity. Host C must authenticate its own invocation, validate the existing command binding, and use the shared authoritative command-resolution mechanism. Node-local caches cannot establish cross-node deduplication.

The retention contract names the period and scope in which replay is supported. A deleted command record cannot silently make an old command identifier a fresh intent. The protocol must either preserve sufficient tombstone/exclusion evidence, use an enforced expiry or epoch contract that rejects expired reuse, or narrow the claimed retry guarantee explicitly. A client must not work around uncertainty by allocating a new command identifier for the same pending intent.

Physical relocation cannot reconstruct an intent whose binding was never retained. A current-state read at host C may establish the Request's present state while leaving the originating participant's command outcome unknown.

Command identity, compatibility, and retention across hosts

The original operation and command binding remains stable across host movement only when its actual retained evidence survives: authority/namespace and target, principal, operation, immutable semantic arguments, command identity, retained outcome and any applicable compatibility or expiry discipline. An HTTP delivery may have a new transport-request ID, and each physical submission has its own attempt identity and one-shot capability lifetime. Neither the new host nor a repeated caller-provided command ID confers fresh authorization.

For a retry through C, the current trusted session/principal and protected result disclosure must be re-established. A compatible **recorded** command is resolved without incorporating again; a new command requires its own current authoritative eligibility and transaction-time guards. Duplicate concurrent deliveries of one command must not both create a new durable effect, under

the qualified atomicity and retained-record assumptions. A changed principal, target, operation or argument binding cannot claim the old result. Missing, expired, inaccessible, or indeterminate evidence cannot silently become a new command or rejection.

The retention guarantee states its supported time horizon, namespace, archival/tombstone or expiry mechanism, and treatment of old commands after node replacement or backend restoration. A finite record TTL without enforceable reuse exclusion does not provide indefinite deduplication. A replaced database incarnation or restored old snapshot does not automatically continue the old command history merely because its authority label or IDs match.

An origin host may fail before submission, after an unobserved submission, after commit but before HTTP settlement, or after result publication. Those are different participant-knowledge states. The successor can return a recorded historical outcome only after acquiring sufficient trusted command-specific evidence; reading a Request's present state alone cannot establish the identity or outcome of Alice's lost command. A process-local attempt journal, response object or callback is not a portable cross-host authority credential.

The selected ordinary topology is deliberately small

The baseline multi-node realization uses:

- at least two independently running application processes behind a request distributor that can route a retry or reconnect to another process;
- one qualified authoritative backend deployment shared by those processes;
- durable publication intent stored atomically with the model operation where the application requires post-commit notice recovery;
- a dispatcher and advisory distribution path that can reach interested sessions on either application process;
- shared or otherwise compatible authentication, command resolution, protocol, artifact, and assembly-compatibility contracts.

These are responsibilities, not a required list of infrastructure products. A separate broker is warranted only when the selected delivery contract needs it. Multiple processes and a shared database are sufficient to test meaningful host replacement; a single process with two labels is not.

Minimal physical realization without prescribing products

A practical first deployment may use two Aleph/Biff processes, a request distributor such as HAProxy, shared qualified XTDB persistence, and a cross-host invalidation transport such as Valkey. A durable log such as Redpanda or another broker is optional and justified only by the selected acknowledgment, replay or routing contract. These are illustrative integration choices, not Gesso protocol requirements, mandatory infrastructure, or proof that the deployment works.

The implementation should first express these responsibilities through stable Gesso semantic, writer, publisher, registration and transport interfaces. An in-process implementation and a multi-process implementation must satisfy the same application contract. A later Rust router or another transport implementation is an optimization or independently qualified adapter, not an excuse to change application semantics or force authors to write a separate multi-node protocol.

Different nodes may concurrently run compatible but nonidentical assembly generations during a rolling deployment. The selected version-compatibility contract must identify admissible old/new operation, command record, credential/session, browser artifact, wire-format, progression, interest-registration and recovery behavior. If compatibility is not established, new admission or adoption is blocked, or a defined retirement/reacquisition path is used. A retained old route, writer callback or browser resource cannot silently cross the compatibility boundary.

The topology qualification identifies concrete services, versions, transport configuration, credentials, failure model, and consistency assumptions. A backend being shared does not prove that two read paths have identical freshness. For that topology, the root ledger records at least:

- actual **distinct** host/process identities, deployed final handler/worker generation, serving sockets and writer entrypoints;
- the shared authoritative backend's real incarnation, version, transaction/consistency contracts, writer domain and connected credentials, including external/direct writers where in scope;
- the request distributor's routing and failure behavior, what session authentication can be reconstructed at a replacement host, and how clock/host-health observations are qualified;
- actual command-record retention, historical outcome readback, compatibility and disclosure, not just process-local one-shot evidence;
- distributed publication, interested-recipient routing, registration/replay/gap detection, authorization, and which observations are durable versus advisory;
- supported concurrent and mixed-generation admission, failure detection, bounded recovery/retirement and liveness/fairness premises, with the first enforcing or observation boundary for every exported guarantee.

The current authoritative read may be served through a different XTDB client or host context. A successful query is not proof that the requested progression frontier was satisfied, and token metadata echoed from a request is not an authenticated observed basis. A qualified insufficient read preserves the browser's outstanding debt and recovery path. Provider system time, transaction ID, domain revision and an SSE cursor are not one interchangeable global sequence.

Invalidation routing is a realization, not a second authority

An authoritative database change, a durable publication intent, an expanded Gesso Live invalidation, a router-forwarded notice, a subscriber's admitted advisory and a domain-participant message are different facts. In particular, an invalidation means that relevant authority may have advanced and a later acquisition must respect its scope/progression. It does **not** mean the client received a choreography message, that protected content may be disclosed, or that a browser may automatically adopt new state.

Indexed local interests may be extended across nodes by one or more routers. Hierarchical/coarse filters, replicated interest views, interleaved routing assignments, and conservative republication are possible realizations; none is prescribed by the semantic model. Any selected topology must specify how interest registration becomes effective, which host/router has enough knowledge to avoid excluding an interested recipient, what happens when that knowledge is stale or a router fails, and how a missed notice is detected or conservatively recovered. An independently healthy backup router is insufficient if it has not acquired the relevant interest or replay frontier. Router process health alone cannot certify complete routing state.

Because invalidations do not command a domain mutation, duplicate or unnecessarily broad **authorized** advisories may be acceptable when coalesced without losing required progression or violating privacy and load constraints. This permits conservative recovery without logging one completion receipt per recipient for every broadcast. But retransmission cannot repair a missed publication or registration race unless a qualified retained source, fence, replay or gap detection mechanism provides the necessary evidence. Neither a dropped notification nor a long silence is proof that no relevant change occurred. The exact outbox/registration and subscriber contracts belong to 10.17; direct, semantically meaningful participant routing belongs to 10.19.

The minimal multi-host contract must also preserve tenant and principal scope: interested routes do not grant permission to read a Request, and a topic or advisory payload cannot leak protected identities, values or counts merely because it contains no document body. Reauthentication, disclosure checks and resource ownership still apply at receiving hosts and browsers.

Placement-specific fault boundaries and evidence

A real multi-host acceptance cannot be performed by renaming two objects inside a single JVM. The test harness must be able to stop one physical process while a second continues, route new requests independently, force a qualified lagging read, change shared authorization state, observe committed command records, and exercise notice loss or an incomplete registration frontier. It must record which process, backend and transport actually supplied each observation.

At minimum, challenge a same-command retry through a surviving host, concurrent duplicate submissions, lost original response, backend incompatibility, command-retention expiry, stale interest/router knowledge, router replacement, interrupted rollout, and one authorized cross-host Refresh. Pair each with a successful ordinary operation rather than passing a reject-all or never-delivered implementation. 10.17–10.19 own the detailed publication, HumanHelp two-host schedule, ephemeral-message contrast and complete acceptance matrix.

Safety is required during permitted interruptions: do not invent commits, successful deliveries, authorized disclosures, progression or UI adoption. Conditional liveness requires explicit assumptions about backend availability, surviving routes, retry/replay retention, eventual dispatch, current authorization and (for HumanHelp remote adoption) a permitted user Refresh. A partition that prevents those premises cannot be used to claim eventual progress. This is a named open guarantee, not a reason to weaken the single-node safety laws.

Qualified progression checks still apply at each host.

Durable facts are the ordinary rendezvous

For HumanHelp's ordinary cross-user work, Alice submits a public authoritative operation, its durable state and command result are incorporated, and a durable outbox intent supports advisory publication. Bob's session may receive an advisory through another host. That advisory records a possible need to acquire new authority; it does not carry permission to adopt protected domain content.

The durable-state contrast is an illustrative semantic path:

```
Alice browser → host A → guarded Request/Claim operation
                              |
                durable Request + command result + outbox intent
                              |
                  cross-host advisory distribution
                              |
Bob browser <- host C <- authorized scoped notification
    |
explicit Refresh → qualified authorized authoritative reread
    |
current browser resource → permitted adoption batch
```

A Request and its assignment are the durable rendezvous, not an in-memory suspended choreography on A. None of these arrows alone proves the next step: an advisory is not permission to read, an observed Request state cannot attribute Alice's lost command, and an acquisition is not an installed browser view.

Bob's explicit Refresh authorizes a board acquisition under the current principal, scope, required progression, and Part VIII policy. Host C reads the qualified authoritative projection. The browser admits the response under its current resource ownership, then installs and records adoption only when the required batch completion succeeds.

The durable Request and assignment facts are the rendezvous. Ordinary Claim, Reassign, lifecycle updates, and later reads do not require a persistent participant-message process or a suspended choreography stack for every user. The public model contracts and short choreographies connect those interactions.

Source-derived publication and atomic recovery obligations

An admitted authoritative operation declares the eligible semantic changes and scopes that require later notification. A primary domain change need not depend on which browser interests were known in the writer's process at commit time: an ephemeral interest cache is not a complete inventory of future subscription/recovery needs. The exact protected operation/writer catalog from original source must be related to its production effect and publication-intent generation; observing one well-behaved writer does not close unregistered or external write paths.

Where the selected application promises recovery of required post-commit notice, the model effect, durable command outcome where applicable and durable outbox intent are incorporated atomically under the qualified backend contract, or through an independently justified equivalent that cannot lose a required intent after commit. An attempted command whose outcome is unknown cannot be used to invent a success advisory or automatically resubmit the effect. Publication failure after a known commit cannot undo it.

Each durable intent binds an independently issued identity, applicable original operation, authorized scope, semantic notification purpose, versioned encoding and source-qualified progression requirement. Command ID, outbox ID, provider basis, domain revision, transport cursor and browser resource are distinct. The outbox records the minimum required scope/notice material, not a snapshot of private Request/User data that bypasses future read authorization. A separately declared best-effort notification may omit durable intent only if loss is genuinely permitted by its weaker application contract.

Outbox dispatch and advisory recovery have separate guarantees

The outbox record establishes durable publication intent, not delivered notice. The dispatcher claims pending work through a qualified concurrency mechanism, retries failures, and records completion according to the chosen publication acknowledgment contract. A crash after publish but before recording completion may duplicate publication. Receivers must handle duplicates without duplicating commands or inventing progression.

A durable outbox identifier, command identifier, provider progression requirement, and subscriber cursor serve different purposes. One may be related to another by checked evidence; they are not the same token.

Dispatch ownership and delivery evidence boundaries

The dispatch state machine names pending, claim/lease, attempt, accepted-by-distribution, completion, retry and retention. A qualifying lease/fencing or comparable concurrency contract prevents a stale worker from illegitimately completing a task taken over by a surviving worker. Lease clocks and transaction assumptions are qualified rather than inferred from process health. A dead host cannot strand publication intent solely in a process-local callback.

The selected acknowledgment must say whether it is a log append, broker acceptance or other durable distribution-boundary event. A send timeout after possible acceptance is not proof of failed delivery: retry may duplicate the **advisory**. The receiver's idempotent/coalescing logic must preserve the strongest required scope/progression debt without issuing another domain command. A publication attempt, broker acceptance, router forwarding, SSE write, browser receipt, current authorized reread and completed DOM adoption are different evidence classes.

Stage	What its observation establishes
Domain commit	Authoritative state and qualified required intent exist
Dispatcher claim	Worker owns one temporary publication attempt
Distribution acknowledgment	Named broker/log/router boundary accepted notice
Router forwarding	Candidate interested host/session route was selected
SSE/HTTP write	Transport attempted delivery, not browser reception
Browser advisory admission	Potential-change debt was registered
Authoritative acquisition	Current permission and provider basis allow a candidate
Adoption batch	Current owned resource installed the permitted candidate

Ordinary advisory recovery need not store a persistent per-recipient completion record for every send: conservative redelivery is permitted under authorized scope, load, progression and retention bounds. A claimed at-least-once **browser** delivery guarantee would need stronger recipient acknowledgment/replay evidence.

The selected delivery contract is at least once to its named distribution boundary under dispatcher availability and retry premises. It must state whether that boundary is a durable log, broker acknowledgment, or another qualified acceptance point. It must not describe this as at-least-once delivery to every browser unless retention, recipient acknowledgment, replay, and authorization actually establish that stronger claim.

Outbox draining alone does not close a subscriber's registration gap. If a notice was published before a new interest became effective, a live-only subscriber may miss it. The baseline subscription protocol therefore establishes a fence or equivalent qualified handshake relating initial observation, registered interest, and replay/gap evidence. A gap is recorded even if no advisory arrives. Reconnect carries the same obligation; silence is not evidence of no change.

Registration fences, late interests and router failover

An initial board read followed by registration can miss an intervening update. Registering first and then reading is sufficient only if effective registration and the observation's coverage are related by a qualified fence or equivalent handshake. A supported protocol relates the authorized first observation, effective interested-recipient registration, retained event interval and replay or conservative reacquisition. If provider and router use different sequence coordinates, a checked bridge must relate them; equal-looking counters/timestamps are not evidence of the same frontier.

A change committed just before a new interest was learned must not disappear solely because no earlier browser listener existed. Likewise a silent dropped notice, late interest update, restarted host, expired cursor, changed filter or reconnect either has qualified event coverage or produces explicit gap/recovery debt. A safe alternative to full replay is conservative authorized reacquisition, subject to HumanHelp's explicit Refresh policy and the selected user-action/recovery contract. Silence is not evidence that the subscriber is current.

Routers may use indexed interests, replicated filter views, coarse/fine hierarchies or interleaved ownership. None is required by the semantic model. A healthy replacement router with an outdated interest view is not evidence of routing completeness. The selected contract must identify registration ownership, effective propagation, filter/version exchange, failure detection, retained publication source, and how undercoverage is avoided or recorded. Overdelivery or duplicate descriptive invalidations may be cheaper than precise per-recipient acknowledgments, but only when scope privacy, bounded fanout and progression composition hold. Merely replaying events for interests remembered by a failed host cannot recover interests the surviving router never learned.

A **change event**, a durable publication intent, an expanded Live invalidation, a broker notice and a Choreo participant message have different meanings. Invalidations do not carry permission to mutate authoritative state, and must not be silently upgraded into participant-message receipts or UI adoption. The exact direct-message contract belongs to 10.19.

Interest filtering is an optimization over authorized scope. It must either avoid missed relevant changes under its stated contract or expose conservative gap/recovery evidence. Stale interest registration, dropped transport messages, retention expiry, or node loss cannot be treated as proof that the user's view is current.

Recovery guarantees and independent fault oracles

Safety is unconditional relative to the declared trusted boundaries: lost, delayed, duplicated or misrouted notices cannot authorize a protected read, manufacture an incorporated command, erase unknown command status, invent an observed basis or silently install remote model changes. Publication and subscription progress are conditional on required retention, working backend, surviving dispatcher, available distribution path and retry fairness. Browser receipt adds connectivity and current scope; HumanHelp remote adoption additionally depends on explicit permitted Refresh. A partition or revoked permission cannot be called a violated unconditional liveness promise.

Positive control	Adversarial control
Atomic Claim commits state, command and outbox intent	Host dies after commit before publication; intent survives
Replacement worker dispatches pending publication	Dead host-local callback was the sole publisher
Broker/log acknowledges at declared boundary	Timeout after possible accept is mislabeled no-delivery
Receiver coalesces repeated scoped notice	Duplicate publication triggers a second Claim
Subscriber registers with qualifying fence/replay	Commit between first read and effective interest is silently missed
Backup router recovers or conservatively detects uncertainty	Healthy router with stale filter drops notice without gap
Expired replay cursor yields recovery debt	Silent reconnect is declared current after retention expiry
Sufficient authorized reread on C clears covered debt	Token echo or lagging read claims a frontier not observed
Explicit Refresh permits one candidate acquisition	Advisory alone auto-adopts protected Request content
Scope-minimal notice reaches authorized destination	Cross-tenant topic or counts leak via routing
Publisher eventually reaches named boundary under fairness	Partition is misreported as unconditional browser delivery
Compatible recorded command is disclosed on retry	Lost HTTP response causes a new domain incorporation

The future tests use actual independent host, provider and transport boundaries with positive controls; in-process source inspection alone cannot establish this multi-node contract.

Advisories and interest topics can themselves disclose protected information. Their payloads and scope references are minimal and authorized; routing membership does not grant permission to read the underlying Request. A notice may carry an opaque progression requirement without revealing protected content or counts.

HumanHelp preserves explicit Refresh across hosts

Part VIII selects explicit Refresh for both remote creation and remote lifecycle changes to already visible cards. This policy is settled for this plan; only the separate response-flow product contract remains unresolved there.

Remote advisories may update scoped outstanding debt or indicate a recovery gap. They do not automatically fetch and adopt a new board. Own-command outcomes and the specifically permitted local rejection/unknown recovery paths remain the exceptions defined in Part VIII. Those exceptions do not authorize unrelated membership changes or clear unrelated remote debt.

The selected product contract and distinct permissions

This is a selected **HumanHelp** application policy, not a general change to Gesso Live's choreography semantics. The authoritative model may commit remote Create, Claim, Unclaim, Take-over/Reassign, On-the-way, Complete or Cancel on another host. The resulting scoped invalidation records potential outstanding authority that Bob could choose to acquire; it never grants a domain read or installs a changed card. Both remote creation and lifecycle changes to a card Bob already sees require explicit Refresh for ordinary collection-content adoption.

The two independently permitted local continuations are narrower than a global Refresh:

- Alice's own accepted command can adopt its **correlated** authorized result and the declared selective affected-card or membership change, subject to physical installation and current ownership. It cannot clear unrelated board advisory debt or pretend that a later individual-card observation refreshed the whole cohort.
- A rejected or indeterminate local command can perform its specifically admitted affected-Request recovery read. This can disclose a current state caused by someone else where current authorization permits, but cannot add unrelated collection members or convert unknown command attribution into a historical success/rejection claim.

Initial page acquisition, user Refresh, an own-command result, and a scoped conflict-recovery read carry different grants and coverage claims. A browser can still display a stale Claim button before an explicit Refresh, but its invocation must pass **current** protected model eligibility and cannot assume the displayed owner or Membership is authoritative. The separate HumanHelp response-interaction product choice in Part VIII remains unresolved; completing this two-host Claim case does not certify that product journey.

One Refresh grant authorizes one board acquisition under that contract. It does not schedule an automatic successor read when a newer advisory arrives during the request. A user action explicitly coalesced under the permitted policy is accounted for as that action, not invented by the scheduler. An actually newer qualified observation may discharge additional covered debt where the scoped progression relation proves it sufficient.

Refresh grants, adversarial arrivals, and scoped debt

One permitted Refresh issues one board acquisition with a stable scope, resource ownership, optional previously observed progression requirements and a bounded lifecycle. If a newer advisory arrives while the acquisition is pending, it records separate or composable scoped debt; it does not silently schedule a successor GET or confer a new user grant. Only a new explicit Refresh/Retry action, or user action explicitly coalesced under the admitted policy, may issue the relevant additional acquisition. If the **already issued** qualified read returns a sufficiently newer coherent observation, it may discharge newer debt that is actually proved covered. Mere chronological arrival, a greater-looking token or a matching Request UUID is not such proof.

A command outcome, invalidation, provider read basis, SSE cursor, board-cohort membership, per-card observation and successful HTMX installation are distinct evidence. Debt merging preserves organization/location/projection and progression-compatibility boundaries; unrelated or incomparable requirements remain separate. When retention, coalescing budget or replay coverage is insufficient, report typed unknown/gap debt rather than dropping it or declaring the board current.

Search, sort, mine-first, unclaimed-first, terminal-visibility and card expansion operate on the **adopted** bounded cohort under Part VIII's local presentation contract. They cannot trigger an undisclosed remote collection reacquisition just because the visible board has newer advisory debt. A successful local action may selectively update the affected card and legitimate membership without upgrading the entire adopted-cohort basis.

A board observation must be coherent across Request, primary assignment, and permitted User projection at its declared provider context. A lagging host that cannot satisfy the required progression returns insufficiency and preserves recovery. It must not echo the requested basis as if it had observed it. A successful HTTP response is not evidence that the required frontier was reached.

Qualifying one coherent board candidate

A complete-board acquisition binds the current trusted principal and allowed Organization/Location, source-owned cohort and completeness contract, RequestAssignment primary membership, Request statuses, permitted User projections, query temporal context and finite collection bounds. These relations must be observed at one qualified common provider basis or carry independent evidence sufficient to justify the advertised coherence. A series of unrelated reads on an advancing database is not a complete snapshot because the last response happened to return HTTP 200.

An insufficient or lagging read through C must preserve the original debt and typed retry/repair affordance. The server must not fabricate sufficiency by echoing a requested frontier into its response. A complete authorized current board can establish current cohort membership; an individual Request recovery read cannot. A missing Request in an incomplete, denied, truncated or differently scoped result does not establish authoritative absence. Counts are facts about the **adopted** cohort, not the latest global database.

The browser treats the received candidate as **not yet adopted**. The current resource instance, authenticated scope, acquisition grant, nonregressing progression and permitted installation batch must agree. For HTMX OOB toolbar and collection changes, physical target installation and batch completion are distinct from HTTP receipt. A failed or partial swap retains unsettled adoption debt and uses the declared compensation/repair protocol without claiming that an older owner regained rights. A delayed callback, aborted-but-completed request or replaced DOM resource cannot install a stale candidate, even when its domain Request ID is still valid.

No acquired board is adopted if its resource is retired, its scope is no longer permitted, or it would regress newer locally installed authority. Installation of toolbar, collection, and associated evidence follows the adoption-batch contract, including repair after partial failure. Reconnect and host replacement do not waive that contract.

A known authorization-context change retires the affected protected projection and subscriptions according to their lifetime contract. It cannot erase data already disclosed to an offline browser. Protected rereads and subscription renewal still require current authorization at the replacement host.

Context changes outrank remote-adoption preference

Logout, switched principal or Organization, revoked read permission and replacement of the admitted board scope are trusted context/lifetime transitions, not ordinary remote Request advisories. They retire incompatible subscriptions, grants, pending installations, provisional effects and protected projections according to the supported-client contract. An in-flight request created for Bob's prior Organization cannot install its late response into the new identity's board. Historical command-result disclosure requires its own **current** principal and permission; reconnect to a different host does not inherit Alice's old browser settlement rights.

No framework can retract protected bytes already copied by an untrusted or offline client. The enforceable boundary is current server-side read, stream, command, historical result and recipient/OOB-drain authorization, plus retirement of controlled client resources. An advisory topic or count must not reveal otherwise protected Request, helper or Organization facts.

A concrete two-host failure schedule

The schedule's authoritative and browser facts must be observed at different physical boundaries. In particular, an outbox publication does not confirm Bob's SSE receipt, Bob's SSE receipt does not authorize Refresh, and an authorized acquisition does not prove a DOM adoption. The Alice/host-A and Bob/host-C flow is the reference scenario. The steps below require a **qualified** shared backend, distinct running host processes, actual current sessions, and the installed Gesso/HTMX/SSE browser path for an end-to-end acceptance claim. A two-host fixture drawn as one JVM with two labels does not establish this physical guarantee.

The following schedule exercises distinct facts; it is an acceptance scenario, not a proof that every deployment execution behaves this way.

Step	Authoritative or transport fact	Permitted participant conclusion
1	Alice's bound Claim reaches host A	Submission may proceed; no commit is inferre
2	Guarded Request, primary assignment, command result, and outbox intent commit	The authoritative command is incorporated
3	Host A dies before delivering Alice's response or dispatching the outbox	Alice may still have `:commit-unknown`; the com
4	A dispatcher on another surviving process publishes the retained intent	Notice reaches the declared distribution bound
5	Bob receives a valid scoped advisory through host C	Record debt; do not automatically adopt the re
6	Alice reconciles or replays the same bound command through host C	After binding and disclosure checks, return th
7	Bob presses Refresh and host C obtains sufficient coherent authority	Acquire a candidate observation; current brow
8	Bob's current resource successfully installs the admitted batch	Record the permitted adoption and discharge

Failure cuts, correct negative conclusions and recovery

Fault point or race	Required observation and forbidden shortcut
A fails before reaching the physical write boundary	No commit inferred from intent; reissue only under the qualified com
A submits but the transaction result is lost	Alice keeps `:commit-unknown` until command-specific retained eviden
A commits but dies before HTTP result or outbox dispatch	Durable domain, command result and required publication intent sur
Outbox is forwarded twice or acknowledgment is lost	Bob may see duplicate advisory debt, never a second Claim or autom
Notice was emitted before Bob's interest became effective	Registration fence, replay or explicit gap/authorized recovery; stream
Router fails with stale interest knowledge	Replacement repairs coverage or records uncertainty; healthy proces
Bob's previous Refresh is pending when another notice arrives	Retain uncovered newer debt; do not schedule an automatic successo
C returns a lagging or noncoherent board	Reject insufficiency as a candidate for the claimed scope, even when
Bob changes Organization or logs out during the read	Retire incompatible owner; drop the old response and recheck server
Browser swaps toolbar but fails collection installation	Keep batch/adoption incomplete and repair within current resource l
Bob's DOM resource is replaced while an old callback is pending	Old callback/effect cannot mutate the new resource or restore stale c
Alice resolves C42 through C after concurrent new domain activity	Historical C42 attribution and current Request content are acquired s
C42 cannot be disclosed after Membership revocation	Nondisclosure is correct; never guess rejection or reveal another user
A Remote Request is created while Bob searches adopted cards	Search changes only adopted presentation; new Request appears only

The positive control for each case includes a real eligible operation and, where the policy permits, a subsequent successful authorized acquisition and installation. A reject-everything handler, empty subscriber fixture, untriggered process fault or stale synthetic adapter trace is not evidence of the relevant production guarantee. Gesso's underlying model checker can explore abstract schedules, but physical two-host correspondence also requires real independent host, provider and browser observations with their versions, scope and open premises recorded.

Variants lose the advisory, expire a cursor, duplicate publication, reject a lagging read, change Membership, or replace Bob's resource between receipt and mutation. Each variant must preserve its unresolved obligations until the appropriate evidence and permission exist. Host C's ability to read the current Request does not by itself attribute Alice's command.

Direct participant routing is an explicit extension

An ephemeral feature can have different semantics. For a typing indicator, the meaningful event may be an admitted participant-message receipt, with no durable business fact to reread. Its contract must say so instead of borrowing the Request invalidation protocol.

Contrasting ephemeral typing-message contract

The durable HumanHelp Claim example in 10.17–10.18 has a current authoritative Request to reread. An ephemeral typing indicator can instead depend on a specific **admitted participant-message receipt** from one choreography execution. The receiver cannot reconstruct that event merely by querying the latest Document, nor may a database-update advisory substitute for the message.

The following Clojure data illustrates this contrast. The compact data below retains the semantic fields without claiming that a routing namespace, function or physical transport implementation is thereby specified:

```
{:message/intent {:message/id :illustrative/message-M
                  :kind :collaboration/typing
                  :document/id :illustrative/document-D}
 :contract {:delivery :best-effort
            :durability :ephemeral
            :duplicates :permitted
            :late-message :discard
            :sender-binding :authenticated-principal
            :recipient-binding :current-collaboration-route
            :authorization :current-document-membership
            :semantic-effect :recipient-observed-admitted-message}}
```

These are **schematic** bound identity placeholders and declarative requirements, not runnable sample API calls, authentication evidence, or checked physical admission. The same browser-supplied message ID, kind and Document ID do not authenticate its sender, designate permitted recipients, or assert actual receipt. The trusted host must bind sender and recipients from current authenticated and authorized collaboration state, and the receiving host/session must independently admit the routed message for its current principal, role, execution, scope and resource owner.

A Gesso Choreo `:communicate` transition is the global semantic declaration; endpoint projection produces role-local send/receive obligations. Actual physical routing must refine the declared message/receipt events. Projection may establish who is supposed to know what; it cannot infer that a remote process actually delivered the envelope, that a browser saw it, or that a protected session was authorized at delivery time. A transport adapter supplies those distinct, source-linked correspondence witnesses.

A minimal typing contract binds sender identity through authentication, resolves recipients from current authorized collaboration scope, limits payload, defines message and execution correlation, admits receipt under the recipient's current resource, and specifies best-effort delivery, duplicate handling, and expiry. The UI effect still passes its final ownership gate. Loss is permitted by that best-effort contract; persistent replay is not automatically useful for an expired typing signal.

Physical correlation, delivery, and admission

Distinguish message identity, choreography execution, role-local send occurrence, transport attempt, broker acknowledgment, recipient-side receipt and actual UI effect. A best-effort transport may lose or duplicate an envelope. A duplicate with the **same admitted delivery identity** must not execute the same role-local semantic receive twice within its declared deduplication scope; different declared messages must not be collapsed just because their payloads or Document selector match. Where a protocol explicitly tolerates repeated typing indications, idempotence still must not mint fresh sender knowledge, permissions or authoritative content.

Receive admission checks intended recipient and execution/role correlation, complete required payload shape, authenticated sender provenance from the trusted routing boundary, current Document membership, compatible plan/protocol generation, expiry, and current resource ownership. A message for a retired or replaced browser target cannot reuse a newer target's rights. If a sender loses membership before the relevant trusted send boundary, old browser intent is not grandfathered. If permission is revoked between routing and receipt, the chosen current-disclosure/recipient-admission policy decides what may still be shown; an old broker queue does not override it.

No router-supplied sender ID, recipient selector, proof-looking map, claimed host identity or timestamp is independently authoritative merely because its syntax matches a declared type. Security and disclosure must be re-established through actual trusted installation boundaries. Recipient-scoped delivery cannot reveal private Document existence, membership or typing activity to a different tenant or session. Malformed, oversize, unsupported, replayed under an incompatible generation, and mistargeted envelopes have typed rejection/ignore behavior under the selected protocol; none may grant an alternative authoritative operation.

If expiry uses clocks across hosts, the contract names the clock and skew assumptions, or uses a receiver-local lifetime with appropriately narrower meaning. A timestamp alone does not prove that a remote message is fresh. A broker's acknowledgment, transport delivery, admitted receipt, rendered indicator, and domain commit are distinct events.

Expiration, optional durability, and non-interchangeable receipts

Typing is a useful example precisely because its best-effort loss is allowed and a missed transient UI effect need not be recovered by durable authoritative reread. The bounded lifetime is scoped to the chosen sender/recipient contract. If clocks across hosts are compared, issuer, offset/skew and admissible expiration behavior are trusted premises. An opaque receiver-local timeout can implement a narrower lifetime without assuming synchronized wall clocks. A stale but syntactically valid typing message is discarded; a late duplicate cannot resurrect a retired resource or start a new execution.

If a product later requires guaranteed receipt, this is a different explicit choreography/message contract: durable retained message identity, recipient-specific authorization and admission, receipt acknowledgment, deduplication horizon, replay/gap and retirement behavior, and the appropriate fairness/availability premises. Broker durability, a publish acknowledgment, a websocket/SSE write, participant semantic receipt, and displayed indicator remain different events. Even a durable routed message is not an authoritative Request commit unless an explicitly admitted domain operation performed that commit.

Stronger direct-message guarantees require explicit persistence, recipient admission, acknowledgment, deduplication, retry, retirement, and version compatibility contracts. Infrastructure reuse is allowed; semantic conflation is not. Direct participant routing must not redefine ordinary authoritative convergence or bypass protected-data authorization.

Assured endpoint correspondence without a second application

Gesso developers continue to define one ordinary Choreo and its public model/admission rules. The framework's projected plans and internal proof/checking machinery derive role knowledge and message obligations; developers do not hand-maintain a second Quint/TLA/ Recife description of the physical router.

A multi-node evidence path relates original `:communicate` source, checked endpoint projection, per-role execution state, trusted sender binding, issued routed envelope, physical delivery attempt, current authenticated recipient admission, and the exact participant receipt/host effect. Each edge is individually qualified. The current portable realization harness can exercise deliberate message drop/reorder and role-local behavior, but a deterministic in-memory queue is not a cross-host authenticity, confidentiality, delivery or resource-ownership proof.

The selected direct-message adapter can reuse infrastructure chosen for Live advisories, but distinct queues, subjects, admission/disclosure rules, acknowledgments and failure semantics remain explicit. The app's ordinary durable Request-convergence guarantee must not depend on a participant typing indicator arriving.

Multi-node assumptions and acceptance

The root assurance graph includes actual host replacement, backend qualification, writer coverage, command retention, outbox recovery, dispatch acknowledgment, interest registration, replay/gap detection, and current authorization contracts. It also includes old/new assembly compatibility and browser artifact availability across rolling activation. Merely sharing a database or load balancer establishes none of these joins.

Source-to-physical acceptance evidence

Whole-application assurance names the original operations, relevant writers, participant messages, hosts, browser resources, routes, installed asset versions, exact admission profiles, and physical backend/transport identities. A successful isolated send/receive projection test or one load-balanced HTTP request does not close this application scope.

A required positive case traverses the **actual** installed producer, transport and consumer path, with two independently running processes where cross-host behavior is claimed. Its adversarial companion challenges the first enforcing boundary for the specific promise, and records independent observable effect/receipt state. A diagnostic report containing matching fabricated host labels or supplied expected outcomes is not physical correspondence evidence. The suite must record source-linked observed, tested, assumed, inconclusive and refuted portions separately.

The multi-node acceptance suite uses real process and transport boundaries and must include:

- mutation through one host becoming authoritatively observable through another;
- compatible retries through a different host, including concurrent deliveries;
- loss before submission, after possible submission, after known commit, and after publication but before dispatch acknowledgment;
- duplicate and reordered advisories with preserved scoped debt;
- initial-subscription and reconnect gaps, including a silent missed notice;
- expired replay retention and explicit recovery without false currentness;
- remote creation and lifecycle change remaining unadopted until permitted Refresh, with own-command exceptions exercised separately;
- insufficient progression at the replacement host without false discharge;
- Membership or scope loss affecting reads, command-result disclosure, and subscription renewal;
- concurrent independent request contexts without capability or identity leakage;
- incompatible plan or assembly replacement following the declared retirement and recovery path;
- eventual successful recovery once the named availability, permission, retention, and user-action premises hold.

Paired distributed and direct-message failure controls

Positive boundary case	Adversarial or forbidden outcome
One authorized sender's typing reaches a current admitted recipient	Forged sender, recipient or Document selector cannot disclose or in
Choreo send and receive correlate within one execution	Same event label in a different execution cannot satisfy receipt
Best-effort typing message displays under current owner	Message for a retired browser resource cannot mutate its replaceme
One scoped delivery ID is admitted once under its contract	Transport duplicate cannot manufacture a second semantic receipt
Recipient's current membership permits the indicator	Revocation before current receipt gate cannot silently retain admiss
Supported receiver-local expiration discards late typing	Spoofed remote wall clock cannot prove freshness across hosts
Declared best-effort loss leaves no claimed recipient receipt	Broker enqueue alone cannot count as delivered or rendered
Qualified stronger durable-message profile supports required replay	Expired or absent replay evidence cannot be called guaranteed rece
Committed Request on A is acquired by permitted read on C	Message routing cannot become a second authoritative Request ow
Same bound command resolves after host replacement	Replayed HTTP attempt cannot create a second committed domain
Durable outbox recovers a crash after authoritative commit	Dead worker cannot permanently strand required publication inten
Interest registration fence/replay closes initial gap	Healthy router with stale filters cannot silently drop advisory cover
Remote HumanHelp update waits for explicit permitted Refresh	Typing, SSE or advisory arrival cannot auto-adopt remote board tru
Lagging C read preserves scoped outstanding debt	Echoed progression token cannot certify a current coherent snapsh
Mixed old/new compatible plan versions follow checked admission	Unqualified wire/asset/assembly generation cannot silently inherit
Concurrent independent requests preserve principal/permit isolation	One request's writer, client, receipt or command result cannot leak
Selected real host fault is observed under declared topology	Two in-JVM labels or untriggered crash cannot prove failover
Liveness under named surviving topology and user action is met	Partition, denied disclosure or no user Refresh cannot be promised

This matrix supplements the multi-node inventory below and the 10.16–10.18 contracts. Positive cases prevent a reject-all implementation from appearing safe, while fault-injection and trace evidence show where the physical system first departed from admitted semantics. An unexecuted test is an open obligation, not certified coverage.

The report separates safety during interruption from progress once those premises hold. If current authorization no longer permits result disclosure, the suite expects nondisclosure rather than assuming eventual command attribution. If explicit Refresh never occurs, the remote-adoption liveness claim is not enabled.

Persisting arbitrary suspended choreography machines remains a separate facility with persistence, migration, ownership, and recovery contracts. Ordinary long waits use durable model state and later short choreographies. Multi-node support does not require converting those waits into serialized process stacks.

Formal correspondence and system-boundary invariant

A claim applies to production only through its stated representation, checking, or refinement relation and the continued applicability of its premises. The executable semantic core, proof kernel, dedicated checkers, external formal checker, physical gateways, and deployed entrypoints have distinct obligations. Source-derived coverage, observed execution, trusted backend behavior, and universal theorems are not interchangeable evidence.

Full host-language implementations remain available through explicit contracts. Finite checking remains finite unless a justified abstraction transfers it. Physical guards and writes enforce current authority; certificates do not. Known commits remain commits despite delivery loss, while participants preserve unknown outcomes until permitted authoritative evidence resolves them.

Every relevant backend or implementation change triggers the appropriate qualification and correspondence checks. Multi-node placement changes hosts, not authority, command meaning, resource ownership, or HumanHelp's adoption policy. The root report preserves the evidence class, scope, assumptions, applicable identities, enforcing boundary, and unresolved obligations of each claim.

Interpret the invariant as a claim-specific transfer judgment

The closing rule is conditional and typed: *claim C about source subject S applies to installed production subject P only when each required interpretation, original-source coverage, admissible proof, implementation refinement, runtime enforcement and environment assumption for that exact C,S,P has been established under the selected scope*. If a mandatory edge is unknown or refuted, the abstract theorem may still be true; its corresponding production or completion claim remains conditional, unsupported or rejected by the selected strict profile. A matching name, digest, code shape or HTTP response is never a substitute for the missing edge.

Claim edge	What must actually be established
Original source to obligation	All relevant declared initializers, operations, business invariants, actors, writers and product requiren
Source to semantic/formal subject	Versioned normalization, interpretation, abstraction and proof dependencies preserve the precise sou
Proof or finite result	Genuine issuer/checker run, exact quantifiers/bounds, source premises and an admissible derivation c
Semantic to runtime transition	Directional refinement with stated abstraction, visible events, stuttering/failures, source/effect and m
Runtime to protected authority	Installed authentic principal, guarded atomic writer, trusted read/disclosure and physical effect-time a
Physical provider and topology	Qualified selected XTDB/backend, codecs, actual installed handlers/writers, host/broker/asset identiti
Client acquisition and adoption	Current scope/permission, genuinely sufficient provider basis, active resource and completed browser
Release and application acceptance	Every required product capability, independently selected policy, positive/adversarial test and deploy

This is an **obligation map**, not a prescribed public map API or evidence that the listed checks have passed. Original-source completeness is judged against the source's full declared requirements, not the smaller catalog returned by an elaborator or a conveniently successful subset of tests. A required obligation omitted from the assembled application remains owed. An externally authored proof language is optional internal framework machinery; normal application authors continue to use Gesso/Clojure declarations.

Evidence remains precise under composition and change

Evidence classes must remain distinct at the root: proved or checker-admitted conditional theorem, exact-finite result, tested correspondence, physically observed enforcement, runtime assumption, application assertion, unsupported, unknown, stale and refuted are not interchangeable. A derived proof inherits every relevant admitted premise, trusted inference rule, external checker assumption, backend condition and coverage gap. The assurance graph does not promote open premises to proved merely because many other nodes passed. A copyable report, signed-looking payload, source fingerprint or issued abstract certificate does not authenticate the current installed server or replace its live authorization checks.

For a safety invariant, the claimed quantifier and transition universe include the necessary initialization, legal steps, relevant conflicts and writer/dependency closure. A real execution may conform through a declared directional abstraction with hidden local steps; byte-identical state traces are not automatically necessary. Conversely, finite exploration, a sampled physical schedule or one causal-looking readback cannot be called an unbounded proof of transaction or browser refinement. Noninterference of optional observation sidecars is not evidence that mandatory runtime guards were applied.

Liveness is separate. Any eventual-command, publication, convergence, participant-message or HumanHelp adoption conclusion names availability, fairness, durable retention, connectedness, current authorization, replay/gap recovery and the **user's explicit Refresh where required**. Safety remains required during supported partitions and delivery uncertainty without pretending the liveness antecedents hold. A known committed effect cannot be relabeled rejected when its response, notice, readback or diagnostic fails. An indeterminate command remains unknown until authorized command-specific evidence settles it; current Request state does not retrospectively identify the successful command.

A relevant source, invariant, backend implementation, physical writer, proof assumption, compiler, browser asset, assembly generation, authenticated context, transport, retention or deployment topology change invalidates or requalifies exactly the dependent claims. An independently checked compatible replacement can retain valid abstract theorems; an unchanged byte digest cannot preserve a claim whose semantic dependency or live boundary has changed. Failure to **observe** an external change is a named assurance limitation, not proof that it did not occur.

Proof hardening and multi-node experiments may proceed iteratively

Complete multi-node productionization is **not** a prerequisite for strengthening the executable semantic core, its proof obligations or source-relative theorems. Conversely, a small real two-process experiment may expose an unstated transaction, observation, routing, identity or failure assumption before the corresponding formalization is considered mature.

The supported feedback loop is: admit the current versioned semantic source; check its obligations; exercise a bounded real deployment; independently classify observed counterexamples and unsupported physical edges; repair the actual source semantics,

implementation or assumption contract under its proper owner; regenerate and rerun the dependent checks. A finding in the physical realization must not be concealed by weakening a formally true requirement, and a changed semantic rule must not inherit an incompatible old theorem. Small physical tests remain tested evidence, never proof of all deployments. The semantic authority is never derived from the host conducting the experiment.

This iterative development permission does not collapse the four separate release milestones below or bypass Part XI's selected candidate, installation and activation requirements.

Four independent completion milestones

A release or progress report must never claim one milestone solely because a different one succeeded:

Milestone	Positive scope
Operational application success	The supported real product capabilities and user-visible workflows work under named deployme
Compositional application certification	The admitted application source, required assembly/contracts, scoped proof obligations and cond
External formal correspondence	The named source-derived formal theorem, its checker/import and necessary representation/refi
Production multi-node completion	The same semantics survive actual host replacement, cross-node command recovery, publication,

A project may legitimately ship a bounded operational profile before the stronger formal or distributed milestone. It must label that smaller achievement and retain its unmet requirements, not relabel the full goal as complete. Conversely, editorial acceptance of this timeless design can occur before these implementation/product evidence gates pass, because the design specifies the obligations without reporting them as facts.

Part XI owns candidate admission, actual installation, activation and milestone release decisions; Part XII owns realization ordering. This final Part X judgment supplies their semantic/physical boundary requirements, not a second release procedure or a replacement for current trusted enforcement.

Closing positive and adversarial correspondence obligations

Positive claim-compatible example	Nearby forbidden inference
A source-complete typed invariant is proved under named abstract premises	Source omission or unchecked opaque writer is silently excl
A genuine closed-kernel derivation is rechecked for the original proposition	A caller-provided proof-looking map or copied report issues
A sound finite model result is reported at its precise domain and bounds	A timeout or enumerated subset is called universal theorem
Qualified physical Claim enforces atomic guards and a complete frame	Handler-only pre-read or successful test map substitutes for
Authenticated current principal invokes a protected operation	A browser ID, stale actor, or certificate substitutes for autho
An installed browser plan executes the admitted effect for its current owner	Retired callback or HTTP 200 is counted as an authorized co
Compatible historical command resolves under current disclosure	Lost response triggers duplicate mutation or current state is
Named outbox/registration recovery closes an eligible advisory gap	Router health, SSE write or silence is called complete delive
Bob's explicit Refresh adopts one coherent authorized board	Remote notice auto-adopts lifecycle content or clears incom
A real mixed-host deployment preserves the same source semantics	Two host labels, shared database alone or broker ack prove
A checked compatible artifact preserves exactly its dependent guarantees	Changed Membership, provider or installed assembly inheri
A release reports each of the four separately evidenced milestones	Operational success, an abstract certificate or editorial appr

These controls require independent conformance and physical tests; they do not report that any such test has passed. Formal correctness of one model, operational success of one workflow and a correctly rendered UI can coexist with an unverified or unsafe production boundary.

Part XI — Release and completion

This part defines incremental evidence reuse, release admission, activation, bounded compatibility, and completion. A release claim concerns an identified application, supported scope, actual artifacts, and declared environment. It is not established by a successful build, an empty error log, a large test count, or a report containing a successful status.

Parts I–X define the semantics and enforcing boundaries. This part makes their acceptance obligations concrete without introducing a second application model. Part XII owns realization ordering. Operational usability, compositional certification, external formal correspondence, and production multi-node support remain four separately reportable completion scopes.

The plan is normative. Examples in this part illustrate release mechanisms and hypothetical records; they do not declare the release complete.

Incremental certification, build, and release

The ordinary application constructor performs structural admission and installs the required production gates. A release profile adds its claim-specific proof, qualification, acceptance, and deployment requirements. Omitting an optional stronger proof profile does not disable structural construction, rendered-surface checking, runtime authorization, or artifact correspondence.

A selected stronger profile must be enforced by the actual candidate and installation path. Defining a strict checker or obtaining its report in a test does not establish that production invoked it. A disabled or absent policy cannot be treated as successful evaluation of that policy.

The selected claim is not the generated gate list

The release request is a declaration of **what is being claimed**, not a summary reconstructed from checks that happened to pass. Its canonical requirements originate in the actual supported application and independently selected product obligations. Construction, elaboration, proof generation, test discovery, and installation **derive** relevant gates from that request; they cannot redefine the requested claim by omitting source operations, writer paths, protected read surfaces, deployment roles, or product journeys that are difficult to establish.

A release candidate has an immutable claim identity during one evaluation. The release's exact subject, qualifying evidence, and intended runtime realization are separately identified: a model/source identity is not a handler implementation identity; an assembly fingerprint is not browser asset bytes; a test fixture's host is not the actual production listener; and a source-checked certificate is not a runtime permission. Reusing a previously admitted result always requires rechecking its declared applicability to these distinct subjects.

The minimum normal construction contract applies regardless of optional stronger milestones. A user may request a bounded operational profile without claiming an external formal proof or multi-node completion. But the ordinary constructor must still check its mandatory whole-application closure, generated/runtime artifact agreement, rendered affordance boundaries and required current-authorization guards. An omitted advanced proof option is not permission to bypass a necessary server/XTDB gate.

Declare the claim before evaluating its gates

A release intent identifies at least:

- the application and canonical requirement-set identity;
- the required product capabilities, independently of the exposed-operation list;
- the requested milestone or explicitly narrower supported scope;
- the versioned assurance profile, mandatory judgments, permitted premise classes, and required runtime enforcement;
- the semantic, implementation, artifact, provider, and configuration subjects;
- the intended deployment topology and supported compatibility directions;
- the required test environments, fault scenarios, resource limits, and product decisions;
- the admission, installation, activation, and retirement conditions.

1. Freeze the intended requirement denominator

   The versioned release intent fixes the canonical requirement inventory and which selected application/product capabilities count as fulfilled, excluded, blocked, or unimplemented **before** expensive checking starts. Its denominator includes obligations that an incomplete implementation fails to discover; discovery is itself an independently checked coverage relation against the original declarations. A discovered empty set of proof tasks, zero failing tests or zero registered business operations is not evidence that the canonical requirement inventory is complete.

   An intentional narrow release is a new identified claim. It names the excluded capabilities and evidence families, why the narrower claim is valid for its supported users and environment, and which original milestones remain unmet. A missing requirement is never relabeled not-applicable merely because its handler is absent. A profile may explicitly accept a trusted premise under Part IX's rules; this is not equivalent to silently deleting a mandatory theorem or physical qualification from the requested milestone.

2. Bind the same candidate across its evidence edges

   A current candidate manifest binds the versioned source/requirement set, normalized semantic and proof-subject identities, admitted operation and application assembly generations, implementation/configuration identities, compiler/codec versions, actual emitted browser artifacts, chosen provider and topology, required runtime gates, and each mandatory test/proof/qualification witness. The manifest is an obligation/identity record, not an invented universal certificate. Physical installation and activation still need independent observations of the actual running handler, worker, writer, served bytes and credentials.

Evidence is applicable only if its checked subject and dependency closure match the **current** candidate within the selected relation. Exact source equality, structural semantic equality, binary artifact identity, implementation conformance and installed host correspondence are different checks. A sound abstract invariant can remain reusable after a compatible implementation change even while previous physical and browser tests become stale. A copied successful report or a digest asserted by the subject under test does not establish these relationships.

The release evaluator must report the origin of every mandatory gate from canonical requirements or admitted application construction, including an explicitly justified dynamic/opaque premise when a static catalog is incomplete. It must record unavailable, unsupported, incomplete and stale required evidence **without** converting it into success. The exact decision vocabulary and distinction between admitted, installed, activated and completed are owned by 11.02, not redefined here.

The release gate derives the concrete obligations from that intent and the canonical application construction. It joins them to the original obligation ledger, rather than replacing the ledger with a list of successful checks. Required capabilities omitted from the implementation remain required.

Changing the requirement set, narrowing a proof domain, removing a capability, or weakening an assurance profile is an explicit change of release intent. It produces a distinguishable claim with its exclusions visible. It cannot make the original full-completion claim pass by shrinking its denominator during gate execution.

3. A concrete scope-bound counterexample

   Suppose the full HumanHelp release still requires a separately selected response-flow interaction, but the candidate demonstrates only Create, Claim and explicit remote-board Refresh. Those features can be valuable positive evidence for an appropriately named partial operational slice. They cannot produce a completed **full HumanHelp** requirement set by deleting response-flow from the discovered feature list. Similarly, a proof of the Claim abstract transition cannot be presented as evidence for all model operations, installed XTDB writers, real-browser behavior and independent multi-node recovery.

   Conversely, a fully specified response-flow product requirement that has not yet been implemented is a missing application capability, not an unsupported proof checker; a checker incapable of evaluating one declared proof judgment is a distinct machinery gap. The release diagnostics and repair owner must reflect the real deficiency rather than steering an application author to rewrite sound product requirements around a tool.

   A missing product decision is different from unsupported framework machinery. Either may block the requested claim, but diagnostics identify the actual owner.

4. Nonvacuous release-intent challenges

Positive candidate	Forbidden alternative
Complete canonical requirement set produces its source-derived gate inventory	Empty or incomplete discovered operation set is d
Named narrow profile explicitly excludes unimplemented journeys	Missing feature is silently omitted while full comp
Current semantic proof is admitted for its checked proposition	Proof-looking report, old digest or mismatched de
Real source/assembly/artifact identities match the qualifying witnesses	Changed physical handler or served JS reuses stale
Actual protected runtime route enforces current admission	Disabled gate or never-installed wrapper is reporte
Original model and application requirements yield all required negative/positive tests	Reject-all or never-run test suite is used as readine
HumanHelp remote changes retain explicit Refresh and protected read policy	Successful SSE send is reported as canonical brows
Selected stricter profile is genuinely evaluated before candidate admission	Strict failure is silently downgraded to a weaker pr
Required backend assumption is qualified within its declared version/scope	Caller-claimed or expired provider status is treated
Accepted operational scope names its four-milestone exclusions	One green build is called formal and multi-node co

Each row is a prospective acceptance obligation; it does not claim that any release has passed it. The real positive case traverses the named construction and (where claimed) physical boundary, preventing a reject-everything implementation from passing a negative-only gate.

HumanHelp's selected remote-adoption policy is already part of Part VIII's requirements. Its response-flow contract remains a product decision until it is substantively selected and tested.

Four release decisions, with different evidence

The release process distinguishes these decisions:

Decision	What must be established	What i
Candidate admission	The constructed semantic/build candidate satisfies its profile and applicable pre-installation gates	That a
Installation eligibility	Actual private host bindings, providers, workers, assets, and enforcement satisfy the installation contract	That al
Activation eligibility	The installation is current, exposure and overlap are permitted, and transition prerequisites hold	The su
Completion claim	Every required capability and evidence gate for the named milestone is satisfied for its scope	Compl

These are architectural decisions, not a requirement for four public API calls. An implementation may combine phases while preserving their obligations.

The four decisions are subject-specific judgments

These decisions cannot be represented by a single ‘:ready‘ bit. The release intent and source-derived requirement ledger of 11.01 identify what was requested before gates execute. Every later phase binds its **own current** subject, evidence, qualified premises and first enforcing boundary.

Decision	Current subject / required observation
Candidate admission	Full source/capability ledger, admitted semantic assembly and required proof/build/asset obligations for the chosen
Installation eligibility	That exact candidate actually bound to this host's Biff handlers, protected writers, provider, worker, credential, asse
Activation eligibility	Current installed generation, actual traffic exposure, overlap and session/command/continuation compatibility, reti
Completion claim	Original requirement set, named supported milestone, real product capabilities, independent acceptance and exact

A future implementation may combine phases in one procedure, but it preserves individual decision subjects, acquired observations, failure results and invalidation conditions. A candidate is not an installation credential; an installation is not a traffic-activation receipt; an activated generation is not a proof that all later requests, real users or full product journeys succeed.

The release ledger retains the exact current normalized assembly, checker/profile and artifact identities where required. It never substitutes an application name, timestamp, Git commit, browser-reported digest or caller-provided host label for an independently qualified subject. A staging test proves only its specified installed staging environment unless a separate checked transfer justifies more.

A candidate can be exercised in a controlled integration environment only after that environment's own admission and installation gates hold. Testing an admitted staging installation is not the same as admitting the final production host. The final host establishes its environment-specific bindings before exposing managed behavior. This avoids both circular acceptance rules and a practice of exposing unchecked application paths merely to see whether they work.

Gate results are closed, explicit, and scoped

The release-report vocabulary distinguishes the following results. An adapter may translate a tool's native statuses into these meanings, but it must not turn an inconclusive result into success.

Result	Meaning	Satisfies a required gat
`:passed`	The required check completed successfully for the applicable subject and scope	Yes, after evidence and
`:failed`	The check established the specified violation or unsuccessful outcome	No
`:incomplete`	Work began but did not establish the required result, including timeout or exhausted exploration	No
`:unavailable`	The required checker, environment, dependency, or resource could not be used	No
`:unsupported`	The mechanism cannot establish the requested judgment for this input or scope	No
`:skipped`	The check was intentionally not executed	No
`:stale`	Existing evidence no longer applies to the current subject or premises	No
`:missing`	No required result or resolvable evidence exists	No

These are release-check results, not model outcomes or evidence classes. `:passed` may describe a test, a kernel check, a qualification run, or a proof check; the record still names which mechanism ran. A passing test does not become a proof because its status shares the same word.

A gate result is not self-authenticating evidence

For every required gate, the report identifies its canonical originating requirement, exact subject, selected mechanism and version, expected proposition and scope, independent evidence/issuer provenance, observed result, dependency applicability and open premises. A map containing ‘:passed‘, a successful test count, an unchecked digest or a copied prior report cannot prove the required checker executed or that the deployed system used its result.

A proof gate needs an actually admitted derivation bound to the current original source and its assumptions. A physical effect gate needs evidence that the actual installed writer, serving handler, provider and artifact were challenged and remain the selected installation. A browser result needs the current resource's completed effect, not an HTTP 200 or attempted SSE send. A required runtime authorization mechanism must be installed at the protected action boundary; a release report cannot preapprove every later principal.

A successful negative test is a positive **test result** only when the specified invalid input reached its required first boundary and produced the expected refusal without forbidden effects. It does not make the tested invalid command an admitted business operation. A later failure of diagnostics after possible XTDB submission cannot be converted into business rejection or permission to resubmit.

An expected rejection passes only a negative test whose input, first enforcing boundary, and expected diagnostic or prohibited effect were specified. The candidate rejected by that test is not thereby admitted. A test that succeeds because an unrelated wrapper rejected earlier does not establish enforcement by the intended boundary.

Closed result vocabulary and complete mandatory inventory

The eight result values above describe what a named checker or test established. They do not themselves decide whether a requirement applies. Gate applicability is derived independently from the canonical source requirements and approved release intent; each required gate must have an accounted result. A missing original-source obligation cannot disappear because the generated gate inventory or the discovered operation list is incomplete.

Unexecuted required work is ‘:missing‘ or ‘:skipped‘ as appropriate, unavailable providers are ‘:unavailable‘, an unsupported checking capability is ‘:unsupported‘, and a bounded or timed-out incomplete exploration is ‘:incomplete‘. None becomes ‘:passed‘ merely because no counterexample was observed. A real failed check remains ‘:failed‘; result provenance is retained so an unsupported checker, missing business rule, failing guard and infrastructure fault have distinct repair owners.

A selected profile may explicitly exclude a gate only through its authorized applicability rules and an independently identifiable narrower claim. An exclusion is a separate scoped decision, **not** a ninth successful checker result. Unknown applicability, undiscovered privileged writers, an unselected required product decision or unqualified source completeness blocks the stronger release claim. A full HumanHelp completion claim cannot be preserved by dropping response-flow or another unmet original capability after evaluating its checks.

A gate can be excluded only through the declared scope and applicability rules. It is recorded as excluded with a reason and requirement reference, not as a passing check. A gate mandatory for the canonical milestone cannot be excluded while retaining that milestone's complete status. Unknown applicability blocks the relevant decision until resolved.

Mandatory obligations and accepted premises are not one counter

The profile decides which obligations require checked derivations, which require runtime mechanisms, which require qualification or test evidence, and which may remain explicit trusted or asserted premises. A conditional admission can be valid while displaying such accepted premises.

The report therefore separates:

- mandatory unmet obligations, which must be empty for the requested decision;
- accepted premises with their exact proposition, scope, owner, and dependent claims;
- runtime obligations, whose enforcing mechanisms must be installed and whose concrete conditions are evaluated when required;
- unsupported, stale, or refuted claims;
- exclusions and narrower scopes;
- original source obligations and the evidence relationships that account for their current disposition.

Exact conditionality and fail-closed mandatory gates

The preselected profile separates mandatory derivations and tests, current runtime enforcement, physical provider qualifications, and specifically permitted trusted or asserted premises. An accepted premise remains in the report with its proposition, provenance, scope, qualifying observation, affected claims and invalidation triggers. An assumption known false or requiring unperformed qualification cannot be called successfully accepted simply because an abstract theorem was checked.

A required proof obligation cannot be moved into the accepted-premise column after its checker fails. A current source-relative conditional theorem can remain valid even when physical installed-host eligibility is unknown, but it does **not** establish that eligibility. Physical XTDB contention tests do not themselves establish independent proof kernel soundness; a closed theorem does not authenticate a production client. Report each evidence class and its current applicability, not an aggregate success percentage.

If the selected strict profile requires a proof, qualified backend, live guard or installed handler, missing, stale, unsupported, incomplete, unavailable or failed evidence withholds that specific release decision. The trusted product owner can request a **different**, expressly narrower release intent; the original request never silently downgrades. After potentially committed work, missing diagnostic evidence cannot undo the effect or turn ‘:commit-unknown‘ into rejection. Mandatory pre-effect safety gates fail closed for new prohibited operations instead.

Required positive/adversarial release decision controls

Positive control	Invalid shortcut to reject
Source-complete candidate checks every mandatory gate	Omitted Claim/response-flow operation disappears from ge
Staging candidate executes valid positive test at admitted staging host	Staging success authorizes an unrelated production listener
Real installed route, writer, provider and served JS match accepted candidate	Checked Biff map or copied artifact receipt is called installe
Active traffic uses a compatible installed generation and recovery context	Old incompatible host/session remains exposed after install
Named full completion includes all required user workflows	Green Claim demo is called all HumanHelp, formal and mul
Invalid action is refused at the required physical gate	A test-only early wrapper refusal is counted as physical cov
Real authorized operation succeeds at the same checked gateway	Reject-all handler passes negative-only acceptance
Exact finite state graph is completely checked within its stated scope	Timeout or truncated search is promoted to universal proof
Approved preview preserves explicit excluded full-milestone debt	Failed strict gate is reclassified as optional after it fails
Current qualified backend premise remains visible and scoped	Stale or known-false atomicity premise is treated as proof
Lost post-submit response retains unknown settlement	Observer exception is reported as rejection and triggers nev
Changed installed handler/provider/artifact requalifies dependent decisions	Old copied successful release report remains perpetual auth

These are physical and semantic acceptance obligations. Written cases do not establish that the corresponding checks have been executed or passed. Part X 10.15 specifies their independent positive/adversarial observation ladder.

“Zero open obligations” is not a substitute for this accounting. It can hide the fact that an implementation premise was merely accepted, or demand an impossible proof of all external systems. Mandatory obligations cannot be moved into the premise list because a proof failed. An accepted backend premise still needs its required qualification and must not be known false.

A current principal, valid command binding, or permission to mutate a resource cannot be frozen as a build-time fact. Release evidence establishes the proper runtime mechanism and its scope, not the truth of every future invocation's premises.

Deterministic regeneration and identity layers

For unchanged complete declared inputs, the normalized semantic construction must reproduce the identities it claims deterministic: semantic subjects, required dependency closure, supported projections and plans, assembly interfaces, and browser manifest. Relevant normalizer, rule, checker, codec, compiler, and format versions enter the appropriate dependencies.

The identities answer different questions:

Identity or relation	Question answered
Semantic subject identity	Is this the same normalized meaning in the supported interpretation?
Required dependency closure	Which exact subjects and premises support this claim?
Implementation identity	Which concrete callable, lowering, or provider realization is bound?
Exact artifact identity	Which actual bytes were built, tested, stored, or served?
Assembly generation	Which immutable admitted whole-application realization owns this invocation?
Continuation compatibility	May this client or execution safely use that target realization?
Activation record	Which admitted realization was exposed in this environment and transition?

Source paths, line numbers, Git commits, timestamps, and human version labels are useful provenance. They are not substitutes for those meanings. Identical source text under a changed primitive or compiler may not preserve the relevant claim. Conversely, a comment edit need not change a semantic subject.

Semantic reproducibility does not imply byte-identical compiler output. Exact reproducible builds additionally control every byte-affecting input. Otherwise the build records and verifies the actual emitted bytes separately. Timestamp or diagnostic exclusions from an identity must be explicit and justified.

1. Determinism has a declared semantic boundary

   Identity is computed from the **complete meaning-bearing input** for the specific subject and interpretation, not merely its displayed name or serialization. The versioned normalizer must declare which fields, primitive definitions, imported contracts, schema types, feature/profile settings and admissible external assumptions affect the relevant meaning. Equivalent ordering of supported unordered maps/sets should not introduce accidental drift; relevant sequence order, literal distinctions and whitespace inside semantic strings must not be erased.

   The identity contract also distinguishes a syntactically normalized interface from denotational equivalence: a structural digest is not a proof that arbitrary Clojure code, a database lowering or two different implementations behave identically.

Changes to interpretation, trusted primitive rules or normalization semantics require versioned reconsideration even when a human-readable rule or identifier looks unchanged.

Compiler, formatter, build toolchain and runtime dependency changes are tracked by the evidence and artifact edges that depend on them. A comment or source location edit that is excluded from the semantic representation need not invalidate a theorem, but it may still affect provenance, diagnostics or an actual byte-producing compiler input. The build either controls every byte-affecting input for reproducible bytes or measures the bytes it **actually** emitted and qualifies their current binding. A timestamp-sensitive artifact cannot inherit a previous byte identity from a matching semantic subject alone.

The identity and dependency ledgers must be compatible with the immutable release intent of 11.01 and the different candidate, installation, activation and completion subjects of 11.02. None of those decisions is represented by one global monotonic release number or a Git commit alone.

A changed dependency requires reconsideration of the affected artifacts; it does not guarantee that every artifact's bytes change. A server-side policy change may leave an executable browser payload unchanged if its complete relevant contract and dependencies remain satisfied. A new manifest or receipt is still required wherever its binding changes. Reuse follows checked facts, not an assumption that every source edit either rebuilds everything or changes nothing.

Incremental invalidation follows actual dependency edges

The dependency index is derived from admitted source and the required claim closure. It includes semantic dependencies and the separate implementation, provider, configuration, build, and qualification dependencies needed by each claim. A structural graph supplied by the caller cannot establish its own completeness.

1. Dependency closure includes the boundaries that can invalidate a claim

   The dependency index must be derived from the canonical admitted application and the required original-source obligation catalog, including invariant, operation, role/knowledge, model read/write, guard, plan, projection, rendered affordance, application assembly, proof/checker/profile and client artifact contracts. Where relevant, the dependent evidence graph also includes the actual bound implementation, physical writer/entrypoint, provider, compiler, configuration, credentials, host installation, activated generation and compatibility/retention assumptions.

   These are not assumed to be one uniform graph. A semantic contract-dependency edge, proof-dependency edge, implementation correspondence edge, build-input edge, physical qualification edge and active-installation edge can have different recheck policies. The checker must establish every required edge of the declared graph; that proves **declared** graph consistency only. Independent closure obligations still govern original source completeness and undiscovered physical writers. A caller-supplied dependency map cannot close its own missing-edge obligation merely because it is acyclic.

   A renamed, removed or newly discovered required subject is a change to the corresponding obligation/closure, not an excuse to retain downstream success because an old identifier disappeared from the graph. Unresolved references, contradictory dependencies, and cycles that cannot be admitted under the selected graph contract are typed construction/release failures rather than silently ignored edges.

   A dynamic permission, current Membership, request status, user input or command outcome should not be frozen as an immutable build dependency. The release dependency is on the **installed mechanism** which checks those facts at the authoritative read/write boundary; actual invocation must still check the current values. A pure build graph cannot replace runtime authorization or liveness.

   When a dependency changes, reconsider its transitive dependents. Retain a result only when the current consumer establishes its applicability, including profile and interpretation compatibility. Rechecking a join may show that an unchanged interface theorem remains reusable while physical evidence must be renewed. The system need not regenerate unrelated subjects because they share a file or Git commit.

   Invalidation records distinguish these actions:

   - reconsider a claim whose dependency changed;
   - reconstruct a semantic subject or executable artifact where required;
   - recheck evidence or its applicability;
   - rerun a concrete qualification or acceptance case;
   - reacquire installation evidence for a changed physical environment;
   - retain an immutable result with the reason its applicability survives.

   They must not collapse all of these into “rebuild” or imply that a retained abstract proof establishes unchanged production behavior.

2. Reconsideration is not synonymous with recompilation

   A changed input expands the candidate **reconsideration cone**, not necessarily the set of artifacts whose bytes must change. For each affected subject, the current consumer determines which action applies:

Action	Required reason
Retain an immutable semantic conclusion	Current source subject, checker/rule contract, original obligations, dependency clos
Reissue or recheck semantic proof evidence	The relevant source, inference/checker, proof dependency, profile or interpretation
Rebuild a plan or browser artifact	Its executable contract or byte-affecting build inputs changed, or reuse cannot be es
Revalidate concrete implementation/XTDB behavior	A handler, transaction lowering, real writer, provider or related physical assumptio
Recheck installation/activation	Exact served assets, registered routes, live host/worker/credential binding or overla
Reacquire product/test evidence	The required capability, execution subject, test harness or assurance scope changed

   An implementation can optimize this by retaining immutable checked results under the cache-admission rules of 11.04. It may not skip required current validation based solely on an old success map. An unavailable required recheck remains unavailable, not a pass. Retrying a read-only proof query is not the same as resubmitting an uncertain committed business command.

3. Executable illustration: the affected dependency closure

   This pure Clojure example computes a reconsideration set over a supplied finite graph. The graph uses subject labels, not cryptographic identities. It assumes that its nodes and edges have already been validated and that the graph is the complete relevant dependency index. It does not check those assumptions, issue a certificate, or decide which reconsidered result can ultimately be reused.

```
(def release-dependencies
  {:claim-rule #{}
   :model-schema #{}
   :claim-contract #{:claim-rule :model-schema}
   :claim-proof #{:claim-contract}
   :claim-adapter #{}
   :claim-conformance #{:claim-contract :claim-adapter}
   :claim-plan #{:claim-contract}
   :search-projection #{}
   :search-plan #{:search-projection}
   :browser-manifest #{:claim-plan :search-plan}
   :application-root #{:claim-proof :claim-conformance :browser-manifest}})

(defn affected-subjects [dependencies changed]
  (loop [affected changed]
    (let [expanded
          (into affected
                (for [[subject inputs] dependencies
                      :when (some #(contains? affected %) inputs)]
                  subject))]
      (if (= affected expanded)
        affected
        (recur expanded)))))

(def policy-change
  (affected-subjects release-dependencies #{:claim-rule}))

(def adapter-change
  (affected-subjects release-dependencies #{:claim-adapter}))

(assert (= #{:claim-rule :claim-contract :claim-proof
             :claim-conformance :claim-plan :browser-manifest
             :application-root}
           policy-change))
(assert (= #{:claim-adapter :claim-conformance :application-root}
```

```
           adapter-change))
(assert (not (contains? policy-change :search-plan)))
(assert (not (contains? adapter-change :claim-proof)))
```

In this graph, the policy change reaches the browser manifest, while an adapter change reaches physical conformance and the application root without invalidating the abstract Claim proof. A real manifest that also binds the adapter would add that edge and therefore be reconsidered. A search implementation that relies on the changed policy would likewise add the dependency. The example does not justify omitting either edge when it exists.

For the illustrative graph, the two assertions test only reachability in a finite, supplied dependency map. A real implementation must independently show that `release-dependencies` was derived from the complete relevant original source and that every implementation, source, browser and environment edge was discovered. The helper eligibility change must also supply a typed, authoritative `:helper/not-suspended` input; merely adding an unvalidated keyword to a set is not an admitted rule.

The example's ‘:claim-adapter‘ is deliberately distinct from ‘:claim-contract‘. Its implementation replacement reconsiders conformance and application assembly while permitting the original abstract proof to survive only after the current checker verifies the unchanged proof subject and dependencies. If the real browser manifest binds adapter identity, the manifest must depend on it: the illustrative absence of that edge cannot justify omission from the production closure.

Reconsidering the manifest does not force recompilation if the actual required payload and build binding can be reused under the current contract. The example computes impact, not an optimal build schedule or a proof of cache validity.

4. Positive and adversarial invalidation controls

Positive control	Prohibited inference or regression
Two equivalent normalized maps reproduce one supported structural subject ID	Unordered input iteration changes
Relevant string, vector-order or primitive-meaning change produces a requalified subject	Normalizer erases a semantic distin
Diagnostic-only edit retains abstract proof where its subject remains the same	Actual byte-affecting build change
Claim eligibility policy change reconsiders Claim proof/plan and root	The unrelated Request search plan
Claim adapter A is replaced by B under the same checked abstract contract	Old A-only physical conformance r
Current proof checker re-admits an unchanged semantic subject and complete dependencies	A copied digest, subject label or ca
Browser manifest changes trigger exact artifact and receipt correspondence checks	Changed manifest is treated as equ
New privileged XTDB writer is discovered and added to physical coverage obligations	Old complete declared semantic gr
Deleted or renamed required capability remains an unresolved original obligation until accounted for	Removed graph node silently disap
Incompatible graph cycle or missing required edge rejects admission with a typed error	Traversal terminates and is mistak
Installation provider, credential or activated host changes requalify affected live claims	Unchanged semantic fingerprint pe
Current Membership is checked at Claim effect time by installed authorization	Build-time cached Membership or a
Optional unrelated file edit retains unaffected evidence when current admissibility is shown	Entire proof graph is regenerated s
Expected unsupported/unavailable proof recheck preserves the original required gate as unsatisfied	Release system silently downgrade

These are design acceptance obligations, not reports of executed tests. They require independently chosen positive and counterexample cases, actual source-derived coverage and physical validation at the boundaries claimed by the selected release profile.

Cache admission preserves the original trust boundary

A persisted cache entry is candidate evidence. Its key, filename, recorded digest, and successful status cannot confer issuance. The trusted consumer reconstructs or validates the current subject and required closure, then applies the relevant source, derivation, assumption, interpretation, and issuer rules from Part IX.

The supported reuse paths are:

Reuse path	Required validation
Immutable in-process issued result	Genuine issuer binding, unchanged subject and required closure, current applicability
Serialized proof/evidence package	Safe decoding, complete original source/proof material, current dependency and profile checks, re
External formal evidence	The supported import/checking contract, exact subject representation, proof/checker identity, and
Previous concrete test or qualification	Exact tested subject/configuration or a justified coverage-preserving relation, with remaining envi
Previous installation record	Only the installation facts it still establishes; a new host or changed binding acquires its own evide

1. Cache lookup and proof admission are different events

   A cache lookup yields candidate evidence, not a checker-issued proof. The trusted consumer constructs its expected subject from the **original current** Gesso source and full declared obligation/dependency closure; it cannot accept a serialized package's own claim about that closure as its sole authority. The present checker contract, required inference rules, proof profile, assumptions and precise proposition determine whether reuse is admissible. A matching cache key, checksum, success flag or subject name is only a lookup/correlation aid.

   A genuine immutable issuer-registered, in-process result can avoid repeated expensive derivation while the current consumer independently establishes its continued applicability. Persisted serialized proof material has crossed a new trust boundary and requires the supported current source/proof checker or external proof-import contract before it can be issued again. The system need not rerun unrelated immutable proofs on every HTTP request, but cannot replace a mandatory current check with a trusted-looking deserialized map.

2. Worked counterexample: a green cache file is not a certificate

   These two maps are **schematic comparison data**, not new callable Gesso APIs or evidence that a checker was executed:

```
(def expected-claim
  {:subject :operation/request-claim-013
   :requires #{:rule/claim-eligible-R8 :schema/request-S9}
   :checker :checker/operation-v7
   :profile :profile/compositional-v3})

(def persisted-candidate
  {:cache/key [:request/claim :profile/compositional-v3]
   :subject :operation/request-claim-012
   :requires #{:rule/claim-eligible-R7 :schema/request-S9}
   :checker :checker/operation-v6
   :status :passed
   :derivation-hash "71ad..."})
```

   The candidate fails current re-admission for **separate reasons**: subject O12 does not match O13, declared rule R7 is stale relative to R8, checker v6 is not the required v7, and the digest is not the proof derivation or genuine live issuer binding. A fail-fast implementation may report the first problem, but cannot infer the rest were satisfied.

   Even copying O13, R8 and v7 into a forged green file does not supply an original source derivation or prove its premises and checking rules. The trusted consumer must derive its expected inputs independently. Corrupted proof bytes can be rejected or reconstructed from admissible originals, never promoted to successful certification.

   An opaque JVM handle is process-local. Printing or serializing it cannot make it a durable certificate. Copying a report into a new map cannot recover its issuer or the origin of its physical observations. The cache must not accept executable reader forms, arbitrary callbacks, or unbounded structures merely because they arrive under an evidence filename; its parser and resource contract are explicit.

3. Closed serialization and observation provenance

   Serialized evidence has a versioned passive-data format, bounded size and depth, permitted reference syntax, supported checker/import versions and strict resource limits. Cache loading cannot evaluate executable EDN reader forms, arbitrary

callbacks, user code, unbounded collections or unknown extensions. Unsupported, truncated and unreadable packages fail with typed diagnostics. A safe cache miss is permission to reconstruct from trusted original input, **not** to ignore a required proof or a known counterexample.

Test and physical-qualification receipts have their own trust rules. A copyable report, checksum or signed-looking record cannot independently prove that a designated test actually ran on an identified physical XTDB provider, browser build, served host or privileged writer. If a selected release profile requires independently attributable execution or current installation, the evidence issuer, tested subject, environment, bounds, method and qualification dependencies must be established at the original boundary. A signature can authenticate a defined statement but cannot make an incomplete statement about writer coverage exhaustive.

Persisted evidence never grants live authority. A cached proof is not a current Membership check, retained command result, principal, permission to disclose, DOM resource-owner capability or permission to submit another mutation. Real protected operations retain their effect-time authorization and post-submit unknown-outcome recovery obligations.

A corrupted or incomplete cache entry may fail admission or trigger fresh reconstruction from trusted inputs. It cannot relax the original check. A reconstruction failure leaves the required gate unsatisfied. Recovering from a cache miss is different from suppressing a counterexample or a known invalid premise.

An unchanged immutable result can avoid repeating expensive work under an admitted cache contract. This does not require reparsing an unchanged proof on every request. It also does not cache current Membership, command disposition, live resource ownership, or permission to disclose a prior result as permanent facts.

4. Proof retention, physical requalification, and cache freshness

   The cache contract must distinguish a proof's **current abstract subject** from a particular Clojure handler, XTDB lowering, compiler, provider, deployment host, issued browser plan and actual served asset. A change to any applicable physical dependency requalifies the corresponding physical claim without automatically invalidating an independently checked unchanged abstract theorem. Matching semantic identity is not sufficient until the current checker validates the source meaning, proof inputs, dependency closure, interpretation, profile and premise scope.

   Cache re-admission does not collapse 11.01's original requirement catalog, 11.02's four release decisions, or 11.03's different reconsideration actions into a permanent green release flag. A previously successful staging report cannot qualify a different production installation. An obsolete artifact receipt cannot qualify newly emitted browser bytes. Unavailable current qualification leaves the required release decision unmet, while unaffected evidence can remain useful under its actual checked applicability.

5. Unchanged contract, changed implementation

   Suppose the normalized Claim contract and proof remain unchanged while its XTDB adapter changes from implementation A to implementation B. The release must separate the following questions:

Item	Reuse decision
Abstract Claim theorem	Reusable if its actual subject, dependencies, interpretation, and profile still apply
A's transaction-lowering correspondence	Does not establish B's correspondence
A's backend qualification	Reusable only for the portion whose complete applicability envelope B preserves; changed ada
Handler and gateway binding	Re-establish for the actual issued B realization and installed host
Browser plan and bytes	Reconsider their actual dependencies; rebuild only where needed
End-to-end Claim evidence	Establish applicability to B; an A-only execution cannot be silently relabeled
Whole-application root	Recheck the joins and report B's remaining premises

   Implementation identity is acquired from the actual bound construction and its relevant dependencies. Assigning a new label in a map does not establish which handler the server executes. A successful source-relative join does not prove complete physical writer coverage or the final serving process's adoption. Those remain the separate production obligations defined in Parts VII and X.

6. Worked implementation contrast: unchanged Claim, changed adapter

   The A44-to-B45 adapter counterexample can be stated without pretending that a public ‘release/reconsider-implementation‘ or ‘certificate/admit-cached‘ API already exists:

Evidence	A44-to-B45 decision
Current abstract Claim theorem	Retain only if the issuer/checker re-establishes its original normalized sul
Original A44 XTDB lowering and guard/frame correspondence	Not evidence of B45's different physical behavior; check B45 independent
Backend isolation, codec and transaction qualification	Retain only the scope demonstrably preserved; requalify changed provide
Installed Ring handler and privileged writer inventory	Acquire evidence of B45's actual bound callable and complete relevant ho
Browser plan, manifest, build receipt and emitted bytes	Reconsider actual dependencies and verify current stamp/byte correspon
End-to-end Claim test	Verify validity on B45 and re-execute any required physical case not cove
Root release and active host claim	Rejoin current conditional proof, B45 implementation/installed host, exac

The useful optimization is preserving a genuine theorem independently of the handler refactor, not retaining A44-only conformance as proof of B45. Conversely, it is wrong to throw away unrelated Request-search proofs or browser plans just because both implementations share a repository commit.

7. Positive and adversarial cache-admission controls

Positive case	Prohibited shortcut or counterexample
Current live issuer handle is bound to its original proof and dependencies	Caller-created handle or copied EDN report becomes a certif
Original O13/R8/v7 proof rejects stale O12/R7/v6 cache candidate	Green filename, digest or ‘:passed‘ flag bypasses validation
Forged package with new labels lacks checkable original derivation	Merely copying a current semantic ID manufactures checker
Closed bounded data package safely decodes under selected version	EDN reader execution or arbitrary callback runs at cache loa
Corrupt stored proof fails or reconstructs from trusted originals	Truncated cache turns mandatory release gate into success
Genuine reusable theorem retains explicit current assumptions	Known-false backend premise becomes discharged through
Qualified physical receipt applies only to its observed subject	Staging test log alone certifies the production deployment
Changed B45 actual transaction requalifies physical conformance	A44's successful XTDB test is relabeled B45
New installed handler and writer closure are qualified at the host	Caller-supplied B45 name authenticates the serving process
Current browser manifest, receipt and actual built bytes agree	Stale JS bundle is reused because semantic Claim name matc
Current Membership and command scope are checked at effect time	Proof cache freezes authorization or resolves unknown comm
Lost command response remains unknown pending historical evidence	Post-submit diagnostic/cache failure becomes rejection and r
Unrelated Request search remains admitted under unchanged dependencies	A Claim-only refactor forces global semantic rebuild
Unavailable mandatory strict re-admission blocks exact profile	Weaker operational success silently substitutes for demande

These are future positive and adversarial acceptance obligations, not recorded test results. Physical claims require the real first enforcing boundary and supported passing operations, not only a test-only wrapper or a reject-all implementation.

Root release evidence and candidate integrity

The root report has machine-readable data and a human-readable explanation of the same records. It is generated from checked construction and observed evidence; it is not a second hand-maintained application registry.

Required report fields

The report identifies:

1. Release intent, canonical requirements, requested milestone/scope, assurance profile, and independent required-capability inventory.
2. Normalized subjects, required dependency closure, original obligation ledger, accepted premises, mandatory unmet obligations, and excluded claims.
3. Actual implementation bindings, source/build provenance, provider/configuration identities, compiler/runtime versions, and qualification applicability.
4. Issued plans, browser manifest, build receipt, exact artifacts, and the admitted assembly generation to which they belong.
5. Every required gate's result, evidence class, evidence reference, subject, environment, bounds, and enforcing boundary where relevant.
6. Candidate, installation, activation, compatibility, and retirement decisions, including the actual serving-host evidence for the claim made.
7. Product decisions, resource limits, retention contracts, security boundaries, and conditional-liveness premises.
8. Completion results for each requested milestone and a change-impact explanation relative to any reused evidence.

The root report carries a typed claim/evidence ledger

A root report has a stable, versioned representation whose human-readable explanation is generated from the **same checked underlying records** as its machine-readable form. It does not reconstruct source or release policy from the checks that happened to pass, and a user-edited human explanation cannot update the checked decision. The report is about a specific named release intent, original requirement denominator, candidate, physical environment, evaluation window and claimed milestone; it is not a global ‘certified‘ flag on an application name.

Each required claim and capability has a source-derived original requirement reference and a current disposition: which checked proposition or physical effect is asserted, what evidence class can support it, who issued or observed that evidence, what exact subject/configuration was examined, and what the selected profile requires. A required obligation absent from the emitted evidence ledger remains **missing**; an admitted narrower scope names the excluded original requirement and its approval. The ledger preserves unknown, failed, unavailable, unsupported, incomplete, stale and intentionally skipped cases according to 11.02 rather than erasing them from the denominator.

Report dimension	Must distinguish and bind
Original source and coverage	Declared business rules, models, operations, invariants, initial states, roles, protected writers, rende
Semantic proofs and issue authority	Exact theorem subject, proof family, original derivation/checker source, proof kernel and dedicated
Assembly and actual implementation	Normalized construction and issued assembly generation, real guarded operation, route, installed h
Browser and build	Plan/manifest, generated bootstrap source, compiler/toolchain/format versions, exact emitted JS ide
Real provider and environment	Selected XTDB/backend profile, transaction lowering and codec, provider configuration/version, ho
Gate observations	Required gate ID and expected boundary, actual test/checker type and result, provenance and execu
Release decisions and rollout	Separately identified candidate admission, specific installation eligibility, exposure/activation and r
Product and milestone	Independently selected product policy, required HumanHelp workflows, positive user-capability ev

The report also classifies the relevant **trusted computing base** (TCB) by responsibility: source semantics and normalization, proof/checker, compiler/build, authenticated application/XTDB writer, installed host, browser/transport and external environment. For each dependent claim, it states which pieces have actual checked evidence, which are explicit trusted premises and which are still unsupported or unknown. A backend premise may be accepted under a selected profile but cannot be presented as an independently proved universal property.

A useful evidence graph contains edges from the original obligation to the admitted semantic/proof subject, concrete implementation, selected artifact, observed deployed gateway and resulting milestone decision. Such edges describe the **specific correspondence evidence** and its limits. Their presence in a report does not establish the edge's truth: the trusted consumer obtains issued proof or physical observation from the owning boundary, then validates the resulting reference and current applicability before any release decision.

Identity and provenance are claim-specific

The canonical release subject from 11.01 remains immutable while evidence is collected. Semantic/proof identities, dependency closure, issuer identity, implementation identity, exact bytes, assembly generation, installation and public activation are distinct coordinates. A Git commit, timestamp, source filename, shared operation name, matching digest or supplied report status cannot substitute for any missing coordinate.

A test made against a fixture assembly, old server implementation or staging host may be valuable evidence **for that stated scope**. It cannot be reused as production evidence merely because the current candidate has the same logical operation name. Concrete test-to-actual-installation correspondence requires an expressly checked applicability relation, including the compiler, emitted artifacts, provider, runtime and resource boundaries required by the claimed behavior.

A built artifact's receipt binds the selected generated bytes to its declared manifest under the build contract. It does not by itself prove that the deployed listener serves those bytes, that a user browser loaded them, or that the browser completed a permitted DOM effect. Installation and activation evidence from the actual serving environment belongs to the separate 11.02 decisions and Part VII's physically enforced lifecycle.

An immutable historical release evidence package is useful for reproducibility, comparison and audit. When source, checker, policy, provider, credential, served artifact, active host or retention condition changes, the report must distinguish historical facts from **currently applicable** evidence. A current release decision reacquires or revalidates exactly the dependent observations required by 11.03–11.04, rather than editing an old successful report to appear current.

Root reports are descriptions, not new authority

The reporting process consumes genuine checked construction, proof-issuer and observation interfaces. A caller-supplied map, test list, original source graph, physical route inventory, reported issuer handle or manually assembled root report is neither a second application registry nor a release authority. The selected trusted owner must obtain the actual source and installed observations from their appropriate boundaries, with unknown physical enumerator completeness recorded as an open obligation rather than assumed away.

The human and machine presentations may arrange information differently, but must represent the same claims, evidence classes, failed or unrun gates, accepted premises, exclusions, enforcing boundaries, and limitations. A concise summary can link to checked detailed evidence, but may not call a weaker status ‘certified‘ or omit a blocking gate. The executable source, proof package and physical installation—not the printed report—remain the authoritative inputs for the corresponding decisions.

Proof/diagnostic values must not be fed into runtime semantic transition decisions, browser effect authorization, Membership evaluation or XTDB guarded writes. Genuine proof of a conditional abstract proposition does not convert it into current physical permission. Runtime guards enforce current authority even when release reports are unreadable or unavailable.

HumanHelp and release-report challenge cases

Valid root-report statement	Forbidden promotion c
The original full HumanHelp capability catalog is named, including unselected response-flow work	The successful Create/
Claim theorem is admitted under its precise original source, kernel version and remaining premises	A copied proof report l
Selected Biff/XTDB handler and complete current writer inventory have their own physical evidence or open gap	A source-derived route
Exact browser manifest, tested built JS bytes and actually served artifact have distinct references	A valid build receipt is
Fixture/browser staging positive and negative tests retain their precise tested subject and environment	A passing staging suite
A named required physical qualification is stale or unavailable and remains blocking	A current human summ
A strict profile lists accepted source assumptions and separately unmet physical obligations	Accepted backend prer
Four release decisions and four completion milestones have separate status/subjects	One green root record
A supported remote HumanHelp lifecycle advisory awaits explicit Refresh and current protected read	An SSE send is labeled
Changed provider or serving handler triggers requalification of its dependent release claim	An old timestamp/dige
A known committed command remains committed after delivery/diagnostic failure	Missing release report
Positive valid Claim succeeds under the actual protected gateway, and forbidden attempts are rejected	A reject-everything ha

These are independent report and physical conformance requirements, not evidence that the tests have run or that any release is certified. The small aggregation illustration in 11.06 intentionally checks no such issuance or physical evidence: its output remains downstream explanatory data.

Every reference must resolve to evidence of the stated subject and scope. A passing fixture test, an old browser bundle, a supplied route inventory, and an actual installed handler are different evidence. The report must preserve the origin and limits identified in Part X rather than combining their green labels into an unsupported production claim.

The root report is explanatory. An admitted candidate or installation uses the genuine checked construction result or supported revalidation contract. It does not trust a caller-edited `:passed` or `:eligible` field. The report never becomes an authorization capability for Claim or a semantic input to browser transitions.

Executable illustration: aggregate status is not evidence admission

The following pure function illustrates only the final aggregation of an already validated, nonempty required-gate set and gate-result records. It does not validate proof evidence, authorize deployment, or replace the profile's source-derived gate construction. Its output explains blockers to a caller.

```
(def release-result-kinds
  #{:passed :failed :incomplete :unavailable :unsupported
    :skipped :stale :missing})

(defn summarize-required-gates [required results]
  (when-not (and (set? required) (seq required) (map? results))
    (throw (ex-info "Expected a nonempty required-gate set and result map."
                    {:reason :invalid-gate-input})))
  (let [blockers
        (into {}
              (keep (fn [gate]
                      (let [present? (contains? results gate)
                            row (get results gate)
                            status (if present? (:status row) :missing)]
                        (cond
                          (not (contains? release-result-kinds status))
                          [gate :invalid-result]

                          (= :passed status) nil

                          :else [gate status]))))
              required)]
    {:summary (if (empty? blockers) :requirements-passed :blocked)
     :blockers blockers}))

(def small-required-set #{:assembly-closure :real-claim :response-contract})
```

```
(def incomplete-results
  {:assembly-closure {:status :passed}
   :real-claim {:status :passed}})

(assert (= {:response-contract :missing}
           (:blockers (summarize-required-gates
                       small-required-set incomplete-results))))

(assert (= :blocked
           (:summary
            (summarize-required-gates
             small-required-set
             (assoc incomplete-results
                    :response-contract {:status :skipped})))))

(assert (= :requirements-passed
           (:summary
            (summarize-required-gates
             small-required-set
             (assoc incomplete-results
                    :response-contract {:status :passed})))))
```

The example's green result is deliberately **non-authoritative**. Two additional pure tests reveal why: (1) a caller can fabricate passing rows without any proof or physical test; (2) the supplied set may be nonempty but omit the actual original requirements. Both produce a ‘:requirements-passed‘ summary even though neither demonstrates admission of the intended release.

```
;; This looks green solely because the caller supplied green rows.
(def invented-green-rows
  (zipmap small-required-set
          (repeat {:status :passed})))

(assert (= :requirements-passed
           (:summary
            (summarize-required-gates
             small-required-set invented-green-rows))))

;; A nonempty subset need not be the complete canonical required set.
(assert (= :requirements-passed
           (:summary
            (summarize-required-gates
             #{:assembly-closure} incomplete-results))))

;; Present malformed rows are distinct from genuinely missing rows.
(assert (= {:response-contract :invalid-result}
           (:blockers
            (summarize-required-gates
             small-required-set
             (assoc incomplete-results
                    :response-contract {:status :unknown})))))

;; Every recognized nonpassing check result remains blocking.
(assert
 (every?
  (fn [status]
    (= {:response-contract status}
       (:blockers
        (summarize-required-gates
         small-required-set
         (assoc incomplete-results
                :response-contract {:status status})))))
  [:failed :incomplete :unavailable :unsupported
   :skipped :stale :missing]))
```

These assertions are **negative illustrations**, not assertions that Gesso has verified any actual release. Their purpose is to show that the same pure helper can report green when its inputs are forged or incomplete. The helper cannot learn the missing requirement from the map it is given; preventing that failure belongs to independent source/requirement closure and current proof/physical evidence validation from 11.01–11.05, not to another success flag in its output.

An arbitrary caller can fabricate the example's `:passed` rows. That is precisely why aggregation is downstream of evidence validation and is not an issuer. The real gate checks each result's evidence, subject, profile, scope, and applicability before it reaches this stage. An extra passing row for an unrelated capability cannot satisfy a missing required gate.

The real required set is derived from the requested canonical scope and must not be replaced with the three-gate illustration. Rejection of an empty set is a small guard against vacuous aggregation, not a proof that the set is complete.

1. Check status, applicability, and authority are not interchangeable

   The example's eight result kinds classify the **outcome of a named required check**. They are not command outcomes, evidence types, exclusions, or issued release decisions. A timed-out exploration is ‘:incomplete‘, not a successful proof from seeing no counterexample. Unavailable or unsupported mandatory machinery blocks the selected claim; the original strict profile cannot be silently downgraded to keep its green label.

   An independently authorized **narrower release intent** may exclude a particular requirement only with its own scope, original requirement reference, responsible decision and visible remaining milestone debt. Exclusion is a separate applicability result, not a ninth check outcome or a passing check. Unknown applicability remains unresolved.

   An expected refusal is a positive result only for a specified **negative test**: it must observe the intended first enforcing boundary and absence of the prohibited physical effect. It is not authorization of that invalid command, nor does a reject-all

implementation meet the ordinary positive operation requirements. Similarly, failure to retrieve release diagnostics after a potentially committed Request mutation does not change its domain outcome to ‘:rejected‘ or authorize duplicate submission.

The release root of 11.05 preserves original requirement coverage, source-bound derivations, proof assumptions, actual test/XTDB and installed Biff/browser evidence, and the candidate/install/activate/complete distinctions. A passing aggregate can be displayed only as a **summary of independently validated evidence under one declared claim**. It cannot produce the evidence or make an incomplete source inventory complete. Even a genuine historical aggregate is not permanent evidence that a changed host or provider remains qualified.

2. Required positive and adversarial acceptance pairs

Positive case	Counterexample to reject
Entire original requirement inventory drives the mandatory gate set	Nonempty reduced set omits failing HumanHelp response
Admitted original proof/checker and real test evidence bind each passing row	All-passing caller-made status map becomes a certificate
Missing required gate remains ‘:missing‘	Absent original requirement disappears from denominator
Malformed present row is reported ‘:invalid-result‘ by sample	Unknown status treated as a passing result
All seven nonpassing statuses block a required gate	Timeout, unsupported checker or stale result is counted as
Positive supported Claim operation succeeds at actual effect boundary	Reject-all handler passes solely negative tests
Defined invalid Claim reaches its designated protecting boundary	Earlier unrelated test wrapper rejection counts as actual pl
Approved preview exposes original full-plan exclusions	A small operational slice is called full HumanHelp complet
Named abstract theorem and accepted backend premises stay distinct	Source proof success claims physical writer and transactio
Candidate, installation, activation and completion retain separate evidence	Pure aggregate green status activates a live host
Real served JS/XTDB evidence is checked for exact current subjects	Copied fixture success authorizes production
Commit-unknown remains pending authorized historical reconciliation	Missing report is read as rejection and triggers duplicate w

These are prospective test requirements, not evidence that the specified conformance tests have run. The executable assertions above test only the arithmetic of a user-supplied finite map, not live proof, release admission, or production behavior.

The tested object must be the activated object

A release acts on the admitted candidate built from the application's original source and selected assurance profile. Candidate identity binds the admitted declarations, operations, source and dependency closure, actual prepared server realizations, route/decoder and provider selections, proof and runtime gates, generated plans, browser manifest, and exact emitted artifacts. The binding is immutable for the claim being checked. A mutable registry, later Var redefinition, stale cached report, or plausible matching name may not replace an admitted component between checking and exposure.

The four judgments in 11.02 apply to different subjects. Candidate admission checks the declared assembly and constructed artifact under its selected profile; installation checks the actual host-bound handler, routes, providers, writers, workers, and asset-serving relationships; activation authorizes specified externally reachable service under a current installation; completion evaluates the separately declared product and assurance milestone. Evidence for one judgment does not issue the next. These judgments may be implemented by one coordinated constructor, but its claimed observations and enforcement boundaries remain distinguishable. A displayed root report is explanatory evidence, not a transferable activation capability.

Establish the real candidate and installation relation

The build binds the generated bootstrap and BrowserAssemblyManifest to the source-derived application assembly. Artifact verification checks the actual bytes and the issuing build receipt, including the relevant compiler and byte-affecting inputs, rather than trusting a filename, embedded stamp, timestamp, digest supplied by the candidate, or a successful status map. A cryptographic digest under its declared assumptions supports byte identity; it does not prove compiler correctness, runtime semantics, or deployment adoption. A selected toolchain remains a named qualification premise unless its correspondence is established by stronger evidence.

The installation then establishes that the exact checked realization is actually selected for serving. It accounts for the final route dispatcher and protected execution gateway, the guarded Ring/Biff handler actually consumed by the serving host, actual writer and provider bindings, background entrypoints, response and render paths, and the exact browser assets made available to clients. A system map containing a correct `:biff.ring/handler` does not prove that the running listener invokes it; a build receipt does not prove that a browser received those bytes. Later route override, unchecked raw Ring HTML, service-worker cache, reverse-proxy rewriting and alternative server entrypoints must either be covered by the qualified installation or remain explicit, potentially blocking, deployment premises.

The selected environment has its own qualified dependencies. Staging results can be reused for a production release only along an explicit checked transfer: the semantic/implementation subject must still match, while production credentials, backend incarnation, endpoint, installed handlers, served assets, provider qualification and topology receive their required current evidence. A successful staging request, a manifest copied to a new host, or equivalent Git source does not by itself establish production installation.

The release gate retains the original predeclared requirement denominator, including required real product paths and privileged writers. A candidate cannot become eligible by failing to discover an operation, excluding a mandatory physical path, changing the requested profile after failure, or relabeling a trusted premise as proof. Conversely, a reusable abstract theorem may remain valid across a qualified implementation change while the changed adapter, emitted asset and installed deployment obtain fresh conformance evidence. These are separate dependency and applicability judgments.

Coherent activation and failure handling

Installation may privately allocate dependencies before activation. The selected lifecycle nevertheless fences every externally effecting application path until its applicable checks are complete. This applies to listeners, HTMX entrypoints, scheduled and background writers, Live publishers, stream callbacks, and managed browser asset delivery. An init phase that opens a listener or starts a worker before a later preflight start step must prevent unadmitted protected behavior at that earlier boundary. Separately qualified health or unavailable responses are allowed; a purported health path must not invoke unchecked Claim, reveal protected content, or submit a privileged write. Private preparation that itself needs a durable effect has its own independently admitted effect contract.

For a selected installation scope, activation chooses one coherent new-entry assembly and its serving relationships. The activated binding contains route selection, actual executors, providers, policy and render checks, guarded output, resource ownership, asset references and the chosen compatibility policy. Publishing these as independent mutable replacements can expose a combination that no candidate admitted. A compare-and-activate transition, a fenced quiescent handoff, or an equivalently checked host mechanism prevents a candidate admitted against an older predecessor from silently superseding a newer active choice. The comparison includes the actual expected active identity and its applicability, not just a coincident version number or a status value in caller-controlled EDN.

The activation authority has an explicit **scope**. One host may switch an immutable dispatch composite without making the entire load-balanced deployment globally atomic. Hosts serving different admitted generations during a rolling release must satisfy the declared placement and directional continuation contract in #req-bounded-release-compatibility; the existence of one successful local switch establishes no cross-host or browser guarantee. A new active host or new provider incarnation cannot inherit a historical installation qualification solely because the operation's semantic ID is unchanged.

An invocation captures the admitted assembly and applicable concrete execution/response context when it enters. A completed Ring return, the end of a lexical scope, or a completed request handler call is not necessarily the end of a future, stream, deferred response, subscription or transferred job. These lifetimes own the necessary generation and resource references through completion, cancellation or an explicitly checked transfer; they do not consult a mutable global lookup for whichever executor, provider or render guard is newest. New invocations use the activated generation. Older work may continue only while its own binding, live authority and declared overlap permission remain valid.

An assembly reference is not a perpetual security grant. Authentication, Membership, source-of-truth model guards, current backend qualification, credential validity, resource incarnation, and effect-time permission are checked at the appropriate current trusted boundary. Retirement or revoked premises fence new protected effects even if an old request still owns memory or a callback retains a handler. Physical cleanup alone is not a substitute for the revocation of semantic execution rights.

Candidate failure before activation leaves the previous assembly exposed **only if the previous assembly still satisfies its own current conditions**. A failed replacement cannot justify continued unsafe service after a known backend or security premise has been revoked. The declared response suspends the affected operation or enters a separately admitted restricted mode; unaffected work may continue under a demonstrably valid scope. Failure after partial physical setup accounts for listeners, workers, credentials, files, publication and any irreversible effects actually acquired or emitted, even when no initialization callback returned successfully.

Cleanup retires the failed candidate's owned resources in dependency-safe order without stopping a surviving generation's shared provider or abandoning admitted transferred durable work. The primary failure remains visible; secondary cleanup failures, leaked resources and required operator actions are separately recorded. An unsuccessful socket close or stale worker stop does not restore permission to make an effect. A pre-activation effect that already escaped the intended fence is an incident requiring authoritative reconciliation, not a clean installation rejection.

A gate failure or response loss after an authoritative command may have been submitted never manufactures `:rejected`. It cannot undo an incorporated transaction or clear committed publication debt. The same stable logical command identity, original binding, and supported historical reconciliation path govern a possibly committed command; a later transport attempt does not silently

create another domain operation. Part IV owns command outcomes and #req-application-lifecycle owns physical lifetime; release activation must preserve both contracts.

Rollback is a new checked activation

Reactivating an older admitted assembly is a **new** installation/activation judgment at the current time, not a reversal of the prior activation receipt. The rollback candidate must again match its actual bytes, handler and provider bindings; pass the currently selected assurance and security gates; and qualify every old/new client-server direction that routing can actually expose. An earlier passing test, historical release report or previous host permit is not perpetual permission. The comparison/fencing rule also applies when an older artifact is intentionally selected as a rollback target.

Rollback does not reverse durable Request state, accepted command records, transaction history, outbox obligations, previously disclosed information, or already installed browser effects. The selected older implementation must be safe for the state, retained work and clients it can actually encounter. Schema/state migrations are authoritative writers in the proof and deployment inventory, even when executed once by an administrative tool. Their admission names the migration's original source owner, preconditions, protected write footprint, invariants, transaction/compensation semantics, qualified backend, and observable committed version or state boundary.

Rollback eligibility is assessed **after observing the actual migration and writer state**. It includes forward/backward data compatibility, concurrent older/newer writers, pending durable work and outbox entries, stable command result decoding and retention, old asset references, client/server envelopes, current authorization, and the resulting invariants. Schema compatibility cannot be inferred from an unchanged API route or a label such as `:backward-compatible`. A compatibility promise may permit an old reader but not an old writer; the admitted deployment must enforce that distinction.

An irreversible migration, a changed mandatory authorization rule, or an unrecoverable command/result format may prohibit activating the old writer. In that case the safe recovery is a qualified forward repair, compatible read-only reconciliation path, or another separately admitted transition; it is not an unchecked code rollback. No universal migration engine or automatic reversal of business effects is required. A retained historical command lookup can remain authorized even when executing its original mutation implementation is no longer safe.

Release activation and rollback acceptance histories

The following are required **positive/adversarial pairs** for any release claim whose profile includes these boundaries. Successful construction alone does not count as physically installed or traffic-active evidence. Actual host adoption, forbidden-effect absence and live client behavior require independent observations at the named boundary.

Valid history	Nearby invalid history to reject or
Qualified candidate's exact browser bytes are served by its selected host	Staging receipt or copied digest is
Actual guarded Biff handler is consumed by the serving listener	Correct system-map handler is by
Private init prepares required resources behind a real effect fence	Init starts a raw XTDB worker or
Both an eligible positive Claim and an unauthorized Claim exercise the real installed gateway	A reject-all wrapper passes only n
Candidate B activates with routes, providers, guards and assets from one admitted generation	New Claim route is exposed with i
B activates after verifying the expected active generation A	Late candidate C, prepared against
Existing A stream completes under its retained authorized context or checked transfer	A delayed callback consults B's ne
Rejected B leaves still-qualified A serving and retires only B-owned resources	Failed B destroys A's shared provi
Revoked A premise fences protected effects while safe restricted service remains available	Rejected B is used as a pretext to k
Partial initialization failure accounts for a socket or publication already created	No returned init value is mistaken
Cleanup failure records the original cause and secondary unretired resource	A cleanup exception masks the ad
Claim commits under A, response is lost, and B resolves retained command evidence	Deployment marks the unknown c
Valid rollback target decodes actual migrated state and retained outcomes under qualified writers	Older code is reactivated based on
Forward-only schema change uses a checked forward repair or restricted reconciliation route	Unrecoverable old writer is activat
Current owner/authorization is rechecked for an A-origin continuation after B activation	Historical assembly, proof or Mem
Allowed mixed-host and sleeping-browser continuations satisfy the declared overlap/retention profile	Local host activation is presented

These cases are design obligations, not automatically passing tests of an implementation. Part VII owns the managed lifecycle and genuine installed handler, Part X the physical/semantic transfer and trusted boundaries, 11.08–11.10 the full directional compatibility and retirement contract, and this section the release decision joining the **actual tested candidate** to the **actual exposed and, if selected, reactivated realization**.

Bounded rolling-release compatibility

A rolling release preserves **one admitted application semantics** while several physical realizations, artifacts, hosts or invocations may remain alive at once. Compatibility is a checked, potentially **directional** relation between a particular originating client/continuation and an admitted target server realization for a specified operation and protocol. It is not a generic property of an application name, a Git branch, or two nearby build numbers. A successful compatibility judgment preserves only its named semantic and physical obligations under its declared premises; it neither grants present authority nor proves that an unobserved deployment uses the checked artifacts.

Name the four distinct release coordinates

A release decision distinguishes four related but non-interchangeable coordinates—three identities/realizations and one directional relation:

- the source-derived semantic identity of the operation, projection or plan, including the conditions that make its continuation meaningful;
- the exact emitted client/server artifact and protocol realization, including actual route/decoder/encoder and browser runtime bindings;
- the admitted application assembly and its concrete installed/activated generation at each applicable host and deployment scope;
- the directional continuation/adaptation relation between an origin and a target for the operation in question.

A semantic ID may be unchanged while generated bytes, provider behavior, trusted writer, decoder, installation or security qualification changes. Conversely, two different generations may support a checked continuation relation. The fourth coordinate is a relation, not another version identity. Neither identity equality nor inequality proves that relation without the relevant source, physical and current-premise evidence.

The normal baseline is a **finite, declared** overlap between the current activated assembly and a selected predecessor. Another finite window may be admitted, but it must name the actual eligible assemblies, origins, target installations, supported operations, duration or retirement condition, and recovery policy. `N` and `N+1` may illustrate activation order; their arithmetic is not an assembly identity and does not itself mean compatible. A third activation does not magically make every previously served page disappear or its command uncertainty resolve. The declared retirement boundary must have an enforcing and recoverable meaning.

Development may build and admit many candidates without activating them. The relevant predecessor is determined by the selected activation history and declared rollout topology, not by earlier commit time, candidate creation order, or a numerical version label. A candidate that was never activated is not silently treated as a predecessor serving real clients. Conversely, a still-reachable older host or sleeping browser cannot be omitted merely because a newer release was admitted and activated elsewhere.

Check every direction the real topology can expose

For a two-assembly overlap A (selected predecessor) and B (selected current assembly), the release gate accounts for all four pairings. ”Permitted” always means **qualified for the specified operation**, not unrestricted permission to invoke arbitrary routes.

Originating client or execution	Target server realization	Required release judgment
A client/plan	A server	Continued actual installation validity and supported lifetime of A/A, or enforc
A client/plan	B server	Separate old-to-new semantic/protocol compatibility, a checked directional ad
B client/plan	A server	Separate new-to-old compatibility, or a physically enforced routing exclusion
B client/plan	B server	Current-pair admission, actual installation and active runtime qualification

The pairs are not interchangeable: an A client may issue an envelope that B's decoder accepts while a B client uses a new field that A's decoder cannot safely interpret. An operation-compatible Claim continuation need not imply compatibility for an unrelated board query, admin writer, Live subscription, or browser effect. Even within one pair, protected commands, read-only reconciliation, authorized collection acquisition and advisory delivery may have different supported directions. Record their individual scopes.

The real reachable matrix is derived from selected deployment/routing rules **and** the actual ways clients can reach a host: mixed instances behind a load balancer, sticky-session expiration, retry and redirect, host loss/failover, reconnecting SSE, stale page assets, service workers, CDN/proxy behavior, in-flight requests, delayed async responses and sleeping browser tabs. A routing rule that excludes B-to-A is an enforced, qualified premise, not a conclusion from an ordinary happy-path load-balancer configuration. If failover or a direct endpoint can reach A, that pairing remains reachable unless an actual checked guard excludes it before the protected continuation.

The release records the true endpoint, installed route and decoder, selected source and plan identity, actual client artifact, host generation, required protocol features, enforcement point and evidence for every permitted or excluded direction. A shared URL, matching operation keyword, apparent schema shape, source hash, browser-supplied generation label or caller-edited compatibility map does not establish the selected target or safe execution. A manifest checked at build time does not prove that an already-running host actually serves it.

Admit compatibility or stop before the effect

A permitted cross-generation continuation must preserve the source-derived operation meaning and the selected client/server representation, including its invocation, result and recovery contract. Its current target must have an admitted installation, supported decoder/encoder, available browser/runtime capabilities, and qualified physical execution or read path. A sound complete normalized continuation contract may support conservative semantic equality, but the actual physical and authority joins remain separate checks. A broader directional transformation requires a stated representation/refinement relation and the evidence or explicitly allowed trusted premise appropriate to the selected assurance profile. Section 11.09 owns that contract and the exact asset/endpoint retention required to realize it.

Unsupported or unqualified skew is detected at the **first boundary** where the incompatible continuation could otherwise acquire protected data, perform an authoritative effect, or install a browser-owned result. The available outcomes are: admitted continuation under current premises; checked directional adaptation; semantic retirement with the declared authorized recovery; or typed refusal where that recovery is unavailable. A silent fallback to an ordinary unmarked HTMX request, an old anonymous decoder, discarded semantic metadata or a guessed operation is never an acceptable fifth outcome.

A refusal is not sufficient merely because some unrelated outer wrapper blocks a request in a test. For the claimed enforcement boundary, negative conformance must show the intended guard rejecting the specific mismatched pairing **before** the protected physical effect or disclosure, while a valid compatible pairing succeeds at that same actual gateway. Unsupported work does not become successful simply because the attempted transport returned HTTP 200, a browser callback ran, or an SSE message was queued.

An existing request or stream remains tied to the originating assembly and its admitted resource context during its declared lifetime. A host switch does not authorize it to fetch an arbitrary new executor, outcome mapper, provider, response guard or asset. A checked transfer may deliberately rebind only the dependencies its compatibility proof covers. Current principal, Membership, model authorization, backend qualification and browser resource ownership are not frozen by either the old assembly or a successful release check. A revoked effect permission is enforced even for an otherwise compatible old continuation.

A command submitted under A may have committed even when its response is lost and B is activated. Failing the A-to-B **mutation continuation** gate must not relabel `:commit-unknown` as `:rejected` or authorize a new command ID. A qualified read-only historical reconciliation path may remain usable after the old mutation implementation is retired; its current principal,

original command binding, result decoding and disclosure checks are independent of permission to retry. Section 11.10 owns the retirement/intent recovery contract and preservation of HumanHelp's explicit remote Refresh policy.

Make the overlap an actual release gate

The candidate and activation gates fix the declared reachable pairings before traffic moves. Installation and runtime qualification then establish the exact host bindings, routing exclusions, asset references and compatible protocol implementations on which the decision depends. A topology or artifact change invalidates the affected direction even if the semantic continuation contract is byte-for-byte unchanged. Unavailable or unsupported required cross-generation checks block that requested overlap profile; a separately approved narrower rollout may instead impose independently enforced exclusions. The original requested strict profile is not silently downgraded.

Rolling releases are not globally atomic merely because one host uses a coherent compare-and-activate transition. Mixed-host operation requires its own finite compatible window, exposure/failover coverage and qualified retirement conditions. For a supported direction, any claimed progress depends on stated fairness, availability, retry and retention premises; safety cannot be replaced by an assertion that a client will eventually reconnect. When the promised overlap expires, the old execution rights are retired by actual enforcement and explicitly recoverable intent is reacquired under the current admitted semantics. Neither elapsed wall time nor server request reference count by itself establishes that all sleeping clients have been handled.

Positive and adversarial rolling-direction acceptance

Positive case to establish	Invalid adjacent case that must not pass
A/A invocation stays within A's qualified lifetime	Old A host continues unsafe writes after its provider
A/B old-to-new Claim uses qualified decoder, command binding and effect gateway	B accepts old envelope by stripping unknown seman
B/A direction is separately shown compatible where permitted	Success of A/B is incorrectly taken to prove symmetr
B/A exclusion is enforced through real dispatch, including failover	Sticky session routes normally to B but error fallback
B/B current installation serves the exact checked runtime/decoder	Current B build receipt is used while host serves stal
Operation-scoped migration allows compatible A-origin read	Safe historical lookup is taken as permission to execu
Source-equivalent semantic plan changes physical adapter and is requalified	Same semantic fingerprint inherits obsolete provider
Old A request completes with retained qualified context during B activation	Deferred A body resolves a newer mutable B provide
Required current Membership is checked for a compatible old command	Stale A principal snapshot authorizes protected B eff
New B client receives explicit typed retirement on unreachable A decoder	Incompatible B envelope is silently treated as an ordi
Blocked direction is rejected before real protected XTDB effect	Test-only wrapper rejects early while the production
Real eligible Claim succeeds beside the required negative skew control	Reject-everything installation passes because only ne
C42 committed under A is reconciled under a qualified B read path	Lost response causes invented rejection and submissi
Browser resource is still current before an admitted cross-generation swap	Retired A callback installs into a replaced B DOM res
Reconnect, host loss and sleeping clients satisfy the declared window	Only one happy-path HTTP request is treated as con
A third activation enforces A retirement and permitted recovery	Old A client is silently granted indefinite execution t
HumanHelp remote board remains advisory until user-authorized Refresh	Reconnect to B automatically adopts unrelated remot
Excluded or unsupported strict direction blocks the original release claim	Unsupported checker is reclassified after failure to ke

These are prospective conformance and physical observations, not automatically satisfied release results. Section 11.07 owns the coherent candidate/installation/activation transition. This section owns the **directional reachable-pair decision and its enforcement**. Section 11.09 owns normalized continuation and physical retention; 11.10 owns semantic retirement, unresolved command intent and selective recovery. They all preserve the semantic ownership, conditional proof and real physical-boundary requirements of Parts IV, VI–VIII and X.

Derive a complete continuation contract

The continuation contract is derived from the **same admitted source** that owns the operation, Choreo projection and executable client plan. It is not a second application-authored catalog of convenient version labels. For an originating continuation and a target realization, the contract identifies every dependency whose change can alter the meaning, authority, physical representation or safe completion of the continuation. An opaque dependency is explicitly an assumption, a separately qualified implementation contract or an unsupported boundary; it is never silently omitted from the source-derived inventory because static analysis cannot inspect its Clojure body.

The normalized contract includes, at the applicable scope:

- original operation, role, placement, rule/invariant interfaces and source-derived semantic identity, with the permitted state/transition family and any open proof or backend premises;
- typed argument and presence semantics, canonical logical command ID binding, deduplication, authority scope, result vocabulary, rejection and uncertain-incorporation/reconciliation obligations;
- event, knowledge/provenance and allowed information-flow conditions, including which participant may receive or use each observation;
- progression and acquisition source/basis, query and coverage contract, advisory versus canonical observation, adoption grants and required authorization for collection membership and rendered content;
- selected optimistic resolution, causal outcome ordering, cancellation, response ownership and browser effect/resource-incarnation constraints;
- the specific endpoint, route/method/template, transport envelope, parameter/codec versions, decoder and encoder, outcome mapping, HTML/HTMX semantics and actual emitted browser runtime requirements;
- resource lifetime, transfer and retirement behavior for requests, streams, event subscriptions, deferred callbacks, old assets and workers;
- persistent document/state and durable-work encodings that the target can encounter, including migration/reader/writer compatibility and historical command-record formats;
- provenance of selected providers, installed executors and runtime capabilities, plus the declared applicability and invalidation edges needed to qualify the physical realization.

This is a **closed and source-derived** relevant-dependency interface for the selected operation and continuation, not a demand that all arbitrary host code be proved. If a required dependency or reachable state/representation is absent, the stronger compatibility judgment is open or rejected under the requested profile. Treating incomplete discovery as an empty dependency set would make every plan appear compatible and is forbidden.

Normalize semantic equality without turning hashes into proofs

A conservative compatibility rule compares complete, type-aware normalized continuation contracts for their **semantic** subject and checks all remaining current physical and trusted-premise edges. Normalization has a versioned definition, accounts for defaults, missing-versus-null fields, role/operation identity and dependent rule semantics, and excludes diagnostic-only source metadata only where the source's declared meaning demonstrably does not depend on it. A digest is a compact identifier for the normalized bytes under its hash assumptions, not an independent derivation or current execution capability.

Equality of source names, Git revisions, route strings or a caller-supplied digest does not establish semantic equality; equal semantic contracts need not imply equal emitted JavaScript, transaction adapters, decoder implementation, host installation or permission to disclose historical results. A changed provider, implementation, backend or artifact may preserve the abstract operation theorem but invalidates its dependent physical correspondence until requalified. A changed current Membership does not change the semantic Claim contract, yet may prohibit the particular operation or protected acquisition at effect time.

A more permissive **directional** adapter may translate an originating plan or protocol into a target representation. It specifies its domain of admitted origins, target assemblies, operation and argument/result/state mapping, preservation obligations, error and unknown outcome behavior, and the point at which the actual transformation is enforced. Required refinement or simulation evidence is bound to those exact subjects and assumptions. A selected profile may explicitly accept a qualified trusted implementation premise where permitted; it may not silently replace required checked refinement with a handwritten `:compatible?` flag. Reversing a proved old-to-new adapter does not infer new-to-old compatibility.

The selected release evidence reports which rule admitted each operation-specific direction: exact semantic equality, checked adapter, explicitly permitted trusted premise, or exclusion. It ties that rule to exact origin/target identities, current dependency and implementation qualification, open assumptions and the concrete first enforcing boundary. A proof-looking map, human report or old successful compiler stamp cannot act as an adapter, proof issuer or server/DOM authorization.

Bind actual route, artifact and browser realization

An admitted continuation can be enacted only when its origin's real client artifact and the target's **actually installed** route, decoder, response encoder and supported runtime form a compatible physical pair. The build manifest, compiled bootstrap, artifact receipt, exact built bytes, selected host and actual served/loaded bytes remain distinct evidence subjects. The actual bytes must match their qualified identity; their digest alone does not prove the compiler's behavioral correctness. A retained file at a build path is not an actually reachable versioned browser asset, and a valid system-map `:biff.ring/handler` does not prove which handler a running host invokes.

Versioned asset references are immutable during the promised lifetime, including all byte-affecting dependencies needed to execute their admitted plan. A mutable URL such as one that always serves only the latest JavaScript cannot be relied on for A's client while B is active unless the release establishes an independent checked content/compatibility and cache relation. Proxies, CDNs, service workers, browser caches and alternative entrypoints are part of the stated actual serving/loading qualification; an origin that self-reports a manifest ID does not authenticate what code the client actually runs.

The release policy identifies exactly which admitted server handlers, route selectors, envelope/codecs, result/command decoders, subscription endpoints, provider contracts and browser artifacts must stay available to service each permitted A/A, A/B, B/A or B/B direction from 11.08. It also names the mechanism ensuring they remain actually reachable, not merely present in a deployment folder. The selected code/runtime compatibility envelope is bounded by a versioned admission policy; old packages are not kept executable forever solely because an ancient tab can wake.

A browser returning from sleep, back-forward cache or offline storage is not assumed still current. Before a protected send, authoritative adoption or stale callback effect, its current resource/assembly contract, permitted server pairing and trusted runtime conditions must be established or the old execution is retired. A compatible client plan does not freeze the current actor, Organization Membership, route authorization, protected disclosure, or DOM target ownership.

Specify distinct retention horizons and their owners

The release derives an explicit lifetime and owner for each required resource and separately records its withdrawal rule:

Retained subject	Required lifetime/owner relation
Exact old browser asset and matching dependency bytes	Named supported client execution/overlap window and serving owne
Old installed route, decoder, encoder and compatible executor	Every permitted origin-to-target operation direction during admitted
Live subscriptions and event/reconnect protocol	Declared overlap, replay/fence and current interest/authorization reco
Resource-owned pending browser/request/stream work	Its valid current operation lifetime or checked transfer; retirement ma
Durable command-result and historical decode/reconciliation path	Declared command retention and authorized resolution horizon, pote
Persistent schema, migration and queued durable work formats	All admitted readers/writers and recoverable work that can encounte
Current protected content acquisition and DOM effects	Only the named current knowledge, principal, grant and resource sco

The overlap horizon, asset availability, command-record horizon, authoritative state/format support, replay/registration windows and required recovery horizon need not coincide. They must be jointly satisfiable for the selected release promise. A release cannot promise recovery from an old command while retiring its only permitted decoder or losing its durable retained record, nor can it call an old browser executable after withdrawing the only compatible runtime endpoint.

When a horizon expires, the policy defines the reachable typed refusal, semantic retirement and current authorized reacquisition path. Expiry must be enforced by a current boundary; the absence of an HTTP request, a zero process-local reference count, a successful cache purge or a wall-clock label does not prove that no old browser exists. A sleeping client may be safely **refused** at the next contact without extending its right to perform effects. Physical garbage collection follows the actual eligible rights and retained recovery obligations rather than creating a silent gap.

Command resolution often outlives executable-plan compatibility. A new generation may expose a qualified, protected **read-only** decoder and resolver for an original C42 command record even after rejecting A's old mutation implementation. Resolution validates the bound operation version, canonical arguments and original target, stable command identity, current authorized principal/disclosure and genuine trusted historical evidence; a browser-held ID or a current Request state does not prove C42 incorporated or identify which attempt caused it. Where evidence is absent or outside the declared retention guarantee, the status remains unknown or explicitly blocked; it is not changed to `:rejected` by expiration or a network exception. Compatible retry of C42, where separately admitted, still must use the original binding and deduplication contract rather than silently creating C43. Part IV owns command truth and 11.10 owns recovering the old participant's intent.

Retention must preserve product acquisition policy

Release reconstruction is not a hidden source of extra authorization or an automatic grant to adopt unrelated remote changes. For HumanHelp, remote Request Create and lifecycle changes remain advisory until Bob's explicitly granted Refresh, except for precisely scoped own-command outcome and affected-card recovery. Asset reload, SSE reconnection, new host selection, plan retirement or recovered command status does not discharge unrelated advisory debt or create a new one-use acquisition grant. A newly initialized page may have its independently declared initial-load grant; it must be distinguished from a transparent successor read performed merely because an old asset expired.

Retained local route, filters and an explicitly supported draft can assist current reconstruction, but none retains old authority. A valid current read must cover the required scope/basis, remain discloseable under present policy, and install only into the new resource's authorized target. Out-of-window execution artifacts, stale optimistic rollback callbacks and obsolete plan-private knowledge cannot be restored by copying bytes from storage. Section 11.10 owns the exact retirement, debt and participant-facing recovery dispositions; this section requires the necessary retained physical paths and evidence to make them feasible.

Positive and adversarial continuation/retention controls

Required positive history	Invalid shortcut or counterexample t
Complete source-derived continuation inventory includes a real rule and its opaque dependencies	Missing rule or opaque body yields v
Typed normalization preserves missing/present argument meaning	Nil, missing and defaulted argument
Equal semantic contract requalifies changed adapter and serving host	Unchanged operation digest is used t
Directional old-to-new adapter has bounded source/target refinement and effect rule	Handwritten compatibility flag or re
Valid old plan uses a genuinely installed compatible route and decoder	Matching URL or handler map hides
Actual A browser bundle is served through its promised immutable reference	CDN/service worker returns incomp
Physical byte receipt and qualified serving/loaded evidence bind an actual browser client	Build hash alone asserts compiler ser
Sleeping A tab awakens and passes current continuation/resource admission before protected work	Cached old allowed bit or bfcache ca
All A/B and B/A permitted directions retain needed codecs and endpoints	Old decoder is deleted after last obse
Released old execution rights are refused at the active boundary after overlap	Successful file purge is mistaken for
Current principal and Membership are rechecked for old-origin lookup	Historical command ID, source proof
Genuine C42 historical record resolves via qualified read-only path on B	Readback of current Request is labele
Retained C42 result decode survives longer than old mutation executor	Resolver is deleted with obsolete wri
Absent command evidence preserves honest unknown and authorized repair	Lost record or expired asset becomes
Migrated state admits only qualified old reader/writer combinations	Schema-compatible old read is taken
Live reconnect meets declared authorized fence/replay/interest contract	SSE transport acceptance is labeled c
Remote HumanHelp advisory remains pending through asset reload	Reload auto-Refreshes unrelated rem
A permitted current acquisition installs into present owned DOM and required basis	Stale A callback overwrites newer B
Expired plan exposes typed retirement and applicable recovery	Missing JS file or decoder exception s
Resource and recovery horizons are jointly valid for the release claim	Offline command promised recoveral

These are prospective test and physical-observation obligations, not a claim that any deployed release currently satisfies them. The release gate must challenge real asset/handler adoption and both valid and invalid runtime paths; a test-only rejecting wrapper, copied stamp, passive status report or one successful staging request is not enough.

Section 11.08 owns **which directional pairings are reachable and admitted**. This section owns *which source-derived contract, physical realization and retained resources make each pairing safe and recoverable*. Section 11.10 owns retiring the execution and restoring only authorized semantic intent. No separate global compatibility registry, arbitrary suspended machine migration or universal durable participant-message fabric is required by these responsibilities.

Retire execution rights while preserving unresolved intent

Release retirement is a semantic transition, not the moment a physical resource finally disappears. It ends the selected obsolete execution's permission to continue, emit protected commands, adopt a browser result, consume a knowledge/acquisition grant, or mutate a resource it no longer owns. The browser coordinator and host activation boundary establish this disposition **before** best-effort cancellation of timers, HTTP requests, listeners, streams, worker callbacks or old DOM resources. Cancellation may race or fail; a surviving physical handle does not recover its revoked effect permission.

Retirement identifies the old assembly and runtime, execution and resource incarnations, affected continuations, and any still-owned physical cleanup obligations. Private callback and provisional-effect rights are retired or explicitly transferred through a checked current owner. A late transport completion may provide admissible evidence about a historical command, but it cannot resume a retired plan or perform the old swap, rollback or refresh. Evidence admission, authoritative command settlement, scoped acquisition and actual DOM installation remain distinct decisions.

Preserve only explicitly recoverable semantic context

A recovery contract derived from admitted application semantics classifies what may survive as **intent**, what may survive as **evidence**, and what must lose execution authority:

Surviving subject	Permitted recovery under a newly admitted owner
Logical page/route and domain Request identity	Locate the currently valid authorized route and subject
Explicitly recoverable local options or draft	Revalidate typed values, user intent and relevant current schema before use
Retained previously adopted observation	Reuse only its still-valid known scope, disclosure and progression where the new c
Stable unresolved command C42 and original binding	Reconcile its historical result or separately re-admit a compatible retry
Live advisory and unresolved acquisition obligation	Transfer the correct scoped debt and user grant under the current coordinator
Durable worker/outbox obligation	Transfer or recover under a qualified durable owner and compatible format

The contract does not demand persistence of arbitrary suspended Choreo machines or restoration of an old browser's whole executable state. A recoverable draft is not an already admitted domain mutation. A stable domain Request ID may select a current protected observation but does not prove that an earlier command incorporated. An invalid new principal, changed Organization membership, removed route or unsupported codec may force protected content and obsolete context to retire rather than remain displayed or executable.

On recovery, each selected capability is re-admitted at its real current boundary: the route and selected plan at construction or serving, authentication and Membership at protected use, model guards at authoritative write, command result at its trusted historical resolver, acquisition/coverage at its permitted read, and resource ownership at physical installation. Matching names, old proof reports, persisted snapshots, application-version labels or browser flags cannot stand in for those current checks.

Keep command uncertainty independent of execution retirement

A logical command and a browser execution have different identities and potentially different lifetimes. The stable `:gesso/command-id` belongs to one original operation, canonical arguments, command binding and authorized source context. A transport retry can have a new `:transport-request-id` or `:http-request-id`; neither changes the command identity, its domain Request target or its historical incorporation question. A process restart, replacement assembly, retired callback or timed-out response does not imply that the trusted operation failed or did not commit.

When an original result is unresolved, the recovery owner retains the applicable command-specific obligation and the exact retained record/decoder/reconciliation scope from 11.09. A current authorized resolver examines trustworthy command evidence under the original binding and current result-disclosure policy. A compatible durable record may justify the permitted historical recorded outcome; a current Request state alone cannot attribute that state to C42. A missing record at one observation, a lost response, failed SSE publication or expired browser plan does not prove nonincorporation. A negative test must distinguish an actual authoritative rejection from an unknown outcome and from a currently nondiscloseable record.

A compatible retry, where the authoritative operation explicitly supports it, preserves C42 and the original immutable binding under the existing deduplication and current permission checks. If the old mutation interface is no longer eligible, a qualified read-only historical lookup can remain supported without executing its old writer. If binding, retention, authorization or provider evidence does not admit either lookup or retry, recovery returns a truthful unknown, unavailable or blocked disposition with any permitted current-state repair path. It does not synthesize `:rejected`, invent C43, claim a successful historical outcome or loop indefinitely. The specific external availability and retention premises for eventual resolution are recorded; safety is required even when those premises fail.

Command outcome, current-state acquisition, durable publication and browser adoption do not collapse into one success. A retained committed C42 record can support an authorized report to Alice while its old callback has no effect rights and its prior card representation is not installed. A currently qualified Request read can repair the visible card without proving C42 caused that state. Publication and remote Live interest recovery retain their independently required obligations after the original server process or browser execution retires.

Release recovery preserves HumanHelp's adoption policy

HumanHelp's product policy distinguishes creator-local Create, own-command settlement, specifically admitted local rejection/unknown card repair, and unrelated remote creation or lifecycle changes. Remote changes are **advisory** to another participant's adopted board until an explicitly permitted Refresh under #decision-humanhelp-remote-lifecycle-adoption. Release retirement, reconnection, a new host, resumed visibility, asset reload and a newer runtime are not themselves authorization to acquire and install an unrelated fresh collection.

The current recovery plan accounts for separately named:

- the adopted board's existing authorized membership and coverage;
- newer current local card/Request observations already installed;
- pending remote advisory and knowledge/progression debt;
- available explicit user Refresh permission and its one-use scope;
- own-command and locally admitted recovery exceptions;
- the effect/target owner to which an authorized response may install.

An old one-use Refresh grant is neither duplicated nor silently restored when its former execution retires. A completed grant may discharge only the obligations actually covered by the completed and authorized acquisition. Unrelated advisory debt remains open, including advisory work arriving during the in-flight read or during a deployment boundary. An unconsumed grant or unresolved read may be transferred only under the declared owner/expiry and continuation contract; an incompatible plan must ask the user for a new authorized action rather than simulate a successful click. A new page initialization may have an independently declared baseline read grant; that is not permission to disguise an unrequested remote Refresh as a silent successor acquisition.

Known principal/Organization authorization changes may require retirement of previously displayed protected content and subscriptions. An earlier adopted observation is not a perpetual right to disclose after an authority context changes. Conversely, a newer already justified local card installation cannot be regressed by a late board response merely because that response was generated by the newly active assembly. Current source, coverage, progression, grant, resource incarnation and physical installation must all justify the proposed adoption.

A representative history is:

```
A serves Alice's eligible Claim C42 for Request R17.
C42 may have incorporated, but its HTTP response is lost.
A's browser execution and callback rights retire as B activates.
B admits a current, authorized historical C42 readback.
The returned historical result is distinguished from R17's current card.
A qualified current card acquisition may repair Alice's own view.
Bob receives the remote lifecycle advisory; no automatic board adoption.
Bob's explicit authorized Refresh can read and adopt its covered scope.
Late A callbacks cannot swap Alice's card or consume Bob's grant.
```

The recovered Claim outcome is truthful only to its recorded historical evidence. Alice's old optimistic rollback is not restored, and Bob's board membership changes only under the product's actual acquisition grant. A lost command-result lookup must not be interpreted as rejection; an unavailable Refresh service retains appropriate blocked/advisory status rather than inventing convergence or a successful installation.

Rolling-release acceptance follows actual physical histories

A release requesting overlap or retirement qualifies the actual reachable A/A, A/B, B/A and B/B pairings specified in 11.08, the source-derived continuation and retained physical resources of 11.09, and the activated and potentially rolled-back assemblies of 11.07. It observes real installed route/decoder and browser artifacts, selected provider and gateway behavior, actual source and target incarnations, and each applicable trusted authority boundary. Positive permitted behavior and nearby forbidden cases must be challenged at their actual corresponding enforcement seams; a passing isolated decoder, report map, fabricated staging fixture or reject-everything wrapper cannot issue a deployed rolling-release result.

Required permitted history	Adversarial history to reject or report honestl
A valid A-origin continuation completes under a retained qualified A or checked B context	A retired callback performs a new protected e
A late A response carries separately admissible C42 evidence	Its obsolete physical response installs into B's
A real C42 record is resolved under current disclosure permission	Browser-held ID or current Request snapshot
No C42 result is visible yet and recovery retains unknown	One negative snapshot is called proof of nonc
A compatible retry uses C42's immutable arguments and deduplication contract	An old command is silently reinterpreted as a

Required permitted history	Adversarial history to reject or report honestl
Qualified B historical lookup outlives A's revoked mutation interface	Reconciliation requires reviving unsafe A wri
In-flight A stream completes only with its checked surviving rights	A's server response adopts B's new runtime/p
New principal revalidates authorized historical result or receives nondisclosing status	Old session or Membership snapshot exposes
Valid source-derived A/B and B/A directions operate with installed artifacts	Sticky routing test hides a failover to an incor
Excluded direction refuses before an actual guarded model write	A test-only early wrapper masks an unchecke
Actual compatible browser asset remains served until eligible overlap expires	CDN purges A's runtime while sleeping A pag
Expired A execution is typed-retired and offered authorized current recovery	Missing JavaScript file or decoder exception s
Failed B preserves still-qualified A; revoked A rights remain fenced	B's failure keeps a known-unsafe old worker a
Old/new workers and migrations preserve admitted durable formats	Backward-compatible read is treated as permi
An owned valid current acquisition installs the selected covered card	Stale old optimistic rollback overwrites a new
Bob's remote advisory waits for explicit permitted Refresh	Asset reload or Live reconnect automatically
One authorized Refresh consumes one correct grant and leaves uncovered debt	Old grant is duplicated on transition or one re
A new page uses its distinct declared baseline grant where permitted	Refresh budget is covertly replenished by sam
Partial cleanup records old resources and does not revive their permissions	Failing cancel or socket close makes a retired
A qualified old-to-new rollback/release preserves actual committed schema state	Historical artifact is activated despite incomp
Positive eligible Claim succeeds at the same actual gateway rejecting invalid Claim	Reject-all implementation or synthetic comma
Multi-host loss and reconnect retain true publication/registration recovery debt	An SSE write or successful local host activatio

The selected release profile lists mandatory observations and real enforcement owners for each reachable case, including resource and asset retention, backend/provider changes, unknown command outcomes, current authorization, durable-work formats and explicit user acquisition. Missing, unsupported, stale or unavailable mandatory evidence blocks the corresponding release claim without silently reclassifying it as success. A narrower rollout may be separately admitted under its explicit exclusions and current enforcement contract. Bounded runtime tests establish only their exercised conditions; universal cross-host or liveness guarantees require their additional assumptions and appropriate stronger evidence.

11.07 owns candidate activation, failed installation and rollback. 11.08 owns the directional reachability and incompatibility gate. 11.09 owns source-derived compatibility, exact artifacts and retention horizons. This section owns *semantic retirement, surviving command/recovery intent, HumanHelp's selective adoption, and their final rolling-release acceptance histories*. It adds no universal suspended-workflow persistence and no second authority for business policy, command results or browser knowledge.

Global non-goals and retained design freedom

Gesso's assurance boundary is deliberately narrower than the set of possible Clojure programs, distributed databases, durable workflows and formal systems. This limits **what the framework requires universally**, not what an application is allowed to use or what a selected release profile may require. A domain can opt into stronger persistence, communication, proof, migration, consistency or verification where its declared semantics demand it. The chosen mechanism then enters the relevant source, dependency, qualification, writer and physical evidence boundaries.

A non-goal is not an exclusion that can be applied after a failed mandatory gate. A required capability remains required by its original business source and selected release intent; an unsupported mechanism makes the corresponding stronger claim open or blocked. Conversely, the presence of a powerful optional mechanism does not automatically upgrade an unrelated operation to a proven or deployed claim.

No universal persisted fact graph or model history

Gesso does not mandate a second persisted directed acyclic graph containing every authoritative fact, provisional presentation, supersession, command, witness and dependency. The domain models and their selected authoritative backend own durable business facts, the identities that make those facts meaningful, and precisely the history and retention required by their operations.

Proof and assembly dependency graphs are **assurance structures**, not a second system of record for arbitrary application state. They must still account for the original-source obligations, applicable assumptions, participating writers and invalidation of dependent evidence when these change. Calling the graph optional does not permit a required operation, outbox effect, command result or recovery obligation to vanish from that account.

Nor is every model required to persist Git-like branches, commit ancestry, three-way merge ancestry, event sourcing or a CRDT. These can be chosen when the domain's consistency and conflict semantics benefit. Gesso can expose a typed, checked interface for conflict detection, eligible resolutions, idempotent handling and reconciliation outcomes; the model supplies the meaning and authorization of those resolutions. The framework never selects a universally valid ”ours wins” policy merely because it has a provisional state and an authoritative observation.

A model that requires an audit trail, idempotent durable command record, compensating operation, migration log or historical outcome lookup must preserve it under the declared backend contract. The absence of a **universal** fact DAG or event log is not permission to erase those selected durable obligations.

No single global revision or automatic knowledge equality

No global integer represents, all at once, authoritative XTDB transaction progression, domain entity revision, valid/effective time, a command's causal history, a participant's knowledge frontier and a browser resource incarnation. These have different owners, comparison relations and retention lifetimes.

A provider may expose a sound scoped monotonic basis or a model may declare and enforce its own entity revision. A composed observation uses only the ordering and coverage relations justified by its provider and query scope; incomparable frontiers are not coerced into one scalar ”freshness”. The browser cannot equate the latest timestamp, transport response or invalidation with authority to install an unobserved collection or prove a command's historical effect.

Gesso also does not require every role to know every fact. Distributed knowledge, relevance, privacy, authorization and permitted acquisition intentionally create different valid projections. Agreement or convergence claims concern only the declared reachable information, justified source and progression frontier, permitted adoption and environmental assumptions. Deliberately undisclosed data is not a disagreement to be repaired by an unauthorized read.

No universal direct participant fabric or durable actors

An ordinary cross-user interaction can rendezvous through authoritative domain state and later short choreographies. Gesso does not require a global directory mapping every actor to a current server node, universally routable participant messages, or persistence of every suspended server/browser machine as an independently durable actor.

If a protocol **does** require direct delivery, confirmed message receipt, durable actor execution or a long-running workflow, its selected module must declare those semantics, failure alternatives, durable owner, authorization, replay/duplicate rules and the evidence on which its claim depends. Those capabilities remain valid application choices rather than mandatory infrastructure for all Gesso deployments.

Single-node and multi-node realization preserve the **same** operation, command, knowledge, Live and recovery semantics. Scaling across nodes may introduce routing, interest discovery, subscription fencing, shared publication and redelivery mechanisms; these do not invent a second model of authority. An implementation may begin with a Clojure router and later replace a physical routing component without changing the source-owned invalidation or permitted acquisition contract.

An SSE advisory is not necessarily a domain command, and delivery of one message is not proof of durable database incorporation or browser adoption. A required invalidation must still have a justified discovery, publication, coverage and recovery path through router loss or late interest registration. A deployment cannot cite this no-global-fabric non-goal as a waiver for lost required Live updates.

No arbitrary stale-machine migration

Gesso does not transparently deserialize or migrate arbitrary suspended browser or server execution state through incompatible releases. The baseline is finite checked release overlap, typed semantic retirement, transfer of explicitly recoverable intent and current authorized reconstruction under the newly admitted plan.

An old execution loses its private timer, callback, resource and optimistic rollback rights when retired. A supported logical route, Request identity, user-owned draft or local choice may survive only with revalidation and current owner admission. A still-unresolved command retains its original identity and historical reconciliation obligation where promised; it does **not** regain old executor permission or become a new command. A current Request read can repair a view but is not proof of the older command's incorporation.

A checked domain-specific state transformation or compatible long-lived workflow is permitted. Its source-derived scope, state/codec interpretation, current principal checks, invariants, and relevant installed effects must be qualified. A release must not pretend that an unavailable old decoder or an expired asset can migrate a private machine through a ”compatible” version label.

For HumanHelp, reconnection, new assets and host replacement do not automatically grant the user a new remote-board Refresh or silently adopt advisory-only changes. Recovery follows the same owner, coverage and one-use grant semantics as an ordinary running application.

No universal proof of arbitrary host-language computation

The finished framework has closed, purpose-built semantic interfaces for selected rules, operations, choreography, projection, application composition and checkable evidence. A Clojure or ClojureScript function can participate through an admitted interface without becoming a theorem simply because it was called from a certified operation.

Gesso does not require a universal Clojure type checker, unrestricted dependent types, arbitrary Clojure/JVM theorem proving or an application author writing Lean, TLA+, Quint or Recife scripts for ordinary development. Application authors declare source business rules and construction, select explicit assurance profiles, and provide genuinely necessary premises or domain-level specifications. Gesso derives supported proof/checker obligations and can invoke appropriate external checking machinery as an implementation mechanism, without making those languages a second mandatory application specification.

A closed checker rejects unsupported expressions **for that judgment**. The application may use a general host-language implementation under an explicitly weaker, qualified interface where the selected profile permits it. A mandatory stronger proof obligation cannot be silently rewritten as a trusted assertion because the checker cannot inspect a function. Likewise, a proof kernel's successfully checked derivation is not proof that a separate opaque executor implements it.

The framework does not claim to have proved the entire JVM, JavaScript engine, browser, HTMX, Biff, XTDB, operating system, hardware, network or every third-party dependency. A precise source theorem can remain valid **conditionally**, with each relevant compiler, adapter, runtime, physical writer and deployment contract separately classified and qualified. ”Unproved” is not the same as ”assumed correct”: the release report names actual unresolved, accepted or unsupported premises and the claims depending on them.

No inference that a coherent business rule is the desired rule

A rule can be well typed, compositionally coherent, preserved by every declared transition, and faithfully executed while still expressing the wrong product choice. Gesso checks the declared semantic and application obligations; it does not infer the product owner's intended meaning from a successfully type-checked syntax tree or the subset of controls the implementation happens to expose.

Independent required capabilities, authorization choices, user-facing interaction and positive acceptance remain part of the original product requirement set. In HumanHelp, missing Response interaction, Reassign or remote Refresh cannot become ”optional” because another feature passed formal or browser checks. The product owner decides open business choices; checking verifies the resulting selected policy, not a guessed policy invented to pass a gate.

No elimination of current enforcement or physical testing

Construction and certificate checks do not freeze future authentication, Membership, current domain guards, backend qualification, browser target ownership or transport compatibility. Those must be evaluated at the corresponding trusted effect, disclosure or

adoption boundary. A proof report does not grant runtime permissions, while the installed runtime guard must implement its selected policy without silently downgrading an unsupported managed action.

Nor do proofs eliminate integration, concurrency, failure-injection, real browser, selected backend, load, startup or deployment tests. The independent production correspondence checks which routes, writers, handlers, assets and provider incarnations actually perform the checked operations. A fabricated passing test map, one successful negative-only fixture or an unused built asset cannot replace a genuine observed boundary. Conversely, hundreds of passing tests cannot turn an unsupported universal theorem into a checked derivation.

No unconditional distributed liveness, recovery or convergence follows from a connected transport or a finite machine check. Any positive progress guarantee retains its availability, fairness, retention, user action, reconnect and disclosure premises. Safety must remain accurate when those premises fail: unknown incorporation cannot become rejection, and inaccessible recovery cannot be described as completed delivery.

Identity, evidence sidecars and backend freedom

Source file, namespace, Git commit, line number, assembly identity, exact asset bytes, normalized semantic subject, issued proof handle and current resource incarnation are different coordinates. Their correctly bound relations can support verification and diagnostics; replacing one with another because two strings look alike is not assurance.

Optional explanatory proof or diagnostic sidecars do not change domain transition meaning or authorize effects. A selected mandatory preflight, issuer, admission or runtime guard **does** enforce its independent boundary using the appropriate validated evidence. Required durable audits, publication outbox writes and command records are actual declared effects, not optional logs simply because they help with diagnosis. No passive report gains the right to mutate a Request or install a protected DOM target.

Database-neutrality is not a mandatory end in itself. The semantic rules are separated from accidental XTDB storage representation, but a qualified XTDB-specific transaction, history, readback or basis-fencing mechanism may be the most direct correct realization. An alternative provider may be installed when its actual atomicity, history, consistency, failure and qualification premises establish the needed relation. It is not accepted merely because it implements a similarly named protocol.

Non-goals do not weaken selected release obligations

Deliberate non-goal	Mandatory boundary that remains
No universal fact DAG	Original business facts, writers, command outcomes, audit/outbox and proof depe
No global revision integer	Every selected source, progression, causal and knowledge ordering is checked in i
No universal event sourcing or CRDT	Model-selected conflict and durable reconciliation semantics are enforced with ap
No global actor directory or cross-node message fabric	Required Live interest/invalidation and durable-work failure/recovery obligations
No arbitrary suspended-machine migration	Supported bounded compatibility, honest retirement and retained command-speci
No arbitrary Clojure theorem prover	Closed-source proof obligations and explicitly classified opaque implementations
No proof of the entire software/hardware stack	Named trusted premises, qualified deployed bindings and independent physical ac
No inference of product intent	Original independent product requirements and positive user capabilities remain
No unconditional convergence	Progress is conditional; safety and truthful blocked/unknown statuses survive out
No mandatory database portability	Each selected backend has qualified, source-compatible transaction and observati
No semantic sidecar authority	Required admission/runtime gates and actual durable effects still enforce their ind

The negative design constraints preserve room for ordinary, lightweight server-rendered Clojure/HTMX applications as well as applications selecting stronger formal, distributed or durable capabilities. What may be omitted is **unselected machinery**, never the source-derived obligations or actual enforcement necessary for a guarantee the release claims.

Progress metrics and completion boundaries

Progress is measured by the original required behavior, supported semantic judgments, usable physical capabilities and justified evidence for the exact subject claimed. Revision counts, file size, passing assertions, the number of emitted reports and the fraction of a **conveniently discovered** operation list do not establish release eligibility or any completion milestone.

A progress measure is a diagnostic view over source-derived requirements and genuinely observed evidence. It is not a separate issuer, an authorization capability, or a way to alter a release profile after an inconvenient failure. Section 11.05 owns the underlying root claim/evidence ledger; 11.02 owns the eight gate-result dispositions and the four independent release decisions. This section specifies what honest counts and comparisons are allowed to say about that ledger.

Freeze the denominator before measuring the numerator

Every coverage measure identifies:

- the original independent product or assurance requirement set, its source owner and immutable identity for the claim being made;
- the exact claim scope, selected profile, required milestone, operation or artifact class, and applicable generation, host, provider and evaluation window;
- the predeclared inventory of required subjects, obligations, successful paths, failure classes and execution environments;
- the admissible evidence type, issuing checker or observing boundary, applicability and trust premises for each item;
- the numerator's actually qualified items, the denominator's complete required items, and an explicit inventory of not-yet-credited, failed, stale, unavailable or unsupported items;
- the validation of denominator completeness against the **original** source or an independently specified product capability contract, not merely a generated catalog; and
- any approved scope change, separate preview profile or genuinely inapplicable item, with the resulting changed claim identity and independently reviewable rationale.

A missing item has not become inapplicable just because the application failed to construct its route, render its button, discover a writer or issue a proof obligation. An implementation with only three exposed operations may close those three correctly and still fail the independently required six-operation product scope. The exposed-operation denominator and the required-user-capability denominator are two distinct metrics; neither silently substitutes for the other.

A denominator may change when an independently authorized business or release requirement actually changes. Reports preserve the previous and new inventories, the approval and effect on claims, and recompute coverage against the new subject. They do not call a shrinking denominator successful implementation progress on the unchanged original goal. A selected stricter assurance profile cannot silently turn a missing proof into a trusted premise, skipped test or new smaller profile after the check fails.

A ratio is reported only where its denominator and classification make sense. For an empty selected inventory, report ”not requested” only if that inventory was validly excluded by the claim; report ”missing requirement inventory” where source completeness cannot be established. Empty, unavailable or unobserved is not automatically 100% coverage. Unknown and incomparable outcomes remain explicit rather than receiving arithmetic credit.

Keep different progress dimensions separate

Metric	Stable required denominator
Usable original product capabilities	Independently selected canonical user capability and decision inventory
Declared assembly closure	Managed source operations, affordances, routes, acquisitions, effects and required depende
Runtime/production enforcement	Required real route, handler, writer, output and privileged effect-boundary classes for the a
Typed semantic judgment coverage	Required type, placement, knowledge, authority, refinement and progression subject/judgr
Required dependency and assumption closure	Original business, proof and application obligations, with their transitive dependencies
Framework theorem/proof-kernel coverage	Predeclared properties and supported semantic artifact classes for the selected assurance p
Formal-to-executable correspondence	Named semantic-to-implementation, writer, transaction, browser, installation and deploym
Mutation/defect rejection quality	Predeclared representative defect classes and intended first rejecting boundaries
Qualified substrate coverage	Actually selected backend, transaction adapter, compiler/browser runtime, host, configurat
Recovery and interruption coverage	Required loss, uncertainty, reconnect, replay, retirement and provider/host failure histories
Release overlap/rollback coverage	Every actually reachable permitted or excluded client/server direction and retained work/f
Evidence reuse/impact accuracy	Applicable unchanged obligations plus affected semantic, physical and environmental char
Test execution inventory integrity	All required jobs, cases, environments, shards and accepted result/receipt records

Each row is a **separate** claim dimension. A release can have complete semantic closure but lack actual installation or user-facing acceptance; it can exercise every browser capability without establishing a theorem; it can prove a model invariant while its full

physical writer set remains unknown. Any combined presentation retains those separate blockers, required evidence classes and dependency links. No weighted ”overall assurance %” can promote a missing mandatory gate to success.

Measure coverage with evidence status, not optimistic arithmetic

The root evidence ledger retains the status of **every** required gate as one of the eight classifications from 11.02: `:passed`, `:failed`, `:incomplete`, `:unavailable`, `:unsupported`, `:skipped`, `:stale` or `:missing`. Passing credit is possible only when the checked evidence actually refers to the intended subject, source, current installation/environment and selected profile. A copied ”passed” map is not evidence and has no numerator authority.

A report distinguishes at least:

Category	Meaning for the original requested claim
Required and verified in scope	Eligible to receive its precise checked credit
Required but not executed/observed	Still missing or incomplete; no success credit
Required but unavailable/unsupported	Blocking unless an independent new profile explicitly changes the claim
Executed and failed	Report failure, intended boundary and reproducible/unknown diagnosis
Previously checked but stale	Re-evaluate dependencies and required physical or source changes before reuse
Skipped by an approved narrower profile	Visible exclusion under a different claim, not completion of the original
Trusted external premise accepted by the selected profile	Conditional assumption disclosed with all dependent claims, not counted as ind
Truly inapplicable under original requirements	Exclude only with a justified, traceable scope rule, not a failing implementation

For example, suppose the independent product requirement set contains Create, Claim, Unclaim, Reassign, Response and Refresh. If four have eligible actual positive-browser evidence and Reassign and Response do not, the product coverage is four of six, with the two missing controls named. If the assembled UI exposes only the four successful operations, it may have four of four **exposed-route closure** but still has only four of six required product capability coverage. The latter denominator does not shrink to make the application look complete. An incomplete Response product decision is an explicit blocker until the owner selects and the application demonstrates its contract.

The same discipline applies to formal proof: a finite check of selected states counts toward its **finite subject** only. It does not count as an induction theorem, universal checker soundness, independent proof-kernel verification or the whole installed XTDB/browser refinement. Counts of assumptions and trusted-base components are audit aids, not a scalar probability of correctness. An accepted environmental premise stays visible and transitively attached to all claims that depend on it.

Diagnose first-boundary rejection and positive usefulness

Rejection quality measures the difference between a defect discovered at the earliest justified owner and one allowed to fail late, at an unrelated wrapper, or only after a protected physical effect. For every included mutation or controlled-defect family, state the intended violation, first sound checking boundary, expected typed diagnostic, physical effect/non-effect oracle and paired valid case. The defect inventory is selected independently of the mutations the implementation happens to reject.

An invalid forged principal must fail at the trusted authorization boundary, not be ”proved safe” because the browser happened not to show the button. An incompatible old-client command must be rejected before the real protected writer, not merely by a test-only interceptor. A malformed model rule rejected by a compiler only after a missing provider crashes has not demonstrated the intended early diagnosis. Conversely, a checker that blocks **all** Claims, or only accepts trivial rules, has excellent rejection arithmetic but fails positive usefulness and the selected capabilities.

Failed or flaky tests remain accountable under the declared test isolation, schedule and evidence policy. A rerun that happens to pass is new observed evidence; it does not erase a genuine unresolved nondeterministic failure or justify excluding its original job from the required inventory. A first-boundary requirement may be structurally proved, physically observed, or still an open transfer obligation; the reporting level is not silently upgraded.

Preserve performance, reproducibility and reuse context

Performance measurements identify hardware, JVM/browser, compiler, storage/backend version and configuration, dataset and data distribution, worker count, routing topology, resource limits, warm/cold state and executed workload. A fast deterministic developer gate is valuable for iteration; it is not a substitute for required complete JVM, finite/formal, Chromium, real XTDB, fault/reconnect or multihost coverage.

Parallel tests retain a deterministic selected inventory, the exact job-to-shard assignment and result accounting, isolated side effects and proof that required preflight ran for the proper scope. Missing, timed-out, duplicated or otherwise unauthenticated

worker completion cannot turn a release gate green. A summary from an orchestration tool must be tied to actual executed tasks and failures under its declared trust boundary; a copyable report cannot authenticate execution by itself.

Incremental proof or test reuse is measured by correct dependency invalidation and current applicability, not only by jobs avoided. When source meaning remains stable but actual browser bytes, handler, provider or backend incarnation changes, the abstract theorem may remain reusable while physical correspondence evidence must be requalified. Source, environment, trusted premise, toolchain and original-required-set changes have distinct invalidation effects. Report both avoided work **and** required reruns; reducing elapsed time by dropping required work is a false optimization.

The release and completion milestones in 11.13–11.18 use these metric definitions to explain partial checkpoints, not to replace the respective mandatory gates. Project management can track iteration count, time and document size as resource measures; these are never evidence that a business behavior, theorem or installed browser effect is correct.

Positive and adversarial measurement controls

Honest measurement	Invalid or misleading nearby measurement
Six independent product capabilities remain required when four are implemented	Four exposed controls are reported as 100% o
Missing Response product decision is shown as a blocker	Response is silently removed from the requir
Exposed operations close against their actual original assembly source	A caller-generated subset is called complete v
Current installed protected routes and writers are independently challenged	A test-only wrapper is counted as production
Supported checked rule form earns its exact theorem/judgment credit	Unsupported arbitrary Clojure is relabeled ch
Trusted premise remains attached to dependent abstract proof	Accepted assumption is counted as discharge
A finite model run credits only its bounded initial/transition domain	Finite sample is shown as universal XTDB/br
Original mutation inventory and positive cases are predeclared	Reject-all verifier records perfect negative tes
Invalid route fails before the intended effect with useful diagnostics	Late crash after write or unrelated wrapper re
A valid real Reassign executes under the same guard challenged by rejection	Disabled or unavailable button is counted as
Actual compiler/host/backend configuration is measured	A passing different adapter or staging deploy
Command C42 remains unknown with explicit permissible recovery	Timeout is scored as a successful rejection or
A/B and B/A actual routing exposures are measured or excluded by tested guards	Happy-path sticky session is called full rollin
New browser asset invalidates required physical checks while reusing sound abstract proof	Cached source-equivalent theorem is treated
Each required parallel job is accounted for and its failures remain inspectable	Missing shard, stale log or rerun success silen
Physical load measurement includes hardware, topology, limits and data	Fast one-worker test is advertised as producti
Original and changed scope are separately recorded after legitimate requirement approval	A failed requirement is dropped, producing a
Empty required inventory without established source completeness is blocked	Empty set is treated as automatic 100% proof
Root report exposes all missing/stale/unsupported obligations	Fabricated passing rows make a release candi
Real browser positive and adversarial histories use deployed source artifacts	Generated fixture with no user control or phy

These are prospective measurement and acceptance obligations; the table is not itself observed test evidence. A complete and accurate report can be useful while the release is blocked. It is the distinction between **what is required**, **what has actually been established** and **what remains open**, not a decorative percentage, that makes progress visible and keeps completion claims honest.

Operational application completion

The first bounded completion milestone is a **usable, structurally closed, actually installed** Gesso application, established through the HumanHelp proving application and its public production-model contracts. The milestone is not the successful creation of an ApplicationAssembly, a passing Claim fixture, a running Biff server, a green release summary, or an acceptance decision about this specification. It requires the independent original product capabilities and the actual supported production paths that make them work.

This milestone is deliberately distinct from full compositional certification, independent external formal verification and production multi-node completion. Those have separate completion subjects in 11.14–11.16. The operational milestone nevertheless honors the ordinary semantic/runtime checks and every stronger assurance obligation explicitly selected by its release profile; it cannot obtain a green result by downgrading an unavailable mandatory theorem or a required physical test after failure.

Bind one original product target to an admitted release

The requirement set is fixed from independent HumanHelp business and interaction obligations in Part VIII, not inferred from the example's currently implemented routes, rendered buttons, or discovered command catalog. The acceptance profile identifies actual required controls, successful user histories, rejection and recovery paths, authoritative model outcomes and real environmental premises. A valid partial checkpoint names what is missing against that same denominator.

The canonical construction closes the source-declared models, rules, operations, choreography, executable plans, HTTP routes, trusted execution and outcome mapping, authoritative acquisitions, rendered affordances, Live subscriptions/publication, generated browser assembly and exact artifact relationships. Part VII owns the underlying construction and its mandatory checks. The installed application must use the **same qualified semantic subject and actual compiled/served realization** for the checked operation. A system map, build receipt or completed constructor does not independently prove a real listener, worker or browser uses those checked bindings.

Completion joins three separately evaluated claims from #req-application-assurance-claims:

Claim subject	Required evidence
Declared closure	Source-complete construction and render/operation-plan relations for the admitted application
Actual production enforcement	Installed route, trusted gateway/model writer, response/asset/HTMX paths and required effect guards be
Required usable behavior	Eligible user actions visibly succeed with authorized source-of-truth results, plus their relevant negative,

A release may pass a structural test yet fail product behavior, or pass one browser scenario while the actual authoritative writer bypasses the checked model. Each failure remains a separate blocker. Root evidence reports do not issue runtime permission, certify externally observed tests merely from caller-provided data, or grant HumanHelp actor authority.

Prove the public production model owns the result

The proving app uses the actual public production User, Membership, Organization, Location, Request and RequestAssignment owners and their declared interfaces; it does not replace them with an example-only Claim model, browser-local permission rules or a separate store invented to make a fixture green. Trusted server authentication binds the acting principal, and production-model guards check current eligibility, actor/target distinction, required Organization/Location association and protected Request and assignment dependencies at the authoritative effect boundary.

For each admitted lifecycle mutation, the public model determines current Request truth, permitted assignment transitions and outcome meaning. The primary assignment and the Request state agree in the authoritative transaction and authorized projection. The completed lifecycle state is `:done`. Reassign selects a **different eligible target** through its real permitted manager/supervisor authority, closes the superseded primary responsibility and establishes the new one under guarded state; it is not a relabeled self-Claim or a fabricated target parameter.

The command path keeps stable logical command identity, canonical binding, authorized effects, compatible incorporation/replay, readback and disclosure distinct from the transport attempt and displayed card. Real XTDB transition behavior must satisfy the selected transaction, contention and consistency assumptions, including the actual production writer inventory. A lost response is `:commit-unknown` where incorporation is not established; a missing readback is not a domain rejection or permission to allocate another command ID. Required durable publication and external writer obligations remain visible even if the browser promptly renders the desired card.

The observed user result is established through the operation's qualified authoritative outcome and acquisition contract. A separate post-commit read is required **where that contract needs one**; it is not an unconditional ritual after every successful transaction. Conversely, a provisional HTMX swap or copied optimistic state is not production-model truth or a substitute for an independently justified installation of current data.

Exercise the full independent HumanHelp capability set

Part VIII owns the exact product meaning and acceptance contracts. The operational profile includes, at minimum, authenticated access and relevant protected disclosure, Create and immediate permitted creator-local visibility, details and local interaction, the six production Request lifecycle actions, response interaction, search and ordering, board visibility/terminal options, Live advisory and explicit Refresh, optimistic feedback, selection/scroll continuity and honest rejection, uncertainty and reconnection recovery. An implementation cannot omit a difficult capability and make its own smaller feature catalog the completed product.

Each lifecycle action has an eligible positive real-browser history and applicable forbidden, stale-state or concurrent histories:

Operation	Required successful user result	Essential contrary bou
Claim	Eligible helper becomes the guarded primary assignee	Two concurrent helpe
Unclaim	Permitted current owner relinquishes responsibility with coherent Request/assignment state	Stale or unauthorized
Mark on the way	Eligible current assignee performs the declared transition	Old or unrelated acto
Complete	Permitted actor reaches authoritative `:done` and closes required assignment state	A provisional comple
Cancel	Authorized requestor performs the allowed cancellation and dependent closure	Unrelated user or inv
Reassign	Authorized manager chooses a distinct eligible helper and persists correct replacement history	Forged target, self-cla

The product's Response interaction is a separate **required but not yet independently selected** product contract at #decision-humanhelp-response-flow. Before full operational completion, the designated product owner must select the observable user intent, eligible actor, model/external-effect ownership, privacy/disclosure, success, error and recovery behavior, then that selected contract must pass independent acceptance. Gesso does not invent email, messaging, contact disclosure or a second Request model merely to fill the gap. A placeholder button or an existing Claim handler is not accepted as Response without the independently authored equivalence and positive user result.

Remote Create and remote lifecycle changes are **advisory** to another participant's board until explicitly adopted under #decision-humanhelp-remote-lifecycle-adoption. An own-command settlement and specified affected-card rejection/unknown recovery have only their declared exceptions. The explicit one-logical-query Refresh grant, its coverage, possible user-authorized successor intent and remaining advisory debt are governed by #req-humanhelp-refresh-coverage. Successful SSE send, reconnect, presence of a new card in storage or server restart is not a new user Refresh grant. A newer justified local card observation must not regress under a late, incomparable board response.

Search, creation ordering, mine-first, unclaimed-first, terminal visibility, selected card/detail state and permitted local input are tested over **adopted** information, not by secretly acquiring unrelated remote changes. Re-render and continuity preserve appropriate local state without retaining revoked principal authority, stale optimistic callback rights or an obsolete browser resource's permission to install content.

Qualify the actual server, browser and backend path

The selected Biff startup and application-module order must install the guarded handler and render-validation path actually consumed by the serving listener. Actual source and render affordances must close to the intended Choreo operation, execution gateway and typed outcome. Alternative routes, raw Ring HTML, scheduled or background writers, migration/admin entrypoints and other selected physical escape paths either receive required coverage or remain named blocking premises for the claim. Private initialization must not expose an unadmitted effect.

Exact generated browser bytes and manifest/receipt relations are checked under their stated compiler and toolchain premises, while actual serving and browser loading are separately observed. The real physical browser path exercises the admitted semantics through HTMX, SSE, resource ownership, continuity and the selected optimistic protocol. A fixture that directly posts hidden command metadata without using the actual rendered control is not a valid substitute for a user's operation. Conversely, a visible successful control does not independently prove every physical XTDB effect or command-record causal relation.

Independent product acceptance runs real Chromium or an equivalently declared supported browser against the selected served assets and actual routes, with appropriate authenticated contexts. Multi-client advisory, concurrent Claim, authorization, session changes and Refresh require genuinely distinct actors and correct source-of-truth observations. The selected XTDB backend, adapter, schema, consistency/progression, command records, provider qualification and deployment premises must match the application being claimed; a passing in-memory fake or unrelated local backend cannot issue this production conclusion.

Representative controlled defects prove that the intended **production** construction and effect paths invoke their promised checks. Invalid operation binding, missing original requirement, stale route, unqualified writer, forged principal, incompatible client generation, stale DOM resource, and incorrect Refresh use should be caught by their first appropriate boundary. Nearby valid cases must work; a reject-all route and a test-only protective wrapper are not successful product or enforcement evidence.

Demand honest interruption and recovery behavior

Usable operation includes the declared interruption and uncertainty paths, not only an uninterrupted positive script. Test at relevant boundaries: concurrent conflicting updates; timeout or response loss after possible commit; unknown incorporation with retained original command identity; Live delivery interruption and required publication recovery; invalidation arriving during Refresh; disconnect and reconnect; stale resource callback, superseded optimistic effect or newer adopted card; principal/Membership changes; and failed preflight, installation, asset or provider qualification. A case may finish with a truthful blocked or unresolved disposition under stated availability assumptions rather than a fabricated successful convergence.

A runtime transition can retire old browser execution rights while preserving only explicitly recoverable semantic intent. Release compatibility, asset/decoder retention and current result disclosure follow 11.07–11.10. An old callback cannot undo a newer authoritative installation, invent a duplicate Claim or consume someone else's Refresh grant. Required resource caps, bounded subscriptions/queues and recoverable user-facing errors remain part of the selected operational acceptance profile.

The operational milestone does not silently include the full general multi-node proof and production deployment profile of 11.16. It **does** require every client, server, backend and failure condition actually selected for its own operating environment, including real multi-client behavior and any physical routing constraints it claims. If an operational profile chooses a multi-host topology, its applicable cross-host premises and acceptance cases cannot be waived merely because 11.16 remains a separately named broader milestone.

Operational completion and falsification histories

Eligible positive acceptance	Nearby invalid or incomplete history that mus
Original HumanHelp feature inventory is independently fixed	Only successfully exposed UI controls determi
All public production Request/model owners supply actual behavior	Example-only fake Request store passes a Clai
Actual authenticated actor and distinct Reassign target are checked at the trusted model	Forged principal or selected helper is trusted f
Concurrent Claims yield one active eligible primary	Both helpers succeed from one stale observed
Real Unclaim/On-the-way/Complete/Cancel/Reassign each work when eligible	Every difficult control is disabled and negative
Complete establishes production `:done` and coherent assignment closure	DOM shows done while Request and primary
Creator-local Create uses admitted result and scope	Creator cannot see own Request until unrelate
Bob sees advisory for Alice's remote mutation until authorized Refresh	Incoming SSE silently replaces Bob's adopted l
One user Refresh installs its covered scope and preserves remaining debt	Read consumes multiple grants or clears an ad
Search/sort/visibility operate on adopted cohort	New remote rows appear through an unreques
Details/selection/scroll survive compatible rerender under current permission	Old DOM snapshot restores revoked content o
Response product contract is independently chosen and exercised	Unselected flow replaced with placeholder or r
Actual current Request/assignment truth is observed after accepted operation	Optimistic card or HTTP success substitutes fo
Lost C42 response is reconciled through authorized historical record or remains unknown	Missing record is called rejection and a new C
Production XTDB transaction and writer conditions are qualified	In-memory fixture or alternate unguarded wri
Biff actual listener uses checked handler and guarded rendered response	Preflight checked a handler that serving code l
Current exact JS is built, served and loaded under selected premises	Matching filename, hash alone or old cached b
User click supplies an actual rendered canonical command	Test fabricates hidden protocol fields without i
Positive protected Claim works and adjacent bad Claim fails at same real gateway	Reject-all wrapper or test-only interceptor is c
Live interruption/unknown/advisory recovery produces truthful statuses	SSE send or HTTP 200 is labeled completed br
Stale callback is denied while new resource can install valid result	Retired optimistic rollback overwrites newer a
Principal/Membership revocation blocks new protected effects and disclosures	Browser keeps using an old proof or cached gr
Required startup/asset/worker failure yields safe behavior and diagnostic	Failed initialization leaks a protected worker in
Selected multi-client contexts verify independent actor outcomes	Single browser script poses as two simultaneo
Root claim distinguishes closure, installed enforcement and usable behavior	Green aggregate map is treated as proof of all
Partial checkpoint retains absent capabilities and unselected Response blocker	Smaller operational preview is advertised as fu

These are **future conformance and physical test requirements**, not assertions that the present app has passed them. The acceptance records cite actual observed operations, required source and environment, failures, first enforcement boundaries and qualified premises. A passing isolated Claim pilot is useful partial progress, not independent proof of the whole HumanHelp product.

The exact operational completion declaration states which original required capabilities work, which authoritative data and browser behaviors were witnessed, which production and external premises are qualified, and which ordinary semantic or stronger selected assurance obligations remain open. The milestone is admitted only when **all mandatory gates for its named profile** succeed under the separate claim rules of 11.01–11.06. A partial checkpoint preserves every missing product or proof obligation and remains partial.

Compositional certification completion

The compositional-certification milestone establishes a **working, non-vacuous and source-complete chain of specified semantic judgments** for its declared artifact classes, operation catalog, application scope and assurance profile. It is neither a generic `:certified` flag nor a promise to prove all Clojure programs. The authoritative definitions of each judgment, its proposition, issuer/checker boundary and dependency obligations are in Part IX; the production-correspondence frontier is in Part X.

A completion decision must name both what was genuinely checked and what remains conditional. A verified derivation, a conditional abstract induction, a qualified model executor, an installed handler, an observed browser interaction and an independently proved checker meta-theorem are **different** claims. Their evidence can be linked through correctly established relations, never identified merely because the application report lists them under one operation name.

Complete the supported source-to-judgment chain

For the selected supported artifact and operation classes the milestone requires the following actual capabilities:

Semantic boundary	Required operational result
Typed rule construction	Closed versioned syntax, value types, short-circuit semantics, safe presence/refinemer
Typed rule and guarded-operation judgments	Exact original source, input/reader paths, permitted transition and frame obligations v
Distributed use-site admissibility	Role and program-point knowledge, provenance, disclosure, authority, refinement, pro
Global/projection/executable-plan relations	Projection and lowering are tied to the original Choreo subject, selected observable be
Proof kernel and dedicated checkers	The admitted closed inference rules and required semantic judgments are actually che
Conditional invariant/induction checking	Checked initial-state and preservation obligations for each invariant and required ope
Source-derived obligation and dependency closure	Original declarations determine every required proposition, branch, producer, depend
Issuance, consumption and currentness	A genuine checked result is bound to the exact proposition, source, profile, checker ve
HumanHelp real Claim pilot	One substantial public-model Claim produces traceable semantic, construction and qu

A checker implementation may fuse these stages or use several focused namespaces. Completion concerns their observable semantics and trusted boundaries, not a mandatory number of compiler passes, a required external proof language or a particular public function name. Arbitrary host-language implementation remains available through an explicit, profile-permitted implementation premise; an unsupported closed judgment cannot silently become a proved theorem.

Require independent source completeness and useful positives

The accepting boundary obtains the **original** model, rule, operation, choreography, projection and application declarations with their actual source owners, relevant imports and independent required-capability inventory. A proposed derivation does not get to define the set of propositions it must prove. Required obligations include every supported operation and applicable branch, read/write/frame condition, role-local use, required initial state, guarded outcome, resource exit, relevant dependency and expressly selected invariant.

Obligation and dependency coverage is checked independently against that original source. A cyclic proof-justification graph is invalid; an acyclic graph that omits Reassign, the rejection branch, an original Organization rule or an opaque implementation dependency is **also** invalid for a claim that requires those subjects. Runtime choreography loops are not automatically circular proofs. An admitted abstract subset may legitimately pass only when its narrower subject and exclusions are explicit; it does not meet a larger original application denominator by changing the reported label.

Certification is not satisfied vacuously by an unsatisfiable initial-state specification, a rule checker that rejects every Claim, unreachable user-facing positive operations, or an inconsistent collection of admitted premises. Where the selected claim requires inhabited initialization, reachable allowed transitions or real participant usefulness, these are separately checked or explicitly qualified. Correct rejection is paired with an eligible positive under the same chosen checker and application boundary. Product policy remains independently owned: a sound derivation can enforce a coherent but unintended HumanHelp rule.

Distinguish derivation, assumption and proof soundness

The small kernel checks exact propositional or other supported derivations under their named hypotheses. Dedicated checkers establish their own defined typing, guarded-transition, finite-model, induction, distributed or composition judgments. A developer or elaborator cannot issue a certificate merely by returning `:valid?`, `:proved`, a matching hash, a copied checker summary or a self-described dependency graph.

A checked derivation says that its conclusion follows from the named premises under the **specified calculus** and the implementation's checker trust contract. It does not prove that every atomic business predicate holds in live Request data, that the selected business policy is desirable, or that the kernel's own algorithm is independently sound. Independent kernel soundness and production-to-formal checker correspondence are separate external formal obligations of 11.15. When that stronger scope is selected they cannot be satisfied by local passing fixture examples alone.

For conditional induction, each declared invariant requires a check of the intended initial-state condition and preservation under **every declared operation** in the required catalog, with the actual transition, simultaneous writes, readable prestate and unchanged frame interpretation. The source-derived catalog cannot shrink when a difficult writer fails. Whether the specified initial states include **all physical initializers** and whether **all actual physical writers** refine the declared operations remain separate qualified claims. A valid conditional theorem must continue to display those premises instead of becoming an unconditional domain invariant.

Assumptions are propagated along the particular accepted justification for the guarantee consumed. A genuine discharge is a checked substitution or other admitted entailment from evidence of the **same proposition and compatible context**, including

source, parameter, principal, authority, temporal basis, implementation and issuer boundaries. Replacing an open P with evidence of P under Q leaves Q open where applicable. A caller-supplied `:discharged?` flag, identical premise label or different tenant's authorization does not remove P. Checked premise identities, dependencies and rejected substitutions remain inspectable.

Bind real issuance and safe evidence reuse

The ordinary construction path checks source shape, typing, elaboration, coverage, dependency closure, proof rules, premise policy and coherent currentness before issuing a source-bound, typed admitted product for the exact supported claim. A human-readable root report is derived from the same genuine checked records but is not an issuer or authorization capability. A raw map or test fixture can describe a proposition; it cannot impersonate the issuer of a checked proof or an admitted executor.

Process-local handles require their intended current issuer/registry and dependency context. Any durable serialized package is treated as untrusted input until decoded and independently re-admitted or authenticated and rechecked under its declared threat, rule-version, source and premise model. A signature can authenticate attributed bytes under its trust contract without proving the signed business proposition. Reuse is valid only when the exact source and applicable semantic/checker dependencies remain qualified; changes to normalization, trust assumptions, domain rules, issued executor, backend, compiled artifact or installed route invalidate the **affected kinds** of evidence. A reusable abstract theorem does not requalify changed physical implementations.

The application assembly may consume the admitted result only for its matching selected operation, plan and source generation. The trusted command, installed Ring/HTMX and browser boundaries separately enforce current authority, execution correspondence and physical resource ownership. Certificates and optional sidecars do not determine runtime business transition outcomes or grant permission to replay a logical command. Mandatory construction/installation gates do retain actual enforcement authority over their own stage; those gates do not derive it from caller-editable reporting fields.

Certify substantial HumanHelp Claim, not a toy surrogate

The mandatory pilot under #req-humanhelp-certification-pilot uses the **actual public production model** ownership for User, Membership, Organization, Location, Request and RequestAssignment where those dependencies matter. It does not invent `:request/helper-id` or rename the completed lifecycle state to `:completed` to simplify a model. The Request status vocabulary includes `:done`, and the active primary assignment is a separately guarded authoritative fact.

The Claim proof/evidence chain includes:

1. An authenticated acting helper and exact Request target under current model/Organization/Membership eligibility, with the target-helper distinction represented where the operation requires it.
2. Request lifecycle guards, primary-assignment existence/absence and cardinality, permitted cross-model dependencies and all original relevant invariant and operation contracts.
3. The abstract guarded multi-record transition, its same-transaction atomicity requirement, write footprint, negative outcomes and frame obligations under the selected model.
4. Stable logical command C42 and exact canonical binding, with independent transport attempt identity, compatible replay and historical result disclosure, and honest submitted-unknown semantics rather than invented rejection.
5. Required durable publication intent/outbox obligations, guarded writer coverage, other lifecycle and administrative entry-points, actual initialization assumptions and physical backend correspondence limits.
6. Choreo roles and knowledge/disclosure placement, source-derived projections and executable plans for committed, rejected, already-incorporated and commit-unknown outcomes.
7. The actual selected application assembly, issued executor, rendered enabled Claim affordance, HTMX route and checked browser asset/response and resource ownership relations.
8. Independently qualified real production transaction and browser/user observations for the stronger operational-realization claim, with scope, physical premises and unresolved gaps explicitly recorded.

A one-scalar Request-state induction or a Clojure map named `:claim/proof` cannot establish multi-record RequestAssignment correctness, actual transaction guards, command attribution, full writer closure, source-to-executor refinement or the real browser user path. Conversely, a genuine conditional source-relative Claim result is useful and may be issued under a profile that exposes its unproved physical edges without inventing a stronger certification.

The pilot's positive control permits an eligible helper to Claim a Request under the public model. Competing helpers and stale role/organization authority must not create two primaries; forged client identity cannot become server authority. An unknown C42 incorporation remains separate from later current Request state. Alice's own result and Bob's remote advisory/one-user-Refresh board acquisition are not conflated. The pilot does not independently complete Unclaim, Reassign, Response or the rich board: the Part VIII original product denominator is unchanged.

Require positive and adversarial conformance

Eligible checked behavior	Invalid adjacent case that must fail or remain ex
Well-typed valid business rule has the intended exact reader/presence semantics	Nil and absent input, short-circuit branch or opa
Closed checker proves supported conclusion from the real premise list	Unknown rule, forward proof reference or fabri
Source-derived obligation closure includes all original Claim/operation branches	A structurally sound graph omits the unfavorab
Valid initializer and enabled Claim provide non-vacuous required controls	Unsatisfiable initial state or reject-all system is p
Real invariant induction uses every required declared transition	Failed operation is removed from the operation
Genuine E establishing P is composed into G with correct inherited assumptions	Same-name P under another actor/basis or evide
A valid typed role-local use receives its qualified allowed knowledge	Forged or stale transported claim is treated as au
Exact source and checked plan bind to issued admitted artifact	Elaborator-generated proof report is copied into
Current checked source-relative evidence is reused only under valid dependencies	Changed rule/normalizer, source or proof check
Process-local certificate or durable package is re-admitted under its real issuer policy	Serialized success map or self-signed report bec
Real public Request and RequestAssignment Claim contract is the proof subject	Toy scalar Claim or invented `:request/helper-`
Concurrent helper Claim obeys one-primary invariant under selected guards	Both helpers win because only pre-read tests we
Genuine C42 retains binding, authorized result and unknown-outcome distinctions	Lost response is relabeled rejection, fresh C43 se
Every required authoritative writer/initializer is covered or expressly open	Abstract catalog is called the complete physical
Browser Claim uses the actual issued plan and resource owner where claimed	Passing synthetic HTTP call certifies generated,
Conditional proof retains backend and runtime premises in root evidence	Root green status converts unproved physical re
Legitimate selected opaque Clojure implementation remains usable with honest premise	Mandatory proof silently turns into an uncheck
Independent true product capabilities are still named and positively exercised	Claim pilot completion deletes Response and otl

The actual checking method, first rejecting boundary, proposition, selected subject, accepted premise, positive oracle and negative counterexample are declared for each required case. An abstract checker cannot claim to have observed an actual XTDB or Chromium effect merely because its fixture passed. A real browser trace cannot establish an unbounded inductive claim merely because one user interaction worked.

Milestone decision and boundaries

Completion requires a non-vacuous checked proof/certificate chain through the selected semantic artifact classes and at least one real source-owned multi-record HumanHelp Claim pilot. Every mandatory obligation in that milestone's original source-derived denominator is either checked or admissibly conditional under its selected profile; missing, unsupported, stale or unexecuted mandatory checks remain blockers. The issuer records the exact scope, premises, checker/proof kinds, tested positive controls, applicable sources and evidence destinations.

The claim ledger distinguishes (a) source-relative typing, judgments and proof derivations; (b) checked issuance and application composition; (c) independent external checker-soundness theorems; (d) abstract-to-physical transaction and writer correspondence; (e) installed host/browser execution and product acceptance. They cannot be reported as one automatically discharged `:certified` state.

The milestone does not require proof of arbitrary host-language code or the full externally verified kernel of 11.15 unless the selected profile explicitly includes that stronger duty. It also does not require all HumanHelp product capabilities of 11.13 to have passed simply because the Claim certification pilot is complete. The genuine original capability and proof catalogs remain visible as separate required denominators. The broader external theorem and physical deployment obligations are owned by 11.15, Part X and their selected release profiles.

The executable semantic core remains the runtime semantic source. The proof kernel checks derivations, dedicated checkers establish named judgments, and the issuer binds their admissible conclusions to exact subjects. None becomes a second source of business rules, a general runtime authorization machine or a passive proof sidecar whose text controls domain outcomes.

External formal verification completion

External formal verification is a **separate milestone with an explicit theorem and implementation-transfer frontier**. It establishes independently checked mathematical results about the exact closed Gesso semantics named by the claim and, where the requested claim includes production acceptance, an adequately justified connection to the real checker and issuer. It does not prove every possible Clojure program, whole HumanHelp product behavior, every XTDB transaction, the deployed browser or every possible multi-node history merely because one formal calculus has been mechanized.

The initial target is the seven-rule **closed propositional proof kernel** and its acceptance relation from Part IX and #req-formal-production-correspondence. It is deliberately small enough to specify independently of the runtime and to remove a specific trust gap. The independently accepted semantics and exact first-target obligations are in Part X, including 10.10; this milestone

describes which results and transfers must exist before completion can be declared. It does not create a second formal semantics or authorize Gesso authors to maintain a separate handwritten policy model merely to obtain checking.

Bind the external theorem to the original semantic subject

The formal package identifies its actual source-owned proposition, inference language, nominal formula/atom identities, premises, proof-step structure, Boolean interpretation, admitted checker rules, proof-goal relation, and the precise versions of normalization and original-source interpretation under examination.

The first calculus consists of exactly the following seven admissible **inference rules**:

Rule	External meaning and side condition
`:premise`	A declared named premise P permits derivation of P, retaining that premise as open
`:truth-introduction`	The distinguished true formula is derivable without assuming an unproved atom
`:implication-reflexivity`	A well-formed P entails P implies P
`:conjunction-introduction`	At least two checked premises yield their ordered conjunction
`:conjunction-elimination`	A checked conjunction yields exactly its permitted indexed constituent
`:implication-elimination`	Checked P implies Q, followed by checked P in the required reference order, yields Q
`:disjunction-introduction`	A checked P yields the declared disjunction containing P in its permitted position

The formal language includes the syntax and valuation of the admitted formula constructors, even when a constructor has no corresponding inference-introduction rule. No unimplemented arithmetic, SMT entailment, arbitrary rewriting, hypothetical-context discharge, excluded-middle rule, atomic-truth axiom or Clojure evaluation can be implicitly added to make a proof pass. Source well-formedness, ordered backward references, no cycles, exact rule arity, index and conclusion, supported limits and the required premise/step-use discipline remain explicit. A stronger inference language requires its own theorem and production correspondence rather than inheriting this theorem by name.

The baseline independent **soundness proposition** is: for every admitted well-formed original-source derivation D of goal P from named premises Gamma and every Boolean valuation v of its opaque atoms, if the inference relation accepts D and each premise **actually used by D** holds under v, then P holds under v. The formal inference relation is defined independently of the production Clojure checker; its acceptance is not a trusted call to that checker. A derivation with remaining open premises is a conditional entailment, not a theorem that those business propositions are true. A proof whose assumptions have no reachable model may still be logically sound but does not establish a useful or satisfiable business operation.

Complete and distinguish the five proof/transfer claims

The milestone ledger identifies the following distinct S0–S4 results and their evidence status:

Claim	Required result
S0 — inference-rule soundness	Independently specified and machine-checked preservation of Boolean entailment by each c
S1 — derivation-system soundness	Machine-checked induction over all well-formed finite ordered derivations, including exact
S2 — production acceptance correspondence	A checked or explicitly qualified relation from every **supported production-accepted inp**
S3 — typed atom/meaning bridge	The supported typed business-rule evaluator, source-reader context, presence/nil and result
S4 — issuer/composition/realization binding	Formal subject, original-source-required obligations, dependency/premise closure, current is

For the initial independent **logical** milestone, S0 and S1 must be genuinely machine-checked, not replaced by ordinary tests or a handwritten assertion that each inference rule appears sound. The declared **production- transferred milestone** additionally requires its exact selected S2–S4 obligations to be discharged by checked relations or clearly named, independently qualified trusted premises **where the original selected profile allows them**. Any still-trusted edge remains visible in the final claim; the milestone cannot call its production correspondence unconditional.

A stronger profile that requires proven S2, S3, S4 or full execution refinement cannot be satisfied by downgrading those duties to tests or assumptions after failure. A narrower S0/S1 theorem package can be reported as valuable partial progress, but it does not meet a selected release claim that requires the production transfer. Neither S2 nor S4 is established by simply pasting the same digest into a Clojure proof report and a formal result.

Establish the correct direction of correspondence

The crucial safety direction is **production acceptance implies formal admissibility** for every concrete input the actual selected production path can accept. That requires sound decoding of all input forms the path admits, source and nominal identity, typed proposition normalization, reference resolution, premise tracking, ordered step checking, resource-limit handling, and the particular certificate/issuer admission behavior. A theorem quantifying over inputs to an **ideal** formal checker does not rule out a real production parser accepting a malformed unsupported input.

The transfer mechanism must name its own trusted boundary: verified implementation/extraction, machine-checked translation, proof-producing compilation with independently checked output, or an explicit qualified translation/checker assumption. An independently formalized derivation is not linked to production simply because both tools accept a few test vectors. Calls from the formal semantics into the unchecked production checker would assume away the very implementation gap the theorem is meant to close.

A soundly **rejecting** production checker may be incomplete relative to the mathematical calculus. Soundness of accepted results does not require it to accept every valid formula, but a release claim for a useful supported fragment also needs the selected positive/control and coverage obligations. Unsupported syntax, invalid derivation, source mismatch, resource exhaustion, tool outage and search timeout remain distinct non-success statuses. A changed accepted proof calculus, checker implementation or parser interpretation needs a new applicable S2 relation even if an earlier S0/S1 theorem remains mathematically valid.

S3 and S4 do not silently close further execution claims. Proving a typed atom's abstract evaluation does not authenticate actual HTTP input or the current principal. Binding the issuer to the operation does not prove the XTDB guard was used, the browser loaded the admitted asset or every physical writer refines its declared transition. Those additional claims require the specific source-to-physical and runtime observations of Part X and the selected operational profile.

For a **safety** theorem transferred from an abstract model to concrete execution, the relevant initial-state coverage, concrete-to-abstract step/trace simulation and property reflection are separate quantified duties. For liveness, fairness, divergence, delivery, environmental availability, retained history and user-action conditions must also be justified. A single successful observed history, or model state equality without history, cannot establish those universal transfer directions.

Import genuine external proof evidence reproducibly

For each S0–S4 claim the evidence package records:

- the original semantic subject, exact theorem statement, quantifiers, formula/rule versions and intended result scope, not just a convenient theorem nickname;
- the generated formal input's source provenance, translation/extraction mechanism, original required proposition and checked normalization/encoding relation;
- proof assistant or checker product, exact version and supported language, its trusted kernel/axioms, imported libraries, options and declared resource budgets;
- the genuine proof term/checkable artifact, observed machine-check result, checking command, environment, reproducible inputs and dependency/certificate fingerprints with their provenance;
- issuer and evidence-import controls, current source, operation, application, toolchain and runtime applicability, versioned accepted premises and named invalidation/rechecking triggers;
- any still-open or explicitly trusted S2–S4 relation, assumptions relevant to installed execution, negative controls and the final conditionality of the exported claim.

A checker may be Lean or another suitable mechanized system; no one vendor, tactic script or notation is part of Gesso's universal application programming contract. Application authors specify their business rules and Choreo/Gesso construction. Generation of appropriate formal subjects, verification tasks and import metadata belongs to the framework and its selected formal backend.

A process exit code, text claiming ”proved”, borrowed screenshot, copied theorem hash or unverified serialized report is not sufficient evidence of checked theorem origin. A local independent checker must validate imported proof material under its declared trust contract, or the output remains explicitly conditional on an authenticated external checking service. Signatures authenticate specified bytes and issuer origin only under their key assumptions; they do not prove the proposition being signed.

The release gate rechecks the source/evidence binding and actual current applicability rather than treating formal verification as permanent authority. Semantic rule changes may invalidate S0/S1; source/proposition or atom-interpretation changes can invalidate S3; checker/parser changes can invalidate S2; new issuer, dependency, operation or installed artifact generations can invalidate S4. A source-compatible abstract proof may survive a physical adapter change while the actual physical transfer becomes stale. An outdated theorem remains historical until the named relation to the new subject is re-established.

Preserve exact finite and conditional evidence classes

An exhaustive finite exploration proves its named property only for the completely enumerated initial-state and transition graph of the selected finite model under its stated domain and history semantics. A bounded-depth search, sampled valuation corpus, incomplete enumeration, timeout, unavailable backend, truncated branch, or unsupported rule cannot be reported as a completed proof. Conversely, a genuine finite counterexample may be useful before enumeration finishes; its exact model and trace remain explicit.

An unbounded theorem needs the corresponding quantified derivation or a separately justified abstraction/refinement from the concrete state/input range. A restricted Boolean atom theorem is not a general arithmetic theorem. A source-relative conditional induction is not a universal physical writer theorem. A tested current backend is not a proved unbounded XTDB implementation. The report preserves those evidence classes and their trusted assumptions without compressing them to one global ”formally verified” bit.

The design must also guard against a common-mode semantic mistake: if an automatically generated formal model and the production checker both implement the same incorrect interpretation, matching test results may conceal the bug. Independent specification of the closed inference rules, a distinct formal semantics, mutation oracles and typed atom counterexamples are required. Their coverage is fixed before selecting only examples that pass.

Positive and adversarial external-formal acceptance

Required accepted history	Adjacent insufficient or invalid history
S0 independently checks all seven rule schemas and legal parameter shapes	One favorite inference instance is offered as proof of
S1 checks every well-formed finite derivation under used open premises	Theorem invokes the production checker as an oracl
Formal P under Gamma stays conditional on actual named premises	Inconsistent or unverified Gamma is silently deleted
Production accepted proof maps imply S1 formal derivations under qualified S2	Malformed accepted parser path is absent from ideal
Real unsupported rule and bad reference are rejected before certificate issuance	Abstract model rejects them but actual Clojure path
Exact normalization preserves nominal, absent/nil and typed reader semantics in S3	Two source-distinct business atoms merge because t
S4 connects theorem to current source, issuer and obligation ledger	Proof for alternate rule catalog or stale application is
Valid compatible source relative conditional proof has a useful positive	Checker rejects everything and claims perfect forma
A formally proved relation is imported through an actual trusted checker	A CLI success string or fabricated theorem package i
A selected trusted translator remains an explicit assumption	Qualifying trust is concealed and unconditional chec
Source-preserving change reuses valid S0/S1 while rechecking changed S2/S4	Old theorem fingerprint requalifies a replaced produ
Typed atom theorem is scoped to its real variable bindings	Propositional atom is asserted to prove current Orga
Complete finite model yields exact finite evidence	Depth-limited sample is labeled unbounded theorem
Source-derived induction includes every declared transition	Failing operation is removed to keep an invariant pr
Abstract-to-physical safety uses initial, step and property transfer	One matching trace is offered as universal XTDB/bro
Selected liveness claim states and discharges fairness/availability premises	Network failure is ignored and eventual completion
Real current issuer/release consumes only properly admitted evidence	Copied root report or untrusted browser proof map g
Genuine positive and negative corpus tests typed rejection and admitted use	Reject-all checker has only negative fixtures and is c
Production service with open physical writer premise reports it as open	Kernel theorem is advertised as all-writers atomic tra
Failed external checking or stale imported proof blocks strict claim	Timeout or unavailable tool is reclassified as proved

These are **prospective** formal and physical conformance obligations; a written example is not machine-checked evidence. Each case specifies its quantified subject, source/checker or physical oracle, expected evidence type and first responsible failure boundary. Formal validity, implementation soundness, issuer admission, and production execution remain separate test or theorem subjects.

Exact milestone disposition and onward ownership

The baseline external milestone is achieved only for the exact named formal kernel and production-transfer scope whose mandatory S0–S4 judgments have genuine admissible results or permitted, explicitly retained premises under the selected profile. Machine-checked S0/S1 and independently accounted S2–S4 are the **minimum** baseline; this does not assert unconditional proof of production acceptance when S2 is still conditional. Any selected stronger requirement for machine-checked implementation correspondence, typed evaluator refinement, all-source induction, knowledge/progression, Choreo projection or browser/XTDB trace refinement requires its own completed proof and transfer, never a silent weakening of the requested theorem set.

A released claim records exactly which formal statements are proved, their remaining premises, which source and runtime interfaces are covered, the current issuer/evidence package, actual checked imports, unsupported or unavailable obligations, and every relevant trusted-base and transfer boundary. A successful **formal baseline** is not a certificate of all Gesso behavior or complete HumanHelp product acceptance. Those remain scoped to 11.13–11.14, 11.16 and the separately specified requirements of Parts VIII–X.

The executable semantic core determines runtime semantic meaning. Formal artifacts justify claimed inferences; the kernel and dedicated checkers validate admitted formal evidence; runtime guards independently enforce current trusted effect and disclosure rights. Neither external proof sidecars nor a root report may become a second business policy or a direct command/DOM authority.

Multi-node production completion

This milestone qualifies the **same admitted application semantics** across actual independent processes, physical host replacement and the declared production topology under #req-multi-node-realization. It does not define a second distributed business-rule system. Source-owned operation meaning, current principal and Organization authorization, stable command identity, knowledge/progression, optimistic settlement and browser acquisition remain the same on one host or several. Distribution adds physical placement, routing, shared-authority, publication, interest, retention and failure correspondence obligations.

The milestone is distinct from operational application completion, compositional certification and external formal verification. A two-process prototype may be useful while those other scopes remain open, but no report may promote a shared database plus a load balancer into a complete distributed production claim without the required real fault and recovery evidence. The claimed envelope names actual hosts, backend, network, clients, routers, build generations, traffic, resource limits and tolerated failures.

One semantic contract, several physical owners

A **choreography role**, authenticated actor, principal, domain authority, command, HTTP attempt, host process, router, installation generation and browser resource are distinct identities. Host A's failure and host C's replacement do not transfer Request authority by process name. C may serve the same qualified public operation only by authenticating its current invocation, selecting the admitted operation/executor, validating the authoritative model guard, and demonstrating the selected backend and physical writer contract.

The requested production envelope accounts for the following as separately qualified physical subjects:

Physical subject	Required completion evidence
Installed hosts and ingress	At least two genuinely independent application processes, actual routing/failover, deployed handlers and
Original source and writers	Exact managed operation/initializer, administrative/migration, scheduled/background and external privile
Shared authority	Qualified selected backend/adapter transaction semantics, visibility, concurrency and read/progression rel
Command incorporation	Same canonical binding and logical ID on every host, atomic guarded transition and deduplication under
Required publication	Durable intent coupled to the qualifying authoritative write where promised, surviving dispatcher owners
Router and interest state	Authorized interest discovery, effective registration, filter/update/failover coverage and observed gaps or
Subscriber/browser continuity	Initial and reconnect fences, replay/retention or re-acquisition, transport/advisory separation, current reso
Mixed-generation deployment	Real A/A, A/B, B/A and B/B directions, enforced exclusions, retained assets/codecs, old execution retireme
Capacity/fault envelope	Measured concurrency, queues, outstanding debt, bounded replay/retention, backpressure, overload and s
Qualified evidence	Independently observed positive/negative histories, source and environment identities, root claim depend

The production implementation may use one or several routing processes, replicated/coarse or hierarchical interest filters, redundant worker assignments, conservative invalidation redelivery, and qualified external brokers. Those are possible physical realizations, not a mandated vendor, routing language or universal distributed participant fabric. Router work can first use Clojure and later another implementation if its wire, failure, authorization, subscription and recovery contracts remain checked. Multi-node placement must not require application authors to describe a new business operation or adoption policy for each physical deployment.

Preserve authoritative command identity across loss

A bound Claim C42 sent through A retains its **one logical command identity** if a later permitted attempt arrives at C. The new HTTP/transport attempt has its own identity and authentication context. The actual authoritative transaction and durable command record, not an A-local cache or C's interpretation of the current Request, decide whether the original command incorporated and which recorded outcome can be disclosed.

The selected operation supplies a real atomic incorporation or separately justified equivalent of its domain transition, command identity/binding and required result history under the qualified backend. Two concurrent hosts cannot both produce a new primary assignment in violation of the same source model, nor may competing retries of C42 create independent authorized business actions. The original required public-model and external writer inventory is considered during proof and physical correspondence. A test of one successful HTTP route cannot attest all XTDB or administrative writers.

A replacement host checks original operation, target, canonical arguments, principal and authoritative binding before compatible retry or historical result lookup. Current authenticated scope and disclosure policy are checked again; the old host or client-supplied command ID grants neither submission nor result access. Expiration of the supported deduplication window is enforced through retained record/tombstone or another admitted reject/reuse policy; deletion cannot silently turn an old ambiguous ID into fresh intent. Loss of a response after possible commit is `:commit-unknown` until adequate historical evidence supports another outcome. An ordinary current-state read cannot attribute C42 to A or C.

The following fault windows have different observable duties:

Fault window	Required classification and retained obligation
Before any trusted submission	No incorporation may be invented; valid unsent intent can be admitte
After possible submission, before trusted result	Preserve original C42 binding and unknown incorporation; do not infe
After authoritative commit, before durable publication	Retain promised atomic publication intent or proven equivalent; doma
After distribution, before dispatcher completion acknowledgment	Permit duplicate advisory dispatch without duplicate domain mutation
During HTTP/SSE result delivery or host failure	Preserve truthful command and publication debt independently of rece
During replacement/expired record retention	Maintain exact permitted reconciliation horizon or typed blocked/unk

A valid command-positive test must reach the actual protected gateway and authoritative backend, alongside forged, duplicated, concurrent and unqualified cases at that same boundary.

Make required publication recoverable without coupling it to live interests

A model change, a required durable publication intent, a Gesso Live invalidation, router forwarding, transport delivery and browser adoption are distinct events. An authoritative writer may generate required durable intent from its original declared semantic change and notification scope without knowing every ephemeral recipient currently interested in the data. A writer-local interest cache is not the authoritative list of every current or future subscriber, and failure to observe a particular browser during commit cannot silently cancel a promised recovery notice.

Where the selected operation promises recoverable publication, its domain effect, durable command result when applicable and outbox intent are incorporated together under the selected backend's qualified atomicity, or via a separately justified equivalent that cannot lose required intent after commit. A best-effort advisory may have different loss rules only where its original declared weaker contract permits loss. Published transport data must contain only the authorized minimal notification scope, not a protected Request snapshot that bypasses current read permission.

A surviving dispatcher claims pending intents under a qualified ownership/lease or equivalent concurrency policy. Failure before/after send or acknowledgment does not strand required work in the dead process. Dispatch may be conservative and at least once to its **named acknowledged boundary**; no unnecessary per-recipient permanent success ledger is imposed when duplicate authorized advisories and bounded replay/recovery suffice. That choice does not license unbounded duplicate amplification, false completion acknowledgments or lost required notification semantics.

The release evidence separately records authoritative commit, retained outbox intent, dispatcher claim, broker/router acceptance, selected interested host, SSE frame attempt, client receipt, authorized acquisition and physical DOM adoption. Neither a broker acknowledgment nor an SSE write demonstrates that every subscriber received or adopted the change. A duplicate invalidation may ask a permitted client to recheck current state; it never creates a second Claim or grants permission to adopt a remote board.

Close interest-registration and router-failover gaps

An initial subscription races with authoritative changes. A committed change may precede an interest being effectively known to the router; a host-local filter may have learned a client the replacement router has not yet learned. The selected protocol must establish a registration fence, replay/gap record, qualified comparison frontier or conservative authorized re-acquisition sufficient for the claimed subscription semantics. The system cannot infer currentness from silence on a stream.

Effective interest, retained publication interval and covered authoritative basis must be related by an independently qualified coordinate or checked bridge when they use different token schemes. A token echoed from the request, a router-local ”seen” bit, or a healthy backup process is not evidence that an interested recipient's interval was covered. Loss, reordered registration, expired replay cursor, changed filter or routing cutover must either preserve the intended coverage or expose an unresolved gap that blocks the stronger currentness/delivery claim.

A distributed interest filter is an optimization over an authorized notification candidate set. A partitioned or hierarchical filter cannot exclude a recipient solely because its local view is incomplete. Replica/filter completeness, version exchange, stale-router fencing, fall-back fanout and bounded conservative redelivery are eligible mechanisms when their own capacity, leakage, deduplication and coverage premises are met. A failed router may be recovered by re-sending more notices than necessary; correctness does not require remembering every earlier successful destination if that larger delivery is safe and its work bound is justified.

Overdelivery may still leak tenant membership, Request existence, participant activity or routing metadata. Current authorization at dispatch, subscription and protected read remains obligatory; ”conservative” does not mean broadcasting protected data to all. A receiver cannot claim delivery of a required invalidation that never reached it merely because another router had a compatible Bloom/vacuum filter or a host reported itself healthy.

After an observed gap, the system may refuse to claim current coverage and retain advisory or recovery debt until a current permitted read at sufficient provider basis discharges the specified scope. It cannot turn an unobserved missing event into a fictitious positive stream receipt or silently clear debt.

Preserve source progression, authorization and selective adoption

A replacement host reads under a qualified provider, transaction basis and query scope; the frontier must be a real observed provider fact and sufficient for the required original semantic progression. Transaction time, valid time, domain revision, command causality, stream cursor, interest-frontier and browser knowledge are not exchangeable scalars. A lagging read or unsupported provider option returns a typed insufficient/ unavailable state rather than reporting that the requested basis was attained.

Each host and router reestablishes current authenticated principal, Organization Membership, operation eligibility, protected subscription, result disclosure and query coverage at their actual trusted boundaries. An old connection, host ID, serialized certificate, source digest or stale permission snapshot cannot authorize a new effect after revocation. A replacement process may retain valid command evidence while properly denying an ineligible principal the result.

For HumanHelp, Alice's own qualified committed Claim can settle her scoped pending action. Bob's remote Request lifecycle change remains **advisory** until his explicit user-authorized Refresh, subject only to the separately declared own-command and affected-card repair exceptions. A new router, host, websocket/SSE reconnect, asset reload or backfilled invalidation never manufactures a Refresh grant. The permitted read must cover the required cohort and progression; an old callback cannot install into a new DOM resource or erase incomparable newer advisory debt. Withheld authorization requires nondisclosure, not fabricated convergence.

Direct participant-message features may have independently specified ephemeral or durable delivery semantics. A best-effort typing message is not a recoverable domain-state invalidation. No universal global actor directory, durable suspended choreography engine or router language is necessary for ordinary Request interactions; if a selected direct-message feature promises stronger receipt or persistence, its own protocol and tests remain mandatory.

Establish real mixed-release, resource and fault evidence

The release profile fixes the actually reachable A/A, A/B, B/A and B/B client/server directions from 11.08, complete source-derived continuation contracts and retained assets from 11.09, and semantic retirement/ command recovery from 11.10. A safe per-host activation does not prove an all-host atomic cutover. Host replacement and failover must exercise compatible request and browser asset paths, pending work ownership, old/new schema reader and writer safety, stale client/stream retirement and current credential/authorization conditions.

The physical acceptance suite uses independently isolated application processes and the actual selected XTDB, transaction adapter, router/broker boundary, installed Ring/Biff handlers, real browser/transport assets and representative authenticated users. Test setups which share in-JVM process state where independence is claimed, intercept only a test wrapper or label two local maps as two hosts cannot establish this milestone. A negative case is credited only when the intended **real** enforcing boundary rejects the invalid action, and a related eligible positive case succeeds through that same gateway.

A production performance and failure envelope identifies host/router/dispatcher count, process and queue limits, interest/filter cardinality, outstanding outbox/debt, event rates, retention, replay pressure, failover latency, load-balancing strategy, target/browser population, backend consistency/availability, network partitions and supported recovery duration. It measures capacity, isolation, overload, queue backpressure, misrouting and leak-free retirement under those specific conditions rather than quoting only peak successful request rate. Unbounded memory, queued callbacks, publication debt or retained assets do not become acceptable because a short throughput test passed.

Recovery/progress evidence names when backend and network service return, router/filter knowledge converges, durable records remain available, current authorization permits disclosure, and a participant actually performs any required Refresh. Without those premises, safety and honest blocked/ unknown status remain the required result; unconditional eventual browser convergence is not promised. Revoked Membership may make correct outcome nondisclosure rather than success. The selected release's stricter progress claims need appropriate fairness and anti-divergence evidence, not just successful finite examples.

Multi-node positive and adversarial production histories

Required qualifying history	Nearby invalid result or claim
Independent A and C processes exercise one original production Claim meaning	Two process labels share one process-local r
Real XTDB model guards permit one of two competing helpers to become primary	Cross-host stale reads permit two active prin
C receives C42 original binding and authorized replay or historical result	C creates fresh command C43 merely becaus
C preserves truthful `:commit-unknown` until sufficient command evidence exists	Current Request state or missing record is fa
Committed Request, command result and required publication intent survive A loss	A dies after commit and sole in-memory call
Surviving dispatcher safely repeats an unacknowledged advisory	Failed acknowledgment doubles the authorit
Broker/router acknowledgment is labeled only for its real boundary	SSE send or broker success is claimed as ever
Subscription registration is fenced with read/replay coverage	Commit between initial read and effective in
Replaced router covers stale or unknown interest via checked recovery	Healthy backup with stale filter silently drop
Conservative authorized redelivery repairs uncertain routes under resource bounds	Replayed event exposes other Organization's
Expired replay cursor produces typed gap and permitted re-acquisition	Stream silence or echoed cursor is reported a
C performs sufficient provider-basis read or reports insufficiency	Lagging snapshot is installed as satisfying a
C authenticates current principal before protected command/result read	Old A session or source digest reauthorizes r
Bob gets remote advisory, then explicitly refreshes authorized covered board	Cross-host notice or reconnect automatically
New owned browser resource installs current result under justified basis	Retired A callback overwrites B's DOM or cl
Genuine compatible mixed A/B client-server routes remain enforced	Sticky routing test misses failover to unsupp
Supported rollback respects actual schema, pending durable work and assets	Prior passing binary is activated against inco
True router or host loss is injected and independently observed	Crash simulation does not terminate claimed
Real positive protected Claim succeeds beside adversarial rejection	Reject-all handler passes negative-only tests
Bounded queued/debt/filter work recovers after declared overload	Unbounded memory/worker leak hidden by
Unauthorized ephemeral typing is discarded under current recipient rights	Best-effort message is treated as durable Req
Root ledger records open physical writer/host premises and exact topology	Local report, copied status or two-host demo
Required liveness progresses once declared fairness/availability/user-action conditions hold	Offline Bob with no Refresh is said to have a
After principal revocation, safe nondisclosure is observed	Repair/replay leaks protected historical com

These are **prospective** physical, source-semantic and deployment tests, not observations that an implementation presently satisfies this milestone. A successful two-host pilot may satisfy selected bounded cases while failing the required production load, failure or route/interest envelope. The complete claim retains the original selected requirements and every missing mandatory case; it cannot shrink the denominator to successful observed paths.

Milestone decision and remaining scope

The multi-node production milestone is admitted only for its exact specified topology, authoritative backend/adapter, installed source and artifact generations, selected fault and resource envelope, exposed routing directions, current security assumptions and authentic observed tests. It verifies that the same Gesso/HumanHelp operation semantics survive the claimed host movements, publication and interest gaps, command uncertainty, authorized acquisition and release overlap, with all required external premises transparently qualified.

The root result identifies original-source required coverage, actual installed host/writer/dispatcher and router evidence, selected provider qualification, positive and negative fault oracles, load and retention bounds, completed gates, still-open assumptions and any unsupported profile requirements. Source theorems, formal proof issuance and current runtime/operation authority retain their own named claims: neither production resilience nor successful fault injection proves general semantic soundness or independent kernel meta-soundness.

A safer, narrower deployment may be separately admitted where its independently selected profile and actually enforced routing/recovery restrictions permit it. A failing strict multi-node claim cannot silently be reported as complete under a different profile. The same semantic machine remains the target of one-node and many-node implementations; later routers, brokers, Rust optimizations or new host layouts are physical replacements requiring their own appropriate correspondence and failure qualification, not a second application policy.

The four scopes do not collapse into one status

Completion is a set of **four separately justified milestone judgments**, not one status called ”Gesso complete” or an ordering constraint on project implementation. The subjects are:

Milestone	What its complete result actually establishes
Operational application	All independently required HumanHelp product capabilities work through qualified installed model, host an
Compositional certification	The named source-complete, non-vacuous typed/checker/issuance chain and genuine multi-record Claim pil
External formal verification	The exact accepted independent formal theorem and qualified S0–S4 correspondence frontier of the selected
Multi-node production	The original single-machine semantics survive the selected actual independent-host, router, recovery, compa

These milestones have independent **requirement denominators** derived from Parts VIII–X and 11.13–11.16. Their work can overlap, and one can be complete before another. They are not aliases for development stage numbers, a cumulative project percentage, or automatic implications in either direction. A successful Claim proof pilot can coexist with missing Response interaction; a usable operational application can coexist with an unproved external checker; a physically resilient two-host application can still have an unestablished semantic theorem.

They are also distinct from the **four release decisions** of 11.02: candidate admission, installation eligibility, activation eligibility and completion. A candidate can be correctly admitted but never installed, installed but not activated, or active under an allowed smaller release intent without meeting a separately requested stronger completion milestone. An abstract completion theorem does not authorize installation or a live command. A permitted operational release does not become a failed release merely because an unselected formal milestone remains unfinished; conversely, it cannot satisfy an originally requested strict formal or multi-node release profile by silently renaming the requested claim.

Completion records bind original requirements and actual evidence

A completion record is a **checked result for one named release intent, milestone, subject and environment**, derived from a source-complete requirement ledger and qualified evidence. It is not an independently authored status word. The root human-readable and machine-readable views use the same validated underlying records; neither can issue a certificate, invoke a protected operation or change a source requirement by editing an explanation.

The record identifies all applicable coordinates:

- **Original subject:** canonical source and independently required product/model obligations, their owner, version and complete denominator, including separately unresolved product choices.
- **Selected claim:** one milestone or an explicitly requested collection of milestones, selected assurance profile, required positive/negative histories and predeclared release intent before checks execute.
- **Semantic candidate:** source-bound normalized rule/operation, Choreo, projected/executable and application-assembly identities; admitted artifacts, exact browser manifest and compiler/receipt relations, with their relevant generation/versions.
- **Installed realization:** actual selected Biff/Ring handler, guarded writers, providers, backend, workers, host/router and serving/loaded asset bindings; site, topology, configuration and evaluation window to which physical evidence applies.
- **Four release decisions:** separately current candidate, installation, activation and completion subjects, evidence and applicability. A stale host or newly revoked backend premise must not inherit an older activation result.
- **Each required gate:** original obligation ID, exact intended proposition or effect, issuer/observer and evidence class, original source, environment, physical/semantic boundary, checked result, qualified premises, dependencies and reproducible or authorized evidence reference.
- **Eight gate-result classifications:** `:passed`, `:failed`, `:incomplete`, `:unavailable`, `:unsupported`, `:skipped`, `:stale` or `:missing`, used only after the corresponding source and evidence record is genuinely checked. A different report vocabulary may summarize milestones without changing gate truth.
- **Current limitations:** mandatory unmet obligations, genuine accepted trusted premises, unselected product decisions, unavailable jobs, exclusions, finite proof bounds, tested physical fault/scale conditions, unresolved command/Live/recovery horizons and explicit liveness premises.
- **Change lineage:** previous claim/evidence subjects, exact source and implementation differences, dependency invalidation, reused still-applicable evidence and separately required requalification.
- **Outcome:** each milestone's computed complete, partial, blocked or otherwise accurately limited disposition, the first responsible failure boundaries and the stronger claims not established.

An explanatory record can display `:partial`, `:prototype` or `:not-complete`, but those labels do not replace the eight gate results and are not proof issuer handles. A complete disposition requires every **mandatory original gate** in the selected profile to have genuine applicable success or a specifically allowed conditional qualified premise under the predeclared policy. Mandatory unproved obligations cannot be reclassified as voluntary assumptions because their checker failed.

The initial denominator is not whatever a generated catalog happens to expose. A missing HumanHelp Response route, an undiscovered external writer, a proof family omitted by an elaborator or an unavailable physical test is still counted when independently required. The record can separately report closure of the currently exposed assembly and coverage of all original required product capabilities; it may not combine their percentages or replace the latter with the former. An approved real change of product scope creates a new identifiable requirement subject and retains the old comparison and authorization for that change.

Applicability is current and claim-specific

A historical completed result states what was established **then**, about the exact release, source, assumptions and environment it actually checked. It is not a permanent property of a repository, branch, application name, model key, deployment label, digest or browser cache. An independently proved theorem can remain mathematically sound after a runtime configuration change while its claimed application to the newly installed runtime becomes stale.

A change-impact computation follows the dependencies of the exact affected claims. A change in a source rule, original requirement, normalization, proof checker, issuer, trusted premise, production writer, backend incarnation, provider, Biff handler, emitted browser artifact, installed router or release compatibility may invalidate different portions of the evidence graph. The affected result is marked stale, incomplete or blocked as appropriate until independent requalification. A genuine unaffected theorem, source relation or valid physical observation may be reused with its original subject and current applicability established. ”No need to rerun everything” is allowed; ”keep every green line after a dependency change” is not.

Known revocation of an accepted principal, Organization Membership, credential, model policy or backend safety premise fences the dependent protected runtime effect or disclosure at the actual trusted boundary. A report's historical success cannot preserve execution permission. Likewise, an invalid previous assembly is not safe fallback merely because a newer installation failed; 11.07 governs current rollback, recovery and restricted service.

Change status also preserves **what kind of evidence survived**. A valid abstract invariant may persist while the source-to-physical writer coverage is stale; a complete bounded model check does not become an unbounded theorem; a sound external kernel theorem does not requalify a changed production parser; an application preflight report does not prove the running listener uses the admitted handler; and a historical successful client does not authenticate the bytes loaded by a later sleeping browser.

A change to documentation can alter the **requirements** and thus the claimed target, with the usual requirement-version and denominator reassessment. Editing documentation cannot itself establish that code has passed a newly described test. The editorial acceptance of a normative Part is not any of the four implementation milestone completion judgments.

Worked record: useful progress with four different answers

Consider a **hypothetical** HumanHelp candidate with some genuinely checked Create, Claim, board option and explicit Refresh paths, but without an independently selected and accepted Response interaction contract. It also has useful conditional proof and two-host experiments. The correct report does not flatten these facts into a single green project result:

Completion subject	Hypothetical established evidence
Operational application	Some actual original production-model and browser capabilities plus installed assembly paths are qualified
Compositional certification	Typed Claim/guard, source-bound conditional checks and part of the issued proof chain are qualified
External formal verification	A bounded derivation or finite exploration has actually been checked
Multi-node production	Two genuinely independent processes can access current shared model state and selected limited advisory b

The following is **schematic explanatory data**, not a concrete Gesso API, a runnable acceptance fixture, an issued certificate or an executed test result. All identifiers are illustrative rather than references to real checked evidence.

```
{:record/kind :illustrative/completion-explanation
 :release/intent :illustrative/humanhelp-operational-intent
 :requirements/original :illustrative/part-viii-required-set
 :candidate
 {:source :illustrative/original-source
  :semantic-assembly :illustrative/assembly-A
  :assembly-generation :illustrative/generation-G
  :browser-artifact :illustrative/exact-bundle-G}
 :installation
 {:host :illustrative/host-H
  :handler :illustrative/guarded-handler-H
  :backend :illustrative/selected-backend}
 :required-product
 {:selected-policy :remote-lifecycle/explicit-refresh
  :open-decisions #{:humanhelp/response-flow}
  :verified-scope #{:request/create :request/claim
                    :board/options :board/explicit-refresh}
  :missing-scope #{:request/respond}}
 :examples-of-gate-results
 {:operation/claim {:status :passed
                    :evidence :illustrative/claim-observation}
  :operation/response {:status :missing
                       :evidence nil}
  :physical/host-loss {:status :missing
                       :evidence nil}}
 :completion-explanation
 {:operational-application :partial
  :compositional-certification :partial
  :external-formal-verification :not-complete
  :multi-node-production :prototype}
 :warning :not-admitted-evidence}
```

Even in this illustrative record, a `:passed` label has **no evidentiary force** without independent issued or observed matching evidence. The `examples-of-gate-results` are not the required complete gate inventory; the missing entries cannot be treated as inapplicable. A real root record must resolve every mandatory gate and its subject, admitted issuer/observer, original source, environment, premises and actual receipt. The `:illustrative/` identifiers authenticate nothing. One actual original-source Response contract cannot be replaced by a placeholder button or a copied result map.

The remote lifecycle policy in the example is an **already selected** explicit Refresh rule, not another unresolved product decision. Remote advisories cannot be silently adopted because a new runtime was installed, and a recovered own-command outcome does not confer unrelated board acquisition authority. If the Response choice is later actually selected and all required operational gates are exercised and qualified, the same product may attain operational completion while the formal and multi-node milestones remain independently partial. A release permitted by its actual operational profile need not await an unselected stronger milestone. A release that originally requested a strict external proof cannot obtain permission by changing the displayed claim label after the proof failed.

Acceptance controls for completion records and applicability

Required valid result	Nearby false claim to reject or conta
Four milestones retain independent original requirement sets	One green "Gesso complete" status
Four release decisions name candidate, installed host, activation and completion separately	Valid candidate assembly map becor
Every mandatory gate has checked evidence or explicit blocking disposition	Missing physical observation disapp
Accepted conditional proof retains named actual trusted premises	A presumed XTDB/issuer premise is
Real HumanHelp Response decision is selected and positively exercised before full operational claim	Claim demo or placeholder Respons
Selected remote lifecycle explicit Refresh remains current and its grant scoped	An unrequested remote reread is tre
Valid old semantic theorem is reused when only physical adapter changes	Reused theorem is presented as new
Changed original rule or required proof checker invalidates dependent evidence	Old certificate handle, report hash c
New browser bytes or serving host requires its relevant physical checks	An old emitted-asset receipt certifie
Known revoked Membership fences protected actions regardless of release badge	Historical completion continues gra
Full requested strict release fails when its mandatory stronger milestone is open	A failing proof is reclassified as an c
Separately approved narrower preview retains original missing scope	Smaller exposed UI operation inven
An operational release remains eligible when unselected external work is partial	All four milestones are incorrectly i
Real two-host observation is credited only to its bounded topology and test	Shared XTDB and two process nam
Finite formal result retains its bounds and independent source	One model run or fabricated externa
Missing command result remains unknown under current disclosure and retention	Cached historical report or Request
Valid release change marks only genuinely dependent claims stale	Every edit discards unrelated sound
Primary gate failure survives cleanup-report processing	A secondary report failure masks th
Root machine-readable and human summaries derive from the same checked ledger	Human text or serialized EDN statu
Prior completed release remains historical with exact environment/provenance	An earlier green completion label is
Conditional liveness requires actual availability and user-action premises	Offline user who never Refreshes is
Genuine positive supported candidate works under the same gate rejecting invalid candidate	Reject-all release mechanism passes

These are prospective first-boundary and positive/adversarial checks, not executed acceptance observations. The record may explain partial progress while every still-required gate remains visible. Its value is that a downstream consumer can identify *which source proposition has been checked, for which actual subject, by whom, under which premises, through what enforcing boundary, and what changed since*. The completed record never becomes an independent source of business rules or a command/DOM authority.

Acceptance for changes to the release mechanism

The release and certification mechanisms are themselves **part of the system being qualified**. An application can have correct rules and still be deployed incorrectly because an obligation was not discovered, a stale certificate was trusted, a source-equivalent adapter change escaped physical requalification, or a tested handler was never installed. This section defines change-driven and adversarial acceptance of the **actual release path**. Parts I–X own the underlying semantic, proof and physical contracts; 11.01–11.17 own candidate admission, evidence, activation, compatibility and completion scopes. This section checks their composition without creating another business policy, issuer or arbitrary host-language proof mechanism.

A release-system test must have an independently fixed original requirement, a declared eligible positive, a controlled invalid or interrupted alternative, the specific expected first rejecting boundary, an observable result/absence-of-effect oracle, and an attributable implementation/environment subject. A passing report produced by the mechanism under test is not an independent oracle that the mechanism was invoked or correctly enforced.

Challenge dependency invalidation and trustworthy evidence reuse

A correct change-impact decision follows the actual meaning and implementation dependencies of each claim. Semantic identity, normalized source, proof and checker rules, premise qualification, concrete callable/provider, exact emitted bytes, installed host/route and client continuation are distinct coordinates. A change can invalidate one layer without changing another. Both unsafe reuse **and unnecessary global invalidation** are rejected when the selected reuse contract promises incrementality.

Controlled change	Requir
Change Claim eligibility or a rule's presence/refinement meaning	Rebuil
Change an opaque implementation, XTDB adapter, model writer or physical provider while retaining semantic operation identity	Requal
Change normalizer, proof rule, checker, proof importer or issuer version	Reche
Corrupt, forge, truncate or replay a cached package or passing report	Safe de
Change compiler, manifest, browser bytes, decoder, handler, deployed route or serving provider	Requal
Change principal, Membership, credential, provider-incarnation or backend safety premise	Fence a
Add, remove or fail to discover an original product capability, non-HTTP writer, proof branch or external obligation	Reche
Reorder physical startup, activate concurrently, replace a host or fail after partial initialization	Qualify

The release ledger records the changed proposition and every dependency reconsidered, which evidence was validly retained, which physical claim became stale, and why unrelated evidence remained applicable. A green cached summary cannot issue or preserve a certificate by itself; an abstract theorem that remains valid does not authenticate a newly deployed adapter, selected handler or browser bytes. Required checks that cannot execute stay explicitly nonpassing, not implicitly accepted because recomputation is costly.

Test the exact decisions and mandatory gate vocabulary

The selected release intent and original independent capability/proof denominator are fixed **before** candidate gate evaluation. Construction then derives its closed required ledger from actual source and selected deployment scope, including branches, writers, protected reads, browser paths and required positive user capabilities. A generated partial catalog, missing UI control, unreachable test, absent proof producer or disabled policy cannot shrink the unchanged intended subject.

Every mandatory gate retains the closed result classification `:passed`, `:failed`, `:incomplete`, `:unavailable`, `:unsupported`, `:skipped`, `:stale` or `:missing`. A malformed result is an error, not a ninth success value. The actual checker/observer and applicable evidence must justify a `:passed` disposition; a copyable status row or unobserved test completion cannot. Failing, unexecuted or expired required gates block the exact requested decision. A valid trusted premise can remain **conditional** if the selected profile permits it; it is not independently proved, and a mandatory failed theorem cannot be moved into a premise list without changing the claim.

The four release decisions are challenged separately: candidate admission; actual installation; traffic activation under coherent overlap and safe existing work; and named milestone completion. A candidate may be correct without being installed, a genuine installed guarded Biff handler may not be selected by a running listener, and a locally activated host does not establish global mixed-node compatibility. The four independently scoped completion milestones of 11.13–11.16 are likewise not aliases for the four release decisions or a single green bit.

Test the required **positive** candidates under the same actual gates as invalid ones. A release mechanism that rejects all candidates, disables every protected button, fails all external proof checks or never emits an executable artifact cannot claim success because its negative safety controls are all green. A legitimate supported conditional proof must retain precisely its permissible premises, while a release that originally requests a stricter proven result must refuse any post-failure silent downgrade to a narrower assurance profile.

Observe activation, rollback and physical failure windows

The qualifying observation ladder reaches the selected physical boundaries: original semantic checker; genuine proof issuer/re-admission; generated application assembly; actual Ring/Biff handler and protected non-HTTP writers; emitted, served and loaded browser artifact; chosen backend transaction; subscription/router transport; and independently authorized browser acquisition and DOM installation. Each observation is credited only at its real boundary. A test-only wrapper, invented receipt or observed HTTP 200 cannot authenticate a different production gateway or prove business command incorporation.

For each possible failure schedule identify the earliest **actual** protected effect, the candidate/host/asset generation and applicable current authority, the fault injected, the intended rejecting/fencing boundary, any effect already committed, and the permitted final user-facing status. An invalid attempt should fail before the forbidden physical effect where the enforcement contract requires it. Detection after an irreversible commit may be diagnostically useful but cannot be counted as prevention of that effect.

Concurrent installation and activation must not expose a half-published mix of routes, executors, guards, providers or browser assets. A stale candidate cannot silently overwrite a newer active generation. Partial initializer failure accounts for acquired sockets, workers and publications even if there is no returned resource map; cleanup failures retain the primary gate error and independently fence unadmitted effects. Fallback to a previous candidate is allowed only while its **current** backend, security and release premises remain valid. Rollback is new checked activation against actual migrated state and retained work, not reversal of committed domain history.

A response timeout or host restart after possible authoritative submission does not imply that the command was rejected. The original logical command binding and identity survive release retirement, compatible reconciliation and any admitted retry. A current Request read does not attribute a prior C42. Durable publication, SSE delivery, current authorized acquisition and visible browser adoption have independently named status and recovery obligations. Retiring old execution removes obsolete effect and DOM rights without erasing truthful historical command evidence.

Require real browser, distributed and product oracles

For every profile that claims real user or installed-system behavior, exercise the actual rendered user control and issued plan, selected protected backend/handler, source-owned outcome, loaded browser/HTMX/SSE runtime and qualified physical read/DOM result. A manually synthesized internal command bypassing the rendered affordance has narrower coverage. A successful optimistic effect, transport event or stream frame is not an independently observed authoritative transaction or remote user's adoption.

HumanHelp retains its independent product capability denominator, public model authority, six lifecycle mutations and still-open Response product choice. The remote lifecycle adoption policy is already selected as advisory until explicit authorized Refresh, subject to its narrow own-command and affected-card recovery. Neither release restart nor router reconnect creates a one-use Refresh grant or clears incomparable newer advisory debt. An outdated browser resource cannot overwrite newer authorized state. A full HumanHelp operational claim must show real eligible success for the required user capabilities, not only rejections or Claim.

A multi-node claim requires independently running processes, shared authoritative semantics under actual selected XTDB premises, protected command result and outbox failure windows, subscription registration fences, stale router/filter failover, authorized replay or qualified re-acquisition, and actual A/A, A/B, B/A, B/B directions. A healthy backup router with an incomplete interest filter is not proof that a required notification was delivered. Conservative redelivery remains eligible when its authorization, resource and recovery constraints are demonstrated. No universal participant fabric or one particular router implementation is demanded.

Conditional liveness retains its backend availability, fairness, history retention, network, reconnect and explicit user-action premises. An offline recipient who has not Refreshed is not reported as having adopted all updates. An unavailable or timed-out physical observer cannot issue a positive result merely because it did not observe a counterexample.

Acceptance matrix: eligible release versus nearby defect

Positive required behavior	Invalid nearby result or mechanism to
Changed Claim rule invalidates complete semantic cone; unrelated search proof remains reusable	Unrelated proof always rebuilt or char
Current abstract theorem reused with changed physical adapter separately qualified	Old adapter's successful XTDB test sil
Genuine proof source/issuer re-admits only supported derivation	Corrupt cached success or copied repo
Exact mandatory operation and product denominator stays fixed	Missing Response or writer disappears
Correct strict profile blocks when required theorem is unavailable	Mandatory proof is called optional or
Explicit allowed assumption stays conditional with dependent claims	Accepted premise is labeled proved or
Each required missing, stale, skipped, incomplete, unsupported, failed or unavailable check blocks	A missing shard, status string, timeout
Legitimate eligible Claim works through actual selected enforcement gate	Reject-all route or test-only guard pass
Genuine checked Biff handler is selected by the live listener	Correct unused handler map is mistak
Current exact compiled JS is served and loaded with applicable receipt	Stale JS, reused filename or forged arti
Concurrent B activation preserves coherent binding and predecessor condition	Late C, built against A, silently overwr
Failed partial setup retains primary error, owned cleanup and still-valid A	B cleanup stops A shared resource or a
Invalid old credential/backend premise fences A even if B installation fails	Previously green A is kept accepting u
Real rollback qualifies old code against current migration and pending work	Historic passing binary is restarted wi
C42 may commit, response is lost, and current authorized resolver preserves uncertainty	Timeout becomes rejection, or retry al
Current protected lookup observes genuine historical command record	Current Request state or user-supplied
Committed effect and required outbox intent survive dispatcher/host loss	Sole in-memory SSE callback disappea
Duplicate delivery asks authorized receiver to recheck state, without repeated Claim	Duplicate invalidation repeats authori
Subscription read/registration and router failover preserve scope and coverage	Healthy router with stale filter silently

Positive required behavior	Invalid nearby result or mechanism to
Permitted A/B and B/A directions have qualified routes, decoders and assets	Happy-path sticky session hides invali
Current principal and browser target rights are checked after release change	Old assembly proof freezes Membersh
Bob's remote board remains advisory until one eligible Refresh	Host restart or SSE message automatic
Independently required real HumanHelp features have positive browser controls	Disabled Reassign/Response is counte
S0/S1 formal subject and S2–S4 conditional transfer remain distinct	Finite fixture or report claims universa
Actual two-host fault and bounded load cases have attributable observations	Simulated two-host labels or fast one-
Required job inventory and test receipt provenance are verified	Unexecuted worker, flaky failure or ca
Legitimate narrower preview has separate original claim identity and excluded obligations	Failed strict release keeps same ID afte
Named conditional-liveness premises are met before progress is claimed	Dead network or missing user Refresh

These are future positive/adversarial conformance requirements. Their existence in a document, test name or coverage table does not establish that they were executed. Positive behavior must use the same real admission/enforcement path whose invalid neighbor is challenged. The acceptance report names original required subjects, actual evidence issuers or observers, selected hardware/software/environment, reproducible fault schedule, verified non-effects, unresolved premises and the first actual enforcing boundary for each relevant case. Production and formal claims are not interchangeable merely because both report success.

Working rule for every semantic or release change

For every framework or application change participating in the compositional chain, ask *what exact proposition this layer establishes, for what original subject, under which premises, through what genuinely checked or observed evidence, and which next layer is entitled to consume it*. If that question cannot be answered at the required interface, the corresponding edge remains an explicit allowed premise or an unsatisfied obligation for the selected claim. It is not silently inferred from matching names, successful status maps or a running server.

Keep a typed handoff at each layer

The ordinary change and release workflow accounts for:

Handoff	Required question before the next claim
Business rule and model to admitted operation	Are the independently selected product meaning, source owner, types, guards, re
Operation to global Choreo and role-local projection	Are placement, knowledge/provenance, authority, progression, outcomes and ter
Projection to browser/server executable construction	Are plans, canonical routes, transport envelopes, effects and resource/acquisition
Typed artifact to derivation and proof checker	Does genuine accepted proof or dedicated checking discharge the exact propositi
Proof/checker result to issuer and application assembly	Are the exact source, dependencies, required obligations, original denominator a
Assembly to installed handler, backend and browser	Does the **actual** listener, protected writer, selected XTDB/provider, built/served/l
Current execution to participant observation/adoption	Do authoritative result, command incorporation, source/progression, user author
Installed realization to activation and release	Are current candidate, installation, traffic/overlap, rollback, resource lifetime and
Prior release to changed release	Which semantic, issuer, trust, physical, product and environment dependencies c

A direct physical shortcut may be a valid independently admitted implementation, but it must satisfy its own actual semantics and release profile. Application authors should declare business rules and Gesso construction, not maintain parallel proof-assistant policies or manually manufacture a certification record. The executable semantic core remains the operational source; a proof kernel checks derivations, dedicated checkers establish their named judgments, and production guards enforce current runtime rights.

Track the exact evidence class and residual frontier

The framework reports which claims are **proved under a named formal system**, *checked by a genuine source-relative proof issuer*, *exact-finite checked within an explicit domain*, **assembly-verified**, **physically qualified**, **runtime-enforced**, **tested through observed paths**, **asserted only**, or **trusted under visible premises**. These are qualifications of specific propositions, not substitutable values on a single ascending ladder. A real installed browser test cannot issue an unbounded mathematical theorem, while a formally proved inference rule does not show that the running server actually uses the corresponding guard.

Evidence propagation retains every undischarged premise along the actually used justification. A trusted backend atomicity assumption must be qualified at its named backend and scope; a proof subject cannot replace a verified actor or current Membership; a report cannot act as an authorization token. An unavailable required checker is blocking under the originally selected profile. A permitted conditional smaller claim must keep its distinct identity and all its declared accepted assumptions.

When a layer changes, the owning application or framework constructor reconsiders only genuinely dependent claims, while still validating source completeness and current applicability. The generation and host binding of the old completed record remain historical. A stale physical receipt does not become fresh through a semantic digest, and sound unaffected theorems need not be reproved solely because a different file changed.

Final rule: admit only the actual compositional subject

A candidate is admitted because the original requirements, selected proof and operational obligations, current issuer and dependency checks actually justify that candidate. An installation is eligible because the **real** selected host, protected writers, browser assets and external premises satisfy their physical qualification. Activation is eligible only under current coherent traffic/continuation, resource and rollback conditions. A milestone is complete only for its exact required capabilities and evidence scope.

These decisions are not reversible historical decorations: their dependent runtime rights can cease when a current principal, backend or release premise becomes false. The release mechanism must fail closed at the first actual enforcing boundary, preserve truthful committed and unknown command state through its own failures, and support permitted positive user operations. No automatic rollback of authoritative domain facts, universal distributed liveness, or proof of arbitrary Clojure/browser/XTDB execution is invented.

The intended system therefore does **not** claim ”everything is formally verified.” It can identify which source-owned guarantees are proved, checked, conditionally justified, physically qualified, runtime-enforced, observed or still open; which current realizations they concern; and exactly when an implementation, premise, application or deployment change requires re-admission. If a required handoff cannot demonstrate its semantic or physical relationship, the requested stronger construction, release or completion claim stops there rather than allowing incompatible premises to drift into an application that merely appears to work.

Part XII — Implementation order and final invariants

This part defines a construction order for the system specified in Parts I–XI. It starts from the required semantics and observable application behavior, then connects their implementation, checking, qualification, and release evidence. It is a design from zero, not a report of completed work or a migration checklist for a particular repository snapshot.

Eighteen stages organize the work. Their numbers are navigation and a useful reading order, not a requirement to finish every lower-numbered stage before beginning a higher-numbered one. The dependency table identifies the branches. The four completion scopes remain those of Part XI; a stage number is not a milestone badge and a sophisticated proof fixture is not a usable application.

Stage order is a **contract dependency and delivery priority**, not a serial project-management lock. Distinguish a semantic prerequisite, a normally earlier implementation task, exploratory feedback that may be obtained sooner, optional early hardening, and a completion dependency for a named claim. Later work may expose a missing earlier contract; repair that contract at its actual owner and reconsider only the dependent evidence. A prototype can inform the design without borrowing the acceptance status of a completed stage.

Delivery discipline

Construct one useful path and strengthen the same objects

The first integrated path is a real HumanHelp Claim using public production models, the canonical application construction, the generated browser plan, the actual trusted invocation and transaction boundary, and an admitted visible result or recovery path. Build only the supported breadth needed for that path before generalizing, while keeping its missing obligations explicit.

The path must include RequestAssignment ownership, current policy dependencies, command identity and replay, and unknown-outcome recovery from the beginning. A single-document demonstration can qualify a narrower mechanism. It cannot be renamed as the completed Claim path while those relationships are absent.

The integrated witness must include an eligible actor authorized by the current trusted model, an actual Request and RequestAssignment with the guarded one-primary relationship, a stable logical command, the installed route and trusted writer that submit the actual effect, and a scoped authoritative observation for the outcome or its unresolved recovery. A declared operation, a generated route, a browser fixture, an optimistic card, and a committed Request are not interchangeable evidence that this complete path worked. Where real backend or application behavior has not yet been observed, the record names the still-open physical or product claim.

Pair that eligible success with nearby failures: a competing Claim, an expired Membership, a missing or incompatible effect capability, a request that may have committed before its response was lost, and a callback from an obsolete browser execution. Reject each defect at its first legitimately informed boundary, or retain uncertainty and the explicit recovery obligation when the boundary cannot decide. A later visible card alone cannot reconstruct which earlier logical command incorporated.

The Claim path is the first integration slice, not the whole HumanHelp product. The independent capability inventory in Part VIII remains fixed while the rest of the application is constructed. Removing a difficult operation from the exposed registry cannot make its requirement disappear. A still-unselected HumanHelp Response interaction likewise remains an explicit product-owned requirement, not a plausible behavior inferred from the working Claim slice. The selected remote-board policy remains advisory until an authorized scope-limited, one-use Refresh or an explicitly narrower local exception; Live delivery and automatic DOM swap do not silently grant adoption.

The ordinary construction must remain useful for arbitrary Clojure and ClojureScript behind explicit contracts. Closed proof languages establish supported properties; opaque implementations retain named premises and their runtime enforcement. There is no second unmanaged application architecture for code that the prover cannot analyze. A supported opaque Clojure function may carry named, checked-at-use runtime obligations; its opacity must not be treated as an implicit proof, automatic exclusion from the source inventory, or permission to bypass the application's admitted execution path.

Establish the real serving boundary before calling a slice runnable

A declared first path requires a compatible, actually installed host substrate. The selected supported Biff lifecycle and FX contracts, the application assembly, its stable startup/preflight gate, and the effective Ring/HTMX routes must be qualified for the path actually served. In particular, a listener must not expose a supposedly admitted operation before the required preflight and startup checks have passed. Startup failure fences newly acquired resources rather than leaving an unchecked handler serving.

This does not demand the final universal closure checker or all later formal theorems before any useful application work begins. It demands a genuine enforcing seam for the selected scope and an honest account of what it does not yet inspect. A green assembly map, successful compilation, manually invoked handler, or test fixture does not by itself show that the deployed listener, active writer, loaded browser asset, and Biff execution flow are the ones whose contracts were checked.

The exact supported Biff and backend profiles are selected and qualified as implementation subjects; they are not extra business-rule owners. Changes to an FX invocation contract, initializer, protected writer, browser bundle, served route or provider configuration trigger reconsideration at the affected physical boundary even if the abstract rule is unchanged.

Start cross-cutting obligations at their first boundary

The following work accompanies construction throughout the roadmap:

Concern	Begins when	Required discipline
Source ownership and identities	The first semantic object is declared	Derived objects retain their actual sou
Obligation and premise accounting	A claim is made	Preserve the original obligation and d
Positive and negative tests	A boundary can admit or reject	Exercise a useful valid case and the re
Physical correspondence	A semantic action first reaches an effect provider	Bind the issued plan, actual invocatio
Diagnostics	A checker or runtime boundary can fail	Identify the failed obligation, scope, s
Artifact and release integrity	A candidate is built or served	Verify the actual plan, manifest, bytes

Concern	Begins when	Required discipline
Resource and recovery limits	Work can wait, retry, queue, or outlive its owner	Bound retained work and preserve un
Backend qualification	A claim uses provider semantics	Name the exact assumption and appli
Original capability inventory	A product or application scope is chosen	Preserve independently required actio
Actor, disclosure, and current authority	A request can observe or change protected facts	Bind the authenticated principal, mod
Actual application admission	The first handler, worker, or browser artifact is installed	Check the real covered path and retai
Publication and command continuity	A commit, notification, retry, or host loss is possible	Separate durable incorporation, notice
Selective presentation adoption	A remote update can reach a client	Distinguish advisory interest, authori

Later stages expand these mechanisms. Their existence does not permit earlier work to ship with placeholder checks, silent fallbacks, or unclassified proof claims. Static preflight checks the declared handling paths for dynamic failures; it does not predict that the network, Membership, or database will remain valid. A dynamic permission change must still be enforced by the authoritative operation or qualified read at the time of use. A failed after-commit notification must not retroactively turn a committed transaction into rejection; the selected publication profile must either retain its durable intent or name the weaker recovery and availability assumptions.

For example, Alice's eligible Claim may incorporate at the trusted transaction boundary while the HTTP result disappears. Her original command remains unresolved to the client until qualified command-specific evidence permits settlement; an arbitrary retry under a new command identity would be a new intent. Bob's Live notification is merely an advisory about a remote change, not automatic authority to replace his adopted board. These obligations begin at the first Claim and first subscription, not at a later roadmap stage devoted to recovery or multi-node deployment.

A stage result is a scoped evidence record

A stage record names its supported subset, source and artifact identities, original obligations, selected profile, accepted premises, actual enforcing boundaries, acceptance results, and unresolved requirements. It distinguishes implemented machinery, exercised behavior, and a completed claimed scope.

A supplied map or a successful-looking report is not an admitted artifact. A checker-issued local object is not durable authority. A test observation is not an unbounded theorem. The evidence classes in Part I and result statuses in Part XI apply without reinterpretation.

The record separates the original required capability set from the supported subset, the chosen conditional or stricter profile from unmet mandatory gates, and source-derived evidence from actual installed enforcement. It names successful eligible witnesses as well as adversarial exclusions. A partial stage can be useful and truthfully complete for a deliberately narrower claim, but it does not make a missing product action, unknown physical writer, unresolved premise, or unexecuted test disappear from the stronger requested claim. Four Part XI release decisions and four independent completion milestones remain different result subjects.

The next useful task is the earliest unmet obligation on the selected delivery path whose prerequisites are available. A physical failure is repaired at the owning boundary, then dependent evidence is reconsidered. Adding another report layer without closing a required boundary is not progress toward that path. Early exploration of a future proof, different provider, or multi-node router is appropriate when it exposes a real missing contract. It must feed that finding back to the original semantic or physical owner, not produce a second architecture whose success is then counted as the missing original application guarantee.

Construction stages

Stage 1 – Semantic ownership, identities, and claim contracts

Establish the vocabulary and construction interfaces shared by runtime code, checkers, application assembly, and evidence reporting. This is the foundation of the permanent executable semantic core, not a promise that all physical behavior has already been formalized.

The source-owned semantic subject includes the actual actor, target, relationship, operation and policy dependencies, not just a friendly function name. A real HumanHelp Claim refers to public Request and RequestAssignment meaning, the current principal and Membership/Organization relation, one-primary responsibility, protected observation and atomic effect. Choreo describes coordination and knowledge; views select affordances; checkers assess named judgments. None of those consumers may create a second, subtly different Claim rule or infer current model authority from its metadata.

Stage 1's semantic-first position does not waive the earlier R4 physical substrate obligation. The actual Biff lifecycle/preflight, model FX invocation, trusted writer and installed host path must be established at the first **claimed runnable** scope, as stated in the delivery discipline; their complete coverage is elaborated in the later host and operation stages. The permanent executable semantic core is neither the small proof kernel nor an invented second runtime for unsupported Clojure code.

Deliver:

- single ownership of domain rules, public operations, choreography, projection, runtime transitions, and application declarations;
- distinct semantic-subject, implementation, artifact, assembly-generation, command, execution, transport, authority, and resource-instance identities;
- versioned claim and judgment records with subject, proposition, scope, assumptions, required dependencies, and evidence mechanism;
- an immutable original obligation ledger with separately derived dispositions;
- contracts for opaque host-language functions and their proof-relevant input, output, authority, state footprint, effects, and failure behavior;
- the independent product-capability inventory and semantic-policy audit shape;
- stable structured diagnostics and a minimal source-derived inspection surface.
- an explicit source-to-consumer mapping for business rule, nominal value, reader/relationship, actor authority, guarded effect and observation-basis dependencies, including which are checked, opaque, asserted or still unknown;
- a source-independent product requirement inventory whose denominator cannot shrink to currently declared operations or currently generated routes;
- the distinction between semantic subject, source syntax, normalized interpretation, checker result, issuer handle, physical implementation, generated asset, deployed host and current execution authority.

The independent HumanHelp requirements retain Create, Claim, Unclaim, other lifecycle operations, real Reassign, and the still-unselected product-owned Response interaction. A proof of one Claim conjunct or a closed but reduced application registry may satisfy a narrower named profile; it may not be reported as the complete product or as coverage of obligations never discovered by the generator.

Acceptance requires that same-shaped objects from different subjects cannot be silently substituted, a premise remains attached to its dependent claim, and a missing product capability remains visible even when the smaller declared graph is internally closed. An unknown inventory is not an empty complete inventory.

Canonical interfaces must be small enough to compose, but must include every meaning-changing dependency used by the consumer. A declaration's name or source location is provenance, not a proof of its semantics or implementation.

For instance, a structurally well-formed description or digest can identify a semantic **interface** without proving its meaning, its source-to-implementation correspondence or a safe effect at the trusted physical boundary. A copied certificate-shaped map, a same-shaped Request from another Organization, an old issuer status and an unbound client actor label must fail to substitute for a qualified current subject and evidence. The original source and its true dependency footprint determine required checking; a missing inventory is **unknown**, never an empty complete set.

A correctness-relevant policy may also be well typed and nevertheless be the **wrong intended policy**. The public model's auditable semantic surface retains explanation, intended actor/target binding and independent product approval. Human or heuristic review may detect an error and propose repair, but cannot manufacture a proof or authority. Ordinary Clojure and ClojureScript functions remain available: unsupported or hidden reads, effects, time dependence, partiality and provider behavior become named opaque obligations or independently enforced exclusions, not silently omitted duties.

Stage 2 – Typed semantic rules and executable judgments

Implement the closed rule language and the versioned judgment contracts of #req-certified-rule-language and Part IX. The rule interpreter and checker operate on the same admitted meaning used by operations and their proofs.

Stage 2 makes four claims distinct: closed **syntax** with an exact reader footprint, intrinsic **typing** under named reader types, **evaluation** over one admitted immutable binding, and **use-site admissibility** with justified knowledge and current authority. Passing one does not discharge the next. The baseline certifiable expression language consists of bounded literals, declared reads and presence tests, strict Boolean negation, ordered short-circuit conjunction/disjunction/implication, typed equality and ordering, and finite-set membership. Its exact arities, input decoding, failure behavior and resource limits are part of the versioned meaning. It does not include arbitrary function calls or EDN reader evaluation.

The full R4 target still includes typed relationship and field access, nominal variant discrimination, exhaustive closed match/case, deterministic derivations and independently specified extension primitives. These are explicit future extensions with their own syntax, type, evaluation, source mapping, effect and proof/premise contracts; they are not secretly certified by the small baseline grammar. Supported application Clojure may be elaborated to the closed term under a checked source-translation contract, or represented by a named opaque dependency. Merely copying an equivalent-looking term is not evidence that it realizes the original public rule.

Deliver:

- closed AST forms, exact reader footprints, operator arity, and semantic types;
- the supported scalar, nominal-enum, optional, and nullable value contracts;
- strict Boolean interpretation and left-to-right short-circuit semantics;
- presence and null refinements scoped to the paths and subjects that justify them;
- conservative normalization that preserves evaluation and access order;
- explicit value codecs and target-runtime limits for portable evaluation;
- typed interfaces for opaque policy or relational extensions, with unproved properties retained as premises;
- executable typing, evaluation, and rule-use judgment definitions with source diagnostics and stable interpretation versions.
- exact separation of absent, present nil, present false and present zero: presence and non-nil are different justified refinements;
- reader/type identity bound to the source, observation subject and evaluation path, with incoming-path joins retaining only shared facts;
- source-to-AST elaboration and conservative interpretation-preserving normalization, with no silent short-circuit reordering or omitted source-meaning dependencies;
- distinct interpretation and proof obligations for JVM/CLJS numerical, codec, overflow, range and external-reader behavior, where selected.

For example, the following rule about one supplied optional, nullable integer binding requires each guard to dominate its dependent read:

```
{:readers #{:case/score}
 :expression
 [:and
  [:present [:read :case/score]]
  [:not= [:read :case/score] [:literal nil]]
  [:≥ [:read :case/score] [:literal 10]]]}

{:case/score {:type :integer :optional? true :nullable? true}}
```

This is illustrative rule and reader-type data, not a complete policy, a production read or a certificate. Moving the final comparison before the presence/non-nil guards is a type error, not ordinary predicate false. Replacing a conjunction with a disjunction may express a **different** well-typed business policy. The interpreter, type checker and any future proof/translation must agree on the actual left-to-right evaluation paths.

Acceptance pairs well-typed successful evaluation with wrong-type, missing-reader, absent/null, invalid enum, unsupported-operator, and malformed-shape cases. A coherent but product-wrong policy remains representable. Typing does not prove its truth, and evaluation to true does not establish current server authorization.

A rule that is intrinsically valid can still be unusable at a choreography point because the role lacks its inputs, provenance, refinement, or authority. Stage 7 composes those distributed judgments; Stage 2 does not invent them from schema acceptance. Arbitrary host functions remain available through their contracts.

Acceptance also distinguishes positive eligibility from reject-everything success: valid typed expressions must evaluate over supported nonvacuous bindings while wrong enum, absent/nil, undeclared read, unsupported operator, malformed AST, overflow/codec mismatch and unsafe refinement-order cases receive exact diagnostics. A timeout, resource limit or unsupported

construct is neither a Boolean false nor a successful theorem. A rule may be typed but violate the product's independently selected policy; a true evaluated rule may rely on stale, fake or unauthorized bindings. These are separate failures.

Normalization may disregard specified irrelevant map/set ordering or equivalent mathematical integer notation, but must preserve nominal domains, reader IDs, optionality, nullability, operator and operand order, failures, interpretation versions and binding identity. A stable abstract rule fingerprint after a provider or installed-browser change cannot reuse stale physical enforcement evidence. The eventual proof kernel, source-elaboration checker, authority gate and physical correspondence tests retain independent responsibilities and premises.

Stage 3 – Small proof kernel and guarded proof mechanisms

Build a small, inspectable **proof-checking** foundation alongside the permanent executable semantic core. These are different components: the semantic core supplies runtime transition meaning; a propositional kernel checks derivations of its own closed logic; dedicated transition, initial-state and induction checkers establish their specified conditional judgments. Proof-producing elaboration proposes derivations but cannot create an inference rule or issue authority by calling a trusted-looking function. The proof stage is a prerequisite for claims that need its judgments, not a universal requirement that every useful application operation await independent mathematical formalization.

Close the initial propositional checking boundary

Use the original, versioned proposition source with an exact subject, nominal variable environment, declared premises, step graph and final goal. The initial calculus has only seven inference-rule families:

- premise use;
- truth introduction;
- implication reflexivity;
- conjunction introduction;
- conjunction elimination;
- implication elimination;
- disjunction introduction.

The checker independently parses and typechecks every source proposition, validates rule-specific arities and exact conclusions, and requires each referenced step to precede its use. A derivation claiming a stronger goal, using the wrong subject/nominal atom, inventing a step, hiding a required premise or relying on an unsupported rule fails. Every required original obligation remains visible independently of the supplied derivation. The supported source format may additionally require every declared premise/step to be used; that completeness discipline does not itself prove that an original application rule or physical writer was inventoried.

There is no initial axiom asserting an arbitrary atomic predicate, unrestricted excluded middle, arithmetic/SMT solving, hypothetical premise discharge, arbitrary host Clojure evaluation or transactional authority. A conditional derivation establishes a conclusion **assuming** its admitted source premises; it does not establish that those premises hold in the current Request, browser, database, or installed application. A checked derivation and its report are not a signed authorization or issuer handle.

Derive guarded consequences without enlarging the logical kernel

Represent a selected typed guarded transition with pre-state and input, current guards, simultaneous post-state assignments and an explicit frame for untouched fields, including presence as well as value where relevant. A dedicated proof mechanism may derive exactly supported guard, assignment, frame, conjunction and equality-substitution consequences. It checks original transition source and matching-relation assumptions. It cannot generalize a one-document assignment to a multi-record RequestAssignment invariant, silently infer hidden external writes, or turn a postcondition into evidence that a particular XTDB effect ran.

For conditional induction, freeze one **original-source** operation catalog, its initial condition and every required invariant. Each invariant needs a justified initial condition and preservation argument for **every** declared operation; a missing operation, malformed guard, changed frame, omitted invariant, unsupported consequence or falsely discharged premise keeps the stronger judgment open. Every claimed operation and initializer also faces independent source/physical completeness challenges. A finite set of tested initial constructors establishes coverage only of its declared finite catalog, not that every physical initializer satisfies the initial predicate. The selected model must retain eligible initial states and meaningful enabled positive operations, rather than pass an unreachable or reject-everything machine.

Do not confuse an induction obligation with an issued universal proof. Successful checker-local reasoning may remain conditional on original-source coverage, initial-state existence, operation-catalog completeness, abstract transition semantics, checker soundness and source-to-installed-effect correspondence. Scoped checking of the complete declared set does not establish the completeness of all potentially bypassing physical writers.

Keep proof, issuance and formal transfer independently accountable

Elaboration and search can construct candidate proofs or finite witnesses, but must submit their **source** to the relevant independent checker. Malformed derivations, stale subjects, step/reference mistakes, unsupported substitutions, skipped branches, result-map forgery and a mutant elaborator must not obtain a stronger accepted claim. Exhaustion, incompleteness, a timeout and an unproved premise are not success statuses. Nearby eligible positive derivations and useful transitions must still pass; a mechanism that rejects everything cannot earn completion for a functional Claim path.

Certificate admission separately binds the checked source, dependency closure, original obligation denominator, issuer scope, validity conditions and unclosed assumptions. A copyable report, old certificate description or fabricated fingerprint cannot mint a current capability. Logical facts in a derivation may be reused under logical inference rules; one-use physical effect rights and resource ownership are not duplicated merely by duplicating a proposition.

Independent mathematical soundness of the kernel, the production checker acceptance implication, typed-atom semantics and actual physical application correspondence remain distinct later formal and release claims. In particular, a sound theorem about a seven-rule abstract calculus does not by itself prove the concrete JVM checker, issuer, Biff/XTDB writer, generated browser plan or deployment. Keep every unproved bridge as a named obligation and never promote finite model checks, successful unit tests or source inspection into such a theorem.

Acceptance of this stage is the precise scoped checking of positive and adverse **source-bound** proofs, guarded consequences and induction candidates with their remaining premises honestly recorded. Complete cross-namespace certification, independently established kernel soundness and production effect qualification remain separate milestones.

Stage 4 – Qualified host, lifecycle, and invocation context

Select and qualify the **actual** Biff, Ring/Aleph, runtime, XTDB, compiler and browser/HTMX versions and interfaces used by the application. “Final Biff 2” is not an API, compatibility contract or reason to wait for an unspecified release. A selected host profile names concrete dependency identity and configuration, effective request/FX value flow, lifecycle semantics, handler adoption, backend behavior and the implementation assumptions on which a claim depends. These are implementation subjects, not new owners of domain rules.

Install the real application boundary before exposing traffic

Deliver:

- explicit current invocation input and prior-output value flow for generated and handwritten Biff/Graph/FX construction; no accidental assumption that previous output is implicitly merged into context;
- stable native module identities, declared resource/dependency owners, correct phase ordering, acquisition, initialization, start and cleanup;
- private candidate construction, required preflight/assembly checks, current handler installation and a qualified admission boundary before the selected application behavior is externally reachable;
- an exact relation between the checked candidate, concrete module, installed Ring handler, server's actual adopted handler, active routes and served/loaded browser artifacts, with unqualified bypasses reported;
- actual protected principal, model, request and transaction context assembled at the trusted boundary, not supplied by client metadata;
- independent concurrent request scopes and explicit supported transfer of a qualified context to deferred responses, streams, workers and subscriptions, with generation/owner-bound retirement;
- selected backend/HTMX/compiler qualification, real valid/invalid conformance tests, and honest residual assumptions for other host paths.

A host may have multiple physical lifecycle phases. In a selected Biff integration where all module initializers run **before** the ordered start phase, putting preflight first in start order does not prevent an earlier initializer from opening a socket, publishing a worker or performing a protected write. Every such initializer must be separately classified, safely private/fenced, or admitted before its relevant effect. The final guarded application must not be exposed through an unchecked listener under startup failure, missing provider or later handler replacement. Safe resource cleanup follows revocation of active effect rights; failed or partial cleanup does not restore them.

Preserve concurrency, asynchronous ownership and real observability

Application admission and a physically installed handler do not automatically qualify every dynamic render, alternative Ring response, migration, external writer, worker or browser entrypoint. An HTML serialization gate covers the canonical responses actually sent through it; a pre-serialized or substituted handler remains an open bypass unless independently instrumented or excluded by a real enforcing boundary. The actual server must be shown to use the guarded handler and asset set before physical correspondence is credited; a staging report or matching handler key is not such proof.

Each request, command, execution, stream, delayed callback and browser resource has its own relevant scope and retirement rules. A single process-wide, one-use writer lease, thread-local dynamic binding or global request context is not a sufficient model for concurrent requests and retained asynchronous effects. A context transferred to a worker needs a declared owner, authority basis, permitted effects, lifetime, cancellation and post-retirement behavior. Current protected reads and writes still require trusted checks at their use points; a formerly admitted request context is not everlasting Membership.

Acceptance challenges startup from a genuine valid candidate as well as missing/misordered modules, an initializer effect before start, host handler substitution, failed preflight after resource acquisition, duplicate module identity, concurrent request cross-talk, stale deferred writes, unsupported context transfer, and asset/route mismatch. The check observes the first real physical effect/serving boundary, not only a helper report claiming the boundary would have blocked it. Eligible normal requests and authorized deferred completions must still work under the selected profile, with observed cleanup and bounded resource behavior.

This stage supplies scoped lifecycle, invocation and enforcement interfaces. It does not assert Stage 10's complete whole-application closure, Stage 11's production release admission, or independent formal/physical correspondence. A health or private probe may use a separately declared and actually enforced narrower scope; it cannot quietly expose ordinary unchecked application routes.

Stage 5 – Public authoritative operations and command resolution

Construct public model operations with explicit preparation, guarded incorporation, and after-commit work. Choreography and views consume their public contracts rather than private model internals. HumanHelp's public Request, Membership, Organization, and RequestAssignment contracts own the relevant business truth; application assembly and the executor cannot define a second Claim policy.

The **source-owned transition** is the original subject. For Claim it includes authenticated actor/target binding, current helper Membership and protected Organization association, eligible Request lifecycle, absence or uniqueness of the primary RequestAssignment, every guarded write and its frame, and the disclosure contract for the returned or later recovered result. A server pre-read may prepare a proposal, but is not itself a current commit-time authorization or a certificate that the write protected all dependencies.

Bind one logical command without confusing it with an attempt

The public operation is declared before any particular command exists. A logical command receives its identity and immutable binding before the first possible transmission; a request, retry, execution, host, transaction, transport attempt and browser resource each have other identities. The trusted ingress must establish the authenticated principal, selected operation and target, canonical argument semantics and allowed authority scope. A client-supplied command ID is a correlation value, never proof of permission, incorporation, current Membership or result disclosure.

The complete binding is retained with the command's authorized historical disposition. A compatible retry or resolution consults that history **before** testing new-incorporation business guards: a successfully incorporated Claim can legitimately leave its Request no longer open. It must be possible to report that original success without executing Claim again or falsely rejecting the historical command. Reusing the same ID for changed arguments, principal, operation or scope is a typed incompatibility, not a new intent.

A newly incorporable command must atomically protect its current business conditions, complete multi-record effects and required durable command outcome. Where the selected reliability profile requires durable notice intent, its record belongs in the same suitable atomic authoritative boundary; asynchronous SSE publication is a subsequent delivery step. Separate after-commit attempts must not make an already incorporated domain transition appear rejected. A command record is not a permission to disclose private history to an unauthorized resolver.

Deliver:

- typed inputs, current authority requirements, guarded dependencies, write and frame contracts, result/disclosure contracts, and durable-work obligations;
- stable `:gesso/command-id` binding before first possible send, covering operation, interface, canonical arguments, principal, and authority scope;
- compatible existing-command lookup before applying a new-command business guard;
- atomic command exclusion, authoritative mutation, result recording, and required publication intent;
- model outcomes `:committed`, `:rejected`, `:already-incorporated`, and `:commit-unknown`, with explicit presentation mappings;
- protected command reconciliation, retry, retention, and expiry behavior;
- public composition of cross-model constraints without pretending that separate transactions form one atomic operation;
- a source-complete ledger of permission, relationship, existence, uniqueness, guarded change, command-result, disclosure and publication requirements, including independently required actions not yet implemented;
- explicit separation of attempt-local transport outcomes, authoritative command incorporation/exclusion, historical recovery, and actor-visible presentation or Live advisory progression;
- canonical outcomes with qualified origin and scope, preserving unknown incorporation after a possible send rather than inferring failure from connection loss, read timeout, absent command record or abort of one attempt;
- the independent server read and authorization required to disclose a retained command result, including retained history, expiry and compatible old-client recovery under a changed host generation.

A command is not safely retryable merely because a new physical attempt is possible. Retained identical command binding and appropriate authoritative resolve-or-incorporate exclusion are needed to prevent duplicate effects; on indeterminate evidence the contract must say who retries, rereads, reconciles or waits, and under which availability and retention premises. A failed individual attempt does not exclude a concurrent other attempt. Nor does current Request/assignment state by itself attribute success to one particular earlier command.

Admit the complete source effect, not a convenient document fragment

Claim's domain mutation, primary RequestAssignment creation or reassignment, canonical command-result creation and required publication intent are one declared authoritative effect where that strong profile is selected. A sequence of individually successful writes is not equivalent to an atomic multi-record transition. Other source-owned operations, including Unclaim, Reassign and terminal transitions, need their own eligible and adversarial execution paths; a successful Claim neither implies their correctness nor shrinks the independent HumanHelp capability denominator.

Outcomes `:committed`, `:rejected`, `:already-incorporated` and `:commit-unknown` remain model/command-level claims with qualified evidence. The browser's provisional/accepted/rejected/unknown presentation does not constitute a database transaction result. A known committed outcome may still have undelivered Live advice or an unadopted remote presentation; a successful SSE send does not grant remote collection Refresh. A postcommit publication failure never retrospectively invalidates the authoritative result. Preflight may reject structurally unsupported operation/effect shapes early, while changing current domain predicates must be rechecked at their trusted incorporation boundary.

Acceptance pairs a genuine successful Claim with typed guard rejection, compatible historical replay, incompatible command reuse, changed Membership or association, simultaneous competing Claim, stale server pre-read, lost response after possible submission,

and recovered command truth after a later lifecycle mutation. It also challenges a forged principal, missing RequestAssignment write, omitted command record or outbox requirement, unknown outcome incorrectly marked rejected, and a retry under a new identity. The first boundary with sufficient legitimate evidence must reject or retain the exact uncertainty; no reject-everything candidate can count as a completed usable operation.

The Claim scope guards Request state, helper Membership, Organization policy, association and primary-assignment conditions together with its writes. A pre-read or a self-asserted eligible flag cannot establish those current facts at incorporation. Replaying C42 after an Unclaim or other later event requires authorized **command-specific** historical evidence, not merely a currently claimed Request. Stage 6 separately qualifies the actual backend, guard semantics, transaction-internal prestate and all physical writing paths.

Stage 6 – Physical transaction mediation and provider qualification

Bind the authoritative plan to the actual physical submission boundary. The supported implementation may use a narrow mechanically checked lowering or an explicit opaque adapter whose remaining assumptions cannot disappear. For a selected strong claim, the admitted source, complete guarded plan, lowered immutable transaction vector, issued executor, exact one-shot physical submission, authenticated authority context and observation provenance must refer to the **same** execution subject. The selected provider/backend profile, transaction options, encoding and trusted host route are distinct identities.

Protect prestate, write footprint and the exact frame

The canonical abstract transition evaluates its guards against a declared pre-write state and applies its supported assignments atomically. A concrete XTDB implementation must show that its ASSERT predicates and transaction operations faithfully implement that transition under the selected API semantics. For a supported ASSERT-before-write lowering, checking ordering and transaction-vector shape is a useful prerequisite, not proof that the provider's **transaction-internal** prestate equals a separately pinned external read. A snapshot token, ledger row, transaction key or later matching card cannot authenticate an unseen internal guard evaluation.

Guard and write footprints are different. A guard may depend on Request status, helper Membership, Organization policy, a relationship's absence or uniqueness, and command history while the admitted writes touch a different set of documents. An opaque predicate or external writer can invalidate the proof unless the selected mechanism establishes its coverage or retains a named premise. Every mutation, effective transaction option, hook, and update/replace semantics must be accounted for.

Whole-document replacement can erase a concurrently added field even when all previously visible values compare equal. A sound frame contract must protect **presence and value**, including newly introduced and deleted fields, using an admitted complete field-count/footprint guard, monotonic history-sensitive revision, suitable partial update operation or other independently qualified mechanism. Full-value comparison alone does not eliminate ABA (A⟶B⟶A); if forbidden intervening history matters, the chosen identity/revision/transaction-history discipline must actually exclude it, including bypassing writers.

Absence, uniqueness, cardinality and one-primary RequestAssignment are predicate/relationship obligations, not merely a cached false value. Their physical realization must survive two concurrent attempts and every admitted writer. No pre-read test or synthetic concurrency schedule is a substitute for a real protected physical contention case.

Deliver:

- immutable admitted plans and complete physical transaction derivation/checking;
- one submission owner that cannot accept appended raw writes or a substituted backend client/callback through its normal interface;
- guard footprints separate from write footprints, including absent/unique relationship predicates and all effective transaction options or hooks;
- frame preservation for whole-document replacement, with complete comparison, a justified revision discipline, or a qualified alternative update mechanism;
- local attempt and provider-return observation correlated to the issued invocation;
- provider-qualified outcome, read-basis, progression, and atomicity interpretation;
- a real writer inventory and mediation contract covering routes, jobs, imports, maintenance, migrations, and external write credentials where the claim requires it;
- explicit backend-assumption qualification and invalidation from Part X;
- one source-to-physical and target-bound submission relation: the caller cannot append raw SQL, alter the protected table/target, change transaction options or swap the backend callback after the admitted plan is issued;
- transaction-internal prestate and actual guard/action correspondence, or a precise conditional premise and reproducible real-provider tests where the provider exposes no independently authenticated internal witness;
- complete protected physical frame and presence/write-footprint evidence, including caller-supplied actual values, absent relationships, monotonic revision policies and history-sensitive conflict assumptions;
- independently qualified process/request/execution/host context, preserving parallelism while limiting individual submit capabilities to the intended one-shot attempt and retiring stale physical effects;
- separate submitted-operation provenance, provider return, ledger/commit status, authoritative command record, read basis, scoped progression, actual actor readback and browser adoption, without treating agreement between two observations as causal proof;
- independent source/assembly and host-observed writer inventories plus mediation or explicitly admitted external writer assumptions for migrations, background tasks, maintenance clients, extra processes and directly credentialed XTDB connections.

Challenge real submission, concurrent writers and evidence limits

Acceptance uses actual backend concurrency and failure windows. It requires at least one eligible physical success and controlled negatives for stale Request status, revoked/changed Membership, changed association, competing primary insertion, duplicate command, additional field erasure, ABA, loss of command record, unauthorized writer, provider exception after possible submission, insufficient read progression, and changed adapter or configuration. The actual transaction's lowered operations and protected targets must agree with the original plan. Supported values distinguish absence, null, false, collections, and physical encodings under selected runtime codecs.

Physical evidence has several independently limited layers. A plan-shaped checker result proves construction of a **form**, not that it ran. A process-local one-shot callback or captured return proves only its correspondingly scoped observation; it is not a cryptographic, durable command record. A pinned poststate and ledger observation may be consistent with the returned key without proving which physical writer caused the observed domain state. A genuinely authenticated protected physical writer, complete internal-guard coverage and actual submission can establish stronger bounded claims; a universal refinement theorem still requires the stated semantics, source completeness and independent proof/correspondence premises.

The selected strong application profile must survey **all relevant writers** rather than only HTTP routes or Gesso-issued operations. Independent host/process/credential coverage is needed before treating a source registry as a physically closed world. Where the environment cannot enumerate or constrain an external writer, the stronger no-bypass claim remains unestablished even when the registered local callbacks all agree. Deployment changes invalidate the appropriate old physical evidence while possibly preserving an unrelated sound abstract theorem.

A pinned external pre-read is not a transaction-internal state witness; later matching post-state is not proof of causality; local return observation is not a universal refinement theorem. Complete field comparison without history does not exclude ABA. The required external behavior, physical prestate relationship, and closed-writer premise remain named and testable.

The stage reports exactly which physical forms, provider versions, writer paths and authority scopes are supported. One-document coverage cannot satisfy HumanHelp's multi-record Claim contract. A host-supplied writer enumerator cannot prove that undisclosed direct XTDB writers are impossible, and a registered one-shot callback need not be the one a deployed route uses. The release and cross-layer checker may claim only the smaller supported subject until actual installed submission, open writer effects, unknown-command recovery, retained publication and protected readback are covered by qualified evidence and nonvacuous testing.

Stage 7 – Portable choreography, distributed admissibility, and projection

Construct one source-owned global choreography and its derived role-local machines around the public operations, typed semantic values and authoritative observation contracts established earlier. The permanent executable semantic core defines global behavior and its distributed observation alphabet; verification decides a selected set of well-formedness, value availability, role knowledge and admissibility obligations; projection emits one canonical role-local `ExecutablePlan`. These are distinguishable judgments, not three independently authored copies of the application.

Establish one meaning across roles and knowledge sources

Deliver:

- distinct role, principal, actor, host, authority, execution and operation identities, including which identity binds a trusted server-side operation;
- a small typed global operation vocabulary, with local action, authoritative invocation, branch, participant communication, environment await and terminal result distinguished by their actual observable meanings;
- source-owned public operation inputs, required capabilities, closed outputs, failure and recovery results, and no browser-provided authorization premise;
- declared messages with required, optional and correlation fields, exact schema and default-closed semantics; open transport extras do not acquire semantic knowledge or bypass disclosure rules;
- role-local knowledge/provenance from entry input, declared communication, qualified authoritative invocation or observation, deterministic derivation, and explicitly identified asserted premises, never from a globally convenient value store or a matching property name;
- source, actor, program point, observation basis and progression-sensitive refinements, with required facts available on every relevant incoming path;
- behavior-sensitive branch discrimination, including who can distinguish continuations through an admitted value, message or authority observation;
- communication/authoritative causal barriers, terminal ownership, waiting, interruption, and declared recovery or durable handoff where later action depends on an effect or participant;
- checked global-to-local projection, canonical plan and compilation identity, portable schema/value codec and separately qualified JVM/Node/Chromium interpretation for the supported language;
- distinct structural verification, projection preservation, runtime trace, physical transport and authorization evidence classes.

A semantic global value may exist without a particular role knowing it. Receiving a closed message can justify declared payload knowledge, but does not authenticate a server, refresh Membership, grant a principal's role or establish a newer authoritative basis. A participant send is not a receive; a browser local `:await` is not participant communication; a reread of authority is not a newly committed domain mutation. Source and compiler IDs may be erased from an executable plan only when the checker/sidecar retains the exact source-to-plan relation; a proof-shaped sidecar cannot change runtime behavior or become an authority token.

Admit behavior-sensitive branches and genuine causal progress

A branch at one role may be private only if other roles' projected behavior does not depend on which branch was taken until they can distinguish it. Identical message keywords or overlapping receive shapes are not sufficient evidence that two continuations are equivalent. The verifier and projection must reject when an undistinguished foreign branch changes a receiver's required action, payload meaning, destination, protected effect or terminal obligation. Closed alternatives need justified disjointness, not a first-matching-case convention. A purported optional value is not definitely known on a path where it was never provided.

An authoritative operation is a global causal event even when some other role has no direct local action at that state. Its projected successors may not execute dependent work before the admitted causal communication, observation or alternative predecessor has occurred. A sender's local completion or a browser role reaching `:return` does not prove a required receiver acted or that the entire global choreography terminated. When the contract requires durable waiting, retry, timeout, participant acknowledgment or a future execution, its owner and actual recovery path must be specified. A later execution cannot inherit another retired browser execution's private resource rights or unresolved command outcome without an independent admission and qualified transfer.

Acceptance pairs a useful global Claim/authoritative result and justified recipient continuation with: private branch leakage, overlapping receive contracts, sender-unknown payload fields, missing declared FX inputs, premature authoritative-success effects, a forged authority assertion, an optional reader used as definite, confused participant/await events, and a disconnected recovery branch. Fail known contradictions at the earliest verifier, projection, assembly or guarded use point; maintain a named conditional premise for genuinely external facts.

For the same admitted source, initial semantic state and normalized event sequence, portable machines must produce the declared equivalent semantic observations under the selected bounded interpretation and representations. That requirement does not assert

identical network schedules, fair delivery, transport authenticity or an unbounded proof of implementation refinement. Stage 13 and later formal work can strengthen proof of projection and cross-layer correspondence without introducing a parallel production runtime. A useful positive role-local trace and actual issued plan must remain independently exercisable; a reject-everything compiler cannot satisfy application delivery.

Stage 8 – Browser semantics and physical effect enforcement

Realize the selected `ExecutablePlan` through a portable pure browser transition function and a narrow explicitly owned physical shell:

```
AdapterState x NormalizedEvent → AdapterState x AbstractEffects
```

The adapter owns semantic execution, normalized-event admission, target/fragment/resource generations, desired abstract effects, provisional/authoritative progression and retirement. The shell owns DOM objects, timers, transport handles, HTMX observations, continuity snapshots and other physical resources, returning exact completion and failure events. Neither layer may silently substitute an independently handwritten business workflow for the admitted plan. An untrusted browser must never execute the server-authoritative operation merely because the same operation name appears in its projection.

Bind events and effects to current resource rights

Deliver:

- distinct logical command, execution, transport attempt, effect instance, resource instance, target incarnation and application-generation identity;
- owner-relative event admission for both execution-private effects and shared fragment/subscription events, including their exact issued-work identity, currentness, payload schema, legal state and provenance class;
- retirement that revokes effect rights before fallible physical cleanup, without erasing separately retained command recovery or current shared resource ownership;
- permission revalidation at the **last protected physical mutation**, not just at an earlier event or scheduling point; delayed, reentrant and secondary effects must obey the same selected scope;
- guarded HTMX request, swap, OOB-target, install and settle boundaries using the actual documented lifecycle of the selected HTMX version, rather than invented generic hooks or a pre-swap report treated as completion;
- logical single-flight and coalesced queued refresh requirements with independent boundedness for metadata, pending callbacks, browser requests, retained snapshots, subscriptions and failed physical cleanup;
- typed provisional state, known rejection, historical incorporation, unknown outcome, supersession, optimistic rollback and canonical installation, with no silent elevation from provisional intent to model authority;
- explicit actual authoritative presence/absence, exact scope/basis progression, partial-installation repair, installed coverage and undisclosed/cannot-read obligations; response receipt is not participant adoption;
- continuity as compatible local UI presentation (search, sorting, selection, scroll, focus), never as current model truth or a retained write permission;
- stale plan/generation compatibility, source-bound asset and route identities, safe retire-or-reconstruct decisions and explicit recovery ownership.

An event arriving early, twice or after retirement may be consumed only under its registered source, generation, current resource owner and legal progression rule. A callback's earlier accepted Boolean is not a future physical permit. A resource reincarnated under the same DOM ID is not the same resource. An abort can fail after physical work, so cleanup failure cannot restore an obsolete permission. Logical slot bounds do not imply bounded physical XHRs, orphaned timers or resource metadata; those require independent limits and honest reclamation or failure assumptions.

Keep physical installation and semantic adoption distinct

The active HTMX/Ring integration must distinguish request admission, transport receipt, response parsing, before-swap decision, OOB secondary swaps, actual installed DOM, post-swap settlement and authoritative acquisition/adoption. A successful HTTP status, newer-looking token, permitted swap, or completed root replacement does not prove every required target was installed or that the result covers the authorized query/progression scope. Record the actual effect completion and correspondence before discharging its installation obligation. A failed partial swap keeps the uncovered debt, repairs or retires the affected resource, and does not expose enabled controls for an unverified inconsistent result.

Live invalidation is an advisory about a possibly relevant change, not an authoritative result and not a user grant. Under HumanHelp's selected remote policy, Bob's other-user Create or lifecycle change may mark the board stale or display an advisory, but does not automatically replace his adopted collection. A **separately authorized**, scoped, one-logical-read explicit Refresh can acquire a qualified result; physical installation and its coverage must then complete before the covered debt is discharged. Failed or partial OOB completion does not mint an additional Refresh or erase an unresolved one. Creator-local adoption and narrowly correlated command recovery are separately selected exceptions, not blanket remote adoption. Reconnect, lost notice, changed browser generation and a rising progression token cannot independently grant acquisition or DOM replacement. An authorized manual acquisition path must still be available when a notice never arrived or an interest was learned after its publication.

A Create projection does not need a fictitious prior entity observation when the entity did not exist. Its provisional intent is bound to an eligible operation and whatever enclosing facts are actually known. Conversely, a newer Request read may justify current presentation repair without proving the historical C42 command outcome. Retiring the optimistic execution or losing its DOM does not erase its independently owned pending resolver, permission checks, retention horizon or authorized recovery path.

Challenge the actual browser and trusted shell

Acceptance uses both portable event-order exploration and independent real Chromium/HTMX tests of the generated physical plan and effect handlers. Required positive histories include a usable Claim, authorized canonical settlement, controlled local continuity, genuine creator-local Create, a delayed but permitted callback, and one explicitly granted remote Refresh that installs its declared scope exactly once. Pair them with stale and duplicate callbacks, recycled resource IDs, failed abort, partial OOB install, a continuation across incompatible assets, a later stale rollback, lost command response, unauthorized automatic remote read, missing required target, incomparable authoritative basis and resource leak under continued retries.

The physical tests must exercise the actual served artifact, lifecycle hooks, DOM mutations, errors and deferred effects at their first enforcing boundaries. A model-only event name, injected trusted-looking settlement, skipped console failure, or a test harness that asserts only final text cannot prove real HTMX integration, browser refinement or current user authorization. Controlled expected failures must not hide unrelated page, request, console or application exceptions. Source-to-physical admission and all relevant writer/read disclosure boundaries retain their own evidence and assumptions.

Stage 8 thus establishes an explicit owned browser realization, not a universal liveness theorem or proof that every current HTMX version and browser schedule behaves alike. Its completion still depends on source/plan correspondence, qualified physical effect shells, actual installation evidence and nonvacuous product acceptance. Choreo proof, browser status, delivery success and model truth remain separate claims.

Stage 9 – Live publication, observation coverage, and selective adoption

Close the authoritative commit → required publication intent → dispatch → subscriber awareness → authorized acquisition → actual browser adoption path without confusing any two of those events. Gesso Live transports advisory invalidation and opaque progression requirements; it does not carry a second copy of domain truth, grant a protected read, or install browser content by publishing a notice. The selected authoritative provider owns the meaning of consistency, basis comparison and read coverage, rather than a universal integer revision or arrival-time ordering.

Derive publication obligations from authoritative source

The public model operation owns which committed changes require notice, the affected domain scopes, permitted recipient disclosure, necessary progression and the durability/recovery profile. Expansion rules may derive invalidation scopes from the original change but cannot invent new authoritative basis evidence or infer recipients' complete interests from a local ephemeral cache.

Where the chosen profile promises publication across writer crash or host replacement, incorporate **durable publication intent** together with the guarded domain effect and command record under a suitable common atomic boundary. Dispatch, retry, duplicate suppression, retention, dequeue, acknowledgment and delivery ownership are later responsibilities with explicit failure windows. An outbox record establishes that a notice was owed, not that every client received it or that a protected projection may now be revealed. A publisher failure after known commit must never be reported as a rejected domain operation; residual notice work remains owned. If a weaker profile is selected, state which crash, retention, availability or user-recovery guarantees are **not** claimed.

Deliver:

- a source-derived change/invalidation/interest/fragment relation, independently qualified for its exact assembly and publication profile;
- an atomic durable intent and qualified dispatcher when required, with exact owner, handoff, retry, idempotency, expiration, failure and recipient/disclosure scopes rather than an implicit best-effort callback;
- per-authority opaque progression, with orderless composition of distinct requirements unless a trusted provider supplies explicit entailment or comparison evidence;
- explicit separation of advisory frontier, authorized acquisition requirements, actually covered read, physically adopted result, current source knowledge and still-outstanding obligation debt;
- initial fragment acquisition or a justified existing observation, with a usable protected read route even when no advisory arrives;
- registration/subscription fencing, replay or qualified gap detection, and reconnect/failover ownership for interests learned after an invalidation or transferred between hosts and routers;
- admissible authorization and scope binding for advisory payload, follow-up acquisition and negative/exclusion observations, including the cases in which an entity is undisclosed rather than absent;
- per-fragment logical single-flight, debt coalescing, retry/backpressure, bounded metadata and separately bounded physical resource remnants, with explicit retention and availability assumptions;
- a convergence or honestly weaker eventual-user-recovery contract that does not depend on receiving every transient notice or inventing provider progression from a successful HTTP response.

Establish the initial-interest and lost-notice recovery seam

A durable outbox by itself cannot close a missed-subscription gap. At initial registration, a subscriber can discover its interest only **after** an invalidation was computed or dispatched; a replacement router can also lack the departed router's interest. The first useful managed projection therefore needs an admitted observation and a registration/acquisition fence, qualified replay, or a documented gap/reacquisition protocol under the source authority. Do not interpret silence from a router as authoritative freshness. A subscription token, published cursor or membership in a coarse routing filter is not automatically equivalent to a complete provider read basis or physical board adoption.

A claimed strong multi-host path must make durable notice recovery, interest registration, failover, redelivery and recipient knowledge separate obligations. Idempotent invalidations may be redelivered conservatively; their duplication cannot mint a second business command, alter current authorization or consume a fresh user grant. No particular broker, hierarchical router topology, Rust process, global durable actor or fixed consumer graph is mandatory in the timeless specification. The chosen topology must establish its own coverage, takeover, ordering/gap and liveness premises. A single-node profile obeys the same semantic contract with fewer physical owners.

An authoritative absence is a **positive qualified negative fact**: it binds the permitted principal, query, entity/membership scope, source basis, negative/disclosure semantics and physically adopted coverage. An absent DOM node, local filter, failed request, undisclosed entity, no matching interest or unfulfilled subscription does not establish domain absence.

Acquire only with an authorized scope and selected product permission

An advisory requirement may make an existing board stale, but it cannot authorize a subsequent database read or replace the DOM. Acquisition checks current trusted principal, source/query scope, permitted provider progression, current fragment/target owner and the **separately selected grant policy**. A qualified observation discharges only those requirements for which the provider proves coverage, scope, temporal compatibility and physical installation. An incomparable or insufficient basis remains outstanding; a successful HTTP 200 or echo of a requested token is not that proof. Newer observations may cover earlier advisory debt without assuming all opaque tokens are linearly ordered.

For HumanHelp's selected ordinary remote Create and lifecycle policy, the incoming notice remains **advisory only**. Bob's one-use explicit Refresh grant authorizes exactly its selected logical board acquisition under the currently admitted applied query and disclosure context. A new notice arriving during that cycle may be covered by its already qualified result if the provider establishes entailment; otherwise its debt survives. It never silently issues a successor acquisition, even if the first physical HTMX attempt completes or fails. A new grant requires the selected new user action or separately declared authorized recovery policy. Creator-local acceptance, a narrowly correlated own-command reread and trusted revocation are different scopes, not general permission to auto-adopt remote content. Local filters and visible cohort membership do not become server read authority.

Acceptance must witness an eligible first acquisition, durable commit-to-advisory path, authorized one-use Refresh and actual scope-complete DOM installation alongside: crash after commit before dispatch, duplicated/reordered notice, early registration race, stale backup filter, failed router, dropped notice with no recorded debt, insufficient or incomparable basis, omitted card, partial OOB installation, authorization change, reconnect and concurrent new notice during Refresh. The recorded result distinguishes incorporated state, owed notice, attempted delivery, recipient interest, permission to read, complete observation and adopted presentation. If availability or fairness is not established, safety may remain valid while convergence remains explicitly conditional.

Stage 10 – Whole-application construction and actual installed closure

The ordinary managed application is built from **one source-owned construction**, not independently hand-maintained route, model, browser, proof, recovery and diagnostic registries that merely use compatible names. The product's independently chosen required capabilities and current public policy are the denominator for closure. A closed discovered graph can be a useful narrower application but cannot define away a required HumanHelp operation, Response-flow decision, protected writer, publication obligation or uninspected correctness-relevant predicate.

Establish source, assembly, and physical-implementation joins

Deliver:

- the typed application graph linking original policy/rule subjects, public model operations, Choreo, role-local plan, generated artifact, route, transport, trusted handler, execution capabilities, transaction/command outcome, notice, fragment and acquisition;
- independently required product-capability and protected-effect inventories, cross-checked against the source-derived admitted operation catalog rather than reduced to what the generator found;
- source-derived requirements and selected evidence mechanisms, distinguishing proven, runtime-enforced, assembly-verified, conditional, trusted/opaque, unsupported, inconclusive, contradicted and uninspected claims at their exact scopes;
- closed required/supplied capability and FX invocation contracts, route method/path identity, collisions with anonymous HTMX, actual provider bindings, model/operation source versions and declared recovery paths for every required dynamic outcome;
- canonical browser manifest, checked build/compile provenance, exact artifact bytes, immutable publication set and serving/ loaded-asset correspondence under generation/release identity;
- actual Biff/Ring lifecycle dependencies, before-exposure preflight, qualified installed handler/executor binding, protected writer/read capabilities and independently examined alternative route, worker, migration and backend paths;
- real dynamic rendered Hiccup/HTML/HTMX affordance checks at the earliest supported pre-serialization boundary, including deferred responses and any selected supported streaming scopes;
- one read-only, source-derived policy/operation/authorization/ disclosure/lifecycle/dependency/assurance audit with exact completeness and opaque-premise annotations;
- stable typed errors, first knowable failure owner, incompatible versus missing resource distinctions, and nonexecuting repair guidance tied back to the same original source.

Fail at the earliest legitimate boundary, not at a convenient report

Static checks reject a known missing operation/view/route/plan/artifact/ model edge, unsupported effect/required recovery contract, inconsistent command binding, ambiguous route or absent provider before installing the affected managed behavior. Dynamic conditions need a **declared, structurally admissible** continuation with a real owner, required knowledge, permitted effects, trigger and terminal/uncertain outcome. Preflight does not assert that Membership, XTDB, network, permissions or clocks will remain valid. The trusted invocation/transaction boundary rechecks facts whose truth becomes knowable only there.

A real rendered response is checked before the supported HTML serialization step; that boundary does not automatically inspect arbitrary pre-serialized Ring HTML or a later substituted handler. The actual Biff module initialization/start order and the server's adoption of ‘:biff.ring/handler‘ (or explicitly selected equivalent) must be qualified independently. One inspected application map, green assembly result or forged preflight report is not evidence that the deployed listener served the checked handler, accepted the issued executor, bound its protected writer or delivered the recorded asset bytes. Under a strong production claim, an uncalled optional checker or an unenumerated physical writer is an open obligation that prevents stronger admission.

A rendering or transport failure **after** a qualified committed Claim does not undo that commit. It may block browser installation, leave unknown attribution at the client or create publication debt, each with its own recovery owner. When canonical rendering fails before response delivery, the supported fail-closed output gate must not release an unchecked affordance. Concurrent startup, reload, deferred output and version change preserve their actual generation and resource lifetimes; stale artifacts and obsolete handlers cannot borrow the successor's admission.

Audit policy honestly, including opaque or unexamined code

The semantic audit is a projection of the original admitted semantic and assembly objects, not a second authority or a complete interpretation of arbitrary Clojure source. Each correctness-relevant predicate affecting authorization, eligibility, disclosure, lifecycle, ownership, command acceptance or transition either has an admitted source-owned contract, a derived proof/inspection relation, or an explicitly open opaque implementation premise. The checker must report what it actually inspected and which other callbacks, handlers, writers and external services remain outside the scope. Zero escapes found in an incomplete scan does not mean zero escapes in the application.

Independent source-policy adequacy is separate from formal internal consistency. A perfectly checked but product-wrong policy cannot earn full product completion. Missing or unselected HumanHelp Response, Reassign or other required lifecycle functionality remains visible even if every discovered operation passes preflight. Genuine positive Claim, Create, lifecycle, search/Refresh and browser scenarios must remain possible under the specified profile; a reject-everything application is not complete.

Acceptance tests the source-to-physical installed path with a real eligible operation, then controlled removal or substitution of every named important edge: missing view/plan/route/provider, unbound clock/principal, unrealizable dynamic recovery, stale build, wrong browser target, anonymous HTMX route collision, replaced Biff handler, pre-serialized response bypass, direct XTDB writer, incorrect CSS/JS publication generation, bogus audit claim and unsupported opaque policy predicate. Errors occur at the first boundary able to know the defect and otherwise retain a typed **open** limitation. Independent observation of actual serving, current authority and protected writes is needed before declaring production-path enforcement. Ordinary genuinely non-Choreo HTMX remains legal under its own explicit contract and may not inherit the managed application's stronger claims.

The later release, product and compositional-certification milestones still require their own independent evidence. Stage 10 establishes the source-owned application construction, exact scope and admissible installed enforcement boundaries; it does not establish all possible arbitrary Clojure behavior, universal backend semantics, full HumanHelp acceptance or actual proof soundness merely by assembling a graph.

Stage 11 – Release integrity, reuse, and bounded compatibility

Implement the four subject-specific release decisions of #req-release-evidence and the directional compatibility contracts of #req-bounded-release-compatibility for the application's independently chosen intent and assurance profile. Admission, installation, activation and completion have different subjects and required observations; they are not four states of one trusted Boolean or four independent completion milestones. This stage integrates obligations already active at the first managed deployment; delaying Stage 11 work does not permit the first installed application to expose unchecked handlers, stale browser assets or unauthorized protected effects.

Bind the original release intent before generating results

The original application, product and deployment requirement denominators, selected evidence policy, source/operation versions, target hosts, protected writer universe and requested milestone are fixed before the associated checks run. A generator must not derive the complete requirement set exclusively from the operations, gates or tests that happen to succeed. A mandatory unselected product decision, unsupported physical writer or unexamined Clojure policy stays an explicit blocker or conditional premise for the **relevant** strong claim; it cannot disappear into a smaller closed report. A deliberately narrower operational release remains legitimate when its stated profile permits it. Its status must not impersonate a stronger requested profile.

Deliver:

- an immutable candidate identity binding original source, public rules, required capability inventory, exact operation/plan and route graph, artifact/manifest/issuer inputs, source dependencies, evidence policy and selected environment;
- four separately admitted and reported judgments: candidate construction/admission, actual host installation, current activation/exposure and named completion milestone;
- exact gate coverage and closed result dispositions, distinguishing proven or checked, passed runtime/physical test, conditional, trusted premise, unsupported, missing, inconclusive, failed, inapplicable and excluded **according to the selected Part XI contract**, without laundering a premise through an aggregate green status;
- dependency-cone invalidation and selective reuse of unaffected abstract theorems, while separately requalifying changed adapters, issuer dependencies, installed handlers, providers, direct writers, browser assets and current authorization;
- robust cache deserialization, original-source rechecking and fresh issuance wherever an in-process proof handle or qualification cannot safely be transferred as raw bytes;
- actual generated/staged/served and client-loaded artifact correspondence, including expected bootstrap, codec, transport, role and executable-plan identities;
- coherent compare-and-activate, bounded old/new overlap, failure fencing, writer/reader capability withdrawal, private installation cleanup and host-level observation of the serving generation;
- source-derived continuation and retirement contracts for old clients, requests, streams, deferred effects, workers, migrations, persistent data and surviving command-recovery ownership, with explicit compatibility and retention limits.

Check directional overlap, not adjacent version labels

Four release coordinates must not be misrepresented as four version identities: source-derived semantic identity, installed physical realization and client artifact/continuation identity are three separately qualified subjects; the **fourth coordinate is a directional compatibility relation** for an origin, target, operation and actual overlap condition. A/A, A/B, B/A and B/B are checked when reachable in the selected routing and deployment topology. Equality of labels, route names, nearby Git revisions or plan digests neither proves cross-direction compatibility nor establishes current read/write permission. One old browser may reach a new server while a new browser reaches an old server, and their decoders, payloads and state transitions may differ.

Before a protected read, authoritative command or DOM installation, the active gateway verifies the applicable origin/target relation and current principal/scope/ resource rights. Missing or unsupported compatibility must refuse the effect or use an independently admitted, non-mutating recovery path; it cannot silently downgrade to an anonymous HTMX route, ignore changed command fields, replace another client's adopted board or run an obsolete unsafe writer. Retain only the old assets, decoders, command records and other resources needed within their declared horizons; retention is not indefinite authorization.

A pending old command C42 may have incorporated before the old execution retired and its response was lost. The target release may supply a new **authorized historical resolver** for the same immutable command ID and binding without admitting the retired mutation implementation, reconstructing the old browser execution, or assuming current Request state attributes C42. If retention, authorization or evidence is insufficient, preserve unknown status and the selected recovery or explicit terminal-exclusion policy. Current-state acquisition, command-specific settlement and browser continuity have different owners.

Activate one safe generation and retain honest failure debt

Installation validates the exact private host resources, Biff/Ring handlers, trusted writers, provider configurations, worker processes and asset-serving paths. Activation then checks the current predecessor and exposed traffic before permitting the successor to serve. A failed candidate cannot borrow an old green report or leave new privileged callbacks alive by relying on best-effort cleanup; revoke their active rights before retiring physical resources. A predecessor may remain active only while its own physical, semantic and security premises still hold. Reactivation or rollback is a **new checked transition**: neither a code rollback nor a browser swap reverses a durable Request/assignment/command history.

A selected production run must test useful valid admissions, installations, activations and unchanged-evidence reuse, not only mutant rejection. Adverse cases include omitted mandatory gate, forged report/issuer, stale cache, changed source/premise, suppressed check, changed XTDB adapter, wrong served or loaded JS, post-check handler substitution, concurrent activation, stale

external writer, unsupported A-to-B/B-to-A continuation, partial cleanup and C42 lost response across retirement. The earliest enforcing boundary rejects, retains conditionality or preserves a recoverable unknown; pass-by-skipping is never success.

An accepted operational release does not automatically prove compositional certification, independent external formal soundness or multi-node production. Those four completion milestones and their separate evidence subjects remain as defined in Part XI, not as a linear sequence of required construction stages.

Stage 12 – Complete HumanHelp proving application

Complete the **independently selected** Part VIII product capability set through the real production public model, admitted Gesso application, actual installed Biff/XTDB/ HTMX/CLJS paths and observable browser behavior. The early Claim vertical slice is a useful architecture test; it is not a replacement for Create, all other lifecycle operations, selected Response interaction, search/details/continuity, recovery and multi-user adoption. Product acceptance is not a consequence of a structurally closed generated registry. HumanHelp must remain a proving **application**, not a special case baked into the framework's generic runtime or proof kernel.

Preserve the original product denominator and authority

The protected model and its public core APIs own Request, RequestAssignment, Organization, Membership, User and Location semantics. Actual sign-in produces separate authenticated principals and current read/ write/disclosure scopes; a browser session label or helper name cannot confer those permissions. Each eligible operation must cross a real rendered control, bound command, trusted server operation, multi-record guarded effect where needed, authoritative result and physical browser adoption/settlement path. One-primary assignment, current helper eligibility and Organization policy apply at incorporation.

The six independently required lifecycle capabilities are **Claim, Unclaim, Mark on the way, Complete, Cancel and Reassign**. Reassign is not ordinary self-Claim: an authorized manager must choose an actual distinct eligible target helper, and the guarded transaction must preserve primary-assignment history and current authority. Create, authentication, Refresh, search/ options, details/continuity, response handling and fault recovery have their own positive product gates. Conditional ineligibility is legitimate, but a permanently disabled control or unimplemented handler cannot satisfy a required reachable positive case.

The Response interaction remains an independently owned, **unselected** HumanHelp product contract at #decision-humanhelp-response-flow. The framework must not invent a default Response workflow just to turn a progress report green. A selected design will specify its actor, lifecycle relation, command/response semantics, rendering, disclosure, completion and positive/adverse tests. Until that choice is made and qualified, the full operational HumanHelp completion claim stays open, even if an independent narrow Claim assurance pilot succeeds.

Use the adopted board cohort and selected user policy

Creator-local Create success may show the creator's new authorized Request promptly under the declared scoped exception, even when it falls outside the currently applied board filter; its membership and projection effect must be explicitly defined rather than changing a global query invisibly. The same exception does not grant acquisition of unrelated remote content or erase pending advisory debt.

For any other actor's remote Create **or lifecycle change**, including changes to an already-visible Request, the ordinary selected HumanHelp policy remains **advisory until explicit Refresh**. The advisory may expose only the permitted pending indicator/status, not protected changed Request contents or automatic collection membership. A user-created grant permits one scoped board acquisition under the current principal, Location/ Organization and applied-query context. Its authorized observation must establish the covered cohort, relevant progression and physically complete adoption before discharging debt. A new notice during Refresh may be covered by that same justified result, but cannot create an ungranted successor request. User-selected reauthorization, rejected/unknown own-command affected-card recovery and trusted loss of disclosure rights are distinct explicitly scoped paths. Reconnect, plan change, delivery success or server push alone does not grant remote adoption.

Search, ordering and presentation options operate on the **already adopted, currently permitted cohort**. They may support newest/oldest, mine-first, unclaimed-first and terminal visibility without a covert read of newer remote membership. Disclosed title/details/location and permitted search labels form the search surface; undisclosed personal data cannot become a side-channel through search matching. The JVM/CLJS normalization, option semantics and current viewer projection must be compatible under their declared profiles. A locally filtered or hidden row is not an authoritative absence.

Details, active selection, focus, editing drafts and scroll continuity are compatible browser-local state, not durable domain facts or a license to restore obsolete provisional DOM. A stale Claim control can remain visible under remote-advisory policy, but a click must still face current Request and Membership guards; the resulting rejected or uncertain command has an honest affected-card recovery path without automatically importing unrelated collection rows.

Prove full product behavior at actual enforcing boundaries

Each of the **fourteen independent Part VIII completion gates** must have its own original requirement, meaningful eligible positive, applicable rejection/ concurrency/fault cases, actual observed result and declared source, build, host, backend, browser, actor/scope and evidence provenance. They cover production model authority, one admitted application source, installed host/writers, actual client assets, real Claim, all six lifecycle actions, rich-board behavior, multi-client selective adoption, explicit Refresh, fault/recovery, real runtime enforcement, semantic/assurance evidence, physical substrate qualification and independent real-browser acceptance. A green browser fixture, passed model check or deployed handler by itself cannot substitute for another gate.

Acceptance uses actual Chromium/Playwright sessions and rendered controls, not invented transport fields injected by a test; real current production models and backend truth, not parallel sample rules; persisted readback after reload, not only optimistic DOM; multiple genuinely distinct users and current Organization/Membership policy; and deterministic contention/failure fixtures with precisely attributed outcomes. Negative tests must fail for the intended reason and unexpected console/page/request errors must still fail the test. An HTTP 200, disabled button, uninstalled browser artifact, fabricated stored state or unimplemented Response cannot count as an eligible successful product capability.

Deliberate adverse scenarios include competing Claim and Reassign, revoked Membership between read and commit, lost C42 result then later lifecycle change, malformed route/manifest, Create during another client's pending remote debt, one-use Refresh with a later advisory, lost SSE/registration fence, incompatible old assets, OOB partial installation, unknown outcome, failed continuity, changed reader permission and unauthorized disclosure. A dropped advisory must have an independently permitted current-state acquisition path: reconnect alone is neither the only trigger nor a user grant. The result distinguishes actual domain truth, authorized observation, installed presentation and unresolved command attribution.

The genuine product can finish its operational milestone while independently stronger compositional, external-formal or multi-node milestones remain open if they were not required by its selected release intent. Conversely, no degree of formal Claim analysis can declare the unfinished product complete. Completion requires all original Part VIII requirements actually discharged, with remaining conditional or unqualified physical claims reported exactly and never hidden by a narrower generated inventory.

Stage 13 – Source-derived certification and the real Claim pilot

Compose the admitted public model semantics, typed rules, guarded operations, global Choreo, role projections, executable plans and whole-application construction into the **source-derived** proof and evidence architecture of Part IX. The checked hierarchy is an interface and dependency relation, not a demand that every namespace implement a nominal certificate class or that a printable graph, source digest or successful test report become a certificate. Its reference subject is the actual original application source and selected claim, not the conveniently checkable subset the proof generator happened to construct.

Compose declared guarantees through an independently required closure

The original source and requested assurance profile determine the required guarantees, source/semantic interpretations, declared operation and invariant catalog, dependencies, current issuer qualifications and independently required product capabilities. For each supported schema/domain/relationship/rule/operation/Choreo/projection/plan/application interface, record the exact exported proposition and subject, premises **used** to justify it, versioned interpretation, producer and consumer, source dependency, and any implementation/physical reliance.

Deliver:

- original-source acquisition and a coherent immutable construction generation, with source, normalized semantic, issued implementation, generated artifact, installed host and live resource identities kept separate under checked compatibility when they change;
- generated required-obligation and dependency manifests, independently checked against original declarations and selected profile rather than accepted merely because a caller-provided DAG is acyclic;
- a small proof kernel for its exact inference rules and separate guarded-transition, finite-checking and conditional-induction mechanisms for their **different** propositions, with proof-producing elaboration always submitting candidate work to the appropriate independent checker;
- precise typed substitution across exported guarantee interfaces, retaining subject, nominal types, actor/Request scope, observation basis, assumption origin and checked semantic equivalence or direction-sensitive compatibility;
- claim-specific used-premise propagation and genuine discharge by independently checked evidence of the **same** proposition in a compatible source, temporal, authority and implementation context;
- closed source-relative proof and dependency admission, in-process checked issuance where supported, exact current consumer rechecks, and safe inert durable packages which undergo new trusted validation/issuance on import;
- controlled reuse of still-valid abstract results with invalidation of every dependent semantic/implementation/physical obligation affected by changed source, provider, callback, writer, host or artifact, without requiring unrelated proofs to be discarded;
- a readable source-derived root proof/assurance ledger distinguishing checked facts, conditional theorems, admitted premises, separately observed behavior, physical installed qualifications, missing, unsupported and inconclusive required checks;
- source-complete positive and adversarial admission oracles, and three separately reported results for the actual HumanHelp Claim pilot defined at #req-humanhelp-certification-pilot.

The source-derived **proof** graph and independently verified **required** dependency manifest must agree; their equality cannot be established by letting the prover write both inputs. A missing Unclaim, Reassign or guarded writer invalidates a whole-catalog Claim invariant argument even if a smaller graph is internally closed. Such a smaller abstract proof remains honest only when its **reduced subject** is named and the stronger original application claim is left open.

Propagate conditional evidence without laundering assumptions

A derivation of proposition P conditional on Q may be accepted as a conditional result with exactly the premises used for P. An assertion with the same name as Q, another actor's verified Membership, a previous read basis, a copied `:passed` report or a structurally valid certificate does not discharge Q. Discharge requires a checked matching proposition with compatible source/subject, authority, temporal, model and provider scope; its own premises remain transitive obligations. Circular proofs, mismatched versions and unestablished current runtime facts do not become unconditional results by composing two green-looking nodes.

For conditional induction, the original catalog specifies a shared initial condition, all required invariants and every admitted operation. Each invariant has an initial case and a preservation obligation under the **combined** invariant hypothesis for every relevant operation. Coverage of actual initializers, external writers, real cross-model interference, physical transaction semantics and independent checker soundness is a further claim or named premise. A vacuous initial condition, disabled Claim or reject-all catalog may pass a narrow mathematical implication while failing a demanded **usable nonvacuous** application proof. A finite model check may close its exact finite state space without proving an unbounded Clojure/XTDB program.

A source-qualified certificate or issued callable is not an installed executable, signed portable permission, issuer-independent theorem, transaction authorization or real-browser result. A durable package is untrusted data until admitted under its current source/profile and issuer context. The proof/evidence sidecar remains runtime-nonsemantic. A failed optional post-write observer must not retroactively reject a potentially committed command; a selected mandatory gate must prevent the prohibited effect at the real first enforcing boundary.

Certify the actual HumanHelp Claim subject in three dimensions

The pilot uses production public `User`, `Membership`, `Organization`, `Location`, `Request` and `RequestAssignment` semantics **where they actually matter**. It does not invent a `:request/helper-id` field, collapse assignment history into a scalar status, manu-

facture a Claim-only model or write duplicate proof-specific business rules. At most one active primary assignment, current actor/Organization eligibility, changed request status, command binding and historical result, source-owned publication intent, supported transaction frame and physical writer coverage are distinct obligations. The invariant/catalog checks include Reassign, Unclaim and all other operations that can affect the protected aggregate, or retain exact gaps.

The independently evaluated pilot outcomes are:

1. **Semantic checking:** the original public model, rule/operation/Choreo/projection/plan contracts and selected source-complete abstract proof judgments are accepted with their exact premises and limits.
2. **Production binding:** the issued operation, actual installed Ring/Biff route, protected XTDB effect, authoritative command outcome, build/served/loaded browser artifact, current principal and qualified scope correspond to that source under independently recorded physical or open-premise evidence.
3. **Real product behavior:** a genuinely enabled browser Claim control sends the bound command, the real current multi-record model authoritatively incorporates or rejects it, and the permitted client settles/adopts from qualified evidence, surviving reload and paired concurrency/recovery scenarios within the selected environment.

A typed scalar Request update cannot substitute for the assignment/relationship/actor proof. A complete abstract Claim theorem cannot attest a deployed Biff writer, and a valid Chromium UI test cannot attest that the checker is sound. C42's compatible historical replay may succeed when a new Claim guard would fail after the Request became claimed; a later Reassign/current card does not prove C42's incorporation. A server response or provisional DOM badge alone satisfies none of the stronger physical/accounting claims.

The independent HumanHelp product inventory of Part VIII still requires all six lifecycle operations, actual manager-target Reassign, Create, scoped creator visibility, selected Response interaction, adopted-cohort controls, remote advisory/one-use Refresh and fourteen independent real product gates. A completed narrow Claim assurance pilot does not remove a missing product capability; an unfinished Response choice stays owned by the product and blocks its affected full operational completion claim.

Acceptance rejects omitted source dependencies, missing operation cases, an invalid premise substitution, forged issued handle, stale cached proof, changed abstract or physical interface, replaced executor, unsupported provider, unqualified external writer or substituted browser asset even if a root report still says successful. It also requires a real eligible Claim and appropriate legal variants to pass. A green report based on no enabled commands cannot satisfy this pilot.

Stage 14 – Systematic mutation and generic boundary coverage

Turn the paired eligible/adverse cases already required at each semantic, application and physical boundary into a reproducible, independently observable mutation corpus under #req-certification-mutation-corpus. The intent is to show **where** a forbidden source or effect is refused, with what specific disposition, and whether the relevant physical effect was prevented, rather than merely obtaining a high mutant-kill percentage. The corpus does not create a second policy database or prove correctness of arbitrary Clojure code by passing a fixed suite.

Make each mutant an exact predeclared claim

Before running a mutant, record its original valid baseline, profile, subject, source and generation, changed assumption or declaration, injection point, expected first legitimately enforcing boundary, exact result/error class, effect or admitted artifact that must not occur, observation mechanism, and an independent valid positive control.

The injection point distinguishes:

- original unadmitted model/rule/operation source or proof-elaborator candidate;
- checked-dependency or imported package input, including a coherent-but-stale source and semantically changed interface;
- application declaration, route, plan, protocol and rendered-affordance source;
- exact generated/served artifact, installed handler, writer, browser effect or resource-generation boundary;
- external fault window, concurrent transaction, transport drop, provider version, router interest gap, expired principal or postcommit delivery failure.

A case rejected at a certificate checksum boundary does not establish that an unreached model authorization or XTDB atomicity boundary was tested. Likewise, a source/type failure should not be reported as a browser robustness success. The verdict is tied to the **actual first responsible boundary** and the independently observed prohibited effect, not a generic downstream `:missing-plan` cascade or the mutant's own claimed `:effect-observed?` flag.

Cover the language, joins and physical fault cuts

Include at least these independently scoped families, extending them as new supported features acquire checking rules:

- schema/nominal type, enum, variant, optionality and absence-versus-nil;
- rule arity, reader declaration, short-circuit path refinement, unsupported primitive and evaluation or representation mismatch;
- public model authorization, business guard, relationship cardinality, RequestAssignment uniqueness and protected disclosure;
- Choreo role-local knowledge, provenance, branch discrimination, communication causality and unpermitted authority assumption;
- global-to-local projection, plan compilation, nonsemantic sidecars, browser event currentness and shared-resource ownership;
- original-source obligation catalog, initial-case existence, invariant preservation and missing operation or required product capability;
- proof-elaboration/derivation error, wrong subject, copied certificate, reused hash, forged issuer, stale dependency and false assumption discharge;
- operation/FX input, command binding, compatible replay, unknown outcome, transaction guard/frame/presence, ABA, competing writer and backend transaction-internal prestate;
- source-to-executor, admitted route, browser asset and actual Biff host/ physical writer correspondence;
- Live outbox/notice, late interest, subscription replay, authorized one-use Refresh, real HTMX swap/OOB completion, stale/suspended browser and delayed physical effects;
- policy completeness and opaque source escapes, missing recovery, skipped mandatory release gate, directional rolling compatibility and failed-resource retirement.

Use generated one-step mutations where their exact semantic effect is well defined, and hand-authored adversarial history for concurrent Claim/Reassign, lost C42 response, a successful commit with failed publication, changed Membership, partial OOB installation, stale writer and lost router registration. Mutation generation may use source annotations but must not independently rewrite the business-rule semantics or select the expected oracle after seeing the result.

Distinguish correct rejection, intended acceptance and incomplete evidence

The runner preserves at least five meaningfully different outcomes:

- **Prevented correctly:** the required boundary refused the mutant before the forbidden construction, write, disclosure, browser adoption or deployment, and an independent qualified observation found no prohibited effect within its actual scope/window.
- **Accepted by design:** a semantically well-formed intentional product-policy variation was admitted under the selected rules; it must not be counted as a missed invalid mutation merely because it changed previous behavior.
- **Conditionally admitted:** the exact selected profile permits specific open premises; those remain attached to the result and are not counted as independently proved.
- **Detected too late or incorrectly:** a forbidden physical effect occurred before diagnosis, the wrong boundary/reason failed, or an unrelated exception replaced the expected controlled error. This fails a prevention claim.
- **Unsupported or incomplete:** the required checker lacks support, the test did not run or finish, resource exhaustion occurred, or instrumentation cannot decide whether the effect happened. It cannot silently raise the supported-rejection success rate.

An explicitly expected `:unsupported` disposition can validate that an unsupported input is **refused** by its actual supported gate; it does not demonstrate correctness of an unimplemented inference or become a passed proof obligation. Counterexample reports, timeouts, missing events, model-only simulations and absence of log lines are not independent proof that a physical mutation was prevented. An optional post-effect observer may accurately detect harm, but it cannot be credited as a before-effect guard.

Keep per-category counts for all required rule families, originally required operation/outcome/branch cases, mutants planned and exercised, valid and accepted variants, rejected/conditionally admitted, unsupported, inconclusive and physical boundary observations. A visited state does not establish that each predicate branch, reader-path, backend writer or browser interruption was exercised. Positive success remains independently required for each applicable gate; a fail-everything checker cannot earn completion by rejecting every mutant.

Demonstrate framework-generic reuse, not new special-case products

Small non-HumanHelp example domains challenge accidental framework coupling: an approval flow tests policy change and disclosure; a reservation or checkout-like operation tests competing command identity and unknown incorporation; an ephemeral presence/typing receipt tests bounded admitted communication without pretending a durable Request-like business fact exists.

These are **contract-driven tests** through the same Gesso semantics, not three new production apps, a mandatory new pub/sub transport or a replacement architecture.

Actual browser cases use the selected real HTMX lifecycle and qualified emitted JavaScript; backend cases use the actual supported XTDB/FX submission; concurrent cases preserve distinct request/command/execution scopes. A fault injection must fail for its predeclared reason without suppressing unrelated console, page, request or server failures. The test's own instrumentation, eligible control and independent physical oracle have separate identity and trust assumptions.

Stage 14's operational mutation checks start as soon as the first relevant boundary exists. Later proof-specific campaigns can expand this corpus independently. Full external soundness checking is not a prerequisite to ordinary application use, and passing mutation coverage does not discharge source completeness, unbounded formal correctness or physical correspondence.

Stage 15 – Explainability, counterexamples, and source-level repair

Complete the read-only source-relative explanation contract established from Stage 1 onward. The explanation is **derived from** the admitted public semantic subjects, operation and policy inventory, dependency and obligation ledger, checker judgments, installed-realization evidence and actually recorded observations. It is not a second maintained business-rule registry, proof issuer, alternative policy interpreter or execution gate. The intended reader may be a developer, auditor, application owner or advisory assistant; none acquires authority merely by reading or editing the explanation.

Explain the first knowable failure and its actual claim

Deliver:

- a compact inspection view of the original model-owned operations, actor/target authorization, guarded lifecycle, relationship and disclosure policy, role-local knowledge, Choreo branches, protected effects, generated plans, routes, artifacts, Live/acquisition debt and independently required product capabilities;
- stable operation and source-span references, dependency paths, exact claimed proposition and subject, selected assurance profile, mandatory judgment, checker/interpretation version, needed versus supplied facts and accepted/open premises;
- a primary first **legitimately informed** blocker, with later unconstructed dependent objects classified as consequences, not competing root causes; independent defects may also be shown with their own distinct sources;
- precise categories for syntax or type error, absent/null refinement, unavailable knowledge/provenance, current authorization/resource loss, proof/derivation error, missing required source coverage, stale dependency, actual counterexample, unsupported judgment, incomplete checking, profile-premise exclusion and advisory business-intent finding;
- evidence origin and disposition: locally checker-verified, conditional, exact-finite, source-only, trusted/opaque, caller-reported, actually observed physical, unsupported, failed, inconclusive or not inspected, without compressing overlapping categories into a fictitious percentage of universal correctness;
- the exact **uninspected** Clojure/host predicate and external writer domains relevant to a claimed policy, even when the checked subset reports zero detected escapes;
- source-linked and privacy-appropriate diagnostic outputs that do not disclose another user's protected Request, Membership, principal or confidential rule inputs;
- an explicit repair owner and minimal affected dependency cone, including changes to source rule, Choreo/knowledge, model/FX, provider/handler binding, browser installation, physical writer or genuinely approved product requirement.

A valid but product-wrong rule cannot be labeled a mechanical type error merely because an assistant suspects a missing Membership guard. Conversely, an observed policy escape in actual host code cannot be counted as mechanically checked by placing its name into an audit. An unknown uninspected source area is **unknown**, not proof that no correctness-relevant branch, bypassing writer or extra authority condition exists. The original product capability denominator and current release gate remain independent of the explanatory inventory.

Attribute abstract counterexamples without inventing production events

A model-checker trace must identify the generated formal/model subject, original semantic source and admitted representation, selected state/input/operation schedule, exact violated property, abstraction and finite bounds, checker result and open environmental assumptions. Reverse links map each formal transition or assumption to the corresponding source rule/operation and required **production** enforcement or correspondence claim. An abstract counterexample falsifies the claimed property of its declared abstract subject and premises, not necessarily the running Clojure/XTDB/browser implementation. A production incident needs its own authenticated observation and source-to-physical link; neither a simulated trace nor a copied test-report map can provide that observation.

The explanation declares whether its trace came from an illustrative fixture, a genuine generated model backend, a bounded independent finite oracle, a concrete physical fault exercise or a separately qualified transfer theorem. Stage 15 may use a representative fixture to verify the diagnostic format before Stage 17 has a generated external backend. A release profile that **claims generated-backend counterexample translation** must actually execute that generated path and preserve the witness/derivation provenance. Do not silently present a fixture as a Recife, Quint, TLA+ or proof-assistant run.

For example, a conditional Claim model may permit two active primary RequestAssignments because the original declared abstract operation catalog omits a relevant Reassign or repair transition. Explain the exact original requirement, omitted source edge and checker assumptions first. Separately ask whether the actual backend writer inventory is complete, the selected XTDB transaction guards protect the relationship, and the installed browser ever observed the state. A successful abstract correction does not, without those further checks, prove that a physical race is prevented.

Repair only at the owning source and recheck what changed

Mechanically invalid source may receive a well-scoped candidate repair; a stale dependency needs source/consumer reconstruction; a missing external witness needs the proper test, checker, physical observation or an explicitly permitted open premise. If an invariant is weakened, a rule is changed, or a premise is added, that is a **different claim**, not an invisible repair that preserves the old proof or product approval. A suggested alternative in an explicit closed enum may be useful only when it preserves the actually selected semantic requirement. Where several valid business policies exist, the product owner chooses; a diagnostic does not invent a uniquely correct answer.

Ordinary Clojure remains the authoring surface. A confirmed new requirement enters its actual public model or declared operation/invariant/contract through the appropriate application authority. Gesso then regenerates the semantic source, dependent

formal model, checked proofs, plans, served assets and affected realization checks according to their dependency cone. Editing generated TLA+, Quint, Recife, cached proof objects, emitted JavaScript or explanatory report content alone is not a repair of the admitted source or installed application.

Human/LLM review is optional and **advisory only**: it may flag asymmetry, surprising positive behavior, missing role/disclosure restrictions or conflicting product rationale, but cannot issue certificates, change admissible operations, discharge premises, relax current authorization or silently modify a client/server effect. The visible explanation must remain read-only and redacted at the same principal/scope boundary as the underlying protected facts.

Acceptance pairs understandable source-local diagnoses with ordinary valid source, actual independent counterexample fixtures and corresponding source repair/recheck. Challenge a wrong-but-well-typed policy, misleading secondary error, copied checker report, source-stale trace, finite/external mismatch, claimed production incident with no physical witness, missing opaque-source inspection, unauthorized diagnostic disclosure and an assistant-produced unapproved policy change. A generated-backend claim has the extra positive obligation to exhibit an actual generated trace and reverse source link.

Stage 16 – Independent formal soundness of the stable kernel

Establish the concrete external soundness target of #req-formal-production-correspondence for the closed seven-rule propositional derivation system. The independently formalized syntax, inference semantics and acceptance relation are small enough to isolate a meaningful proof from arbitrary Clojure, Biff, XTDB and browser behavior. Their truth is not supplied by reusing the same Clojure implementation as both specification and checker. This stage shrinks a **specified** trusted assumption rather than declaring the entire Gesso framework proved.

Prove the exact initial calculus with open premises

The initial seven inference families remain:

- named premise use with its precise **open** source assumption and proposition;
- truth introduction;
- implication reflexivity;
- conjunction introduction, with its supported minimum arity and exact source operand order;
- indexed conjunction elimination;
- implication elimination with the admitted implication/antecedent reference pattern;
- indexed disjunction introduction.

Formalize finite proposition trees, inert declared opaque Boolean atoms, nominal subject/context, well-formed premises, ordered derivation steps, reference/currentness restrictions and the final goal. State the satisfaction and entailment relation over **all** supported Boolean atom valuations. If Gamma entails P is proved, Gamma still names the accepted premises; a premise assumption is not an unconditional atomic truth axiom. The calculus does not gain arbitrary embedded Clojure evaluation, unbounded arithmetic, SMT reasoning, hypothetical-context discharge, classical excluded middle or new inference rules because the formula language contains an operator with a similar name.

For the selected mathematical rule system, each admitted inference step must preserve entailment. Derivation soundness then follows by induction over any finite accepted derivation, not from testing one representative of each rule tag. Explicitly quantify over permitted formula shapes, conjunction arities, reference orders, indexes, source premises and all Boolean valuations. An independent proof must state any axioms, trusted checker/kernel, definition versions and semantic interpretation assumed.

Preserve five separate conclusions and their trust gaps

The external program reports at least five distinguished objectives:

1. **S0 — Inference-rule soundness:** the seven formal rules preserve the defined Boolean entailment under their exact arity/premise/typing restrictions.
2. **S1 — Derivation-system soundness:** any admitted finite proof term derives its goal from exactly the legally used source premises by a valid ordered derivation under the formal acceptance relation.
3. **S2 — Actual Clojure checker correspondence:** the real source parser, validator, normalizer, checker and certificate admission accept only concrete inputs that correspond to S1 under a qualified versioned decoding/source-to-formal relation.
4. **S3 — Typed atomic-proposition correspondence:** the selected source typed-rule evaluations, reader footprint, absent/nil/false behavior, immutable bindings and failures genuinely realize the opaque Boolean atoms in the formal system in their declared scope.
5. **S4 — Issuer/composition and source-to-production transfer:** the original application rule and obligation ledger, trusted issuer, dependencies, admitted operation and current installed Clojure/XTDB/HTMX execution bear the required checked relationships, retaining any physical writer, current authority and environmental premises.

S0 and S1 may be independently established before S2–S4. That is a valuable real theorem but not a theorem that the production Clojure checker is sound, that an arbitrary Membership predicate is true, that the issuer is safe against hostile JVM code, or that the installed HumanHelp Claim is correct. A finite truth table over the actual bounded set of atoms is separate exact-finite evidence; it cannot prove the unbounded all-derivations result or transfer opaque typed atoms/physical effects. No single `:proved` field collapses the five objectives.

Bind the theorem to an actually checkable artifact

The proof evidence includes its exact theorem statement, canonical source and formal encodings, admitted axioms or trusted primitives, versioned syntax/ rules/semantics, model and proof-tool versions, checked proof object or equivalent reproducible derivation, actual verification disposition, resources/bounds, and all open applicability conditions. Raw tool exit status, an EDN report, a digest or a claimed theorem name without checked content cannot issue independent proof authority.

A sound checker is not necessarily complete for all valid derivations under finite production resource limits. Its advertised supported domain, plain-data/metadata and type decoding, formula depth, step/premise count, atom encoding, conclusion identity, normalization and failure behavior must be specified and challenged. A conservative resource-limit rejection may preserve safety but cannot be counted as proof of the rejected statement or as a successful eligible product case.

An actual proof-assistant/kernel result, an incomplete proof, counterexample to an admitted lemma, unsupported syntax, tool interruption, timeout and malformed imported proof are different evidence outcomes. If a strict selected profile requires a currently available S2/issuer-qualified theorem, a missing or invalid result blocks **new** affected admission without downgrading its profile. It does not retroactively change a committed command's history; a still-current previously admitted deployment may continue only within its own qualification/lifetime.

Independent adversarial checks include omitted premise, false truth axiom, changed nominal atom, reversed implication references, invalid index, conjunction arity, cycle, transformed normalization, tampered formal artifact, changed axiom, unsupported tool result and false S2 checker correspondence. Pair each with valid proof derivations and an admitted usable application operation. The prohibition is at the actual mathematical/issuer boundary, not a copied report's status.

Grow formal assurance along real transfer edges

After the small theorem, add typed-rule and absent/nil interpretation, guarded atomic transitions and frames, conditional induction/source catalog coverage, role-local knowledge/projection, partial progression and browser/XTDB refinement only through named formal subjects and independently qualified transfer premises. Correctness of S0/S1 does not automatically establish soundness of these additional logics, the runtime framework, arbitrary host Clojure, real provider serialization or all multi-node schedules.

Application developers continue declaring business rules, model operations and Choreo in supported Clojure interfaces; Gesso owns generation and checking of formal representations. Lean, another proof assistant, Recife, Quint or TLA+ may serve a precisely chosen checking/exploration role; no one tool and no handwritten parallel HumanHelp formal specification is made mandatory.

Operational release, real-browser product testing and authoritative transaction safety remain active while these theorems are developed. Completion of Stage 16 is a specifically evidenced external soundness result for its admitted formal subject, not a blanket “framework formally verified” status.

Stage 17 – Formal-to-production transfer and release-bound evidence

Connect independently checked formal and source-derived results to the **actual** production checker, issuer and execution subjects that consume them. The correspondence direction, observation alphabet, source representation, selected environment, complete original obligation inventory, remaining trust and invalidation triggers must be stated for each claim. A genuine abstract theorem is still valuable when a physical bridge is open; its production applicability then remains conditional or unestablished rather than being silently promoted by a successful test or report label.

State the direction of each transferred guarantee

The source-owned model, executable Gesso semantics, generated formal subject, production checker, admitted operation, protected writer and installed browser are related through explicit **distinct** judgments:

- **Admission soundness:** every admitted concrete source/checker result in the claimed supported domain denotes a valid formal judgment under the specified translation, decoding, versioned semantics and premises. This is a direction from actual accepted inputs to the formal relation, not the assertion that all formal judgments are implementable or accepted.
- **Admission completeness/usability:** legitimate source cases in the separately advertised supported domain have the intended admissible path. A conservative checker may preserve soundness while refusing valid constructions; that cannot satisfy positive application gates.
- **Execution safety refinement:** every relevant physical execution or observed protected effect, under an explicit initial-state and step/observation relation and environmental premises, is represented by an allowed abstract behavior, possibly with justified stuttering. Proving a formal step is allowed is not proof the XTDB/host step implements that atomic step.
- **Progress and liveness transfer:** only under explicitly selected scheduling, fairness, continuing authority, user action, delivery, retention and resource assumptions. Safety simulation alone cannot establish convergence, execution completion or message receipt.
- **Reverse traceability:** every relevant original formal assumption, atomicity claim, source operation, knowledge requirement, provider read and environmental premise points to the exact corresponding production enforcement/observation duty. This direction of explanation is **not** automatically a reverse simulation theorem.

These relations bind the same original source, subject, actor, authority, version, reader/guard footprint, operation catalog, initial state, current proof scope and selected physical environment. Distinct source-derived semantic identity, implementation identity, physical installation and live execution generations are not interchangeable because a digest, route label or proposition spelling matches.

Carry every formal assumption back to its enforcing boundary

For each transferred proof or model result, derive an exact production obligation ledger: theorem, formal syntax and axioms, original model rule/operation/invariant, abstraction map, concrete entry path, guarded reads/writes, full frame and relationship footprint, decoder/normalizer, issued executor, authenticated principal and actual installed handler, provider/transaction contract, browser/Live interpretation and applicable environmental assumption. Mark each edge as proven, source-checked, physically observed within a bounded scope, independently qualified trusted, open, contradicted, unsupported or incomplete as appropriate. One opened edge does not become discharged merely because its dependent high-level abstract theorem remains valid.

The first S0/S1 seven-rule formal result does not itself prove S2 actual Clojure checker correspondence, S3 typed-atom/reader meaning, or S4 issuer and source-to-production transfer. The production normalizer's acceptance of a changed atom or omitted premise is a concrete S2 failure even if the abstract rule calculus is sound. A finite generated-model example or independently implemented bounded oracle can falsify a claim; it is not a quantified implementation refinement theorem without a justified abstraction/coverage relation.

A formal atomic Claim must correspond to a selected physically atomic guarded multi-document Request/RequestAssignment transition, including current Membership and Organization conditions, one-primary/cardinality, all relevant writer paths, presence/frame, historical command result and required publication intent. A pre-transaction external read does not attest the transaction-internal prestate, a returned XTDB transaction key does not identify every internal guard input, and matching readback cannot alone establish which writer caused the state. If the selected backend cannot independently expose such a witness, preserve the precise trusted serialization/refinement premise and test that selected backend and allowed physical forms rather than claim all XTDB has been proved.

Every original application initializer, bypassing writer, external provider, direct database client, asynchronous host path, loaded browser artifact and resource-generation change that matters to the proof remains an independent coverage or physical assumption. The original source's operation/invariant and product denominators are independent of a generated checked subgraph. A profile demanding proved transfer cannot relabel an opaque assumption, absent physical observation or unexecuted test as an admissible theorem. A narrower explicitly selected conditional operational profile may remain legitimate.

Qualify generated evidence, source repair and release use

Generate formal artifacts, checked plans, proof/evidence packages and explanatory sidecars deterministically from the admitted original semantics and selected toolchain/versions. The issuer rechecks the actual current source, complete obligations, theorem/axiom/derivation evidence and allowed premises; cached proof-shaped maps and external tool exit codes do not grant permission by themselves. A changed checker, translator, provider, route, writer, served JavaScript or installed generation invalidates the affected **applicability** evidence without necessarily disproving an unchanged abstract theorem. Rebuild the smallest sound dependency cone while preserving mandatory checks and actual physical requalification.

A formal counterexample with an actual generated-backend claim must come from that generated backend and retain its model, trace, assumption, abstraction and source-reference provenance. A fixture or synthetic schedule is useful only under its accurately declared evidence class. Editing generated TLA+, Quint, Recife, proof data or code-emission output to avoid a counterexample cannot repair the original source; an authorized model/operation/requirement change and its dependent rechecks are required.

Acceptance pairs eligible current checker admission, a real permitted authoritative Claim and a correctly bound physical/browser result with stale theorem, changed source/normalizer/issuer, forged proof report, omitted obligation, invalid abstraction, uncovered physical writer, swapped asset, unsupported transaction guard and an unavailable mandatory external checker. The first **actually enforcing** boundary must prevent the disallowed effect; an after-effect diagnostic does not establish prevention. Finite conformance vectors, a successful test suite and source inspection are separate scoped evidence and cannot replace a required universal S2/S4 transfer theorem.

Stage 17 strengthens the release machinery already required by Stages 4, 6, 8, 10 and 11. It does not introduce source freshness, installed handler safety, real-browser tests or actual protected authorization for the first time. Ordinary useful applications may be accepted under their explicitly narrower profiles while unproved stronger formal scope remains open.

Stage 18 – Qualified multi-node realization

Realize the **same admitted Gesso semantic machine** across actual independently failing application processes, shared authoritative storage, physical router/dispatcher instances, browsers and compatible release generations. Single-node and multi-node differ in **placement and environmental correspondence**, not in the public model's Claim authority, logical command identity, Choreo meaning, provisional browser status, opaque provider progression or HumanHelp selective adoption policy. The baseline and separate production milestone are defined by #req-multi-node-realization and #milestone-multi-node-production.

Separate actor, command, host and authority identities

An authenticated principal, model actor, Choreo participant, browser execution, server process, interest router, database authority, HTTP attempt, logical command, physical submission and resource owner are different identities. Moving a request from host A to C does not transfer authorization to C as a business rule or mint a new command. Each new trusted read/write revalidates the selected principal and scope at its current boundary; client and router assertions do not supply Membership.

One logical Claim C42 can be attempted on A, lose the response after possible submission, and be resolved or retried through C under the **same immutable** original command binding and current authorized disclosure. A process- local success/abort report at A is not distributed command history; C must consult qualified retained authoritative command records or preserve unknown. A later Request state and matching card may show current model truth but cannot attribute C42's incorporation. A retry under C43 is a new intent, not historical resolution.

Deliver at least two genuinely independent application processes under a declared topology, actual load balancing, controlled host replacement, a selected qualified shared authoritative backend, protected writer and reader coverage, current session routing, browser assets and directional old/new protocol compatibility. A two-process prototype with a shared DB can test meaningful failure schedules before all other milestones are complete, but is not thereby a full production distributed release.

Make publication durable without assuming delivery

For a selected strong durability profile, derive notice intent from source-owned committed changes and incorporate that intent, canonical command result and guarded domain effect in one suitable authoritative atomic boundary. Production publication must not depend on the write host's transient memory or on a complete snapshot of every browser's current interest; a missing interest at write time must not erase required change/recovery information.

A dispatcher claims durable work through a declared concurrency/lease/fencing or equivalent ownership discipline, publishes with bounded retry/backpressure and records only acknowledgments whose meaning and failure boundary have been selected. A successful broker acceptance, router forward, connection write and final browser receipt/adoption are distinct facts. No per-recipient successful-send ledger is required merely because notices can duplicate: the selected scheme may redeliver a conservative superset after probable router failure if it preserves recipient disclosure, idempotency, bounded-resource and recovery premises. A guessed failure window alone is not a durability or eventual-delivery proof.

Publishers may crash between commit and dispatch, after dispatch before accounting, and during router handoff. Duplicate invalidations must never incorporate duplicate domain operations or silently consume a fresh one-use user Refresh. Sender completeness, routing filters, late registration, received progress, protected acquisition and actual DOM adoption each have separate proofs, observations or declared conditional gaps. Publication after commit may create outstanding work or advisories, but cannot retroactively reject the already incorporated domain transaction.

Fence registration, route interested scopes and recover gaps

A distributed router cannot infer that its interest graph is complete merely because its own local process knows no subscriber. A partitioned coarse/fine interest filter, replicated filter table or hierarchical routing graph may be used, but the actual topology must establish where interests are stored, propagated and fenced, which router can lose information, and how replacement/partial propagation recovers missed changes. A successor router with a stale filter must not silently certify that no subscriber cares about a notice.

An initial authenticated, scoped authoritative observation and interest registration require an appropriate qualified fence, replay, gap-detection/reacquisition relation or an explicitly weaker user-recovery guarantee. The design must cover the interval when an invalidation occurs before a web node learns or shares its new interest. An outbox alone does not solve that gap: persistence of a change is not proof of the receiver's eventual interest or ability to reconstruct its advisory requirement. A lost notice, router crash or failover must not make the next authorized acquisition path depend on receipt of that very notice.

Safe duplicate forwarding may be an intentional optimization. Its semantics are advisory only, subject to current recipient scope, authorization/disclosure, honest provider progression and declared work/memory/connection bounds. Neither a coarse “maybe interested” match nor the highest numeric token authorizes private detail or grants an automatic board read.

Preserve selective acquisition and convergence across hosts

For HumanHelp, remote Create and remote lifecycle changes remain advisory to Bob on any host. A permitted one-use explicit Refresh authorizes one scoped board acquisition; a second notice during the cycle cannot create another user grant. A qualified newer result may cover earlier debt when the trusted provider proves its exact scope and progression. An incomparable basis, missing Request, partial OOB installation or failed disclosure check retains its corresponding debt or blocks adoption. Authorized initial acquisition and an explicit Refresh path remain usable when a notice was never received.

Creator-local and narrowly scoped own-command settlement/recovery are separately admitted exceptions, not automatic remote board updates. A later node cannot use a background SSE delivery to grant browser adoption, force independent user action, disclose currently revoked Membership or resurrect an old execution's one-shot writer/DOM capability.

The strong convergence property must quantify the relevant current authorized scope, retained records, registration/replay, provider progression, actual read availability, continued principal rights, fair scheduling, bounded recovery resources and **user action when the product deliberately requires it**. If Bob never chooses Refresh, the selected policy cannot promise automatic visible convergence. It can promise bounded advisory or independently available authorized acquisition under its actual stated environmental premises.

Test physical faults, realistic workloads and honest proof scope

The production milestone requires a measured fault envelope and actual multi-process, multi-client, authoritative-backend, browser and resource observations:

- host A dies before submission, after possible C42 commit and before HTTP result; C attempts exact authorized resolution;
- commit and outbox succeed, publisher dies before notice, then replacement dispatcher drains/replays under the selected acknowledgment model;
- notice forwards twice or to a stale coarse filter; the recipient neither leaks protected data nor duplicates a user grant or command;
- B registers interest between source change and router propagation; fence/replay/gap recovery prevents silent false freshness;
- two routers fail or transfer ownership with incomplete interest knowledge, while eligible independent acquisition still works;
- Bob reconnects through C with appropriate principal binding, old/new client protocol and non-adoption until his permitted Refresh;
- provider basis comparison fails or is incomparable; the system retains unresolved debt, never invents order;
- Membership is revoked during fault recovery; protected reads/old command results remain gated by current disclosure rights;
- delayed and duplicated physical effects from an obsolete node/client fail at their actual fencing and resource-owner boundary;
- unrelated routers, ordinary traffic, long-lived SSE subscriptions and repeated failures stay within the selected memory, queue, connection, replay and latency envelope.

Non-HumanHelp examples should use the same distributed semantics, not a new service- specific invalidation engine. A Clojure implementation first and a later Rust router are permissible engineering choices, not two frameworks or a different semantic contract. Direct participant messaging, durable suspended workflows and a universal distributed fabric remain separate optional capabilities, not prerequisites for ordinary authoritative rendezvous.

Real fault tests and finite model exploration can expose bad schedules and provide bounded production evidence; a universal distributed safety/refinement or liveness theorem needs its own formal subject and justified environmental transfer. Multi-node production completion is distinct from operational HumanHelp, compositional certification and external kernel verification. The accepted release must state exactly which processes, clients, routers, backend, assets, workloads and failure windows were physically qualified, which premises remain open, and where the stronger requested completion is not met.

Dependency structure and delivery checkpoints

The eighteen stages provide **contract navigation and normal delivery priority**, not a strict serial project plan, implementation-progress percentage or evidence that every earlier stage has passed all of its strongest exit gates. One admitted application can be built and evaluated in an explicit supported scope while larger proof, product or multi-node work proceeds separately. Every result must retain the original subject, the selected claim, actual positive behavior, missing mandatory duties and exact evidentiary strength. Part XI independently owns release and completion decisions; this section only maps their inputs.

Classify dependency edges before using the stage table

The roadmap distinguishes five different relationships:

- **Semantic prerequisite:** a consumer cannot legitimately interpret or use a subject without the earlier exact semantic contract and required facts.
- **Implementation priority:** completing this construction first usually reduces rework, although independent exploration may begin sooner.
- **Exploratory feedback:** bounded prototype or finite/model/browser test may reveal a missing source contract; its result does not gain the stronger qualification of a finished downstream system.
- **Optional early hardening:** cheap identity, proof, instrumentation, mutation, lifecycle or release safety mechanisms may close an immediate real defect. Their early presence does not certify all later stage responsibilities.
- **Completion dependency:** the requested **named** milestone/release profile cannot pass until every independently required gate has actual applicable evidence or an explicitly allowed and declared premise.

A dependency in a checker source, proof DAG, generated route registry or document stage list is not automatically evidence that the **original** required capability, writer, initializer, policy predicate or physical path inventory is complete. Independently establish the relevant denominator before declaring source-closed coverage. The checked subset and complete requested application/product must remain distinguishable.

The delivery priority is: establish original public semantics, typed rules and actual Biff/XTDB host; make one real, permitted Claim work through admitted operation, Choreo, browser, guarded transaction and scoped recovery; close the same application's source/physical path; expand the real HumanHelp product and generic adverse coverage; compose proof claims and independent mathematical obligations; then qualify production-formal transfer and actual multi-host fault envelope. A small external proof, two-host experiment, or source-derived diagnostic can start as soon as its real prerequisites exist. No advanced proof fixture repairs a missing enabled browser Claim or bypassed writer, and no working Claim excuses an unproved mandatory source/correspondence obligation.

Principal contract graph for the eighteen stages

The table names the **minimum construction contract normally consumed**, not full completion of every predecessor. "Selected slice" means the specific admitted source/implementation profile named in the claim. The full original milestone still needs all applicable stronger gates.

Stage	Principal contract inputs	Scope of this edge
1	Original public model and independent requirement owner	Define source semantics, stable subjects, policy/actor identiti
2	1	Closed/versioned rule judgments and explicit opaque Clojure
3	1, selected 2	Propositional and guarded/induction proof mechanisms only
4	1 and selected host/provider interfaces	Actual Biff/FX/XTDB and browser invocation/lifecycle sema
5	1, relevant 2, 4	Source-owned operation/command and current trusted guar
6	4, 5	Exact supported transaction/writer/reader realization and its
7	1, relevant 2, 5	Global Choreo, role-local knowledge, justified projection and
8	4, 7	Pure adapter and qualified physical browser effects/HTMX; l
9	5, 6 for publication; 8 additionally for browser acquisition/adoption	Durable notice, opaque progression, interest/replay, permissi
10	Selected 4–9 plus original 1 and applicable 2–3	One admitted, actually installed application scope with sourc
11	Selected 10 and exact release/profile/host intent	Four current release decisions and directional compatibility;
12	Selected 5–11 and independent Part VIII product inventory	Real HumanHelp all-required-capability and fourteen-gate co
13	1–3, 5, 7 and selected 6, 8–10 as Claim realization requires	Actual source-complete compositional Claim pilot: abstract c
14	Each boundary under test; 3 and 13 for proof-specific cases	Mutations start early with positive control, first enforcing ga
15	1, 2 and actual currently available boundary evidence	Source-local first-blocker and policy review begin early; gene
16	Stable 3 calculus and 2 where typed-atom transfer is claimed	Independent S0/S1 theorem may precede S2–S4 production/i
17	Relevant 3, 6, 10–11, 13 and 16 under selected formal profile	Distinguish checker admission, source/physical safety simula
18	Selected real 4–11 semantics, host/authority and recovery profile	Exploratory multi-host path may precede 12–17 completion;

This graph is not a demand to implement every capability of Stage 9 before **publishing a first durable notice**, or every feature of Stage 8 before **exploring multi-node command failover**. The publication half consumes authoritative operation/transaction contracts; physical browser adoption adds browser rights, user grant and actual installation. Similarly a conditional seven-rule kernel theorem need not await production HumanHelp Response, while S4 transfer to its installed writer cannot be claimed without corresponding physical coverage.

A runnable Claim slice consumes the actual trusted route/FX and protected transaction path before its first observed user-facing effect, even if Stage 10's **general** closure and Stage 11's **complete** release management are unfinished. An early narrow staging instance needs its own valid installation and enforced safety scope; a green in-process assembly is never evidence the tested/production listener used the checked handler. The same distinction applies to the actual emitted/served/loaded browser artifact, not just a correctly shaped manifest or generated file name.

Avoid apparent construction and evidence cycles

The selected source, checked subject, physical plan, actually installed executor and observed command are different objects. Their relationships can be built in increments without allowing a later report to retroactively manufacture an earlier right:

- A **startup** preflight seam exists before broad whole-app closure; its first served profile rejects the known missing inputs and unsafe early initializer/handler effects. Full source/writer/ alternative-route coverage is a separately checked later claim.
- A **source-derived** operation and provisional browser plan may exist before all formal proofs; first actual user acceptance still requires current model authority, correct physical effects, honest error/unknown handling and independent positive evidence.
- An **initial** Live read and registration mechanism precedes mature durable routing. Multi-node late-interest fencing, publication replay and user-authorized acquisition may be explored early, but a prototype cannot claim full availability or remote automatic adoption from an advisory.
- A **first** diagnostic/finite counterexample can be checked before generated external-formal semantics; an illustrative fixture cannot be relabeled an executed generated-backend counterexample or a theorem about production.
- Source-proof/issuer admission and physically installed realization are linked by selected correspondence relations, not by matching digests. An incomplete actual writer inventory is an open premise even when the submitted proof DAG is closed.

When downstream exploration exposes a missing rule, actor distinction, guard/frame, source obligation, dynamic recovery branch, interest fence or incompatibility, repair the **original owning contract**. Rebuild only the actually affected semantic, proof, plan, handler, browser, routing and installation evidence dependency cone; do not copy new policy into a separate prover, router or browser machine to evade the source. A genuine unmodified abstract theorem may be reused while its previous physical applicability becomes stale.

Useful integrated checkpoints and honest positive witnesses

A checkpoint names the **first independently useful integrated result**, its allowed evidence and its still-open stronger obligations. It is neither a schedule promise nor a permanent success status.

Checkpoint	Required useful result and first admissible positive
Public semantic operation	Real model-owned Claim with actor, Request/RequestAssignment, current guard/effect/command and
Physically guarded command	Eligible Claim incorporates through selected actual XTDB writer, with competing/unauthorized nega
Actual browser Claim slice	Installed emitted artifact presents enabled control, reaches trusted Biff route, settles/recovers permitte
Live selected acquisition	Initial authorized board can be acquired; advisory and explicit one-use Refresh are separately exercise
Closed installed application profile	Original selected source/capability edges, preflight, handler, route, asset, effect and required recovery
Real HumanHelp product	All Part VIII independent capabilities and fourteen positive/adverse product gates, including six lifecy
Compositional Claim certificate	Same original model/relationship/operation/Choreo/plan and real pilot bound by accepted source/issu
Independent kernel theorem	Actual checked S0/S1 theorem for the stable selected calculus with reproducible proof, premises and a
Qualified formal transfer	Selected S2–S4 obligations and concrete-to-abstract execution relation are proved or accurately qualif
Production multi-node	Actual independently failing hosts, original command/history and durable notice recovery, fence/repl

Each checkpoint has paired eligible/adversarial tests at its **first legitimately enforcing** boundary. A defect prevented only after an unauthorized write or browser disclosure cannot count as before-effect prevention. A reject-everything implementation does not earn credit for its negative results without a usable valid operation. A resource, backend or test that has not been physically exercised remains an explicit limitation, not a counterfactual pass.

A complete real Claim path includes compatible replay and `:commit-unknown` after possible transmission, changed current Membership, genuine competing assignment, and ordinary creator- local/remote-board distinctions. A new browser execution,

reconnect or host switch does not authorize the old one-shot effect or erase command-specific resolution. Strong physical readback, proof, browser installation and model authority each report their own observations and assumptions.

Completion mapping and honest progress reporting

Part XI owns the original **four independent completion milestones**, their subject-specific qualification and the **four release decisions** (candidate, installation, activation, completion). The release decisions do not themselves determine which milestone has been achieved; stage labels do not establish either result. The following is a dependency map, not a second completion policy or an ordered percentage scale.

Independent milestone	Evidence path required for its chosen profile
Operational application	Real installed 1, 2, selected 4–12, plus applicable adverse/runtime evidence from 14–15 and all independent P
Compositional certification	Selected 1–3 and 13 with original source/operation/invariant coverage, genuine Claim path, current issuer ar
External formal verification	16–17 for **the requested S0–S4 frontier**, actual theorem/tool identity, conditional premises and correspond
Production multi-node	18 plus selected 4–11 authority/release and actual topology, failure, recovery, workload, security and browse

Do not infer that the **operational** milestone has to wait for every S0–S4 theorem or the final multi-node topology, or that a small real two-host fault experiment can be counted as the **production multi-node** milestone. A release explicitly requesting those stronger claims must meet its independently predeclared requirements. A claim-specific conditional proof may be genuinely valuable without satisfying an independent product-completeness milestone.

For every progress measure, freeze the original required numerator/ denominator subjects **before** checking the currently exposed application. Report eligible product capabilities, source/assembly closure, semantic judgments, physical installed-path enforcement, formal transfer, mutation first-gate quality, fault recovery, asset/host compatibility and executed tests as distinct dimensions. Distinguish `:passed`, `:failed`, `:incomplete`, `:unavailable`, `:unsupported`, `:skipped`, `:stale` and `:missing` at the actual Part XI gate boundary. Do not infer that uninspected == empty or that unavailable == inapplicable. A stricter requested profile may not be silently replaced after a failing check by a new smaller one.

A checkpoint report binds its source, selected claim, known environment, admitted model/plan, actual installed host/writer, served browser artifact, issued/observed evidence and remaining premises. An original product owner can approve a **new** requirement subject, with old and new counts preserved; a generated registry cannot lower the old denominator itself. A previously valid abstract theorem can survive an unrelated CSS change while the deployed asset/generation still needs independent physical compatibility checking. Conversely, a changed protected writer can invalidate physical applicability without retroactively disproving the unchanged theorem.

The next useful work item is the missing **highest-consequence currently required boundary** for the chosen claim, not automatically the next stage number or the easiest proof/report to generate. Exploratory discoveries may increase the list of named open work without reversing genuine completed applicable evidence. Progress remains net improvement against the stable obligations, not a forecast based on revisions, tests or checked-file counts.

Worked Claim path: one meaning through the stages

Follow **one** HumanHelp Claim source through construction, actual execution, recovery and increasingly strong evidence. This is an acceptance contract and illustrative failure history, not a collection of eighteen differently implemented Claim applications and not an invented Gesso API. R4's sketches of calls named `admit`, `issue`, `prepare` or `check` were conceptual checking boundaries, not guarantees that those exact callable functions exist. The description here names the source, proposition, authority and physical observation that each real supported implementation must establish.

Assume an authenticated, currently eligible helper **Alice** claims an existing open Request R from its permitted Organization. The original authoritative model owns Request, Membership, Organization and the separate RequestAssignment relationship. The active primary assignment is **not** a persisted `:request/helper-id` invented for a convenient demonstration. **Bob** is another authenticated user with a separately authorized board and the selected remote-advisory-only presentation policy. Alice's Claim intent is logical command **C42**; it may have multiple actual HTTP/ server/transaction attempts. Its operation, request target, arguments and original principal binding must remain immutable and compatible through recovery. The names are narrative identifiers, not prescribed string, UUID or keyword encodings.

One original operation, several identities and evidence subjects

The static public Claim declaration records the original subject, canonical arguments, business policy/guards, expected input readers, protected relationships, guarded effects, results, permitted disclosure and recovery. It has **no particular actor, command ID, execution ID, transport attempt, source read basis or browser resource** yet. Its generated Choreo and view attach an affordance to this same model operation without becoming a second implementation of Claim.

The following coordinates remain distinct throughout the trace:

Coordinate	What it identifies
Public `:request/claim` operation and model source	Original business transition, authoritatively owned policy and versioned dep
Request R and RequestAssignment aggregate	Target and protected one-primary/relationship history
Authenticated Alice and target Organization	Actual trusted principal/relationship for one permissible disclosure or effect
Logical C42 and canonical binding	Exactly one intended model command across compatible attempts
HTTP request/transport attempt and host A or C	A particular delivery, connection or serving process
Execution and one-shot physical submission	One admitted attempt, actual plan, provider and bounded effect permission
Browser plan, target/resource instance and effect generation	Exact installed and currently permitted local behavior
Assembly, browser artifact and release generation	Checked source-to-plan/route/build/host interpretation
Provider basis/qualified read and original command record	Scoped authoritative observation or C42-specific historical evidence, respect
Proof, issuer, test and physical observation identities	Evidence of exactly their checked propositions and environments

The browser may transmit C42, a route target and protocol data; an actor label, client-provided basis, certificate-shaped map or displayed Claim button is never trusted identity or authority. Trusted server admission reconstructs the authenticated principal, the selected original operation, canonical target, clock, required current model/Graph inputs, permitted disclosure, execution context and applicable handler/provider profile. Structural admission still cannot promise that Alice's Membership or R's open status will remain true at actual incorporation.

Stage-by-stage construction remains one trace

The table names the minimum meaningful **same-source** construction/evidence join. Later stages strengthen the scope instead of reimplementing the business transition.

Stage	Same Claim subject and added work
1 — semantic ownership	Original HumanHelp Request/RequestAssignment model, Alice/R/Organization policy and independently re
2 — typed rules	Source-derived supported guard/read/nominal/absence interpretation and explicitly opaque Clojure policy c
3 — small proof mechanisms	Original Claim abstract guarded transition, initial/invariant obligations and eligible proof candidate
4 — qualified host	Actual selected Biff/Ring/FX/XTDB invocation, lifecycle, clock/principal and trusted effect context
5 — authoritative operation	Static Claim, immutable C42 binding, resolve-before-new-incorporation and current business guard/effect sp
6 — physical mediation	Complete guarded multi-record transaction, command-result and selected durable notice intent through act
7 — global Choreo	Alice/authority roles, required knowledge, authoritative action, causal result/recovery, projected role-local p
8 — browser realization	Pure adapter, generated plan, exact target/effect rights, real HTMX/DOM install and optional optimism
9 — Live/adoption	Source-derived change/debt, selected durable dispatch, qualified read basis and explicit user-granted remote
10 — application closure	Original source-to-operation/plan/route/model/handler/asset/acquisition graph and first physical preflight
11 — release	Candidate, installation, activation, compatibility and evidence/currentness of the exact served generation
12 — HumanHelp product	Actual enabled Claim, all other lifecycle operations, rich board, Response selection, selective Refresh and pr
13 — compositional pilot	Source-complete Claim obligations and three separate semantic, installed and real-browser conclusions
14 — mutation	Positive Claim plus competing, revoked, lost-response, stale-writer, stale-browser and false-evidence mutati
15 — explanation	Source-relative first blocker, open physical premise and actual/illustrative counterexample provenance
16 — independent kernel	Formal S0/S1 seven-rule soundness under named axioms, proof tool and assumptions
17 — formal transfer	S2/S3 and S4 issuance/source-to-physical links, concrete-to-abstract execution refinement when selected
18 — multi-node	Same C42 and model truth across A/C hosts, retained outbox, interest fencing/failover and Bob's selected or

The operation is not “proved” just because a stage's table row exists. Each strengthening is attributable to an actually executed checker, approved source, issued handle, qualified external assumption, installed physical boundary or observed result. Claim completion and the independently required Part VIII capabilities have different denominators. In particular, real manager Reassign can later affect the same aggregate and must appear in a claimed full invariant operation catalog even if this specific trace follows Alice's self-Claim. The independently selected Response interaction at #decision-humanhelp-response-flow remains an open product requirement until the owning application makes and qualifies it.

Trusted incorporation: one command, complete guards and writes

A genuinely eligible positive path needs a rendered and enabled Claim control bound to the admitted semantic operation and installed plan; the generated browser sends C42 to the actual effective route. The serving handler qualifies its original model/FX inputs and authenticated Alice, binds R's target independently of client role claims and checks a compatible historical command record **before** treating C42 as a new incorporation attempt.

For a genuinely new C42, the actual atomic guard protects R's open/eligible status, current Alice Membership/Organization association, absence or permitted history/cardinality of active primary RequestAssignment, the correct immutable command binding/exclusion, and all relevant source-owned policy reads. A selected strong profile atomically applies the real Request lifecycle mutation, creates or updates the appropriate **separate** assignment/history, retains the C42 canonical result and records required durable publication intent in the same suitable authoritative boundary. Optional after-commit dispatch is a separate effect and cannot turn an incorporated Claim into a rejected command.

The selected XTDB transaction must actually implement the full guard/effect/frame, including presence and previously unseen fields, uniqueness, revision or history-sensitive ABA exclusions when required, and other relevant writers. The actual statement or immutable lowered transaction, exact provider, authentic submission and qualified transaction-internal prestate relation are different pieces of evidence. An external pre-read, a syntactically valid ASSERT/PUT, a process-local one-shot return or a matching later Request card does not independently prove all four. If a required physical witness or all-writer enforcement is unavailable, record the precise open provider premise or fail a profile that mandates proof.

A successful incorporation must support an eligible positive physical readback after appropriate provider progression. A competing valid helper's attempt, revoked Membership, association change, same-ID incompatible binding, missing primary guard, duplicate assignment or stale snapshot must receive its intended typed rejection/unknown outcome; a runner that only proves such mutants fail while no real Claim works has not demonstrated the operation.

Failure schedule C42: authoritative truth and actor knowledge diverge

Assume the selected strong profile has an atomic domain/command/ outbox record and the requisite shared authority. The following is a **prospective controlled execution schedule**, not a report that the current HumanHelp app or multi-host runtime has run it.

Step	Physical or user event
0	Alice has the permitted R affordance and creates C42 before its first possible send
1	Host A admits an attempt; C42's new-command guarded transaction actually incorporates
2	A's response is lost after possible physical submission
3	A dies before dispatching the durable notice and her old browser plan/resource retires
4	Host C receives a permitted recovery/resolution for **the same bound C42**
5	Replacement dispatcher claims/retries original notice intent; a router redelivers an advisory
6	Bob's router learns or restores his scoped interest; he may receive the advisory late, twice, or not at all
7	Bob is shown an allowed pending indicator and explicitly selects Refresh
8	An authorized provider read returns a qualified basis/coverage result and the actual browser finishes all required HTMX/OOB instal
9	A subsequent Reassign/Unclaim occurs or Bob's Membership/visibility changes

At step 4, **successful compatible historical replay is not new incorporation**. A new Claim guard may now reject because R is already claimed; the resolver must first check the retained exact binding and independently authorize disclosure of the old result. If C42's record is absent, unreadable, expired, not yet visible at a qualified basis or outside current disclosure, a new current card cannot manufacture attribution; the status remains unknown until appropriately established or excluded by the selected protocol. An attempt on A failing or aborting does not prove a concurrent attempt on C never incorporated. A different C43 is a different intended command, not a safe replay of C42.

At steps 5–8, Live invalidation is **advisory** for Bob, including remote lifecycle changes to already-visible cards. An eligible manual Refresh must remain possible even when the advisory is lost entirely. The current user grant covers one logical acquisition, not one new acquisition for each arriving notice or one per network retry. Provider scope/basis and actual physical OOB/DOM completion establish which debt it covered. A newer-looking token or HTTP 200 does not create provider entailment; an advisory arriving mid-Refresh cannot silently mint a successor read. Creator-local Create and narrowed own-command recovery are separately declared exceptions, not automatic Bob-board replacement.

Alternate schedules expose the necessary distinctions

A complete acceptance corpus also includes these nearby cases:

- **No incorporation:** Alice's Membership is revoked, R is no longer eligible or another primary Claim wins **at the actual guarded transaction**. The corresponding new command rejects with the typed actual reason; a rejected fresh command is not an inferred outcome for earlier unknown C42.
- **Potential incorporation without available history:** an HTTP timeout or host crash leaves command record access unavailable. Preserve `:commit-unknown` or the precisely permitted unresolved result; do not treat "no record seen" as exclusion without a qualified fence.
- **Same command, multiple attempts:** attempt A1 aborts under contention while A2 incorporates C42; the command outcome is not the Boolean result of A1. Incompatible actor, arguments, operation or scope reuse under C42 is rejected rather than replayed.
- **Bad browser continuation:** a delayed callback from Alice's retired target or an old release tries to write a new DOM incarnation; last-effect ownership prevents it even if an earlier scheduling check passed.
- **Publication/interest gap:** commit persists, dispatcher fails, backup router carries an incomplete coarse filter, or Bob registers interest after notice generation. The admitted recovery path must not certify false freshness from silence, nor disclose protected details to a merely **possibly** interested recipient.
- **Proof and installation substitutions:** a conditional abstract Claim theorem is supplied with an omitted Reassign operation; a checked artifact receipt is offered for bytes another host served; or a valid local wrapper is bypassed by deployed routing. Each requires source/inventory or actual physical rejection before claiming full certification.

The test oracle records its **original requirement**, injected change, eligible positive, first legitimate rejection point, actual observed forbidden physical effects and applicable limitations. Seeing a failure afterward or reading a copied report is not the same as prevention or independent physical conformance.

Certification uses the same source, not a proof-only Claim

The source-owned Claim policy, original complete relevant operation and invariant catalog, supported closed typed rules, guarded relation, Choreo, projected plan and application assembly are the source subjects of Part IX's #req-humanhelp-certification-pilot. The proof checker and dependency issuer establish only their own propositions and exact open premises; source dependency completeness cannot be inferred from an applicant-provided DAG. The original model still owns opaque Organization policy and real multi-record RequestAssignment meaning.

The pilot keeps **three distinct outcomes**:

1. Semantic source checking and conditional abstract proof: verified derivations/finite assertions over the selected original operation catalog, with correct nonvacuous initial/Claim examples, declared premises and bounds.
2. Production binding: current issued operation and active trusted Biff/Ring route, guarded XTDB writer, authenticated principal, command result, selected outbox/Live and built/served/loaded browser artifact match the admitted source under appropriate independently checked or explicitly still-open physical correspondence.
3. Real product behavior: an actual enabled Claim in supported Chromium/HTMX, authenticated Request and Membership, guarded persistence, reload, permitted settlement and controlled failure/recovery occur through the real production-backed model/installed UI.

A small propositional kernel proof may establish S0/S1 over its seven inference rules; it neither establishes S2 actual Clojure checker correspondence, S3 typed policy atom interpretation, nor S4 source-to-installed physical transfer. Formal admission soundness and concrete execution safety simulation are separate propositions. A changed provider, verifier, compiler, asset or host can invalidate physical **applicability** while leaving an unmodified abstract theorem sound. An early finite example, a successful model-only fixture or a browser test by itself never closes the missing intermediate edge.

Evidence labels must identify original source, current subject, checker and actual proof/run status, issuer, premises, provider/host/asset qualification, observation window and selected release profile. A copyable certificate-shaped map, named theorem, fingerprint or locally matching readback cannot be substituted for an independently checked issuer or physical fact. The four release decisions and separate operational, compositional, external-formal and multi-node **completion** milestones retain their Part XI definitions; a narrow Claim proof pilot cannot silently finish the independent HumanHelp product inventory.

Source changes preserve the claim and invalidate its dependents

A legitimate change in HumanHelp's Claim eligibility or Organization policy is made through its original public model/requirement owner. The dependent rule, operation, Choreo knowledge/projection, plan, application closure, relevant proof and issuance, currently served browser bytes, runtime guard and selected real-user/fault evidence must be rechecked when their semantics or

qualified implementation dependencies change. Unrelated Search evidence and an unaffected abstract theorem may be reused only after checking their actual dependency/interpretation scopes.

A new XTDB adapter, added direct writer, Biff handler, host version, browser build or router topology need not change the original mathematical Claim meaning, but does require the relevant physical/writer/asset/fault requalification. A copied old receipt or cached result cannot mint a new current release admission. The corrective action belongs at the first owning source or physical boundary, not by teaching the browser, router or proof generator a second Claim policy.

Across all eighteen stages this is **one continually strengthened Claim**: the model supplies truth and policy; Choreo supplies justified interaction and role knowledge; the browser owns only provisional and adopted presentation; the physical writer owns authoritative incorporation; command history owns C42-specific attribution; Live owns advisory publication/recovery; checkers own their exact abstract propositions; release admission owns current installed applicability. Every stronger claim needs evidence at its own actual boundary, and every missing original HumanHelp capability remains visible outside the successful Claim slice.

Cross-stage acceptance invariants

Gates A–H apply from the **first boundary at which their subjects are constructed, interpreted, installed or used**. They are not a final ceremonial release checklist, nor a fresh, competing policy layer. The owning Parts and original source define their full semantics. Each gate names: the original admitted operation or requirement, the boundary that can first know/enforce it, the permitted positive behavior, an independently falsifying neighbor, the actual checker or physical observation, and any residual premise. Completing a generated graph does not prove that its original obligation denominator, deployed writer or independent product inventory was complete.

Gates A–H therefore connect the permanent executable semantic core to its selected implementation, runtime and evidence boundaries. Gates I–P additionally govern admission by representation, product completeness, outcome knowledge, adoption rights, proof/evidence scope, source audit, formal transfer and human intent review; the two groups must be jointly satisfied for a release profile that requires them.

Gate A – No silent semantic downgrade

A public semantic operation retains its original model-owned command, required inputs, role/authority, typed results, recovery and effect contract through view construction, Choreo projection, generated plan, route, browser event and trusted invocation. A missing or incompatible binding is rejected at the first knowable boundary or the affordance is explicitly unavailable. It may not become an unmarked ordinary HTTP/HTMX request, anonymous route, browser-only imitation or direct backend mutation simply because the managed metadata is absent.

The source-derived application and generated artifact must distinguish deliberately ordinary, separately declared HTMX from a semantic operation whose managed origin was stripped. A valid ordinary form remains permitted under its own narrower guarantee, but cannot claim a managed operation's assurance or authorization. An explicitly unavailable managed affordance may be structurally sound while still failing an independent positive product gate requiring an actual usable Claim/Reassign action.

Positive/adverse oracle: a genuine enabled authorized Claim reaches the matching trusted route and model; removing its `:choreo/op` or plan/route binding does not preserve a working anonymous Claim. Check the actual generated/rendered surface and installed route, not only a local ”unavailable” report.

Gate B – One connected semantic subject

Runtime, tests, source proof/finite-check subjects, global Choreo, role-local projection, executable plans, issuer results, generated assets and their actual installed consumers derive from **one original admitted semantic meaning**, or have a specifically checked, direction-sensitive representation/refinement relation. The subject need not have identical data layouts in every representation. It must not change business guards, scope, actors, command identity or allowed outcomes without preserving a justified relation.

The permanent executable semantic core is not replaced by a proof kernel or one-off test reference machine. A seven-rule propositional checker, guarded transition checker, independent formal backend, browser adapter and physical XTDB writer own different propositions. Nonsemantic sidecars may explain their source correspondence but must not change runtime execution or issue permission. A valid abstract Claim theorem is useful under its premises without proving that the deployed handler performs the same guarded transaction.

Positive/adverse oracle: original Request/RequestAssignment Claim with current Membership runs through the supported plan; a proof-only model using invented `:request/helper-id` or omitting Reassign is refused as a full-subject correspondence even if its reduced model checks internally.

Gate C – Source ownership stays single

Public model/core interfaces own Request, RequestAssignment, Membership, Organization and other domain business semantics. Choreo owns participant interaction and justified knowledge; application construction joins the public semantics to actual plans, routes, effects and protected acquisition. Proof obligations, operation summaries, policy audit, code generation, diagnostics and tests derive from the **same** original declarations rather than maintaining plausibly synchronized business copies. The browser and router are not alternative owners of Claim or remote read permission.

Ordinary application Clojure may express genuinely complex domain policies. Where a correctness-relevant host function cannot be checked in the supported closed rule language, its original owner supplies the typed/effect/dependency contract, selected trusted boundary and an honestly open conformance premise. An empty source-audit catalog, a smaller proof DAG or an opaque implementation body cannot be relabeled complete mechanical inspection. HumanHelp-specific rules may occur in the HumanHelp product model, not be hard-coded into generic Gesso.

Positive/adverse oracle: a changed Organization Claim policy changes its source-owned affected judgments and dependent execution/assurance; a hand-maintained ”helper eligible” predicate in the browser/prover/router that disagrees with the model is

rejected or recorded as a source-conformance gap, not accepted because both names match.

Gate D – Authority remains at the trusted boundary

A browser-supplied role, principal, request/tenant identifier, plan, certificate, basis, correlation ID, DOM field or `already-authorized` flag grants **no** protected authority. Trusted ingress binds the authenticated actor and exact operation/target; a real authoritative model/read boundary checks current Organization/Membership, scope, disclosure and operation policy at its applicable use. Each protected write must guard changing conditions at incorporation, and each protected read, subscription or historical command resolution must independently establish current permitted disclosure.

A source-checked proof is not a runtime authorization token. A legal HTML affordance is not protection against a manually constructed hostile request. An old accepted membership observation does not continue authorizing operations after a relevant revocation. An opaque provider progression witness may justify the **scope of an observation** when its authority was independently qualified, but cannot manufacture the actor's right to perform a read or adopt a remote collection.

Positive/adverse oracle: authenticated eligible Alice Claims R using the current trusted model; a forged client helper role or revoked Membership fails at the protected model/transaction boundary even when the previously served UI still displays Claim. Test the direct hostile route, not only the disappearing button.

Gate E – Exact artifact and implementation binding

Where a release or proof claim depends on exact correspondence, preserve the subject and immutable interpretation through source, issued plan/callable, generated manifest, emitted bytes, served bytes, actual client-loaded bootstrap, selected handler, live provider and installed generation. A matching title, timestamp, route name, local receipt, copied hash or source similarity is not an authenticated production observation.

A build receipt may bind the approved manifest to locally emitted JavaScript bytes; it does not prove a server served those bytes, a browser loaded them, or a later substituted Biff handler used the source-checked route. Actual installation/activation and user-facing entrypoint evidence remain separate. Exact byte identity and checked semantic compatibility are different relations; A-to-B old/new compatibility does not imply B-to-A. Changed implementation invalidates affected physical applicability without necessarily invalidating an unaffected mathematical theorem.

Positive/adverse oracle: a qualified browser build loads its actual current plan and the server invokes the matching checked handler; a stale CDN asset, forged manifest, post- preflight Ring replacement or changed XTDB adapter cannot reuse a green candidate report.

Gate F – Dynamic premises remain dynamic

Static preflight rejects **structurally known** missing, incompatible or unsupported realization and requires the actual declared handling/recovery path for a relevant dynamic outcome, as owned by #req-preflight-dynamic-handling-coverage. It cannot guarantee future Membership, Request state, network delivery, scheduler fairness, transaction prestate, principal revocation or resource lifetime. Declared handling means a real source-derived continuation with owner, trigger, required knowledge, permitted effects and terminal or explicitly unresolved outcome; a boolean `:recovery-exists?` or copied success report is not structural coverage.

The trusted invocation, guarded incorporation, live acquisition, browser last-effect mutation and historical command resolver enforce actual current facts at their respective boundaries. An aborted HTTP attempt after possible C42 submission does not prove C42 rejected; missing history or unknown outcome must retain its uncertainty. A lost Live advisory must not make manual authorized acquisition impossible; an advisory alone grants no automatic user Refresh. Safety can hold when optional liveness/fairness assumptions do not.

Positive/adverse oracle: current eligible Claim and its admitted unknown-result resolution work; a fake recovery branch hides missing issuer/read authority, or preflight claims that the future provider succeeds. Reject the missing structural contract early; after an actual possible submission, retain precisely scoped `:commit-unknown` rather than invented rejection.

Gate G – Real physical integration remains required

Portable semantic/projection checks, finite exploration, source inspection and proof checking cannot by themselves establish actual Ring/Biff handler adoption, XTDB guarded transaction, transaction-internal prestate, complete physical writers, current principal, real Chromium/HTMX request/swap/OOB installation or cross-host fault recovery. A release or completion claim records exactly which **physical** subjects, versions, sessions, entrypoints, bytes, witnesses and fault windows were exercised, with untested or trusted premises retained.

A required check that is failed, skipped, missing, stale, unavailable, unsupported or incomplete does not pass. A successful **negative** test must reach the intended real enforcing boundary and independently observe that the forbidden write/disclosure/DOM mutation did not occur within the stated observation scope. A different earlier exception or a post-write diagnostic cannot count as prevention. Every applicable negative has a nonvacuous eligible positive through the same installed path. If the requested assurance requires an independent external proof rather than bounded test evidence, a physical test cannot substitute for that proof.

Positive/adverse oracle: real authenticated Claim succeeds, survives reload and is visible through a permitted installed browser; competing physical Claim, missing FX clock, stale served artifact and actual OOB partial swap have distinct appropriately typed results. A successful mocked model-only browser fixture cannot certify the installed production path.

Gate H – Reject contradictions when they become knowable

A contradiction is refused at the *first boundary with sufficient information and legitimate authority* to recognize and enforce it. A malformed typed operation, unrealizable Choreo knowledge, missing route/FX capability, incompatible plan/asset or required recovery contract should fail construction or pre-installation checks, before the associated managed effect is exposed. A malformed rendered Hiccup/HTMX affordance fails at the supported real pre-serialization boundary; an invalid invocation fails at trusted ingress; a stale or unauthorized guarded transaction fails before incorporation; an expired browser callback fails before its physical DOM mutation.

A static checker must not pretend to know future permissions, race outcome, physical network state or target incarnation. Nor may a late diagnostic describe itself as preventing an already incorporated command or previously disclosed response. A failure after an authoritative commit may block HTML installation, create publication/recovery debt or leave client knowledge unknown, but cannot retroactively turn the committed model transition into rejected truth. The observed first blocker must not be a cascading `:missing-plan` error when the original cause was an earlier invalid rule or lost permission.

Positive/adverse oracle: one genuine valid Claim reaches the appropriate physical model/DOM boundaries; missing `:biff.fx/now` is detected before the required invocation can issue a protected effect, revoked Membership is rejected at current incorporation, an invalid render blocks actual response delivery, and a retired callback is stopped at its last mutating effect. If the discovered bug is only detectable after an effect, report late detection and surviving uncertainty rather than a false prevention claim.

The gates apply to every supported entrypoint and effect path within the selected guarantee. A narrow declared legacy HTMX path may remain valid, but cannot inherit stronger managed proof, installation, policy or runtime claims without the corresponding checks. Their report preserves which original source, installed generation, profile, eligible positive, adverse case, actual observer and open obligations justify each successful gate.

Gate I – Representation does not confer admission

A represented value is not an admitted semantic subject or privileged capability merely because it has the expected tag, nominal identifier, map keys, opaque-looking bytes, source digest, certificate-shaped fields or serialized form. Every elevation of status belongs to the responsible **actual** boundary: original-source type/semantic checking, trusted model/principal acquisition, canonical command-history admission, issuer-controlled proof checking, current resource/effect admission or qualified physical persistence and installation.

Distinguish at least the following from each other:

- original model rule or operation declaration, checked semantic term, generated plan and actually issued executable callback;
- proof candidate/report, source-rechecked in-process proof handle, independent checked theorem artifact and transferred production proof;
- browser claim/role/correlation token, currently authenticated actor and authoritatively verified permission to invoke or disclose;
- command ID and canonical immutable binding, physical attempt, retained original-command result and current authorized resolution;
- proposed basis/progression witness, trusted provider-qualified observation and authorized physically adopted fragment;
- local function/plan object, installed host binding and actual protected backend or DOM effect.

A versioned decoder may reject malformed input without thereby proving that all well-shaped data is trusted. A JVM-local checked handle prevents ordinary copy-and-paste substitution inside its admitted scope; it is not a hostile-JVM sandbox, transferable capability, signature or runtime authorization. Importing a durable report or proof package requires fresh checked source/issuer and applicable premise admission; one printable field cannot confer a stronger evidence class. An adversary supplying their own compatible-looking graph does not establish the required original-source dependency closure of the intended application.

Positive/adverse oracle: an actually issued Claim executor and a current original-source proof handle are accepted for the exact supported subjects; a copied EDN certificate/report, forged role, replayed stale resource generation, mismatched C42 binding or browser-supplied provider witness cannot cross the corresponding trusted admission boundary. The eligible positive must remain usable after rejecting its counterfeit.

Gate J – Capability completeness is independent of closure

The required product and original assurance inventories are **independent inputs**, not whatever the implementation currently registers, compiles, projects, tests or proves. A closed Choreo graph, source-derived report, finite operation catalog or internally consistent proof DAG can certify its explicitly smaller declared subject without satisfying the original stronger application or HumanHelp completion claim. The generator may not define its own denominator by discarding difficult capabilities, positive cases, external writers, invariants or dynamic recovery paths.

For HumanHelp, the independent positive product set includes Create; Claim, Unclaim, Mark on the way, Complete, Cancel and manager-target Reassign; search/order/options/details/continuity on the authorized adopted cohort; current authentication/Membership/Organization policy; multi-client advisory-only remote lifecycle; one-use explicit Refresh; eligible unknown-command recovery; and the separately selected Response interaction. The fourteen independent Part VIII gates also include real production-model authority, installed Biff/XTDB/HTMX paths, physical assets, true browser acceptance and failures. The Response decision remains product-owned and **unselected** until its authorized requirements are chosen and implemented. A placeholder, deliberately disabled control or missing registry entry does not count as a completed positive.

A genuinely narrower selected Claim pilot, external S0/S1 theorem or bounded prototype may pass its **named** partial profile; it must not relabel that profile as complete HumanHelp, all-writer closure, external formal transfer or multi-node production. The original requested release profile and requirements are fixed before evaluation. An unsupported or missing mandatory gate cannot be silently made inapplicable after it fails.

Positive/adverse oracle: all required original product actions have genuine eligible paths and their separate negative cases before a full operational claim passes; removing Reassign from the exposed registry and the checker catalog, or omitting Response and its positive oracle, does not make the full requirement set smaller. A legitimate partial Claim release must be reported as partial rather than refused as if no useful scoped work were possible.

Gate K – Incorporation, delivery, and knowledge stay distinct

The authoritative database's actual incorporation, a command's durable historical attribution, an HTTP attempt's delivery/return, a subscriber's advisory knowledge, the user's permitted authoritative read and the browser's adopted presentation are separate facts and evidence subjects. A connection timeout, abort, host loss, retired execution or absent immediate response does **not** prove a possibly submitted command was rejected. A known committed operation stays committed after postcommit render, publication or transport failure.

One logical command C42 retains its immutable principal/operation/ target/arguments binding across compatible retries and multiple physical attempts. The trusted resolver checks its exact retained historical result and **current** permission to disclose that result before using a fresh operation's eligibility conditions. A Claim may be historically incorporated even when Request R is no longer open for new Claim. Reusing C42 for different actors/arguments is a typed incompatibility; creating C43 is a different intent, not automatic safe replay of C42. Failure of A1 does not exclude incorporation by concurrent A2.

An authorized read of current R can repair a stale optimistic card without proving which previous logical command produced that state. A missing, unreadable, expired or insufficiently progressed command-result record must not be replaced by a guess from the card, request status, basis token or a timeout. The selected resolve/retry/fence protocol preserves `:commit-unknown` until valid command-specific evidence establishes a compatible original outcome or its qualified exclusion. A local result receipt cannot authenticate all other hosts' physical attempts.

Positive/adverse oracle: a real C42 Claim incorporates, loses its HTTP response and later permits historical `:already-incorporated` resolution under current disclosure, without a duplicate primary RequestAssignment. A newer post-Reassign card or A1 abort does not falsely prove that C42 succeeded or failed. A fresh eligible command still has a valid permitted success.

Gate L – Recovery preserves scope and adoption permission

An advisory is a scoped statement that a relevant authoritative observation may need refreshing; it is not an authority token, permission to issue a protected read, permission to replace a collection or guarantee of physical installation. Source, principal, Organization/Location, actual applied query, fragment, target/resource owner, provider basis and required coverage remain bound through retry, reconnect, failover and release retirement. Unrelated changes and unresolved obligations must not disappear because one newer or locally successful result was installed.

Under the selected HumanHelp remote policy, Bob's other-user Create and lifecycle changes, including changes to cards he already sees, remain advisory until he explicitly grants a scoped one-use Refresh. Its default grant permits **one logical authorized acquisition with one physical read budget and no automatic successor**. A separate selected recovery policy may authorize a bounded retry under its actual remaining physical budget, or a new explicit user Refresh/Retry may issue a new grant; an SSE notice, failed attempt, queued intent or reconnect cannot authorize another physical read by itself.

A new notice during the granted acquisition may be covered by the already authorized observation only when the trusted provider establishes its progression, scope and coverage and the browser finishes every required actual HTMX/OOB installation. An insufficient or incomparable basis, missing target or partial swap retains the uncovered debt; it cannot create a successor grant or silently widen the original query.

A permitted manual Refresh and initial acquisition remain available even when no notice arrived or no interest was registered in time. A subscription reconnect, routing filter hit, local Create exception, unrelated own-command settlement or new plan generation cannot implicitly grant Bob remote collection adoption. Creator-local accepted Create and a narrowly correlated authorized own-command reread are separate selected scopes. Revocation blocks future protected disclosures and effects at the current boundary; already disclosed offline data cannot be retroactively erased.

An authoritative absence requires a qualified **negative** observation under the correct query, principal, disclosure and basis, not simply an absent DOM node, local filter or failed network request.

Positive/adverse oracle: Bob can choose one authorized Refresh without a preceding SSE event, and the complete installed result discharges only covered debt. An O2 advisory arriving during A1 cannot issue an ungranted A2; a lost notice, unmatched OOB target or recycled DOM resource cannot authorize remote content or declare false freshness.

Gate M – Evidence retains its scope through composition

Each assurance conclusion keeps its original proposition, normalized semantic subject, actor/operation/ state scope, premises used, evidence and checker class, quantification domain, relevant version/implementation/host generation and actual observation window. Source completeness is checked against the **independently required** original obligation manifest, not inferred from the elaborator's candidate graph. Copied identity fields, a replayed successful report or matching source hash do not establish that the current issuer, installed host, physical writer or browser was admitted.

Maintain distinct strength for: a propositional derivation, guarded abstract transition, conditional induction, exact finite exhaustion within its declared state/input domain, bounded test, externally checked theorem, source/assembly preflight, scoped actual physical observation and implementation refinement. A genuine S0/S1 soundness theorem need not be dismissed because S2 production checker, S3 typed atoms or S4 issuer-to-physical transfer are open; it equally may not impersonate those missing claims. Tests that exercise one physical schedule do not prove unbounded scheduling, transaction-internal prestate or all-writer closure. A proven abstract theorem may survive an unrelated source change while old installed physical evidence becomes stale.

The four Part XI release decisions—candidate, installation, activation, completion—keep separate subjects. The four independent operational, compositional, external formal and multi-node milestones keep separate denominators. The selected profile's required

results are recorded using the complete declared gate vocabulary: passed, failed, incomplete, unavailable, unsupported, skipped, stale and missing. A reported `:passed` without authenticity/applicability checking is still not accepted evidence.

Positive/adverse oracle: a qualified unchanged abstract Claim theorem is reused under an actual valid dependency relation, while the changed XTDB adapter and emitted browser asset receive new physical checks. A copied certificate map, finite truth-table result, unexecuted browser test, narrow proof graph or receipt from an older host cannot make a stronger current release green.

Gate N – Policy visibility has an honest inspection boundary

Correctness-relevant application semantics are inspectable as original source-owned public rules, operations, identities, actor/relationship/disclosure constraints, guarded effects, recovery, lifecycle invariants, required capability inventory, dependency graph and explicitly named opaque or external conformance premises. The audit and explanatory report are **read-only derived views** of the same construction and supplied evidence, not a second business policy registry, separate runtime interpreter or new admission issuer. Real domain complexity is not a defect; no arbitrary ratio of policy lines to total implementation is an acceptance criterion.

Inspection states precisely which Clojure rule bodies, callbacks, model predicates, FX, writer paths, administrative operations, startup/replacement handlers and protected acquisition points were actually examined. A condition influencing authorization, eligibility, disclosure, command acceptance, relationship uniqueness or observable lifecycle cannot appear mechanically verified merely because some other part of the application graph is closed. Where Gesso cannot inspect the relevant host code, require an original source-owned typed/effect/ dependency contract and a named implementation- conformance premise, or refuse a stronger selected profile that requires its discharge.

A known opaque branch must be visible at the first boundary capable of identifying its influence; unexamined code is reported as not inspected, not as absence of policy. The original operation inventory and physical writer universe remain independent: no list of observed Gesso executors proves that outside XTDB clients cannot mutate the same business state. A privacy-scoped view never discloses protected actor or organization facts to a reader unauthorized to see them.

Positive/adverse oracle: a real public model-owned Membership guard is listed with its source, dependencies and qualified coverage; an arbitrary Clojure callback with uninspected extra authorization is not reported as fully proved because the semantic-audit operation list is empty or contains no detected escapes. A valid complex domain policy must remain expressible with honest assumptions.

Gate O – Formal assumptions create production obligations

A formal proof is about its precise source-derived subject, definitions, axioms and premises. Every correctness- relevant assumption needed to apply its result creates an **owned production obligation**, not merely a diagnostic note. For a model that treats Claim as an atomic guarded action, the selected real Request/RequestAssignment writer must implement the same protected transition or the correspondence must retain a named, unproved physical premise. Passing model exploration does not establish that arbitrary Clojure or XTDB executes the checked transition.

The transfer ledger binds the exact original initializer/invariants and complete operation catalog, declared actor and current Membership/ Organization policy, read/guard footprint, relationship absence or one-primary cardinality, multi-document write footprint and protected frame (including new field presence), command binding/retained outcome, publication intent, provider consistency/ progression, model-to- concrete state relation, installed handler/executor, and every relevant physical writer. A pinned external read is not a transaction- internal prestate witness; an ASSERT-before-PUT shape, local submission receipt, ledger key or matching later card is not independently a universal backend refinement theorem. Full-value endpoint comparison does not exclude ABA without a history-sensitive mechanism when required.

If the formal subject assumes authentic communication, fair delivery, resource currentness, authorized observation or eventual browser convergence, the actual selected transport, router, read permission, HTMX effect and user-action policy carry separate physical/ environmental duties. HumanHelp's advisory remote policy cannot prove unconditional automatic visible convergence when Bob does not grant Refresh.

Positive/adverse oracle: a qualifying actual guarded multi-record Claim and relevant external writer inventory support the stated concrete-to- abstract safety relation; concurrent duplicate primary assignment, unprotected newly added field, missing trusted Membership, incorrect XTDB prestate claim or lost router interest leaves the exact assumption open or fails its requested physical admission. An unaffected abstract theorem remains a valid theorem under its original premises, not proof of the broken production step.

Gate P – Intent review and repair preserve the evidence boundary

Human, model-assisted or LLM-assisted intent review may identify an internally well-typed policy as surprising, asymmetric, under- constrained, inequitable, incomplete or inconsistent with independently stated product needs. Such a finding is **advisory**.

It cannot be promoted into a proof result, new runtime authorization, automatic source mutation, denied valid program, discharged formal premise or modified release gate solely because a reviewer expressed confidence.

Only the legitimate product/policy owner may decide whether a new requirement changes business intent. A confirmed decision becomes an original public model rule, declared invariant, operation/authorization/ disclosure contract or explicit requirement. Gesso then checks the supported meaning and regenerates affected Choreo, proof obligations, application plans, build assets and current physical qualification. A changed assumption, weakened invariant or removed capability is a **new claim** whose original/revised denominators and evidence stay distinguishable. A generator or review report cannot edit the required product inventory until it appears complete.

A mechanistically invalid rule, unavailable knowledge, missing dynamic recovery, or unsafe deployed effect belongs to its first appropriate typed semantic, preflight, trusted model, issuer or physical owner. An unprovable opaque Clojure policy remains a declared premise or unsupported assurance request until independently qualified, rather than silently being rewritten into a convenient rule that can be proved. Ordinary Clojure remains the developer authoring surface; Gesso owns generation of formal models, not a requirement that HumanHelp developers handwrite a second TLA+/Quint/ Recife business model.

Positive/adverse oracle: a product owner approves a revised current Membership requirement at the original model source, and the affected proof/plan/physical evidence is rechecked without discarding unaffected sound results. A reviewer who merely dislikes a valid policy cannot modify it through a diagnostic report, weaken the invariant in generated formal code, alter a green status or conceal the unselected Response decision.

Together Gates A–P remain *independently scoped recurring checks*, not one undifferentiated completion badge. They reject an invalid candidate at its first real owner, preserve valid usable positives and leave genuinely unknown, conditional, uninspected and unsupported duties visible. A final accepted release must meet its original selected profile without changing its requirements after seeing failures.

Worked stopped candidate: a green-looking release is not admission

This is a **controlled acceptance counterexample** spanning all Gates A–P. It shows why passing local semantic checks, building JavaScript, and observing several browser successes cannot make a defective candidate an admitted production release. The candidate is schematic and deliberately inconsistent. It is not executable Gesso API, a real HumanHelp defect report, or evidence that the corresponding checks have run. The literal names below identify narrative subjects, not supported constructor functions or wire encodings.

Assume the original approved release intent requests a real HumanHelp operational application, source-derived compositional Claim assurance, and a selected external-formal/checker correspondence profile. The required gates and product capabilities are fixed **before** candidate generation. Some deliberately narrower release could be independently requested, but this candidate cannot change the original intent when its stronger requirements fail.

The candidate reports success for an abstract Claim derivation, a compiled browser bundle, an earlier Chromium journey and a source-derived application graph. Those observations can all be true within their own scopes while the current candidate remains inadmissible.

The illustrative candidate and its conflicting subjects

The following is **illustrative EDN data only**, not a Gesso constructor, accepted schema, current API or test result:

```
{:candidate/semantic-plan :claim/P24
 :candidate/browser-artifact
 {:declared-plan :claim/P23
  :bytes-observed :previous-build}
 :candidate/trusted-providers
 {:required #{:principal/current :clock/current
              :request/current :membership/current}
  :supplied #{:principal/current :request/current}
  :on-missing :ordinary-htmx-fallback}
 :candidate/browser-envelope
 {:claimed-principal :helper-17
  :claimed-role :helper
  :claimed-basis 42}
 :candidate/command-recovery
 {:original-command :C42
  :response :timeout
  :current-request-status :claimed
  :inferred-outcome :committed}
 :candidate/remote-adoption
 {:source :reconnect
  :policy :adopt-all-newer-cards}
 :candidate/operation-inventory
 {:original-required #{:claim :unclaim :mark-on-the-way
                       :complete :cancel :reassign :create
                       :response}
  :generated #{:claim :unclaim :mark-on-the-way
               :complete :cancel :create}}
 :candidate/proof-evidence
 {:original-claim-subject :claim/P23
  :claimed-production-subject :claim/P24
  :browser-tested-artifact :previous-build}
 :candidate/preflight
 {:network-remains-available true
  :on-transport-loss nil}}
```

The ‘:response‘ entry means only that the **independent** product requirement is still owed. Its precise Response interaction semantics remain a HumanHelp product decision at #decision-humanhelp-response-flow; this example does not select a workflow. The synthetic provider list is likewise an example of required/supplied mismatch, not a complete real Biff FX contract.

The operation's actual RequestAssignment history, current Membership, original command record, transaction write footprint, installation/activation identity, subscription registration, and trusted progression are intentionally missing from this green-looking

report. Their absence is **not** proof that they were safely implemented elsewhere.

Classify each defect at its earliest legitimate owner

Do not feed the entire broken candidate to one late release check and treat the first random exception as coverage of every gate. Record individual mutations of a previously eligible baseline, the expected first enforcing boundary, the required result and the independently observed absence of forbidden effects. A defect rejected during source assembly prevents later candidate installation; a separate controlled eligible instance is needed to challenge a runtime-only or physical fault.

Defect or false inference	First capable owner
Missing trusted clock causes silent ordinary HTMX fallback	Source-derived required FX/route and actual trusted invoc
P23 browser manifest or old bytes offered for P24	Current compiler/build/assembly, then actual serving/load
Browser-supplied principal, helper role or basis accepted as authority	Trusted ingress, current protected read and model incorpo
Independent model or proof uses scalar `:request/helper-id`	Original model/operation and source-checked proof subje
New Organization/Claim policy paired with old certificate	Original source, dependency issuer and selected evidence
Proof DAG omits Reassign, initializer or original invariant operation	Independently required source/operation/invariant invent
Response and manager-target Reassign omitted from exposed actions	Independent Part VIII product requirement admission
Model-owned Claim eligibility duplicated or changed in browser/router	Public model source and source-to-plan/policy correspond
Uninspected Clojure predicate or raw XTDB writer called proved absent	Source-audit and independent physical writer scope
Old Membership pre-read substituted for current commit authorization	Trusted guarded transaction and selected provider consist
Submitted XTDB effect appends an unaccounted write	Physical statement/gateway and relevant all-writer footpr
Replacement document erases newly added field or permits duplicate primary	Transaction guard/frame/cardinality and all relevant write
Same C42 rejected by new open-Request guard despite stored success	Trusted original-command binding/history resolver
A1 abort/HTTP timeout/current claimed card used to infer C42 result	Authoritative command history and qualified disclosure
Dispatcher dies after commit, notification debt erased	Original atomic publication-intent and selected dispatcher
Late subscriber or failed backup router silently reports freshness	Registration fence, replay, gap recovery and authorized ac
SSE/reconnect changes Bob's board without a user Refresh	Scoped read/adoption policy and last physical browser effe
O2 notice creates ungranted A2 while A1 is pending	One-logical-read grant, qualified progression and installed
Partial OOB install or retired callback declared settled	Real HTMX completion and target/effect generation
Preflight predicts future network or recovery flag has no real branch	Original dynamic outcome/recovery structure
Checked local Ring wrapper but production listener uses old handler	Actual Biff host installation and current activation
External theorem covers different checker or copied proof map	Independent formal import and production S2–S4 relation
Skipped required test or prior Chromium fixture credited as current	Named release gate/evidence issuer and actual browser ins
Human/LLM flags an odd but valid business rule	Product intent owner, not mechanical proof checker

The defect matrix is a **falsification plan**, not a claim that one running candidate visits all these boundaries. Every chosen mutant starts with its corresponding real eligible baseline, then changes one owned condition while holding other known prerequisites fixed enough to reach the declared check. If an earlier unrelated contradiction masks it, the test is inconclusive for that target boundary rather than a successful kill. A post-effect observer can diagnose harm but cannot prove prevention.

A physical check reports the **actual** submitted transaction, executed handler or browser effect and its independent observation window. Silence in logs, absence of a rendered card, an EDN `:passed` status, expected test-runner exception or a disconnected fixture does not independently demonstrate that a forbidden write or disclosure never happened.

Stop the candidate without changing model truth

The above release intent has **four distinct decisions** under Part XI:

Decision subject	This defective candidate's disposition
Candidate construction/admission	Block the selected strong request on known incompatible artifact, missing FX/recovery, original capab
Host installation	No permission to present this candidate as a valid installed generation; a separately admitted repaired
Activation/public exposure	No permission for this candidate to take over traffic or continue incompatible old callbacks
Requested completion	Operational HumanHelp, compositional Claim and selected external formal qualifications remain inde

The full original requirements cannot be silently relabeled `:inapplicable` after a blocker. For a mandatory result, distinguish Part XI's `:failed`, `:incomplete`, `:unavailable`, `:unsupported`, `:skipped`, `:stale` and `:missing` instead of replacing all with one `:false`. A genuinely `:passed` local gate still retains its precise subject and applicability; it does not make the combined strong release eligible.

A candidate whose static assembly is known invalid is stopped before newly exposing its managed effects. That does **not** assert that every possible dynamic race was exercised. If an earlier separately admitted generation already incorporated C42 before a later response, render, notice or host failure, the authoritative commit is **not rolled back**, and C42's client outcome remains uncertain until valid command-specific evidence establishes it. Retire this candidate's unissued or obsolete effects and keep separately owned command/disclosure, publication, and recovery obligations according to their true histories.

An old generation may remain active only while its own qualified policy, writer, principal and release assumptions still hold. Changing the planned artifact does not resurrect a stale browser callback; rollback of code does not undo a durable RequestAssignment or authorize replaying an unknown command as a new C43.

Reconstruct at the original owner, not by editing success flags

The repaired candidate is a **new qualified construction**, not the old candidate with renamed file digests or hand-edited ‘:passed=‘ fields. Re-establish the following owned sequence; independently satisfy its original selected profile rather than allowing the release report to choose a smaller denominator.

1. **Source and requirements.** Reacquire current public HumanHelp model and original product decisions, including RequestAssignment, Membership, Organization, actual Reassign target selection, all six lifecycle actions, Create, search/options, scoped Refresh, pending Response decision, original operation/invariant catalog and protected writer obligations. Select the Response contract through the product owner before claiming full operational completion.
2. **Construction and type/Choreo admission.** Regenerate the supported typed rules, source-owned operations, global/local Choreo, executable plans, route/FX requirements and real declared recovery branches. Reject new contradictions **before** installation; do not invent a browser-specific Claim policy or silent legacy route.
3. **Trusted providers and backend.** Bind actual clock, actor, Request, Membership, Organization and selected protected models to the real serving handler. Establish current principal, immutable logical command binding, transaction guard/frame/cardinality, exact submitted writer and required durable command/publication storage. The physical provider's internal prestate and all-writer coverage require their actual qualified evidence or disclosed premise.
4. **Browser and publication.** Rebuild from the current semantic manifest; check emitted/staged bytes, actually served/loaded bootstrap, current Biff handler, real HTMX request/swap/OOB completion and resource permissions. Preserve Bob's remote advisory-only policy, one-use scoped Refresh, independent manual acquisition if no notice arrived and honest uncovered debt.
5. **Proof and release evidence.** Regenerate original-source obligation closure and certificate candidates; recheck issued handles, used premises, changed implementation dependencies, mandatory external checker S0–S4 edges and real observed browser/ backend cases as the **selected** profile requires. A bounded Claim proof is useful but not full product or S4 transfer.
6. **Reevaluation.** Re-run each applicable independent positive and targeted negative test at its legitimate first boundary with actual physical observation, then independently evaluate candidate, installation, activation and named completion decisions. Do not call a repaired subgraph passed merely because it now avoids an earlier obvious failure.

The sequence above describes **dependencies and evidence subjects**, not functions named `prepare!`, `compose-application!`, `admit-generation!` or `release/admit!`. R4 used API-shaped examples to make the intent readable; it explicitly did not assert those names were installed Gesso entrypoints. Production implementations may combine phases while retaining the separate subject and check/observation obligations.

Demonstrate nonvacuous recovery and bounded claims

A fixed source is not enough: the same accepted generation must permit real legitimate Claim/Unclaim and Reassign operations and, when full HumanHelp completion is requested, the independently selected Response behavior. The relevant actual

Chromium/Biff/XTDB paths must be tested through rendered controls and current authenticated actors. Include an eligible Creator-local Create path; remote Create or lifecycle change remains merely advisory to Bob until his authorized Refresh.

For the original C42 failure window, use at least one controlled legitimate incorporation followed by a lost HTTP response, and an independently qualified historical resolution. Challenge another helper competing for the primary assignment, revoked Membership after pre-read, C42 incompatible binding, missing durable notice, late subscriber, router failover, no advisory before manual Refresh, partial OOB installation, resource retirement and incompatible old assets. Each outcome retains known committed, known rejected, already incorporated or unknown **business** meaning, separate from release gate `:passed=/`:failed=.

A narrow explicitly declared operational demonstration, a conditional abstract theorem, an independent external kernel theorem, a source-linked diagnosis and a tested physical fault scenario are valuable **different** achievements. Their results may be retained as valid for their actual subjects even while this strong candidate is stopped. The prohibited move is promoting one into a stronger completion claim without its correspondence.

The acceptance result for this example is therefore not that a single root status says ”rejected”; it is that each contradiction is attributed to its actual first owner, forbidden effects are blocked where claimed, permitted positive paths remain usable, postcommit truth is not rewritten, and the original release profile's unmet obligations remain visible until genuine independent evidence discharges them.

Final unified invariants

The unified design is one coherent account of what an application **means**, which participants may learn and do, how authoritative facts change, which physical effects implement that meaning, and exactly what the available evidence establishes. Its purpose is not to replace ordinary Clojure web development with a proof-only application language. It makes the supported, correctness-relevant web domain narrower and more explicit than arbitrary computation, so construction, source-derived checking and real enforcement can work together without giving either a fictional authority.

These are the closing invariants of Parts I–XII, not a second model language, an additional runtime policy registry, or new completion criteria. Each applies only to its admitted subject and selected assurance/release profile; a stronger conclusion requires the independently named supporting source, theorem, observation and physical correspondence. The final sixteen Gates A–P and four independent Part XI completion milestones retain their existing owners and cannot be replaced by this summary.

Application meaning and executable construction

A Gesso application has **one original public semantic construction**. Models own business policy, actor/relationship eligibility, disclosure, authoritative state and permitted effects. Choreo describes typed interactions and the knowledge participants must possess; its global machine admits no operation whose projected local participant cannot justify its relevant inputs, causal order or branch choice. Projection constructs local machines from that same meaning. Views attach those operations to actual affordances and presentation; application construction joins models, plans, routes, effects, transports, browser artifacts, acquisition and corresponding checking into one source-derived object, not independently hand-maintained copies.

Knowledge is not the same thing as data existing somewhere in a globally described application. A participant knows only what its declared local input, valid communication, authorized observation, deterministic derivation or explicitly admitted trusted assertion establishes. Communication is one way to produce knowledge, not the definition of distributed knowledge. Role, participant, principal, resource, transport attempt and server host have distinct identities; none may be inferred or authorized from another merely by giving them matching names.

The portable **executable semantic core** is the permanent source of the supported machine behavior. It is not identical to the small propositional **proof kernel**: the former runs defined Gesso semantics, while the latter checks specific derivations under an explicit logic. Finite checking, theorem elaboration, independent proof assistants, generated model checking and test instrumentation have distinct jobs; they neither replace the executable core nor become new owners of HumanHelp's business rules. A pure browser transition boundary and the relevant Clojure/JVM implementation can be compared only through declared semantics and qualified correspondence, not through parallel human-written versions of the same policy.

Ordinary Clojure remains the application author's medium, including complex domain behavior and existing supported libraries. Gesso generates accepted internal rules, obligations, plans and formal representations where its precise language permits; authors need not maintain handwritten parallel TLA+, Quint, Recife or proof-assistant specifications of their app. Host-language policies outside the closed checked language require honest public typed, effect, dependency and authority contracts with explicit opaque or trusted implementation-conformance premises. Neither an unknown Clojure predicate nor an unsupported program is made verified by calling it a rule. The checked subset must not define away independently required application operations.

The source semantic owner remains single when the original business policy changes: change the model/requirement and regenerate the affected Choreo, projections, proof subjects, application closure, artifacts and actual installation qualifications. A browser, router, auditor, report or formal generator may not silently repair itself by inventing a second policy owner. Source-derived audit and readable first-blocker diagnosis are projections of that same original source, with their actual inspection coverage and unexamined code made explicit.

Authority, commands, and observations

The browser is adversarial relative to authoritative model truth. It cannot grant server rights by posting a principal, role, tenant, Request identifier, capability-shaped map, proof report, claimed basis, plan label, provenance or saved client state. Trusted server boundaries authenticate the principal, select the admitted operation/target and reauthorize **current** read, subscription, command-result disclosure and mutation. A presented affordance, nominal value or checked theorem does not substitute for the guarded authoritative action.

The authoritative transition enforces the current original model policy and all required source-owned writes in its declared atomic frame. For HumanHelp Claim that includes the actual Request, independent RequestAssignment one-primary relationship, current Membership/Organization eligibility, command history and any selected durable publication intent. A stale external read, typed transaction plan, locally accepted write receipt, matching later card or checked ASSERT/PUT syntax does not independently attest transaction-internal prestate, all relevant writers, preserved unrelated fields, ABA history or physically committed outcome. The selected backend's exact enforcement and any open provider premises remain visible. XTDB may be deliberately chosen as

the implementation substrate when it simplifies real correctness; semantic concepts should not be confused with incidental XTDB representation or burdened by speculative portability requirements.

A static operation, one logical command, a particular attempt, physical submission, transport request, browser execution and later observation are **not** interchangeable identities. A command preserves its immutable original binding across admitted retries. Incorporation, durable historical attribution, transaction/provider result, HTTP delivery, participant knowledge, current state and physically adopted presentation remain distinct. A lost response after possible submission leaves a genuine unknown until qualified command-specific evidence establishes the original result or valid exclusion. A later Request state cannot establish that C42 incorporated, even when it looks compatible; another attempt's abort does not exclude a concurrent incorporation. Historical compatible replay may be permitted even after a new-command Claim guard becomes false, but current result disclosure remains authorized independently.

An authoritative read identifies its principal, scope, original query/relationship meaning, projection, trusted provider position and coverage. An authoritative **absence** is a qualified negative observation under the appropriate disclosure semantics, not a missing DOM row, local filter, silent subscriber or failed network request. Domain revision, transaction basis, valid time, provider progression, observation, command, request, resource instance and assembly generation remain separate coordinates. Generic Live cannot invent total ordering from opaque basis strings, timestamps, invalidation arrival or source names. Only the chosen trusted authority can establish the progression and coverage relation needed by a read or command-resolution claim.

Presentation, resources, and recovery

Optimism is typed, nonauthoritative, provisional knowledge of one command or user intent under its admitted source and available basis. It may display expected consequences but cannot claim incorporation, authorize new actions or suppress unknown command outcomes. A creator's Create intent does not require a fictional earlier observation of an entity that does not yet exist. Provisional settlement, rejection, unknown status, supersession and postcommit UI failure have different meanings and recovery owners.

The browser realizes its local machine through a small portable pure state/event-to-abstract-effect transition boundary and a deliberately narrow host-effect shell. HTMX retains the ordinary request/response, out-of-band target, swap, settlement and lifecycle work it already owns. Gesso gates or observes the documented boundaries needed for semantic safety; a bespoke HTMX extension or replacement client network stack is not an architectural prerequisite. Gesso-owned timers, DOM effects, deferred callbacks, continuity, optimistic presentation and related browser resource effects have explicit ownership and observable completion.

Semantic retirement revokes the relevant execution/ resource owner's effect rights **before** best-effort physical cleanup. Shared resources independently owned by other live executions retain their own legitimate rights. A callback, HTTP response or timer must recheck its actual current effect/ target ownership at the last physical mutation, not only when scheduled. A reused logical DOM ID, failed abort, replaced plan or newer browser generation cannot resurrect retired rights. Installation is recorded after the selected actual physical effect completes, not merely on HTTP 200, response parsing or an intended swap. Selection, scroll, focus, compatible drafts and details continuity preserve local presentation only; they do not freeze model truth or grant obsolete continuation authority.

Gesso Live transports scope-bound **advisory** information that relevant authority may have changed. It is not a second database, automatic read permission, authentic command result, subscription receipt or DOM installation. A selected durable profile records the required original publication intent in a suitable authoritative commit boundary and separately owns dispatch, retry, deduplication and failure accounting. Neither an outbox nor a locally complete router filter proves that a late subscriber cannot miss an invalidation. Initial observation, registration fencing, qualified replay/gap detection, failover and independently permitted reacquisition have explicit owners and reliability premises. Duplicated advisory redelivery is safe under its actual disclosure, resource and idempotency contract; it never duplicates business incorporation.

Acquisition and **visible adoption** require their own current authorization, source/query scope, trusted observation coverage, selected user grant when applicable, and actual completed browser installation. For HumanHelp, remote Create and remote lifecycle changes, including changes to already- visible cards, remain advisory until Bob explicitly grants one scoped logical Refresh under the selected ordinary policy. A notice, reconnect, router failover or later advisory arriving during that cycle cannot mint an unrequested successor read. A provider- qualified already covered newer observation may discharge the corresponding debt; incomparable basis, incomplete OOB install or uncovered query retains it. Creator- local accepted Create and narrowly correlated authorized own-command recovery are separately declared exceptions, not general remote auto-adoption.

Lost command outcome, lost notice, expired authority, invalidated browser resources, provider failure and changed release need separately owned recovery. Convergence and liveness are conditional on the named fairness, retention, availability, scope, authenticated user action and environmental premises. In particular a policy requiring Bob to press Refresh cannot promise automatic visible convergence if he never does so.

Evidence, release, and topology

Every accepted conclusion keeps its exact source, proposition, scope, original required obligation inventory, selected profile, premise set, checker or test mechanism, observation/evidence provenance, interpretation version and actual implementation and installation applicability. Original product/ operation/invariant requirements are independent of the currently discovered proof, route, browser or writer graph. A closed smaller graph proves or checks only its named smaller subject. A generator cannot silently remove Reassign, Response, another relevant writer, a required dynamic recovery or a positive operation and thereby complete the requested larger claim.

Mechanically derived proof status is reserved for what an accepted formal semantics and actually checked derivation establish under named assumptions. Conditional induction, finite state exhaustion, production test, original-source type/preflight, physical writer observation, independent mathematical theorem and formal-to-production refinement are distinct evidence classes. Exact finite checking establishes its finite bounded subject; unbounded transfer requires sound abstraction or another qualified theorem. Tests remain essential for trusted host interfaces, libraries, actual Chromium/HTMX, Biff/XTDB, deployment, security, faults and scale; they are not a synonym for proof. Likewise a certificate-shaped map, hash, copyable report or tool exit code is not proof issuance, runtime authorization or an installed execution capability.

The closed seven-rule kernel's possible external theorem has explicit S0 rule-level soundness and S1 derivation- system soundness subjects. S2 actual Clojure checker correspondence, S3 typed source-atom interpretation and S4 issuer/application to physical transfer are separately qualified. A sound abstract theorem can remain true when a backend, handler, artifact or external writer changes; its **physical applicability** must then be requalified. A source- derived proof hierarchy cannot invent its own complete application catalog or authenticate untested physical observations.

A released application is not defined by one green map. The original selected requirements and permitted evidence mechanisms precede evaluation; the candidate, actual installed host, current activation and named completion claim require four subject-specific decisions. Application safety is enforced from the first served managed effect, not postponed until full formal or distributed qualification. Directional old/new client/server compatibility, retired effect rights, real served/loaded artifact identities, protected writers, command history, source generation and failure recovery are independently checked where relevant. Rollback of code neither reverses a durable transaction nor erases a pending historical command.

Operational HumanHelp completion, compositional certification, external formal verification and production multi-node realization are four distinct milestone claims with their own original requirements and evidence. The full HumanHelp product includes six lifecycle actions, Create, manager-target Reassign, current model authorization, rich adopted- board behavior, selected Response interaction, explicit Refresh and fourteen independent positive/adverse acceptance gates. A narrow Claim proof does not complete the product; an operational application need not claim all external metatheory, and a two- host prototype does not qualify every production topology. A reject- everything candidate or a never- executed mandatory test cannot satisfy a usable completion gate.

Multiple physical hosts realize the same logical authority, command, Choreo and browser contract. They do not make a new business semantics. Durable model facts remain the ordinary cross-host rendezvous; selected authoritative command history, publication recovery, subscriber interest, authenticated acquisition, router failover and browser adoption carry their real failure windows. A node's interchangeability does not prove exactly- once effects, complete interest knowledge, all-message receipt or participant liveness. A Clojure router and a later Rust optimization may implement the same semantic contract; neither a particular router topology nor universal durable participant messaging is mandatory for ordinary single-node Gesso.

Source-owned audit, intent, and repair

An application's correctness-relevant policy is visible as named original rules, operation inputs and effects, authority and disclosure, lifecycle transitions, dependencies, admitted assumptions and required capabilities. The audit identifies the code and physical paths it actually inspected, and which Clojure predicates, callbacks, FX, external services and writers remain opaque or uninspected. “No detected escape in the inspected subset” does not mean “all source policy was inspected.” The report is a read-only explanation under the same disclosure policy, not a second source of authority or an issuer.

A diagnostic finds the earliest legitimately knowable blocker and retains the dependent failures as consequences. A model counterexample refers to its actual formal subject, trace, abstraction, bounds and premises; a fixture cannot be passed off as a generated external checker run or a real XTDB incident. Mutants need a predeclared first enforcing gate, a usable valid positive, and independent observation of the prohibited effect within the actual scope. A late detection is not before-effect prevention, and no accepted status may be manufactured by silently relaxing the selected profile.

Human or AI-assisted review may identify surprising intent, missing business restrictions or reasonable alternative policies. It does not prove a well-typed policy wrong, issue admission, grant new rights or change source. Only the authorized product owner can select a new business requirement and promote it into the original public model/semantic contract. Repair then regenerates and rechecks only the affected source, proof, plan, adapter, host, asset and physical evidence cone. Weakening an invariant, adding an axiom or removing a required feature is a new claim, not an invisible way to make the old claim pass.

Final design principle

A Gesso choreography is a typed, source-owned description of how participants acting for principals interact with evolving authoritative facts, communicate justified knowledge, and execute permitted local behavior.

One executable semantic machine joins public model truth, global and local Choreo, declarative HTMX-facing views, typed provisional browser behavior, trustworthy guarded transactions and scoped advisory recovery. The framework generates the proof and assembly machinery around that meaning rather than asking application authors to write separate business rules for every runtime or theorem prover.

The same design is valid on one node or across multiple hosts. Its logical meaning is stable while physical registration, publication, command history, installation and recovery carry explicit real-world premises.

Gesso may prove the supported claims it has actually checked, and may accept explicitly qualified trusted assumptions, but it may not transform a typed label, finite test, diagnostic, report, compilation, browser success or source-consistent graph into a stronger certificate or production fact without the corresponding evidence and actual enforcing boundary.

Success is usable, authorized product behavior with its true source, physical effects, proof scope and remaining obligations visible. No apparently green subset, missing required capability, unchecked writer, silent downgrade, fabricated command outcome, unauthorized adoption or unexamined assumption can stand in for that result.